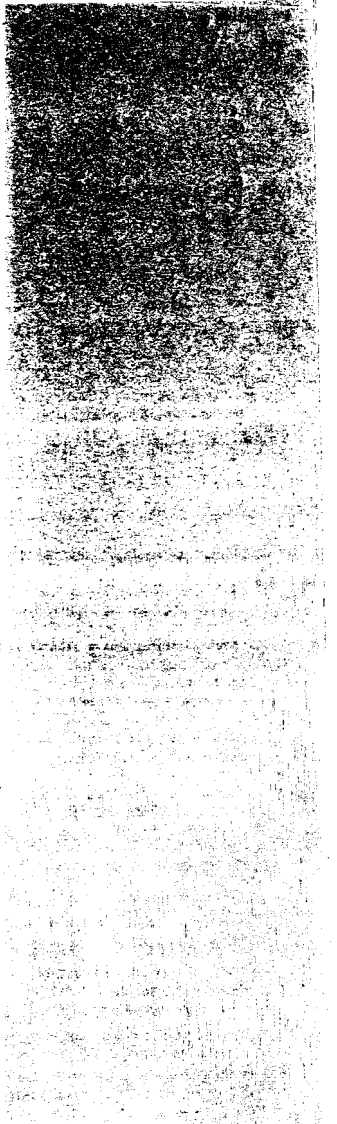
15765



MATEMÁTICAS DISCRETAS

Cuarta edición

Richard Johnsonbaugh
DePaul University, Chicago

BIBLIOTECA
FACULTAD
DE EXACTAS, INGENIERIA
Y AGRIMENSURA
ROSARIO

TRADUCCIÓN:
Óscar Alfredo Palmas Velasco
Doctor en Matemáticas
Instituto de Matemáticas

REVISIÓN TÉCNICA:
Víctor Hugo Ibarra Mercado
Lic. en Física y Matemáticas, ESFM-IPN
Catedrático de la Escuela de Actuaría
Universidad Anáhuac

PRENTICE
HALL

MÉXICO • NUEVA YORK • BOGOTÁ • LONDRES • MADRID
MUNICH • NUEVA DELHI • PARÍS • RÍO DE JANEIRO
SINGAPUR • SYDNEY • TOKIO • TORONTO • ZURICH

o/c. FONEC-UNR Dir. Gral. Cnt. c/c 130 Rcl. 21 Ningromo.
Precio $ 20,80.-
Prov. COMIPA - FOMEC
Fecha 11-5-01
Nº/orden 24385

Datos de catalogación bibliográfica

JOHNSONBAUGH, RICHARD
Matemáticas discretas, 4a. ed.
PRENTICE HALL, México, 1999

ISBN: 970-17-0253-0
Área: Universitarios

Formato: 20 × 25.5 cm Páginas: 720

EDICIÓN EN ESPAÑOL:
EDITOR: PABLO EDUARDO ROIG VÁZQUEZ
SUPERVISORA DE TRADUCCIÓN: ROCÍO CABAÑAS CHÁVEZ
SUPERVISOR DE EDICIÓN: MAGDIEL GÓMEZ MARINA

EDICIÓN EN INGLÉS:
Editorial Director: *Tim Bozik*
Editor-in-Chief: *Jerome Grant*
Acquisition Editor: *George Lobell*
Director, Production and Manufacturing: *David Riccardi*
Executive Managing Editor: *Kathleen Schiaparelli*
Managing Editor: *Linda Mihatov Behrens*
Editorial/Production Supervision: *Nicholas Romanelli*
Manufacturing Manager: *Trudy Pisciotti*
Manufacturing Buyer: *Alan Fischer*
Creative Director: *Paula Maylahn*
Art Director: *Amy Rosen*
Assistant to Art Director: *Rod Hernandez*
Interior Designer: *Donna Wickes*
Cover Designer: *Bruce Kenselaar*
Art Manager: *Gus Vibal*
Editorial Assistants: *Gale Epps/Nancy Bauer*
Director of Marketing: *John Tweeddale*
Marketing Assistant: *Diana Penha*

JOHNSONBAUGH: MATEMÁTICAS DISCRETAS, 4a. ed.

Traducido de la cuarta edición en inglés de la obra: **Discrete Mathematics.**

All rights reserved. Authorized translation from English language edition published by Prentice Hall, Inc.

Todos los derechos reservados. Traducción autorizada de la edición en inglés publicada por Prentice Hall, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage and retrieval system, without permission in writing from the publisher.

Prohibida la reproducción total o parcial de esta obra, por cualquier medio o método sin autorización por escrito del editor.

Derechos reservados © 1999 respecto a la primera edición en español publicada por:
PRENTICE HALL HISPANOAMERICANA, S. A.
Calle 4 Núm. 25-2º piso, Fracc. Industrial Alce Blanco
53370 Naucalpan de Juárez, Edo. de México

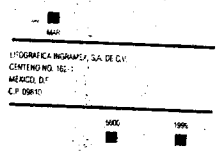
ISBN 970-17-0253-0

Miembro de la Cámara Nacional de la Industria Editorial, Reg. Núm. 1524.

Original English Language Edition Published by Prentice Hall, Inc.
Copyright © 1997
All rights reserved

ISBN 0-13-518242-5

IMPRESO EN MÉXICO / PRINTED IN MEXICO



CONTENIDO

1 LÓGICA Y DEMOSTRACIONES 1

1.1 Proposiciones 2
1.2 Proposiciones condicionales y equivalencia lógica 8
1.3 Cuantificadores 18
1.4 Demostraciones 34
†1.5 Demostraciones por resolución 42
1.6 Inducción matemática 46
Rincón de solución de problemas: Inducción matemática 56
Notas 59
Conceptos básicos del capítulo 59
Autoevaluación del capítulo 61

2 EL LENGUAJE DE LAS MATEMÁTICAS 63

2.1 Conjuntos 64
2.2 Sucesiones y cadenas 73
2.3 Sistemas numéricos 84
2.4 Relaciones 91
Rincón de solución de problemas: Relaciones 102
2.5 Relaciones de equivalencia 104
Rincón de solución de problemas: Relaciones de equivalencia 111
2.6 Matrices de relaciones 114
†2.7 Bases de datos relacionales 118
2.8 Funciones 125
Notas 136
Conceptos básicos del capítulo 137
Autoevaluación del capítulo 139

† Las secciones precedidas por un símbolo † pueden omitirse sin perder la continuidad de la lectura.

3 ALGORITMOS 142

3.1 Introducción 143
3.2 Notación para los algoritmos 144
3.3 El algoritmo de Euclides 151
3.4 Algoritmos recursivos 157
3.5 Complejidad de los algoritmos 166
Rincón de solución de problemas: Diseño y análisis de un algoritmo 182
3.6 Análisis del algoritmo de Euclides 186
3.7 El sistema criptográfico con clave pública RSA 189
Notas 193
Conceptos básicos del capítulo 193
Autoevaluación del capítulo 194

4 MÉTODOS DE CONTEO Y EL PRINCIPIO DE LA PICHONERA 197

4.1 Principios básicos 197
Rincón de solución de problemas: Conteo 207
4.2 Permutaciones y combinaciones 210
Rincón de solución de problemas: Combinaciones 225
4.3 Algoritmos para generar permutaciones y combinaciones 228
4.4 Permutaciones y combinaciones generalizadas 235
4.5 Coeficientes binomiales e identidades combinatorias 242
4.6 El principio de la pichonera 248
Notas 253
Conceptos básicos del capítulo 253
Autoevaluación del capítulo 254

5 RELACIONES DE RECURRENCIA 256

5.1 Introducción 256
5.2 Solución de relaciones de recurrencia 270
Rincón de solución de problemas: Relaciones de recurrencia 284
5.3 Aplicaciones al análisis de algoritmos 287
Notas 302
Conceptos básicos del capítulo 302
Autoevaluación del capítulo 302

6 TEORÍA DE GRÁFICAS 304

6.1 Introducción 305
6.2 Caminos y ciclos 316
Rincón de solución de problemas: Gráficas 330
6.3 Ciclos hamiltonianos y el problema del agente de ventas viajero 331
6.4 Un algoritmo para la ruta más corta 338
6.5 Representaciones de gráficas 344
6.6 Isomorfismos de gráficas 350
6.7 Gráficas planas 359
† 6.8 Locura instantánea 366
Notas 371
Conceptos básicos del capítulo 371
Autoevaluación del capítulo 372

7 ÁRBOLES 376

7.1 Introducción 377
7.2 Terminología y caracterizaciones de los árboles 385
Rincón de solución de problemas: Árboles 390
7.3 Árboles de expansión 392
7.4 Árboles de expansión mínimos 400
7.5 Árboles binarios 408
7.6 Recorridos de un árbol 415
7.7 Árboles de decisión y el tiempo mínimo para el ordenamiento 422
7.8 Isomorfismos de árboles 429
† 7.9 Árboles de juegos 440
Notas 450
Conceptos básicos del capítulo 450
Autoevaluación del capítulo 451

8 MODELO DE REDES Y REDES DE PETRI 455

8.1 Modelos de redes 456
8.2 Un algoritmo de flujo máximo 462
8.3 El teorema del flujo máximo y corte mínimo 473
8.4 Acoplamiento 478
Rincón de solución de problemas: Acoplamiento 484
8.5 Redes de Petri 486
Notas 496
Conceptos básicos del capítulo 497
Autoevaluación del capítulo 498

9 ÁLGEBRAS BOOLEANAS Y CIRCUITOS COMBINATORIOS 500

9.1 Circuitos combinatorios 500
9.2 Propiedades de circuitos combinatorios 509
9.3 Álgebras booleanas 516
Rincón de solución de problemas: Álgebras booleanas 522
9.4 Funciones booleanas y simplificación de circuitos 524
9.5 Aplicaciones 531
Notas 542
Conceptos básicos del capítulo 542
Autoevaluación del capítulo 543

10 AUTÓMATAS, GRAMÁTICAS Y LENGUAJES 546

10.1 Circuitos secuenciales y máquinas de estado finito 546
10.2 Autómatas de estado finito 554
10.3 Lenguajes y gramáticas 562
10.4 Autómatas de estado finito no deterministas 573
10.5 Relaciones entre lenguajes y autómatas 582
Notas 589
Conceptos básicos del capítulo 590
Autoevaluación del capítulo 590

11 GEOMETRÍA COMPUTACIONAL 593

11.1 El problema del par más cercano 593
† 11.2 Una cota inferior para el problema del par más cercano 598
11.3 Un algoritmo para calcular la cubierta convexa 601
Notas 608
Conceptos básicos del capítulo 609
Autoevaluación del capítulo 609

APÉNDICE: MATRICES 610

REFERENCIAS 615

SUGERENCIAS Y SOLUCIONES DE EJERCICIOS SELECCIONADOS 621

ÍNDICE 687

PREFACIO

Este libro está planeado para un curso de introducción a las matemáticas discretas, con una duración de uno o dos semestres. Los requisitos de matemáticas son mínimos: *no* es necesario un conocimiento del cálculo. Tampoco se exigen requisitos de computación. El libro incluye ejemplos, ejercicios, figuras, tablas, secciones relativas a la solución de problemas, notas, repaso y autoevaluación de cada capítulo que ayudarán al estudiante a dominar las matemáticas discretas básicas.

Panorama de la obra

A principios de la década de los ochenta casi no había libros adecuados para un curso de introducción de matemáticas discretas. Al mismo tiempo, existía la necesidad de un curso que ampliase la madurez matemática y la capacidad de los estudiantes para trabajar con la abstracción, y que incluyera temas útiles, como la combinatoria, los algoritmos y las gráficas. La edición original de este libro (1984) buscaba cubrir esta necesidad. Posteriormente, los cursos de matemáticas discretas se enriquecieron con diversos tipos de audiencias, como estudiantes de matemáticas y computación. Un grupo de expertos de la Mathematical Association of America apoyó el establecimiento de un curso de un año sobre matemáticas discretas. El Consejo de Actividades Educativas del Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) ha recomendado un curso de matemáticas discretas para estudiantes del primer año de licenciatura. Los criterios de acreditación de la ACM (Association for Computing Machinery) y del IEEE piden un curso de matemáticas discretas. Esta edición, al igual que las anteriores, incluye temas como algoritmos, combinatoria, conjuntos, funciones e inducción matemática, sugeridos por esos grupos. También busca la comprensión y la construcción de demostraciones y, en general, la ampliación de la madurez matemática.

Acerca de este libro

Este libro incluye:

- Lógica (incluyendo cuantificadores), demostraciones, demostraciones por resolución e inducción matemática (capítulo 1).
- Conjuntos, sucesiones, cadenas, notaciones para la suma y el producto, sistemas numéricos, relaciones y funciones, incluyendo ejemplos motivantes, como una aplicación de los órdenes parciales a la planeación de tareas (sección 2.4), las bases de datos relacionales (sección 2.7) y una introducción a las funciones de dispersión (sección 2.8).
- Una discusión amplia de los algoritmos, de los algoritmos recursivos y del análisis de algoritmos (capítulo 3). Además, en todo el libro se adopta un punto de vista algorítmico. Los algoritmos se escriben en una forma flexible de seudocódigo. (Este libro no presupone requisitos de cursos de computación, la descripción del seudocódigo utilizado se incluye en el mismo texto.) Entre los algoritmos presentados están el algoritmo de Euclides para determinar el máximo común divisor (sección 3.3), los mosaicos (sección 3.4), el algoritmo de criptografía con clave pública RSA (sección 3.7), la generación de combinaciones y permutaciones (sección 4.3), el ordenamiento por fusión (sección 5.3), el algoritmo del camino más corto de Dijkstra (sección 6.4), los algoritmos con retroceso (sección 7.3), la búsqueda a lo ancho y en profundidad (sección 7.3), los recorridos de los árboles (sección 7.6), la evaluación de un árbol de juegos (sección 7.9), la determinación del flujo máximo en una red (sección 8.2), la determinación de un par cercano de puntos (sección 11.1) y el cálculo de la cubierta convexa (sección 11.3).
- Un análisis completo de las notaciones "O mayúscula", omega y theta para el crecimiento de funciones (sección 3.5). Al disponer de todas estas notaciones, es posible dar enunciados precisos acerca del crecimiento de funciones y la complejidad de los algoritmos.
- Combinaciones, permutaciones y el principio de las casillas (capítulo 4).
- Relaciones de recurrencia y su uso en el análisis de algoritmos (capítulo 5).
- Gráficas, incluyendo los modelos de computadoras en paralelo, el recorrido de un caballo de ajedrez, los ciclos hamiltonianos, los isomorfismos de gráficas y las gráficas planas (capítulo 6). El teorema 6.4.3 proporciona una demostración simple, breve y elegante de que el algoritmo de Dijkstra es correcto.
- Árboles, incluyendo árboles binarios, recorridos de árboles, árboles de expansión mínimos, árboles de decisión, el tiempo mínimo para un ordenamiento y los isomorfismos de árboles (capítulo 7).
- El algoritmo del flujo máximo, acoplamiento y las redes de Petri (capítulo 8).
- Un tratamiento de las álgebras booleanas que enfatiza la relación de las álgebras booleanas con los circuitos combinatorios (capítulo 9).
- Un estudio de los autómatas que enfatiza la modelación y las aplicaciones (capítulo 10). El circuito flip-flop *SR* se analiza en el ejemplo 10.1.11. Los fractales, incluyendo el copo de nieve de von Koch, se describen mediante algunos tipos especiales de gramáticas (ejemplo 10.3.19).
- Una introducción a la geometría computacional (capítulo 11).
- Un apéndice sobre matrices.

- Un gran énfasis de la relación entre los diversos temas. Como ejemplo, la inducción matemática está íntimamente ligada con los algoritmos recursivos (sección 3.4); la sucesión de Fibonacci se utiliza en el análisis del algoritmo de Euclides (sección 3.6); muchos ejercicios de todo el libro utilizan la inducción matemática; mostramos cómo caracterizar los componentes de una gráfica definiendo una relación de equivalencia en el conjunto de vértices (véase el análisis después del ejemplo 6.2.13) y contamos el número de árboles binarios con n vértices (teorema 7.8.12).
- Un vehemente énfasis en la lectura y realización de demostraciones. La mayor parte de las demostraciones de los teoremas se ilustran mediante figuras. En las secciones independientes *Rincón de solución de problemas* se muestra cómo resolver problemas y cómo realizar demostraciones.
- Numerosos ejemplos resueltos en todo el libro. (Existen más de 430 ejemplos resueltos.)
- Un gran número de aplicaciones, en particular a la computación.
- Cerca de 2400 ejercicios, con respuestas a casi la tercera parte de ellos al final del libro. (Los ejercicios con números en **negritas** tienen su respuesta al final del libro.)
- Más de 650 figuras y tablas para ilustrar los conceptos, para mostrar el funcionamiento de los algoritmos, para aclarar las demostraciones y para motivar el material.
- Secciones de *Notas* con sugerencias de lecturas posteriores.
- Secciones de *Repaso del capítulo.*
- Secciones de *Autoevaluación del capítulo.*
- Una sección de bibliografía con más de 100 referencias.
- En los forros del libro se resume la notación matemática y algorítmica utilizada en esta obra.

Cambios de la tercera edición

- Se han agregado once secciones denominadas *Rincón de solución de problemas*. Estas secciones muestran a los estudiantes formas de enfrentar y resolver los problemas, y cómo realizar demostraciones.
- La demostración por resolución es el tema de la nueva sección 1.5. Esta técnica de demostración, la cual se puede automatizar, y que por tanto es importante en el campo de la inteligencia artificial, ayuda a los estudiantes a tener una mejor perspectiva de la lógica, en general, y de la lectura y construcción de demostraciones, en particular.
- Se ha agregado una nueva sección acerca de los sistemas numéricos binario y hexadecimal (sección 2.3). Se presentan estos sistemas numéricos y se analiza la conversión entre los diversos sistemas. También se estudia la aritmética en los diversos sistemas.
- En la nueva sección 3.7 se estudia el sistema de criptografía de clave pública RSA, que recibe el nombre de sus autores, Ronald L. Rivest, Adi Shamir y Leonard M. Adleman. En el sistema RSA, cada participante hace pública una clave de ciframiento y oculta una clave de desciframiento. Para enviar un mensaje, se busca la clave de ciframiento del receptor en la tabla distribuida en forma pública. El receptor descifra entonces el mensaje utilizando la clave oculta.

- Se han agregado varias figuras para ilustrar demostraciones de teoremas. Ahora todas las figuras tienen leyendas, y las leyendas de las figuras que ilustran demostraciones proporcionan una explicación adicional y dan una mejor idea de las demostraciones.
- Se han agregado algunos libros y artículos recientes a la bibliografía.
- El número de ejemplos resueltos se ha incrementado hasta ser más de 430.
- El número de ejercicios se ha incrementado hasta ser casi 2400.
- Se ha establecido un sitio en la World Wide Web para proporcionar un apoyo actualizado para este libro.

Estructura de cada capítulo

Cada capítulo se organiza de la manera siguiente:

- Panorama
- Sección
- Ejercicios de la sección
- Sección
- Ejercicios de la sección

...

- Notas
- Repaso del capítulo
- Autoevaluación del capítulo

Las secciones *Notas* contienen sugerencias para lecturas posteriores. Las secciones *Conceptos básicos del capítulo* proporcionan listas de referencia para los conceptos clave de cada capítulo. Las secciones *Autoevaluación del capítulo* contienen cuatro ejercicios por cada sección, cuyas respuestas aparecen al final del libro. Además, la mayor parte de los capítulos tienen secciones *Rincón de solución de problemas.*

Ejercicios

Este libro contiene casi 2400 ejercicios. Los ejercicios que podrían ser más difíciles que el promedio se han indicado mediante una estrella, ☆. Los números de los ejercicios en **negritas** (aproximadamente la tercera parte de los ejercicios) indican que el ejercicio tiene una sugerencia o solución al final del libro. En algunos ejercicios claramente identificados se necesitan algunos conocimientos de cálculo. Sin embargo, en el cuerpo principal del libro no se utilizan conceptos del cálculo y, excepto por los ejercicios indicados, no se necesita saber cálculo para resolver los ejercicios. El final de las demostraciones se indica mediante el símbolo ■.

Ejemplos

El libro contiene más de 430 ejemplos resueltos. Estos ejemplos muestran a los estudiantes la forma de enfrentar problemas de matemáticas discretas, demuestran aplicaciones de la teoría, aclaran demostraciones y ayudan a motivar el material. El final de los ejemplos se indica mediante el símbolo □.

Rincones de solución de problemas

Las nuevas secciones *Rincón de solución de problemas* ayudan a los estudiantes a enfrentar y resolver problemas, y también muestran cómo hacer demostraciones. Escritas de manera informal, cada una de estas secciones es autosuficiente y continúa el análisis del tema del problema. En vez de simplemente presentar una demostración o la solución a un problema, la intención de estas secciones es mostrar diversas formas de enfrentar un problema, analizar aquello que debe buscarse para obtener la solución de un problema, y presentar técnicas de solución de problemas y de demostraciones.

Cada *Rincón de solución de problemas* comienza con el enunciado de un problema. Después de enunciar el problema, se analizan algunas formas de resolverlo. A este análisis le siguen las técnicas para determinar una solución. Después de hallar una respuesta, se presenta una solución formal para mostrar la forma correcta de redactar ésta. Por último, se resumen las técnicas para solución de problemas utilizadas en la sección. Además, algunas de estas secciones concluyen con una subsección *Comentarios*, la cual analiza la relación con otros temas de matemáticas y ciencias de la educación, proporciona una motivación del problema y enumera algunas referencias para lecturas posteriores relacionadas con el problema.

[redacted]

El sitio http://condor.depaul.edu/~ rjohnson contiene información acerca del libro, incluyendo programas de computadora, transparencias, ejercicios para computadora, un programa para generar gráficas aleatorias de diversos tipos y una fe de erratas de la edición en inglés.

Agradecimientos

He recibido útiles comentarios de muchas personas, entre las que se incluyen a Gary Andrus, Robert Busby, David G. Cantor, Tim Carroll, Joseph P. Chan, Hon-Wing Cheng, Robert Crawford, Henry D'Angelo, Jerry Delazzer, Br. Michael Driscoll, Carl E. Eckberg, Susanna Epp, Gerald Gordon, Jerrold Grossman, Mark Herbster, Steve Jost, Nicholas Krier, Warren Krueger, Glenn Lancaster, Donald E. G. Malm, Kevin Phelps, James H. Stoddard, Michael Sullivan, Edward J. Williams y Hanyi Zhang.

Agradezco de manera especial a Martin Kalin sus comentarios acerca de la nueva sección *Rincón de solución de problemas* y por su apoyo relativo a la nueva sección acerca de las demostraciones por resolución; a Gregory Brewster y a I-Ping Chu por nuestras discusiones acerca de los flujos en redes de computadoras; a Gregory Bachelis por revisar la nueva sección relativa al sistema de criptografía RSA y a Sam Stueckle, de Northeastern University; Towanna Roller, de Asbury College; Feng-Eng Lin, de George Mason University; Gordon D. Prichett, de Babson College; y Donald Bein, de Fairleigh Dickinson University por revisar el manuscrito de esta edición.

Estoy en deuda con Helmut Epp, decano de la escuela de computación, telecomunicaciones y sistemas de información en DePaul University, por prestar su tiempo y apoyo para el desarrollo de esta edición y sus predecesoras.

He recibido un apoyo constante del equipo en Prentice Hall. Agradezco de manera especial la ayuda de George Lobell, editor ejecutivo, y Nicholas Romanelli, supervisor de producción.

R.J.

1

LÓGICA Y DEMOSTRACIONES

1.1 PROPOSICIONES
1.2 PROPOSICIONES CONDICIONALES Y EQUIVALENCIA LÓGICA
1.3 CUANTIFICADORES
1.4 DEMOSTRACIONES
†1.5 DEMOSTRACIONES POR RESOLUCIÓN
1.6 INDUCCIÓN MATEMÁTICA
RINCÓN DE SOLUCIÓN DE PROBLEMAS: INDUCCIÓN MATEMÁTICA
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Querido Sr. Marlowe, observo en usted una tendencia desagradable hacia las transiciones bruscas. Una característica de su generación, pero, en este caso, le debo pedir que siga algún orden lógico.

— de Murder, My Sweet

La **lógica** es el estudio del razonamiento; en particular, se analiza si un razonamiento es correcto. La lógica se centra en las relaciones entre los enunciados y no en el contenido de un enunciado particular. Por ejemplo, considérese el siguiente argumento:

Todos los matemáticos utilizan sandalias.
Cualquier persona que utilice sandalias es algebrista.
Por tanto, todos los matemáticos son algebristas.

Desde el punto de vista técnico, la lógica no permite determinar si estos enunciados son verdaderos; sin embargo, si los dos primeros enunciados fuesen verdaderos, la lógica garantizaría que el enunciado

Todos los matemáticos son algebristas.

también es verdadero.

Los métodos lógicos se utilizan en matemáticas para demostrar teoremas, y en computación para demostrar que los programas hacen precisamente lo que deberían hacer.

† Esta sección puede omitirse sin pérdida de continuidad.

En la última parte del capítulo analizaremos algunos métodos generales para realizar demostraciones, uno de los cuales, la inducción matemática, se utiliza en matemáticas y en computación. La inducción matemática es particularmente útil en las matemáticas discretas.

1.1 PROPOSICIONES

¿Cuál de los siguientes enunciados es verdadero o falso (pero no ambos)?

(a) Los únicos enteros positivos que dividen a 7 son 1 y el propio 7.

(b) Alfred Hitchcock ganó un Premio de la Academia en 1940 por dirigir *Rebecca*.

(c) Para todo entero positivo n, existe un número primo mayor que n.

(d) La Tierra es el único planeta en el universo que tiene vida.

(e) Compre dos boletos para el concierto de rock de Unhinged Universe para el viernes.

La afirmación (a) es verdadera. Un entero n es *primo* si $n > 1$ y los únicos enteros positivos que dividen a n son 1 y el propio n. La afirmación (a) es otra forma de decir que 7 es primo.

La afirmación (b) es falsa. Aunque *Rebecca* ganó el Premio de la Academia como mejor película en 1940, John Ford ganó el premio al mejor director por *The Grapes of Wrath*. Es sorprendente, pero Alfred Hitchcock nunca ganó un Premio de la Academia como director.

La afirmación (c) es verdadera; es otra forma de decir que existe una infinidad de primos.

La afirmación (d) puede ser verdadera o falsa (pero no ambas), pero nadie podría decidir esto en este momento.

La afirmación (e) no es verdadera ni falsa [en realidad es una orden].

Una afirmación que es verdadera o falsa, pero no ambas, es una **proposición**. Las afirmaciones (a)-(d) son proposiciones, mientras que la afirmación (e) no lo es. En general, una proposición se expresa como una afirmación declarativa (y no como una pregunta, una instrucción, etc.). Las proposiciones son los bloques de construcción básicos para cualquier teoría de la lógica.

Utilizaremos letras minúsculas, como p, q y r, para representar las proposiciones. También utilizaremos la notación

$$p\colon\ 1 + 1 = 3$$

para indicar que p es la proposición $1 + 1 = 3$.

Al hablar o escribir en forma ordinaria, combinamos las proposiciones mediante conectivos como *y* y *o*. Por ejemplo, las proposiciones "Está lloviendo" y "Llevaré mi paraguas" pueden combinarse para formar una única proposición "Está lloviendo y llevaré mi paraguas". A continuación se definen formalmente *y* y *o*.

DEFINICIÓN 1.1.1

Sean p y q proposiciones.

La *conjunción* de p y q, denotada $p \wedge q$, es la proppsición

$$p \text{ y } q.$$

La *disyunción* de p y q, denotada $p \vee q$, es la proposición

$$p \text{ o } q.$$

Las proposiciones (como $p \wedge q$ y $p \vee q$) resultantes de combinar proposiciones son **proposiciones compuestas**.

EJEMPLO 1.1.2

Si

p: $1 + 1 = 3$,
q: Un decenio tiene 10 años,

entonces la conjunción de p y q es

$p \wedge q$: $1 + 1 = 3$ y un decenio tiene 10 años.

La disyunción de p y q es

$p \vee q$: $1 + 1 = 3$ o un decenio tiene 10 años. □

Los valores de verdad de proposiciones como las conjunciones y disyunciones pueden describirse mediante **tablas de verdad**. Una tabla de verdad de una proposición P formada por las proposiciones $p_1, \ldots, p_n$ enumera todas las combinaciones posibles de los valores de verdad para $p_1, \ldots, p_n$, donde V indica verdadero y F falso, de modo que para cada una de estas combinaciones se indica el valor de verdad de P.

DEFINICIÓN 1.1.3

El valor de verdad de la proposición compuesta $p \wedge q$ queda definido mediante la tabla de verdad

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

Observe que en la tabla de verdad de la definición 1.1.3 aparecen las cuatro combinaciones posibles de las asignaciones de verdad para p y q.

La definición 1.1.3 establece que la conjunción $p \wedge q$ es verdadera si p y q son ambas verdaderas; en cualquier otro caso, $p \wedge q$ es falsa.

EJEMPLO 1.1.4

Si

p: $1 + 1 = 3$,
q: Un decenio tiene 10 años,

entonces p es falsa, q es verdadera, y la conjunción

$p \wedge q$: $1 + 1 = 3$ y un decenio tiene 10 años

es falsa. □

EJEMPLO 1.1.5

Si

p: Benny Goodman grabó música clásica,
q: Los Orioles de Baltimore eran los Cafés de San Luis,

entonces p y q son verdaderas. Aunque Benny Goodman es mejor conocido por sus grabaciones de jazz, grabó mucha música clásica (por ejemplo, los conciertos para clarinete de Weber, números 1 y 2, con la Orquesta Sinfónica de Chicago). Los Cafés de San Luis se mudaron a Baltimore en 1954 y cambiaron su nombre por Orioles. La conjunción

$p \wedge q$: Benny Goodman grabó música clásica y los Orioles de Baltimore eran los Cafés de San Luis

es verdadera. □

EJEMPLO 1.1.6

Si

p: $1 + 1 = 3$,
q: Minneápolis es la capital de Minnesota,

entonces p y q son falsas y la conjunción

$p \wedge q$: $1 + 1 = 3$ y Minneápolis es la capital de Minnesota

es falsa. □

DEFINICION 1.1.7

El valor de verdad de la proposición compuesta $p \vee q$ se define mediante la tabla de verdad

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

El *o* en la disyunción $p \vee q$ se utiliza en el sentido inclusivo; es decir, $p \vee q$ es verdadera si p o q o ambas son verdaderas, y $p \vee q$ es falsa sólo si p y q son falsas. Existe también una **o exclusiva** (véase el ejercicio 31), donde p *oex* q es verdadera si p o q es verdadera, pero *no* ocurre que ambas sean verdaderas.

EJEMPLO 1.1.8

Si

p: $1 + 1 = 3$,
q: Un decenio tiene 10 años,

entonces p es falsa, q es verdadera, y la disyunción

$p \vee q$: $1 + 1 = 3$ o un decenio tiene 10 años

es verdadera. □

EJEMPLO 1.1.9

Si

p: Benny Goodman grabó música clásica,
q: Los Orioles de Baltimore eran los Cafés de San Luis,

entonces p y q son ambas verdaderas y la disyunción

$p \vee q$: Benny Goodman grabó música clásica o los Orioles de Baltimore eran los Cafés de San Luis

también es verdadera. □

EJEMPLO 1.1.10

Si

p: $1 + 1 = 3$,
q: Minneápolis es la capital de Minnesota,

entonces p y q son falsas y la disyunción

$p \vee q$: $1 + 1 = 3$ o Minneápolis es la capital de Minnesota

es falsa. □

La última operación sobre una proposición p analizada en esta sección es la **negación** de p.

DEFINICIÓN 1.1.11

La *negación* de p, denotada por $\bar{p}$, es la proposición

no p.

El valor de verdad de la proposición $\bar{p}$ se define mediante la tabla de verdad

p	$\bar{p}$
V	F
F	V

EJEMPLO 1.1.12

Si

p: Cary Grant estelarizó *Rear Window*,

la negación de p es la proposición

$\bar{p}$: No es cierto que Cary Grant haya estelarizado *Rear Window*.

Como p es falsa, $\bar{p}$ es verdadera. (James Stewart interpretó el papel principal en la película *Rear Window*.) La negación se escribiría normalmente:

Cary Grant no estelarizó *Rear Window*. □

EJEMPLO 1.1.13

Sean

p: Blas Pascal inventó varias máquinas calculadoras,

q: La primera computadora digital completamente electrónica fue construida en el siglo XX.

r: π se calculó hasta 1 millón de cifras decimales en 1954.

Representar la proposición

Blas Pascal inventó varias máquinas calculadoras y no es cierto que la primera computadora digital completamente electrónica haya sido construida en el siglo XX; o bien π se calculó hasta 1 millón de cifras decimales en 1954,

en forma simbólica y determinar si es verdadera o falsa.

La proposición puede escribirse en forma simbólica como

$$(p \wedge \bar{q}) \vee r.$$

Primero observamos que p y q son verdaderas y que r es falsa. (No fue sino hasta 1973 que se calculó 1 millón de cifras decimales de π. Posteriormente se han calculado más de dos mil millones de cifras decimales.) Si reemplazamos cada símbolo por su valor de verdad, tenemos que

$$\begin{aligned}(p \wedge \bar{q}) \vee r &= (\mathrm{V} \wedge \overline{\mathrm{V}}) \vee \mathrm{F} \\ &= (\mathrm{V} \wedge \mathrm{F}) \vee \mathrm{F} \\ &= \mathrm{F} \vee \mathrm{F} \\ &= \mathrm{F}.\end{aligned}$$

Por tanto, la proposición dada es falsa. □

Ejercicios

Determine si cada una de las afirmaciones de los ejercicios 1-8 es una proposición. Si la afirmación es una proposición, escriba su negación. (No se le piden los valores de verdad de las afirmaciones que son proposiciones.)

† **1.** $2 + 5 = 19$.

2. Mesero, ¿puede traer las nueces? Es decir, ¿puede servir las nueces a los invitados?
3. Para algún entero positivo n, $19340 = n \cdot 17$.
4. Audrey Meadows fue la "Alicia" original en "The Honeymooners".
5. Pélame una uva.
6. La frase "Hazlo de nuevo, Sam" aparece en la película *Casablanca*.
7. Todo entero par mayor que 4 es la suma de dos primos.
8. La diferencia de dos primos.

Evalúe cada proposición en los ejercicios 9-14 con los valores de verdad

$$p = \mathrm{F}, \quad q = \mathrm{V}, \quad r = \mathrm{F}.$$

9. $p \vee q$

10. $\bar{p} \vee \bar{q}$

11. $\bar{p} \vee q$

12. $\bar{p} \vee \overline{(q \wedge r)}$

13. $\overline{(p \vee q)} \wedge (\bar{p} \vee r)$

14. $(p \vee \bar{r}) \wedge \overline{(q \vee r)} \vee \overline{(r \vee p)}$

Escriba la tabla de verdad para cada una de las proposiciones de los ejercicios 15-22.

15. $p \wedge \bar{q}$

16. $(\bar{p} \vee \bar{q}) \vee p$

17. $(p \vee q) \wedge \bar{p}$

18. $(p \wedge q) \wedge \bar{p}$

19. $(p \wedge q) \vee (\bar{p} \vee q)$

20. $\overline{(p \wedge q)} \vee (r \wedge \bar{p})$

21. $(p \vee q) \wedge (\bar{p} \vee q) \wedge (p \vee \bar{q}) \wedge (\bar{p} \vee \bar{q})$

22. $\overline{(p \wedge q)} \vee (q \vee r)$

† Los ejercicios con números en negritas indican que tienen una sugerencia o una solución al final del libro, en la sección posterior a la Bibliografía.

En los ejercicios 23-25, represente la proposición dada de manera simbólica con

$$p: 5<9, \qquad q: 9<7, \qquad r: 5<7.$$

Determine si cada proposición es verdadera o falsa.

23. $5<9$ y $9<7$.
24. No es cierto que $(5<9$ y $9<7)$.
25. $5<9$ o no es cierto que $(9<7$ y $5<7)$.

En los ejercicios 26-30, describa la expresión simbólica con palabras, utilizando

p: Hoy es lunes.

q: Está lloviendo.

r: Hace calor.

26. $p \vee q$
27. $\bar{p} \wedge (q \vee r)$
28. $\overline{p \vee q} \wedge r$
29. $(p \wedge q) \wedge \overline{(r \vee p)}$
30. $(p \wedge (q \vee r)) \wedge (r \vee (q \vee p))$
31. Proporcione el valor de verdad para el o exclusivo de p y q, donde p *oex* q es verdadera si p o q es verdadera, pero no ambas.
32. En alguna época, la siguiente ley regía en Naperville, Illinois: "No se permitirá que una persona tenga más de tres [3] perros y tres [3] gatos dentro de la ciudad". ¿Violó la ley Charles Marko, quien tenía cinco perros y ningún gato? Explique.

1.2 PROPOSICIONES CONDICIONALES Y EQUIVALENCIA LÓGICA

El decano ha anunciado que

> Si el Departamento de Matemáticas obtiene \$20,000 adicionales, entonces podrá contratar un nuevo miembro. (1.2.1)

La afirmación (1.2.1) establece que, bajo la condición de que el Departamento de Matemáticas obtenga \$20,000 adicionales, entonces se podrá contratar un nuevo miembro. Una proposición como (1.2.1) es una **proposición condicional**.

DEFINICIÓN 1.2.1

Si p y q son proposiciones, la proposición compuesta

$$\text{si } p \text{ entonces } q \tag{1.2.2}$$

es una *proposición condicional* y se denota

$$p \rightarrow q.$$

La proposición p es la *hipótesis* (o *antecedente*) y la proposición q es la *conclusión* (o *consecuente*).

EJEMPLO 1.2.2

Si definimos

p: El Departamento de Matemáticas obtiene \$20,000 adicionales,
q: El Departamento de Matemáticas contrata un nuevo miembro,

entonces la afirmación (1.2.1) tiene la forma (1.2.2). La hipótesis es la afirmación "El Departamento de Matemáticas obtiene \$20,000 adicionales", y la conclusión es la afirmación "El Departamento de Matemáticas contrata un nuevo miembro". □

Algunas afirmaciones que no tienen la forma (1.2.2) pueden formularse como proposiciones condicionales, como en el siguiente ejemplo.

EJEMPLO 1.2.3

Enuncie cada proposición en forma de una proposición condicional (1.2.2).

(a) María será una buena estudiante si estudia mucho.

(b) Juan puede cursar cálculo sólo si está en su segundo, tercer o cuarto año de estudio de licenciatura.

(c) Cuando cantas, me duelen los oídos.

(d) Una condición necesaria para que los Cachorros ganen la Serie Mundial es que consigan un lanzador relevista derecho.

(e) Una condición suficiente para que Rafael visite California es que vaya a Disneylandia.

(a) La hipótesis es la cláusula posterior al condicional *si*, de modo que una formulación equivalente es

> Si María estudia mucho, entonces será una buena estudiante.

(b) La cláusula *sólo si* es la conclusión; es decir,

$$\text{si } p \text{ entonces } q$$

es considerada desde el punto de vista lógico como igual a

$$p \text{ sólo si } q.$$

Una formulación equivalente es

> Si Juan cursa cálculo, entonces está en su segundo, tercer o cuarto año de estudio de licenciatura.

La formulación "si p entonces q" enfatiza la hipótesis, mientras que la formulación "p sólo si q" enfatiza la conclusión: la diferencia es sólo de estilo.

(c) *Cuando* significa lo mismo que *si*, de modo que una formulación equivalente es

> Si cantas, entonces me duelen los oídos.

(d) La conclusión expresa una **condición necesaria**; de este modo, una formulación equivalente es

Si los Cachorros ganan la Serie Mundial, entonces han contratado un lanzador relevista derecho.

(e) La hipótesis expresa una **condición suficiente**, por lo que una formulación equivalente es

Si Rafael va a Disneylandia, entonces estará visitando California. □

Consideremos el problema de asignar un valor de verdad a la afirmación del decano

Si el Departamento de Matemáticas obtiene $20,000 adicionales, entonces podrá contratar un nuevo miembro.

En realidad, la afirmación sólo es de interés cuando la hipótesis "el Departamento de Matemáticas obtiene $20,000 adicionales" es verdadera. Si es cierto que el Departamento de Matemáticas obtiene $20,000 adicionales y también es cierto que el Departamento contrata un nuevo miembro, podemos considerar que la afirmación del decano es verdadera. Por otro lado, si es cierto que el Departamento de Matemáticas obtiene $20,000 adicionales pero es falso que el Departamento contrata un nuevo miembro, podemos considerar que la afirmación del decano es falsa. Cuando la hipótesis es verdadera, el valor de verdad de la proposición condicional, como un todo, depende del valor de verdad de la conclusión. En general, cuando la hipótesis p es verdadera, la proposición condicional $p \rightarrow q$ es verdadera si q es verdadera, y falsa si q es falsa. Si la hipótesis p es falsa, el único punto intuitivamente claro es que el valor de verdad del enunciado condicional $p \rightarrow q$ no debe basarse en el valor de verdad de la conclusión. No podemos considerar que la afirmación del decano sea falsa sólo porque el Departamento de Matemáticas no hubiese conseguido $20,000 adicionales. Sin embargo, la proposición condicional, al igual que cualquier otra proposición, debe tener un valor de verdad, aunque la hipótesis sea falsa. La definición usual establece que $p \rightarrow q$ es verdadera si p es falsa. Este análisis se resume en la siguiente definición.

DEFINICIÓN 1.2.4

El valor de verdad de la proposición condicional $p \rightarrow q$ se define mediante la siguiente tabla de verdad:

p	q	$p \rightarrow q$
V	V	V
V	F	F
F	V	V
F	F	V

EJEMPLO 1.2.5

Sean

$$p\text{: } 1 > 2, \qquad q\text{: } 4 < 8.$$

Entonces p es falsa y q es verdadera. Por tanto,

$p \rightarrow q$ es verdadera, $q \rightarrow p$ es falsa. □

EJEMPLO 1.2.6

Si p es verdadera, q es falsa y r es verdadera, determine el valor de verdad de cada proposición.

(a) $(p \wedge q) \rightarrow r$

(b) $(p \vee q) \rightarrow \overline{r}$

(c) $p \wedge (q \rightarrow r)$

(d) $p \rightarrow (q \rightarrow r)$

Reemplazamos cada símbolo p, q y r por su valor de verdad para obtener el valor de verdad de la proposición:

(a) $(\mathrm{V} \wedge \mathrm{F}) \rightarrow \mathrm{V} = \mathrm{F} \rightarrow \mathrm{V}$ = verdadera

(b) $(\mathrm{V} \vee \mathrm{F}) \rightarrow \overline{\mathrm{V}} = \mathrm{V} \rightarrow \mathrm{F}$ = falsa

(c) $\mathrm{V} \wedge (\mathrm{F} \rightarrow \mathrm{V}) = \mathrm{V} \wedge \mathrm{V}$ = verdadera

(d) $\mathrm{V} \rightarrow (\mathrm{F} \rightarrow \mathrm{V}) = \mathrm{V} \rightarrow \mathrm{V}$ = verdadera □

En el lenguaje común, lo usual es que la hipótesis y la conclusión de una proposición condicional tengan cierta relación; pero en lógica, no es necesario que la hipótesis y la conclusión se refieran al mismo tema. Por ejemplo, en lógica se permiten proposiciones como:

Si $5 < 3$, entonces Nelson Rockefeller fue presidente de Estados Unidos.

La lógica estudia la forma de las proposiciones y las relaciones entre éstas, y no con el tema en cuestión. (De hecho, como la hipótesis es falsa, esta proposición es verdadera. Observe que una proposición condicional verdadera es distinta de una proposición condicional con una conclusión verdadera.)

El ejemplo 1.2.5 muestra que la proposición $p \rightarrow q$ puede ser verdadera aunque la proposición $q \rightarrow p$ sea falsa. La proposición $q \rightarrow p$ es la **recíproca** de la proposición $p \rightarrow q$. Así, una proposición condicional puede ser verdadera y su recíproca ser falsa.

EJEMPLO 1.2.7

Escriba cada proposición condicional de manera simbólica. Escriba el recíproco de cada enunciado en forma simbólica y con palabras. Además, determine el valor de verdad de cada proposición condicional y de su recíproca.

(a) Si $1 < 2$, entonces $3 < 6$.

(b) Si $1 > 2$, entonces $3 < 6$.

(a) Sean

$$p\colon 1 < 2, \qquad q\colon 3 < 6.$$

La afirmación dada puede escribirse de manera simbólica como

$$p \to q.$$

Como p y q son verdaderas, esta proposición es verdadera. La recíproca puede escribirse de manera simbólica como

$$q \to p$$

y con palabras como

Si $3 < 6$, entonces $1 < 2$.

Como p y q son verdaderas, la recíproca $q \to p$ es verdadera.

(b) Sean

$$p\colon 1 > 2, \qquad q\colon 3 < 6.$$

La afirmación dada puede escribirse de manera simbólica como

$$p \to q.$$

Como p es falsa y q es verdadera, esta afirmación es verdadera. La recíproca puede escribirse de manera simbólica como

$$q \to p$$

y con palabras como

Si $3 < 6$, entonces $1 > 2$.

Como q es verdadera y p es falsa, la recíproca $q \to p$ es falsa. □

Otra proposición compuesta útil es

$$p \text{ si y sólo si } q. \tag{1.2.3}$$

Esta afirmación se considera verdadera precisamente cuando p y q tienen los mismos valores de verdad (es decir, p y q son ambas verdaderas o ambas falsas).

DEFINICIÓN 1.2.8

Si p y q son proposiciones, la proposición compuesta

$$p \text{ si y sólo si } q$$

es una *proposición bicondicional* y se denota

$$p \leftrightarrow q.$$

El valor de verdad de la proposición $p \leftrightarrow q$ se define mediante la siguiente tabla de verdad:

p	q	$p \leftrightarrow q$
V	V	V
V	F	F
F	V	F
F	F	V

Una forma alternativa de afirmar "p si y sólo si q" es "p es una condición necesaria y suficiente para q". A veces, "p si y sólo si q" se escribe "p ssi q".

EJEMPLO 1.2.9

La afirmación

$$1 < 5 \text{ si y sólo si } 2 < 8 \tag{1.2.4}$$

puede escribirse de manera simbólica como

$$p \leftrightarrow q$$

si definimos

$$p\colon 1 < 5, \qquad q\colon 2 < 8.$$

Como p y q son verdaderas, la afirmación $p \leftrightarrow q$ es verdadera. □

Una forma alternativa de (1.2.4) es: Una condición necesaria y suficiente para $1 < 5$ es que $2 < 8$.

En algunos casos, es posible que dos proposiciones compuestas tengan los mismos valores de verdad, sin importar los valores de verdad de sus proposiciones constituyentes. Tales proposiciones son **lógicamente equivalentes**.

DEFINICIÓN 1.2.10

Supongamos que las proposiciones compuestas P y Q están formadas por las proposiciones $p_1, \ldots, p_n$. Decimos que P y Q son *lógicamente equivalentes* y escribimos

$$P \equiv Q,$$

siempre que dados cualesquiera valores de verdad de $p_1, \ldots, p_n$, P y Q sean ambas verdaderas o ambas falsas.

EJEMPLO 1.2.11 *Leyes de De Morgan para la lógica*

Verificaremos la primera de las **leyes de De Morgan**

$$\overline{p \vee q} \equiv \bar{p} \wedge \bar{q}\,, \quad \overline{p \wedge q} \equiv \bar{p} \vee \bar{q}$$

y dejaremos la segunda como ejercicio (véase el ejercicio 44).

Al escribir las tablas de verdad para $P = \overline{p \vee q}$ y $Q = \bar{p} \wedge \bar{q}$, podemos verificar que dados cualesquiera valores de verdad de p y q, P y Q son ambas verdaderas o ambas falsas:

p	q	$\overline{p \vee q}$	$\bar{p} \wedge \bar{q}$
V	V	F	F
V	F	F	F
F	V	F	F
F	F	V	V

Por tanto, P y Q son lógicamente equivalentes. □

Nuestro ejemplo siguiente proporciona una forma lógicamente equivalente de la negación de $p \rightarrow q$.

EJEMPLO 1.2.12

Muestre que la negación de $p \rightarrow q$ es lógicamente equivalente a $p \wedge \bar{q}$.

Debemos mostrar que

$$\overline{p \rightarrow q} \equiv p \wedge \bar{q}\,.$$

Al escribir las tablas de verdad para $P = \overline{p \rightarrow q}$ y $Q = p \wedge \bar{q}$, verificamos que dados cualesquiera valores de verdad de p y q, P y Q son ambas verdaderas o ambas falsas:

p	q	$\overline{p \rightarrow q}$	$p \wedge \bar{q}$
V	V	F	F
V	F	V	V
F	V	F	F
F	F	F	F

Por tanto, P y Q son lógicamente equivalentes. □

Ahora mostraremos que de acuerdo con nuestras definiciones, $p \leftrightarrow q$ es lógicamente equivalente a $p \rightarrow q$ y $q \rightarrow p$.

EJEMPLO 1.2.13

La tabla de verdad muestra que

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)\,.$$

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$
V	V	V	V	V	V
V	F	F	F	V	F
F	V	F	V	F	F
F	F	V	V	V	V

□

Concluimos esta sección definiendo la **contrapositiva** de una proposición condicional. Veremos (teorema 1.2.16) que la contrapositiva es una forma alternativa, lógicamente equivalente, de la proposición condicional. El ejercicio 45 muestra otra forma lógicamente equivalente de la proposición condicional.

DEFINICION 1.2.14

La *contrapositiva* (o *transposición*) de una proposición condicional $p \rightarrow q$ es la proposición $\bar{q} \rightarrow \bar{p}$.

Observe la diferencia entre la contrapositiva y la recíproca. La recíproca de una proposición condicional solamente cambia los papeles de p y q, mientras que la contrapositiva cambia los papeles de p y q y niega cada una de ellas.

EJEMPLO 1.2.15

Escriba la proposición

Si $1 < 4$, entonces $5 > 8$

en forma simbólica. Escriba la recíproca y la contrapositiva de manera simbólica y con palabras. Determine el valor de verdad de cada proposición.

Si definimos

$$p\colon 1 < 4, \qquad q\colon 5 > 8,$$

entonces podemos escribir la proposición de manera simbólica como

$$p \rightarrow q.$$

La recíproca es

$$q \rightarrow p,$$

o, con palabras,

Si $5 > 8$, entonces $1 < 4$.

La contrapositiva es

$$\bar{q} \rightarrow \bar{p},$$

o, con palabras,

Si 5 no es mayor que 8, entonces 1 no es menor que 4.

Vemos que $p \to q$ es falsa, $q \to p$ es verdadera y $\bar{q} \to \bar{p}$ es falsa. □

Un hecho importante es que una proposición condicional y su contrapositiva son lógicamente equivalentes.

TEOREMA 1.2.16

La proposición condicional $p \to q$ y su contrapositiva $\bar{q} \to \bar{p}$ son lógicamente equivalentes.

Demostración. La tabla de verdad

p	q	$p \to q$	$\bar{q} \to \bar{p}$
V	V	V	V
V	F	F	F
F	V	V	V
F	F	V	V

muestra que $p \to q$ y $\bar{q} \to \bar{p}$ son lógicamente equivalentes. ■

Ejercicios

En los ejercicios 1-7, enuncie cada proposición en la forma (1.2.2) de una proposición condicional.

1. José aprobará el examen de matemáticas discretas si estudia mucho.
2. Rosa podrá graduarse si tiene 160 créditos.
3. Una condición necesaria para que Fernando compre una computadora es que tenga $2000.
4. Una condición suficiente para que Karina lleve el curso de algoritmos es que apruebe matemáticas discretas.
5. Cuando se construyan mejores automóviles, Buick los construirá.
6. El auditorio se dormirá si el presidente imparte la conferencia.
7. El programa es legible sólo si está bien estructurado.
8. Escriba la recíproca de cada proposición en los ejercicios 1-7.
9. Escriba la contrapositiva de cada proposición en los ejercicios 1-7.

Suponiendo que p y r son falsas y que q y s son verdaderas, determine los valores de verdad de cada proposición en los ejercicios 10-17.

10. $p \to q$
11. $\bar{p} \to \bar{q}$

12. $\overline{p \to q}$
13. $(p \to q) \wedge (q \to r)$
14. $(p \to q) \to r$
15. $p \to (q \to r)$
16. $(s \to (p \wedge \bar{r})) \wedge ((p \to (r \vee q)) \wedge s)$
17. $((p \wedge \bar{q}) \to (q \wedge r)) \to (s \vee \bar{q})$

En los ejercicios 18-21, represente la afirmación dada de manera simbólica, haciendo

p: $4 < 2$, q: $7 < 10$, r: $6 < 6$.

18. Si $4 < 2$, entonces $7 < 10$.
19. Si ($4 < 2$ y $6 < 6$), entonces $7 < 10$.
20. Si no es cierto que ($6 < 6$ y 7 no es menor que 10), entonces $6 < 6$.
21. $7 < 10$ si y sólo si ($4 < 2$ y 6 no es menor que 6).

En los ejercicios 22-27, formule la expresión simbólica con palabras utilizando

p: Hoy es lunes,

q: Está lloviendo,

r: Hace calor.

22. $p \to q$
23. $\bar{q} \to (r \wedge p)$
24. $\bar{p} \to (q \vee r)$
25. $\overline{(p \vee q)} \leftrightarrow r$
26. $(p \wedge (q \vee r)) \to (r \vee (q \wedge p))$
27. $(p \vee (\bar{p} \wedge \overline{(q \vee r)})) \to (p \vee \overline{(r \vee q)})$

En los ejercicios 28-31, escriba cada proposición condicional en forma simbólica. Escriba la recíproca y la contrapositiva de cada afirmación de manera simbólica y con palabras. Además, determine el valor de verdad de cada proposición condicional, su recíproca y su contrapositiva.

28. Si $4 < 6$, entonces $9 > 12$.
29. Si $4 > 6$, entonces $9 > 12$.
30. $|1| < 3$ si $-3 < 1 < 3$.
31. $|4| < 3$ si $-3 < 4 < 3$.

Para cada par de proposiciones P y Q en los ejercicios 32-41, indique si $P \equiv Q$.

32. $P = p, Q = p \vee q$
33. $P = p \wedge q, Q = \bar{p} \vee \bar{q}$
34. $P = p \to q, Q = \bar{p} \vee q$
35. $P = p \wedge (\bar{q} \vee r), Q = p \vee (q \wedge \bar{r})$
36. $P = p \wedge (q \vee r), Q = (p \vee q) \wedge (p \vee r)$
37. $P = p \to q, Q = \bar{q} \to \bar{p}$

38. $P = p \to q, Q = q \leftrightarrow p$

39. $P = (p \to q) \wedge (q \to r), Q = p \to r$

40. $P = (p \to q) \to r, Q = p \to (q \to r)$

41. $P = (s \to (p \wedge \bar{r})) \wedge ((p \to (r \vee q)) \wedge s), Q = p \vee t$

42. Defina la tabla de verdad para *impl* como

p	q	p *impl* q
V	V	V
V	F	F
F	V	F
F	F	V

Muestre que

$$p \textit{ impl } q \equiv q \textit{ impl } p.$$

43. Defina la tabla de verdad para *imp2* como

p	q	p *imp2* q
V	V	V
V	F	F
F	V	V
F	F	F

(a) Muestre que

$$(p \textit{ imp2 } q) \wedge (q \textit{ imp2 } p) \not\equiv p \leftrightarrow q. \tag{1.2.5}$$

(b) Muestre que (1.2.5) sigue siendo verdadera si modificamos *imp2*, de modo que si p es falsa y q es verdadera, entonces p *imp2* q es falsa.

44. Verifique la segunda ley de De Morgan, $\overline{p \wedge q} \equiv \bar{p} \vee \bar{q}$.

45. Muestre que $(p \to q) \equiv (\bar{p} \vee q)$.

1.3 CUANTIFICADORES

La lógica de las secciones 1.1 y 1.2 que trata con proposiciones, no puede describir la mayoría de las que se manejan en matemáticas y en computación.

Por ejemplo, consideremos la proposición:

$$p\text{: } n \text{ es un entero impar.} \tag{1.3.1}$$

Una proposición es una afirmación que es verdadera o falsa. La afirmación p no es una proposición, ya que el hecho de que p sea verdadera o falsa depende del valor de n. Por ejemplo, p es verdadera si $n = 103$ y falsa si $n = 8$. Como muchas afirmaciones en matemáticas y computación utilizan variables, debemos ampliar el sistema lógico para incluir estas afirmaciones.

DEFINICION 1.3.1

Sea $P(x)$ un enunciado que contiene la variable x y sea D un conjunto. P es una *función proposicional* (con respecto de D) si para cada x en D, $P(x)$ es una proposición. D es el *dominio de discurso* de P.

EJEMPLO 1.3.2

Sea $P(n)$ la afirmación

$$n \text{ es un entero impar}$$

y sea D el conjunto de enteros positivos. Entonces P es una función proposicional con dominio de discurso D ya que para cada n en D, $P(n)$ es una proposición (es decir, para cada n en D, $P(n)$ es verdadera o falsa, pero no ambas). Por ejemplo, si $n = 1$, obtenemos la proposición

$$1 \text{ es un entero impar}$$

(que es verdadera). Si $n = 2$, obtenemos la proposición

$$2 \text{ es un entero impar}$$

(que es falsa). □

Una función proposicional P, por sí misma, no es verdadera ni falsa. Sin embargo, para toda x en su dominio de discurso, $P(x)$ es una proposición y, por tanto, es verdadera o falsa. Podemos pensar que una función proposicional define una clase de proposiciones, una por cada elemento de su dominio de discurso. Por ejemplo, si P es una función proposicional con dominio de discurso igual al conjunto de enteros positivos, obtenemos la clase de proposiciones

$$P(1), P(2), \ldots.$$

Cada $P(1), P(2), \ldots$ es verdadera o falsa.

EJEMPLO 1.3.3

Las siguientes son funciones proposicionales.

(a) $n^2 + 2n$ es un entero impar (dominio de discurso = conjunto de enteros positivos).

(b) $x^2 - x - 6 = 0$ (dominio de discurso = conjunto de números reales).

(c) El jugador de béisbol bateó arriba de .300 en 1974 (dominio de discurso = conjunto de jugadores de béisbol).

(d) El restaurante se catalogó como de "dos estrellas" en la revista *Chicago* (dominio de discurso = restaurantes catalogados en la revista *Chicago*).

En la afirmación (a), obtenemos una proposición para cada entero positivo n; por tanto, la afirmación (a) es una función proposicional.

De igual manera, en la afirmación (b), obtenemos una proposición para cada número real x; por tanto, la afirmación (b) es una función proposicional.

Podemos considerar que la variable en la afirmación (c) es "jugador de béisbol". Al sustituir un jugador de béisbol particular en vez de la variable "jugador de béisbol", la afirmación es una proposición. Por ejemplo, si sustituimos "Willie Stargell" en vez de "jugador de béisbol", la afirmación (c) es

Willie Stargell bateó arriba de .300 en 1974

que es verdadera. Si sustituimos "Carlton Fisk" por "jugador de béisbol" la afirmación (c) es

Carlton Fisk bateó arriba de .300 en 1974

que es falsa. Así, la afirmación (c) es una función proposicional.

La afirmación (d) tiene una forma similar a la afirmación (c): en este caso, la variable es "restaurante". Al sustituir un restaurante catalogado en la revista *Chicago* en vez de la variable "restaurante", la afirmación es una proposición. Por ejemplo, si sustituimos "Yugo Inn" en vez de "restaurante", la afirmación (d) es

Yugo Inn se catalogó como de "dos estrellas" en la revista *Chicago*

que es falsa. Si sustituimos "Le Français" en vez de "restaurante", la afirmación (d) es

Le Français se catalogó como de "dos estrellas" en la revista *Chicago*

que es verdadera. Así, la afirmación (d) es una función proposicional. □

Muchas afirmaciones en matemáticas y computación utilizan términos como "para todo" y "para algún". Por ejemplo, en matemáticas tenemos el teorema:

Para todo triángulo T, la suma de los ángulos de T es igual a 180°.

En computación, tenemos el teorema:

Para algún programa P, la salida de P es el propio P.

Ahora ampliaremos el sistema lógico de las secciones 1.1 y 1.2 para poder utilizar las afirmaciones que incluyan "para todo" y "para algún".

DEFINICION 1.3.4

Sea P una función proposicional con dominio de discurso D. La afirmación

para toda x, $P(x)$

es una *afirmación cuantificada universalmente*. El símbolo $\forall$ significa "para toda". Así, la afirmación

para toda x, $P(x)$

puede escribirse como

$$\forall x, P(x).$$

El símbolo $\forall$ es un *cuantificador universal*.

La afirmación

para toda x, $P(x)$

es verdadera si $P(x)$ es verdadera para toda x en D. La afirmación

para toda x, $P(x)$

es falsa si $P(x)$ es falsa para al menos una x en D.

La afirmación

para alguna x, $P(x)$

es una *afirmación cuantificada existencialmente*. El símbolo $\exists$ significa "para alguna". Así, la afirmación

para alguna x, $P(x)$

puede escribirse como

$$\exists x, P(x).$$

El símbolo $\exists$ es un *cuantificador existencial*.

La afirmación

para alguna x, $P(x)$

es verdadera si $P(x)$ es verdadera para al menos una x en D. La afirmación

para alguna x, $P(x)$

es falsa si $P(x)$ es falsa para toda x en D.

La variable x en una función proposicional $P(x)$ es una *variable libre*. (La idea es que x tiene la "libertad" de recorrer el dominio de discurso.) La variable x en la afirmación cuantificada universalmente

$$\forall x, P(x) \tag{1.3.2}$$

o en la afirmación cuantificada existencialmente

$$\exists x, P(x) \tag{1.3.3}$$

es una *variable acotada*. (La idea es que x queda "acotada" por el cuantificador $\forall$ o $\exists$.) Anteriormente señalamos que una función proposicional no tiene un valor de verdad. Por otro lado, la definición 1.3.4 asigna un valor de verdad a las afirmaciones cuantificadas (1.3.2) y (1.3.3). En resumen, una afirmación con variables libres no cuantificadas no es una proposición y una afirmación sin variables libres (variables cuantificadas) es una proposición.

La afirmación

para cada x, $P(x)$

también puede escribirse como

para toda x, $P(x)$

y

para cualquier x, $P(x)$.

El símbolo $\forall$ se lee "para cada", "para toda" o "para cualquier".

La afirmación

$$\text{para alguna } x,\ P(x)$$

también puede escribirse como

$$\text{para al menos una } x,\ P(x)$$

o como

$$\text{existe } x \text{ tal que } P(x).$$

El símbolo $\exists$ se lee "para algún", "para al menos un" o "existe".

A veces, para especificar el dominio de discurso D, escribimos una afirmación cuantificada universalmente como

$$\text{para toda } x \text{ en } D,\ P(x),$$

y una afirmación cuantificada existencialmente como

$$\text{para alguna } x \text{ en } D,\ P(x).$$

EJEMPLO 1.3.5

La afirmación

$$\text{para cada número real } x,\ x^2 \geq 0$$

es una afirmación cuantificada universalmente. El dominio de discurso es el conjunto de números reales. La afirmación es verdadera, pues *para cada* número real x, es cierto que el cuadrado de x es positivo o cero. □

EJEMPLO 1.3.6

La afirmación cuantificada universalmente

$$\text{para cada número real } x, \text{ si } x > 1, \text{ entonces } x + 1 > 1$$

es verdadera. Esta vez debemos verificar que la afirmación

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

sea verdadera *para cada* número real x.

Sea x cualquier número real. Es cierto que para cualquier número real x, $x \leq 1$ o $x > 1$. Si $x \leq 1$, la proposición condicional

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

es verdadera, pues la hipótesis $x > 1$ es falsa. (Recuerde que cuando la hipótesis es falsa, la proposición condicional es verdadera, sin importar que la conclusión sea verdadera o falsa.)

Ahora, supongamos que $x > 1$. Sin importar el valor específico de x, $x + 1 > x$. Como

$$x + 1 > x \quad \text{y} \quad x > 1,$$

concluimos que $x + 1 > 1$, de modo que la conclusión es verdadera. Si $x > 1$, la hipótesis y la conclusión son verdaderas, por lo que la proposición condicional

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

es verdadera.

Hemos mostrado que para cada número real x, la proposición

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

es verdadera. Por tanto, la afirmación cuantificada universalmente

$$\text{para cada número real } x, \text{ si } x > 1, \text{ entonces } x + 1 > 1$$

es verdadera. □

El ejemplo 1.3.6 proporciona una motivación más para definir una proposición condicional $p \rightarrow q$ como verdadera cuando p es falsa. Para que la afirmación cuantificada universalmente

$$\text{para cada número real } x, \text{ si } x > 1, \text{ entonces } x + 1 > 1$$

sea verdadera, debe ocurrir que la proposición condicional

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

sea verdadera *sin importar el valor de x*. En particular, la proposición

$$\text{si } x > 1, \text{ entonces } x + 1 > 1$$

debe ser verdadera si $x > 1$ es falsa.

De acuerdo con la definición 1.3.4, la afirmación cuantificada universalmente

$$\text{para cada } x,\ P(x)$$

es falsa si *para al menos una x* en el dominio de discurso, la proposición $P(x)$ es falsa. Un valor x en el dominio de discurso que haga falsa a $P(x)$ es un **contraejemplo** a la afirmación

$$\text{para cada } x,\ P(x).$$

EJEMPLO 1.3.7

La afirmación cuantificada universalmente

$$\text{para cada número real } x,\ x^2 - 1 > 0$$

es falsa, pues, si $x = 1$, la proposición

$$1^2 - 1 > 0$$

es falsa. El valor 1 es un contraejemplo a la afirmación

$$\text{para cada número real } x,\ x^2 - 1 > 0.$$

□

Para mostrar que la afirmación cuantificada universalmente

$$\text{para cada } x,\ P(x)$$

es *falsa*, es suficiente determinar *un* valor x en el dominio de discurso para el cual la proposición $P(x)$ sea falsa. El método para refutar la afirmación

$$\text{para cada } x,\ P(x)$$

es un poco diferente del método utilizado para demostrar que la afirmación es verdadera. Para probar que

$$\text{para cada } x,\ P(x)$$

es *verdadera*, hay que examinar *todos* los valores de x en el dominio de discurso y mostrar que para cada x, $P(x)$ es verdadera.

EJEMPLO 1.3.8

La afirmación cuantificada universalmente

$$\text{para cada entero positivo } n, \text{ si } n \text{ es par, entonces } n^2 + n + 19 \text{ es primo}$$

es falsa; obtenemos un contraejemplo al considerar $n = 38$. La proposición condicional

$$\text{si } 38 \text{ es par, entonces } 38^2 + 38 + 19 \text{ es primo}$$

es falsa, pues la hipótesis

$$38 \text{ es par}$$

es verdadera, pero la conclusión

$$38^2 + 38 + 19 \text{ es primo}$$

es falsa. $38^2 + 38 + 19$ no es primo pues puede factorizarse como sigue:

$$38^2 + 38 + 19 = 38 \cdot 38 + 38 + 19 = 19(2 \cdot 38 + 2 + 1) = 19 \cdot 79. \qquad \square$$

Ahora analizaremos las afirmaciones cuantificadas existencialmente. Según la definición 1.3.4, la afirmación cuantificada existencialmente

$$\text{para alguna } x \text{ en } D,\ P(x)$$

es verdadera si $P(x)$ es verdadera para *al menos una* x en D. Si $P(x)$ es verdadera para algunos valores de x, podría ocurrir que $P(x)$ sea falsa para otros valores de x.

EJEMPLO 1.3.9

La afirmación cuantificada existencialmente

$$\text{para algún número real } x,\quad \frac{x}{x^2+1} = \frac{2}{5}$$

es verdadera, pues es posible determinar *al menos un* número real x para el cual la proposición

$$\frac{x}{x^2+1} = \frac{2}{5}$$

sea verdadera. Por ejemplo, si $x = 2$, obtenemos la proposición verdadera

$$\frac{2}{2^2+1} = \frac{2}{5}.$$

No es cierto que *todo* valor de x produzca una proposición verdadera. Por ejemplo, la proposición

$$\frac{1}{1^2+1} = \frac{2}{5}$$

es falsa. $\square$

EJEMPLO 1.3.10

La afirmación cuantificada existencialmente

$$\text{para algún entero positivo } n, \text{ si } n \text{ es primo, entonces } n+1, n+2, n+3 \text{ y } n+4 \text{ no son primos}$$

es verdadera, pues podemos determinar *al menos un* entero n que haga la proposición condicional

$$\text{si } n \text{ es primo, entonces } n+1, n+2, n+3 \text{ y } n+4 \text{ no son primos}$$

verdadera. Por ejemplo, si $n = 23$, obtenemos la proposición verdadera

$$\text{si } 23 \text{ es primo, entonces } 24, 25, 26 \text{ y } 27 \text{ no son primos.}$$

(Esta proposición condicional es verdadera pues tanto la hipótesis "si 23 es primo" como la conclusión "24, 25, 26 y 27 no son primos" son verdaderas.) Algunos valores de n hacen que la proposición condicional sea verdadera (por ejemplo, $n = 23$, $n = 4$, $n = 47$), mientras que otras hacen que sea falsa (por ejemplo, $n = 2$, $n = 101$). El hecho es que hemos determinado *un* valor que hace verdadera a la proposición condicional

$$\text{si } n \text{ es primo, entonces } n+1, n+2, n+3 \text{ y } n+4 \text{ no son primos.}$$

Por esta razón, la afirmación cuantificada universalmente

$$\text{para algún entero positivo } n, \text{ si } n \text{ es primo, entonces } n+1, n+2, n+3 \text{ y } n+4 \text{ no son primos}$$

es verdadera. $\square$

Según la definición 1.3.4, la afirmación cuantificada existencialmente

$$\text{para alguna } x,\ P(x)$$

es falsa si para toda x en el dominio de discurso, la proposición $P(x)$ es falsa.

EJEMPLO 1.3.11

Para verificar que la afirmación cuantificada existencialmente

$$\text{para algún número real } x,\quad \frac{1}{x^2+1} > 1$$

es falsa, debemos mostrar que

$$\frac{1}{x^2+1} > 1$$

es falsa para cada número real x. Ahora,

$$\frac{1}{x^2+1} > 1$$

es falsa precisamente cuando

$$\frac{1}{x^2+1} \leq 1$$

es verdadera. Así, debemos mostrar que

$$\frac{1}{x^2+1} \leq 1$$

es verdadera para cada número real x. Para esto, sea x cualquier número real. Como $0 \leq x^2$, podemos sumar 1 a ambos lados de esta desigualdad para obtener $1 \leq x^2 + 1$. Si dividimos ambos lados de esta última desigualdad entre $x^2 + 1$, obtenemos

$$\frac{1}{x^2+1} \leq 1.$$

Por tanto, la afirmación

$$\frac{1}{x^2+1} \leq 1$$

es verdadera para cada número real x. Así, la afirmación

$$\frac{1}{x^2+1} > 1$$

es falsa para cada número real x. Hemos mostrado que la afirmación cuantificada existencialmente

$$\text{para alguna } x,\quad \frac{1}{x^2+1} > 1$$

es falsa. □

En el ejemplo 1.3.11, mostramos que una afirmación cuantificada existencialmente era falsa demostrando que una afirmación cuantificada universalmente relacionada con aquella era verdadera. El siguiente teorema precisa esta relación. El teorema generaliza las leyes de De Morgan para la lógica (ejemplo 1.2.11).

TEOREMA 1.3.12 *Leyes de De Morgan generalizadas para la lógica*

Si P es una función proposicional, cada par de proposiciones en (a) y (b) tiene el mismo valor de verdad (es decir, ambas son verdaderas o falsas).

(a) $\overline{\forall x, P(x)}\ ;\ \exists x, \overline{P(x)}$

(b) $\overline{\exists x, P(x)}\ ;\ \forall x, \overline{P(x)}$

Demostración. Sólo demostraremos el inciso (a) y dejaremos la demostración del inciso (b) al lector (ejercicio 50).

Supongamos que la proposición $\overline{\forall x, P(x)}$ es verdadera. Entonces la proposición $\forall x, P(x)$ es falsa. Por la definición 1.3.4, la proposición $\forall x, P(x)$ es falsa precisamente cuando $P(x)$ es falsa para al menos una x en el dominio de discurso. Pero si $P(x)$ es falsa para al menos una x en el dominio de discurso, $\overline{P(x)}$ es verdadera para al menos una x en el dominio de discurso. De nuevo, por la definición 1.3.4, cuando $\overline{P(x)}$ es verdadera para al menos una x en el dominio de discurso, la proposición $\exists x, \overline{P(x)}$ es verdadera. Así, si la proposición $\overline{\forall x, P(x)}$ es verdadera, la proposición $\exists x, \overline{P(x)}$ es verdadera. De manera análoga, si la proposición $\overline{\forall x, P(x)}$ es falsa, la proposición $\exists x, \overline{P(x)}$ es falsa.

Por tanto, las dos proposiciones en el inciso (a) tienen siempre el mismo valor de verdad. ■

EJEMPLO 1.3.13

Sea $P(x)$ la afirmación

$$\frac{1}{x^2+1} > 1.$$

En el ejemplo 1.3.11 mostramos que

$$\text{para algún número real } x,\ P(x)$$

es falsa verificando que

$$\text{para cada número real } x,\ \overline{P(x)} \tag{1.3.4}$$

es verdadera.

La técnica puede justificarse apelando al teorema 1.3.12. Después de demostrar que la proposición (1.3.4) es verdadera, podemos negarla y concluir que

$$\overline{\text{para cada número real } x,\ \overline{P(x)}}$$

es falsa. Por el inciso (a) del teorema 1.3.12,

$$\text{para algún número real } x,\ \overline{\overline{P(x)}}$$

o bien, en forma equivalente,

$$\text{para algún número real } x,\ P(x)$$

también es falsa. □

Una proposición cuantificada universalmente generaliza la proposición compuesta

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \tag{1.3.5}$$

en el sentido de que las proposiciones individuales $P_1, P_2, \ldots, P_n$ se reemplazan mediante una familia arbitraria $P(x)$, donde x es un elemento del dominio de discurso, y (1.3.5) se reemplaza mediante

$$\text{para cada } x,\ P(x). \tag{1.3.6}$$

La proposición (1.3.5) es verdadera si y sólo si P_i es verdadera para cada $i = 1, \ldots, n$. El valor de verdad de la proposición (1.3.6) se define de manera análoga: (1.3.6) es verdadera si y sólo si $P(x)$ es verdadera para cada x en el dominio de discurso.

De manera similar, una proposición cuantificada existencialmente generaliza la proposición compuesta

$$P_1 \vee P_2 \vee \cdots \vee P_n \tag{1.3.7}$$

en el sentido de que las proposiciones individuales $P_1, P_2, \ldots, P_n$ se reemplazan mediante una familia arbitraria $P(x)$, donde x es un elemento del dominio de discurso, y (1.3.7) se reemplaza mediante

$$\text{para alguna } x,\ P(x).$$

Estas observaciones explican la forma en que el teorema 1.3.12 generaliza las leyes de De Morgan para la lógica (ejemplo 1.2.11). Recuerde que la primera ley de De Morgan para la lógica establece que las proposiciones

$$\overline{P_1 \vee P_2 \vee \cdots \vee P_n} \quad \text{y} \quad \overline{P}_1 \wedge \overline{P}_2 \wedge \cdots \wedge \overline{P}_n$$

tienen los mismos valores de verdad. En el inciso (b) del teorema 1.3.12,

$$\overline{P}_1 \wedge \overline{P}_2 \wedge \cdots \wedge \overline{P}_n$$

se reemplaza por

$$\forall x,\ \overline{P(x)}$$

y

$$\overline{P_1 \vee P_2 \vee \cdots \vee P_n}$$

se reemplaza por

$$\overline{\exists x,\ P(x)}.$$

EJEMPLO 1.3.14

Ciertas frases pueden tener más de una interpretación. Como ejemplo, consideremos la famosa cita de Shakespeare

Todo lo que brilla no es oro. (All that glitters is not gold.)

Una posible interpretación de esta cita es: Nada que brille es oro (es decir, un objeto de oro nunca brilla). Sin embargo, seguramente esto no es lo que quiso decir Shakespeare. La interpretación correcta es: Algo que brilla no es oro.

Si $P(x)$ es la función proposicional "x brilla" y $Q(x)$ es la función proposicional "x es oro", la primera interpretación es

$$\text{para toda } x,\ P(x) \rightarrow \overline{Q(x)}, \tag{1.3.8}$$

y la segunda interpretación es

$$\text{para alguna } x,\ P(x) \wedge \overline{Q(x)}.$$

Con el resultado del ejemplo 1.2.12, se ve que los valores de verdad de

$$\text{para alguna } x,\ P(x) \wedge \overline{Q(x)}$$

y

$$\text{para alguna } x,\ \overline{P(x) \rightarrow Q(x)}$$

son los mismos. Por el teorema 1.3.12, los valores de verdad de

$$\text{para alguna } x,\ \overline{P(x) \rightarrow Q(x)}$$

y

$$\overline{\text{para toda } x,\ P(x) \rightarrow Q(x)}$$

son los mismos. Así, una forma equivalente de representar la segunda interpretación es

$$\overline{\text{para toda } x,\ P(x) \rightarrow Q(x)}. \tag{1.3.9}$$

Al comparar (1.3.8) y (1.3.9), se ve que la ambigüedad surge del hecho de que la negación se aplique a $Q(x)$ (la primera interpretación) o bien a toda la afirmación

$$\text{para toda } x,\ P(x) \rightarrow Q(x)$$

(la segunda interpretación). La interpretación correcta de la afirmación

Todo lo que brilla no es oro

surge de negar toda la afirmación.

En las afirmaciones positivas, "cualquier", "todo" y "cada uno" tienen el mismo significado. En las afirmaciones negativas, la situación cambia:

No cualquier C_1 es C_2

No toda C_1 es C_2

No cada C_1 es C_2

se considera que estas afirmaciones tienen el mismo significado que

Alguna C_1 no es C_2

mientras que

Ninguna C_1 es C_2

significa que

No hay C_1 que sea C_2.

Véanse otros ejemplos en los ejercicios 47 y 48. □

Nuestro siguiente ejemplo muestra la forma de combinar cuantificadores universales y existenciales dentro de una única afirmación, y también para cuantificar más de una variable.

EJEMPLO 1.3.15

Suponga que el dominio de discurso es el conjunto de números reales. Considere la afirmación

$$\text{para cada } x, \text{ para alguna } y, x + y = 0.$$

El significado de esta afirmación es que para cualquier x, existe al menos una y, que puede depender de la elección de x, tal que $x + y = 0$. Podemos mostrar que la afirmación

$$\text{para cada } x, \text{ para alguna } y, x + y = 0$$

es verdadera. Para cualquier x, podemos encontrar al menos una y, a saber, $y = -x$, tal que $x + y = 0$ es verdadera. □

Supongamos que modificamos la afirmación del ejemplo 1.3.15 de la manera siguiente:

$$\text{para alguna } y, \text{ para cada } x, x + y = 0.$$

Si esta afirmación fuese cierta, entonces es posible elegir algún valor de y tal que la afirmación

$$\text{para cada } x, x + y = 0$$

sea verdadera. Sin embargo, podemos demostrar que esta última afirmación no es verdadera con un contraejemplo. Es decir, podemos considerar $x = 1 - y$. Entonces obtenemos la afirmación falsa

$$1 - y + y = 0.$$

Por tanto, la afirmación

$$\text{para alguna } y, \text{ para cada } x, x + y = 0$$

es falsa.

EJEMPLO 1.3.16

Sea $P(x, y)$ la afirmación

$$\text{si } x^2 < y^2, \text{ entonces } x < y.$$

El dominio de discurso es el conjunto de números reales.

La afirmación

$$\text{para cada } x, \text{ para cada } y, P(x, y)$$

es falsa. Un contraejemplo es $x = 1$, $y = -2$. En este caso, obtenemos la proposición falsa

$$\text{si } 1^2 < (-2)^2, \text{ entonces } 1 < -2.$$

La afirmación

$$\text{para cada } x, \text{ para alguna } y, P(x, y)$$

es verdadera. Mostraremos que para cada x, la proposición

$$\text{para alguna } y, \text{ si } x^2 < y^2, \text{ entonces } x < y$$

es verdadera exhibiendo un valor de y para el cual

$$\text{si } x^2 < y^2, \text{ entonces } x < y$$

sea verdadera. De hecho, si $y = 0$, obtenemos la proposición verdadera

$$\text{si } x^2 < 0, \text{ entonces } x < 0.$$

(La proposición condicional es verdadera, pues la hipótesis $x^2 < 0$ es falsa.)

La afirmación

$$\text{para cada } y, \text{ para alguna } x, P(x, y)$$

es verdadera. Mostraremos que para cada y, la proposición

$$\text{para alguna } x, \text{ si } x^2 < y^2, \text{ entonces } x < y$$

es verdadera exhibiendo un valor de x para el cual

$$\text{si } x^2 < y^2, \text{ entonces } x < y$$

sea verdadera. De hecho, si $x = |y| + 1$, obtenemos la proposición verdadera

$$\text{si } (|y| + 1)^2 < y^2, \text{ entonces } |y| + 1 < y.$$

(La proposición condicional es verdadera, pues la hipótesis es falsa.) □

Resumimos estas reglas para demostrar o refutar las afirmaciones cuantificadas en forma universal o existencial:

- Para demostrar que la afirmación cuantificada universalmente

  $$\text{para cada } x, P(x)$$

  es verdadera, hay que mostrar que para *cada* x en el dominio de discurso, la proposición $P(x)$ es verdadera. El hecho de mostrar que $P(x)$ es verdadera para un valor *particular* x *no* demuestra que

  $$\text{para cada } x, P(x)$$

  sea verdadera.
- Para demostrar que la afirmación cuantificada existencialmente

  $$\text{para alguna } x, P(x)$$

  es verdadera, hay que determinar *un* valor x en el dominio de discurso para el cual $P(x)$ sea verdadera. Basta *un* valor.
- Para demostrar que la afirmación cuantificada universalmente

  $$\text{para cada } x, P(x)$$

  es falsa, hay que determinar *un* valor de x (un contraejemplo) en el dominio de discurso para el cual $P(x)$ sea falsa.
- Para demostrar que la afirmación cuantificada existencialmente

  $$\text{para alguna } x, P(x)$$

  es falsa, hay que mostrar que para *cada* x en el dominio de discurso, la proposición $P(x)$ es falsa. El hecho de mostrar que $P(x)$ es falsa para un valor *particular* x *no* demuestra que

  $$\text{para cada } x, P(x)$$

  sea falsa.

Ejercicios

En los ejercicios 1-6, indique si la afirmación es una función proposicional. Para cada afirmación que sea una función proposicional, indique el dominio de discurso.

1. $(2n + 1)^2$ es un entero impar.

2. Elija un entero entre 1 y 10.

3. Sea x un número real.

4. La película ganó el Premio de la Academia como la mejor de 1955.

5. $1 + 3 = 4$.

6. Existe x tal que $x < y$ (x, y números reales).

Sea $P(n)$ la función proposicional "n divide a 77". Escriba cada proposición de los ejercicios 7-11 con palabras e indique si es verdadera o falsa. El dominio de discurso es el conjunto de enteros positivos.

7. $P(11)$ 8. $P(1)$ 9. $P(3)$ **10.** Para cada n, $P(n)$.

11. Para alguna n, $P(n)$.

Sea $T(x, y)$ la función proposicional "x es más alto que y". El dominio de discurso consta de tres estudiantes: Garth, quien mide 5 pies y 11 pulgadas; Erin, quien mide 5 pies y 6 pulgadas; y Marty, quien mide 6 pies. Escriba cada proposición en los ejercicios 12-15 con palabras e indique si es verdadera o falsa.

12. $\forall x \forall y T(x, y)$ 13. $\forall x \exists y T(x, y)$

14. $\exists x \forall y T(x, y)$ **15.** $\exists x \exists y T(x, y)$

16. Escriba la negación de cada proposición en los ejercicios 12-15 con palabras y en forma simbólica.

Sea $L(x, y)$ la función proposicional "x ama a y". El dominio de discurso es el conjunto de todas las personas vivas. Escriba cada proposición en los ejercicios 17-20 en forma simbólica. ¿Cuáles cree que sean verdaderas?

17. Alguien ama a todos.

18. Todos aman a todos.

19. Alguien ama a alguien.

20. Todos aman a alguien.

21. Escriba la negación de cada proposición en los ejercicios 17-20 con palabras y en forma simbólica.

Determine el valor de verdad de cada afirmación en los ejercicios 22-45. El dominio de discurso es el conjunto de números reales. Justifique sus respuestas.

22. Para cada x, $x^2 > x$.

23. Para alguna x, $x^2 > x$.

24. Para cada x, si $x > 1$, entonces $x^2 > x$.

25. Para alguna x, si $x > 1$, entonces $x^2 > x$.

26. Para cada x, si $x > 1$, entonces $x/(x^2 + 1) < \frac{1}{3}$.

27. Para alguna x, si $x > 1$, entonces $x/(x^2 + 1) < \frac{1}{3}$.

28. Para cada x, para cada y, $x^2 < y + 1$.

29. Para cada x, para alguna y, $x^2 < y + 1$.

30. Para alguna x, para cada y, $x^2 < y + 1$.

31. Para alguna x, para alguna y, $x^2 < y + 1$.

32. Para alguna y, para cada x, $x^2 < y + 1$.

33. Para cada y, para alguna x, $x^2 < y + 1$.

34. Para cada x, para cada y, $x^2 + y^2 = 9$.

35. Para cada x, para alguna y, $x^2 + y^2 = 9$.

36. Para alguna x, para cada y, $x^2 + y^2 = 9$.

37. Para alguna x, para alguna y, $x^2 + y^2 = 9$.

38. Para cada x, para cada y, $x^2 + y^2 \geq 0$.

39. Para cada x, para alguna y, $x^2 + y^2 \geq 0$.

40. Para alguna x, para cada y, $x^2 + y^2 \geq 0$.

41. Para alguna x, para alguna y, $x^2 + y^2 \geq 0$.

42. Para cada x, para cada y, si $x < y$, entonces $x^2 < y^2$.

43. Para cada x, para alguna y, si $x < y$, entonces $x^2 < y^2$.

44. Para alguna x, para cada y, si $x < y$, entonces $x^2 < y^2$.

45. Para alguna x, para alguna y, si $x < y$, entonces $x^2 < y^2$.

46. Escriba la negación de cada proposición en los ejercicios 22-45.

47. La siguiente proposición apareció en la columna *Querida Abby*: Todos los hombres no engañan a sus esposas. ¿Cuál es el significado exacto de esta afirmación? ¿Piensa que la proposición es verdadera o falsa?

48. El economista Robert J. Samuelson fue citado diciendo "Cada problema ambiental no es una tragedia". ¿Cuál es el significado exacto de esta afirmación? Aclare la afirmación reformulándola.

49. (a) Utilice una tabla de verdad para demostrar que si p y q son proposiciones, alguna de las afirmaciones $p \rightarrow q$ o $q \rightarrow p$ es verdadera.

(b) Sea $P(x)$ la función proposicional "x es un número racional" y sea $Q(x)$ la función proposicional "x es un número positivo". El dominio de discurso es el conjunto de todos los números reales. Comente el siguiente argumento, que supuestamente demuestra que todos los números racionales son positivos o que todos los números reales positivos son racionales.

Por el inciso (a),

$$\forall x(P(x) \rightarrow Q(x)) \vee (Q(x) \rightarrow P(x))$$

es verdadera. Con palabras: Para toda x, si x es racional, entonces x es positiva, o si x es positiva, entonces x es racional. Por tanto, todos los números racionales son positivos o todos los números reales positivos son racionales.

50. Demuestre el inciso (b) del teorema 1.3.12.

1.4 DEMOSTRACIONES

Un **sistema matemático** consta de **axiomas, definiciones y términos no definidos**. Se suponen verdaderos los axiomas. Las definiciones se utilizan para crear conceptos nuevos en términos de los existentes. Algunos términos no se definen en forma explícita, sino que se definen en forma implícita mediante los axiomas. Dentro de un sistema matemático es posible deducir teoremas. Un **teorema** es una proposición cuya verdad se ha demostrado. Algunos tipos especiales de teoremas se conocen como lemas y corolarios. Un **lema** es un teorema que por lo general no es interesante en sí mismo sino que es útil para demostrar otro teorema. Un **corolario** es un teorema que se sigue rápidamente de otro teorema.

Un argumento que establece la verdad de un teorema es una **demostración**. La lógica es una herramienta para el análisis de las demostraciones. En esta sección describiremos algunos métodos generales de demostración y utilizaremos la lógica para analizar los argumentos válidos y los no válidos. En las secciones 1.5 y 1.6 analizaremos la resolución y la inducción matemática, que son técnicas especiales de demostración. Primero daremos algunos ejemplos de sistemas matemáticos.

EJEMPLO 1.4.1

La geometría euclidiana proporciona un ejemplo de sistema matemático. Entre los axiomas están

- Dados dos puntos distintos, existe exactamente una recta que los contiene.
- Dada una recta y un punto que no está sobre la recta, existe exactamente una recta paralela a la primera recta y que pasa por el punto.

Los términos *punto* y *recta* son términos no definidos que quedan definidos de manera implícita mediante los axiomas que describen sus propiedades.

Entre las definiciones están

- Dos triángulos son *congruentes* si sus vértices pueden ponerse en correspondencia de modo que los lados correspondientes y los ángulos correspondientes sean iguales.
- Dos ángulos son *suplementarios* si la suma de sus medidas es 180°. □

EJEMPLO 1.4.2

Los números reales proporcionan otro ejemplo de sistema matemático. Entre los axiomas están

- Para todos los números reales x y y, $xy = yx$.
- Existe un subconjunto **P** de números reales que satisface
  (a) Si x y y están en **P**, entonces $x + y$ y xy están en **P**.
  (b) Si x es un número real, entonces exactamente una de las siguientes afirmaciones es verdadera:

$$x \text{ está en } \mathbf{P} \qquad x = 0 \qquad -x \text{ está en } \mathbf{P}.$$

La multiplicación se define de manera implícita mediante el primer axioma y otros axiomas que describen las propiedades que se supone tiene la multiplicación.

Entre las definiciones están

- Los elementos en **P** (del axioma anterior) son los *números reales positivos*.
- El *valor absoluto* $|x|$ de un número real x es x si x es positivo o 0 y en caso contrario es $-x$. □

Daremos varios ejemplos de teoremas, corolarios y lemas de la geometría euclidiana y del sistema de números reales.

EJEMPLO 1.4.3

Algunos ejemplos de teoremas de geometría euclidiana son

- Si dos lados de un triángulo son iguales, entonces los ángulos opuestos a ellos son iguales.
- Si las diagonales de un cuadrilátero se bisecan mutuamente, entonces el cuadrilátero es un paralelogramo. □

EJEMPLO 1.4.4

Un ejemplo de corolario en geometría euclidiana es

- Si un triángulo es equilátero, entonces es equiangular.

Este corolario se sigue de manera inmediata a partir del primer teorema del ejemplo 1.4.3. □

EJEMPLO 1.4.5

Algunos ejemplos de teoremas relativos a los números reales son

- $x \cdot 0 = 0$ para cada número real x.
- Para todos los números reales x, y y z, si $x \leq y$ y $y \leq z$, entonces $x \leq z$. □

EJEMPLO 1.4.6

Un ejemplo de lema relativo a los números reales es

- Si n es un entero positivo, entonces $n - 1$ es un entero positivo o $n - 1 = 0$.

Seguramente este resultado no es interesante en sí mismo, pero puede utilizarse para demostrar otros resultados. □

Con frecuencia, los teoremas tienen la forma

Para toda $x_1, x_2, \ldots, x_n$, si $p(x_1, x_2, \ldots, x_n)$, entonces $q(x_1, x_2, \ldots, x_n)$.

Esta afirmación cuantificada universalmente es verdadera siempre que la proposición condicional

$$\text{si } p(x_1, x_2, \ldots, x_n), \text{ entonces } q(x_1, x_2, \ldots, x_n) \tag{1.4.1}$$

sea verdadera para toda $x_1, x_2, \ldots, x_n$ en el dominio de discurso. Para demostrar (1.4.1), suponemos que $x_1, x_2, \ldots, x_n$ son elementos arbitrarios del dominio de discurso. Si $p(x_1, x_2, \ldots, x_n)$ es falsa, por la definición 1.2.4, (1.4.1) es verdadera; así, sólo es necesario considerar el caso en que $p(x_1, x_2, \ldots, x_n)$ sea verdadera. Una **demostración directa** supone que $p(x_1, x_2, \ldots, x_n)$ es verdadera y entonces, utilizando $p(x_1, x_2, \ldots, x_n)$ y otros axiomas, definiciones o teoremas demostrados con anterioridad, muestra directamente que $q(x_1, x_2, \ldots, x_n)$ es verdadera.

EJEMPLO 1.4.7

Daremos una demostración directa de la siguiente afirmación. Para todos los números reales d, d_1, d_2 y x,

$$\text{Si } d = \text{mín}\{d_1, d_2\} \text{ y } x \le d, \text{ entonces } x \le d_1 \text{ y } x \le d_2.$$

Demostración. Suponemos que d, d_1, d_2 y x son números reales arbitrarios. El análisis anterior muestra que basta suponer que

$$d = \text{mín}\{d_1, d_2\} \text{ y } x \le d$$

es verdadera y demostrar entonces que

$$x \le d_1 \quad \text{y} \quad x \le d_2$$

es verdadera.

Por la definición del mínimo, se tiene que $d \le d_1$ y $d \le d_2$. De $x \le d$ y $d \le d_1$, podemos deducir $x \le d_1$ de un teorema anterior (el segundo teorema del ejemplo 1.4.5). De $x \le d$ y $d \le d_2$, puede deducirse $x \le d_2$ por el mismo teorema anterior. Por tanto, $x \le d_1$ y $x \le d_2$. ■ □

Una segunda técnica de demostración es la **demostración por contradicción**. Una demostración por contradicción establece (1.4.1) suponiendo que la hipótesis p es verdadera y que la conclusión q es falsa, para entonces, utilizando p y $\bar{q}$, así como otros axiomas, definiciones y teoremas demostrados con anterioridad, deducir una **contradicción**. Una contradicción es una proposición de la forma $r \wedge \bar{r}$ (r puede ser cualquier proposición). Una demostración por contradicción se llama a veces **demostración indirecta**, pues para establecer (1.4.1) mediante una demostración por contradicción, se sigue un camino indirecto: se deduce $r \wedge \bar{r}$ y se concluye que (1.4.1) es verdadera.

La única diferencia entre las suposiciones en una demostración directa y una demostración por contradicción es la conclusión negada. En una demostración directa, no se supone la conclusión negada, mientras que en una demostración por contradicción se supone la conclusión negada.

La demostración por contradicción puede justificarse observando que las proposiciones

$$p \to q \quad \text{y} \quad p \wedge \bar{q} \to r \wedge \bar{r}$$

son equivalentes. La equivalencia es inmediata observando la tabla de verdad:

p	q	r	$p \to q$	$p \wedge \bar{q}$	$r \wedge \bar{r}$	$p \wedge \bar{q} \to r \wedge \bar{r}$
V	V	V	V	F	F	V
V	V	F	V	F	F	V
V	F	V	F	V	F	F
V	F	F	F	V	F	F
F	V	V	V	F	F	V
F	V	F	V	F	F	V
F	F	V	V	F	F	V
F	F	F	V	F	F	V

EJEMPLO 1.4.8

Daremos una demostración por contradicción de la siguiente afirmación:

Para todos los números reales x y y, si $x + y \ge 2$, entonces $x \ge 1$ o $y \ge 1$.

Demostración. Supongamos que la conclusión es falsa. Entonces $x < 1$ y $y < 1$. (Recuerde que al negar un "o" se obtiene un "y"; véase el ejemplo 1.2.11, las leyes de De Morgan para la lógica.) Un teorema anterior permite sumar estas desigualdades para obtener

$$x + y < 1 + 1 = 2.$$

En este momento, hemos obtenido la contradicción $p \wedge \bar{p}$, donde

$$p: x + y \ge 2.$$

De esta manera, concluimos que la afirmación es verdadera. ■ □

Suponga que damos una demostración por contradicción de (1.4.1) en la que, como en el ejemplo 1.4.8, deducimos $\bar{p}$. Entonces habremos demostrado

$$\bar{q} \to \bar{p}. \tag{1.4.2}$$

Este caso particular de demostración por contradicción se llama **demostración por contrapositiva**.

Al construir una demostración, debemos asegurarnos de que los argumentos utilizados sean **válidos**. En el resto de esta sección precisaremos el concepto de argumento válido y exploraremos este concepto con cierto detalle.

Consideremos la siguiente serie de proposiciones.

El problema está en el módulo 17 o en el módulo 81.
El problema es un error numérico.
El módulo 81 no tiene un error numérico. (1.4.3)

Suponiendo que estas afirmaciones son verdaderas, es razonable concluir:

El problema está en el módulo 17. (1.4.4)

Este proceso de extracción de una conclusión a partir de una serie de proposiciones se llama **razonamiento deductivo**. Las proposiciones dadas, como (1.4.3), son las **hipótesis** o **premisas** y la proposición que se sigue de las hipótesis, como (1.4.4), es la **conclusión**. Un **argumento (deductivo)** consta de ciertas hipótesis con una conclusión. Muchas demostraciones en matemáticas y computación utilizan argumentos deductivos.

Un argumento tiene la forma

$$\text{Si } p_1 \text{ y } p_2 \text{ y} \cdots \text{y } p_n, \text{ entonces } q. \tag{1.4.5}$$

El argumento (1.4.5) es válido si las conclusiones se siguen de la hipótesis; es decir, si p_1 y p_2 y $\cdots$ y p_n son verdaderas, entonces q también debe ser verdadera. Este análisis motiva la siguiente definición.

DEFINICIÓN 1.4.9

Un *argumento* es una serie de proposiciones que se escriben

$$\begin{array}{l} p_1 \\ p_2 \\ \vdots \\ p_n \\ \hline \therefore q \end{array}$$

o

$$p_1, p_2, \ldots, p_n / \therefore q.$$

Las proposiciones $p_1, p_2, \ldots, p_n$ son las *hipótesis* (o *premisas*) y la proposición q es la *conclusión*. El argumento es *válido* si siempre que p_1 y p_2 y $\cdots$ y p_n sean todas verdaderas, entonces q deberá también ser verdadera; en caso contrario, el argumento *no* es *válido* (es una *falacia*).

En un argumento válido, a veces decimos que la conclusión se sigue de las hipótesis. Observe que no estamos diciendo que la conclusión sea verdadera; sólo estamos diciendo que si se garantizan las hipótesis, entonces se tiene garantizada la conclusión. Un argumento es válido debido a su forma, no a su contenido.

EJEMPLO 1.4.10

Determine si el argumento

$$\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$$

es válido.

[Primera solución.] Construimos una tabla de verdad para todas las proposiciones que aparecen en el argumento:

p	q	$p \rightarrow q$	p	q
V	V	V	V	V
V	F	F	V	F
F	V	V	F	V
F	F	V	F	F

Observamos que siempre que las hipótesis $p \rightarrow q$ y p son verdaderas, la conclusión q también lo es; por tanto, el argumento es válido.

[Segunda solución.] Podemos dejar de lado la tabla de verdad, verificando directamente que siempre que las hipótesis sean verdaderas, la conclusión también es verdadera.

Supongamos que $p \rightarrow q$ y p son verdaderas. Entonces q debe ser verdadera, ya que en caso contrario $p \rightarrow q$ sería falsa. Por tanto, el argumento es válido. □

EJEMPLO 1.4.11

Represente el argumento

$$\begin{array}{l} \text{Si } 2 = 3, \text{ entonces me comí mi sombrero.} \\ \text{Me comí mi sombrero.} \\ \hline \therefore 2 = 3 \end{array}$$

en forma simbólica y determinar si el argumento es válido.

Si hacemos

$$p\text{: } 2 = 3, \quad q\text{: Me comí mi sombrero.}$$

el argumento puede escribirse como

$$\begin{array}{l} p \rightarrow q \\ q \\ \hline \therefore p \end{array}$$

Si el argumento es válido, entonces siempre que $p \rightarrow q$ y q sean ambas verdaderas, p debería ser verdadera. Suponga que $p \rightarrow q$ y q son verdaderas. Esto es posible si p es falsa y q es verdadera. En este caso, p no es verdadera; así, el argumento no es válido. □

También podemos determinar la validez del argumento en el ejemplo 1.4.11 examinando la tabla de verdad del ejemplo 1.4.10. En el tercer renglón de la tabla, las hipótesis son verdaderas y la conclusión falsa; así, el argumento no es válido.

Ejercicios

1. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.1) de un axioma de la geometría euclidiana.
2. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.2) de un axioma del sistema de números reales.
3. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.1) de una definición de la geometría euclidiana.
4. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.2) de una definición del sistema de números reales.
5. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.3) de un teorema de la geometría euclidiana.
6. Proporcione un ejemplo (diferente de los dados en el ejemplo 1.4.5) de un teorema del sistema de números reales.
7. Justifique cada paso de la siguiente demostración indirecta, la cual muestra que si x es un número real, entonces $x \cdot 0 = 0$. Suponga que los siguientes son teoremas previos: Si a, b y c son números reales, entonces $b + 0 = b$ y $a(b + c) = ab + ac$. Si $a + b = a + c$, entonces $b = c$.

   Demostración. $x \cdot 0 + 0 = x \cdot 0$ $x \cdot (0 + 0) = x \cdot 0 + x \cdot 0$; por tanto, $x \cdot 0 = 0$. ■

8. Justifique cada paso de la siguiente demostración por contradicción, la cual muestra que si $xy = 0$, entonces $x = 0$ o $y = 0$. Suponga que si a, b y c son números reales tales que $ab = ac$ y $a \neq 0$, entonces $b = c$.

   Demostración. Suponga que $xy = 0$ y $x \neq 0$ y $y \neq 0$. Como $xy = 0 = x \cdot 0$ y $x \neq 0$; tenemos que $y = 0$, lo cual es una contradicción. ■

9. Muestre, mediante una demostración por contradicción, que si se colocan 100 bolas en nueve cajas, alguna caja contiene 12 o más bolas.

Formule los argumentos de los ejercicios 10-14 en forma simbólica y determine si cada uno es válido. Sean

p: Estudio mucho. q: Obtengo un 10. r: Me vuelvo rico.

10. Si estudio mucho, entonces obtengo un 10.
    Estudio mucho.
    ∴ Obtengo un 10.
11. Si estudio mucho, entonces obtengo un 10.
    Si no me vuelvo rico, entonces no obtengo un 10.
    ∴ Me vuelvo rico.
12. Estudio mucho si y sólo si me vuelvo rico.
    Me vuelvo rico.
    ∴ Estudio mucho.
13. Si estudio mucho o me vuelvo rico, entonces obtengo un 10.
    Obtengo un 10.
    ∴ Si no estudio mucho, entonces me vuelvo rico.
14. Si estudio mucho, entonces obtengo un 10 o me vuelvo rico.
    No obtengo un 10 y no me vuelvo rico.
    ∴ No estudio mucho.

En los ejercicios 15-19, escriba el argumento dado con palabras y determine si cada argumento es válido. Sean

p: 64K es mejor que no tener memoria alguna.
q: Compraremos más memoria.
r: Compraremos una nueva computadora.

15. $p \rightarrow r$
    $\underline{p \rightarrow q}$
    $\therefore p \rightarrow (r \wedge q)$
16. $p \rightarrow (r \vee q)$
    $\underline{r \rightarrow \bar{q}}$
    $\therefore p \rightarrow r$
17. $p \rightarrow r$
    $\underline{r \rightarrow q}$
    $\therefore q$
18. $\bar{r} \rightarrow \bar{p}$
    $\underline{r}$
    $\therefore p$
19. $p \rightarrow r$
    $r \rightarrow q$
    $\underline{p}$
    $\therefore q$

Determine si cada argumento en los ejercicios 20-24 es válido.

20. $p \rightarrow q$
    $\underline{\bar{p}}$
    $\therefore q$
21. $p \rightarrow q$
    $\underline{\bar{q}}$
    $\therefore \bar{p}$
22. $\underline{p \wedge \bar{p}}$
    $\therefore q$
23. $p \rightarrow (q \rightarrow r)$
    $\underline{q \rightarrow (p \rightarrow r)}$
    $\therefore (p \vee q) \rightarrow r$
24. $(p \rightarrow q) \wedge (r \rightarrow s)$
    $\underline{p \vee r}$
    $\therefore q \vee s$
25. Muestre que si

    $$p_1, p_2 / \therefore p \quad \text{y} \quad p, p_3, \ldots, p_n / \therefore c$$

    son argumentos válidos, el argumento

    $$p_1, p_2, \ldots, p_n / \therefore c$$

    también es válido.
26. Comente acerca del siguiente argumento:

    El espacio de almacenamiento en disco flexible es mejor que nada.
    Nada es mejor que una unidad de disco duro.
    ∴ El espacio de almacenamiento en disco flexible es mejor que una unidad de disco duro.

† 1.5 DEMOSTRACIONES POR RESOLUCIÓN

En esta sección escribiremos $a \wedge b$ como ab.

La **resolución** es una técnica de demostración propuesta por J. A. Robinson en 1965 (véase [Robinson]) que depende de una única regla:

$$\text{Si } p \vee q \text{ y } \bar{p} \vee r \text{ son verdaderas, entonces } q \vee r \text{ es verdadera.} \qquad (1.5.1)$$

Podemos verificar (1.5.1) mediante la tabla de verdad (véase el ejercicio 1). Como la resolución depende sólo de esta sencilla regla, es la base de muchos programas de computadora que realizan razonamientos y demostraciones de teoremas.

En una demostración por resolución, las hipótesis y la conclusión se escriben como **cláusulas**. Una cláusula consta de términos separados por o, donde cada término es una variable o la negación de una variable.

EJEMPLO 1.5.1

La expresión

$$a \vee b \vee \bar{c} \vee d$$

es una cláusula, pues los términos a, b, $\bar{c}$ y d están separadas por o, y cada término es una variable o la negación de una variable. □

EJEMPLO 1.5.2

La expresión

$$xy \vee w \vee \bar{z}$$

no es una cláusula, pues aunque los términos están separados por o, el término xy consta de dos variables, y no de una sola. □

EJEMPLO 1.5.3

La expresión

$$p \rightarrow q$$

no es una cláusula, pues los términos están separados por $\rightarrow$. Sin embargo, cada término es una variable. □

Una demostración directa por resolución se realiza aplicando varias veces (1.5.1) a pares de afirmaciones, para deducir nuevas afirmaciones, hasta que se obtenga la conclusión. Al aplicar (1.5.1), p debe ser una sola variable, pero q y r pueden ser expresiones. Observe que al aplicar (1.5.1) a las cláusulas, el resultado $q \vee r$ es una cláusula. (Como q y r constan de términos separados por o, donde cada término es una variable o la negación de una variable, $q \vee r$ también consta de términos separados por o, donde cada término es una variable o la negación de una variable.)

† Esta sección puede omitirse sin pérdida de continuidad.

EJEMPLO 1.5.4

Demostraremos lo siguiente mediante resolución

1. $a \vee b$
2. $\bar{a} \vee c$
3. $\bar{c} \vee d$

$\therefore b \vee d$

Al aplicar (1.5.1) a las expresiones 1 y 2, deducimos

4. $b \vee c$

Al aplicar (1.5.1) a las expresiones 3 y 4, deducimos

5. $b \vee d$

la conclusión deseada. Dadas las hipótesis 1, 2 y 3, hemos demostrado la conclusión $b \vee d$. □

Algunos casos particulares de (1.5.1) son

$$\text{Si } p \vee q \text{ y } \bar{p} \text{ son verdaderas, entonces } q \text{ es verdadera.} \qquad (1.5.2)$$

$$\text{Si } p \text{ y } \bar{p} \vee r \text{ son verdaderas, entonces } r \text{ es verdadera.}$$

EJEMPLO 1.5.5

Demostraremos lo siguiente mediante resolución

1. a
2. $\bar{a} \vee c$
3. $\bar{c} \vee d$

$\therefore d$

Al aplicar (1.5.2) a las expresiones 1 y 2, deducimos

4. c

Al aplicar (1.5.2) a las expresiones 3 y 4, deducimos

5. d

la conclusión deseada. Dadas las hipótesis 1, 2 y 3, hemos demostrado la conclusión d. □

Si una hipótesis no es una cláusula, debe reemplazarse por una expresión equivalente que sea una cláusula o la conjunción de varias cláusulas. Por ejemplo, supongamos que una de las hipótesis es $\overline{a \vee b}$. Como la barra está sobre más de una variable, utilizamos la primera ley de De Morgan (véase el ejemplo 1.2.11)

$$\overline{a \vee b} \equiv \bar{a}\,\bar{b}, \qquad \overline{ab} \equiv \bar{a} \vee \bar{b} \qquad (1.5.3)$$

para obtener una expresión equivalente con una barra sobre las variables individuales

$$\overline{a \vee b} \equiv \bar{a}\,\bar{b}.$$

Luego reemplazamos la hipótesis original $\overline{a \vee b}$ por las dos hipótesis $\bar{a}$ y $\bar{b}$. Este reemplazo se justifica recordando que las hipótesis individuales h_1 y h_2 son equivalentes a $h_1 h_2$ (véase la definición 1.4.9 y el análisis anterior a ésta). Al utilizar varias veces las leyes de De Morgan se logra que cada barra se aplique sólo a una variable.

Una expresión que consta de términos separados por o, donde cada término consta de la *conjunción* de varias variables puede reemplazarse mediante una expresión equivalente que consta de las conjunciones de cláusulas, utilizando la equivalencia

$$a \vee bc \equiv (a \vee b)(a \vee c). \tag{1.5.4}$$

En este caso, podemos reemplazar la hipótesis $a \vee bc$ por las dos hipótesis $a \vee b$ y $a \vee c$. Al utilizar primero las leyes de De Morgan (1.5.3) y luego (1.5.4), podemos obtener hipótesis equivalentes, de modo que cada una de ellas sea una cláusula.

EJEMPLO 1.5.6

Demostraremos lo siguiente mediante resolución

1. $a \vee \overline{bc}$
2. $\overline{a \vee d}$

$\therefore \bar{b}$

Utilizamos (1.5.4) para reemplazar la hipótesis 1 con las dos hipótesis

$$a \vee \bar{b}$$

$$a \vee c$$

Utilizamos la primera ley de De Morgan (1.5.3) para reemplazar la hipótesis 2 con las dos hipótesis

$$\bar{a}$$

$$\bar{d}$$

El argumento se convierte en

1. $a \vee \bar{b}$
2. $a \vee c$
3. $\bar{a}$
4. $\bar{d}$

$\therefore \bar{b}$

Al aplicar (1.5.1) a las expresiones 1 y 3, de inmediato obtenemos la conclusión

$$\bar{b}$$

□

En los sistemas de razonamiento automatizado, la demostración por resolución se combina con la demostración por contradicción. Escribimos la conclusión negada como cláusulas y agregamos estas cláusulas a las hipótesis. Luego utilizamos varias veces (1.5.1) hasta obtener una contradicción.

EJEMPLO 1.5.7

Volveremos a resolver el ejemplo 1.5.4 combinando la resolución con la demostración por contradicción.

Primero negamos la conclusión y utilizamos la primera ley de De Morgan (1.5.3) para obtener

$$\overline{b \vee d} \equiv \bar{b}\,\bar{d}\,.$$

Luego agregamos las cláusulas $\bar{b}$ y $\bar{d}$ a las hipótesis para obtener

1. $a \vee b$
2. $\bar{a} \vee c$
3. $\bar{c} \vee d$
4. $\bar{b}$
5. $\bar{d}$

Al aplicar (1.5.1) a las expresiones 1 y 2, obtenemos

6. $b \vee c$

Al aplicar (1.5.1) a las expresiones 3 y 6, obtenemos

7. $b \vee d$

Al aplicar (1.5.1) a las expresiones 4 y 7, obtenemos

8. d

Ahora, podemos combinar 5 y 8 para obtener una contradicción, con lo que concluye la demostración. □

Puede mostrarse que la resolución es *correcta* y *completa con respecto de la refutación*. El hecho de que la resolución sea correcta significa que sólo puede obtenerse una contradicción a partir de un conjunto de cláusulas inconsistentes (es decir, un conjunto de cláusulas tales que no todas pueden ser verdaderas). El hecho de que la resolución sea completa con respecto de la refutación significa que si un conjunto de cláusulas es inconsistente, entonces la resolución podrá obtener una contradicción. Así, si se obtiene una conclusión a partir de un conjunto de hipótesis, la resolución podrá obtener una contradicción a partir de las hipótesis y la negación de la conclusión. Por desgracia, la resolución no nos indica cuáles son las cláusulas que debemos combinar para deducir la contradicción. Un reto fundamental al automatizar un sistema de razonamiento es ayudar a guiar la búsqueda de las cláusulas que deben combinarse. La bibliografía acerca de la resolución y el razonamiento automatizado es [Gallier; Genesereth; y Wos].

Ejercicios

1. Escriba una tabla de verdad que demuestre (1.5.1).

Utilice la resolución para deducir cada conclusión en los ejercicios 2-6. *Sugerencia:* En los ejercicios 5 y 6, reemplace $\rightarrow$ y $\leftrightarrow$ con expresiones lógicamente equivalentes que utilicen *o* e *y*.

2. $\bar{p} \vee q \vee r$, $\bar{q}$, $\bar{r}$ / $\therefore \bar{p}$

3. $\bar{p} \vee r$, $\bar{r} \vee q$, p / $\therefore q$

4. $\bar{p} \vee t$, $\bar{q} \vee s$, $\bar{r} \vee st$, $p \vee q \vee r \vee u$ / $\therefore s \vee t \vee u$

5. $p \rightarrow q$, $p \vee q$ / $\therefore q$

6. $p \leftrightarrow r$, r / $\therefore p$

7. Utilice la resolución y la demostración por contradicción para resolver de nuevo los ejercicios 2-6.
8. Utilice la resolución y la demostración por contradicción para resolver de nuevo el ejemplo 1.5.6.

1.6 *INDUCCIÓN MATEMÁTICA*

Supongamos que una serie de cubos numerados 1, 2, ... están sobre una mesa (infinitamente) larga (véase la figura 1.6.1) y que algunos cubos están marcados con una "X". (Todos los cubos visibles en la figura 1.6.1 están marcados.) Supongamos que

El primer cubo está marcado. (1.6.1)

Si todos los cubos anteriores al cubo $(n + 1)$ están marcados, entonces el cubo $(n + 1)$ también lo está. (1.6.2)

Mostraremos que (1.6.1) y (1.6.2) implican que cada cubo está marcado, examinando los cubos uno por uno.

La afirmación (1.6.1) establece de manera explícita que el cubo 1 está marcado. Consideremos el cubo 2. Todos los cubos anteriores al cubo 2, a saber, el cubo 1, están marcados; así, de acuerdo con (1.6.2), el cubo 2 también está marcado. Consideremos el cubo 3. Todos los cubos anteriores al cubo 3, a saber, los cubos 1 y 2, están marcados; así, de acuerdo con (1.6.2), el cubo 3 también está marcado. De esta forma, podemos mostrar que cada cubo está marcado. Por ejemplo, supongamos que hemos verificado que los cubos 1 a 5 están marcados, como muestra la figura 1.6.1. Para mostrar que el cubo 6, que no aparece en la figura 1.6.1, está marcado, observamos que todos los cubos anteriores al cubo 6 están marcados, de modo que por (1.6.2), el cubo 6 también está marcado.

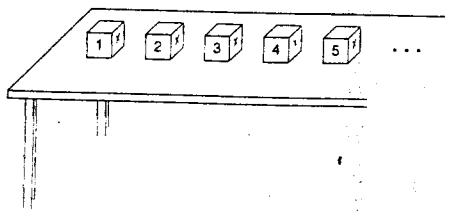


FIGURA 1.6.1 Cubos numerados sobre una mesa.

El ejemplo anterior ilustra el **principio de inducción matemática**. Para mostrar cómo se puede utilizar la inducción matemática de manera más profunda, sea S_n la suma de los primeros n enteros positivos

$$S_n = 1 + 2 + 3 + \cdots + n. \tag{1.6.3}$$

Supongamos que alguien afirma que

$$S_n = \frac{n(n+1)}{2} \qquad \text{para } n = 1, 2, \ldots \tag{1.6.4}$$

En realidad, se establece una serie de afirmaciones, a saber,

$$S_1 = \frac{1(2)}{2} = 1$$
$$S_2 = \frac{2(3)}{2} = 3$$
$$S_3 = \frac{3(4)}{2} = 6$$
$$\vdots$$

Supongamos que cada ecuación verdadera tiene una "×" junto a ella (véase la figura 1.6.2). Como la primera ecuación es verdadera, está marcada. Ahora, supongamos que podemos mostrar que si todas las ecuaciones anteriores a una ecuación particular, digamos, la ecuación $(n + 1)$, están marcadas, entonces la ecuación $(n + 1)$ también está marcada. Entonces, como en el ejemplo de los cubos, todas las ecuaciones están marcadas; es decir, todas las ecuaciones son verdaderas y se verifica la fórmula (1.6.4).

$$\begin{aligned} S_1 &= \frac{1(2)}{2} && \times \\ S_2 &= \frac{2(3)}{2} && \times \\ &\vdots \\ S_{n-1} &= \frac{(n-1)n}{2} && \times \\ S_n &= \frac{n(+1)}{2} && \times \\ S_{n+1} &= \frac{(n+1)(n+2)}{2} && ? \\ &\vdots \end{aligned}$$

FIGURA 1.6.2
Una serie de afirmaciones. Las afirmaciones verdaderas se señalan con ×.

Debemos mostrar que si todas las ecuaciones anteriores a la ecuación $(n + 1)$ son verdaderas, entonces la ecuación $(n + 1)$ también lo es. Suponiendo que todas las ecuaciones anteriores a la ecuación $(n + 1)$ son verdaderas, entonces, en particular, la ecuación n es verdadera:

$$S_n = \frac{n(n+1)}{2}. \tag{1.6.5}$$

Debemos mostrar que la ecuación $(n + 1)$

$$S_{n+1} = \frac{(n+1)(n+2)}{2}$$

es verdadera. De acuerdo con la definición (1.6.3),

$$S_{n+1} = 1 + 2 + \cdots + n + (n + 1).$$

Observamos que S_n está contenida dentro de S_{n+1}, en el sentido de que

$$\begin{aligned} S_{n+1} &= 1 + 2 + \cdots + n + (n + 1) \\ &= S_n + (n + 1). \end{aligned} \tag{1.6.6}$$

Debido a (1.6.5) y (1.6.6), tenemos

$$S_{n+1} = S_n + n + 1 = \frac{n(n+1)}{2} + n + 1 = \frac{(n+1)(n+2)}{2}.$$

Nuestra demostración por inducción matemática constó de dos pasos. En primer lugar, verificamos que la afirmación correspondiente a $n = 1$ era verdadera. En segundo lugar, *supusimos* que las afirmaciones $1, 2, \ldots, n$ eran verdaderas y *demostramos* que la afirmación $(n + 1)$ también lo era. Al demostrar la afirmación $(n + 1)$, podíamos utilizar las afirmaciones $1, 2, \ldots, n$; de hecho, el truco para construir una demostración por inducción matemática consiste en relacionar las afirmaciones $1, 2, \ldots, n$ con la afirmación $(n + 1)$.

A continuación enunciamos de manera formal el principio de inducción matemática.

Principio de inducción matemática

Supongamos que para cada entero positivo n tenemos una afirmación $S(n)$ que es verdadera o falsa. Supongamos que

$S(1)$ *es verdadera;* (1.6.7)

si $S(i)$ *es verdadera, para toda* $i < n + 1$, *entonces* $S(n + 1)$ *es verdadera.* (1.6.8)

Entonces $S(n)$ *es verdadera para cada entero positivo n.*

La condición (1.6.7) se llama el **paso base** y la condición (1.6.8) se llama el **paso inductivo**. De aquí en adelante, "inducción" significa "inducción matemática".

En este momento ilustraremos el principio de inducción matemática mediante otro ejemplo.

EJEMPLO 1.6.1

Utilice la inducción para mostrar que

$$n! \geq 2^{n-1} \quad \text{para } n = 1, 2, \ldots. \tag{1.6.9}$$

PASO BASE. [Condición (1.6.7)] Debemos mostrar que (1.6.9) es verdadera si $n = 1$. Esto es fácil de verificar, pues $1! = 1 \geq 1 = 2^{1-1}$.

PASO INDUCTIVO. [Condición (1.6.8)] Debemos mostrar que si $i! \geq 2^{i-1}$ para $i = 1, \ldots, n$, entonces

$$(n + 1)! \geq 2^n. \tag{1.6.10}$$

Supongamos que $i! \geq 2^{i-1}$ para $i = 1, \ldots, n$. Entonces, en particular, para $i = n$, tenemos

$$n! \geq 2^{n-1}. \tag{1.6.11}$$

Podemos relacionar (1.6.10) y (1.6.11) observando que

$$(n + 1)! = (n + 1)(n!).$$

Ahora,

$$\begin{aligned} (n + 1)! &= (n + 1)(n!) \\ &\geq (n + 1)2^{n-1} && \text{por (1.6.11)} \\ &\geq 2 \cdot 2^{n-1} && \text{pues } n + 1 \geq 2 \\ &= 2^n. \end{aligned}$$

Por tanto, (1.6.10) es verdadera. Hemos concluido el paso inductivo.

Como hemos verificado el paso base y el paso inductivo, el principio de inducción matemática nos dice que (1.6.9) es verdadera para cada entero positivo n. □

Para verificar el paso inductivo (1.6.8), suponemos que $S(i)$ es verdadera para toda $i < n + 1$ y luego demostramos que $S(n + 1)$ es verdadera. Esta formulación de la inducción matemática se llama la **forma fuerte de la inducción matemática**. Con frecuencia, como en el caso de los ejemplos anteriores, podemos deducir $S(n + 1)$ suponiendo solamente $S(n)$. En realidad, con frecuencia se enuncia el paso inductivo de la manera siguiente:

Si $S(n)$ es verdadera, entonces $S(n + 1)$ es verdadera.

En estas dos formulaciones, el paso base no se modifica. Puede mostrarse (véase el ejercicio 45) que las dos formas de inducción matemática son lógicamente equivalentes.

Si queremos verificar que las afirmaciones

$$S(n_0), S(n_0 + 1), \ldots,$$

donde $n_0 \neq 1$, son verdaderas, debemos cambiar el paso base a

$$S(n_0) \text{ es verdadera.}$$

El paso inductivo no se modifica.

EJEMPLO 1.6.2 ***Suma geométrica***

Utilice la inducción para mostrar que si $r \neq 1$,

$$a + ar^1 + ar^2 + \cdots + ar^n = \frac{a(r^{n+1} - 1)}{r - 1} \tag{1.6.12}$$

para $n = 0, 1, \ldots$.

La suma de la izquierda se llama **suma geométrica**. En una suma geométrica, la razón entre los términos consecutivos $(ar^{i+1}/ar^i = r)$ es constante.

PASO BASE. El paso base, que en este caso se obtiene haciendo $n = 0$, es

$$a = \frac{a(r^1 - 1)}{r - 1},$$

lo cual es verdadero.

PASO INDUCTIVO. Supongamos que la afirmación (1.6.12) es verdadera para n. Ahora

$$\begin{aligned} a + ar^1 + ar^2 + \cdots + ar^n + ar^{n+1} &= \frac{a(r^{n+1} - 1)}{r - 1} + ar^{n+1} \\ &= \frac{a(r^{n+1} - 1)}{r - 1} + \frac{ar^{n+1}(r - 1)}{r - 1} \\ &= \frac{a(r^{n+2} - 1)}{r - 1}. \end{aligned}$$

Como hemos verificado el paso base modificado y el paso inductivo, el principio de inducción matemática nos dice que (1.6.12) es verdadera para $n = 0, 1, \ldots$. □

Como ejemplo del uso de la suma geométrica, si hacemos $a = 1$ y $r = 2$ en (1.6.12), obtenemos la fórmula

$$1 + 2 + 2^2 + 2^3 + \cdots + 2^n = \frac{2^{n+1} - 1}{2 - 1} = 2^{n+1} - 1.$$

El lector habrá notado que para demostrar la fórmula anterior, primero hay que contar con las fórmulas correctas. Una pregunta razonable es: ¿cómo obtener estas fórmulas? Existen muchas respuestas a esta pregunta. Una técnica para deducir una fórmula es experimentar con valores pequeños e intentar descubrir un patrón. Por ejemplo, consideremos la suma $1 + 3 + \cdots + (2n - 1)$. La siguiente tabla proporciona los valores de esta suma para $n = 1, 2, 3, 4$.

n	$1 + 3 + \cdots + (2n - 1)$
1	1
2	4
3	9
4	16

Como la segunda columna consta de cuadrados, conjeturamos que

$$1 + 3 + \cdots + (2n - 1) = n^2 \qquad \text{para cada entero positivo } n.$$

La conjetura es correcta y la fórmula puede demostrarse por inducción matemática (véase el ejercicio 1).

Nuestros dos últimos ejemplos muestran que la inducción no se limita a demostrar fórmulas para sumas y verificar desigualdades.

EJEMPLO 1.6.3

Utilice inducción para mostrar que $5^n - 1$ es divisible entre 4 para $n = 1, 2, \ldots$.

PASO BASE. Si $n = 1$, $5^n - 1 = 5^1 - 1 = 4$, que es divisible entre 4.

PASO INDUCTIVO. Supongamos que $5^n - 1$ es divisible entre 4. Debemos mostrar que $5^{n+1} - 1$ es divisible entre 4. Para relacionar el caso $(n + 1)$ con el caso n, escribimos

$$5^{n+1} - 1 = 5 \cdot 5^n - 1 = (5^n - 1) + 4 \cdot 5^n.$$

Por hipótesis, $5^n - 1$ es divisible entre 4, y como $4 \cdot 5^n$ es divisible entre 4, la suma

$$(5^n - 1) + 4 \cdot 5^n = 5^{n+1} - 1$$

es divisible entre 4. Como hemos verificado el paso base y el paso inductivo, el principio de inducción matemática nos dice que $5^n - 1$ es divisible entre 4 para $n = 1, 2, \ldots$. □

EJEMPLO 1.6.4 *Un problema de mosaicos*

Un *triominó recto*, que de aquí en adelante llamaremos sólo *triominó*, es un objeto formado por tres cuadrados, como muestra la figura 1.6.3. Un triominó es un tipo de poliminó. Desde que los poliminós fueron ideados por Solomon W. Golomb en 1954 (véase [Golomb, 1954]), han sido un tema predilecto de las matemáticas recreativas. Un *poliminó de orden s* consta de s cuadrados unidos por sus aristas. Un triominó es un poliminó de orden 3. El otro tipo de poliminó de orden 3 está formado por una fila de tres cuadrados. (Nadie ha determinado una fórmula sencilla para el número de poliminós de orden s.) Se han diseñado varios problemas con poliminós (véase [Martin]).

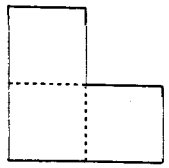


FIGURA 1.6.3
Un triominó.

Daremos la demostración inductiva de Golomb (véase [Golomb, 1954]) de que si eliminamos un cuadrado de un tablero de $n \times n$, donde n es una potencia de 2, podemos formar un mosaico sobre los demás cuadrados con triominós rectos (véase la figura 1.6.4). Formar un *mosaico* sobre una figura con triominós quiere decir cubrir de manera exacta una figura mediante triominós, sin que éstos se traslapen o rebasen la figura. Un tablero al que le falta un cuadrado se llama *tablero deficiente*.

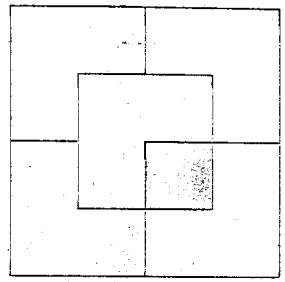


FIGURA 1.6.4
Formación de un mosaico sobre un tablero deficiente 4×4 con triominós.

Ahora, utilizaremos la inducción sobre k para demostrar que podemos formar un mosaico sobre un tablero deficiente de $2^k \times 2^k$ con triominós.

PASO BASE. Si $k = 1$, el tablero deficiente 2×2 es en sí un triominó y por tanto puede formarse un mosaico con un triominó.

PASO INDUCTIVO. Supongamos que podemos formar un mosaico sobre un tablero deficiente $2^k \times 2^k$. Mostraremos que podemos formar un mosaico sobre un tablero deficiente $2^{k+1} \times 2^{k+1}$.

Consideremos un tablero deficiente $2^{k+1} \times 2^{k+1}$. Dividimos el tablero en cuatro tableros $2^k \times 2^k$, como muestra la figura 1.6.5. Giramos el tablero de modo que el cuadrado faltante esté en el cuadrante superior izquierdo. Según la hipótesis inductiva, podemos formar un mosaico sobre el tablero superior izquierdo $2^k \times 2^k$. Colocamos un triominó T en el centro, como muestra la figura 1.6.5, de modo que cada cuadrado de T esté en cada uno de los demás cuadrantes. Si consideramos que los cuadrados cubiertos por T son faltantes, entonces cada uno de estos cuadrantes es un tablero deficiente $2^k \times 2^k$. De nuevo, por la hipótesis de inducción, podemos formar un mosaico sobre estos tableros. Ahora hemos formado un mosaico sobre el tablero $2^{k+1} \times 2^{k+1}$. Por el principio de inducción matemática, se sigue que podemos formar un mosaico sobre cualquier tablero deficiente $2^k \times 2^k$ con triominós, para $k = 1, 2, \ldots$.

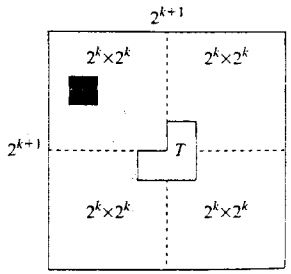


FIGURA 1.6.5
Uso de la inducción matemática para formar un mosaico sobre un tablero deficiente $2^{k+1} \times 2^{k+1}$ con triominós.

Si podemos formar un mosaico sobre un tablero deficiente $n \times n$, donde n no necesariamente es una potencia de 2, entonces el número de cuadrados, $n^2 - 1$, debe ser divisible entre 3. [Chu] mostró que la recíproca es verdadera, excepto cuando n es igual a 5. Más precisamente, si $n \neq 5$, puede formarse un mosaico sobre cualquier tablero deficiente $n \times n$ con triominós si y sólo si 3 divide a $n^2 - 1$. [Es posible formar un mosaico sobre algunos tableros deficientes 5×5 y sobre otros no (véanse los ejercicios 32 y 33)]. □

Ejercicios

En los ejercicios 1-11, utilice inducción para verificar que cada ecuación es verdadera para cada entero positivo n.

1. $1 + 3 + 5 + \cdots + (2n - 1) = n^2$
2. $1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + \cdots + n(n + 1) = \dfrac{n(n + 1)(n + 2)}{3}$

3. $1(1!)+2(2!)+\cdots+n(n!)=(n+1)!-1$

4. $1^2+2^2+3^2+\cdots+n^2=\dfrac{n(n+1)(2n+1)}{6}$

5. $1^2-2^2+3^2-\cdots+(-1)^{n+1}n^2=\dfrac{(-1)^{n+1}n(n+1)}{2}$

6. $1^3+2^3+3^3+\cdots+n^3=\left[\dfrac{n(n+1)}{2}\right]^2$

7. $\dfrac{1}{1\cdot3}+\dfrac{1}{3\cdot5}+\dfrac{1}{5\cdot7}+\cdots+\dfrac{1}{(2n-1)(2n+1)}=\dfrac{n}{2n+1}$

8. $\dfrac{1}{2\cdot4}+\dfrac{1\cdot3}{2\cdot4\cdot6}+\dfrac{1\cdot3\cdot5}{2\cdot4\cdot6\cdot8}+\cdots+\dfrac{1\cdot3\cdot5\cdots(2n-1)}{2\cdot4\cdot6\cdots(2n+2)}=\dfrac{1}{2}-\dfrac{1\cdot3\cdot5\cdots(2n+1)}{2\cdot4\cdot6\cdots(2n+2)}$

9. $\dfrac{1}{2^2-1}+\dfrac{1}{3^2-1}+\cdots+\dfrac{1}{(n+1)^2-1}=\dfrac{3}{4}-\dfrac{1}{2(n+1)}-\dfrac{1}{2(n+2)}$

†★ **10.** $\cos x+\cos 2x+\cdots+\cos nx=\dfrac{\cos[(x/2)(n+1)]\operatorname{sen}(nx/2)}{\operatorname{sen}(x/2)}$ siempre que $\operatorname{sen}(x/2)\neq 0$.

☆ 11. $1\operatorname{sen}x+2\operatorname{sen}2x+\cdots+n\operatorname{sen}nx=$

$$\frac{\operatorname{sen}[(n+1)x]}{4\operatorname{sen}^2(x/2)}-\frac{(n+1)\cos\left(\frac{2n+1}{2}x\right)}{2\operatorname{sen}(x/2)}$$

siempre que $\operatorname{sen}(x/2)\neq 0$.

En los ejercicios 12-17, utilice inducción para verificar la desigualdad.

12. $\dfrac{1}{2n}\leq\dfrac{1\cdot3\cdot5\cdots(2n-1)}{2\cdot4\cdot6\cdots(2n)}$, $n=1, 2, \ldots$

☆ 13. $\dfrac{1\cdot3\cdot5\cdots(2n-1)}{2\cdot4\cdot6\cdots(2n)}\leq\dfrac{1}{\sqrt{n+1}}$, $n=1, 2, \ldots$

14. $2n+1\leq 2^n$, $n=3, 4, \ldots$

★ **15.** $2^n\geq n^2$, $n=4, 5, \ldots$

☆ 16. $(a_1a_2\cdots a_{2^n})^{1/2^n}\leq\dfrac{a_1+a_2+\cdots+a_{2^n}}{2^n}$, $n=1, 2, \ldots$ y las a_i son números positivos.

17. $(1+x)^n\geq 1+nx$, para $x\geq -1$ y $n=1, 2, \ldots$

En los ejercicios 18-21, utilice inducción para demostrar la afirmación.

18. 7^n-1 es divisible entre 6, para $n=1, 2, \ldots$.

19. 11^n-6 es divisible entre 5, para $n=1, 2, \ldots$.

20. $6\cdot7^n-2\cdot3^n$ es divisible entre 4, para $n=1, 2, \ldots$.

★ **21.** 3^n+7^n-2 es divisible entre 8, para $n=1, 2, \ldots$.

22. Experimente con valores pequeños de n, conjeture una fórmula para la suma

$$\frac{1}{1\cdot2}+\frac{1}{2\cdot3}+\cdots+\frac{1}{n(n+1)},$$

y luego utilice inducción para verificar su fórmula.

† Un ejercicio con una estrella indica un problema con dificultad superior al promedio.

23. Utilice inducción para mostrar que n líneas rectas en el plano dividen a éste en $(n^2+n+2)/2$ regiones. Suponga que no hay pares de rectas paralelas y que no hay tres rectas que tengan un punto en común.

24. Muestre que cualquier tarifa postal de 5 centavos o más puede cobrarse utilizando sólo estampillas de 2 y 5 centavos.

25. Muestre que cualquier tarifa postal de 24 centavos o más puede cobrarse utilizando sólo estampillas de 5 y 7 centavos.

Los antiguos egipcios expresaban una fracción como una suma de fracciones con numerador igual a 1. Por ejemplo, 5/6 puede expresarse como

$$\frac{5}{6}=\frac{1}{2}+\frac{1}{3}.$$

Decimos que una fracción p/q, donde p y q son enteros positivos, está en *forma egipcia* si

$$\frac{p}{q}=\frac{1}{n_1}+\frac{1}{n_2}+\cdots+\frac{1}{n_k}, \tag{1.6.13}$$

donde $n_1, n_2, \ldots, n_k$ son enteros positivos que satisfacen $n_1<n_2<\cdots n_k$.

26. Muestre que la representación (1.6.13) no necesariamente es única, representando $\frac{5}{6}$ de dos maneras distintas.

★ 27. Muestre que la representación (1.6.13) nunca es única.

28. Realice los siguientes pasos para demostrar, por inducción sobre p, que cada fracción p/q con $0<p/q<1$ puede expresarse en forma egipcia.

(a) Verifique el paso base ($p=1$).

(b) Suponga que $0<p/q<1$ y que todas las fracciones i/q', con $1\leq i<p$ y q' arbitrarios, pueden expresarse en forma egipcia. Elija el menor entero positivo n tal que $1/n\leq p/q$. Muestre que

$$n>1 \quad \text{y} \quad \frac{p}{q}<\frac{1}{n-1}.$$

(c) Muestre que si $p/q=1/n$, la demostración concluye.

(d) Suponga que $1/n<p/q$. Sean

$$p_1=np-q \quad \text{y} \quad q_1=nq.$$

Muestre que

$$\frac{p_1}{q_1}=\frac{p}{q}-\frac{1}{n}, \qquad 0<\frac{p_1}{q_1}<1 \qquad \text{y} \qquad p_1<p.$$

Concluya que

$$\frac{p_1}{q_1}=\frac{1}{n_1}+\frac{1}{n_2}+\cdots+\frac{1}{n_k}$$

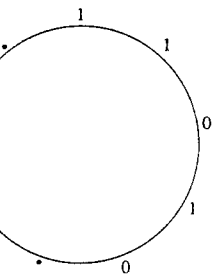
con $n_1, n_2, \ldots, n_k$ distintos.

(e) Muestre que $p_1/q_1<1/n$.

(f) Muestre que

$$\frac{p}{q}=\frac{1}{n}+\frac{1}{n_1}+\cdots+\frac{1}{n_k}$$

y $n, n_1, \ldots, n_k$ son distintos.



29. Utilice el método del ejercicio 28 para determinar formas egipcias para 3/8, 5/7 y 13/19.

★ **30.** Muestre que cualquier fracción p/q, donde p y q son enteros positivos, puede escribirse en forma egipcia. (No estamos suponiendo que $p/q < 1$.)

31. Dado un número igual de ceros y unos distribuidos en un círculo (véase la figura anexa), muestre que es posible comenzar en algún número y recorrer el círculo hasta llegar a la posición inicial de modo que, en cualquier punto del ciclo, uno haya visto al menos tantos ceros como unos.

32. Forme un mosaico sobre un tablero 5×5 con triominós, donde falte el cuadrado superior izquierdo.

33. Muestre un tablero deficiente 5×5 sobre el cual no pueda formarse un mosaico con triominós. Explique la razón de este hecho.

34. Muestre que cualquier tablero $(2i) \times (3j)$, donde i y j son enteros positivos, sin que falten cuadrados, puede cubrirse mediante triominós.

☆ 35. Muestre que sobre cualquier tablero deficiente 7×7 puede formarse un mosaico con triominós.

★ **36.** Muestre que sobre cualquier tablero deficiente $n \times n$ puede formarse un mosaico con triominós si n es impar, $n > 5$, y 3 divide a $n^2 - 1$.

Un *heptaminó tridimensional* es un cubo tridimensional $2 \times 2 \times 2$ al que se le ha quitado un cubo $1 \times 1 \times 1$ en una esquina. Un *cubo deficiente* es un cubo $k \times k \times k$ al que se le ha quitado un cubo $1 \times 1 \times 1$.

37. Demuestre que un cubo deficiente $2^n \times 2^n \times 2^n$ puede cubrirse mediante heptaminós tridimensionales.

38. Demuestre que si puede formarse un mosaico con heptaminós tridimensionales sobre un cubo deficiente $k \times k \times k$, entonces 7 divide a $k - 1$, $k - 2$ y $k - 4$.

39. Suponga que $S_n = (n + 2)(n - 1)$ se propone (incorrectamente) como una fórmula para

$$2 + 4 + \cdots + 2n.$$

(a) Muestre que se satisface el paso inductivo pero no el paso base.

☆ (b) Si S'_n es una expresión arbitraria que satisface el paso inductivo, ¿qué forma debe tener?

☆ 40. ¿En qué punto falla el siguiente argumento, el cual mostraría que cualesquiera dos enteros positivos son iguales?

Utilizamos inducción sobre n para "demostrar" que si a y b son enteros positivos y $n = \text{máx}\{a, b\}$, entonces $a = b$.

PASO BASE. $(n = 1)$. Si a y b son enteros positivos y $1 = \text{máx}\{a, b\}$, debemos tener $a = b = 1$.

PASO INDUCTIVO. Supongamos que si a' y b' son enteros positivos y $n = \text{máx}\{a', b'\}$, entonces $a' = b'$. Supongamos que a y b son enteros positivos y que $n + 1 = \text{máx}\{a, b\}$. Ahora, $n = \text{máx}\{a - 1, b - 1\}$. Por la hipótesis de inducción, $a - 1 = b - 1$. Por tanto, $a = b$.

Como hemos verificado el paso base y el paso inductivo, el principio de inducción matemática implica que ¡cualesquiera dos enteros positivos son iguales!

41. ¿Qué falla en la siguiente "demostración" de que

$$\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1} \neq \frac{n^2}{n+1}$$

para toda $n \geq 2$?

Supongamos, por contradicción, que

$$\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1} = \frac{n^2}{n+1}. \tag{1.6.14}$$

Entonces también

$$\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1} + \frac{n+1}{n+2} = \frac{(n+1)^2}{n+2}.$$

Podríamos demostrar la afirmación (1.6.14) por inducción. En particular, el paso inductivo sería

$$\left(\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1}\right) + \frac{n+1}{n+2} = \frac{n^2}{n+1} + \frac{n+1}{n+2}.$$

Por tanto,

$$\frac{n^2}{n+1} + \frac{n+1}{n+2} = \frac{(n+1)^2}{n+2}.$$

Al multiplicar cada lado de esta última ecuación por $(n + 1)(n + 2)$ se obtiene

$$n^2(n + 2) + (n + 1)^2 = (n + 1)^3.$$

Podemos escribir esta última ecuación como

$$n^3 + 2n^2 + n^2 + 2n + 1 = n^3 + 3n^2 + 3n + 1$$

o

$$n^3 + 3n^2 + 2n + 1 = n^3 + 3n^2 + 3n + 1,$$

lo cual es una contradicción. Por tanto,

$$\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1} \neq \frac{n^2}{n+1}$$

como se afirmaba.

42. Utilice la inducción matemática para demostrar que

$$\frac{1}{2} + \frac{2}{3} + \cdots + \frac{n}{n+1} < \frac{n^2}{n+1}$$

para toda $n \geq 2$. Esta desigualdad proporciona una demostración correcta de la afirmación del ejercicio 41.

☆ 43. Suponga que la forma del paso inductivo en la inducción matemática es: Si $S(n)$ es verdadera, entonces $S(n + 1)$ es verdadera. Demuestre el principio de buen orden para enteros positivos.

Principio de buen orden para enteros positivos

Si X es un conjunto no vacío de enteros positivos, entonces X contiene un elemento mínimo.

Suponga que no existe un entero positivo menor que 1 y que si n es un entero positivo, no existe un entero positivo entre n y $n + 1$.

★ **44.** Suponga válido el principio de buen orden para los enteros positivos (véase el ejercicio 43). Demuestre la forma fuerte del principio de inducción matemática.

☆ 45. Muestre que la forma fuerte del principio de inducción matemática y la forma de la inducción matemática en que el paso inductivo es "si $S(n)$ es verdadera, entonces $S(n + 1)$ es verdadera" son equivalentes; es decir, suponga la forma fuerte y demuestre la forma alternativa; después, suponga la forma alternativa y demuestre la forma fuerte.

46. Muestre que si se distribuyen 40 monedas en nueve bolsas, al menos dos bolsas contienen el mismo número de monedas.

☆ 47. [Carmony] Suponga que $n > 1$ personas se colocan de modo que cada una tenga un único vecino más cercano. Además, suponga que cada persona tiene un pastel que arroja sobre su vecino más cercano. Un *sobreviviente* es una persona que no es alcanzada por un pastel.

(a) Proporcione un ejemplo para mostrar que si n es par, podría no haber sobrevivientes.

(b) Utilice inducción sobre n para mostrar que si n es impar, siempre habrá al menos un sobreviviente.

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

INDUCCIÓN MATEMÁTICA

Problema

Defina

$$H_k = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k} \tag{1}$$

para $k \geq 1$. Demuestre que

$$H_{2^n} \geq 1 + \frac{n}{2} \tag{2}$$

para toda $n \geq 0$.

Para enfrentar el problema

Con frecuencia, es buena idea analizar algunos ejemplos concretos de las expresiones en cuestión. Analizaremos H_k para algunos valores pequeños de k. El menor valor de k para el cual H_k está definido es $k = 1$. En este caso, el último término $1/k$ en la definición de H_k es igual a $1/1 = 1$. Como el primer y el último términos coinciden,

$$H_1 = 1$$

Para $k = 2$, el último término $1/k$ en la definición de H_k es igual a $1/2$, de modo que

$$H_2 = 1 + \frac{1}{2}$$

De manera análoga, vemos que

$$H_3 = 1 + \frac{1}{2} + \frac{1}{3}$$

$$H_4 = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4}$$

Observamos que H_1 aparece como primer término de H_2, H_3 y H_4, que H_2 aparece como los dos primeros términos de H_3 y H_4, y que H_3 aparece como los tres primeros términos de H_4. En general, H_m aparece como los m primeros términos de H_n, si $m \leq k$. Esta observación nos ayudará posteriormente, pues el paso inductivo en una demostración por inducción debe relacionar las instancias menores de un problema con las instancias mayores.

En general, es una buena estrategia retrasar la combinación de términos y la simplificación mientras se pueda; ésta es la razón por la cual, por ejemplo, hemos dejado H_4 como la suma de cuatro términos en vez de escribir $H_4 = 25/12$. Como dejamos a H_4 expresada como la suma de cuatro términos, pudimos observar que H_1, H_2 y H_3 aparecían en la expresión para H_4.

Determinación de una solución

El paso base consiste en demostrar la siguiente afirmación para el menor valor de n, que en este caso es $n = 0$. Para $n = 0$, la desigualdad (2) que debemos demostrar se convierte en

$$H_{2^0} \geq 1 + \frac{0}{2} \tag{3}$$

Ya hemos observado que $H_1 = 1$. Así, la desigualdad (3) es verdadera cuando $n = 0$; de hecho, se convierte en una igualdad. (Recuerde que por definición, si $x = y$ es verdadera, entonces $x \geq y$ también lo es.)

Veamos ahora el paso inductivo. Es bueno escribir lo que se supone (aquí es el caso n)

$$H_{2^n} \geq 1 + \frac{n}{2} \tag{4}$$

y lo que hay que demostrar (aquí es el caso $n + 1$)

$$H_{2^{n+1}} \geq 1 + \frac{n+1}{2} \tag{5}$$

También es bueno escribir las fórmulas de todas las expresiones que han aparecido. Al utilizar la ecuación (1), podemos escribir

$$H_{2^n} = 1 + \frac{1}{2} + \cdots + \frac{1}{2^n} \tag{6}$$

y

$$H_{2^{n+1}} = 1 + \frac{1}{2} + \cdots + \frac{1}{2^{n+1}}$$

La última ecuación no muestra tan claramente que H_{2^n} aparezca como los primeros 2^n términos de $H_{2^{n+1}}$. Escribimos la última ecuación como

$$H_{2^{n+1}} = 1 + \frac{1}{2} + \cdots + \frac{1}{2^n} + \frac{1}{2^n + 1} + \cdots + \frac{1}{2^{n+1}} \tag{7}$$

para hacer evidente que H_{2^n} aparece como los primeros 2^n términos de $H_{2^{n+1}}$.

Para mayor claridad, hemos escrito el término posterior a $1/2^n$. Observe que los denominadores aumentan de uno en uno, de modo que el término posterior a $1/2^n$ es $1/(2^n + 1)$. Además, observe que existe una gran diferencia entre $1/(2^n + 1)$, el término posterior a $1/2^n$, y $1/2^{n+1}$, el último término en la ecuación (7).

Al utilizar las ecuaciones (6) y (7), podemos relacionar H_{2^n} con $H_{2^{n+1}}$ de manera explícita escribiendo

$$H_{2^{n+1}} = H_{2^n} + \frac{1}{2^n + 1} + \cdots + \frac{1}{2^{n+1}} \tag{8}$$

Al combinar (4) y (8), obtenemos

$$H_{2^{n+1}} \geq 1 + \frac{n}{2} + \frac{1}{2^n + 1} + \cdots + \frac{1}{2^{n+1}} \tag{9}$$

Esta desigualdad muestra que $H_{2^{n+1}}$ es mayor o igual que

$$1 + \frac{n}{2} + \frac{1}{2^n + 1} + \cdots + \frac{1}{2^{n+1}}$$

pero nuestro objetivo (5) es mostrar que $H_{2^{n+1}}$ es mayor o igual que $1 + (n + 1)/2$. Lograremos esto si mostramos que

$$1+\frac{n}{2}+\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}\geq 1+\frac{n+1}{2}.$$

En general, para demostrar una desigualdad, reemplazamos algunos términos en la expresión más grande mediante términos más pequeños, de modo que la expresión resultante sea igual a la expresión menor, o bien reemplazamos algunos términos en la expresión menor con términos más grandes de modo que la expresión resultante sea igual a la expresión mayor. En este caso, reemplazaremos cada uno de los términos de la suma

$$\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}$$

mediante el término menor $1/2^{n+1}$. Obtenemos

$$\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}\geq\frac{1}{2^{n+1}}+\cdots+\frac{1}{2^{n+1}}.$$

Como existen 2^n términos en esta última suma, cada uno de los cuales es igual a $1/2^{n+1}$, podemos escribir esta última desigualdad como

$$\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}\geq\frac{1}{2^{n+1}}+\cdots+\frac{1}{2^{n+1}}=2^n\frac{1}{2^{n+1}}=\frac{1}{2}. \qquad (10)$$

Combinamos (9) y (10) para obtener

$$H_{2^{n+1}}\geq 1+\frac{n}{2}+\frac{1}{2}=1+\frac{n+1}{2}. \qquad (11)$$

Tenemos el resultado deseado, con lo que concluye el paso inductivo.

Solución formal

Podemos escribir la solución formal como sigue.

PASO BASE ($n = 0$)

$$H_{2^0}=1\geq 1=1+\frac{0}{2}$$

PASO INDUCTIVO. Supongamos (2). Ahora bien

$$\begin{aligned}H_{2^{n+1}}&=1+\frac{1}{2}+\cdots+\frac{1}{2^n}+\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}\\&=H_{2^n}+\frac{1}{2^n+1}+\cdots+\frac{1}{2^{n+1}}\geq 1+\frac{n}{2}+\frac{1}{2^{n+1}}+\cdots+\frac{1}{2^{n+1}}\\&=1+\frac{n}{2}+2^n\frac{1}{2^{n+1}}=1+\frac{n}{2}+\frac{1}{2}=1+\frac{n+1}{2}.\end{aligned}$$

Resumen de técnicas para resolver problemas

- Analice algunos ejemplos concretos de las expresiones en cuestión, por lo general con valores pequeños de las variables.
- Busque las expresiones para valores pequeños de n que aparezcan dentro de expresiones para valores mayores de n. En particular, el paso inductivo depende de la habilidad para relacionar el caso n con el caso $n + 1$.
- Retrase la agrupación y simplificación de términos lo más posible, para poder descubrir las relaciones entre las expresiones.
- Escriba con detalle aquello que debe demostrar, en específico, el valor más pequeño en el paso base, el caso n supuesto en el paso inductivo y el caso $n + 1$ por demostrar en el paso inductivo. Escriba las fórmulas para las diversas expresiones necesarias.
- Para demostrar una desigualdad, reemplace los términos en la expresión mayor con términos más pequeños, de modo que la expresión resultante sea igual a la expresión menor, o bien reemplace los términos de la expresión menor con términos más grandes de modo que la expresión resultante sea igual a la expresión mayor.

Comentarios

Los números H_k son los *números armónicos*, y la serie

$$1+\frac{1}{2}+\frac{1}{3}+\cdots$$

que aparece en cálculo es la *serie armónica*. La desigualdad (2) muestra que los números armónicos crecen sin límite. En la terminología del cálculo, la serie armónica *diverge*.

NOTAS

La bibliografía general relativa a las matemáticas discretas es [Dossey; Graham, 1988; Liu, 1985; Ross; Tucker]. [Knuth, 1973, volúmenes 1 y 3; 1981] es la referencia clásica para una gran parte del material.

[Barker; Copi; Edgar] son libros de texto de introducción a la lógica. Un análisis más avanzado aparece en [Davis]. El primer capítulo del libro de geometría de [Jacobs] está dedicado a la lógica básica. [Solow] estudia el problema de la construcción de demostraciones. Para una historia de la lógica, véase [Kline]. El papel del razonamiento lógico en los programas de cómputo se analiza en [Gries].

La formación de mosaicos con polimínos es el tema del libro de [Martin].

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 1.1
Lógica
Proposición
Conjunción: p y q, $p \wedge q$
Disyunción: p o q, $p \vee q$
Negación: no p, $\bar{p}$
Proposición compuesta
Tabla de verdad
O exclusivo de proposiciones p, q: p o q, pero no ambos

Sección 1.2

Proposición condicional: si p, entonces q; $p \rightarrow q$
Hipótesis
Conclusión
Condición necesaria
Condición suficiente
Recíproca de $p \rightarrow q$: $q \rightarrow p$
Proposición bicondicional: p si y sólo si q, $p \leftrightarrow q$
Equivalencia lógica: $P \equiv Q$
Leyes de De Morgan para la lógica: $\overline{p \vee q} \equiv \overline{p} \wedge \overline{q}$, $\overline{p \wedge q} \equiv \overline{p} \vee \overline{q}$
Contrapositiva de $p \rightarrow q$: $\overline{q} \rightarrow \overline{p}$

Sección 1.3

Función proposicional
Dominio de discurso
Cuantificador universal
Afirmación cuantificada universalmente
Contraejemplo
Cuantificador existencial
Afirmación cuantificada existencialmente
Leyes de De Morgan generalizadas para la lógica:
$\overline{\forall x, P(x)}$ y $\exists x, \overline{P(x)}$ tienen los mismos valores de verdad.
$\overline{\exists x, P(x)}$ y $\forall x, \overline{P(x)}$ tienen los mismos valores de verdad.
Para demostrar que la afirmación cuantificada universalmente

para cada x, $P(x)$

es verdadera, muestre que para cada x en el dominio de discurso, la proposición $P(x)$ es verdadera.
Para demostrar que la afirmación cuantificada universalmente

para alguna x, $P(x)$

es verdadera, determine un valor de x en el dominio de discurso para la cual la proposición $P(x)$ sea verdadera.
Para demostrar que la afirmación cuantificada universalmente

para cada x, $P(x)$

es falsa, determine un valor de x (un contraejemplo) en el dominio de discurso para el cual la proposición $P(x)$ sea falsa.
Para demostrar que la afirmación cuantificada universalmente

para alguna x, $P(x)$

es falsa, muestre que para cada x en el dominio de discurso, la proposición $P(x)$ es falsa.

Sección 1.4

Sistema matemático
Axioma
Definición
Término no definido
Teorema
Demostración
Lema
Demostración directa
Demostración por contradicción
Demostración indirecta
Demostración por contrapositiva
Razonamiento deductivo
Hipótesis
Premisas
Conclusión
Argumento
Argumento válido
Argumento no válido

Sección 1.5

Demostración por resolución; utiliza: Si $p \vee q$ y $\overline{p} \vee r$ son verdaderas, entonces $q \vee r$ es verdadera.
Cláusula: consta de términos separados por o, donde cada término es una variable o la negación de una variable.

Sección 1.6

Principio de inducción matemática
Paso base: demostrar la verdad de la afirmación para la primera instancia
Paso inductivo: suponer ciertas todas las instancias menores que n y demostrar que es cierta para n
Fórmula para la suma de los primeros n enteros positivos:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Suma geométrica:

$$a + ar + ar^2 + \cdots + ar^n = \frac{a(r^{n+1} - 1)}{r - 1}$$

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 1.1

1. Si p, q y r son verdaderas, determine el valor de verdad de la proposición $(p \vee q) \wedge \overline{((\overline{p} \wedge r) \vee q)}$.
2. Escriba la tabla de verdad de la proposición $\overline{(p \wedge q)} \vee (p \vee \overline{r})$.
3. Formule la proposición $p \wedge (\overline{q} \vee r)$ con palabras, utilizando

   p: Mi área es la administración hotelera.
   q: Mi área es la supervisión de diversiones.
   r: Mi área es la cultura popular.

4. Suponga que a, b y c son números reales. Represente la afirmación

   $$a < b \text{ o } (b < c \text{ y } a \geq c)$$

   en forma simbólica, haciendo

   $$p: a < b, \quad q: b < c, \quad r: a < c.$$

Sección 1.2

5. Enuncie la afirmación "Una condición necesaria para que Leah obtenga una buena calificación en matemáticas discretas es que estudie mucho" como una proposición condicional.
6. Escriba la recíproca y la contrapositiva de la proposición del ejercicio 5.
7. Si p es verdadera y q y r son falsas, determine el valor de verdad de la proposición

   $$(p \vee q) \rightarrow \overline{r}.$$

8. Represente la afirmación

   $$\text{Si } (a \geq c \text{ o } b < c), \text{ entonces } b \geq c$$

en forma simbólica, utilizando las definiciones del ejercicio 4.

Sección 1.3

9. ¿Es una proposición la siguiente afirmación?

   El equipo ganó el campeonato de la Asociación Nacional de Básquetbol en 1996.

   Justifique su respuesta.

10. ¿Es la afirmación del ejercicio 9 una función proposicional? Explique.

Sea $P(n)$ la afirmación

$$n \text{ y } n + 2 \text{ son primos.}$$

En los ejercicios 11 y 12, escriba la afirmación con palabras e indique si es verdadera o falsa.

11. Para todos los enteros positivos n, $P(n)$.
12. Para algún entero positivo n, $P(n)$.

Sección 1.4

13. Muestre, dando una demostración por contradicción, que si cuatro equipos juegan siete juegos, algún par de equipos juega al menos dos veces.
14. Distinga entre los términos *axioma* y *definición*.
15. ¿Cuál es la diferencia entre una demostración directa y una demostración por contradicción?

BIBLIOTECA
FACULTAD
Cs. EXACTAS, INGENIERÍA
Y AGRIMENSURA
ROSARIO

16. Determine si el siguiente argumento es válido.

$$\begin{array}{l} p \to q \vee r \\ p \vee \overline{q} \\ \hline \therefore q \end{array}$$

Sección 1.5

17. Determine una expresión, que sea la *conjunción* de cláusulas, equivalente a $(p \vee q) \to r$.

18. Determine una expresión, que sea la *conjunción* de cláusulas, equivalente a $(p \vee \overline{q}) \to \overline{r}s$.

19. Utilice resolución para demostrar

$$\begin{array}{l} \overline{p} \vee q \\ \overline{q} \vee \overline{r} \\ p \vee \overline{r} \\ \hline \therefore \overline{r} \end{array}$$

20. Demuestre de nuevo el ejercicio 19 utilizando la resolución y la demostración por contradicción.

Sección 1.6

Utilice inducción matemática para demostrar que las afirmaciones de los ejercicios 21-24 son verdaderas para cada entero positivo n.

21. $2 + 4 + \cdots + 2n = n(n+1)$

22. $2^2 + 4^2 + \cdots + (2n)^2 = \dfrac{2n(n+1)(2n+1)}{3}$

23. $\dfrac{1}{2!} + \dfrac{2}{3!} + \cdots + \dfrac{n}{(n+1)!} = 1 - \dfrac{1}{(n+1)!}$

24. $2^{n+1} < 1 + (n+1)2^n$

16. Determine si el siguiente argumento es válido.

$$\begin{array}{l} p \rightarrow q \vee r \\ p \vee \bar{q} \\ \hline \therefore q \end{array}$$

Sección 1.5

17. Determine una expresión, que sea la *conjunción* de cláusulas, equivalente a $(p \vee q) \rightarrow r$.

18. Determine una expresión, que sea la *conjunción* de cláusulas, equivalente a $(p \vee \bar{q}) \rightarrow \bar{r}s$.

19. Utilice resolución para demostrar

$$\begin{array}{l} \bar{p} \vee q \\ \bar{q} \vee \bar{r} \\ \hline p \vee \bar{r} \\ \hline \therefore \bar{r} \end{array}$$

20. Demuestre de nuevo el ejercicio 19 utilizando la resolución y la demostración por contradicción.

Sección 1.6

Utilice inducción matemática para demostrar que las afirmaciones de los ejercicios 21-24 son verdaderas para cada entero positivo n.

21. $2+4+\cdots+2n = n(n+1)$

22. $2^2+4^2+\cdots+(2n)^2 = \dfrac{2n(n+1)(2n+1)}{3}$

23. $\dfrac{1}{2!}+\dfrac{2}{3!}+\cdots+\dfrac{n}{(n+1)!} = 1-\dfrac{1}{(n+1)!}$

24. $2^{n+1} < 1+(n+1)2^n$

2

EL LENGUAJE DE LAS MATEMÁTICAS

2.1 CONJUNTOS
2.2 SUCESIONES Y CADENAS
2.3 SISTEMAS NUMÉRICOS
2.4 RELACIONES
RINCÓN DE SOLUCIÓN DE PROBLEMAS: RELACIONES
2.5 RELACIONES DE EQUIVALENCIA
RINCÓN DE SOLUCIÓN DE PROBLEMAS: RELACIONES DE EQUIVALENCIA
2.6 MATRICES DE RELACIONES
†2.7 BASES DE DATOS RELACIONALES
2.8 FUNCIONES
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Cuando YO utilizo una palabra, ésta quiere decir precisamente lo que yo quiero decir, ni más ni menos.

—tomado de Alicia en el país de las maravillas

Este capítulo trata acerca del lenguaje de las matemáticas. Los temas, algunos de los cuales son familiares para el lector, son los conjuntos, las sucesiones, los sistemas numéricos, las relaciones y las funciones. Todas las matemáticas, así como las áreas que se basan en éstas, como las ciencias de la computación y la ingeniería, hacen uso de estos conceptos fundamentales.

Un conjunto es una colección de objetos. Las matemáticas discretas trabajan con estructuras como gráficas (conjuntos de vértices y aristas) y álgebras booleanas (conjuntos con ciertas operaciones definidas sobre ellos).

A diferencia de un conjunto, una sucesión toma en cuenta el orden. Una lista de las letras, conforme éstas aparecen en una palabra, es un ejemplo de sucesión. (Es claro que en este caso es importante el orden, pues, por ejemplo, *caso* y *cosa* son palabras diferentes.)

Entre los sistemas numéricos están el familiar sistema decimal (base 10), así como los sistemas binario (base 2) y hexadecimal (base 16).

† Esta sección puede omitirse sin pérdida de continuidad.

Una relación es un conjunto de pares ordenados. La presencia del par ordenado (a, b) en una relación indica una relación entre a y b. El modelo de base de datos relacional que ayuda a los usuarios a obtener información en una base de datos (una colección de registros controlada por una computadora) se basa en el concepto de relación.

Una función, que es un tipo particular de relación, asigna a cada miembro de un conjunto X exactamente un miembro de un conjunto Y. Las funciones se utilizan ampliamente en las matemáticas discretas; por ejemplo, las funciones sirven para analizar el tiempo necesario para ejecutar los algoritmos.

2.1 CONJUNTOS

El concepto de conjunto es fundamental en todas las matemáticas y en las aplicaciones matemáticas. Un **conjunto** es simplemente una colección arbitraria de objetos. Si un conjunto es finito y no demasiado grande, podemos describirlo enumerando sus elementos. Por ejemplo, la ecuación

$$A = \{1, 2, 3, 4\} \tag{2.1.1}$$

describe un conjunto A formado por los cuatro elementos 1, 2, 3 y 4. Un conjunto queda determinado mediante sus elementos y no por algún orden particular en que se enumeren dichos elementos. Así, A también puede especificarse como

$$A = \{1, 3, 4, 2\}.$$

Se supone que los elementos que conforman un conjunto son distintos, y aunque por alguna razón podríamos tener duplicados en nuestra lista, sólo una ocurrencia de cada elemento está en el conjunto. Por esta razón, también podríamos describir al conjunto A definido en (2.1.1) como

$$A = \{1, 2, 2, 3, 4\}.$$

Si un conjunto es finito pero grande, o bien es infinito, podemos describirlo enunciando una propiedad necesaria para la pertenencia a dicho conjunto. Por ejemplo, la ecuación

$$B = \{x \mid x \text{ es un entero positivo par}\} \tag{2.1.2}$$

describe al conjunto B formado por todos los enteros positivos pares; es decir, B consta de los enteros 2, 4, 6, y así sucesivamente. La barra vertical "$\mid$" se lee "tal que". La ecuación (2.1.2) se leería entonces "B es igual al conjunto de todas las x tales que x es un entero positivo par". En este caso, la propiedad necesaria para la pertenencia es "es un entero positivo par". Observe que la propiedad aparece después de la barra vertical.

Si X es un conjunto finito, sea

$$|X| = \text{número de elementos en } X.$$

Dada una descripción de un conjunto X, como (2.1.1) o (2.1.2) y un elemento x, podemos determinar si éste pertenece o no a X. Si los miembros de X se enumeran como en (2.1.1), sólo revisamos la lista para ver si el elemento x aparece en la lista. En una descripción como (2.1.2), verificamos si el elemento x tiene la propiedad indicada. Si x está en el conjunto X, escribimos $x \in X$, y si x no está en X, escribimos $x \notin X$. Por ejemplo, si $x = 1$, entonces $x \in A$, pero $x \notin B$, donde A y B están dados por las ecuaciones (2.1.1) y (2.1.2).

El conjunto sin elementos es el **conjunto vacío** y se denota $\varnothing$. Así, $\varnothing = \{\ \}$.

Dos conjuntos X y Y son **iguales** y escribimos $X = Y$ si X y Y tienen los mismos elementos. Dicho de otra forma, $X = Y$ si siempre que $x \in X$, entonces $x \in Y$ y siempre que $x \in Y$, entonces $x \in X$.

EJEMPLO 2.1.1

Si

$$A = \{x \mid x^2 + x - 6 = 0\}, \qquad B = \{2, -3\},$$

entonces $A = B$. □

Supongamos que X y Y son conjuntos. Si todo elemento de X es un elemento de Y, decimos que X es un **subconjunto** de Y y escribimos $X \subseteq Y$.

EJEMPLO 2.1.2

Si

$$C = \{1, 3\} \qquad \text{y} \qquad A = \{1, 2, 3, 4\},$$

entonces C es un subconjunto de A. □

Cualquier conjunto X es un subconjunto de sí mismo, pues cualquier elemento en X está en X. Si X es un subconjunto de Y y X no es igual a Y, decimos que X es un **subconjunto propio** de Y. El conjunto vacío es un subconjunto de cualquier conjunto (véase el ejercicio 56). El conjunto de todos los subconjuntos (propios o no) de un conjunto X, denotado $P(X)$, es el **conjunto potencia** de X.

EJEMPLO 2.1.3

Si $A = \{a, b, c\}$, los miembros de $P(A)$ son

$$\varnothing, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}.$$

Todos los subconjuntos, excepto $\{a, b, c\}$, son subconjuntos propios de A. Para este ejemplo,

$$|A| = 3, \quad |P(A)| = 2^3 = 8.$$ □

Daremos una demostración por inducción de que el resultado del ejemplo 2.1.3 es válido en general; es decir, el conjunto potencia de un conjunto con n elementos tiene 2^n elementos.

TEOREMA 2.1.4

Si $|X| = n$, entonces

$$|P(X)| = 2^n. \tag{2.1.3}$$

Demostración. La demostración es por inducción sobre n.

PASO BASE. Si $n = 0$, X es el conjunto vacío. El único subconjunto del conjunto vacío es el propio conjunto vacío; así,

$$|P(X)| = 1 = 2^0 = 2^n.$$

Así, (2.1.3) es verdadera para $n = 0$.

PASO INDUCTIVO. Supongamos que (2.1.3) es válida para n. Sea X un conjunto con $n+1$ elementos. Elijamos $x \in X$. Afirmamos que exactamente la mitad de los subconjuntos de X contienen a x, y exactamente la mitad de los subconjuntos de X no contienen a x. Para ver esto, observe que cada subconjunto S de X que contenga a x puede asociarse de manera única con el subconjunto obtenido al eliminar x de S (véase la figura 2.1.1). Así, exactamente la mitad de los subconjuntos de X contienen a x, y exactamente la mitad de los subconjuntos de X no contienen a x.

Si Y es el conjunto obtenido de X al eliminar x, Y tiene n elementos. Por la hipótesis de inducción, $|P(Y)| = 2^n$. Pero los subconjuntos de Y son precisamente los subconjuntos de X que no contienen a x. Por el argumento del párrafo anterior, concluimos que

$$|P(Y)| = \frac{|P(X)|}{2}.$$

Por lo tanto,

$$|P(X)| = 2|P(Y)| = 2 \cdot 2^n = 2^{n+1}.$$

Así, (2.1.3) es válida para $n+1$ y esto concluye el paso inductivo. Por el principio de inducción matemática, (2.1.3) es válida para toda $n \geq 0$. ■

En la sección 4.1 (véase el ejemplo 4.1.4) se dará otra demostración del teorema 2.1.4.

Subconjuntos de X que contienen a a	*Subconjuntos de X que no contienen a a*
$\{a\}$	$\emptyset$
$\{a, b\}$	$\{b\}$
$\{a, c\}$	$\{c\}$
$\{a, b, c\}$	$\{b, c\}$

FIGURA 2.1.1
Los subconjuntos de $X = \{a, b, c\}$ divididos en dos clases: aquellos que contienen a a y aquellos que no contienen a a. Cada subconjunto de la derecha se obtiene del conjunto correspondiente en la columna de la izquierda eliminando el elemento a de éste.

Dados dos conjuntos X y Y, existen varias formas de combinar X y Y para formar un nuevo conjunto. El conjunto

$$X \cup Y = \{x \mid x \in X \text{ o } x \in Y\}$$

es la **unión** de X y Y. La unión consta de todos los elementos que pertenecen a X o a Y (o a ambos).

El conjunto

$$X \cap Y = \{x \mid x \in X \text{ y } x \in Y\}$$

es la **intersección** de X y Y. La intersección consta de todos los elementos que pertenecen a X y a Y.

Los conjuntos X y Y son **ajenos** si $X \cap Y = \emptyset$. Una colección de conjuntos $\mathcal{S}$ es ajena por pares si siempre que X y Y sean conjuntos distintos en $\mathcal{S}$, X y Y son ajenos.

El conjunto

$$X - Y = \{x \mid x \in X \text{ y } x \notin Y\}$$

es la **diferencia** (o **complemento relativo**). La diferencia $X - Y$ consta de todos los elementos en X que no están en Y.

EJEMPLO 2.1.5

Si $A = \{1, 3, 5\}$ y $B = \{4, 5, 6\}$, entonces

$$A \cup B = \{1, 3, 4, 5, 6\}$$
$$A \cap B = \{5\}$$
$$A - B = \{1, 3\}$$
$$B - A = \{4, 6\}.$$

□

EJEMPLO 2.1.6

Los conjuntos

$$\{1, 4, 5\} \quad \text{y} \quad \{2, 6\}$$

son ajenos. La colección de conjuntos

$$\mathcal{S} = \{\{1, 4, 5\}, \{2, 6\}, \{3\}, \{7, 8\}\}$$

es ajena por pares. □

A veces trabajaremos con varios conjuntos, todos los cuales serán subconjuntos de un conjunto U. Este conjunto U es un **conjunto universal** o **universo**. El conjunto U debe darse en forma explícita o inferirse del contexto. Dado un conjunto universal U y un subconjunto X de U, el conjunto $U - X$ es el **complemento** de X y se denota $\overline{X}$.

EJEMPLO 2.1.7

Sea $A = \{1, 3, 5\}$. Si el conjunto universal U se especifica como $U = \{1, 2, 3, 4, 5\}$, entonces $\overline{A} = \{2, 4\}$. Por otro lado, si el conjunto universal U se especifica como $U = \{1, 3, 5, 7, 9\}$, entonces $\overline{A} = \{7, 9\}$. Es claro que el complemento depende del universo con el cual estemos trabajando. □

Nuestro siguiente teorema resume algunas propiedades útiles de los conjuntos. La demostración se deja al lector (véase el ejercicio 70).

TEOREMA 2.1.8

Sean U un conjunto universal y A, B y C subconjuntos de U. Se cumplen las siguientes propiedades.

(a) Leyes asociativas:

$$(A \cup B) \cup C = A \cup (B \cup C), \quad (A \cap B) \cap C = A \cap (B \cap C)$$

(b) Leyes conmutativas:

$$A \cup B = B \cup A, \quad A \cap B = B \cap A$$

(c) Leyes distributivas:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$
$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

(d) Leyes del neutro y del idéntico:

$$A \cup \emptyset = A, \quad A \cap U = A$$

(e) Leyes de complementos:

$$A \cup \overline{A} = U, \quad A \cap \overline{A} = \varnothing$$

(f) Leyes de idempotencia:

$$A \cup A = A, \quad A \cap A = A$$

(g) Leyes de acotación:

$$A \cup U = U, \quad A \cap \varnothing = \varnothing$$

(h) Leyes de absorción:

$$A \cup (A \cap B) = A, \quad A \cap (A \cup B) = A$$

(i) Ley de involución:

$$\overline{\overline{A}} = A$$

(j) Leyes del 0/1:

$$\overline{\varnothing} = U, \quad \overline{U} = \varnothing$$

(k) Leyes de De Morgan para conjuntos:

$$\overline{(A \cup B)} = \overline{A} \cap \overline{B}, \quad \overline{(A \cap B)} = \overline{A} \cup \overline{B}$$

Demostración. Véase el ejercicio 70. ■

Definimos la unión de una familia arbitraria de conjuntos $\mathcal{S}$ como aquellos elementos x que pertenecen al menos a un conjunto X en $\mathcal{S}$. De manera formal,

$$\cup\mathcal{S} = \{\, x \mid x \in X \text{ para algún } X \in \mathcal{S} \,\}.$$

De manera análoga, definimos la intersección de una familia arbitraria $\mathcal{S}$ de conjuntos como aquellos elementos x que pertenecen a cada conjunto X en $\mathcal{S}$. De manera formal,

$$\cap\mathcal{S} = \{\, x \mid x \in X \text{ para todo } X \in \mathcal{S} \,\}.$$

Si

$$\mathcal{S} = \{A_1, A_2, \ldots, A_n\},$$

escribimos

$$\bigcup\mathcal{S} = \bigcup_{i=1}^{n} A_i, \qquad \bigcap\mathcal{S} = \bigcap_{i=1}^{n} A_i$$

y si

$$\mathcal{S} = \{A_1, A_2, \ldots\},$$

escribimos

$$\bigcup\mathcal{S} = \bigcup_{i=1}^{\infty} A_i, \qquad \bigcap\mathcal{S} = \bigcap_{i=1}^{\infty} A_i.$$

EJEMPLO 2.1.9

Si

$$A_n = \{n, n+1, \ldots\} \quad \text{y} \quad \mathcal{S} = \{A_1, A_2, \ldots\},$$

entonces

$$\bigcup_{i=1}^{\infty} A_i = \bigcup\mathcal{S} = \{1, 2, \ldots\}, \qquad \bigcap_{i=1}^{\infty} A_i = \bigcap\mathcal{S} = \varnothing. \qquad \square$$

Una partición de un conjunto X divide a X en subconjuntos que no se traslapan. Más formalmente, una colección $\mathcal{S}$ de subconjuntos no vacíos de X es una **partición** del conjunto X si todo elemento de X pertenece exactamente a un miembro de $\mathcal{S}$. Observe que si $\mathcal{S}$ es una partición de X, $\mathcal{S}$ es ajena por pares y $\cup\mathcal{S} = X$.

EJEMPLO 2.1.10

Como cada elemento de

$$X = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

está exactamente en un miembro de

$$\mathcal{S} = \{\{1, 4, 5\}, \{2, 6\}, \{3\}, \{7, 8\}\},$$

$\mathcal{S}$ es una partición de X. $\square$

Al inicio de esta sección señalamos que un conjunto es una colección no ordenada de elementos; es decir, un conjunto queda determinado por sus elementos y no por algún orden particular de enumerar éstos. Sin embargo, a veces es necesario tomar en cuenta el orden. Un **par ordenado** de elementos, que se escribe (a, b), se considera distinto del par ordenado (b, a), a menos, por supuesto, que $a = b$. Dicho de otra forma, $(a, b) = (c, d)$ si y sólo si $a = c$ y $b = d$. Si X y Y son conjuntos, $X \times Y$ denota el conjunto de todos los pares ordenados (x, y) tales que $x \in X$ y $y \in Y$. $X \times Y$ es el **producto cartesiano** de X y Y.

EJEMPLO 2.1.11

Si $X = \{1, 2, 3\}$ y $Y = \{a, b\}$, entonces

$$X \times Y = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$$
$$Y \times X = \{(a, 1), (b, 1), (a, 2), (b, 2), (a, 3), (b, 3)\}$$
$$X \times X = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)\}$$
$$Y \times X = \{(a, a), (a, b), (b, a), (b, b)\}. \qquad \square$$

El ejemplo 2.1.11 muestra que, en general, $X \times Y \neq Y \times X$. Observe que $|X \times Y| = |X| \cdot |Y|$.

EJEMPLO 2.1.12

Un restaurante sirve cuatro entradas

$$r = \text{costillas}, \quad n = \text{nachos}, \quad s = \text{camarón}, \quad f = \text{queso fundido}$$

y tres platos principales

$$c = \text{pollo}, \quad b = \text{filete de res}, \quad t = \text{trucha}.$$

Si $A = \{r, n, s, f\}$ y $M = \{c, b, t\}$, el producto cartesiano $A \times M$ indica las 12 posibles comidas que constan de una entrada y un plato principal. □

Las listas ordenadas no tienen por qué ser de dos elementos. Una ***n*-ada**, que se escribe $(a_1, a_2, \ldots, a_n)$, toma en cuenta el orden:

$$(a_1, a_2, \ldots, a_n) = (b_1, b_2, \ldots, b_n)$$

si y sólo si

$$a_1 = b_1, a_2 = b_2, \ldots, a_n = b_n.$$

El producto cartesiano de conjuntos $X_1, X_2, \ldots, X_n$ se define como el conjunto de todas las n-adas $(x_1, x_2, \ldots, x_n)$, donde $x_i \in X_i$ para $i = 1, \ldots, n$.

EJEMPLO 2.1.13

Si

$$X = \{1, 2\}, \qquad Y = \{a, b\}, \qquad Z = \{\alpha, \beta\},$$

entonces

$$X \times Y \times Z = \{(1, a, \alpha), (1, a, \beta), (1, b, \alpha), (1, b, \beta), (2, a, \alpha),$$
$$(2, a, \beta), (2, b, \alpha), (2, b, \beta)\}.$$ □

Observe que en el ejemplo 2.1.13, $|X \times Y \times Z| = |X| \cdot |Y| \cdot |Z|$. En general, tenemos

$$|X_1 \times X_2 \times \cdots \times X_n| = |X_1| \cdot |X_2| \cdots |X_n|. \tag{2.1.4}$$

Esta última afirmación puede demostrarse por inducción sobre el número n de conjuntos (véase el ejercicio 71).

EJEMPLO 2.1.14

Si A es un conjunto de entradas, M un conjunto de platos principales y D un conjunto de postres, el producto cartesiano $A \times M \times D$ enumera todas las comidas posibles que constan de una entrada, un plato principal y un postre. □

Ejercicios

En los ejercicios 1-16, considere como universo al conjunto $U = \{1, 2, 3, \ldots, 10\}$. Sean $A = \{1, 4, 7, 10\}$, $B = \{1, 2, 3, 4, 5\}$ y $C = \{2, 4, 6, 8\}$. Enumere los elementos de cada conjunto.

1. $A \cup B$
2. $B \cap C$
3. $A - B$
4. $B - A$
5. $\overline{A}$
6. $U - C$
7. $\overline{U}$
8. $A \cup \varnothing$
9. $B \cap \varnothing$
10. $A \cup U$
11. $B \cap U$
12. $A \cap (B \cup C)$
13. $\overline{B} \cap (C - A)$
14. $(A \cap B) - C$
15. $\overline{A \cap B} \cup C$
16. $(A \cup B) - (C - B)$

En los ejercicios 17-20, sean $X = \{1, 2\}$ y $Y = \{a, b, c\}$. Enumere los elementos de cada conjunto.

17. $X \times Y$
18. $Y \times X$
19. $X \times X$
20. $Y \times Y$

En los ejercicios 21-24, sean $X = \{1, 2\}$, $Y = \{a\}$ y $Z = \{\alpha, \beta\}$. Enumere los elementos de cada conjunto.

21. $X \times Y \times Z$
22. $X \times Y \times Y$
23. $X \times X \times X$
24. $Y \times X \times Y \times Z$

En los ejercicios 25-28, enumere todas las particiones del conjunto.

25. $\{1\}$
26. $\{1, 2\}$
27. $\{a, b, c\}$
28. $\{a, b, c, d\}$

En los ejercicios 29-32, responda cierto o falso.

29. $\{x\} \subseteq \{x\}$
30. $\{x\} \in \{x\}$
31. $\{x\} \in \{x, \{x\}\}$
32. $\{x\} \subseteq \{x, \{x\}\}$

En los ejercicios 33-37, determine si cada par de conjuntos son iguales.

33. $\{1, 2, 3\}, \{1, 3, 2\}$

34. $\{1, 2, 2, 3\}, \{1, 2, 3\}$

35. $\{1, 1, 3\}, \{3, 3, 1\}$

36. $\{x \mid x^2 + x = 2\}, \{1, -2\}$

37. $\{x \mid x$ es un número real y $0 < x \leq 2\}, \{1, 2\}$

38. Enumere los elementos de $P(\{a, b\})$. ¿Cuáles son subconjuntos propios de $\{a, b\}$?

39. Enumere los elementos de $P(\{a, b, c, d\})$. ¿Cuáles son subconjuntos propios de $\{a, b, c, d\}$?

40. Si X tiene 10 elementos, ¿cuántos miembros tiene $P(X)$? ¿Cuántos subconjuntos propios tiene X?

41. Si X tiene n elementos, ¿cuántos subconjuntos propios tiene X?

42. Si X y Y son conjuntos no vacíos y $X \times Y = Y \times X$, ¿qué podemos concluir acerca de X y Y?

En cada uno de los ejercicios 43-55, escriba "verdadero" si la afirmación es verdadera; en caso contrario, proporcione un contraejemplo. Los conjuntos X, Y y Z son subconjuntos de un conjunto universal U. Suponga que el universo para los productos cartesianos es $U \times U$.

43. Para cualesquiera conjuntos X y Y, X es un subconjunto de Y o Y es un subconjunto de X.

44. $X \cap (Y - Z) = (X \cap Y) - (X \cap Z)$ para todos los conjuntos X, Y y Z

45. $(X - Y) \cap (Y - X) = \emptyset$ para todos los conjuntos X y Y

46. $X - (Y \cup Z) = (X - Y) \cup Z$ para todos los conjuntos X, Y y Z

47. $\overline{X - Y} = \overline{Y - X}$ para todos los conjuntos X y Y

48. $\overline{X \cap Y} \subseteq X$ para todos los conjuntos X y Y

49. $(X \cap Y) \cup (Y - X) = X$ para todos los conjuntos X y Y

50. $X \times (Y \cup Z) = (X \times Y) \cup (X \times Z)$ para todos los conjuntos X, Y y Z

51. $\overline{X \times Y} = \overline{X} \times \overline{Y}$ para todos los conjuntos X y Y

52. $X \times (Y - Z) = (X \times Y) - (X \times Z)$ para todos los conjuntos X, Y y Z

53. $X - (Y \times Z) = (X - Y) \times (X - Z)$ para todos los conjuntos X, Y y Z

54. $X \cap (Y \times Z) = (X \cap Y) \times (X \cap Z)$ para todos los conjuntos X, Y y Z

55. $X \times \emptyset = \emptyset$. para todo conjunto X

56. Muestre que para cualquier conjunto X, $\emptyset \subseteq X$.

Para cada una de las condiciones de los ejercicios 57-60, ¿cuál es la relación que debe existir entre los conjuntos A y B?

57. $A \cap B = A$ 58. $A \cup B = A$

59. $\overline{A} \cap U = \emptyset$ **60.** $\overline{A \cap B} = \overline{B}$

La *diferencia simétrica* de dos conjuntos A y B es el conjunto

$$A \Delta B = (A \cup B) - (A \cap B).$$

61. Si $A = \{1, 2, 3\}$ y $B = \{2, 3, 4, 5\}$, determine $A \Delta B$.

62. Describa la diferencia simétrica de los conjuntos A y B con palabras.

63. Dado un universo U, describa $A \Delta A$, $A \Delta \overline{A}$, $U \Delta A$ y $\emptyset \Delta A$.

64. Demuestre o dé un contraejemplo: Si A, B y C son conjuntos que satisfacen $A \Delta C = B \Delta C$, entonces $A = B$.

65. Muestre que

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

66. Determine una fórmula para $|A \cup B \cup C|$ similar a la fórmula del ejercicio 65. Muestre que su fórmula es válida para cualesquiera conjuntos A, B y C.

67. Sea C un círculo y D el conjunto de todos los diámetros de C. ¿Qué es $\cap D$?

68. Sea P el conjunto de los enteros mayores que 1. Para $i \geq 2$, defina

$$X_i = \{ik \mid k \geq 2, k \in P\}.$$

Describa $P - \bigcup_{i=2}^{\infty} X_i$.

59. Utilice la inducción para mostrar que si $X_1, \ldots, X_n$ y X son conjuntos, entonces

(a) $X \cap (X_1 \cup X_2 \cup \cdots \cup X_n) = (X \cap X_1) \cup (X \cap X_2) \cup \cdots \cup (X \cap X_n)$.

(b) $\overline{X_1 \cap X_2 \cap \cdots \cap X_n} = \overline{X}_1 \cup \overline{X}_2 \cup \cdots \cup \overline{X}_n$.

70. Demuestre el teorema 2.1.8.

71. Utilice la inducción para demostrar la afirmación (2.1.4).

2.2 *SUCESIONES Y CADENAS*

La compañía Blue Taxi cobra \$1 por la primera milla y 50 centavos por cada milla adicional. La tabla anexa muestra el costo de recorrido de 1 a 10 millas. En general, el costo C_n de recorrido de n millas es 1.00 (el costo de recorrido de la primera milla) más 0.50 veces el número $(n - 1)$ de millas adicionales. Es decir,

Millaje	*Costo*
1	\$1.00
2	1.50
3	2.00
4	2.50
5	3.00
6	3.50
7	4.00
8	4.50
9	5.00
10	5.50

$$C_n = 1 + 0.5(n - 1).$$

Por ejemplo,

$$C_1 = 1 + 0.5(1 - 1) = 1 + 0.5 \cdot 0 = 1,$$

$$C_5 = 1 + 0.5(5 - 1) = 1 + 0.5 \cdot 4 = 1 + 2 = 3.$$

Una **sucesión** es una lista donde se toma en cuenta el orden. En el ejemplo anterior, la lista de tarifas

$$1.00, \quad 1.50, \quad 2.00, \quad 2.50, \quad 3.00, \ldots$$

es una sucesión. Observe que el orden realmente es importante. Por ejemplo, si se intercambian el primero y el quinto números, la tarifa por una milla sería \$3.00, un poco distinta a la tarifa de \$1.00.

Si s es una sucesión, con frecuencia denotamos el primer elemento de la sucesión como s_1, el segundo elemento de la sucesión como s_2, y así sucesivamente. En general, s_n denota el n-ésimo elemento de la sucesión. n es el **índice** de la sucesión.

EJEMPLO 2.2.1

La lista ordenada

$$2, \quad 4, \quad 6, \ldots, \quad 2n, \ldots$$

es una sucesión. El primer elemento de la sucesión es 2, el segundo es 4, y así sucesivamente. El n-ésimo elemento de la sucesión es $2n$. Si s denota esta sucesión, tenemos

$$s_1 = 2, \quad s_2 = 4, \quad s_3 = 6, \ldots, \quad s_n = 2n, \ldots. \qquad \square$$

EJEMPLO 2.2.2

La lista ordenada

$$a, \quad a, \quad b, \quad a, \quad b$$

es una sucesión. El primer elemento de la sucesión es a, el segundo es a, y así sucesivamente. Si t denota esta sucesión, tenemos

$$t_1 = a, \quad t_2 = a, \quad t_3 = b, \quad t_4 = a, \quad t_5 = b. \qquad \square$$

El ejemplo 2.2.2 muestra que una sucesión (a diferencia de un conjunto) puede tener repeticiones. Una sucesión puede tener una infinidad de elementos (como la sucesión del ejemplo 2.2.1) o un número finito de elementos (como la sucesión del ejemplo 2.2.2).

Una notación alternativa para la sucesión s es $\{s_n\}$. En este caso, s o $\{s_n\}$ denota la sucesión completa

$$s_1, \quad s_2, \quad s_3, \quad \ldots.$$

Utilizamos la notación s_n para denotar al n-ésimo elemento de la sucesión s.

EJEMPLO 2.2.3

Defina una sucesión $\{t_n\}$ mediante la regla

$$t_n = n^2 - 1, \quad n \geq 1.$$

Los primeros cinco términos de esta sucesión son

$$0, \quad 3, \quad 8, \quad 15, \quad 24.$$

El término número 55 es

$$t_{55} = 55^2 - 1 = 3024. \qquad \square$$

EJEMPLO 2.2.4

Defina una sucesión u mediante la regla, u_n está dada por la n-ésima letra de la palabra *digital*. Entonces $u_1 = d, u_2 = u_4 = i$ y $u_7 = l$. Esta sucesión es finita. $\square$

Aunque en este libro se denotará al primer elemento de una sucesión s por lo general como s_1, este primer elemento puede quedar indicado por cualquier entero. Por ejemplo, si v es una sucesión cuyo primer elemento es v_0, los elementos de v serían

$$v_0, \quad v_1, \quad v_2, \quad \ldots.$$

Cuando queremos mencionar de manera explícita el índice inicial de una sucesión finita s, escribimos $\{s_n\}_{n=1}^{\infty}$. Una sucesión infinita v cuyo índice inicial es 0 se denota $\{v_n\}_{n=0}^{\infty}$. Una sucesión finita x con índices de -1 a 4 se denota $\{x_n\}_{n=-1}^{4}$.

EJEMPLO 2.2.5

Si x es la sucesión definida por

$$x_n = 1/2^n, \quad -1 \leq n \leq 4,$$

los elementos de x son

$$2, \quad 1, \quad 1/2, \quad 1/4, \quad 1/8, \quad 1/16. \qquad \square$$

Dos tipos importantes de sucesiones son las sucesiones crecientes y las decrecientes.† Una sucesión s es **creciente** si $s_n \leq s_{n+1}$ para toda n. Una sucesión s es **decreciente** si $s_n \geq s_{n+1}$ para toda n. Observe que en ambas definiciones se permite la igualdad entre los términos sucesivos de la sucesión.

EJEMPLO 2.2.6

La sucesión 2, 4, 6, . . . , del ejemplo 2.2.1 es creciente, pues $s_n = 2n \leq 2(n+1) = s_{n+1}$ para toda n. $\square$

EJEMPLO 2.2.7

La sucesión 2, 1, 1/2, . . . , del ejemplo 2.2.5 es decreciente, pues $x_n = 1/2^n \geq 1/2^{n+1} = x_{n+1}$ para toda n. $\square$

EJEMPLO 2.2.8

La sucesión s

$$3, \quad 5, \quad 5, \quad 7, \quad 8, \quad 8, \quad 13$$

es creciente, pues $s_n \leq s_{n+1}$ para toda n. Observe que $s_2 = s_3$ y $s_5 = s_6$ (suponemos que el índice del primer término de la sucesión es 1). La igualdad se permite en la definición de *sucesión creciente*. $\square$

Una forma de crear una nueva sucesión a partir de una sucesión dada es conservar solamente algunos términos de la sucesión original, manteniendo el orden de los términos en la sucesión dada. La sucesión resultante es una **subsucesión** de la sucesión original.

DEFINICIÓN 2.2.9

Sea $\{s_n\}$ una sucesión definida para $n = m, m+1, \ldots$, y sea $n_1, n_2, \ldots$ una sucesión creciente que satisface $n_k < n_{k+1}$, para toda k, cuyos valores están en el conjunto $\{m, m+1, \ldots\}$. Decimos que la sucesión $\{s_{n_k}\}$ es una *subsucesión* de $\{s_n\}$.

EJEMPLO 2.2.10

La sucesión

$$b, c \tag{2.2.1}$$

es una subsucesión de la sucesión

$$t_1 = a, \quad t_2 = a, \quad t_3 = b, \quad t_4 = c, \quad t_5 = q. \tag{2.2.2}$$

† En algunos libros, lo que llamamos *creciente* se llama *no decreciente* o lo que llamamos *decreciente* se llama *no creciente*.

La subsucesión (2.2.1) se obtiene de la sucesión (2.2.2) eligiendo el tercer y cuarto términos. La expresión n_k de la definición 2.2.9 nos dice cuáles términos de (2.2.2) debemos elegir para obtener la subsucesión (2.2.1); así, $n_1 = 3$, $n_2 = 4$. La subsucesión (2.2.1) es

$$t_3,\ t_4 \quad \text{o} \quad t_{n_1},\ t_{n_2}.$$

Observe que la sucesión

$$c, b$$

no es una subsucesión de la sucesión (2.2.2), pues no se conserva el orden de los términos en la sucesión (2.2.2). □

EJEMPLO 2.2.11

La sucesión

$$2,\quad 4,\quad 8,\quad 16,\quad \ldots,\quad 2^k,\quad \ldots \tag{2.2.3}$$

es una subsucesión de la sucesión

$$2,\quad 4,\quad 6,\quad 8,\quad 10,\quad 12,\quad 14,\quad 16,\ldots,\quad 2n,\ldots. \tag{2.2.4}$$

La subsucesión (2.2.3) se obtiene de la sucesión (2.2.4) eligiendo el primer, segundo, cuarto, octavo, etc., términos; así, el valor de n_k de la definición 2.2.9 es $n_k = 2^{k-1}$. Si definimos la sucesión (2.2.4) como $s_n = 2n$, la subsucesión (2.2.3) queda definida por

$$s_{n_k} = s_{2^{k-1}} = 2 \cdot 2^{k-1} = 2^k.$$

Para crear nuevas sucesiones a partir de sucesiones numéricas, dos formas importantes consisten en sumar y multiplicar los términos entre ellos.

DEFINICIÓN 2.2.12

Si $\{a_i\}_{i=m}^{n}$ es una sucesión, definimos

$$\sum_{i=m}^{n} a_i = a_m + a_{m+1} + \cdots + a_n, \qquad \prod_{i=m}^{n} a_i = a_m \cdot a_{m+1} \cdots a_n.$$

El formalismo

$$\sum_{i=m}^{n} a_i \tag{2.2.5}$$

es la *notación* de *suma* (o *sigma*) y

$$\prod_{i=m}^{n} a_i \tag{2.2.6}$$

es la *notación producto*.

En (2.2.5) o (2.2.6), i es el *índice*, m es el *límite inferior* y n es el *límite superior*.

EJEMPLO 2.2.13

Sea a la sucesión definida mediante $a_n = 2n$, $n \geq 1$. Entonces

$$\sum_{i=1}^{3} a_i = a_1 + a_2 + a_3 = 2 + 4 + 6 = 12,$$

$$\prod_{i=1}^{3} a_i = a_1 \cdot a_2 \cdot a_3 = 2 \cdot 4 \cdot 6 = 48.$$

□

EJEMPLO 2.2.14

La suma geométrica (véase el ejemplo 1.6.2)

$$a + ar + ar^2 + \cdots + ar^n$$

puede escribirse de manera más compacta mediante la notación de suma, como

$$\sum_{i=0}^{n} ar^i.$$

□

El nombre del índice en (2.2.5) o (2.2.6) no es importante. Por ejemplo,

$$\sum_{i=1}^{n} a_i = \sum_{j=1}^{n} a_j \qquad \text{y} \qquad \prod_{i=1}^{n} a_i = \prod_{x=1}^{n} a_x.$$

A veces no sólo es útil cambiar el nombre del índice, sino también los límites. (El proceso es análogo a cambiar la variable en una integral.)

EJEMPLO 2.2.15 ***Cambio del índice y los límites en una suma***

Reescriba la suma

$$\sum_{i=0}^{n} ir^{n-i},$$

reemplazando el índice i por j, donde $i = j - 1$.

Como $i = j - 1$, el término ir^{n-i} se convierte en

$$(j-1)r^{n-(j-1)} = (j-1)r^{n-j+1}.$$

Como $j = i + 1$, cuando $i = 0$, $j = 1$. Así, el límite inferior para j es 1. De manera análoga, cuando $i = n$, $j = n + 1$ y el límite superior para j es $n + 1$. Por lo tanto,

$$\sum_{i=0}^{n} ir^{n-i} = \sum_{j=1}^{n+1} (j-1)r^{n-j+1}.$$

□

EJEMPLO 2.2.16

Sea a la sucesión definida mediante la regla $a_n = 2(-1)^n$, $n \geq 0$. Determinar una fórmula para la sucesión definida como

$$s_n = \sum_{i=0}^{n} a_i.$$

Vemos que

$$s_n = 2(-1)^0 + 2(-1)^1 + 2(-1)^2 + \cdots + 2(-1)^n$$
$$= 2 - 2 + 2 - \cdots \pm 2 = \begin{cases} 2, & \text{si } n \text{ es par} \\ 0, & \text{si } n \text{ es impar.} \end{cases}$$

□

Las notaciones de suma y producto pueden modificarse para denotar sumas y productos indicados por conjuntos arbitrarios de enteros. Formalmente, si S es un conjunto de enteros y a es una sucesión,

$$\sum_{i \in S} a_i$$

denota la suma de los elementos $\{a_i \mid i \in S\}$. De manera análoga,

$$\prod_{i \in S} a_i$$

denota el producto de los elementos $\{a_i \mid i \in S\}$.

EJEMPLO 2.2.17

Si S denota el conjunto de números primos menores que 20,

$$\sum_{i \in S} \frac{1}{i} = \frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \frac{1}{11} + \frac{1}{13} + \frac{1}{17} + \frac{1}{19} = 1.455.$$

□

En ciertos contextos, una sucesión finita se llama **cadena**.

DEFINICIÓN 2.2.18

Una *cadena sobre X* es una sucesión finita de elementos de X.

EJEMPLO 2.2.19

Sea $X = \{a, b, c\}$. Si

$$\beta_1 = b, \quad \beta_2 = a, \quad \beta_3 = a, \quad \beta_4 = c,$$

obtenemos una cadena sobre X. Esta cadena se escribe $baac$. □

Como una cadena es una sucesión, es importante el orden. Por ejemplo, la cadena $baac$ es diferente de la cadena $acab$.

Las repeticiones en una cadena pueden especificarse mediante superíndices. Por ejemplo, la cadena $bbaaac$ puede escribirse b^2a^3c.

La cadena sin elementos es la **cadena nula** y se denota λ. Sea X^* el conjunto de todas las cadenas sobre X, incluyendo la cadena nula, y sea X^+ el conjunto de todas las cadenas no nulas sobre X.

EJEMPLO 2.2.20

Sea $X = \{a, b\}$. Algunos elementos de X^* son

$$\lambda, \quad a, \quad b, \quad abab, \quad b^{20}a^5ba.$$

□

La **longitud** de una cadena α es el número de elementos en α. La longitud de α se denota $|\alpha|$.

EJEMPLO 2.2.21

Si $\alpha = aabab$ y $\beta = a^3b^4a^{32}$, entonces

$$|\alpha| = 5 \quad \text{y} \quad |\beta| = 39.$$

□

Si α y β son dos cadenas, la cadena formada por α seguida de β, la cual se escribe $\alpha\beta$, es la **concatenación** de α y β.

EJEMPLO 2.2.22

Si $\gamma = aab$ y $\theta = cabd$, entonces

$$\gamma\theta = aabcabd, \quad \theta\gamma = cabdaab, \quad \gamma\lambda = \gamma = aab, \quad \lambda\gamma = \gamma = aab.$$

□

Ejercicios

1. Responda (a)-(c) para la sucesión s definida por

$$c, d, d, c, d, c.$$

(a) Determine s_1. (b) Determine s_4.
(c) Escriba s como una cadena.

2. Responda (a)-(k) para la sucesión t definida por
$$t_n = 2n - 1, \quad n \geq 1.$$
(a) Determine t_3. (b) Determine t_7.
(c) Determine t_{100}. (d) Determine t_{2077}.
(e) Determine $\sum_{i=1}^{3} t_i$. (f) Determine $\sum_{i=3}^{7} t_i$.
(g) Determine $\prod_{i=1}^{3} t_i$. (h) Determine $\prod_{i=3}^{6} t_i$.
(i) Determine una fórmula que represente esta sucesión como aquella en la que el índice inferior sea 0.
(j) ¿Es t creciente? (k) ¿Es t decreciente?

3. Responda (a)-(f) para la sucesión v definida como
$$v_n = (n-1)\cdots 2 \cdot 1 + 2, \quad n \geq 1.$$
(a) Determine v_3. (b) Determine v_4.
(c) Determine $\sum_{i=1}^{4} v_i$. (d) Determine $\sum_{i=3}^{3} v_i$.
(e) ¿Es v creciente? (f) ¿Es v decreciente?

4. Calcule la cantidad dada utilizando la sucesión a definida como
$$a_n = n^2 - 3n + 3, \quad n \geq 1.$$
(a) $\sum_{i=1}^{4} a_i$ (b) $\sum_{j=3}^{5} a_j$
(c) $\sum_{i=4}^{4} a_i$ (d) $\sum_{k=1}^{6} a_k$
(e) $\prod_{i=1}^{2} a_i$ (f) $\prod_{i=1}^{3} a_i$
(g) $\prod_{n=2}^{3} a_n$ (h) $\prod_{x=3}^{4} a_x$

5. Responda (a) y (b) para la sucesión a del ejercicio 4.
(a) ¿Es a creciente? (b) ¿Es a decreciente?

6. Responda (a)-(f) para la sucesión b definida como $b_n = n(-1)^n$.
(a) Determine $\sum_{i=1}^{4} b_i$. (b) Determine $\sum_{i=1}^{10} b_i$.
(c) Determine una fórmula para la sucesión c definida como
$$c_n = \sum_{i=1}^{n} b_i.$$
(d) Determine una fórmula para la sucesión d definida como
$$d_n = \prod_{i=1}^{n} b_i.$$
(e) ¿Es b creciente? (f) ¿Es b decreciente?

7. Responda (a)-(f) para la sucesión Ω definida como $\Omega_n = 3$ para toda n.
(a) Determine $\sum_{i=1}^{3} \Omega_i$. (b) Determine $\sum_{i=1}^{10} \Omega_i$.
(c) Determine una fórmula para la sucesión c definida como
$$c_n = \sum_{i=1}^{n} \Omega_i.$$
(d) Determine una fórmula para la sucesión d definida como
$$d_n = \prod_{i=1}^{n} \Omega_i.$$
(e) ¿Es Ω creciente? (f) ¿Es Ω decreciente?

8. Responda (a)-(e) para la sucesión x definida como
$$x_1 = 2 \quad x_n = 3 + x_{n-1}, \quad n \geq 2.$$
(a) Determine $\sum_{i=1}^{3} x_i$. (b) Determine $\sum_{i=1}^{10} x_i$.
(c) Determine una fórmula para la sucesión c definida como
$$c_n = \sum_{i=1}^{n} x_i.$$
(d) ¿Es x creciente? (e) ¿Es x decreciente?

9. Responda (a)-(f) para la sucesión w definida como
$$w_n = \frac{1}{n} - \frac{1}{n+1}, \quad n \geq 1.$$
(a) Determine $\sum_{i=1}^{3} w_i$. (b) Determine $\sum_{i=1}^{10} w_i$.
(c) Determine una fórmula para la sucesión c definida como
$$c_n = \sum_{i=1}^{n} w_i.$$
(d) Determine una fórmula para la sucesión d definida como
$$d_n = \prod_{i=1}^{n} w_i.$$
(e) ¿Es w creciente? (f) ¿Es w decreciente?

10. Sea u la sucesión definida como
$$u_1 = 3 \quad u_n = 3 + u_{n-1}, \quad n \geq 2.$$
Determine una fórmula para la sucesión d definida como
$$d_n = \prod_{i=1}^{n} u_i.$$

11. Defina $\{s_n\}$ mediante la regla

$$s_n = 2n - 1, \quad n \geq 1.$$

Considere la subsucesión de s obtenida con el primer, tercer, quinto, . . . términos.

(a) Enumere los primeros siete términos de s.

(b) Enumere los primeros siete términos de la subsucesión.

(c) Determine una fórmula para la expresión n_k de la definición 2.2.9.

(d) Determine una fórmula para el k-ésimo término de la subsucesión.

12. Defina $\{t_n\}$ mediante la regla

$$t_n = 2^n, \quad n \geq 1.$$

Considere la subsucesión de t obtenida al considerar el primer, segundo, cuarto, séptimo, undécimo, . . . términos.

(a) Enumere los primeros siete términos de t.

(b) Enumere los primeros siete términos de la subsucesión.

(c) Determine una fórmula para la expresión n_k de la definición 2.2.9.

(d) Determine una fórmula para el k-ésimo término de la subsucesión.

13. Responda (a)-(d) con las sucesiones y y z definidas por

$$y_n = 2^n - 1, \quad z_n = n(n-1).$$

(a) Determine $\left(\sum_{i=1}^{3} y_i\right)\left(\sum_{i=1}^{3} z_i\right)$.

(b) Determine $\left(\sum_{i=1}^{5} y_i\right)\left(\sum_{i=1}^{4} z_i\right)$.

(c) Determine $\sum_{i=1}^{3} y_i z_i$.

(d) Determine $\left(\sum_{i=3}^{4} y_i\right)\left(\prod_{i=2}^{4} z_i\right)$.

14. Responda (a)-(h) para la sucesión r definida como

$$r_n = 3 \cdot 2^n - 4 \cdot 5^n, \quad n \geq 0.$$

(a) Determine r_0.

(b) Determine r_1.

(c) Determine r_2.

(d) Determine r_3.

(e) Determine una fórmula para r_p.

(f) Determine una fórmula para r_{n-1}.

(g) Determine una fórmula para r_{n-2}.

(h) Muestre que $\{r_n\}$ satisface

$$r_n = 7r_{n-1} - 10r_{n-2}, \quad n \geq 2.$$

15. Responda (a)-(h) para la sucesión z definida como

$$z_n = (2 + n)3^n, \quad n \geq 0.$$

(a) Determine z_0.

(b) Determine z_1.

(c) Determine z_2.

(d) Determine z_3.

(e) Determine una fórmula para z_i.

(f) Determine una fórmula para z_{n-1}.

(g) Determine una fórmula para z_{n-2}.

(h) Muestre que $\{z_n\}$ satisface

$$z_n = 6z_{n-1} - 9z_{n-2}, \quad n \geq 2.$$

16. Determine b_i, $i = 1, \ldots, 6$, donde

$$b_n = 2[1 + (n-1)(n-2)(n-3)(n-4)(n-5)] + \frac{(n-1)n}{2}.$$

17. Reescriba la suma

$$\sum_{i=1}^{n} i^2 r^{n-1}$$

reemplazando el índice i por k, donde $i = k + 1$.

18. Reescriba la suma

$$\sum_{k=1}^{n} C_{k-1} C_{n-k}$$

reemplazando el índice k por i, donde $k = i + 1$.

19. Sean a y b sucesiones, y sea

$$s_k = \sum_{i=1}^{k} a_i.$$

Muestre que

$$\sum_{k=1}^{n} a_k b_k = \sum_{k=1}^{n} s_k (b_k - b_{k+1}) + s_n b_{n+1}.$$

Esta ecuación, conocida como la *fórmula de suma por partes*, es el análogo discreto para la fórmula de integración por partes del cálculo.

20. Podemos generalizar el concepto de sucesión definido en esta sección utilizando índices más generales. Supongamos que $\{a_{ij}\}$ es una sucesión cuyos índices es una pareja de enteros positivos. Muestre que

$$\sum_{i=1}^{n}\left(\sum_{j=1}^{n} a_{ij}\right) = \sum_{j=1}^{n}\left(\sum_{i=1}^{j} a_{ij}\right).$$

21. Calcule lo que se pide utilizando las cadenas

$$\alpha = baab, \quad \beta = caaba, \quad \gamma = bbab.$$

(a) $\alpha\beta$ (b) $\beta\alpha$ (c) $\alpha\alpha$ (d) $\beta\beta$

(e) $|\alpha\beta|$ (f) $|\beta\alpha|$ (g) $|\alpha\alpha|$ (h) $|\beta\beta|$

(i) $\alpha\lambda$ (j) $\lambda\beta$ (k) $\alpha\beta\gamma$ (l) $\beta\beta\gamma\alpha$

22. Enumere todas las cadenas de longitud 2 sobre $X = \{0, 1\}$.

23. Enumere todas las cadenas de longitud 2 o menor sobre $X = \{0, 1\}$.

24. Enumere todas las cadenas de longitud 3 sobre $X = \{0, 1\}$.

25. Enumere todas las cadenas de longitud 3 o menor sobre $X = \{0, 1\}$.

26. Una cadena s es una **subcadena** de t si existen cadenas u y v tales que $t = usv$. Determine todas las subcadenas de la cadena *babc*.

27. Determine todas las subcadenas de la cadena *aabaabb*.

28. Utilice inducción para mostrar que

$$\sum \frac{1}{n_1 \cdot n_2 \cdots n_k} = n,$$

donde la suma se toma sobre todos los subconjuntos no vacíos $\{n_1, n_2, \ldots, n_k\}$ de $\{1, 2, \ldots, n\}$.

2.3 SISTEMAS NUMÉRICOS

Un **bit** es un dígito binario (la palabra bit proviene de *b*inary dig*it*), es decir, un 0 o un 1. En una computadora digital, los datos y las instrucciones se codifican mediante bits. (El término *digital* se refiere al uso de los dígitos 0 y 1.) La tecnología determina la forma física de representar los bits dentro de un sistema de cómputo. El hardware actual se basa en el estado de un circuito electrónico para representar un bit. El circuito debe poder estar en dos estados (uno que represente 1, y el otro 0). En esta sección analizaremos el **sistema numérico binario**, el cual representa a los enteros mediante bits, y el **sistema numérico hexadecimal**, el cual representa los enteros mediante 16 símbolos. El **sistema numérico octal**, que representa a los enteros mediante ocho símbolos, se analiza antes del ejercicio 35.

Para representar los enteros en el sistema numérico decimal, utilizamos los diez símbolos 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9. Al representar un entero, es importante la posición de los símbolos; al leer de derecha a izquierda, el primer símbolo representa el número de unidades, el siguiente símbolo el número de decenas, el siguiente símbolo el número de centenas, y así sucesivamente (véase la figura 2.3.1). En general, el símbolo en la posición n (donde el símbolo de la extrema derecha está en la posición 0) representa el número de magnitud 10^n. Como $10^0 = 1$, el símbolo en la posición 0 representa el número de magnitud 10^0, o unidades; como $10^1 = 10$, el símbolo en la posición 1 representa el número de magnitud 10^1, o decenas; como $10^2 = 100$, el símbolo en la posición 2 representa el número de magnitud 10^2, o centenas; y así sucesivamente. El valor sobre el cual se basa el sistema (10 en el caso del sistema decimal) es la **base** del sistema numérico.

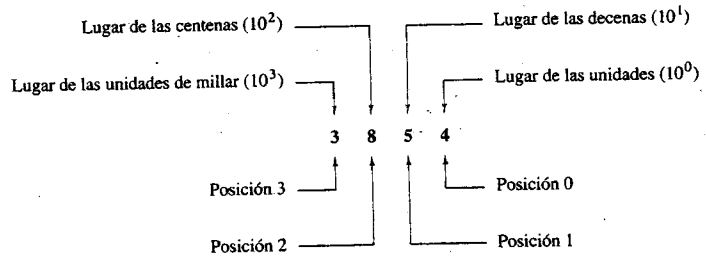


FIGURA 2.3.1 Sistema numérico decimal.

En el sistema numérico binario (base 2), sólo necesitamos dos símbolos (0 y 1) para representar los enteros. En la representación de un entero, leída de derecha a izquierda, el primer símbolo representa el número de unos, el siguiente símbolo el número de doses, el siguiente símbolo el número de cuatros, el siguiente símbolo el número de ochos, y así sucesivamente (véase la figura 2.3.2). En general, el símbolo en la posición n (donde el símbolo de la extrema derecha ocupa la posición 0) representa el número de magnitud 2^n. Como $2^0 = 1$, el símbolo en la posición 0 representa el número de 2^0, o de unos; como $2^1 = 2$, el símbolo en la posición 1 representa el número de 2^1, o de doses; como $2^2 = 4$, el símbolo en la posición 2 representa el número de 2^2, o de cuatros; y así sucesivamente.

Sin saber cuál sistema numérico se esté utilizando, una representación es ambigua; por ejemplo, 101101 representa un número en decimal y otro número muy distinto en binario. Con frecuencia, el contexto indica el sistema numérico en uso, pero cuando se quiere ser absolutamente claro, colocamos un número como subíndice para especificar la base (el subíndice 10 denota el sistema decimal y el subíndice 2 denota el sistema binario). Por ejemplo, el número binario 101101_2 puede escribirse 101101_2.

EJEMPLO 2.3.1 ***De binario a decimal***

El número binario 101101_2 representa al número que consta de un 1, ningún 2, un 4, un 8, ningún 16 y un 32 (véase la figura 2.3.2). Esta representación puede expresarse como

$$101101_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

Al calcular el lado derecho en decimal, tenemos que

$$\begin{aligned} 101101_2 &= 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 \\ &= 32 + 8 + 4 + 1 + 45_{10}. \end{aligned}$$

□

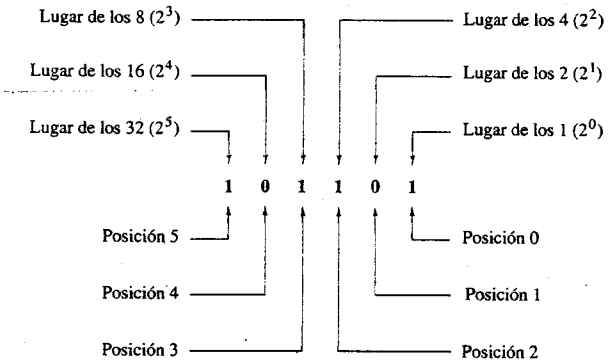


FIGURA 2.3.2 Sistema numérico binario.

El ejemplo 2.3.1 muestra la forma de convertir un número binario a decimal. Consideremos el problema inverso, convertir un número decimal a binario. Supongamos, por ejemplo, que queremos convertir el número decimal 91 a binario. Si dividimos 91 entre 2, obtenemos

$$\begin{array}{r} 45 \\ 2\overline{)91} \\ \underline{8} \\ 11 \\ \underline{10} \\ 1 \end{array}$$

Este cálculo muestra que

$$91 = 2 \cdot 45 + 1. \tag{2.3.1}$$

Comenzamos expresando 91 en potencias de 2. Si a continuación dividimos 45 entre 2, tenemos

$$45 = 2 \cdot 22 + 1. \tag{2.3.2}$$

Al sustituir esta expresión para 45 en (2.3.1), obtenemos

$$\begin{aligned} 91 &= 2 \cdot 45 + 1 \\ &= 2 \cdot (2 \cdot 22 + 1) + 1 \\ &= 2^2 \cdot 22 + 2 + 1. \end{aligned} \tag{2.3.3}$$

Si ahora dividimos 22 entre 2, tenemos que

$$22 = 2 \cdot 11.$$

Al sustituir esta expresión para 22 en (2.3.3), obtenemos

$$\begin{aligned} 91 &= 2^2 \cdot 22 + 2 + 1 \\ &= 2^2 \cdot (2 \cdot 11) + 2 + 1 \\ &= 2^3 \cdot 11 + 2 + 1. \end{aligned} \tag{2.3.4}$$

Si ahora dividimos 11 entre 2, tenemos que

$$11 = 2 \cdot 5 + 1.$$

Al sustituir esta expresión para 11 en (2.3.4), obtenemos

$$91 = 2^4 \cdot 5 + 2^3 + 2 + 1. \tag{2.3.5}$$

Si ahora dividimos 5 entre 2, tenemos

$$5 = 2 \cdot 2 + 1.$$

Al sustituir esta expresión para 5 en (2.3.5), obtenemos

$$\begin{aligned} 91 &= 2^5 \cdot 2 + 2^4 + 2^3 + 2 + 1 \\ &= 2^6 + 2^4 + 2^3 + 2 + 1 \\ &= 1011011_2. \end{aligned}$$

El cálculo anterior muestra que los *residuos*, cuando N se divide de manera sucesiva entre 2, proporcionan los bits en la representación binaria de N. La primera división entre (2.3.1) proporciona el bit de los unos; la segunda división entre 2 en (2.3.2) proporciona el bit de los 2, y así sucesivamente. Ilustraremos esto con otro ejemplo.

EJEMPLO 2.3.2 *De decimal a binario*

Escribir el número decimal 130 en binario.

El cálculo muestra las divisiones sucesivas entre 2, con los residuos registrados a la derecha.

2)130	residuo = 0	bit de los 1
2)65	residuo = 1	bit de los 2
2)32	residuo = 0	bit de los 4
2)16	residuo = 0	bit de los 8
2)8	residuo = 0	bit de los 16
2)4	residuo = 0	bit de los 32
2)2	residuo = 0	bit de los 64
2)1	residuo = 1	bit de los 128
0		

Podemos concluir el proceso cuando el dividendo es 0. Al recordar que el primer residuo proporciona el número de los unos, el segundo residuo proporciona el número de los 2, y así sucesivamente, obtenemos

$$130_{10} = 10000010_2. \qquad \square$$

Ahora analizaremos la suma de números con bases arbitrarias. El mismo método utilizado para sumar números decimales puede utilizarse para sumar números binarios; sin embargo, debemos reemplazar la tabla de la suma decimal con la tabla de suma binaria

+	**0**	**1**
0	0	1
1	1	10

(En decimal, $1 + 1 = 2$, y $2_{10} = 10_2$; así, en binario, $1 + 1 = 10$.)

EJEMPLO 2.3.3 *Suma binaria*

Sumar los números binarios 10011011 y 1011011.

Escribimos el problema como

$$\begin{array}{r} 10011011 \\ +\ 1011011 \\ \hline \end{array}$$

Como en la suma decimal, comenzamos por la derecha, sumando 1 y 1. La suma es 10_2; así, escribimos 0 y llevamos 1. En este momento el cálculo es

$$\begin{array}{r} 1 \\ 10011011 \\ +\ 1011011 \\ \hline 0 \end{array}$$

A continuación, sumamos 1 más 1 más 1, que es 11_2. Escribimos 1 y llevamos 1. En este momento, el cálculo es

$$\begin{array}{r} \ 1 \\ 10011011 \\ +\ \ 1011011 \\ \hline \ \ 10 \end{array}$$

Continuamos de esta manera, y obtenemos

$$\begin{array}{r} 10011011 \\ +\ \ 1011011 \\ \hline 11110110 \end{array}$$

□

EJEMPLO 2.3.4

El problema de suma del ejemplo 2.3.3, en decimal, es

$$\begin{array}{r} \ 155 \\ +\ \ 91 \\ \hline \ 246 \end{array}$$

□

Otras bases importantes para los sistemas numéricos en las ciencias de la computación son la base 8 u **octal** y la base 16 o **hexadecimal** (a veces se abrevia como **hex**). Analizaremos el sistema hexadecimal y dejaremos el sistema octal para los ejercicios (véanse los ejercicios 35-40).

En el sistema numérico hexadecima utilizamos los símbolos 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E y F para representar los enteros. Los símbolos A-F se interpretan como los decimales 10-15. (En general, en el sistema numérico de base N, se necesitan N símbolos distintos, los cuales representan $0, 1, 2, \ldots, N-1$.) En la representación de un entero, comenzando por la derecha, el primer símbolo representa el número de 1, el siguiente símbolo el número de 16, el siguiente símbolo el número de 16^2, y así sucesivamente (véase la figura 2.3.3). En general, el símbolo en la posición n (donde el símbolo de la extrema derecha está en la posición 0) representa el número de $16n$.

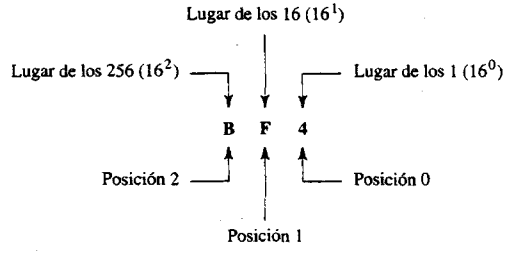


FIGURA 2.3.3 Sistema numérico hexadecimal.

EJEMPLO 2.3.5 *De hexadecimal a decimal*

Convierta el número hexadecimal B4F a decimal.

Obtenemos

$$B4F_{16} = 11 \cdot 16^2 + 4 \cdot 16^1 + 15 \cdot 16^0$$

$$= 11 \cdot 256 + 4 \cdot 16 + 15 = 2816 + 64 + 15 = 2895_{10}.$$

□

Para convertir un número decimal a hexadecimal, dividimos entre 16 de manera sucesiva. Los residuos proporcionan los símbolos hexadecimales.

EJEMPLO 2.3.6 *De decimal a hexadecimal*

Convierta el número decimal 20385 a hexadecimal.

El cálculo muestra las sucesivas divisiones entre 16, con los residuos a la derecha.

$16\overline{)20385}$	residuo = 1	lugar de los 1
$16\overline{)1274}$	residuo = 10	lugar de los 16
$16\overline{)79}$	residuo = 15	lugar de los 16^2
$16\overline{)4}$	residuo = 4	lugar de los 16^3
0		

Podemos concluir el proceso cuando el dividendo es 0. El primer residuo proporciona el número de 1, el segundo residuo proporciona el número de 16, y así sucesivamente, con lo cual obtenemos

$$20385_{10} = 4FA1_{16}.$$

□

Nuestro siguiente ejemplo muestra que podemos sumar números hexadecimales de la misma forma en que sumamos números decimales o binarios.

EJEMPLO 2.3.7 *Suma hexadecimal*

Sume los números hexadecimales 84F y 42EA.

El problema puede escribirse

$$\begin{array}{r} \ 84F \\ +\ 42EA \\ \hline \end{array}$$

Comenzamos con la primera columna de la derecha, sumando F y A. Como F es 15_{10} y A es 10_{10}, $F + A = 15_{10} + 10_{10} = 25_{10} = 19_{16}$. Escribimos 9 y llevamos 1:

$$\begin{array}{r} \ \ 1 \\ \ 84F \\ +\ 42EA \\ \hline \ \ 9 \end{array}$$

Ahora sumamos 1, 4 y E, obteniendo 13_{16}. Escribimos 3 y llevamos 1:

$$\begin{array}{r} {\scriptstyle 1} \\ 84F \\ +\ 42EA \\ \hline 39 \end{array}$$

Continuamos de esta forma para obtener

$$\begin{array}{r} 84F \\ +\ 42EA \\ \hline 4B39 \end{array}$$

□

EJEMPLO 2.3.8

El problema de la suma del ejemplo 2.3.7 escrito en decimal es

$$\begin{array}{r} 2127 \\ +\ 17130 \\ \hline 19257 \end{array}$$

□

Ejercicios

En los ejercicios 1-6, exprese cada número binario en decimal.

1. 1001 2. 11011 3. 11011011

4. 100000 5. 11111111 6. 110111011011

En los ejercicios 7-12, exprese cada número decimal en binario.

7. 34 8. 61 9. 223

10. 400 11. 1024 12. 12,340

En los ejercicios 13-18, sume los números binarios.

13. 1001 + 1111 14. 11011 + 1101

15. 110110 + 101101

16. 101101 + 11011

17. 110110101 + 1101101

18. 1101 + 101100 + 11011011

En los ejercicios 19-24, exprese cada número hexadecimal en decimal.

19. 3A 20. 1E9 21. 3E7C

22. A03 23. 209D 24. 4B07A

25. Exprese cada número decimal de los ejercicios 7-12 en hexadecimal.

26. Exprese cada número binario de los ejercicios 1-6 en hexadecimal.

27. Exprese cada número hexadecimal de los ejercicios 19, 20 y 22 en binario.

En los ejercicios 28-32, sume los números hexadecimales.

28. 4A + B4 29. 195 + 76E 30. 49F7 + C66

31. 349CC + 922D 32. 82054 + AEFA3

33. ¿Representa 2010 un número en binario?, ¿en decimal?, ¿en hexadecimal?

34. ¿Representa 1101010 un número en binario?, ¿en decimal?, ¿en hexadecimal?

En el sistema numérico octal (base 8) utilizamos los símbolos 0, 1, 2, 3, 4, 5, 6 y 7 para representar un entero; leyendo desde la derecha, el primer símbolo representa el número de 1, el siguiente símbolo el número de 8, el siguiente símbolo el número de 8^2, y así sucesivamente. En general, el símbolo en la posición n (donde el símbolo de la extrema derecha está en la posición 0) representa el número de 8^n. En los ejercicios 35-40, exprese cada número octal en decimal.

35. 63 36. 7643 37. 7711

38. 10732 39. 1007 40. 537261

41. Exprese cada número decimal en los ejercicios 7-12 en octal.

42. Exprese cada número binario en los ejercicios 1-6 en octal.

43. Exprese cada número hexadecimal en los ejercicios 19-24 en octal.

44. Exprese cada número octal en los ejercicios 35-40 en hexadecimal.

45. ¿Representa 1101010 un número en octal?

46. ¿Representa 30470 un número en binario?, ¿en octal?, ¿en decimal?, ¿en hexadecimal?

47. ¿Representa 9450 un número en binario?, ¿en octal?, ¿en decimal?, ¿en hexadecimal?

48. Sea T_n la mayor potencia de 2 que divida a n. Muestre que $T_{mn} = T_m + T_n$ para toda m, $n \geq 1$.

49. Sea S_n el número de unos en la representación binaria. Utilice inducción para demostrar que $T_{n!} = n - S_n$ para toda $n \geq 1$. (T_n se define en el ejercicio 48.)

2.4 *RELACIONES*

Una **relación** puede pensarse como una tabla que enumera la relación de algunos elementos con otros (véase la tabla 2.4.1). La tabla 2.4.1 muestra cuáles estudiantes están asistiendo a cuáles cursos. Por ejemplo, Bill está cursando Ciencias de la computación y Arte, y Mary está cursando Matemáticas. En la terminología de las relaciones, podríamos decir que Bill está relacionado con Ciencias de la computación y Arte, y que Mary está relacionada con Matemáticas.

Por supuesto, la tabla 2.4.1 es tan sólo un conjunto de pares ordenados. De manera abstracta, *definimos* una relación como un conjunto de pares ordenados. En este contexto, consideramos que el primer elemento del par ordenado se relaciona con el segundo elemento del par ordenado.

TABLA 2.4.1
Relación de los estudiantes con los cursos

Estudiante	*Curso*
Bill	Ciencias de la computación
Mary	Matemáticas
Bill	Arte
Beth	Historia
Beth	Ciencias de la computación
Dave	Matemáticas

DEFINICIÓN 2.4.1

Una *relación (binaria) R de un conjunto X en un conjunto Y* es un subconjunto del producto cartesiano $X \times Y$. Si $(x, y) \in R$, escribimos $x\ R\ y$ y decimos que *x está relacionada con y*. Si $X = Y$, decimos que R es una *relación (binaria) sobre X*.

El conjunto

$$\{x \in X \mid (x, y) \in R \text{ para algún } y \in Y\}$$

es el *dominio* de R. El conjunto

$$\{y \in Y \mid (x, y) \in R \text{ para algún } x \in X\}$$

es el *rango* de R.

Si una relación se indica mediante una tabla, el dominio está formado por los miembros de la primera columna y el rango consta de los miembros de la segunda columna.

EJEMPLO 2.4.2

Si

$$X = \{\text{Bill, Mary, Beth, Dave}\}$$

y

$$Y = \{\text{Ciencias de la computación, Matemáticas, Arte, Historia}\}$$

nuestra relación R de la tabla 2.4.1 puede escribirse

$$R = \{(\text{Bill, Ciencias de la computación}),\ (\text{Mary, Matemáticas}),\ (\text{Bill, Arte}),\ (\text{Beth, Historia}),\ (\text{Beth, Ciencias de la computación}),\ (\text{Dave, Matemáticas})\}.$$

Como (Beth, Historia) $\in R$, podemos escribir Beth R Historia. El dominio (primera columna) de R es el conjunto X y el rango (segunda columna) de R es el conjunto Y. □

El ejemplo 2.4.2 muestra que para indicar una relación basta especificar los pares ordenados que pertenecen a dicha relación. Nuestro siguiente ejemplo muestra que a veces es posible definir una relación proporcionando una regla para la pertenencia a la relación.

EJEMPLO 2.4.3

Sean

$$X = \{2, 3, 4\} \quad \text{y} \quad Y = \{3, 4, 5, 6, 7\}.$$

Si definimos una relación R de X en Y como

$$(x, y) \in R \text{ si } x \text{ divide a } y \text{ (con residuo igual a cero)},$$

obtenemos

$$R = \{(2, 4), (2, 6), (3, 3), (3, 6), (4, 4)\}.$$

Si escribimos R como una tabla, obtenemos

X	Y
2	4
2	6
3	3
3	6
4	4

El dominio de R es el conjunto $\{2, 3, 4\}$ y el rango de R es el conjunto $\{3, 4, 6\}$. □

EJEMPLO 2.4.4

Sea R la relación sobre $X = \{1, 2, 3, 4\}$ definida como $(x, y) \in R$ si $x \leq y$, $x, y \in X$. Entonces

$$R = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (2, 3), (2, 4), (3, 3), (3, 4), (4, 4)\}.$$

Tanto el dominio como el rango de R son iguales a X. □

Una manera útil de graficar una relación sobre un conjunto utiliza su **digráfica**. (Las digráficas se analizan con detalle en el capítulo 6. Por ahora, sólo mencionaremos las digráficas en conexión con las relaciones.) Para trazar la digráfica de una relación sobre un conjunto X, primero dibujamos puntos o **vértices** para representar los elementos de X. En la figura 2.4.1 hemos trazado cuatro vértices para representar los elementos del conjunto X del ejemplo 2.4.4. A continuación, si el elemento (x, y) está en la relación, trazamos una flecha (llamada **arista dirigida**) de x a y. En la figura 2.4.1 hemos trazado las aristas dirigidas que representan los miembros de la relación R del ejemplo 2.4.4. Observe que un elemento de la forma (x, x) en una relación corresponde a una arista dirigida de x a x. Dicha arista es un **lazo**. En cada vértice de la figura 2.4.1 aparece un lazo.

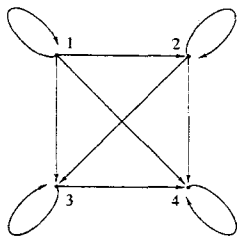


FIGURA 2.4.1
La digráfica de la relación del ejemplo 2.4.4.

EJEMPLO 2.4.5

La relación R sobre $X = \{a, b, c, d\}$ dada por la digráfica de la figura 2.4.2 es

$$R = \{(a, a), (b, c), (c, b), (d, d)\}.$$ □

A continuación definimos varias propiedades de las relaciones.

DEFINICIÓN 2.4.6

Una relación R sobre un conjunto X es *reflexiva* si $(x, x) \in R$ para cada $x \in X$.

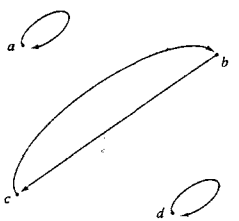


FIGURA 2.4.2
La digráfica de la relación del ejemplo 2.4.5.

EJEMPLO 2.4.7

La relación R sobre $X = \{1, 2, 3, 4\}$ del ejemplo 2.4.4 es reflexiva, pues para cada elemento $x \in X$, $(x, x) \in R$; específicamente, $(1, 1)$, $(2, 2)$, $(3, 3)$ y $(4, 4)$ están en R. La digráfica de una relación reflexiva tiene un lazo en cada vértice. Observe que la digráfica de la relación reflexiva del ejemplo 2.4.4 (véase la figura 2.4.1) tiene un lazo en cada vértice. □

EJEMPLO 2.4.8

La relación R sobre $X = \{a, b, c, d\}$ del ejemplo 2.4.5 no es reflexiva. Por ejemplo, $b \in X$, pero $(b, b) \notin R$. El hecho de que esta relación no sea reflexiva también se ve en su digráfica (véase la figura 2.4.2); el vértice b no tiene un lazo. □

DEFINICIÓN 2.4.9

Una relación R sobre un conjunto X es *simétrica* si para toda $x, y \in X$, si $(x, y) \in R$, entonces $(y, x) \in R$.

EJEMPLO 2.4.10

La relación del ejemplo 2.4.5 es simétrica, pues para toda x, y, si $(x, y) \in R$, entonces $(y, x) \in R$. Por ejemplo, (b, c) está en R y (c, b) también está en R. La digráfica de una relación simétrica tiene la siguiente propiedad: siempre que exista una arista dirigida de v a w, también existe una arista dirigida de w a v. Observe que la digráfica de la relación del ejemplo 2.4.5 (véase la figura 2.4.2) tiene la propiedad de que para cada arista dirigida de v a w, también existe una arista dirigida de w a v. □

EJEMPLO 2.4.11

La relación del ejemplo 2.4.4 no es simétrica. Por ejemplo, $(2, 3) \in R$, pero $(3, 2) \notin R$. La digráfica de esta relación (véase la figura 2.4.1) tiene una arista dirigida de 2 a 3, pero no tiene una arista dirigida de 3 a 2. □

DEFINICIÓN 2.4.12

Una relación R sobre un conjunto X es *antisimétrica* si para toda $x, y \in X$, si $(x, y) \in R$ y $x \neq y$, entonces $(y, x) \notin R$.

EJEMPLO 2.4.13

La relación del ejemplo 2.4.4 es antisimétrica, pues para toda x, y, si $(x, y) \in R$ y $x \neq y$, entonces $(y, x) \notin R$. Por ejemplo $(1, 2) \in R$, pero $(2, 1) \notin R$. La digráfica de una relación antisimétrica tiene la propiedad de que entre cualesquiera dos vértices existe a lo más una arista dirigida. Observe que la digráfica de la relación del ejemplo 2.4.4 (véase la figura 2.4.1) tiene a lo más una arista dirigida entre cada par de vértices. □

EJEMPLO 2.4.14

La relación del ejemplo 2.4.5 no es antisimétrica pues (b, c) y (c, b) están en R. Observe que en la digráfica de la relación del ejemplo 2.4.5 (véase la figura 2.4.2) existen dos aristas dirigidas entre b y c. □

EJEMPLO 2.4.15

Si una relación R sobre X no tiene elementos de la forma (x, y) con $x \neq y$, entonces R es antisimétrica. En este caso, si x y y son elementos arbitrarios en X, la proposición

$$\text{si } (x, y) \in R \text{ y } x \neq y, \text{ entonces } (y, x) \notin R$$

es verdadera, pues la hipótesis es falsa. Por ejemplo,

$$R = \{(a, a), (b, b), (c, c)\}$$

sobre $X = \{a, b, c\}$ es antisimétrica. La digráfica de R, que aparece en la figura 2.4.3, tiene a lo más una arista dirigida entre cada par de vértices. Observe que R también es reflexiva y simétrica. Este ejemplo muestra que "antisimétrica" no es lo mismo que "no simétrica". □

FIGURA 2.4.3 La digráfica de la relación del ejemplo 2.4.15.

DEFINICIÓN 2.4.16

Una relación R sobre un conjunto X es *transitiva* si para toda $x, y, z \in X$, si (x, y) y $(y, z) \in R$, entonces $(x, z) \in R$.

EJEMPLO 2.4.17

La relación R del ejemplo 2.4.4 es transitiva, pues para toda x, y, z, si (x, y) y $(y, z) \in R$, entonces $(x, z) \in R$. Para verificar formalmente que esta relación satisface la definición 2.4.16, debemos enumerar todas las parejas de pares de la forma (x, y) y (y, z) en R, y luego verificar que, en cada caso, $(x, z) \in R$:

Pares de la forma			*Pares de la forma*		
(x, y),	(y, z)	(x, z)	(x, y),	(y, z)	(x, z)
(1, 1)	(1, 1)	(1, 1)	(2, 2)	(2, 2)	(2, 2)
(1, 1)	(1, 2)	(1, 2)	(2, 2)	(2, 3)	(2, 3)
(1, 1)	(1, 3)	(1, 3)	(2, 2)	(2, 4)	(2, 4)
(1, 1)	(1, 4)	(1, 4)	(2, 3)	(3, 3)	(2, 3)
(1, 2)	(2, 2)	(1, 2)	(2, 3)	(3, 4)	(2, 4)
(1, 2)	(2, 3)	(1, 3)	(2, 4)	(4, 4)	(2, 4)
(1, 2)	(2, 4)	(1, 4)	(3, 3)	(3, 3)	(3, 3)
(1, 3)	(3, 3)	(1, 3)	(3, 3)	(3, 4)	(3, 4)
(1, 3)	(3, 4)	(1, 4)	(3, 4)	(4, 4)	(3, 4)
(1, 4)	(4, 4)	(1, 4)	(4, 4)	(4, 4)	(4, 4)

Cuando hay que determinar si una relación R es transitiva directamente a partir de la definición 2.4.16, en el caso $x = y$ o $y = z$ no hay que verificar de manera explícita la condición

$$\text{si } (x, y) \text{ y } (y, z) \in R, \text{ entonces } (x, z) \in R,$$

pues ésta se satisface de manera automática. Supongamos, por ejemplo, que $x = y$ y que (x, y) y (y, z) están en R. Como $x = y$, $(x, z) = (y, z)$ está en R y se satisface la condición. Al eliminar los casos $x = y$ y $y = z$ sólo hay que verificar lo siguiente de manera explícita, para ver que la relación del ejemplo 2.4.4 es transitiva:

Pares de la forma (x, y), (y, z)		(x, z)
(1, 2)	(2, 3)	(1, 3)
(1, 2)	(2, 4)	(1, 4)
(1, 3)	(3, 4)	(1, 4)
(2, 3)	(3, 4)	(2, 4)

La digráfica de una relación transitiva tiene la propiedad de que siempre que existan aristas dirigidas de x a y y de y a z, también existe una arista dirigida de x a z. Observe que la digráfica de la relación del ejemplo 2.4.4 (véase la figura 2.4.1) tiene esta propiedad. □

EJEMPLO 2.4.18

La relación del ejemplo 2.4.5 no es transitiva. Por ejemplo, (b, c) y (c, b) están en R, pero (b, b) no está en R. Observe que en la digráfica (véase la figura 2.4.2) de la relación del ejemplo 2.4.5 existen aristas dirigidas de b a c y de c a b, pero no existe una arista dirigida de b a b. □

Las relaciones pueden utilizarse para ordenar los elementos de un conjunto. Por ejemplo, la relación R definida sobre el conjunto de los enteros como

$$(x, y) \in R \quad \text{si } x \leq y$$

ordena a los enteros. Observe que la relación R es reflexiva, antisimétrica y transitiva. Tales relaciones se llaman **órdenes parciales**.

DEFINICIÓN 2.4.19

Una relación R sobre un conjunto X es un *orden parcial* si R es reflexiva, antisimétrica y transitiva.

EJEMPLO 2.4.20

Como la relación R definida sobre los enteros positivos mediante

$$(x, y) \in R \quad \text{si } x \text{ divide a } y \text{ (exactamente)}$$

es reflexiva, antisimétrica y transitiva, tenemos que R es un orden parcial. □

Si R es un orden parcial sobre un conjunto X, a veces se utiliza la notación $x \leq y$ para indicar que $(x, y) \in R$. Esta notación sugiere que estamos interpretando la relación como un orden sobre los elementos de X.

Supongamos que R es un orden parcial sobre un conjunto X. Si $x, y \in X$ y $x \leq y$ o $y \leq x$, decimos que x y y son **comparables**. Si $x, y \in X$ y $x \not\leq y$ y $x \not\leq y$, decimos que x y y **no son comparables**. Si cada par de elementos en X son comparables, decimos que R es un **orden total**. La relación menor o igual sobre los enteros positivos es un orden total, pues si x y y son enteros, entonces $x \leq y$ o $y \leq x$. La razón del término "orden parcial" es que, en general, algunos elementos de X pueden no ser comparables. La relación "divide" sobre los enteros positivos (véase el ejemplo 2.4.20) tiene elementos comparables y elementos no comparables. Por ejemplo, 2 y 3 no son comparables (pues 2 no divide a 3 y 3 no divide a 2), pero 3 y 6 sí lo son (pues 3 divide a 6).

Una aplicación de los órdenes parciales es la planeación de tareas.

EJEMPLO 2.4.21 *Planeación de tareas*

Consideremos el conjunto T de tareas que pueden realizarse para tomar una fotografía con flash en un interior.

1. Quitar la tapa de la lente.
2. Enfocar la cámara.
3. Quitar la cerradura de seguridad.
4. Activar la unidad de flash.
5. Oprimir el botón para tomar la fotografía.

Algunas de estas tareas deben realizarse antes que otras. Por ejemplo, la tarea 1 debe llevarse a cabo antes de la tarea 2. Por otro lado, otras tareas pueden efectuarse en cualquier orden. Por ejemplo, las tareas 2 y 3 pueden realizarse en cualquier orden.

La relación R' definida sobre T como

$$iR'j \text{ si y sólo si la tarea } i \text{ debe realizar antes que la tarea } j$$

ordena las tareas. Aunque R' es antisimétrica y transitiva, no es reflexiva, de modo que R' no es un orden parcial. Podemos obtener un orden parcial si añadimos todos los pares (i, i) para $i = 1, \ldots, 5$. Desde el punto de vista formal, la relación

$$R' \cup \{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5)\}$$

es un orden parcial sobre T. Una solución al problema de planeación de las tareas necesarias para tomar una fotografía es establecer un orden total de las tareas, consistente con el orden parcial. Más precisamente, necesitamos un orden total de las tareas

$$t_1, \quad t_2, \quad t_3, \quad t_4, \quad t_5$$

tal que si $t_i R' t_j$, entonces t_i precede a t_j en la lista. □

Dada una relación R de X a Y, podemos definir una relación de Y a X invirtiendo el orden de cada par ordenado en R. A continuación damos la definición formal.

DEFINICIÓN 2.4.22

Sea R una relación de X en Y. La *inversa* de R, que se denota R^{-1}, es la relación de Y a X definida como

$$R^{-1} = \{(y, x) \mid (x, y) \in R\}.$$

EJEMPLO 2.4.23

La inversa de la relación R del ejemplo 2.4.3 es

$$R^{-1} = \{(4, 2), (6, 2), (3, 3), (6, 3), (4, 4)\}.$$

Podemos expresar esta relación en palabras, como "es divisible entre". □

Si tenemos una relación R_1 de X a Y y una relación R_2 de Y a Z, podemos formar una relación de X a Z, aplicando primero la relación R_1 y luego la relación R_2. La relación resultante se denota $R_2 \circ R_1$. Observe el orden en que se escriben las relaciones. A continuación damos la definición formal.

DEFINICIÓN 2.4.24

Sea R_1 una relación de X a Y y R_2 una relación de Y a Z. La *composición* de R_1 y R_2, que se denota $R_2 \circ R_1$, es la relación de X a Z definida como

$$R_2 \circ R_1 = \{(x, z) \mid (x, y) \in R_1 \text{ y } (y, z) \in R_2 \text{ para alguna } y \in Y\}.$$

EJEMPLO 2.4.25

La composición de las relaciones

$$R_1 = \{(1, 2), (1, 6), (2, 4), (3, 4), (3, 6), (3, 8)\}$$

y

$$R_2 = \{(2, u), (4, s), (4, t), (6, t), (8, u)\}$$

es

$$R_2 \circ R_1 = \{(1, u), (1, t), (2, s), (2, t), (3, s), (3, t), (3, u)\}.$$ □

Ejercicios

En los ejercicios 1-4, escriba la relación como un conjunto de pares ordenados.

1.

8840	Martillo
9921	Pinzas
452	Pintura
2207	Tapiz

2.

a	3
b	1
b	4
c	1

3.

Sally	Matemáticas
Ruth	Física
Sam	3Economía

4.

a	a
b	b

En los ejercicios 5-8, escriba la relación como una tabla.

5. $R = \{(a, 6), (b, 2), (a, 1), (c, 1)\}$
6. $R = \{(\text{Roger, Música}), (\text{Pat, Historia}), (\text{Ben, Matemáticas}), (\text{Pat, Ciencias políticas})\}$
7. La relación R sobre $\{1, 2, 3, 4\}$ definida como $(x, y) \in R$ si $x^2 \geq y$.
8. La relación R del conjunto X de estados de la Unión Americana cuyos nombres comienzan con la letra "M" al conjunto Y de ciudades, definida como $(S, C) \in X \times Y$ si C es la capital de S.

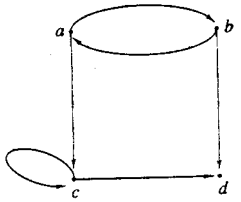
En los ejercicios 9-12, trace la digráfica de la relación.

9. La relación del ejercicio 4 sobre $\{a, b, c\}$.
10. La relación $R = \{(1, 2), (2, 1), (3, 3), (1, 1), (2, 2)\}$ sobre $X = \{1, 2, 3\}$.

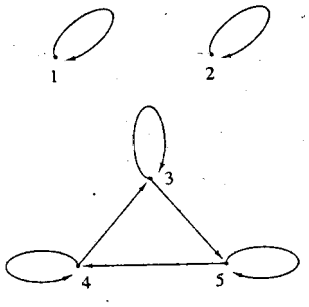
11. La relación $R = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$ sobre $\{1, 2, 3, 4\}$.
12. La relación del ejercicio 7.

En los ejercicios 13-16, escriba la relación como un conjunto de pares ordenados.

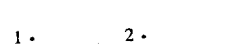
13.



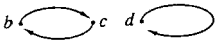
14.



15.



16.



17. Determine el dominio y rango de cada una de las relaciones en los ejercicios 1-16.
18. Determine la inversa (como conjunto de pares ordenados) de cada relación en los ejercicios 1-16.

Los ejercicios 19-24 se refieren a la relación R sobre el conjunto $\{1, 2, 3, 4, 5\}$ definida mediante la regla $(x, y) \in R$ si 3 divide a $x - y$.

19. Enumere los elementos de R.
20. Enumere los elementos de R^{-1}.
21. Determine el dominio de R.
22. Concluya el rango de R.
23. Delimite el dominio de R^{-1}.
24. Determine el rango de R^{-1}.
25. Repita los ejercicios 19-24 para la relación R sobre el conjunto $\{1, 2, 3, 4, 5\}$ definida mediante la regla $(x, y) \in R$ si $x + y \leq 6$.
26. Repita los ejercicios 19-24 para la relación R sobre el conjunto $\{1, 2, 3, 4, 5\}$ definida mediante la regla $(x, y) \in R$ si $x = y - 1$.
27. ¿Es la relación del ejercicio 25 reflexiva, simétrica, antisimétrica, transitiva o un orden parcial?
28. ¿Es la relación del ejercicio 26 reflexiva, simétrica, antisimétrica, transitiva o un orden parcial?

En los ejercicios 29-33, determine si cada relación definida sobre el conjunto de enteros positivos es reflexiva, simétrica, antisimétrica, transitiva o un orden parcial.

29. $(x, y) \in R$ si $x = y^2$.
30. $(x, y) \in R$ si $x > y$.
31. $(x, y) \in R$ si $x \geq y$.
32. $(x, y) \in R$ si $x = y$.
33. $(x, y) \in R$ si 3 divide a $x - y$.
34. Sea X un conjunto no vacío. Defina una relación sobre $P(X)$, el conjunto potencia de X, como $(A, B) \in R$ si $A \subseteq B$. ¿Es esta relación reflexiva, simétrica, antisimétrica, transitiva o un orden parcial?

35. Sea X el conjunto de todas las cadenas de cuatro bits (por ejemplo, 0011, 0101, 1000). Defina una relación R sobre X como $s_1 R s_2$ si alguna subcadena de s_1 de longitud 2 es igual a alguna subcadena de s_2 de longitud 2. Por ejemplo: $0111 R 1010$ (pues 0111 y 1010 contienen a 01). $1110 \not R 0001$ (pues 1110 y 0001 no comparten una subcadena común de longitud 2). ¿Es esta relación reflexiva, simétrica, antisimétrica, transitiva o un orden parcial?
36. Suponga que R_i es un orden parcial sobre X_i, $i = 1, 2$. Muestre que R es un orden parcial sobre $X_1 \times X_2$ si definimos

$$(x_1, x_2)\, R\, (x_1', x_2') \quad \text{si } x_1 R_1 x_1' \text{ y } x_2 R_2 x_2'.$$

37. Sean R_1 y R_2 las relaciones sobre $\{1, 2, 3, 4\}$ dadas por

$$R_1 = \{(1, 1), (1, 2), (3, 4), (4, 2)\}$$
$$R_2 = \{(1, 1), (2, 1), (3, 1), (4, 4), (2, 2)\}.$$

Enumere los elementos de $R_1 \circ R_2$ y $R_2 \circ R_1$.

Proporcione ejemplos de relaciones sobre $\{1, 2, 3, 4\}$ que tengan las propiedades especificadas en los ejercicios 38-42.

38. Reflexiva, simétrica y no transitiva
39. Reflexiva, no simétrica y no transitiva
40. Reflexiva, antisimétrica y no transitiva
41. No reflexiva, simétrica, no antisimétrica y transitiva
42. No reflexiva, no simétrica y transitiva

Sean R y S relaciones sobre X. Determine si cada afirmación en los ejercicios 43-58 es verdadera o falsa. Si la afirmación es falsa, proporcione un contraejemplo.

43. Si R y S son transitivas, entonces $R \cup S$ es transitiva.
44. Si R y S son transitivas, entonces $R \cap S$ es transitiva.
45. Si R y S son transitivas, entonces $R \circ S$ es transitiva.
46. Si R es transitiva, entonces R^{-1} es transitiva.
47. Si R y S son reflexivas, entonces $R \cup S$ es reflexiva.
48. Si R y S son reflexivas, entonces $R \cap S$ es reflexiva.
49. Si R y S son reflexivas, entonces $R \circ S$ es reflexiva.
50. Si R es reflexiva, entonces R^{-1} es reflexiva.
51. Si R y S son simétricas, entonces $R \cup S$ es simétrica.
52. Si R y S son simétricas, entonces $R \cap S$ es simétrica.
F 53. Si R y S son simétricas, entonces $R \circ S$ es simétrica. $R = \{(1,2), (2,1)\} \wedge S = \{(2,3), (1,3)\}$
V 54. Si R es simétrica, entonces R^{-1} es simétrica.
55. Si R y S son antisimétricas, entonces R ∪ S es antisimétrica.
56. Si R y S son antisimétricas, entonces R ∩ S es antisimétrica.
57. Si R y S son antisimétricas, entonces $R \circ S$ es antisimétrica.
58. Si R es antisimétrica, entonces R^{-1} es antisimétrica.
F 59. ¿Qué es incorrecto en el siguiente argumento, el cual supuestamente muestra que para cualquier relación R sobre X que sea simétrica y transitiva es reflexiva?

Sea $x \in X$. Utilizamos la simetría para tener (x, y) y (y, x) ambos en R. Como (x, y), $(y, x) \in R$, por transitividad tenemos $(x, x) \in R$. Por lo tanto, R es reflexiva.

$(x, y) \in R \rightarrow$ no es reflexiva.

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

RELACIONES

Problema

Defina la relación R sobre el conjunto de enteros positivos como

$(x, y) \in R$ si el máximo común divisor de x y y es 1.

Determine si R es reflexiva, simétrica, antisimétrica, transitiva o un orden parcial.

Para enfrentar el problema

La relación R consta de pares ordenados de enteros positivos. Clasificaremos algunos pares ordenados como estando o no en R. Enumeraremos de manera sistemática todos los pares en orden creciente, en el sentido de que primero enumeramos todos los pares (x, y) donde $x + y = 2$, luego todos los pares (x, y) donde $x + y = 3$, y así sucesivamente:

Par ordenado (x, y)	$x + y$	*Máximo común divisor*	*¿Está en R?*
(1, 1)	2	1	Sí
(1, 2)	3	1	Sí
(2, 1)	3	1	Sí
(1, 3)	4	1	Sí
(2, 2)	4	2	No
(3, 1)	4	1	Sí
(4, 1)	5	1	Sí
(3, 2)	5	1	Sí
(2, 3)	5	1	Sí
(1, 4)	5	1	Sí

El lector debe agregar las entradas para $x + y = 6$ antes de continuar.

Al construir la tabla, surgen varios patrones. En primer lugar, vemos que el máximo común divisor de (x, y) y (y, x) son iguales. Así, si (x, y) está en R (es decir, si el máximo común divisor de x y y es 1), entonces (y, x) también está en R (pues el máximo común divisor de y y x también es 1). Además, (x, x) está en R sólo si $x = 1$.

Determinación de una solución

La primera pregunta es si R es reflexiva. Recordemos (véase la definición 2.4.6) que R es reflexiva sobre el conjunto de enteros positivos si (x, x) está en R para toda x en el conjunto de enteros positivos. Nuestra tabla muestra que $(2, 2)$ no está en R. Así, $(2, 2)$ es un contraejemplo que muestra que R no es reflexiva.

La siguiente pregunta es si R es simétrica. Recordemos (véase la definición 2.4.9) que R es simétrica si para toda x y y, si (x, y) está en R, entonces (y, x) también está en R. En la subsección anterior observamos que esta condición es válida para R. Así, R es simétrica.

La siguiente pregunta es si R es antisimétrica. Recordemos (véase la definición 2.4.12) que R es antisimétrica si para toda x y y, si (x, y) está en R y $x \neq y$, entonces (y, x) no está en R. Nuestra tabla muestra que $(2, 1)$ está en R, y es claro que $2 \neq 1$, pero $(1, 2)$ está en R. Así, $(2, 1)$ es un contraejemplo que muestra que R no es antisimétrica.

La siguiente pregunta es si R es transitiva. Recordemos (véase la definición 2.4.16) que R es transitiva si para toda x, y y z, si (x, y) y (y, z) están en R, entonces (x, z) está en R. Nuestra tabla muestra que $(2, 1)$ y $(1, 2)$ están en R, pero $(2, 2)$ no está en R. Así, si consideramos $x = 2$, $y = 1$ y $z = 2$, tenemos un contraejemplo que muestra que R no es transitiva.

La última pregunta es si R es un orden parcial. Recordemos (véase la definición 2.4.19) que R es un orden parcial si R es reflexiva, antisimétrica y transitiva. Ya hemos mostrado que R no es reflexiva, R no es antisimétrica, y R no es transitiva. Por lo tanto, R no es un orden parcial. Si R no hubiese cumplido con sólo uno de los tres criterios (ser reflexiva, antisimétrica o transitiva), R no sería un orden parcial. En este caso, R no cumple con los tres criterios.

Solución formal

Hemos determinado que R no es reflexiva, es simétrica, no es antisimétrica, no es transitiva, ni es un orden parcial.

Resumen de técnicas para la solución de un problema

- En un problema que se refiere a una relación concreta, comience enumerando varios pares ordenados y clasifíquelos como pertenecientes o no a la relación.
- Al enumerar los pares ordenados, hágalo de una manera sistemática. Su método de enumeración de pares ordenados debe generar todos los pares como si éstos continuasen de manera indefinida.
- Al enumerar los pares ordenados, busque patrones. Por ejemplo, en este problema ya habíamos observado que (x, y) y (y, x) estaban ambas en R o no estaban ambas en R.
- No debe olvidar que un contraejemplo demuestra que una propiedad no se cumple para una relación.
- Para demostrar que una propiedad es válida para una relación, es necesario considerar un elemento arbitrario de la relación y demostrar la propiedad para éste. Por ejemplo, para demostrar que una relación es simétrica, se hace que (x, y) represente un par ordenado *arbitrario* en R y se proporciona un argumento para mostrar que (x, y) está en R.

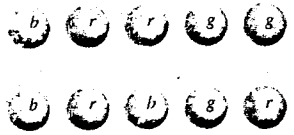


FIGURA 2.5.1
Un conjunto de bolas con color.

2.5 *RELACIONES DE EQUIVALENCIA*

Suponga que tenemos un conjunto X de 10 bolas rojas (R), azules (B) y verdes (G) (véase la figura 2.5.1). Si dividimos las bolas en los conjuntos R, B y G de acuerdo con su color, la familia $\{R, B, G\}$ es una partición de X. (Recuerde que en la sección 2.1 definimos una partición de un conjunto X como una colección $\mathcal{S}$ de subconjuntos no vacíos de X tal que todo elemento en X pertenece exactamente a un miembro de $\mathcal{S}$.)

Una partición puede utilizarse para definir una relación. Si $\mathcal{S}$ es una partición de X, podemos decir que $x\,R\,y$ si para algún conjunto $S \in \mathcal{S}$, x y y pertenecen a S. Para el ejemplo de la figura 2.5.1, la relación obtenida podría describirse como "tiene el mismo color que". El siguiente teorema muestra que esta relación siempre es reflexiva, simétrica y transitiva.

TEOREMA 2.5.1

Sea $\mathcal{S}$ una partición de un conjunto X. Decimos que $x\,R\,y$ si para algún conjunto S en $\mathcal{S}$, x y y pertenecen a S. Entonces R es reflexiva, simétrica y transitiva.

Demostración. Sea $x \in X$. Por la definición de partición, x pertenece a algún miembro S de $\mathcal{S}$. Así, $x\,R\,x$ y R es reflexiva.

Supongamos que $x\,R\,y$. Entonces x y y pertenecen a algún conjunto $S \in \mathcal{S}$. Como y y x pertenecen a S, $y\,R\,x$ y R es simétrica.

Por último, supongamos que $x\,R\,y$ y $y\,R\,z$. Entonces x y y pertenecen a algún conjunto $S \in \mathcal{S}$ y y y z pertenecen a algún conjunto $T \in \mathcal{S}$. Como y pertenece exactamente a un miembro de $\mathcal{S}$, se debe cumplir que $S = T$. Por lo tanto, x y z pertenecen a S y $x\,R\,z$. Hemos mostrado que R es transitiva. ■

EJEMPLO 2.5.2

Consideremos la partición

$$\mathcal{S} = \{\{1, 3, 5,\}, \{2, 6\}, \{4\}\}$$

de $X = \{1, 2, 3, 4, 5, 6\}$. La relación R sobre X dada por el teorema 2.5.1 contiene los pares ordenados $(1, 1)$, $(1, 3)$ y $(1, 5)$, pues $\{1, 3, 5\}$ está en $\mathcal{S}$. La relación completa es

$$R = \{(1, 1), (1, 3), (1, 5), (3, 1), (3, 3), (3, 5), (5, 1), (5, 3), (5, 5), (2, 2), (2, 6), (6, 2), (6, 6), (4, 4)\}.$$ □

Sean $\mathcal{S}$ y R como en el teorema 2.5.1. Si $S \in \mathcal{S}$, podemos considerar a los miembros de S como equivalentes, en el sentido de la relación R. Por esta razón, las relaciones que son reflexivas, simétricas y transitivas se llaman **relaciones de equivalencia**. En el ejemplo de la figura 2.5.1, la relación es "tiene el mismo color que"; por lo tanto, *equivalente* quiere decir "tiene el mismo color que". Cada conjunto en la partición consta de todas las bolas de un color particular.

DEFINICIÓN 2.5.3

Una relación que sea reflexiva, simétrica y transitiva sobre un conjunto X es una *relación de equivalencia sobre X*.

EJEMPLO 2.5.4

La relación R del ejemplo 2.5.2 es una relación de equivalencia sobre $\{1, 2, 3, 4, 5, 6\}$ debido al teorema 2.5.1. También podemos verificar directamente que R es reflexiva, simétrica y transitiva.

La digráfica de la relación R del ejemplo 2.5.2 aparece en la figura 2.5.2. De nuevo, vemos que R es reflexiva (existe un lazo en cada vértice), simétrica (para cada arista dirigida de v a w, también existe una arista dirigida de w a v) y transitiva (si existe una arista dirigida de x a y y una arista dirigida de y a z, entonces existe una arista dirigida de x a z). □

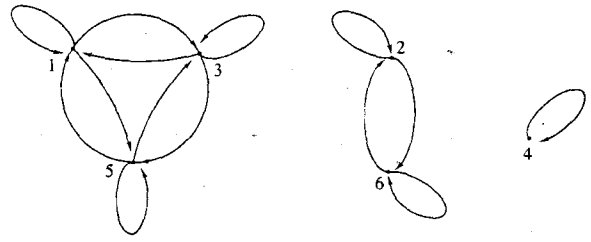


FIGURA 2.5.2 La digráfica de la relación del ejemplo 2.5.2.

EJEMPLO 2.5.5

Consideremos la relación

$$R = \{(1, 1), (1, 3), (1, 5), (2, 2), (2, 4), (3, 1), (3, 3), (3, 5), (4, 2), (4, 4), (5, 1), (5, 3), (5, 5)\}$$

sobre $\{1, 2, 3, 4, 5\}$. R es reflexiva pues $(1, 1), (2, 2), (3, 3), (4, 4), (5, 5) \in R$. R es simétrica, pues siempre que (x, y) esté en R, (y, x) también está en R. Por último, R es transitiva, pues siempre que (x, y) y (y, z) estén en R, (x, z) también está en R. Como R es reflexiva, simétrica y transitiva, R es una relación de equivalencia sobre $\{1, 2, 3, 4, 5\}$. □

EJEMPLO 2.5.6

La relación R del ejemplo 2.4.4 no es una relación de equivalencia, pues R no es simétrica. □

EJEMPLO 2.5.7

La relación R del ejemplo 2.4.5 no es una relación de equivalencia, pues R no es reflexiva ni transitiva. □

EJEMPLO 2.5.8

La relación R del ejemplo 2.4.15 es una relación de equivalencia, pues R es reflexiva, simétrica y transitiva. □

Dada una relación de equivalencia sobre un conjunto X, podemos separar a X agrupando los miembros de X relacionados entre ellos. Los elementos relacionados pueden pensarse como equivalentes unos con otros. El siguiente teorema proporciona los detalles.

TEOREMA 2.5.9

Sea R una relación de equivalencia sobre un conjunto X. Para cada $a \in X$, sea

$$[a] = \{x \in X \mid xRa\}.$$

Entonces

$$\mathcal{S} = \{[a] \mid a \in X\}$$

es una partición de X.

Demostración. Debemos mostrar que cada elemento de X pertenece a exactamente un miembro de $\mathcal{S}$.

Sea $a \in X$. Como $a\,R\,a$, $a \in [a]$. Así, cada elemento en X pertenece *al menos a un* elemento de $\mathcal{S}$. Ahora debemos mostrar que cada elemento de X pertenece a *exactamente* un miembro de $\mathcal{S}$; es decir,

$$\text{si } x \in X \text{ y } x \in [a] \cap [b], \quad \text{entonces } [a] = [b]. \tag{2.5.1}$$

Primero mostramos que si $a\,R\,b$, entonces $[a] = [b]$. Supongamos que $a\,R\,b$. Sea $x \in [a]$. Entonces $x\,R\,a$. Como $a\,R\,b$ y R es transitiva, $x\,R\,b$. Por lo tanto, $x \in [b]$ y $[a] \subseteq [b]$. El argumento para mostrar que $[b] \subseteq [a]$ es igual al último argumento, pero cambiando los papeles de a y b. Así, $[a] = [b]$.

Ahora demostraremos (2.5.1). Supongamos que $x \in X$ y $x \in [a] \cap [b]$. Entonces $x\,R\,a$ y $x\,R\,b$. Nuestro resultado anterior muestra que $[x] = [a]$ y $[x] = [b]$. Así, $[a] = [b]$. ■

DEFINICIÓN 2.5.10

Sea R una relación de equivalencia sobre un conjunto X. Los conjuntos $[a]$ definidos en el teorema 2.5.9 son las *clases de equivalencia de X dadas por la relación R*.

EJEMPLO 2.5.11

Consideremos la relación de equivalencia R del ejemplo 2.5.2. La clase de equivalencia $[1]$ que contiene a 1 consta de todas las x tales que $(x, 1) \in R$. Por lo tanto,

$$[1] = \{1, 3, 5\}.$$

Las demás clases de equivalencia se determinan de manera similar:

$$[3] = [5] = \{1, 3, 5\}, \quad [2] = [6] = \{2, 6\}, \quad [4] = \{4\}.$$ □

EJEMPLO 2.5.12

Las clases de equivalencia aparecen con claridad en la digráfica de una relación de equivalencia. Las tres clases de equivalencia de la relación R del ejemplo 2.5.2 aparecen en la digráfica de R (que aparece en la figura 2.5.2) como las tres subgráficas cuyos vértices son $\{1, 3, 5\}$, $\{2, 6\}$ y $\{4\}$. Una subgráfica G que representa a una clase de equivalencia es una subgráfica más grande de la digráfica original, con la propiedad de que para cualesquiera vértices v y w en G, existe una arista dirigida de v a w. Por ejemplo, si $v, w \in \{1, 3, 5\}$, existe una arista dirigida de v a w. Además, no pueden agregarse vértices a 1, 3, 5 de modo que el conjunto de vértices resultante tenga una arista dirigida entre cada par de vértices. □

EJEMPLO 2.5.13

Existen dos clases de equivalencia para la relación de equivalencia del ejemplo 2.5.5, a saber,

$$[1] = [3] = [5] = \{1, 3, 5\}, \quad [2] = [4] = \{2, 4\}.$$ □

EJEMPLO 2.5.14

Las clases de equivalencia para la relación de equivalencia del ejemplo 2.4.15 son

$$[a] = \{a\}, \quad [b] = \{b\}, \quad [c] = \{c\}.$$ □

EJEMPLO 2.5.15

Sea $X = \{1, 2, \ldots, 10\}$. Decimos que $x\,R\,y$ si 3 divide a $x - y$. Podemos verificar rápidamente que la relación R es reflexiva, simétrica y transitiva. Así, R es una relación de equivalencia sobre X.

Ahora determinaremos los miembros de las clases de equivalencia. La clase de equivalencia $[1]$ consta de todas las x tales que $x\,R\,1$. Así,

$$[1] = \{x \in X \mid 3 \text{ divide a } x - 1\} = \{1, 4, 7, 10\}.$$

De manera análoga,

$$[2] = \{2, 5, 8\}$$
$$[3] = \{3, 6, 9\}.$$

Estos tres conjuntos separan a X. Observe que

$$[1] = [4] = [7] = [10], \quad [2] = [5] = [8], \quad [3] = [6] = [9].$$

Para esta relación, *equivalencia* es "tiene el mismo residuo al dividir entre 3". □

Concluimos esta sección demostrando un resultado especial que necesitaremos posteriormente (véanse las secciones 4.2 y 4.4). La demostración se ilustra en la figura 2.5.3.

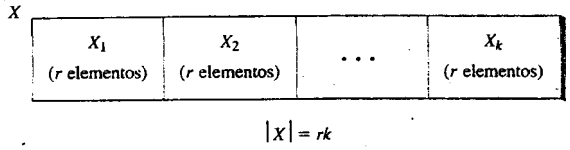


FIGURA 2.5.3 La demostración del teorema 2.5.16.

TEOREMA 2.5.16

Sea R una relación de equivalencia sobre un conjunto finito X. Si cada clase de equivalencia tiene r elementos, entonces existen $|X|/r$ clases de equivalencia.

Demostración. Sean $X_1, X_2, \ldots, X_k$ las distintas clases de equivalencia. Como éstos conjuntos separan a X,

$$|X| = |X_1| + |X_2| + \cdots + |X_k| = r + r + \cdots + r = kr$$

de donde se sigue la conclusión. ■

Ejercicios

En los ejercicios 1-8, determine si la relación dada es una relación de equivalencia sobre {1, 2, 3, 4, 5}. Si la relación es una relación de equivalencia, enumere las clases de equivalencia. (En los ejercicios 5-8, $x, y \in \{1, 2, 3, 4, 5\}$.)

1. $\{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (1, 3), (3, 1)\}$
2. $\{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (1, 3), (3, 1), (3, 4), (4, 3)\}$
3. $\{(1, 1), (2, 2), (3, 3), (4, 4)\}$
4. $\{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (1, 5), (5, 1), (3, 5), (5, 3), (1, 3), (3, 1)\}$
5. $\{(x, y) \mid 1 \leq x \leq 5, 1 \leq y \leq 5\}$
6. $\{(x, y) \mid 4 \text{ divide a } x - y\}$
7. $\{(x, y) \mid 3 \text{ divide a } x + y\}$
8. $\{(x, y) \mid x \text{ divide a } 2 - y\}$

En los ejercicios 9-14, enumere los miembros de la relación de equivalencia sobre {1, 2, 3, 4} definida mediante la partición dada (como en el teorema 2.5.1). Además, determine las clases de equivalencia [1], [2], [3] y [4].

9. $\{\{1, 2\}, \{3, 4\}\}$
10. $\{\{1\}, \{2\}, \{3, 4\}\}$
11. $\{\{1\}, \{2\}, \{3\}, \{4\}\}$
12. $\{\{1, 2, 3\}, \{4\}\}$
13. $\{\{1, 2, 3, 4\}\}$
14. $\{\{1\}, \{2, 4\}, \{3\}\}$

En los ejercicios 15-17, sean $X = \{1, 2, 3, 4, 5\}$, $Y = \{3, 4\}$ y $C = \{1, 3\}$. Defina la relación R sobre $P(X)$, el conjunto de todos los subconjuntos de X, como

$$A \, R \, B \text{ si y sólo si } A \cup Y = B \cup Y.$$

15. Muestre que R es una relación de equivalencia.
16. Enumere los elementos de $[C]$, la clase de equivalencia que contiene a C.
17. ¿Cuántas clases de equivalencia distintas existen?
18. Sea

$$X = \{\text{San Francisco, Pittsburgh, Chicago, San Diego, Filadelfia, Los Ángeles}\}.$$

Defina una relación R sobre X como $x \, R \, y$ si x y y están en el mismo estado.

(a) Muestre que R es una relación de equivalencia.

(b) Enumere las clases de equivalencia de X.

19. Muestre que si R es una relación de equivalencia sobre X, entonces

$$\text{dominio } R = \text{rango } R = X.$$

20. Si una relación de equivalencia tiene sólo una clase de equivalencia, ¿cómo debe verse dicha relación?
21. Si R es una relación de equivalencia sobre un conjunto X y $|X| = |R|$, ¿cómo debe verse dicha relación? la relacion debe ser solamente reflexiva.
22. Proporcione un ejemplo de una relación de equivalencia sobre $\{1, 2, 3, 4, 5, 6\}$ con exactamente cuatro clases de equivalencia, enumerando sus pares ordenados.
23. ¿Cuántas relaciones de equivalencia existen sobre el conjunto $\{1, 2, 3\}$? 7
24. Sea $X = \{1, 2, \ldots, 10\}$. Defina una relación R sobre $X \times X$ como $(a, b) \, R \, (c, d)$ si $a + d = b + c$.

(a) Muestre que R es una relación de equivalencia sobre $X \times X$.

(b) Enumere un miembro de cada clase de equivalencia de $X \times X$.

25. Sea $X = \{1, 2, \ldots, 10\}$. Defina una relación R sobre $X \times X$ como $(a, b) \, R \, (c, d)$ si $ad = bc$.

(a) Muestre que R es una relación de equivalencia sobre $X \times X$.

(b) Enumere un miembro de cada clase de equivalencia de $X \times X$.

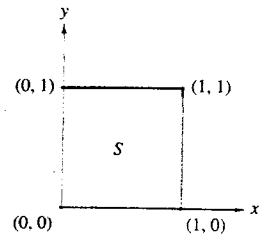
(c) Describa la relación R en términos familiares.

26. Sea R una relación reflexiva y transitiva sobre X. Muestre que $R \cap R^{-1}$ es una relación de equivalencia sobre X.
27. Sean R_1 y R_2 relaciones de equivalencia sobre X.

(a) Muestre que $R_1 \cap R_2$ es una relación de equivalencia sobre X.

(b) Describa las clases de equivalencia de $R_1 \cap R_2$ en términos de las clases de equivalencia de R_1 y las clases de equivalencia de R_2. $[x] = \{y \mid x R_1 y \wedge x R_2 y\}$

28. Suponga que $\mathcal{S}$ es una colección de subconjuntos de un conjunto X y $X = \cup \mathcal{S}$. (No suponemos que la familia $\mathcal{S}$ es ajena por pares.) Definimos: $x \, R \, y$ si para algún conjunto $S \in \mathcal{S}$, x y y están en S. ¿Es R necesariamente reflexiva, simétrica o transitiva?



29. Sea S un cuadrado unitario, incluyendo el interior, como muestra la figura anexa. Defina una relación R sobre S como $(x, y)\ R\ (x', y')$ si $(x = x' \text{ y } y = y')$ o $(y = y' \text{ y } x = 0 \text{ y } x' = 1)$ o $(y = y' \text{ y } x = 1 \text{ y } x' = 0)$.

(a) Muestre que R es una relación de equivalencia sobre S.

(b) Si los puntos en la misma clase de equivalencia se pegan, ¿cómo describiría la figura formada?

30. Sea S un cuadrado unitario, incluyendo el interior (como en el ejercicio 29). Defina una relación R' sobre S como $(x, y)\ R'\ (x', y')$ si $(x = x' \text{ y } y = y')$ o $(y = y' \text{ y } x = 0 \text{ y } x' = 1)$ o $(y = y' \text{ y } x = 1 \text{ y } x' = 0)$ o $(x = x' \text{ y } y = 0 \text{ y } y' = 1)$ o $(x = x' \text{ y } y = 1 \text{ y } y' = 0)$. Sea

$$R = R' \cup \{((0, 0), (1, 1)), ((0, 1), (1, 0)), ((1, 0), (0, 1)), ((1, 1), (0, 0))\}.$$

(a) Muestre que R es una relación de equivalencia sobre S.

(b) Si los puntos en la misma clase de equivalencia se pegan, ¿cómo describiría la figura formada?

Sea R una relación sobre un conjunto X. Defina

$$\rho(R) = R \cup \{(x, x) \mid x \in X\}$$
$$\sigma(R) = R \cup R^{-1}$$
$$R^n = R \circ R \circ R \circ R \circ \cdots \circ R \quad (n \text{ erres})$$
$$\tau(R) = \cup \{R^n \mid n = 1, 2, \ldots\}.$$

La relación $\tau(R)$ es la *cerradura transitiva* de R.

31. Para las relaciones R_1 y R_2 del ejercicio 37, sección 2.4, determine $\rho(R_i)$, $\sigma(R_i)$, $\tau(R_i)$, y $\tau(\sigma(\rho(R_i)))$, para $i = 1, 2$.

32. Muestre que $\rho(R)$ es reflexiva.

33. Muestre que $\sigma(R)$ es simétrica.

34. Muestre que $\tau(R)$ es transitiva.

☆ 35. Muestre que $\tau(\sigma(\rho(R)))$ es una relación de equivalencia que contiene a R.

☆ 36. Muestre que $\tau(\sigma(\rho(R)))$ es la menor relación de equivalencia sobre X que contiene a R; es decir, muestre que si R' es una relación de equivalencia sobre X y $R' \supseteq R$, entonces $R' \supseteq \tau(\sigma(\rho(R)))$.

★ 37. Muestre que R es transitiva si y sólo si $\tau(R) = R$.

En los ejercicios 38-44, escriba "verdadero" si la afirmación es verdadera para todas las relaciones R_1 y R_2 sobre un conjunto arbitrario X; en caso contrario, proporcione un contraejemplo.

38. $\rho(R_1 \cup R_2) = \rho(R_1) \cup \rho(R_2)$

39. $\sigma(R_1 \cap R_2) = \sigma(R_1) \cap \sigma(R_2)$

40. $\tau(R_1 \cup R_2) = \tau(R_1) \cup \tau(R_2)$

41. $\tau(R_1 \cap R_2) = \tau(R_1) \cap \tau(R_2)$

42. $\sigma(\tau(R_1)) = \tau(\sigma(R_1))$

43. $\sigma(\rho(R_1)) = \rho(\sigma(R_1))$

44. $\rho(\tau(R_1)) = \tau(\rho(R_1))$

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

RELACIONES DE EQUIVALENCIA

Problema

Responda las siguientes preguntas para la relación R definida sobre el conjunto de cadenas de ocho bits como $s\ R\ s'$ si los primeros cuatro bits de s y s' coinciden.

(a) Muestre que R es una relación de equivalencia.

(b) Enumere un miembro de cada clase de equivalencia.

(c) ¿Cuántas clases de equivalencia existen?

Para enfrentar el problema

Primero analizaremos algunas cadenas específicas de ocho bits relacionadas de acuerdo con la relación R. Consideremos una cadena arbitraria 01111010 y determinaremos las cadenas relacionadas con ella. Una cadena s está relacionada con 01111010 si los primeros cuatro bits de 01111010 y s coinciden. Esto significa que s debe comenzar con 0111 y los cuatro últimos dígitos pueden ser arbitrarios. Un ejemplo es s = 01111000.

Enumeraremos todas las cadenas relacionadas con 01111010. Al hacerlo, debemos tener cuidado en agregar a 0111 todas las cadenas posibles de cuatro bits:

01110000, 01110001, 01110010, 01110011,
01110100, 01110101, 01110110, 01110111,
01111000, 01111001, 01111010, 01111011,
01111100, 01111101, 01111110, 01111111.

Supondremos por el momento que R es una relación de equivalencia; en este caso, la clase de equivalencia que contiene a 01111010, denotada [01111010], consta de todas las cadenas relacionadas con 01111010. Por lo tanto, lo que acabamos de calcular son los miembros de [01111010].

Observe que si consideramos cualquier cadena en [01111010], digamos 01111100, y calculamos su clase de equivalencia [01111100], obtendremos exactamente el mismo conjunto de cadenas, a saber, el conjunto de cadenas de ocho bits que comienzan con 0111.

Para obtener un ejemplo diferente, tendríamos que comenzar con una cadena cuyos primeros cuatro bits son distintos de 0111, como 1011. Por ejemplo, las cadenas relacionadas con 10110100 son

10110000, 10110001, 10110010, 10110011,
10110100, 10110101, 10110110, 10110111,
10111000, 10111001, 10111010, 10111011,
10111100, 10111101, 10111110, 10111111.

Lo que acabamos de calcular son los miembros de [10110100].

Antes de continuar, calcule los miembros de alguna otra clase de equivalencia.

Vemos que [01111010] y [10110100] no tienen miembros en común. Siempre ocurre que dos clases de equivalencia son idénticas o que no tienen miembros en común (véase el teorema 2.5.9).

Determinación de una solución

Para mostrar que R es una relación de equivalencia, debemos mostrar que R es reflexiva, simétrica y transitiva (véase la definición 2.5.3). Para cada propiedad, iremos directamente a la definición y verificaremos que se cumplen las condiciones dadas en ella.

Para que R sea reflexiva, debemos tener que $s\,R\,s$ para cada cadena de ocho bits s. Para que $s\,R\,s$ sea cierto, los primeros cuatro bits de s y s deben coincidir. ¡Esto claramente es cierto!

Para que R sea simétrica, para todas las cadenas de ocho bits s_1 y s_2, si $s_1\,R\,s_2$, entonces $s_2\,R\,s_1$. Utilizamos la definición de R para traducir esta condición como: Si los primeros cuatro bits de s_1 y de s_2 coinciden, entonces los primeros cuatro bits de s_2 y de s_1 coinciden, lo cual ¡nuevamente es cierto!

Para que R sea transitiva, para todas las cadenas de ocho bits s_1, s_2 y s_3, si $s_1\,R\,s_2$ y $s_2\,R\,s_3$, entonces $s_1\,R\,s_3$. De nuevo utilizamos la definición de R para traducir esta condición como sigue: Si los primeros cuatro bits de s_1 y de s_2 coinciden y los primeros cuatro bits de s_2 y de s_3 coinciden, entonces los primeros cuatro bits de s_1 y de s_3 coinciden. ¡Esto también es cierto! Hemos demostrado que R es una relación de equivalencia.

En nuestro análisis anterior, vimos que cada cadena de cuatro bits determina una clase de equivalencia. Por ejemplo, la cadena 0111 determina la clase de equivalencia que consta de todas las cadenas de ocho bits que comienzan con 0111. Por lo tanto, el número de clases de equivalencia es igual al número de cadenas de cuatro bits. Podemos enumerarlas de la manera siguiente:

0000, 0001, 0010, 0011,
0100, 0101, 0110, 0111,
1000, 1001, 1010, 1011,
1100, 1101, 1110, 1111

y luego contarlas. Existen 16 clases de equivalencia.

Consideremos el problema de enumerar un miembro de cada clase de equivalencia. Las 16 cadenas de cuatro bits ya enumeradas determinan las 16 clases de equivalencia. La primera cadena 0000 determina la clase de equivalencia que consta de todas las cadenas de ocho bits que comienzan con 0000; la segunda cadena 0001 determina la clase de equivalencia que consta de todas las cadenas de ocho bits que comienzan con 0001; y así sucesivamente. Así, para enumerar un miembro de cada clase de equivalencia, sólo necesitamos agregar alguna cadena de cuatro bits a cada una de las cadenas en la lista anterior:

00000000, 00010000, 00100000, 00110000,
01000000, 01010000, 01100000, 01110000,
10000000, 10010000, 10100000, 10110000,
11000000, 11010000, 11100000, 11110000.

Solución formal

(a) Ya hemos presentado una demostración formal de que R es una relación de equivalencia.

(b)
00000000, 00010000, 00100000, 00110000,
01000000, 01010000, 01100000, 01110000,
10000000, 10010000, 10100000, 10110000,
11000000, 11010000, 11100000, 11110000.

enumera un miembro de cada clase de equivalencia.

(c) Existen 16 clases de equivalencia.

Resumen de técnicas para resolver problemas

- Enumerar los elementos relacionados entre ellos.
- Calcular algunas clases de equivalencia; es decir, enumerar *todos* los elementos relacionados con un elemento particular.
- Puede ser de utilidad resolver las partes de un problema en un orden distinto al dado en el enunciado del problema. En nuestro ejemplo, al analizar algunos casos concretos fue útil *suponer* que la relación era una relación de equivalencia antes de demostrar que realmente lo era.
- Para mostrar que una relación particular R es una relación de equivalencia, vaya directamente a las definiciones. Muestre que R es reflexiva, simétrica y transitiva, verificando directamente que R satisface las definiciones correspondientes.
- Si el problema consiste en contar el número de elementos que satisfacen cierta propiedad (por ejemplo, en nuestro problema se pedía contar el número de clases de equivalencia) y el número es pequeño, sólo enumere todos los elementos y cuéntelos en forma directa.

Comentarios

En los lenguajes de programación, lo usual es que sólo cierto número de caracteres de los nombres de las variables y los términos especiales (desde el punto de vista técnico se llaman *identificadores*) son significativos. Por ejemplo, en el lenguaje de programación C, sólo los primeros 31 caracteres de los identificadores son significativos. Esto quiere decir que si dos identificadores comienzan con los mismos 31 caracteres, el sistema puede considerarlos idénticos.

Si definimos una relación R sobre el conjunto de identificadores en C como $s_1\,R\,s_2$ siempre que los primeros 31 caracteres de s_1 y s_2 coincidan, entonces R es una relación de equivalencia. Una clase de equivalencia consta de los identificadores que el sistema puede considerar idénticos.

2.6 MATRICES DE RELACIONES

Una matriz es una forma conveniente de representar una relación R de X a Y. Una computadora puede utilizar esta representación para analizar una relación. Etiquetamos los renglones con los elementos de X (con cierto orden arbitrario) y las columnas con los elementos de Y (de nuevo, con cierto orden arbitrario). Entonces colocamos un 1 en el renglón x y la columna y si $x\,R\,y$ o un 0 en caso contrario. Esta matriz es la **matriz de la relación R** (con respecto del orden dado para X y Y.)

EJEMPLO 2.6.1

La matriz de la relación

$$R = \{(1, b), (1, d), (2, c), (3, c), (3, b), (4, a)\}$$

de $X = \{1, 2, 3, 4\}$ a $Y = \{a, b, c, d\}$ con respecto de los órdenes 1, 2, 3, 4 y a, b, c, d es

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \end{array}.$$

□

EJEMPLO 2.6.2

La matriz de la relación R del ejemplo 2.6.1 con respecto de los órdenes 2, 3, 4, 1 y d, b, a, c es

$$\begin{array}{c} \\ 2 \\ 3 \\ 4 \\ 1 \end{array} \begin{array}{c} \begin{array}{cccc} d & b & a & c \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} \end{array}.$$

Es claro que la matriz de la relación de X a Y depende de los órdenes de X y Y. □

EJEMPLO 2.6.3

La matriz de la relación R de $\{2, 3, 4\}$ a $\{5, 6, 7, 8\}$, con respecto de los órdenes 2, 3, 4 y 5, 6, 7, 8 definida como

$$x\,R\,y \quad \text{si } x \text{ divide a } y$$

es

$$\begin{array}{c} \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 5 & 6 & 7 & 8 \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}.$$

□

Al escribir la matriz de una relación R sobre un conjunto X (es decir, de X en X), utilizamos el mismo orden para los renglones y para las columnas.

EJEMPLO 2.6.4

La matriz de la relación

$$R = \{(a, a), (b, b), (c, c), (d, d), (b, c), (c, b)\}$$

sobre $\{a, b, c, d\}$ con respecto del orden a, b, c, d, es

$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}.$$

□

Observe que la matriz de una relación sobre un conjunto X siempre es una matriz cuadrada.

Podemos determinar con rapidez si una relación R sobre un conjunto X es reflexiva analizando la matriz A de R (con respecto de cierto orden). La relación R es reflexiva si y sólo si A tiene unos en la diagonal principal. (La diagonal principal de una matriz cuadrada consta de las entradas sobre una línea que va de la esquina superior izquierda a la esquina inferior derecha.) La relación R es reflexiva si y sólo si $(x, x) \in R$ para toda $x \in X$. Pero esta última condición se cumple precisamente cuando la diagonal principal consta de unos. Observe que la relación R del ejemplo 2.6.4 es reflexiva y que la diagonal principal de la matriz de R consta de unos.

También podemos determinar con rapidez si una relación R sobre un conjunto X es simétrica, analizando la matriz A de R (con respecto de cierto orden). La relación R es simétrica si y sólo si para toda i y j, la ij-ésima entrada de A es igual a la j i-ésima entrada de A. (De manera menos formal, R es simétrica si y sólo si A es simétrica con respecto de la diagonal principal.) La razón es que R es simétrica si y sólo si siempre que (x, y) esté en R, entonces (y, x) también está en R. Pero esta última condición se cumple precisamente cuando A es simétrica con respecto de la diagonal principal. Observe que la relación R del ejemplo 2.6.4 es simétrica y que la matriz de R es simétrica con respecto de la diagonal principal.

También podemos determinar con rapidez si una relación R es antisimétrica analizando la matriz de R (con respecto de cierto orden; véase el ejercicio 10). Por desgracia, no existe una forma sencilla de verificar si una relación R sobre X es transitiva analizando la matriz de R.

Concluimos esta sección mostrando la forma en que la multiplicación matricial se relaciona con la composición de relaciones.

EJEMPLO 2.6.5

Sea R_1 la relación de $X = \{1, 2, 3\}$ en $Y = \{a, b\}$ definida por

$$R_1 = \{(1, a), (2, b), (3, a), (3, b)\}$$

y sea R_2 la relación de Y en $Z = \{x, y, z\}$ definida por

$$R_2 = \{(a, x), (a, y), (b, y), (b, z)\}.$$

La matriz de R_1 con respecto de los órdenes 1, 2, 3 y a, b es

$$A_1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\!\!\begin{array}{c} \begin{array}{cc} a & b \end{array} \\ \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \end{array}$$

y la matriz de R_2 con respecto de los órdenes a, b y x, y, z es

$$A_2 = \begin{array}{c} \\ a \\ b \end{array}\!\!\begin{array}{c} \begin{array}{ccc} x & y & z \end{array} \\ \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \end{array}.$$

El producto de estas matrices es

$$A_1A_2 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}.$$

Ahora interpretaremos este producto.

La ik-ésima entrada en A_1A_2 se calcula así:

$$\begin{array}{c} \\ i \end{array}\!\!\begin{array}{c} \begin{array}{cc} a & b \end{array} \\ \begin{pmatrix} s & t \end{pmatrix} \end{array}\!\!\begin{array}{c} \begin{array}{c} k \end{array} \\ \begin{pmatrix} u \\ v \end{pmatrix} \end{array} = su + tv.$$

Si este valor es distinto de cero, entonces su o tv es distinto de cero. Supongamos que $su \neq 0$. (El argumento es similar si $tv \neq 0$.) Entonces $s \neq 0$ y $u \neq 0$. Esto significa que $(i, a) \in R_1$ y $(a, k) \in R_2$. Esto implica que $(i, k) \in R_2 \circ R_1$. Hemos mostrado que si la ik-ésima entrada en A_1A_2 es distinta de cero, entonces $(i, k) \in R_2 \circ R_1$. El recíproco también es verdadero, como mostramos a continuación.

Supongamos que $(i, k) \in R_2 \circ R_1$. Entonces

1. $(i, a) \in R_1$ y $(a, k) \in R_2$

o

2. $(i, b) \in R_1$ y $(b, k) \in R_2$.

Si 1 se cumple, entonces $s = 1$ y $u = 1$, de modo que $su = 1$ y $su + tv$ es distinto de cero. De manera análoga, si se cumple 2, $tv = 1$ y de nuevo tenemos que $su + tv$ es distinto de cero. Hemos mostrado que si $(i, k) \in R_2 \circ R_1$, entonces la ik-ésima entrada en A_1A_2 es distinta de cero.

Hemos mostrado que $(i, k) \in R_2 \circ R_1$ si y sólo si la ik-ésima entrada en A_1A_2 es distinta de cero; así, A_1A_2 "casi" es la matriz de la relación $R_2 \circ R_1$. Para obtener la matriz de la relación $R_2 \circ R_1$, sólo hay que cambiar todas las entradas distintas de cero en A_1A_2 por unos. Así, la matriz de la relación $R_2 \circ R_1$, con respecto de los órdenes previamente elegidos 1, 2, 3 y x, y, z es

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\!\!\begin{array}{c} \begin{array}{ccc} x & y & z \end{array} \\ \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \end{array}.$$

□

El argumento dado en el ejemplo 2.6.5 es válido para cualesquiera relaciones. Resumimos este resultado como el teorema 2.6.6.

TEOREMA 2.6.6

Sea R_1 una relación de X en Y y sea R_2 una relación de Y en Z. Elíjanse órdenes para X, Y y Z. Todas las matrices de las relaciones están dadas respecto de estos órdenes. Sea A_1 la matriz de R_1 y A_2 la matriz de R_2. La matriz de la relación $R_2 \circ R_1$ se obtiene reemplazando cada término distinto de cero en el producto de matrices A_1A_2 por 1.

Demostración. La demostración aparece antes del enunciado del teorema. ■

Ejercicios

En los ejercicios 1-3, determine la matriz de la relación R de X en Y con respecto de los órdenes dados.

1. $R = \{(1, \delta), (2, \alpha), (2, \Sigma), (3, \beta), (3, \Sigma)\}$
   Orden de X: 1, 2, 3
   Orden de Y: $\alpha, \beta, \Sigma, \delta$
2. R como en el ejercicio 1
   Orden de X: 3, 2, 1
   Orden de Y: $\Sigma, \beta, \alpha, \delta$
3. $R = \{(x, a), (x, c), (y, a), (y, b), (z, d)\}$
   Orden de X: x, y, z
   Orden de Y: a, b, c, d

En los ejercicios 4-6, determine la matriz de la relación R sobre X con respecto del orden dado.

4. $R = \{(1, 2), (2, 3), (3, 4), (4, 5)\}$
   Orden de X: 1, 2, 3, 4, 5
5. R como en el ejercicio 4
   Orden de X: 5, 3, 1, 2, 4
6. $R = \{(x, y) \mid x < y\}$
   Orden de X: 1, 2, 3, 4

En los ejercicios 7-9, escriba la relación R, dada por la matriz, como un conjunto de pares ordenados.

7. $$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array}\!\!\begin{array}{c} \begin{array}{cccc} w & x & y & z \end{array} \\ \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

8. $$\begin{array}{c} \\ 1 \\ 2 \end{array}\!\!\begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

9. $$\begin{array}{c} \\ w \\ x \\ y \\ z \end{array}\!\!\begin{array}{c} \begin{array}{cccc} w & x & y & z \end{array} \\ \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

10. ¿Cómo podemos determinar, rápidamente, si una relación R es antisimétrica, examinando la matriz de R (con respecto de cierto orden)?

11. Indique si la relación del ejercicio 9 es reflexiva, simétrica, transitiva, antisimétrica, un orden parcial o una relación de equivalencia.

12. Dada la matriz de una relación R de X en Y, ¿cómo podemos determinar la matriz de la relación inversa R^{-1}?

13. Determine la matriz de la inversa de cada una de las relaciones de los ejercicios 7 y 8.

En los ejercicios 14-16, determine

(a) La matriz A_1 de la relación R_1 (con respecto de los órdenes dados).
(b) La matriz A_2 de la relación R_2 (con respecto de los órdenes dados).
(c) El producto matricial A_1A_2.
(d) Utilice el resultado de la parte (c) para determinar la matriz de la relación $R_2 \circ R_1$.
(e) Utilice el resultado de la parte (d) para determinar la relación $R_2 \circ R_1$ (como un conjunto de pares ordenados).

14. $R_1 = \{(1, x), (1, y), (2, x), (3, x)\}$
$R_2 = \{(x, b), (y, b), (y, a), (y, c)\}$
Órdenes: 1, 2, 3; x, y; a, b, c

15. $R_1 = \{(x, y) \mid x \text{ divide a y}\}$; R_1 es de X en Y
$R_2 = \{(y, z) \mid y > z\}$; R_2 es de Y en Z
Órdenes de X y Y: 2, 3, 4, 5
Orden de Z: 1, 2, 3, 4

16. $R_1 = \{(x, y) \mid x + y \leq 6\}$; R_1 es de X en Y
$R_2 = \{(y, z) \mid y = z + 1\}$; R_2 es de Y en Z
Órdenes de X, Y y Z: 1, 2, 3, 4, 5

17. Dada la matriz de una relación de equivalencia R sobre X, ¿cómo podemos determinar con facilidad la clase de equivalencia que contiene al elemento $x \in X$?

★ **18.** Sean R_1 una relación de X en Y y R_2 una relación de Y en Z. Elija órdenes para X, Y y Z. Todas las matrices de las relaciones son con respecto de estos órdenes. Sea A_1 la matriz de R_1 y A_2 la matriz de R_2. Muestre que la ik-ésima entrada en el producto matricial A_1A_2 es igual al número de elementos en el conjunto

$$\{m \mid (i, m) \in R_1 \text{ y } (m, k) \in R_2\}.$$

† 2.7 *BASES DE DATOS RELACIONALES*

El prefijo "bi" en una relación binaria R refleja el hecho de que R tiene dos columnas cuando se escribe como una tabla. Con frecuencia, es útil permitir que una tabla tenga un número arbitrario de columnas. Si una tabla tiene n columnas, la relación correspondiente es una **relación n-aria**.

EJEMPLO 2.7.1

La tabla 2.7.1 representa una relación 4-aria. Esta tabla expresa la relación entre los números de identificación, los nombres, las posiciones y las edades. □

También podemos expresar una relación n-aria como una colección de n-adas.

† Esta sección puede omitirse sin pérdida de continuidad.

TABLA 2.7.1
JUGADOR

Número de identificación	*Nombre*	*Posición*	*Edad*
22012	Johnsonbaugh	c	22
93831	Glover	of	24
58199	Battey	p	18
84341	Cage	c	30
01180	Homer	1b	37
26710	Score	p	22
61049	Johnsonbaugh	of	30
39826	Singleton	2b	31

EJEMPLO 2.7.2

La tabla 2.7.1 puede expresarse como el conjunto

{(22012, Johnsonbaugh, c, 22), (93831, Glover, of, 24),
(58199, Battey, p, 18), (84341, Cage, c, 30),
(01180, Homer, 1b, 37), (26710, Score, p, 22),
(61049, Johnsonbaugh, of, 30), (39826, Singleton, 2b, 31)}

de 4-adas. □

Una **base de datos** es una colección de registros controlada mediante una computadora. Por ejemplo, una base de datos de una línea aérea podría contener los registros de las reservaciones, programación de los vuelos, equipo, etc. Los sistemas de cómputo permiten guardar grandes cantidades de información en bases de datos. Los datos están disponibles para su uso mediante varias aplicaciones. Los **sistemas de administración de bases de datos** son programas que permiten a los usuarios tener acceso a la información contenida en las bases de datos. El **modelo de base de datos relacional**, ideado por E. F. Codd en 1970, se basa en el concepto de una relación n-aria. A continuación presentaremos algunas de las ideas fundamentales en la teoría de las bases de datos relacionales. Para más detalles, el lector puede consultar [Codd; Date; y Kroenke]. Comenzaremos con un poco de terminología.

Las columnas de una relación n-aria se llaman **atributos** (o campos). El dominio de un atributo es un conjunto al cual pertenecen todos los elementos en ese atributo. Por ejemplo, en la tabla 2.7.1, el atributo Edad se podría considerar como el conjunto de todos los enteros positivos menores que 100. El atributo Nombre se podría considerar como todas las cadenas sobre el alfabeto con longitud menor o igual a 30.

Un atributo individual o una combinación de atributos para una relación es una **clave** si los valores de los atributos definen de manera única una n-ada. Por ejemplo, en la tabla 2.7.1, podemos considerar al atributo Número de identificación como una clave. (Suponemos que cada persona tiene un número de identificación único.) El atributo Nombre no es una clave, pues personas diferentes pueden tener el mismo nombre. Por esta misma razón, no podemos considerar a los atributos Posición o Edad como una clave. La combinación de Nombre y Posición puede servir como clave para la tabla 2.7.1, pues en nuestro ejemplo, cada jugador queda definido de manera única por su nombre y su posición.

Un sistema de administración de bases de datos responde a **consultas**. Una consulta es una solicitud de información en la base de datos. Por ejemplo, "Determinar todas las personas que juegan en el jardín" es una consulta significativa para la relación dada por la tabla 2.7.1. Analizaremos varias operaciones relacionales que pueden utilizarse para responder a consultas en el modelo de base de datos relacional.

EJEMPLO 2.7.3 ***Selección***

El operador de selección elige ciertas n-adas de una relación. Las elecciones se realizan dando condiciones sobre los atributos. Por ejemplo, para la relación JUGADOR dada en la tabla 2.7.1,

JUGADOR [Posición = c]

seleccionará las n-adas

(22012, Johnsonbaugh, c, 22), (84341, Cage, c, 30). □

EJEMPLO 2.7.4 ***Proyección***

Mientras que el operador de selección elige los renglones de una relación, el operador de proyección elige las columnas. Además, se eliminan los duplicados. Por ejemplo, para la relación JUGADOR dada en la tabla 2.7.1,

JUGADOR [Nombre, Posición]

seleccionará las n-adas

(Johnsonbaugh, c), (Glover, of), (Battey, p), (Gage, c),

(Homer, 1b), (Score, p), (Johnsonbaugh, of), (Singleton, 2b). □

TABLA 2.7.2
ASIGNACIÓN

NIJ	*Equipo*
39826	Blue Sox
26710	Mutts
58199	Jackalopes
01180	Mutts

EJEMPLO 2.7.5 ***Fusión***

Los operadores de selección y de proyección controlan una única relación; el operador fusión controla dos relaciones. La operación de fusión sobre las relaciones R_1 y R_2 comienza examinando todos los pares de n-adas, uno de R_1 y otro de R_2. Si se satisface la condición de fusión, las n-adas se combinan para formar una nueva. La condición de fusión especifica una relación entre un atributo de R_1 y un atributo de R_2. Por ejemplo, realicemos una operación de fusión sobre las tablas 2.7.1 y 2.7.2. Como condición consideramos

Número de identificación = Número de identificación del jugador.

Consideremos un renglón de la tabla 2.7.1 y un renglón de la tabla 2.7.2; si Número de identificación = Número de identificación del jugador, combinamos los renglones. Por ejemplo, el Número de identificación 01180 en el primer renglón (01180, Homer, 1b, 37) de la tabla 2.7.1 concuerda con el Número de identificación del jugador en el cuarto renglón (01180, Mutts) de la tabla 2.7.2. Estas n-adas se combinan escribiendo primero la n-ada de la tabla 2.7.1, seguida de la n-ada de la tabla 2.7.2 y eliminando las entradas iguales en los atributos dados para obtener

(01180, Homer, 1b, 37, Mutts).

Esta operación se expresa como

JUGADOR [Número de identificación = NIJ] ASIGNACIÓN.

La relación obtenida al realizar esta fusión aparece en la tabla 2.7.3. □

TABLA 2.7.3
JUGADOR [Número de identificación = NIJ] ASIGNACIÓN

Número de identificación	*Nombre*	*Posición*	*Edad*	*Equipo*
58199	Battey	p	18	Jackalopes
01180	Homer	1b	37	Mutts
26710	Score	p	22	Mutts
39826	Singleton	2b	31	Blue Sox

La mayor parte de las consultas a una base de datos relacional necesitan de varias operaciones para proporcionar la respuesta.

EJEMPLO 2.7.6

Describa las operaciones que proporcionen la respuesta a la consulta "Determinar los nombres de todas las personas que juegan para algún equipo".

Si primero realizamos la fusión de las relaciones dadas por las tablas 2.7.1 y 2.7.2 sujeta a la condición Número de identificación = Número de identificación del jugador, obtenemos la tabla 2.7.3, la cual enumera todas las personas que juegan para algún equipo, junto con otra información. Para obtener los nombres, basta proyectar sobre el atributo Nombre. Obtenemos la relación

Nombre
Battey
Homer
Score
Singleton

Formalmente, podemos especificar estas operaciones como

TEMP := JUGADOR [Número de identificación = Número de identificación del jugador] ASIGNACIÓN
TEMP [Nombre] □

EJEMPLO 2.7.7

Describa las operaciones que proporcionen la respuesta a la consulta "Determinar los nombres de todas las personas que juegan para los Mutts".

Si primero utilizamos el operador de selección para elegir los renglones de la tabla 2.7.2 que hacen referencia a los jugadores de los Mutts, obtenemos la relación

TEMP1

NIJ	*Equipo*
26710	Mutts
01180	Mutts

Si ahora realizamos la fusión de la tabla 2.7.1 y la relación TEMP1 sujeta a Número de identificación = Número de identificación del jugador, obtenemos la relación

TEMP2

Número de identificación	*Nombre*	*Posición*	*Edad*	*Equipo*
01180	Homer	1b	37	Mutts
26710	Score	p	22	Mutts

Si proyectamos la relación TEMP2 sobre al atributo Nombre, obtenemos la relación

Podríamos especificar formalmente estas operaciones como sigue:

Nombre
Homer
Score

TEMP1 := ASIGNACIÓN [Equipo = Mutts]
TEMP2 := JUGADOR [Número de identificación = Número de identificación del jugador] TEMP1
TEMP2 [Nombre] □

Observe que las operaciones

TEMP1 := JUGADOR [Número de identificación = Número de identificación del jugador] ASIGNACIÓN
TEMP2 := TEMP1 [Equipo = Mutts]
TEMP2 [Nombre]

también responderían a la consulta del ejemplo 2.7.7.

Ejercicios

1. Exprese la relación dada por la tabla 2.7.4 como un conjunto de *n*-adas.

TABLA 2.7.4
EMPLEADO

Número de identificación	*Nombre*	*Jefe*
1089	Suzuki	Zamora
5620	Kaminski	Jones
9354	Jones	Yu
9551	Ryan	Washington
3600	Beaulieu	Yu
0285	Schmidt	Jones
6684	Manacotti	Jones

2. Exprese la relación dada por la tabla 2.7.5 como un conjunto de *n*-adas.

TABLA 2.7.5
DEPARTAMENTO

Departamento	*Jefe*
23	Jones
04	Yu
96	Zamora
66	Washington

3. Exprese la relación dada por la tabla 2.7.6 como un conjunto de *n*-adas.

TABLA 2.7.6
PROVEEDOR

Departamento	*Número de parte*	*Cantidad*
04	335B2	220
23	2A	14
04	8C200	302
66	42C	3
04	900	7720
96	20A8	200
96	1199C	296
23	772	39

4. Exprese la relación dada por la tabla 2.7.7 como un conjunto de n-adas.

TABLA 2.7.7
COMPRADOR

Nombre	*Número de parte*
United Supplies	2A
ABC Unlimited	8C200
United Supplies	1199C
JCN Electronics	2A
United Supplies	335B2
ABC Unlimited	772
Danny's	900
United Supplies	772
Underhanded Sales	20A8
Danny's	20A8
DePaul University	42C
ABC Unlimited	20A8

En los ejercicios 5-20, escriba una serie de operaciones para responder la consulta. Además, proporcione una respuesta a la consulta. Utilice las tablas 2.7.4 a 2.7.7.

5. Determine los nombres de todos los empleados. (No incluya a los jefes.)
6. Determine los nombres de todos los jefes.
7. Determine todos los números de parte.
8. Determine los nombres de todos los compradores.
9. Determine los nombres de todos los empleados que son dirigidos por Jones.
10. Determine todos los números de partes proporcionadas por el departamento 96.
11. Determine todos los compradores de la parte 20A8.
12. Determine todos los empleados del departamento 04.
13. Determine los números de parte de las partes cuya existencia es al menos de 100 unidades.
14. Determine todos los números de departamento de los departamentos que proporcionan partes a Danny's.
15. Determine los números de parte y la cantidad de partes adquiridas por United Supplies.
16. Determine todos los jefes de departamento que producen partes para ABC Unlimited.
17. Determine los nombres de todos los empleados que trabajan en departamentos que proporcionan partes para JCN Electronics.
18. Determine todos los compradores que adquieren partes en el departamento dirigido por Jones.
19. Determine todos los compradores que adquieren partes producidas por el departamento donde trabaja Suzuki.
20. Determine todos los números de parte y cantidades para el departamento de Zamora.
21. Cree al menos tres relaciones n-arias con datos artificiales que se puedan utilizar en una base de datos médicos. Ilustre la forma en que se podría utilizar su base de datos, planteando y resolviendo dos consultas. Además, escriba una serie de operaciones que podrían servir para responder las consultas.
22. Describa una operación *unión* sobre una base de datos relacional. Ilustre la forma en que funciona el operador, respondiendo la siguiente consulta, utilizando las relaciones de las tablas 2.7.4 a 2.7.7: Determine los nombres de todos los empleados que trabajan en el departamento 23 o en el departamento 96. Además, escriba una serie de operaciones que pudiesen servir para responder la consulta.
23. Describa una operación *intersección* sobre una base de datos relacional. Ilustre la forma en que funciona el operador, respondiendo la siguiente consulta, utilizando las relaciones de las tablas 2.7.4 a 2.7.7: Determine los nombres de todos los compradores que adquieren las partes 2A y 1199C. Además, escriba una serie de operaciones que pudiesen servir para responder la consulta.
24. Describa una operación *diferencia* sobre una base de datos relacional. Ilustre la forma en que funciona el operador, respondiendo la siguiente consulta, utilizando las relaciones de las tablas 2.7.4 a 2.7.7: Determine los nombres de todos los empleados que no trabajan en el departamento 04. Además, escriba una serie de operaciones que pudiesen servir para responder la consulta.

2.8 *FUNCIONES*

Una **función** es un tipo especial de relación. Recuerde (véase la definición 2.4.1) que una relación R de X en Y es un subconjunto del producto cartesiano $X \times Y$ y que

$$\text{dominio } R = \{x \in X \mid (x, y) \in R \text{ para alguna } y \in Y\}.$$

Si f es una relación de X en Y, para que f sea además una función, el dominio de f debe ser igual a X y si (x, y) y (x, y') están en f, debemos tener $y = y'$.

DEFINICIÓN 2.8.1

Una *función* f de X en Y es una relación de X en Y con las siguientes propiedades:

1. El dominio de f es X.
2. Si $(x, y), (x, y') \in f$, entonces $y = y'$.

A veces, una función de X en Y se denota como $f: X \to Y$.

EJEMPLO 2.8.2

La relación

$$f = \{(1, a), (2, b), (3, a)\}$$

de $X = \{1, 2, 3\}$ en $Y = \{a, b, c\}$ es una función de X en Y. El dominio de f es X y el rango de f es $\{a, b\}$. [Recuerde (véase la definición 2.4.1) que el rango de una relación R es el conjunto

$$\{y \in Y \mid (x, y) \in R \text{ para alguna } x \in X\}.] \quad \square$$

Una sucesión (véase la sección 2.2) es un tipo particular de función. Una sucesión cuyo menor índice es 1 es una función cuyo dominio es el conjunto de todos los enteros positivos o un conjunto de la forma $\{1, \ldots, n\}$. La sucesión del ejemplo 2.2.2 tiene dominio $\{1, 2, 3, 4, 5\}$. La sucesión del ejemplo 2.2.1 tiene como dominio al conjunto de todos los enteros positivos.

EJEMPLO 2.8.3

La relación

$$R = \{(1, a), (2, a), (3, b)\} \tag{2.8.1}$$

de $X = \{1, 2, 3, 4\}$ en $Y = \{a, b, c\}$ no es una función de X en Y. No se cumple la propiedad 1 de la definición 2.8.1. El dominio de R, $\{1, 2, 3\}$, no es igual a X. Si (2.8.1) se considera como una relación de $X' = \{1, 2, 3\}$ en $Y = \{a, b, c\}$, sería una función de X' en Y. □

EJEMPLO 2.8.4

La relación

$$R = \{(1, a), (2, b), (3, c), (1, b)\}$$

de $X = \{1, 2, 3\}$ en $Y = \{a, b, c\}$ no es una función de X en Y. No se cumple la propiedad 2 de la definición 2.8.1. Tenemos que $(1, a)$ y $(1, b)$ están en R pero $a \neq b$. □

Dada una función f de X en Y, de acuerdo con la definición 2.8.1, para cada elemento x del dominio X, existe exactamente una $y \in Y$ tal que $(x, y) \in f$. Este único valor y se denota $f(x)$. En otras palabras, $y = f(x)$ es otra forma de escribir $(x, y) \in f$.

EJEMPLO 2.8.5

Para la función f del ejemplo 2.8.2, podemos escribir

$$f(1) = a, \quad f(2) = b, \quad f(3) = a.$$ □

Las funciones que implican al **operador módulo** juegan un importante papel en las matemáticas y las ciencias de la computación.

DEFINICIÓN 2.8.6

Si x es un entero no negativo y y es un entero positivo, definimos x mod y como el residuo que queda al dividir x entre y.

EJEMPLO 2.8.7

$$6 \bmod 2 = 0, \quad 5 \bmod 1 = 0, \quad 8 \bmod 12 = 8, \quad 199673 \bmod 2 = 1$$ □

EJEMPLO 2.8.8

¿Qué día de la semana será 365 días después de un miércoles?

Siete días después del miércoles será miércoles de nuevo; 14 días después del miércoles será miércoles de nuevo; y en general, si n es un entero positivo, después de $7n$ días será miércoles de nuevo. Así, debemos eliminar tantos sietes de 365 como sea posible y ver cuántos días restan. Pero esto es $365 \bmod 7 = 1$. Así, 365 días después del miércoles será un día de la semana después, es decir, jueves. Esto explica por qué, excepto por los años bisiestos en que se agrega un día a febrero, el día y mes idénticos en años consecutivos se mueve hacia adelante un día de la semana. □

EJEMPLO 2.8.9 ***Números estándar internacionales de libros***

Un número estándar internacional de un libro (ISBN) es un código de 10 caracteres separados por guiones, como 0-8065-0959-7. Un ISBN consta de cuatro partes: un código de grupo, un código de editor, un código que identifica de manera única al libro entre aquellos publicados por el editor particular, y un carácter de verificación. Este último sirve para validar el ISBN.

Para el ISBN 0-8065-0959-7, el código de grupo es 0, el cual identifica al libro como correspondiente a un país de habla inglesa. El código del editor 8065 identifica al libro como publicado por Citadel Press. El código 0959 identifica de manera única al libro entre aquellos publicados por Citadel Press (Brode: *Woody Allen: His Films and Career*, en este caso). El carácter de verificación es s mod 11, donde s es la suma del primer dígito más dos veces el segundo dígito más tres veces el tercer dígito, . . . , más nueve veces el noveno dígito. Si este valor es 10, el carácter de verificación es X. Por ejemplo, la suma s para el ISBN 0-8065-0959-7 es

$$s = 0 + 2 \cdot 8 + 3 \cdot 0 + 4 \cdot 6 + 5 \cdot 5 + 6 \cdot 0 + 7 \cdot 9 + 8 \cdot 5 + 9 \cdot 9 = 249.$$

Así, el carácter de verificación es $249 \bmod 11 = 7$. □

EJEMPLO 2.8.10 ***Funciones de dispersión***

Suponga que tenemos ciertas celdas en la memoria de una computadora, con índices de 0 a 10 (véase la figura 2.8.1). Queremos guardar y recuperar enteros no negativos arbitrarios en estas celdas. Nuestro método consiste en utilizar una **función de dispersión** (hash). Una función de dispersión considera un elemento de datos por guardar o recuperar y calcula la primera opción para la posición del elemento. Por ejemplo, en nuestro problema, para guardar o recuperar el número n, podríamos considerar como primera opción para la posición al número n mod 11. Nuestra función de dispersión es

$$h(n) = n \bmod 11.$$

La figura 2.8.1 muestra el resultado de guardar 15, 558, 32, 132, 102 y 5, en ese orden, en unas celdas originalmente vacías.

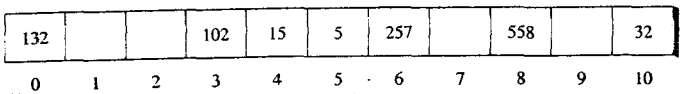


FIGURA 2.8.1 Celdas en la memoria de una computadora.

Ahora suponga que queremos guardar 257. Como $h(257) = 4$, tendríamos que guardar 257 en la posición 4; sin embargo, esta posición ya está ocupada. En este caso decimos que ha ocurrido una **colisión**. Más precisamente, ocurre una colisión para una función de dispersión H si $H(x) = H(y)$, pero $x \neq y$. Para controlar las colisiones se necesita una **política para resolver colisiones**. Una política sencilla para resolver colisiones es determinar la siguiente celda mayor (suponiendo que 0 va después de 10) no ocupada. Si utilizamos esta política para resolver colisiones, podríamos guardar 257 en la posición 6 (véase la figura 2.8.1).

Si queremos localizar un valor ya guardado n, calculamos $m = h(n)$ y comenzamos a buscar en la posición m. Si n no está en esa posición, buscamos en la siguiente posición mayor (de nuevo, suponiendo que 0 va después de 10); si n no está en esa posición, pasamos a la siguiente posición, etc. Si llegamos a una celda vacía o regresamos a la posición original, concluimos que n no está presente; en caso contrario, obtenemos la posición de n.

Si las colisiones no ocurren con frecuencia, y si cuando una de ellas ocurre se resuelve con rapidez, entonces la dispersión proporciona un método muy rápido para guardar y recuperar datos. Como ejemplo, los datos relativos al personal se guardan y recuperan con frecuencia mediante la dispersión de los números de identificación de los empleados. □

A continuación definimos el **piso** y el **techo** de un número real.

DEFINICION 2.8.11

El *piso* de x, denotado $\lfloor x \rfloor$, es el mayor entero menor o igual a x. El *techo* de x, denotado $\lceil x \rceil$, es el menor entero mayor o igual que x.

EJEMPLO 2.8.12

$$\lfloor 8.3 \rfloor = 8, \qquad \lceil 9.1 \rceil = 10,$$
$$\lfloor -8.7 \rfloor = -9, \qquad \lceil -11.3 \rceil = -11,$$
$$\lceil 6 \rceil = 6, \qquad \lceil -8 \rceil = -8$$ □

El piso de x "redondea x hacia abajo", mientras que el techo de x "redondea x hacia arriba". En todo el libro utilizaremos las funciones piso y techo.

EJEMPLO 2.8.13

En 1996, la tarifa postal de primera clase en Estados Unidos para pesos de hasta 11 onzas, era de 32 centavos por la primera onza o fracción y de 23 centavos por cada onza o fracción adicional. La tarifa postal $P(w)$ como una función del peso w está dada por la ecuación

$$P(w) = 32 + 23\lceil w - 1 \rceil, \quad 11 \geq w > 0.$$

La expresión $\lceil w - 1 \rceil$ cuenta el número de onzas adicionales arriba de 1 donde una fracción cuenta como una onza adicional. Como ejemplos,

$$P = (3.7) = 32 + 23\lceil 3.7 - 1 \rceil = 32 + 23\lceil 2.7 \rceil = 32 + 23 \cdot 3 = 101,$$
$$P(2) = 32 + 23\lceil 2 - 1 \rceil = 32 + 23\lceil 1 \rceil = 32 + 23 \cdot 1 = 55.$$ □

DEFINICION 2.8.14

Una función f de X en Y es *uno a uno* (o *inyectiva*) si para cada $y \in Y$ existe a lo más una $x \in X$ tal que $f(x) = y$.

La condición dada en la definición 2.8.14 para que una función sea uno a uno es equivalente a: Si $x, x' \in X$ y $f(x) = f(x')$, entonces $x = x'$.

Debido a que la cantidad de datos potenciales es por lo general mucho mayor que la memoria disponible, es usual que las funciones de dispersión no sean uno a uno. En otras palabras, la mayor parte de las funciones de dispersión provocan colisiones.

EJEMPLO 2.8.15

La función

$$f = \{(1, b), (3, a), (2, c)\}$$

de $X = \{1, 2, 3\}$ en $Y = \{a, b, c, d\}$ es uno a uno. □

EJEMPLO 2.8.16

La función del ejemplo 2.8.2 no es uno a uno, pues $f(1) = a = f(3)$. □

Si el rango de una función f es Y, decimos que la función es **sobre** Y.

DEFINICION 2.8.17

Si f es una función de X en Y y el rango de f es Y, f es *sobre* Y (o una *función suprayectiva*).

EJEMPLO 2.8.18

La función

$$f = \{(1, a), (2, c), (3, b)\}$$

de $X = \{1, 2, 3\}$ en $Y = \{a, b, c\}$ es uno a uno y sobre Y. □

EJEMPLO 2.8.19

La función f del ejemplo 2.8.15 *no* es sobre $Y = \{a, b, c, d\}$. Es sobre $\{a, b, c\}$. □

DEFINICIÓN 2.8.20

Una función que es uno a uno y sobre es una *biyección.*

EJEMPLO 2.8.21

La función f del ejemplo 2.8.18 es una biyección. □

Supongamos que f es una función uno a uno y sobre de X en Y. Puede mostrarse (véase el ejercicio 60) que la relación inversa

$$\{(y, x) \mid (x, y) \in f\}$$

es una función de Y en X. Esta nueva función, denotada f^{-1}, es llamada f **inversa**.

EJEMPLO 2.8.22

Para la función f del ejemplo 2.8.18, tenemos

$$f^{-1} = \{(a, 1), (c, 2), (b, 3)\}.$$ □

Como las funciones son tipos especiales de relaciones, podemos formar la **composición** de dos funciones. En específico, supongamos que g es una función de X en Y y que f es una función de Y en Z. Dada $x \in X$, podemos aplicarle g para determinar un único elemento $y = g(x) \in Y$. Luego podemos aplicar f para determinar un único elemento $z = f(y) = f(g(x)) \in Z$. La función resultante de X en Z es la *composición de f con g* y se denota $f \circ g$.

EJEMPLO 2.8.23

Dadas

$$g = \{(1, a), (2, a), (3, c)\},$$

una función de $X = \{1, 2, 3\}$ en $Y = \{a, b, c\}$, y

$$f = \{(a, y), (b, x), (c, z)\},$$

una función de Y en $Z = \{x, y, z\}$. la composición de X en Z es la función

$$f \circ g = \{(1, y), (2, y), (3, z)\}.$$ □

Un **operador binario** sobre un conjunto X asocia a cada par ordenado de elementos en X un elemento en X.

DEFINICIÓN 2.8.24

Una función de $X \times X$ en X es un *operador binario* sobre X.

EJEMPLO 2.8.25

Sea $X = \{1, 2, \ldots\}$. Si definimos

$$f(x, y) = x + y,$$

entonces f es un operador binario sobre X. □

EJEMPLO 2.8.26

Sea $X = \{a, b, c\}$. Si definimos

$$f(s, t) = st,$$

donde s y t son cadenas sobre X y st es la concatenación de s y t, entonces f es un operador binario sobre X^*. □

Un **operador unario** sobre un conjunto X asocia a cada elemento particular de X un elemento en X.

DEFINICIÓN 2.8.27

Una función de X en X es un *operador unario* sobre X.

EJEMPLO 2.8.28

Sea U un conjunto universal. Si definimos

$$f(X) = \overline{X}, \quad X \subseteq U,$$

entonces f es un operador unario sobre $P(U)$. □

Ejercicios

Determine si cada relación en los ejercicios 1-5 es una función de $X = \{1, 2, 3, 4\}$ en $Y = \{a, b, c, d\}$. Si es una función, determine su dominio y rango y determine si es uno a uno o sobre. Si es uno a uno y sobre, proporcione una descripción de la función inversa como un conjunto de pares ordenados y proporcione el dominio y rango de la función inversa.

1. $\{(1, a), (2, a), (3, c), (4, b)\}$
2. $\{(1, c), (2, a), (3, b), (4, c), (2, d)\}$
3. $\{(1, c), (2, d), (3, a), (4, b)\}$

4. $\{(1, d), (2, d), (4, a)\}$
5. $\{(1, b), (2, b), (3, b), (4, b)\}$
6. Dé un ejemplo de una función que sea uno a uno pero no sobre.
7. Dé un ejemplo de una función que sea sobre pero no uno a uno.
8. Dé un ejemplo de una función que no sea uno a uno ni sobre.
9. Dadas

$$g = \{(1, b), (2, c), (3, a)\},$$

una función de $X = \{1, 2, 3\}$ en $Y = \{a, b, c, d\}$, y

$$f = \{(a, x), (b, x), (c, z), (d, w)\},$$

una función de Y en $Z = \{w, x, y, z\}$, escriba $f \circ g$ como un conjunto de pares ordenados.

10. Dada

$$f = \{(x, x^2) \mid x \in X\},$$

una función de $X = \{-5, -4, \ldots, 4, 5\}$ al conjunto de enteros, escriba f como un conjunto de pares ordenados. ¿Es f uno a uno o sobre?

11. ¿Cuántas funciones existen de $\{1, 2\}$ en $\{a, b\}$? ¿Cuáles son uno a uno? ¿Cuáles son sobre?
12. Dada

$$f = \{(a, b), (b, a), (c, b)\},$$

una función de $X = \{a, b, c\}$ en X:

(a) Escriba $f \circ f$ y $f \circ f \circ f$ como conjuntos de pares ordenados.
(b) Defina

$$f^n = f \circ f \circ \cdots \circ f$$

como la composición de n copias de f. Determine f^9 y f^{623}.

13. Sea f la función de $X = \{0, 1, 2, 3, 4\}$ en X definida por

$$f(x) = 4x \bmod 5.$$

Escriba f como un conjunto de pares ordenados. ¿Es f uno a uno o sobre?

14. Sea f la función de $X = \{0, 1, 2, 3, 4, 5\}$ en X definida por

$$f(x) = 4x \bmod 6.$$

Escriba f como un conjunto de pares ordenados. ¿Es f uno a uno o sobre?

★ 15. Sean m y n enteros positivos. Sea f la función de

$$X = \{0, 1, \ldots, m-1\}$$

en X definida por

$$f(x) = nx \bmod m.$$

Determine condiciones sobre m y n que garanticen que f es uno a uno y sobre.

16. Verifique el carácter de verificación ISBN de este libro.

17. Los códigos universales de productos (UPC) son los familiares códigos de barras que identifican a los productos, de modo que se pueda revisar su precio de manera automática al pagar en una caja. Un UPC es un código de 12 dígitos, donde el primer dígito identifica el tipo de producto (0 identifica un artículo común de abarrotes, 2 es un artículo vendido por peso, 3 es un artículo médico, 4 es un artículo especial, 5 es un cupón, y 6 y 7 son artículos que no se venden al menudeo). Los siguientes cinco dígitos identifican al fabricante, los siguientes cinco dígitos identifican al producto y el último dígito es un dígito de verificación. (Todos los códigos UPC tienen un dígito de verificación. Siempre está presente en el código de barras, pero podría no aparecer en la versión impresa.) Por ejemplo, el UPC para un paquete de 10 tacos Ortega es 0-54400-00800-5. El primer cero indica que éste es un artículo común de abarrotes, los siguientes cinco dígitos identifican al fabricante Nabisco Foods, y los siguientes cinco dígitos 00800 identifican al producto como un paquete de 10 tacos Ortega.

El dígito de verificación se calcula como sigue. Primero se calcula s, donde s es tres veces la suma de los dígitos que ocupan los lugares impares, más la suma de los dígitos que ocupan los lugares pares, excepto el dígito de verificación. El dígito de verificación es el número c, entre 0 y 9, que satisface $(c + s) \bmod 10 = 0$. Para el código en el paquete de tacos, tendríamos

$$s = 3(0 + 4 + 0 + 0 + 8 + 0) + 5 + 4 + 0 + 0 + 0 = 45.$$

Como $(5 + 45) \bmod 10 = 0$, el dígito de verificación es 5.

Determine el dígito de verificación para el UPC cuyos primeros 11 dígitos son 3-41280-21414.

Para cada función de dispersión en los ejercicios 18-21, muestre la forma en que los datos serían insertados, en el orden dado, en celdas inicialmente vacías. Utilice la política para resolver colisiones del ejemplo 2.8.10.

18. $h(x) = x \bmod 11$; las celdas van del 0 al 10; datos: 53, 13, 281, 743, 377, 20, 10, 796
19. $h(x) = x \bmod 17$; las celdas van del 0 al 16; datos: 714, 631, 26, 373, 775, 906, 509, 2032, 42, 4, 136, 1028
20. $h(x) = x^2 \bmod 11$; las celdas y los datos como en el ejercicio 18
21. $h(x) = (x^2 + x) \bmod 17$; las celdas y los datos como en el ejercicio 19
22. Suponga que guardamos y recuperamos datos según lo descrito en el ejemplo 2.8.10. ¿Surgirá algún problema si eliminamos algunos datos? Explique.
23. Suponga que guardamos datos según lo descrito en el ejemplo 2.8.10 y que nunca guardamos más de 10 artículos. ¿Surgirá algún problema al recuperar datos si detenemos la búsqueda al encontrar una celda vacía? Explique.
24. Suponga que guardamos datos según lo descrito en el ejemplo 2.8.10 y recuperamos los datos como en el ejercicio 23. ¿Surgirá algún problema si eliminamos algunos datos? Explique.

Sea g una función de X en Y y sea f una función de Y en Z. Para cada afirmación en los ejercicios 25-30, escriba "verdadero" si la afirmación es verdadera. Si la afirmación es falsa, proporcione un contraejemplo.

25. Si f es uno a uno, entonces $f \circ g$ es uno a uno.
26. Si f y g son sobre, entonces $f \circ g$ es sobre.
27. Si f y g son uno a uno y sobre, entonces $f \circ g$ es uno a uno y sobre.
28. Si $f \circ g$ es uno a uno, entonces f es uno a uno.
29. Si $f \circ g$ es uno a uno, entonces g es uno a uno.
30. Si $f \circ g$ es sobre, entonces f es sobre.

Si f es una función de X en Y y $A \subseteq X$ y $B \subseteq Y$, definimos

$$f(A) = \{f(x) \mid x \in A\}, \quad f^{-1}(B) = \{x \in X \mid f(x) \in B\}.$$

Decimos que $f^{-1}(B)$ es la *imagen inversa* de B bajo f.

31. Sea

$$g = \{(1, a), (2, c), (3, c)\}$$

una función de $X = \{1, 2, 3\}$ en $Y = \{a, b, c, d\}$. Sean $S = \{1\}$, $T = \{1, 3\}$, $U = \{a\}$ y $V = \{a, c\}$. Determine $g(S)$, $g(T)$, $g^{-1}(U)$ y $g^{-1}(V)$.

☆ **32.** Sea f una función de X en Y. Demuestre que f es uno a uno si y sólo si

$$f(A \cap B) = f(A) \cap f(B)$$

para todos los subconjuntos A y B de X.

33. Sea f una función de X en Y. Defina una relación R sobre X como

$$x \, R \, y \quad \text{si} \quad f(x) = f(y).$$

Muestre que R es una relación de equivalencia sobre X.

34. Sea f una función de X en Y. Sea

$$\mathcal{S} = \{f^{-1}(\{y\}) \mid y \in Y\}.$$

[La definición de $f^{-1}(B)$, donde B es un conjunto, aparece antes del ejercicio 31.] Muestre que $\mathcal{S}$ es una partición de X. Describa una relación de equivalencia que dé lugar a esta partición.

35. Sea R una relación de equivalencia sobre un conjunto A. Defina una función f de A en el conjunto de clases de equivalencia de A mediante la regla

$$f(x) = [x].$$

¿Cuándo ocurre que $f(x) = f(y)$?

36. Sea R una relación de equivalencia sobre un conjunto A. Suponga que g es una función de A en un conjunto X con la propiedad de que si $x \, R \, y$, entonces $g(x) = g(y)$. Muestre que

$$h([x]) = g(x)$$

define una función del conjunto de clases de equivalencia de A en X. [Lo que hay que mostrar es que h asigna un valor de manera *única* a $[x]$: es decir, que si $[x] = [y]$, entonces $g(x) = g(y)$.]

★ **37.** Sea f una función de X en Y. Muestre que f es uno a uno si, y sólo si, siempre que g sea una función uno a uno de cualquier conjunto A en X, $f \circ g$ es uno a uno.

☆ **38.** Sea f una función de X en Y. Muestre que f es sobre Y si, y sólo si, siempre que g sea una función de Y sobre cualquier conjunto Z, $g \circ f$ es sobre Z.

Sea U un conjunto universal y sea $X \subseteq U$. Defina

$$C_X(x) = \begin{cases} 1 & \text{si } x \in X \\ 0 & \text{si } x \notin X. \end{cases}$$

C_X es la *función característica* de X (en U).

39. Muestre que $C_{X \cap Y}(x) = C_X(x) C_Y(x)$ para toda $x \in U$.

40. Muestre que $C_{X \cup Y}(x) = C_X(x) + C_Y(x) - C_X(x) C_Y(x)$ para toda $x \in U$.

41. Muestre que $C_{\bar{X}}(x) = 1 - C_X(x)$ para toda $x \in U$.

42. Muestre que $C_{X - Y}(x) = C_X(x)\,[1 - C_Y(x)]$ para toda $x \in U$.

43. Muestre que si $X \subseteq Y$, entonces $C_X(x) \leq C_Y(x)$ para toda $x \in U$.

44. Muestre que $C_{X \cup Y}(x) = C_X(x) + C_Y(x)$ para toda $x \in U$ si y sólo si $X \cap Y = \varnothing$.

45. Determine una fórmula para $C_{X \Delta Y}$. ($X \Delta Y$ es la diferencia simétrica de X y Y. La definición aparece antes del ejercicio 61, sección 2.1.)

46. Muestre que la función f de $\mathcal{P}(U)$ en el conjunto de funciones características en U, definida como

$$f(X) = C_X$$

es uno a uno y sobre.

47. Sea f una función característica en X. Defina una relación R sobre X como $x \, R \, y$ si $f(x) = f(y)$. De acuerdo con el ejercicio 33, R es una relación de equivalencia. ¿Cuáles son las clases de equivalencia?

Si X y Y son conjuntos, decimos que X es *equivalente* a Y si existe una función uno a uno y sobre de X en Y.

48. Muestre que la equivalencia de conjuntos es una relación de equivalencia.

49. Si X y Y son conjuntos finitos y X es equivalente a Y, ¿qué nos dice esto acerca de X y de Y?

50. Muestre que los conjuntos $\{1, 2, \ldots\}$ y $\{1, 4, \ldots\}$ son equivalentes.

★ **51.** Muestre que para cualquier conjunto X, X no es equivalente a $\mathcal{P}(X)$, el conjunto potencia de X.

52. Sean X y Y conjuntos. Muestre que existe una función uno a uno de X en Y si y sólo si existe una función de Y sobre X.

Un operador binario f sobre un conjunto X es *conmutativo* si $f(x, y) = f(y, x)$ para toda $x, y \in X$. En los ejercicios 53-57, indique si la función dada f es un operador binario sobre el conjunto X. Si f no es un operador binario, indique por qué. Si f es un operador binario, indique si es conmutativo o no.

53. $f(x, y) = x + y, \quad X = \{1, 2, \ldots\}$

54. $f(x, y) = x - y, \quad X = \{1, 2, \ldots\}$

55. $f(s, t) = st$, X es el conjunto de cadenas sobre $\{a, b\}$

56. $f(x, y) = x/y, \quad X = \{0, 1, 2, \ldots\}$

57. $f(x, y) = x^2 + y^2 - xy, \quad X = \{1, 2, \ldots\}$

En los ejercicios 58 y 59, proporcione un ejemplo de un operador unario (diferente de $f(x) = x$, para toda x) sobre el conjunto dado.

58. $\{\ldots, -2, -1, 0, 1, 2, \ldots\}$

59. El conjunto de cadenas sobre $\{a, b\}$

60. Muestre que si f es una función uno a uno y sobre de X en Y, entonces

$$\{(y, x) \mid (x, y) \in f\}$$

es un función uno a uno y sobre de Y en X.

61. ¿Cómo podemos determinar con rapidez si una relación R es una función, examinando la matriz de R (con respecto de algún orden)?

62. Sea A la matriz de una función f de X en Y (con respecto al orden dado de X y Y). ¿Qué condiciones debe satisfacer A para que f sea sobre Y?

63. Sea A la matriz de una función f de X en Y (con respecto del orden dado de X y Y). ¿Qué condiciones debe satisfacer A para que f sea uno a uno?

En los ejercicios 64-66, escriba "verdadero" si la afirmación es verdadera para todos los números reales. Si es falsa, proporcione un contraejemplo.

64. $\lceil x + 3 \rceil = \lceil x \rceil + 3$

65. $\lceil x + y \rceil = \lceil x \rceil + \lceil y \rceil$

66. $\lfloor x + y \rfloor = \lfloor x \rfloor + \lceil y \rceil$

67. Muestre que si n es un entero impar,

$$\left\lfloor \frac{n^2}{4} \right\rfloor = \left(\frac{n-1}{2}\right)\left(\frac{n+1}{2}\right).$$

68. Muestre que si n es un entero impar,

$$\left\lceil \frac{n^2}{4} \right\rceil = \frac{n^2+3}{4}.$$

El 1 de enero del año x se presenta en el día de la semana mostrado en la segunda columna del renglón

$$y = \left(x + \left\lfloor \frac{x-1}{4} \right\rfloor - \left\lfloor \frac{x-1}{100} \right\rfloor + \left\lfloor \frac{x-1}{400} \right\rfloor\right) \bmod 7$$

en la siguiente tabla (véase [Ritter]).

y	*1 de enero*	*Año no bisiesto*	*Año bisiesto*
0	Domingo	enero, octubre	enero, abril, julio
1	Lunes	abril, julio	septiembre, diciembre
2	Martes	septiembre, diciembre	junio
3	Miércoles	junio	marzo, noviembre
4	Jueves	febrero, marzo, noviembre	febrero, agosto
5	Viernes	agosto	mayo
6	Sábado	mayo	octubre

Los meses con viernes 13 en el año x se determinan en el renglón y de la columna adecuada.

69. Determine los meses con viernes 13 en 1945.

70. Determine los meses con viernes 13 en este año.

71. Determine los meses con viernes 13 en el año 2000.

72. Sea X el conjunto de sucesiones con dominio finito. Defina una relación R sobre X como $s\,R\,t$ si $|\text{dominio } s| = |\text{dominio } t|$ y, si el dominio de s es $\{m, m+1, \ldots, m+k\}$ y el dominio de t es $\{n, n+1, \ldots, n+k\}$, entonces $s_{m+i} = t_{n+i}$ para $i = 0, \ldots, k$.

(a) Muestre que R es una relación de equivalencia.
(b) Explique con palabras lo que significa que dos sucesiones en X sean equivalentes bajo la relación R.
(c) Una sucesión es una función y por lo tanto es un conjunto de pares ordenados. Dos sucesiones son iguales si los dos conjuntos de pares ordenados son iguales. Compare la diferencia entre dos sucesiones equivalentes en X y dos sucesiones iguales en X.

NOTAS

La mayor parte de la bibliografía general en matemáticas discretas se refieren a los temas de este capítulo. [Halmos; Lipschutz; y Stoll] son recomendables para el lector que desee estudiar teoría de conjuntos, relaciones y funciones con mayor detalle. [Codd; Date; Kroenke; y Ullman] son referencias recomendables acerca de las bases de datos en general y el modelo relacional en particular.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 2.1

Conjunto: cualquier colección de objetos
Notación para conjuntos:
$\{x \mid x \text{ tiene la propiedad } P\}$
$|X|$: el número de elementos en el conjunto X
$x \in X$: x es un elemento del conjunto X
$x \notin X$: x no es un elemento del conjunto X
Conjunto vacío: $\varnothing$ o $\{\}$
$X = Y$, donde X y Y son conjuntos: X y Y tienen los mismos elementos
$X \subseteq Y$, X es un subconjunto de Y: todo elemento de X es también elemento de Y
X es un subconjunto propio de Y: $X \subseteq Y$ y $X \neq Y$
$P(X)$, el conjunto potencia de X: conjunto de todos los subconjuntos de X
$|P(X)| = 2^{|X|}$
$X \cup Y$, X unión Y: conjunto de elementos en X o Y
Unión de una familia $\mathcal{S}$ de conjuntos:
$\cup \mathcal{S} = \{x \mid x \in X \text{ para algún } X \in \mathcal{S}\}$
$X \cap Y$, X intersección Y: conjunto de elementos en X y Y
Intersección de una familia $\mathcal{S}$ de conjuntos: $\cap \mathcal{S} = \{x \mid x \in X \text{ para toda } X \in \mathcal{S}\}$
Conjuntos ajenos X y Y: $X \cap Y = \varnothing$
Familia de conjuntos ajenos por pares
$X - Y$, diferencia de X y Y, complemento relativo: conjunto de elementos en X pero no en Y
Conjunto universal, universo
$\overline{X}$, complemento de X: $U - X$, donde U es un conjunto universal
Propiedades de conjuntos (véase el teorema 2.1.8)
Leyes de De Morgan para conjuntos:

$$\overline{(A \cup B)} = \overline{A} \cap \overline{B},$$
$$\overline{(A \cap B)} = \overline{A} \cup \overline{B}$$

Partición de X: una colección $\mathcal{S}$ de subconjuntos no vacíos de X tales que todo elemento en X pertenece exactamente a un miembro de $\mathcal{S}$
Par ordenado: (x, y)
Producto cartesiano de X y Y:
$X \times Y = \{(x, y) \mid x \in X, y \in Y\}$
Producto cartesiano de $X_1, X_2, \ldots, X_n$:
$X_1 \times X_2 \times \cdots \times X_n = \{(a_1, a_2, \ldots, a_n) \mid a_i \in X_i\}$

Sección 2.2

Sucesión: lista donde se toma en cuenta el orden
Índice: en la sucesión $\{s_n\}$, n es el índice
Sucesión creciente: $s_n \leq s_{n+1}$ para toda n
Sucesión decreciente: $s_n \geq s_{n+1}$ para toda n
Subsucesión s_{n_k} de la sucesión $\{s_n\}$
Notación de suma o sigma:

$$\sum_{i=m}^{n} a_i = a_m + a_{m+1} + \cdots + a_n$$

Notación producto:

$$\prod_{i=m}^{n} a_i = a_m \cdot a_{m+1} \cdots a_n$$

Cadena: sucesión finita
Cadena nula, λ: cadena sin elementos
X^*: conjunto de todas las cadenas sobre X, incluyendo la cadena nula
X^+: conjunto de todas las cadenas no nulas sobre X
Longitud de la cadena α, $|\alpha|$: número de elementos en α
Concatenación de cadenas α y β, $\alpha\beta$: α seguida de β

Sección 2.3

Sistema numérico decimal
Sistema numérico binario
Sistema numérico hexadecimal
Base de un sistema numérico
Conversión de binario a decimal
Conversión de hexadecimal a decimal
Conversión de decimal a hexadecimal
Suma de números binarios
Suma de números hexadecimales

Sección 2.4

Relación binaria de X en Y: conjunto de pares ordenados (x, y), con $x \in X$, $y \in Y$

Dominio de una relación binaria R: $\{x \mid (x, y) \in R\}$

Rango de una relación binaria R: $\{y \mid (x, y) \in R\}$

Digráfica de una relación binaria

Relación reflexiva R sobre X: $(x, x) \in R$ para toda $x \in X$

Relación simétrica R sobre X: para toda $x, y \in X$, si $(x, y) \in R$, entonces $(y, x) \in R$

Relación antisimétrica R sobre X: para toda $x, y \in X$, si $(x, y) \in R$ y $x \neq y$, entonces $(y, x) \notin R$

Relación transitiva R en X: para toda $x, y, z \in X$, si (x, y) y (y, z) están en R, entonces $(x, z) \in R$

Orden parcial: relación que es reflexiva, antisimétrica y transitiva

Relación inversa R^{-1}: $\{(y, x) \mid (x, y) \in R\}$

Composición de relaciones $R_2 \circ R_1$: $\{(x, z) \mid (x, y) \in R_1, (y, z) \in R_2\}$

Sección 2.5

Relación de equivalencia: relación que es reflexiva, antisimétrica y transitiva

Clase de equivalencia que contiene a a, dada por la relación de equivalencia R: $[a] = \{x \mid x\, R\, a\}$

Las clases de equivalencia crean una partición del conjunto (teorema 2.5.9)

Sección 2.6

Matriz de una relación

R es una relación reflexiva si y sólo si la diagonal principal de la matriz de R consta de unos.

R es una relación simétrica si y sólo si la matriz de R es simétrica con respecto de la diagonal principal.

Si A_1 es la matriz de la relación R_1 y A_2 es la matriz de la relación A_2, la matriz de la relación $R_2 \circ R_1$ se obtiene reemplazando cada término distinto de cero en la matriz producto A_1A_2 por 1.

Sección 2.7

Relación n-aria: Conjunto de n-adas

Sistema de administración de bases de datos

Base de datos relacional

Clave

Consulta

Selección

Proyección

Fusión

Sección 2.8

Función de X en Y, $f: X \to Y$: relación de X en Y que satisface que el dominio de $f = X$ y si $(x, y), (x, y') \in f$, entonces $y = y'$

x mod y: residuo cuando x es dividido entre y

Función de dispersión

Colisión para una función de dispersión H: $H(x) = H(y)$

Política de resolución de colisiones

Piso de x, $\lfloor x \rfloor$: mayor entero menor o igual a x

Techo de x, $\lceil x \rceil$: menor entero mayor o igual a x

Función f uno a uno: si $f(x) = f(x')$, entonces $x = x'$

Función sobre f de X en Y: rango de $f = Y$

Biyección: función uno a uno y sobre

Inversa f^{-1} de una función f uno a uno y sobre: $\{(y, x) \mid (x, y) \in f\}$

Composición de funciones: $f \circ g = \{(x, z) \mid (x, y) \in g\}$ y $(y, z) \in f\}$

Operador binario sobre X: función de $X \times X$ sobre X

Operador unario sobre X: función de X sobre X

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 2.1

1. Si $A = \{1, 3, 4, 5, 6, 7\}$, $B = \{x \mid x$ es un entero par$\}$, $C = \{2, 3, 4, 5, 6\}$, determine $(A \cap B) - C$.

2. Si X es un conjunto y $|X| = 8$, ¿cuántos miembros tiene $P(X)$? ¿Cuántos subconjuntos propios tiene X?

3. Si $A \cup B = B$, ¿qué relación debe haber entre A y B?

4. ¿Son iguales los conjuntos

$$\{3, 2, 2\}, \quad \{x \mid x \text{ es un entero y } 1 < x \leq 3\}?$$

Explique.

Sección 2.2

5. Para la sucesión a definida por $a_n = 2n + 2$, determine

(a) a_6 (b) $\sum_{i=1}^{3} a_i$ (c) $\prod_{i=1}^{3} a_i$

(d) una fórmula para la subsucesión de a obtenida al elegir un término sí y uno no en a, comenzando por el primero.

6. Reescriba la suma

$$\sum_{i=1}^{n} (n - i)r^i$$

reemplazando el índice i por k, donde $i = k + 2$.

7. Sea

$$b_n = \sum_{i=1}^{n} (i + 1)^2 - i^2.$$

(a) Determine b_5 y b_{10}. (b) Determine una fórmula para b_n.
(c) ¿Es b creciente? (d) ¿Es b decreciente?

8. Sean $\alpha = ccddc$ y $\beta = c^3d^2$. Determine

(a) $\alpha\beta$ (b) $\beta\alpha$ (c) $|\alpha|$ (d) $|\alpha\alpha\beta\alpha|$

Sección 2.3

9. Escriba el número binario 10010110 en decimal.

10. Escriba el número decimal 430 en binario y en hexadecimal.

11. Sume los números binarios 11001 y 101001.

12. Escriba el número hexadecimal C39 en decimal.

Sección 2.4

En los ejercicios 13 y 14, determine si la relación definida sobre el conjunto de enteros positivos es reflexiva, simétrica, antisimétrica, transitiva o un orden parcial.

13. $(x, y) \in R$ si 2 divide a $x + y$ **14.** $(x, y) \in R$ si 3 divide a $x + y$

15. Proporcione un ejemplo de una relación sobre $\{1, 2, 3, 4\}$ que sea reflexiva, pero no antisimétrica ni transitiva.

16. Suponga que R es una relación sobre X, simétrica y transitiva, pero no reflexiva. Suponga también que $|X| \geq 2$. Defina la relación $\bar{R}$ sobre X como

$$\bar{R} = X \times X - R.$$

¿Cuáles de las siguientes afirmaciones son verdaderas? Para cada afirmación falsa, proporcione un contraejemplo.

(a) $\bar{R}$ es reflexiva. (b) $\bar{R}$ es simétrica.
(c) $\bar{R}$ es no antisimétrica. (d) $\bar{R}$ es transitiva.

Sección 2.5

17. ¿La relación

$$\{(1, 1), (1, 2), (2, 2), (4, 4), (2, 1), (3, 3)\}$$

es una relación de equivalencia sobre $\{1, 2, 3, 4\}$? Explique.

18. Si la relación

$$\{(1, 1), (2, 2), (3, 3), (4, 4), (1, 2), (2, 1), (3, 4), (4, 3)\}$$

es una relación de equivalencia sobre $\{1, 2, 3, 4\}$, determine $[3]$, la clase de equivalencia que contiene a 3. ¿Cuántas clases de equivalencia (distintas) existen?

19. Determine la relación de equivalencia (como un conjunto de pares ordenados) sobre $\{a, b, c, d, e\}$ cuyas clases de equivalencia sean

$$\{a\}, \quad \{b, d, e\}, \quad \{c\}.$$

20. Sea R la relación definida sobre el conjunto de cadenas de ocho bits como $s_1 R s_2$ siempre que s_1 y s_2 tengan el mismo número de ceros.
(a) Muestre que R es una relación de equivalencia.
(b) ¿Cuántas clases de equivalencia existen?
(c) Enumere un miembro de cada clase de equivalencia.

Sección 2.6

Los ejercicios 21-24 se refieren a las relaciones

$$R_1 = \{(1, x), (2, x), (2, y), (3, y)\}, \qquad R_2 = \{(x, a), (x, b), (y, a), (y, c)\}.$$

21. Determine la matriz A_1 de la relación R_1 con respecto de los órdenes

$$1, 2, 3; \quad x, y.$$

22. Determine la matriz A_2 de la relación R_2 con respecto de los órdenes

$$x, y; \quad a, b, c.$$

23. Determine el producto matricial $A_1 A_2$.

24. Utilice el resultado del ejercicio 23 para determinar la matriz de la relación $R_2 \circ R_1$.

Sección 2.7

En los ejercicios 25-28, escriba una serie de operaciones que respondan la consulta. Además, proporcione una respuesta a la consulta. Utilice las tablas 2.7.1 y 2.7.2.

25. Determine todos los equipos.

26. Determine los nombres y edades de todos los jugadores.

27. Determine los nombres de todos los equipos que tienen un lanzador (pitcher, p).

28. Determine los nombres de todos los equipos que tengan jugadores de 30 años o más.

Sección 2.8

29. Sean X el conjunto de cadenas sobre $\{a, b\}$ de longitud 4 y Y el conjunto de cadenas sobre $\{a, b\}$ de longitud 3. Defina una función f de X en Y mediante la regla

$$f(\alpha) = \text{cadena que consta de los primeros tres caracteres de } \alpha.$$

¿Es f uno a uno? ¿Es f sobre?

30. Determine los números reales que satisfacen $\lfloor x \rfloor \lfloor y \rfloor = \lfloor xy \rfloor - 1$.

31. Proporcione ejemplos de funciones f y g tales que $f \circ g$ sea sobre, pero que g no sea sobre.

32. Para la función de dispersión

$$h(x) = x \bmod 13,$$

muestre la forma en que los datos

$$784, \quad 281, \quad 1141, \quad 18, \quad 1, \quad 329, \quad 620, \quad 43, \quad 31, \quad 684$$

se insertarían en el orden dado en celdas inicialmente vacías, con índices del 0 al 12.

Es muy sencillo:

Paso 1: Encontramos la peor obra del mundo, un fracaso seguro.

Paso 2: Consigo un millón de dólares: aún existen muchas ancianas en el mundo.

Paso 3: Trabajas con el directorio y hablas con muchos patrocinadores: uno para el gobierno, y otro para nosotros. Sé que lo lograrás, eres un genio.

Paso 4: Estrenamos en Broadway, y en menos de lo que canta un gallo...

Paso 5: Cerramos la temporada en Broadway.

Paso 6: Nos llevamos el millón de dólares para Río de Janeiro.

— de Los Productores

3

ALGORITMOS

3.1 INTRODUCCIÓN
3.2 NOTACIÓN PARA LOS ALGORITMOS
3.3 EL ALGORITMO DE EUCLIDES
3.4 ALGORITMOS RECURSIVOS
3.5 COMPLEJIDAD DE LOS ALGORITMOS
RINCÓN DE SOLUCIÓN DE PROBLEMAS: DISEÑO Y ANÁLISIS DE UN ALGORITMO
3.6 ANÁLISIS DEL ALGORITMO DE EUCLIDES
† 3.7 EL SISTEMA CRIPTOGRÁFICO CON CLAVE PÚBLICA RSA
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Un algoritmo es un método, descrito paso a paso, para resolver algún problema. La receta de las carpas de Adlai Stevenson es un ejemplo de algoritmo:

1. Compre una carpa de 1 a 2 libras de peso y permítale nadar en agua limpia durante 24 horas.
2. Quítele las escamas y rebánela en filetes.
3. Unte los filetes con mantequilla y sazone con sal y pimienta.
4. Coloque el pescado en un molde y hornee a temperatura moderada durante 20 minutos.
5. Tire la carpa y cómase el molde.

Históricamente, existen diversos ejemplos de algoritmos, como en la antigua Babilonia. En realidad, la palabra "algoritmo" surge del nombre del matemático árabe del siglo XI al-Khowārizmī. Los algoritmos basados en principios matemáticos sólidos desempeñan un papel central en las matemáticas y las ciencias de la computación. Para que la solución a un problema se pueda realizar mediante una computadora, hay que describirla como una serie de pasos precisos.

† Esta sección puede omitirse sin pérdida de continuidad.

Después de presentar los algoritmos y nuestra notación para ellos, analizaremos el algoritmo del máximo común divisor, un antiguo algoritmo griego que se utiliza hasta la fecha. Después estudiaremos la complejidad de los algoritmos, es decir, el espacio y el tiempo necesarios para ejecutarlos, y analizaremos los recursos necesarios para ciertos algoritmos. Concluiremos analizando el sistema criptográfico con clave pública RSA, un método para codificar y decodificar mensajes cuya seguridad se basa principalmente en que no existe un algoritmo eficiente para determinar los divisores primos de un entero arbitrario.

3.1 *INTRODUCCIÓN*

Un **algoritmo** es un conjunto finito de instrucciones con las siguientes características:

- ***Precisión.*** Los pasos se enuncian con precisión.
- ***Unicidad.*** Los resultados intermedios en cada paso quedan definidos de manera única y sólo dependen de las entradas y los resultados de los pasos anteriores.
- ***Carácter finito.*** El algoritmo se detiene después de ejecutar un número finito de instrucciones.
- ***Entrada.*** El algoritmo recibe una entrada.
- ***Salida.*** El algoritmo produce una salida.
- ***Generalidad.*** El algoritmo se aplica a un conjunto de entradas.

Por ejemplo, consideremos el siguiente algoritmo:

1. $x := a$
2. Si $b > x$, entonces $x := b$.
3. Si $c > x$, entonces $x := c$.

el cual determina el máximo de tres números a, b y c. La idea del algoritmo es revisar los números uno por uno y copiar el máximo valor observado en una variable x. Al concluir el algoritmo, x será igual al mayor de los tres números.

La notación $y := z$ significa "copiar el valor de z en y" o, de manera equivalente, "reemplazar el valor actual de y por el valor de z". Al ejecutar $y := z$, el valor de z no cambia. $:=$ es el **operador de asignación.**

Ahora mostraremos cómo se aplica el algoritmo anterior a algunos valores específicos de a, b y c. Tal simulación es un **rastreo** (o seguimiento). Primero suponemos que

$$a = 1, \qquad b = 5, \qquad c = 3.$$

En la línea 1, asignamos x a a (1). En la línea 2, $b > x$ (5 > 1) es verdadero, por lo que asignamos x a b (5). En la línea 3, $c > x$ (3 > 5) es falso, por lo que no hacemos nada. En este momento, x es 5, el máximo de a, b y c.

Supongamos que

$$a = 6, \qquad b = 1, \qquad c = 9.$$

En la línea 1, a x le asignamos el valor de a (6). En la línea 2, $b > x$ (1 > 6) es falso, por lo que no hacemos nada. En la línea 3, $c > x$ (9 > 6) es verdadero, por lo que asignamos 9 a x. En este momento, x es igual a 9, el máximo de a, b y c.

Observemos que este ejemplo de algoritmo tiene el conjunto de propiedades establecidas al principio de esta sección.

Los pasos de un algoritmo deben establecerse con precisión. Los pasos del ejemplo anterior tienen la precisión como para poder escribirse en un lenguaje de programación y ejecutarse en una computadora.

Dados los valores de entrada, cada paso intermedio de un algoritmo produce un resultado único. Por ejemplo, dados los valores

$$a = 1, \qquad b = 5, \qquad c = 3,$$

en la línea 2 del ejemplo, se asigna 5 a x sin importar la persona o máquina que ejecute el algoritmo.

Un algoritmo termina después de un número finito de pasos que responden la pregunta solicitada. Por ejemplo, el algoritmo del ejemplo se detiene después de tres pasos y proporciona el máximo de los tres valores dados.

Un algoritmo recibe entradas y produce salidas. El algoritmo del ejemplo recibe, como entrada, los valores a, b y c, y produce, como salida, el valor x.

Un algoritmo debe ser general. El algoritmo del ejemplo puede determinar el máximo valor de *cualesquiera* tres números.

Nuestra descripción de un algoritmo es suficiente para las necesidades en este libro. Sin embargo, es posible dar una definición matemática más precisa de "algoritmo" (véanse las notas del capítulo 10).

En la sección 3.2 presentaremos una manera más formal de especificar los algoritmos y daremos más ejemplos de éstos.

Ejercicios

1. Escriba un algoritmo que determine el menor elemento entre a, b y c.
2. Escriba un algoritmo que determine el segundo elemento más pequeño entre a, b y c. Suponga que los valores de a, b y c son distintos.
3. Escriba el método usual, el que se enseña en matemáticas elementales, para la suma de dos enteros positivos como un algoritmo.
4. Consulte en el directorio telefónico las instrucciones para hacer una llamada de larga distancia. ¿Cuáles propiedades de un algoritmo (precisión, unicidad, carácter finito, entrada, salida, generalidad) tienen estas instrucciones? ¿Cuáles no?

3.2 *NOTACIÓN PARA LOS ALGORITMOS*

Aunque a veces el lenguaje común es adecuado para especificar un algoritmo, muchos investigadores en matemáticas y ciencias de la computación prefieren un **seudocódigo**, por su precisión, estructura y universalidad. El seudocódigo recibe ese nombre pues se asemeja al código real (programas) de lenguajes como Pascal y C. Existen muchas versiones de seudocódigo. A diferencia de los verdaderos lenguajes de computación, que se preocupan demasiado por los signos de puntuación, las letras mayúsculas y minúsculas, las palabras especiales, etc., cualquier versión de seudocódigo es aceptable, siempre que sus instrucciones no sean ambiguas y tenga la forma, aunque no la sintaxis exacta, del seudocódigo descrito en esta sección.

Como primer ejemplo de seudocódigo, escribiremos de nuevo el algoritmo de la sección 3.1, que determina el máximo de tres números.

ALGORITMO 3.2.1 ***Determinación del máximo de tres números***

Este algoritmo determina el máximo de los números a, b y c.

Entrada: Tres números a, b y c

Salida: x, el máximo de a, b y c

```
1.  procedure* max(a, b, c)
2.    x := a
3.    if b > x then    // si b es mayor que x, actualizar x
4.      x := b
5.    if c > x then    // si c es mayor que x, actualizar x
6.      x := c
7.    return(x)
8.  end max
```

Nuestro algoritmo consta de un título, una breve descripción del algoritmo, la entrada y salida del algoritmo, y los procedimientos con las instrucciones del algoritmo. El algoritmo 3.2.1 tiene un solo procedimiento. Para una fácil referencia a las líneas individuales dentro de un procedimiento, a veces las numeraremos. El procedimiento del algoritmo 3.2.1 tiene ocho líneas numeradas. La primera línea de un procedimiento tendrá la palabra **procedure**, después el nombre del procedimiento y, entre paréntesis, los parámetros del procedimiento, los cuales describen los datos, variables, arreglos, etc., que se encuentran disponibles para el procedimiento. En el algoritmo 3.2.1, los parámetros del procedimiento son los números a, b y c. La última línea de un procedimiento tiene la palabra **end** seguida por el nombre del procedimiento. Entre las líneas **procedure** y **end** se encuentran las líneas ejecutables del procedimiento. Las líneas 2-7 son las líneas ejecutables del procedimiento en el algoritmo 3.2.1.

Al ejecutar el procedimiento del algoritmo 3.2.1, en la línea 2, asignamos x a a. En la línea 3, comparamos b con x. Si b es mayor que x, ejecutamos la línea 4

$$x := b$$

pero si b no es mayor que x, pasamos a la línea 5, en la cual comparamos c con x. Si c es mayor que x, ejecutamos la línea 6

$$x := c$$

pero si c no es mayor que x, pasamos a la línea 7. Así, cuando llegamos a la línea 7, x contendrá correctamente al máximo de a, b y c.

En la línea 7 regresamos el valor de x, que es igual al máximo de los números a, b y c, a quien haya llamado al procedimiento, y concluimos. El algoritmo 3.2.1 ha encontrado en forma correcta el máximo de los tres números.

* En un seudocódigo, las palabras *procedure*, *if*, *then*, etc., pueden escribirse en español, aunque se ha hecho el uso en inglés.

En general, en la estructura **if-then**

if *p* **then**
  acción

si la condición *p* es verdadera, se ejecuta *acción* y el control pasa a la proposición posterior a *acción*. Si la condición *p* es falsa, el control pasa directamente a la proposición posterior a *acción*.

Una forma alternativa es la estructura **if-then-else**

if *p* **then**
  acción 1
else
  acción 2

si la condición *p* es verdadera, se ejecuta *acción* 1 (pero no *acción* 2) y el control pasa a la proposición posterior a *acción* 2. Si la condición *p* es falsa, se ejecuta *acción* 2 (pero no *acción* 1) y el control pasa a la proposición posterior a *acción* 2.

Como se muestra, utilizamos las sangrías para identificar las proposiciones que conforman *acción*. Además, si *acción* consta de varias proposiciones, las delimitamos con las palabras **begin** y **end**. Un ejemplo de una *acción* con varias proposiciones en un enunciado if es

if $x \geq 0$ **then**
  begin
  $x := x - 1$
  $a := b + c$
  end

Las dos diagonales // indican el inicio de un **comentario**, el cual se extiende hasta el final de la línea. Un ejemplo de comentario en el algoritmo 3.2.1 es

// si *b* es mayor que *x*, actualizar *x*

Los comentarios ayudan al lector a entender el algoritmo, pero no se ejecutan.

La proposición **return**(*x*) concluye un procedimiento y regresa el valor de *x* a quien llamó al procedimiento. La proposición **return** [sin (*x*)] sólo termina un procedimiento. Si no existe una proposición return, el proceso termina justo antes de la línea **end**.

Un procedimiento que contiene una proposición **return**(*x*) es una función. El dominio consta de todos los valores válidos para los parámetros y el rango es el conjunto de valores que puede regresar el procedimiento.

Al utilizar un seudocódigo, utilizaremos las operaciones aritméticas comunes +, −, * (para la multiplicación) y /, así como los operadores de relación =, ≠, <, >, ≤, ≥ y los operadores lógicos **and**, **or** y **not**. Utilizaremos = para denotar el operador de igualdad y := para la asignación. A veces utilizaremos proposiciones menos formales (*por ejemplo*, "Elegir un elemento *x* en *S*") si lo contrario pudiese esconder el significado. En general, las soluciones de los ejercicios que requieren algoritmos deben escribirse como en la forma ilustrada en el algoritmo 3.2.1.

Las líneas de un procedimiento, que se ejecutan en forma secuencial, por lo general son proposiciones de asignación, condicionales (if), ciclos, return y combinaciones de éstas. Una estructura cíclica útil es el **ciclo while**.

while *p* **do**
  acción

en donde *acción* se ejecuta varias veces, mientras *p* sea verdadera. El **cuerpo** de un ciclo es *acción*. Como en la proposición if, si *acción* consta de varias proposiciones, las delimitamos mediante las palabras **begin** y **end**. Ilustramos el ciclo while en el algoritmo 3.2.2, el cual determina el valor máximo en una sucesión. Como en el algoritmo 3.2.1, recorremos los números uno por uno y actualizamos la variable que contiene al máximo. Utilizamos el ciclo while para recorrer los números.

ALGORITMO 3.2.2 ***Determinación del elemento máximo en una sucesión finita***

Este algoritmo determina el número máximo en la sucesión $s_1, s_2, \ldots, s_n$. Esta versión utiliza un ciclo while.

Entrada: La sucesión $s_1, s_2, \ldots, s_n$ y la longitud *n* de la sucesión

Salida: *large*, el máximo elemento en esta sucesión

```
1.  procedure find_large(s, n)
2.     large := s_1
3.     i := 2
4.     while i ≤ n do
5.        begin
6.        if s_i > large then    // se ha encontrado un valor más grande
7.           large := s_i
8.        i := i + 1
9.        end
10.    return(large)
11. end find_large
```

Seguiremos el algoritmo 3.2.2 cuando $n = 4$ y *s* es la sucesión

$$s_1 = -2, \quad s_2 = 6, \quad s_3 = 5, \quad s_4 = 6.$$

En la línea 2 hacemos *large* igual a s_1; es decir, *large* tiene el valor −2. Después, en la línea 3, asignamos 2 a *i*. En la línea 4 probamos si $i \leq n$; en este caso, vemos si $2 \leq 4$. Como esta condición es verdadera, ejecutamos el cuerpo del ciclo while (líneas 5 a 9). En la línea 6 probamos si $s_i > large$; en este caso, probamos si $s_2 > large$ $(6 > -2)$. Como la condición es verdadera, ejecutamos la línea 7; y asignamos 6 a *large*. En la línea 8, hacemos *i* igual a 3. Entonces regresamos a la línea 4.

De nuevo verificamos si $i \leq n$; en este caso, verificamos si $3 \leq 4$. Como esta condición es verdadera, ejecutamos el cuerpo del ciclo while. En la línea 6, probamos si $s_i > large$; en este caso, probamos si $s_3 > large$ $(5 > 6)$. Como la condición es falsa, pasamos a la línea 7. En la línea 8, hacemos *i* igual a 4. Entonces regresamos a la línea 4.

De nuevo verificamos si $i \leq n$; en este caso, verificamos si $4 \leq 4$. Como esta condición es verdadera, ejecutamos el cuerpo del ciclo while. En la línea 6, probamos si $s_i > large$; en este caso, probamos si $s_4 > large$ $(6 > 6)$. Como la condición es falsa, pasamos a la línea 7. En la línea 8, hacemos *i* igual a 5. Entonces regresamos a la línea 4.

De nuevo verificamos si $i \leq n$; en este caso, verificamos si $5 \leq 4$. Como la condición es falsa, concluimos el ciclo while y llegamos a la línea 10, donde regresamos *large* (6). Hemos encontrado el máximo elemento en la sucesión.

En el algoritmo 3.2.2 recorrimos una sucesión utilizando la variable i, la cual tomó los valores enteros de 1 a n. Este tipo de ciclo es tan común que con frecuencia se utiliza un ciclo especial, el **ciclo for**, en vez del ciclo while. La forma del ciclo for es

for *var* := *init* **to** *limit* **do**
  acción

Como en los casos de los enunciados if y while, si *acción* consta de varios enunciados, los delimitamos con las palabras **begin** y **end**. Al ejecutar el ciclo for, la *acción* se ejecuta para los valores de *var* desde *init* hasta *limit*. Más precisamente, *init* y *limit* son expresiones que tienen valores enteros. La variable *var* empieza con el valor *init*. Si *var* ≤ *limit*, ejecutamos acción y luego sumamos 1 a *var*. Luego se repite el proceso, hasta que *var* > *limit*. Observe que si *init* > *limit*, *acción* no se ejecutará.

Podemos escribir el algoritmo 3.2.2 de la siguiente forma con un ciclo for.

ALGORITMO 3.2.3 ***Determinación del elemento máximo en una sucesión finita***

Este algoritmo determina el número máximo en la sucesión $s_1, s_2, \ldots, s_n$. Esta versión utiliza un ciclo for.

Entrada: La sucesión $s_1, s_2, \ldots, s_n$ y la longitud n de la sucesión

Salida: *large*, el máximo elemento en esta sucesión

```
1. procedure find_large(s, n)
2.   large := s_1
3.   for i := 2 to n do
4.     if s_i > large then    // se ha determinado un valor más grande
5.       large := s_i
6.   return(large)
7. end find_large
```

Durante el desarrollo de un algoritmo, con frecuencia es recomendable descomponer el problema original en dos o más subproblemas. Puede desarrollarse un procedimiento para resolver cada subproblema, después de lo cual estos procedimientos pueden combinarse para proporcionar una solución del problema original. Nuestros últimos algoritmos ilustran estas ideas.

Supongamos que necesitamos un algoritmo para determinar el mínimo número primo mayor que un entero positivo dado. Más precisamente, el problema es: Dado un entero positivo n, determinar el mínimo primo p tal que $p > n$. Podemos descomponer este problema al menos en dos subproblemas. Primero podríamos desarrollar un algoritmo para determinar si un entero positivo es primo. Luego podríamos utilizar este algoritmo para determinar el mínimo primo mayor que un entero positivo dado.

El algoritmo 3.2.4 verifica si un entero positivo m es primo. Sólo verificamos si algún entero entre 2 y $m - 1$ divide a m. Si determinamos un entero entre 2 y $m - 1$ que divida a m, entonces m no es primo. Si no podemos determinar un entero entre 2 y $m - 1$ que divida a m, entonces m es primo. (El ejercicio 17 muestra que basta verificar los enteros entre 2 y $\sqrt{m}$ como posibles divisores.) El algoritmo 3.2.4 muestra que los procedimientos pueden regresar los valores **true** (*verdadero*) o **false** (*falso*).

ALGORITMO 3.2.4 ***Verificar si un entero positivo es primo***

Este algoritmo verifica si el entero positivo m es primo. La salida es **true** (*verdadero*) si m es primo y **false** (*falso*) si m no es primo.

Entrada: m, un entero positivo

Salida: **true**, si m es primo; **false**, si m no es primo

```
procedure is_prime(m)
  for i := 2 to m − 1 do
    if m mod i = 0 then    // i divide a m
      return(false)
  return(true)
end is_prime
```

El algoritmo 3.2.5 determina el mínimo primo mayor que el entero positivo n y utiliza al algoritmo 3.2.4. Para llamar a un procedimiento que regrese un valor, como el algoritmo 3.2.4, basta llamarlo por su nombre. Para llamar un procedimiento, digamos, *proc*, que no regresa algún valor, escribimos

call $proc(p_1, p_2, \ldots, p_k)$,

donde $p_1, p_2, \ldots, p_k$ son los argumentos transferidos a *proc*.

ALGORITMO 3.2.5 ***Determinar un primo mayor que un entero dado***

Este algoritmo determina el mínimo primo mayor que el entero positivo n.

Entrada: n, un entero positivo

Salida: m, el menor primo mayor que n

```
procedure large_prime(n)
  m := n + 1
  while not is_prime(m) do
    m := m + 1
  return(m)
end large_prime
```

Como el número de primos es infinito (véase el ejercicio 18), el procedimiento del algoritmo 3.2.5 terminará en algún momento.

Ejercicios

Escriba todos los algoritmos con el estilo de los algoritmos 3.2.1 a 3.2.5.

1. Escriba un algoritmo cuya salida sea el menor elemento de la sucesión

$$s_1, \quad \ldots, \quad s_n.$$

2. Escriba un algoritmo cuya salida sean el primero y segundo elementos mayores de la sucesión

$$s_1, \ldots, s_n.$$

3. Escriba un algoritmo cuya salida sean el primero y segundo elementos menores de la sucesión

$$s_1, \ldots, s_n.$$

4. Escriba un algoritmo cuya salida sean los elementos máximo y mínimo de la sucesión

$$s_1, \ldots, s_n.$$

5. Escriba un algoritmo cuya salida sea el índice de la primera ocurrencia del elemento máximo de la sucesión

$$s_1, \ldots, s_n.$$

Ejemplo: Si la sucesión fuese

6.2 8.9 4.2 8.9,

el algoritmo tendría como salida el valor 2.

6. Escriba un algoritmo cuya salida sea el índice de la última ocurrencia del elemento máximo de la sucesión

$$s_1, \ldots, s_n.$$

Ejemplo: Si la sucesión fuese

6.2 8.9 4.2 8.9,

el algoritmo tendría como salida el valor 4.

7. Escriba un algoritmo cuya salida sea el índice de la primera ocurrencia del valor *key* en la sucesión

$$s_1, \ldots, s_n.$$

Si *key* no está en la sucesión, el algoritmo tiene como salida el valor 0.
Ejemplo: Si la sucesión fuese

'MARY' 'JOE' 'MARK' 'RUDY',

y *key* fuese 'MARK', el algoritmo tendría como salida el valor 3.

8. Escriba un algoritmo cuya salida sea el índice de la última ocurrencia del valor *key* en la sucesión

$$s_1, \ldots, s_n.$$

Si *key* no está en la sucesión, el algoritmo tiene como salida el valor 0.

9. Escriba un algoritmo cuya salida sea el índice del primer elemento que sea menor que su antecesor en la sucesión

$$s_1, \ldots, s_n.$$

Si los elementos están en orden creciente, el algoritmo tiene como salida el valor 0.
Ejemplo: Si la sucesión fuese

'AMY' 'BRUNO' 'ELIE' 'DAN' 'ZEKE',

el algoritmo tendría como salida el valor 4.

10. Escriba un algoritmo cuya salida sea el índice del primer elemento que sea mayor que su antecesor en la sucesión

$$s_1, \ldots, s_n.$$

Si los elementos están en orden decreciente, el algoritmo tiene como salida el valor 0.

11. Escriba un algoritmo que invierta la sucesión

$$s_1, \ldots, s_n.$$

Ejemplo: Si la sucesión fuese

'AMY' 'BRUNO' 'ELIE',

la sucesión invertida sería

'ELIE' 'BRUNO' 'AMY'.

12. Escriba el método usual, que se enseña en la escuela primaria, para multiplicar dos enteros positivos como un algoritmo.
13. Escriba un algoritmo que reciba como entrada la matriz de una relación R y verifique si R es reflexiva.
14. Escriba un algoritmo que reciba como entrada la matriz de una relación R y verifique si R es antisimétrica.
15. Escriba un algoritmo que reciba como entrada la matriz de una relación R y verifique si R es una función.
16. Escriba un algoritmo que reciba como entrada la matriz de una relación R y produzca como salida la matriz de la relación inversa R^{-1}.
17. Muestre que el entero positivo $m \geq 2$ es primo si y sólo si ningún entero entre 2 y $\sqrt{m}$ divide a m.
18. Muestre que el número de primos es infinito, completando el siguiente argumento:
Basta mostrar que si p es primo, entonces existe un primo mayor que p. Sean $p_1 < p_2 < \cdots < p_k = p$ los primos menores o iguales a p, y sea $n = p_1 p_2 \cdots p_k + 1$. Muestre que cualquier primo que divida a n es mayor que p.

3.3 *EL ALGORITMO DE EUCLIDES*

Un antiguo y famoso algoritmo para determinar el máximo común divisor de dos enteros es el **algoritmo de Euclides**. El máximo común divisor de dos enteros m y n (no ambos cero) es el máximo entero positivo que divide a m y a n. Por ejemplo, el máximo común divisor de 4 y 6 es 2, y el máximo común divisor de 3 y 8 es 1. Utilizamos el concepto de máximo común divisor cuando queremos verificar si una fracción m/n, donde m y n son enteros, está reducida a su mínima expresión. Si el máximo común divisor de m y n es 1, m/n está reducida a su mínima expresión; en caso contrario, podemos reducir m/n. Por ejemplo, 4/6 no está reducida a su mínima expresión, pues el máximo común divisor de 4 y 6 es 2, no 1. (Podemos dividir 4 y 6 entre 2.) La fracción 3/8 está reducida a su mínima expresión pues el máximo común divisor de 3 y 8 es 1. Después de analizar la divisibilidad de los enteros, examinaremos el máximo común divisor con detalle y presentaremos el algoritmo de Euclides.

Si a, b y q son enteros, $b \neq 0$, y satisfacen $a = bq$, decimos que b **divide** a a y escribimos $b \mid a$. En este caso, decimos que q es el **cociente**, y b es un **divisor** de a. Si b no divide a a, escribimos $b \nmid a$.

EJEMPLO 3.3.1

Como $21 = 3 \cdot 7$, 3 divide a 21 y escribimos $3 \mid 21$. El cociente es 7. □

Sean m y n enteros tales que no son ambos cero. Entre todos los enteros que dividen a m y n, existe un divisor más grande, conocido como el **máximo común divisor** de m y n.

DEFINICION 3.3.2

Sean m y n enteros tales que no son ambos cero. Un *divisor común* de m y n es un entero que divide a m y a n. El *máximo común divisor*, que se escribe

$$\text{mcd}(m, n)$$

es el mayor divisor común de m y n.

EJEMPLO 3.3.3

Los divisores positivos de 30 son

$$1, \quad 2, \quad 3, \quad 5, \quad 6, \quad 10, \quad 15, \quad 30$$

y los divisores positivos de 105 son

$$1, \quad 3, \quad 5, \quad 7, \quad 15, \quad 21, \quad 35, \quad 105;$$

así, los divisores positivos comunes de 30 y 105 son

$$1, \quad 3, \quad 5, \quad 15.$$

Esto implica que el máximo común divisor de 30 y 105, mcd(30, 105), es 15. □

Las propiedades de los divisores comunes dadas en el siguiente teorema serán útiles en nuestro trabajo posterior en esta sección.

TEOREMA 3.3.4

Sean m,n y c enteros.

(a) Si c es un divisor común de m y n, entonces

$$c \mid (m + n).$$

(b) Si c es un divisor común de m y n, entonces

$$c \mid (m - n).$$

(c) Si $c \mid m$, entonces $c \mid mn$.

Demostración. (a) Sea c un divisor común de m y n. Como $c \mid m$,

$$m = cq_1 \tag{3.3.1}$$

para algún entero q_1. De manera análoga, como $c \mid n$,

$$n = cq_2 \tag{3.3.2}$$

para algún entero q_2. Si sumamos las ecuaciones (3.3.1) y (3.3.2), obtenemos

$$m + n = cq_1 + cq_2 = c(q_1 + q_2).$$

Por tanto, c divide a $m + n$ (con cociente $q_1 + q_2$). Hemos demostrado la parte (a).

Las demostraciones de las partes (b) y (c) se dejan al lector (véanse los ejercicios 17 y 18). ■

Si dividimos el entero no negativo a entre el entero positivo b, obtenemos un cociente q y un residuo r que satisface

$$a = bq + r, \quad 0 \le r < b, \quad q \ge 0. \tag{3.3.3}$$

EJEMPLO 3.3.5

Ilustremos el cociente q y el residuo r en (3.3.3) para diversos valores de a y b:

$$\begin{array}{llllll} a = 22, & b = 7, & q = 3, & r = 1; & 22 = 7 \cdot 3 + 1 & \\ a = 24, & b = 8, & q = 3, & r = 0; & 24 = 8 \cdot 3 + 0 & (3.3.4) \\ a = 103, & b = 21, & q = 4, & r = 19; & 103 = 21 \cdot 4 + 19 & \\ a = 4895, & b = 87, & q = 56, & r = 23; & 4895 = 87 \cdot 56 + 23 & \\ a = 0, & b = 47, & q = 0, & r = 0; & 0 = 47 \cdot 0 + 0. & (3.3.5) \end{array}$$

□

En (3.3.4) y (3.3.5), el residuo r es cero y $b \mid a$. En los demás casos, $b \nmid a$.

Ahora supongamos que a es un entero no negativo y que b es un entero positivo. Podemos dividir a entre b para obtener

$$a = bq + r, \quad 0 \le r < b.$$

Mostraremos que el conjunto de divisores comunes de a y b es igual al conjunto de divisores comunes de b y r.

Sea c un divisor común de a y b. Por el teorema 3.3.4c, $c \mid bq$. Como $c \mid a$ y $c \mid bq$, por el teorema 3.3.4b, $c \mid a - bq\ (= r)$. Así, c es un divisor común de b y r. Recíprocamente, si c es un divisor común de b y r, entonces $c \mid bq$ y $c \mid bq + r\ (= a)$ y c es un divisor común de a y b. Así, el conjunto de divisores comunes de a y b es igual al conjunto de divisores comunes de b y r. Esto implica que

$$\text{mcd}(a, b) = \text{mcd}(b, r).$$

Resumimos este resultado como un teorema.

TEOREMA 3.3.6

Si a es un entero no negativo, b es un entero positivo y

$$a = bq + r, \qquad 0 \le r < b,$$

entonces

$$\text{mcd}(a, b) = \text{mcd}(b, r).$$

Demostración. La demostración aparece antes del enunciado del teorema. ■

EJEMPLO 3.3.7

Si dividimos 105 entre 30, obtenemos

$$105 = 30 \cdot 3 + 15.$$

El residuo es 15. Por el teorema 3.3.6,

$$\text{mcd}(105, 30) = \text{mcd}(30, 15).$$

Si dividimos 30 entre 15, obtenemos

$$30 = 15 \cdot 2 + 0.$$

El residuo es 0. Por el teorema 3.3.6,

$$\text{mcd}(30, 15) = \text{mcd}(15, 0).$$

Por inspección, mcd(15, 0) = 15. Por tanto,

$$\text{mcd}(105, 30) = \text{mcd}(30, 15) = \text{mcd}(15, 0) = 15.$$ □

En el ejemplo 3.3.3 obtuvimos el máximo común divisor de 105 y 30 enumerando todos los divisores de 105 y 30. Al utilizar el teorema 3.3.6, dos divisiones sencillas nos proporcionan el máximo común divisor. Este cálculo ilustra el algoritmo de Euclides.

En general, el algoritmo de Euclides determina el máximo común divisor de a y b utilizando varias veces el teorema 3.3.6 para reemplazar el problema original, determinar el máximo común divisor de a y b, por el problema de determinar el máximo común divisor de números más pequeños. En última instancia, reducimos el problema original al problema de determinar el máximo común divisor de dos números, uno de los cuales es 0. Como mcd(m, 0) = m, hemos resuelto el problema original. Ahora precisaremos esta técnica.

Sean r_0 y r_1 enteros no negativos, con r_1 distinto de cero. Si dividimos r_0 entre r_1, obtenemos

$$r_0 = r_1 q_2 + r_2, \qquad 0 \le r_2 < r_1.$$

Por el teorema 3.3.6,

$$\text{mcd}(r_0, r_1) = \text{mcd}(r_1, r_2).$$

Si $r_2 \neq 0$, podemos dividir r_1 entre r_2 para obtener

$$r_1 = r_2 q_3 + r_3, \qquad 0 \le r_3 < r_2.$$

Por el teorema 3.3.6,

$$\text{mcd}(r_1, r_2) = \text{mcd}(r_2, r_3).$$

Continuamos dividiendo r_i entre r_{i+1}, siempre que $r_{i+1} \neq 0$. Como $r_1, r_2, \ldots$ son enteros no negativos y

$$r_1 > r_2 > r_3 > \cdots,$$

en algún momento r_i será cero. Sea r_n el primer residuo igual a cero. Entonces

$$\begin{aligned}\text{mcd}(r_0, r_1) &= \text{mcd}(r_1, r_2) = \text{mcd}(r_2, r_3) = \cdots \\ &= \text{mcd}(r_{n-1}, r_n) = \text{mcd}(r_{n-1}, 0).\end{aligned}$$

El máximo común divisor de r_{n-1} y 0 es r_{n-1}; por tanto,

$$\text{mcd}(r_0, r_1) = \text{mcd}(r_{n-1}, 0) = r_{n-1}.$$

Así, el máximo común divisor de r_0 y r_1 será el último residuo distinto de cero.

Enunciamos el algoritmo de Euclides como el algoritmo 3.3.8.

ALGORITMO 3.3.8 — ***Algoritmo de Euclides***

Este algoritmo determina el máximo común divisor de los enteros no negativos a y b, no ambos nulos.

Entrada: a y b (enteros no negativos, no ambos cero)

Salida: Máximo común divisor de a y b

```
1.  procedure mcd(a, b)
2.    // hacemos que a sea el mayor
3.    if a < b then-
4.       intercambiar(a, b)
         // es decir, hacemos
         // temp := a
         // a := b
         // b := temp
5.    while b ≠ 0 do
6.       begin
7.       dividir a entre b para obtener a = bq + r, 0 ≤ r < b
8.       a := b
9.       b := r
10.      end
11.   return(a)
12. end mcd
```

EJEMPLO 3.3.9

Ahora mostremos la forma en que el algoritmo 3.3.8 determina mcd(504, 396).

Sean $a = 504$ y $b = 396$. Como $a > b$, pasamos a la línea 5. Como $b \neq 0$, pasamos a la línea 7, donde dividimos a (504) entre b (396) para obtener

$$504 = 396 \cdot 1 + 108.$$

Ahora pasamos a la línea 8. Hacemos a igual a 396 y b igual a 108 y regresamos a la línea 5. Como $b \neq 0$, pasamos a la línea 7, donde dividimos a (396) entre b (108) para obtener

$$396 = 108 \cdot 3 + 72.$$

Ahora pasamos a la línea 8. Hacemos a igual a 108 y b igual a 72 y regresamos a la línea 5. Como $b \neq 0$, pasamos a la línea 7, donde dividimos a (108) entre b (72) para obtener

$$108 = 72 \cdot 1 + 36.$$

Ahora pasamos a la línea 8. Hacemos a igual a 72 y b igual a 36 y regresamos a la línea 5. Como $b \neq 0$, pasamos a la línea 7, donde dividimos a (72) entre b (36) para obtener

$$72 = 36 \cdot 2 + 0.$$

Ahora pasamos a la línea 8. Hacemos a igual a 36 y b igual a 0 y regresamos a la línea 5. Ahora, $b = 0$, por lo que pasamos a la línea 11, donde regresamos a (36) como el máximo común divisor de 396 y 504. □

Ejercicios

En los ejercicios 1-6, determine enteros q y r tales que $a = bq + r$, con $0 \leq r < b$.

1. $a = 45$, $b = 6$
2. $a = 106$, $b = 12$
3. $a = 66$, $b = 11$
4. $a = 221$, $b = 17$
5. $a = 0$, $b = 31$
6. $a = 0$, $b = 47$

Utilice el algoritmo de Euclides para determinar el máximo común divisor de cada par de enteros en los ejercicios 7-16.

7. 60, 90
8. 110, 273
9. 220, 1400
10. 315, 825
11. 20, 40
12. 331, 993
13. 2091, 4807
14. 2475; 32,670
15. 67,942; 4209
16. 490,256; 337

17. Sean m, n y c enteros. Muestre que si c es un divisor común de m y n, entonces $c \mid (m - n)$.
18. Sean m, n y c enteros. Muestre que si $c \mid m$, entonces $c \mid mn$.
19. Suponga que a, b y c son enteros positivos. Muestre que si $a \mid b$ y $b \mid c$, entonces $a \mid c$.
20. Si a y b son enteros positivos, muestre que $\text{mcd}(a, b) = \text{mcd}(a, a + b)$.

21. Utilice la notación en el texto posterior al ejemplo 3.3.7 y muestre que podemos escribir de manera sucesiva

$$r_{n-1} = s_{n-3}r_{n-2} + t_{n-3}r_{n-3}$$
$$r_{n-1} = s_{n-4}r_{n-3} + t_{n-4}r_{n-4}$$
$$\vdots$$
$$r_{n-1} = s_0 r_1 + t_0 r_0$$
$$\text{mcd}(r_0, r_1) = s_0 r_1 + t_0 r_0,$$

donde los s y t son enteros.

22. Utilice el método del ejercicio 21 para escribir el máximo común divisor de cada par de enteros a y b en los ejercicios 7 a 16 en la forma $ta + sb$.
23. Muestre que si p es un número primo, a y b son enteros positivos y $p \mid ab$, entonces $p \mid a$ o $p \mid b$.
24. Proporcione un ejemplo de enteros positivos a, b y c tales que $a \mid bc$, $a \nmid b$ y $a \nmid c$.
25. Muestre que si $a > b \geq 0$, entonces

$$\text{mcd}(a, b) = \text{mcd}(a - b, b).$$

26. Utilice el ejercicio 25 y escriba un algoritmo para calcular el máximo común divisor de dos enteros no negativos a y b, no ambos cero, que utilice la resta y no la división.

3.4 *ALGORITMOS RECURSIVOS*

Un **procedimiento recursivo** es aquel que se llama a sí mismo. Un **algoritmo recursivo** es un algoritmo que contiene un procedimiento recursivo. La recursión es una forma poderosa, elegante y natural de resolver una amplia clase de problemas. Un problema de esta clase puede resolverse mediante una técnica *divide y vencerás* en la cual el problema se descompone en problemas del mismo tipo del problema original. Cada subproblema, a su vez, se descompone de nuevo hasta que el proceso produce subproblemas que se puedan resolver de manera directa. Por último, las soluciones de los subproblemas se combinan para obtener una solución del problema original.

EJEMPLO 3.4.1

El **factorial de n**, o **n factorial**, se define como

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n(n-1)(n-2)\cdots 2 \cdot 1 & \text{si } n \geq 1. \end{cases}$$

Es decir, si $n \geq 1$, $n!$ es igual al producto de todos los enteros entre 1 y n, inclusive. 0! se define como 1. Por ejemplo,

$$3! = 3 \cdot 2 \cdot 1 = 6, \qquad 6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720.$$

Observe que n factorial puede escribirse "en términos de sí mismo" pues si "quitamos" n, el producto restante es tan sólo $(n - 1)!$; es decir,

$$n! = n(n-1)(n-2)\cdots 2 \cdot 1 = n \cdot (n-1)!$$

Por ejemplo,

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5 \cdot 4!$$

La ecuación

$$n! = n \cdot (n - 1)!$$

que es cierta incluso cuando $n = 1$, muestra la forma de descomponer el problema original (calcular $n!$) en subproblemas cada vez más sencillos [calcular $(n - 1)!$, calcular $(n - 2)!$, . . .] hasta que el proceso llegue al problema más simple, el cálculo de 0!. En ese momento, las soluciones de estos subproblemas pueden combinarse (multiplicando) para resolver el problema original.

Por ejemplo, el problema de calcular 5! se reduce a calcular 4!; el problema de calcular 4! se reduce a calcular 3!; y así sucesivamente. La tabla 3.4.1 resume este proceso.

TABLA 3.4.1
Descomposición del problema del factorial

Problema	*Problema simplificado*
5!	5 · 4!
4!	4 · 3!
3!	3 · 2!
2!	2 · 1!
1!	1 · 0!
0!	Ninguno

Una vez que el problema de calcular 5! se ha reducido a resolver subproblemas, la solución del subproblema más sencillo puede utilizarse para resolver el siguiente subproblema más sencillo, y así sucesivamente, hasta resolver el problema original. La tabla 3.4.2 muestra la forma en que los subproblemas se combinan para calcular 5!.

TABLA 3.4.2
Combinación de los subproblemas del problema del factorial

Problema	*Solución*
0!	1
1!	1 · 0! = 1
2!	2 · 1! = 2
3!	3 · 2! = 3 · 2 = 6
4!	4 · 3! = 4 · 6 = 24
5!	5 · 4! = 5 · 24 = 120

□

A continuación escribimos un algoritmo recursivo que calcula factoriales. El algoritmo es una traducción directa de la ecuación

$$n! = n \cdot (n - 1)!.$$

ALGORITMO 3.4.2 *Cálculo de n factorial*

Este algoritmo recursivo calcula $n!$.

Entrada: n, un entero mayor o igual que 0

Salida: $n!$

```
1. procedure factorial(n)
2.     if n = 0 then
3.         return(1)
4.     return(n * factorial(n - 1))
5. end factorial
```

Ahora mostraremos la forma en que el algoritmo 3.4.2 calcula $n!$ para diversos valores de n. Si $n = 0$, el procedimiento regresa, en la línea 3, el valor correcto 1.

Si $n = 1$, como $n \neq 0$, pasamos a la línea 4. Utilizamos este procedimiento para calcular 0!. Ya hemos observado que el procedimiento calcula 1 como el valor de 0!. En la línea 4, el procedimiento calcula en forma correcta el valor 1!

$$(n - 1)! \cdot n = 0! \cdot 1 = 1 \cdot 1 = 1.$$

Si $n = 2$, como $n \neq 0$, pasamos a la línea 4. Utilizamos este procedimiento para calcular 1!. Ya hemos observado que el procedimiento calcula 1 como el valor de 1!. En la línea 4, el procedimiento calcula en forma correcta el valor 2!

$$(n - 1)! \cdot n = 1! \cdot 2 = 1 \cdot 2 = 2.$$

Si $n = 3$, como $n \neq 0$, pasamos a la línea 4. Utilizamos este procedimiento para calcular 2!. Ya hemos observado que el procedimiento calcula 2 como el valor de 2!. En la línea 4, el procedimiento calcula en forma correcta el valor 3!

$$(n - 1)! \cdot n = 2! \cdot 3 = 2 \cdot 3 = 6.$$

Los argumentos anteriores pueden generalizarse mediante inducción matemática para *demostrar* que el algoritmo 3.4.2 tiene como salida correcta el valor de $n!$ para cualquier entero no negativo n.

TEOREMA 3.4.3

El algoritmo 3.4.2 produce como salida el valor de $n!$, $n \geq 0$.

Demostración.

PASO BASE ($n = 0$). Ya hemos observado que si $n = 0$, el algoritmo produce correctamente como salida el valor 1 de 0!.

PASO INDUCTIVO. Supongamos que el algoritmo 3.4.2 produce correctamente como salida el valor de $(n - 1)!$, $n > 0$. Ahora supongamos que n es la entrada del algoritmo 3.4.2. Como $n \neq 0$, al ejecutar el procedimiento en el algoritmo 3.4.2 pasamos a la línea 4. Por la hipótesis de inducción, el procedimiento calcula correctamente el valor de $(n - 1)!$. En la línea 4, el procedimiento calcula correctamente el valor $(n - 1)! \cdot n = n!$.

Por tanto, el algoritmo 3.4.2 produce correctamente como salida el valor de $n!$ para cada entero $n \geq 0$. ■

Deben existir ciertas situaciones en las que un procedimiento recursivo no se llame a sí mismo; en caso contrario, se llamaría a sí mismo por siempre. En el algoritmo 3.4.2, si $n = 0$, el procedimiento no se llama a sí mismo. Los valores para los cuales un procedimiento recursivo no se llama a sí mismo son los *casos base*. Para resumir, cada procedimiento recursivo debe tener casos base.

Hemos mostrado que la inducción matemática puede utilizarse para demostrar que un algoritmo recursivo calcula el valor que afirmaba calcular. El vínculo entre la inducción matemática y los algoritmos recursivos es mucho más profundo. Con frecuencia, una demostración por inducción matemática puede considerarse como un algoritmo para calcular un valor o realizar una construcción particular. El paso base de una demostración por inducción matemática corresponde a los casos base de un procedimiento recursivo y el paso inductivo de una demostración por inducción matemática corresponde a la parte de un procedimiento recursivo donde éste se llama a sí mismo.

En el ejemplo 1.6.4 dimos una demostración por inducción matemática de que dado un tablero deficiente $n \times n$ (un tablero sin un cuadrado) donde n es una potencia de 2, podemos formar un mosaico sobre el tablero con triominós rectos (tres cuadrados que forman una "L"; véase la figura 1.6.3). Ahora podemos traducir la demostración inductiva en un algoritmo recursivo para construir un mosaico con triominós rectos sobre un tablero deficiente $n \times n$, donde n es una potencia de 2.

ALGORITMO 3.4.4 ***Mosaico sobre un tablero deficiente con triominós***

Este algoritmo cubre con triominós rectos un tablero deficiente $n \times n$, donde n es una potencia de 2.

Entrada: n, una potencia de 2 (el tamaño del tablero) y la posición L del cuadrado faltante

Salida: La cubierta por medio de triominós de un tablero deficiente $n \times n$

procedure *tile*(n, L)
1. **if** $n = 2$ **then**
   // el tablero es un triominó recto T
2. **return**(T)
3. dividir el tablero en cuatro tableros $(n/2) \times (n/2)$
4. girar el tablero de modo que el cuadrado faltante esté en el cuadrante superior izquierdo
5. colocar un triominó recto en el centro // como en la figura 1.6.5
   // considerar cada uno de los cuadrados cubiertos por el triominó central como faltante y denotar los cuadrados faltantes como m_1, m_2, m_3, m_4
6. **call** *tile*$(n/2, m_1)$
7. **call** *tile*$(n/2, m_2)$
8. **call** *tile*$(n/2, m_3)$
9. **call** *tile*$(n/2, m_4)$

end *tile*

A continuación daremos un algoritmo recursivo para el cálculo del máximo común divisor de dos enteros no negativos, no ambos cero.

El teorema 3.3.6 establece que si a es un entero no negativo, b es un entero positivo, y

$$a = bq + r, \qquad 0 \le r < b,$$

entonces

$$\text{mcd}(a, b) = \text{mcd}(b, r). \tag{3.4.1}$$

[mcd(x, y) denota el máximo común divisor de x y y.] De manera inherente la ecuación (3.4.1) es recursiva; reduce el problema de calcular el máximo común divisor de a y b a un problema menor, el de calcular el máximo común divisor de b y r. El algoritmo recursivo 3.4.5, que calcula el máximo común divisor, se basa en la ecuación (3.4.1).

BIBLIOTECA
FACULTAD
Cs. EXACTAS, INGENIERIA
Y AGRIMENSURA
ROSARIO

ALGORITMO 3.4.5 ***Cálculo recursivo del máximo común divisor***

Este algoritmo determina de manera recursiva el máximo común divisor de los enteros no negativos a y b, no ambos cero. (El algoritmo 3.3.8 proporciona un algoritmo no recursivo para calcular el máximo común divisor.)

Entrada: a y b (enteros no negativos, no ambos cero)

Salida: Máximo común divisor de a y b

procedure *mcd_recurs*(a, b)
   // se hace que a sea el más grande
1. **if** $a < b$ **then**
2.   intercambiar(a, b)
3. **if** $b = 0$ **then**
4.   **return**(a)
5. dividir a entre b para obtener $a = bq + r, 0 \le r < b$
6. **return**(*mcd_recurs*(b, r))

end *mcd_recurs*

Ahora presentamos un último ejemplo de algoritmo recursivo.

EJEMPLO 3.4.6

Un robot puede dar pasos de 1 o 2 metros. Escribiremos un algoritmo para el cálculo del número de formas en que el robot puede recorrer n metros. Por ejemplo:

Distancia	*Serie de pasos*	*Número de formas de recorrerlos*
1	1	1
2	1, 1 o 2	2
3	1, 1, 1 o 1, 2 o 2, 1	3
4	1, 1, 1, 1 o 1, 1, 2 o 1, 2, 1 o 2, 1, 1 o 2, 2	5

Sea walk(n) el número de formas en que el robot puede recorrer n metros. Hemos observado que

$$\text{walk}(1) = 1, \qquad \text{walk}(2) = 2.$$

Ahora supongamos que $n > 2$. El robot puede comenzar dando un paso de 1 metro o un paso de 2 metros. Si el robot comienza dando un paso de 1 metro, resta una distancia de $n - 1$ metros; pero, por definición, el resto de la caminata puede completarse de walk$(n - 1)$ formas. De manera análoga, si el robot comienza dando un paso de 2 metros, resta una distancia de $n - 2$ metros y, en este caso, el resto de la caminata puede concluirse de walk$(n - 2)$ formas. Como la caminata debe comenzar con un paso de 1 o de 2 metros, hemos abarcado todas las formas de recorrer n metros. Obtenemos la fórmula

$$\text{walk}(n) = \text{walk}(n - 1) + \text{walk}(n - 2).$$

Por ejemplo,

$$\text{walk}(4) = \text{walk}(3) + \text{walk}(2) = 3 + 2 = 5.$$

Podemos escribir un algoritmo recursivo para calcular walk(n) traduciendo la ecuación

$$\text{walk}(n) = \text{walk}(n - 1) + \text{walk}(n - 2)$$

directamente a un algoritmo. Los casos base son $n = 1$ y $n = 2$. □

ALGORITMO 3.4.7 ***Caminata de un robot***

Este algoritmo calcula la función definida como

$$\text{walk}(n) = \begin{cases} 1, & n = 1 \\ 2, & n = 2 \\ \text{walk}(n-1) + \text{walk}(n-2), & n > 2 \end{cases}$$

Entrada: n

Salida: walk(n)

```
procedure robot_walk(n)
  if n = 1 or n = 2 then
    return(n)
  return(robot_walk(n − 1) + robot_walk(n − 2))
end robot_walk
```

La sucesión

$$\text{walk}(1),\quad \text{walk}(2),\quad \text{walk}(3),\quad \ldots,$$

cuyos valores comienzan con

$$1,\quad 2,\quad 3,\quad 5,\quad 8,\quad 13,\quad \ldots,$$

es llamada **sucesión de Fibonacci*** en honor de Leonardo Fibonacci (hacia 1170-1250), comerciante y matemático italiano. En lo sucesivo, denotaremos la sucesión de Fibonacci como

$$f_1,\quad f_2,\quad \ldots.$$

* Los dos primeros elementos de la sucesión de Fibonacci son 1 y 1, y no1 y 2.

Esta sucesión queda definida mediante las ecuaciones

$$f_1 = 1$$
$$f_2 = 2$$
$$f_n = f_{n-1} + f_{n-2}, \qquad n \geq 3.$$

La sucesión de Fibonacci surgió originalmente de un acertijo relacionado con conejos (véanse los ejercicios 13 y 14). Después de regresar del Oriente en 1202, Fibonacci escribió su obra más famosa, *Liber Abaci*, que además de contener lo que ahora llamamos la sucesión de Fibonacci abogaba en favor del uso de los numerales indoarábigos. Este libro fue una de las principales influencias para llevar el sistema numérico decimal a Europa Occidental. Fibonacci firmó gran parte de su trabajo como "Leonardo Bigollo". *"Bigollo"* se traduce como "viajero" o "necio". Existe cierta evidencia de que Fibonacci disfrutaba que sus contemporáneos lo considerasen un necio por apoyar el nuevo sistema numérico.

La sucesión de Fibonacci surge en los lugares menos esperados. Por ejemplo, el número de espirales realizadas en el sentido de las manecillas del reloj y el número de las espirales realizadas en sentido contrario al de las manecillas del reloj formadas por las semillas de ciertas variedades de girasoles aparecen en la sucesión de Fibonacci. La figura 3.4.1 muestra un girasol hipotético, con 13 espirales en el sentido de las manecillas y 8 espirales en sentido contrario. En la sección 3.6, la sucesión de Fibonacci aparece en el análisis del algoritmo de Euclides.

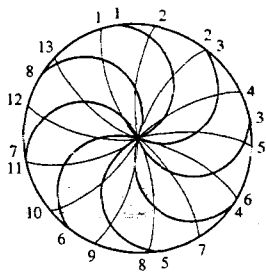


FIGURA 3.4.1
Un girasol hipotético.

Ejercicios

1. Rastree el algoritmo 3.4.2 para $n = 4$.
2. Rastree el algoritmo 3.4.4 cuando $n = 4$ y el cuadrado faltante sea el de la esquina superior izquierda.
3. Rastree el algoritmo 3.4.4 cuando $n = 8$ y el cuadrado faltante esté a cuatro cuadrados de la izquierda y a seis de la parte superior.
4. Rastree el algoritmo 3.4.5 para $a = 5$ y $b = 0$.
5. Rastree el algoritmo 3.4.5 para $a = 55$ y $b = 20$.
6. (a) Utilice las fórmulas

$$s_1 = 1, \qquad s_n = s_{n-1} + n, \qquad n \geq 2.$$

para escribir un algoritmo recursivo que calcule

$$s_n = 1 + 2 + 3 + \cdots + n.$$

(b) Proporcione una demostración por inducción matemática de que su algoritmo de la parte (a) es correcto.

7. (a) Utilice las fórmulas

$$s_1 = 2, \qquad s_n = s_{n-1} + 2n, \qquad n \geq 2,$$

para escribir un algoritmo recursivo que calcule

$$s_n = 2 + 4 + 6 + \cdots + 2n.$$

(b) Haga una demostración por inducción matemática de que su algoritmo de la parte (a) es correcto.

8. (a) Un robot puede dar pasos de 1, 2 o 3 metros. Escriba un algoritmo recursivo para calcular el número de formas en que el robot puede caminar n metros.

(b) Proporcione una demostración por inducción matemática de que su algoritmo de la parte (a) es correcto.

9. Escriba un algoritmo recursivo que calcule el máximo común divisor de dos enteros no negativos, no ambos nulos, que utilice restas en vez de divisiones (véase el ejercicio 25 de la sección 3.3).

10. Escriba un algoritmo no recursivo para calcular $n!$.

☆ **11.** Un robot puede dar pasos de 1 o 2 metros. Escriba un algoritmo para enumerar todas las formas en que el robot puede recorrer n metros.

☆ **12.** Un robot puede dar pasos de 1, 2 o 3 metros. Escriba un algoritmo para enumerar todas las formas en que el robot puede recorrer n metros.

Los ejercicios 13-25 se refieren a la sucesión de Fibonacci $\{f_n\}$.

13. Supongamos que al inicio del año, existe una pareja de conejos y que cada mes cada pareja produce una nueva pareja que puede reproducirse después de un mes. Supongamos además que no ocurre muerte alguna. Sea a_n el número de parejas de conejos al final del n-ésimo mes. Muestre que $a_1 = 1$, $a_2 = 2$ y $a_n - a_{n-1} = a_{n-2}$. Explique por qué $a_n = f_n$, $n \geq 1$.

14. La pregunta original de Fibonacci fue: Bajo las condiciones del ejercicio 13, ¿cuántas parejas de conejos existen después de un año? Responda la pregunta de Fibonacci.

15. Utilice inducción matemática para mostrar que

$$\sum_{k=1}^{n} f_k = f_{n+2} - 2, \qquad n \geq 1.$$

16. Utilice inducción matemática para mostrar que

$$f_n^2 = f_{n-1} f_{n+1} + (-1)^n, \qquad n \geq 2.$$

17. Muestre que

$$f_{n+2}^2 - f_{n+1}^2 = f_n f_{n+3}, \qquad n \geq 1.$$

18. Utilice inducción matemática para mostrar que

$$\sum_{k=1}^{n} f_k^2 = f_n f_{n+1} - 1, \qquad n \geq 1.$$

19. Utilice inducción matemática para mostrar que f_n es par si y sólo si $n + 1$ es divisible entre 3, $n \geq 1$.

20. Utilice inducción matemática para mostrar que para $n \geq 5$,

$$f_n > \left(\frac{3}{2}\right)^n.$$

21. Utilice inducción matemática para mostrar que para $n \geq 1$,

$$f_n < 2^n.$$

22. Utilice inducción matemática para mostrar que para $n \geq 1$,

$$\sum_{k=1}^{n} f_{2k-1} = f_{2n} - 1, \qquad \sum_{k=1}^{n} f_{2k} = f_{2n+1} - 1.$$

23. Utilice inducción matemática para mostrar que cada entero $n \geq 1$ puede expresarse como la suma de números de Fibonacci distintos y no consecutivos.

24. Muestre que la representación del ejercicio 23 es única.

25. Muestre que para $n \geq 2$,

$$f_n = \frac{f_{n-1} + \sqrt{5 f_{n-1}^2 + 4(-1)^n}}{2}.$$

Observe que esta fórmula proporciona el valor de f_n en términos de un predecesor en vez de dos predecesores, como en la definición original.

26. [Este ejercicio requiere conocimientos de cálculo.] Suponga válida la fórmula para derivar productos:

$$\frac{d(fg)}{dx} = f\frac{dg}{dx} + g\frac{df}{dx}.$$

Utilice inducción matemática para demostrar que

$$\frac{dx^n}{dx} = nx^{n-1} \qquad \text{para } n = 1, 2, \ldots.$$

27. [Este ejercicio requiere conocimientos de cálculo.] Explique por qué la siguiente fórmula proporciona un algoritmo recursivo para integrar $\log^n |x|$:

$$\int \log^n |x|\, dx = x \log^n |x| - n \int \log^{n-1} |x|\, dx.$$

Proporcione otros ejemplos de fórmulas recursivas de integración.

3.5 COMPLEJIDAD DE LOS ALGORITMOS

Un programa de computadora, aunque sea consecuencia de un algoritmo correcto, podría no servir para ciertos tipos de entrada, pues el tiempo necesario para ejecutar el programa o el espacio necesario para almacenar los datos, las variables del programa, etc., pueden ser demasiado grandes. El **análisis de un algoritmo** se refiere al proceso de estimación del tiempo y espacio necesarios para ejecutar el algoritmo. La **complejidad de un algoritmo** se refiere a la cantidad de tiempo y espacio necesarios para ejecutar el algoritmo. En esta sección estudiaremos el problema de estimación del tiempo necesario para ejecutar un algoritmo.

Supongamos dado un conjunto X con n elementos, algunos con la etiqueta "rojo" y otros con la etiqueta "negro" y que queremos determinar el número de subconjuntos de X que contienen al menos un elemento rojo. Supongamos que se desea construir un algoritmo que examine a todos los subconjuntos de X y que cuente aquellos que contengan al menos un elemento rojo, para después poner en práctica este algoritmo como un programa de computadora. Como un conjunto con n elementos tiene 2^n elementos (teorema 2.1.4), el programa necesitaría al menos 2^n unidades de tiempo para su ejecución. Sin importar cuáles sean las unidades de tiempo, 2^n crece tan rápido, conforme n crece (véase la tabla 3.5.1) que, excepto para valores pequeños de n, no sería factible ejecutar el programa.

La determinación de los parámetros de rendimiento de un programa de computadora es una tarea difícil y depende de varios factores, como la computadora utilizada, la forma de representar los datos, y la forma en que el programa se traduce en instrucciones de máquina. Aunque la estimación precisa del tiempo de ejecución de un programa debe tomar en cuenta estos factores, puede obtenerse información útil analizando la complejidad del tiempo del algoritmo subyacente.

El tiempo necesario para ejecutar un algoritmo es una función que depende de la entrada. Por lo general, es difícil obtener una fórmula explícita de esta función y utilizaremos menos que esto. En vez de trabajar directamente con la entrada, utilizamos parámetros que caracterizan el tamaño de la entrada. Buscamos el tiempo mínimo necesario para ejecutar el algoritmo con todas las entradas de tamaño n. Este tiempo es el **tiempo en el mejor de los casos** para entradas de tamaño n. También podemos buscar el tiempo máximo necesario para ejecutar el algoritmo con todas las entradas de tamaño n. Este tiempo es el **tiempo en el peor de los casos** para entradas de tamaño n. Otro caso importante es el **tiempo en el caso promedio**, el tiempo promedio necesario para ejecutar el algoritmo sobre un conjunto finito de entradas, todas de tamaño n.

Podríamos medir el tiempo necesario para ejecutar un algoritmo contando el número de instrucciones ejecutadas. Otra alternativa consiste en utilizar una estimación menos precisa del tiempo, como el número de veces que se ejecuta cada ciclo. Si la actividad principal de un algoritmo consiste en realizar comparaciones, como podría ocurrir en una rutina de ordenamiento, podríamos contar el número de comparaciones. Lo usual es que nos interesen las estimaciones generales, pues como ya hemos observado, el desempeño real de la implantación de un algoritmo en un programa depende de muchos factores.

EJEMPLO 3.5.1

Una definición razonable del tamaño de entrada para el algoritmo 3.2.2 que determina el valor máximo en una sucesión finita es el número de elementos en la sucesión de entrada. Una definición razonable del tiempo de ejecución es el número de iteraciones del ciclo while. Con estas definiciones, los tiempos en el peor de los casos, en el mejor de los casos y en el caso promedio para el algoritmo 3.2.2 para una entrada de tamaño n son $n - 1$ cada uno, pues el ciclo siempre se ejecuta $n - 1$ veces. □

TABLA 3.5.1
Tiempo necesario para ejecutar un algoritmo, si cada paso se realiza en un microsegundo

Número de pasos hasta concluir el algoritmo para una entrada de tamaño n	**Tiempo de ejecución si n =**				
	3	*6*	*9*	*12*	
1	10^{-6} seg	10^{-6} seg	10^{-6} seg	10^{-6} seg	
lg lg n	10^{-6} seg	10^{-6} seg	2×10^{-6} seg	2×10^{-6} seg	
lg n	2×10^{-6} seg	3×10^{-6} seg	3×10^{-6} seg	4×10^{-6} seg	
n	3×10^{-6} seg	6×10^{-6} seg	9×10^{-6} seg	10^{-5} seg	
n lg n	5×10^{-6} seg	2×10^{-5} seg	3×10^{-6} seg	4×10^{-5} seg	
n^2	9×10^{-6} seg	4×10^{-5} seg	8×10^{-6} seg	10^{-4} seg	
n^3	3×10^{-5} seg	2×10^{-4} seg	7×10^{-6} seg	2×10^{-3} seg	
2^n	8×10^{-6} seg	6×10^{-5} seg	5×10^{-6} seg	4×10^{-3} seg	
	50	*100*	*1000*	*10^5*	*10^6*
1	10^{-6} seg	10^{-6} seg	10^{-6} seg	10^{-6} seg	10^{-6} seg
lg lg n	2×10^{-6} seg	3×10^{-6} seg	3×10^{-6} seg	4×10^{-6} seg	4×10^{-6} seg
lg n	6×10^{-6} seg	7×10^{-6} seg	10^{-5} seg	2×10^{-5} seg	2×10^{-5} seg
n	5×10^{-5} seg	10^{-4} seg	10^{-3} seg	0.1 seg	1 seg
n lg n	3×10^{-4} seg	7×10^{-4} seg	10^{-2} seg	2 seg	20 seg
n^2	3×10^{-3} seg	0.01 seg	1 seg	3 h	12 días
n^3	0.13 seg	1 seg	16.7 min	32 años	31,710 años
2^n	36 años	4×10^{16} años	3×10^{287} años	3×10^{30089} años	3×10^{301016} años

Con frecuencia estamos menos interesados en los tiempos exactos en el peor o en el mejor de los casos para un algoritmo, que en la forma de incremento del tiempo en ambos casos, cuando el tamaño de la entrada se incrementa. Por ejemplo, supongamos que el tiempo de un algoritmo en el peor de los casos es

$$t(n) = 60n^2 + 5n + 1 \qquad (3.5.1)$$

para una entrada de tamaño n. Para n grande, el término $60n^2$ es aproximadamente igual a $t(n)$ (véase la tabla 3.5.2). En este sentido, $t(n)$ crece como $60n^2$.

TABLA 3.5.2
Comparando el crecimiento de $t(n)$ con $60n^2$

n	$t(n) = 60n^2 + 5n + 1$	$60n^2$
10	6,051	6,000
100	600,501	600,000
1,000	60,005,001	60,000,000
10,000	6,000,050,001	6,000,000,000

Si (3.5.1) mide el tiempo, en segundos, en el peor de los casos para una entrada de tamaño n, entonces

$$T(n) = n^2 + \frac{5}{60}n + \frac{1}{60}$$

mide el tiempo, en minutos, en el peor de los casos para una entrada de tamaño n. Este cambio de unidades no afecta la forma en que el tiempo en el peor de los casos crece cuando el tamaño de la entrada se incrementa, sino sólo las unidades con las cuales se mide el tiempo en el peor de los casos para una entrada de tamaño n. Así, al describir la forma en que el tiempo en el mejor de los casos o en el peor de los casos crece cuando el tamaño de la entrada crece, no sólo buscamos el término dominante [por ejemplo, $60n^2$ en (3.5.1)] sino que podemos ignorar los coeficientes constantes. Bajo estas hipótesis, $t(n)$ crece como n^2 cuando n crece. Decimos que $t(n)$ es de **orden** n^2 y escribimos

$$t(n) = \Theta(n^2),$$

lo cual se lee "$t(n)$ es theta de n^2". La idea básica consiste en reemplazar una expresión como $t(n) = 60n^2 + 5n + 1$ con una expresión más sencilla, como n^2, que crece con la misma razón que $t(n)$. A continuación proporcionamos la definición formal.

DEFINICIÓN 3.5.2

Sean f y g funciones con dominio $\{1, 2, 3, \ldots\}$.

Escribimos

$$f(n) = O(g(n))$$

y decimos que $f(n)$ *es de orden a lo más* $g(n)$ si existe una constante positiva C_1 tal que

$$|f(n)| \leq C_1 |g(n)|$$

para todos los enteros positivos n, excepto para un número finito.

Escribimos

$$f(n) = \Omega(g(n))$$

y decimos que $f(n)$ es de *orden por lo menos* $g(n)$ si existe una constante positiva C_2 tal que

$$|f(n)| \geq C_2 |g(n)|$$

para todos los enteros positivos n, excepto para un número finito.

Escribimos

$$f(n) = \Theta(g(n))$$

y decimos que $f(n)$ es de *orden* $g(n)$ si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

La definición 3.5.2 puede expresarse de la manera siguiente. $f(n) = O(g(n))$ si, excepto por constantes y un número finito de excepciones, f está acotada superiormente por g. $f(n) = \Omega(g(n))$ si, excepto por constantes y un número finito de excepciones, f está acotada inferiormente por g. $f(n) = \Theta(g(n))$ si, excepto por constantes y un número finito de excepciones, f está acotada inferior y superiormente por g.

Una expresión de la forma $f(n) = O(g(n))$ se conoce como una **notación o mayúscula** para f. De manera análoga, $f(n) = \Omega(g(n))$ se conoce como una **notación omega** para f y $f(n) = \Theta(g(n))$ se conoce como una **notación theta** para f.

De acuerdo con la definición, si $f(n) = O(g(n))$, lo único que uno puede concluir es que excepto por constantes y un número finito de excepciones, f está acotada *superiormente* por g, de modo que g crece al menos tan rápido como f. Por ejemplo, si $f(n) = n$ y $g(n) = 2^n$, entonces $f(n) = O(g(n))$, aunque g crece mucho más rápido que f. La afirmación $f(n) = O(g(n))$ no dice nada acerca de una cota *inferior* para f. Por otro lado, si $f(n) = \Theta(g(n))$, uno puede concluir que, excepto por constantes y un número finito de excepciones, f está acotada *superior* e *inferiormente* por g, de modo que f y g crecen con la misma razón. Observe que $n = O(2^n)$, pero $n \neq \Theta(2^n)$. Por desgracia, no es difícil encontrar bibliografía donde la notación o mayúscula se utiliza como si fuese notación theta.

EJEMPLO 3.5.3

Como

$$60n^2 + 5n + 1 \leq 60n^2 + 5n^2 + n^2 = 66n^2 \quad \text{para } n \geq 1,$$

podemos hacer $C_1 = 66$ en la definición 3.5.2 para obtener

$$60n^2 + 5n + 1 = O(n^2).$$

Como

$$60n^2 + 5n + 1 \geq 60n^2 \quad \text{para } n \geq 1,$$

podemos hacer $C_2 = 60$ en la definición 3.5.2 para obtener

$$60n^2 + 5n + 1 = \Omega(n^2).$$

Como $60n^2 + 5n + 1 = O(n^2)$ y $60n^2 + 5n + 1 = \Omega(n^2)$,

$$60n^2 + 5n + 1 = \Theta(n^2).$$

□

Podemos utilizar el método del ejemplo 3.5.3 para mostrar que un polinomio en n, de grado k con coeficientes no negativos es $\Theta(n^k)$. [De hecho, *cualquier* polinomio en n de grado k es $\Theta(n^k)$, aunque algunos de sus coeficientes sean negativos. Para demostrar este resultado más general, hay que modificar el método del ejemplo 3.5.3.]

TEOREMA 3.5.4

Sea

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0$$

un polinomio en n de grado k, donde cada a_i *es no negativo. Entonces*

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 = \Theta(n^k).$$

Demostración. Sea

$$C = a_k + a_{k-1} + \cdots + a_1 + a_0.$$

Entonces

$$\begin{aligned} a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 &\leq a_k n^k + a_{k-1} n^k + \cdots + a_1 n^k + a_0 n^k \\ &= (a_k + a_{k-1} + \cdots + a_1 + a_0) n^k = Cn^k. \end{aligned}$$

Por tanto,

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 = O(n^k).$$

Como

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 \geq a_k n^k,$$

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 = \Omega(n^k).$$

Así,

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 = \Theta(n^k). \quad \blacksquare$$

EJEMPLO 3.5.5

En este libro, lg n denota $\log_2 n$ (el logaritmo de n en base 2). Como lg $n < n$ para $n \geq 1$ (véase la figura 3.5.1),

$$2n + 3 \lg n < 2n + 3n = 5n \qquad \text{para } n \geq 1;$$

así,

$$2n + 3 \lg n = O(n).$$

Además,

$$2n + 3 \lg n \geq 2n \qquad \text{para } n \geq 1,$$

de modo que

$$2n + 3 \lg n = \Omega(n).$$

Por tanto,

$$2n + 3 \lg n = \Theta(n). \quad \square$$

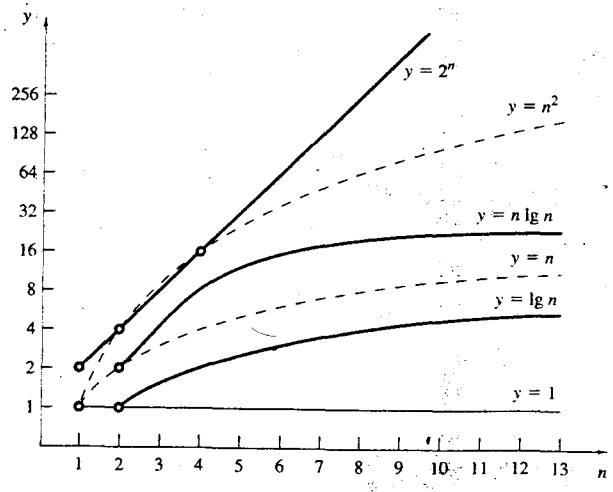


FIGURA 3.5.1 Crecimiento de algunas funciones comunes.

EJEMPLO 3.5.6

Si reemplazamos cada entero 1, 2, . . . , n por n en la suma $1 + 2 + \cdots + n$, la suma no decrece y tenemos que

$$1 + 2 + \cdots + n \leq n + n + \cdots + n = n \cdot n = n^2 \qquad (3.5.2)$$

para $n \geq 1$. Esto implica que

$$1 + 2 + \cdots + n = O(n_2).$$

Para obtener una cota inferior, podemos imitar el argumento anterior y reemplazar cada entero 1, 2, . . . , n por 1 en la suma $1 + 2 + \cdots + n$ para obtener

$$1 + 2 + \cdots + n \geq 1 + 1 + \cdots + 1 = n.$$

En este caso concluimos que

$$1 + 2 + \cdots + n = \Omega(n),$$

y aunque la expresión anterior es cierta, no podemos deducir una estimación Θ para $1 + 2 + \cdots + n$, pues la cota superior n^2 y la cota inferior n no son iguales. Debemos trabajar con más cuidado para obtener una cota inferior.

Una forma de obtener una mejor cota inferior es argumentar como antes, pero eliminando la primera mitad de los términos, para obtener

$$\begin{aligned} 1 + 2 + \cdots + n &\geq \lceil n/2 \rceil + \cdots + (n-1) + n \\ &\geq \lceil n/2 \rceil + \cdots + \lceil n/2 \rceil + \lceil n/2 \rceil \\ &= \lceil (n+1)/2 \rceil \lceil n/2 \rceil \geq (n/2)(n/2) = n^2/4. \end{aligned} \qquad (3.5.3)$$

Ahora podemos concluir que

$$1 + 2 + \cdots + n = \Omega(n^2).$$

Por tanto,

$$1 + 2 + \cdots + n = \Theta(n^2). \quad \square$$

EJEMPLO 3.5.7

Si k es un entero positivo y, como en el ejemplo 3.5.6, reemplazamos cada entero 1, 2, . . . , n por n, tenemos

$$1^k + 2^k + \cdots + n^k \leq n^k + n^k + \cdots + n^k = n \cdot n^k = n^{k+1}$$

para $n \geq 1$; por tanto,

$$1^k + 2^k + \cdots + n^k = O(n^{k+1}).$$

También podemos obtener una cota inferior como en el ejemplo 3.5.6:

$$\begin{aligned}1^k + 2^k + \cdots + n^k &\geq \lceil n/2 \rceil^k + \cdots + (n-1)^k + n^k \\ &\geq \lceil n/2 \rceil^k + \cdots + \lceil n/2 \rceil^k + \lceil n/2 \rceil^k \\ &= \lceil (n+1)/2 \rceil \lceil n/2 \rceil^k \geq (n/2)(n/2)^k = n^{k+1}/2^{k+1}.\end{aligned}$$

Concluimos que

$$1^k + 2^k + \cdots + n^k = \Omega(n^{k+1}),$$

y por tanto

$$1^k + 2^k + \cdots + n^k = \Theta(n^{k+1}).$$ □

Observe la diferencia entre el polinomio

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0$$

del teorema 3.5.4 y la expresión

$$1^k + 2^k + \cdots + n^k$$

del ejemplo 3.5.7. Un polinomio tiene un número fijo de términos, mientras que el número de términos en la expresión del ejemplo 3.5.7 depende del valor de n. Además, el polinomio del teorema 3.5.4 es $\Theta(n^k)$, pero la expresión del ejemplo 3.5.7 es $\Theta(n^{k+1})$.

Nuestro siguiente ejemplo proporciona una notación theta para $\lg n!$.

EJEMPLO 3.5.8

Mostremos que

$$\lg n! = \Theta(n \lg n)$$

utilizando un argumento similar al del ejemplo 3.5.6.

Por las propiedades de los logaritmos, tenemos

$$\lg n! = \lg n + \lg(n-1) + \cdots + \lg 2 + \lg 1.$$

Como lg es una función creciente,

$$\lg n + \lg(n-1) + \cdots + \lg 2 + \lg 1 \leq \lg n + \lg n + \cdots + \lg n + \lg n = n \lg n.$$

Concluimos que

$$\lg n! = O(n \lg n).$$

Ahora,

$$\begin{aligned}&\lg n + \lg(n-1) + \cdots + \lg 2 + \lg 1 \\ &\quad \geq \lg n + \lg(n-1) + \cdots + \lg \lceil n/2 \rceil \\ &\quad \geq \lg \lceil n/2 \rceil + \cdots + \lg \lceil n/2 \rceil \\ &\quad = \lceil (n+1)/2 \rceil \lg \lceil n/2 \rceil \geq (n/2) \lg (n/2).\end{aligned}$$

Una demostración por inducción matemática (véase el ejercicio 46) muestra que si $n \geq 4$,

$$(n/2) \lg(n/2) \geq (n \lg n)/4.$$

Combinamos estas desigualdades para obtener

$$\lg n + \lg(n-1) + \cdots + \lg 2 + \lg 1 \geq (n \lg n)/4$$

para $n \geq 4$. Por tanto,

$$\lg n! = \Omega(n \lg n).$$

Esto implica que

$$\lg n! = \Theta(n \lg n).$$ □

A continuación definimos lo que se entiende por el hecho de que el tiempo en el mejor de los casos, en el peor de los casos y en el caso promedio de un algoritmo sea de orden a lo más $g(n)$.

DEFINICIÓN 3.5.9

Si un algoritmo necesita $t(n)$ unidades de tiempo para terminar en el mejor de los casos para una entrada de tamaño n y

$$t(n) = O(g(n)),$$

decimos que el *tiempo necesario para el algoritmo en el mejor de los casos es de orden a lo más* $g(n)$ o que el *tiempo requerido por el algoritmo en el mejor de los casos* es $O(g(n))$.

Si un algoritmo necesita $t(n)$ unidades de tiempo para terminar en el peor de los casos para una entrada de tamaño n y

$$t(n) = O(g(n)),$$

decimos que el *tiempo necesario para el algoritmo en el peor de los casos es de orden a lo más* $g(n)$ o que el *tiempo requerido por el algoritmo en el peor de los casos* es $O(g(n))$.

Si un algoritmo necesita $t(n)$ unidades de tiempo para terminar en el caso promedio para una entrada de tamaño n y

$$t(n) = O(g(n)),$$

decimos que el *tiempo necesario para el algoritmo en el caso promedio es de orden a lo más* $g(n)$ o que el *tiempo requerido por el algoritmo en el caso promedio* es $O(g(n))$.

Al reemplazar O por Ω y "a lo más" por "por lo menos" en la definición 3.5.9, obtenemos la definición de lo que se entiende por el hecho de que el tiempo necesario para un algoritmo en el mejor de los casos, en el peor de los casos y en el caso promedio sea de orden al menos $g(n)$. Si el tiempo necesario para un algoritmo en el mejor de los casos es $O(g(n))$ y $\Omega(g(n))$, decimos que el tiempo necesario para un algoritmo en el mejor de los casos es $\Theta(g(n))$. Se aplica una definición análoga para el tiempo necesario para un algoritmo en el peor de los casos y en el caso promedio.

EJEMPLO 3.5.10

Supongamos que se sabe que un algoritmo tarda

$$60n^2 + 5n + 1$$

unidades de tiempo para concluir en el peor de los casos para una entrada de tamaño n. En el ejemplo 3.5.3 mostramos que

$$60n^2 + 5n + 1 = \Theta(n^2).$$

Así, el tiempo necesario para el algoritmo en el peor de los casos es $\Theta(n^2)$. □

EJEMPLO 3.5.11

Determine una notación theta en términos de n para el número de veces que se ejecuta la instrucción $x := x + 1$.

```
1. for i := 1 to n do
2.   for j := 1 to i do
3.     x := x + 1
```

En primer lugar, i toma el valor de 1, y cuando j corre de 1 a 1, la línea 3 se ejecuta una vez. A continuación, i es igual a 2, j corre de 1 a 2 y la línea 3 se ejecuta dos veces, etc. Así, el número total de veces que se ejecuta la línea 3 es (véase el ejemplo 3.5.6)

$$1 + 2 + \cdots + n = \Theta(n^2).$$

Así, una notación theta para el número de veces que se ejecuta la instrucción $x := x + 1$ es $\Theta(n^2)$. □

EJEMPLO 3.5.12

Determine una notación theta en términos de n para el número de veces que se ejecuta la instrucción $x := x + 1$.

```
1. j := n
2.   while j ≥ 1 do
3.     begin
4.     for i := 1 to j do
5.       x := x + 1
6.     j := ⌊j/2⌋
7.     end
```

Sea $t(n)$ el número de veces que ejecutamos la instrucción $x := x + 1$. La primera vez que llegamos al cuerpo del ciclo while, la instrucción $x := x + 1$ se ejecuta n veces. Por tanto, $t(n) \geq n$ y $t(n) = \Omega(n)$.

A continuación deducimos una notación o mayúscula para $t(n)$. Después de hacer j igual a n, llegamos al ciclo while por primera vez. La instrucción $x := x + 1$ se ejecuta n veces. En la línea 6, j se reemplaza por $\lfloor n/2 \rfloor$; por tanto, $j \leq n/2$. Si $j \geq 1$, ejecutaremos $x := x + 1$ a lo más $n/2$ veces adicionales en la siguiente iteración del ciclo while, y así sucesivamente. Si k denota el número de veces que ejecutamos el cuerpo del ciclo while, el número de veces que ejecutamos $x := x + 1$ es a lo más

$$n + \frac{n}{2} + \frac{n}{4} + \cdots + \frac{n}{2^{k-1}}.$$

Esta suma geométrica (véase el ejemplo 1.6.2) es igual a

$$\frac{n\left(1 - \frac{1}{2^k}\right)}{1 - \frac{1}{2}}.$$

Ahora,

$$t(n) \leq \frac{n\left(1 - \frac{1}{2^k}\right)}{1 - \frac{1}{2}} = 2n\left(1 - \frac{1}{2^k}\right) \leq 2n,$$

de modo que $t(n) = O(n)$. Así, una notación theta para el número de veces que ejecutamos $x := x + 1$ es $\Theta(n)$. □

EJEMPLO 3.5.13

Determine, en notación theta, los tiempos necesarios para ejecutar el algoritmo 3.5.14 en el mejor de los casos, en el peor de los casos y en el caso promedio. Suponga que el tamaño de la entrada es n y que el tiempo de ejecución del algoritmo es el número de comparaciones realizadas en la línea 3. Además, suponga que las $n + 1$ posibilidades de que *key* esté en cierta posición particular de la sucesión o que no esté en la sucesión son igualmente probables.

Podemos analizar el tiempo en el mejor de los casos como sigue. Si $s_1 = key$, la línea 3 se ejecuta una vez. Así, el tiempo para el algoritmo 3.5.14 en el mejor de los casos es

$$\Theta(1).$$

El tiempo necesario para ejecutar el algoritmo 3.5.14 en el peor de los casos puede analizarse como sigue. Si *key* no está en la sucesión, la línea 3 se ejecutará n veces, de modo que el tiempo del algoritmo 3.5.14 en el peor de los casos es

$$\Theta(n).$$

Por último, consideremos el tiempo del algoritmo 3.5.14 en el caso promedio. Si *key* se encuentra en la i-ésima posición, la línea 3 se ejecuta i veces, y si *key* no está en la sucesión, la línea 3 se ejecuta n veces. Así, el número promedio de veces que la línea 3 se ejecuta es

$$\frac{(1 + 2 + \cdots + n) + n}{n + 1}.$$

Ahora,

$$\frac{(1 + 2 + \cdots + n) + n}{n + 1} \leq \frac{n^2 + n}{n + 1} \quad \text{por (3.5.2)}$$
$$= \frac{n(n+1)}{n+1} = n.$$

Por tanto, el tiempo del algoritmo 3.5.14 en el caso promedio es

$$O(n).$$

Además,

$$\frac{(1 + 2 + \cdots + n) + n}{n + 1} \geq \frac{n^2/4 + n}{n + 1} \quad \text{por (3.5.3)}$$
$$\geq \frac{n^2/4 + n/4}{n + 1} = \frac{n}{4}.$$

Por tanto, el tiempo del algoritmo 3.5.14 en el caso promedio es

$$\Omega(n).$$

Así, el tiempo del algoritmo 3.5.14 en el caso promedio es

$$\Theta(n).$$

Para este algoritmo, los tiempos en el peor de los casos y en el caso promedio son ambos $\Theta(n)$. □

ALGORITMO 3.5.14 *Búsqueda en una sucesión no ordenada*

Dada la sucesión

$$s_1, \quad s_2, \quad \cdots, \quad s_n,$$

y un valor *key*, este algoritmo determina la posición de *key*. Si *key* no se encuentra, el algoritmo produce como salida 0.

Entrada: $s_1, s_2, \ldots, s_n, n$ y *key* (el valor buscado)

Salida: La posición de *key*, o bien, 0 si *key* no se encuentra

```
1.  procedure linear_search(s, n, key)
2.    for i := 1 to n do
3.      if key = s_i then
4.        return(i)   // búsqueda exitosa
5.    return(0)   // búsqueda no exitosa
6.  end linear_search
```

En la sección 3.6 consideraremos un ejemplo más complejo, el tiempo en el peor de los casos para el algoritmo de Euclides (algoritmo 3.3.8).

Las constantes que se suprimen en la notación theta pueden ser importantes. Aunque para cualquier entrada de tamaño n, el algoritmo A requiere exactamente $C_1 n$ unidades de tiempo y el algoritmo B requiere exactamente $C_2 n^2$ unidades de tiempo, para ciertos tamaños de entrada el algoritmo B puede ser superior. Por ejemplo, supongamos que para cualquier tamaño de entrada n, el algoritmo A requiere $300n$ unidades de tiempo y el algoritmo B requiere $5n^2$ unidades de tiempo. Para un tamaño de entrada de $n = 5$, el algoritmo A requiere 1500 unidades de tiempo y el algoritmo B requiere 125 unidades de tiempo, por lo que el algoritmo B es más rápido. Por supuesto, para entradas suficientemente grandes, el algoritmo A es mucho más rápido que el algoritmo B.

Ciertas formas aparecen con tanta frecuencia que tienen nombres especiales, como muestra la tabla 3.5.3. Las formas de la tabla 3.5.3, con la excepción de $\Theta(n^m)$, están ordenadas de modo que si $\Theta(f(n))$ está arriba de $\Theta(g(n))$, entonces $f(n) \leq g(n)$ para todo entero positivo n excepto para un número finito. Así, si los algoritmos A y B tienen tiempos de ejecución que sean $\Theta(f(n))$ y $\Theta(g(n))$, respectivamente, y si $\Theta(f(n))$ está arriba de $\Theta(g(n))$ en la tabla 3.5.3, entonces, para entradas suficientemente grandes, el algoritmo A es más eficiente en tiempo que el algoritmo B.

Es importante desarrollar una idea intuitiva de los tamaños relativos de las funciones que aparecen en la tabla 3.5.3. En la figura 3.5.1 hemos graficado algunas de estas funciones. Otra forma de desarrollar una idea de los tamaños relativos de las funciones $f(n)$ en la tabla 3.5.3 es determinar cuánto tardaría en concluir un algoritmo cuyo tiempo de ejecución sea exactamente $f(n)$. Para esto, supongamos que disponemos de una computadora que pueda ejecutar un paso en un microsegundo (10^{-6} segundos). La tabla 3.5.1 muestra los tiempos de ejecución para diversos tamaños de entrada, bajo estas hipótesis. Observe que es factible implantar un algoritmo que requiera 2^n pasos para una entrada de tamaño n sólo para tamaños de entrada muy pequeños. Los algoritmos que requieren n^2 o n^3 pasos también dejan de ser factibles, pero sólo para tamaños de entrada relativamente grandes. Además, observe la drástica mejoría resultante al pasar de n^2 pasos a $n \lg n$ pasos.

Un problema que tiene un algoritmo con tiempo polinomial en el peor de los casos se considera un algoritmo "bueno"; la interpretación de esto es que dicho problema tiene una solución eficiente. Por supuesto, si el tiempo necesario para resolver un problema en el peor de los casos es proporcional a un polinomio de grado alto, la solución del problema podría tardar mucho tiempo. Por fortuna, en muchos casos importantes la cota polinomial tiene un grado pequeño.

TABLA 3.5.3
Funciones de crecimiento comunes

Forma theta†	*Nombre*
$\Theta(1)$	Constante
$\Theta(\lg \lg n)$	Log log
$\Theta(\lg n)$	Logarítmica
$\Theta(n)$	Lineal
$\Theta(n \lg n)$	$n \log n$
$\Theta(n^2)$	Cuadrática
$\Theta(n^3)$	Cúbica
$\Theta(n^m)$	Polinomial
$\Theta(m^n), m \geq 2$	Exponencial
$\Theta(n!)$	Factorial

†lg = log de base 2;
m es un entero no negativo fijo.

Un problema que no tiene un algoritmo con tiempo polinomial en el peor de los casos es **intratable**. El tiempo de ejecución, para el peor de los casos, de cualquier algoritmo, si existe, que resuelva un problema intratable, es muy largo incluso para tamaños de entrada modestos.

Ciertos problemas son tan difíciles que no disponen de algoritmo alguno. Un problema para el que no hay algoritmo se dice que es **irresoluble**. Se conoce un gran número de problemas que son irresolubles, algunos de ellos con una importancia práctica considerable. Uno de los primeros problemas que se demostró ser irresoluble es el **problema de terminación**: Dado un programa arbitrario y un conjunto de entradas, ¿se detendrá el programa en algún momento?

Un gran número de problemas solubles tienen un estado hasta ahora indeterminado; se supone que son intratables, pero esto no se ha demostrado. (Estos problemas pertenecen a la clase NP; véase los detalles en [Hopcroft].) Un ejemplo de problema soluble que se piensa que es intratable, aunque esto no es claro aún, es:

> Dada una colección C de conjuntos finitos y un entero positivo $k \leq |C|$, ¿contiene C al menos k conjuntos mutuamente ajenos?

Otros problemas solubles de los cuales no se sabe si sean tratables o no son el problema del agente de ventas viajero y el problema del ciclo hamiltoniano (véase la sección 6.3).

Ejercicios

Seleccione una notación theta de la tabla 3.5.3 para cada expresión en los ejercicios 1-12.

1. $6n + 1$
2. $2n^2 + 1$
3. $6n^3 + 12n^2 + 1$
4. $3n^2 + 2n \lg n$
5. $2 \lg n + 4n + 3n \lg n$
6. $6n^6 + n + 4$
7. $2 + 4 + 6 + \cdots + 2n$
8. $(6n + 1)^2$
9. $(6n + 4)(1 + \lg n)$
10. $\dfrac{(n+1)(n+3)}{n+2}$
11. $\dfrac{(n^2 + \lg n)(n+1)}{n + n^2}$
12. $2 + 4 + 8 + 16 + \cdots + 2^n$

En los ejercicios 13-15, seleccione una notación theta para $f(n) + g(n)$.

13. $f(n) = \Theta(1), \quad g(n) = \Theta(n^2)$
14. $f(n) = 6n^3 + 2n^2 + 4, \quad g(n) = \Theta(n \lg n)$
15. $f(n) = \Theta(n^{3/2}), \quad g(n) = \Theta(n^{5/2})$

En los ejercicios 16-26, seleccione una notación theta entre

$$\Theta(1), \quad \Theta(\lg n), \quad \Theta(n), \quad \Theta(n \lg n), \quad \Theta(n^2), \quad \Theta(n^3), \quad \Theta(2^n) \quad \text{o} \quad \Theta(n!)$$

para el número de veces que se ejecuta la instrucción $x := x + 1$.

16. **for** $i := 1$ **to** $2n$ **do**
    $x := x + 1$

17. $i := 1$
    while $i \leq 2n$ **do**
    begin
    $x := x + 1$
    $i := i + 2$
    end

18. **for** $i := 1$ **to** n **do**
    for $j := 1$ **to** n **do**
    $x := x + 1$

19. **for** $i := 1$ **to** $2n$ **do**
    for $j := 1$ **to** n **do**
    $x := x + 1$

20. **for** $i := 1$ **to** n **do**
    for $j := 1$ **to** $\lfloor i/2 \rfloor$ **do**
    $x := x + 1$

21. **for** $i := 1$ **to** n **do**
    for $j := 1$ **to** n **do**
    for $k := 1$ **to** n **do**
    $x := x + 1$

22. **for** $i := 1$ **to** n **do**
    for $j := 1$ **to** n **do**
    for $k := 1$ **to** i **do**
    $x := x + 1$

23. **for** $i := 1$ **to** n **do**
    for $j := 1$ **to** i **do**
    for $k := 1$ **to** j **do**
    $x := x + 1$

24. $j := n$
    while $j \geq 1$ **do**
    begin
    for $i := 1$ **to** j **do**
    $x := x + 1$
    $j := \lfloor j/3 \rfloor$
    end

25. $i := n$
    while $i \geq 1$ **do**
    begin
    $x := x + 1$
    $i := \lfloor i/2 \rfloor$
    end

26. $i := n$
    while $i \geq 1$ **do**
    begin
    for $j := 1$ **to** n **do**
    $x := x + 1$
    $i := \lfloor i/2 \rfloor$
    end

27. Determine una notación theta para el número de veces que se ejecuta la instrucción $x := x + 1$.

```
i := 2
while i < n do
 begin
 i := i^2
 x := x + 1
 end
```

28. Determine el número exacto de comparaciones (líneas 12, 18, 20, 28 y 30) necesarias para el siguiente algoritmo cuando n es par y cuando n es impar. Determine una notación theta para este algoritmo.

Entrada: $s_1, s_2, \ldots, s_n, n$

Salida: *large* (el elemento más grande en $s_1, s_2, \ldots, s_n$)
small (el elemento más pequeño en $s_1, s_2, \ldots, s_n$)


```
1.  procedure large_small (s, n, large, small)
2.   if n = 1 then
3.      begin
4.      large := s_1
5.      small := s_1
6.      return
7.      end
8.   m := 2⌊n/2⌋
9.   i := 1
10.  while i ≤ m − 1 do
11.     begin
12.     if s_i > s_{i+1} then
13.       intercambiar(s_i, s_{i+1})
14.     i := i + 2
15.     end
16.  if n > m then
17.     begin
18.     if s_{m−1} > s_n then
19.       intercambiar(s_{m−1}, s_n)
20.     if s_n > s_m then
21.       intercambiar(s_m, s_n)
22.     end
23.  small := s_1
24.  large := s_2
25.  i := 3
26.  while i ≤ m − 1 do
27.     begin
28.     if s_i < small then
29.       small := s_i
30.     if s_{i+1} > large then
31.       large := s_{i+1}
32.     i := i + 2
33.     end
34.  end large_small
```

29. Suponga que $a > 1$ y que $f(n) = \Theta(\log_a n)$. Muestre que $f(n) = \Theta(\lg n)$.
30. Muestre que $n! = O(n^n)$.
31. Muestre que $2^n = O(n!)$.
32. Suponga que $g(n) > 0$ para $n = 1, 2, \ldots$ y para toda n, $f(n)$ es distinta de cero. Muestre que $f(n) = \Theta(g(n))$ si y sólo si existen constantes positivas c_1 y c_2 tales que

$$c_1 g(n) \leq |f(n)| \leq c_2 g(n) \qquad \text{para } \textit{toda } n = 1, 2, \ldots.$$

Determine si cada afirmación en los ejercicios 33-42 es verdadera o falsa. Si la afirmación es falsa, proporcione un contraejemplo. Suponga que las funciones f, g y h sólo toman valores positivos.

33. Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $f(n) = \Theta(h(n))$.
34. Si $f(n) = \Theta(h(n))$ y $g(n) = \Theta(h(n))$, entonces $f(n) + g(n) = \Theta(h(n))$.
35. Si $f(n) = \Theta(g(n))$, entonces $cf(n) = \Theta(g(n))$ para toda $c \neq 0$.
36. Si $f(n) = \Theta(g(n))$, entonces $2^{f(n)} = \Theta(2^{g(n)})$.
37. Si $f(n) = \Theta(g(n))$, entonces $\lg f(n) = \Theta(\lg g(n))$. Suponga que $f(n) \geq 1$ y $g(n) \geq 1$ para toda $n = 1, 2, \ldots$.
38. Si $f(n) = O(g(n))$, entonces $g(n) = O(f(n))$.

39. Si $f(n) = O(g(n))$, entonces $g(n) = \Omega(f(n))$.

40. Si $f(n) = \Theta(g(n))$, entonces $g(n) = \Theta(f(n))$.

41. $f(n) + g(n) = \Theta(h(n))$, donde $h(n) = \max\{f(n), g(n)\}$

42. $f(n) + g(n) = \Theta(h(n))$, donde $h(n) = \min\{f(n), g(n)\}$

★ **43.** Determine funciones f y g que satisfagan

$$f(n) \neq O(g(n)) \qquad \text{y} \qquad g(n) \neq O(f(n)).$$

44. Determine funciones f, g, h y t que satisfagan

$$f(n) = \Theta(g(n)), \qquad h(n) = \Theta(t(n)), \qquad f(n) - h(n) \neq \Theta(g(n) - t(n)).$$

45. ¿Dónde está el error en el siguiente razonamiento? Suponga que el tiempo de un algoritmo en el peor de los casos es $\Theta(n)$. Como $2n = \Theta(n)$, el tiempo de ejecución del algoritmo en el peor de los casos con entrada de tamaño $2n$ será aproximadamente igual al tiempo de ejecución del algoritmo en el peor de los casos con entrada de tamaño n.

46. Muestre que si $n \geq 4$,

$$\frac{n}{2}\lg\frac{n}{2} \geq \frac{n \lg n}{4}.$$

47. ¿Define la ecuación

$$f(n) = O(g(n))$$

una relación de equivalencia sobre el conjunto de funciones con valores reales definidas sobre $\{1, 2, \ldots\}$?

48. ¿Define la ecuación

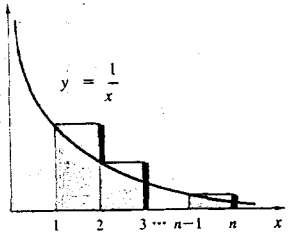
$$f(n) = \Theta(g(n))$$

una relación de equivalencia sobre el conjunto de funciones con valores reales definidas sobre $\{1, 2, \ldots\}$?

49. [Requiere conocimientos de cálculo integral.]

(a) Muestre, consultando la figura, que

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} < \log_e n.$$



(b) Muestre, consultando la figura, que

$$\log_e n < 1 + \frac{1}{2} + \cdots + \frac{1}{n-1}.$$

(c) Utilice las partes (a) y (b) para mostrar que

$$1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\lg n).$$

50. [Requiere conocimientos de cálculo integral.] Utilice un argumento similar al del ejercicio 49 para mostrar que

$$\frac{n^{m+1}}{m+1} < 1^m + 2^m + \cdots + n^m < \frac{(n+1)^{m+1}}{m+1},$$

donde m es un entero positivo.

51. ¿Cuál es el error en la siguiente "demostración" de que cualquier algoritmo tiene un tiempo de ejecución que es $O(n)$?

Debemos mostrar que el tiempo necesario para una entrada de tamaño n es a lo más una constante por n.

PASO BASE. Suponga que $n = 1$. Si el algoritmo tarda C unidades de tiempo para una entrada de tamaño 1, el algoritmo tarda a lo más $C \cdot 1$ unidades de tiempo. Así, la afirmación es verdadera para $n = 1$.

PASO INDUCTIVO. Suponga que el tiempo necesario para una entrada de tamaño n es a lo más $C'n$ y que el tiempo necesario para procesar un elemento adicional es C''. Sea C el máximo de C' y C''. Entonces el tiempo total necesario para una entrada de tamaño $n + 1$ es a lo más

$$C'n + C'' \leq Cn + C = C(n + 1)$$

Se ha verificado el paso inductivo.

Por inducción, para una entrada de tamaño n, el tiempo necesario es a lo más Cn. Por tanto, el tiempo de ejecución es $O(n)$.

52. [Requiere conocimientos de cálculo.] Determine si cada afirmación es verdadera o falsa. Si la afirmación es falsa, proporcione un contrajemplo. Se supone que f y g son funciones con valores reales, definidas sobre el conjunto de enteros positivos y que $g(n) \neq 0$ para $n \geq 1$.

(a) Si

$$\lim_{n\to\infty} \frac{f(n)}{g(n)}$$

existe y es igual a algún número real, entonces $f(n) = O(g(n))$.

(b) Si $f(n) = O(g(n))$, entonces

$$\lim_{n\to\infty} \frac{f(n)}{g(n)}$$

existe y es igual a algún número real.

(c) Si

$$\lim_{n\to\infty} \frac{f(n)}{g(n)}$$

existe y es igual a algún número real, entonces $f(n) = \Theta(g(n))$.

(d) Si

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1,$$

entonces $f(n) = \Theta(g(n))$.

(e) Si $f(n) = \Theta(g(n))$, entonces.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)}$$

existe y es igual a algún número real.

☆ **53.** Utilice inducción para demostrar que

$$\lg n! \geq \frac{n}{2}\lg\frac{n}{2}.$$

54. [Requiere conocimientos de cálculo.] Sea $\ln x$ el logaritmo natural ($\log_e x$) de x. Utilice la integral para obtener la estimación

$$n \ln n - n \leq \sum_{k=1}^{n} \ln k = \ln n!, \qquad n \geq 1.$$

55. Utilice el resultado del ejercicio 54 y la fórmula para cambio de base de los logaritmos para obtener la fórmula

$$n \lg n - n \lg e \leq \lg n!, \; n \geq 1.$$

56. Deduzca

$$\lg n! \geq \frac{n}{2}\lg\frac{n}{2}$$

a partir de la desigualdad del ejercicio 55.

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

DISEÑO Y ANÁLISIS DE UN ALGORITMO

1. Problema

Desarrollar y analizar un algoritmo que produzca como salida la suma máxima de valores consecutivos en la sucesión numérica

$$s_1, \ldots, s_n$$

En notación matemática, el problema es determinar la máxima suma de la forma $s_i + s_{i+1} + \cdots + s_j$. *Ejemplo:* Si la sucesión es

27 6 −50 21 −3 14 16 −8 42 33 −21 9

el algoritmo produce como salida 115, la suma de

21 −3 14 16 −8 42 33

Si todos los números de una sucesión son negativos, la suma máxima de valores consecutivos se define como 0. (La idea es que el máximo de 0 se alcanza considerando una suma "vacía".)

Para enfrentar el problema

Al desarrollar un algoritmo, es bueno comenzar con la pregunta: "¿Cómo resolvería este problema a mano?" Puede utilizarse un punto de vista directo, al menos inicialmente. Aquí enumeramos las sumas de todos los valores consecutivos y elegimos la más grande. Para la sucesión del ejemplo, las sumas son como sigue:

	j											
i	1	2	3	4	5	6	7	8	9	10	11	12
1	27											
2	33	6										
3	−17	−44	−50									
4	4	−23	−29	21								
5	1	−26	−32	18	−3							
6	15	−12	−18	32	11	14						
7	31	4	−2	48	27	30	16					
8	23	−4	−10	40	19	22	8	−8				
9	65	38	32	82	61	64	50	34	42			
10	98	71	65	115	94	97	83	67	75	33		
11	77	50	44	94	73	76	62	46	54	12	−21	
12	86	59	53	103	82	85	71	55	63	21	−12	9

La entrada en la columna j, renglón i, es la suma

$$s_j + \cdots + s_i$$

Por ejemplo, la entrada en la columna 4, renglón 7, es 48, la suma

$$s_4 + s_5 + s_6 + s_7 = 21 + -3 + 14 + 16 = 48.$$

Por inspección, vemos que 115 es la suma más grande.

Determinación de una solución

Primero escribimos un seudocódigo para el algoritmo directo que calcula todas las sumas consecutivas y determina la más grande.

Entrada: $s_1, \ldots, s_n$

Salida: max

```
procedure max_sum1(s, n)
  // sum_ij es la suma s_j + ... + s_i
  for i := 1 to n do
    begin
    for j := 1 to i − 1 do
      sum_ij := sum_i,j−1 + s_i
    sum_ii := s_i
    end

  // se recorren sum_ij y se determina el máximo
  max := 0
  for i := 1 to n do
    for j := 1 to i do
      if sum_ij > max then
        max := sum_ij
  return(max)
  end max_sum1
```

Los primeros ciclos for anidados calculan las sumas

$$sum_{ij} = s_j + \cdots + s_i$$

El cálculo se basa en el hecho de que

$$sum_{ij} = s_j + \cdots + s_i = s_j + \cdots + s_{i-1} + s_i = sum_{i,i-1} + s_i$$

Los segundos ciclos for anidados recorren los términos sum_{ij} y determinan el valor máximo.

Como cada uno de los ciclos for anidados tarda un tiempo $\Theta(n^2)$, el tiempo de *max_sum1* es $\Theta(n^2)$.

Podemos mejorar el tiempo real, pero no la complejidad del algoritmo, calculando el máximo dentro de los mismos ciclos for anidados en los que calculamos sum_{ij}.

```
Entrada:  s_1, ..., s_n
Salida:   max

procedure max_sum2(s, n)
  // sum_ij es la suma s_j + ... + s_i
  max := 0
  for i := 1 to n do
    begin
    for j := 1 to i - 1 do
      begin
      sum_ji := sum_j,i-1 + s_i
      if sum_ji > max then
        max := sum_ji
      end
    sum_ii := s_i
    if sum_ii > max then
      max := sum_ii
    end
  return(max)
end max_sum2
```

Como los ciclos for anidados tardan un tiempo $\Theta(n^2)$, el tiempo de *max_sum2* es $\Theta(n^2)$. Para reducir la complejidad del tiempo, necesitamos analizar con detalle el seudocódigo para ver en dónde puede mejorarse.

Existen dos observaciones fundamentales que permiten mejorar el tiempo. En primer lugar, como sólo estamos buscando la suma *máxima*, no hay necesidad de registrar todas las sumas; basta guardar la suma máxima que termina en el índice *i*. En segundo lugar, la línea

$$sum_{ji} := sum_{j,i-1} + s_i$$

muestra la forma en que una suma consecutiva que termina en el índice $i-1$ se relaciona con una suma consecutiva que termina en el índice *i*. El máximo puede calcularse utilizando una fórmula similar. Si *sum* es la suma consecutiva máxima que termina en el índice $i-1$, la suma consecutiva máxima que termina en el índice *i* se obtiene sumando s_i a *sum*, siempre que $sum + s_i$ sea positivo. (Si alguna suma de términos consecutivos que termina en el índice *i* excede a $sum + s_i$, podríamos eliminar el *i*-ésimo término y obtener una suma de términos consecutivos que termine en el índice $i-1$ que exceda a *sum*, lo cual es imposible.) Si $sum + s_i \leq 0$, la suma consecutiva máxima que termina en el índice *i* se obtiene sin considerar término alguno y tiene valor 0. Así, podemos calcular la suma consecutiva máxima que termina en el índice *i* ejecutando

```
if sum + s_i > 0 then
    sum := sum + s_i
else
    sum := 0
```


Solución formal

```
Entrada:  s_1, ..., s_n
Salida:   max

procedure max_sum3(s, n)
  // max es la suma máxima vista hasta el momento.
  // Después de la i-ésima iteración del ciclo for, sum es la máxima suma
  consecutiva que termina en la posición i.
  max := 0
  sum := 0
  for i := 1 to n do
    begin
    if sum + s_i > 0 then
      sum := sum + s_i
    else
      sum := 0
    if sum > max then
      max := sum
    end
  return(max)
end max_sum3
```

Como este algoritmo tiene un único ciclo for que se ejecuta desde 1 hasta *n*, el tiempo de *max_sum3* es $\Theta(n)$. La complejidad del tiempo para este algoritmo no puede mejorarse más. Para resolver este problema, debemos analizar al menos cada elemento de la sucesión s_i, lo cual tarda un tiempo $\Theta(n)$.

Resumen de técnicas de solución de problemas

- Al desarrollar un algoritmo, una buena forma de comenzar es preguntarse: "¿Cómo resolvería este problema a mano?"
- Al desarrollar un algoritmo, primero adopte un punto de vista directo.
- Después de desarrollar un algoritmo, analice con detalle el seudocódigo para ver los puntos donde puede mejorarse. Analice las partes que realizan los cálculos fundamentales para intuir dónde puede mejorarse la eficiencia del algoritmo.
- Como en la inducción matemática, extienda la solución de un problema más pequeño a un problema más grande. (En este problema, extendimos una suma que termina en el índice $i-1$ a una suma que termina en el índice *i*.)
- No repita los cálculos. (En este problema, extendemos una suma que termina en el índice $i-1$ a una suma que termina en el índice *i* agregando un término adicional, en vez de calcular la suma que termina en el índice *i* partiendo de cero. Este último método implicaría volver a calcular la suma que termina en el índice $i-1$.)

Comentarios

De acuerdo con [Bentley], el problema analizado en esta sección es la versión unidimensional del problema original bidimensional que trata de la concordancia de patrones en las imágenes digitales. El problema original era determinar la suma máxima en una submatriz rectangular de una matriz $n \times n$ de números reales.

EJERCICIOS

1. Modifique *max_sum3* de modo que no sólo calcule la suma máxima de valores consecutivos, sino también los índices del primer y último términos de una subsucesión con suma máxima. Si no existe una subsucesión con suma máxima (lo cual ocurriría, por ejemplo, si todos los valores de la sucesión fuesen negativos), el algoritmo haría el primer y último índices iguales a cero.

3.6 ANÁLISIS DEL ALGORITMO DE EUCLIDES

En esta sección analizaremos el desempeño en el peor de los casos del algoritmo de Euclides para determinar el máximo común divisor de dos enteros no negativos, no ambos cero (algoritmo 3.3.8). Como referencia, resumiremos el algoritmo:

Entrada: a y b (enteros no negativos, no ambos cero)

Salida: Máximo común divisor de a y b

```
1.  procedure mcd(a, b)
2.      // hacemos que a sea el mayor
3.      if a < b then
4.          intercambiar(a, b)
5.      while b ≠ 0 do
6.          begin
7.          dividir a entre b para obtener a = bq + r, 0 ≤ r < b
8.          a := b
9.          b := r
10.         end
11.     return(a)
12. end mcd
```

Definimos el tiempo requerido por el algoritmo de Euclides como el número de divisiones ejecutadas en la línea 7. La tabla 3.6.1 enumera el número de divisiones necesarias para ciertos valores pequeños de entrada.

El peor de los casos en el algoritmo de Euclides ocurre cuando el número de divisiones es tan grande como sea posible. En relación con la tabla 3.6.1, podemos determinar la pareja de entradas $a, b, a > b$, con a tan pequeño como sea posible, que requiere n divisiones para $n = 0, \ldots, 4$. Los resultados aparecen en la tabla 3.6.2.

Recordemos que la sucesión de Fibonacci $\{f_n\}$ (véase el ejemplo 3.4.6) se define mediante las ecuaciones

$$f_1 = 1; \quad f_2 = 2; \quad fn = f_{n-1} + f_{n-2}, \quad n \geq 3.$$

TABLA 3.6.1
Número de divisiones necesarias para el algoritmo de Euclides para diversos valores de la entrada

a \ b	0	1	2	3	4	5	6	7	8
0	–	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1
2	0	1	1	2	1	2	1	2	1
3	0	1	2	1	2	3	1	2	3
4	0	1	1	2	1	2	2	3	1
5	0	1	2	3	2	1	2	3	4
6	0	1	1	1	2	2	1	2	2
7	0	1	2	2	3	3	2	1	2
8	0	1	1	3	1	4	2	2	1

La sucesión de Fibonacci comienza así:

$$1, \quad 2, \quad 3, \quad 5, \quad 8, \quad 13, \quad \ldots.$$

En la tabla 3.6.2 aparece un patrón sorprendente: La columna a es el principio de la sucesión de Fibonacci y, excepto por el primer valor, la columna b ¡también es el principio de la sucesión de Fibonacci! Conjeturamos entonces que si la pareja $a, b, a > b$, se introduce como entrada en el algoritmo de Euclides y requiere $n \geq 1$ divisiones, entonces $a \geq f_{n+1}$ y $b \geq f_n$. Como mayor evidencia de esta conjetura, si calculamos la menor pareja de entrada que requiere cinco divisiones, obtenemos $a = 13$ y $b = 8$. Los valores para seis divisiones son $a = 21$ y $b = 13$. Nuestro siguiente teorema confirma que nuestra conjetura es correcta. La demostración de este teorema se ilustra en la figura 3.6.1.

TABLA 3.6.2
Menor pareja de entrada que requiere n divisiones en el algoritmo de Euclides

a b	n (= número de divisiones)
1 0	0
2 1	1
3 2	2
5 3	3
8 5	4

$91 = 57 \cdot 1 + 34$	(1 división)
34, 57 requieren 4 divisiones	(para hacer un total de 5)
$57 \geq f_5$ y $34 \geq f_4$	(por hipótesis de inducción)
$\therefore 91 = 57 \cdot 1 + 34 \geq 57 + 34 \geq f_5 + f_4 = f_6$	

FIGURA 3.6.1 La demostración del teorema 3.6.1. La pareja 57, 91, que requiere $n + 1 = 5$ divisiones, es una entrada del algoritmo de Euclides.

TEOREMA 3.6.1

Suponga que la pareja $a, b, a > b$, requiere $n \geq 1$ divisiones cuando se utiliza como entrada del algoritmo de Euclides. Entonces $a \geq f_{n+1}$ y $b \geq f_n$, donde $\{f_n\}$ denota la sucesión de Fibonacci.

Demostración. La demostración es por inducción sobre n.

PASO BASE ($n = 1$). Ya hemos observado que el teorema es verdadero si $n = 1$.

PASO INDUCTIVO. Suponga que el teorema es cierto para $n \geq 1$. Debemos mostrar que el teorema es verdadero para $n + 1$.

Supongamos que la pareja $a, b, a > b$, requiere $n + 1$ divisiones al utilizarse como entrada para el algoritmo de Euclides. En la línea 7, dividimos a entre b para obtener

$$a = bq + r, \quad 0 \leq r < b. \tag{3.6.1}$$

El algoritmo se repite entonces, utilizando los valores b y $r, b > r$. Estos valores requieren n divisiones adicionales. Por la hipótesis de inducción,

$$b \geq f_{n+1} \quad \text{y} \quad r \geq f_n. \tag{3.6.2}$$

Al combinar (3.6.1) y (3.6.2), obtenemos

$$a = bq + r \geq b + r \geq f_{n+1} + f_n = f_{n+2}. \tag{3.6.3}$$

[La primera desigualdad en (3.6.3) es válida pues $q > 0$; q no puede ser igual a 0, pues $a > b$.] Las desigualdades (3.6.2) y (3.6.3) implican

$$a \geq f_{n+2} \quad \text{y} \quad b \geq f_{n+1}.$$

Con esto concluye el paso inductivo y la demostración. ■

Podemos utilizar el teorema 3.6.1 para analizar el rendimiento del algoritmo de Euclides en el peor de los casos.

TEOREMA 3.6.2

Si en el algoritmo de Euclides se utilizan enteros en el rango 0 a m, $m \geq 8$, no ambos cero, entonces se necesitan a lo más

$$\log_{3/2} \frac{2m}{3}$$

divisiones.

Demostración. Sea n el número máximo de divisiones necesarias para realizar el algoritmo de Euclides para enteros en el rango 0 a m, $m \geq 8$. Sea a, b una pareja de entradas en el rango 0 a m que requieran n divisiones. La tabla 3.6.1 muestra que $n \geq 4$ y que $a \neq b$. Podemos suponer que $a > b$. (El intercambio de los valores de a y b no altera el número de divisiones requeridas.) Por el teorema 3.6.1, $a \geq f_{n+1}$. Así,

$$f_{n+1} \leq m.$$

Como $n + 1 \geq 5$, el ejercicio 20 de la sección 3.4 implica que

$$\left(\frac{3}{2}\right)^{n+1} < f_{n+1}.$$

Al combinar estas últimas desigualdades, se obtiene

$$\left(\frac{3}{2}\right)^{n+1} < m.$$

Al calcular el logaritmo en base 3/2, obtenemos

$$n + 1 < \log_{3/2} m.$$

Por tanto,

$$n < \log_{3/2} m - 1 = \log_{3/2} m - \log_{3/2} 3/2 = \log_{3/2} \frac{2m}{3}.$$ ■

Debido a que la función logarítmica crece muy lentamente, el teorema 3.6.2 nos dice que el algoritmo de Euclides es más o menos eficiente, incluso para valores grandes de la entrada. Por ejemplo, como

$$\log_{3/2} \frac{2(1,000,000)}{3} = 33.07\ldots,$$

el algoritmo de Euclides requiere a lo más 33 divisiones para calcular el máximo común divisor de cualquier pareja de enteros, no ambos cero, en el rango de 0 a 1,000,000.

Ejercicios

1. Extienda las tablas 3.6.1 y 3.6.2 al rango de 0 a 13.

2. ¿Exactamente cuántas divisiones son necesarias en el algoritmo de Euclides en el peor de los casos para números en el rango de 0 a 1,000,000?

3. ¿Cuántas restas son necesarias en el algoritmo del ejercicio 26, sección 3.3, en el peor de los casos, en el rango de 0 a m? (Este algoritmo determina el máximo común divisor mediante restas en lugar de divisiones.)

4. Demuestre que cuando la pareja f_{n+1}, f_n se introduce como entrada en el algoritmo de Euclides, $n \geq 1$, se necesitan exactamente n divisiones.

5. Muestre que para cualquier entero $k > 1$, el número de divisiones necesarias para realizar el algoritmo de Euclides y calcular mcd(a, b) es igual al número de divisiones necesarias para calcular mcd(ka, kb).

6. Muestre que mcd$(f_n, f_{n+1}) = 1, n \geq 1$.

† 3.7 *EL SISTEMA CRIPTOGRÁFICO CON CLAVE PÚBLICA RSA*

La **criptología** es el estudio de sistemas llamados **criptosistemas**, para las comunicaciones seguras. En un criptosistema, el emisor transforma el mensaje antes de transmitirlo de modo que, en teoría, sólo un receptor autorizado pueda reconstruir el mensaje original (es decir, el mensaje antes de ser transformado). Se dice que el emisor **cifra** el mensaje, y que el receptor **descifra** el mensaje. Si el criptosistema es seguro, las personas no autorizadas no podrán descubrir la técnica de cifrado, así que aunque lean el mensaje cifrado, no podrán descifrarlo. Los criptosistemas son importantes para las grandes organizaciones (por ejemplo, el gobierno o el ejército) y también para los individuos. Por ejemplo, si un número de tarjeta de crédito se envía a través de una red de computadoras, es importante que el número sea leído sólo por el receptor indicado. En esta sección, examinaremos ciertos algoritmos que permiten la comunicación segura.

† Esta sección puede omitirse sin pérdida de continuidad.

En uno de los sistemas más antiguos y sencillos, el emisor y el receptor tienen cada uno una clave que define un carácter sustituto por cada carácter potencial por ser enviado. Además, el emisor y el receptor no revelan la clave. Tales claves son *privadas*.

EJEMPLO 3.7.1

Si se define una clave como

carácter:	ABCDEFGHIJKLMNOPQRSTUVWXYZ
se reemplaza por:	EIJFUAXVHWP GSRKOBTQYDMLZNC

el mensaje SEND MONEY se cifraría como QARUESKRAN. El mensaje cifrado SKRANEKRELIN se descifraría como MONEY ON WAY. □

Los sistemas sencillos, como el del ejemplo 3.7.1, se descubren con facilidad, pues ciertas letras (por ejemplo, en el inglés la letra E) o combinaciones de letras (por ejemplo, ER en el inglés) aparecen con más frecuencia que otras. Además, un problema general con las claves privadas es que las claves se tienen que enviar de manera segura al emisor y al receptor antes de poder enviar los mensajes. Dedicaremos el resto de esta sección al **sistema criptográfico con clave pública RSA**, que recibe el nombre de sus inventores, Ronald L. Rivest, Adi Shamir y Leonard M. Adleman, el cual se cree que es seguro. En el sistema RSA, cada participante hace pública una clave de cifrado y oculta una clave de descifrado. Para enviar un mensaje, todo lo que debe hacer es buscar la clave de cifrado del receptor en una tabla distribuida públicamente. El receptor descifra entonces el mensaje con la clave oculta de descifrado.

En el sistema RSA, los mensajes se representan mediante números. Por ejemplo, cada carácter se podría representar como un número. Si el espacio en *blanco* se representa como 1, A como 2, B como 3, y así sucesivamente, el mensaje SEND MONEY se representaría como 20, 6, 15, 5, 1, 14, 16, 15, 6, 26. Si se desea, los enteros se podrían combinar en un único entero

$$20061505011416150626.$$

A continuación describimos el funcionamiento del sistema RSA, presentaremos un ejemplo concreto, y luego analizaremos su funcionamiento. Cada posible receptor elige dos primos p y q y calcula $z = pq$. Como la seguridad del sistema RSA se basa principalmente en la incapacidad de que alguien que conozca el valor de z descubra los números p y q, por lo general p y q se eligen de modo que cada uno tenga 100 o más dígitos. A continuación, el posible receptor calcula $\phi = (p - 1)(q - 1)$ y elige un entero n tal que $\text{mcd}(n, \phi) = 1$. En la práctica, con frecuencia se elige n como un primo. Entonces se hace pública la pareja z, n. Por último, el posible receptor calcula el único número s, $0 < s < \phi$ que satisface $ns \bmod \phi = 1$. El número s se mantiene en secreto y se utiliza para descifrar los mensajes.

Para enviar el entero a, $0 \leq a \leq z - 1$, al poseedor de la clave pública z, n, el emisor calcula $c = a^n \bmod z$ y envía c. Para descifrar el mensaje, el receptor calcula $c^s \bmod z$, lo cual es igual a a, como puede mostrarse.

EJEMPLO 3.7.2

Supongamos que se elige $p = 23$, $q = 31$ y $n = 29$. Entonces $z = pq = 713$ y $\phi = (p - 1)(q - 1) = 660$. Ahora, $s = 569$, pues $ns \bmod \phi = 29 \cdot 569 \bmod 660 = 16501 \bmod 660 = 1$. La pareja z, $n = 713, 29$ se pone a disposición del público.

Para transmitir $a = 572$ al poseedor de la clave pública 713, 29, el emisor calcula $c = a^n \bmod z = 572^{29} \bmod 713 = 113$ y envía 113. El receptor calcula $cs \bmod z = 113^{569} \bmod 713 = 572$ para descifrar el mensaje. □

Parecería que hay que calcular números enormes para cifrar y descifrar mensajes mediante el sistema RSA. Por ejemplo, el número 572^{29} en el ejemplo 3.7.2 tiene 80 dígitos, y si p y q tienen 100 o más dígitos, los números serían mucho mayores. La clave para simplificar los cálculos consiste en observar que la aritmética se realiza módulo z. Puede mostrarse que

$$ab \bmod z = [(a \bmod z)(b \bmod z)] \bmod z \tag{3.7.1}$$

(véase el ejercicio 10). Mostraremos la forma de utilizar (3.7.1) para calcular $572^{29} \bmod 713$.

EJEMPLO 3.7.3

Utilizaremos (3.7.1) para calcular $572^{29} \bmod 713$. Observemos que

$$29 = 16 + 8 + 4 + 1$$

(que es justamente la representación en base 2 de 29), de modo que calculamos 572 elevado a cada una de las potencias 16, 8, 4 y 1, mod 713, elevando al cuadrado y multiplicando varias veces, módulo 713:

$$572^2 \bmod 713 = 327184 \bmod 731 = 630$$

$$572^4 \bmod 713 = 630^2 \bmod 713 = 396900 \bmod 713 = 472$$

$$572^8 \bmod 713 = 472^2 \bmod 713 = 222784 \bmod 713 = 328$$

$$572^{16} \bmod 713 = 328^2 \bmod 713 = 107584 \bmod 713 = 634$$

$$572^{24} \bmod 713 = 572^{16} \cdot 572^8 \bmod 713 = 634 \cdot 328 \bmod 713$$
$$= 207952 \bmod 713 = 469$$

$$572^{28} \bmod 713 = 572^{24} \cdot 572^4 \bmod 713 = 469 \cdot 472 \bmod 713$$
$$= 221368 \bmod 713 = 338$$

$$572^{29} \bmod 713 = 572^{28} \cdot 572^1 \bmod 713 = 338 \cdot 572 \bmod 713$$
$$= 193336 \bmod 713 = 113.$$

El método puede convertirse fácilmente en un algoritmo (véase el ejercicio 11). □

Un posible receptor puede utilizar el algoritmo de Euclides para calcular con eficiencia el número único s, $0 < s < \phi$, que satisface $ns \bmod \phi = 1$ (véase el ejercicio 12). El principal resultado que hace funcionar el cifrado y el descifrado es que

$$a^u \bmod z = a \quad \text{para toda } 0 \leq a < z \text{ y } u \bmod \phi = 1$$

(para una demostración, véase [Cormen: teorema 33.36, página 834]). Utilizamos este resultado y (3.7.1) para mostrar que el descifrado produce el resultado correcto. Como $ns \bmod \phi = 1$,

$$c^s \bmod z = (a^n \bmod z)^s \bmod z = (a^n)^s \bmod z = a^{ns} \bmod z = a.$$

La seguridad del sistema de cifrado RSA reside principalmente en el hecho de que hasta el momento no existe un algoritmo eficiente conocido para factorizar enteros; es decir, actualmente no se conoce un algoritmo que factorice enteros con d dígitos en un tiempo polinomial, $O(d^k)$. Así, si los primos p y q se eligen suficientemente grandes, no es práctico calcular la factorización $z = pq$. Si una persona que intercepta el mensaje puede determinar la factorización, podría descifrar el mensaje al igual que lo hace un receptor autorizado. Hasta la fecha, no se conoce un método práctico para factorizar enteros con 200 o más dígitos, de modo que si p y q se eligen de modo que cada uno tenga 100 o más dígitos, pq tendría entonces cerca de 200 o más dígitos, lo cual hace que el sistema RSA sea seguro.

La primera descripción del sistema de cifrado RSA apareció en una columna de Martin Gardner en la edición de febrero de 1977 del *Scientific American* (véase [Gardner, 1977]). En esta columna se incluyó un mensaje codificado utilizando la clave z, n, donde z era el producto de primos de 64 y 65 dígitos y $n = 9007$, junto con un premio de $100 para la primera persona que descifrase el código. Cuando se escribió el artículo, se estimaba que se necesitarían 40,000 billones de años para factorizar z. De hecho, en abril de 1994, Arjen Lenstra, Paul Leyland, Michael Graff y Derek Atkins, con la ayuda de 600 voluntarios de 25 países utilizando más de 1600 computadoras, factorizaron z (véase [Taubes]). El trabajo fue coordinado mediante Internet.

Otra forma de interceptar y descifrar el mensaje sería considerar la n-ésima raíz de c mod z, donde c es el valor cifrado enviado. Como $c = a^n \bmod z$, la n-ésima raíz de c mod z daría a, el valor descifrado. De nuevo, hasta el momento no existe un algoritmo con tiempo polinomial conocido para calcular raíces n-ésimas mod z. También pueden existir otras formas de descifrar un mensaje mediante otros métodos distintos de la factorización de enteros o el cálculo de raíces n-ésimas mod z. Por ejemplo, en 1996, Paul Kocher propuso una forma de descifrar el RSA con base en el tiempo que tarda en descifrar mensajes (véase [English]). La idea es que diversas claves secretas requieren diferentes cantidades de tiempo para descifrar mensajes y, utilizando esta información del tiempo, una persona no autorizada podría descubrir la clave secreta y con ello descifrar el mensaje. Las personas que han implantado RSA han dado pasos para alterar el tiempo observado en el descifrado de mensajes para prevenir tales ataques.

Ejercicios

1. Cifre el mensaje COOL BEAVIS utilizando la clave del ejemplo 3.7.1.
2. Descifre el mensaje UTWR ENKDTEKMIGYWRA utilizando la clave del ejemplo 3.7.1.
3. Descifre 333 utilizando la clave pública 713, 29 del ejemplo 3.7.2.
4. Descifre 411 utilizando $s = 569$ como en el ejemplo 3.7.2.

En los ejercicios 5-9, suponga que hemos elegido los primos $p = 17$, $q = 23$ y $n = 31$.

5. Calcule z.
6. Calcule ϕ.
7. Verifique que $s = 159$.
8. Cifre 101 utilizando la clave pública z, n.
9. Descifre 250.
10. Demuestre la ecuación (3.7.1).
11. Proporcione un algoritmo eficiente para calcular $a^n \bmod z$.
12. Muestre la forma de calcular de manera eficiente el valor de s dados n y ϕ; es decir, dados enteros positivos n y ϕ, con $\text{mcd}(n, \phi) = 1$, proporcione un algoritmo eficiente para calcular enteros positivos s y t, con $0 < s < \phi$, tal que $ns - t\phi = 1$ y, en particular, $ns \bmod \phi = 1$.

    Sugerencia: Utilice el método del ejercicio 21, sección 3.3, para calcular de manera eficiente enteros s' y t' tales que $s'n + t'\phi = 1$. Si $s' > 0$, se elige $s' = s'$. Si $s' < 0$, se elige

    $$s = -s'(\phi - 1) \bmod \phi.$$

13. Muestre que el número s del ejercicio 12 es único.
14. Muestre la forma de utilizar el método del ejercicio 12 para calcular el valor s del ejemplo 3.7.2.
15. Muestre la forma de utilizar el método del ejercicio 12 para calcular el valor s del ejercicio 7.

NOTAS

Los libros de Knuth [1973, volúmenes 1 a 3; 1981] son los primeros tres libros de un conjunto proyectado de siete volúmenes. La primera mitad del volumen 1 presenta el concepto de algoritmo y diversos temas matemáticos, incluyendo la inducción matemática. La segunda mitad del volumen 1 está dedicada a las estructuras de datos. Estos volúmenes son clásicos en el área de los algoritmos y están entre los mejores ejemplos de literatura técnica.

La mayor parte de la bibliografía general relativa a las ciencias de la computación contienen un análisis de los algoritmos. Los libros específicos sobre algoritmos son [Aho; Baase; Brassard; Cormen; Knuth 1973, volúmenes 1 y 3, 1981; Manber; Nievergelt; y Reingold]. [McNaughton] contiene un amplio análisis a nivel introductorio de lo que es un algoritmo. También son recomendables el artículo explicativo de Knuth acerca de los algoritmos ([Knuth, 1977]) y su artículo acerca del papel de los algoritmos en las ciencias matemáticas ([Knuth, 1985]). [Gardner, 1979] contiene un capítulo acerca de la sucesión de Fibonacci.

Todos los detalles del criptosistema RSA aparecen en [Cormen]. [Pfleeger] está dedicado a la seguridad en computación.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 3.1
Algoritmo
Propiedades de un algoritmo: Precisión, unicidad, carácter finito, entrada, salida, generalidad
Enunciado de asignación: $x := y$
Rastreo

Sección 3.2
Seudocódigo
Procedimiento
Estructura if-then:
if *p* **then**
acción

Estructura if-then-else:
if *p* **then**
acción 1
else
acción 2

Comentario: Información no ejecutable. Un comentario comienza con // y continúa hasta el final de la línea.
Enunciados de retorno: **return** o **return**(*x*)
Ciclo while:
while *p* **do**
acción

Ciclo for:
for *var* := *init* **to** *limit* **do**
acción
Enunciado call (para llamar):
call $proc(p_1, p_2, \ldots, p_k)$

Sección 3.3

b divide a a : $b \mid a$
b es un divisor de a
Cociente
Residuo
Divisor común
Máximo común divisor
Algoritmo de Euclides

Sección 3.4

Algoritmo recursivo
Procedimiento recursivo
Técnica divide y vencerás
n factorial, $n!$: $n(n-1)\cdots 2 \cdot 1$
Casos base: Situaciones en las que un procedimiento recursivo no se llama a sí mismo
Sucesión de Fibonacci: $\{f_n\}$: $f_1 = 1$, $f_2 = 2, f_n = f_{n-1} + f_{n-2}, n \geq 3$

Sección 3.5

Análisis de algoritmos
Complejidad de algoritmos
Tiempo de un algoritmo en el peor de los casos
Tiempo de un algoritmo en el mejor de los casos
Tiempo de un algoritmo en el caso promedio
Notación o mayúscula: $f(n) = O(g(n))$
Notación omega: $f(n) = \Omega(g(n))$
Notación theta: $f(n) = \Theta(g(n))$

Sección 3.6

Si la pareja a, b, $a > b$, requiere $n \geq 1$ divisiones al servir como entrada del algoritmo de Euclides, entonces $a \geq f_{n+1}$ y $b \geq f_n$, donde $\{f_n\}$ denota la sucesión de Fibonacci.

Si los enteros en el rango de 0 a m, $m \geq 8$, no ambos cero, se utilizan como entrada en el algoritmo de Euclides, entonces se necesitan a lo más

$$\log_{3/2} \frac{2m}{3}$$

divisiones.

Sección 3.7

Criptología
Criptosistema
Cifrar un mensaje
Descifrar un mensaje
Criptosistema RSA con clave pública: Para cifrar a y enviarla al poseedor de la clave pública z, n, se calcula $c = a^n \bmod z$, y se envía c. Para descifrar el mensaje se calcula $c^s \bmod z$, lo cual es igual a a, como puede demostrarse.
$ab \bmod z = [(a \bmod z)(b \bmod z)] \bmod z$
La seguridad del sistema de cifrado RSA se basa principalmente en el hecho de que hasta la fecha no existe un algoritmo eficiente conocido para factorizar enteros.

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 3.1

1. Rastree el algoritmo "find max" de la sección 3.1 para los valores $a = 12$, $b = 3$ y $c = 0$.
2. Escriba un algoritmo que reciba como entrada los números distintos a, b y c y asigne los valores a, b y c a las variables x, y y z de modo que
$$x < y < z.$$
3. Escriba un algoritmo que proporcione como salida "Sí" si los valores de a, b y c son distintos, y "No" en caso contrario.
4. ¿Cuáles de las propiedades de precisión, unicidad, carácter finito, entrada, salida y generalidad fallan en este ejemplo? Explique.

Entrada: S, un conjunto de enteros; m, un entero
Salida: Todos los subconjuntos de S que suman m

1. Enumere todos los subconjuntos de S y sus sumas.
2. Recorra los subconjuntos enumerados en 1 y proporcione como salida aquellos cuya suma sea m.

Sección 3.2

5. Rastree el algoritmo 3.2.2 para la entrada
$$s_1 = 7, \quad s_2 = 9, \quad s_3 = 17, \quad s_4 = 7.$$
6. Escriba un algoritmo que reciba como entrada la matriz de una relación R y verifique si R es simétrica.
7. Escriba un algoritmo que reciba como entrada la matriz A $n \times n$ y que proporcione como salida la transpuesta A^T.
8. Escriba un algoritmo que reciba como entrada la sucesión
$$s_1, \ldots, s_n$$
ordenada de manera creciente y que imprima los valores que aparecen más de una vez. *Ejemplo*: Si la sucesión fuese
1 1 1 5 8 8 9 12
la salida debería ser
1 8.

Sección 3.3

9. Si $a = 333$ y $b = 24$, determine enteros q y r tales que $a = bq + r$, con $0 \leq r < b$.
10. Utilice el algoritmo de Euclides para determinar el máximo común divisor de los enteros 396 y 480.
11. Utilice el algoritmo de Euclides para determinar el máximo común divisor de los enteros 2390 y 4326.
12. Llene el espacio en blanco para obtener una afirmación verdadera: Si a y b son enteros que satisfacen $a > b > 0$ y $a = bq + r$, $0 \leq r < b$, entonces $\text{mcd}(a, b) =$ ——.

Sección 3.4

13. Rastree el algoritmo 3.4.4 (el algoritmo para cubrir con trominós) cuando $n = 8$ y el cuadrado faltante está a cuatro cuadrados de la izquierda y a dos cuadrados de la parte superior.

Los ejercicios 14-16 se refieren a la *sucesión de tribonacci* (Fibonacci de orden tres) definida mediante las ecuaciones
$$t_1 = t_2 = t_3 = 1; \qquad t_n = t_{n-1} + t_{n-2} + t_{n-3}, \qquad n \geq 4.$$

14. Determine t_4 y t_5.
15. Escriba un algoritmo recursivo para calcular t_n, $n \geq 1$.
16. Proporcione una demostración por inducción matemática de que su algoritmo para el ejercicio 15 es correcto.

os

Sección 3.5

Seleccione una notación theta entre $\Theta(1)$, $\Theta(n)$, $\Theta(n^2)$, $\Theta(n^3)$, $\Theta(n^4)$, $\Theta(2^n)$ o $\Theta(n!)$ para cada una de las expresiones en los ejercicios 17 y 18.

17. $4n^3 + 2n - 5$ **18.** $1^3 + 2^3 + \cdots + n^3$

19. Seleccione una notación theta entre $\Theta(1)$, $\Theta(n)$, $\Theta(n^2)$, $\Theta(n^3)$, $\Theta(2^n)$ o $\Theta(n!)$ para el número de veces que se ejecuta la línea $x := x + 1$.

```
for i := 1 to n do
  for j := 1 to n do
    x := x + 1
```

20. Escriba un algoritmo que verifique si dos matrices $n \times n$ son iguales y determine una notación theta para el peor de los casos.

Sección 3.6

21. ¿Exactamente cuántas divisiones necesita el algoritmo de Euclides en el peor de los casos, para números en el rango de 0 a 1000?

22. ¿Exactamente cuántas divisiones necesita el algoritmo de Euclides para calcular mcd(2, 76652913)?

23. ¿Exactamente cuántas divisiones necesita el algoritmo de Euclides para calcular mcd(f_{324}, f_{323})? ($\{f_n\}$ denota la sucesión de Fibonacci.)

24. Dado que $\log_{3/2}100 = 11.357747$, proporcione una cota superior para el número de divisiones necesarias para el algoritmo de Euclides, para enteros en el rango de 0 a 100,000,000.

Sección 3.7

En los ejercicios 25-28, suponga que elegimos los primos $p = 13$, $q = 17$ y $n = 19$.

25. Calcule z y ϕ.

26. Verifique que $s = 91$.

27. Cifre 144 utilizando la clave pública z, n.

28. Descifre 28.

4

MÉTODOS DE CONTEO Y EL PRINCIPIO DE LA PICHONERA

4.1 PRINCIPIOS BÁSICOS
RINCÓN DE SOLUCIÓN DE PROBLEMAS: CONTEO
4.2 PERMUTACIONES Y COMBINACIONES
RINCÓN DE SOLUCIÓN DE PROBLEMAS: COMBINACIONES
4.3 ALGORITMOS PARA GENERAR PERMUTACIONES Y COMBINACIONES
4.4 PERMUTACIONES Y COMBINACIONES GENERALIZADAS
4.5 COEFICIENTES BINOMIALES E IDENTIDADES COMBINATORIAS
4.6 EL PRINCIPIO DE LA PICHONERA
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Sobre existen demasiadas mitos en una baraja de naipes.

de Shane

En muchos problemas discretos debemos enfrentar un problema de conteo. Por ejemplo, en la sección 3.5 vimos que para estimar el tiempo de ejecución de un algoritmo, había que contar el número de veces que ciertos pasos o ciclos se ejecutaban. El conteo también tiene un papel crucial en la teoría de la probabilidad. Debido a la importancia del conteo, se han desarrollado varias formas para realizarlo, algunas de ellas un tanto complejas. En este capítulo desarrollaremos varias herramientas para el conteo. Estas técnicas pueden utilizarse para desarrollar el teorema del binomio. El capítulo concluye con un análisis del principio de la pichonera, el cual permite, con frecuencia, demostrar la existencia de un objeto con ciertas propiedades.

4.1 *PRINCIPIOS BÁSICOS*

El menú de Quick Lunch de Kay aparece en la figura 4.1.1. Como puede verse, tiene dos entradas, tres platos principales, y cuatro bebidas. ¿Cuántas comidas diferentes constan de un plato principal y una bebida?

ENTRADAS	
Nachos (N)	2.15
Ensalada (E)	1.90
PLATOS PRINCIPALES	
Hamburgesa (H)	3.25
Hamburguesa con queso (Q)	3.65
Filete de pescado	3.15
BEBIDAS	
Té	0.70
Leche	0.85
Cola	0.75
Cerveza de raíz	0.75

FIGURA 4.1.1
Menú del Quick Lunch de Kay.

Si enumeramos todas las comidas posibles que constan de un plato principal y una bebida:

HT, HL, HC, HR, QT, QL, QC, QR, FT, FL, FC, FR,

vemos que hay 12 comidas diferentes. (La comida que consta de un plato principal cuya primera letra es X y una bebida cuya primera letra es Y se denota XY. Por ejemplo, QR se refiere a la comida que consta de una hamburguesa con queso y cerveza de raíz.) Observe que existen tres platos principales y cuatro bebidas, y que $12 = 3 \cdot 4$.

Existen 24 posibles comidas que constan de una entrada, un plato principal y una bebida:

NHT, NHL, NHC, NHR, NQT, NQL, NQC, NQR,

NFT, NFL, NFC, NFR, EHT, EHL, EHC, EHR,

EQT, EQL, EQC, EQR, EFT, EFL, EFC, EFR.

(La comida que consta de una entrada cuya primera letra es X, un plato principal cuya primera letra es Y y una bebida cuya primera letra es Z se denota XYZ.) Observe que hay dos entradas, tres platos principales y cuatro bebidas, y que $24 = 2 \cdot 3 \cdot 4$.

En cada uno de estos ejemplos vimos que el número total de comidas era igual al producto de los números de cada platillo. Estos ejemplos ilustran el **principio de multiplicación**.

PRINCIPIO DE MULTIPLICACIÓN

Si una actividad puede construirse en t pasos sucesivos y el paso 1 puede realizarse de n_1 formas; el paso 2 puede realizarse de n_2 formas; ...; y el paso t puede realizarse de n_t formas, entonces el número de diferentes actividades posibles es $n_1 \cdot n_2 \cdots n_t$.

En el problema de conteo de la cantidad de comidas que constan de un plato principal y una bebida, el primer paso consiste en "elegir el plato principal" y el segundo paso es "elegir la bebida". Así, $n_1 = 3$ y $n_2 = 4$, y, por el principio de multiplicación, el número total de comidas es $3 \cdot 4 = 12$. La figura 4.1.2 muestra por qué multiplicamos 3 por 4 (tenemos tres grupos de cuatro objetos).

Podemos resumir el principio de multiplicación diciendo que multiplicamos el número de formas de realizar cada paso cuando una actividad se construye mediante pasos sucesivos.

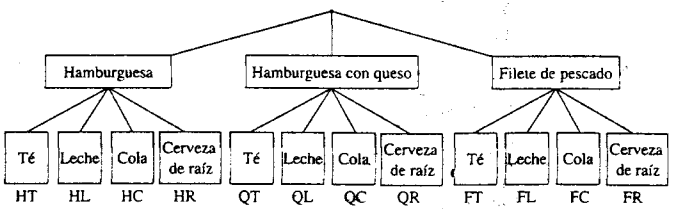


FIGURA 4.1.2 Una ilustración del principio de multiplicación.

EJEMPLO 4.1.1

¿Cuántas comidas están disponibles en Quick Lunch que consten de un plato principal y una bebida *opcional*?

Podemos formar una comida que consta de un plato principal y una bebida opcional mediante un proceso de dos pasos. El primer paso consiste en "elegir el plato principal" y el segundo paso es "elegir una bebida opcional". Existen $n_1 = 3$ formas de elegir el plato principal (hamburguesa, hamburguesa con queso, filete de pescado) y $n_2 = 5$ formas de elegir la bebida opcional (té, leche, cola, cerveza de raíz, ninguna). Por el principio de multiplicación, existen $3 \cdot 5 = 15$ comidas. Como confirmación, estas comidas se enumeran a continuación:

HT, HL, HC, HR, HN, QT, QL,

QC, QR, QN, FT, FL, FC, FR, FN. □

EJEMPLO 4.1.2

(a) ¿Cuántas cadenas de longitud 4 pueden formarse mediante las letras *ABCDE* si no se permiten repeticiones?

(b) ¿Cuántas cadenas de la parte (a) comienzan con la letra *B*?

(c) ¿Cuántas cadenas de la parte (a) no comienzan con la letra *B*?

(a) Utilizamos el principio de multiplicación. Una cadena de longitud 4 puede construirse en cuatro pasos sucesivos: Se elige la primera letra, luego la segunda, luego la tercera y finalmente la cuarta. La primera letra puede escogerse de cinco maneras. Una vez elegida la primera letra, la segunda puede seleccionarse de cuatro formas. Una vez elegida la segunda letra, la tercera puede escogerse de tres formas. Una vez elegida la tercera letra, la cuarta puede seleccionarse de dos formas. Por el principio de multiplicación, existen

$$5 \cdot 4 \cdot 3 \cdot 2 = 120$$

cadenas.

(b) Las cadenas que comienzan con la letra *B* pueden construirse en cuatro pasos: Se elige la primera letra, luego la segunda, luego la tercera y por último la cuarta. La primera letra (*B*) puede escogerse de una manera, la segunda de cuatro formas, la tercera de tres y la cuarta de dos formas. Así, por el principio de multiplicación, existen

$$1 \cdot 4 \cdot 3 \cdot 2 = 24$$

cadenas que comienzan con la letra *B*.

(c) La parte (a) muestra que existen 120 cadenas de longitud 4 que pueden formarse mediante las letras *ABCDE* y la parte (b) muestra que 24 de ellas comienzan con la letra *B*. Esto implica que existen

$$120 - 24 = 96$$

cadenas que no comienzan con la letra *B*. □

EJEMPLO 4.1.3

En una imagen digital, queremos codificar la cantidad de luz en cada punto como una cadena de ocho bits. ¿Cuántos valores son posibles en cada punto?

Podemos construir un código de ocho bits en ocho pasos: Se elige el primer bit, luego el segundo, . . . , y finalmente el octavo. Como existen dos formas de escoger cada bit, por el principio de multiplicación el número total de códigos de ocho bits es

$$2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^8 = 256.$$ □

A continuación daremos una demostración, utilizando el principio de multiplicación, de que un conjunto con n elementos tiene 2^n subconjuntos. Anteriormente dimos una demostración de este resultado utilizando inducción matemática (teorema 2.1.4).

EJEMPLO 4.1.4

Utilice el principio de multiplicación para mostrar que un conjunto $\{x_1, \ldots, x_n\}$ con n elementos tiene 2^n subconjuntos.

Un subconjunto puede construirse en n pasos: Se elige o no x_1, se elige o no x_2, . . . , se elige o no x_n. Cada paso puede realizarse de dos formas. Así, el número de subconjuntos posibles es

$$\underbrace{2 \cdot 2 \cdots 2}_{n \text{ factores}} = 2^n.$$ □

EJEMPLO 4.1.5

Sea X un conjunto con n elementos. ¿Cuántos pares ordenados (A, B) satisfacen $A \subseteq B \subseteq X$?

Dado un par ordenado (A, B) que satisface $A \subseteq B \subseteq X$, cada elemento de X está exactamente en alguno de los conjuntos A, $B - A$, o $X - B$. Recíprocamente, si asignamos cada elemento de X a alguno de los tres conjuntos A (y por hipótesis, también a B y X), $B - A$ (y por hipótesis, también a X), o $X - B$, obtenemos un único par ordenado (A, B) que satisface $A \subseteq B \subseteq X$. Así, el número de pares ordenados (A, B) que satisfacen $A \subseteq B \subseteq X$ es igual al número de formas de asignar los elementos de X a los tres conjuntos A, $B - A$ y $X - B$. Podemos realizar tales asignaciones mediante el siguiente proceso de n pasos: Asignamos el primer elemento de X a A, $B - A$ o $X - B$; asignamos el segundo elemento de X a A, $B - A$ o $X - B$; . . . ; asignamos el n-ésimo elemento de X a A, $B - A$ o $X - B$. Como cada paso puede realizarse de tres formas, el número de pares ordenados (A, B) que satisfacen $A \subseteq B \subseteq X$ es

$$\underbrace{3 \cdot 3 \cdots 3}_{n \text{ factores}} = 3^n.$$ □

A continuación ilustraremos el principio de la suma mediante un ejemplo, para después presentar el principio.

EJEMPLO 4.1.6

¿Cuántas cadenas de ocho bits comienzan con 101 o 111?

Una cadena de ocho bits que comienza con 101 puede construirse en cinco pasos: Se elige el cuarto bit, se elige el quinto bit, . . . , se elige el octavo bit. Como cada uno de los cinco bits puede elegirse de dos formas, por el principio de multiplicación, existen

$$2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^5 = 32$$

cadenas de ocho bits que comienzan con 101. Podemos utilizar el mismo argumento para mostrar que existen 32 cadenas de ocho bits que comienzan con 111. Como existen 32 cadenas de ocho bits que comienzan con 101 y 32 cadenas de ocho bits que comienzan con 111, existen $32 + 32 = 64$ cadenas de ocho bits que comienzan con 101 o con 111. □

En el ejemplo 4.1.6 sumamos los números de cadenas de ocho bits (32 y 32) de cada tipo para determinar el resultado final. El **principio de la suma** nos dice cuándo debemos sumar para calcular el número total de posibilidades.

PRINCIPIO DE LA SUMA

Supóngase que $X_1, \ldots, X_t$ son conjuntos y que el i-ésimo conjunto X_i tiene n_i elementos. Si $\{X_1, \ldots, X_t\}$ es una familia ajena por pares (es decir, si $i \neq j$, $X_i \cap X_j = \emptyset$), el número de elementos posibles que pueden elegirse de X_1 o X_2 o . . . o X_t es

$$n_1 + n_2 + \cdots + n_t.$$

(En forma equivalente, la unión $X_1 \cup X_2 \cup \cdots \cup X_t$ contiene $n_1 + n_2 + \cdots + n_t$ elementos.)

En el ejemplo 4.1.6, X_1 podría ser el conjunto de cadenas de ocho bits que comienzan con 101 y X_2 el conjunto de cadenas de ocho bits que comienzan con 111. Como X_1 es ajeno de X_2, de acuerdo con el principio de la suma, el número de cadenas de ocho bits de cualquiera de estos tipos, que es el número de elementos en $X_1 \cup X_2$, es $32 + 32 = 64$.

Podemos resumir el principio de la suma diciendo que sumamos la cantidad de elementos en cada subconjunto cuando los elementos por contar pueden descomponerse en subconjuntos ajenos.

Si estamos contando objetos que se construyen por pasos, utilizamos el principio de multiplicación. Si tenemos conjuntos ajenos de objetos y queremos conocer la cantidad total de objetos, utilizamos el principio de la suma. Es importante reconocer cuándo aplicar cada principio. Esta habilidad se adquiere con la práctica y un razonamiento cuidadoso en cada problema.

Concluimos esta sección con ejemplos que ilustran ambos principios de conteo.

EJEMPLO 4.1.7

¿De cuántas formas podemos elegir dos libros de diferentes materias entre cinco libros distintos de computación, tres libros distintos de matemáticas y dos libros distintos de arte?

Por el principio de multiplicación, vemos que podemos elegir dos libros, uno de computación y uno de matemáticas, de $5 \cdot 3 = 15$ formas.

De manera análoga, podemos elegir dos libros, uno de computación y uno de arte, de $5 \cdot 2 = 10$ formas, y podemos elegir dos libros, uno de matemáticas y uno de arte, de $3 \cdot 2 = 6$ formas. Como estos conjuntos de selecciones son ajenos por pares, podemos utilizar el principio de la suma para concluir que existen

$$15 + 10 + 6 = 31$$

formas de elegir dos libros de diversos temas entre los libros de computación, matemáticas y arte. □

EJEMPLO 4.1.8

Un comité de seis personas compuesto por Alice, Ben, Connie, Dolph, Egbert y Francisco debe elegir un presidente, un secretario y un tesorero.

(a) ¿De cuántas formas puede hacerse esto?

(b) ¿De cuántas formas puede hacerse esto si debe ser presidente Alice o Ben?

(c) ¿De cuántas formas puede hacerse esto si Egbert debe ocupar uno de los puestos?

(d) ¿De cuántas formas puede hacerse esto si Dolph y Francisco deben tener alguno de los puestos?

(a) Utilizamos el principio de multiplicación. Los ocupantes de los puestos pueden elegirse en tres pasos: Se elige el presidente, luego el secretario y finalmente el tesorero. El presidente puede elegirse de seis formas. Una vez electo el presidente, el secretario puede elegirse de cinco formas. Después de la selección del presidente y del secretario, el tesorero puede elegirse de cuatro formas. Por tanto, el número total de posibilidades es

$$6 \cdot 5 \cdot 4 = 120.$$

(b) Argumentaremos como en la parte (a): Si Alice es presidente, existen $5 \cdot 4 = 20$ formas de seleccionar a los demás ocupantes de los puestos. De manera análoga, si Ben es presidente, existen 20 formas de elegir a los demás. Como estos casos son ajenos, por el principio de la suma, existen

$$20 + 20 = 40$$

posibilidades.

(c) [Primera solución.] Argumentamos como en la parte (a): Si Egbert es presidente, existen 20 formas de elegir a los demás. De manera análoga, si Egbert es secretario, existen 20 posibilidades, y si Egbert es tesorero, existen 20 posibilidades. Como estos tres casos son ajenos por pares, por el principio de la suma, existen

$$20 + 20 + 20 = 60$$

posibilidades.

[Segunda solución.] Vemos que la actividad de asignar a Egbert y otras dos personas los puestos se conforma con tres pasos: Asignar a Egbert un puesto, ocupar el siguiente puesto más alto, ocupar el último puesto restante. Existen tres formas de asignar a Egbert un puesto. Una vez asignado el puesto a Egbert, existen tres formas de ocupar el siguiente puesto más alto. Una vez asignados los puestos de Egbert y el siguiente puesto más alto, existen cuatro formas de ocupar el último puesto restante. Por el principio de multiplicación, existen

$$3 \cdot 5 \cdot 4 = 60$$

posibilidades.

(d) Vemos que la actividad de asignar puestos a Dolph, Francisco y alguna otra persona consta de tres pasos: Asignar un puesto a Dolph, asignar un puesto a Francisco, ocupar el puesto restante. Existen tres formas de asignar un puesto a Dolph. Una vez asignado un puesto a Dolph, existen dos formas de asignar un puesto a Francisco. Una vez asignados puestos a Dolph y a Francisco, existen cuatro formas de ocupar el puesto restante. Por el principio de multiplicación, existen

$$3 \cdot 2 \cdot 4 = 24$$

posibilidades. □

Ejercicios

Determine el número de comidas en Quick Lunch (figura 4.1.1) que satisfagan las condiciones de los ejercicios 1-3.

1. Una entrada y una bebida.
2. Una entrada, un plato principal y una bebida opcional.
3. Una entrada opcional, un plato principal y una bebida opcional.
4. Un hombre tiene ocho camisas, cuatro pares de pantalones y cinco pares de zapatos. ¿Cuántas combinaciones de ropa puede hacer?
5. Las opciones disponibles para un modelo particular de automóvil son cinco colores para el interior, seis colores para el exterior, dos tipos de asientos, tres tipos de motor y tres tipos de radios. ¿Cuántas posibilidades diferentes tiene un consumidor?
6. El sistema Braille para representación de caracteres fue desarrollado a principios del siglo XIX por Louis Braille. Los caracteres, utilizados por los ciegos, constan de puntos en altorrelieve. Las posiciones de los puntos se eligen en dos columnas verticales, de tres puntos cada una. Debe aparecer al menos un punto en altorrelieve. ¿Cuántos caracteres Braille son posibles?

En los ejercicios 7-15 se lanzan dos dados, uno azul y otro rojo.

7. ¿Cuántos resultados son posibles?
8. ¿En cuántos resultados la suma es 4?
9. ¿Cuántos resultados son dobles? (Un doble ocurre cuando ambos dados muestran el mismo número.)

10. ¿En cuántos resultados la suma es 7 u 11?

11. ¿En cuántos resultados el dado azul muestra el 2?

12. ¿En cuántos resultados ocurre que exactamente uno de los dados muestre un 2?

13. ¿En cuántos resultados al menos uno de los dados muestra un 2?

14. ¿En cuántos resultados ninguno de los dados muestra un 2?

15. ¿En cuántos resultados se obtiene una suma par?

En los ejercicios 16-18, suponga que existen 10 caminos de Oz a Mid Earth y cinco caminos de Mid Earth a la Isla de la Fantasía.

16. ¿Cuántas rutas existen de Oz a la Isla de la Fantasía que pasen por Mid Earth?

17. ¿Cuántas rutas de viaje redondo son de la forma Oz - Mid Earth - Isla de la Fantasía - Mid Earth - Oz?

18. ¿Cuántas rutas de viaje redondo son de la forma Oz - Mid Earth - Isla de la Fantasía - Mid Earth - Oz y en las cuales el viaje de regreso no sea igual a la ruta original de Oz a la Isla de la Fantasía?

19. ¿Cuántas placas de automóvil pueden construirse si las placas contienen tres letras seguidas de dos dígitos, y se permiten las repeticiones? ¿Y si no se permiten repeticiones?

20. ¿Cuántas cadenas de ocho bits comienzan con 1100?

21. ¿Cuántas cadenas de ocho bits comienzan y terminan con 1?

22. ¿Cuántas cadenas de ocho bits tienen el segundo o el cuarto bit (o ambos) igual a 1?

23. ¿Cuántas cadenas de ocho bits tienen exactamente un 1?

24. ¿Cuántas cadenas de ocho bits tienen exactamente dos 1?

25. ¿Cuántas cadenas de ocho bits tienen al menos un 1?

26. ¿Cuántas cadenas de ocho bits se leen igual en ambas direcciones? (Un ejemplo de este tipo de cadena es 01111110. Tales cadenas se llaman *palíndromos*.*)

En los ejercicios 27-32, un comité de seis personas compuesto por Alice, Ben, Connie, Dolph, Egbert y Francisco debe elegir un presidente, un secretario y un tesorero.

27. ¿Cuántas selecciones excluyen a Connie?

28. ¿En cuántas selecciones Ben y Francisco no tienen puestos?

29. ¿En cuántas selecciones Ben y Francisco tienen puestos?

30. ¿En cuántas selecciones Dolph tiene un puesto y Francisco no?

31. ¿En cuántas selecciones Dolph es presidente o no tiene un puesto?

32. ¿En cuántas selecciones Ben es presidente o tesorero?

* Palíndromo es una palabra que se lee igual de izquierda a derecha que de derecha a izquierda.

En los ejercicios 33-40, las letras $ABCDE$ se utilizan para formar cadenas de longitud 3.

33. ¿Cuántas cadenas pueden formarse si permitimos repeticiones?

34. ¿Cuántas cadenas pueden formarse si no permitimos repeticiones?

35. ¿Cuántas cadenas comienzan con A, permitiendo repeticiones?

36. ¿Cuántas cadenas comienzan con A, sin permitir repeticiones?

37. ¿Cuántas cadenas no contienen a A, permitiendo repeticiones?

38. ¿Cuántas cadenas no contienen a A, sin permitir repeticiones?

39. ¿Cuántas cadenas contienen a A, permitiendo repeticiones?

40. ¿Cuántas cadenas contienen a A, sin permitir repeticiones?

Los ejercicios 41-51 se refieren a los enteros del 5 al 200, inclusive.

41. ¿Cuántos números de este tipo existen?

42. ¿Cuántos son pares?

43. ¿Cuántos son impares?

44. ¿Cuántos son divisibles entre 5?

45. ¿Cuántos son mayores que 72?

46. ¿Cuántos constan de dígitos distintos?

47. ¿Cuántos contienen el dígito 7?

48. ¿Cuántos no contienen al dígito 0?

49. ¿Cuántos son mayores que 101 y no contienen al dígito 6?

50. ¿Cuántos tienen los dígitos en orden estrictamente creciente? (Por ejemplo, 13, 147, 8.)

51. ¿Cuántos son de la forma xyz, donde $0 \neq x < y$ y $y > z$?

52. (a) ¿De cuántas formas pueden ser distintos los meses de los cumpleaños de cinco personas?

(b) ¿Cuántas posibilidades existen para los meses de los cumpleaños de cinco personas?

(c) ¿De cuántas formas pueden tener su cumpleaños en el mismo mes al menos dos personas de las cinco?

Los ejercicios 53-57 se refieren a un conjunto de cinco libros distintos de computación, tres libros distintos de matemáticas y dos libros distintos de arte.

53. ¿De cuántas formas pueden ordenarse estos libros en un estante?

54. ¿De cuántas formas pueden ordenarse estos libros en un estante si los cinco libros de computación debe estar a la izquierda y los dos libros de arte deben estar a la derecha?

55. ¿De cuántas formas pueden ordenarse estos libros en un estante si los cinco libros de computación debe estar a la izquierda?

56. ¿De cuántas formas pueden ordenarse estos libros en un estante si todos los libros de la misma disciplina se deben colocar juntos?

☆ 57. ¿De cuántas formas pueden ordenarse estos libros en un estante si los dos libros de arte no deben colocarse juntos?

58. En algunas versiones de FORTRAN, un identificador consta de una cadena de uno a seis caracteres alfanuméricos que comienzan con una letra. (Un carácter *alfanumérico* es una letra, de la A a la Z, o un dígito, del 0 al 9.) ¿Cuántos identificadores válidos en FORTRAN existen?

59. Si X es un conjunto con n elementos y Y es un conjunto con m elementos, ¿cuántas funciones existen de X en Y?

★ **60.** Existen 10 copias de un libro y una copia de otros 10 libros. ¿De cuántas formas podemos elegir 10 libros?

61. ¿Cuántos términos hay en el desarrollo de

$$(x + y)(a + b + c)(e + f + g)(h + i)\,?$$

☆ 62. ¿Cuántos subconjuntos de un conjunto con $(2n + 1)$ elementos tienen n elementos o menos?

63. ¿Cuántas relaciones antisimétricas existen sobre un conjunto con n elementos?

64. Si X y Y no son subconjuntos ajenos, no podemos sumar $|X|$ con $|Y|$ para calcular el número de elementos en $X \cup Y$. Demuestre que

$$|X \cup Y| = |X| + |Y| - |X \cap Y|$$

para cualesquiera conjuntos X y Y.

Utilice el resultado del ejercicio 64 para resolver los ejercicios 65-69.

65. ¿Cuántas cadenas de ocho bits comienzan con 100 o tienen el cuarto bit igual a 1?

66. ¿Cuántas cadenas de ocho bits comienzan con 1 o terminan con 1?

En los ejercicios 67 y 68, un comité de seis personas compuesto por Alice, Ben, Connie, Dolph, Egbert y Francisco debe elegir un presidente, un secretario y un tesorero.

67. ¿En cuántas selecciones Ben es presidente o Alice es secretario?

68. ¿En cuántas selecciones Connie es presidente o Alice tiene un puesto?

69. Se arrojan dos dados, uno azul y otro rojo. ¿En cuántos resultados el dado azul muestra un 3 o se obtiene una suma par?

70. ¿Cuántos operadores binarios existen sobre $\{1, 2, \ldots, n\}$?

71. ¿Cuántos operadores binarios conmutativos existen sobre $\{1, 2, \ldots, n\}$?

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

CONTEO

Problema

Determinar el número de ternas o tercias ordenadas de conjuntos X_1, X_2, X_3 que satisfagan

$$X_1 \cup X_2 \cup X_3 = \{1, 2, 3, 4, 5, 6, 7, 8\} \text{ y } X_1 \cap X_2 \cap X_3 = \emptyset.$$

Por *terna ordenada* entendemos que el orden de los conjuntos X_1, X_2, X_3 se toma en cuenta. Por ejemplo, las ternas

$$\{1, 2, 3\},\ \{1, 4, 8\},\ \{2, 5, 6, 7\}$$

y

$$\{1, 4, 8\},\ \{1, 2, 3\},\ \{2, 5, 6, 7\}$$

se consideran distintas.

Para resolver el problema

Sería bueno comenzar a enumerar las ternas, pero son tantas que difícilmente se lograría cierta intuición a partir del análisis de unas cuantas ternas. Simplificaremos el problema haciéndolo más pequeño. Reemplacemos

$$\{1, 2, 3, 4, 5, 6, 7, 8\}$$

por $\{1\}$. ¿Qué podría ser más sencillo que $\{1\}$? (Bueno, tal vez $\emptyset$, ¡pero es demasiado sencillo!) Ahora podemos enumerar todas las ternas ordenadas de conjuntos X_1, X_2, X_3 que satisfacen $X_1 \cup X_2 \cup X_3 = \{1\}$ y $X_1 \cap X_2 \cap X_3 = \emptyset$. Debemos colocar 1 en al menos uno de los conjuntos X_1, X_2, X_3 (de modo que la unión sea $\{1\}$), pero no debemos colocar 1 en los tres conjuntos X_1, X_2, X_3 (ya que en caso contrario, la intersección no sería vacía). Así, 1 estará en exactamente uno o dos de los conjuntos X_1, X_2, X_3. La lista completa de ternas ordenadas es:

$$X_1 = \{1\},\ X_2 = \emptyset,\ X_3 = \emptyset;\quad X_1 = \emptyset,\ X_2 = \{1\},\ X_3 = \emptyset;$$
$$X_1 = \emptyset,\ X_2 = \emptyset,\ X_3 = \{1\};\quad X_1 = \{1\},\ X_2 = \{1\},\ X_3 = \emptyset;$$
$$X_1 = \{1\},\ X_2 = \emptyset,\ X_3 = \{1\};\quad X_1 = \emptyset,\ X_2 = \{1\},\ X_3 = \{1\}.$$

Así, existen seis ternas ordenadas de conjuntos X_1, X_2, X_3 que satisfacen

$$X_1 \cup X_2 \cup X_3 = \{1\} \quad \text{y} \quad X_1 \cap X_2 \cap X_3 = \emptyset.$$

Pasemos a otro nivel y enumeremos todas las ternas ordenadas de conjuntos X_1, X_2, X_3 que satisfacen $X_1 \cup X_2 \cup X_3 = \{1, 2\}$ y $X_1 \cap X_2 \cap X_3 = \emptyset$. Como antes, debemos colocar 1 en al menos uno de los conjuntos X_1, X_2, X_3 (de modo que 1 esté en la unión), pero no debemos colocar 1 en los tres conjuntos X_1, X_2, X_3 (en caso contrario, la intersección no sería vacía). Esta vez, también debemos colocar a 2 en al menos uno de los conjuntos X_1, X_2, X_3 (de modo que 2 esté también en la unión), pero no debemos colocar a 2 en los tres conjuntos X_1, X_2, X_3 (en caso contrario, la intersección no sería vacía). Así, cada uno de los elementos 1 y 2 estará exactamente en uno o dos de los conjuntos X_1, X_2, X_3. Enumeramos los conjuntos de manera sistemática, de modo que podamos reconocer algún patrón. La lista completa de ternas ordenadas es:

1 está en	*2 está en*	*1 está en*	*2 está en*	*1 está en*	*2 está en*
X_1	X_1	X_1	X_2	X_1	X_3
X_2	X_1	X_2	X_2	X_2	X_3
X_3	X_1	X_3	X_2	X_3	X_3
X_1, X_2	X_1	X_1, X_2	X_2	X_1, X_2	X_3
X_1, X_3	X_1	X_1, X_3	X_2	X_1, X_3	X_3
X_2, X_3	X_1	X_2, X_3	X_2	X_2, X_3	X_3
X_1	X_1, X_2	X_1	X_1, X_3	X_1	X_2, X_3
X_2	X_1, X_2	X_2	X_1, X_3	X_2	X_2, X_3
X_3	X_1, X_2	X_3	X_1, X_3	X_3	X_2, X_3
X_1, X_2	X_1, X_2	X_1, X_2	X_1, X_3	X_1, X_2	X_2, X_3
X_1, X_3	X_1, X_2	X_1, X_3	X_1, X_3	X_1, X_3	X_2, X_3
X_2, X_3	X_1, X_2	X_2, X_3	X_1, X_3	X_2, X_3	X_2, X_3

Por ejemplo, la entrada superior izquierda, X_1 X_1, especifica que 1 está en X_1 y 2 está en X_1, por tanto, esta entrada proporciona la terna ordenada

$$X_1 = \{1, 2\}, \quad X_2 = \emptyset, \quad X_3 = \emptyset.$$

Como se muestra, existen 36 ternas ordenadas de conjuntos X_1, X_2, X_3 que satisfacen

$$X_1 \cup X_2 \cup X_3 = \{1, 2\} \quad \text{y} \quad X_1 \cap X_2 \cap X_3 = \emptyset.$$

Vemos que existen seis formas de asignar 1 a los conjuntos X_1, X_2, X_3, lo cual implica la existencia de seis líneas por bloque. De manera análoga, existen seis formas de asignar 2 a los conjuntos X_1, X_2, X_3, por lo cual existen seis bloques.

Antes de continuar, ¿podría usted estimar el número de ternas ordenadas de conjuntos X_1, X_2, X_3 que satisfacen

$$X_1 \cup X_2 \cup X_3 = \{1, 2, 3\} \quad \text{y} \quad X_1 \cap X_2 \cap X_3 = \emptyset?$$

Aquí surge el patrón. Si $X = \{1, 2, \ldots, n\}$, existen seis formas de asignar cada uno de los elementos $1, 2, \ldots, n$ a los conjuntos X_1, X_2, X_3. Por el principio de multiplicación, el número de ternas ordenadas es 6^n.

Determinación de otra solución

Hemos determinado una solución al problema partiendo de un problema más sencillo y luego descubriendo y justificando el patrón que apareció.

Otro enfoque consiste en analizar un problema similar e imitar su solución. El problema del ejemplo 4.1.5 es similar a nuestro problema en cuestión, en el sentido de que también es un problema de conteo relacionado con conjuntos:

> Sea X un conjunto con n elementos. ¿Cuántos pares ordenados (A, B) satisfacen $A \subseteq B \subseteq X$?

(En este momento, sería bueno regresar a releer el ejemplo 4.1.5.) La solución dada en el ejemplo 4.1.5 contaba el número de formas de asignar elementos de X con exactamente uno de los conjuntos A, $B - A$, o $X - B$.

Podemos resolver nuestro problema mediante un enfoque similar. Cada elemento de X está exactamente en uno de los conjuntos

$$\overline{X_1} \cap X_2 \cap X_3, \quad X_1 \cap \overline{X_2} \cap X_3, \quad X_1 \cap X_2 \cap \overline{X_3},$$
$$\overline{X_1} \cap \overline{X_2} \cap X_3, \quad \overline{X_1} \cap X_2 \cap \overline{X_3}, \quad X_1 \cap \overline{X_2} \cap \overline{X_3}.$$

Como cada miembro de X puede ser asignado a uno de estos conjuntos de seis formas, por el principio de multiplicación, el número de ternas ordenadas es 6^n.

Observe que aunque este enfoque de solución del problema es distinto al de la sección anterior, el argumento final es esencialmente el mismo.

Solución formal

Cada elemento en X está exactamente en uno de los conjuntos

$$Y_1 = \overline{X_1} \cap X_2 \cap X_3, \quad Y_2 = X_1 \cap \overline{X_2} \cap X_3, \quad Y_3 = X_1 \cap X_2 \cap \overline{X_3},$$
$$Y_4 = \overline{X_1} \cap \overline{X_2} \cap X_3, \quad Y_5 = \overline{X_1} \cap X_2 \cap \overline{X_3}, \quad Y_6 = X_1 \cap \overline{X_2} \cap \overline{X_3}.$$

Podemos construir una terna ordenada mediante el siguiente proceso de ocho pasos: se elige j, $1 \le j \le 6$, y se coloca 1 en Y_j; se elige j, $1 \le j \le 6$, y se coloca 2 en Y_j; ...; se elige j, $1 \le j \le 6$, y se coloca 8 en Y_j. Por ejemplo, para construir la terna

$$\{1, 2, 3\}, \quad \{1, 4, 8\}, \quad \{2, 5, 6, 7\},$$

primero elegimos $j = 3$ y colocamos 1 en $Y_3 = X_1 \cap X_2 \cap \overline{X_3}$. Luego elegimos $j = 2$ y colocamos 2 en $Y_2 = X_1 \cap \overline{X_2} \cap X_3$. Las demás elecciones de j son $j = 6, 5, 4, 4, 4, 5$.

Cada elección de j puede hacerse de seis formas. Por el principio de multiplicación, el número de ternas ordenadas es

$$6 \cdot 6 \cdot 6 \cdot 6 \cdot 6 \cdot 6 \cdot 6 \cdot 6 = 6^8 = 1{,}679{,}616.$$

Resumen de técnicas para resolver problemas

- Reemplazar el problema original con un problema más sencillo. Una forma de hacer esto consiste en reducir el tamaño del problema original.
- Enumerar directamente los elementos por contar.
- Enumerar los elementos de manera sistemática, de modo que surjan los patrones.
- Buscar patrones.
- Buscar un problema similar e imitar su solución.

4.2 PERMUTACIONES Y COMBINACIONES

Existen cuatro candidatos para un puesto de elección: Zeke, Yung, Xeno y Wilma. Para que las posiciones de los nombres en la boleta electoral no influyan sobre los votantes, es necesario imprimir boletas con los nombres enumerados en todos los órdenes posibles. ¿Cuántas boletas distintas puede haber?

Podemos utilizar el principio de multiplicación. Podemos formar una boleta en cuatro pasos: Se elige el primer nombre por enumerar, se elige el segundo nombre, se elige el tercer nombre, y finalmente se elige el cuarto. El primer nombre puede elegirse de cuatro formas. Una vez elegido el primer nombre, el segundo nombre puede elegirse de tres formas. Una vez elegido el segundo nombre, el tercero puede elegirse de dos formas. Una vez elegido el tercer nombre, el cuarto puede elegirse de una forma. Por el principio de multiplicación, el número de boletas es

$$4 \cdot 3 \cdot 2 \cdot 1 = 24.$$

Un ordenamiento de objetos, como los nombres en la boleta, es una **permutación**.

DEFINICIÓN 4.2.1

Una *permutación* de n elementos distintos $x_1, \ldots, x_n$ es un ordenamiento de los n elementos $x_1, \ldots, x_n$.

EJEMPLO 4.2.2

Existen seis permutaciones de tres elementos. Si los elementos se denotan como A, B, C, las seis permutaciones son

$$ABC, \quad ACB, \quad BAC, \quad BCA, \quad CAB, \quad CBA. \qquad \square$$

Hemos visto que existen 24 formas de ordenar cuatro candidatos en una boleta; así, existen 24 permutaciones de cuatro objetos. El método que utilizamos para contar el número de boletas distintas que contienen cuatro nombres puede utilizarse para deducir una fórmula para el número de permutaciones de n elementos.

La demostración del siguiente teorema para $n = 4$ se ilustra en la figura 4.2.1.

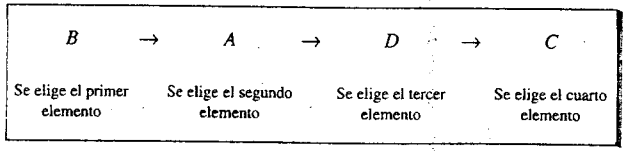


FIGURA 4.2.1 La demostración del teorema 4.2.3 para $n = 4$. Una permutación de $ABCD$ se construye seleccionando de manera sucesiva el primer elemento, luego el segundo, después el tercero y, finalmente, el cuarto elemento.

TEOREMA 4.2.3

Existen $n!$ permutaciones de n elementos.

Demostración. Utilizamos el principio de multiplicación. Una permutación de n elementos puede construirse en n pasos: Se elige el primer elemento, luego el segundo, . . . , y finalmente el último elemento. El primer elemento puede elegirse de n formas. Una vez elegido el primer elemento, el segundo elemento puede elegirse de $n - 1$ formas. Una vez elegido el segundo elemento, el tercer elemento puede elegirse de $n - 2$ formas, y así sucesivamente. Por el principio de multiplicación, existen

$$n(n-1)(n-2)\cdots 2 \cdot 1 = n!$$

permutaciones de n elementos. ■

EJEMPLO 4.2.4

Existen

$$10! = 10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 3{,}628{,}800$$

permutaciones de 10 elementos. □

EJEMPLO 4.2.5

¿Cuántas permutaciones de las letras $ABCDEF$ contiene la subcadena DEF?

Para garantizar la presencia del patrón DEF en la subcadena, estas tres letras deben estar juntas y en este orden. Las demás letras, A, B y C, pueden colocarse de manera arbitraria. Podemos pensar que construimos las permutaciones de las letras $ABCDEF$ que contienen el patrón DEF permutando cuatro elementos, uno con la etiqueta DEF y los otros con las etiquetas A, B y C (véase la figura 4.2.2). Por el teorema 4.2.3, existen 4! permutaciones de cuatro objetos. Así, el número de permutaciones de las letras $ABCDEF$ que contienen a la subcadena DEF es

$$4! = 24. \qquad \square$$

FIGURA 4.2.2
Permutación de cuatro elementos.

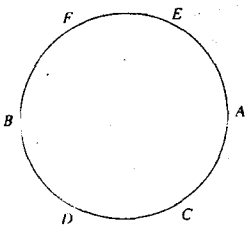
EJEMPLO 4.2.6

¿Cuántas permutaciones de las letras $ABCDEF$ contienen las letras DEF juntas en cualquier orden?

Podemos resolver este problema mediante un procedimiento de dos pasos: Se elige un ordenamiento de las letras DEF, se construye una permutación de $ABCDEF$ que contenga el orden dado de las letras DEF. Por el teorema 4.2.3, el primer paso puede realizarse de $3! = 6$ formas, y, de acuerdo con el ejemplo 4.2.5, el segundo paso puede realizarse de 24 formas. Por el principio de multiplicación, el número de permutaciones de las letras $ABCDEF$ que contienen las letras DEF juntas y con un orden arbitrario es

$$6 \cdot 24 = 144. \qquad \square$$

EJEMPLO 4.2.7



¿De cuántas formas pueden sentarse seis personas en torno de una mesa circular? Si una forma de sentarse se obtiene de otra forma de sentarse haciendo que cada persona se mueva n asientos en el sentido de las manecillas del reloj, dichas formas se consideran idénticas.

Denotemos las personas por A, B, C, D, E y F. Como las formas de sentarse obtenidas mediante rotaciones se consideran idénticas, podríamos sentar a A de manera arbitraria. Para sentar a las otras cinco personas, podemos ordenarlas y luego sentarlas en este orden, en el sentido de las manecillas del reloj, a partir de A. Por ejemplo, la permutación $CDBFE$ definiría la forma de sentarse de la figura anexa. Como existen $5! = 120$ permutaciones de cinco elementos, existen 120 formas en que seis personas pueden sentarse en torno de una mesa circular.

Podemos utilizar el mismo argumento para mostrar que existen $(n-1)!$ formas en que n personas pueden sentarse en torno de una mesa circular. □

A veces queremos considerar un ordenamiento de r elementos, elegidos entre n elementos disponibles. Tal ordenamiento es una **r-permutación**.

DEFINICIÓN 4.2.8

Una *r-permutación* de n elementos (distintos) $x_1, \ldots, x_n$ es un ordenamiento de un subconjunto de r elementos de $\{x_1, \ldots, x_n\}$. El número de r-permutaciones de un conjunto de n elementos distintos se denota $P(n, r)$.

EJEMPLO 4.2.9

Los siguientes son ejemplos de 2-permutaciones de a, b, c:

$$ab, \quad ba, \quad ca.$$ □

Si $r = n$ en la definición 4.2.8, obtenemos un ordenamiento de los n elementos. Así, una n-permutación de n elementos es lo que antes hemos llamado una permutación. El teorema 4.2.3 dice que $P(n, n) = n!$. El número $P(n, r)$ de r-permutaciones de un conjunto con n elementos cuando $r < n$ puede deducirse como en la demostración del teorema 4.2.3. La demostración del teorema para $r = 6$ y $r = 3$ se ilustra en la figura 4.2.3.

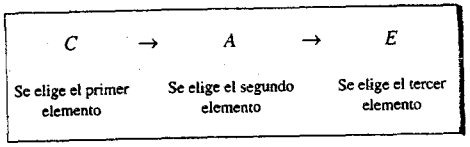


FIGURA 4.2.3 La demostración del teorema 4.2.10 para $n = 6$ y $r = 3$. Una r-permutación de $ABCDEF$ se construye eligiendo de manera sucesiva el primer elemento, luego el segundo y finalmente el tercero.

TEOREMA 4.2.10

El número de r-permutaciones de un conjunto de n objetos distintos es

$$P(n, r) = n(n-1)(n-2)\cdots(n-r+1), \quad r \le n.$$

Demostración. Debemos contar el número de formas de ordenar r elementos elegidos de un conjunto con n elementos. El primer elemento puede elegirse de n formas. Una vez elegido el primer elemento, el segundo elemento puede elegirse de $n-1$ formas. Continuamos eligiendo elementos hasta que, después de elegir el $(r-1)$-ésimo elemento, elegimos el r-ésimo elemento. Este último elemento puede elegirse de $n-r+1$ formas. Por el principio de multiplicación, el número de r-permutaciones de un conjunto con n objetos distintos es

$$n(n-1)(n-2)\cdots(n-r+1).$$ ■

EJEMPLO 4.2.11

De acuerdo con el teorema 4.2.10, el número de 2-permutaciones de $X = \{a, b, c\}$ es

$$P(3, 2) = 3 \cdot 2 = 6.$$

Estas seis 2-permutaciones son

$$ab, \quad ac, \quad ba, \quad bc, \quad ca, \quad cb.$$ □

EJEMPLO 4.2.12

¿De cuántas formas podemos elegir un presidente, un vicepresidente, un secretario y un tesorero de un grupo de 10 personas?

Debemos contar el número de ordenamientos de cuatro personas elegidas de un grupo de 10, pues un ordenamiento elige (de manera única) un presidente (primera elección), un vicepresidente (segunda elección), un secretario (tercera elección) y un tesorero (cuarta elección). Por el teorema 4.2.10, la solución es

$$P(10, 4) = 10 \cdot 9 \cdot 8 \cdot 7 = 5040.$$ □

También podríamos haber resuelto el ejemplo 4.2.12 apelando directamente al principio de multiplicación.

$P(n, r)$ puede escribirse en términos de factoriales:

$$P(n,r) = n(n-1)\cdots(n-r+1)$$

$$= \frac{n(n-1)\cdots(n-r+1)(n-r)\cdots 2\cdot 1}{(n-r)\cdots 2\cdot 1} = \frac{n!}{(n-r)!}. \qquad (4.2.1)$$

EJEMPLO 4.2.13

Utilizamos (4.2.1) para escribir la solución del ejemplo 4.2.12 como

$$P(10,4) = \frac{10!}{(10-4)!} = \frac{10!}{6!}. \qquad \square$$

EJEMPLO 4.2.14

¿De cuántas formas pueden formarse en una fila siete marcianos distintos y cinco jupiterianos distintos si ninguna pareja de jupiterianos puede estar junta?

Podemos formar a los marcianos y los jupiterianos mediante un proceso de dos pasos: Formamos a los marcianos y formamos a los jupiterianos. Los marcianos pueden formarse de $7! = 5040$ maneras. Una vez formados los marcianos (por ejemplo, en las posiciones $M_1 - M_7$), como ninguna pareja de jupiterianos pueden estar juntos, los jupiterianos tienen ocho posiciones posibles en las cuales formarse (indicadas mediante espacios en blanco):

$$_M_1_M_2_M_3_M_4_M_5_M_6_M_7_.$$

Así, los jupiterianos pueden formarse de $P(8,5) = 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 = 6720$ maneras. Por el principio de multiplicación, el número de maneras en que siete marcianos distintos y cinco jupiterianos distintos pueden formarse de modo que no queden jupiterianos juntos es

$$5040 \cdot 6720 = 33{,}868{,}800. \qquad \square$$

Ahora veremos las combinaciones. Una selección de objetos en la cual no importa el orden es una **combinación**.

DEFINICIÓN 4.2.15

Dado un conjunto $X = \{x_1, \ldots, x_n\}$ con n elementos (distintos),

(a) Una *r-combinación* de X es una selección no ordenada de r elementos de X (es decir, un subconjunto de X con r elementos).

(b) El número de r-combinaciones de un conjunto de n elementos distintos se denota $C(n, r)$ o $\binom{n}{r}$.

EJEMPLO 4.2.16

Un grupo de cinco estudiantes, Mary, Boris, Rosa, Ahmad y Nguyen, ha decidido hablar con la jefa del departamento de matemáticas para que el departamento ofrezca más cursos de matemáticas discretas. La jefa ha avisado que hablará con tres estudiantes. ¿De cuántas maneras pueden elegir estos cinco estudiantes a tres de ellos para hablar con la jefa?

Para resolver este problema, *no* debemos tomar en cuenta el orden. (Por ejemplo, no importa si la jefa habla con Mary, Ahmad y Nguyen o con Nguyen, Mary y Ahmad.) Si enumeramos las posibilidades, vemos que existen 10 maneras en que los cinco estudiantes pueden elegir a tres de ellos para hablar con la jefa:

MBR, MBA, MRA, BRA, MBN, MRN, BRN, MAN, BAN, RAN.

Con la terminología de la definición 4.2.15, el número de maneras en que los cinco estudiantes pueden elegir a tres de ellos para hablar con la jefa es $C(5, 3)$, el número de 3-combinaciones de cinco elementos. Hemos determinado que

$$C(5, 3) = 10. \qquad \square$$

A continuación deducimos una fórmula para $C(n, r)$, contando el número de r-permutaciones de un conjunto con n elementos de dos formas. La primera forma sólo utiliza la fórmula $P(n, r)$. La segunda forma de contar el número de r-permutaciones de un conjunto con n elementos implica el uso de $C(n, r)$. El cálculo de los dos valores nos permitirá derivar una fórmula de $C(n, r)$.

Podemos construir las r-permutaciones de un conjunto X con n elementos en dos pasos: Primero se elige una r-combinación de X (un subconjunto no ordenado con r elementos), y luego se ordena ésta. Por ejemplo, para construir una 2-permutación de $\{a, b, c, d\}$, podemos elegir primero una 2-combinación y luego ordenarla. La figura 4.2.4 muestra cómo se obtienen de esta manera todas las 2-permutaciones de $\{a, b, c, d\}$. El principio de multiplicación nos dice que el número de r-permutaciones es el producto del número de r-combinaciones y el número de ordenamientos de r elementos. Es decir,

$$P(n, r) = C(n, r)r!.$$

Por tanto,

$$C(n, r) = \frac{P(n, r)}{r!}.$$

Nuestro siguiente teorema establece este resultado y proporciona varias formas de escribir $C(n, r)$.

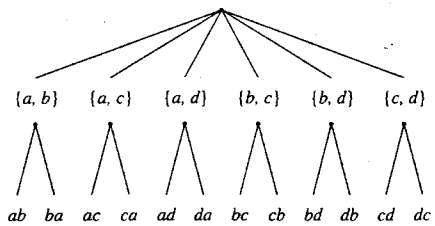


FIGURA 4.2.4 2-permutaciones de $\{a, b, c, d\}$.

TEOREMA 4.2.17

El número de r-combinaciones de un conjunto de n objetos distintos es

$$C(n,r) = \frac{P(n,r)}{r!} = \frac{n(n-1)\cdots(n-r+1)}{r!} = \frac{n!}{(n-r)!\,r!}, \quad r \le n.$$

Demostración. La demostración de la primera ecuación está dada antes del enunciado del teorema. Las otras formas de la ecuación son consecuencia del teorema 4.2.10 y de la ecuación (4.2.1). ■

EJEMPLO 4.2.18

¿De cuántas formas puede elegirse un comité de tres entre un grupo de 10 personas distintas?

Como un comité es un grupo no ordenado de personas, la respuesta es

$$C(10,3) = \frac{10 \cdot 9 \cdot 8}{3!} = 120.$$ □

EJEMPLO 4.2.19

¿De cuántas formas puede elegirse un comité de dos mujeres y tres hombres de un grupo de cinco mujeres distintas y seis hombres distintos?

Como en el ejemplo 4.2.18, vemos que las dos mujeres pueden elegirse de

$$C(5, 2) = 10$$

formas y que los tres hombres pueden elegirse de

$$C(6, 3) = 20$$

formas. El comité puede construirse en dos pasos sucesivos. Se eligen las mujeres, y luego se eligen los hombres. Por el principio de multiplicación, el número total de comités es

$$10 \cdot 20 = 200.$$ □

EJEMPLO 4.2.20

¿Cuántas cadenas de ocho bits contienen exactamente cuatro unos?

Una cadena de ocho bits que contiene cuatro unos queda determinada de manera única si indicamos cuáles bits contienen 1. Pero esto puede hacerse de

$$C(8, 4) = 70$$

formas. □

EJEMPLO 4.2.21

Una baraja ordinaria de 52 cartas consta de cuatro palos

tréboles, diamantes, corazones y espadas

con 13 denominaciones cada una

as, 2-10, jack, reina, rey.

(a) ¿Cuántas manos de póquer de cinco cartas (no ordenadas) pueden elegirse de una baraja ordinaria con 52 cartas?

(b) ¿Cuántas manos de póquer tienen todas las cartas de un mismo palo?

(c) ¿Cuántas manos de póquer tienen tres cartas de una denominación y dos cartas de una segunda denominación?

(a) La respuesta está dada por la fórmula de combinación

$$C(52, 5) = 2{,}598{,}960.$$

(b) Una mano con todas las cartas del mismo palo puede construirse en dos pasos: Se elige un palo, se eligen cinco cartas del palo elegido. El primer paso puede realizarse de cuatro formas y el segundo paso de $C(13, 5)$ formas. Por el principio de multiplicación, la respuesta es

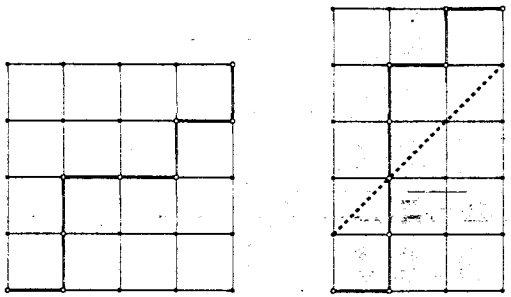
$$4 \cdot C(13, 5) = 5148.$$

(c) Una mano con tres cartas de una denominación y dos cartas de una segunda denominación puede construirse en cuatro pasos: Se elige la primera denominación, se escoge también la segunda, se eligen tres cartas de la primera denominación, se eligen dos cartas de la segunda denominación. La primera denominación puede elegirse de 13 formas. Una vez elegida la primera denominación, podemos elegir la segunda de 12 formas. Podemos elegir tres cartas de la primera denominación de $C(4, 3)$ formas y podemos elegir dos cartas de la segunda denominación de $C(4, 2)$ formas. Por el principio de multiplicación, la respuesta es

$$13 \cdot 12 \cdot C(4, 3) \cdot C(4, 2) = 3744.$$ □

EJEMPLO 4.2.22

¿Cuántas rutas existen desde la esquina inferior izquierda de una retícula cuadrada de $n \times n$ hasta la esquina superior derecha si sólo podemos viajar hacia la derecha y hacia arriba? Una de estas rutas aparece en una retícula de 4×4 en la figura 4.2.5a.



(a) (b)

FIGURA 4.2.5 (a) Una retícula 4×4 con una ruta de la esquina inferior izquierda a la esquina superior derecha. (b) La ruta en (a) transformada en una ruta en una retícula 5×3.

Cada ruta puede describirse mediante una cadena de n letras D (derecha) y n letras A (arriba). Por ejemplo, la ruta que aparece en la figura 4.2.5a puede describirse mediante la cadena *DAADDADA*. Cada una de estas cadenas puede obtenerse eligiendo n posiciones para las D, sin importar el orden de selección, de entre las $2n$ posiciones disponibles en la cadena y luego ocupando las demás posiciones con A. Así, existen $C(2n, n)$ rutas posibles. □

EJEMPLO 4.2.23

¿Cuántas rutas existen desde la esquina inferior izquierda de una retícula cuadrada de $n \times n$ hasta la esquina superior derecha si sólo podemos viajar hacia la derecha y hacia arriba y si sólo se permite tocar pero no ir por arriba de una recta diagonal que va de la esquina inferior izquierda a la esquina superior derecha?

Diremos que una ruta que toca pero no pasa sobre la diagonal es una *ruta buena*, y una ruta que rebasa la diagonal es una *ruta mala*. Nuestro problema consiste en contar el número de rutas buenas. Sea G_n el número de rutas buenas y B_n el número de rutas malas. En el ejemplo 4.2.22 se mostró que

$$G_n + B_n = C(2n, n);$$

así, basta calcular el número de rutas malas.

Una ruta en una retícula $(n + 1) \times (n - 1)$ que va de la esquina inferior izquierda a la esquina superior derecha (sin restricciones) es una ruta $(n + 1) \times (n - 1)$. La figura 4.2.5b muestra una ruta 5×3. Mostraremos que el número de rutas malas es igual al número de rutas $(n + 1) \times (n - 1)$ describiendo una función uno a uno y sobre del conjunto de rutas malas al conjunto de rutas $(n + 1) \times (n - 1)$.

Dada una ruta mala, determinamos el primer movimiento (partiendo de la esquina inferior izquierda) que la lleva por arriba de la diagonal. A partir de ese momento reemplazamos cada movimiento a la derecha por un movimiento hacia arriba y cada movimiento hacia arriba por un movimiento hacia la derecha. Por ejemplo, la ruta de la figura 4.2.5a se transforma en la ruta de la figura 4.2.5b. Esta transformación también puede realizarse girando la parte de la ruta posterior al primer movimiento arriba de la diagonal, en torno de la línea punteada que aparece en la figura 4.2.5b. Vemos que esta transformación realmente asigna a cada ruta mala una ruta $(n + 1) \times (n - 1)$.

Para mostrar que nuestra función es sobre, consideremos cualquier ruta $(n + 1) \times (n - 1)$. Como esta ruta termina sobre la diagonal, existe un primer movimiento que la lleva arriba de la diagonal. Entonces podemos girar el resto de la ruta en torno de la línea punteada que aparece en la figura 4.2.5b para obtener una ruta mala. La imagen de esta ruta mala bajo nuestra función es la ruta $(n + 1) \times (n - 1)$ con la que comenzamos. Por tanto, nuestra función es sobre. Nuestra función también es uno a uno, pues podemos verificar con facilidad que la función transforma rutas malas distintas en rutas $(n + 1) \times (n - 1)$ distintas. Por tanto, existe el mismo número de rutas malas y de rutas $(n + 1) \times (n - 1)$.

Un argumento similar al del ejemplo 4.2.22 muestra que el número de rutas $(n + 1) \times (n - 1)$ es igual a $C(2n, n - 1)$. Así, el número de rutas buenas es igual a

$$\begin{aligned} C(2n,n) - B_n &= C(2n,n) - C(2n,n-1) = \frac{(2n)!}{n!n!} - \frac{(2n)!}{(n-1)!(n+1)!} \\ &= \frac{(2n)!}{n!(n-1)!}\left(\frac{1}{n} - \frac{1}{n+1}\right) = \frac{(2n)!}{n!(n-1)!} \cdot \frac{1}{n(n+1)} \\ &= \frac{(2n)!}{(n+1)n!n!} = \frac{C(2n,n)}{n+1}. \end{aligned}$$

□

Los números $C(2n, n)/(n + 1)$ son llamados **números de Catalan** en honor del matemático belga Eugène-Charles Catalan (1814-1894), quien descubrió una deducción elemental de la fórmula $C(2n, n)/(n + 1)$. El primer problema investigado por Catalan aparece en el ejercicio 29 de la sección 5.1. Catalan publicó un gran número de artículos de análisis, combinatoria, álgebra, geometría, probabilidad y teoría de números. La conjetura de Catalan de que 0,1 y 8, 9 son las únicas parejas de enteros no negativos consecutivos que son potencia, permanece abierta (véase P. Ribenboim, "Catalan's conjeture", Am. núm., 103 [1996], 529–538).

En este libro denotaremos los números de Catalan $C(2n, n)/(n + 1)$ como C_n, $n \geq 1$, y definimos C_0 como 1. Los primeros números de Catalan son

$$C_0 = 1, \quad C_1 = 1, \quad C_2 = 2, \quad C_3 = 5, \quad C_4 = 14, \quad C_5 = 42.$$

Al igual que los números de Fibonacci, los números de Catalan pueden aparecer en lugares insospechados (por ejemplo, véanse los ejercicios 28 y 29 de la sección 5.1).

Concluimos esta sección con otra demostración del teorema 4.2.17, el cual proporciona una fórmula para el número de subconjuntos con r elementos, de un conjunto con n elementos. La demostración se ilustra en la figura 4.2.6. Sea X un conjunto con n elementos. Supondremos válida la fórmula $P(n, r) = n(n-1)\cdots(n-r+1)$, que cuenta el número de ordenamientos de subconjuntos con r elementos elegidos de X. Para contar el número de subconjuntos con r elementos de X, *no* queremos tomar en cuenta el orden; queremos considerar como *equivalentes* las permutaciones del mismo subconjunto. Formalmente, definimos una relación R sobre el conjunto S de r-permutaciones de X mediante la regla: $p_1 R p_2$ si p_1 y p_2 son permutaciones del mismo subconjunto con r elementos de X. Se verifica de manera directa que R es una relación de equivalencia sobre S.

Si p es una r-permutación de X, entonces p es una permutación de algún subconjunto con r elementos X_r de X; así, la clase de equivalencia que contiene a p consta de todas las permutaciones de X_r. Vemos que cada clase de equivalencia tiene $r!$ elementos. Una clase de equivalencia queda determinada mediante el subconjunto de X con r elementos que es permutado para obtener sus miembros. Por tanto, existen $C(n, r)$ clases de equivalencia. Como el conjunto S tiene $P(n, r)$ elementos, por el teorema 2.5.16, $C(n, r) = P(n, r)/r!$.

FIGURA 4.2.6 La demostración alternativa del teorema 4.2.17 para $n = 4$ y $r = 2$. Cada caja contiene una clase de equivalencia para la relación R sobre el conjunto de 2-permutaciones de $X = \{a, b, c, d\}$ definida como $p_1 R p_2$ si p_1 y p_2 son permutaciones del mismo subconjunto con 2 elementos de X. Existen $P(4, 2) = 12$ 2-permutaciones de X y 2 formas de permutar cada 2-permutación. Como cada clase de equivalencia corresponde a un subconjunto de X, $12/2 = C(4, 2)$.

Ejercicios

1. ¿Cuántas permutaciones existen de a, b, c, d?
2. Enumere las permutaciones de a, b, c, d.
3. ¿Cuántas 3-permutaciones existen de a, b, c, d?
4. Enumere las 3-permutaciones de a, b, c, d.
5. ¿Cuántas permutaciones existen de 11 objetos distintos?
6. ¿Cuántas 5-permutaciones existen de 11 objetos distintos?
7. ¿De cuántas formas puede elegirse un presidente, un vicepresidente y un secretario de actas de un grupo de 11 personas?
8. ¿De cuántas formas puede elegirse un presidente, un vicepresidente un secretario y un tesorero de un grupo de 12 personas?
9. En una carrera de 12 caballos, ¿de cuántas formas puede terminar la carrera (sólo importa el orden primero, segundo y tercero)?

En los ejercicios 10-18, determine cuántas cadenas pueden formarse ordenando las letras *ABCDE* sujetas a las condiciones dadas.

10. Que contengan la subcadena *ACE*.
11. Que contengan las letras *ACE*, juntas y en cualquier orden.
12. Que contengan las subcadenas *DB* y *AE*.
13. Que contengan la subcadena *AE* o la subcadena *EA*.
14. Que *A* aparezca antes que *D*. *Ejemplos*: *BCAED*, *BCADE*.
15. Que no contenga las subcadenas *AB*, *CD*.
16. Que no contenga las subcadenas *AB*, *BE*.
17. Que *A* aparezca antes que *C* y que *C* aparezca antes que *E*.
18. Que contenga la subcadena *DB* o la subcadena *BE*.
19. ¿De cuántas maneras puede formarse una fila con cinco marcianos distintos y ocho jupiterianos distintos si ningún par de marcianos debe estar junto?
20. ¿De cuántas maneras puede formarse una fila con cinco marcianos distintos, 10 venusinos distintos y ocho jupiterianos distintos si ningún par de marcianos debe estar junto?
21. ¿De cuántas maneras puede formarse una fila con cinco marcianos distintos y cinco jupiterianos distintos?
22. ¿De cuántas formas pueden sentarse en torno de una mesa redonda cinco marcianos distintos y cinco jupiterianos distintos?
23. ¿De cuántas formas pueden sentarse en torno de una mesa redonda cinco marcianos distintos y cinco jupiterianos distintos si ningún par de marcianos debe estar junto?
24. ¿De cuántas formas pueden sentarse en torno de una mesa redonda cinco marcianos distintos y ocho jupiterianos distintos si ningún par de marcianos debe estar junto?

En los ejercicios 25-27, sea $X = \{a, b, c, d\}$.

25. Calcule el número de 3-combinaciones de X.
26. Enumere las 3-combinaciones de X.
27. Muestre la relación entre las 3-permutaciones y las 3-combinaciones de X mediante un diagrama como el de la figura 4.2.4.
28. ¿De cuántas formas podemos elegir un comité de 3 personas entre un grupo de 11?
29. ¿De cuántas formas podemos elegir un comité de 4 personas entre un grupo de 12?
30. En el juego Lotto de la lotería estatal de Illinois, una persona debe elegir seis números (en cualquier orden) entre 44 números. ¿De cuántas formas puede hacerse esto? El estado piensa modificar el juego, de modo que una persona deba elegir seis números de 48. ¿De cuántas formas puede hacerse esto?

Los ejercicios 31-36 se refieren a un club que consta de seis hombres distintos y siete mujeres distintas.

31. ¿De cuántas formas puede elegirse un comité de cinco personas?
32. ¿De cuántas formas puede elegirse un comité de tres hombres y cuatro mujeres?
33. ¿De cuántas formas puede elegirse un comité de cuatro personas que tenga al menos una mujer?
34. ¿De cuántas formas puede elegirse un comité de cuatro personas que tenga al menos un hombre?
35. ¿De cuántas formas puede elegirse un comité de cuatro personas que tenga personas de ambos sexos?
36. ¿De cuántas formas puede elegirse un comité de cuatro personas de modo que Mabel y Ralph no queden juntos?
37. ¿De cuántas formas puede elegirse un comité de cuatro republicanos, tres demócratas y dos independientes de un grupo de 10 republicanos distintos, 12 demócratas distintos y cuatro independientes distintos?
38. ¿Cuántas cadenas de ocho bits contienen exactamente tres ceros?

39. ¿Cuántas cadenas de ocho bits contienen tres ceros consecutivos y cinco unos?

★ **40.** ¿Cuántas cadenas de ocho bits contienen al menos dos ceros consecutivos?

En los ejercicios 41-49, determine el número de manos de póquer de cinco cartas (sin orden), elegidas de una baraja común con 52 cartas, con las propiedades indicadas.

41. Con cuatro ases.

42. Con cuatro cartas del mismo tipo, es decir, cuatro cartas de la misma denominación.

43. Que todas las cartas sean espadas.

44. Con cartas de exactamente dos palos.

45. Con cartas de todos los palos.

46. De la forma A2345 del mismo palo.

47. Consecutivas y del mismo palo. (Suponga que el as es la menor denominación.)

48. Consecutivas. (Suponga que el as es la menor denominación.)

49. Con dos cartas de una denominación, dos cartas de otra denominación y una carta de una tercera denominación.

50. Determine el número de manos de bridge de 13 cartas (no ordenadas) elegidas de una baraja normal con 52 cartas.

51. ¿Cuántas manos de bridge tienen todas las cartas del mismo palo?

52. ¿Cuántas manos de bridge contienen exactamente dos palos?

53. ¿Cuántas manos de bridge tienen los cuatro ases?

54. ¿Cuántas manos de bridge tienen cinco espadas, cuatro corazones, tres tréboles y un diamante?

55. ¿Cuántas manos de bridge tienen cinco cartas de un palo, cuatro de otro palo, tres de otro palo y una de otro palo?

56. ¿Cuántas manos de bridge tienen cuatro cartas de tres palos y una carta del cuarto palo?

57. ¿Cuántas manos de bridge no tienen cartas 10, J, Q, K y A?

En los ejercicios 58-62, una moneda se arroja 10 veces.

58. ¿Cuántos resultados son posibles? [Un *resultado* es una lista de 10 caras (C) y cruces (X) que proporciona el resultado de cada uno de los 10 lanzamientos. Por ejemplo,

CCXCXCCCXC

representa 10 lanzamientos, en los que se obtuvo una cara en los dos primeros lanzamientos, luego una cruz, luego una cara, etcétera.]

59. ¿Cuántos resultados tienen exactamente tres caras?

60. ¿Cuántos resultados tienen a lo más tres caras?

61. ¿Cuántos resultados tienen una cara en el quinto lanzamiento?

62. ¿Cuántos resultados tienen tantas caras como cruces?

Los ejercicios 63-66 se refieren a un embarque de 50 microprocesadores de los cuales cuatro están defectuosos.

63. ¿De cuántas formas puede elegirse un conjunto de cuatro microprocesadores?

64. ¿De cuántas formas puede elegirse un conjunto de cuatro microprocesadores no defectuosos?

65. ¿De cuántas formas puede elegirse un conjunto de cuatro microprocesadores con exactamente dos microprocesadores defectuosos?

66. ¿De cuántas formas puede elegirse un conjunto de cuatro microprocesadores que contenga al menos un microprocesador defectuoso?

☆ 67. Muestre que el número de cadenas binarias de longitud $n \geq 4$ que contengan exactamente dos ocurrencias de 10 es $C(n + 1, 5)$.

☆ 68. Muestre que el número de cadenas de n bit con exactamente k ceros, sin que haya ceros consecutivos, es $C(n - k + 1, k)$.

☆ 69. Muestre que el producto de cualquier entero positivo y sus $k - 1$ sucesores es divisible entre $k!$.

70. Muestre que existen $(2n - 1)(2n - 3) \cdots 3 \cdot 1$ formas de elegir n pares de $2n$ elementos distintos.

71. Suponga que tenemos n objetos, r distintos y $n - r$ idénticos. Proporcione otra manera de deducir la fórmula

$$P(n, r) = r!C(n, r)$$

contando el número de ordenamientos de los n objetos de dos maneras:

- Contando los ordenamientos, eligiendo primero las posiciones de los r objetos distintos.
- Contando los ordenamientos, eligiendo primero las posiciones de los $n - r$ objetos idénticos.

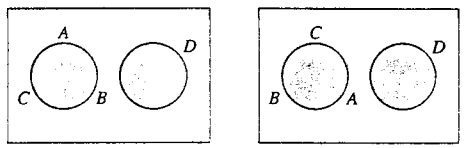
72. ¿Cuál es el error del siguiente argumento, el cual supuestamente muestra que $4C(39, 13)$ manos de bridge contienen tres o menos palos?

$C(39, 13)$ manos contienen sólo tréboles, diamantes y espadas. De hecho, para cualesquiera tres palos, existen $C(39, 13)$ manos que sólo contienen estos tres palos. Como existen cuatro 3-combinaciones de los palos, la respuesta es $4C(39, 13)$.

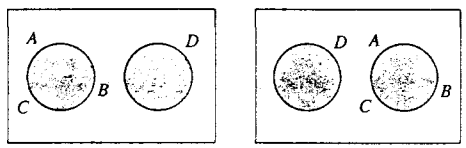
73. ¿Cuál es el error del siguiente argumento, el cual supuestamente muestra que $13^4 \cdot 48$ manos de póquer de cinco cartas (sin ordenar) contienen cartas de todos los palos?

Se elige una carta de cada palo. Esto puede realizarse de $13 \cdot 13 \cdot 13 \cdot 13 = 13^4$ formas. Como la quinta carta puede elegirse de 48 formas, la respuesta es $13^4 \cdot 48$.

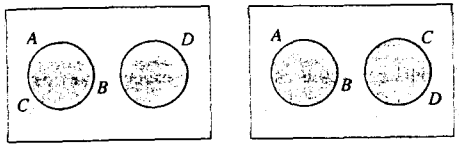
74. Sea $s_{n,k}$ el número de formas de sentar a n personas en torno de k mesas redondas, con al menos una persona en cada mesa. (Los números $s_{n,k}$ son los *números de Stirling de primer tipo.*) El orden de las tablas *no* se toma en cuenta. El orden del arreglo en una mesa *se* toma en cuenta, salvo rotaciones. *Ejemplos*: El siguiente par *no* es distinto:



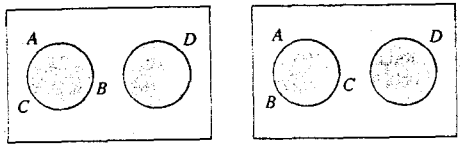
El siguiente par *no* es distinto:



El siguiente par *es* distinto:



El siguiente par *es* distinto:



(a) Muestre que $s_{n,k} = 0$ si $k > n$.
(b) Muestre que $s_{n,n} = 1$ para toda $n \geq 1$.
(c) Muestre que $s_{n,1} = (n-1)!$ para toda $n \geq 1$.
(d) Muestre que $s_{n,n-1} = C(n, 2)$ para toda $n \geq 2$.
(e) Muestre que

$$s_{n,2} = (n-1)!\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1}\right) \qquad \text{para toda } n \geq 2.$$

(f) Muestre que

$$\sum_{k=1}^{n} s_{n,k} = n! \qquad \text{para toda } n \geq 1.$$

(g) Determine una fórmula para $s_{n,n-2}$, $n \geq 3$, y demuéstrela.

75. Sea $S_{n,k}$ el número de formas de separar un conjunto de n elementos en exactamente k subconjuntos no vacíos. El orden de los subconjuntos no se toma en cuenta. (Los números $S_{n,k}$ son los *números de Stirling de segundo tipo.*)
(a) Muestre que $S_{n,k} = 0$ si $k > n$.
(b) Muestre que $S_{n,n} = 1$ para toda $n \geq 1$.
(c) Muestre que $S_{n,1} = 1$ para toda $n \geq 1$.
(d) Muestre que $S_{3,2} = 3$.
(e) Muestre que $S_{4,2} = 7$.
(f) Muestre que $S_{4,3} = 6$.
(g) Muestre que $S_{n,2} = 2^{n-1} - 1$ para toda $n \geq 2$.
(h) Muestre que $S_{n,n-1} = C(n, 2)$ para toda $n \geq 2$.
(i) Determine una fórmula para $S_{n,n-2}$, $n \geq 3$, y demuéstrela.

76. Muestre que existen

$$\sum_{k=1}^{n} S_{n,k}$$

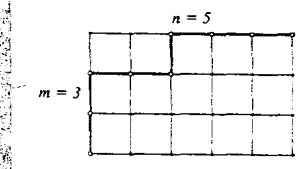
relaciones de equivalencia sobre un conjunto de n elementos. [Los números $S_{n,k}$ son los números de Stirling del segundo tipo; véase el ejercicio 75.]

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

COMBINACIONES

Problema

(a) ¿Cuántas rutas existen desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula $m \times n$, la cual sólo puede recorrerse hacia la derecha o hacia arriba? Por ejemplo, la figura anexa es una retícula 3×5 en la cual se muestra una ruta.



(b) Divida las rutas en clases, basándose en el momento en que la ruta llega primero a la arista superior, para deducir la fórmula

$$\sum_{k=0}^{n} C(k+m-1, k) = C(m+n, m).$$

Para enfrentar el problema

En el ejemplo 4.2.22 se contó el número de trayectorias desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula $n \times n$, la cual sólo se podía recorrer hacia la derecha o hacia arriba. La solución a ese problema codificaba cada ruta como una cadena de n letras D (derecha) y n letras A (arriba). Entonces, el problema se convirtió en el problema de contar el número de tales cadenas. Una de estas cadenas puede obtenerse eligiendo n posiciones para las letras D, sin importar el orden de selección, entre las $2n$ posiciones disponibles en la cadena, para después llenar las demás posiciones con letras A. Así, el número de cadenas y el número de rutas son iguales a $C(2n, n)$.

En este nuevo problema, podemos codificar cada ruta como una cadena de n letras D (derecha) y m letras A (arriba). Como en el teorema anterior, debemos contar el número de tales cadenas. Una de estas cadenas puede obtenerse eligiendo n posiciones para las letras D, sin importar el orden de selección, entre las $n + m$ posiciones disponibles en la cadena y llenando después las posiciones restantes con letras A. Así, el número de cadenas y el número de rutas son iguales a $C(n + m, n)$. Con esto hemos respondido a la parte (a).

En la parte (b) se nos proporciona una sugerencia importante: Dividir las rutas en clases de acuerdo con el momento en que la ruta llega a la arista superior. Una ruta puede llegar por primera vez a la arista superior en cualquiera de $n + 1$ posiciones. En la figura anterior, la ruta que se muestra toca por vez primera a la arista superior en la tercera posición a partir de la izquierda. Antes de continuar, tal vez el lector se pregunte la razón de dividir las rutas en clases.

Observe que al dividir las rutas en clases con base en el primer momento en que la ruta toca la arista superior:

- Las clases son *ajenas*.

(Una ruta no puede tocar a la arista superior en dos o más posiciones distintas.) Observe también que todas las rutas llegan a la arista superior en algún momento, de modo que

- Cada ruta está en alguna clase.

Con la terminología de la sección 2.1 (véase el ejemplo 2.1.10 y el análisis anterior a éste), las clases establecen una *partición* del conjunto de rutas. Debido a este hecho,

podemos aplicar el principio de la suma, de modo que la suma del número de rutas en cada clase es igual al número total de rutas. (Ninguna ruta se cuenta dos veces, pues las clases no se traslapan, y cada ruta se cuenta una vez, pues cada ruta está en alguna clase.) Es claro que la ecuación que debemos demostrar surge de igualar la suma de los números de rutas en cada clase con el número total de rutas.

Determinación de una solución

Ya hemos resuelto la parte (a). Para la parte (b), observemos la retícula 3×5. Existe exactamente una ruta que llega por vez primera a la arista superior en la primera posición a partir de la izquierda. Existen tres rutas que llegan por vez primera a la arista superior en la segunda posición a partir de la izquierda.

Observe que la única variación en las figuras anteriores ocurre entre el inicio y el

punto marcado con un círculo. Dicho de otra forma, después de que una ruta llega a dicho punto, sólo existe una forma de concluir el recorrido. Por tanto, basta contar el número de rutas desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula 2×1. ¡Pero ya hemos resuelto esto en la parte (a)! El número de rutas desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula 2×1 es igual a $C(2 + 1, 1) = 3$. De manera análoga, vemos que el número de rutas que llegan por vez primera a la arista superior en la tercera posición desde la izquierda es igual al número de rutas desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula 2×2 [a saber, $C(2 + 2, 2) = 6$]. Al sumar obtenemos todas las rutas

$$C(5 + 3, 5) = C(0 + 2, 0) + C(1 + 2, 1) + C(2 + 2, 2) + C(3 + 2, 3) + C(4 + 2, 4) + C(5 + 2, 5).$$

Si reemplazamos cada término $C(k + 3 - 1, k)$ por su valor, obtenemos

$$56 = 1 + 3 + 6 + 10 + 15 + 21.$$

El lector debe verificar esta fórmula, determinar las seis rutas que llegan por vez primera en la tercera posición desde la izquierda, y ver por qué el número de tales rutas es igual al número de rutas desde la esquina inferior izquierda hasta la esquina superior derecha de una retícula 2×2.

Solución formal

(a) Podemos codificar cada ruta como una cadena de n letras D (derecha) y m letras A (arriba). Cualquiera de estas cadenas puede obtenerse eligiendo n posiciones para las letras D, sin importar el orden de selección, entre las $n + m$ posiciones disponibles en la cadena, y luego ocupando las demás posiciones con letras A. Así, el número de rutas es igual a $C(n + m, n)$.

(b) Cada cadena puede describirse como una cadena que contiene n letras D y m letras A. La última A de la cadena señala el punto donde la ruta toca por vez primera a la arista superior. Contaremos las cadenas, dividiéndolas en clases que constan de cadenas que terminan en A, AD, ADD, y así sucesivamente. Existen

$$C(n + m - 1, n)$$

cadenas que terminan en A, pues debemos elegir n espacios entre los primeros $n + m - 1$ espacios para las n letras D. Existen

$$C((n - 1) + m - 1, n - 1)$$

cadenas que terminan en AD, pues debemos elegir $n - 1$ espacios entre los primeros $(n - 1) + m - 1$ espacios para las $n - 1$ letras D. En general, existen $C(k + m - 1, k)$ cadenas que terminan en AD^{n-k}. Como en total existen $C(m + n, m)$ cadenas, obtenemos el resultado deseado.

Resumen de técnicas para resolver problemas

- Busque un problema similar e imite su solución.
- Con frecuencia, el conteo del número de miembros de un conjunto de dos maneras distintas conduce a una ecuación. En particular, si $\{X_1, X_2, \ldots, X_n\}$ es una partición de X, podemos aplicar el principio de la suma y

$$|X| = \sum_{i=1}^{n} |X_i|.$$

- Enumere en forma directa algunos de los elementos por contar.
- Busque patrones.

Comentarios

Antes de aplicar el principio de la suma, es importante verificar que una supuesta partición realmente lo sea. Si X es el conjunto de cadenas de cinco bits y X_i es el conjunto de cadenas de cinco bits que no contienen i ceros consecutivos, el principio de la suma *no* se aplica; los conjuntos X_i *no* son ajenos por pares. Por ejemplo, $00001 \in X_2 \cap X_3$. Como ejemplo de una partición de X, podríamos definir X_i como el conjunto de cadenas de cinco bits que contienen exactamente i ceros.

Ejercicios

1. Divida las rutas en clases, con base en el primer momento en que la ruta cruza una recta vertical, y utilice el principio de la suma para deducir una fórmula como la demostrada en esta sección.
2. Divida las rutas en clases, con base en el momento en que la ruta cruza la línea inclinada de la figura.

Luego, utilice el principio de la suma para deducir una fórmula como la demostrada en esta sección.

4.3 *ALGORITMOS PARA GENERAR PERMUTACIONES Y COMBINACIONES*

El grupo de rock Unhinged Universe ha grabado n videos, cuyos tiempos de ejecución son

$$t_1, t_2, \ldots, t_n$$

segundos. Hay que lanzar al mercado una cinta que contenga C segundos. Como ésta es la primera cinta del grupo, ellos quieren incluir la mayor cantidad posible de material. Así, el problema consiste en elegir un subconjunto $\{i_1, \ldots, i_k\}$ de $\{1, 2, \ldots, n\}$ tal que la suma

$$\sum_{j=1}^{k} t_{i_j} \tag{4.3.1}$$

no exceda a C y sea lo más grande posible. Un método directo consiste en examinar todos los subconjuntos de $\{1, 2, \ldots, n\}$ y elegir un subconjunto de modo que la suma (4.3.1) no exceda a C y sea lo más grande posible. Para implantar este método, necesitamos un algoritmo que genere todas las combinaciones de un conjunto con n elementos. En esta sección desarrollaremos algoritmos para generar combinaciones y permutaciones.

Como existen 2^n subconjuntos de un conjunto con n elementos, el tiempo de ejecución de un algoritmo que examine todos los subconjuntos es $\Omega(2^n)$. Como hemos visto en la sección 3.5, la ejecución de tales algoritmos no es práctica, salvo para valores pequeños de n. Por desgracia, existen problemas (un ejemplo de los cuales es el problema de la cinta en cuestión) para los cuales no se conoce un método mejor que el de enumerar todas las posibilidades.

Nuestro algoritmo enumera las permutaciones y las combinaciones en **orden lexicográfico**. Este orden generaliza el orden común de los diccionarios.

Dadas dos palabras distintas, para determinar cuál precede a la otra en el diccionario, comparamos las letras en las palabras. Existen dos posibilidades:

1. Las palabras tienen diferentes longitudes, y cada letra de la palabra más corta es idéntica a la letra correspondiente en la palabra más larga.
2. Las palabras tienen igual o distinta longitud, y en alguna posición, las letras de las palabras son diferentes. (4.3.2)

Si se cumple 1, entonces la palabra más corta precede a la más larga. (Por ejemplo, "gas" precede a "gaseoso" en el diccionario.) Si se cumple 2, localizamos la primera posición p (partiendo de la izquierda) donde las letras son distintas. El orden de las palabras se determina mediante el orden de las letras en la posición p. [Por ejemplo, "gladiador" precede a "gladiola" en el diccionario. En la primera posición (desde la izquierda) donde las letras son distintas, tenemos una "a" en "gladiador" y una "o" en "gladiola", y "a" precede a "o" en el alfabeto.]

El orden lexicográfico generaliza el orden común de los diccionarios, reemplazando el alfabeto por cualquier conjunto de símbolos para los cuales se ha definido un orden. Nosotros trabajaremos con cadenas de enteros.

DEFINICIÓN 4.3.1

Sean $\alpha = s_1 s_2 \cdots s_p$ y $\beta = t_1 t_2 \cdots t_q$ cadenas sobre $\{1, 2, \ldots, n\}$. Decimos que α es *lexicográficamente menor que* β y escribimos $\alpha < \beta$ si

(a) $p < q$ y $s_i = t_i$ para $i = 1, \ldots, p$

o

(b) para algún i, $s_i \neq t_i$ y para el mínimo de tales i, tenemos $s_i < t_i$.

En la definición 4.3.1, el caso (a) corresponde a la posibilidad 1 de (4.3.2) y el caso (b) corresponde a la posibilidad 2 de (4.3.2).

EJEMPLO 4.3.2

Sean $\alpha = 132$ y $\beta = 1324$ cadenas sobre $\{1, 2, 3, 4\}$. En la notación de la definición 4.3.1, $p = 3, q = 4, s_1 = 1, s_2 = 3, s_3 = 2, t_1 = 1, t_2 = 3, t_3 = 2$ y $t_4 = 4$. Como $p = 3 < 4 = q$ y $s_i = t_i$ para $i = 1, 2, 3$, se cumple la condición (a) de la definición 4.3.1. Por tanto, $\alpha < \beta$. □

EJEMPLO 4.3.3

Sean $\alpha = 13246$ y $\beta = 1342$ cadenas sobre $\{1, 2, 3, 4, 5, 6\}$. En la notación de la definición 4.3.1, $p = 5, q = 4, s_1 = 1, s_2 = 3, s_3 = 2, s_4 = 4, s_5 = 6, t_1 = 1, t_2 = 3, t_3 = 4$ y $t_4 = 2$. El menor i para el cual $s_i \neq t_i$ es $i = 3$. Como $s_3 < t_3$, por la condición (b) de la definición 4.3.1, $\alpha < \beta$. □

EJEMPLO 4.3.4

Sean $\alpha = 1324$ y $\beta = 1342$ cadenas sobre $\{1, 2, 3, 4\}$. En la notación de la definición 4.3.1, $p = q = 4, s_1 = 1, s_2 = 3, s_3 = 2, s_4 = 4, t_1 = 1, t_2 = 3, t_3 = 4$ y $t_4 = 2$. El menor i para el cual $s_i \neq t_i$ es $i = 3$. Como $s_3 < t_3$, por la condición (b) de la definición 4.3.1, $\alpha < \beta$. □

EJEMPLO 4.3.5

Sean $\alpha = 13542$ y $\beta = 21354$ cadenas sobre $\{1, 2, 3, 4, 5\}$. En la notación de la definición 4.3.1, $s_1 = 1$, $s_2 = 3$, $s_3 = 5$, $s_4 = 4$, $s_5 = 2$, $t_1 = 2$, $t_2 = 1$, $t_3 = 3$, $t_4 = 5$ y $t_5 = 4$. El menor i para el cual $s_i \neq t_i$ es $i = 1$. Como $s_1 < t_1$, por la condición (b) de la definición 4.3.1, $\alpha < \beta$. □

Para las cadenas de la misma longitud sobre $\{1, 2, \ldots, 9\}$, el orden lexicográfico es igual al orden numérico de los enteros positivos, si interpretamos las cadenas como números decimales (véanse los ejemplos 4.3.4 y 4.3.5). Para las cadenas de distinta longitud, el orden lexicográfico puede ser distinto del orden numérico (véase el ejemplo 4.3.3). En el resto de esta sección, orden se refiere al *orden* lexicográfico.

Primero, consideremos el problema de enumerar todas las r-combinaciones de $\{1, 2, \ldots, n\}$. En nuestro algoritmo, enumeraremos la r-combinación $\{x_1, \ldots, x_r\}$ como la cadena $s_1 \cdots s_r$, donde $s_1 < s_2 < \cdots < s_r$ y $\{x_1, \ldots, x_r\} = \{s_1, \ldots, s_r\}$. Por ejemplo, la 3-combinación $\{6, 2, 4\}$ se enumera 246.

Enumeraremos las r-combinaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico. Así, la primera cadena enumerada será $12 \cdots r$ y la última será $(n - r + 1) \cdots n$.

EJEMPLO 4.3.6

Consideremos el orden en que se enumerarán las 5-combinaciones de $\{1, 2, 3, 4, 5, 6, 7\}$. La primera cadena es 12345, seguida por 12346 y 12347. La siguiente cadena es 12356 seguida por 12357. La última cadena será 34567. □

EJEMPLO 4.3.7

Determine la cadena posterior a 13467 al enumerar las 5-combinaciones de $X = \{1, 2, 3, 4, 5, 6, 7\}$.

Ninguna cadena que comience con 134 y represente una 5-combinación de X excede a 13467. Así, la cadena posterior a 13467 debe comenzar con 135. Como 13567 es la menor cadena que comienza con 135 y que representa una 5-combinación de X, la respuesta es 13567. □

EJEMPLO 4.3.8

Determine la cadena posterior a 2367 al enumerar las 4-combinaciones de $X = \{1, 2, 3, 4, 5, 6, 7\}$.

Ninguna cadena que comience con 23 y represente una 4-combinación de X excede a 2367. Así, la cadena posterior a 2367 debe comenzar con 24. Como 2456 es la menor cadena que comienza con 24 y representa una 4-combinación de X, la respuesta es 2456. □

Comienza a aparecer un patrón. Dada una cadena $\alpha = s_1 \cdots s_r$ que represente a la r-combinación $\{s_1, \ldots, s_r\}$, para determinar la siguiente cadena $\beta = t_1 \cdots t_r$, buscamos el elemento de más a la derecha que no tenga su máximo valor (s_r puede tener el valor máximo n, s_{r-1} puede tener el valor máximo $n - 1$, y así sucesivamente). Entonces

$$t_i = s_i \quad \text{para } i = 1, \ldots, m - 1.$$

El elemento t_m es igual a $s_m + 1$. Para el resto de la cadena β tenemos

$$t_{m+1} \cdots t_r \ (s_m + 2)(s_m + 3) \cdots .$$

A continuación damos el algoritmo.

ALGORITMO 4.3.9 ***Generación de combinaciones***

Este algoritmo enumera todas las r-combinaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico creciente.

Entrada: r, n

Salida: Todas las r-combinaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico creciente.

```
1.  procedure combination(r, n)
2.    for i := 1 to r do
3.      s_i := i
4.    print s_1, ..., s_r   // se imprime la primera r-combinación
5.    for i := 2 to C(n, r) do
6.      begin
7.      m := r
8.      max_val := n
9.      while s_m = max_val do
10.       // se determina el elemento más a la derecha, que no tenga su máximo valor
11.       begin
12.       m := m - 1
13.       max_val := max_val - 1
14.       end
15.     // se incrementa el elemento más a la derecha
16.     s_m := s_m + 1
17.     // el resto de los elementos son los sucesores de s_m
18.     for j := m + 1 to r do
19.       s_j := s_{j-1} + 1
20.     print s_1, ..., s_r   // se imprime la i-ésima combinación
21.     end
22. end combination
```

EJEMPLO 4.3.10

Mostramos la forma en que el algoritmo 4.3.9 genera la 5-combinación de $\{1, 2, 3, 4, 5, 6, 7\}$ posterior a 23467. Estamos suponiendo que

$$s_1 = 2, \quad s_2 = 3, \quad s_3 = 4, \quad s_4 = 6, \quad s_5 = 7.$$

En la línea 15, vemos que s_3 es el elemento más a la derecha que no alcanza su máximo valor. En la línea 16, s_3 se iguala a 5. En las líneas 18 y 19, s_4 se iguala a 6 y s_5 se iguala a 7. En este momento,

$$s_1 = 2, \quad s_2 = 3, \quad s_3 = 5, \quad s_4 = 6, \quad s_5 = 7.$$

Hemos generado la 5-combinación 23567 posterior a 23467. □

EJEMPLO 4.3.11

Las 4-combinaciones de {1, 2, 3, 4, 5, 6} enumeradas según el algoritmo 4.3.9 son

1234, 1235, 1236, 1245, 1246, 1256, 1345, 1346,
1356, 1456, 2345, 2346, 2356, 2456, 3456. □

Al igual que el algoritmo para generación de las r-combinaciones, el algoritmo para generar permutaciones enumerará las permutaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico. (En el ejercicio 16 se pide un algoritmo que genere todas las r-permutaciones de un conjunto con n elementos.)

EJEMPLO 4.3.12

Para construir la permutación de {1, 2, 3, 4, 5, 6} posterior a 163542, debemos mantener idénticos el mayor número posible de dígitos de la izquierda.

¿Podría la permutación posterior a la permutación dada tener la forma 1635_ _? Como la única permutación de la forma 1635_ _ distinta de la permutación dada es 163524 y 163524 es menor que 163542, la permutación posterior a la permutación dada no es de la forma 1635_ _.

¿Podría la permutación posterior a la permutación dada tener la forma 163_ _ _? Los últimos tres dígitos deben ser una permutación de {2, 4, 5}. Como 542 es la permutación más grande de {2, 4, 5}, cualquier permutación que comience con 163 es menor que la permutación dada. Así, la permutación posterior a la permutación dada no es de la forma 163_ _ _.

La razón por la cual la permutación posterior a la permutación dada no puede comenzar con 1635 o 163 es que en cualquier caso, los dígitos restantes en la permutación dada (42 y 542, respectivamente) *decrecen*. Por tanto, trabajando desde el lado derecho, debemos determinar el primer dígito d cuyo vecino derecho satisfaga $d < r$. En nuestro caso, el tercer dígito, 3, tiene esta propiedad. Así, la permutación posterior a la permutación dada comenzará con 16.

El dígito posterior a 16 debe ser mayor que 3. Como queremos determinar la permutación siguiente más pequeña, el siguiente dígito es 4, el menor dígito disponible. Así, la permutación deseada comienza con 164. Los demás dígitos 235 deben aparecer en orden creciente para obtener el valor mínimo. Por tanto, la permutación posterior a la permutación dada es 164235. □

Vemos que para generar todas las permutaciones de $\{1, 2, \ldots, n\}$, podemos comenzar con la permutación $12 \cdots n$ y luego utilizar varias veces el método del ejemplo 4.3.12 para generar la siguiente permutación. El proceso concluye cuando se genera la permutación $n(n-1) \cdots 21$.

EJEMPLO 4.3.13

El método del ejemplo 4.3.12 permite enumerar las permutaciones de {1, 2, 3, 4} en orden lexicográfico como

1234, 1243, 1324, 1342, 1423, 1432, 2134, 2143,
2314, 2341, 2413, 2431, 3124, 3142, 3214, 3241,
3412, 3421, 4123, 4132, 4213, 4231, 4312, 4321. □

A continuación damos el algoritmo.

ALGORITMO 4.3.14 *Generación de permutaciones*

Este algoritmo enumera todas las permutaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico creciente.

Entrada: n

Salida: Todas las permutaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico creciente.

```
1.  procedure permutation(n)
2.    for i := 1 to n do
3.      s_i := i
4.    print s_1, ..., s_n // se imprime la primera permutación
5.    for i := 2 to n! do
6.      begin
7.      m := n − 1
8.      while s_m > s_{m+1} do
9.        // se determina la primera disminución trabajando desde la derecha
10.       m := m − 1
11.     k := n
12.     while s_m > s_k do
13.       // se determina el elemento más a la derecha s_k con s_m < s_k
14.       k := k − 1
15.     swap(s_m, s_k)
16.     p := m + 1
17.     q := n
18.     while p < q do
19.       // se intercambian s_{m+1} y s_n, se intercambian s_{m+2} y s_{n−1}, y así sucesivamente
20.       begin
21.       swap(s_p, s_q)
22.       p := p + 1
23.       q := q − 1
24.       end
25.     print s_1, ..., s_n    // se imprime la i-ésima permutación
26.     end
27.   end permutation
```

EJEMPLO 4.3.15

Ahora mostraremos la forma en que el algoritmo 4.3.14 genera la permutación posterior a 163542. Supongamos que

$$s_1 = 1, \quad s_2 = 6, \quad s_3 = 3, \quad s_4 = 5, \quad s_5 = 4, \quad s_6 = 2$$

y que estamos en la línea 7. El máximo índice m que satisface $s_m < s_{m+1}$ es 3. En las líneas 11 a 14, vemos que el máximo índice k que satisface $s_k > s_m$ es 5. En la línea 15, intercambiamos s_m y s_k. En este momento, tenemos $s = 164532$. En las líneas 16-24, invertimos el orden de los elementos $s_4 s_5 s_6 = 532$. Obtenemos la permutación deseada 164235. □

Ejercicios

En los ejercicios 1-3, determine la r-combinación que será generada por el algoritmo 4.3.9 con $n = 7$, posterior a la r-combinación dada.

1. 1356 2. 12367 3. 14567

En los ejercicios 4-6, determine la permutación que será generada por el algoritmo 4.3.14 posterior a la permutación dada.

4. 12354 5. 625431 6. 12876543

7. Para cada una de las cadenas de los ejercicios 1-3, explique (como en el ejemplo 4.3.10) exactamente la forma en que el algoritmo 4.3.9 genera la siguiente r-combinación.

8. Para cada una de las cadenas de los ejercicios 4-6, explique (como en el ejemplo 4.3.15) exactamente la forma en que el algoritmo 4.3.14 genera la siguiente permutación.

9. Muestre la salida del algoritmo 4.3.9 cuando $n = 6$ y $r = 3$.

10. Muestre la salida del algoritmo 4.3.9 cuando $n = 6$ y $r = 2$.

11. Muestre la salida del algoritmo 4.3.9 cuando $n = 7$ y $r = 5$.

12. Muestre la salida del algoritmo 4.3.14 cuando $n = 2$.

13. Muestre la salida del algoritmo 4.3.14 cuando $n = 3$.

14. Modifique el algoritmo 4.3.9 para eliminar la línea 5

    5. **for** $i := 2$ **to** $C(n, r)$ **do**

    se elimina. Base la condición de conclusión en el hecho de que la última r-combinación tiene todos los elementos s_i iguales a su máximo valor.

15. Modifique el algoritmo 4.3.14 para eliminar la línea 5

    5. **for** $i := 2$ **to** $n!$ **do**

    se elimina. Base la condición de conclusión en el hecho de que la última permutación tiene los elementos s_i en orden decreciente.

16. Escriba un algoritmo que genere todas las r-permutaciones de un conjunto con n elementos.

★ 17. Escriba un algoritmo recursivo que genere todas las r-combinaciones del conjunto $\{s_1, s_2, \ldots, s_n\}$. Divida el problema en dos subproblemas:

- Enumere todas las r-combinaciones que contienen a s_1.
- Enumere todas las r-combinaciones que no contienen a s_1.

☆ 18. Escriba un algoritmo recursivo que genere todas las permutaciones del conjunto $\{s_1, s_2, \ldots, s_n\}$. Divida el problema en n subproblemas:

- Enumere las permutaciones que comienzan con s_1.
- Enumere las permutaciones que comienzan con s_2.

  ⋮

- Enumere las permutaciones que comienzan con s_n.

4.4 *PERMUTACIONES Y COMBINACIONES GENERALIZADAS*

En las secciones 4.2 y 4.3 trabajamos con ordenamientos y selecciones sin permitir repeticiones. En esta sección consideraremos ordenamientos de sucesiones que contienen repeticiones y selecciones no ordenadas donde se permiten las repeticiones.

EJEMPLO 4.4.1

¿Cuántas cadenas pueden formarse mediante las siguientes letras?

M I S S I S S I P P I

Debido a la duplicación de letras, la respuesta no es 11!, sino algún número menor que 11!.

Consideremos el problema de llenar 11 espacios en blanco,

_ _ _ _ _ _ _ _ _ _ _ ,

con las letras dadas. Existen $C(11, 2)$ formas de elegir posiciones para las dos letras P. Una vez elegidas las posiciones de las P, existen $C(9, 4)$ formas de elegir posiciones para las cuatro S. Una vez elegidas las posiciones de las S, existen $C(5, 4)$ formas de elegir posiciones para las cuatro I. Una vez realizadas estas selecciones, existe una única posición para la M. Por el principio de multiplicación, el número de formas de ordenar las letras es

$$C(11, 2)C(9, 4)C(5, 4) = \frac{11!}{2!9!}\frac{9!}{4!5!}\frac{5!}{4!1!} = \frac{11!}{2!4!4!1!} = 34,650. \quad \square$$

La solución del ejemplo 4.4.1 adopta una forma agradable. El número 11 que aparece en el numerador es el número total de letras. Los valores en el denominador proporcionan las cantidades de duplicados de cada letra. El método puede utilizarse para establecer una fórmula general.

TEOREMA 4.4.2

Suponga que una sucesión S de n elementos tiene n_1 objetos idénticos de tipo 1, n_2 objetos idénticos de tipo 2, . . . , y n_t objetos idénticos de tipo t. Entonces el número de ordenamientos de S es

$$\frac{n!}{n_1!n_2!\cdots n_t!}.$$

Demostración. Asignamos posiciones a cada uno de los n elementos para crear un ordenamiento de S. Podemos asignar posiciones a los n_1 elementos de tipo 1 de $C(n, n_1)$ formas. Una vez realizadas estas asignaciones, podemos asignar posiciones a los n_2 elementos de tipo 2 de $C(n - n_1, n_2)$ formas, y así sucesivamente. Por el principio de multiplicación, la cantidad de ordenamientos es

$$\begin{aligned} &C(n, n_1)C(n - n_1, n_2)C(n - n_1 - n_2, n_3)\cdots C(n - n_1 - \cdots - n_{t-1}, n_t) \\ &= \frac{n!}{n_1!(n - n_1)!}\frac{(n - n_1)!}{n_2!(n - n_1 - n_2)!}\cdots\frac{(n - n_1 - \cdots - n_{t-1})!}{n_t!0!} \\ &= \frac{n!}{n_1!n_2!\cdots n_t!}. \end{aligned}$$

EJEMPLO 4.4.3

¿De cuántas formas pueden dividirse ocho libros distintos entre tres estudiantes si Bill debe tener cuatro libros y Shizuo y Marian dos libros cada una?

Se colocan los libros en algún orden fijo. Ahora, consideremos los ordenamientos de cuatro letras B, dos S y dos M. Un ejemplo es

$$B\ B\ B\ S\ M\ B\ M\ S.$$

Cada ordenamiento de este tipo determina una distribución de los libros. Para el ordenamiento del ejemplo, Bill tendrá los libros 1, 2, 3 y 6, Shizuo tendrá los libros 4 y 8 y Marian los libros 5 y 7. Así, el número de formas de ordenar *BBBBSSMM* es el número de formas de distribuir los libros. Por el teorema 4.4.2, este número es

$$\frac{8!}{4!2!2!} = 420. \qquad \square$$

Podemos dar una demostración alternativa del teorema 4.4.2 mediante relaciones. Supongamos que una sucesión S de n elementos tiene n_i objetos idénticos de tipo i para $i = 1, \ldots, t$. Sea X el conjunto de n elementos obtenidos de S considerando los n_i objetos de tipo i *distintos* para $i = 1, \ldots, t$. Por ejemplo, si S es la sucesión de letras

$$M\ I\ S\ S\ I\ S\ S\ I\ P\ P\ I,$$

X sería el conjunto

$$\{M, I_1, S_1, S_2, I_2, S_3, S_4, I_3, P_1, P_2, I_4\}.$$

Definimos una relación R sobre el conjunto de todas las permutaciones de X mediante la regla: $p_1 R p_2$ si p_2 se obtiene de p_1 permutando el orden de los objetos de tipo 1 (pero sin cambiar su posición) o permutando el orden de los objetos de tipo 2 (pero sin cambiar su posición) $\cdots$ y/o permutando el orden de los objetos de tipo t (pero sin cambiar su posición); por ejemplo,

$$I_1S_1S_2I_2S_3S_4I_3P_1P_2I_4MRI_2S_3S_2I_1S_4S_1I_3P_1P_2I_4M.$$

En forma directa puede verificarse que R es una relación de equivalencia sobre el conjunto de todas las permutaciones de X.

La clase de equivalencia que contiene a la permutación p consta de todas las permutaciones de X que son idénticas si consideramos idénticos los objetos de tipo i para $i = 1, \ldots, t$. Así, cada clase de equivalencia tiene $n_1!n_2!\cdots n_t!$ elementos. Como una clase de equivalencia queda determinada por un ordenamiento de S, el número de ordenamientos de S es igual al número de clases de equivalencia. Existen $n!$ permutaciones de X, de modo que, por el teorema 2.5.16, el número de ordenamientos de S es

$$\frac{n!}{n_1!n_2!\cdots n_t!}.$$

Ahora analizaremos el problema de contar las selecciones no ordenadas donde se permiten repeticiones.

EJEMPLO 4.4.4

Consideremos tres libros: un libro de computación, un libro de física y uno de historia. Supongamos que la biblioteca tiene al menos seis copias de cada uno de estos libros. ¿De cuántas formas pueden elegirse seis libros?

El problema consiste en obtener selecciones no ordenadas de seis elementos, entre el conjunto {computación, física, historia}, permitiendo repeticiones. Una selección queda determinada de manera única mediante el número de libros elegidos de cada tipo. Denotemos una selección particular como

Computación	Física	Historia
× × ×	| × × |	×

Aquí hemos designado la selección que consta de tres libros de computación, dos libros de física y un libro de historia. Otro ejemplo de selección es

Computación	Física	Historia
	| × × × × |	× ×

lo cual denota la selección que consta de ningún libro de computación, cuatro libros de física y dos libros de historia. Vemos que cada orden de seis símbolos × y dos símbolos | denota una selección. Así, nuestro problema consiste en contar el número de tales ordenamientos. Pero esto es precisamente el número de formas

$$C(8, 2) = 28$$

de elegir dos posiciones para los símbolos | entre ocho posibles posiciones. Así, existen 28 formas de elegir seis libros. $\square$

El método utilizado en el ejemplo 4.4.4 puede utilizarse para deducir un resultado general.

TEOREMA 4.4.5

Si X es un conjunto con t elementos, el número de selecciones no ordenadas de k elementos de X, permitiendo repeticiones, es

$$C(k + t - 1, t - 1) = C(k + t - 1, k).$$

Demostración. Sea $X = \{a_1, \ldots, a_t\}$. Consideremos los $k + t - 1$ espacios

_ _ _ _ _ _ _ _

y $k + t - 1$ símbolos que constan de k símbolos $\times$ y $t - 1$ símbolos $|$. Cada colocación de estos símbolos en los espacios determina una selección. El número n_1 de $\times$ hasta el primer $|$ representa la selección de n_1 símbolos a_1; el número n_2 de $\times$ entre el primer y el segundo $|$ representa la selección de n_2 símbolos a_2; y así sucesivamente. Como existen $C(k + t - 1, t - 1)$ formas de elegir las posiciones para los símbolos $|$, también existen $C(k + t - 1, t - 1)$ selecciones. Esto es igual a $C(k + t - 1, k)$, el número de formas de elegir las posiciones para los símbolos $\times$; por tanto, existen

$$C(k + t - 1, t - 1) = C(k + t - 1, k)$$

selecciones no ordenadas de k elementos de X, permitiendo repeticiones. ■

EJEMPLO 4.4.6

Supongamos que existen pilas de pelotas rojas, azules y verdes, y que cada pila contiene al menos ocho pelotas.

(a) ¿De cuántas formas podemos elegir ocho pelotas?

(b) ¿De cuántas formas podemos elegir ocho pelotas si debemos tener al menos una pelota de cada color?

Por el teorema 4.4.5, el número de formas de elegir ocho pelotas es

$$C(8 + 3 - 1, 3 - 1) = C(10, 2) = 45.$$

También podemos utilizar el teorema 4.4.5 para resolver la parte (b), si primero elegimos una pelota de cada color. Para completar la selección, debemos elegir otras cinco pelotas. Esto puede realizarse de

$$C(5 + 3 - 1, 3 - 1) = C(7, 2) = 21$$

formas. □

EJEMPLO 4.4.7

¿De cuántas formas pueden distribuirse 12 libros idénticos de matemáticas entre los estudiantes Anna, Beth, Candy y Dan?

Podemos utilizar el teorema 4.4.5 para resolver este problema, si lo consideramos como el problema de etiquetar cada libro con el nombre del estudiante que lo recibe. Esto es lo mismo que elegir 12 elementos (los nombres de los estudiantes) del conjunto {Anna, Beth, Candy, Dan}, permitiendo repeticiones. Por el teorema 4.4.5, el número de formas de hacer esto es

$$C(12 + 4 - 1, 4 - 1) = C(15, 3) = 455.$$ □

EJEMPLO 4.4.8

(a) ¿Cuántas soluciones enteras no negativas tiene la ecuación

$$x_1 + x_2 + x_3 + x_4 = 29? \tag{4.4.1}$$

(b) ¿Cuántas soluciones enteras tiene (4.4.1) y que satisfagan $x_1 > 0, x_2 > 1, x_3 > 2, x_4 \geq 0$?

(a) Cada solución de (4.4.1) es equivalente a elegir 29 elementos, x_i de tipo i, $i = 1, 2, 3, 4$. De acuerdo con el teorema 4.4.5, el número de selecciones es

$$C(29 + 4 - 1, 4 - 1) = C(32, 3) = 4960.$$

(b) Cada solución de (4.4.1) y que satisfaga las condiciones dadas es equivalente a seleccionar 29 elementos, x_i de tipo i, $i = 1, 2, 3, 4$, donde, además, debemos tener al menos un elemento de tipo 1, al menos dos elementos de tipo 2, y al menos tres elementos de tipo 3. En primer lugar, se elige un elemento de tipo 1, dos elementos de tipo 2 y tres elementos de tipo 3. Luego, se eligen otros 23 elementos. Por el teorema 4.4.5, esto puede hacerse de

$$C(23 + 4 - 1, 4 - 1) = C(26, 3) = 2600$$

formas. □

EJEMPLO 4.4.9

¿Cuántas veces se ejecuta la instrucción print?

```
for i_1 := 1 to n do
  for i_2 := 1 to i_1 do
    for i_3 := 1 to i_2 do
      .
      .
      .
        for i_k := 1 to i_{k-1} do
          print i_1, i_2, ..., i_k
```

Observe que cada línea de salida consta de k enteros

$$i_1 i_2 \cdots i_k, \tag{4.4.2}$$

donde

$$n \geq i_1 \geq i_2 \geq \cdots \geq i_k \geq 1, \tag{4.4.3}$$

y que aparece cada sucesión (4.4.2) que satisfaga (4.4.3). Así, el problema consiste en contar el número de formas de elegir k enteros, permitiendo repeticiones, en el conjunto $\{1, 2, \ldots, n\}$. [Cualquier selección de este tipo puede ordenarse para producir (4.4.3).] Por el teorema 4.4.5, el número total de selecciones posibles es

$$C(k + n - 1, k).$$ □

Ejercicios

En los ejercicios 1-3, determine el número de cadenas que pueden formarse ordenando las letras dadas.

1. *GUIDE* 2. *SCHOOL* 3. *SALESPERSONS*
4. ¿De cuántas formas pueden repartirse 10 libros distintos entre tres estudiantes, si el primero debe tener cinco libros, el segundo tres y el tercero dos?

Los ejercicios 5-11 se refieren a pilas de pelotas rojas, azules y verdes, idénticas, donde cada pila contiene al menos 10 pelotas.

5. ¿De cuántas formas pueden elegirse 10 pelotas?
6. ¿De cuántas formas pueden elegirse 10 pelotas si al menos se debe elegir una roja?
7. ¿De cuántas formas pueden elegirse 10 pelotas si al menos debe elegirse una pelota roja, al menos dos azules y al menos tres verdes?
8. ¿De cuántas formas pueden elegirse 10 pelotas si hay que elegir exactamente una pelota roja?
9. ¿De cuántas formas pueden elegirse 10 pelotas si hay que elegir al exactamente una pelota roja y al menos una pelota azul?
10. ¿De cuántas formas pueden elegirse 10 pelotas si a lo más se elige una pelota roja?
11. ¿De cuántas formas pueden elegirse 10 pelotas si hay que elegir el doble de pelotas rojas en relación con las pelotas verdes?

En los ejercicios 12-17, determine el número de soluciones enteras de

$$x_1 + x_2 + x_3 = 15$$

sujetas a las condiciones dadas.

12. $x_1 \geq 0, x_2 \geq 0, x_3 \geq 0$
13. $x_1 \geq 1, x_2 \geq 1, x_3 \geq 1$
14. $x_1 = 1, x_2 \geq 0, x_3 \geq 0$
15. $x_1 \geq 0, x_2 > 0, x_3 = 1$
16. $0 \leq x_1 \leq 6, x_2 \geq 0, x_3 \geq 0$
☆ 17. $0 \leq x_1 < 6, 1 \leq x_2 < 9, x_3 \geq 0$
★ 18. Determine el número de soluciones enteras de

$$x_1 + x_2 + x_3 + x_4 = 12$$

que satisfacen $0 \leq x_1 \leq 4, 0 \leq x_2 \leq 5, 0 \leq x_3 \leq 8$ y $0 \leq x_4 \leq 9$.

19. ¿Cuántos enteros entre 1 y 1,000,000 cumplen que la suma de sus dígitos es igual a 15?
☆ 20. ¿Cuántos enteros entre 1 y 1,000,000 cumplen que la suma de sus dígitos es igual a 20?
21. ¿Cuántas partidas de bridge existen? (Una partida consiste en repartir una baraja de 52 cartas en cuatro manos, cada una de las cuales contiene 13 cartas.)
22. ¿De cuántas formas pueden elegirse tres equipos con cuatro, dos y dos personas, respectivamente, en un grupo de ocho personas?
23. Un *dominó* es un rectángulo dividido en dos cuadrados, donde cada cuadrado tiene un número elegido entre $0, 1, \ldots, 6$, permitiendo repeticiones. ¿Cuántos dominós distintos existen?

Los ejercicios 24-29 se refieren a una bolsa que contiene 20 pelotas (seis rojas, seis verdes y ocho moradas).

24. ¿De cuántas formas podemos elegir cinco pelotas si las pelotas se consideran distintas unas de otras?
25. ¿De cuántas formas podemos elegir cinco pelotas si las pelotas del mismo color se consideran idénticas?

26. ¿De cuántas formas podemos extraer de la bolsa dos pelotas rojas, tres pelotas verdes y dos pelotas moradas si las pelotas se consideran distintas?
27. Extraemos cinco pelotas, luego las reemplazamos, y finalmente extraemos cinco pelotas más. ¿De cuántas formas puede hacerse esto si las bolas se consideran distintas?
28. Sacamos cinco pelotas sin reemplazarlas, y luego extraemos cinco pelotas más. ¿De cuántas formas puede hacerse esto si las bolas se consideran distintas?
29. Extraemos cinco pelotas donde al menos una es roja. Luego las reemplazamos. A continuación sacamos cinco pelotas y a lo más una es verde. ¿De cuántas formas puede hacerse esto si las pelotas se consideran distintas?
30. ¿De cuántas formas pueden distribuirse 15 libros idénticos de matemáticas entre seis estudiantes?
31. ¿De cuántas formas pueden distribuirse 15 libros idénticos de computación y 10 libros idénticos de psicología entre cinco estudiantes?
32. ¿De cuántas formas podemos colocar 10 pelotas idénticas en 12 cajas, si cada caja sólo puede contener una pelota?
33. ¿De cuántas formas podemos colocar 10 pelotas idénticas en 12 cajas, si cada caja puede contener 10 pelotas?
34. Muestre que $(kn)!$ es divisible entre $(n!)^k$.
35. Considere

```
for i1 := 1 to n do
    for i2 := 1 to i1 do
        print i1, i2
```

y el ejemplo 4.4.9 para deducir que

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

★ 36. Utilice el ejemplo 4.4.9 para demostrar la fórmula

$$C(k-1, k-1) + C(k, k-1) + \cdots + C(n+k-2, k-1) = C(k+n-1, k).$$

37. Escriba un algoritmo para enumerar todas las soluciones enteras no negativas de

$$x_1 + x_2 + x_3 = n.$$

38. ¿Cuál es el error del siguiente argumento, el cual supuestamente cuenta el número de particiones de un conjunto con 10 elementos en ocho subconjuntos (no vacíos)?

Enumeramos los elementos del conjunto con espacios en blanco entre ellos:

$$x_1 _ x_2 _ x_3 _ x_4 _ x_5 _ x_6 _ x_7 _ x_8 _ x_9 _ x_{10}.$$

Cada vez que ocupamos siete de los nueve espacios en blanco con siete barras verticales, obtenemos una partición de $\{x_1, \ldots, x_{10}\}$ en ocho subconjuntos. Por ejemplo, la partición $\{x_1\}, \{x_2\}, \{x_3, x_4\}, \{x_5\}, \{x_6\}, \{x_7, x_8\}, \{x_9\}, \{x_{10}\}$ se representaría como

$$x_1 \mid x_2 \mid x_3 x_4 \mid x_5 \mid x_6 \mid x_7 x_8 \mid x_9 \mid x_{10}.$$

Así, la solución a este problema es $C(9, 7)$.

4.5 *COEFICIENTES BINOMIALES E IDENTIDADES COMBINATORIAS*

A primera vista, la expresión $(a + b)^n$ no parece tener mucho que ver con las combinaciones, pero como veremos en esta sección, podemos obtener una fórmula para desarrollar $(a + b)^n$ utilizando la fórmula para el número de r-combinaciones de n objetos. Con frecuencia, podemos relacionar una expresión algebraica con algún proceso de conteo. Varias técnicas avanzadas de conteo utilizan estos métodos (véanse [Riordan y Tucker]).

El **teorema del binomio** proporciona una fórmula para los coeficientes en el desarrollo de $(a + b)^n$. Como

$$(a+b)^n = \underbrace{(a+b)(a+b)\cdots(a+b)}_{\text{factores de } n}, \tag{4.5.1}$$

el desarrollo surge al elegir a o b en cada uno de los n factores, multiplicando las selecciones entre ellas, y luego sumando todos los productos obtenidos de esta manera. Por ejemplo, en el desarrollo de $(a + b)^3$, se elige a o b en el primer factor $(a + b)$; a o b en el segundo factor $(a + b)$, y a o b en el tercer factor $(a + b)$; se multiplican las selecciones entre ellas y luego se suman los productos obtenidos. Si elegimos a en todos los factores y multiplicamos, obtenemos el término aaa. Si elegimos a en el primer factor, b en el segundo factor y a en el tercer factor y multiplicamos, obtenemos el término aba. La tabla 4.5.1 muestra todas las posibilidades. Si sumamos los productos de todas las selecciones, obtenemos

$$\begin{aligned}(a+b)^3 &= (a+b)(a+b)(a+b)\\ &= aaa + aab + aba + abb + baa + bab + bba + bbb\\ &= a^3 + a^2b + a^2b + ab^2 + a^2b + ab^2 + ab^2 + b^3\\ &= a^3 + 3a^2b + 3ab^2 + b^3.\end{aligned}$$

En (4.5.1), un término de la forma $a^{n-k}b^k$ surge de elegir b en k factores y a de los otros $n - k$ factores. Pero esto puede realizarse de $C(n, k)$ formas, pues $C(n, k)$ cuenta el número de formas de elegir k cosas de n elementos. Así, $a^{n-k}b^k$ aparece $C(n, k)$ veces. Esto implica que

$$\begin{aligned}(a+b)^n = {} & C(n,0)a^n b^0 + C(n,1)a^{n-1}b^1 + C(n,2)a^{n-2}b^2\\ & + \cdots + C(n,n-1)a^1b^{n-1} + C(n,n)a^0b^n.\end{aligned} \tag{4.5.2}$$

Este resultado se conoce como el teorema del binomio.

TEOREMA 4.5.1 *Teorema del binomio*

Si a y b son números reales y n es un entero positivo, entonces

$$(a+b)^n = \sum_{k=0}^{n} C(n,k)a^{n-k}b^k.$$

Demostración. La demostración aparece antes del enunciado del teorema. ■

El teorema del binomio también puede demostrar por inducción sobre n (véase el ejercicio 16).

TABLA 4.5.1
Cálculo de $(a + b)^3$

Selección del primer factor $(a + b)$	*Selección del segundo factor* $(a + b)$	*Selección del tercer factor* $(a + b)$	*Producto de selecciones*
a	a	a	$aaa = a^3$
a	a	b	$aab = a^2b$
a	b	a	$aba = a^2b$
a	b	b	$abb = ab^2$
b	a	a	$baa = a^2b$
b	a	b	$bab = ab^2$
b	b	a	$bba = ab^2$
b	b	b	$bbb = b^3$

Los números $C(n, r)$ se conocen como los **coeficientes binomiales**, pues aparecen en el desarrollo (4.5.2) del binomio elevado a una potencia.

EJEMPLO 4.5.2

Al considerar $n = 3$ en el teorema 4.5.1, obtenemos

$$\begin{aligned}(a+b)^3 &= C(3,0)a^3b^0 + C(3,1)a^2b^1 + C(3,2)a^1b^2 + C(3,3)a^0b^3\\ &= a^3 + 3a^2b + 3ab^2 + b^3.\end{aligned}$$

□

EJEMPLO 4.5.3

Desarrollar $(3x - 2y)^4$ utilizando el teorema del binomio.

Si hacemos $a = 3x$, $b = -2y$ y $n = 4$ en el teorema 4.5.1, obtenemos

$$\begin{aligned}(3x-2y)^4 &= (a+b)^4\\ &= C(4,0)a^4b^0 + C(4,1)a^3b^1 + C(4,2)a^2b^2\\ &\quad + C(4,3)a^1b^3 + C(4,4)a^0b^4\\ &= C(4,0)(3x)^4(-2y)^0 + C(4,1)(3x)^3(-2y)^1\\ &\quad + C(4,2)(3x)^2(-2y)^2 + C(4,3)(3x)^1(-2y)^3\\ &\quad + C(4,4)(3x)^0(-2y)^4\\ &= 3^4x^4 + 4\cdot 3^3x^3(-2y) + 6\cdot 3^2x^2(-2)^2y^2\\ &\quad + 4(3x)(-2)^3y^3 + (-2)^4y^4\\ &= 81x^4 - 216x^3y + 216x^2y^2 - 96xy^3 + 16y^4.\end{aligned}$$

□

EJEMPLO 4.5.4

Determine el coeficiente de a^5b^4 en el desarrollo de $(a + b)^9$.

El término relacionado con a^5b^4 aparece en el teorema del binomio considerando $n = 9$ y $k = 4$:

$$C(n, k)a^{n-k}b^k = C(9, 4)a^5b^4 = 126a^5b^4.$$

Así, el coeficiente de a^5b^4 es 126. □

EJEMPLO 4.5.5

Determine el coeficiente de $x^2y^3z^4$ en el desarrollo de $(x + y + z)^9$.

Como

$$(x + y + z)^9 = (x + y + z)(x + y + z)\cdots(x + y + z) \quad \text{(nueve términos)},$$

obtenemos $x^2y^3z^4$ cada vez que multiplicamos dos x elegidas de los nueve términos, tres y elegidas de los nueve términos y cuatro z elegidas de los nueve términos. Podemos elegir dos términos para las x de $C(9, 2)$ formas. Una vez realizada esta selección, podemos elegir tres términos para las y de $C(7, 3)$ formas. Esto deja los cuatro términos restantes para las z. Así, el coeficiente de $x^2y^3z^4$ en el desarrollo de $(x + y + z)^9$ es

$$C(9,2)\,C(7,3) = \frac{9!}{2!7!}\frac{7!}{3!4!} = \frac{9!}{2!3!4!} = 1260.$$ □

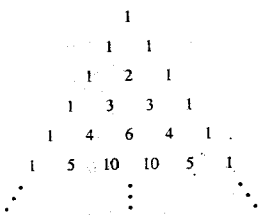


FIGURA 4.5.1
Triángulo de Pascal.

Podemos escribir los coeficientes binomiales en una forma triangular conocida como el **triángulo de Pascal** (véase la figura 4.5.1). El borde consta de unos y cualquier valor interior es la suma de los dos números arriba de él. Esta relación se enuncia de manera formal en el siguiente teorema. La demostración es un argumento combinatorio. Una identidad que surge de algún proceso de conteo es una **identidad combinatoria** y el argumento que conduce a su formulación es un **argumento combinatorio**.

TEOREMA 4.5.6

$$C(n + 1, k) = C(n, k - 1) + C(n, k)$$

para $1 \le k \le n$.

Demostración. Sea X un conjunto con n elementos. Sea $a \notin X$. Entonces $C(n + 1, k)$ es el número de subconjuntos de $Y = X \cup \{a\}$ con k elementos. Ahora, los subconjuntos de Y con k elementos pueden separar en dos clases ajenas:

1. Los subconjuntos de Y que no contienen a a.
2. Los subconjuntos de Y que contienen a a.

Los subconjuntos de la clase 1 son precisamente los subconjuntos de X con k elementos y existen $C(n, k)$ de ellos. Cada subconjunto de la clase 2 consta de un subconjunto de X con $(k - 1)$ elementos, junto con a, y existen $C(n, k - 1)$ de ellos. Por tanto,

$$C(n + 1, k) = C(n, k - 1) + C(n, k).$$ ■

El teorema 4.5.6 también puede demostrar mediante el teorema 4.2.17 (ejercicio 17).

Concluimos esta sección mostrando la forma en que el teorema del binomio (teorema 4.5.1) y el teorema 4.5.6 pueden utilizarse para deducir otras identidades combinatorias.

EJEMPLO 4.5.7

Utilice el teorema del binomio para deducir la ecuación

$$\sum_{k=0}^{n} C(n, k) = 2^n.$$

La suma es igual a la suma del teorema del binomio

$$\sum_{k=0}^{n} C(n, k)\,a^{n-k}b^k.$$

excepto que falta la expresión $a^{n-k}b^k$. Una forma de "eliminar" esta expresión es considerar $a = b = 1$, en cuyo caso el teorema del binomio se convierte en

$$2^n = (1+1)^n = \sum_{k=0}^{n} C(n, k)1^{n-k}1^k = \sum_{k=0}^{n} C(n, k). \tag{4.5.3}$$

□

La ecuación (4.5.3) también puede demostrarse mediante un argumento combinatorio. Dado un conjunto X con n elementos, $C(n, k)$ cuenta el número de subconjuntos con k elementos. Así, el lado derecho de la ecuación (4.5.3) cuenta el número de subconjuntos de X. Pero el número de subconjuntos de X es 2^n; con esto hemos demostrado de nuevo la ecuación (4.5.3).

EJEMPLO 4.5.8

Utilice el teorema 4.5.6 para mostrar que

$$\sum_{i=k}^{n} C(i, k) = C(n+1, k+1). \tag{4.5.4}$$

Utilizamos el teorema 4.5.6 en la forma

$$C(i, k) = C(i + 1, k + 1) - C(i, k + 1)$$

para obtener

$$\begin{aligned} &C(k, k) + C(k + 1, k) + C(k + 2, k) + \cdots + C(n, k) \\ &\quad = 1 + C(k + 2, k + 1) - C(k + 1, k + 1) + C(k + 3, k + 1) \\ &\quad\quad - C(k + 2, k + 1) + \cdots + C(n + 1, k + 1) - C(n, k + 1) \\ &\quad = C(n + 1, k + 1). \end{aligned}$$ □

El ejercicio 36 de la sección 4.4 muestra otra forma de demostrar (4.5.4).

EJEMPLO 4.5.9

Utilice la ecuación (4.5.4) para determinar la suma

$$1 + 2 + \cdots + n.$$

Podemos escribir

$$\begin{aligned} 1 + 2 + \cdots + n &= C(1, 1) + C(2, 1) + \cdots + C(n, 1) \\ &= C(n + 1, 2) \quad \text{por la ecuación (4.5.4)} \\ &= \frac{(n + 1)n}{2}. \end{aligned}$$

□

Ejercicios

1. Desarrolle $(x + y)^4$ mediante el teorema del binomio.
2. Desarrolle $(2c - 3d)^5$ mediante el teorema del binomio.

En los ejercicios 3-9, determine el coeficiente que tendrá el término indicado al desarrollar la expresión.

3. $x^4y^7; (x + y)^{11}$
4. $s^6t^6; (2s - t)^{12}$
5. $x^2y^3z^5; (x + y + z)^{10}$
6. $w^2x^3y^2z^5; (2w + x + 3y + z)^{12}$
7. $a^2x^3; (a + x + c)^2(a + x + d)^3$
8. $a^2x^3; (a + ax + x)(a + x)^4$
9. $a^3x^4; (a + \sqrt{ax} + x)^2(a + x)^5$

En los ejercicios 10-12, determine el número de términos en el desarrollo de cada expresión.

10. $(x + y + z)^{10}$
11. $(w + x + y + z)^{12}$
☆ 12. $(x + y + z)^{10}(w + x + y + z)^2$
13. Determine el siguiente renglón del triángulo de Pascal, dado el renglón

$$1 \quad 7 \quad 21 \quad 35 \quad 35 \quad 21 \quad 7 \quad 1.$$

14. (a) Muestre que $C(n, k) < C(n, k + 1)$ si y sólo si $k < (n - 1)/2$.
    (b) Utilice la parte (a) para deducir que el máximo de $C(n, k)$ para $k = 0, 1, \ldots, n$ es $C(n, \lfloor n/2 \rfloor)$.
15. Utilice el teorema del binomio para demostrar que

$$0 = \sum_{k=0}^{n} (-1)^k C(n, k).$$

16. Utilice inducción sobre n para demostrar el teorema del binomio.
17. Demuestre el teorema 4.5.6 utilizando el teorema 4.2.17.
18. Proporcione un argumento combinatorio para demostrar que

$$C(n, k) = C(n, n - k).$$

★ 19. Demuestre la ecuación (4.5.4) dando un argumento combinatorio.

20. Determine la suma

$$1 \cdot 2 + 2 \cdot 3 + \cdots + (n - 1)n.$$

☆ 21. Utilice la ecuación (4.5.4) para deducir una fórmula para

$$1^2 + 2^2 + \cdots + n^2.$$

22. Utilice el teorema del binomio para mostrar que

$$\sum_{k=0}^{n} 2^k C(n, k) = 3^n.$$

23. Suponga que n es par. Demuestre que

$$\sum_{k=0}^{n/2} C(n, 2k) = 2^{n-1} = \sum_{k=1}^{n/2} C(n, 2k - 1).$$

24. Demuestre

$$(a + b + c)^n = \sum_{0 \le i+j \le n} \frac{n!}{i!\,j!(n - i - j)!} a^i b^j c^{n-i-j}.$$

25. Utilice el ejercicio 24 para escribir el desarrollo de $(x + y + z)^3$.
26. Demuestre

$$3^n = \sum_{0 \le i-j \le n} \frac{n!}{i!\,j!(n - i - j)!}.$$

☆ 27. Proporcione un argumento combinatorio para demostrar que

$$\sum_{k=0}^{n} C(n, k)^2 = C(2n, n).$$

28. Demuestre

$$n(1 + x)^{n-1} = \sum_{k=1}^{n} C(n, k) k x^{k-1}.$$

29. Utilice el resultado del ejercicio 28 para mostrar que

$$n2^{n-1} = \sum_{k=1}^{n} kC(n, k). \tag{4.5.5}$$

☆ 30. Demuestre la ecuación (4.5.5) por inducción.
31. Una *sucesión suavizante* $b_0, \ldots, b_{k-1}$ es una sucesión (finita) que satisface $b_i \ge 0$ para $i = 0, \ldots, k - 1$ y $\sum_{i=0}^{k-1} b_i = 1$. Una *suavización de la sucesión (infinita)* $a_1, a_2, \ldots$ *por medio de la sucesión suavizante* $b_0, \ldots, b_{k-1}$ es la sucesión $\{a'_j\}$ definida como

$$a'_j = \sum_{i=0}^{k-1} a_{i+j} b_i.$$

La idea es que al tomar promedios se suavizan los datos con ruido.

El *suavizante binomial de tamaño k* es la sucesión

$$\frac{B_0}{2^n}, \ldots, \frac{B_{k-1}}{2^n},$$

donde $B_0, \ldots, B_{k-1}$ es el n-ésimo renglón del triángulo de Pascal (el renglón 0 es el superior).

Sea c_0, c_1 la sucesión suavizante definida como $c_0 = c_1 = \frac{1}{2}$. Muestre que si una sucesión a es suavizada por c, luego la sucesión resultante suavizada por c, y así sucesivamente k veces; entonces, la sucesión resultante puede obtenerse mediante suavización, valga la repetición, del suavizante binomial de tamaño $k + 1$ sobre a.

32. En el ejemplo 4.1.5 mostramos que existen 3^n pares ordenados (A, B) que satisfacen $A \subseteq B \subseteq X$, donde X es un conjunto con n elementos. Deduzca este resultado considerando los casos $|A| = 0, |A| = 1, \ldots, |A| = n$, y utilizando después el teorema del binomio.

4.6 EL PRINCIPIO DE LA PICHONERA

El **principio de la pichonera** (también conocido como *principo de las casillas, principio de las gavetas de Dirichlet* o *principio de las cajas de zapatos*) sirve en ciertas ocasiones para responder la pregunta: ¿Existe un elemento con una propiedad dada? Si puede aplicar el principio de la pichonera, este principio sólo dice que el objeto existe; el principio no indica la forma de determinar al objeto o el número de ellos.

Nuestra primera versión del principio de la pichonera afirma que si n palomas vuelan a k pichoneras y $k < n$, alguna pichonera contendrá al menos dos palomas (véase la figura 4.6.1). La razón de la verdad de este enunciado puede verse argumentando por contradicción. Si la conclusión fuese falsa, cada pichonera contendría a lo más una paloma y, en este caso, habría a lo más k palomas. Como existen n palomas y $n > k$, tenemos una contradicción.

PRINCIPIO DE LA PICHONERA (*Primera forma*)

Si n palomas vuelan hacia k pichoneras y $k < n$, alguna pichonera contiene al menos dos palomas.

Observemos que el principio de la pichonera no nos dice cómo localizar la pichonera que contiene dos o más palomas. Sólo afirma la *existencia* de una pichonera con dos o más palomas.

Para aplicar el principio de la pichonera, debemos decidir cuáles objetos juegan el papel de las palomas y cuáles objetos juegan el papel de los palomares. Nuestro primer ejemplo ilustra una posibilidad.

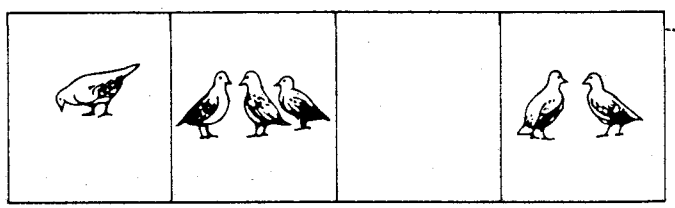


FIGURA 4.6.1 $n = 6$ palomas en $k = 4$ pichoneras. Alguna pichonera contiene al menos dos palomas.

EJEMPLO 4.6.1

Los nombres de diez personas son Alice, Bernard y Charles, mientras que sus apellidos son Lee, McDuff y Ng. Muestre que al menos dos personas tienen los mismos nombres y apellidos.

Existen nueve nombres posibles para las 10 personas. Si pensamos que las personas son las palomas y los nombres son los palomares, podemos considerar la asignación de nombres a las personas como la asignación de palomares a las palomas. Por el principio de la pichonera, algún nombre (pichonera) se asigna por lo menos a dos personas (palomas). □

A continuación enunciamos el principio de la pichonera de forma alternativa.

PRINCIPIO DE LA PICHONERA (*Segunda forma*)

Si f es una función de un conjunto finito X a un conjunto finito Y y $|X| > |Y|$, entonces $f(x_1) = f(x_2)$ para $x_1, x_2 \in X$, $x_1 \neq x_2$.

La segunda forma del principio de la pichonera puede reducirse a la primera, si X es el conjunto de palomas y Y el conjunto de pichoneras. Asignamos la paloma x a la pichonera $f(x)$. Por la primera forma del principio de la pichonera, al menos dos palomas, $x_1, x_2 \in X$, se asignan a la misma pichonera; es decir, $f(x_1) = f(x_2)$ para $x_1, x_2 \in X$, $x_1 \neq x_2$.

Nuestro siguiente ejemplo ilustra el uso de la segunda forma del principio de la pichonera.

EJEMPLO 4.6.2

Si se conectan unos con otros 20 procesadores, muestre que al menos dos procesadores se conectan directamente al mismo número de procesadores.

Designemos los procesadores como $1, 2, \ldots, 20$. Sea a_i el número de procesadores a los cuales está directamente conectado el procesador i. Debemos mostrar que $a_i = a_j$ para algún $i \neq j$. El dominio de la función a es $X = \{1, 2, \ldots, 20\}$ y el rango Y es algún subconjunto de $\{0, 1, \ldots, 19\}$. Por desgracia, $|X| = |\{0, 1, \ldots, 19\}|$ y no podemos utilizar de manera inmediata la segunda forma del principio de la pichonera.

Examinemos la situación con más detalle. Observe que no podemos tener $a_i = 0$, para algún i, y $a_j = 19$, para algún j, ya que entonces tendríamos un procesador (el i-ésimo) no conectado a los demás, mientras que, al mismo tiempo, algún otro procesador (el j-ésimo) estaría conectado a todos los demás procesadores (incluyendo al i-ésimo). Así, el rango Y es un subconjunto de $\{0, 1, \ldots, 18\}$ o $\{1, 2, \ldots, 19\}$. En cualquier caso, $|Y| < 20 = |X|$. Por la segunda forma del principio de la pichonera, $a_i = a_j$ para algún $i \neq j$, como se deseaba demostrar. □

EJEMPLO 4.6.3

Muestre que si se eligen 151 cursos distintos entre 300 cursos de computación numerados del 1 al 300 inclusive, al menos dos están numerados en forma consecutiva.

Digamos que los números de los cursos elegidos son

$$c_1, \quad c_2, \quad \ldots, \quad c_{151}. \tag{4.6.1}$$

Los 302 números que constan de (4.6.1) junto con

$$c_1 + 1, \quad c_2 + 1, \quad \ldots, \quad c_{151} + 1. \tag{4.6.2}$$

varían entre 1 y 301. Por la segunda forma del principio de la pichonera, al menos dos de estos valores coinciden. Los números (4.6.1) son todos distintos y por tanto los números (4.6.2) también son distintos. Entonces, debe ocurrir que uno de los números (4.6.1) y uno de los números (4.6.2) sean iguales. Así, tenemos

$$c_i = c_j + 1$$

y por supuesto, c_i es el sucesor de c_j. □

EJEMPLO 4.6.4

Un inventario consta de una lista de 80 artículos, cada uno marcado como "disponible" o "no disponible". Existen 45 artículos disponibles. Muestre que existen al menos dos artículos disponibles en la lista a exactamente nueve artículos de distancia. (Por ejemplo, los artículos disponibles en las posiciones 13 y 22 o en las posiciones 69 y 78 satisfacen la condición.)

Sea a_i la posición del i-ésimo artículo disponible. Debemos mostrar que $a_i - a_j = 9$ para algunos i y j. Consideremos los números

$$a_1, \quad a_2, \quad \ldots, \quad a_{45} \tag{4.6.3}$$

y

$$a_1 + 9, \quad a_2 + 9, \quad \ldots, \quad a_{45} + 9. \tag{4.6.4}$$

Los 90 números en (4.6.3) y (4.6.4) sólo pueden variar entre 1 y 89 inclusive. Por la segunda forma del principio de la pichonera, dos de los números deben coincidir. No podemos tener dos números de (4.6.3) o de (4.6.4) idénticos; así, algún número en (4.6.3) es igual a algún número de (4.6.4). Por tanto, $a_i - a_j = 9$ para algunos i y j, como se deseaba. □

A continuación enunciamos otra forma más del principio de la pichonera.

PRINCIPIO DE LA PICHONERA *(Tercera forma)*

Sea f una función de un conjunto finito X en un conjunto finito Y. Suponga que $|X| = n$ y $|Y| = m$. Sea $k = \lceil n/m \rceil$. Entonces existen al menos k valores $a_1, \ldots, a_k \in X$ tales que

$$f(a_1) = f(a_2) = \cdots = f(a_k).$$

Para demostrar la tercera forma del principio de la pichonera, argumentamos por contradicción. Sea $Y = \{y_1, \ldots, y_m\}$. Supongamos que la conclusión es falsa. Entonces existen a lo más $k - 1$ valores $x \in X$ con $f(x) = y_1$; existen a lo más $k - 1$ valores $x \in X$ con $f(x) = y_2$; ...; existen a lo más $k - 1$ valores $x \in X$ con $f(x) = y_m$. Así, existen a lo más $m(k - 1)$ miembros en el dominio de f. Pero

$$m(k-1) < m\frac{n}{m} = n,$$

lo cual es una contradicción. Por tanto, existen al menos k valores $a_1, \ldots, a_k \in X$ tales que

$$f(a_1) = f(a_2) = \cdots = f(a_k).$$

Nuestro último ejemplo ilustra el uso de la tercera forma del principio de la pichonera.

EJEMPLO 4.6.5

Una característica útil de las fotos en blanco y negro es la brillantez promedio de la foto. Decimos que dos fotos son similares si su brillantez promedio no difiere más allá de cierto valor fijo. Muestre que entre seis fotos, hay tres que son mutuamente similares o tres que no son mutuamente similares.

Denotemos las fotos por $P_1, P_2, \ldots, P_6$. Cada uno de los cinco pares

$$(P_1, P_2), \quad (P_1, P_3), \quad (P_1, P_4), \quad (P_1, P_5), \quad (P_1, P_6),$$

tiene el valor "similar" o "no similar". Por la tercera forma del principio de la pichonera, existen al menos $\lceil 5/2 \rceil = 3$ pares con el mismo valor; es decir, existen tres pares de fotos

$$(P_1, P_i), \quad (P_1, P_j), \quad (P_1, P_k)$$

todas similares o todas no similares. Supongamos que cada par es similar. (El caso en que cada par no es similar está en el ejercicio 8.) Si cualquier par

$$(P_i, P_j), \quad (P_i, P_k), \quad (P_j, P_k) \tag{4.6.5}$$

es similar, entonces estas dos fotos, junto con P_1, son mutuamente similares, con lo que hemos encontrado tres fotos mutuamente similares. En caso contrario, cada uno de los pares (4.6.5) no es similar y hemos determinado tres fotos que no son mutuamente similares. □

Ejercicios

1. Trece personas tienen por nombres Dennis, Evita y Ferdinand, y por apellidos Oh, Pietro, Quine y Rostenkowski. Muestre que al menos dos personas tienen el mismo nombre y apellido.
2. 18 personas tienen por nombres Alfie, Ben y Cissi, y por apellidos Dumont y Elm. Muestre que al menos tres personas tienen el mismo nombre y apellido.
3. El profesor Euclides recibe su pago por semanas alternadas, en viernes. Muestre que en algún mes se le paga en tres ocasiones.
4. ¿Es posible conectar cinco procesadores unos con otros, de modo que exactamente dos procesadores estén conectados de manera directa con un número idéntico de procesadores? Explique.
5. Un inventario consta de una lista de 115 artículos, cada uno marcado "disponible" o "no disponible". Existen 60 artículos disponibles. Muestre que existen al menos dos artículos disponibles en la lista que están a cuatro artículos de distancia.
6. Un inventario consta de una lista de 100 artículos, cada uno marcado "disponible" o "no disponible". Existen 55 artículos disponibles. Muestre que existen al menos dos artículos disponibles en la lista que están a nueve artículos de distancia.
7. ★ Un inventario consta de una lista de 80 artículos, cada uno marcado "disponible" o "no disponible". Existen 50 artículos disponibles. Muestre que existen al menos dos artículos no disponibles en la lista que están a tres o a seis artículos de distancia.
8. Complete el ejemplo 4.6.5, mostrando que si los pares $(P_1, P_i), (P_1, P_j), (P_1, P_k)$ no son similares, entonces existen tres fotos que son mutuamente similares o que no son mutuamente similares.
9. ¿Necesariamente se cumple la conclusión del ejemplo 4.6.5 si existen menos de seis fotos? Explique.
10. ¿Necesariamente se cumple la conclusión del ejemplo 4.6.5 si existen más de seis fotos? Explique.

Responda los ejercicios 11-14 para dar un argumento que muestre que si X es cualquier subconjunto de $\{1, 2, \ldots, 2n + 1\}$ con $(n + 2)$ elementos y m es el máximo elemento en X, entonces existen i y j distintos en X tales que $m = i + j$.

Para cada elemento $k \in X - \{m\}$, sea

$$a_k = \begin{cases} k & \text{si } k \le \dfrac{m}{2} \\ m - k & \text{si } k > \dfrac{m}{2}. \end{cases}$$

11. ¿Cuántos elementos tiene el dominio de a?
12. Muestre que el rango de a está contenido en $\{1, 2, \ldots, n\}$.
13. Explique por qué los ejercicios 11 y 12 implican que $a_i = a_j$ para algunos $i \neq j$.
14. Explique por qué el ejercicio 13 implica que existen i y j distintos en X tales que $m = i + j$.
15. Proporcione un ejemplo de un subconjunto de $\{1, 2, \ldots, 2n + 1\}$ con $(n + 1)$ elementos con la siguiente propiedad: No existen $i, j \in X$ tales que $i + j \in X$.

Responda los ejercicios 16-19 para proporcionar un argumento que demuestre el siguiente resultado.

> Una sucesión $a_1, a_2, \ldots, a_{n^2+1}$ de $n^2 + 1$ números distintos contiene una subsucesión creciente de longitud $n + 1$ o una subsucesión decreciente de longitud $n + 1$.

Supongamos, por contradicción, que toda subsucesión creciente o decreciente tiene longitud n o menor. Sea b_i la longitud de la subsucesión creciente más grande, que comience en a_i y sea c_i la longitud de la subsucesión decreciente más pequeña que comienza en a_i.

16. Muestre que los pares ordenados (b_i, c_i), $i = 1, \ldots, n^2 + 1$, son distintos.
17. ¿Cuántos pares ordenados (b_i, c_i) existen?
18. Explique por qué $1 \le b_i \le n$ y $1 \le c_i \le n$.
19. ¿Cuál es la contradicción?

Responda los ejercicios 20-23 para dar un argumento que muestre que en un grupo de 10 personas existen al menos dos tales que la suma o resta de sus edades es divisible entre 16. Suponga que las edades se dan como números enteros no negativos.

Sean $a_1, \ldots, a_{10}$ las edades. Sea $r_i = a_i \bmod 16$ y sea

$$s_i = \begin{cases} r_i & \text{si } r_i \le 8 \\ 16 - r_i & \text{si } r_i > 8. \end{cases}$$

20. Muestre que $s_1, \ldots, s_{10}$ varían entre 0 y 8.
21. Explique por qué $s_j = s_k$ para algún $j \neq k$.
22. Explique por qué si $s_j = r_j$ y $s_k = r_k$ o $s_j = 16 - r_j$ y $s_k = 16 - r_k$, entonces 16 divide a $a_j - a_k$.
23. Muestre que si no se cumplen las condiciones del ejercicio 22, entonces 16 divide a $a_j + a_k$.
24. Muestre que en el desarrollo decimal del cociente de dos enteros, en algún momento se repite un bloque de dígitos. *Ejemplos*:

$$\frac{1}{6} = 0.1\underline{6}66\ldots, \qquad \frac{217}{660} = 0.32\underline{878}787\ldots$$

25. Doce jugadores de básquetbol, cuyos uniformes están numerados del 1 al 12, se paran en torno del centro de una cancha en un orden arbitrario. Muestre que existen tres jugadores consecutivos tales que la suma de sus números es al menos 20.
26. Para la situación del ejercicio 25, determine y demuestre una estimación de la magnitud de la suma de los números de cuatro jugadores consecutivos.
27. Sea f una función uno a uno de $X = \{1, 2, \ldots, n\}$ sobre X. Sea $f^k = f \circ f \circ \cdots \circ f$ la k-ésima composición de f con sí misma. Muestre que existen enteros positivos distintos i y j tales que $f^i(x) = f^j(x)$ para todo $x \in X$. Muestre que para algún entero positivo k, $f^k(x) = x$ para todo $x \in X$.
28. Un rectángulo 3×7 se divide en 21 cuadrados, cada uno de los cuales se colorea rojo o negro. Demuestre que el tablero contiene un rectángulo no trivial (que no sea $1 \times k$ ni $k \times 1$) cuyas cuatro esquinas cuadradas sean todas negras o todas rojas.
29. Demuestre que si p unos y q ceros se colocan en torno de un círculo de manera arbitraria, donde p, q y k son enteros positivos tales que $p \ge kq$, el arreglo debe contener al menos k unos consecutivos.
30. Escriba un algoritmo tal que, dada una sucesión a, determine la longitud de la máxima subsucesión creciente de a.

NOTAS

Un libro elemental relativo a los métodos de conteo es [Niven]. La bibliografía para combinatoria es [Brualdi; Even, 1973; Liu, 1968; Riordan; y Roberts]. [Vilenkin] contiene muchos ejemplos combinatorios resueltos. Las principales referencias relativas a la matemática discreta [Liu, 1985; y Tucker] dedican varias secciones a los temas del capítulo 4. [Even, 1973; Hu; y Reingold] analizan los algoritmos combinatorios.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 4.1

Principio de multiplicación

Principio de la suma

Sección 4.2

Permutación de $x_1, \ldots, x_n$: Ordenamiento de $x_1, \ldots, x_n$

$n!$ = número de permutaciones de un conjunto con n elementos

r-permutación de $x_1, \ldots, x_n$: Ordenamiento de r elementos de $x_1, \ldots, x_n$

$P(n, r)$: Número de r-permutaciones de un conjunto con n elementos; $P(n, r) = n(n-1)\cdots(n-r+1)$

r-combinación de $\{x_1, \ldots, x_n\}$: Subconjunto (no ordenado) de $\{x_1, \ldots, x_n\}$ con r elementos

$C(n, r)$: Número de r-combinaciones de un conjunto con n elementos; $C(n, r) = P(n, r)/r! = n!/[(n-r)!r!]$

Sección 4.3

Orden lexicográfico

Algoritmo para generar r-combinaciones: Algoritmo 4.3.9

Algoritmo para generar permutaciones: Algoritmo 4.3.14

Sección 4.4

Número de ordenamientos de n elementos de t tipos, con n_i objetos idénticos del tipo $i = n!/[n_1! \cdots n_t!]$

Número de selecciones de k elementos, sin considerar el orden, extraídos de un conjunto con t elementos, permitiendo repeticiones $= C(k + t - 1, k)$

Sección 4.5

Teorema del binomio:

$$(a+b)^n = \sum_{k=0}^{n} C(n,k)\, a^{n-k} b^k$$

Triángulo de Pascal: $C(n + 1, k) = C(n, k - 1) + C(n, k)$

Sección 4.6

Principio de la pichonera (tres formas)

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 4.1

1. ¿Cuántas cadenas de ocho bits comienzan con 0 y terminan con 101?
2. ¿De cuántas formas podemos elegir tres libros, cada uno de un tema distinto, de un conjunto de seis libros diferentes de historia, nueve libros distintos de literatura clásica, siete libros diferentes de derecho y cuatro libros distintos de enseñanza?
3. ¿Cuántas funciones existen de un conjunto de n elementos sobre $\{0, 1\}$?
4. Un comité de siete personas compuesto por Greg, Hwang, Isaac, Jasmine, Kirk, Lynn y Manuel debe elegir un presidente, un vicepresidente, un presidente de eventos sociales, un secretario y un tesorero. ¿De cuántas formas pueden otorgarse los puestos si Greg es el secretario o bien no tiene un puesto?

Sección 4.2

5. ¿Cuántas 3-combinaciones de seis objetos existen?
6. ¿Cuántas cadenas pueden formarse ordenando las letras $ABCDEF$ si A aparece antes que C y E aparece antes que C?
7. ¿Cuántas manos de seis cartas extraídas de una baraja normal con 52 cartas contienen tres cartas de un palo y tres cartas de otro palo?
8. Un embarque de 100 discos compactos contiene cinco discos defectuosos. ¿De cuántas formas puede elegirse un conjunto de cuatro discos compactos que contenga más discos defectuosos que no defectuosos?

Sección 4.3

9. Determine la 5-combinación posterior a 12467 que será generada por el algoritmo 4.3.9, si $n = 7$.
10. Determine la 6-combinación posterior a 145678 que será generada por el algoritmo 4.3.9, si $n = 8$.
11. Determine la permutación posterior a 6427135 que será generada por el algoritmo 4.3.14.
12. Determine la permutación posterior a 625431 que será generada por el algoritmo 4.3.14.

Sección 4.4

13. ¿Cuántas cadenas pueden formarse ordenando las letras de la palabra *ILLINOIS*?
14. ¿Cuántas cadenas pueden formarse ordenando las letras de la palabra *ILLINOIS* si alguna I aparece antes de alguna L?
15. ¿De cuántas formas pueden repartirse 12 libros distintos entre cuatro estudiantes, si cada estudiante debe tener cuatro libros?
16. ¿Cuántas soluciones enteras de

$$x_1 + x_2 + x_3 + x_4 = 17$$

satisfacen $x_1 \geq 0, x_2 \geq 1, x_3 \geq 2, x_4 \geq 3$?

Sección 4.5

17. Desarrolle la expresión $(s - r)^4$ mediante el teorema del binomio.
18. Determine el coeficiente de x^3yz^4 en el desarrollo de $(2x + y + z)^8$.
19. Utilice el teorema del binomio para demostrar que

$$\sum_{k=0}^{n} 2^{n-k}(-1)^k C(n,k) = 1.$$

20. Gire el triángulo de Pascal en contra del sentido de las manecillas del reloj, de modo que el renglón superior conste de unos. Explique por qué el segundo renglón enumera los enteros positivos en orden: $1, 2, \ldots$.

Sección 4.6

21. Muestre que todo conjunto de 15 calcetines elegidos entre 14 pares de calcetines tiene al menos un par correcto.
22. 19 personas tienen por nombre Zeke, Wally o Linda, por segundo nombre Lee y David, y por apellido Yu, Zamora y Smith. Muestre que al menos dos personas tienen los mismos primeros nombres, segundos nombres y apellidos.
23. Un inventario consta de una lista de 200 artículos, cada uno marcado como "disponible" o "no disponible". Existen 110 artículos disponibles. Muestre que existen al menos dos artículos disponibles en la lista a exactamente 19 artículos de distancia.
24. Sea $P = \{p_1, p_2, p_3, p_4, p_5\}$ un conjunto de cinco puntos (distintos) en el plano euclidiano ordinario, cada uno de los cuales tiene coordenadas enteras. Muestre que para algún par, su punto medio tiene coordenadas enteras.

¿Ya me vas a decir? ¿Decirte qué? Que es lo que quieres saber. Sabes, es gracioso. Estás tratando de saber aquello que debo averiguar para tu padre y yo estoy tratando de descubrir por qué lo quieres saber. Podrías continuar así por siempre, ¿no?

—The Big Sleep.

5

RELACIONES DE RECURRENCIA

5.1 INTRODUCCIÓN
5.2 SOLUCIÓN DE RELACIONES DE RECURRENCIA
RINCÓN DE SOLUCIÓN DE PROBLEMAS: RELACIONES DE RECURRENCIA
5.3 APLICACIONES AL ANÁLISIS DE ALGORITMOS
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Este capítulo ofrece una introducción a las relaciones de recurrencia, las cuales son útiles en ciertos problemas de conteo. Una relación de recurrencia relaciona el n-ésimo elemento de una sucesión con sus predecesores. Debido a que las relaciones de recurrencia están íntimamente relacionadas con los algoritmos recursivos, dichas relaciones surgen de manera natural en el análisis de este tipo de algoritmos.

5.1 *INTRODUCCIÓN*

Consideremos las siguientes instrucciones para generar una sucesión:

1. Comenzar con 5.
2. Dado cualquier término, sumarle 3 para obtener el siguiente término.

Si enumeramos los términos de la sucesión, obtenemos

$$5, \quad 8, \quad 11, \quad 14, \quad 17, \quad \ldots. \tag{5.1.1}$$

El primero término es igual a 5 debido a la primera instrucción. El segundo término es 8 debido a que la instrucción 2 dice que debemos sumar 3 a 5 para obtener el siguiente término, 8. El tercer término es 11 debido a que la instrucción 2 dice que debemos sumar 3 a 8 para obtener el siguiente término, 11. Si seguimos las instrucciones 1 y 2, podemos calcular cualquier término de la sucesión. Las instrucciones 1 y 2 no proporcionan una fórmula explícita para el n-ésimo término de la sucesión, en el sentido de proporcionar una fórmula en que podamos "sustituir n" para obtener el valor del n-ésimo término, sino que al ir calculando término a término podemos obtener cualquier término de la sucesión.

Si denotamos la sucesión (5.1.1) como $a_1, a_2, \ldots$, podemos enunciar la instrucción 1 como

$$a_1 = 5 \tag{5.1.2}$$

y la instrucción 2 como

$$a_n = a_{n-1} + 3, \quad n \geq 2. \tag{5.1.3}$$

Si hacemos $n = 2$ en (5.1.3), obtenemos

$$a_2 = a_1 + 3.$$

Por (5.1.2), $a_1 = 5$; así,

$$a_2 = a_1 + 3 = 5 + 3 = 8.$$

Si hacemos $n = 3$ en (5.1.3), obtenemos

$$a_3 = a_2 + 3.$$

Como $a_2 = 8$,

$$a_3 = a_2 + 3 = 8 + 3 = 11.$$

(5.1.2) y (5.1.3) permiten calcular cualquier término de la sucesión, de la misma manera que lo hicimos utilizando las instrucciones 1 y 2. Vemos que (5.1.2) y (5.1.3) son equivalentes a las instrucciones 1 y 2.

La ecuación (5.1.3) proporciona un ejemplo de **relación de recurrencia**. Una relación de recurrencia define una sucesión dando el n-ésimo valor en términos de algunos de sus predecesores. En (5.1.3), el n-ésimo valor está dado en términos del valor inmediato anterior. Para que una relación de recurrencia como (5.1.3) defina una sucesión, hay que dar cierto valor o valores "de arranque", como (5.1.2). Estos valores de arranque son llamados **condiciones iniciales**. A continuación damos las definiciones formales.

DEFINICIÓN 5.1.1

Una *relación de recurrencia* para la sucesión $a_0, a_1, \ldots$ es una ecuación que relaciona a_n con algunos de sus predecesores $a_0, a_1, \ldots, a_{n-1}$.

Las *condiciones iniciales* para la sucesión $a_0, a_1, \ldots$ son valores dados en forma explícita para un número finito de términos de la sucesión.

Hemos visto que es posible definir una sucesión mediante una relación de recurrencia, junto con ciertas condiciones iniciales. Daremos varios ejemplos de relaciones de recurrencia.

EJEMPLO 5.1.2

La sucesión de Fibonacci (véase el análisis después del algoritmo 3.4.7) se define mediante la relación de recurrencia

$$f_n = f_{n-1} + f_{n-2}, \quad n \geq 3$$

y las condiciones iniciales

$$f_1 = 1, \quad f_2 = 2.$$

□

EJEMPLO 5.1.3

Una persona invierte \$1000 a 12% compuesto anualmente. Si A_n representa la cantidad al final de n años, determinar una relación de recurrencia y condiciones iniciales que definan la sucesión $\{A_n\}$.

Al final de $n-1$ años, la cantidad es A_{n-1}. Después de un año más, tendremos la cantidad A_{n-1} más los intereses. Así,

$$A_n = A_{n-1} + (0.12)A_{n-1} = (1.12)A_{n-1}, \quad n \geq 1. \tag{5.1.4}$$

Para aplicar esta relación de recurrencia a $n = 1$, necesitamos saber el valor de A_0. Como A_0 es la cantidad del principio, tenemos la condición inicial

$$A_0 = 1000. \tag{5.1.5}$$

□

La condición inicial (5.1.5) y la relación de recurrencia (5.1.4) nos permiten calcular el valor de A_n para cualquier n. Por ejemplo,

$$\begin{aligned} A_3 &= (1.12)A_2 = (1.12)(1.12)A_1 \\ &= (1.12)(1.12)(1.12)A_0 = (1.12)^3(1000) = 1404.93. \end{aligned} \tag{5.1.6}$$

Así, al final del tercer año, la cantidad es \$1404.93.

El cálculo (5.1.6) se puede realizar para un valor arbitrario de n para obtener

$$\begin{aligned} A_n &= (1.12)A_{n-1} \\ &\vdots \\ &= (1.12)^n(1000). \end{aligned}$$

Vemos que en ciertas ocasiones podemos deducir una fórmula explícita a partir de una relación de recurrencia y las condiciones iniciales. La determinación de fórmulas explícitas a partir de las relaciones de recurrencia es el tema de la sección 5.2.

Aunque es fácil obtener una fórmula explícita a partir de la relación de recurrencia y la condición inicial para la sucesión del ejemplo 5.1.3, no es tan inmediata la forma de obtener una fórmula explícita para la sucesión de Fibonacci. En la sección 5.2 daremos un método que proporcionará una fórmula explícita para la sucesión de Fibonacci.

Las relaciones de recurrencia, los algoritmos recursivos y la inducción matemática tienen una relación muy estrecha. En las tres, se suponen conocidos casos anteriores del caso en cuestión. Una relación de recurrencia utiliza valores anteriores en una sucesión para calcular el valor actual. Un algoritmo recursivo utiliza instancias menores de la entrada actual para calcular ésta. El paso inductivo en una demostración por inducción matemática supone la verdad de instancias anteriores del enunciado, para demostrar la verdad del enunciado en cuestión.

Una relación de recurrencia que define una sucesión se puede convertir de manera directa en un algoritmo para el cálculo de la sucesión. Por ejemplo, el algoritmo 5.1.4, deducido de la relación de recurrencia (5.1.4) y la condición inicial (5.1.5), calcula la sucesión del ejemplo 5.1.3.

ALGORITMO 5.1.4 *Cálculo del interés compuesto*

Este algoritmo recursivo calcula la cantidad de dinero al final de n años, suponiendo un capital inicial de \$1000 y una tasa de interés de 12% compuesto anualmente.

Entrada: n, el número de años

Salida: La cantidad de dinero al cabo de n años

```
1. procedure compound_interest(n)
2.   if n = 0 then
3.     return(1000)
4.   return(1.12 * compound_interest(n − 1))
5. end compound_interest
```

El algoritmo 5.1.4 es una traducción directa de las ecuaciones (5.1.4) y (5.1.5) que definen a la sucesión $A_0, A_1, \ldots$. Las líneas 2 y 3 corresponden a la condición inicial (5.1.5) y la línea 4 corresponde a la relación de recurrencia (5.1.4).

EJEMPLO 5.1.5

Sea S_n el número de subconjuntos de un conjunto con n elementos. Como el paso de un conjunto con $(n-1)$ elementos a un conjunto con n elementos duplica el número de subconjuntos (véase el teorema 2.1.4), obtenemos la relación de recurrencia

$$S_n = 2S_{n-1}.$$

La condición inicial es

$$S_0 = 1.$$

□

Una de las principales razones para el uso de las relaciones de recurrencia es que a veces es más fácil determinar el n-ésimo término de una sucesión a partir de sus predecesores que determinar una fórmula explícita para el n-ésimo término en términos de n. Los siguientes ejemplos pretenden ilustrar esta tesis.

EJEMPLO 5.1.6

Sea S_n el número de cadenas de n bits que no contienen el patrón 111. Desarrollar una relación de recurrencia para $S_1, S_2, \ldots$, y las condiciones iniciales que definen la sucesión S.

Contaremos el número de cadenas de n bits que no contienen el patrón 111

(a) que comienzan con 0;

(b) que comienzan con 10;

(c) que comienzan con 11.

Como los conjuntos de cadenas de los tipos (a), (b) y (c) son ajenos, por el principio de la suma, S_n será igual a la suma de las cantidades de cadenas de los tipos (a), (b) y (c). Supongamos que una cadena de n bits comienza con 0 y que no contiene al patrón 111. Entonces, la cadena de $(n-1)$ bits que va después del 0 inicial no contiene al patrón 111. Como después del 0 inicial puede aparecer cualquier cadena de $(n-1)$ bits que no contenga 111, existen S_{n-1} cadenas de tipo (a). Si una cadena de n bits comienza con 10 y no contiene al patrón 111, entonces la cadena de $(n-2)$ bits posterior al 10 inicial no puede contener al patrón 111; por tanto, existen S_{n-2} cadenas de tipo (b). Si una cadena de n bits comienza con 11 y no contiene al patrón 111, entonces el tercer bit debe ser 0. La cadena de $(n-3)$ bits posterior al 110 inicial no puede contener el patrón 111; por tanto, existen S_{n-3} cadenas de tipo (c). Así,

$$S_n = S_{n-1} + S_{n-2} + S_{n-3}, \quad n \geq 4.$$

Encontramos las condiciones iniciales por inspección:

$$S_1 = 2, \quad S_2 = 4, \quad S_3 = 7. \qquad \square$$

EJEMPLO 5.1.7

El lector debe recordar (véase el ejemplo 4.2.23) que el número de Catalan C_n es igual al número de rutas que van de la esquina inferior izquierda de una retícula de $n \times n$ a la esquina superior derecha si sólo podemos recorrerla hacia la derecha o hacia arriba y si sólo se permite tocar pero no rebasar una recta diagonal, de la esquina inferior izquierda a la esquina superior derecha. Una ruta de este tipo será una *ruta buena*. Daremos una relación de recurrencia para los números de Catalan.

Separaremos las rutas buenas en clases, con base en el punto donde la ruta toca la diagonal después de salir de la esquina inferior izquierda. Por ejemplo, la ruta de la figura 5.1.1 corta por vez primera la diagonal en el punto (3, 3). Consideraremos que las rutas que tocan por vez primera la diagonal en (k, k) se construyen mediante un proceso de dos pasos: En primer lugar, se construye la parte de (0, 0) a (k, k). En segundo lugar se construye

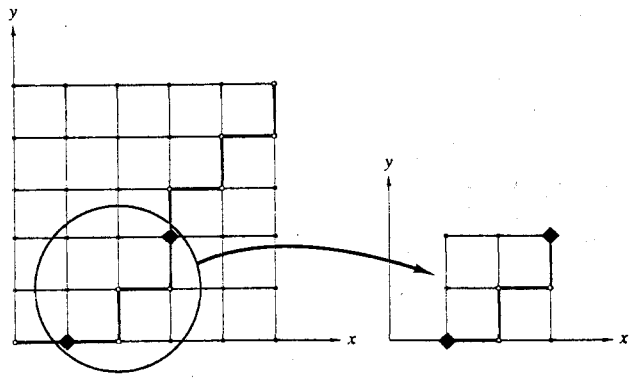


FIGURA 5.1.1 Descomposición de una ruta buena.

la parte de (k, k) a (n, n). Una ruta buena siempre sale de (0, 0) yendo hacia la derecha, a (1, 0) y siempre llega a (k, k) moviéndose hacia arriba desde $(k, k-1)$. Los movimientos de (1, 0) a $(k, k-1)$ proporcionan una ruta buena en la retícula $(k-1) \times (k-1)$ con esquinas en (1, 0), $(1, k-1)$, $(k, k-1)$ y (k, 0). [En la figura 5.1.1, hemos marcado los puntos (1, 0) y $(k, k-1)$, $k = 3$, con diamantes, y hemos separado la subretícula $(k-1) \times (k-1)$.] Así, existen C_{k-1} rutas de (0, 0) a (k, k) que cortan por vez primera la diagonal en (k, k). La parte de (k, k) a (n, n) es una ruta buena en la retícula $(n-k) \times (n-k)$ con esquinas en (k, k), (k, n), (n, n) y (n, k) (véase la figura 5.1.1). Existen C_{n-k} de estas rutas. Por el principio de multiplicación, existen $C_{k-1}C_{n-k}$ rutas buenas en una retícula $n \times n$ que cortan por vez primera la diagonal en (k, k). Las rutas buenas que cortan por vez primera la diagonal en (k, k) son distintas de aquellas que cortan por vez primera la diagonal en (k', k'), si $k \neq k'$. Así, podemos utilizar el principio de la suma para obtener una relación de recurrencia para la cantidad total de rutas buenas en una retícula de $n \times n$:

$$C_n = \sum_{k=1}^{n} C_{k-1}C_{n-k}. \qquad \square$$

BIBLIOTECA FACULTAD Cs. EXACTAS, INGENIERÍA Y AGRIMENSURA ROSARIO

EJEMPLO 5.1.8 *Torre de Hanoi*

La Torre de Hanoi es un juego que consta de tres postes montados sobre un tablero y n discos de diversos tamaños con agujeros en sus centros (véase la figura 5.1.2). Se supone que si un disco está en algún poste, sólo se puede colocar sobre tal disco otro con diámetro menor. Dados todos los discos apilados en un poste, como en la figura 5.1.2, el problema consiste en transferir los discos a otro poste, moviendo un disco a la vez.

Daremos una solución y luego determinaremos una relación de recurrencia y una condición inicial para la sucesión $c_1, c_2, \ldots$, donde c_n denota el número de movimientos necesarios en nuestra solución del problema con n discos. Luego mostraremos que nuestra solución es óptima; es decir, mostraremos que ninguna otra solución utiliza menos movimientos.

Daremos un algoritmo recursivo. Si sólo existe un disco, basta moverlo al poste deseado. Si tenemos $n > 1$ discos en el poste 1, como en la figura 5.1.2, primero llamamos de manera recursiva a nuestro algoritmo, para mover los $n-1$ discos superiores al poste 2 (véase la figura 5.1.3). Durante estos movimientos, el disco inferior en el poste 1 permanece fijo. A continuación movemos el disco restante del poste 1 al poste 3. Por último, de nuevo llamamos de manera recursiva a nuestro algoritmo para mover los $n-1$ discos del poste 2 al poste 3. Con esto hemos podido mover n discos del poste 1 al poste 3.

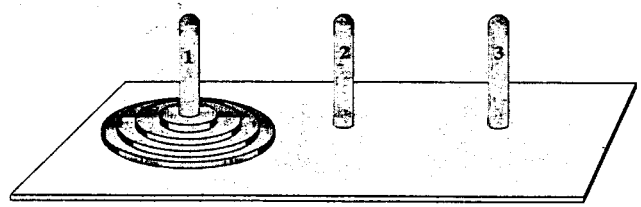


FIGURA 5.1.2 Torre de Hanoi.

Si $n > 1$, resolvemos dos veces el problema con $(n-1)$ discos y movemos de manera explícita un disco. Por tanto,

$$c_n = 2c_{n-1} + 1, \quad n > 1.$$

La condición inicial es

$$c_1 = 1.$$

En la sección 5.2 mostraremos que $c_n = 2^n - 1$.

A continuación mostraremos que nuestra solución es óptima. Sea d_n el número de movimientos necesarios por una solución óptima. Utilizaremos la inducción matemática para mostrar que

$$c_n = d_n, \quad n \geq 1. \tag{5.1.7}$$

PASO BASE ($n = 1$). Por inspección,

$$c_1 = 1 = d_1;$$

de modo que (5.1.7) es verdadera para $n = 1$.

PASO INDUCTIVO. Supongamos que (5.1.7) es verdadera para $n - 1$. Consideremos el momento de una solución óptima para el problema con n discos en que el disco de mayor tamaño se mueve por vez primera. Este disco debe estar sólo en un poste (para que pueda moverse) y otro poste debe estar vacío (de modo que este poste pueda recibir el disco de mayor tamaño). Así, los $n-1$ discos menores deben estar apilados en un tercer poste (véase la figura 5.1.3). En otras palabras, debe haberse resuelto el problema con $n-1$ discos, lo cual requería al menos d_{n-1} movimientos. Luego se mueve el disco mayor, para lo cual se requiere un movimiento adicional. Por último, en algún momento, los $n-1$ discos se colocan sobre el disco mayor, lo que requiere al menos d_{n-1} movimientos adicionales. Esto implica que

$$d_n \geq 2d_{n-1} + 1.$$

Por la hipótesis de inducción, $c_{n-1} = d_{n-1}$. Así,

$$d_n \geq 2d_{n-1} + 1 = 2c_{n-1} + 1 = c_n. \tag{5.1.8}$$

La última igualdad es consecuencia de la relación de recurrencia para la sucesión $c_1, c_2, \ldots$. Por definición, ninguna solución puede realizarse en menos movimientos que la solución óptima, de modo que

$$c_n \geq d_n. \tag{5.1.9}$$

Combinamos las desigualdades (5.1.8) y (5.1.9) para obtener

$$c_n = d_n.$$

Con esto concluye el paso inductivo. Por tanto, nuestra solución es óptima. □

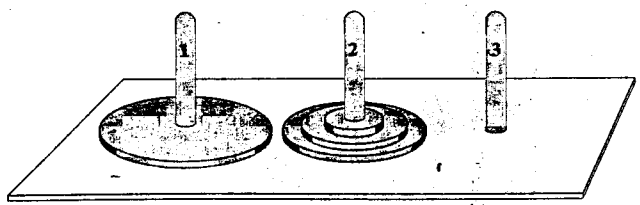


FIGURA 5.1.3 Estado de la Torre de Hanoi después de mover de manera recursiva los $n-1$ discos superiores del poste 1 al poste 2.

El juego de la Torre de Hanoi fue ideado por el matemático francés Édouard Lucas a finales del siglo XIX. (Lucas fue la primera persona que llamó a la sucesión 1, 2, 3, 5, ... sucesión de Fibonacci.) Se creó la siguiente leyenda para acompañar al juego (y, suponemos, para apoyar su comercialización). Se decía que el juego se había deducido de una mítica torre de oro con 64 discos. Los 64 discos debían ser trasladados por monjes, de acuerdo con las reglas ya establecidas. Se decía que antes de que los monjes terminasen de mover la torre, ésta caería y el mundo llegaría a su fin. Como al menos se necesitan $2^{64} - 1 = 18{,}446{,}744{,}073{,}709{,}551{,}615$ movimientos para resolver el juego de la Torre de Hanoi con 64 discos, podemos estar seguros de que algo ocurriría a la torre antes de moverla por completo.

EJEMPLO 5.1.9 ***La telaraña en la economía***

Supongamos un modelo económico en el cual la oferta y la demanda están dadas mediante ecuaciones lineales (véase la figura 5.1.4). En forma específica, la demanda está dada por la ecuación

$$p = a - bq,$$

donde p es el precio, q es la cantidad y a y b son parámetros positivos. La idea es que si el precio aumenta, los consumidores demandan menos producto. La oferta está dada por la ecuación

$$p = kp,$$

donde p es el precio, q es la cantidad y k es un parámetro positivo. La idea es que si el precio se incrementa, el fabricante está dispuesto a ofrecer mayores cantidades del producto.

También suponemos que existe un cierto retraso en la reacción de la oferta a los cambios. (Por ejemplo, se necesita cierto tiempo para fabricar bienes o para que crezcan las cosechas.) Denotamos los intervalos de tiempo discretos como $n = 0, 1, \ldots$. Suponemos que la demanda está dada por la ecuación

$$p_n = a - bq_n;$$

es decir, en el instante n, la cantidad q_n del producto se venderá al precio p_n. Suponemos que la oferta está dada por la ecuación

$$p_n = kq_{n+1}; \tag{5.1.10}$$

es decir, se necesita una unidad de tiempo para que el fabricante ajuste la cantidad q_{n+1}, en el instante $n+1$, al precio p_n, en el instante anterior n.

Si despejamos q_{n+1} en (5.1.10) y sustituimos el valor obtenido en la ecuación de la demanda para el instante $n+1$,

$$p_{n+1} = a - bq_{n+1},$$

obtenemos la relación de recurrencia

$$p_{n+1} = a - \frac{b}{k} p_n$$

para el precio. En la sección 5.2 resolveremos esta relación de recurrencia.

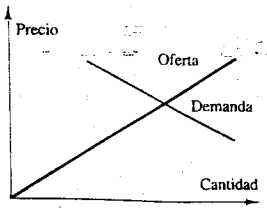


FIGURA 5.1.4
Un modelo económico.

```
1.  procedure insertion_sort(s, n)
2.    if n = 1 then
3.      return
4.    insertion_sort(s, n − 1)
5.    i := n − 1
6.    temp := s_n
7.    while i ≥ 1 and s_i > temp do
8.      begin
9.      s_{i+1} := s_i
10.     i := i − 1
11.     end
12.   s_{i+1} := temp
13. end insertion_sort
```

Sea b_n el número de veces que se realiza, en el peor de los casos, la comparación $s_i > temp$ en la línea 7. Suponga que si $i < 1$, la comparación $s_i > temp$ no se realiza.

39. Explique por qué el algoritmo 5.3.14 ordena la sucesión.
40. ¿Cuál entrada produce el comportamiento en el peor de los casos para el algoritmo 5.3.14?
41. Determine b_1, b_2 y b_3.
42. Determine una relación de recurrencia para la sucesión $\{b_n\}$.
43. Resuelva la relación de recurrencia del ejercicio 42.

Los ejercicios 44-46 se refieren al algoritmo 5.3.15.

ALGORITMO 5.3.15

Entrada: $s_1, \ldots, s_n, n$
Salida: $s_1, \ldots, s_n$

```
procedure algor(s, n)
  i := n
  while i ≥ 1 do
    begin
    s_i := s_i + 1
    i := ⌊i/2⌋
    end
  n := ⌊n/2⌋
  if n ≥ 1 then
    algor(s, n)
end algor
```

Sea b_n el número de veces que se ejecuta el enunciado $s_i := s_i + 1$.

44. Determine una relación de recurrencia para la sucesión $\{b_n\}$ y calcule b_1, b_2 y b_3.
45. Resuelva la relación de recurrencia del ejercicio 44 cuando n sea una potencia de 2.
46. Demuestre que $b_n = \Theta((\lg n)^2)$.
47. Resuelva la relación de recurrencia

$$a_n = 3a_{\lfloor n/2 \rfloor} + n, \quad n > 1,$$

cuando n sea una potencia de 2. Suponga que $a_1 = 1$.

48. Muestre que $a_n = \Theta(n^{\lg 3})$, donde a_n es como en el ejercicio 47.

Los ejercicios 49-56 se refieren a un algoritmo que acepta como entrada la sucesión

$$s_i, \ldots, s_j.$$

Si $j > i$, los subproblemas

$$s_i, \ldots, s_{\lfloor (i+j)/2 \rfloor} \quad \text{y} \quad s_{\lfloor (i+j)/2+1 \rfloor}, \ldots, s_j$$

se resuelven de manera recursiva. Las soluciones de los subproblemas de tamaños m y k se pueden combinar en un tiempo $c_{m,k}$ para resolver el problema original. Sea b_n el tiempo necesario para que el algoritmo procese una entrada de tamaño n.

49. Escriba una relación de recurrencia para b_n, suponiendo que $c_{m,k} = 3$.
50. Escriba una relación de recurrencia para b_n, suponiendo que $c_{m,k} = m + k$.
51. Resuelva la relación de recurrencia del ejercicio 49 para el caso en que n sea una potencia de 2, suponiendo que $b_1 = 0$.
52. Resuelva la relación de recurrencia del ejercicio 49 para el caso en que n sea una potencia de 2, suponiendo que $b_1 = 1$.
53. Resuelva la relación de recurrencia del ejercicio 50 para el caso en que n sea una potencia de 2, suponiendo que $b_1 = 0$.
54. Resuelva la relación de recurrencia del ejercicio 50 para el caso en que n sea una potencia de 2, suponiendo que $b_1 = 1$.
☆ **55.** Suponga que si $m_1 \geq m_2$ y $k_1 \geq k_2$, entonces $c_{m_1,k_1} \geq c_{m_2,k_2}$. Muestre que la sucesión $b_1, b_2, \ldots$ es creciente.
★ 56. Suponga que $c_{m,k} = m + k$ y $b_1 = 0$, y muestre que $b_n \leq 4n \lg n$.

Los ejercicios 57-62 se refieren a la siguiente situación. Sea P_n un problema particular de tamaño n. Si P_n se divide en subproblemas de tamaños i y j, existe un algoritmo que combina las soluciones de estos dos subproblemas en una solución de P_n en un tiempo a lo más de $2 + \lg(ij)$. Suponga que ya se ha resuelto un problema de tamaño 1.

57. Escriba un algoritmo recursivo para resolver P_n, similar al algoritmo 5.3.8.
58. Sea a_n el tiempo, en el peor de los casos, para resolver P_n mediante el algoritmo del ejercicio 57. Muestre que

$$a_n \leq a_{\lfloor n/2 \rfloor} + a_{\lfloor (n+1)/2 \rfloor} + 2 \lg n.$$

59. Sea b_n la relación de recurrencia obtenida a partir del ejercicio 58 reemplazando "≤" por "=". Suponga que $b_1 = a_1 = 0$. Muestre que si n es una potencia de 2,

$$b_n = 4n - 2 \lg n - 4.$$

60. Muestre que $a_n \leq b_n$ para $n = 1, 2, 3, \ldots$.
61. Muestre que $b_n \leq b_{n+1}$ para $n = 1, 2, 3, \ldots$.
62. Muestre que $a_n \leq 8n$ para $n = 1, 2, 3, \ldots$.
63. Suponga que $\{a_n\}$ es una sucesión creciente y que siempre que m divida a n,

$$a_n = a_{n/m} + d,$$

donde d es un número real positivo y m es un entero que satisface $m > 1$. Muestre que $a_n = \Theta(\lg n)$.
★ 64. Suponga que $\{a_n\}$ es una sucesión creciente y que siempre que m divida a n,

$$a_n = ca_{n/m} + d,$$

donde c y d son números reales positivos que satisfacen $c > 1$ y $d > 0$, y m es un entero que satisface $m > 1$. Muestre que $a_n = \Theta(n^{\log_m c})$.
65. [Proyecto] Investigue otros algoritmos de ordenamiento. Considere de manera específica la complejidad, los análisis empíricos y las características particulares de los algoritmos (véase [Knuth, 1973, vol. 3]).

NOTAS

Las relaciones de recurrencia se analizan con más detalle en [Liu, 1985; Roberts; y Tucker]. [Cormen] presenta varias aplicaciones al análisis de algoritmos.

[Cull] proporciona algoritmos para resolver ciertos problemas del tipo de la Torre de Hanoi con un mínimo de complejidad en el espacio y el tiempo. [Hinz] es un amplio análisis de la Torre de Hanoi con 50 referencias.

La telaraña de la economía apareció por vez primera en [Ezekiel].

Todos los libros relativos a las estructuras de datos y los algoritmos contienen amplios análisis de la búsqueda y el ordenamiento (véase, por ejemplo, [Brassard; Cormen; Knuth, 1973, vol. 3; Kruse; y Nyhoff]).

Las relaciones de recurrencia también se llaman ecuaciones en diferencias. [Goldberg] contiene un análisis de las ecuaciones en diferencias y sus aplicaciones.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 5.1

Relación de recurrencia

Condición inicial

Interés compuesto

Torre de Hanoi

Telaraña en la economía

Función de Ackermann

Sección 5.2

Solución de una relación de recurrencia por iteración

Relación de recurrencia, lineal y homogénea de orden n con coeficientes constantes y la forma de resolver una relación de recurrencia de segundo orden

Crecimiento de poblaciones

Sección 5.3

Cómo determinar una relación de recurrencia que describa el tiempo necesario para ejecutar un algoritmo recursivo

Ordenamiento por selección

Búsqueda binaria

Fusión de sucesiones

Ordenamiento por fusión

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 5.1

1. Responda las partes (a)-(c) para la sucesión definida mediante las reglas:
   1. El primer término es 3.
   2. El n-ésimo término es n más el término anterior.
      (a) Escriba los cuatro primeros términos de la sucesión.
      (b) Determine una condición inicial para la sucesión.
      (c) Determine una relación de recurrencia para la sucesión.
2. Suponga que una persona invierte \$4000 a 17% compuesto anualmente. Sea A_n la cantidad al final de n años. Determine una relación de recurrencia y una condición inicial para la sucesión A0, A1, ...
3. Sea P_n el número de particiones de un conjunto con n elementos. Muestre que la sucesión $P_0, P_1, \ldots$ satisface la relación de recurrencia

$$P_n = \sum_{k=0}^{n-1} C(n-1, k)\, P_k.$$

4. Suponga que tenemos un tablero rectangular $2 \times n$ dividido en $2n$ cuadrados. Sea a_n el número de formas de cubrir exactamente este tablero con dominóes 1×2. Muestre que la sucesión $\{a_n\}$ satisface la relación de recurrencia

$$a_n = a_{n-1} + a_{n-2}.$$

Muestre que $a_n = f_n$, donde $\{f_n\}$ es la sucesión de Fibonacci.

Sección 5.2

5. ¿Es la relación de recurrencia

$$a_n = a_{n-1} + a_{n-3}$$

una relación de recurrencia lineal homogénea con coeficientes constantes?

En los ejercicios 6 y 7, resuelva la relación de recurrencia sujeta a las condiciones iniciales.

6. $a_n = -4a_{n-1} - 4a_{n-2}; \quad a_0 = 2, \quad a_1 = 4$
7. $a_n = 3a_{n-1} + 10a_{n-2}; \quad a_0 = 4, \quad a_1 = 13$
8. Sea c_n el número de cadenas sobre $\{0, 1, 2\}$ de longitud n que tienen una cantidad par de unos. Escriba una relación de recurrencia y una condición inicial que defina la sucesión $c_1, c_2, \ldots$. Resuelva la relación de recurrencia para obtener una fórmula explícita para c_n.

Sección 5.3

Los ejercicios 9-12 se refieren al siguiente algoritmo.

ALGORITMO ***Evaluación de un polinomio***

Este algoritmo evalúa el polinomio

$$p(x) = \sum_{k=0}^{n} c_k x^{n-k}$$

en el punto t.

Entrada: La sucesión de coeficientes $c_0, c_1, \ldots, c_n$, el valor t y n
Salida: $p(t)$

procedure *poly*(c, n, t)
 if $n = 0$ **then**
  return(c_0)
 return$(t \cdot poly(c, n-1, t) + c_n)$
end *poly*

Sea b_n el número de multiplicaciones necesarias para calcular $p(t)$.

9. Determine una relación de recurrencia y una condición inicial para la sucesión $\{b_n\}$.
10. Calcule b_1, b_2 y b_3.
11. Resuelva la relación de recurrencia del ejercicio 9.
12. Suponga que calculamos $p(t)$ mediante una técnica directa que requiere $n - k$ multiplicaciones para obtener $c_k t^{n-k}$. ¿Cuántas multiplicaciones se necesitarían para calcular $p(t)$? ¿Preferiría usted este método o el algoritmo anterior? Explique.

```
1.  procedure merge_sort (s, i, j)
2.    // caso base: i = j
3.    if i = j then
4.      return
5.    // dividir la sucesión y ordenar
6.    m := ⌊(i + j)/2⌋
7.    call merge_sort (s, i, m)
8.    call merge_sort (s, m + 1, j)
9.    // fusión
10.   call merge (s, i, m, j, c)
11.   // copiar c, la salida de la fusión, en s
12.   for k := i to j do
13.     s_k := c_k
14. end merge_sort
```

EJEMPLO 5.3.9

La figura 5.3.2 muestra la forma en que el algoritmo 5.3.8 ordena la sucesión

12, 30, 21, 8, 6, 9, 1, 7. □

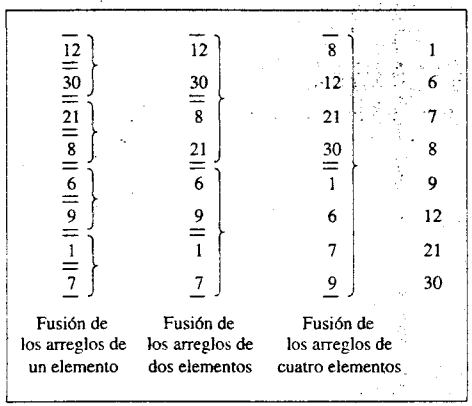


FIGURA 5.3.2 Ordenamiento por fusión.

Concluimos esta sección mostrando que el ordenamiento por fusión (algoritmo 5.3.8) es $\Theta(n \lg n)$ en el peor de los casos. El método de demostración es igual al utilizado para mostrar que la búsqueda binaria es $\Theta(\lg n)$ en el peor de los casos.

TEOREMA 5.3.10

El ordenamiento por fusión (algoritmo 5.3.8) es $\Theta(n \lg n)$ en el peor de los casos.

Demostración. Sea a_n el número de comparaciones necesarias para que el algoritmo 5.3.8 ordene n elementos en el peor de los casos. Entonces $a_1 = 0$. Si $n > 1$, a_n es a lo más la suma de los números de comparaciones en el peor de los casos resultante de las llamadas recursivas en las líneas 7 y 8, y el número de comparaciones en el peor de los casos necesarias para la fusión en la línea 10. Es decir,

$$a_n \le a_{\lfloor n/2 \rfloor} + a_{\lfloor (n+1)/2 \rfloor} + n - 1.$$

De hecho, esta cota superior se puede alcanzar (véase el ejercicio 11), de modo que

$$a_n = a_{\lfloor n/2 \rfloor} + a_{\lfloor (n+1)/2 \rfloor} + n - 1.$$

Primero resolvemos la anterior relación de recurrencia cuando n es una potencia de 2, digamos, $n = 2^k$. La ecuación se convierte en

$$a_{2^k} = 2a_{2^{k-1}} + 2^k - 1.$$

Podemos resolver esta última ecuación mediante el método de iteración (véase la sección 5.2):

$$\begin{aligned}
a_{2^k} &= 2a_{2^{k-1}} + 2^k - 1 \\
&= 2[2a_{2^{k-2}} + 2^{k-1} - 1] + 2^k - 1 \\
&= 2^2 a_{2^{k-2}} + 2 \cdot 2^k - 1 - 2 \\
&= 2^2[2a_{2^{k-3}} + 2^{k-2} - 1] + 2 \cdot 2^k - 1 - 2 \\
&= 2^3 a_{2^{k-3}} + 3 \cdot 2^k - 1 - 2 - 2^2 \\
&\vdots \\
&= 2^k a_{2^0} + k \cdot 2^k - 1 - 2 - 2^2 - \cdots - 2^{k-1} \\
&= k \cdot 2^k - (2^k - 1) \\
&= (k-1)2^k + 1.
\end{aligned} \tag{5.3.8}$$

Un valor arbitrario de n está entre dos potencias de 2, digamos,

$$2^{k-1} < n \le 2^k. \tag{5.3.9}$$

Como la sucesión a es creciente (véase el ejercicio 14),

$$a_{2^{k-1}} \le a_n \le a_{2^k}. \tag{5.3.10}$$

Observe que (5.3.9) implica

$$k - 1 < \lg n \le k. \tag{5.3.11}$$

(5.3.8), (5.3.10) y (5.3.11) implican que

$$\begin{aligned}
\Omega(n \lg n) = (-2 + \lg n)\frac{n}{2} &< (k-2)2^{k-1} + 1 = a_{2^{k-1}} \\
&\le a_n \le a_{2^k} \le k2^k + 1 \le (1 + \lg n)2n + 1 = O(n \lg n).
\end{aligned}$$

Por tanto, $a_n = \Theta(n \lg n)$, de modo que el ordenamiento por fusión es $\Theta(n \lg n)$ en el peor de los casos. ■

Como ya hemos observado, en la sección 7.7 mostraremos que cualquier algoritmo de ordenamiento basado en comparaciones es $\Omega(n \lg n)$ en el peor de los casos. Este resultado implica, en particular, que el ordenamiento por fusión es $\Omega(n \lg n)$ en el peor de los casos. Si ya hubiéramos demostrado este resultado, para probar que el ordenamiento por fusión es $\Theta(n \lg n)$ en el peor de los casos, sería suficiente demostrar que el ordenamiento por fusión es $O(n \lg n)$ en el peor de los casos.

Aunque el ordenamiento por fusión (algoritmo 5.3.8) es óptimo, podría no ser el algoritmo a elegir para un problema de ordenamiento particular. Habría que tomar en cuenta algunos factores, como el tiempo en el caso promedio, el número de elementos por ordenar, la memoria disponible, las estructuras de datos por utilizar, el hecho de que los elementos por ordenar estén en la memoria o en dispositivos de almacenamiento periférico como discos o cintas, el hecho de que los elementos por ordenar estén "casi" ordenados, o el hardware por utilizar.

Ejercicios

Los ejercicios 1-4 se refieren a la sucesión

$$s_1 = \text{'C'}, \quad s_2 = \text{'G'}, \quad s_3 = \text{'J'}, \quad s_4 = \text{'M'}, \quad s_5 = \text{'X'}.$$

1. Muestre la forma en que el algoritmo 5.3.2 se ejecuta cuando $key = \text{'G'}$.
2. Muestre la forma en que el algoritmo 5.3.2 se ejecuta cuando $key = \text{'P'}$.
3. Muestre la forma en que el algoritmo 5.3.2 se ejecuta cuando $key = \text{'C'}$.
4. Muestre la forma en que el algoritmo 5.3.2 se ejecuta cuando $key = \text{'Z'}$.
5. Sea a_n el tiempo de la búsqueda binaria (algoritmo 5.3.2) en el peor de los casos. Demuestre que $a_n \leq a_{n+1}$ para $n \geq 1$.
6. Demuestre que si a_n es el número de veces que se llama, en el peor de los casos, al algoritmo de búsqueda binaria (algoritmo 5.3.2) para una sucesión con n elementos, entonces
$$a_n = 2 + \lfloor \lg n \rfloor$$
para cada entero positivo n.
7. Supongamos que el algoritmo A necesita $\lceil n \lg n \rceil$ comparaciones para ordenar n elementos y el algoritmo B necesita $\lceil n^2/4 \rceil$ comparaciones para ordenar n elementos. ¿Para cuáles n el algoritmo B es mejor que el algoritmo A?
8. Muestre la forma en que el ordenamiento por fusión (algoritmo 5.3.8) ordena la sucesión 1, 9, 7, 3.
9. Muestre la forma en que el ordenamiento por fusión (algoritmo 5.3.8) ordena la sucesión 2, 3, 7, 2, 8, 9, 7, 5, 4.
10. Suponga que tenemos dos sucesiones, cada una de tamaño n, ordenadas en forma creciente.
    (a) ¿Bajo qué condiciones se alcanza el número máximo de comparaciones en el algoritmo 5.3.5?
    (b) ¿Bajo qué condiciones se alcanza el número mínimo de comparaciones en el algoritmo 5.3.5?
11. Sea a_n como en la demostración del teorema 5.3.10. Describa la entrada para la cual
$$a_n = a_{\lfloor n/2 \rfloor} + a_{\lfloor (n-1)/2 \rfloor} + n - 1.$$
12. ¿Cuál es el número mínimo de comparaciones necesarias para que el algoritmo 5.3.8 ordene un arreglo de tamaño 6?
13. ¿Cuál es el número máximo de comparaciones necesarias para que el algoritmo 5.3.8 ordene un arreglo de tamaño 6?

14. Sea a_n como en la demostración del teorema 5.3.10. Demuestre que $a_n \leq a_{n+1}$ para toda $n \geq 1$.
15. Sea a_n el número de comparaciones necesarias para el ordenamiento por fusión en el peor de los casos. Muestre que $a_n \leq 3n \lg n$ para $n = 1, 2, 3, \ldots$.
16. Muestre que en el mejor de los casos, el ordenamiento por fusión necesita $\Theta(n \lg n)$ comparaciones.

Los ejercicios 17-21 se refieren al algoritmo 5.3.11.

ALGORITMO 5.3.11 ***Cálculo de una exponencial***

Este algoritmo calcula a^n de manera recursiva, donde a es un número real y n es un entero positivo.

Entrada: a (un número real), n (un entero positivo)
Salida: a^n

```
1. procedure expl(a, n)
2.   if n = 1 then
3.     return(a)
4.   m := ⌊n/2⌋
5.   return(expl(a, m) · expl(a, n − m))
6. end expl
```

Sea b_n el número de multiplicaciones (línea 5) necesarias para calcular a^n.

17. Explique por qué el algoritmo 5.3.11 calcula a^n.
18. Determine una relación de recurrencia y condiciones iniciales para la sucesión $\{b_n\}$.
19. Calcule b_2, b_3 y b_4.
20. Resuelva la relación de recurrencia del ejercicio 18, cuando n es una potencia de 2.
21. Demuestre que $b_n = n - 1$ para cada entero positivo n.

Los ejercicios 22-27 se refieren al algoritmo 5.3.12.

ALGORITMO 5.3.12 ***Cálculo de una exponencial***

Este algoritmo calcula a^n de manera recursiva, donde a es un número real y n es un entero positivo.

Entrada: a (un número real), n (un entero positivo)
Salida: a^n

```
1. procedure exp2(a, n)
2.   if n = 1 then
3.     return(a)
4.   m := ⌊n/2⌋
5.   power := exp2(a, m)
6.   power := power · power
7.   if n is even then
8.     return(power)
9.   else
10.    return(power · a)
11. end exp2
```

Sea b_n el número de multiplicaciones (líneas 6 y 10) necesarias para calcular a^n.

22. Explique por qué el algoritmo 5.3.12 calcula a^n.
23. Muestre que

$$b_n = \begin{cases} b_{(n-1)/2} + 2, & \text{si } n \text{ es impar;} \\ b_{n/2} + 1, & \text{si } n \text{ es par.} \end{cases}$$

24. Determine b_1, b_2, b_3 y b_4.
25. Resuelva la relación de recurrencia del ejercicio 23 cuando n es una potencia de 2.
26. Muestre, mediante un ejemplo, que b no es creciente.
☆ 27. Demuestre que $b_n = \Theta(\lg n)$.

Los ejercicios 28-33 se refieren al algoritmo 5.3.13.

ALGORITMO 5.3.13 ***Determinación de los elementos máximo y mínimo de una sucesión***

Este algoritmo recursivo determina los elementos máximo y mínimo de una sucesión.

Entrada: *si*, . . . , *sj*, *i* y *j*

Salida: *large* (el elemento máximo de una sucesión), *small* (el elemento mínimo de una sucesión)

```
1.  procedure large_small(s, i, j, large, small)
2.    if i = j then
3.      begin
4.      large := s_i
5.      small := s_i
6.      return
7.      end
8.    m := ⌊(i + j)/2⌋
9.    large_small(s, i, m, large_left, small_left)
10.   large_small(s, m + 1, j, large_right, small_right)
11.   if large_left > large_right then
12.     large := large_left
13.   else
14.     large := large_right
15.   if small_left > small_right then
16.     small := small_right
17.   else
18.     small := small_left
19. end large_small
```

Sea b_n el número de comparaciones (líneas 11 y 15) necesarias para una entrada de tamaño n.

28. Explique por qué el algoritmo 5.3.13 determina los elementos máximo y mínimo.
29. Muestre que $b_1 = 0$ y $b_2 = 2$.
30. Determine b_3.
31. Establezca la relación de recurrencia

$$b_n = b_{\lfloor n/2 \rfloor} + b_{\lfloor (n+1)/2 \rfloor} + 2 \qquad (5.3.12)$$

para $n > 1$.

32. Resuelva la relación de recurrencia (5.3.12) cuando n es una potencia de 2, para obtener

$$b_n = 2n - 2, \quad n = 1, 2, 4, \ldots$$

33. Utilice inducción para mostrar que

$$b_n = 2n - 2$$

para cada entero positivo n.

Los ejercicios 34-37 se refieren al algoritmo 5.3.13, con las líneas siguientes introducidas después de la línea 7.

```
7a.  if j = i + 1 then
7b.    begin
7c.    if s_i > s_j then
7d.      begin
7e.      large := s_i
7f.      small := s_j
7g.      end
7h.    else
7i.      begin
7j.      small := s_i
7k.      large := s_j
7l.      end
7m.    return
7n.  end
```

Sea b_n el número de comparaciones (líneas 7c, 11 y 15) para una entrada de tamaño n.

34. Muestre que $b_1 = 0$ y $b_2 = 1$.
35. Calcule b_3 y b_4.
36. Muestre que la relación de recurrencia (5.3.12) es válida para $n > 2$.
37. Resuelva la relación de recurrencia (5.3.12) cuando n es una potencia de 2 para obtener

$$b_n = \frac{3n}{2} - 2, \quad n = 2, 4, 8, \ldots.$$

★ **38.** Modifique el algoritmo 5.3.13 insertando las líneas anteriores al ejercicio 34 después de la línea 7 y reemplazando la línea 8 con lo siguiente.

```
8a.  if j − i is odd and (1 + j − i)/2 is odd then
8b.    m := ⌊(i + j)/2⌋ − 1
8c.  else
8d.    m := ⌊(i + j)/2⌋
```

Muestre que en el peor de los casos, este algoritmo modificado necesita a lo más $\lceil (3n/2) - 2 \rceil$ comparaciones para determinar los elementos máximo y mínimo en un arreglo de tamaño n.

Los ejercicios 39-41 se refieren al algoritmo 5.3.14.

ALGORITMO 5.3.14 ***Ordenamiento por inserción***

Este algoritmo ordena la sucesión

$$s_1, s_2, \ldots, s_n$$

en orden creciente, ordenando de manera recursiva los primeros $n - 1$ elementos y luego insertando s_n en la posición correcta.

Entrada: $s_1, s_2, \ldots, s_n$ y la longitud n de la sucesión
Salida: $s_1, s_2, \ldots, s_n$ ordenada de manera creciente

en orden creciente, seleccionando primero al elemento máximo para colocarlo al final, y luego ordenar de manera recursiva los demás elementos.

Entrada: $s_1, s_2, \ldots, s_n$ y la longitud n de la sucesión

Salida: $s_1, s_2, \ldots, s_n$, en orden creciente

```
1.  procedure selection_sort(s, n)
2.    // caso base
3.    if n = 1 then
4.      return
5.    // se encuentra el máximo
6.    max_index := 1    // se supone al inicio que s_1 es el máximo
7.    for i := 2 to n do
8.      if s_i > s_max_index then    // se encontró uno mayor, así que debe actualizarse
9.        max_index := i
10.   // se mueve el máximo al final
11.   swap(s_n, s_max_index)
12.   call selection_sort(s, n − 1)
13. end selection_sort
```

Para medir el tiempo necesario para este algoritmo, contaremos el número de comparaciones b_n en la línea 8 necesarias para ordenar n elementos. (Observe que los tiempos en el mejor de los casos, en el caso promedio y en el peor de los casos son todos iguales para este algoritmo.) De inmediato obtenemos la condición inicial

$$b_1 = 0.$$

Para obtener una relación de recurrencia para la sucesión $b_1, b_2, \ldots$, simulamos la ejecución del algoritmo para una entrada arbitraria de tamaño $n > 1$. Contamos el número de comparaciones en cada línea y luego sumamos estos números para obtener la cantidad total de comparaciones b_n. En las líneas 1-7, no hay comparaciones (del tipo que estamos contando). En la línea 8, existen $n - 1$ comparaciones (pues la línea 7 hace que la línea 8 se ejecute $n - 1$ veces). No hay comparaciones en las líneas 9-11. La llamada recursiva aparece en la línea 12, donde llamamos a este algoritmo con una entrada de tamaño $n - 1$. Pero por definición, este algoritmo requiere b_{n-1} comparaciones para una entrada de tamaño $n - 1$. Así, existen b_{n-1} comparaciones en la línea 12. Por tanto, la cantidad total de comparaciones es

$$b_n = n - 1 + b_{n-1},$$

lo cual da la relación de recurrencia deseada.

Nuestra relación de recurrencia se puede resolver por iteración:

$$\begin{aligned} b_n &= b_{n-1} + n - 1 \\ &= (b_{n-2} + n - 2) + (n - 1) \\ &= (b_{n-3} + n - 3) + (n - 2) + (n - 1) \\ &\vdots \\ &= b_1 + 1 + 2 + \cdots + (n - 2) + (n - 1) \\ &= 0 + 1 + 2 + \cdots + (n - 1) = \frac{(n-1)n}{2} = \Theta(n^2). \end{aligned}$$

Así, el tiempo necesario para el algoritmo 5.3.1 es $\Theta(n^2)$.

Nuestro siguiente algoritmo (algoritmo 5.3.2) es la **búsqueda binaria**. La búsqueda binaria busca un valor en una sucesión *ordenada* y regresa el índice del valor si lo encuentra, o 0, en caso contrario. El algoritmo utiliza el enfoque divide y vencerás. La sucesión se divide en dos partes casi iguales (línea 4). Si el elemento se encuentra en el punto de división (línea 5), el algoritmo termina. Si el elemento no se encuentra, como la sucesión está ordenada, una comparación adicional (línea 7) localizará la mitad de la secuencia en la que el elemento aparecerá si está presente. Luego llamamos de manera recursiva a la búsqueda binaria (línea 11) para continuar la búsqueda.

ALGORITMO 5.3.2 *Búsqueda binaria*

Este algoritmo busca un valor en una sucesión creciente y regresa el índice del valor si se encuentra, o 0 en caso contrario.

Entrada: Una sucesión $s_i, s_i + 1, \ldots, s_j, i \geq 1$, en orden creciente, un valor *key* (elemento que se busca), i y j.

Salida: La salida es un índice k para el cual $s_k = key$, o si *key* no está en la sucesión, la salida es el valor 0.

```
1.  procedure binary_search (s, i, j, key)
2.    if i > j then    // no encontrado
3.      return (0)
4.    k := ⌊(i + j) / 2⌋
5.    if key = s_k then    // encontrado
6.      return (k)
7.    if key < s_k then    // busca en la mitad izquierda
8.      j := k − 1
9.    else    // busca en la mitad derecha
10.     i := k + 1
11.   return (binary_search (s, i, j, key))
12. end binary_search
```

EJEMPLO 5.3.3

Ilustramos el algoritmo 5.3.2 para la entrada

$$s_1 = \text{'B'}, \quad s_2 = \text{'D'}, \quad s_3 = \text{'F'}, \quad s_4 = \text{'S'},$$

y *key* = 'S'. En la línea 2, como $i > j$ $(1 > 4)$ es falso, pasamos a la línea 4, donde hacemos k igual a 2. En la línea 5, como *key* ('S') no es igual a s_2 ('D'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('S' < 'D') es falsa, de modo que en la línea 10, hacemos i igual a 3. Luego llamamos a este algoritmo con $i = 3, j = 4$ para buscar *key* en

$$s_3 = \text{'F'}, \quad s_4 = \text{'S'}.$$

En la línea 2, como $i > j$ $(3 > 4)$ es falso, pasamos a la línea 4, donde hacemos k igual a 3. En la línea 5, como *key* ('S') no es igual a s_3 ('F'), pasamos a la línea 7. En la línea 7,

key $< s_k$ ('*S*' < '*F*') es falso, de modo que en la línea 10 hacemos *i* igual a 4. Luego llamamos de nuevo a este algoritmo con $i = j = 4$ para buscar *key* en

$$s_4 = \text{'}S\text{'}.$$

En la línea 2, como $i > j$ $(4 > 4)$ es falso, pasamos a la línea 4, donde hacemos *k* igual a 4. En la línea 5, como *key* ('*S*') es igual a s_4 ('*S*'), regresamos 4, el índice de *key* en la sucesión *s*. □

Ahora analizaremos el peor de los casos de la búsqueda binaria. Definimos el tiempo necesario para la búsqueda binaria en el peor de los casos como el número de veces que se llama al algoritmo en el peor de los casos para una sucesión que contiene *n* elementos. Sea a_n el tiempo en el peor de los casos.

Supongamos que *n* es 1; es decir, supongamos que la sucesión consta de un elemento s_i e $i = j$. En el peor de los casos, el elemento no está en la línea 5, de modo que el algoritmo será llamado una segunda vez en la línea 11. Sin embargo, al llamarlo por segunda vez, tendremos $i > j$ y el algoritmo terminará sin éxito en la línea 3. Hemos mostrado que si *n* es 1, el algoritmo se llama dos veces. Obtenemos la condición inicial

$$a_1 = 2. \tag{5.3.1}$$

Ahora, supongamos que $n > 1$. En este caso, $i < j$, de modo que la condición en la línea 2 es falsa. En el peor de los casos, el elemento no se encontrará en la línea 5, de modo que el algoritmo será llamado en la línea 11. Por definición, la llamada en la línea 11 requerirá *m* total de a_m llamadas, donde *m* es el tamaño de la secuencia que se introduce en la línea 11. Como los tamaños de los lados izquierdo y derecho de la sucesión original son $\lfloor (n-1)/2 \rfloor$ y $\lfloor n/2 \rfloor$ y el peor de los casos ocurre con la sucesión de mayor tamaño, el número total de llamadas en la línea 11, será $a_{\lfloor n/2 \rfloor}$. La llamada original, junto con las llamadas en la línea 11 proporcionan el total de llamadas; así, obtenemos la relación de recurrencia

$$a_n = 1 + a_{\lfloor n/2 \rfloor}. \tag{5.3.2}$$

La relación de recurrencia (5.3.2) es típica de aquellas que son resultado de algoritmos del tipo divide y vencerás. Por lo general tales relaciones de recurrencia no se resuelven tan fácilmente en forma explícita (sin embargo, véase el ejercicio 6). En vez de esto, uno estima el crecimiento de la sucesión correspondiente mediante la notación theta. Nuestro método para deducir una notación theta para la sucesión definida mediante (5.3.1) y (5.3.2) ilustra un método general de manejo de tales relaciones de recurrencia. Primero *resolvemos de manera explícita* (5.3.2) en el cado de que *n* sea una potencia de 2. Cuando *n* no es una potencia de 2, *n* está entre dos potencias de 2, digamos que 2^{k-1} y 2^k y a_n está entre $a_{2^{k-1}}$ y a_{2^k}. Como se conocen fórmulas explícitas para $a_{2^{k-1}}$ y a_{2^k}, podemos *estimar* a_n y de este modo deducir una notación theta para a_n.

En primer lugar, resolvemos la relación de recurrencia (5.3.2) en caso de que *n* sea una potencia de 2. Si $n = 2^k$, (5.3.2) se escribe

$$a_{2^k} = 1 + a_{2^{k-1}}, \quad k = 1, 2, \ldots.$$

Si hacemos $b_k = a_{2^k}$, obtenemos la relación de recurrencia

$$b_k = 1 + b_{k-1}, \quad k = 1, 2, \ldots. \tag{5.3.3}$$

y la condición inicial

$$b_0 = 2.$$

La relación de recurrencia (5.3.3) se puede resolver mediante el método iterativo:

$$b_k = 1 + b_{k-1} = 2 + b_{k-2} = \cdots = k + b_0 = k + 2.$$

Así, si $n = 2^k$,

$$a_n = 2 + \lg n. \tag{5.3.4}$$

Un valor arbitrario de *n* está entre dos potencias de 2, digamos

$$2^{k-1} < n \le 2^k. \tag{5.3.5}$$

Como la sucesión *a* es creciente (hecho que se puede *demostrar* por inducción, véase el ejercicio 5),

$$a_{2^{k-1}} \le a_n \le a_{2^k}. \tag{5.3.6}$$

Observe que (5.3.5) implica

$$k - 1 < \lg n \le k. \tag{5.3.7}$$

De (5.3.4), (5.3.6) y (5.3.7), deducimos que

$$\lg n < 1 + k = a_{2^{k-1}} \le a_n \le a_{2^k} = 2 + k < 3 + \lg n = O(\lg n).$$

Por tanto, $a_n = \Theta(\lg n)$, de modo que la búsqueda binaria es $\Theta(\lg n)$ en el peor de los casos. Este resultado es lo bastante importante como para resaltarse como teorema.

TEOREMA 5.3.4

El tiempo en el peor de los casos de la búsqueda binaria para una entrada de tamaño n es $\Theta(lg\ n)$.

Demostración. La demostración está antes del enunciado del teorema. ■

Como último ejemplo, presentamos y analizamos otro algoritmo de ordenamiento conocido como el **ordenamiento por fusión** (algoritmo 5.3.8). Mostraremos que el ordenamiento por fusión tiene un tiempo de ejecución en el peor de los casos de $\Theta(n \lg n)$, de modo que para entradas de gran tamaño, el ordenamiento por fusión es mucho más rápido que el ordenamiento por selección (algoritmo 5.3.1), el cual tiene un tiempo de ejecución en el peor de los casos $\Theta(n^2)$. En la sección 7.7 mostraremos que *cualquier* algoritmo de ordenamiento que compare los elementos y, con base en el resultado de una comparación, mueva los elementos dentro de un arreglo, tiene un tiempo de ejecución $\Omega(n \lg n)$ en el peor de los casos; así, el ordenamiento por fusión es óptimo dentro de esta clase de algoritmos de ordenamiento.

En el ordenamiento por fusión, la sucesión por ordenar,

$$s_i, \ldots, s_j,$$

se divide en dos sucesiones casi iguales,

$$s_i, \ldots, s_m, \quad s_{m+1}, \ldots, s_j,$$

donde $m = \lfloor (i + j)/2 \rfloor$. Cada una de estas sucesiones se ordena de manera recursiva, después de lo cual éstas se combinan para producir un arreglo ordenado de la sucesión original. El proceso de combinación de dos sucesiones ordenadas es una **fusión**.

ALGORITMO 5.3.5 *Fusión de dos sucesiones*

Este algoritmo combina dos sucesiones crecientes en una única sucesión creciente.

Entrada: Dos sucesiones crecientes $s_i, \ldots, s_m$ y $s_{m+1}, \ldots, s_j$, y los índices i, m y j

Salida: La sucesión $c_i, \ldots, c_j$ que consta de los elementos $s_i, \ldots, s_m$ y $s_{m+1}, \ldots, s_j$ combinados en una sucesión creciente

```
1.  procedure merge(s, i, m, j, c)
2.    // p es la posición en la sucesión s_i, ..., s_m
3.    // q es la posición en la sucesión s_{m+1}, ..., s_j
4.    // r es la posición en la sucesión c_i, ..., c_j
5.    p := i
6.    q := m + 1
7.    r := i
8.    // copiar el menor entre s_p y s_q
9.    while p ≤ m and q ≤ j do
10.     begin
11.     if s_p < s_q then
12.       begin
13.       c_r := s_p
14.       p := p + 1
15.       end
16.     else
17.       begin
18.       c_r := s_q
19.       q := q + 1
20.       end
21.     r := r + 1
22.     end
23.   // copiar el resto de la primera sucesión
24.   while p ≤ m do
25.     begin
26.     c_r := s_p
27.     p := p + 1
28.     r := r + 1
29.     end
30.   // copiar el resto de la segunda sucesión
31.   while q ≤ j do
32.     begin
33.     c_r := s_q
34.     q := q + 1
35.     r := r + 1
36.     end
37.   end merge
```


EJEMPLO 5.3.6

La figura 5.3.1 muestra la forma en que el algoritmo 5.3.5 fusiona las sucesiones

1, 3, 4 2, 4, 5, 6. □

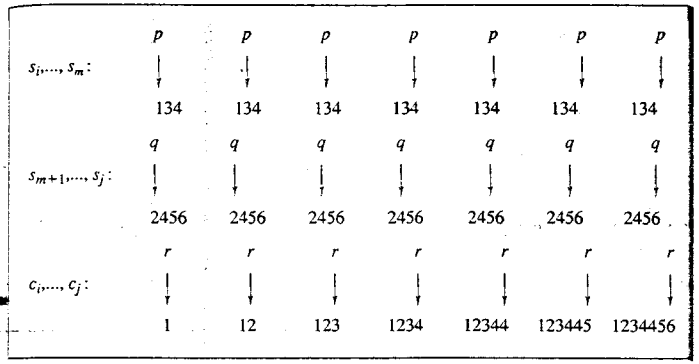


FIGURA 5.3.1 Fusión de $s_i, \ldots, s_m$ y $s_{m+1}, \ldots, s_j$. El resultado es $c_i, \ldots, c_j$.

El teorema 5.3.7 muestra que en el peor de los casos, se necesitan $n - 1$ comparaciones para fusionar dos sucesiones tales que la suma de sus longitudes es n.

TEOREMA 5.3.7

En el peor de los casos, el algoritmo 5.3.5 *requiere* $j - i$ *comparaciones. En particular, en el peor de los casos, se necesitan* $n - 1$ *comparaciones para fusionar dos sucesiones tales que la suma de sus longitudes es* n.

Demostración. En el algoritmo 5.3.5, la comparación de los elementos de la sucesión ocurre en el ciclo while de la línea 11. El ciclo while se ejecutará mientras $p \leq m$ y $q \leq j$. Así, en el peor de los casos, el algoritmo 5.3.5 necesita $j - i$ comparaciones. ■

A continuación utilizamos el algoritmo 5.3.5 (fusión) para construir el ordenamiento por fusión.

ALGORITMO 5.3.8 *Ordenamiento por fusión*

Este algoritmo recursivo ordena una sucesión de manera creciente utilizando el algoritmo 5.3.5, el cual fusiona dos sucesiones crecientes.

Entrada: $s_i, \ldots, s_j$, i y j

Salida: $s_i, \ldots, s_j$, ordenados de manera creciente

☆ 31. $A(n, m) = 1 + A(n-1, m-1) + A(n-1, m)$; $n-1 \geq m \geq 1$, $n \geq 2$; $A(n, 0) = A(n, n) = 1$, $n \geq 0$

32. Muestre que

$$f_n \geq \left(\frac{1+\sqrt{5}}{2}\right)^{n-1}, \qquad n \geq 1,$$

donde f denota la sucesión de Fibonacci.

33. La ecuación

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + f(n) \tag{5.2.20}$$

es una relación de **recurrencia lineal no homogénea de segundo orden con coeficientes constantes**.

Sea $g(n)$ una solución de (5.2.20). Muestre que cualquier solución U de (5.2.20) es de la forma

$$U_n = V_n + g(n), \tag{5.2.21}$$

donde V es una solución de la ecuación homogénea (5.2.13).

Si $f(n) = C$ en (5.2.20), se puede mostrar que $g(n) = C'$ en (5.2.21). Además, si $f(n) = Cn$, $g(n) = C_1'n + C_0'$; si $f(n) = Cn^2$, $g(n) = C_2'n^2 + C_1'n + C_0'$; y si $f(n) = C^n$, $g(n) = C'C^n$. Utilice estos hechos y el ejercicio 33 para determinar las soluciones generales de las relaciones de recurrencia de los ejercicios 34-39.

34. $a_n = 6a_{n-1} - 8a_{n-2} + 3$

35. $a_n = 7a_{n-1} - 10a_{n-2} + 16n$

36. $a_n = 2a_{n-1} + 8a_{n-2} + 81n^2$

37. $2a_n = 7a_{n-1} - 3a_{n-2} + 2^n$

38. $a_n = -8a_{n-1} - 16a_{n-2} + 3n$

39. $9a_n = 6a_{n-1} - a_{n-2} + 5n^2$

40. La ecuación

$$a_n = f(n)a_{n-1} + g(n)a_{n-2} \tag{5.2.22}$$

es una **relación de recurrencia lineal homogénea de segundo orden**. Los coeficientes $f(n)$ y $g(n)$ no necesariamente son constantes. Muestre que si S y T son soluciones de (5.2.22), entonces $bS + dT$ también es solución de (5.2.22).

41. Suponga que ambas raíces de

$$t^2 - c_1 t - c^2 = 0$$

son iguales a r y supongamos que a_n satisface

$$a_n = c_1 a_{n-1} + c_2 a_{n-2}, \qquad a_0 = C_0, \qquad a_1 = C_1.$$

Muestre que existen constantes b y d tales que

$$a_n = br^n + dnr^n, \qquad n = 0, 1, \ldots,$$

lo cual completa la demostración del teorema 5.2.14.

42. Sea a_n el número mínimo de enlaces necesarios para resolver el problema de comunicación de n nodos (véase el ejercicio 48 de la sección 5.1). Utilice la iteración para mostrar que $a_n \leq 2n - 4$, $n \geq 4$.

El juego de la Torre de Hanoi con cuatro postes y n discos tiene las mismas reglas que el juego con tres postes; la única diferencia es que existe un poste más. Los ejercicios 43-46 se refieren al siguiente algoritmo para resolver el juego de la Torre de Hanoi con cuatro postes y n discos.

Suponga que los postes están numerados 1, 2, 3, 4 y que el problema consiste en mover los discos, que inicialmente están apilados en el poste 1, hasta el poste 4. Si $n = 1$, se mueve el disco al poste 4 y se concluye. Si $n > 1$, sea k_n el máximo entero que satisface

$$\sum_{i=1}^{k_n} i \leq n.$$

Fijemos k_n en la parte inferior del poste 1. Se llama este algoritmo en forma recursiva para mover los $n - k_n$ discos en la parte superior del poste 1 al poste 2. Durante esta parte del algoritmo, los k_n discos inferiores en el poste 1 permanecen fijos. A continuación, se mueven los k_n discos del poste 1 al poste 4 llamando al algoritmo óptimo del caso de tres postes (véase el ejemplo 5.1.8) utilizando sólo los postes 1, 3 y 4. Por último, de nuevo se llama recursivamente a este algoritmo para mover los $n - k_n$ discos del poste 2 al poste 4. Durante esta parte del algoritmo, los k_n discos del poste 4 permanecen fijos. Sea $T(n)$ el número de movimientos necesarios para este algoritmo.

Este algoritmo, aunque se sabe que no es óptimo, utiliza la menor cantidad de movimientos entre todos los algoritmos propuestos para el problema de las cuatro espigas.

43. Deduzca la relación de recurrencia

$$T(n) = 2T(n - k_n) + 2^{k_n} - 1.$$

44. Calcule $T(n)$ para $n = 1, \ldots, 10$. Compare estos valores con el número óptimo de movimientos necesarios para resolver el problema de los tres postes.

☆ 45. Sea

$$r_n = n - \frac{k_n(k_n+1)}{2}.$$

Utilice inducción o algún otro método para demostrar que

$$T(n) = (k_n + r_n - 1)2^{k_n} + 1.$$

★ **46.** Muestre que $T(n) = O(4^{\sqrt{n}})$.

RINCÓN DE SOLUCIÓN DE PROBLEMAS: RELACIONES DE RECURRENCIA

Problema

(a) En una comida, n personas dejan su abrigo en el guardarropa. Al salir, los abrigos se regresan al azar; por desgracia, nadie recibe el abrigo correcto. Sea D_n el número de formas en que n personas pueden recibir todas los abrigos equivocados. Muestre que la sucesión $D_1, D_2, \ldots$ satisface la relación de recurrencia

$$D_n = (n-1)(D_{n-1} + D_{n-2}).$$

(b) Resuelva la relación de recurrencia de la parte (a) realizando la sustitución

$$C_n = D_n - nD_{n-1}.$$

Para enfrentar el problema

Antes de enfrentar el problema, veamos qué es lo necesario en la parte (a). Para demostrar la relación de recurrencia, debemos reducir el problema de los abrigos incorrectos para n personas a los problemas de los abrigos incorrectos para $(n-1)$ y $(n-2)$ personas (pues en la fórmula, D_n está dado en términos de D_{n-1} y D_{n-2}). Así, al revisar de manera sistemática algunos ejemplos, debemos intentar ver la forma en que el caso para n personas se relaciona con los casos para $n-1$ y $n-2$ personas. La situación es similar a la de la inducción matemática y los algoritmos recursivos, en donde una instancia dada de un problema se relaciona con instancias menores del mismo problema.

Ahora veamos algunos ejemplos. El mínimo caso es $n = 1$. Una única persona debe recibir el abrigo correcto, de modo que $D_1 = 0$. Para $n = 2$, sólo hay una forma en que todas reciban el abrigo equivocado: la persona 1 recibe el abrigo 2 y la persona 2 recibe el abrigo 1. Así, $D_2 = 1$. Antes de continuar, desarrollaremos algo de notación para la distribución de abrigos. Una notación elegida en forma cuidadosa puede ayudar a resolver un problema.

Escribiremos

$$c_1, \quad c_2, \ldots, \quad c_n$$

para indicar que la persona 1 recibió el abrigo c_1, que la persona 2 recibió el abrigo c_2, y así sucesivamente. La única forma de que dos personas obtengan los abrigos incorrectos se denota 2, 1.

Si $n = 3$, la persona 1 puede recibir el abrigo 2 o 3, de modo que las posibilidades son 2, ?, ? y 3, ?, ?. Veamos ahora el caso de los números faltantes. Supongamos que la persona 1 recibe el abrigo 2. La persona 2 no puede recibir el abrigo 1 (pues entonces la persona 3 recibiría el abrigo correcto); así, la persona 2 recibe el abrigo 3. Esto deja el abrigo 1 para la persona 3. Así, si la persona 1 recibe el abrigo 2, la única posibilidad es

$$2, 3, 1.$$

Supongamos que la persona 1 recibe el abrigo 3. La persona 3 no puede recibir el abrigo 1 (ya que entonces la persona 2 recibiría el abrigo correcto); así, la persona 3 recibe el abrigo 2. Esto deja el abrigo 1 para la persona 2. Así, si la persona 1 recibe el abrigo 3, la única posibilidad es

$$3, 1, 2.$$

Así, $D_3 = 2$.

Verifiquemos si la relación de recurrencia es válida para $n = 3$:

$$D_3 = 2 = 2(1 + 0) = (3-1)(D_2 + D_1).$$

Si $n = 4$, la persona 1 recibe el abrigo 2, 3 o 4, de modo que las posibilidades son 2, ?, ?, ?; 3, ?, ?, ?; y 4, ?, ?, ?. (Si hay n personas, la persona 1 recibe el abrigo 2 o 3 o $\ldots$ o $n-1$. Estas $n-1$ posibilidades implican la presencia del factor $n-1$ en la relación de recurrencia.) Veamos cuáles son los números faltantes. Supongamos que la persona 1 recibe el abrigo 2. Si la persona 2 recibe el abrigo 1, entonces la persona 3 recibe el abrigo 4 y la persona 4 recibe el abrigo 3, lo cual da 2, 1, 4, 3. Si la persona 2 no recibe el abrigo 1, las posibilidades son 2, 3, 4, 1 y 2, 4, 1, 3. Así, si la persona 1 recibe el abrigo 2, existen tres posibilidades. De manera análoga, si la persona 1 recibe el abrigo 3, existen tres posibilidades, y si la persona 1 recibe el abrigo 4, existen tres posibilidades. Usted debe enumerar las posibilidades para confirmar esta última afirmación. Así, $D_4 = 9$.

Verifiquemos que la relación de recurrencia también es válida para $n = 4$:

$$D_4 = 9 = 3(2 + 1) = (4-1)(D_3 + D_2).$$

Antes de continuar, el lector debe analizar el caso $n = 5$. Enumere sólo las posibilidades cuando la persona 1 reciba el abrigo 2. (Existen demasiadas posibilidades como para enumerarlas.) También verifique la relación de recurrencia para $n = 5$.

Observe que si la persona 1 recibe el abrigo 2 y la persona 2 recibe el abrigo 1 (es decir, si las personas 1 y 2 intercambian sus abrigos), el número de formas en que las demás personas reciben el abrigo equivocado es D_{n-2} (las restantes $n-2$ personas deben recibir abrigos equivocados). Por esto aparece D_{n-2} en la relación de recurrencia. Tendremos una solución, siempre que el término D_{n-1} aparezca en el caso restante: cuando la persona 1 recibe el abrigo 2, pero la persona 2 no recibe el abrigo 1 (es decir, si las personas 1 y 2 no intercambian sus abrigos).

Determinación de una solución

Supongamos que existen n personas. Resumiremos el argumento que hemos desarrollado a través de los ejemplos. La persona 1 puede recibir el abrigo 2, o 3, $\ldots$, o n; así, existen $n-1$ formas posibles de que la persona 1 reciba el abrigo equivocado. Supongamos que la persona 1 recibe el abrigo 2. Existen dos posibilidades: la persona 2 recibe el abrigo 1 o no recibe el abrigo 1. Si la persona 2 recibe el abrigo 1, el número de formas en que las demás personas reciben el abrigo equivocado es D_{n-2}. El caso restante es que la persona 2 no reciba el abrigo 1.

Escribamos con cuidado lo que debemos contar. Las personas 2, 3, $\ldots$, n tienen entre ellas los abrigos 1, 3, 4, $\ldots$, n (falta el abrigo 2 pues la persona 1 lo tiene). Queremos determinar el número de formas en que las personas 2, 3, $\ldots$, n recibirán el abrigo equivocado y que la persona 2 no reciba el abrigo 1. Éste es casi el problema de que $n-1$ personas no reciban los abrigos correctos. Podemos transformarlo en este problema si decimos a las personas 2, 3, $\ldots$, n que el abrigo 1 es el abrigo 2. (¡Basta coserle una etiqueta temporal!) Ahora, la persona 2 no tendrá el abrigo 1, pues la persona 2 piensa que es el abrigo 2. Como existen $n-1$ personas, existen D_{n-1} formas en que las personas 2, 3, $\ldots$, n recibirán el abrigo equivocado y la persona 2 no recibirá el abrigo 1. Esto implica que existen $D_{n-1} + D_{n-2}$ formas en que la persona 1 recibirá el abrigo 2 de modo que las demás personas tengan los abrigos equivocados. Como existen $n-1$ formas posibles en que la persona 1 obtenga el abrigo equivocado, obtenemos la relación de recurrencia deseada.

La relación de recurrencia define a D_n en términos de D_{n-1} y D_{n-2}, de modo que no se puede resolver mediante iteración. Además, la relación de recurrencia no tiene coeficientes constantes (aunque es lineal), de modo que no puede resolverse mediante el teorema 5.2.11 o 5.2.14. Esto explica la necesidad de realizar la sustitución en la parte (b). Es claro que después de hacer la sustitución, podemos resolver la relación de recurrencia en términos de C_n mediante los métodos de la sección 5.2.

Si desarrollamos

$$D_n = (n-1)(D_{n-1} + D_{n-2}),$$

obtenemos

$$D_n = nD_{n-1} - D_{n-1} + (n-1)D_{n-2}.$$

Si entonces pasamos nD_{n-1} al lado izquierdo de la ecuación (para obtener una expresión igual a C_n), obtenemos

$$D_n - nD_{n-1} = -D_{n-1} + (n-1)D_{n-2}.$$

Ahora, el lado izquierdo de la ecuación es igual a C_n y el lado derecho es igual a $-C_{n-1}$. Así, obtenemos la relación de recurrencia

$$C_n = -C_{n-1}.$$

Esta ecuación se puede resolver mediante iteración.

Solución formal

Parte (a): Supongamos que las n personas tienen los abrigos equivocados. Consideremos el abrigo que tiene una persona p. Supongamos que p tiene el abrigo de q. Consideremos dos casos: q tiene el abrigo de p y q no tiene el abrigo de p.

Existen D_{n-2} distribuciones en las que q tiene el abrigo de p dado que los $n-2$ abrigos restantes están en posesión de las restantes $n-2$ personas, pero cada una tiene el abrigo equivocado.

Demostramos que hay D_{n-1} distribuciones en las cuales q no tiene el abrigo de p. Observe que el conjunto de abrigos C que tienen las $n-1$ personas (excluyendo a p) incluye a todos los abrigos menos el de q (pues p lo tiene). Asignemos por un momento la propiedad del abrigo de p a q. Entonces, cualquier distribución de C entre las $n-1$ personas en las que nadie tiene su propio abrigo proporciona una distribución en la cual q no tiene el abrigo que realmente es de p. Como existen D_{n-1} de tales distribuciones, existen D_{n-1} distribuciones en las que q no tiene el abrigo de p.

Esto implica que existen $D_{n-1} + D_{n-2}$ distribuciones en las que p tiene el abrigo de q. Como p puede tener cualquiera de $n-1$ abrigos, obtenemos la relación de recurrencia.

Parte (b): Al hacer la sustitución dada, obtenemos

$$C_n = -C_{n-1}.$$

Al utilizar iteración obtenemos

$$C_n = (-1)^1 C_{n-1} = (-1)^2 C_{n-2} = \cdots$$
$$= (-1)^{n-2} C_2 = (-1)^n C_2 = (-1)^n (D_2 - 2D_1) = (-1)^n.$$

Por tanto,

$$D_n - nD_{n-1} = (-1)^n.$$

Al resolver esta última relación de recurrencia mediante iteración, obtenemos

$$\begin{aligned} D_n &= (-1)^n + nD_{n-1} \\ &= (-1)^n + n[(-1)^{n-1} + (n-1)D_{n-2}] \\ &= (-1)^n + n(-1)^{n-1} + n(n-1)[(-1)^{n-2} + (n-2)D_{n-3}] \\ &\cdots \\ &= (-1)^n + n(-1)^{n-1} + n(n-1)(-1)^{n-2} + \cdots \\ &\quad -[n(n-1)\cdots 4] + [n(n-1)\cdots 3]. \end{aligned}$$

Resumen de técnicas para resolver problemas

- Cuando los ejemplos tienen un lenguaje complicado, desarrolle una notación para describirlos en forma concisa. La elección cuidadosa de una notación puede ayudar en gran medida a resolver un problema.
- Al analizar los ejemplos, intente ver la forma en que el problema en cuestión se relaciona con instancias menores del mismo problema.
- Con frecuencia es útil escribir con cuidado aquello que se debe contar.
- A veces es posible convertir una relación de recurrencia que no es una ecuación lineal homogénea con coeficientes constantes en una ecuación lineal homogénea con coeficientes constantes. Dicha relación de recurrencia se puede resolver entonces mediante los métodos de la sección 5.2.

Comentario

El nombre técnico de una permutación en la que ningún elemento queda en su posición original es un **desordenamiento**.

5.3 *APLICACIONES AL ANÁLISIS DE ALGORITMOS*

En esta sección utilizamos las relaciones de recurrencia para analizar el tiempo necesario utilizado por los algoritmos. La técnica consiste en desarrollar una relación de recurrencia y las condiciones iniciales que definan una sucesión $a_1, a_2, \ldots$, donde a_n es el tiempo (en el mejor de los casos, en el caso promedio o en el peor de los casos) necesario para que un algoritmo procese una entrada de tamaño n. Al resolver la relación de recurrencia, podemos determinar el tiempo necesario utilizado por el algoritmo.

Nuestro primer algoritmo es una versión del algoritmo de ordenamiento por selección. Este algoritmo selecciona el elemento máximo y lo coloca al final, para luego repetir este proceso de manera recursiva.

ALGORITMO 5.3.1 *Ordenamiento por selección*

Este algoritmo ordena la sucesión

$$s_1, s_2, \ldots, s_n$$

entonces U es una solución de (5.2.9).

Para satisfacer las condiciones iniciales (5.2.10), debemos tener

$$7 = U_0 = b2^0 + d3^0 = b + d, \quad 16 = U_1 = b2^1 + d3^1 = 2b + 3d.$$

Al resolver estas ecuaciones en términos de b y d, obtenemos

$$b = 5, \quad d = 2.$$

Por tanto, la sucesión U definida como

$$U_n = 5 \cdot 2^n + 2 \cdot 3^n$$

satisface la relación de recurrencia (5.2.9) y las condiciones iniciales (5.2.10). Concluimos que

$$a_n = U_n = 5 \cdot 2^n + 2 \cdot 3^n, \quad \text{para } n = 0, 1, \ldots.$$

Ahora haremos un resumen y justificaremos las técnicas utilizadas para resolver la relación de recurrencia anterior.

TEOREMA 5.2.11

Sea

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} \tag{5.2.13}$$

una relación de recurrencia lineal homogénea de segundo orden con coeficientes constantes.

Si S y T son soluciones de (5.2.13), *entonces $U = bS + dT$ también es solución de* (5.2.13).

Si r es una raíz de

$$t^2 - c_1 t - c_2 = 0, \tag{5.2.14}$$

entonces la sucesión r^n, $n = 0, 1, \ldots$, es una solución de (5.2.13).

Si a es la sucesión definida mediante (5.2.13),

$$a_0 = C_0, \quad a_1 = C_1, \tag{5.2.15}$$

y r_1 y r_2 son raíces de (5.2.14) con $r_1 \neq r_2$, entonces existen constantes b y d tales que

$$a_n = br_1^n + dr_2^n, \quad n = 0, 1, \ldots.$$

Demostración. Como S y T son soluciones de (5.2.13),

$$S_n = c_1 S_{n-1} + c_2 S_{n-2}, \quad T_n = c_1 T_{n-1} + c_2 T_{n-2}.$$

Si multiplicamos la primera ecuación por b y la segunda por d y sumamos, obtenemos

$$\begin{aligned} U_n = bS_n + dT_n &= c_1(bS_{n-1} + dT_{n-1}) + c_2(bS_{n-2} + dT_{n-2}) \\ &= c_1 U_{n-1} + c_2 U_{n-2}. \end{aligned}$$

Por tanto, U es una solución de (5.2.13).

Como r es una raíz de (5.2.14),

$$r^2 = c_1 r + c_2.$$

Ahora,

$$c_1 r^{n-1} + c_2 r^{n-2} = r^{n-2}(c_1 r + c_2) = r^{n-2} r^2 = r^n;$$

de modo que la sucesión r^n, $n = 0,1, \ldots$, es una solución de (5.2.13).

Si hacemos $U_n = br_1^n + dr_2^n$, entonces U es una solución de (5.2.13). Para cumplir con las condiciones iniciales (5.2.15), debemos tener

$$U_0 = b + d = C_0, \quad U_1 = br_1 + dr_2 = C_1.$$

Si multiplicamos la primera ecuación por r_1 y restamos, obtenemos

$$d(r_1 - r_2) = r_1 C_0 - C_1.$$

Como $r_1 - r_2 \neq 0$, podemos despejar d. De manera análoga, podemos determinar b. Con estas elecciones de b y d, tenemos

$$U_0 = C_0, \quad U_1 = C_1.$$

Sea a la sucesión definida mediante (5.2.13) y (5.2.15). Como U también satisface (5.2.13) y (5.2.15), esto implica que $U_n = a_n$, $n = 0,1, \ldots$. ■

EJEMPLO 5.2.12 ***Más sobre crecimiento de poblaciones***

Suponga que la población de venados de Rustic County es de 200 en el instante $n = 0$ y de 220 en el instante $n = 1$ y que el incremento del instante $n - 1$ al instante n es el doble del incremento del instante $n - 2$ al instante $n - 1$. Escribir una relación de recurrencia y una condición inicial que definan la población de venados en el instante n y luego resolver la relación de recurrencia.

Sea d_n la población de venados en el instante n. Tenemos las condiciones iniciales

$$d_0 = 200, \quad d_1 = 220.$$

El incremento del instante $n - 1$ al instante n es $d_n - d_{n-1}$ y el incremento del instante $n - 2$ al instante $n - 1$ es $d_{n-1} - d_{n-2}$. Así, obtenemos la relación de recurrencia

$$d_n - d_{n-1} = 2(d_{n-1} - d_{n-2}),$$

la cual se puede escribir como

$$d_n = 3d_{n-1} - 2d_{n-2}.$$

Para resolver esta relación de recurrencia, primero resolvemos la ecuación cuadrática

$$t^2 - 3t + 2 = 0$$

para obtener las raíces 1 y 2. La sucesión d es de la forma

$$d_n = b \cdot 1^n + c \cdot 2^n = b + c2^n.$$

Para cumplir con las condiciones iniciales, debemos tener

$$200 = d_0 = b + c, \quad 220 = d_1 = b + 2c.$$

Al despejar b y c, tenemos que $b = 180$ y $c = 20$. Así, d_n está dada por

$$d_n = 180 + 20 \cdot 2^n.$$

Como en el ejemplo 5.2.3, el crecimiento es exponencial. □

EJEMPLO 5.2.13

Determine una fórmula explícita para la sucesión de Fibonacci.

La sucesión de Fibonacci se define mediante la relación de recurrencia lineal homogénea de segundo orden

$$f_n - f_{n-1} - f_{n-2} = 0, \quad n \geq 3,$$

y las condiciones iniciales

$$f_1 = 1, \quad f_2 = 2.$$

Primero utilizamos la fórmula cuadrática para resolver

$$t^2 - t - 1 = 0.$$

Las soluciones son

$$t = \frac{1 \pm \sqrt{5}}{2}.$$

Así, la solución es de la forma

$$f_n = b\left(\frac{1+\sqrt{5}}{2}\right)^n + d\left(\frac{1-\sqrt{5}}{2}\right)^n.$$

Para satisfacer las condiciones iniciales, debemos tener

$$b\left(\frac{1+\sqrt{5}}{2}\right) + d\left(\frac{1-\sqrt{5}}{2}\right) = 1$$

$$b\left(\frac{1+\sqrt{5}}{2}\right)^2 + d\left(\frac{1-\sqrt{5}}{2}\right)^2 = 2.$$

Al resolver estas ecuaciones en términos de b y d, obtenemos

$$b = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right), \qquad d = -\frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right).$$

Por tanto, una fórmula explícita para la sucesión de Fibonacci es

$$f_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^{n+1} - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^{n+1}$$

Es sorprendente que aunque f_n sea un entero, la fórmula anterior implica el uso del número irracional $\sqrt{5}$. □

El teorema 5.2.11 establece que cualquier solución de (5.2.13) puede darse en términos de dos soluciones básicas r_1^n y r_2^n. Sin embargo, si (5.2.14) tiene dos raíces iguales r, sólo obtenemos una solución básica r^n. El siguiente teorema muestra que en este caso, nr^n proporciona la otra solución básica.

TEOREMA 5.2.14

Sea

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} \tag{5.2.16}$$

una relación de recurrencia lineal homogénea de segundo orden con coeficientes constantes.

Sea a la sucesión que satisface (5.2.16) *y*

$$a_0 = C_0, \quad a_1 = C_1.$$

Si ambas raíces de

$$t^2 - c_1 t - c_2 = 0 \tag{5.2.17}$$

son iguales a r, *entonces existen constantes b y d tales que*

$$a_n = br^n + dnr^n, \quad n = 0, 1, \ldots.$$

Demostración. La demostración del teorema 5.2.11 muestra que la sucesión r^n, $n = 0, 1, \ldots$, es una solución de (5.2.16). Mostraremos que la sucesión nr^n, $n = 0, 1, \ldots$, también es una solución de (5.2.16).

Como r es la única solución de (5.2.17), debemos tener

$$t^2 - c_1 t - c_2 = (t - r)^2.$$

Esto implica que

$$c_1 = 2r, \quad c_2 = -r^2.$$

Ahora,

$$c_1[(n-1)r^{n-1}] + c_2[(n-2)r^{n-2}] = 2r(n-1)r^{n-1} - r^2(n-2)r^{n-2}$$
$$= r^n[2(n-1) - (n-2)] = nr^n.$$

Por tanto, la sucesión nr^n, $n = 0, 1, \ldots$, es una solución de (5.2.16).

Por el teorema 5.2.11, la sucesión U definida como $U_n = br^n + dnr^n$ es una solución de (5.2.16).

La demostración de que existen constantes b y d tales que $U_0 = C_0$ y $U_1 = C_1$ es similar al argumento dado en el teorema 5.2.11 y se deja como ejercicio (ejercicio 41). Esto implica que $U_n = a_n$, $n = 0, 1, \ldots$. ■

EJEMPLO 5.2.15

Resuelva la relación de recurrencia

$$d_n = 4(d_{n-1} - d_{n-2}) \tag{5.2.18}$$

sujeta a las condiciones iniciales

$$d_0 = 1 = d_1.$$

De acuerdo con el teorema 5.2.11, $S_n = r^n$ es una solución de (5.2.18), donde r es una solución de

$$t^2 - 4t + 4 = 0. \tag{5.2.19}$$

Así, obtenemos la solución

$$S_n = 2n$$

de (5.2.18). Como 2 es la única solución de (5.2.19), por el teorema 5.2.14,

$$T_n = n2^n$$

también es una solución de (5.2.18). Así, la solución general de (5.2.18) es de la forma

$$U = aS + bT.$$

Debemos tener

$$U_0 = 1 = U_1.$$

Estas últimas ecuaciones se convierten en

$$aS_0 + bT_0 = a + 0b = 1, \quad aS_1 + bT_1 = 2a + 2b = 1.$$

Al despejar a y b, obtenemos

$$a = 1, \quad b = -\tfrac{1}{2}.$$

Por tanto, la solución de (5.2.18) es

$$d_n = 2^n - n2^{n-1}.$$

Para la relación de recurrencia general lineal homogénea de orden k con coeficientes constantes (5.2.5), si r es una raíz de

$$t^k - c_1t^{k-1} - c_2t^{k-2} - \cdots - c_k = 0$$

de multiplicidad m, se puede mostrar que

$$r^n, \quad nr^n, \quad \cdots, \quad n^{m-1}r^n$$

son soluciones de (5.2.5). Este hecho se puede utilizar, al igual que en los ejemplos anteriores para las relaciones de recurrencia de orden 2, para resolver una relación de recurrencia lineal homogénea de orden k con coeficientes constantes. Para un enunciado preciso y una demostración del resultado general, véase [Brualdi].

Ejercicios

Indique si cada relación de recurrencia en los ejercicios 1-10 es una relación de recurrencia lineal homogénea con coeficientes constantes. Proporcione el orden de cada relación de recurrencia lineal homogénea con coeficientes constantes.

1. $a_n = -3a_{n-1}$

2. $a_n = 2na_{n-1}$

3. $a_n = 2na_{n-2} - a_{n-1}$

4. $a_n = a_{n-1} + n$

5. $a_n = 7a_{n-2} - 6a_{n-3}$

6. $a_n = a_{n-1} + 1 + 2^{n-1}$

7. $a_n = (\lg 2n)a_{n-1} - [\lg(n-1)]a_{n-2}$

8. $a_n = 6a_{n-1} - 9a_{n-2}$

9. $a_n = -a_{n-1} - a_{n-2}$

10. $a_n = -a_{n-1} + 5a_{n-2} - 3a_{n-3}$

En los ejercicios 11-25, resuelva la relación de recurrencia dada para las condiciones iniciales dadas.

11. Ejercicio 1; $a_0 = 2$

12. Ejercicio 2; $a_0 = 1$

13. Ejercicio 4; $a_0 = 0$

14. $a_n = 6a_{n-1} - 8a_{n-2}$; $a_0 = 1$, $a_1 = 0$

15. $a_n = 7a_{n-1} - 10a_{n-2}$; $a_0 = 5$, $a_1 = 16$

16. $a_n = 2a_{n-1} + 8a_{n-2}$; $a_0 = 4$, $a_1 = 10$

17. $2a_n = 7a_{n-1} - 3a_{n-2}$; $a_0 = a_1 = 1$

18. Ejercicio 6; $a_0 = 0$

19. Ejercicio 8; $a_0 = a_1 = 1$

20. $a_n = -8a_{n-1} - 16a_{n-2}$; $a_0 = 2$, $a_1 = -20$

21. $9a_n = 6a_{n-1} - a_{n-2}$; $a_0 = 6$, $a_1 = 5$

22. La sucesión de Lucas

$$L_n = L_{n-1} + L_{n-2}, \quad n \geq 3; \quad L_1 = 1, \quad L_2 = 3$$

23. Ejercicio 50, sección 5.1

24. Ejercicio 52, sección 5.1

25. La relación de recurrencia anterior al ejercicio 53, sección 5.1

26. Suponga que la población de venados de Rustic County es 0 en el instante $n = 0$. Supongamos que en el instante n, se llevan $100n$ venados a Rustic County y que la población crece 20% cada año. Escriba una relación de recurrencia y una condición inicial que defina la población de venados en el instante n y luego resuelva la relación de recurrencia. La siguiente fórmula puede ser útil:

$$\sum_{i=1}^{n-1} ix^{i-1} = \frac{(n-1)x^n - nx^{n-1} + 1}{(x-1)^2}.$$

En ciertas ocasiones, una relación de recurrencia que no es una ecuación lineal homogénea con coeficientes constantes se puede transformar en una ecuación lineal homogénea con coeficientes constantes. En los ejercicios 27 y 28, realice la sustitución dada y resuelva la relación de recurrencia resultante, y luego determine la solución de la relación de recurrencia original.

27. Resuelva la relación de recurrencia

$$\sqrt{a_n} = \sqrt{a_{n-1}} + 2\sqrt{a_{n-2}}$$

con condiciones iniciales $a_0 = a_1 = 1$ realizando la sustitución $b_n = \sqrt{a_n}$.

28. Resuelva la relación de recurrencia

$$a_n = \sqrt{\frac{a_{n-2}}{a_{n-1}}}$$

con condiciones iniciales $a_0 = 8$, $a_1 = 1/(2\sqrt{2})$ calculando el logaritmo de ambos lados y realizando la sustitución $b_n = \lg a_n$.

En los ejercicios 29-31, resuelva la relación de recurrencia para las condiciones iniciales dadas.

29. $a_n = -2na_{n-1} + 3n(n-1)a_{n-2}$; $a_0 = 1$, $a_1 = 2$

30. $c_n = 2 + \Sigma_{i=1}^{n-1} c_i$, $n \geq 2$; $c_1 = 1$

☆ 63. Muestre que

$$S_{n,k} = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i(k-i)^n C(k,i),$$

donde $S_{n,k}$ denota un número de Stirling del segundo tipo (véase el ejercicio 75 de la sección 4.2).

64. Suponga que una persona invierte una suma de dinero a r% compuesto anualmente. Explique la siguiente regla: Para estimar el tiempo necesario para duplicar la inversión, divida 70 entre r.

65. Deduzca una relación de recurrencia para el número de multiplicaciones necesarias para evaluar un determinante $n \times n$ mediante el método de los cofactores.

Una *permutación sube/baja* es una permutación p de $1, 2, \ldots, n$ que satisface

$$p(i) < p(i+1) \quad \text{para } i = 1, 3, 5, \ldots$$

y

$$p(i) > p(i+1) \quad \text{para } i = 2, 4, 6, \ldots$$

Por ejemplo, existen cinco permutaciones sube/baja de 1, 2, 3, 4:

$$1,3,2,4; \quad 1,4,2,3; \quad 2,3,1,4; \quad 2,4,1,3; \quad 3,4,1,2.$$

Sea E_n el número de permutaciones sube/baja de $1, 2, \ldots, n$. (Defina $E_0 = 1$.) Los números $E_0, E_1, E_2, \ldots$ son los *números de Euler.*

66. Enumere todas las permutaciones sube/baja de 1, 2, 3. ¿Cuál es el valor de E_3?

67. Enumere todas las permutaciones sube/baja de 1, 2, 3, 4, 5. ¿Cuál es el valor de E_5?

68. Muestre que en una permutación sube/baja de $1, 2, \ldots, n$, n debe aparecer en la posición $2i$, para alguna i.

★ **69.** Utilice el ejercicio 68 para deducir la relación de recurrencia

$$E_n = \sum_{j=1}^{\lfloor n/2 \rfloor} C(n-1, 2j-1)E_{2j-1}E_{n-2j}.$$

☆ 70. Analice el lugar donde debe aparecer el 1 en una permutación sube/baja y deduzca la relación de recurrencia

$$E_n = \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} C(n-1, 2j)E_{2j}E_{n-2j-1}.$$

☆ 71. Demuestre que

$$E_n = \frac{1}{2}\sum_{j=1}^{n-1} C(n-1, j)E_j E_{n-j-1}.$$

5.2 *SOLUCIÓN DE RELACIONES DE RECURRENCIA*

Resolver una relación de recurrencia asociada a la sucesión $a_0, a_1, \ldots$ consiste en determinar una fórmula explícita para el término general a_n. En esta sección analizaremos dos métodos para resolver relaciones de recurrencia: la **iteración** y un método especial que se aplica a las **relaciones de recurrencia lineales homogéneas con coeficientes constantes.** Para otros métodos más poderosos, como aquellos que utilizan funciones generatrices, el lector puede consultar [Brualdi].

Para resolver una relación de recurrencia asociada a la sucesión $a_0, a_1, \ldots$ por iteración, utilizamos la relación de recurrencia para escribir el n-ésimo término a_n en términos de algunos de sus predecesores $a_{n-1}, \ldots, a_0$. Luego utilizamos de manera sucesiva la relación de recurrencia para reemplazar cada uno de los términos $a_{n-1}, \ldots$ por algunos de sus predecesores. Continuamos hasta obtener una fórmula explícita. Utilizamos el método iterativo para resolver la relación de recurrencia del ejemplo 5.1.3.

EJEMPLO 5.2.1

Podemos resolver la relación de recurrencia

$$a_n = a_{n-1} + 3, \tag{5.2.1}$$

sujeta a la condición inicial

$$a_1 = 2,$$

por iteración. Al reemplazar n con $n-1$ en (5.2.1), obtenemos

$$a_{n-1} = a_{n-2} + 3.$$

Si sustituimos esta expresión para a_{n-1} en (5.2.1), obtenemos

$$\begin{aligned} a_n &= \boxed{a_{n-1}} && +3 \\ &\downarrow \\ &= \boxed{a_{n-2}+3} && +3 \\ &= a_{n-2} + 2 \cdot 3. \end{aligned} \tag{5.2.2}$$

Al reemplazar n con $n-2$ en (5.2.1), obtenemos

$$a_{n-2} = a_{n-3} + 3.$$

Si sustituimos esta expresión para a_{n-2} en (5.2.2), obtenemos

$$\begin{aligned} a_n &= \boxed{a_{n-2}} && +2 \cdot 3 \\ &\downarrow \\ &= \boxed{a_{n-3}+3} && +2 \cdot 3 \\ &= a_{n-3} + 3 \cdot 3. \end{aligned}$$

En general, tenemos

$$a_n = a_{n-k} + k \cdot 3.$$

Si hacemos $k = n-1$ en esta última expresión, tenemos

$$a_n = a_1 + (n-1) \cdot 3.$$

Como $a_1 = 2$, obtenemos la fórmula explícita

$$a_n = 2 + 3(n-1)$$

para la sucesión a. □

EJEMPLO 5.2.2

Podemos resolver la relación de recurrencia

$$S_n = 2S_{n-1}$$

del ejemplo 5.1.5, sujeta a la condición inicial

$$S_0 = 1,$$

por iteración:

$$S_n = 2S_{n-1} = 2(2S_{n-2}) = \cdots = 2^n S_0 = 2^n. \qquad \square$$

EJEMPLO 5.2.3 *Crecimiento de poblaciones*

Suponga que la población de venados en Rustic County es 1000 en el instante $n = 0$ y que el incremento desde el instante $n - 1$ hasta el instante n es 10% del tamaño en el instante $n - 1$. Escriba una relación de recurrencia y una condición inicial que defina la población de venados en el instante n, y luego resuelva la relación de recurrencia.

Sea d_n la población de venados en el instante n. Tenemos la condición inicial

$$d_0 = 1000.$$

El incremento del instante $n - 1$ al instante n es $d_n - d_{n-1}$. Como este incremento es igual a 10% del tamaño en el instante $n - 1$, obtenemos la relación de recurrencia

$$d_n - d_{n-1} = 0.1d_{n-1},$$

que se puede escribir como

$$d_n = 1.1d_{n-1}.$$

La relación de recurrencia se puede resolver por iteración:

$$d_n = 1.1d_{n-1} = 1.1(1.1d_{n-2}) = (1.1)^2(d_{n-2})$$
$$= \cdots = (1.1)^n d_0 = (1.1)^n 1000.$$

La hipótesis implica un crecimiento exponencial de la población. $\square$

EJEMPLO 5.2.4

Determine una fórmula explícita para c_n, el número mínimo de movimientos en que puede resolverse el juego de la Torre de Hanoi con n discos (véase el ejemplo 5.1.8).

En el ejemplo 5.1.8 obtuvimos la relación de recurrencia

$$c_n = 2c_{n-1} + 1 \qquad (5.2.3)$$

y la condición inicial

$$c_1 = 1.$$

Al aplicar el método iterativo a (5.2.3), obtenemos

$$\begin{aligned} c_n &= 2c_{n-1} + 1 \\ &= 2(2c_{n-2} + 1) + 1 \\ &= 2^2 c_{n-2} + 2 + 1 \\ &= 2^2(2c_{n-3} + 1) + 2 + 1 \\ &= 2^3 c_{n-3} + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1} c_1 + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 \\ &= 2^n - 1. \end{aligned}$$

El último paso surge de la fórmula para la suma geométrica (véase el ejemplo 1.6.2). $\square$

EJEMPLO 5.2.5

Podemos resolver la relación de recurrencia

$$p_n = a - \frac{b}{k} p_{n-1}$$

para el precio p_n en el modelo económico del ejemplo 5.1.9 por iteración. Para que la notación sea más sencilla, hacemos $s = -b/k$.

$$\begin{aligned} p_n &= a + sp_{n-1} \\ &= a + s(a + sp_{n-2}) \\ &= a + as + s^2 p_{n-2} \\ &= a + as + s^2(a + sp_{n-3}) \\ &= a + as + as^2 + s^3 p_{n-3} \\ &\vdots \\ &= a + as + as^2 + \cdots + as^{n-1} + s^n p_0 \\ &= \frac{a - as^n}{1 - s} + s^n p_0 \\ &= s^n\left(\frac{-a}{1-s} + p_0\right) + \frac{a}{1-s} \\ &= \left(-\frac{b}{k}\right)^n \left(\frac{-ak}{k+b} + p_0\right) + \frac{ak}{k+b}. \end{aligned} \qquad (5.2.4)$$

Vemos que si $b/k < 1$, el término

$$\left(-\frac{b}{k}\right)^n \left(\frac{-ak}{k+b} + p_0\right)$$

es cada vez más pequeño conforme n crece, de modo que el precio tiende a estabilizarse en aproximadamente $ak/(k + b)$. Si $b/k = 1$, (5.2.4) muestra que p_n oscila entre p_0 y p_1. Si $b/k > 1$, (5.2.4) muestra que las diferencias entre los precios sucesivos aumentan. Anteriormente habíamos observado estas propiedades de manera gráfica (véase el ejemplo 5.1.9). □

Ahora veremos una clase particular de relaciones de recurrencia.

DEFINICIÓN 5.2.6

Una relación de recurrencia lineal homogénea de orden k con coeficientes constantes es una relación de recurrencia de la forma

$$a_n = c_1a_{n-1} + c_2a_{n-2} + \cdots + c_ka_{n-k}, \qquad c_k \neq 0. \tag{5.2.5}$$

Observe que una relación de recurrencia lineal homogénea de orden k con coeficientes constantes (5.2.5), junto con las k condiciones iniciales

$$a_0 = C_0, \qquad a_1 = C_1, \ldots, \qquad a_{k-1} = C_{k-1},$$

definen de manera única una sucesión $a_0, a_1, \ldots$.

EJEMPLO 5.2.7

Las relaciones de recurrencia

$$S_n = 2S_{n-1} \tag{5.2.6}$$

del ejemplo 5.2.2 y

$$f_n = f_{n-1} + f_{n-2}, \tag{5.2.7}$$

que define la sucesión de Fibonacci, son ambas relaciones de recurrencia lineales homogéneas con coeficientes constantes. La relación de recurrencia (5.2.6) es de orden 1 y (5.2.7) es de orden 2. □

EJEMPLO 5.2.8

La relación de recurrencia

$$a_n = 3a_{n-1}a_{n-2} \tag{5.2.8}$$

no es una relación de recurrencia lineal homogénea con coeficientes constantes. En una relación de recurrencia lineal homogénea con coeficientes constantes, cada término es de la forma ca_k. Los términos como $a_{n-1}a_{n-2}$ no están permitidos. Las relaciones de recurrencia como (5.2.8) son *no lineales*. □

EJEMPLO 5.2.9

La relación de recurrencia

$$a_n - a_{n-1} = 2n$$

no es una relación de recurrencia lineal homogénea con coeficientes constantes, debido a que la expresión del lado derecho de la ecuación no es igual a cero. (Tal ecuación es *no homogénea*. Las relaciones de recurrencia lineales no homogéneas con coeficientes constantes se analizan en los ejercicios 33-39.) □

EJEMPLO 5.2.10

La relación de recurrencia

$$a_n = 3na_{n-1}$$

no es una relación de recurrencia lineal homogénea con coeficientes constantes debido a que el coeficiente $3n$ no es constante. Es una relación de recurrencia lineal homogénea con coeficientes no constantes. □

Ilustraremos el método general de resolución de las relaciones de recurrencia lineales homogéneas con coeficientes constantes determinando una fórmula explícita para la sucesión definida mediante la relación de recurrencia

$$a_n = 5a_{n-1} - 6a_{n-2} \tag{5.2.9}$$

y condiciones iniciales

$$a_0 = 7, \qquad a_1 = 16. \tag{5.2.10}$$

En matemáticas, al intentar resolver una instancia más difícil de algún problema, con frecuencia comenzamos con una expresión que resuelve una versión más sencilla. Para la relación de recurrencia de primer orden (5.2.6), vimos en el ejemplo 5.2.2 que la solución era de la forma

$$S_n = t^n;$$

así, para nuestro primer intento de determinar una solución de la relación de recurrencia de segundo orden (5.2.9), buscaremos una solución de la forma $V_n = t^n$.

Si $V_n = t^n$ fuese solución de (5.2.9), debemos tener

$$V_n = 5V_{n-1} - 6V_{n-2}$$

o

$$t^n = 5t^{n-1} - 6t^{n-2}$$

o

$$t^n - 5t^{n-1} + 6t^{n-2} = 0.$$

Al dividir entre t^{n-2}, obtenemos la ecuación equivalente

$$t^2 - 5t + 6 = 0. \tag{5.2.11}$$

Al resolver (5.2.11), tenemos las soluciones

$$t = 2, \qquad t = 3.$$

En este momento, tenemos dos soluciones S y T de (5.2.9), dadas por

$$S_n = 2^n, \qquad T_n = 3^n. \tag{5.2.12}$$

Podemos verificar (véase el teorema 5.2.11) que si S y T son soluciones de (5.2.9), entonces $bS + dT$, donde b y d son números arbitrarios, también es una solución de (5.2.9). En nuestro caso, si definimos la sucesión U mediante la ecuación

$$U_n = bS_n + dT_n$$
$$= b2^n + d3^n,$$

Los cambios del precio con respecto del tiempo se pueden observar en una gráfica. Si el precio inicial es p_0, el fabricante estará dispuesto a ofrecer la cantidad q_1, en el instante $n = 1$. Localizamos esta cantidad moviéndonos de manera horizontal hasta la curva de oferta (véase la figura 5.1.5). Sin embargo, las fuerzas del mercado obligan al precio a bajar hasta p_1, como podemos ver al movernos de manera vertical hasta la curva de demanda. En el precio p_1, el fabricante estará dispuesto a ofrecer la cantidad q_2 en el instante $n = 2$, como podemos ver moviéndonos en forma horizontal hasta la curva de oferta. Ahora, las fuerzas del mercado obligan al precio a subir hasta p_2, como podemos ver al movernos en forma vertical hasta la curva de demanda. Al continuar este proceso, obtenemos la "telaraña" que aparece en la figura 5.1.5.

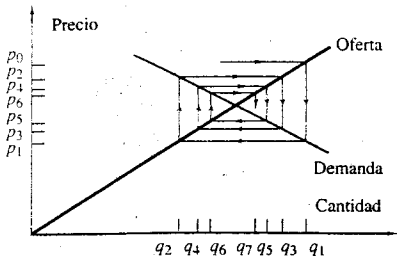


FIGURA 5.1.5 Una telaraña con un precio estabilizante.

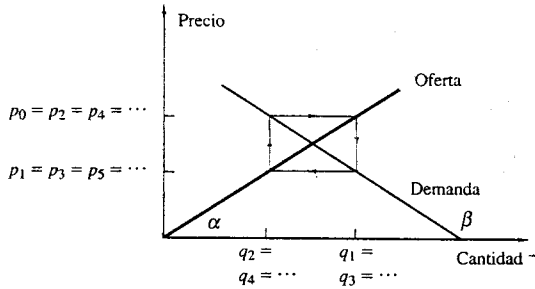


FIGURA 5.1.6 Una telaraña con un precio fluctuante.

Para las funciones de oferta y de demanda de la figura 5.1.5, el precio tiende al precio dado por la intersección de las curvas de oferta y de demanda. Sin embargo, esto no siempre ocurre. Por ejemplo, en la figura 5.1.6, el precio fluctúa entre p_0 y p_1, mientras que en la figura 5.1.7, el precio comienza a oscilar en forma cada vez más pronunciada. El comportamiento queda determinado por las pendientes de las rectas de oferta y de demanda. Para producir el comportamiento fluctuante de la figura 5.1.6, los ángulos α y β deben sumar 180°. Las pendientes de las curvas de oferta y de demanda son $\tan \alpha$ y $\tan \beta$, respectivamente, de modo que en la figura 5.1.6 tenemos

$$k = \tan \alpha = -\tan \beta = b.$$

Hemos mostrado que el precio fluctúa entre dos valores cuando $k = b$. Un análisis similar muestra que el precio tiende al dado por la intersección de las curvas de oferta y de demanda (figura 5.1.5) cuando $b < k$ y el caso del precio oscilante y creciente (figura 5.1.7)

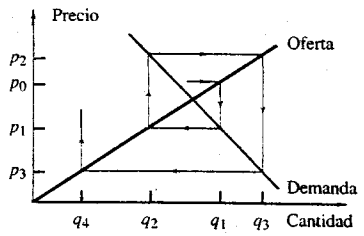


FIGURA 5.1.7 Una telaraña con oscilaciones crecientes del precio.

ocurre cuando $b > k$ (véanse los ejercicios 35 y 36). En la sección 5.2 analizaremos el comportamiento del precio con respecto del tiempo, mediante una fórmula explícita para el precio p_n. □

La definición de relación de recurrencia se puede ampliar para incluir funciones con índices dados por n-adas de enteros positivos. Nuestro último ejemplo es de esta forma.

EJEMPLO 5.1.10 ***Función de Ackermann***

La función de Ackermann se puede definir mediante las relaciones de recurrencia

$$A(m, 0) = A(m-1, 1), \qquad m = 1, 2, \ldots, \tag{5.1.11}$$

$$A(m, n) = A(m-1, A(m, n-1)), \qquad m = 1, 2, \ldots, \quad n = 1, 2, \ldots, \tag{5.1.12}$$

y las condiciones iniciales

$$A(0, n) = n + 1, \qquad n = 0, 1, \ldots. \tag{5.1.13}$$

La función de Ackermann tiene importancia teórica debido a su rápida tasa de crecimiento. Las funciones relacionadas con la función de Ackermann aparecen en la complejidad del tiempo de ciertos algoritmos, como el tiempo para ejecutar algoritmos de unión y búsqueda (véase [Tarjan, páginas 22-29]).

El cálculo

$$\begin{aligned} A(1, 1) &= A(0, A(1, 0)) && \text{por (5.1.12)} \\ &= A(0, A(0, 1)) && \text{por (5.1.11)} \\ &= A(0, 2) && \text{por (5.1.13)} \\ &= 3 && \text{por (5.1.13)} \end{aligned}$$

ilustra el uso de las ecuaciones (5.1.11)-(5.1.13). □

Ejercicios

En los ejercicios 1-3, determine una relación de recurrencia y las condiciones iniciales que generan una sucesión que comience con los términos dados.

1. 3, 7, 11, 15, ...
2. 3, 6, 9, 15, 24, 39, ...
3. 1, 1, 2, 4, 16, 128, 4096, ...

En los ejercicios 4-8, suponga que una persona invierte $2000 al 14% compuesto anualmente. Sea A_n la cantidad al final de n años.

4. Determine una relación de recurrencia para la sucesión $A_0, A_1, \ldots$.
5. Determine una condición inicial para la sucesión $A_0, A_1, \ldots$.
6. Determine A_1, A_2 y A_3.
7. Determine una fórmula explícita para A_n.
8. ¿Cuánto tiempo tardará una persona en duplicar la inversión inicial?

Si una persona invierte en una anualidad protegida contra impuestos, la cantidad invertida, al igual que los intereses devengados, no están sujetos a impuestos hasta ser retirados de la cuenta. En los ejercicios 9-12, suponga que una persona invierte $2000 cada año en una anualidad protegida contra impuestos, a 10% compuesto anualmente. Sea A_n la cantidad al final de n años.

9. Determine una relación de recurrencia para la sucesión $A_0, A_1, \ldots$.
10. Determine una condición inicial para la sucesión $A_0, A_1, \ldots$.
11. Determine A_1, A_2 y A_3.
12. Determine una fórmula explícita para A_n.

En los ejercicios 13-17, suponga que una persona invierte $3000 a 12% de interés anual compuesto en forma trimestral. Sea A_n la cantidad al final de n años.

13. Determine una relación de recurrencia para la sucesión $A_0, A_1, \ldots$.
14. Determine una condición inicial para la sucesión $A_0, A_1, \ldots$.
15. Determine A_1, A_2 y A_3.
16. Determine una fórmula explícita para A_n.
17. ¿Cuánto tiempo tardará una persona en duplicar la inversión inicial?
18. Sea S_n el número de cadenas de n bits que no contienen al patrón 000. Determine una relación de recurrencia y condiciones iniciales para la sucesión $\{S_n\}$.

Los ejercicios 19-21 se refieren a la sucesión S, donde S_n denota el número de cadenas de n bits que no contienen al patrón 00.

19. Determine una relación de recurrencia y condiciones iniciales para la sucesión $\{S_n\}$.
20. Muestre que $S_n = f_{n+1}$, $n = 1, 2, \ldots$, donde f denota la sucesión de Fibonacci.
21. Considere el número de cadenas de n bits con exactamente i ceros y el ejercicio 20 para mostrar que

$$f_{n+1} = \sum_{i=0}^{\lfloor (n+1)/2 \rfloor} C(n+1-i,\ i), \qquad n = 1,\ 2,\ \ldots,$$

donde f denota la sucesión de Fibonacci.

Los ejercicios 22-24 se refieren a la sucesión $S_1, S_2, \ldots$, donde S_n denota el número de cadenas de n bits que no contienen al patrón 010.

22. Calcule S_1, S_2, S_3 y S_4.
23. Considere el número de cadenas de n bits que no contienen el patrón 010 y que no comienzan con 0; aquellas que comienzan con un único 0 (es decir, que comienzan con 01), aquellas que comienzan con 0, y así sucesivamente, para deducir la relación de recurrencia

$$S_n = S_{n-1} + S_{n-3} + S_{n-4} + S_{n-5} + \cdots + S_1 + 3. \tag{5.1.14}$$

24. Reemplace n con $n-1$ en (5.1.14) y escriba una fórmula para S_{n-1}. Reste la fórmula para S_{n-1} de la fórmula para S_n y utilice el resultado para deducir la relación de recurrencia

$$S_n = 2S_{n-1} - S_{n-2} + S_{n-3}.$$

En los ejercicios 25-30, $C_0, C_1, C_2, \ldots$ denota la sucesión de números de Catalan.

25. Dado que $C_0 = C_1 = 1$ y $C_2 = 2$, calcule C_3, C_4 y C_5 utilizando la relación de recurrencia del ejemplo 5.1.7.

26. Muestre que los números de Catalan están dados por la relación de recurrencia

$$(n+2)\,C_{n+1} = (4n+2)\,C_n, \quad n \geq 0$$

y la condición inicial $C_0 = 1$.

27. Demuestre que

$$C_n \geq \frac{4^{n-1}}{n^2} \qquad \text{para toda } n \geq 1.$$

28. Deduzca una relación de recurrencia y una condición inicial para el número de formas de colocar paréntesis en el producto

$$a_1 * a_2 * \cdots * a_n, \quad n \geq 2.$$

Ejemplos: Existe una forma de colocar paréntesis en $a_1 * a_2$, a saber, $(a_1 * a_2)$. Existen dos formas de colocar paréntesis a $a_1 * a_2 * a_3$, a saber, $((a_1 * a_2) * a_3)$ y $(a_1 * (a_2 * a_3))$. Deduzca que el número de formas de colocar paréntesis en el producto de n elementos es C_{n-1}, $n \geq 2$.

★ 29. Éste es el problema analizado originalmente por Catalan.

Deduzca una relación de recurrencia y una condición inicial para el número de formas de dividir un polígono convexo de $(n+2)$ lados, $n \geq 1$, en triángulos, trazando $n-1$ líneas a través de los vértices y de modo que no se intersequen en el interior del polígono. (Un polígono es convexo si cualquier recta que una dos puntos del polígono está completamente dentro del polígono.) Por ejemplo, existen cinco formas de dividir un pentágono convexo en triángulos, trazando dos rectas ajenas a través de los vértices:

Deduzca que el número de formas de dividir un polígono convexo de $(n+2)$ lados en triángulos trazando $n-1$ rectas ajenas a través de los vértices es C_n, $n \geq 1$.

30. Considere las rutas desde la esquina inferior izquierda hasta la esquina superior derecha en una retícula $(n+1) \times (n+1)$, donde sólo se puede ir hacia la derecha o hacia arriba; separe dichas rutas en clases con base en el momento en que, después de salir de la esquina inferior izquierda, la ruta toca por vez primera la diagonal que va de la esquina inferior izquierda a la esquina superior derecha y deduzca la relación de recurrencia

$$C_n = \frac{1}{2}C(2(n+1), n+1) - \sum_{k=0}^{n-1} C_k C(2(n-k), n-k).$$

En los ejercicios 31 y 32, sea S_n el número de rutas desde la esquina inferior izquierda hasta la esquina superior derecha en una retícula $n \times n$, donde sólo se puede ir hacia la derecha, hacia arriba o en diagonal hacia el noreste [es decir, de (i, j) hasta $(i+1, j+1)$] y en donde se permite tocar pero no rebasar la diagonal que va de la esquina inferior izquierda a la esquina superior derecha. Los números $S_0, S_1, \ldots$ se llaman *números de Schröder*.

31. Muestre que $S_0 = 1$, $S_1 = 2$, $S_2 = 6$ y $S_3 = 22$.
32. Deduzca una relación de recurrencia para la sucesión de números de Schröder.
33. Escriba las soluciones explícitas del juego de la Torre de Hanoi para $n = 3, 4$.
34. ¿A qué valores tienden el precio y la cantidad en el ejemplo 5.1.9 cuando $b < k$?
35. Muestre que cuando $b < k$ en el ejemplo 5.1.9, el precio tiende al dado por la intersección de las curvas de oferta y de demanda.
36. Muestre que cuando $b > k$ en el ejemplo 5.1.9, las diferencias entre los precios sucesivos aumentan.

Los ejercicios 37-43 se refieren a la función de Ackermann $A(m, n)$.

37. Calcule $A(2, 2)$ y $A(2, 3)$.

38. Utilice la inducción para mostrar que

$$A(1, n) = n + 2, \quad n = 0, 1, \ldots.$$

39. Utilice la inducción para mostrar que

$$A(2, n) = 3 + 2n, \quad n = 0, 1 \ldots.$$

40. Haga una conjetura acerca de una fórmula para $A(3, n)$ y demuéstrela por inducción.

☆ **41.** Demuestre que $A(m, n) > n$ para toda $m \geq 0$, $n \geq 0$, por inducción sobre m. El paso inductivo utilizará inducción sobre n.

42. Utilice el ejercicio 41 o algún otro recurso, para demostrar que $A(m, n) > 1$ para toda $m \geq 1, n \geq 0$.

43. Utilice el ejercicio 41 o algún otro recurso, para demostrar que $A(m, n) < A(m, n + 1)$ para toda $m \geq 0, n \geq 0$.

Lo que hemos llamado función de Ackermann en realidad se deduce de la función original de Ackermann dada por

$$\begin{aligned} AO(0, y, z) &= z + 1, & y, z &\geq 0 \\ AO(1, y, z) &= y + z, & y, z &\geq 0 \\ AO(2, y, z) &= yz, & y, z &\geq 0 \\ AO(x + 3, y, 0) &= 1, & x, y &\geq 0 \\ AO(x + 3, y, z + 1) &= AO(x + 2, y, AO(x + 3, y, z)), & x, y, z &\geq 0. \end{aligned}$$

Los ejercicios 44-47 se refieren a la función AO y a la función de Ackermann A.

44. Muestre que $A(x, y) = AO(x, 2, y + 3) - 3$ para $y \geq 0$ y $x = 0, 1, 2$.

45. Muestre que $AO(x, 2, 1) = 2$ para $x \geq 2$.

46. Muestre que $AO(x, 2, 2) = 4$ para $x \geq 2$.

★ **47.** Muestre que $A(x, y) = AO(x, 2, y + 3) - 3$ para $x, y \geq 0$.

48. Una red consta de n nodos. Cada nodo tiene cierta capacidad de comunicación y de almacenamiento local. En forma periódica, hay que compartir todos los archivos. Un *enlace* consta de dos nodos que comparten archivos. En forma específica, al enlazar los nodos A y B, A transmite todos sus archivos a B y viceversa. Sólo existe un enlace a la vez, y después de establecer un enlace y compartir los archivos, el enlace se elimina. Sea a_n el número mínimo de enlaces necesarios para n nodos, de modo que todos los archivos sean conocidos por todos los nodos.
(a) Muestre que $a_2 = 1, a_3 \leq 3, a_4 \leq 4$.
(b) Muestre que $a_n \leq a_{n-1} + 2, n \geq 3$.

49. Si P_n denota el número de permutaciones de n objetos distintos, determine una relación de recurrencia y una condición inicial para la sucesión $P_1, P_2, \ldots$.

50. Suponga que tenemos n dólares y que cada día compramos jugo de naranja ($1), leche ($2), o cerveza ($2). Si R_n es el número de formas de gastar el dinero, muestre que

$$R_n = R_{n-1} + 2R_{n-2}.$$

El orden se toma en cuenta. Por ejemplo, existen 12 formas de gastar cuatro dólares: *LC, CL, JJL, JJC, JLJ, JCJ, LJJ, CJJ, JJJJ, LL, CC.*

51. Suponga que tenemos n dólares y que cada día compramos cintas ($1), papel ($1), plumas ($2), lápices ($2) o carpetas ($3). Si R_n es el número de formas de gastar todo el dinero, deduzca una relación de recurrencia para la sucesión $R_1, R_2, \ldots$.

52. Sea R_n el número de regiones en que queda dividido el plano mediante n rectas. Suponga que cada par de rectas se interseca en un punto, pero que no existen tres rectas que se intersequen en un punto. Deduzca una relación de recurrencia para la sucesión $R_1, R_2, \ldots$.

Los ejercicios 53 y 54 se refieren a la sucesión S_n definida como

$$S_1 = 0, \quad S_2 = 1, \quad S_n = \frac{S_{n-1} + S_{n-2}}{2}, \quad n = 3, 4, \ldots.$$

53. Calcule S_3 y S_4.

☆ **54.** Haga una conjetura acerca de una fórmula para S_n y muestre que es correcta utilizando inducción.

★ **55.** Sea F_n el número de funciones f de $X = \{1, \ldots, n\}$ en X con la propiedad de que si i está en el rango de f, entonces $1, 2, \ldots, i - 1$ también están en el rango de f. (Haga $F_0 = 1$.) Muestre que la sucesión $F_0, F_1, \ldots$ satisface la relación de recurrencia

$$F_n = \sum_{j=0}^{n-1} C(n, j)F_j.$$

56. Si α es una cadena de bits, sea $C(\alpha)$ el máximo número de ceros consecutivos en α. [*Ejemplos*: $C(10010) = 2$, $C(00110001) = 3$.] Sea S_n el número de cadenas de n bits α con $C(\alpha) \leq 2$. Desarrolle una relación de recurrencia para $S_1, S_2, \ldots$.

57. Deduzca una relación de recurrencia para $C(n, k)$, el número de subconjuntos con k elementos de un conjunto con n elementos. Específicamente, escriba $C(n + 1, k)$ en términos de $C(n, i)$ para i adecuada.

58. Deduzca una relación de recurrencia para $S(k, n)$, el número de formas de elegir k elementos, permitiendo repeticiones, de n tipos disponibles. Específicamente, escriba $S(k, n)$ en términos de $S(k - 1, i)$ para i adecuada.

59. Sea $S(n, k)$ el número de funciones de $\{1, \ldots, n\}$ sobre $\{1, \ldots, k\}$. Muestre que $S(n, k)$ satisface la relación de recurrencia

$$S(n, k) = k^n - \sum_{i=1}^{k-1} C(k, i)S(n, i).$$

60. La *sucesión de Lucas* $L_1, L_2, \ldots$ (la cual recibe el nombre de Édouard Lucas, el inventor del juego de la Torre de Hanoi) se define mediante la relación de recurrencia

$$L_n = L_{n-1} + L_{n-2}, \quad n \geq 3,$$

y las condiciones iniciales

$$L_1 = 1, \qquad L_2 = 3.$$

(a) Determine los valores de L_3, L_4 y L_5.
(b) Muestre que

$$L_{n+2} = f_n + f_{n+2}, \quad n \geq 1,$$

donde $f_1, f_2, \ldots$ denota la sucesión de Fibonacci.

61. Establezca la relación de recurrencia

$$s_{n+1,k} = s_{n,k-1} + ns_{n,k}$$

para los números de Stirling del primer tipo (véase el ejercicio 74 de la sección 4.2).

62. Establezca la relación de recurrencia

$$S_{n+1,k} = S_{n,k-1} + kS_{n,k}$$

para los números de Stirling del segundo tipo (véase el ejercicio 75 de la sección 4.2).

Bueno, me fui por el camino, hacia el norte, a Providence. Para encontrarme con el alcalde, ¡el alcalde de Providence! Él estaba sentado en el lobby del hotel. ¿Qué dijo? Él dijo: "¡Hola!" Y le dije: "Oye, alcalde, está muy bonita tu ciudad". Y entonces se tomó un café conmigo. Luego fui para Waterbury. Waterbury es una ciudad bella. Una ciudad con un reloj enorme, el famoso reloj de Waterbury. Hice buenos negocios ahí. Y luego a Boston (Boston es la cuna de la revolución.) Bonita ciudad. Luego algunos otros lugares de Massachusetts, de ahí a Portland, a Bangor y ¡directo a casa!

– de Death of a Salesman

6

TEORÍA DE GRÁFICAS *

6.1 INTRODUCCIÓN
6.2 CAMINOS Y CICLOS
RINCÓN DE SOLUCIÓN DE PROBLEMAS: GRÁFICAS
6.3 CICLOS HAMILTONIANOS Y EL PROBLEMA DEL AGENTE DE VENTAS VIAJERO
6.4 UN ALGORITMO PARA LA RUTA MÁS CORTA
6.5 REPRESENTACIONES DE GRÁFICAS
6.6 ISOMORFISMOS DE GRÁFICAS
6.7 GRÁFICAS PLANAS
† 6.8 LOCURA INSTANTÁNEA
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Aunque el primer artículo en teoría de gráficas apareció en 1736 (véase el ejemplo 6.2.16) y varios resultados importantes en la teoría de gráficas se obtuvieron en el siglo XIX, sólo fue a partir de la década de 1920 que hubo un interés sostenido, amplio e intenso en la teoría de gráficas. De hecho, el primer texto sobre teoría de gráficas ([König]) apareció en 1936. Sin duda, una de las razones de este reciente interés en la teoría de gráficas es su capacidad de aplicación en campos muy diversos, incluyendo las ciencias de la computación, la química, la investigación de operaciones, la ingeniería eléctrica, la lingüística y la economía.

Comenzaremos con cierta terminología básica y ejemplos de gráficas. Luego analizaremos algunos conceptos importantes en la teoría de gráficas, incluyendo los caminos y los ciclos. Presentamos un algoritmo para determinar el camino (o ruta) más corto, con el cual se obtiene la trayectoria más breve entre dos puntos dados. Luego consideraremos dos problemas clásicos de gráficas, la existencia de ciclos hamiltonianos y el problema del agente viajero. Después de dar opciones para la representación de las gráficas, estudiaremos la cuestión de cuándo dos gráficas son esencialmente iguales (es decir, cuándo dos gráficas son isomorfas) y cuándo una gráfica se puede trazar en el plano sin que se crucen sus aristas. Concluimos presentando una solución con base en un modelo de gráfica para el juego Locura instantánea.

* Aunque en algunos libros se ha traducido la palabra *graph* como grafo, es preferible emplear el vocablo castizo gráfica. (Nota del traductor.)

† Esta sección se puede omitir sin pérdida de continuidad.

6.1 *INTRODUCCIÓN*

La figura 6.1.1 muestra el sistema de carreteras en Wyoming, el cual debe ser inspeccionado por una persona. En específico, este inspector de carreteras debe recorrer cada uno de estos caminos y crear un archivo con información acerca de las condiciones de cada camino, la visibilidad de las líneas en los caminos, el estado de las señales de tráfico, etc. Como el inspector de carreteras vive en Greybull, la forma más económica de inspeccionar todos los caminos será comenzar en Greybull, recorrer todos los caminos exactamente una vez, y regresar a Greybull. ¿Es esto posible? Vea si puede decidir antes de continuar.

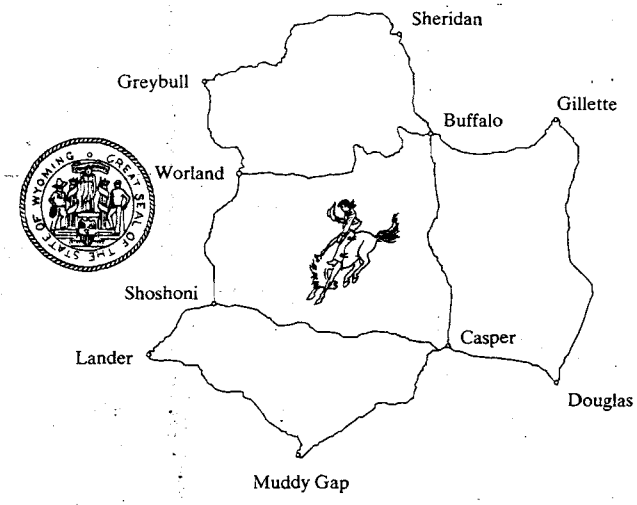


FIGURA 6.1.1 Parte del sistema de carreteras de Wyoming.

El problema se puede modelar mediante una **gráfica**. De hecho, como las gráficas se trazan con puntos y líneas, parecen como mapas de carreteras. En la figura 6.1.2 hemos trazado una gráfica G que modela el mapa de la figura 6.1.1. Los puntos de la figura 6.1.2 se llaman **vértices** y las líneas que unen los vértices se llaman **aristas**. (Posteriormente, en esta sección definiremos todos estos términos con cuidado.) Hemos etiquetado cada vértice con las tres primeras letras de la ciudad a la cual corresponde y las aristas como $e_1, \ldots, e_{13}$. Al trazar una gráfica, la única información importante es saber cuáles vértices están unidos

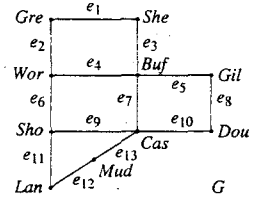


FIGURA 6.1.2
Un modelo de gráfica del sistema de carreteras de la figura 6.1.1.

Sección 6.5

Matriz de adyacencia
Si A es la matriz de adyacencia de una gráfica simple, la entrada ij-ésima de A^n es igual al número de caminos de longitud n del vértice i al vértice j.
Matriz de incidencia

Sección 6.6

Las gráficas $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$ son isomorfas si existen funciones uno a uno y sobre $f: V_1 \to V_2$ y $g: E_1 \to E_2$ tales que $e \in E_1$ es incidente en v y $w \in V_1$ si y sólo si $g(e)$ es incidente en $f(v)$ y $f(w)$.
Las gráficas G_1 y G_2 son isomorfas si y sólo si, para algún orden de sus vértices y aristas, sus matrices de incidencia son idénticas.
Modelo de red para la computación paralela
Invariante

Sección 6.7

Gráfica plana
Cara
Ecuación de Euler para gráficas planas conexas: $f = e - v + 2$
Aristas en serie
Reducción de serie
Gráficas homeomorfas
Teorema de Kuratowski: Una gráfica es plana si y sólo si no contiene una subgráfica homeomorfa a K_5 o $K_{3,3}$.

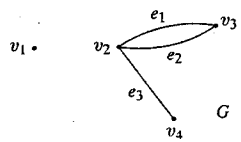
Sección 6.8

Locura Instantánea
Cómo resolver un juego de Locura Instantánea

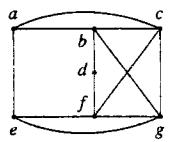
AUTOEVALUACIÓN DEL CAPÍTULO

Sección 6.1

1. Para la gráfica $G = (V, E)$, determine V, E, todas las aristas paralelas, todos los lazos, todos los vértices aislados, e indique si G es una gráfica simple. Además, indique los vértices sobre los cuales incide la arista e_3 y las aristas sobre las cuales incide el vértice v_2.



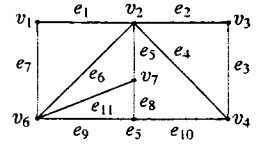
2. Explique por qué la gráfica no tiene un camino de a a a que pase por cada arista exactamente una vez.



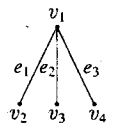
3. Trace $K_{2,5}$ la gráfica bipartita completa con 2 y 5 vértices.
4. Demuestre que el n-cubo es bipartita para toda $n \geq 1$.

Sección 6.2

5. Indique si el camino $(v_2, v_3, v_4, v_2, v_6, v_1, v_2)$ en la gráfica es un camino simple, un ciclo, un ciclo simple, o ninguno de los anteriores.



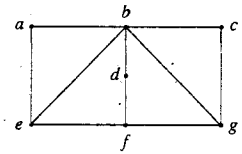
6. Trace todas las subgráficas de la gráfica siguiente que tengan exactamente dos aristas.



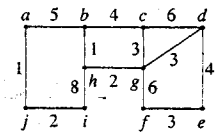
7. Determine una subgráfica conexa de la gráfica del ejercicio 5 que contenga todos los vértices de la gráfica original y tenga el menor número posible de aristas.
8. ¿Contiene la gráfica del ejercicio 5 un ciclo de Euler? Explique.

Sección 6.3

9. Determine un ciclo hamiltoniano para la gráfica del ejercicio 5.
10. Determine un ciclo hamiltoniano en el 3-cubo.
11. Muestre que la gráfica siguiente no tiene un ciclo hamiltoniano.

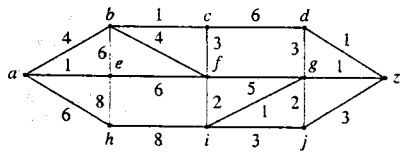


12. Muestre que el ciclo $(a, b, c, d, e, f, g, h, i, j, a)$ proporciona una solución al problema del agente viajero para la siguiente gráfica.



Sección 6.4

Los ejercicios 13-16 se refieren a la siguiente gráfica.



13. Determine la longitud de una ruta más corta de a a i.
14. Determine la longitud de una ruta más corta de a a z.
15. Determine una ruta más corta de a a z.
16. Determine la longitud de una ruta más corta de a a z que pase por c.

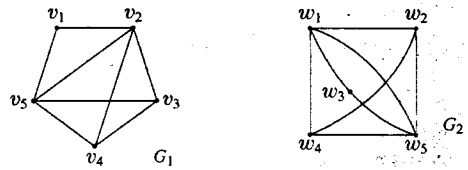
Sección 6.5

17. Escriba la matriz de adyacencia de la gráfica del ejercicio 5.
18. Escriba la matriz de incidencia de la gráfica del ejercicio 5.
19. Si A es la matriz de adyacencia de la gráfica del ejercicio 5, ¿qué representa la entrada del renglón v_2 y la columna v_3 de A^3?
20. ¿Puede una columna de una matriz de incidencia constar sólo de ceros? Explique.

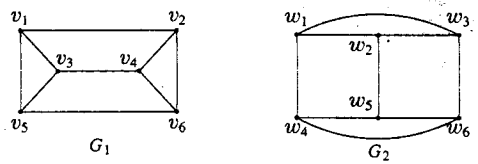
Sección 6.6

En los ejercicios 21 y 22, determine si las gráficas G_1 y G_2 son isomorfas. Si las gráficas son isomorfas, proporcione un orden para los vértices que produzca matrices de adyacencia iguales. Si las gráficas no son isomorfas, proporcione un invariante no compartido por las gráficas.

21.



22.

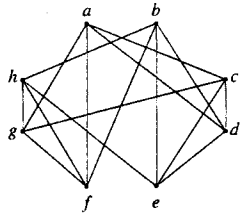


23. Trace todas las gráficas simples, no isomorfas, que tengan exactamente cinco vértices y dos aristas.
24. Trace todas las gráficas simples, no isomorfas, que tengan exactamente cinco vértices, dos componentes, sin ciclos.

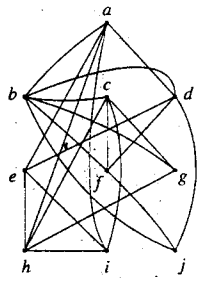
Sección 6.7

En los ejercicios 25 y 26, determine si la gráfica es plana. Si la gráfica es plana, vuelva a trazarla de modo que las aristas no se crucen; en caso contrario, determine una subgráfica homeomorfa a K_5 o $K_{3,3}$.

25.



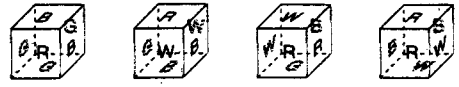
26.



27. Muestre que cualquier gráfica simple, conexa, con 31 aristas y 12 vértices no es plana.
28. Muestre que el n-cubo es plano si $n \leq 3$ y no es plano si $n > 3$.

Sección 6.8

29. Represente el siguiente juego Locura Instantánea mediante una gráfica.



1 2 3 4

30. Determine una solución al juego del ejercicio 29.
31. Determine todas las subgráficas de la gráfica del ejercicio 29 que satisfagan las propiedades (6.8.1) y (6.8.2).
32. Utilice el ejercicio 31 para determinar el número de soluciones del juego del ejercicio 29.

†6.8 *LOCURA INSTANTÁNEA*

Locura Instantánea es un juego formado por cuatro cubos, cada una de cuyas caras está pintada de uno de cuatro colores, rojo, blanco, azul o verde (R, W, B, G; véase la figura 6.8.1). (Existen varias versiones del juego, dependiendo de cuáles caras están pintadas con cuáles colores.) El problema consiste en apilar los cubos, uno sobre el otro, de modo que uno vea los cuatro colores desde el frente, por detrás, por la izquierda o por la derecha (véase la figura 6.8.2). Como existen 331,776 formas diferentes de apilar los cubos (véase el ejercicio 12), no es práctica una solución a mano mediante prueba y error. Presentamos una solución, mediante un modelo de gráfica, que permite descubrir una solución, si existe, ¡en unos cuantos minutos!

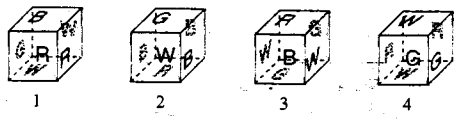


FIGURA 6.8.1 Un juego de Locura Instantánea.

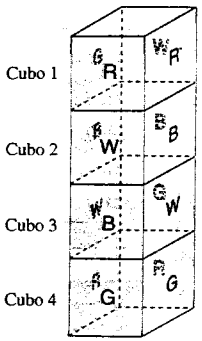
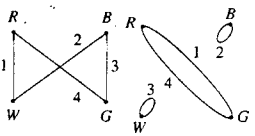


FIGURA 6.8.2
Una solución al juego de Locura Instantánea de la figura 6.8.1.

En primer lugar, observemos que cualquier forma particular de apilar los cubos se puede representar mediante dos gráficas, una de las cuales muestra los colores del frente y por detrás, y la otra exhibe los colores de la izquierda y la derecha. Por ejemplo, en la figura 6.8.3, hemos representado la forma de apilar los cubos de la figura 6.8.2. Los vértices representan los colores, y una arista une dos vértices si las caras opuestas tienen dichos colores. Por ejemplo, en la gráfica frente/atrás, la arista etiquetada 1 une *R* con *W*, pues las caras del frente y la parte posterior del cubo 1 son rojas y blancas. Como otro ejemplo, en la gráfica izquierda/derecha, *W* tiene un lazo, pues las caras izquierda y derecha del cubo 3 son blancas.



(a) Frente/vuelta (b) Izquierda/derecha

FIGURA 6.8.3
Gráficas que representan las formas de apilar los cubos de la figura 6.8.2.

También podemos apilar los cubos a partir de un par de gráficas como las de la figura 6.8.3, la cual representa una solución al juego Locura Instantánea. Comencemos con la gráfica frente/detrás. El cubo 1 debe tener las caras opuestas de color rojo y blanco. Asignamos de manera arbitraria uno de estos colores, digamos el rojo, al frente. Entonces el cubo 1 tiene la cara posterior de color blanco. La otra arista incidente en *W* es 2, así que hacemos que la cara frontal del cubo 2 sea blanca. Esto hace que la cara posterior del cubo 2 sea azul. La otra arista incidente en *B* es 3, de modo que hacemos que la cara frontal del cubo 3 sea azul. Esto hace que la cara posterior del cubo 3 sea verde. La otra arista incidente en *G* es 4. El cubo 4 tiene entonces una cara frontal verde y una cara posterior roja. Ahora están asignados correctamente los colores frontal y posterior. En este momento, las caras izquierda y derecha están ordenadas de manera aleatoria; sin embargo, mostraremos la forma de orientar correctamente las caras izquierda y derecha sin alterar los colores de las caras frontal y posterior.

El cubo 1 debe tener los colores rojo y verde opuestos en las caras izquierda y derecha. Asignamos de manera arbitraria uno de estos colores, digamos el verde, a la izquierda. Entonces el cubo 1 tiene una cara derecha de color rojo. Observe que al rotar el cubo, podemos obtener esta orientación izquierda/derecha sin modificar los colores frontal y posterior (véase la figura 6.8.4). De manera análoga, podemos orientar los cubos 2, 3 y 4. Observe que los cubos 2 y 3 tienen los mismos colores en los lados opuestos. Así, hemos reconstruido la pila de la figura 6.8.2.

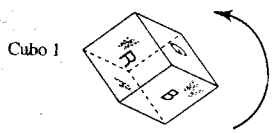


FIGURA 6.8.4
La rotación de un cubo para obtener una orientación izquierda/derecha, sin modificar los colores frontal/posterior.

El análisis anterior muestra que podemos obtener una solución del juego Locura Instantánea si podemos determinar dos gráficas como las de la figura 6.8.3. Las propiedades necesarias son

† Esta sección se puede omitir sin pérdida de continuidad.

- Cada vértice debe tener grado 2. (6.8.1)
- Cada cubo debe representarse mediante una arista exactamente una vez en cada gráfica. (6.8.2)
- Las dos gráficas no deben tener aristas en común. (6.8.3)

La propiedad (6.8.1) garantiza que cada color se puede utilizar dos veces, una en el frente (o izquierda) y una vez en la parte posterior (o derecha). La propiedad (6.8.2) garantiza que cada cubo se utiliza exactamente una vez. La propiedad (6.8.3) garantiza que, después de orientar los lados frontal y posterior, podemos orientar con éxito los lados izquierdo y derecho.

Para obtener una solución, primero trazamos una gráfica *G* que representa todas las caras de todos los cubos. Los vértices de *G* representan los cuatro colores y una arista con etiqueta *i* conecta dos vértices (colores) si las caras opuestas del cubo *i* tienen dichos colores. En la figura 6.8.5 hemos trazado la gráfica que representa los cubos de la figura 6.8.1. Entonces, por inspección, determinamos dos subgráficas de *G* que satisfacen las propiedades (6.8.1)-(6.8.3). El lector debe utilizar este método para encontrar otra solución del juego representado por la figura 6.8.5.

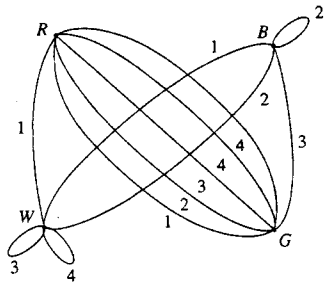


FIGURA 6.8.5 Una representación mediante una gráfica del juego Locura Instantánea de la figura 6.8.1.

EJEMPLO 6.8.1

Determine una solución del juego Locura Instantánea de la figura 6.8.6.

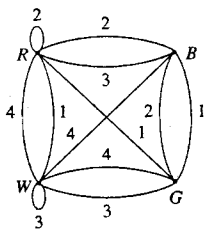


FIGURA 6.8.6 El juego Locura Instantánea del ejemplo 6.8.1.

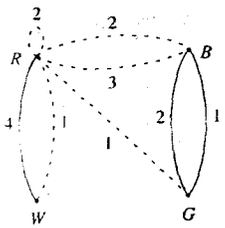


FIGURA 6.8.7
Intento por determinar una subgráfica de la figura 6.8.6 que satisfaga (6.8.1) y (6.8.2).

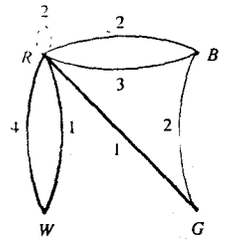


FIGURA 6.8.8
Otro intento por determinar una subgráfica de la figura 6.8.6 que satisfaga (6.8.1) y (6.8.2).

Primero intentamos construir una subgráfica con las propiedades (6.8.1) y (6.8.2). Elegimos de manera arbitraria un vértice, digamos B, y elegimos dos aristas incidentes en el vértice B. Suponga que hemos elegido las dos aristas que aparecen como líneas continuas en la figura 6.8.7. Ahora consideramos el problema de elegir dos aristas incidentes en el vértice R. Ninguna de las aristas incidentes en B o G puede elegirse, pues B y G deben tener grado 2 cada uno. Como cada cubo debe aparecer en cada subgráfica exactamente una vez, no podemos elegir las aristas etiquetadas con 1 o 2, pues ya hemos elegido aristas con estas etiquetas. Las aristas incidentes en R que no se pueden elegir aparecen como líneas punteadas en la figura 6.8.7. Esto sólo deja la arista etiquetada 4. Como necesitamos dos aristas incidentes en R, nuestra selección inicial de aristas incidentes en B debe revisarse.

Para nuestro siguiente intento por elegir dos aristas incidentes en el vértice B, elegimos las aristas etiquetadas 2 y 3, como muestra la figura 6.8.8. Como esta opción incluye una arista incidente en R, debemos elegir una arista adicional incidente en R. Tenemos tres posibilidades para elegir la arista adicional (que aparece remarcada en la figura 6.8.8). (El lazo incidente en R cuenta como dos aristas y por tanto no se puede elegir.) Si elegimos la arista 1 incidente en R y G, necesitaríamos un lazo en W etiquetado 4. Como no existe tal lazo, no elegimos esta arista. Si elegimos la arista etiquetada 1 incidente en R y W, entonces podemos elegir la arista etiquetada 4 incidente en W y G (véase la figura 6.8.9). Hemos obtenido una de las gráficas.

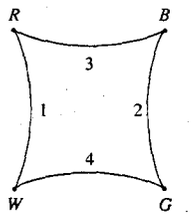


FIGURA 6.8.9
Una subgráfica de la figura 6.8.6 que satisface (6.8.1) y (6.8.2).

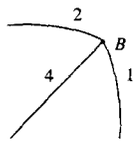


FIGURA 6.8.10
Aristas incidentes en B no utilizadas en la figura 6.8.9.

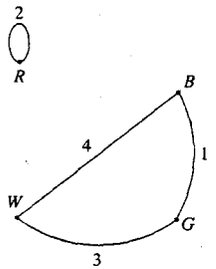


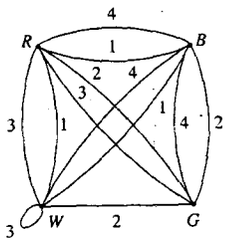
FIGURA 6.8.11 Otra subgráfica de la figura 6.8.6, sin aristas en común con la figura 6.8.9, que satisface (6.8.1) y (6.8.2). Esta gráfica y la de la figura 6.8.9 resuelven el juego Locura Instantánea de la figura 6.8.6.

Ahora buscamos una segunda gráfica sin aristas comunes con la gráfica recién determinada. De nuevo comenzamos eligiendo dos aristas incidentes en B. Como no podemos reutilizar las aristas, nuestras opciones quedan limitadas a tres aristas (véase la figura 6.8.10). Al elegir las aristas etiquetadas 1 y 4 obtenemos la gráfica de la figura 6.8.11. Las gráficas de las figuras 6.8.9 y 6.8.11 resuelven el juego Locura Instantánea de la figura 6.8.6. □

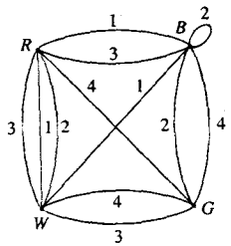
Ejercicios

Determine soluciones a los siguientes juegos de Locura Instantánea.

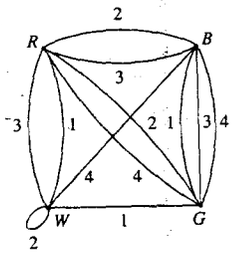
1.



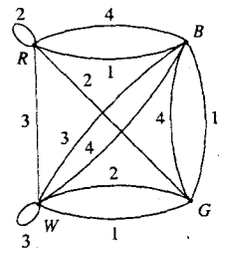
2.



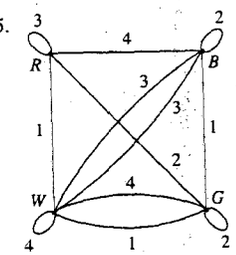
3.



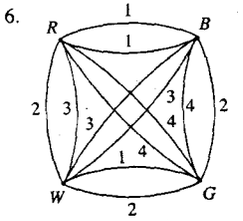
4.



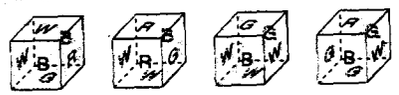
5.



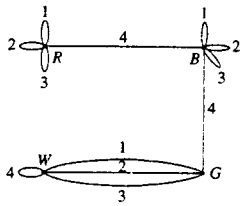
6.



7. (a) Determine todas las subgráficas de la figura 6.8.5 que satisfagan (6.8.1) y (6.8.2).
   (b) Determine todas las soluciones del juego Locura Instantánea de la figura 6.8.5.
8. (a) Represente el juego Locura Instantánea mediante una gráfica.



   (b) Determine una solución del juego.
   (c) Determine todas las subgráficas de la gráfica de la parte (a) que satisfagan las propiedades (6.8.1) y (6.8.2).
   (d) Utilice (c) para mostrar que el juego tiene una única solución.



9. Muestre que el juego Locura Instantánea anexo no tiene solución, proporcionando un argumento para mostrar que ninguna subgráfica satisface las propiedades (6.8.1) y (6.8.2). Observe que no existe una solución, aunque cada cubo contuviese los cuatro colores.

☆ 10. Proporcione un ejemplo de un juego Locura Instantánea que satisfaga:

(a) No existe una solución.

(b) Cada cubo contiene los cuatro colores.

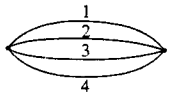
(c) Existe una subgráfica que satisface las propiedades (6.8.1) y (6.8.2).

11. Muestre que existen 24 orientaciones de un cubo.

12. Numere los cubos de un juego Locura Instantánea como 1, 2, 3 y 4. Muestre que el número de formas de apilar los cubos en el orden 1, 2, 3 y 4, leídos de arriba hacia abajo, son 331,776.

★ **13.** ¿Cuántas gráficas de Locura Instantánea existen? Es decir, ¿cuántas gráficas existen con cuatro vértices y 12 aristas (tres de ellas de uno de cuatro tipos)?

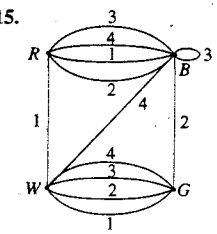
Los ejercicios 14-21 se refieren a una versión modificada del juego Locura Instantánea, donde una solución se define como una pila tal que, vista desde el frente, por detrás, por la izquierda o por la derecha, muestre un único color. (El frente, la parte posterior, la parte izquierda y la derecha tienen colores diferentes.)



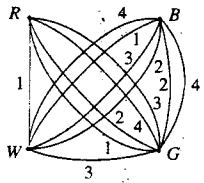
14. Proporcione un argumento que muestre que si graficamos el juego como un juego regular de Locura Instantánea, una solución al juego modificado consta de dos subgráficas de la forma que aparece en la figura, sin aristas ni vértices en común.

Determine soluciones de la Locura Instantánea modificada para los siguientes juegos.

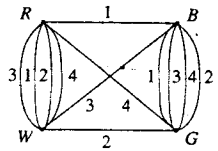
15.



16.



17.



18. La gráfica del ejercicio 6.

19. Muestre que para la figura 6.8.5, la Locura Instantánea dada en el texto tiene una solución, pero la versión modificada no tiene solución.

☆ 20. Muestre que si la Locura Instantánea tiene una solución, la versión dada en el texto debe tener una solución.

21. ¿Es posible que ninguna de las versiones de Locura Instantánea tenga solución, aunque cada cubo contenga los cuatro colores? Si la respuesta es afirmativa, demuéstrelo; en caso contrario, proporcione un contraejemplo.

NOTAS

Prácticamente cualquier libro de matemáticas discretas tiene uno o más capítulos sobre teoría de gráficas. Algunos libros específicos de teoría de gráficas son [Berge; Bondy; Chartrand; Deo; Even, 1979; Gibbons; Harary; König; Ore; y Wilson]. [Deo; Even, 1979; y Gibbons] enfatizan los algoritmos de gráficas. [Brassard y Cormen] también estudian las gráficas y los algoritmos de gráficas.

[Fukunaga; Gose; y Nadler] son textos relativos al reconocimiento de patrones.

[Akl; Leighton; Lester; Lewis; y Quinn] analizan las computadoras paralelas y los algoritmos para computadoras paralelas. Nuestros resultados acerca de las subgráficas del hipercubo son de [Saad].

El artículo original de Euler acerca de los puentes de Königsberg, editado por J. R. Newman, fue reimpreso en [Newman].

En [Gardner, 1959], los ciclos hamiltonianos se relacionan con el juego de la Torre de Hanoi.

En muchos casos, los métodos llamados de *ramificación y acotamiento* (véase, por ejemplo, [Tucker]) proporcionan soluciones al problema del agente viajero de manera más eficiente que la búsqueda exhaustiva.

El algoritmo de la ruta más corta de Dijkstra aparece en [Dijkstra, 1959].

La complejidad del problema de isomorfismo de gráficas se analiza en [Köbler].

Appel y Haken publicaron su solución al problema de los cuatro en [Appel].

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 6.1

Gráfica de similaridad
Función de disimilaridad
Reconocimiento de patrones
n-cubo (hipercubo)
Computadora en serie
Algoritmo en serie
Computadora paralela
Algoritmo paralelo

Sección 6.2

Camino
Camino simple
Ciclo
Ciclo simple
Gráfica conexa
Subgráfica
Componente de una gráfica
Grado de un vértice $\delta(v)$
Problema de los puentes de Königsberg
Ciclo de Euler
Una gráfica G tiene un ciclo de Euler si y sólo si G es conexa y todo vértice tiene grado par.
La suma de los grados de todos los vértices de una gráfica es igual a dos veces el número de aristas.
En cualquier gráfica existe un número par de vértices de grado impar.
Una gráfica tiene un camino sin aristas repetidas de v a w ($v \neq w$) que contiene a todas las aristas y todos los vértices si y sólo si es conexa y v y w son los únicos vértices que tienen grado impar.
Si una gráfica G contiene un ciclo de v a v, entonces G contiene un ciclo simple de v a v.

Sección 6.3

Ciclo hamiltoniano
Problema del agente viajero
Modelo de anillo para la computación paralela
Código Gray

Sección 6.4

Algoritmo de la ruta más corta de Dijkstra

DEFINICIÓN 6.7.3

Si una gráfica G tiene un vértice v de grado 2 y aristas (v, v_1) y (v, v_2) con $v_1 \neq v_2$, decimos que las aristas (v, v_1) y (v, v_2) están en *serie*. Una *reducción de serie* consiste en la eliminación del vértice v de la gráfica G y el reemplazo de las aristas (v, v_1) y (v, v_2) por la arista (v_1, v_2). Decimos que la gráfica resultante G' se *obtiene de G mediante una reducción de serie*. Por convención, se dice que G se puede obtener de sí misma mediante una reducción de serie.

EJEMPLO 6.7.4

En la gráfica G de la figura 6.7.4, las aristas (v, v_1) y (v, v_2) están en serie. La gráfica G' de la figura 6.7.4 se obtiene de G mediante una reducción de serie.

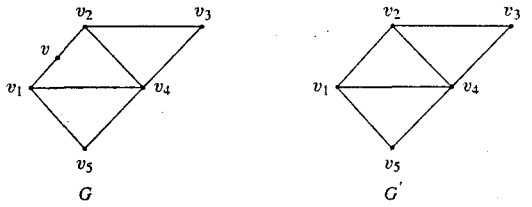


FIGURA 6.7.4 G' se obtiene de G mediante una reducción de serie. □

DEFINICIÓN 6.7.5

Las gráficas G_1 y G_2 son *homeomorfas* si G_1 y G_2 se pueden reducir a gráficas isomorfas realizando varias reducciones de serie.

De acuerdo con las definiciones 6.7.3 y 6.7.5, cualquier gráfica es homeomorfa a sí misma. Además, las gráficas G_1 y G_2 son homeomorfas si G_1 se puede reducir a una gráfica isomorfa a G_2 o si G_2 se puede reducir a una gráfica isomorfa a G_1.

EJEMPLO 6.7.6

Las gráficas G_1 y G_2 de la figura 6.7.5 son homeomorfas, pues ambas se pueden reducir a la gráfica G' de la figura 6.7.5 mediante varias reducciones de serie.

Si definimos una relación R sobre un conjunto de gráficas mediante la regla $G_1 R G_2$ si G_1 y G_2 son homeomorfas, R es una relación de equivalencia. Cada clase de equivalencia consta de un conjunto de gráficas homeomorfas entre ellas.

Ahora establecemos una condición necesaria y suficiente para que una gráfica sea plana. Kuratowski estableció por vez primera el teorema en 1930. La demostración aparece en [Even, 1979].

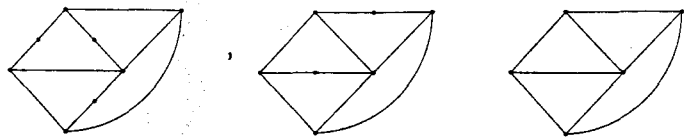


FIGURA 6.7.5 G_1 y G_2 son homeomorfas; cada una se puede reducir a G'.

BIBLIOTECA FACULTAD DE EXACTAS, INGENIERÍA Y AGRIMENSURA ROSARIO

TEOREMA 6.7.7 ***Teorema de Kuratowski***

Una gráfica G es plana si y sólo si G no contiene una subgráfica homeomorfa a K_5 *o* $K_{3,3}$. ■

EJEMPLO 6.7.8

Utilizando el teorema de Kuratowski, muestre que la gráfica G de la figura 6.7.6 no es plana.

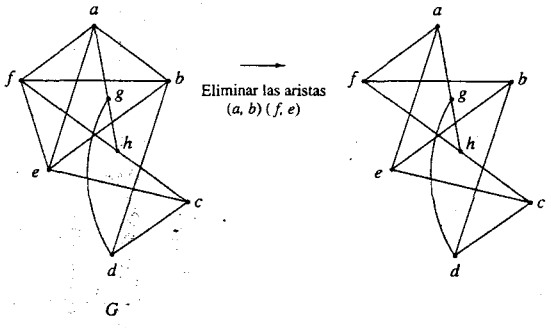


FIGURA 6.7.6 Eliminación de aristas para obtener una subgráfica.

Intentemos encontrar $K_{3,3}$ en la gráfica G de la figura 6.7.6. Primero observemos que los vértices a, b, f y e tienen cada uno grado 4. En $K_{3,3}$ cada vértice tiene grado 3, de modo que eliminaremos las aristas (a, b) y (f, e) para que todos los vértices tengan grado 3 (véase la figura 6.7.6). Observemos que si eliminamos una arista más, obtendremos dos vértices de grado 2 y luego podremos realizar dos reducciones de serie. La gráfica resultante tendrá nueve aristas, y como $K_{3,3}$ tiene nueve aristas, este punto de vista parece promisorio. Mediante prueba y error, vemos finalmente que si eliminamos la arista (g, h) y realizamos las reducciones de serie, obtenemos una copia isomorfa de $K_{3,3}$ (véase la figura 6.7.7). Por tanto, la gráfica G de la figura 6.7.6 no es plana, pues contiene una subgráfica homeomorfa a $K_{3,3}$. □

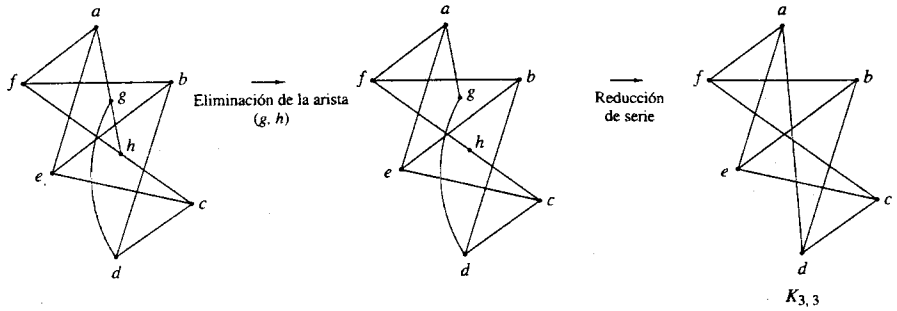


FIGURA 6.7.7 Eliminación de una arista para obtener una subgráfica, seguida por reducciones de serie.

Aunque el teorema 6.7.7 proporciona una elegante caracterización de las gráficas planas, no conduce a un algoritmo eficiente para el reconocimiento de gráficas planas. Sin embargo, se conocen algoritmos que pueden determinar si una gráfica con n vértices es plana en un tiempo $O(n)$ (véase [Even,1979]).

Concluiremos esta sección demostrando la fórmula de Euler.

TEOREMA 6.7.9 ***Fórmula de Euler para gráficas***

Si G es una gráfica plana, conexa, con e aristas, v vértices y f caras, entonces

$$f = e - v + 2. \tag{6.7.3}$$

$f = 1, e = 1, v = 2$

$f = 2, e = 1, v = 1$

FIGURA 6.7.8
El paso base del teorema 6.7.9.

Demostración. Utilizaremos inducción sobre el número de aristas.

Supongamos que $e = 1$. Entonces G es una de las dos gráficas que aparecen en la figura 6.7.8. En cualquier caso, la fórmula es válida. Hemos verificado el paso base.

Suponga que la fórmula es válida para las gráficas planas, conexas, con n aristas. Sea G una gráfica con $n + 1$ aristas. En primer lugar, suponga que G no contiene ciclos. Elegimos un vértice v y trazamos un camino que comienza en v. Como G no tiene ciclos, cada vez que trazamos una arista, llegamos a un nuevo vértice. En cierto momento, llegaremos a un vértice a, con grado 1, del cual no podremos salir (véase la figura 6.7.9). Eliminamos

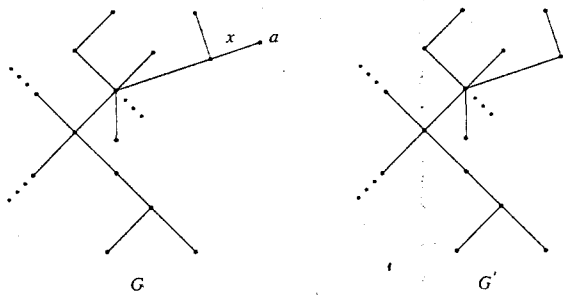


FIGURA 6.7.9 La demostración del teorema 6.7.9 para el caso en que G no tiene ciclos. Encontramos un vértice a de grado 1 y eliminamos a y la arista x incidente en él.

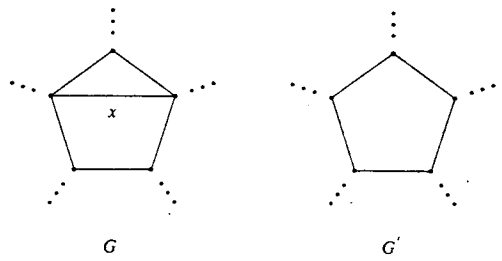


FIGURA 6.7.10 La demostración del teorema 6.7.9 para el caso en que G tiene un ciclo. Eliminamos una arista x en un ciclo.

a y la arista x incidente en a de la gráfica G. La gráfica resultante G' tiene n aristas; por tanto, por la hipótesis inductiva, (6.7.3) es válida para G'. Como G tiene una arista más que G', un vértice más que G', y el mismo número de caras que G', esto implica que (6.7.3) también es válida para G.

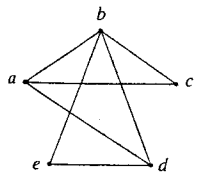
Ahora supongamos que G contiene un ciclo. Sea x una arista en un ciclo (véase la figura 6.7.10). Ahora, x es parte de una frontera para dos caras. Esta vez eliminamos la arista x, pero no los vértices, para obtener la gráfica G' (véase la figura 6.7.10). De nuevo, G' tiene n aristas; por tanto, por la hipótesis de inducción, (6.7.3) es válida para G'. Como G tiene una cara más que G', una arista más que G', y el mismo número de vértices que G', esto implica que (6.7.3) también es válida para G.

Como hemos verificado el paso inductivo, por el principio de inducción matemática, el teorema queda demostrado. ■

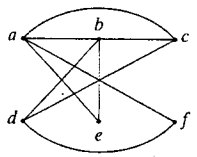
Ejercicios

En los ejercicios 1-3, muestre que cada gráfica es plana, volviéndola a trazar de modo que no haya cruces entre las aristas.

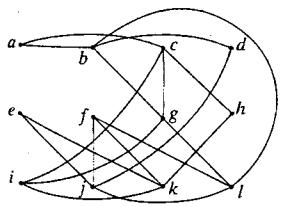
1.



2.

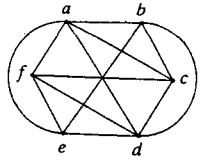


3.

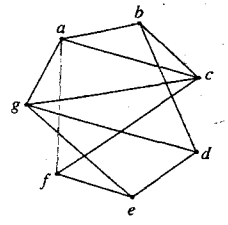


En los ejercicios 4 y 5, muestre que cada gráfica no es plana, encontrando una subgráfica homeomorfa a K_5 o $K_{3,3}$.

4.

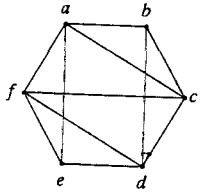


5.

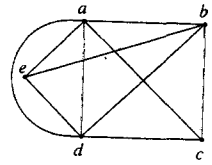


En los ejercicios 6-8, determine si cada gráfica es plana. Si la gráfica es plana, vuelva a trazar de modo que no haya cruces entre las aristas; en caso contrario, determine una subgráfica homeomorfa a K_5 o $K_{3,3}$.

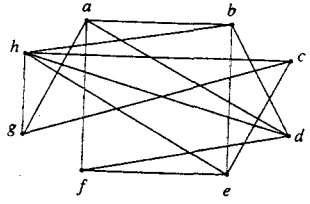
6.



7.



8.



9. Una gráfica plana, conexa, tiene nueve vértices con grados 2, 2, 2, 3, 3, 3, 4, 4 y 5. ¿Cuántas aristas existen? ¿Cuántas caras existen?

10. Muestre que al agregar o eliminar lazos, aristas paralelas o aristas en serie, no se afecta la planaridad de una gráfica.

11. Muestre que cualquier gráfica con cuatro o menos vértices es plana.

12. Muestre que cualquier gráfica con cinco o menos vértices y un vértice de grado 2 es plana.

13. Muestre que en cualquier gráfica simple, conexa, plana, $e \leq 3v - 6$.

14. Proporcione un ejemplo de una gráfica simple, conexa, no plana, para la cual $e \leq 3v - 6$.

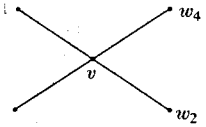
15. Utilice el ejercicio 13 para mostrar que K_5 no es plana.

☆ 16. Muestre que si una gráfica simple G tiene 11 o más vértices, entonces G o su complemento $\overline{G}$ no es plana.

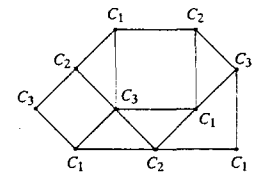
☆ 17. Demuestre que si una gráfica plana tiene un ciclo de Euler, entonces tiene un ciclo de Euler sin cruces. Un camino P en una gráfica plana tiene un cruce si un vértice v aparece al menos dos veces en P y P se cruza a sí mismo en v; es decir,

$$P = (\ldots, w_1, v, w_2, \ldots, w_3, v, w_4, \ldots),$$

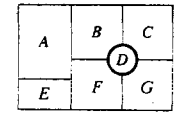
donde los vértices están ordenados de modo que w_1, v, w_2 cruza a w_3, v, w_4 en v como en la figura anexa.



Una *coloración* de una gráfica G mediante los colores $C_1, C_2, \ldots, C_n$ asigna a cada vértice un color C_i de modo que cualquier vértice tenga un color diferente al de cualquiera de sus vértices adyacentes. Por ejemplo, la gráfica anexa se colorea con tres colores. El resto de los ejercicios trata de la coloración de gráficas planas.



Un *mapa plano* es una gráfica plana en la cual las caras se interpretan como países, las aristas se interpretan como fronteras entre los países y los vértices representan la intersección de las fronteras. El problema de coloración de un mapa plano G, de modo que no haya países con fronteras contiguas con el mismo color, se puede reducir al problema de coloración de una gráfica, construyendo primero la *gráfica dual* G' de G de la siguiente manera. Los vértices de la gráfica dual G' constan de un punto en cada cara de G, incluyendo la cara no acotada. Una arista en G' conecta dos vértices si las caras correspondientes en G están separadas por una frontera. La coloración del mapa G es equivalente a la coloración de los vértices de la gráfica dual G'.



Ejercicio 18

18. Determine el dual del mapa adyacente.

19. Muestre que el dual de un mapa plano es una gráfica plana.

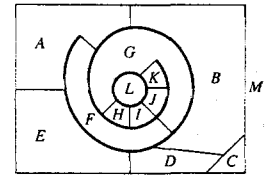
20. Muestre que cualquier coloración del mapa del ejercicio 18, excluyendo la región no acotada, requiere al menos tres colores.

21. Establezca una coloración del mapa del ejercicio 18, excluyendo la región no acotada, utilizando tres colores.

22. Determine el dual del mapa adyacente.

23. Muestre que cualquier coloración del mapa del ejercicio 22, excluyendo la región no acotada, requiere al menos cuatro colores.

24. Establezca una coloración del mapa del ejercicio 22, excluyendo la región no acotada, utilizando cuatro colores.



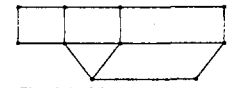
Ejercicio 22

Una *triangulación* de una gráfica plana simple G se obtiene de G uniendo el mayor número posible de vértices, conservando la planaridad y sin introducir lazos ni aristas paralelas.

25. Determine una triangulación de la gráfica adyacente.

26. Muestre que si una triangulación G' de una gráfica plana, simple G se puede colorear con n colores, entonces se puede hacer lo mismo con G.

27. Muestre que en una triangulación de una gráfica plana, simple, $3f = 2e$.



Ejercicio 25

Appel y Haken demostraron (véase [Appel]) que toda gráfica plana, simple, se puede colorear con cuatro colores. El problema había sido planteado a mitad del siglo XIX y durante años nadie había logrado establecer una demostración. Las personas que trabajan actualmente en el problema de los cuatro colores en los últimos años tienen una ventaja sobre sus predecesores: el uso de rápidas computadoras electrónicas. Los siguientes ejercicios muestran la forma en que comienza la demostración.

Suponga que existe una gráfica plana, simple, que requiera más de cuatro colores para colorearse. Entre todas las gráficas de este tipo, existe una con la menor cantidad de vértices. Sea G una triangulación de esta gráfica. Entonces G también tiene un número mínimo de vértices y por el ejercicio 26, G requiere más de cuatro colores para colorearse.

28. Si el dual de un mapa tiene un vértice de grado 3, ¿cómo debe verse el mapa original?

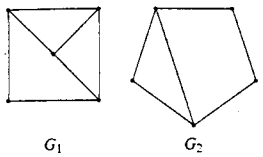
29. Muestre que G no puede tener un vértice de grado 3.

☆ 30. Muestre que G no puede tener un vértice de grado 4.

★ **31.** Muestre que G tiene un vértice de grado 5.

La contribución de Appel y Haken fue mostrar que sólo había que considerar un número finito de casos relacionados con el vértice de grado 5, analizar todos estos casos y mostrar que todos se podrían colorear mediante cuatro colores. La reducción a un número finito de casos fue facilitada por el uso de computadoras para poder determinar los casos por analizar. Luego se volvieron a utilizar las computadoras para analizar los casos restantes.

☆ 32. Muestre que cualquier gráfica plana simple se puede colorear mediante cinco colores.



G_1 G_2

FIGURA 6.6.3
Gráficas no isomorfas. G_1 tiene siete aristas y G_2 tiene seis aristas.

EJEMPLO 6.6.7

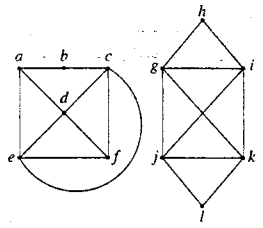
Las gráficas G_1 y G_2 de la figura 6.6.3 no son isomorfas, pues G_1 tiene siete aristas y G_2 tiene seis aristas, y "tener siete aristas" es un invariante. □

EJEMPLO 6.6.8

Muestre que si k es un entero positivo, entonces "tener un vértice de grado k" es un invariante.

Suponga que G_1 y G_2 son gráficas isomorfas y que f (respectivamente, g) es una función uno a uno y sobre de los vértices (respectivamente, las aristas) de G_1 sobre los vértices (respectivamente, las aristas) de G_2. Suponga que G_1 tiene un vértice v de grado k. Entonces existen k aristas $e_1, \ldots, e_k$ incidentes en v. Por la definición 6.6.1, $g(e_1), \ldots, g(e_k)$ son incidentes en $f(v)$. Como g es uno a uno, $\delta(f(v)) \geq k$.

Sea E una arista incidente en $f(v)$ en G_2. Como g es sobre, existe una arista e en G_1 tal que $g(e) = E$. Como $g(e)$ es incidente en $f(v)$ en G_2, la definición 6.6.1 implica que e es incidente en v en G_1. Como $e_1, \ldots, e_k$ son las únicas aristas en G_1 incidentes en v, $e = e_i$ para algún $i \in \{1, \ldots, k\}$. Ahora, $g(e_i) = g(e) = E$. Así, $\delta(f(v)) = k$, de modo que G_2 tiene un vértice, a saber, $f(v)$, de grado k. □



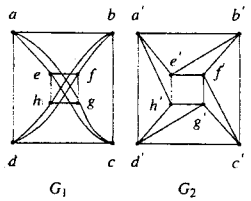
G_1 G_2

FIGURA 6.6.4
Gráficas no isomorfas. G_1 tiene vértices de grado 3, pero G_2 no tiene vértices de grado 3.

EJEMPLO 6.6.9

Como "tener un vértice de grado 3" es un invariante, las gráficas G_1 y G_2 de la figura 6.6.4 no son isomorfas; G_1 tiene vértices (a y f) de grado 3, pero G_2 no tiene un vértice de grado 3. Observe que G_1 y G_2 tienen el mismo número de aristas y de vértices. □

Otro invariante que es útil en ciertas ocasiones es "tener un ciclo simple de longitud k". Dejaremos la demostración de que esta propiedad es un invariante para los ejercicios (ejercicio 12).



G_1 G_2

FIGURA 6.6.5
Gráficas no isomorfas. G_2 tiene un ciclo simple de longitud 3, pero G_1 no tiene ciclos simples de longitud 3.

EJEMPLO 6.6.10

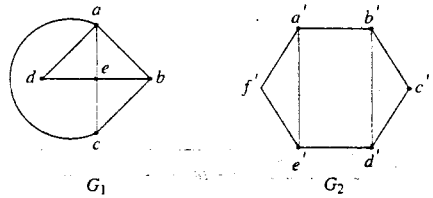
Como "tener un ciclo simple de longitud 3" es un invariante, las gráficas G_1 y G_2 de la figura 6.6.5 no son isomorfas; la gráfica G_2 tiene un ciclo simple de longitud 3, pero todos los ciclos simples en G_1 tienen longitud al menos 4. Observe que G_1 y G_2 tienen el mismo número de aristas y de vértices y que cada vértice en G_1 o G_2 tiene grado 4. □

Sería fácil verificar si un par de gráficas es isomorfo o no, si pudiéramos verificar rápidamente un número pequeño de invariantes, los cuales fuesen compartidos por las gráficas isomorfas y *sólo* por ellas. Por desgracia, ninguna persona ha podido determinar tal conjunto de invariantes.

Ejercicios

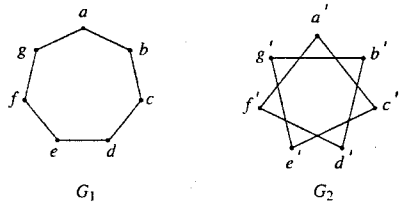
En los ejercicios 1-10, determine si las gráficas G_1 y G_2 son isomorfas. Si las gráficas son isomorfas, determine las funciones f y g de la definición 6.6.1; en caso contrario, proporcione un invariante no compartido por las gráficas.

1.



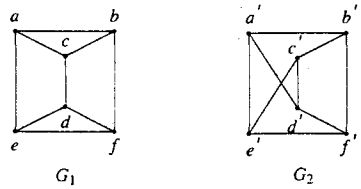
G_1 G_2

2.



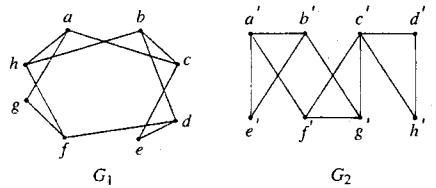
G_1 G_2

3.



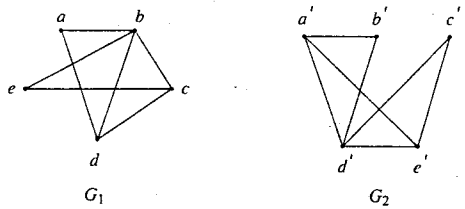
G_1 G_2

4.



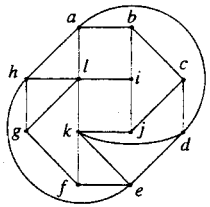
G_1 G_2

5.

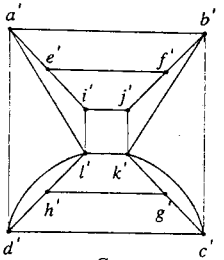


G_1 G_2

6.

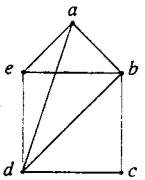


G_1

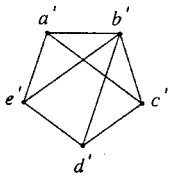


G_2

7.

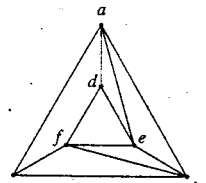


G_1

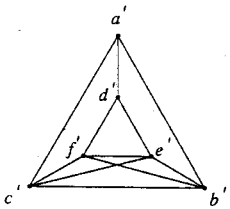


G_2

☆ 8.

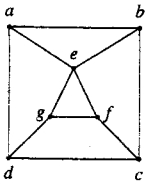


G_1

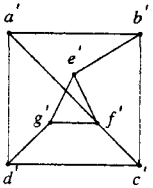


G_2

☆ 9.

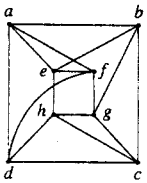


G_1

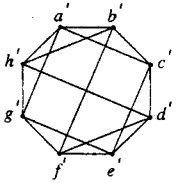


G_2

★ 10.



G_1



G_2

11. Muestre que si M es una red $p_1 \times p_2 \times \cdots \times p_k$, donde $p_i \leq 2^{t_i}$, para $i = 1, \ldots, k$, entonces el $(t_1 + t_2 + \cdots + t_k)$-cubo contiene una subgráfica isomorfa a M.

En los ejercicios 12-16, muestre que la propiedad dada es un invariante.

12. Tiene un ciclo simple de longitud k

13. Tiene n vértices de grado k

14. Es conexa

15. Tiene n ciclos simples de longitud k

16. Tiene una arista (v, w), donde $\delta(v) = i$ y $\delta(w) = j$

17. Determine un invariante no dado en esta sección o en los ejercicios 12-16. Demuestre que su propiedad es un invariante.

En los ejercicios 18-20, indique si cada propiedad es un invariante o no. Si la propiedad es un invariante, demuestre que lo es; en caso contrario, proporcione un contraejemplo.

18. Tiene un ciclo de Euler

19. Tiene un vértice dentro de un ciclo simple

20. Es bipartita

21. Trace todas las gráficas simples no isomorfas que tengan tres vértices.

22. Trace todas las gráficas simples no isomorfas que tengan cuarto vértices.

23. Trace todas las gráficas conexas, sin ciclos, no isomorfas que tengan cinco vértices.

24. Trace todas las gráficas conexas, sin ciclos, no isomorfas que tengan seis vértices.

25. Muestre que las gráficas simples G_1 y G_2 son isomorfas si y sólo si sus vértices se pueden ordenar de modo que sus matrices de adyacencia sean idénticas.

El *complemento* de una gráfica simple G es la gráfica simple $\overline{G}$ con los mismos vértices que G. Una arista existe en $\overline{G}$ si y sólo si no existe en G.

26. Trace el complemento de la gráfica G_1 del ejercicio 1.

27. Trace el complemento de la gráfica G_2 del ejercicio 1.

☆ 28. Muestre que si G es una gráfica simple, entonces G o $\overline{G}$ es conexa.

29. Una gráfica simple G es **autocomplementaria** si G y $\overline{G}$ son isomorfas.

(a) Determine una gráfica autocomplementaria con cinco vértices.

(b) Determine otra gráfica autocomplementaria.

30. Sean G_1 y G_2 gráficas simples. Muestre que G_1 y G_2 son isomorfas si y sólo si $\overline{G}_1$ y $\overline{G}_2$ son isomorfas.

31. Dadas dos gráficas G_1 y G_2, suponga que existe una función uno a uno y sobre, f, de los vértices de G_1 a los vértices de G_2 y una función uno a uno y sobre, g, de las aristas de G_1 a las aristas de G_2, de modo que si una arista e es incidente en v y w en G_1, la arista $g(e)$ es incidente en $f(v)$ y $f(w)$ en G_2. ¿Son G_1 y G_2 isomorfas?

Un *homomorfismo* de una gráfica G_1 en una gráfica G_2 es una función f del conjunto de vértices de G_1 en el conjunto de vértices de G_2 con la propiedad de que si v y w son adyacentes en G_1, entonces $f(v)$ y $f(w)$ son adyacentes en G_2.

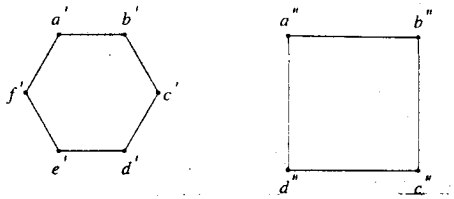
32. Suponga que G_1 y G_2 son gráficas simples. Muestre que si f es un homomorfismo de G_1 en G_2 y f es uno a uno y sobre, G_1 y G_2 son isomorfas.

En los ejercicios 33-37, para cada par de gráficas, proporcione un ejemplo de homomorfismo de G_1 en G_2.

33.

G_1 G_2

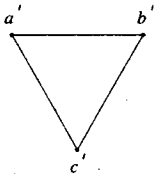
34.



G_1 G_2

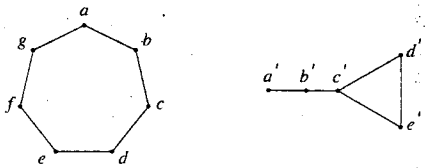
35. $G_1 = G_1$ del ejercicio 34; $G_2 = G_1$ del ejercicio 33

36. $G_1 = G_1$ del ejercicio 33



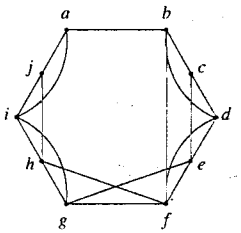
G_2

37.



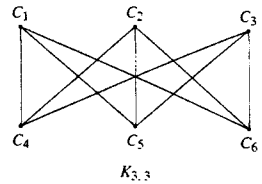
G_1 G_2

☆ 38. [Hell] Muestre que el único homomorfismo de la siguiente gráfica en sí misma es la función idéntica.



6.7 *GRÁFICAS PLANAS*

Hay que unir tres ciudades, C_1, C_2 y C_3, de manera directa, mediante autopistas que van hacia otras tres ciudades, C_4, C_5 y C_6. ¿Puede diseñarse este sistema de carreteras de modo que las autopistas no se crucen? En la figura 6.7.1 se muestra un sistema en la cual se cruzan los caminos. Si usted intenta trazar un sistema en el cual los caminos no se crucen, pronto se convencerá de que esto no se puede hacer. Posteriormente en esta sección explicaremos con cuidado este hecho.



$K_{3,3}$

FIGURA 6.7.1
Ciudades unidas por autopistas.

DEFINICION 6.7.1

Una gráfica es *plana* si se puede trazar en el plano sin que se crucen sus aristas.

En el diseño de circuitos impresos es recomendable tener el menor número posible de cruces de líneas; así, el diseñador de circuitos impresos enfrenta el problema de la planaridad.

Si una gráfica conexa y plana se traza en el plano, el plano se divide en regiones contiguas llamadas **caras**. Una cara queda caracterizada por el ciclo que forma su frontera. Por ejemplo, en la gráfica de la figura 6.7.2, la cara A está acotada por el ciclo (5, 2, 3, 4, 5) y la cara C está acotada por el ciclo (1, 2, 5, 1). La cara exterior D se considera acotada por el ciclo (1, 2, 3, 4, 6, 1). La gráfica de la figura 6.7.2 tiene $f = 4$ caras, $e = 8$ aristas y $v = 6$ vértices. Observe que f, e y v satisfacen la ecuación

$$f = e - v + 2. \tag{6.7.1}$$

En 1752, Euler demostró que la ecuación (6.7.1) es válida para cualquier gráfica plana conexa. Al final de esta sección mostraremos la idea de la demostración de (6.7.1), pero por ahora sólo la utilizaremos para mostrar que ciertas gráficas no son planas.

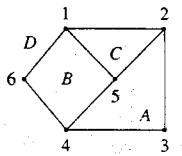


FIGURA 6.7.2
Una gráfica plana conexa con $f = 4$ caras (A, B, C, D), $e = 8$ aristas y $v = 6$ vértices; $f = e - v + 2$.

EJEMPLO 6.7.2

Muestre que la gráfica $K_{3,3}$ de la figura 6.7.1 no es plana.

Suponga que $K_{3,3}$ es plana. Como cada ciclo tiene al menos cuatro aristas, cada cara está acotada por al menos cuatro aristas. Así, el número de aristas que acotan caras es al menos $4f$. En una gráfica plana, cada arista pertenece a lo más a dos ciclos acotados por ella. Por tanto,

$$2e \geq 4f.$$

Utilizamos (6.7.1) para obtener

$$2e \geq 4\,(e - v + 2). \tag{6.7.2}$$

Para la gráfica de la figura 6.7.1, $e = 9$ y $v = 6$, de modo que (6.7.2) se convierte en

$$18 = 2 \cdot 9 \geq 4\,(9 - 6 + 2) = 20,$$

lo cual es una contradicción. Por tanto, $K_{3,3}$ no es plana. □

Mediante un tipo similar de argumento (véase el ejercicio 15) podemos mostrar que la gráfica K_5 de la figura 6.7.3 no es plana.

Es claro que si una gráfica contiene a $K_{3,3}$ o K_5 como una subgráfica, no puede ser plana. El recíproco casi es verdadero. Para establecer la situación con precisión, debemos presentar algunos términos nuevos.

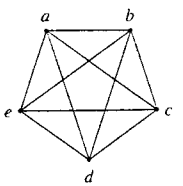


FIGURA 6.7.3
La gráfica no plana K_5.

tenemos que

$$A^4 = \begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array} \begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 9 & 3 & 11 & 1 & 6 \\ 3 & 15 & 7 & 11 & 8 \\ 11 & 7 & 15 & 3 & 8 \\ 1 & 11 & 3 & 9 & 6 \\ 6 & 8 & 8 & 6 & 8 \end{pmatrix} \end{array}.$$

La entrada del renglón d, columna e es 6, lo cual significa que existen seis caminos de longitud 4 de d a e. Por inspección, vemos que estos caminos son

$$(d, a, d, c, e), \quad (d, c, d, c, e), \quad (d, a, b, c, e),$$
$$(d, c, e, c, e), \quad (d, c, e, b, e), \quad (d, c, b, c, e).$$

□

Otra útil representación matricial de una gráfica es la **matriz de incidencia**.

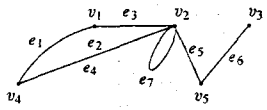


FIGURA 6.5.4
La gráfica del ejemplo 6.5.5.

EJEMPLO 6.5.5 *Matriz de incidencia*

Para obtener la matriz de incidencia de la gráfica de la figura 6.5.4, etiquetamos los renglones con los vértices y las columnas con las aristas (en algún orden arbitrario). La entrada del renglón v y la columna e es 1 si e es incidente en v y 0 en caso contrario. Así, la matriz de incidencia de la gráfica de la figura 6.5.4 es

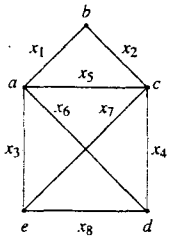
$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{ccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 \end{array} \\ \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \end{array}.$$

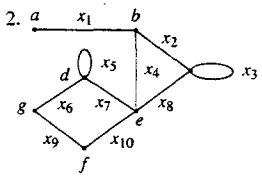
Una columna como e_7 representa un lazo. □

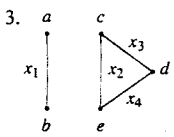
La matriz de incidencia nos permite representar las aristas paralelas y los lazos. Observe que en una gráfica sin lazos, cada columna tiene dos 1 y que la suma de los elementos de un renglón proporciona el grado del vértice identificado con ese renglón.

Ejercicios

En los ejercicios 1-6, escriba la matriz de adyacencia de cada gráfica.

1.


2.


3.


4. La gráfica de la figura 6.2.2
5. La gráfica completa de cinco vértices K_5
6. La gráfica bipartita completa $K_{2,3}$

En los ejercicios 7-12, escriba la matriz de incidencia de cada gráfica.

7. La gráfica del ejercicio 1
8. La gráfica del ejercicio 2
9. La gráfica del ejercicio 3
10. La gráfica del ejercicio 6.2.1
11. La gráfica completa de cinco vértices K_5
12. La gráfica bipartita completa $K_{2,3}$

En los ejercicios 13-17, trace la gráfica representada por cada matriz de adyacencia.

13.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array} \begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

14.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array} \begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

15.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \\ f \end{array} \begin{array}{c} \begin{array}{cccccc} a & b & c & d & e & f \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

16.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \\ f \end{array} \begin{array}{c} \begin{array}{cccccc} a & b & c & d & e & f \end{array} \\ \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \end{pmatrix} \end{array}$$

17. La matriz 7×7 cuya entrada ij-ésima es 1 si $i + 1$ divide a $j + 1$ o $j + 1$ divide a $i + 1$ y cuya entrada ij-ésima es 0 en caso contrario.
18. Escriba las matrices de adyacencia de los componentes de las gráficas dadas por las matrices de adyacencia de los ejercicios 13-17.
19. Calcule los cuadrados de las matrices de adyacencia de K_5 y las gráficas de los ejercicios 1 y 3.
20. Sea A la matriz de adyacencia para la gráfica del ejercicio 1. ¿Cuál es la entrada en el renglón a, columna d de A^5?
21. Suponga que una gráfica tiene una matriz de adyacencia de la forma

$$A = \left(\begin{array}{c|c} & A' \\ \hline A'' & \end{array}\right),$$

donde todas las entradas de las submatrices A' y A'' son 0. ¿Cómo debe verse la gráfica?
22. Repita el ejercicio 21 reemplazando "adyacencia" por "incidencia".
23. ¿Cómo podría modificarse la definición de matriz de adyacencia para poder representar las aristas paralelas?
24. Sea A una matriz de adyacencia de una gráfica. ¿Por qué A^n es simétrica con respecto de la diagonal principal para cada entero positivo n?

En los ejercicios 25 y 26, trace las gráficas representadas por las matrices de incidencia.

25.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array}\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

26.
$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array}\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

27. ¿Cómo debe verse una gráfica si algún renglón de su matriz de incidencia consta sólo de ceros?

28. Sea A la matriz de adyacencia de una gráfica G con n vértices. Sea

$$Y = A + A^2 + \cdots + A^{n-1}.$$

Si alguna entrada fuera de la diagonal principal de la matriz Y es igual a cero, ¿qué podría decir de la gráfica G?

Los ejercicios 29-32 se refieren a la matriz de adyacencia A de K_5.

29. Sea n un entero positivo. Explique por qué todos los elementos en la diagonal de A^n son iguales entre ellos y todos los elementos fuera de la diagonal de A^n son iguales unos con otros.

Sea d_n el valor común de los elementos de la diagonal de A^n y sea a_n el valor común de los elementos fuera de la diagonal de A^n.

☆ 30. Muestre que

$$d_{n+1} = 4a_n; \quad a_{n-1} = d_n + 3a_n; \quad a_{n+1} = 3a_n + 4a_{n-1}.$$

☆ 31. Muestre que

$$a_n = \frac{1}{5}[4^n + (-1)^{n+1}].$$

32. Muestre que

$$d_n = \frac{4}{5}[4^{n-1} + (-1)^n].$$

☆ 33. Deduzca resultados similares a los de los ejercicios 30-32 para la matriz de adyacencia A de la gráfica K_m.

☆ 34. Sea A la matriz de adyacencia de la gráfica $K_{m,n}$. Determine una fórmula para las entradas de A^2.

6.6 *ISOMORFISMOS DE GRÁFICAS*

Se proporcionan las siguientes instrucciones a dos personas, quienes no pueden ver la hoja de la otra persona: "Trace y etiquete cinco vértices como a, b, c, d y e. Una a con b, b con c, c con d, d con e y a con e." Las gráficas producidas aparecen en la figura 6.6.1. Es seguro que estas figuras definen la misma gráfica aunque parezcan diferentes. Tales gráficas son **isomorfas**.

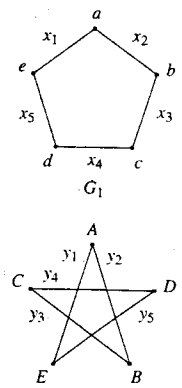


FIGURA 6.6.1
Gráficas isomorfas.

DEFINICIÓN 6.6.1

Las gráficas G_1 y G_2 son *isomorfas* si existe una función uno a uno y sobre, f, de los vértices de G_1 a los vértices de G_2 y una función uno a uno y sobre, g, de las aristas de G_1 a las aristas de G_2, de modo que una arista e es incidente en v y w en G_1 si y sólo si la arista $g(e)$ es incidente en $f(v)$ y $f(w)$ en G_2. El par de funciones f y g es un *isomorfismo* de G_1 sobre G_2.

EJEMPLO 6.6.2

Un isomorfismo para las gráficas G_1 y G_2 de la figura 6.6.1 se define como

$$f(a) = A, \quad f(b) = B, \quad f(c) = C, \quad f(d) = D, \quad f(e) = E$$
$$g(x_i) = y_i, \quad i = 1, \ldots, 5. \qquad \square$$

Si definimos una relación R en un conjunto de gráficas mediante la regla $G_1 R G_2$ si G_1 y G_2 son isomorfas, entonces R es una relación de equivalencia. Cada clase de equivalencia consta de un conjunto de gráficas isomorfas entre ellas.

EJEMPLO 6.6.3 ***El modelo de red para la computación paralela***

Anteriormente consideramos el problema de cuándo el n-cubo podría simular un modelo de anillo para la computación paralela (véase el ejemplo 6.3.5). Ahora veremos cuándo el n-cubo puede simular el **modelo de red para la computación paralela**.

El modelo de red para la computación paralela en dos dimensiones descrito mediante una gráfica consta de un arreglo rectangular de vértices unidos como se muestra (véase la figura 6.6.2). El problema "¿Cuándo puede un n-cubo simular una red bidimensional?" se puede expresar en terminología de la teoría de gráficas como "¿Cuándo contiene un n-cubo una subgráfica isomorfa a una red bidimensional?" Mostraremos que si M es una red de p por q vértices, donde $p \leq 2^i$ y $q \leq 2^j$, entonces el $(i + j)$-cubo contiene una subgráfica isomorfa a M. (En la figura 6.6.2, consideramos $p = 6$, $q = 4$, $i = 3$ y $j = 2$. Así, nuestro resultado muestra que el 5-cubo contiene una subgráfica isomorfa a la gráfica de la figura 6.6.2.)

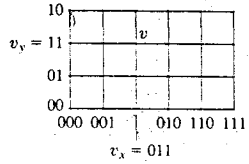


FIGURA 6.6.2
Modelo de red para la computación paralela.

Sea M una red con p vértices por q vértices, donde $p \leq 2^i$ y $q \leq 2^j$. Consideramos a M como un arreglo rectangular en el espacio ordinario de dos dimensiones con p vértices en la dirección horizontal y q vértices en la dirección vertical (véase la figura 6.6.2). Utilizamos los elementos de los códigos Gray como las coordenadas de los vértices. (Los códigos Gray se analizaron en el ejemplo 6.3.5.) Las coordenadas en la dirección horizontal son los primeros p miembros de un código Gray de i bits y las coordenadas en la dirección vertical son los primeros q miembros de un código Gray de j bits (véase el ejemplo 6.6.2). Si un vértice v está en la red, sean v_x la coordenada horizontal y v_y la coordenada vertical de v. Ahora definimos una función f sobre los vértices de M como

$$f(v) = v_x v_y.$$

(La cadena $v_x v_y$ es la cadena v_x seguida de la cadena v_y.) Observe que f es uno a uno.

Si (v, w) es una arista en M, las cadenas de bits $v_x v_y$ y $w_x w_y$ difieren exactamente en un bit. Así, $(v_x v_y, w_x w_y)$ es una arista en el $(i + j)$-cubo. Definimos una función g sobre las aristas de M como

$$g((v, w)) = (v_x v_y, w_x w_y).$$

Observe que g es uno a uno. El par de funciones f, g es un isomorfismo de M sobre la subgráfica (V, E) del $(i+j)$-cubo, donde

$$V = \{ f(v) \mid v \text{ es un vértice en } M \}, \quad E = \{ g(e) \mid e \text{ es una arista en } M \}.$$

Por tanto, si M es una red de p por q vértices, donde $p \leq 2^i$ y $q \leq 2^j$, el $(i + j)$-cubo contiene una subgráfica isomorfa a M.

El argumento dado se puede ampliar a un número arbitrario de dimensiones (véase el ejercicio 11); es decir, si M es una red $p_1 \times p_2 \times \cdots$ por p_k, donde $p_i \leq 2^{t_i}$ para $i = 1, \ldots, k$, entonces el $(t_1 + t_2 + \cdots + t_k)$-cubo contiene una subgráfica isomorfa a M. □

Suponga que las gráficas G_1 y G_2 son isomorfas. Si f es la función de la definición 6.6.1, la definición implica que los vértices v y w en G_1 son adyacentes si y sólo si los vértices $f(v)$ y $f(w)$ son adyacentes en G_2. El recíproco es verdadero para gráficas *simples*.

TEOREMA 6.6.4

Sean G_1 y G_2 gráficas simples. Las siguientes afirmaciones son equivalentes.

(a) G_1 y G_2 son isomorfas.

(b) Existe una función uno a uno y sobre, f, del conjunto de vértices de G_1 al conjunto de los vértices de G_2 que satisface: Los vértices v y w son adyacentes en G_1 si y sólo si los vértices $f(v)$ y $f(w)$ son adyacentes en G_2.

Demostración. Por la definición 6.6.1, tenemos de manera directa que (a) implica (b).

Demostremos que (b) implica (a). Suponga que existe una función uno a uno y sobre, f, del conjunto de vértices de G_1 al conjunto de vértices de G_2 que satisface la siguiente condición: Los vértices v y w son adyacentes en G_1 si y sólo si los vértices $f(v)$ y $f(w)$ son adyacentes en G_2.

Definimos una función, g, de las aristas de G_1 a las aristas de G_2 mediante la regla

$$g((v, w)) = (f(v), f(w)).$$

Como G_1 y G_2 son gráficas simples, ninguna de ellas tiene aristas paralelas, de modo que la notación (v', w') designa sin ambigüedades una arista. Observe que el rango de g es un subconjunto de las aristas de G_2, pues si (v, w) es una arista en G_1, v y w son adyacentes, lo cual implica que $f(v)$ y $f(w)$ son adyacentes; es decir, que $(f(v), f(w))$ es una arista en G_2.

Dejaremos al lector los detalles de la verificación de que g es uno a uno y sobre y que una arista e es incidente en v y w en G_1 si y sólo si la arista $g(e)$ es incidente en $f(v)$ y $f(w)$ en G_2. Esto implica que G_1 y G_2 son isomorfas. ■

El teorema 6.6.4 implica de inmediato que las gráficas simples son isomorfas si y sólo si para cierto orden de sus vértices, sus matrices de adyacencia son idénticas.

TEOREMA 6.6.5

Las gráficas simples G_1 y G_2 son isomorfas si y sólo si para cierto orden de sus vértices, las matrices de adyacencia son iguales.

Demostración. Véase el ejercicio 25. ■

EJEMPLO 6.6.6

La matriz de adyacencia de la gráfica G_1 en la figura 6.6.1 con respecto del orden de los vértices a, b, c, d, e,

$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array} \begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix} \end{array},$$

es igual a la matriz de adyacencia de la gráfica G_2 en la figura 6.6.1 con respecto del orden de los vértices A, B, C, D, E,

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix} \end{array}.$$

De nuevo vemos que G_1 y G_2 son isomorfas. □

Un problema interesante consiste en determinar si dos gráficas son isomorfas o no. Aunque todos los algoritmos conocidos para verificar si dos gráficas son isomorfas necesita un tiempo exponencial o factorial en el peor de los casos, existen algoritmos que permiten determinar cuándo un par de gráficas son isomorfas en un tiempo lineal en el caso promedio (véase [Read] y [Babai]).

La siguiente es una forma de demostrar que dos gráficas simples G_1 y G_2 *no* son isomorfas. Se determina una propiedad de G_1 que G_2 *no* tenga, pero que G_2 *tendría* si G_1 y G_2 fuesen isomorfas. Tal propiedad es un **invariante**. Más precisamente, una propiedad P es un invariante, si siempre que G_1 y G_2 sean gráficas isomorfas:

Si G_1 tiene la propiedad P, entonces G_2 también tiene la propiedad P.

Según la definición 6.6.1, si las gráficas G_1 y G_2 son isomorfas, entonces existen funciones uno a uno y sobre de las aristas (respectivamente, vértices) de G_1 en las aristas (respectivamente, vértices) de G_2. Así, si G_1 y G_2 son isomorfas, entonces G_1 y G_2 tienen el mismo número de aristas y el mismo número de vértices. Por tanto, si e y n son enteros no negativos, las propiedades "tiene e aristas" y "tiene n vértices" son invariantes.

EJEMPLO 6.4.4

Determine una ruta más corta de a a z y su longitud para la gráfica de la figura 6.4.7.

Aplicaremos el algoritmo 6.4.1 con una ligera modificación. Además de encerrar un vértice en un círculo, también lo etiquetaremos con el nombre del vértice a partir del cual fue etiquetado.

La figura 6.4.7 muestra el resultado de ejecutar las líneas 2-4 del algoritmo 6.4.1. Primero encerramos a en un círculo (véase la figura 6.4.8). A continuación, etiquetamos los vértices b y d adyacentes a a. El vértice b se etiqueta "a, 2" para indicar su valor y el hecho de que fue etiquetado a partir de a. De manera análoga, el vértice d se etiqueta "a, 1".

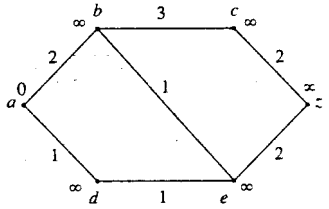


FIGURA 6.4.7 Inicialización en el algoritmo de la ruta más corta de Dijkstra.

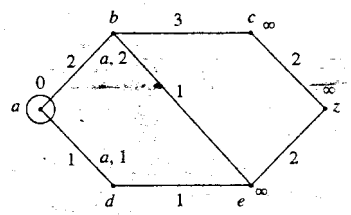


FIGURA 6.4.8 La primera iteración del algoritmo de la ruta más corta de Dijkstra.

A continuación, encerramos en un círculo al vértice d y actualizamos la etiqueta del vértice e adyacente a d (véase la figura 6.4.9). A continuación, encerramos en un círculo al vértice b y actualizamos las etiquetas de los vértice c y e (véase la figura 6.4.10). A continuación, encerramos en un círculo al vértice e y actualizamos la etiqueta del vértice z (véase la figura 6.4.11). En este momento, podemos encerrar a z dentro de un círculo, con lo cual concluye el algoritmo. La longitud de una ruta más corta de a a z es 4. A partir de z, podemos seguir las etiquetas para determinar la ruta más corta

$$(a, d, e, z).$$ □

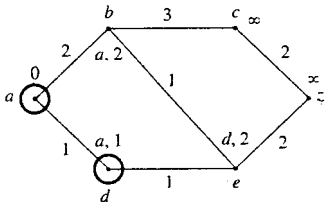


FIGURA 6.4.9 La segunda iteración del algoritmo de la ruta más corta de Dijkstra.

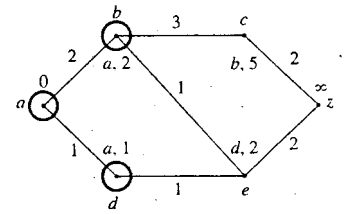


FIGURA 6.4.10 La tercera iteración del algoritmo de la ruta más corta de Dijkstra.

Nuestro siguiente teorema muestra que el algoritmo de Dijkstra es $\Theta(n^2)$ en el peor de los casos.

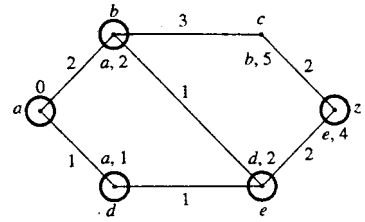


FIGURA 6.4.11 La conclusión del algoritmo de la ruta más corta de Dijkstra.

TEOREMA 6.4.5

Para una entrada consistente en una gráfica simple, conexa, con pesos y n vértices, el algoritmo de Dijkstra (algoritmo 6.4.1) tiene un tiempo de ejecución $\Theta(n^2)$ en el peor de los casos.

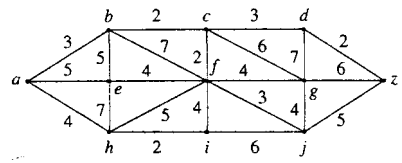
Demostración. Consideremos el tiempo ocupado en los ciclos, el cual proporciona una cota superior para el tiempo total. La línea 4 se ejecuta $O(n)$ veces. Dentro del ciclo while, la línea 10 tarda un tiempo $O(n)$ [podríamos determinar el mínimo $L(v)$ examinando todos los vértices en T]. El cuerpo del ciclo for (línea 13) tarda un tiempo $O(n)$. Como las líneas 10 y 13 están anidadas dentro de un ciclo while, el cual tarda un tiempo $O(n)$, el tiempo total para las líneas 10 y 13 es $O(n^2)$. Así, el algoritmo de Dijkstra se ejecuta en un tiempo $O(n^2)$.

De hecho, para una elección adecuada de z, el tiempo es $\Omega(n^2)$ para K_n, la gráfica completa con n vértices, debido a que cada vértice es adyacente a todos los demás. Así, el tiempo de ejecución en el peor de los casos es $\Theta(n^2)$. ■

Cualquier algoritmo para determinar una ruta más corta y que reciba como entrada K_n, la gráfica completa con n vértices, debe examinar todas las aristas de K_n al menos una vez. Como K_n tiene $n(n-1)/2$ aristas (véase el ejercicio 11, sección 6), su tiempo de ejecución en el peor de los casos debe ser al menos $n(n-1)/2 = \Omega(n^2)$. El teorema 6.4.5 implica entonces que el algoritmo 6.4.1 es óptimo.

Ejercicios

En los ejercicios 1-5, determine la longitud de una ruta más corta y de una ruta más corta entre cualesquiera par de vértices en la siguiente gráfica con pesos.



1. a, f 2. a, g 3. a, z **4.** b, j 5. h, d

6. Escriba un algoritmo que determine la longitud de una ruta más corta entre dos vértices dados en una gráfica conexa con pesos, y que también determine una ruta más corta.

7. Escriba un algoritmo que determine las longitudes de las rutas más cortas desde un vértice dado hasta cualquier otro vértice en una gráfica simple, conexa, con pesos, G.

☆ 8. Escriba un algoritmo que determine las longitudes de las rutas más cortas entre todos los pares de vértices en una gráfica simple, conexa, con pesos, con n vértices en un tiempo $O(n^3)$.

9. Modifique el algoritmo 6.4.1 de modo que acepte una gráfica con pesos que no necesariamente sea conexa. Al concluir, ¿cuál es el valor de $L(z)$ si no existe un camino de a a z?

10. ¿Verdadero o falso? Al introducir como dato una gráfica conexa con pesos y los vértices a y z en el siguiente algoritmo, éste regresa la longitud de una ruta más corta de a a z. Si el algoritmo es correcto, demuéstrelo; en caso contrario, proporcione un ejemplo de una gráfica conexa con pesos y vértices a y z para los cuales falle.

ALGORITMO 6.4.6

```
procedure algor(w, a, z)
  length := 0
  v := a
  T := conjunto de todos los vértices
  while v ≠ z do
    begin
    T := T − {v}
    elegir x ∈ T con w(v, x) mínimo
    length := length + w(v, x)
    v := x
    end
  return (length)
end algor
```

11. ¿Verdadero o falso? El algoritmo 6.4.1 determina la longitud de una ruta más corta en una gráfica conexa con pesos, incluso si algunos pesos son negativos. De ser cierto, demuéstrelo; en caso contrario, proporcione un contraejemplo.

6.5 *REPRESENTACIONES DE GRÁFICAS*

En las secciones anteriores representamos una gráfica mediante un diagrama. En ciertas ocasiones, como por ejemplo, al utilizar una computadora para analizar una gráfica, es necesaria una representación más formal. Nuestro primer método de representación de una gráfica utiliza la **matriz de adyacencia**.

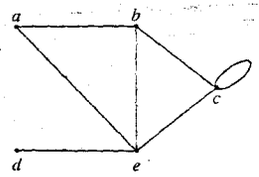


FIGURA 6.5.1
La gráfica del ejemplo 6.5.1.

EJEMPLO 6.5.1 *Matriz de adyacencia*

Consideremos la gráfica de la figura 6.5.1. Para obtener la matriz de adyacencia de esta gráfica, primero elegimos un orden para los vértices, digamos, a, b, c, d, e. A continuación etiquetamos los renglones y las columnas de una matriz con los vértices ordenados. La entrada en esta matriz es 1 si los vértices del renglón y la columna son adyacentes y 0 en caso contrario. La matriz de adyacencia de esta gráfica es

$$\begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array}\!\!\begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix} \end{array}.$$

□

Observe que podemos obtener el grado de un vértice v en una gráfica simple G sumando el renglón v o la columna v en la matriz de adyacencia de G. Además, observe que aunque la matriz de adyacencia nos permite representar lazos, no nos permite representar aristas paralelas; sin embargo, si modificamos la definición de matriz de adyacencia para que ésta pueda contener enteros no negativos arbitrarios, podemos representar las aristas paralelas. En la matriz de adyacencia modificada, interpretamos la entrada ij-ésima especificando el número de aristas entre i y j.

La matriz de adyacencia no es una manera muy eficiente para representar una gráfica. Como la matriz es simétrica con respecto de la diagonal principal (los elementos sobre la línea que va de la esquina superior izquierda a la inferior derecha), la información aparece dos veces (excepto aquella que se encuentra en la diagonal principal).

EJEMPLO 6.5.2

La matriz de adyacencia de la gráfica simple de la figura 6.5.2 es

$$A = \begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array}\!\!\begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} \end{array}.$$

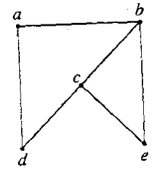


FIGURA 6.5.2
La gráfica del ejemplo 6.5.2.

□

Mostraremos que si A es la matriz de adyacencia de una gráfica simple G, las potencias de A,

$$A, A^2, A^3, \ldots,$$

cuentan el número de caminos de diversas longitudes. Más precisamente, si los vértices de G se etiquetan $1, 2, \ldots,$ la entrada ij-ésima en la matriz A^n es igual al número de caminos de i a j de longitud n. Por ejemplo, supongamos que obtenemos el cuadrado de la matriz A del ejemplo 6.5.2:

$$A^2 = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} = \begin{array}{c} \\ a \\ b \\ c \\ d \\ e \end{array}\!\!\begin{array}{c} \begin{array}{ccccc} a & b & c & d & e \end{array} \\ \begin{pmatrix} 2 & 0 & 2 & 0 & 1 \\ 0 & 3 & 1 & 2 & 1 \\ 2 & 1 & 3 & 0 & 1 \\ 0 & 2 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{pmatrix} \end{array}.$$

Consideremos la entrada del renglón a, columna c en A^2, obtenida al multiplicar por pares las entradas del renglón a por las entradas de la columna c de la matriz A y sumando:

$$\begin{matrix} & & & & & c & \\ a & \begin{pmatrix} b & & d & & \end{pmatrix} & \begin{matrix} \\ \\ \\ \\ \end{matrix} \\ & \begin{pmatrix} 1 & 0 & 1 & 0 \end{pmatrix} & \begin{matrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} \end{matrix} \begin{matrix} b \\ \\ \\ \\ d \end{matrix} \end{matrix} = 0\cdot 0 + 1\cdot 1 + 0\cdot 0 + 1\cdot 1 + 0\cdot 1 = 2.$$

La única forma en que un producto distinto de cero podría aparecer en esta suma es cuando ambas entradas por multiplicar son iguales a 1. Esto ocurre si existe un vértice v cuya entrada en el renglón a es 1 y cuya entrada en la columna c es 1. En otras palabras, debe haber aristas de la forma (a, v) y (v, c). Tales aristas forman un camino (a, v, c) de longitud 2 de a a c y cada camino incrementa la cantidad en 1. En este ejemplo, la suma es 2 pues existen dos caminos

$$(a, b, c), \quad (a, d, c)$$

de longitud 2 de a a c. En general, la entrada en el renglón x y la columna y de la matriz A^2 es el número de caminos de longitud 2 del vértice x al vértice y.

Las entradas de la diagonal principal de A^2 proporcionan los grados de los vértices (cuando la gráfica es una gráfica simple). Consideremos, por ejemplo, el vértice c. El grado de c es 3, pues c es incidente en las tres aristas (c, b), (c, d) y (c, e). Pero cada una de estas aristas se puede convertir en un camino de longitud 2 de c a c:

$$(c, b, c), \quad (c, d, c), \quad (c, e, c).$$

De manera análoga, un camino de longitud 2 de c a c define una arista incidente en c. Así, el número de caminos de longitud 2 de c a c es 3, el grado de c.

Ahora utilizaremos la inducción para mostrar que las entradas en la n-ésima potencia de una matriz de adyacencia proporcionan el número de caminos de longitud n.

TEOREMA 6.5.3

Si A es la matriz de adyacencia de una gráfica simple, la entrada ij-ésima de A^n es igual al número de caminos de longitud n del vértice i al vértice j, $n = 1, 2, \ldots$.

Demostración. Utilizaremos la inducción sobre n.

En el caso $n = 1$, A^1 es simplemente A. La entrada ij-ésima es 1 si existe una arista de i a j, que es un camino de longitud 1, y 0 en caso contrario. Así, el teorema es verdadero en el caso $n = 1$. Hemos verificado el paso base.

Supongamos que el teorema es verdadero para n. Ahora,

$$A^{n+1} = A^n A$$

de modo que la entrada ik-ésima de A^{n+1} se obtiene al multiplicar por pares los elementos del renglón i-ésimo de A^n por los elementos de la columna k-ésima de A y sumando:

$$\begin{matrix} & & & & k\text{-ésima columna de } A & \\ i\text{-ésimo renglón de } A^n(s_1, s_2, \ldots, s_j, \ldots, s_m) & & \begin{matrix} \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_j \\ \vdots \\ t_m \end{pmatrix} \end{matrix} \end{matrix}$$

$$= s_1t_1 + s_2t_2 + \cdots + s_jt_j + \cdots + s_mt_m$$

$$= \text{entrada } ik\text{-ésima de } A^{n+1}.$$

Por inducción, s_j proporciona el número de caminos de longitud n de i a j en la gráfica G. Ahora, t_j es 0 o 1. Si t_j es 0, no existe una arista de j a k, de modo que existen $s_jt_j = 0$ caminos de longitud $n + 1$ de i a k, donde la última arista es (j, k). Si t_j es 1, existe una arista del vértice j al vértice k (véase la figura 6.5.3). Como existen s_j caminos de longitud n del vértice i al vértice j, existen $s_jt_j = s_j$ caminos de longitud $n + 1$ de i a k, donde la última arista es (j, k) (véase la figura 6.5.3). Al sumar sobre todas las j, contamos todos los caminos de longitud $n + 1$ de i a k. Así, la entrada ik-ésima en A^{n+1} proporciona el número de caminos de longitud $n + 1$ de i a k y se verifica el paso inductivo.

El teorema queda establecido por el principio de inducción matemática. ■

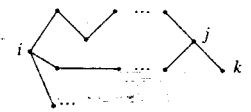


FIGURA 6.5.3
La demostración del teorema 6.5.3. Un camino de i a k de longitud $n + 1$ cuyo penúltimo vértice es j consta de un camino de longitud n de i a j seguido de la arista (j, k). Si existen s_j caminos de longitud n de i a j y t_j es igual a 1 si la arista (j, k) existe y 0 en caso contrario, entonces la suma de s_jt_j sobre todas las j proporciona el número de caminos de longitud $n + 1$ de i a k.

EJEMPLO 6.5.4

Después del ejemplo 6.5.2, mostramos que si A es la matriz de la gráfica de la figura 6.5.2,

$$A^2 = \begin{matrix} \\ a \\ b \\ c \\ d \\ e \end{matrix} \begin{matrix} \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{pmatrix} 2 & 0 & 2 & 0 & 1 \\ 0 & 3 & 1 & 2 & 1 \\ 2 & 1 & 3 & 0 & 1 \\ 0 & 2 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{pmatrix} \end{matrix}.$$

Al multiplicar,

$$A^4 = A^2A^2 = \begin{pmatrix} 2 & 0 & 2 & 0 & 1 \\ 0 & 3 & 1 & 2 & 1 \\ 2 & 1 & 3 & 0 & 1 \\ 0 & 2 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 0 & 2 & 0 & 1 \\ 0 & 3 & 1 & 2 & 1 \\ 2 & 1 & 3 & 0 & 1 \\ 0 & 2 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{pmatrix},$$

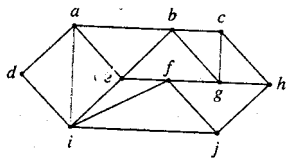
mos que v_1 corresponde al cuadro superior izquierdo. Llamamos a los ocho cuadros de la parte superior y la parte inferior los *cuadros exteriores*, y a los otros ocho les llamaremos *cuadros interiores*. Observe que el caballo debe llegar a un cuadro exterior desde un cuadro interior y que el caballo debe moverse desde un cuadro exterior hacia un cuadro interior. Así, en el ciclo C, cada vértice correspondiente a un cuadro exterior debe ser precedido y seguido por un vértice correspondiente a un cuadro interior. Como existen iguales cantidades de cuadros exteriores e interiores, los vértices v_i, donde i es impar corresponden a cuadros exteriores, y los vértices v_i, donde i es par, corresponden a cuadros interiores. Pero al observar los movimientos del caballo, vemos que los vértices v_i, donde i es impar corresponden a cuadros blancos, y los vértices v_i, donde i es par, corresponden a cuadros negros. Por tanto, los únicos cuadros exteriores visitados son blancos y los únicos cuadros interiores visitados son negros. Así, C no es un ciclo hamiltoniano. Esta contradicción completa la demostración de que GK_4 no tiene un ciclo hamiltoniano. Este argumento fue utilizado por Louis Pósa cuando era adolescente.

La gráfica GK_6 tiene un ciclo hamiltoniano. Este hecho se puede demostrar exhibiendo uno (véase el ejercicio 21). Se puede mostrar mediante métodos elementales que GK_n tiene un ciclo hamiltoniano para todo $n \geq 6$ par (véase [Schwenk]). La demostración construye ciclos hamiltonianos de manera explícita para ciertos tableros menores y luego pega los tableros menores entre ellos para obtener ciclos hamiltonianos para los tableros mayores. □

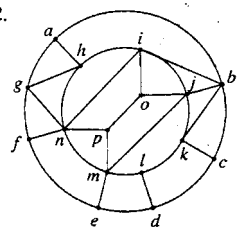
Ejercicios

Para cada gráfica determine un ciclo hamiltoniano.

1.

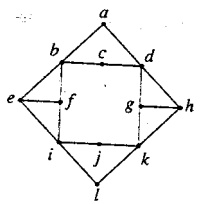


2.

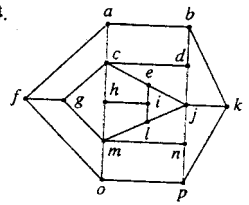


Muestre que ninguna de las siguientes gráficas contiene un ciclo hamiltoniano.

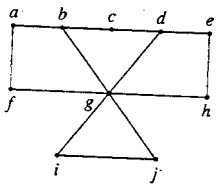
3.



4.

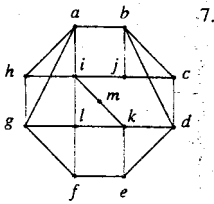


5.

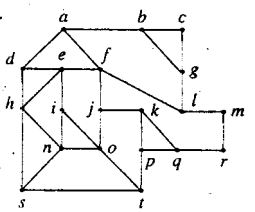


Determine si cada gráfica contiene o no un ciclo hamiltoniano. Si existe un ciclo hamiltoniano, exhíbalo; en caso contrario, proporcione un argumento que muestre que no existe tal ciclo hamiltoniano.

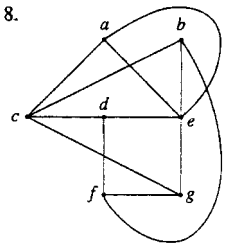
6.



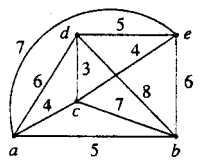
7.



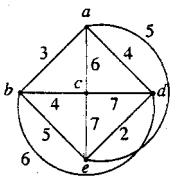
8.



9. Proporcione un ejemplo de una gráfica que tenga un ciclo de Euler pero no un ciclo hamiltoniano.
10. Proporcione un ejemplo de una gráfica que tenga un ciclo de Euler que también sea un ciclo hamiltoniano.
11. Proporcione un ejemplo de una gráfica que tenga un ciclo de Euler y un ciclo hamiltoniano pero que no sean idénticos.
★ 12. ¿Para cuáles valores de m y n tiene la gráfica del ejercicio 37, sección 6.2, un ciclo hamiltoniano?
13. Modifique la gráfica del ejercicio 37, sección 6.2, insertando una arista entre el vértice del renglón i, columna 1, y el vértice en el renglón i, columna m, para $i = 1, \ldots, n$. Muestre que la gráfica resultante siempre tiene un ciclo hamiltoniano.
14. Muestre que si $n \geq 3$, la gráfica completa de n vértices K_n contiene un ciclo hamiltoniano.
15. ¿Cuándo contiene un ciclo hamiltoniano la gráfica bipartita completa $K_{m,n}$?
16. Muestre que el ciclo (e, b, a, c, d, e) proporciona una solución al problema del agente viajero para la siguiente gráfica.



17. Resuelva el problema del agente viajero para la siguiente gráfica.



★ 18. Sean m y n enteros que satisfacen $1 \leq m \leq 2^n$. Demuestre que el n-cubo tiene un ciclo simple de longitud m si y sólo si $m \geq 4$ y m es par.

19. Utilice el teorema 6.3.6 para calcular el código Gray G_4.

20. Sea G una gráfica bipartita con conjuntos ajenos de vértices V_1 y V_2, como en la definición 6.1.10. Muestre que si G tiene un ciclo hamiltoniano, entonces V_1 y V_2 tienen el mismo número de elementos.

21. Determine un ciclo hamiltoniano en GK_6 (véase el ejemplo 6.3.9).

22. Describa un modelo de gráfica adecuado para resolver el siguiente problema: ¿Podrían ordenarse las permutaciones de $\{1, 2, \ldots, n\}$ de modo que las permutaciones adyacentes

$$p: \quad p_1, \quad \ldots, \quad p_n \quad \text{y} \quad q: \quad q_1, \quad \ldots, \quad q_n$$

satisfagan $p_i \neq q_i$ para $i = 1, \ldots, n$?

23. Resuelva el problema del ejercicio 22 para $n = 1, 2, 3, 4$. (La respuesta a la pregunta es "sí" para $n \geq 5$; véase el [problema 1186] en la bibliografía.)

6.4 UN ALGORITMO PARA LA RUTA MÁS CORTA

Recuerde (véase la sección 6.1) que una gráfica con pesos es una gráfica en la cual se asignan valores a las aristas y que la longitud de una ruta (camino) en una gráfica con pesos es la suma de los pesos de las aristas en la ruta. Sea $w(i, j)$ el peso de la arista (i, j). En las gráficas con pesos, con frecuencia queremos determinar la **ruta más corta** (es decir, un camino de longitud mínima) entre dos vértices dados. El algoritmo 6.4.1, debido a E. W. Dijkstra, que resuelve de manera eficiente este problema, es el tema de esta sección.

Edsger W. Dijkstra (1930–) nació en los Países Bajos. Él fue una de las primeras personas que propuso la programación como una ciencia. Estaba tan dedicado a la programación que al casarse, en 1957, indicó que su profesión era programador. Sin embargo, las autoridades holandesas dijeron que no existía tal profesión, de modo que tuvo que modificar el dato por "físico teórico". Ganó el prestigiado Premio Turing de la Association for Computing Machinery en 1972 y obtuvo la Schlumberger Centennial Chair en Computación en la Universidad de Texas en Austin, en 1984.

En esta sección, G denota una gráfica conexa, con pesos. Suponemos que los pesos son números positivos y que queremos determinar un camino más corto del vértice a al vértice z. Podemos eliminar la hipótesis de que G sea conexa (véase el ejercicio 9).

En el algoritmo de Dijkstra se asignan etiquetas a los vértices. Sea $L(v)$ la etiqueta del vértice v. En cualquier momento, algunos vértices tienen etiquetas temporales y el resto poseen permanentes. Sea T el conjunto de vértices que tienen etiquetas temporales. Al ilustrar el algoritmo, encerraremos en un círculo los vértices que tienen etiquetas permanentes. Mostraremos posteriormente que si $L(v)$ es la etiqueta permanente del vértice v, entonces $L(v)$ es la longitud de una ruta más corta de a a v. En principio, todos los vértices tienen etiquetas temporales. Cada iteración del algoritmo modifica el estado de una etiqueta, de temporal a permanente; así, podemos concluir el algoritmo cuando z recibe una etiqueta temporal. En este momento, $L(z)$ proporciona la longitud de una ruta más corta de a a z.

ALGORITMO 6.4.1 *Algoritmo de la ruta más corta de Dijkstra*

Este algoritmo determina la longitud de una ruta más corta del vértice a al vértice z en una gráfica conexa con pesos. El peso de la arista (i, j) es $w(i, j) > 0$ y la etiqueta del vértice x es $L(x)$. Al concluir, $L(z)$ es la longitud de una ruta más corta de a a z.

Entrada: Una gráfica conexa, con pesos, en la cual todos los pesos son positivos. Los vértices a y z.

Salida: $L(z)$, la longitud de una ruta más corta de a a z.

1. **procedure** *dijkstra* (w, a, z, L)
2. $L(a) := 0$
3. **for** todos los vértices $x \neq a$ **do**
4. $\quad L(x) := \infty$
5. $T :=$ conjunto de todos los vértices
6. // T es el conjunto de vértices cuya distancia más corta a a
7. // no ha sido determinada
8. **while** $z \in T$ **do**
9. $\quad$ **begin**
10. $\quad$ elegir $v \in T$ con $L(v)$ mínimo
11. $\quad T := T - \{v\}$
12. $\quad$ **for** cada $x \in T$ adyacente a v **do**
13. $\quad\quad L(x) := \min\{L(x), L(v) + w(v, x)\}$
14. $\quad$ **end**
15. **end** *dijkstra*

EJEMPLO 6.4.2

Ahora mostraremos la forma en que el algoritmo 6.4.1 determina una ruta más corta de a a z en la gráfica de la figura 6.4.1. (Los vértices en T no están encerrados en un círculo y tienen etiquetas temporales. Los vértices encerrados en un círculo tienen etiquetas permanentes.) La figura 6.4.2 muestra el resultado de ejecutar las líneas 2-5. En la línea 8, z no está encerrado en un círculo. Continuamos con la línea 10, donde elegimos el vértice a (el vértice no encerrado en un círculo y que tiene la etiqueta menor) y lo encerramos en un círcu-

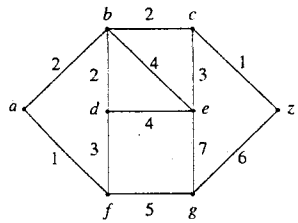


FIGURA 6.4.1 La gráfica para el ejemplo 6.4.2.

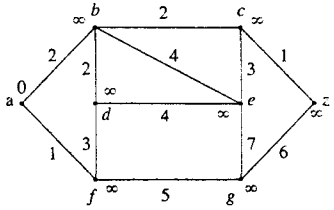


FIGURA 6.4.2 Inicialización en el algoritmo de la ruta más corta de Dijkstra.

lo (véase la figura 6.4.3). En las líneas 12 y 13 actualizamos cada uno de los vértices no encerrados en un círculo, b y f, adyacentes a a. Obtenemos las nuevas etiquetas

$$L(b) = \text{mín}\{\infty, 0 + 2\} = 2, \quad L(f) = \text{mín}\{\infty, 0 + 1\} = 1$$

(véase la figura 6.4.3). En este momento, regresamos a la línea 8.

Como z no está encerrado en un círculo, pasamos a la línea 10, donde elegimos el vértice f (el vértice no encerrado en un círculo y que tiene la menor etiqueta) y lo encerramos en un círculo (véase la figura 6.4.4). En las líneas 12 y 13 actualizamos las etiquetas de los vértices no encerrados en un círculo, d y g, adyacentes a f. Obtenemos las etiquetas que aparecen en la figura 6.4.4.

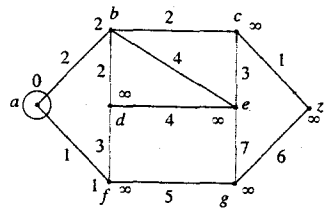


FIGURA 6.4.3 La primera iteración del algoritmo de la ruta más corta de Dijkstra.

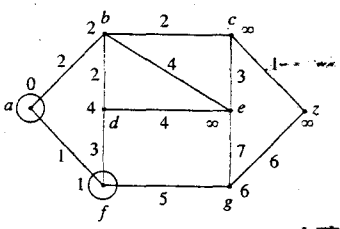


FIGURA 6.4.4 La segunda iteración del algoritmo de la ruta más corta de Dijkstra.

El lector debe verificar que la siguiente iteración del algoritmo produce las etiquetas que aparecen en la figura 6.4.5 y que al concluir el algoritmo, z tiene la etiqueta 5, lo cual indica que la longitud de una ruta más corta de a a z es 5. Una ruta más corta es (a, b, c, z).

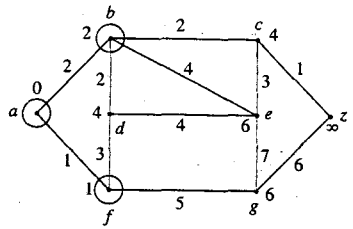


FIGURA 6.4.5 La tercera iteración del algoritmo de la ruta más corta de Dijkstra. □

Ahora mostraremos que el algoritmo 6.4.1 es correcto. La demostración se basa en el hecho de que el algoritmo de Dijkstra determina las longitudes de las rutas más cortas a partir de a en orden creciente.

TEOREMA 6.4.3

El algoritmo de la ruta más corta de Dijkstra (algoritmo 6.4.1) determina de manera correcta la longitud de una ruta más corta de a a z.

Demostración. Utilizamos la inducción matemática sobre i para demostrar que la i-ésima vez que llegamos a la línea 10, $L(v)$ es la longitud de una ruta más corta de a a v. Al demostrar esto, tendremos que el algoritmo es correcto, pues al elegir z en la línea 10, $L(z)$ dará la longitud de una ruta más corta de a a z.

PASO BASE ($i = 1$). La primera vez que llegamos a la línea 10, debido a los pasos de inicialización (líneas 2-4), $L(a)$ es igual a cero y todos los demás valores de L son ∞. Así, a se elige la primera vez que llegamos a la línea 10. Como $L(a)$ es igual a cero, $L(a)$ es la longitud de una ruta más corta de a hasta a.

PASO INDUCTIVO. Supongamos que para toda $k < i$, la k-ésima vez que llegamos a la línea 10, $L(v)$ es la longitud de una ruta más corta de a a v.

Suponga que estamos en la línea 10 por la i-ésima vez y que elegimos v en T con el valor mínimo $L(v)$.

Primero mostraremos que si existe un camino de a a un vértice w, cuya longitud sea menor que $L(v)$, entonces w no está en T (es decir, w fue elegido antes en la línea 10). Supongamos por contradicción que w está en T. Sean P una ruta más corta de a a w, x el vértice más cercano a a sobre P y que pertenezca a T, y u el predecesor de x en P (véase la figura 6.4.6). Entonces u no está en T, por lo que u fue elegido en la línea 10 durante una iteración anterior del ciclo while. Por la hipótesis de inducción, $L(u)$ es la longitud de una ruta más corta de a a u. Ahora,

$$L(x) \leq L(u) + w(u, x) \leq \text{longitud de } P < L(v).$$

Pero esta desigualdad muestra que v no es el vértice en T con $L(v)$ mínimo [$L(x)$ es menor]. Esta contradicción concluye la demostración de que si existe una ruta de a a un vértice w cuya longitud sea menor que $L(v)$, entonces w no está en T.

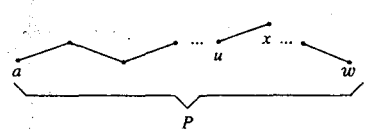


FIGURA 6.4.6 La demostración del teorema 6.4.3. P es una ruta más corta de a a w, x es el vértice más cercano a a sobre P y que pertenece a T, y u es el predecesor de x sobre P.

El resultado anterior muestra, en particular, que si existiera un camino de a a v con longitud menor que $L(v)$, v ya hubiera sido elegido en la línea 10 y eliminado de T. Por tanto, cada camino de a a v tiene longitud al menos $L(v)$. Por construcción, existe un camino de a a v de longitud $L(v)$, de modo que éste es una ruta más corta de a a v. Esto concluye la demostración. ■

El algoritmo 6.4.1 determina la longitud de una ruta más corta de a a z. En la mayor parte de las aplicaciones, también quisiéramos identificar una ruta más corta. Una ligera modificación del algoritmo 6.4.1 nos permite determinar una ruta más corta.

RINCÓN DE SOLUCIÓN DE PROBLEMAS:
GRÁFICAS

Problema

¿Es posible que en un departamento de 25 personas, clasificadas según su desacuerdo, cada persona congenie con exactamente otras cinco?

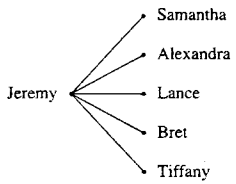
Para enfrentar el problema

¿Dónde comenzar? Como este problema está en el capítulo 6, que trata de las gráficas, es probable que una buena idea sea intentar modelar el problema como una gráfica. Si este problema no estuviese asociado con una sección o capítulo del libro, podríamos intentar varios enfoques, uno de los cuales podría ser modelar el problema como una gráfica. Muchos problemas discretos se pueden resolver modelándolos mediante una gráfica. Esto no quiere decir que éste sea el único enfoque posible. Casi siempre, al adoptar diversos enfoques, un único problema se puede resolver de varias formas. (Un buen ejemplo es [Wagon].)

Determinación de una solución

Un elemento fundamental al construir un modelo de gráfica es imaginar lo que ésta debe ser. ¿Cuáles son los vértices y cuáles las aristas? En este problema, no tenemos muchas opciones; tenemos personas y desacuerdos. Intentemos utilizar a las personas como vértices. En un modelo de gráfica, es común que las aristas indiquen una *relación* entre los vértices. En este caso, la relación es "congeniar con", de modo que escribiremos una arista entre dos vértices (personas) si ellas congenian.

Ahora supongamos que cada persona congenia con exactamente otras cinco. Por ejemplo, en la figura adyacente, que muestra parte de la gráfica, Jeremy congenia con Samantha, Alexandra, Lance, Bret y Tiffany, y nadie más.



Esto implica que el grado de cada vértice es 5. Ahora, la situación se resume así: Tenemos 25 vértices y cada vértice tiene grado 5. Antes de continuar, veamos si esto es posible.

El corolario 6.2.22 dice que existe un número par de vértices de grado impar. Tenemos una contradicción, pues existe un número *impar* de vértices de grado impar. Por tanto, no es posible que en un departamento de 25 personas clasificadas según sus desacuerdos, cada persona congenie con exactamente otras cinco.

Solución formal

No. No es posible que en un departamento de 25 personas clasificadas según sus desacuerdos, cada persona congenie con exactamente otras cinco. Supongamos por contradicción que esto es posible. Consideremos una gráfica donde los vértices sean las personas y una arista conecte dos vértices (personas) si las personas congenian. Como cada vértice tiene grado impar, existe un número impar de vértices de grado impar, lo cual es una contradicción.

Resumen de técnicas para resolver problemas

- Muchos problemas discretos se pueden resolver modelándolos mediante gráficas.
- Para construir un modelo de gráfica, determine lo que representan los vértices y las aristas.
- En un modelo de gráfica, es muy común que las aristas indiquen una relación entre los vértices.

6.3 *CICLOS HAMILTONIANOS Y EL PROBLEMA DEL AGENTE DE VENTAS VIAJERO*

Sir William Rowan Hamilton lanzó al mercado a mediados del siglo XIX un juego en forma de dodecaedro (véase la figura 6.3.1). Cada esquina llevaba el nombre de una ciudad y el problema era partir de cualquier ciudad, recorrer las aristas, visitar cada ciudad exactamente una vez, y regresar a la ciudad inicial. La gráfica de las aristas del dodecaedro aparece en la figura 6.3.2. Podemos resolver el juego de Hamilton si podemos determinar un ciclo en la gráfica de la figura 6.3.2 que contenga a cada vértice exactamente una vez (excepto por el vértice inicial y final que aparece dos veces). El lector debe intentar hallar una solución antes de observar la solución dada en la figura 6.3.3.

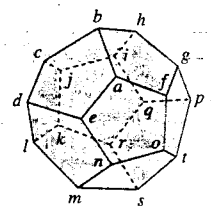


FIGURA 6.3.1
Juego de Hamilton.

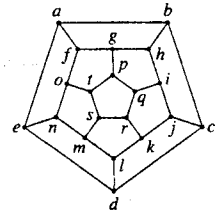


FIGURA 6.3.2
La gráfica del juego de Hamilton.

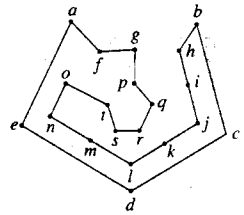


FIGURA 6.3.3
Visita de cada vértice una vez en la gráfica de la figura 6.3.2.

En honor de Hamilton, decimos que un ciclo en una gráfica G que contiene cada vértice en G exactamente una vez, excepto por el vértice inicial y final que aparece dos veces, es un **ciclo hamiltoniano**.

Hamilton (1805-1865) fue uno de los más grandes sabios de Irlanda. Fue profesor de astronomía en la Universidad de Dublín, donde publicó artículos en física y matemáticas. En esta última rama, Hamilton es más conocido por idear los cuaternios, una generalización del sistema de números complejos. Los cuaternios inspiraron el desarrollo del álgebra abstracta moderna. A este respecto, Hamilton introdujo el término *vector*.

EJEMPLO 6.3.1

El ciclo (a, b, c, d, e, f, g, a) de la gráfica de la figura 6.3.4 es un ciclo hamiltoniano. □

El problema de determinar un ciclo hamiltoniano en una gráfica parece similar al de determinar un ciclo de Euler en una gráfica. Un ciclo de Euler visita cada arista una vez, mientras que un ciclo hamiltoniano visita cada vértice una vez; sin embargo, en realidad estos problemas son un poco distintos. Por ejemplo, la gráfica de la figura 6.3.4 no tiene un ciclo de Euler, pues existen vértices de grado impar, aunque en el ejemplo 6.3.1 se vio que G tiene un ciclo hamiltoniano. Además, a diferencia de la situación para los ciclos de Euler (véanse los teoremas 6.2.17 y 6.2.18), no se conocen condiciones necesarias y suficientes fácilmente verificables para la existencia de un ciclo hamiltoniano en una gráfica.

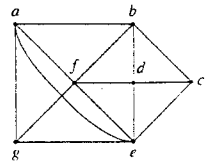


FIGURA 6.3.4
Una gráfica con un ciclo hamiltoniano.

Los siguientes ejemplos muestran que a veces podemos argumentar que una gráfica no contiene un ciclo hamiltoniano.

EJEMPLO 6.3.2

Muestre que la gráfica de la figura 6.3.5 no contiene un ciclo hamiltoniano.

Como existen cinco vértices, un ciclo hamiltoniano debe tener cinco aristas. Suponga que pudiéramos eliminar algunas aristas de la gráfica, para quedarnos sólo con un ciclo hamiltoniano. Tendríamos que eliminar una arista incidente en v_2 y una arista incidente en v_4, pues cada vértice en un ciclo hamiltoniano tiene grado 2. Pero esto sólo deja cuatro aristas, que no son suficientes para un ciclo hamiltoniano de longitud 5. Por tanto, la gráfica de la figura 6.3.5 no contiene un ciclo hamiltoniano. □

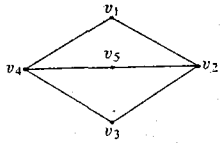


FIGURA 6.3.5
Una gráfica *sin* un ciclo hamiltoniano.

Al utilizar un argumento similar al del ejemplo 6.3.2 para mostrar que una gráfica no tiene un ciclo hamiltoniano, debemos tener cuidado de no contar una arista eliminada más de una vez. En el ejemplo 6.3.2 (el cual se refiere a la figura 6.3.5), si eliminamos una arista incidente en v_2 y una arista incidente en v_4, estas aristas son distintas. Por tanto, es correcto el razonamiento en el sentido de que debemos eliminar dos aristas de la gráfica de la figura 6.3.5 para producir un ciclo hamiltoniano.

Como ejemplo de doble conteo, consideremos el siguiente argumento *incorrecto* que supuestamente muestra que la gráfica de la figura 6.3.6 no tiene un ciclo hamiltoniano. Como existen cinco vértices, un ciclo hamiltoniano debe tener cinco aristas. Suponga que pudiésemos eliminar algunas aristas de la gráfica para obtener un ciclo hamiltoniano. Tendríamos que eliminar dos aristas incidentes en c y una arista incidente en cada uno de los vértices a, b, d y e. Esto deja sólo dos aristas, que no bastan para tener un ciclo hamiltoniano. Por tanto, la gráfica de la figura 6.3.6 no tiene un ciclo hamiltoniano. El error en este argumento es que si eliminamos dos aristas incidentes en c (como debemos hacerlo), también eliminamos aristas incidentes en dos de los vértices a, b, d o e. No debemos contar de nuevo las dos aristas eliminadas incidentes en los dos vértices. Observe que la gráfica de la figura 6.3.6 sí tiene un ciclo hamiltoniano.

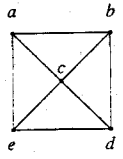


FIGURA 6.3.6
Una gráfica *con* un ciclo hamiltoniano.

EJEMPLO 6.3.3

Muestre que la gráfica G de la figura 6.3.7 no contiene un ciclo hamiltoniano.

Suponga que G tiene un ciclo hamiltoniano H. Las aristas (a, b), (a, g), (b, c) y (c, k) deben estar en H, pues cada vértice de un ciclo hamiltoniano tiene grado 2. Así, las aristas (b, d) y (b, f) no están en H. Por tanto, las aristas (g, d), (d, e), (e, f) y (f, k) están en H. Ahora, sabemos que las aristas que están en H forman un ciclo C. Al agregar una arista más a C tendríamos un vértice en H con grado mayor que 2. Esta contradicción muestra que G no tiene un ciclo hamiltoniano. □

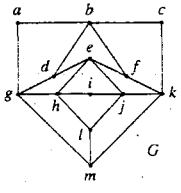


FIGURA 6.3.7
Una gráfica *sin* un ciclo hamiltoniano.

El **problema del agente viajero** se relaciona con el de determinar un ciclo hamiltoniano en una gráfica. (En la sección 6.1 nos referimos con brevedad a una variante del problema de agente viajero.) El problema es: Dada una gráfica con pesos G, determinar un ciclo hamiltoniano de longitud mínima en G. Si pensamos en los vértices de una gráfica con pesos y que los pesos en las aristas son distancias, entonces el problema del agente viajero consiste en determinar una ruta más corta mediante la cual el agente pueda visitar cada ciudad una vez, partiendo de y regresando a la misma ciudad.

EJEMPLO 6.3.4

El ciclo $C = (a, b, c, d, a)$ es un ciclo hamiltoniano para la gráfica G de la figura 6.3.8. El reemplazo de cualquiera de las aristas de C por cualquiera de las aristas con la etiqueta 11 incrementaría la longitud de C; así, C es un ciclo hamiltoniano de longitud mínima para G. Así, C resuelve el problema del agente viajero para G. □

Aunque existen algoritmos (véase, por ejemplo, [Even, 1979]) para determinar un ciclo de Euler (si éste existe) en un tiempo $\Theta(n)$ para una gráfica con n aristas, todos los algoritmos conocidos que permiten determinar un ciclo hamiltoniano requieren un tiempo exponencial o factorial en el peor de los casos. Es por esto que en los problemas donde hay que hallar una solución al del agente viajero se utilizan con frecuencia métodos que producen ciclos de longitud cercana al mínimo. La fama y la fortuna esperan al descubridor de un algoritmo de tiempo polinomial que resuelva el problema del ciclo hamiltoniano (o el problema del agente viajero) o que dé una demostración de que no existe un algoritmo de tiempo polinomial para estos problemas.

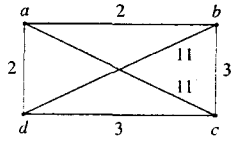


FIGURA 6.3.8
Una gráfica para el problema del agente viajero.

Concluimos esta sección buscando ciclos hamiltonianos en el n-cubo.

EJEMPLO 6.3.5 *Códigos Gray y ciclos hamiltonianos en el n-cubo*

Consideremos un **modelo de anillo** para la computación paralela; la representación de este modelo mediante una gráfica es un ciclo simple (véase la figura 6.3.9). Los vértices representan los procesadores. Una arista entre los procesadores p y q indica que p y q se pueden comunicar de manera directa entre sí. Vemos que cada procesador se puede comunicar de manera directa con exactamente dos procesadores más. Los procesadores no adyacentes se comunican enviando mensajes.

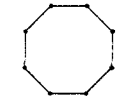


FIGURA 6.3.9
El modelo de anillo para la computación paralela.

El n-cubo (véase el ejemplo 6.1.7) es otro modelo para la computación paralela. El n-cubo tiene un mayor grado de conectividad entre sus procesadores. Consideremos la pregunta de cuándo un n-cubo puede simular un modelo de anillo con 2^n procesadores. En la terminología de las gráficas, queremos ver cuándo un n-cubo contiene un ciclo simple con 2^n vértices como una subgráfica, o bien, como el n-cubo tiene 2^n procesadores, cuándo el n-cubo contiene un ciclo hamiltoniano. [Dejaremos para los ejercicios la pregunta de cuándo un n-cubo puede simular un modelo de anillo con un número arbitrario de procesadores (véase el ejercicio 18).]

Primero observemos que si el n-cubo contiene un ciclo hamiltoniano, debemos tener que $n \geq 2$, pues el 1-cubo no tiene ciclos.

Recordemos (véase el ejemplo 6.1.7) que podemos etiquetar los vértices del n-cubo $0, 1, \ldots, 2^n - 1$ de tal modo que una arista conecte dos vértices si y sólo si la representación binaria de sus etiquetas difieren en exactamente un bit. Así, el n-cubo tiene un ciclo hamiltoniano si y sólo si $n \geq 2$ y existe una sucesión,

$$s_1, \quad s_2, \quad \ldots, \quad s_{2^n} \tag{6.3.1}$$

donde cada s_i es una cadena de n bits, de modo que:

- Cada cadena de n bits aparece en alguna parte de la sucesión.
- s_i y s_{i+1} difieren en exactamente un bit, $i = 1, \ldots, 2^n - 1$.
- s_{2^n} y s_1 difieren en exactamente un bit.

Una sucesión (6.3.1) es un **código Gray**. Cuando $n \geq 2$, un código Gray (6.3.1) corresponde al ciclo hamiltoniano

$$s_1, \quad s_2, \quad \ldots, \quad s_{2^n}, \quad s_1$$

pues todos los vértices aparecen y las aristas (s_i, s_{i+1}), $i = 1, \ldots, 2^n - 1$ y (s_{2^n}, s_1) son distintas. Cuando $n = 1$, el código Gray 0, 1, corresponde al camino (0, 1, 0), que no es un ciclo, pues la arista (0, 1) se repite.

Los códigos Gray se han estudiado con amplitud en otros contextos. Por ejemplo, se han utilizado en la conversión de información análoga a una forma digital (véase [Deo]). Ahora mostraremos la forma de construir un código Gray para cada entero positivo n, con lo cual se demuestra que el n-cubo tiene un ciclo hamiltoniano para cada entero positivo $n \geq 2$. □

TEOREMA 6.3.6

Sea G_1 la sucesión 0, 1. Definimos G_n en términos de G_{n-1} mediante las reglas siguientes:

(a) Sea G_{n-1}^R la sucesión G_{n-1} escrita al revés.

(b) Sea G'_{n-1} la sucesión obtenida al agregar 0 al principio de G_{n-1}.

(c) Sea G''_{n-1} la sucesión obtenida al agregar 1 al principio de G_{n-1}^R.

(d) Sea G_n la sucesión formada por G'_{n-1} seguida de G''_{n-1}.

Entonces G_n es un código Gray para cada entero positivo n.

Demostración. Demostramos el teorema por inducción sobre n.

PASO BASE ($n = 1$). Como la sucesión 0, 1 es un código Gray, el teorema es verdadero cuando n es igual a 1.

PASO INDUCTIVO. Supongamos que G_{n-1} es un código Gray. Cada cadena en G'_{n-1} comienza con 0, de modo que cualquier diferencia entre las cadenas consecutivas debe surgir de bits diferentes en las cadenas correspondientes en G_{n-1}. Pero como G_{n-1} es un código Gray, cada par consecutivo de cadenas en G_{n-1} difieren en exactamente un bit. Por tanto, cada par consecutivo de cadenas en G'_{n-1} difieren en exactamente un bit. De manera análoga, cada par consecutivo de cadenas en G''_{n-1} difieren en exactamente un bit.

Sea α la última cadena en G'_{n-1} y sea β la primera cadena en G''_{n-1}. Si eliminamos el primer bit de α y el primer bit de β, las cadenas resultantes son idénticas. Como el primer bit en α es 0 y el primer bit en β es 1, la última cadena en G'_{n-1} y la primera cadena en G''_{n-1} difieren en exactamente un bit. De manera análoga, la primera cadena en G'_{n-1} y la última cadena en G''_{n-1} difieren en exactamente un bit. Por tanto, G_n es un código Gray. ■

COROLARIO 6.3.7

El n-cubo tiene un ciclo hamiltoniano para cada entero positivo $n \geq 2$.

EJEMPLO 6.3.8

Utilizaremos el teorema 6.3.6 para construir el código Gray G_3 comenzando en G_1.

G_1:	0	1						
G_1^R:	1	0						
G'_1:	00	01						
G''_1:	11	10						
G_2:	00	01	11	10				
G_2^R:	10	11	01	00				
G'_2:	000	001	011	010				
G''_2:	110	111	101	100				
G_3:	000	001	011	010	110	111	101	100

□

Concluimos esta sección examinando un problema con una antigüedad aproximada de 200 años.

EJEMPLO 6.3.9 ***El recorrido del caballo***

En el ajedrez, el movimiento del caballo consta de un movimiento horizontal o vertical de dos cuadros y luego de un movimiento de un cuadro en la dirección perpendicular. Por ejemplo, en la figura 6.3.10, un caballo en el cuadro marcado con K se puede mover a cualquiera de los cuadros marcados con X. Un **recorrido del caballo en un tablero $n \times n$** comienza en algún cuadro, visita cada cuadro exactamente una vez (mediante movimientos válidos) y regresa al cuadro inicial. El problema consiste en determinar los valores de n para los cuales existe un recorrido del caballo.

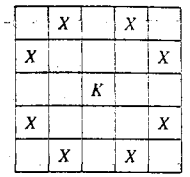


FIGURA 6.3.10
Los movimientos válidos del caballo (K) del ajedrez.

Podemos utilizar una gráfica para modelar este problema. Digamos que los cuadrados del tablero, pintados en blanco y negro de la manera usual, son los vértices de la gráfica y coloquemos una arista entre dos vértices si los cuadros correspondientes del tablero representan un movimiento válido del caballo (véase la figura 6.3.11). Denotamos la gráfica como GK_n. Entonces existe un recorrido del caballo en el tablero $n \times n$ si y sólo si GK_n tiene un ciclo hamiltoniano.

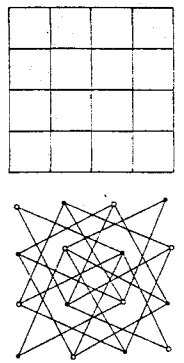


FIGURA 6.3.11
Un tablero 4 × 4 y la gráfica GK_4.

Mostraremos que si GK_n tiene un ciclo hamiltoniano, n es par. Para ver esto, observemos que GK_n es bipartita. Separemos los vértices en los conjuntos V_1, aquellos correspondientes a los cuadros blancos, y V_2, aquellos correspondientes a los cuadros negros; cada arista es incidente en un vértice en V_1 y V_2. Como cualquier ciclo debe alternar entre un vértice en V_1 y uno en V_2, cualquier ciclo en GK_n debe tener longitud par. Pero como un ciclo hamiltoniano debe visitar cada vértice exactamente una vez, un ciclo hamiltoniano en GK_n debe tener longitud n^2. Así, n debe ser par.

En vista del resultado anterior, el menor tablero posible que podría tener un recorrido del caballo es el tablero 2 × 2, pero no puede tener tal recorrido, pues el tablero es tan pequeño que el caballo no tiene movimientos válidos. El siguiente tablero menor que podría tener un recorrido del caballo es el tablero 4 × 4, aunque, como mostraremos, tampoco tiene un recorrido de caballo.

Argumentaremos por contradicción para mostrar que GK_4 no tiene un ciclo hamiltoniano. Supongamos que GK_4 tiene un ciclo hamiltoniano $C = (v_1, v_2, \ldots, v_{17})$. Supondre-

Concluimos esta sección demostrando un resultado especial que utilizaremos en la sección 7.2.

TEOREMA 6.2.24

Si una gráfica G contiene un ciclo de v a v, entonces G contiene un ciclo simple de v a v.

Demostración. Sea

$$C = (v_0, e_1, v_1, \ldots, e_1, v_1, e_{i+1}, \ldots, e_j, v_j, e_{j+1}, \ldots, e_n, v_n)$$

un ciclo de v a v, donde $v = v_0 = v_n$ (véase la figura 6.2.11). Si C no es un ciclo simple, entonces $v_i = v_j$ para algún $i < j < n$. Podemos reemplazar C con el ciclo

$$C' = (v_0, e_1, v_1, \ldots, e_i, v_i, e_{j+1}, \ldots, e_n, v_n).$$

Si C' no es un ciclo simple de v a v, repetimos el procedimiento anterior. En algún momento obtenemos un ciclo simple de v a v. ■

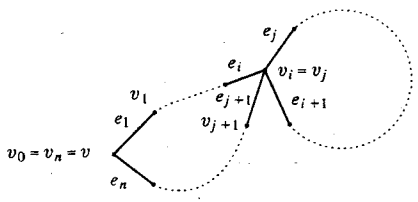
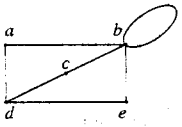


FIGURA 6.2.11 Un ciclo que es simple o se puede reducir a uno simple.

Ejercicios



En los ejercicios 1-9, indique si el camino dado en la gráfica es

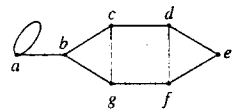
(a) Un camino simple **(b) Un ciclo** **(c) Un ciclo simple**

1. (b, b)
2. (e, d, c, b)
3. (a, d, c, d, e)
4. (d, c, b, e, d)
5. $(b, c, d, a, b, e, d, c, b)$
6. (b, c, d, e, b, b)
7. (a, d, c, b, e)
8. (d)
9. (d, c, b)

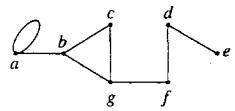
En los ejercicios 10-18, trace una gráfica con las propiedades dadas o explique por qué no existe tal gráfica.

10. Seis vértices, cada uno de grado 3
11. Cinco vértices, cada uno de grado 3

12. Cuatro vértices, cada uno de grado 1
13. Seis vértices; cuatro aristas
14. Cuatro aristas; cuatro vértices con grados 1, 2, 3, 4
15. Cuatro vértices con grados 1, 2, 3, 4
16. Gráfica simple; seis vértices con grados 1, 2, 3, 4, 5, 5
17. Gráfica simple; cinco vértices con grados 2, 3, 3, 4, 4
18. Gráfica simple; cinco vértices con grados 2, 2, 4, 4, 4
19. Determine todos los ciclos simples de la siguiente gráfica.



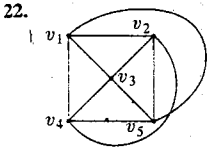
20. Determine todos los caminos simples de a a e en la gráfica del ejercicio 19.



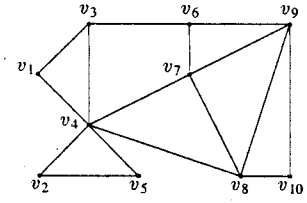
21. Determine todas las subgráficas de la gráfica anexa que contengan a todos los vértices de la gráfica original y tengan el menor número posible de aristas. ¿Cuáles son caminos simples? ¿Cuáles son ciclos? ¿Cuáles son ciclos simples?

Determine el grado de cada vértice para las siguientes gráficas.

22.

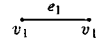


23.



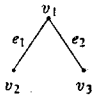
En los ejercicios 24-27, determine todas las subgráficas que tienen al menos un vértice de la gráfica dada.

24.

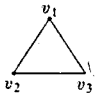


25.

26.



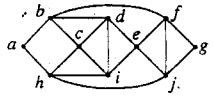
★ **27.**



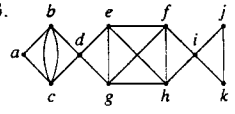
En los ejercicios 28-33, decida si la gráfica tiene un ciclo de Euler. Si la gráfica tiene un ciclo de Euler, exhiba alguno.

28. Ejercicio 21
29. Ejercicio 22
30. Ejercicio 23
31. Figura 6.2.4

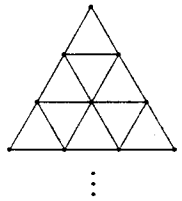
32.



33.



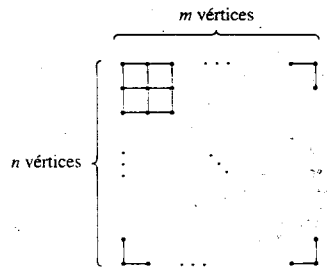
34. La siguiente gráfica continúa hasta una profundidad arbitraria pero finita. ¿Contiene esta gráfica un ciclo de Euler? Si la respuesta es sí, describa uno.



35. ¿En qué casos la gráfica completa K_n contiene un ciclo de Euler?

36. ¿En qué casos la gráfica bipartita completa $K_{m,n}$ contiene un ciclo de Euler?

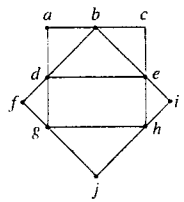
37. ¿Para cuáles valores de m y n la siguiente gráfica contiene un ciclo de Euler?



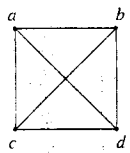
38. ¿Para cuáles valores de n contiene el n-cubo un ciclo de Euler?

En los ejercicios 39 y 40, verifique que la gráfica tiene un número par de vértices de grado impar.

39.



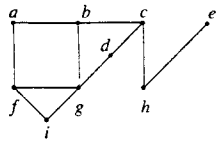
40.



41. Para la gráfica del ejercicio 39, determine un camino sin aristas repetidas de d a e que contenga a todas las aristas.

42. Sea G una gráfica conexa con cuatro vértices v_1, v_2, v_3 y v_4 de grado impar. Muestre que existen caminos sin aristas repetidas de v_1 a v_2 y de v_3 a v_4 tales que cada arista en G está en exactamente uno de los caminos.

43. Ilustre el ejercicio 42 mediante la gráfica anexa.



44. Enuncie y demuestre una generalización del ejercicio 42 para un número arbitrario de vértices de grado impar.

En los ejercicios 45 y 46, indique si cada afirmación es verdadera o falsa. En caso de que sea falsa, proporcione un contraejemplo y, de ser verdadera, explique.

45. Sea G una gráfica y sean v y w vértices distintos. Si existe un camino de v a w, existe un camino simple de v a w.

46. Si una gráfica contiene un ciclo que incluya todas las aristas, el ciclo es un ciclo de Euler.

47. Sea G una gráfica conexa. Suponga que una arista e está en un ciclo. Muestre que G con e eliminada sigue siendo conexa.

48. Proporcione un ejemplo de una gráfica conexa de modo que la eliminación de cualquier arista produzca una gráfica que no sea conexa. (Suponga que la eliminación de una arista no elimina vértice alguno.)

☆ 49. ¿Podría un caballo de ajedrez moverse en un tablero y regresar a su posición original realizando cada movimiento exactamente una vez? (Un movimiento se considera efectuado cuando el movimiento se hace en cualquier dirección.)

50. Muestre que si G' es una subgráfica conexa de una gráfica G, entonces G' está contenida en un componente.

51. Muestre que si una gráfica G se divide en subgráficas conexas de modo que cada arista y cada vértice de G pertenezca a una de las subgráficas, entonces las subgráficas son componentes.

52. Sea G una gráfica dirigida y sea G' la gráfica no dirigida obtenida a partir de G ignorando la dirección de las aristas de G. Suponga que G es conexa. Si v es un vértice en G, decimos que la *paridad de* v es *par* si el número de aristas de la forma (v, w) es par; la *paridad impar* se define de manera análoga. Demuestre que si v y w son vértices de G que tienen paridad impar, es posible cambiar la orientación de algunas aristas de G de modo que v y w tengan paridad par y que la paridad del resto de los vértices de G no se modifique.

★ **53.** Muestre que el número máximo de aristas en una gráfica simple disconexa con n vértices es $(n-1)(n-2)/2$.

☆ 54. Muestre que el número máximo de aristas en una gráfica bipartita simple con n vértices es $\lfloor n^2/4 \rfloor$.

Un vértice v en una gráfica conexa G es un *punto de articulación* si la eliminacion de v y de todas las aristas incidentes en v desconecta a G.

55. Proporcione un ejemplo de una gráfica con seis vértices que tenga exactamente dos puntos de articulación.

56. Proporcione un ejemplo de una gráfica con seis vértices y que no tenga puntos de articulación.

57. Muestre que un vértice v en una gráfica conexa G es un punto de articulación si y sólo si existen vértices w y x en G tales que cada camino de w a x pasa por v.

Sea G una gráfica dirigida y v un vértice en G. El *grado interno* (o grado de entrada) de v, $\text{in}(v)$, es el número de aristas de la forma (w, v). El *grado de salida* de v, $\text{out}(v)$, es el número de aristas de la forma (v, w). Un *ciclo de Euler dirigido* en G es una sucesión de aristas de la forma

$$(v_0, v_1), (v_1, v_2), \ldots, (v_{n-1}, v_n),$$

donde $v_0 = v_n$, cada arista en G aparece exactamente una vez, y aparecen todos los vértices.

58. Muestre que una gráfica dirigida G contiene un ciclo de Euler dirigido si y sólo si la gráfica no dirigida obtenida al ignorar las direcciones de las aristas de G es conexa e $\text{in}(v) = \text{out}(v)$ para cada vértice v en G.

Una *sucesión de de Bruijn* para n (en ceros y unos) es una sucesión

$$a_1, \quad \ldots, \quad a_{2^n}$$

de 2^n bits con la propiedad de que si s es una cadena de bits de longitud n, entonces para alguna m,

$$s = a_m a_{m+1} \cdots a_{m+n-1}. \tag{6.2.2}$$

En (6.2.2), definimos $a_{2^n+i} = a_i$ para $i = 1, \ldots, 2^n - 1$.

59. Verifique que 00011101 es una sucesión de de Bruijn para $n = 3$.

60. Sea G una gráfica dirigida con vértices correspondientes a todas las cadenas de bits de longitud $n - 1$. Existe una arista dirigida del vértice $x_1 \cdots x_{n-1}$ a $x_2 \cdots x_n$. Muestre que un ciclo de Euler dirigido en G corresponde a una sucesión de de Bruijn.

★ **61.** Muestre que existe una sucesión de de Bruijn para cada $n = 1, 2, \ldots$.

☆ 62. Un *camino cerrado* es un camino de v a v. Muestre que una gráfica conexa G es bipartita si y sólo si cada camino cerrado en G tiene longitud par.

63. ¿Cuántos caminos de longitud $k \geq 1$ existen en K_n?

64. Muestre que existen

$$\frac{n(n-1)[(n-1)^k - 1]}{n-2}$$

caminos cuyas longitudes están entre 1 y k, inclusive, en K_n, $n > 2$.

65. Sean v y w vértices distintos en K_n. Sea p_m el número de caminos de longitud m de v a w en K_n, $1 \leq m \leq n$.

(a) Deduzca una relación de recurrencia para p_m.

(b) Determine una fórmula explícita para p_m.

66. Sean v y w vértices distintos en K_n, $n \geq 2$. Muestre que el número de caminos simples de v a w es

$$(n-2)! \sum_{k=0}^{n-2} \frac{1}{k!}.$$

☆ 67. [Requiere conocimientos de cálculo.] Muestre que existen $\lfloor n!e - 1 \rfloor$ caminos simples en K_n. ($e = 2.71828\ldots$ es la base del logaritmo natural.)

68. Sea G una gráfica. Defina una relación R sobre el conjunto V de vértices de G como vRw si existe un camino de v a w. Demuestre que R es una relación de equivalencia en V.

69. Demuestre que una gráfica conexa con uno o dos vértices, cada uno de los cuales tiene grado par, tiene un ciclo de Euler.

Sea G una gráfica conexa. La *distancia* entre los vértices v y w en G, $\text{dist}(v, w)$, es la longitud del camino más corto de v a w. El *diámetro* de G es

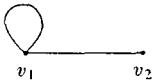
$$d(G) = \text{máx}\{\text{dist}(v, w) \mid v \text{ y } w \text{ son vértices en } G\}.$$

70. Determine el diámetro de la gráfica de la figura 6.2.10.

71. Determine el diámetro del n-cubo. En el contexto de la computación paralela, ¿cuál es el significado de este valor?

72. Determine el diámetro de K_n, la gráfica completa de n vértices.

73. Muestre que el número de caminos en la gráfica anexa, de v_1 a v_1 de longitud n es igual al n-ésimo número de Fibonacci f_n.



74. Sea G una gráfica simple con n vértices en la cual cada vértice tiene grado k y

$$k \geq \frac{n-3}{2} \qquad \text{si } n \bmod 4 = 1,$$
$$k \geq \frac{n-1}{2} \qquad \text{si } n \bmod 4 \neq 1.$$

Muestre que G es conexa.

Un *ciclo* en una gráfica dirigida simple [es decir, una gráfica dirigida en la que existe a lo más una arista de la forma (v, w) y que no tiene aristas de la forma (v, v)] es una sucesión de tres o más vértices

$$(v_0, v_1, \ldots, v_n)$$

en la cual (v_{i-1}, v_i) es una arista para $i = 1, \ldots, n$ y $v_0 = v_n$. Una *gráfica acíclica dirigida* (*gad*) es una gráfica simple dirigida sin ciclos.

75. Muestre que una gad tiene al menos un vértice sin aristas de entrada [es decir, existe al menos un vértice tal que no existen aristas de la forma (v, w)].

76. Muestre que el número máximo de aristas en una gad de n vértices es $n(n-1)/2$.

77. Un *conjunto independiente* en una gráfica G es un subconjunto S de los vértices de G con la propiedad de que ningún par de vértices en S son adyacentes. (Observe que $\varnothing$ es un conjunto independiente para cualquier gráfica.) Demuestre el siguiente resultado debido a [Prodinger].

Sea P_n la gráfica dada por un camino simple con n vértices. Demuestre que el número de conjuntos independientes en P_n es igual a f_{n+1}, $n = 1, 2, \ldots$, donde $\{f_n\}$ es la sucesión de Fibonacci.

EJEMPLO 6.2.10

Determine todas las subgráficas de la gráfica G de la figura 6.2.5 que tengan al menos un vértice.

Si no elegimos arista alguna, podemos elegir uno o ambos vértices para obtener las subgráficas G_1, G_2 y G_3 que aparecen en la figura 6.2.6. Si elegimos la única arista disponible e_1, debemos seleccionar los dos vértices sobre los cuales incide e_1. En este caso, obtenemos la subgráfica G_4 de la figura 6.2.6. Así, G tiene las cuatro subgráficas que aparecen en la figura 6.2.6.

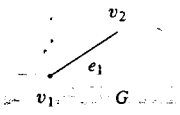


FIGURA 6.2.5 La gráfica del ejemplo 6.2.10.

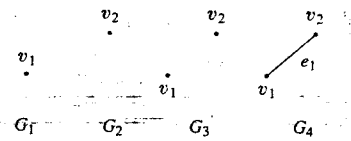


FIGURA 6.2.6 Las cuatro subgráficas de la gráfica de la figura 6.2.5.

□

Ahora podemos definir los componentes.

DEFINICIÓN 6.2.11

Sea G una gráfica y v un vértice en G. La subgráfica G' de G que consta de todas las aristas y vértices en G que están contenidas en algún camino que comienza en v es el *componente* de G que contiene a v.

EJEMPLO 6.2.12

La gráfica G de la figura 6.2.1 tiene un componente, a saber, ella misma. De hecho, una gráfica es conexa si y sólo si tiene exactamente un componente. □

EJEMPLO 6.2.13

Sea G la gráfica de la figura 6.2.2. El componente de G que contiene a v_3 es la subgráfica

$$G_1 = (V_1, E_1), \quad V_1 = \{v_1, v_2, v_3\}, \quad E_1 = \{e_1, e_2, e_3\}.$$

El componente de G que contiene a v_4 es la subgráfica

$$G_2 = (V_2, E_2), \quad V_2 = \{v_4\}, \quad E_2 = \varnothing.$$

El componente de G que contiene a v_5 es la subgráfica

$$G_3 = (V_3, E_3), \quad V_3 = \{v_5, v_6\}, \quad E_3 = \{e_4\}.$$ □

Otra caracterización de los componentes de una gráfica $G = (V, E)$ se obtiene definiendo una relación R sobre el conjunto de vértices V mediante la regla

$v_1 R\, v_2$, si existe un camino de v_1 a v_2.

Se puede mostrar (ejercicio 68) que R es una relación de equivalencia sobre V y que si $v \in V$, el conjunto de vértices en el componente que contiene a v es la clase de equivalencia

$$[v] = \{w \in V \mid w\, R\, v\}.$$

Observe que la definición de camino permite repetir los vértices o las aristas (o ambos). En el camino (6.2.1), el vértice 2 aparece dos veces.

Obtenemos subclases de caminos prohibiendo la repetición de vértices y de aristas, o haciendo que los vértices v_0 y v_n de la definición 6.2.1 sean idénticos.

DEFINICIÓN 6.2.14

Sean v y w vértices en una gráfica G.

Un *camino simple* de v a w es un camino de v a w sin vértices repetidos.

Un *ciclo* (o *circuito*) es un camino de longitud distinta de cero de v a v, sin aristas repetidas.

Un *ciclo simple* es un ciclo de v a v en el cual no existen vértices repetidos, excepto por los vértices inicial y final, que son iguales a v.

EJEMPLO 6.2.15

Para la gráfica de la figura 6.2.1 tenemos

Camino	*¿Es un camino simple?*	*¿Es un ciclo?*	*¿Es un ciclo simple?*
(6, 5, 2, 4, 3, 2, 1)	No	No	No
(6, 5, 2, 4)	Sí	No	No
(2, 6, 5, 2, 4, 3, 2)	No	Sí	No
(5, 6, 2, 5)	No	Sí	Sí
(7)	Sí	No	No

□

A continuación volvemos a analizar el problema presentado en la sección 6.1: determinar un ciclo en una gráfica que recorra cada arista exactamente una vez.

EJEMPLO 6.2.16 ***Problema de los puentes de Königsberg***

El primer artículo en teoría de gráficas fue el escrito por Leonhard Euler en 1736. El artículo presentó una teoría general que incluía una solución a lo que ahora se llama el problema de los puentes de Königsberg.

Dos islas en el río Pregel, en Königsberg (ahora Kaliningrado, Rusia) se unen entre ellas y con la tierra firme mediante puentes, como muestra la figura 6.2.7. El problema consiste en partir de cualquier lugar (A, B, C o D), caminar sobre cada puente exactamente una vez y luego regresar a la posición inicial.

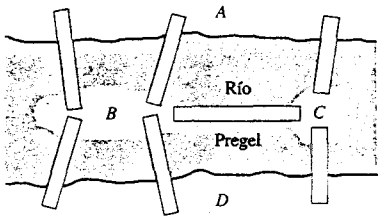


FIGURA 6.2.7 Los puentes de Königsberg.

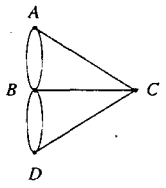


FIGURA 6.2.8 Un modelo de gráfica de los puentes de Königsberg.

La configuración de los puentes se puede modelar mediante una gráfica, como muestra la figura 6.2.8. Los vértices representan las posiciones y las aristas representan los puentes. Entonces, el problema de los puentes de Königsberg se reduce a determinar un ciclo en la gráfica de la figura 6.2.8 que incluya todas las aristas y todos los vértices. En honor de Euler, un ciclo de una gráfica G que incluya todas las aristas y los vértices de G es un **ciclo de Euler**[†] (también se le conoce como paseo euleriano). Por el análisis de la sección 6.1, vemos que no existe un ciclo de Euler en la gráfica de la figura 6.2.8, pues hay un número impar de aristas incidentes en el vértice A. (De hecho, en la gráfica de la figura 6.2.8, cada vértice es incidente en un número impar de aristas.) □

La solución al problema de existencia de ciclos de Euler se puede establecer mediante el grado de un vértice. El **grado de un vértice** v, $\delta(v)$, es el número de aristas incidentes en v. (Por definición, cada lazo en v contribuye en 2 al grado de v.) En la sección 6.1 vimos que si una gráfica G tiene un ciclo de Euler, entonces cada vértice en G tiene grado par. También podemos demostrar que G es conexa.

TEOREMA 6.2.17

Si una gráfica G tiene un ciclo de Euler, entonces G es conexa y cada vértice tiene grado par.

Demostración. Supongamos que G tiene un ciclo de Euler. En la sección 6.1 argumentamos que cada vértice en G tiene grado par. Si v y w son vértices en G, la parte del ciclo de Euler que nos lleva de v a w sirve como un camino de v a w. Por tanto, G es conexa. ■

El recíproco del teorema 6.2.17 también es cierto. Daremos una demostración por inducción matemática debida a [Fowler].

[†] Por razones técnicas, si G consta de un vértice v y no tiene aristas, también diremos que el camino (v) es un ciclo de Euler para G.

TEOREMA 6.2.18

Si G es una gráfica conexa y todo vértice tiene grado par, entonces G tiene un ciclo de Euler.

Demostración. La demostración es por inducción sobre el número n de aristas en G.

PASO BASE ($n = 0$). Como G es conexa, si G no tiene aristas, G consta de un único vértice. Un ciclo de Euler consta de este único vértice y ninguna arista.

PASO INDUCTIVO. Supongamos que G tiene n aristas, $n > 0$, y que cada gráfica conexa con k aristas, $k < n$, en la cual todo vértice tenga grado par, tiene un ciclo de Euler.

Se puede verificar directamente que una gráfica conexa con uno o dos vértices, cada uno de los cuales tiene grado par, tiene un ciclo de Euler (véase el ejercicio 69); así, supondremos que la gráfica tiene al menos tres vértices.

Como G es conexa, existen vértices v_1, v_2, v_3 en G de modo que la arista e_1 sea incidente en v_1 y v_2 y la arista e_2 sea incidente en v_2 y v_3. Eliminamos las aristas e_1 y e_2, pero no los vértices, y agregamos una arista e incidente en v_1 y v_3 para obtener la gráfica G' [véase la figura 6.2.9(a)]. Observe que cada componente de la gráfica G' tiene menos de n aristas y que en cada componente de la gráfica G', cada vértice tiene grado par. Mostraremos que G' tiene uno o dos componentes.

Sea v un vértice. Como G es conexa, existe un camino P en G de v a v_1. Sea P' la parte del camino P que comienza en v cuyas aristas también estén en G'. Ahora, P' termina en v_1, v_2 o v_3, pues la única forma en que P podría dejar de ser un camino en G' sería que P contuviera alguna de las aristas eliminadas e_1 o e_2. Si P' termina en v_1, entonces v está en el mismo componente que v_1 en G'. Si P' termina en v_3 [véase la figura 6.2.9(b)], entonces v está en el mismo componente que v_3 en G', que está en el mismo componente que v_1 en

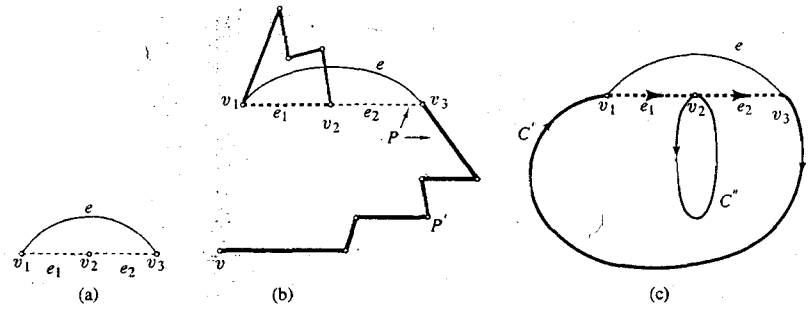


FIGURA 6.2.9 La demostración del teorema 6.2.18. En (a), las aristas e_1 y e_2 se eliminan y se agrega la arista e. En (b), P (remarcado) es un camino en G de v a v_1, y P' (en tono más oscuro) es la parte de P que comienza en v y cuyas aristas también están en G'. Como se muestra, P' termina en v_3. Como la arista e está en G', existe un camino en G' de v a v_1. Así, v y v_1 están en el mismo componente. En (c), C' (que aparece con una línea más gruesa) es un ciclo de Euler para un componente, y C'' (que aparece con una línea continua más delgada) es un ciclo de Euler para el otro componente. Si reemplazamos e en C' por e_1, C'', e_2, obtenemos un ciclo de Euler (en tono más oscuro o remarcado) para G.

G' (pues la arista e en G' es incidente en v_1 y v_3). Si P' termina en v_2, entonces v_2 está en el mismo componente que v. Por tanto, todo vértice en G' está en el mismo componente que v_1 o v_2. Así, G' tiene uno o dos componentes.

Si G' tiene un componente, es decir, si G' es conexa, podemos aplicar la hipótesis inductiva para concluir que G' tiene un ciclo de Euler C'. Este ciclo de Euler se puede modificar para obtener un ciclo de Euler en G: basta reemplazar la ocurrencia de la arista e en C' por las aristas e_1 y e_2.

Supongamos que G' tiene dos componentes [véase la figura 6.2.9(c)]. Por la hipótesis de inducción, el componente que contiene a v_1 tiene un ciclo de Euler C' y el componente que contiene a v_2 tiene un ciclo de Euler C'' que comienza y termina en v_2. Obtenemos un ciclo de Euler en G modificando C', reemplazando (v_1, v_3) en C' por (v_1, v_2) seguido de C'' seguido de (v_2, v_3) o reemplazando (v_3, v_1) en C' por (v_3, v_2) seguido de C'' seguido de (v_2, v_1). Con esto concluimos el paso inductivo y G tiene un ciclo de Euler. ■

Si G es una gráfica conexa y cada vértice tiene grado par, donde G tiene pocas aristas, por lo general podemos encontrar un ciclo de Euler por inspección.

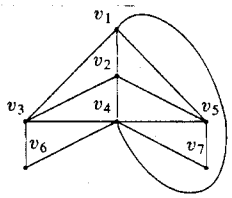


FIGURA 6.2.10
La gráfica del ejemplo 6.2.19.

EJEMPLO 6.2.19

Sea G la gráfica de la figura 6.2.10. Utilizar el teorema 6.2.18 para verificar que G tiene un ciclo de Euler. Determinar un ciclo de Euler para G.

Observamos que G es conexa y que

$$\delta(v_1) = \delta(v_2) = \delta(v_3) = \delta(v_5) = 4, \quad \delta(v_4) = 6, \quad \delta(v_6) = \delta(v_7) = 2.$$

Como el grado de cada vértice es par, por el teorema 6.2.18, G tiene un ciclo de Euler. Por inspección, determinamos el ciclo de Euler

$$(v_6, v_4, v_7, v_5, v_1, v_3, v_4, v_1, v_2, v_5, v_4, v_2, v_3, v_6).$$ □

EJEMPLO 6.2.20

Un dominó es un rectángulo dividido en dos cuadrados, donde cada cuadrado tiene un número entre 0, 1, . . . , 6. Dos cuadrados de un mismo dominó pueden tener el mismo número. Mostraremos que los distintos dominós se pueden ordenar en un círculo, de modo que los dominós adyacentes tengan cuadrados con números idénticos.

Modelamos la situación como una gráfica G con siete vértices etiquetados 0, 1, . . . , 6. Las aristas representan los dominós: existe una arista entre cada par de vértices distintos y existe un lazo en cada vértice. Observe que G es conexa. Ahora, los dominós se pueden ordenar en un círculo, de modo que los dominós adyacentes tengan cuadrados con números idénticos, si y sólo si G contiene un ciclo de Euler. Como el grado de cada vértice es 8 (recuerde que un lazo contribuye en 2 unidades al grado), cada vértice tiene grado par. Por el teorema 6.2.18, G tiene un ciclo de Euler. Por tanto, los dominós se pueden ordenar en un círculo, de modo que los dominós adyacentes tengan cuadrados con números idénticos. □

¿Qué se puede decir acerca de una gráfica en la que no todos los vértices tengan grado par? La primera observación (corolario 6.2.22) es que existe un número par de vértices de grado impar. Esto es consecuencia del hecho (teorema 6.2.21) de que la suma de todos los grados en una gráfica es un número par.

TEOREMA 6.2.21

Si G es una gráfica con m aristas y vértices $\{v_1, v_2, \ldots, v_n\}$, entonces

$$\sum_{i=1}^{n} \delta(v_i) = 2m.$$

En particular, la suma de los grados de todos los vértices en una gráfica es par.

Demostración. Al sumar los grados de todos los vértices, contamos cada arista (v_i, v_j) dos veces [una al contarla como (v_i, v_j) en el grado de v_i y otra más al contarla como (v_j, v_i) en el grado de v_j]. De aquí se sigue el resultado. ■

COROLARIO 6.2.22

En cualquier gráfica, existe un número par de vértices de grado impar.

Demostración. Dividimos los vértices en dos grupos, aquellos con grado par $x_1, \ldots, x_m$ y aquellos con grado impar $y_1, \ldots, y_n$. Sean

$$S = \delta(x_1) + \delta(x_2) + \cdots + \delta(x_m), \qquad T = \delta(y_1) + \delta(y_2) + \cdots + \delta(y_n).$$

Por el teorema 6.2.21, $S + T$ es par. Como S es la suma de números pares, S es par. Así, T es par. Pero T es la suma de n números impares, por tanto n es par. ■

Supongamos que una gráfica conexa G tiene exactamente dos vértices v y w de grado impar. Insertemos de manera temporal una arista e de v a w. La gráfica resultante G' es conexa y cada vértice tiene grado par. Por el teorema 6.2.18, G' tiene un ciclo de Euler. Si eliminamos e de este ciclo de Euler, obtenemos un camino sin aristas repetidas de v a w que contiene a las demás aristas y vértices de G. Hemos mostrado entonces que si una gráfica tiene exactamente dos vértices v y w de grado impar, existe un camino sin aristas repetidas que contiene a todas las aristas y vértices de v a w. El recíproco se puede demostrar de manera similar.

TEOREMA 6.2.23

Una gráfica tiene un camino sin aristas repetidas de v a w $(v \neq w)$ que contiene a todas las aristas y vértices si y sólo si es conexa y v y w son los únicos vértices que tienen grado impar.

Demostración. Supongamos que una gráfica tiene un camino P sin aristas repetidas de v a w que contiene a todas las aristas y vértices. Es claro que la gráfica es conexa. Si agregamos la arista de v a w, la gráfica resultante tiene un ciclo de Euler, a saber, el camino P junto con la arista agregada. Por el teorema 6.2.17, cada vértice tiene grado par. La eliminación de la arista agregada sólo afecta a los grados de v y w, que se ven reducidos en 1 cada uno. Así, en la gráfica original, v y w tienen grado impar y los demás vértices tienen grado par.

El recíproco fue analizado justo antes del enunciado del teorema. ■

Los ejercicios 42 y 44 tienen generalizaciones del teorema 6.2.23.

El n-cubo es un modelo importante de la computación, pues varias de estas máquinas ya se han construido y están en operación. Además, el hipercubo puede simular otros modelos de computación paralela. Este último punto se considera con más detalle en los ejemplos 6.3.5 y 6.6.3. □

Concluimos esta sección introductoria definiendo algunas gráficas particulares que aparecen con frecuencia en la teoría de gráficas.

DEFINICIÓN 6.1.8

La *gráfica completa de n vértices*, que se denota K_n, es la gráfica simple con n vértices en la cual existe una arista entre cada par de vértices distintos.

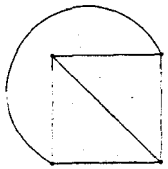


FIGURA 6.1.11
La gráfica completa K_4.

EJEMPLO 6.1.9

La figura 6.1.11 muestra la gráfica completa de 4 vértices, K_4. □

DEFINICIÓN 6.1.10

Una gráfica $G = (V, E)$ es *bipartita* si el conjunto de vértices V se puede separar en dos subconjuntos V_1 y V_2 de modo que cada arista en E sea incidente en un vértice de V_1 y un vértice de V_2.

EJEMPLO 6.1.11

La gráfica de la figura 6.1.12 es bipartita, pues si hacemos

$$V_1 = \{v_1, v_2, v_3\} \quad \text{y} \quad V_2 = \{v_4, v_5\},$$

cada arista es incidente en un vértice de V_1 y un vértice de V_2. □

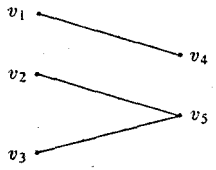


FIGURA 6.1.12 Una gráfica bipartita.

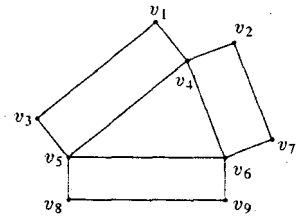


FIGURA 6.1.13 Una gráfica no bipartita.

Observe que la definición 6.1.10 establece que si e es una arista en una gráfica bipartita, entonces e es incidente en un vértice de V_1 y un vértice de V_2. *No* establece que si v_1 es un vértice de V_1 y v_2 es un vértice de V_2, entonces exista una arista entre v_1 y v_2. Por ejemplo, la gráfica de la figura 6.1.12 es bipartita, pues cada arista es incidente en un vértice de $V_1 = \{v_1, v_2, v_3\}$ y un vértice de $V_2 = \{v_4, v_5\}$. Sin embargo, no todas las aristas entre vértices de V_1 y V_2 están en la gráfica. Por ejemplo, no está presente la arista (v_1, v_5).

EJEMPLO 6.1.12

La gráfica de la figura 6.1.13 *no* es bipartita. Con frecuencia es más fácil mostrar que una gráfica no es bipartita argumentando por contradicción.

Supongamos que la gráfica de la figura 6.1.13 es bipartita. Entonces el conjunto de vértices se puede separar en dos subconjuntos V_1 y V_2 de modo que cada arista sea incidente en un vértice de V_1 y un vértice de V_2. Ahora, consideremos los vértices v_4, v_5 y v_6. Como v_4 y v_5 son adyacentes, uno está en V_1 y el otro en V_2. Podemos suponer que v_4 está en V_1 y que v_5 está en V_2. Como v_5 y v_6 son adyacentes y v_5 está en V_2, v_6 está en V_1. Como v_4 y v_6 son adyacentes y v_4 está en V_1, v_6 está en V_2. Pero entonces v_6 está en V_1 y V_2, lo cual es una contradicción, pues V_1 y V_2 son ajenos. Por tanto, la gráfica de la figura 6.1.13 no es bipartita. □

DEFINICIÓN 6.1.13

La *gráfica bipartita completa con m y n vértices*, denotada $K_{m,n}$, es la gráfica simple cuyo conjunto de vértices está dividido en conjuntos V_1 con m vértices y V_2 con n vértices, en los cuales existe una arista entre cada par de vértices v_1 y v_2, donde v_1 está en V_1 y v_2 está en V_2.

EJEMPLO 6.1.14

La gráfica bipartita completa con dos y cuatro vértices, $K_{2,4}$, aparece en la figura 6.1.14.

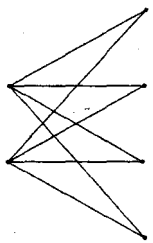
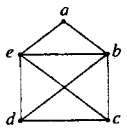
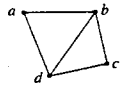


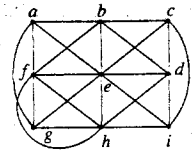
FIGURA 6.1.14 La gráfica bipartita completa $K_{2,4}$. □

Ejercicios

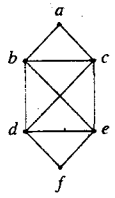
Explique por qué ninguna de las gráficas de los ejercicios 1-3 tiene un camino de a a a que pase por cada arista exactamente una vez.

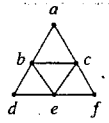
1. 

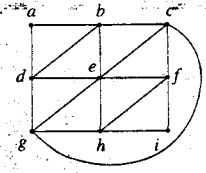
2. 

3. 

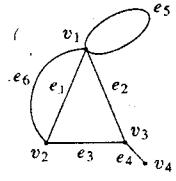
Muestre que cada gráfica en los ejercicios 4-6 tiene un camino de a a a que pasa por cada arista exactamente una vez, determinando tal camino por inspección.

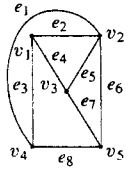
4. 

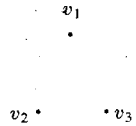
5. 

6. 

Para cada gráfica $G = (V, E)$ en los ejercicios 7-9, determine V, E, todas las aristas paralelas, todos los lazos, todos los vértices aislados, e indique si G es una gráfica simple. Además, indique los vértices donde es incidente la arista e_1.

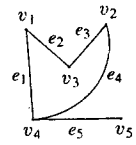
7. 

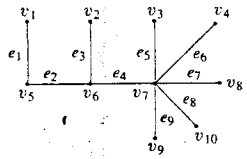
8. 

9. 

10. Trace K_3 y K_5.
11. Determine una fórmula para el número de aristas en K_n.
12. Proporcione un ejemplo de gráfica bipartita diferente a la figura 6.1.12. Especifique los conjuntos ajenos de vértices.

Indique cuáles de las gráficas de los ejercicios 13-19 son bipartitas. Si la gráfica es bipartita, especifique los conjuntos ajenos de vértices.

13. 

14. 

15. Figura 6.1.2
16. Figura 6.1.5
17. Ejercicio 7
18. Ejercicio 8

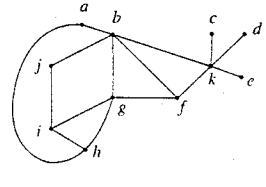
19. Ejercicio 9
20. Trace $K_{2,3}$ y $K_{3,3}$.
21. Determine una fórmula para el número de aristas de $K_{m,n}$.

En los ejercicios 22-24, determine un camino de longitud mínima de v a w en la gráfica de la figura 6.1.7 que pase por cada vértice exactamente una vez.

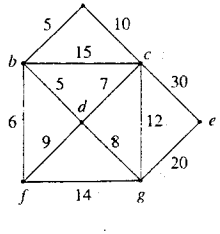
22. $v = b, w = e$ 23. $v = c, w = d$ 24. $v = a, w = b$

25. Trace la gráfica de similaridad que resulta de hacer $S = 40$ en el ejemplo 6.1.6. ¿Cuántas clases existen?
26. Trace la gráfica de similaridad que resulta de hacer $S = 50$ en el ejemplo 6.1.6. ¿Cuántas clases existen?
27. En general, ¿"es similar" es una relación de equivalencia?
28. Para el ejemplo 6.1.6, sugiera otras propiedades que podrían ser útiles al comparar programas.
29. ¿Cómo se podría automatizar la selección de S para agrupar datos en clases utilizando una gráfica de similaridad?
30. Trace un 2-cubo.
31. Trace una imagen como la de la figura 6.1.10 para mostrar la forma de construir un 3-cubo mediante dos 2-cubos.
32. Demuestre que la construcción recursiva del ejemplo 6.1.7 produce un n-cubo.
33. ¿Cuántas aristas son incidentes en un vértice de un n-cubo?
34. ¿Cuántas aristas tiene un n-cubo?
☆ 35. ¿De cuántas formas se pueden etiquetar los vértices de un n-cubo como $0, \ldots, 2^n - 1$ de modo que exista una arista entre dos vértices si y sólo si la representación binaria de sus etiquetas difiere precisamente en un bit?

Los ejercicios 36-38 se refieren a la gráfica anexa. Los vértices representan oficinas. Una arista conecta dos oficinas si existe un enlace de comunicación entre ambas. Observe que una oficina se puede comunicar con cualquier otra de manera directa, mediante un enlace, o haciendo que otras oficinas retransmitan el mensaje.



36. Muestre, mediante un ejemplo, que la comunicación entre todas las oficinas es posible aunque se rompan algunos enlaces de comunicación.
37. ¿Cuál es el número máximo de enlaces de comunicación que pueden romperse de modo que continúe siendo posible la comunicación entre todas las oficinas?
38. Muestre una configuración en que se rompa el número máximo de enlaces de comunicación de modo que continúe siendo posible la comunicación entre todas las oficinas.
39. En la gráfica anexa, los vértices representan ciudades y los números sobre las aristas representan los costos de construcción de los caminos indicados. Determine el sistema de carreteras más barato que una todas las ciudades.



6.2 CAMINOS Y CICLOS

Si pensamos los vértices de una gráfica como ciudades y las aristas como carreteras, un camino corresponde a un viaje que comienza en cierta ciudad, pasa por varias ciudades, y termina en alguna ciudad. Primero daremos una definición formal de camino.

DEFINICIÓN 6.2.1

Sean v_0 y v_n vértices de una gráfica. Un *camino* (ruta) de v_0 a v_n de longitud n es una sucesión alternante de $n + 1$ vértices y n aristas que comienza con el vértice v_0 y termina con el vértice v_n,

$$(v_0, e_1, v_1, e_2, v_2, \ldots, v_{n-1}, e_n, v_n),$$

en donde la arista e_i es incidente sobre los vértices v_{i-1} y v_i para $i = 1, \ldots, n$.

El formalismo de la definición 6.2.1 significa que se parte del vértice v_0, se sigue la arista e_1 hasta v_1, se sigue la arista e_2 hasta v_2, y así sucesivamente.

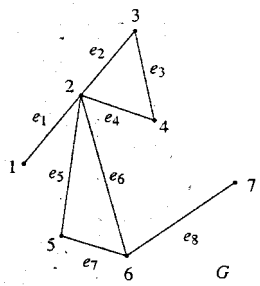


FIGURA 6.2.1
Una gráfica conexa con los caminos $(1, e_1, 2, e_2, 3, e_3, 4, e_4, 2)$ de longitud 4 y (6) de longitud 0.

EJEMPLO 6.2.2

En la gráfica de la figura 6.2.1,

$$(1, e_1, 2, e_2, 3, e_3, 4, e_4, 2) \qquad (6.2.1)$$

es un camino de longitud 4 del vértice 1 al vértice 2. □

EJEMPLO 6.2.3

En la gráfica de la figura 6.2.1, el camino (6) que sólo consta del vértice 6 es un camino de longitud 0 del vértice 6 al vértice 6. □

Si no hay aristas paralelas, al denotar un camino podemos omitir las aristas. Por ejemplo, el camino (6.2.1) se podría escribir

$$(1, 2, 3, 4, 2).$$

Una **gráfica conexa** es aquella en la cual podemos ir de cualquier vértice a cualquier otro por un camino. A continuación damos la definición formal.

DEFINICIÓN 6.2.4

Una gráfica G es *conexa* si dados cualesquiera dos vértices v y w en G, existe un camino de v a w.

EJEMPLO 6.2.5

La gráfica G de la figura 6.2.1 es conexa, pues dados cualesquiera dos vértices v y w en G, existe un camino de v a w. □

EJEMPLO 6.2.6

La gráfica G de la figura 6.2.2 no es conexa, pues por ejemplo, no existe un camino del vértice v_2 al vértice v_5. □

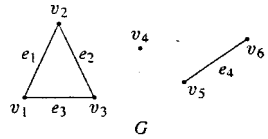


FIGURA 6.2.2
Una gráfica que no es conexa.

EJEMPLO 6.2.7

Sea G la gráfica cuyo conjunto de vértices consta de los 50 estados de Estados Unidos. Colocamos una arista entre los estados v y w si ellos tienen una frontera común. Por ejemplo, existe una arista entre California y Oregon y entre Illinois y Missouri. No existe una arista entre Georgia y Nueva York, ni tampoco una arista entre Utah y Nuevo México. (Tocarse no basta; los estados deben compartir una frontera.) La gráfica G no es conexa pues no existe un camino de Hawaii a California (o de Hawaii a cualquier otro estado). □

Como podemos ver en las figuras 6.2.1 y 6.2.2, una gráfica conexa consta de una "pieza", mientras que una gráfica no conexa consta de dos o más "piezas". Estas "piezas" son **subgráficas** de la gráfica original y se llaman **componentes**. Daremos las definiciones formales, comenzando por la de subgráfica.

Una subgráfica G' de una gráfica G se obtiene eligiendo ciertas aristas y vértices de G, sujetos a la restricción de que si elegimos una arista e en G que sea incidente en los vértices v y w, entonces debemos incluir a v y w en G'. La restricción permite garantizar que G' es realmente una gráfica. A continuación damos la definición formal.

DEFINICIÓN 6.2.8

Sea $G = (V, E)$ una gráfica. (V', E') es una *subgráfica* de G si

(a) $V' \subseteq V$ y $E' \subseteq E$.

(b) Para cada arista $e' \in E'$, si e' es incidente en v' y w', entonces $v', w' \in V'$.

EJEMPLO 6.2.9

La gráfica $G' = (V', E')$ de la figura 6.2.3 es una subgráfica de la gráfica $G = (V, E)$ de la figura 6.2.4, pues $V' \subseteq V$ y $E' \subseteq E$.

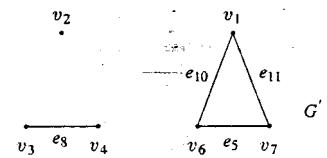


FIGURA 6.2.3 Una subgráfica de la gráfica de la figura 6.2.4.

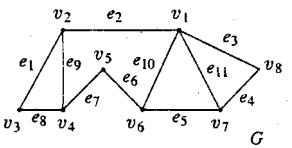


FIGURA 6.2.4 Una gráfica, una de cuyas subgráficas aparece en la figura 6.2.3.

□

por cuáles aristas. Por esta razón, la gráfica de la figura 6.1.2 también podría haberse trazado como en la figura 6.1.3.

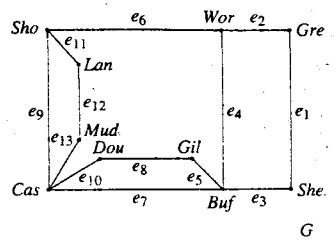


FIGURA 6.1.3 Un modelo de gráfica alternativa, pero equivalente, al modelo de gráfica del sistema de carreteras de la figura 6.1.1.

Si partimos de un vértice v_0, recorremos una arista hasta el vértice v_1, recorremos otra arista hasta el vértice v_2, y así sucesivamente, y en cierto momento llegamos al vértice v_n, llamamos a todo el **recorrido** un camino (o ruta) de v_0 a v_n. El camino que comienza en *She*, luego va para *Buf*, y termina en *Gil* corresponde a un viaje sobre el mapa de la figura 6.1.1 que comienza en Sheridan, va hacia Buffalo y termina en Gillette. El problema del inspector de carreteras se puede parafrasear para el modelo de gráfica G de la siguiente manera: ¿Existe un camino del vértice *Gre* al vértice *Gre* que recorra cada arista exactamente una vez?

Podemos mostrar que el inspector de carreteras no puede partir de Greybull, recorrer cada una de las carreteras exactamente una vez y regresar a Greybull. Para poner la respuesta en términos de gráficas, no existe un camino del vértice *Gre* al vértice *Gre* en la figura 6.1.2 que recorra cada arista exactamente una vez. Para ver esto, supongamos que existe tal camino y consideremos el vértice *Wor*. Cada vez que lleguemos a *Wor* por alguna arista, debemos salir de *Wor* por alguna arista diferente. Además, hay que utilizar cada una de las aristas que tocan a *Wor*. Como tres aristas tocan a *Wor*, tenemos una contradicción. Por tanto, no existe un camino del vértice *Gre* al vértice *Gre* de la figura 6.1.2 que recorra cada arista exactamente una vez. El argumento se puede aplicar a una gráfica arbitraria G. Si G tiene un camino del vértice v a v que recorra cada arista exactamente una vez, un número par de aristas debe tocar a cada vértice. En la sección 6.2 analizaremos este problema con detalle.

Ahora daremos algunas definiciones formales.

DEFINICIÓN 6.1.1

Una *gráfica* (o *gráfica no dirigida*) consta de un conjunto V de *vértices* (o *nodos*) y un conjunto E de *aristas* (*arcos* o *lados*) tales que cada arista $e \in E$ queda asociada a un par no ordenado de vértices. Si existe una única arista e asociada con los vértices v y w, escribimos $e = (v, w)$ o $e = (w, v)$. En este contexto, (v, w) denota una arista entre v y w en una gráfica no dirigida y *no* un par ordenado.

Una *gráfica dirigida* (o *digráfica*) G consta de un conjunto V de *vértices* (o *nodos*) y un conjunto E de *aristas* (*arcos* o *lados*) tales que cada arista $e \in E$ se asocia con un par ordenado de vértices. Si existe una única arista e asociada con el par ordenado (v, w) de vértices, escribimos $e = (v, w)$, lo cual denota una arista de v a w.

Una arista e en una gráfica (dirigida o no) asociada con el par de vértices v y w es *incidente* en v y w, y se dice que v y w son *incidentes en* e y que son *vértices adyacentes*.

Si G es una gráfica (dirigida o no) con vértices V y aristas E, escribimos $G = (V, E)$.

A menos que se indique lo contrario, supondremos que los conjuntos E y V son finitos y que V es no vacío.

EJEMPLO 6.1.2

En la figura 6.1.2 la gráfica (no dirigida) G consta del conjunto

$$V = \{Gre, She, Wor, Buf, Gil, Sho, Cas, Dou, Lan, Mud\}$$

de vértices y el conjunto

$$E = \{e_1, e_2, \ldots, e_{13}\}$$

de aristas. La arista e_1 se asocia con el par no ordenado $\{Gre, She\}$ de vértices y la arista e_{10} se asocia con el par no ordenado $\{Cas, Dou\}$ de vértices. La arista e_1 se denota (Gre, She) o (She, Gre) y la arista e_{10} se denota (Cas, Dou) o (Dou, Cas). La arista e_4 es incidente en *Wor* y *Buf* y los vértices *Wor* y *Buf* son adyacentes. □

EJEMPLO 6.1.3

La figura 6.1.4 muestra una gráfica dirigida. Las aristas dirigidas se indican mediante flechas. La arista e_1 se asocia con el par ordenado (v_2, v_1) de vértices y la arista e_7 se asocia con el par ordenado (v_6, v_6) de vértices. La arista e_1 se denota (v_2, v_1) y la arista e_7 se denota (v_6, v_6). □

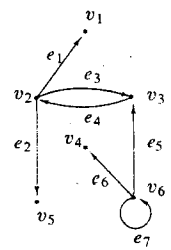


FIGURA 6.1.4 Una gráfica dirigida.

La definición 6.1.1 permite asociar aristas distintas con el mismo par de vértices. Por ejemplo, en la figura 6.1.5 siguiente, las aristas e_1 y e_2 están asociadas con el par de vértices $\{v_1, v_2\}$. Tales **aristas** son **paralelas**. Una arista incidente en un único vértice es un **lazo**. Por ejemplo, en la figura 6.1.5, la arista $e_3 = (v_2, v_2)$ es un lazo. Un vértice, como el vértice v_4 de la figura 6.1.5, que no es incidente en arista alguna es un **vértice aislado**. Una gráfica sin lazos ni aristas paralelas es una **gráfica simple**.

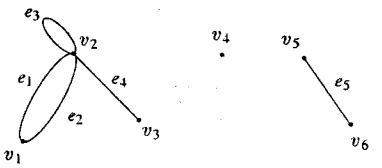


FIGURA 6.1.5 Una gráfica con aristas paralelas y lazos.

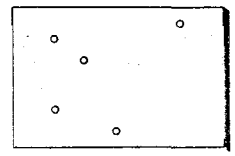


FIGURA 6.1.6 Una hoja de metal con agujeros para tornillos.

EJEMPLO 6.1.4

Como la gráfica de la figura 6.1.2 no tiene aristas paralelas ni lazos, es una gráfica simple. □

Algunos autores no permiten la existencia de lazos o aristas paralelas al definir las gráficas. Sería de esperar que si no existe un acuerdo en la definición de gráfica, muchos otros términos en teoría de gráficas no tendrán definiciones estándar. Esto realmente ocurre. Al leer artículos o libros relativos a las gráficas, es necesario verificar las definiciones utilizadas.

Ahora veremos un ejemplo que muestra la forma de utilizar un modelo de gráfica para analizar un problema de producción.

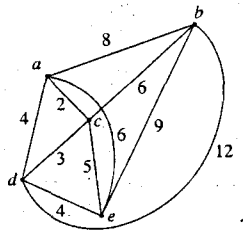


FIGURA 6.1.7
Un modelo de gráfica de la hoja de metal de la figura 6.1.6. El peso de cada arista es el tiempo necesario para mover el taladro.

EJEMPLO 6.1.5

En un proceso de producción, con frecuencia es necesario realizar muchos agujeros en hojas de metal (véase la figura 6.1.6). Los componentes se pueden atornillar entonces en estas hojas de metal. Los agujeros se pueden realizar bajo el control de una computadora. Para ahorrar tiempo y dinero, el taladro debe moverse lo más rápido posible. Modelaremos esta situación como una gráfica.

Los vértices de la gráfica corresponden a los agujeros (véase la figura 6.1.7). Cada par de vértices se une mediante una arista. En cada arista escribimos el tiempo necesario para mover el taladro entre los agujeros correspondientes. Una gráfica con números sobre las aristas (como la gráfica de la figura 6.1.7) se llama una **gráfica con pesos**. Si la arista e tiene la etiqueta k, decimos que el **peso de la arista** e es k. Por ejemplo, en la figura 6.1.7, el peso de la arista (c, e) es 5. En una gráfica con pesos, la **longitud de un camino** es la suma de los pesos de las aristas en el camino. Por ejemplo, en la figura 6.1.7, la longitud del camino que comienza en a, visita c y termina en b es 8. En este problema, la longitud de un camino que comienza en el vértice v_1, luego visita $v_2, v_3, \ldots$, en ese orden, y termina en v_n representa el tiempo que tarda el taladro en partir del agujero h_1, luego pasar por los agujeros $h_2, h_3, \ldots$, en ese orden, y terminar en h_n, donde el agujero h_i corresponde al vértice v_i. Un camino de longitud mínima que visite cada vértice exactamente una vez representa el camino óptimo que debe seguir el taladro.

Supongamos que en este problema se pide que el camino comience en el vértice a y termine en el vértice e. Podemos determinar el camino de longitud mínima enumerando todos los caminos posibles de a a e que pasen por cada vértice exactamente una vez y elegir el más corto (véase la tabla 6.1.1). Vemos que el camino que visita los vértices a, b, c, d, e, en ese orden, tiene longitud mínima. Por supuesto, un par diferente de vértices inicial y final podría producir un camino más corto. □

TABLA 6.1.1
Caminos en la gráfica de la figura 6.1.7 de a a e que pasan por cada vértice exactamente una vez, y sus longitudes

Camino	*Longitud*
a, b, c, d, e	21
a, b, d, c, e	28
a, c, b, d, e	24
a, c, d, b, e	26
a, d, b, c, e	27
a, d, c, b, e	22

La enumeración de todos los caminos del vértice v al vértice w, como en el ejemplo 6.1.5, es una forma un tanto lenta de determinar un camino de longitud mínima de v a w que visite cada vértice exactamente una vez. Por desgracia, no se conoce un método mucho más práctico para gráficas arbitrarias. Este problema es una versión del **problema del agente de ventas viajero**. Analizaremos este problema en la sección 6.3.

EJEMPLO 6.1.6 ***Gráficas de similaridad***

Este ejemplo se refiere al problema de agrupar objetos "semejantes" en clases, con base en las propiedades de los objetos. Por ejemplo, supongamos que un algoritmo particular se implanta en C por cierto grupo de personas y que queremos agrupar a los programas "semejantes" en clases con base en ciertas propiedades de los programas (véase la tabla 6.1.2). Supongamos que elegimos como propiedades

1. El número de líneas en el programa.
2. El número de enunciados return en el programa.
3. El número de llamadas a función en el programa.

TABLA 6.1.2
Programas en C que implantan el mismo algoritmo

Programa	*Número de líneas del programa*	*Número de enunciados* return	*Número de llamadas a función*
1	66	20	1
2	41	10	2
3	68	5	8
4	90	34	5
5	75	12	14

Construimos una **gráfica de similaridad** G como sigue. Los vértices corresponden a los programas. Un vértice se denota (p_1, p_2, p_3), donde p_i es el valor de la propiedad i. Definimos una **función de disimilaridad** s como sigue. Para cada par de vértices $v = (p_1, p_2, p_3)$ y $w = (q_1, q_2, q_3)$, hacemos

$$s(v, w) = |p_1 - q_1| + |p_2 - q_2| + |p_3 - q_3|.$$

Si v_i es el vértice correspondiente al programa i, obtenemos

$$s(v_1, v_2) = 36, \quad s(v_1, v_3) = 24, \quad s(v_1, v_4) = 42, \quad s(v_1, v_5) = 30,$$
$$s(v_2, v_3) = 38, \quad s(v_2, v_4) = 76, \quad s(v_2, v_5) = 48, \quad s(v_3, v_4) = 54,$$
$$s(v_3, v_5) = 20, \quad s(v_4, v_5) = 46.$$

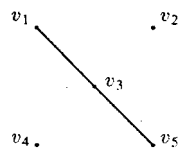


FIGURA 6.1.8
Una gráfica de similaridad correspondiente a los programas de la tabla 6.1.2 con $S = 25$.

Si v y w son vértices correspondientes a dos programas, $s(v, w)$ es una medida de la disimilaridad de los programas. Un valor grande de $s(v, w)$ indica una disimilaridad y un valor pequeño indica una similaridad.

Para un número fijo S, insertamos una arista entre los vértices v y w si $s(v, w) < S$. (En general, habrá diferentes gráficas de similaridad para diversos valores de S.) Decimos que v y w están **en la misma clase** si $v = w$ o si existe un camino de v a w. En la figura 6.1.8 mostramos la gráfica correspondiente a los programas de la tabla 6.1.2 con $S = 25$. En esta gráfica, los programas se agrupan en tres clases: $\{1, 3, 5\}$, $\{2\}$ y $\{4\}$. En un problema real, se podría elegir un valor adecuado de S por prueba y error, o bien elegir el valor de S de manera automática según ciertos criterios determinados con anterioridad. □

El ejemplo 6.1.6 pertenece al tema del **reconocimiento de patrones**, el cual trata de la agrupación de datos en clases con base en propiedades de los datos. El reconocimiento de patrones por computadora tiene mucha importancia práctica. Por ejemplo, se han programado computadoras para detectar el cáncer mediante rayos X, para elegir las declaraciones de impuestos por revisar, para analizar imágenes obtenidas por satélites, para reconocer textos y para predecir el clima.

EJEMPLO 6.1.7 ***El n-cubo (hipercubo)***

La computadora tradicional, llamada **computadora en serie**, ejecuta una instrucción a la vez. Nuestra definición de algoritmo también supone la ejecución de una instrucción a la vez. Tales algoritmos son **algoritmos en serie**. En los últimos años, al disminuir el costo del hardware, se ha vuelto factible construir **computadoras paralelas** con muchos procesadores, las cuales pueden ejecutar varias instrucciones a la vez. Con frecuencia, las gráficas son modelos convenientes para la descripción de estas máquinas. Los algoritmos asociados se conocen como **algoritmos paralelos**. Muchos problemas se pueden resolver más rápido mediante computadoras en paralelo. Analizaremos un modelo para la computación paralela conocido como el ***n*-cubo** o **hipercubo**.

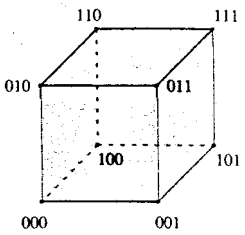


FIGURA 6.1.9
El 3-cubo.

El n-cubo tiene 2^n procesadores, $n \geq 1$, que se representan mediante vértices (véase la figura 6.1.9) con las etiquetas $0, 1, \ldots, 2^n - 1$. Cada procesador tiene su propia memoria local. Una arista conecta dos vértices si la representación binaria de sus etiquetas difiere en exactamente un bit. Durante una unidad de tiempo, todos los procesadores del n-cubo pueden ejecutar una instrucción de manera simultánea y luego comunicarse con un procesador adyacente. Si un procesador necesita comunicarse con un procesador no adyacente, el primero envía un mensaje con la ruta hasta el receptor. Tal vez se necesiten varias unidades de tiempo para comunicarse con un procesador no adyacente.

También podemos describir al n-cubo de manera recursiva. El 1-cubo tiene dos procesadores, etiquetados 0 y 1, y una arista. Sean H_1 y H_2 dos $(n-1)$-cubos cuyos vértices están etiquetados en binario $0, \ldots, 2^{n-1} - 1$ (véase la figura 6.1.10). Colocamos una arista entre cada par de vértices, uno de H_1 y uno de H_2, siempre que los vértices tengan etiquetas idénticas. Luego cambiamos la etiqueta L de cada vértice de H_1 por $0L$ y la etiqueta L de cada vértice de H_2 por $1L$. Obtenemos un n-cubo (ejercicio 32).

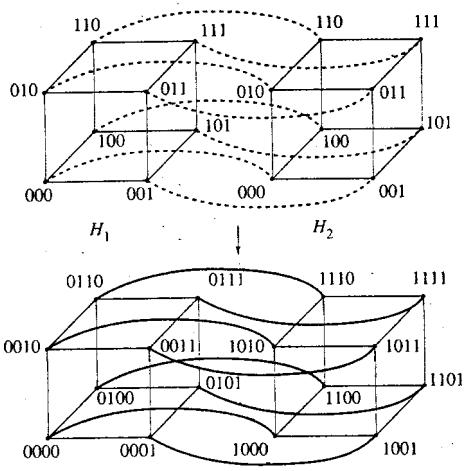
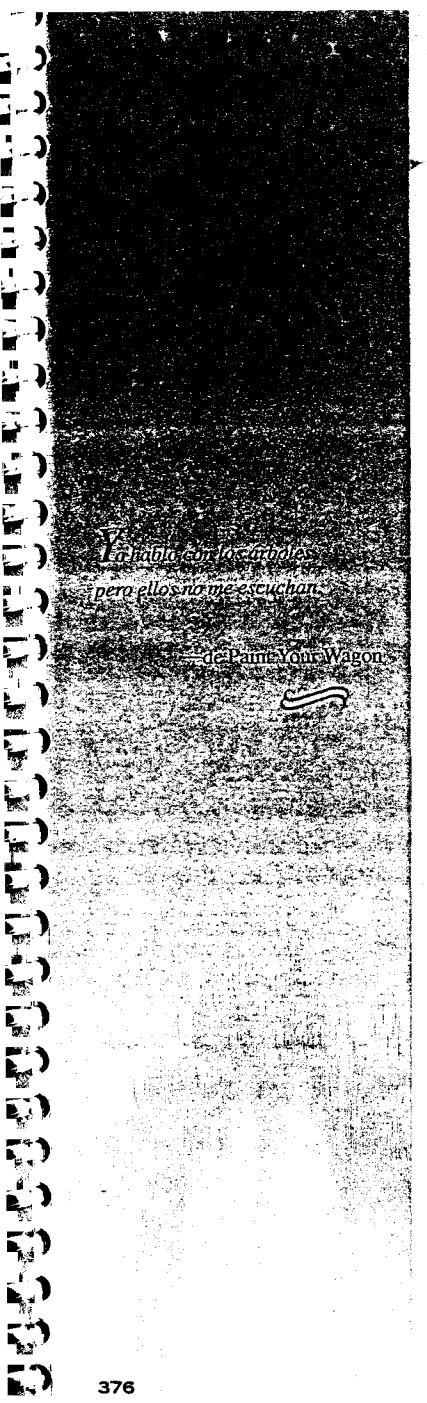


FIGURA 6.1.10 Combinación de dos 3-cubos para obtener un 4-cubo.



7 ÁRBOLES

7.1 INTRODUCCIÓN
7.2 TERMINOLOGÍA Y CARACTERIZACIONES DE LOS ÁRBOLES
RINCÓN DE SOLUCIÓN DE PROBLEMAS: ÁRBOLES
7.3 ÁRBOLES DE EXPANSIÓN
7.4 ÁRBOLES DE EXPANSIÓN MÍNIMOS
7.5 ÁRBOLES BINARIOS
7.6 RECORRIDOS DE UN ÁRBOL
7.7 ÁRBOLES DE DECISIÓN Y EL TIEMPO MÍNIMO PARA EL ORDENAMIENTO
7.8 ISOMORFISMOS DE ÁRBOLES
† 7.9 ÁRBOLES DE JUEGOS
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Los árboles forman una de las subclases de las gráficas de uso más amplio. En particular, en la computación se hace un uso amplio de los árboles. En este terreno, los árboles sirven para organizar y relacionar los datos en una base de datos (véase el ejemplo 7.1.6). También surgen en problemas teóricos, como el tiempo óptimo para el ordenamiento (véase la sección 7.7).

En este capítulo estudiaremos primero la terminología necesaria. Veremos algunas subclases de árboles (por ejemplo, árboles con raíz y árboles binarios) y muchas aplicaciones de los árboles (por ejemplo, árboles de expansión o generadores, árboles de decisión y árboles de juegos). Nuestro análisis de los isomorfismos de árboles amplía el análisis de la sección 6.6 acerca de los isomorfismos de gráficas.

† Esta sección se puede omitir sin pérdida de continuidad.

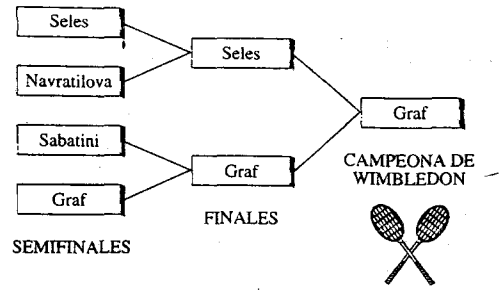


FIGURA 7.1.1 Semifinales y finales en Wimbledon.

7.1 *INTRODUCCIÓN*

La figura 7.1.1 muestra los resultados de las semifinales y finales de una competencia de tenis en Wimbledon, en la cual participaron cuatro de las mejores jugadoras en la historia del tenis. En Wimbledon, cuando una jugadora pierde, queda fuera del torneo. Las ganadoras continúan jugando hasta que sólo queda una, la campeona. (Esta competencia se llama *torneo de eliminación simple*.) La figura 7.1.1 muestra que en las semifinales, Mónica Seles derrotó a Martina Navratilova y Steffi Graf derrotó a Gabriela Sabatini. Luego jugaron las ganadoras, Seles y Graf, y Graf derrotó a Seles. Steffi Graf, al ser la única jugadora que no fue derrotada, se convirtió en campeona de Wimbledon.

Si consideramos el torneo de eliminación simple de la figura 7.1.1 como una gráfica (véase la figura 7.1.2), obtenemos un **árbol**. Si giramos la figura 7.1.2, se ve como un árbol natural (véase la figura 7.1.3). A continuación damos la definición formal.

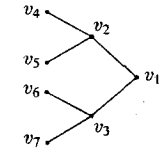


FIGURA 7.1.2 El torneo de la figura 7.1.1 como un árbol.

DEFINICIÓN 7.1.1

Un *árbol (libre)* T es una gráfica simple que satisface: Si v y w son vértices en T, entonces existe un único camino simple de v a w.

Un *árbol con raíz* es un árbol en el cual un vértice particular se designa como la raíz.

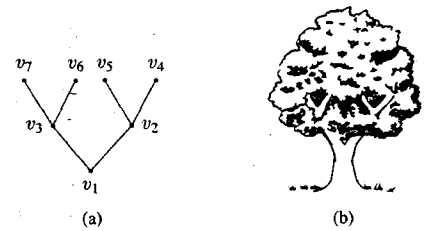


FIGURA 7.1.3 El árbol de la figura 7.1.2 girado (a) comparado con un árbol natural (b).

EJEMPLO 7.1.2

Si designamos a la ganadora como la raíz, el torneo de eliminación simple de la figura 7.1.1 (o la figura 7.1.2) es un árbol con raíz. Observe que si v y w son vértices de esta gráfica, existe un único camino simple de v a w. Por ejemplo, el único camino simple de v_2 a v_7 es (v_2, v_1, v_3, v_7). □

En contraste con los árboles naturales, que tienen su raíz en la parte inferior, en la teoría de gráficas se acostumbra trazar los árboles con raíz con su raíz en la parte superior. La figura 7.1.4 muestra la forma en que se trazaría el árbol de la figura 7.1.2 (con v_1 como raíz). En primer lugar, colocamos la raíz v_1 en la parte superior. Debajo de la raíz y en el mismo nivel, colocamos los vértices v_2 y v_3, los cuales se pueden alcanzar desde la raíz mediante un camino simple de longitud 1. Debajo de estos vértices y en el mismo nivel, colocamos los vértices v_4, v_5, v_6 y v_7, los cuales se pueden alcanzar desde la raíz mediante un camino simple de longitud 2. Continuamos de esta forma hasta trazar todo el árbol. Como el camino simple de la raíz a cualquier vértice dado es único, cada vértice está en un nivel determinado de manera única. Decimos que el nivel de la raíz es el nivel 0. Los vértices debajo de la raíz están en el nivel 1, y así sucesivamente. Así, el **nivel de un vértice** v es la longitud del camino simple de la raíz a v. La **altura** de un árbol con raíz es el número máximo de nivel que aparece en dicho árbol.

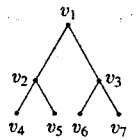


FIGURA 7.1.4
El árbol de la figura 7.1.3(a) con la raíz en la parte superior.

EJEMPLO 7.1.3

Los vértices $v_1, v_2, v_3, v_4, v_5, v_6, v_7$ en el árbol con raíz de la figura 7.1.4 están en los niveles 0, 1, 1, 2, 2, 2, 2, respectivamente. La altura del árbol es 2. □

EJEMPLO 7.1.4

Si designamos e como la raíz del árbol T de la figura 7.1.5, obtenemos el árbol con raíz T' que aparece en la figura 7.1.5. Los vértices $a, b, c, d, e, f, g, h, i, j$ están en los niveles 2, 1, 2, 1, 0, 1, 1, 2, 2, 3, respectivamente. La altura de T' es 3. □

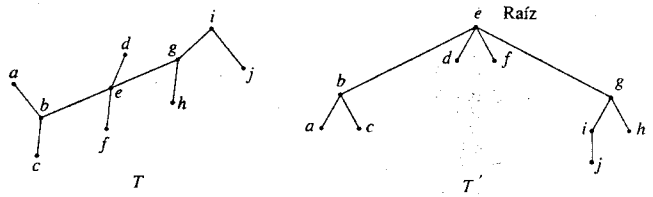


FIGURA 7.1.5 Un árbol T y un árbol con raíz T'. T' se obtiene de T designando a e como la raíz.

EJEMPLO 7.1.5

Con frecuencia, un árbol con raíz se utiliza para especificar relaciones jerárquicas. Cuando un árbol se utiliza de esta manera, si el vértice a está en el siguiente nivel arriba del vértice b y a y b son adyacentes, entonces a está "justo arriba" de b y existe una relación lógica entre a y b: a domina a b o b está subordinado a a de alguna manera. La figura 7.1.6 exhibe un ejemplo de este tipo de árbol, el organigrama de una universidad hipotética. □

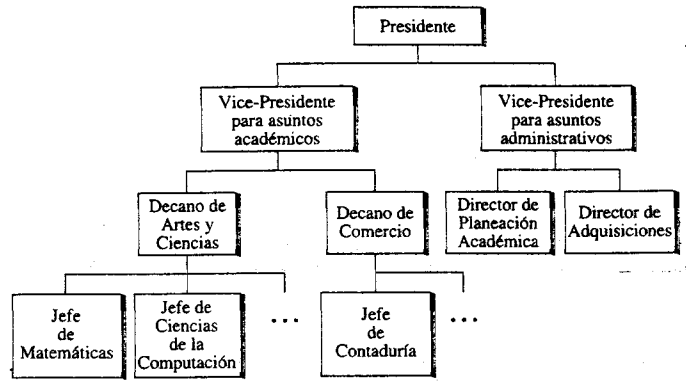


FIGURA 7.1.6 Ejemplo de un organigrama.

EJEMPLO 7.1.6 *Árboles de definición jerárquica*

La figura 7.1.7 es un ejemplo de un **árbol de definición jerárquica**. Tales árboles se utilizan para mostrar las relaciones lógicas entre los registros en una base de datos. [Recuerde (véase la sección 2.7) que una base de datos es una colección de registros controlados por una computadora.] El árbol de la figura 7.1.7 se podría utilizar como modelo para configurar una base de datos para los registros de libros existentes en diversas bibliotecas. □

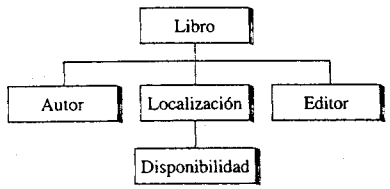


FIGURA 7.1.7 Un árbol de definición jerárquica.

EJEMPLO 7.1.7 *Códigos de Huffman*

La forma más común de representar los caracteres internamente en una computadora es utilizar cadenas de bits de longitud fija. Por ejemplo, ASCII (Código americano estándar

para el intercambio de información) representa cada carácter mediante una cadena de siete bits. Algunos ejemplos aparecen en la tabla 7.1.1.

TABLA 7.1.1
Una parte de la tabla ASCII

Carácter	*Código ASCII*
A	100 0001
B	100 0010
C	100 0011
1	011 0001
2	011 0010
!	010 0001
*	010 1010

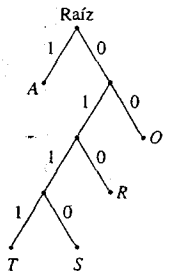


FIGURA 7.1.8
Un código de Huffman.

Los **códigos de Huffman** representan los caracteres mediante cadenas de bits de longitud variable y proporcionan una alternativa para ASCII y otros códigos de longitud fija. La idea consiste en utilizar cadenas cortas de bits para los caracteres de uso frecuente y utilizar cadenas de bits de mayor tamaño para representar caracteres de uso menos frecuente. De esta forma, por lo general es posible representar cadenas de caracteres, como texto y programas, en un menor espacio comparado con el espacio necesario al utilizar ASCII. VCR Plus+, un dispositivo que programa de manera automática una grabadora de videocasetes, utiliza un código de Huffman para generar los números mediante los cuales el usuario elige los programas que desea grabar. Los números se publican en la programación de la televisión.

Un código de Huffman se define fácilmente mediante un árbol con raíz (véase la figura 7.1.8). Para decodificar una cadena de bits, comenzamos en la raíz y nos movemos hacia abajo en el árbol hasta encontrar un carácter. El bit 0 o 1 nos indica si debemos movernos a la izquierda o a la derecha. Por ejemplo, decodifiquemos la cadena

$$01010111. \tag{7.1.1}$$

Comenzamos en la raíz. Como el primer bit es 0, el primer movimiento es hacia la derecha. A continuación, nos movemos a la izquierda y luego a la derecha. En este momento, encontramos el primer carácter *R*. Para decodificar el siguiente carácter, comenzamos de nuevo en la raíz. El siguiente bit es 1, de modo que nos movemos hacia la izquierda y encontramos el siguiente carácter *A*. Los últimos bits 0111 se decodifican como *T*. Por tanto, la cadena de bits (7.1.1) representan la palabra *RAT*.

Dado un árbol que define un código de Huffman, como la figura 7.1.8, cualquier cadena de bits [por ejemplo, (7.1.1)] se puede decodificar de manera única, aunque los caracteres sean representados mediante cadenas de bits de longitud variable. Para el código de Huffman definido mediante el árbol de la figura 7.1.8, el carácter *A* se representa mediante una cadena de bits de longitud 1, mientras que *S* y *T* se representan mediante cadenas de bits de longitud 4. (*A* se representa como 1, *S* como 0110 y *T* como 0111.) □

Huffman dio un algoritmo (algoritmo 7.1.8) para construir un código de Huffman a partir de una tabla con la frecuencia de aparición de los caracteres que se desea representar, de modo que el código represente cadenas de caracteres en un mínimo espacio, siempre que las cadenas por representar tengan la frecuencia de los caracteres idéntica a la frecuencia de los caracteres en la tabla. Una demostración de que el código construido es óptimo aparece en [Cormen].

ALGORITMO 7.1.8 ***Construcción de un código de Huffman óptimo***

Este algoritmo construye un código de Huffman óptimo a partir de una tabla que contiene la frecuencia de aparición de los caracteres por representar. La salida es un árbol con raíz tal que los vértices en los niveles más bajos se etiquetan con las frecuencias y las aristas se etiquetan con bits, como en la figura 7.1.8. El árbol de codificación se obtiene reemplazando cada frecuencia por un carácter que tenga esa frecuencia.

Entrada: Una serie de n frecuencias, $n \geq 2$

Salida: Un árbol con raíz que define un código de Huffman óptimo

procedure *huffman* (f, n)
  if $n = 2$ **then**
    begin
    sean f_1 y f_2 las frecuencias
    sea T como en la figura 7.1.9
    return(T)
    end
  sean f_i y f_j las frecuencias menores
  reemplazar f_i y f_j en la lista f por $f_i + f_j$
  $T' :=$ *huffman* (f, $n - 1$)
  reemplazar un vértice en T' etiquetado con $f_i + f_j$ por el árbol que aparece en la figura 7.1.10 para obtener el árbol T
  return(T)
end *huffman*

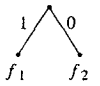


FIGURA 7.1.9
El caso $n = 2$ para el algoritmo 7.1.8.

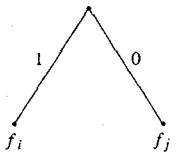


FIGURA 7.1.10 El caso $n > 2$ para el algoritmo 7.1.8.

EJEMPLO 7.1.9

Ahora mostraremos la forma en que el algoritmo 7.1.8 construye un código de Huffman óptimo para la tabla 7.1.2.

TABLA 7.1.2
Una parte de la tabla ASCII

Carácter	*Frecuencia*
!	2
@	3
#	7
$	8
%	12

El algoritmo comienza reemplazando varias veces las dos frecuencias menores con su suma, hasta obtener una sucesión de dos elementos:

$$2, 3, 7, 8, 12 \rightarrow 2 + 3, 7, 8, 12$$
$$5, 7, 8, 12 \rightarrow 5 + 7, 8, 12$$
$$8, 12, 12 \rightarrow 8 + 12, 12$$
$$12, 20$$

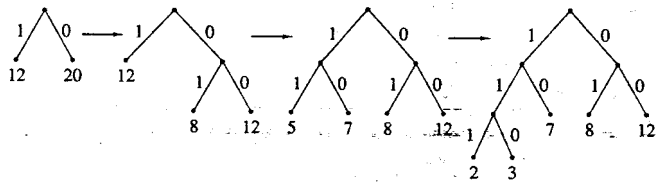


FIGURA 7.1.11 Construcción de un código de Huffman óptimo.

Luego, el algoritmo construye árboles trabajando hacia atrás, comenzando con la sucesión de dos elementos 12, 20 que aparece en la figura 7.1.11. Por ejemplo, el segundo árbol se obtiene del primero reemplazando el vértice con la etiqueta 20 por el árbol de la figura 7.1.12, pues 20 aparece como la suma de 8 y 12. Por último, para obtener el árbol de codificación de Huffman óptimo, reemplazamos cada frecuencia por un carácter que tenga esa frecuencia (véase la figura 7.1.13).

FIGURA 7.1.12
El árbol que reemplaza al vértice con la etiqueta 20 en la figura 7.1.11.

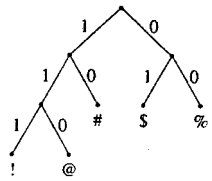


FIGURA 7.1.13
El árbol final de la figura 7.1.11, donde cada frecuencia ha sido reemplazada por un carácter que tenga esa frecuencia.

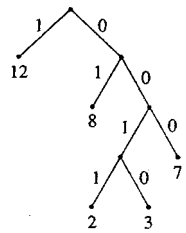
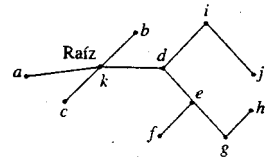


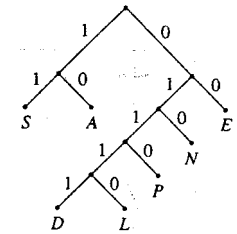
FIGURA 7.1.14
Otro árbol de Huffman óptimo para el ejemplo 7.1.9.

Observe que el árbol de Huffman para la tabla 7.1.2 no es único. Al reemplazar 12 por 5, 7, pues hay dos vértices etiquetados 12, hay que tomar una decisión. En la figura 7.1.11, elegimos de manera arbitraria uno de los vértices etiquetados con 12. Si elegimos el otro vértice etiquetado con 12, obtenemos el árbol de la figura 7.1.14. Cualquier árbol de Huffman proporciona un código óptimo; es decir, cualquier árbol de Huffman codificará un texto que tenga las frecuencias de la tabla 7.1.2 exactamente en el mismo espacio (óptimo). □

Ejercicios



1. Determine el nivel de cada vértice en el árbol de la figura anexa.
2. Determine la altura del árbol de la figura anexa.
3. Trace el árbol T de la figura 7.1.5 como un árbol con raíz, con a como raíz. ¿Cuál es la altura del árbol resultante?
4. Trace el árbol T de la figura 7.1.5 como un árbol con raíz, con b como raíz. ¿Cuál es la altura del árbol resultante?
5. Proporcione un ejemplo similar al ejemplo 7.1.5 de un árbol que se utilice para especificar relaciones jerárquicas.
6. Proporcione un ejemplo distinto al ejemplo 7.1.6 de un árbol de definición jerárquica.



Decodifique cada cadena de bits mediante el código de Huffman de la figura anexa.

7. 011000010
8. 01110100110
9. 01111001001110
10. 1110011101001111

Codifique cada palabra mediante el código de Huffman anterior.

11. DEN
12. NEED
13. LEADEN
14. PENNED

15. ¿Cuáles factores (además de la cantidad de memoria utilizada) deben tomarse en cuenta al elegir un código, como el ASCII o un código de Huffman, para representar los caracteres en una computadora?
16. ¿Cuáles técnicas, además del uso de códigos de Huffman, se podrían utilizar para ahorrar memoria al guardar un texto?
17. Construya un código de Huffman óptimo para el conjunto de letras en la tabla.

Letra	*Frecuencia*
α	5
β	6
γ	6
δ	11
ε	20

18. Construya un código de Huffman óptimo para el conjunto de letras en la tabla.

Letra	*Frecuencia*
I	7.5
U	20.0
B	2.5
S	27.5
C	5.0
H	10.0
M	2.5
P	25.0

19. Utilice el código desarrollado en el ejercicio 18 para codificar las siguientes palabras (las cuales tienen frecuencias consistentes con la tabla del ejercicio 18):

BUS, CUPS, MUSH, PUSS, SIP, PUSH, CUSS, HIP, PUP, PUPS, HIPS.

20. Construya dos árboles de codificación de Huffman óptimos para la tabla del ejercicio 17, de diferentes alturas.

21. El profesor Gig A. Byte necesitaba guardar texto formado mediante los caracteres A, B, C, D, E, los cuales aparecen con las siguientes frecuencias:

Carácter	*Frecuencia*
A	6
B	2
C	3
D	2
E	8

El profesor Byte sugiere el uso de los códigos de longitud variable

Carácter	*Código*
A	1
B	00
C	01
D	10
E	0

los cuales, argumenta, guardan el texto en un espacio menor que el utilizado por un código de Huffman óptimo. ¿Tiene razón el profesor? Explique.

22. Muestre que cualquier árbol con dos o más vértices tiene un vértice de grado 1.
23. Muestre que un árbol es una gráfica plana.
24. Muestre que un árbol es una gráfica bipartita.
25. Muestre que los vértices de un árbol pueden distinguirse con dos colores, de modo que cada arista sea incidente en vértices de colores distintos.

La *excentricidad* de un vértice v en un árbol T es la longitud máxima de un camino simple que comienza en v.

26. Determine la excentricidad de cada vértice en el árbol de la figura 7.1.5.

Un vértice v en un árbol T es un *centro* para T si la excentricidad de v es mínima.

27. Determine el centro (o centros) del árbol de la figura 7.1.5.
☆ 28. Muestre que un árbol tiene uno o dos centros.
☆ 29. Muestre que si un árbol tiene dos centros, éstos son adyacentes.
30. Defina el radio r de un árbol mediante los conceptos de excentricidad y centro. El diámetro d de cualquier gráfica se definió antes del ejercicio 70 de la sección 6.2. ¿Es siempre cierto, de acuerdo con su definición de radio, que $2r = d$? Explique.
31. Proporcione un ejemplo de árbol T que no satisfaga la propiedad: Si v y w son vértices en T, existe un único camino de v a w.

7.2 *TERMINOLOGÍA Y CARACTERIZACIONES DE LOS ÁRBOLES*

Una parte del árbol genealógico de los antiguos dioses griegos aparece en la figura 7.2.1. (No aparecen todos los hijos.) Como se muestra, un árbol genealógico se puede considerar como un árbol con raíz. Los vértices adyacentes a un vértice v y en el siguiente nivel hacia abajo son los hijos de v. Por ejemplo, los hijos de Cronos son Zeus, Poseidón, Hades y Ares. La terminología adaptada de un árbol genealógico se utiliza de manera normal en cualquier árbol con raíz. A continuación damos las definiciones formales.

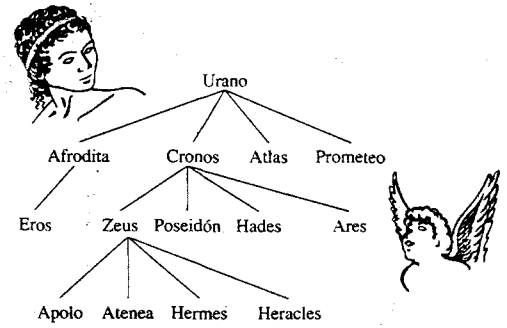


FIGURA 7.2.1 Una parte del árbol genealógico de los antiguos dioses griegos.

DEFINICIÓN 7.2.1

Sea T un árbol con raíz v_0. Suponga que x, y y z son vértices en T y que $(v_0, v_1, \ldots, v_n)$ es un camino simple en T. Entonces

(a) v_{n-1} es el *padre* de v_n.

(b) $v_0, \ldots, v_{n-1}$ son *ancestros* de v_n.

(c) v_n es un *hijo* de v_{n-1}.

(d) Si x es un ancestro de y, y es un *descendiente* de x.

(e) Si x y y son hijos de z, x y y son *hermanos*.

(f) Si x no tiene hijos, x es un *vértice terminal* (o una *hoja*).

(g) Si x no es un vértice terminal, x es un *vértice interno* (o una *rama*).

(h) El *subárbol de T con raíz en x* es la gráfica con conjunto de vértices V y conjunto de aristas E, donde V es x junto con los descendientes de x y

$$E = \{e \mid e \text{ es una arista en un camino simple de } x \text{ a algún vértice en } V\}.$$

EJEMPLO 7.2.2

En el árbol con raíz de la figura 7.2.1,

(a) El padre de Eros es Afrodita.

(b) Los ancestros de Hermes son Zeus, Cronos y Urano.

(c) Los hijos de Zeus son Apolo, Atenea, Hermes y Heracles.

(d) Los descendientes de Cronos son Zeus, Poseidón, Hades, Ares, Apolo, Atenea, Hermes y Heracles.

(e) Afrodita y Prometeo son hermanos.

(f) Los vértices terminales son Eros, Apolo, Atenea, Hermes, Heracles, Poseidón, Hades, Ares, Atlas y Prometeo.

(g) Los vértices internos son Urano, Afrodita, Cronos y Zeus.

(h) El subárbol con raíz en Cronos aparece en la figura 7.2.2. □

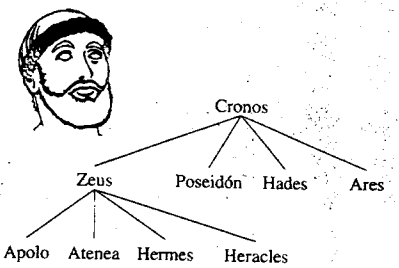


FIGURA 7.2.2 El subárbol con raíz en Cronos del árbol de la figura 7.2.1.

Dedicamos el resto de esta sección a algunas caracterizaciones alternativas de los árboles. Sea T un árbol. Observamos que T es conexo, pues existe un camino simple de cualquier vértice a cualquier otro vértice. Además, podemos mostrar que T no tiene ciclos. Para ver esto, suponga que T tiene un ciclo C'. Por el teorema 6.2.24, T tiene un ciclo simple (véase la figura 7.2.3)

$$C = (v_0, \ldots, v_n),$$

con $v_0 = v_n$. Como T es una gráfica simple, C no puede ser un lazo; así, C contiene al menos dos vértices distintos v_i y v_j, con $i < j$. Ahora,

$$(v_i, v_{i+1}, \ldots, v_j), \quad (v_i, v_{i-1}, \ldots, v_0, v_{n-1}, \ldots, v_j)$$

son caminos simples distintos de v_i a v_j, lo cual contradice la definición de árbol. Por tanto, un árbol no puede contener un ciclo.

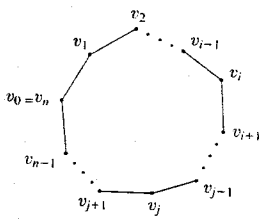


FIGURA 7.2.3
Un ciclo simple.

Una gráfica sin ciclos es una **gráfica acíclica**. Hemos mostrado que un árbol es una gráfica conexa acíclica. El recíproco también es cierto; toda gráfica conexa acíclica es un árbol. El siguiente teorema da ésta y otras caracterizaciones de los árboles.

TEOREMA 7.2.3

Sea T una gráfica con n vértices. Las siguientes afirmaciones son equivalentes.

(a) T es un árbol.

(b) T es conexa y acíclica.

(c) T es conexa y tiene $n - 1$ aristas.

(d) T es acíclica y tiene $n - 1$ aristas.

Demostración. Para mostrar que (a)−(d) son equivalentes, demostraremos cuatro resultados: si (a), entonces (b); si (b), entonces (c); si (c), entonces (d); y si (d), entonces (a).

[Si (a), entonces (b).] La demostración de este resultado se dio antes del enunciado del teorema.

[Si (b), entonces (c).] Suponga que T es conexa y acíclica. Demostraremos que T tiene $n - 1$ aristas por inducción sobre n.

Si $n = 1$, T consta de un vértice y cero aristas, por lo que el resultado es verdadero si $n = 1$.

Ahora suponga que el resultado es válido para una gráfica conexa, acíclica con n vértices. Sea T una gráfica conexa, acíclica, con $n + 1$ vértices. Sea P un camino simple de longitud máxima. Como T es acíclica, P no es un ciclo. Por tanto, P contiene un vértice v de grado 1 (véase la figura 7.2.4). Sea T^* el árbol obtenido de T al eliminar v y la arista incidente en v. Entonces T^* es conexa y acíclica, y como T^* contiene n vértices, por la hipótesis de inducción T^* contiene $n - 1$ aristas. Por tanto, T tiene n aristas. El argumento inductivo está completo y concluye esta parte de la demostración.

[Si (c), entonces (d).] Suponga que T es conexa y que tiene $n - 1$ aristas. Debemos mostrar que T es acíclica.

Supongamos que T tiene al menos un ciclo. Como la eliminación de una arista de un ciclo no desconecta una gráfica, podemos eliminar las aristas, pero no los vértices, de los ciclos de T hasta que la gráfica resultante T^* sea conexa y acíclica. Ahora, T^* es una gráfica conexa acíclica con n vértices. Podemos utilizar el resultado recién demostrado, (b) implica (c), para concluir que T^* tiene $n - 1$ aristas. Pero entonces T tiene más de $n - 1$ aristas. Esto es una contradicción. Por tanto, T es acíclica y concluye esta parte de la demostración.

[Si (d), entonces (a).] Suponga que T es acíclica y tiene $n - 1$ aristas. Debemos mostrar que T es un árbol; es decir, que T es una gráfica simple y que T tiene un único camino simple desde cualquier vértice hasta cualquier otro vértice.

La gráfica T no puede contener lazos, pues los lazos son ciclos y T es acíclica. De manera análoga, T no puede tener aristas distintas e_1 y e_2 incidentes en v y w, pues entonces tendríamos el ciclo (v, e_1, w, e_2, v). Por tanto, T es una gráfica simple.

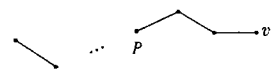


FIGURA 7.2.4
La demostración del teorema 7.2.3 [Si (b), entonces (c).]. P es un camino simple v y la arista incidente en v se eliminan, de modo que se pueda utilizar la hipótesis inductiva.

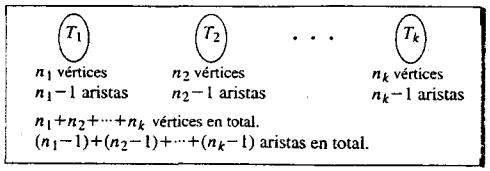


FIGURA 7.2.5 La demostración del teorema 7.2.3 [Si (d), entonces (a).]. Los T_i son componentes de T. T_i tiene n_i vértices y n_i-1 aristas. Surge una contradicción del hecho de que el número total de aristas debe ser igual a $n-1$.

Supongamos, por contradicción, que T no es conexa (véase la figura 7.2.5). Sean

$$T_1, \quad T_2, \quad \ldots, \quad T_k$$

los componentes de T. Como T no es conexa, $k > 1$. Supongamos que T_i tiene n_i vértices. Cada T_i es conexa y acíclica, de modo que podemos utilizar el resultado ya demostrado, (b) implica (c), para concluir que T_i tiene n_i-1 aristas. Entonces

$$\begin{aligned} n-1 &= (n_1-1)+(n_2-1)+\cdots+(n_k-1) && \text{(contando las aristas)} \\ &< (n_1+n_2)+\cdots+n_k)-1 && \text{(pues } k>1\text{)} \\ &= n-1, && \text{(contando los vértices)} \end{aligned}$$

lo cual es imposible. Por tanto, T es conexa.

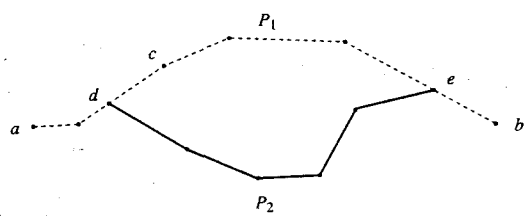


FIGURA 7.2.6 La demostración del teorema 7.2.3 [Si (d), entonces (a).]. P_1 (que aparece punteado) y P_2 (que aparece en línea continua) son caminos simples distintos de a a b. c es el primer vértice posterior a a en P_1 que no está en P_2. d es el vértice que precede a c en P_1. e es el primer vértice posterior a d en P_1 que también está en P_2. Como se muestra, aparece un ciclo, lo cual da una contradicción.

Suponga que existen caminos simples distintos P_1 y P_2 de a a b en T (véase la figura 7.2.6). Sea c el primer vértice posterior a a en P_1 que no está en P_2; sea d el vértice que precede a c en P_1; y sea e el primer vértice posterior a d en P_1 que también está en P_2. Sea

$$(v_0, v_1, \ldots, v_{n-1}, v_n)$$

la parte de P_1 de $d = v_0$ a $e = v_n$. Sea

$$(w_0, w_1, \ldots, w_{m-1}, w_m)$$

la parte de P_2 de $d = w_0$ a $e = w_m$. Entonces

$$(v_0, \ldots, v_n = w_m, w_{m-1}, \ldots, w_1, w_0) \qquad (7.2.1)$$

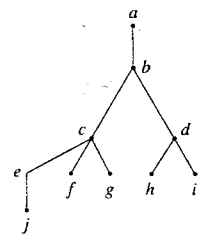
es un ciclo en T, lo cual es una contradicción. [De hecho, (7.2.1) es un ciclo simple, pues ningún vértice está repetido, excepto v_0 y w_0.] Así, existe un único camino simple desde cualquier vértice hasta cualquier otro vértice de T. Por tanto, T es un árbol. Esto concluye la demostración. ■

Ejercicios

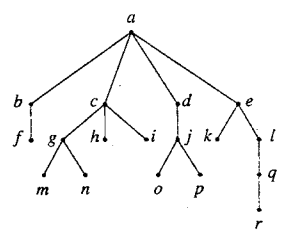
Responda las preguntas de los ejercicios 1-6 para el árbol de la figura 7.2.1.

1. Determine el padre de Poseidón.
2. Determine los ancestros de Eros.
3. Determine los hijos de Urano.
4. Determine los descendientes de Zeus.
5. Determine los hermanos de Ares.
6. Trace el subárbol con raíz en Afrodita.

Responda las preguntas de los ejercicios 7-15 para el árbol anexo.



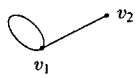
7. Determine los padres de c y de h.
8. Determine los ancestros de c y de j.
9. Determine los hijos de d y de e.
10. Determine los descendientes de c y de e.
11. Determine los hermanos de f y de h.
12. Determine los vértices terminales.
13. Determine los vértices internos.
14. Trace el subárbol con raíz en j.
15. Trace el subárbol con raíz en e.
16. Responda las preguntas de los ejercicios 7-15 para el árbol anexo.



17. ¿Qué podría decir acerca de dos vértices en un árbol con raíz que tengan el mismo padre?
18. ¿Qué podría decir acerca de dos vértices en un árbol con raíz que tengan los mismos ancestros?
19. ¿Qué podría decir acerca de un vértice en un árbol con raíz que no tenga ancestros?
20. ¿Qué podría decir acerca de dos vértices en un árbol con raíz que tengan un descendiente común?
21. ¿Qué podría decir acerca de un vértice en un árbol con raíz que no tenga descendientes?

En los ejercicios 22-26, trace una gráfica con las propiedades dadas o explique por qué no existe tal gráfica.

22. Seis aristas; ocho vértices
23. Acíclica; cuatro aristas, seis vértices
24. Árbol; todos los vértices de grado 2
25. Árbol; seis vértices con grados 1, 1, 1, 1, 3, 3
26. Árbol; cuatro vértices internos; seis vértices terminales
27. Explique por qué si permitimos la existencia de ciclos de longitud 0, una gráfica que consta de un único vértice, sin aristas, no es acíclica.
28. Explique por qué si permitimos que se repitan aristas en los ciclos, una gráfica que consta de una única arista y dos vértices no es acíclica.



29. La gráfica conexa que se muestra aquí tiene un único camino simple de cualquier vértice a cualquier otro vértice pero no es un árbol. Explique.

Un *bosque* es una gráfica simple sin ciclos.

30. Explique por qué un bosque es una unión de árboles.
31. Si un bosque F consta de m árboles y n vértices, ¿cuántas aristas tiene F?
32. Si $P_1 = (v_0, \ldots, v_n)$ y $P_2 = (w_0, \ldots, w_m)$ son caminos simples distintos de a a b en una gráfica simple G, ¿es

$$(v_0, \ldots, v_n = w_m, w_{m-1}, \ldots, w_1, w_0)$$

necesariamente un ciclo? Explique. (Este ejercicio es importante para el último párrafo de la demostración del teorema 7.2.3.)

33. Muestre que una gráfica G con n vértices y menos de $n - 1$ aristas no es conexa.
☆ 34. Demuestre que T es un árbol si y sólo si T es conexa y tal que al agregar una arista entre cualesquiera dos vértices, se crea exactamente un ciclo.
35. Muestre que si G es un árbol, cada vértice de grado mayor o igual que 2 es un punto de articulación. ("Punto de articulación" se definió antes del ejercicio 55 de la sección 6.2.)
36. Proporcione un ejemplo para mostrar que el recíproco del ejercicio 35 es falso, incluso suponiendo que G es conexa.

RINCÓN DE SOLUCIÓN DE PROBLEMAS: ÁRBOLES

Problema

Sea T una gráfica simple. Demuestre que T es un árbol si y sólo si T es conexa, pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta.

Para enfrentar el problema

Debemos saber claramente lo que debemos demostrar. Como la afirmación es "si y sólo si", debemos demostrar dos afirmaciones:

Si T es un árbol, entonces T es conexa pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta. (1)

Si T es conexa pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta, entonces T es un árbol. (2)

En (1), partiendo de la hipótesis de que T es un árbol, debemos demostrar que T es conexa, pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta. En (2), a partir de la hipótesis de que T es conexa pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta, debemos deducir que T es un árbol.

Al desarrollar una demostración, por lo general es útil revisar las definiciones y otros resultados relacionados con las afirmaciones por demostrar. En este caso, la definición de árbol y el teorema 7.2.3, el cual proporciona condiciones equivalentes para que una gráfica sea un árbol, tienen una relación directa con lo que debemos demostrar.

La definición 7.1.1 establece:

Un árbol T es una gráfica simple que satisface: Si v y w son vértices en T, existe un único camino simple de v a w. (3)

El teorema 7.2.3 establece que las siguientes afirmaciones son equivalentes para una gráfica T con n vértices:

T es un árbol. (4)

T es conexa y acíclica. (5)

T es conexa y tiene $n - 1$ aristas. (6)

T es acíclica y tiene $n - 1$ aristas. (7)

Determinación de una solución

Primero intentaremos demostrar (1). Suponemos que T es un árbol. Debemos demostrar dos cosas: que T es conexa y que al eliminar cualquier arista de T (pero no los vértices), T se desconecta.

Las afirmaciones (5) y (6) nos indican de inmediato que T es conexa. Ninguna de las afirmaciones (3) a (7) nos dicen de manera directa algo acerca de la eliminación de aristas o de una gráfica que no sea conexa. Sin embargo, si argumentamos por contradicción y suponemos que al eliminar cualquier arista de T (pero no los vértices), T no se desconecta, entonces al eliminar esa arista de T, la gráfica resultante T' es conexa. En este caso, para la gráfica T', (5) es verdadera, pero (6) y (7) son falsas, lo cual es una contradicción, pues (5), (6) y (7) son todas verdaderas (y la gráfica es un árbol) o todas falsas (y la gráfica no es un árbol).

Ahora, intentemos demostrar (2). Suponemos que T es conexa y que al eliminar cualquier arista de T (pero no los vértices), T se desconecta. Debemos mostrar que T es un árbol. Intentemos mostrar que T es conexa y acíclica. Entonces podemos apelar a (5) para concluir que T es un árbol.

Como T es conexa, lo único que debemos hacer es mostrar que T es acíclica. De nuevo, nuestro método será por contradicción. Supongamos que T tiene un ciclo. Recordando nuestra hipótesis (la eliminación de cualquier arista de T desconecta a T), el lector debe tratar de ver cuál será la contradicción por el hecho de suponer que T tiene un ciclo antes de continuar.

Si eliminamos una arista del ciclo de T, T seguirá siendo conexa. Esta contradicción muestra que T es acíclica. Por (5), T es un árbol.

Solución formal

Suponga que T tiene n vértices.

Suponga que T es un árbol. Entonces, por el teorema 7.2.3, T es conexa y tiene $n - 1$ aristas. Suponga que podemos eliminar una arista de T para obtener T', de modo que T' sea conexa. Como T no tiene ciclos, T' tampoco tiene ciclos. Por el teorema 7.2.3, T' es un árbol. De nuevo por el teorema 7.2.3, T' tiene $n - 1$ aristas. Esto es una contradicción. Por tanto, T es conexa pero al eliminar cualquier arista de T (pero no los vértices), T se desconecta.

Si T es conexa y al eliminar cualquier arista de T (pero no los vértices), T se desconecta, entonces T no contiene ciclos. Por el teorema 7.2.3, T es un árbol.

Resumen de técnicas para resolver problemas

- Al intentar construir una demostración, escriba con cuidado lo que se está suponiendo y lo que se debe demostrar.
- Al intentar construir una demostración, considere el uso de definiciones y teoremas relacionados con aquello que se debe demostrar.
- Al intentar construir una demostración, revise las demostraciones de teoremas similares y relacionados con lo que se desea demostrar.
- Si ninguna de las condiciones de las definiciones y teoremas potencialmente útiles se puede aplicar, intente la demostración por contradicción. Al suponer la negación de las hipótesis, dispondrá de afirmaciones adicionales que podrían volver aplicables las condiciones de las definiciones y los teoremas.

7.3 ÁRBOLES DE EXPANSIÓN

En esta sección consideraremos el problema de determinar una subgráfica T de una gráfica G de modo que T sea un árbol con todos los vértices de G; es decir, un **árbol de expansión**. Veremos que los métodos para determinar árboles de expansión se pueden aplicar también a otros problemas.

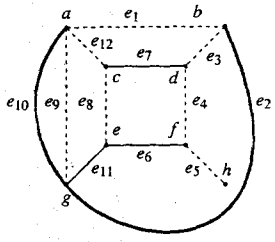


FIGURA 7.3.1
Una gráfica y un árbol de expansión en líneas punteadas.

DEFINICIÓN 7.3.1

Un árbol T es un *árbol de expansión* de una gráfica G si T es una subgráfica de G que contiene a todos los vértices de G.

EJEMPLO 7.3.2

Un árbol de expansión de la gráfica G de la figura 7.3.1 aparece en líneas punteadas. □

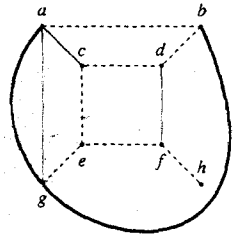


FIGURA 7.3.2
Otro árbol de expansión (en líneas punteadas) de la gráfica de la figura 7.3.1.

EJEMPLO 7.3.3

En general, una gráfica tendrá varios árboles de expansión. Otro árbol de expansión de la gráfica G de la figura 7.3.1 aparece en la figura 7.3.2. □

Suponga que una gráfica G tiene un árbol de expansión T. Sean a y b vértices de G. Como a y b son también vértices de T y T es un árbol, existe un camino P de a a b. Sin embargo, P también sirve como un camino de a a b en G; así, G es conexa. El recíproco también es verdadero.

TEOREMA 7.3.4

Una gráfica G tiene un árbol de expansión si y sólo si G es conexa.

Demostración. Ya hemos mostrado que si G tiene un árbol de expansión, entonces G es conexa. Suponga que G es conexa. Si G es acíclica, por el teorema 7.2.3, G es un árbol.

Suponga que G contiene un ciclo. Eliminamos una arista (pero no los vértices) de este ciclo. La gráfica producida sigue siendo conexa. Si es acíclica, nos detenemos. Si contiene un ciclo, eliminamos una arista de este ciclo. Continuamos de esta manera hasta obtener una subgráfica acíclica, conexa. Por el teorema 7.2.3, T es un árbol. Como T contiene a todos los vértices de G, T es un árbol de expansión de G. ■

Un algoritmo para determinar un árbol de expansión con base en la demostración del teorema 7.3.4 no sería muy eficiente; implicaría el lento proceso de determinación de ciclos. Pero podemos hacer algo mucho mejor. Ilustraremos el primer algoritmo para determinar un árbol de expansión por medio de un ejemplo y luego enunciaremos el algoritmo.

EJEMPLO 7.3.5

Determinar un árbol de expansión para la gráfica G de la figura 7.3.1.

Utilizaremos un método llamado **búsqueda a lo ancho** (algoritmo 7.3.6). La idea de la búsqueda a lo ancho es procesar todos los vértices de un nivel dado antes de pasar al siguiente nivel superior.

Primero, elegimos un orden, digamos *abcdefgh*, de los vértices de G. Elegimos el primer vértice a y lo etiquetamos como la raíz. Sea T la gráfica formada por el único vértice a, sin aristas. Agregamos a T todas las aristas (a, x) y los vértices sobre los cuales son incidentes, desde $x = b$ hasta h, que no produzcan un ciclo al agregarse a T. Así, agregamos a T las aristas (a, b), (a, c) y (a, g). (Podemos usar cualesquiera de las aristas paralelas incidentes en a y g.) Repetimos este procedimiento con los vértices del nivel 1, examinándolos en orden:

b: Incluir (b, d).
c: Incluir (c, e).
g: Ninguno.

Repetimos el procedimiento con los vértices del nivel 2:

d: Incluir (d, f).
e: Ninguno.

Repetimos el procedimiento con los vértices del nivel 3:

f: Incluir (f, h).

Como no se pueden agregar más aristas a h, el único vértice del nivel 4, el procedimiento termina. Hemos determinado el árbol de expansión de la figura 7.3.1. □

Formalizamos el método del ejemplo 7.3.5 como el algoritmo 7.3.6.

ALGORITMO 7.3.6 ***Búsqueda a lo ancho para obtener un árbol de expansión***

Este algoritmo determina un árbol de expansión mediante el método de búsqueda a lo ancho.

Entrada: Una gráfica conexa G con los vértices ordenados

$$v_1, \quad v_2, \quad \ldots, \quad v_n$$

Salida: Un árbol de expansión T

```
procedure bfs (V, E)
 // V = vértices ordenados v1, . . . , vn; E = aristas
 // V' = vértices del árbol de expansión T; E' = aristas del árbol de expansión T
 // v1 es la raíz del árbol de expansión
 // S es una lista ordenada
 S := (v1)
 V' := {v1}
 E' := ∅
  while true do
    begin
    for cada x ∈ S, en orden, do
      for cada y ∈ V − V', en orden, do
        if (x, y) es una arista then
          agregar arista (x, y) a E' y y a V'
    if no se agregó arista alguna then
      return (T)
    S := hijos de S ordenados de manera consistente, con el orden original
         de los vértices
    end
end bfs
```

El ejercicio 16 consiste en dar un argumento para mostrar que el algoritmo 7.3.6 determina de manera correcta un árbol de expansión.

La búsqueda a lo ancho se puede utilizar para verificar si una gráfica arbitraria G con n vértices es conexa (véase el ejercicio 26). Utilizamos el método del algoritmo 7.3.6 para producir un árbol T. Entonces G es conexa si y sólo si T tiene n vértices.

La búsqueda a lo ancho también se puede utilizar para determinar caminos de longitud mínima en una gráfica sin pesos, que vayan de un vértice dado v a todos los demás vértices (véase el ejercicio 20). Utilizamos el método del algoritmo 7.3.6 para obtener un árbol de expansión con raíz en v. Observamos que la longitud de un camino más corto de v a un vértice en le nivel i del árbol de expansión es i. El algoritmo para el camino (o ruta) más corto de Dijkstra para gráficas con pesos (algoritmo 6.4.1) se puede considerar como una generalización de la búsqueda a lo ancho (véase el ejercicio 21).

Una alternativa a la búsqueda a lo ancho es la **búsqueda a profundidad**, la cual pasa a los niveles sucesivos de un árbol en la primera oportunidad posible.

ALGORITMO 7.3.7 ***Búsqueda a profundidad de un árbol de expansión***

Este algoritmo determina un árbol de expansión mediante el método de búsqueda a profundidad.

Entrada: Una gráfica conexa G con los vértices ordenados

$$v_1, \quad v_2, \quad \ldots, \quad v_n$$

Salida: Un árbol de expansión T

```
procedure dfs(V, E)
 // V' = vértices del árbol de expansión T; E' = aristas del árbol de expansión T
 // v1 es la raíz del árbol de expansión
 V' := {v1}
 E' := ∅
 w := v1
 while true do
    begin
    while existe una arista (w, v) tal que al agregarse a T no se crea un ciclo en T do
    begin
    se elige la arista (w, vk) con k mínimo tal que al agregarse a T no crea un ciclo en T
    se agrega (w, vk) a E'
    se agrega vk a V'
    w := vk
    end
   if w = v1 then
    return(T)
   w := padre de w en T // retroceso
   end
end dfs
```

El ejercicio 17 consiste en dar un argumento para mostrar que el algoritmo 7.3.7 determina de manera correcta un árbol de expansión.

EJEMPLO 7.3.8

Utilizar la búsqueda a profundidad (algoritmo 7.3.7) para determinar un árbol de expansión para la gráfica de la figura 7.3.2 con el orden *abcdefgh* de los vértices.

Elegimos el primer vértice a y lo llamamos la raíz (véase la figura 7.3.2). A continuación, agregamos a nuestro árbol la arista (a, x), con x mínimo. En nuestro caso, agregamos la arista (a, b).

Repetimos este proceso. Agregamos las aristas (b, d), (d, c), (c, e), (e, f) y (f, h). En este momento no podemos agregar una arista de la forma (h, x), así que retrocedemos al padre f de h e intentamos agregar una arista de la forma (f, x). De nuevo, no podemos lograr esto, de modo que regresamos al padre e de f. Esta vez podemos agregar la arista (e, g). En este momento no se pueden agregar más aristas, de modo que finalmente retrocedemos hasta la raíz y el procedimiento concluye. □

Debido a la línea en el algoritmo 7.3.7 donde regresamos a lo largo de una arista hacia la raíz elegida al principio, la búsqueda a profundidad también se llama **retroceso**. En el siguiente ejemplo utilizamos el retroceso para resolver un juego.

EJEMPLO 7.3.9 *El problema de las cuatro reinas*

El problema de las cuatro reinas consiste en colocar cuatro fichas en una cuadrícula 4 × 4 de modo que no haya dos fichas en el mismo renglón, la misma columna o en diagonal. Construir un algoritmo de retroceso para resolver el problema de las cuatro reinas. (Para utilizar la terminología del ajedrez, éste es el problema de colocar cuatro reinas en un tablero 4 × 4 de modo que las reinas no se ataquen entre sí.)

La idea del algoritmo consiste en colocar las fichas de manera sucesiva en las columnas. Cuando no sea posible colocar una ficha en una columna, retrocedemos y ajustamos la ficha de la columna anterior. □

ALGORITMO 7.3.10 *Resolución del problema de las cuatro reinas mediante retroceso*

Este algoritmo utiliza el retroceso para buscar un arreglo de cuatro fichas en una cuadrícula 4 × 4 de modo que no haya dos fichas en el mismo renglón, la misma columna, o en la diagonal.

Entrada: Un arreglo *row* de tamaño 4

Salida: **true**, si existe una solución
false, si no existe solución
[Si existe una solución, la *k*-ésima reina está en la columna *k*, y el renglón *row* (*k*).]

```
procedure four_queens (row)
 k := 1     // se comienza en la columna 1
 // row (k) se incrementa antes de utilizarse, de modo que comenzamos en el renglón 1
 row (1) := 0
 while k > 0 do
  begin
  row (k) := row (k) + 1
  // se busca un movimiento válido en la columna k
  while row (k) ≤ 4 and hay conflicto en la columna k, renglón row (k) do
   // se intenta con el siguiente renglón
   row (k) := row (k) + 1
  if row (k) ≤ 4 then     // se ha determinado un movimiento válido en la columna k
   if k = 4 then     // solución concluida
    return(true)
   else     // siguiente columna
    begin
    k := k + 1
    row (k) := 0
    end
  else     // se retrocede a la columna anterior
   k := k − 1
  end
 return(false)     // no existe solución
end four_queens
```

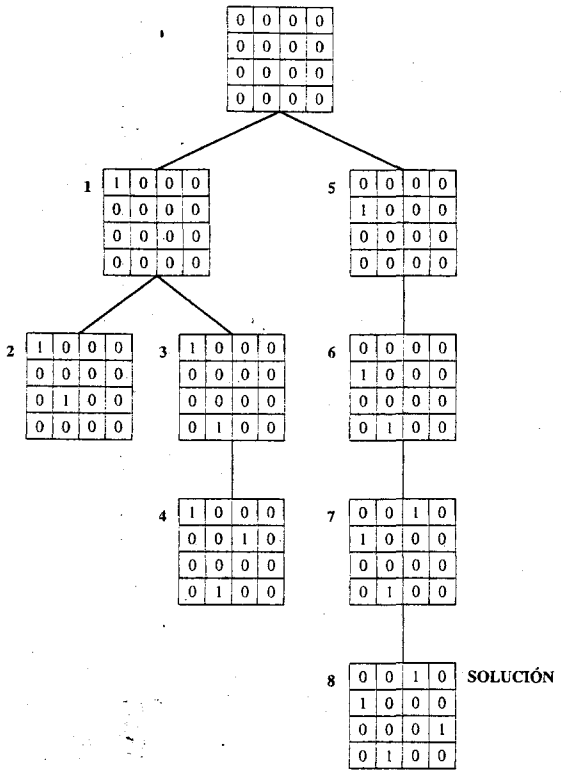


FIGURA 7.3.3 El árbol generado por el algoritmo de retroceso (algoritmo 7.3.10) en búsqueda de una solución al problema de las cuatro reinas.

El árbol generado por el algoritmo 7.3.10 aparece en la figura 7.3.3. Los números indican el orden en que se generan los vértices. La solución aparece en el vértice 8.

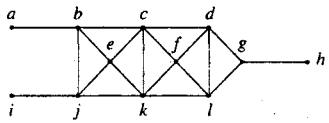
El problema de las *n* reinas consiste en colocar *n* fichas en una cuadrícula *n* × *n* de modo que no haya dos fichas en el mismo renglón, la misma columna, o la misma diagonal. Se puede verificar directamente que no existe una solución para el problema de dos o tres reinas (véase el ejercicio 10). Acabamos de ver que el algoritmo 7.3.10 genera una solución al problema de las cuatro reinas. Se han dado muchas construcciones para generar soluciones al problema de las *n* reinas para toda $n \geq 4$ (véase, por ejemplo, [Erbas]).

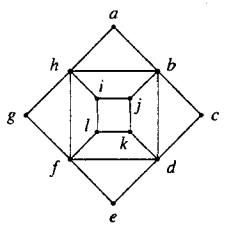
El retroceso o búsqueda a profundidad es de particular interés en un problema como el del ejemplo 7.3.9, donde lo único que se quería era una solución. Como una solución (si existe) aparece en un vértice terminal, al pasar a los vértices terminales lo más pronto posible, en general se evita la generación de muchos vértices innecesarios.

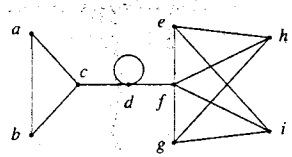
Ejercicios

1. Utilice la búsqueda a lo ancho (algoritmo 7.3.6) con el orden *hgfedcba* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.
2. Utilice la búsqueda a lo ancho (algoritmo 7.3.6) con el orden *hfdbgeca* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.
3. Utilice la búsqueda a lo ancho (algoritmo 7.3.6) con el orden *chbgadfe* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.
4. Utilice la búsqueda a profundidad (algoritmo 7.3.7) con el orden *hgfedcba* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.
5. Utilice la búsqueda a profundidad (algoritmo 7.3.7) con el orden *hfdbgeca* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.
6. Utilice la búsqueda a profundidad (algoritmo 7.3.7) con el orden *dhcbefag* de los vértices para determinar un árbol de expansión de la gráfica G de la figura 7.3.1.

En los ejercicios 7-9, determine un árbol de expansión para cada gráfica.

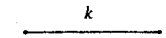
7. 

8. 

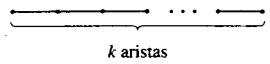
9. 

10. Muestre que no existe solución para los problemas de dos o tres reinas.
11. Determine una solución para los problemas de cinco y seis reinas.
12. ¿Verdadero o falso? Si G es una gráfica conexa y T es un árbol de expansión para G. existe un orden de los vértices de G tal que el algoritmo 7.3.6 produce a T como árbol de expansión. De ser cierto, demuestre este hecho; en caso contrario, proporcione un contraejemplo.
13. ¿Verdadero o falso? Si G es una gráfica conexa y T es un árbol de expansión para G. existe un orden de los vértices de G tal que el algoritmo 7.3.7 produce a T como árbol de expansión. De ser cierto, demuestre este hecho; en caso contrario, proporcione un contraejemplo.
14. Muestre, mediante un ejemplo, que el algoritmo 7.3.6 puede producir árboles de expansión idénticos para una gráfica conexa G partiendo de dos órdenes distintos de los vértices de G.
15. Muestre, mediante un ejemplo, que el algoritmo 7.3.7 puede producir árboles de expansión idénticos para una gráfica conexa G partiendo de dos órdenes distintos de los vértices de G.

16. Demuestre que el algoritmo 7.3.6 es correcto.
17. Demuestre que el algoritmo 7.3.7 es correcto.
18. ¿Bajo qué condiciones está una arista de una gráfica conexa G contenida en todo árbol de expansión de G?
19. Sean T y T' dos árboles de expansión de una gráfica conexa G. Suponga que una arista x está en T pero no en T'. Muestre que existe una arista y en T' pero no en T tal que $(T - \{x\}) \cup \{y\}$ y $(T' - \{y\}) \cup \{x\}$ son árboles de expansión de G.
20. Escriba un algoritmo con base en la búsqueda a lo ancho que determine la longitud mínima de cada camino en una gráfica sin pesos que parta de un vértice fijo v a los demás vértices.
21. Sea G una gráfica con pesos en la que el peso de cada arista es un entero positivo. Sea G' la gráfica obtenida de G reemplazando cada arista



en G de peso k por k aristas sin peso en serie:



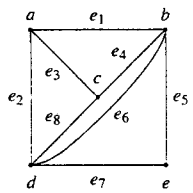
Muestre que el algoritmo de Dijkstra para determinar la longitud mínima de cada camino en la gráfica con pesos G partiendo de un vértice fijo v a los demás vértices (algoritmo 6.4.1) y la realización de una búsqueda a lo ancho en la gráfica sin pesos G' partiendo del vértice v son, en realidad, el mismo proceso.

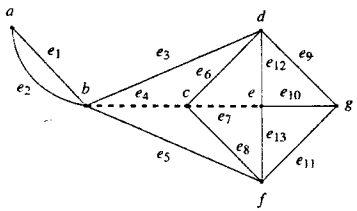
22. Sea T un árbol de expansión para una gráfica G. Muestre que si una arista en G, pero no en T, se agrega a T, se produce un único ciclo.

Un ciclo como el descrito en el ejercicio 22 es un *ciclo fundamental*. La *matriz de ciclos fundamentales* de una gráfica G tiene sus renglones indicados por los ciclos fundamentales de G con respecto de un árbol de expansión T de G y sus columnas indicadas por las aristas de G. La entrada ij es 1 si la arista j está en el i-ésimo ciclo fundamental y 0 en caso contrario. Por ejemplo, la matriz de ciclos fundamentales de la gráfica G de la figura 7.3.1 con respecto del árbol de expansión que aparece en la figura 7.3.1 es

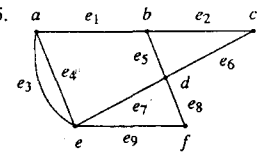
$$\begin{array}{c} \\ (abdca) \\ (efdbace) \\ (ageca) \\ (aga) \\ (abga) \end{array} \begin{array}{c} \begin{array}{cccccccccccc} e_7 & e_6 & e_{11} & e_{10} & e_2 & e_1 & e_3 & e_4 & e_5 & e_8 & e_9 & e_{12} \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \end{array}$$

Determine la matriz de ciclos fundamentales de cada gráfica. El árbol de expansión por utilizar aparece en líneas punteadas.

23. 

24. 

25.



26. Escriba un algoritmo de búsqueda a lo ancho para verificar si una gráfica es conexa.

27. Escriba un algoritmo de búsqueda a profundidad para verificar si una gráfica es conexa.

28. Escriba un algoritmo de búsqueda a profundidad que determine todas las soluciones al problema de cuatro reinas.

7.4 ÁRBOLES DE EXPANSIÓN MÍNIMOS

La gráfica con pesos, G, de la figura 7.4.1 muestra seis ciudades y los costos de construcción de carreteras entre ciertos pares de ellas. Queremos construir el sistema de carreteras de menor costo que una estas seis ciudades. La solución se puede representar mediante una subgráfica. Esta subgráfica debe ser un árbol de expansión, pues debe contener a todos los vértices (de modo que cada ciudad esté en el sistema de carreteras), debe ser conexa (de modo que se pueda llegar a cualquier ciudad partiendo de otra) y debe tener un único camino simple entre cada par de vértices (pues una gráfica con varios caminos simples entre un par de vértices no podría representar un sistema con costo mínimo). Así, lo que se necesita es un árbol de expansión cuya suma de pesos sea mínima. Tal árbol es un **árbol de expansión mínimo**.

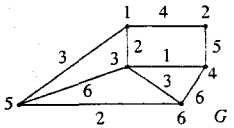


FIGURA 7.4.1
Seis ciudades (a, b, c, d, e, f) y los costos de construcción de carreteras entre ciertos pares de ellas.

DEFINICIÓN 7.4.1

Sea G una gráfica con pesos. Un *árbol de expansión mínimo* de G es un árbol de expansión de G con peso mínimo.

EJEMPLO 7.4.2

El árbol T' de la figura 7.4.2 es un árbol de expansión de la gráfica G de la figura 7.4.1. El peso de T' es 20. Éste no es un árbol de expansión mínimo, pues el árbol de expansión T de la figura 7.4.3 tiene peso 12. Veremos más adelante que T es un árbol de expansión mínimo de G. □

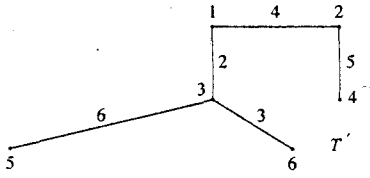


FIGURA 7.4.2 Un árbol de expansión de peso 20 para la gráfica de la figura 7.4.1.

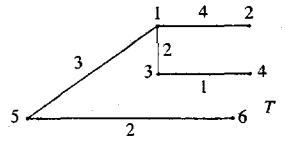


FIGURA 7.4.3 Un árbol de expansión de peso 12 para la gráfica de la figura 7.4.1.

Analizaremos un algoritmo para determinar un árbol de expansión mínimo conocido como **algoritmo de Prim** (algoritmo 7.4.3). Este algoritmo construye un árbol agregando aristas de manera iterativa hasta obtener un árbol de expansión mínimo. En cada iteración agregamos una arista de peso mínimo que no forme un ciclo en el árbol en cuestión. Otro algoritmo para determinar un árbol de expansión mínimo, el **algoritmo de Kruskal**, se presenta en los ejercicios (véanse los ejercicios 20-22).

ALGORITMO 7.4.3 *Algoritmo de Prim*

Este algoritmo determina un árbol de expansión mínimo en una gráfica conexa con pesos.

Entrada: Una gráfica conexa con pesos, con vértices $1, \ldots, n$ y vértice inicial s. Si (i, j) es una arista, $w(i, j)$ es igual al peso de (i, j); si (i, j) no es una arista, entonces $w(i, j)$ es igual a ∞ (un valor mayor que cualquier peso real).

Salida: El conjunto de aristas E en un árbol de expansión mínimo.

```
    procedure prim (w, n, s)
     // v (i) = 1 si el vértice i se ha agregado al árbol de expansión mínimo
     // v (i) = 0 si el vértice i no se ha agregado al árbol de expansión mínimo
1.   for i := 1 to n do
2.     v (i) := 0
     // se agrega el vértice inicial al árbol de expansión mínimo
3.   v (s) := 1
     // se comienza con un conjunto de aristas vacío
4.   E := ∅
     // se colocan n − 1 aristas en el árbol de expansión mínimo
5.   for i := 1 to n − 1 do
6.    begin
      // se agrega la arista de peso mínimo que tenga un vértice en el árbol de expansión
        mínimo
      // y otro vértice que no esté en el árbol de expansión mínimo
7.    min := ∞
8.    for j := 1 to n do
9.     if v (j) = 1 then     // j es un vértice en el árbol de expansión mínimo
10.      for k = 1 to n do
11.       if v (k) = 0 and w (j, k) , < min then
12.         begin
13.         add_vertex := k
14.         e := (j, k)
15.         min := w (j, k)
16.         end
      // se agregan el vértice y la arista al árbol de expansión mínimo
17.   v (add_vertex) := 1
18.   E := E ∪ {e}
19.   end
20.  return(E)
21. end prim
```

EJEMPLO 7.4.4

Mostrar la forma en que el algoritmo de Prim determina un árbol de expansión mínimo para la gráfica de la figura 7.4.1. Suponer que el vértice inicial s es 1.

En la línea 3, agregamos el vértice 1 al árbol de expansión mínimo. La primera vez que ejecutamos el ciclo for en las líneas 8–16, las aristas con un vértice en el árbol y un vértice que no está en el árbol son

Arista	*Peso*
(1, 2)	4
(1, 3)	2
(1, 5)	3

Se elige la arista con menor peso, (1, 3). En las líneas 17 y 18, el vértice 3 se agrega al árbol de expansión mínimo y la arista (1, 3) se agrega a E.

La siguiente vez que ejecutamos el ciclo for en las líneas 8–16, las aristas con un vértice en el árbol y un vértice que no está en el árbol son

Arista	*Peso*
(1, 2)	4
(1, 5)	3
(3, 4)	1
(3, 5)	6
(3, 6)	3

Se elige la arista con menor peso, (3, 4). En las líneas 17 y 18, el vértice 4 se agrega al árbol de expansión mínimo y la arista (3, 4) se agrega a E.

La siguiente vez que ejecutamos el ciclo for en las líneas 8–16, las aristas con un vértice en el árbol y un vértice que no está en el árbol son

Arista	*Peso*
(1, 2)	4
(1, 5)	3
(2, 4)	5
(3, 5)	6
(3, 6)	3
(4, 6)	6

Esta vez, dos aristas tienen el peso mínimo 3. Sin importar la arista elegida, se obtendrá un árbol de expansión mínimo. En esta versión, se elige la arista (1, 5). En las líneas 17 y 18, el vértice 5 se agrega al árbol de expansión mínimo y la arista (1, 5) se agrega a E.

La siguiente vez que ejecutamos el ciclo for en las líneas 8–16, las aristas con un vértice en el árbol y un vértice que no está en el árbol son

Arista	*Peso*
(1, 2)	4
(2, 4)	5
(3, 6)	3
(4, 6)	6
(5, 6)	2

Se elige la arista con menor peso, (5, 6). En las líneas 17 y 18, el vértice 6 se agrega al árbol de expansión mínimo y la arista (5, 6) se agrega a E.

La última vez que ejecutamos el ciclo for en las líneas 8–16, las aristas con un vértice en el árbol y un vértice que no está en el árbol son

Arista	*Peso*
(1, 2)	4
(2, 4)	5

Se elige la arista con menor peso, (1, 2). En las líneas 17 y 18, el vértice 2 se agrega al árbol de expansión mínimo y la arista (1, 2) se agrega a E. El árbol de expansión mínimo construido de esta manera aparece en la figura 7.4.3. □

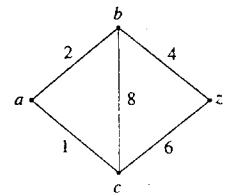


FIGURA 7.4.4
Esta gráfica muestra que la elección de una arista de peso mínimo incidente en el vértice agregado de manera más reciente *no* necesariamente produce un camino más corto. Al partir de a, obtenemos (a, c, z), pero el camino más corto de a a z es (a, b, z).

El algoritmo de Prim es un ejemplo de **algoritmo codicioso**; es decir, aquel que optima la elección de cada iteración, sin considerar las elecciones anteriores. El principio se puede resumir como "hacer lo mejor localmente". En el algoritmo de Prim, como queremos determinar un árbol de expansión mínimo, en cada iteración sólo agregamos una arista disponible con peso mínimo.

La optimación en cada iteración no necesariamente proporciona una solución óptima del problema original. Más adelante (teorema 7.4.5) mostraremos que el algoritmo de Prim es correcto: en realidad obtenemos un árbol de expansión mínimo. Como ejemplo de un algoritmo codicioso que no conduce a una solución óptima, consideremos un "algoritmo del camino más corto", en el cual en cada paso elegimos una arista disponible con peso mínimo, incidente en el vértice agregado de manera más reciente. Si aplicamos este algoritmo a la gráfica con pesos de la figura 7.4.4 para determinar un camino más corto de a a z, elegiríamos la arista (a, c) y luego la arista (c, z). Por desgracia, éste no es el camino más corto de a a z.

Ahora mostraremos que el algoritmo de Prim es correcto.

TEOREMA 7.4.5

El algoritmo de Prim (algoritmo 7.4.3*) es correcto; es decir, al concluir el algoritmo* 7.4.3, *T es un árbol de expansión mínimo.*

Demostración. Sea T_i la gráfica construida por el algoritmo 7.4.3 después de la i-ésima iteración del ciclo for, líneas 5-19. Más precisamente, el conjunto de aristas de T_i es el conjunto E construido después de la i-ésima iteración del ciclo for, líneas 5–19, y el conjunto de vértices de T_i es el conjunto de vértices donde inciden las aristas de E. Sea T_0 la gráfica construida por el algoritmo 7.4.3 justo antes de entrar al ciclo for en la línea 5 por primera vez; T_0 consta del único vértice s, sin aristas. En esta demostración, eliminaremos el conjunto de vértices y haremos referencia a una gráfica especificando su conjunto de aristas.

Por construcción, al concluir el algoritmo 7.4.3, la gráfica construida T_{n-1} es una subgráfica conexa, acíclica, de la gráfica dada G y que contiene a todos los vértices de G; por tanto, T_{n-1} es un árbol de expansión de G.

Utilizamos inducción para mostrar que para toda $i = 0, \ldots, n-1$, T_i está contenida en un árbol de expansión mínimo. Esto implica que, al concluir, T_{n-1} es un árbol de expansión mínimo.

Si $i = 0$, T_0 consta de un único vértice. En este caso, T_0 está contenida en todo árbol de expansión mínimo. Hemos verificado el paso base.

A continuación, supongamos que T_i está contenida en un árbol de expansión mínimo T'. Sea V el conjunto de vértices en T_i. El algoritmo 7.4.3 elige una arista (j, k) de peso mínimo, tal que $j \in V$ y $k \notin V$ y la agrega a T_i para producir T_{i+1}. Si (j, k) está en T', entonces T_{i+1} está contenida en el árbol de expansión mínimo T'. Si (j, k) no está en T', $T' \cup \{(j, k)\}$ contiene un ciclo C. Se elige una arista (x, y) en C, distinta de (j, k), con $x \in V$ y $y \notin V$. Entonces

$$w(x, y) \geq w(j, k). \tag{7.4.1}$$

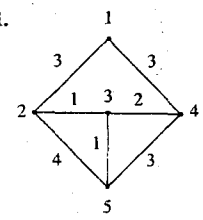
Debido a (7.4.1), la gráfica $T'' = [T' \cup \{(j, k)\}] - \{(x, y)\}$ tiene peso menor o igual al peso de T'. Como T'' es un árbol de expansión, T'' es un árbol de expansión mínimo. Como T_{i+1} está contenido en T'', hemos verificado el paso inductivo y hemos concluido la demostración. ■

Nuestra versión del algoritmo de Prim examina $\Theta(n^3)$ aristas en el peor de los casos (véase el ejercicio 6) para determinar un árbol de expansión mínimo para una gráfica con n vértices. Es posible (véase el ejercicio 8) implantar el algoritmo de Prim de modo que sólo examine $\Theta(n^2)$ aristas en el peor de los casos. Como K_n tiene $\Theta(n^2)$ aristas, esta última versión es óptima.

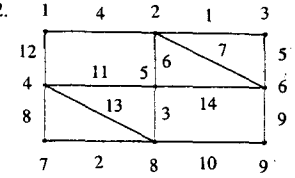
Ejercicios

En los ejercicios 1-5, determine el árbol de expansión mínimo dado por el algoritmo 7.4.3 para cada gráfica.

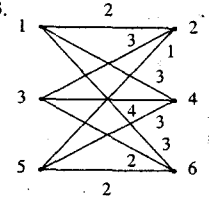
1.



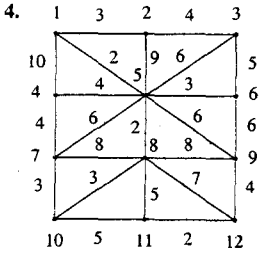
2.



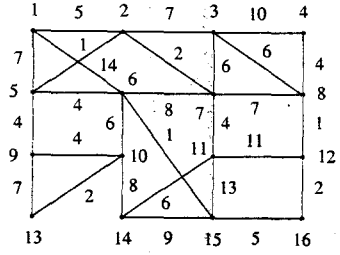
3.



4.



5.



6. Muestre que el algoritmo 7.4.3 examina $\Theta(n^3)$ aristas en el peor de los casos.

Los ejercicios 7-9 se refieren a una versión alternativa del algoritmo de Prim (algoritmo 7.4.6).

ALGORITMO 7.4.6 ***Versión alternativa del algoritmo de Prim***

Este algoritmo determina un árbol de expansión mínimo en una gráfica conexa, con pesos, G. En cada paso, algunos vértices tienen etiquetas temporales y otros tienen etiquetas permanentes. La etiqueta del vértice i se denota L_i.

Entrada: Una gráfica conexa con pesos, con vértices $1, \ldots, n$ y vértice inicial s. Si (i, j) es una arista, $w(i, j)$ es igual al peso de (i, j); si (i, j) no es una arista, entonces $w(i, j)$ es igual a ∞ (un valor mayor que cualquier peso real).

Salida: Un árbol de expansión mínimo T.

```
procedure prim_alternate (w, n, s)
 sea T la gráfica con vértice s, sin aristas
 for j := 1 to n do
  begin
  L_j := w(s, j) // estas etiquetas son temporales
  back (j) := s
  end
 L_s := 0
 hacer L_s permanente
 while existen etiquetas temporales do
  begin
  elegir la menor etiqueta temporal L_i
  hacer L_i permanente
  agregar arista (i, back (i)) a T
  agregar vértice i a T
  for cada etiqueta temporal L_k do
   if w (i, k) < L_k then
    begin
    L_k := w (i, k)
    back(k) := i
     end
   end
  return(T)
 end prim_alternate
```

7. Muestre la forma en que el algoritmo 7.4.6 determina un árbol de expansión mínimo para las gráficas de los ejercicios 1-5.
8. Muestre que el algoritmo 7.4.6 examina $O(n^2)$ aristas en el peor de los casos.
9. Demuestre que el algoritmo 7.4.6 es correcto; es decir, que al concluir el algoritmo 7.4.6, T es un árbol de expansión mínimo.
10. Sea G una gráfica conexa con pesos, sea v un vértice en G y e una arista de peso mínimo incidente en v. Muestre que e está contenida en algún árbol de expansión mínimo.
11. Sea G una gráfica conexa con pesos y v un vértice en G. Suponga que los pesos de las aristas incidentes en v son distintos. Sea e la arista de peso mínimo incidente en v. ¿Debe e estar contenida en algún árbol de expansión mínimo?
12. Muestre que cualquier algoritmo que determine un árbol de expansión mínimo en K_n, donde todos los pesos son iguales, debe examinar cada arista en K_n.
13. Muestre que si todos los pesos de una gráfica conexa G son distintos, entonces G tiene un único árbol de expansión mínimo.

En los ejercicios 14-16, decida si la afirmación es verdadera o falsa. Si la afirmación es verdadera, demuéstrela; en caso contrario, proporcione un contraejemplo. En cada ejercicio, G es una gráfica conexa con pesos.

14. Si todos los pesos en G son distintos, los diversos árboles de expansión de G tendrán pesos distintos.
15. Si e es una arista en G cuyo peso es menor que el peso de cualquier otra arista, e está en cada árbol de expansión mínimo de G.

16. Si T es un árbol de expansión mínimo de G, existe un etiquetado de los vértices de G de modo que el algoritmo 7.4.3 produce a T.
17. Sea G una gráfica conexa con pesos. Muestre que si, mientras sea posible, eliminamos una arista de G con peso máximo y cuya eliminación no desconecte a G, el resultado es un árbol de expansión mínimo de G.

☆ 18. Escriba un algoritmo que determine un árbol de expansión máximo en una gráfica conexa con pesos.

19. Demuestre que su algoritmo del ejercicio 18 es correcto.

El *algoritmo de Kruskal* determina un árbol de expansión mínimo en una gráfica conexa con pesos G con n vértices de la manera siguiente. La gráfica T consta inicialmente de los vértices de G y ninguna arista. En cada iteración, agregamos una arista e a T con peso mínimo y que no complete un ciclo en T. Cuando T tenga $n - 1$ aristas, nos detenemos.

20. Enuncie de manera formal el algoritmo de Kruskal.
21. Muestre la forma en que el algoritmo de Kruskal determina árboles de expansión mínimos para las gráficas de los ejercicios 1-5.
22. Muestre que el algoritmo de Kruskal es correcto; es decir, que al concluir el algoritmo de Kruskal, T es un árbol de expansión mínimo.
23. Sea V un conjunto de n vértices y s una "función de disimilaridad" en $V \times V$ (véase el ejemplo 6.1.6). Sea G la gráfica completa con pesos con conjunto de vértices V y pesos $w(v_i, v_j) = s(v_i, v_j)$. Modifique el algoritmo de Kruskal de modo que agrupe los datos en clases. Esta modificación se conoce como el **método de los vecinos más cercanos** (véase [Gose]).

Los ejercicios 24-30 se refieren a la siguiente situación. Suponga que tenemos estampillas de diversas denominaciones y que queremos elegir la menor cantidad de estampillas para pagar una tarifa postal. Consideremos un algoritmo codicioso que selecciona las estampillas eligiendo primero la mayor cantidad posible de estampillas de la mayor denominación, luego la mayor cantidad posible de estampillas de la segunda mayor denominación, etcétera.

24. Muestre que si las denominaciones disponibles son 1, 8 y 10 centavos, el algoritmo no siempre produce la menor cantidad de estampillas para pagar cierta tarifa postal.

☆ 25. Muestre que si las denominaciones disponibles son 1, 5 y 25 centavos, el algoritmo produce la menor cantidad de estampillas para pagar cierta tarifa postal.

26. Determine enteros positivos a_1 y a_2, tales que $a_1 > 2a_2 > 1$, a_2 no divide a a_1, y que el algoritmo con denominaciones $1, a_1, a_2$, no siempre produce la menor cantidad de estampillas para pagar cierta tarifa postal.

★ 27. Determine enteros positivos a_1 y a_2, tales que $a_1 > 2a_2 > 1$, a_2 no divide a a_1, y que el algoritmo con denominaciones $1, a_1, a_2$, produce la menor cantidad de estampillas para pagar cierta tarifa postal. Demuestre que sus valores proporcionan una solución óptima.

☆ 28. Suponga que las denominaciones posibles son

$$1 = a_1 < a_2 < \cdots < a_n.$$

Muestre, mediante contraejemplos, que la condición

$$a_i \geq 2a_{i-1} - a_{i-2}, \quad 3 \leq i \leq n$$

no es necesaria ni suficiente para que el algoritmo codicioso sea óptimo.

☆ 29. Muestre que el algoritmo codicioso es óptimo para las denominaciones

$$1 = a_1 < a_2 < a_3$$

si y sólo si el algoritmo codicioso es óptimo para $n = 1, 2, \ldots, a_2a_3 - 1$.

★ **30.** Muestre que el algoritmo codicioso es óptimo para las denominaciones

$$1 = a_1 < a_2 < \cdots < a_m$$

si y sólo si el algoritmo codicioso es óptimo para $n = 1, 2, \ldots, k$, donde

$$k = \sum_{i=1}^{m-1} \left(\frac{a_{i+1}}{\text{mcd}(a_{i+1}, a_i)} - 1 \right) a_i.$$

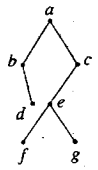


FIGURA 7.5.1
Un árbol binario.

7.5 ÁRBOLES BINARIOS

Los **árboles binarios** son de los tipos particulares más importantes de árboles con raíz. Cada vértice de un árbol binario tiene a lo más dos hijos (véase la figura 7.5.1). Además, cada hijo se designa como **hijo izquierdo** o **hijo derecho**. Al trazar un árbol binario, un hijo izquierdo se dibuja a la izquierda y un hijo derecho se dibuja a la derecha. A continuación damos la definición formal.

DEFINICIÓN 7.5.1

Un *árbol binario* es un árbol con raíz en el cual cada vértice tiene cero, uno o dos hijos. Si un vértice tiene un hijo, ese hijo se designa como un hijo izquierdo o un hijo derecho (pero no ambos). Si un vértice tiene dos hijos, uno de ellos de designa como un hijo izquierdo y el otro se designa como un hijo derecho.

EJEMPLO 7.5.2

En el árbol binario de la figura 7.5.1, el vértice b es el hijo izquierdo del vértice a y el vértice c es el hijo derecho del vértice a. El vértice d es el hijo derecho del vértice b; el vértice b no tiene un hijo izquierdo. El vértice e es el hijo izquierdo del vértice c; el vértice c no tiene un hijo derecho. □

EJEMPLO 7.5.3

Un árbol que define un código de Huffman es un árbol binario. Por ejemplo, en el árbol del código de Huffman de la figura 7.1.8, el paso de un vértice a un hijo izquierdo corresponde a utilizar el bit 1 y el paso de un vértice a un hijo derecho corresponde al uso del bit 0. □

Un **árbol binario completo** es un árbol binario en el cual cada vértice tiene dos o cero hijos. Un resultado fundamental acerca de los árboles binarios completos es el siguiente teorema.

TEOREMA 7.5.4

Si T es un árbol binario completo con i vértices internos, entonces T tiene $i + 1$ vértices terminales y $2i + 1$ vértices en total.

Demostración. Los vértices de T constan de los vértices que son hijos (de algún padre) y los vértices que no son hijos (de ningún padre). Existe un vértice que no es hijo de nadie: la raíz. Como existen i vértices internos, cada uno de los cuales tiene dos hijos, existen $2i$ hijos. Así, la cantidad total de vértices de T es $2i + 1$ y el número de vértices terminales es

$$(2i + 1) - i = i + 1.$$ ■

EJEMPLO 7.5.5

Un torneo de eliminación simple es aquél en que cada competidor se elimina después de perder una vez. La gráfica de un torneo de eliminación simple es un árbol binario completo (véase la figura 7.5.2). Los nombres de los competidores aparecen a la izquierda. Los ganadores siguen avanzando hacia la derecha. En cierto momento, existe un único ganador en la raíz. Si el número de competidores no es una potencia de 2, algunos competidores pueden pasar a la siguiente ronda sin tener que jugar; en la figura 7.5.2, el competidor 7 está en este caso.

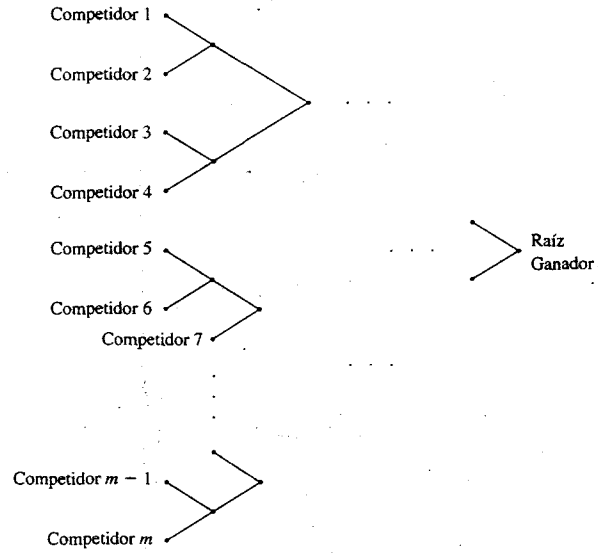


FIGURA 7.5.2 La gráfica (árbol binario completo) de un torneo de eliminación simple.

Mostraremos que si existen n competidores en un torneo de eliminación simple, se juegan un total de $n - 1$ encuentros.

El número de competidores es igual al número de vértices terminales y el número de encuentros i es igual al número de vértices internos. Así, por el teorema 7.5.4,

$$n + i = 2i + 1,$$

de modo que $i = n - 1$. □

Nuestro siguiente resultado relativo a los árboles binarios relaciona el número de vértices terminales con la altura.

TEOREMA 7.5.6

Si un árbol binario de altura h tiene t vértices terminales, entonces

$$\lg t \leq h. \tag{7.5.1}$$

Demostración. Demostraremos la desigualdad equivalente

$$t \leq 2^h \tag{7.5.2}$$

por inducción sobre h. La desigualdad (7.5.1) se obtiene de (7.5.2) tomando el logaritmo en base 2 de ambos lados de (7.5.2).

Si $h = 0$, el árbol binario consta de un único vértice. En este caso, $t = 1$ y entonces (7.5.2) es verdadero.

Suponga que el resultado es válido para un árbol binario cuya altura sea menor que h. Sea T un árbol binario de altura $h > 0$ con t vértices terminales. Suponga primero que la raíz de T tiene sólo un hijo. Si eliminamos la raíz y la arista incidente en la raíz, el árbol resultante tiene altura $h - 1$ y el mismo número de vértices terminales que T. Por inducción, $t \leq 2^{h-1}$. Como $2^{h-1} < 2^h$, (7.5.2) queda establecida para este caso.

Ahora supongamos que la raíz de T tiene hijos v_1 y v_2. Sea T_i el subárbol con raíz en v_i y suponga que T_i tiene altura h_i y t_i vértices terminales, $i = 1, 2$. Por inducción,

$$t_i \leq 2^{h_i}, \quad i = 1, 2. \tag{7.5.3}$$

Los vértices terminales de T constan de los vértices terminales de T_1 y T_2. Por tanto,

$$t = t_1 + t_2. \tag{7.5.4}$$

Al combinar (7.5.3) y (7.5.4), obtenemos

$$t = t_1 + t_2 \leq 2^{h_1} + 2^{h_2} \leq 2^{h-1} + 2^{h-1} = 2^h.$$

Hemos verificado el paso inductivo, con lo cual concluye la demostración. ■

EJEMPLO 7.5.7

El árbol binario de la figura 7.5.3 tiene altura $h = 3$ y el número de vértices terminales es $t = 8$. Para este árbol, la desigualdad (7.5.1) se convierte en una igualdad. □

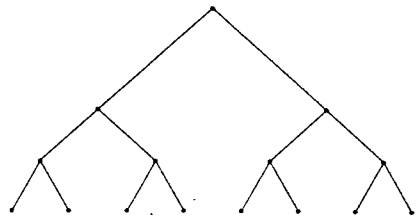


FIGURA 7.5.3 Un árbol binario de altura $h = 3$ con $t = 8$ vértices terminales. Para este árbol binario, $\lg t = h$.

Suponga que tenemos un conjunto S cuyos elementos se pueden ordenar. Por ejemplo, si S consta de números, podemos utilizar el orden común definido sobre los números, y si S consta de cadenas de caracteres alfabéticos, podemos utilizar el orden lexicográfico. Los árboles binarios se utilizan ampliamente en las ciencias de la computación para guardar los elementos de un conjunto ordenado, como un conjunto de número o un conjunto de cadenas. Si el elemento dato $d(v)$ se guarda en el vértice v y el elemento de dato $d(w)$ se guarda en el vértice w, entonces, si v es un hijo izquierdo (o un hijo derecho) de w, se puede garantizar que existe cierta relación de orden entre $d(v)$ y $d(w)$. Un ejemplo es un **árbol de búsqueda binaria**.

DEFINICIÓN 7.5.8

Un *árbol de búsqueda binaria* es un árbol binario T en el cual se asocian ciertos datos con los vértices. Los datos están ordenados de modo que, para *cada* vértice v en T, cada elemento de dato en el subárbol izquierdo de v sea menor que el elemento de dato en v y cada elemento de dato en el subárbol derecho de v es mayor que el elemento de dato en v.

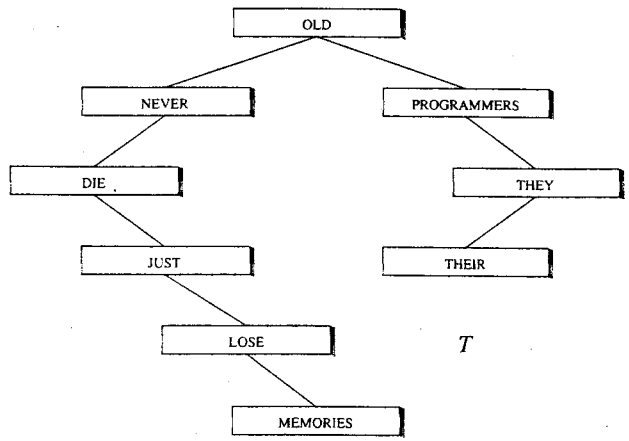


FIGURA 7.5.4 Un árbol de búsqueda binaria.

EJEMPLO 7.5.9

Las palabras

OLD PROGRAMMERS NEVER DIE

THEY JUST LOSE THEIR MEMORIES (7.5.5)

(Los viejos programadores nunca mueren; sólo pierden su memoria) se pueden colocar en un árbol de búsqueda binaria, como muestra la figura 7.5.4. Observe que para cualquier vértice v, cada elemento de dato en el subárbol izquierdo de v es menor (es decir, precede alfabéticamente) que el elemento de dato en v y cada elemento de dato en el subárbol derecho de v es mayor que el elemento de dato en v.

En general, habrá muchas formas de colocar datos en un árbol de búsqueda binaria. La figura 7.5.5 muestra otro árbol de búsqueda binaria que guarda las palabras (7.5.5).

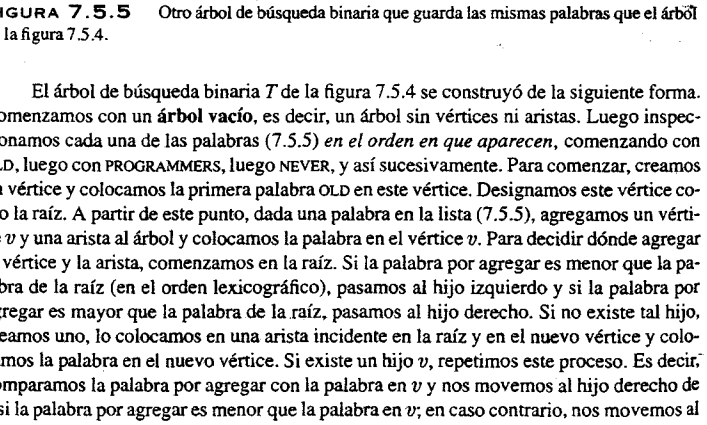


FIGURA 7.5.5 Otro árbol de búsqueda binaria que guarda las mismas palabras que el árbol de la figura 7.5.4.

El árbol de búsqueda binaria T de la figura 7.5.4 se construyó de la siguiente forma. Comenzamos con un **árbol vacío**, es decir, un árbol sin vértices ni aristas. Luego inspeccionamos cada una de las palabras (7.5.5) *en el orden en que aparecen*, comenzando con OLD, luego con PROGRAMMERS, luego NEVER, y así sucesivamente. Para comenzar, creamos un vértice y colocamos la primera palabra OLD en este vértice. Designamos este vértice como la raíz. A partir de este punto, dada una palabra en la lista (7.5.5), agregamos un vértice v y una arista al árbol y colocamos la palabra en el vértice v. Para decidir dónde agregar el vértice y la arista, comenzamos en la raíz. Si la palabra por agregar es menor que la palabra de la raíz (en el orden lexicográfico), pasamos al hijo izquierdo y si la palabra por agregar es mayor que la palabra de la raíz, pasamos al hijo derecho. Si no existe tal hijo, creamos uno, lo colocamos en una arista incidente en la raíz y en el nuevo vértice y colocamos la palabra en el nuevo vértice. Si existe un hijo v, repetimos este proceso. Es decir, comparamos la palabra por agregar con la palabra en v y nos movemos al hijo derecho de v si la palabra por agregar es menor que la palabra en v; en caso contrario, nos movemos al hijo derecho de v. Si no existe un hijo al cual movernos, creamos uno, lo colocamos en una arista incidente en v y el nuevo vértice y colocamos la palabra en el nuevo vértice. Si existe un hijo al cual movernos, repetimos este proceso. En algún momento colocamos la palabra en el árbol. Luego vamos por la siguiente palabra en la lista, la comparamos con la raíz, nos movemos hacia la izquierda o la derecha, la comparamos con el nuevo vértice, nos movemos hacia la izquierda o hacia la derecha, y así sucesivamente, de modo que en algún momento lo guardamos en el árbol. De esta forma, guardamos todas las palabras en el árbol y así creamos un árbol de búsqueda binaria. Establecemos este método de construcción de un árbol de búsqueda binaria de manera formal como el algoritmo 7.5.10.

ALGORITMO 7.5.10 ***Construcción de un árbol de búsqueda binaria***

Este algoritmo construye un árbol de búsqueda binaria. La entrada se lee en el orden con el cual fue enviada. Después de leer cada palabra, ésta se inserta en el árbol.

Entrada: Una sucesión $w_1, \ldots, w_n$ de palabras distintas y la longitud n de la sucesión

Salida: Un árbol de búsqueda binaria T

```
procedure make_bin_search_tree(w, n)
  sea T el árbol con un vértice, root
  guardar w_1 en root
  for i := 2 to n do
    begin
    v := root
    search := true     // encontrar un lugar para w_i
    while search do
      begin
      s := palabra en v
      if w_i < s then
        if v no tiene hijo izquierdo then
          begin
          agregar un hijo izquierdo l a v
          guardar w_i en l
          search := false     // fin de la búsqueda
          end
        else
          v := hijo izquierdo de v
      else  // w_i > s
        if v no tiene hijo derecho then
          begin
          agregar un hijo derecho r a v
          guardar w_i en r
          search := false     // fin de la búsqueda
          end
        else
          v := hijo derecho de v
      end     // mientras
    end     // para
  return(T)
end make_bin_search_tree
```

Los árboles de búsqueda binaria son útiles para localizar datos. Es decir, dado un elemento D, podemos determinar con facilidad si D está en un árbol de búsqueda binaria y, de estar presente, conocer su posición. Para determinar si un elemento de dato D está en un árbol de búsqueda binaria, comenzaríamos en la raíz. Luego compararíamos de manera su-

cesiva D con el elemento de dato en el vértice en cuestión. Si D es igual al elemento de dato en el vértice en cuestión, hemos encontrado a D, por lo cual habremos concluido. Si D es menor que el elemento de dato en el vértice en cuestión v, nos movemos al hijo izquierdo de v y repetimos el proceso. Si D es mayor que el elemento de dato en el vértice en cuestión v, nos movemos al hijo derecho de v y repetimos el proceso. Si en algún momento no existe un hijo al cual moverse, podemos concluir que D no está en el árbol. (El ejercicio 2 pide establecer de manera formal este proceso.)

El tiempo utilizado en la búsqueda de un elemento en un árbol de búsqueda binaria es máximo cuando el elemento no está presente y seguimos el camino de mayor longitud desde la raíz. Así el tiempo máximo para buscar un elemento en un árbol de búsqueda binaria es aproximadamente proporcional a la altura del árbol. Por tanto, si la altura de un árbol de búsqueda binaria es pequeña, la búsqueda en el árbol será siempre muy rápida (véase el ejercicio 21). Se conocen muchas formas de minimizar la altura de un árbol de búsqueda binaria (véase, por ejemplo, [Cormen]).

Estableceremos un enunciado más preciso acerca de la búsqueda en un árbol de búsqueda binaria en el peor de los casos. Sea T un árbol de búsqueda binaria con n vértices y sea T^* el árbol binario completo obtenido de T al agregar hijos izquierdos y derechos a los vértices existentes de T, cuando esto sea posible. En la figura 7.5.6, mostramos el árbol binario completo resultante de modificar el árbol de búsqueda binaria de la figura 7.5.4. Los vértices agregados se dibujan como cajas. Una búsqueda sin éxito en T corresponde a llegar a un vértice agregado (caja) en T^*. Definiremos el tiempo necesario para realizar el procedimiento de búsqueda en el peor de los casos como la altura h del árbol T^*. Por el teorema 7.5.6, $\lg t \le h$, donde t es el número de vértices terminales en T^*. El árbol binario completo T^* tiene n vértices internos, de modo que por el teorema 7.5.4, $t = n + 1$. Así, en el peor de los casos, el tiempo será igual al menos a $\lg t = \lg(n + 1)$. El ejercicio 3 muestra que si se minimiza la altura de T, el peor de los casos requiere un tiempo igual a $\lceil \lg(n + 1) \rceil$. Por ejemplo, como

$$\lceil \lg(2{,}000{,}000 + 1) \rceil = 21,$$

es posible guardar 2 millones de datos en un árbol de búsqueda binaria y encontrar uno de estos elementos, o determinar que no está presente en el árbol, en a lo más 21 pasos.

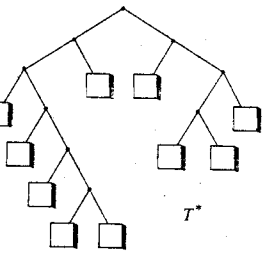


FIGURA 7.5.6
Ampliación de un árbol de búsqueda binaria a un árbol binario completo.

Ejercicios

1. Coloque las palabras FOUR SCORE AND SEVEN YEARS AGO OUR FOREFATHERS BROUGHT FORTH. en orden de aparición, en un árbol de búsqueda binaria.
2. Escriba un algoritmo formal para la búsqueda en un árbol de búsqueda binaria.
3. Escriba un algoritmo que guarde n palabras distintas en un árbol de búsqueda binaria T de altura mínima. Muestre que el árbol derivado T^*, según lo descrito en el texto, tiene altura $\lceil \lg(n + 1) \rceil$.
4. ¿Verdadero o falso? Sea T un árbol binario. Si para cada vértice v en T el elemento de dato en v es mayor que el elemento de dato en el hijo izquierdo de v, y el elemento de dato en v es menor que el elemento de dato en el hijo derecho de v, entonces T es un árbol de búsqueda binaria. Explique.

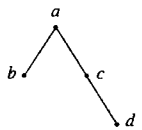
En los ejercicios 5-7, trace una gráfica con las propiedades dadas o explique por qué no existe tal gráfica.

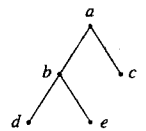
5. Árbol binario completo; cuatro vértices internos; cinco vértices terminales
6. Árbol binario completo; altura = 3; nueve vértices terminales
7. Árbol binario completo; altura = 4; nueve vértices terminales

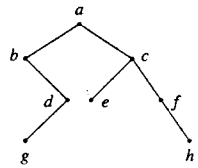
8. Un **árbol m-ario completo** es un árbol con raíz tal que cada padre tiene m hijos ordenados. Si T es un árbol m-ario completo con i vértices internos, ¿cuántos vértices tiene T? ¿Cuántos vértices terminales tiene T? Demuestre sus resultados.
9. Proporcione un algoritmo para construir un árbol binario completo con $n > 1$ vértices terminales.
10. Proporcione un algoritmo recursivo para insertar una palabra en un árbol de búsqueda binaria.
11. Determine la altura máxima de un árbol de búsqueda binaria con t vértices terminales.
12. Escriba un algoritmo que verifique si un árbol binario en cuyos vértices se guardan datos es un árbol de búsqueda binaria.
13. Sea T un árbol binario completo. Sea I la suma de las longitudes de los caminos simples de la raíz a los vértices internos. I es la *longitud interna*. Sea E la suma de las longitudes de los caminos simples de la raíz a los vértices terminales. E es la *longitud externa*. Demuestre que si T tiene n vértices internos, entonces $E = I - 2n$.

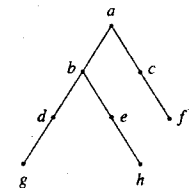
Un árbol binario T está *balanceado* si para cada vértice v en T, las alturas de los subárboles izquierdo y derecho de v difieren a lo más en 1. (En este caso, la altura de un árbol vacío se define como -1.)

Indique si cada árbol de los ejercicios 14-17 es balanceado o no.

14.


15.


16.


17.


En los ejercicios 18-20, N_h se define como el menor número de vértices en un árbol binario balanceado de altura h y $f_1, f_2, \ldots$ denota la sucesión de Fibonacci.

18. Muestre que $N_0 = 1$, $N_1 = 2$ y $N_2 = 4$.
19. Muestre que $N_h = 1 + N_{h-1} + N_{h-2}$, para $h \ge 0$.
20. Muestre que $N_h = f_{h+2} - 1$, para $h \ge 0$.

★ **21.** Muestre que la altura h de un árbol binario balanceado con n vértices satisface $h = O(\lg n)$. Este resultado muestra que el tiempo de búsqueda en un árbol binario balanceado con n vértices es $O(\lg n)$, en el peor de los casos.

☆ 22. Demuestre que si un árbol binario de altura h tiene $n \ge 1$ vértices, entonces $\lg n < h + 1$. Este resultado, junto con el ejercicio 21, muestra que el tiempo de búsqueda en un árbol binario balanceado con n vértices es $\Theta(\lg n)$, en el peor de los casos.

7.6 *RECORRIDOS DE UN ÁRBOL*

La búsqueda a lo ancho y la búsqueda a profundidad proporcionan formas de "recorrer" un árbol, es decir, de recorrerlo de manera sistemática de modo que cada vértice sea visitado exactamente una vez. En esta sección consideraremos otros tres métodos para recorrer un árbol. Definiremos estos recorridos de manera recursiva.

ALGORITMO 7.6.1 *Recorrido en preorden*

Este algoritmo recursivo procesa los vértices de un árbol binario utilizando el recorrido en preorden.

Entrada: *PT*, la raíz de un árbol binario

Salida: Depende de la interpretación de "procesar" en la línea 3

```
    procedure preorder (PT)
1.    if PT es vacío then
2.       return
3.    procesar PT
4.    l := hijo izquierdo de PT
5.    preorder (l)
6.    r := hijo derecho de PT
7.    preorder (r)
    end preorder
```

Examinaremos el algoritmo 7.6.1 para algunos casos sencillos. Si el árbol binario es vacío, nada se procesa pues, en este caso, el algoritmo simplemente regresa en la línea 2.

Suponga que la entrada consta de un árbol con un único vértice. Hacemos *PT* igual a la raíz y llamamos a *preorder*(*PT*). Como *PT* no es vacío, pasamos a la línea 3, donde procesamos la raíz. En la línea 5, llamamos a *preorder* con *PT* igual al hijo izquierdo (vacío) de la raíz. Sin embargo, hemos visto que cuando la entrada es un árbol vacío, *preorder* no procesa nada. De manera análoga, en la línea 7 utilizamos un árbol vacío como entrada de *preorder* y nada se procesa. Así, cuando la entrada consta de un árbol con un único vértice, procesamos la raíz y regresamos.

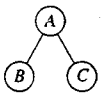


FIGURA 7.6.1
Entrada del algoritmo 7.6.1.

FIGURA 7.6.2
En la línea 5 del algoritmo 7.6.1, donde la entrada es el árbol de la figura 7.6.1.

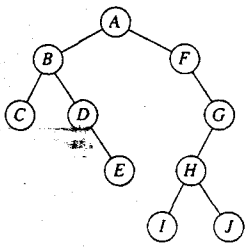


FIGURA 7.6.3
Un árbol binario. El preorden es *A B C D E F G H I J*. El entreorden es *C B D E A F I H J G*. El posorden es *C E D B I J H G F A*.

Ahora, supongamos que la entrada es el árbol de la figura 7.6.1. Hacemos *PT* igual a la raíz y llamamos a *preorder*(*PT*). Como *PT* no es vacío, pasamos a la línea 3, donde procesamos a la raíz. En la línea 5 llamamos a *preorder*, con *PT* igual al hijo izquierdo de la raíz (véase la figura 7.6.2). Acabamos de ver que si la entrada de *preorder* consta de un único vértice, *preorder* procesa ese vértice. Así, a continuación procesamos el vértice *B*. De manera análoga, en la línea 7, procesamos el vértice *C*. Así, los vértices se procesan en el orden *ABC*.

EJEMPLO 7.6.2

¿En qué orden se procesan los vértices del árbol de la figura 7.6.3 si se utiliza el recorrido en preorden?

Si seguimos las líneas 3-7 (raíz/izquierdo/derecho) del algoritmo 7.6.1, el recorrido se realiza como muestra la figura 7.6.4. Así, el orden de procesamiento es

A B C D E F G H I J. □

Los recorridos en entreorden y en posorden se obtienen cambiando la posición de la línea 3 (raíz) en el algoritmo 7.6.1. Los prefijos pre, entre y pos se refieren a la posición de la raíz en el recorrido; es decir, "preorden" significa que va primero la raíz; "entreorden", quiere decir que la raíz va en segundo lugar, y "posorden" que la raíz va al final.

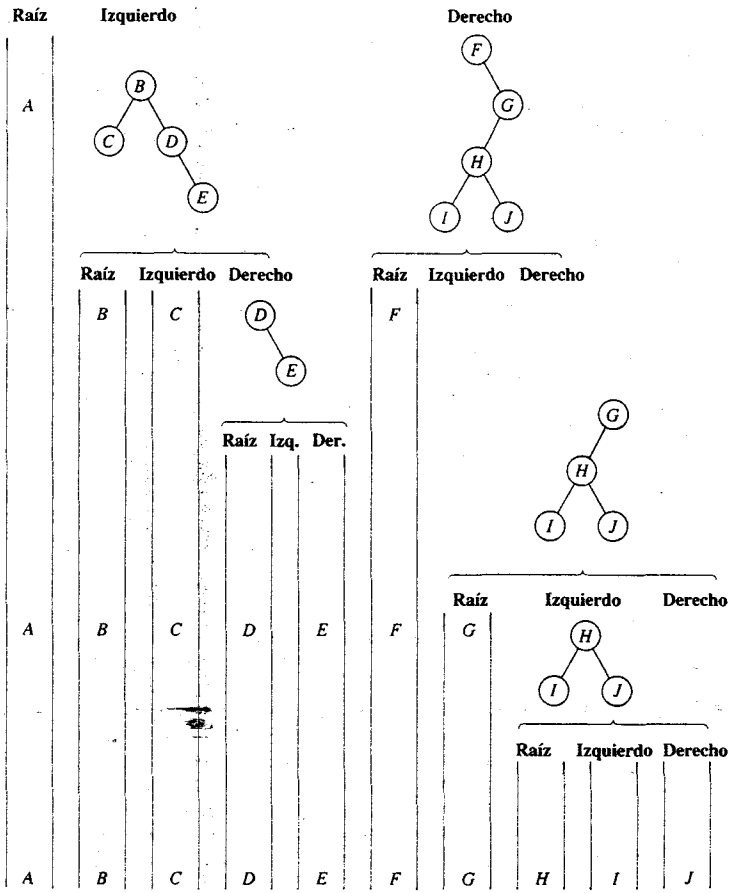


FIGURA 7.6.4 Recorrido en preorden del árbol de la figura 7.6.3.

ALGORITMO 7.6.3 ***Recorrido en entreorden***

Este algoritmo recursivo procesa los vértices de un árbol binario utilizando el recorrido en entreorden.

Entrada: *PT*, la raíz de un árbol binario

Salida: Depende de la interpretación de "procesar" en la línea 5

```
    procedure inorder(PT)
1.    if PT es vacío then
2.      return
3.    l := hijo izquierdo de PT
4.    inorder(l)
5.    procesar PT
6.    r := hijo derecho de PT
7.    inorder(r)
    end inorder
```

EJEMPLO 7.6.4

¿En qué orden se procesan los vértices del árbol de la figura 7.6.3 si se utiliza el recorrido en entreorden?

Si seguimos las líneas 3-7 (izquierdo/raíz/derecho) del algoritmo 7.6.3, obtenemos la enumeración en entreorden *CBDEAFIHJG*. □

ALGORITMO 7.6.5 ***Recorrido en posorden***

Este algoritmo recursivo procesa los vértices de un árbol binario utilizando el recorrido en posorden.

Entrada: *PT*, la raíz de un árbol binario

Salida: Depende de la interpretación de "procesar" en la línea 7

```
    procedure postorder(PT)
1.    if PT es vacío then
2.      return
3.    l := hijo izquierdo de PT
4.    postorder(l)
5.    r := hijo derecho de PT
6.    postorder(r)
7.    procesar PT
    end postorder
```

EJEMPLO 7.6.6

¿En qué orden se procesan los vértices del árbol de la figura 7.6.3 si se utiliza el recorrido en posorden?

Si seguimos las líneas 3-7 (izquierdo/derecho/raíz) del algoritmo 7.6.5, obtenemos la enumeración en posorden *CEDBIJHGFA*. □

Observe que el recorrido en preorden se puede obtener siguiendo la ruta que aparece en la figura 7.6.5, y que el recorrido en posorden inverso se puede obtener siguiendo la ruta que aparece en la figura 7.6.6.

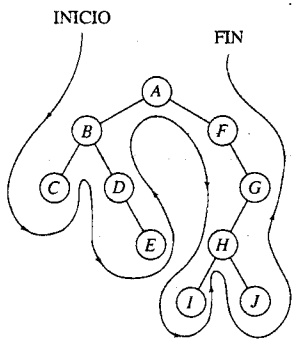


FIGURA 7.6.5
Recorrido en preorden.

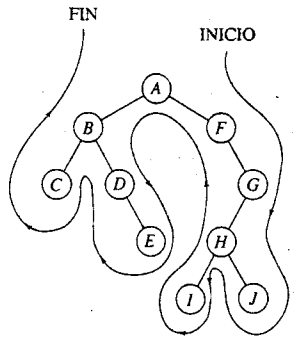


FIGURA 7.6.6
Recorrido en posorden inverso.

Si se guardan datos en un árbol de búsqueda binaria, como se describió en la sección 7.5, el recorrido en entreorden procesará los datos en orden, pues la secuencia izquierdo/raíz/derecho coincide con el orden de los datos en el árbol.

En el resto de esta sección consideraremos la representación de expresiones aritméticas mediante árboles binarios. Tales representaciones facilitan la evaluación de expresiones mediante computadoras.

Restringiremos nuestros operadores a +, −, * y /. Un ejemplo de expresión que utiliza estos operadores es

$$(A + B) * C - D/E. \tag{7.6.1}$$

Esta manera canónica de representar las expresiones es la **forma entrefija de una expresión**. Las variables *A*, *B*, *C*, *D* y *E* se conocen como **operandos**. Los **operadores** +, −, * y / operan sobre pares de operandos o expresiones. En la forma entrefija de una expresión, un operador aparece entre sus operandos.

Una expresión como (7.6.1) se puede representar como un árbol binario. Los vértices terminales corresponden a los operandos, y los vértices internos corresponden a los operadores. La expresión (7.6.1) se representaría como en la figura 7.6.7. En la representación de una expresión mediante un árbol binario, un operador opera sobre sus subárboles izquierdo y derecho. Por ejemplo, en el subárbol cuya raíz es / en la figura 7.6.7, el operador de división opera sobre los operandos *D* y *E*; es decir, hay que dividir *D* entre *E*. En el subárbol cuya raíz es * en la figura 7.6.7, el operador de multiplicación opera sobre el subárbol encabezado por +, que a su vez representa una expresión, y *C*.

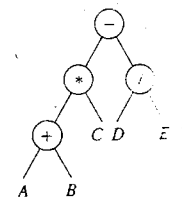


FIGURA 7.6.7
La representación mediante un árbol binario de la expresión $(A + B) * C - D/E$.

En un árbol binario, hacemos una distinción entre los subárboles izquierdo y derecho de un vértice. Los subárboles izquierdo y derecho de un vértice corresponden a los operandos izquierdo y derecho o a expresiones. Esta distinción es importante en las expresiones. Por ejemplo, 4-6 y 6-4 son diferentes.

Si recorremos el árbol binario de la figura 7.6.7 en entreorden, e insertamos un par de paréntesis para cada operación, obtenemos

$$(((A + B) * C) - (D / E)).$$

Esta forma de una expresión se llama la **forma con todos los paréntesis de la expresión**. De esta forma, no tenemos que especificar cuáles operaciones (como la multiplicación) deben realizarse antes que las demás (como la suma), pues los paréntesis indican el orden de las operaciones sin ambigüedad.

Si recorremos el árbol de la figura 7.6.7 en posorden, obtenemos

$$A\ B + C * D\ E / -.$$

Esta forma de la expresión es la **forma posfija de la expresión** (o **notación polaca inversa**). En la forma posfija, el operador aparece después de sus operandos. Por ejemplo, los primeros tres símbolos $A\ B+$ indican que A y B deben sumarse. La ventaja de la forma posfija sobre la forma entrefija es que en la notación posfija no se necesitan paréntesis ni convenciones en relación con el orden de las operaciones. La expresión se evaluará sin ambigüedad. Por éstas y otras razones, muchos compiladores traducen las expresiones entrefijas a su forma posfija. Además, algunas calculadoras necesitan que las expresiones sean introducidas en forma posfija.

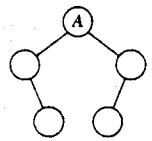
Se puede obtener una tercera forma de una expresión aplicando el recorrido en preorden a una representación de una expresión mediante un árbol binario. En este caso, el resultado es la **forma prefija de la expresión** (o **notación polaca**). Como en la forma posfija, no se necesitan paréntesis ni convenciones acerca del orden de las operaciones. La forma prefija de (7.6.1), obtenida al aplicar el recorrido en preorden al árbol de la figura 7.6.7, es

$$-*+ABC/DE.$$

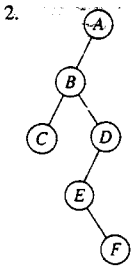
Ejercicios

En los ejercicios 1-5, indique el orden en que se procesan los vértices en un recorrido en preorden, entreorden y posorden.

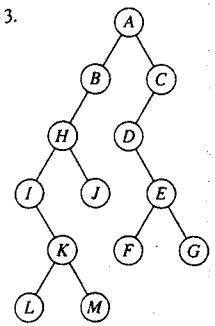
1.



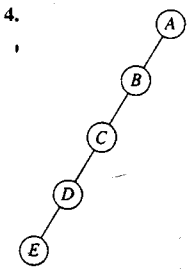
2.



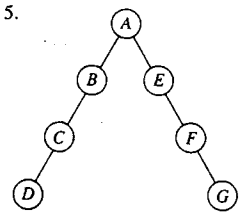
3.



4.



5.



En los ejercicios 6-10, represente la expresión como un árbol binario y escriba las formas prefija y posfija de la expresión.

6. $(A + B) * (C - D)$
7. $((A - C) * D) / (A + (B + D))$
8. $(A * B + C * D) - (A/B - (D + E))$
9. $(((A + B) * C + D) * E) - ((A + B) * C - D)$
10. $(A * B - C/D + E) + (A - B - C - D * D) / (A + B + C)$

En los ejercicios 11-15, represente la expresión posfija como un árbol binario y escriba la forma prefija, la forma entrefija usual y la forma entrefija con todos los paréntesis de la expresión.

11. $AB + C -$
12. $ABC+-$
13. $ABCD+*/E-$
14. $ABC**CDE+/-$
15. $AB + CD*EF/--A*$

En los ejercicios 16-21, determine el valor de la expresión posfija si $A = 1$, $B = 2$, $C = 3$ y $D = 4$.

16. $ABC+-$
17. $AB+C-$
18. $AB+CD*AA/--B*$
19. $ABC**ABC++-$
20. $ABAB*+*D*$
21. $ADBCD*-+*$

22. Muestre, mediante un ejemplo, que distintos árboles binarios con vértices A, B y C pueden tener la misma enumeración en preorden $A\ B\ C$.
23. Muestre que existe un único árbol binario con seis vértices cuya enumeración de los vértices en preorden es $A\ B\ C\ E\ F\ D$ y cuya enumeración de los vértices en entreorden es $A\ C\ F\ E\ B\ D$.
☆ 24. Escriba un algoritmo que reconstruya el árbol binario, dadas sus enumeraciones de los vértices en preorden y en entreorden.
25. Proporcione ejemplos de árboles binarios distintos, B_1 y B_2, cada uno con dos vértices, tales que la enumeración de los vértices de B_1 en preorden sea igual a la enumeración de los vértices de B_2 en preorden y que la enumeración de los vértices de B_1 en posorden sea igual a la enumeración de los vértices de B_2 en posorden.
26. Sean P_1 y P_2 permutaciones de $A\ B\ C\ D\ E\ F$. ¿Existe un árbol binario con vértices A, B, C, D, E y F cuya enumeración en preorden sea P_1 y cuya enumeración en entreorden sea P_2? Explique.
27. Escriba un algoritmo recursivo que imprima el contenido de los vértices terminales de un árbol binario de izquierda a derecha.

28. Escriba un algoritmo recursivo que intercambie todos los hijos izquierdos y derechos de un árbol binario.
29. Escriba un algoritmo recursivo que inicialice cada vértice de un árbol binario con el número de sus descendientes.

En los ejercicios 30 y 31, cada expresión utiliza sólo los operandos $A, B, \ldots, Z$ y los operadores $+, -, *, /$.

★ **30.** Proporcione una condición necesaria y suficiente para que una cadena de símbolos sea una expresión posfija válida.
31. Escriba un algoritmo tal que. dada la representación de una expresión mediante un árbol binario, proporcione como salida la forma entrefija de la expresión con todos sus paréntesis.
32. Escriba un algoritmo que imprima los caracteres y sus códigos dado un árbol de codificación de Huffman (véase el ejemplo 7.1.7). Suponga que cada vértice terminal guarda un carácter y su frecuencia.
★ **33.** Una *cubierta de vértices* de una gráfica $G = (V, E)$ es un subconjunto W de V tal que para cada arista $(v, w) \in E$, entonces $v \in W$ o $w \in W$. El *tamaño* de una cubierta de vértices W es el número de vértices en W. Escriba un algoritmo que determina una cubierta de vértices de tamaño mínimo para un árbol $T = (V, E)$ cuyo tiempo en el peor de los casos sea $\Theta(|E|)$.

7.7 ÁRBOLES DE DECISIÓN Y EL TIEMPO MÍNIMO PARA EL ORDENAMIENTO

El árbol binario de la figura 7.7.1 proporciona un algoritmo para elegir un restaurante. Cada vértice interno pregunta algo. Si partimos de la raíz, respondemos cada pregunta, y seguimos la arista adecuada, llegaremos a un vértice terminal donde se elige un restaurante. Tal árbol es un **árbol de decisión**. En esta sección utilizaremos los árboles de decisión para especificar algoritmos y para obtener cotas inferiores para un ordenamiento en el peor de los casos, así como para resolver ciertos juegos con monedas. Comenzaremos con los juegos de monedas.

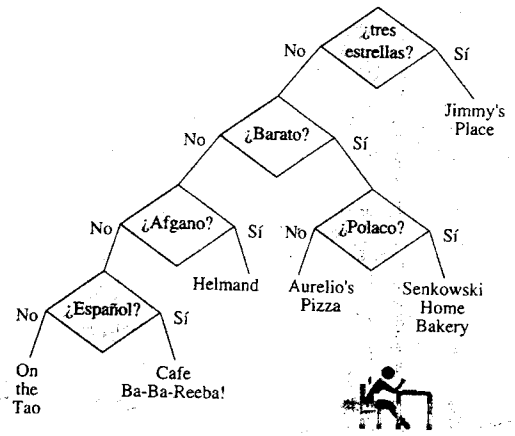


FIGURA 7.7.1 Un árbol de decisión.

EJEMPLO 7.7.1 ***Juego de cinco monedas***

Cinco monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. El problema es identificar la moneda distinta y si es más pesada o más ligera que las demás, utilizando una balanza de platillos (véase la figura 7.7.2), la cual compara el peso de dos conjuntos de monedas.

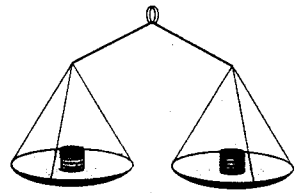


FIGURA 7.7.2 Una balanza de platillos para comparar pesos de monedas.

En la figura 7.7.3 aparece un algoritmo para resolver este problema en forma de árbol de decisión. Las monedas se etiquetan como C_1, C_2, C_3, C_4, C_5. Como se muestra, comenzamos en la raíz y colocamos la moneda C_1 en el platillo izquierdo y la moneda C_2 en el platillo derecho. Una arista etiquetada ↙ indica que el lado izquierdo de la balanza es más pesado que el lado derecho. De manera análoga, una arista etiquetada ↘ indica que el lado derecho de la balanza es más pesado que el lado izquierdo y una arista etiquetada → indica que los dos lados están en equilibrio. Por ejemplo, en la raíz, al comparar C_1 con C_2, si el lado izquierdo es más pesado que el lado derecho, sabemos que C_1 es la moneda pesada o que C_2 es la moneda ligera. En este caso, como muestra el árbol de decisión, continuamos comparando C_1 con C_5 (que sabemos es una moneda buena) y de inmediato determinamos si la moneda mala es C_1 o C_2 y si es más ligera o más pesada. Los vértices terminales proporcionan la solución. Por ejemplo, si al comparar C_1 con C_5 los platillos se equilibran, seguimos la arista hasta el vértice terminal etiquetado C_2, L, lo cual nos dice que la moneda mala es C_2 y que es más ligera que las demás. □

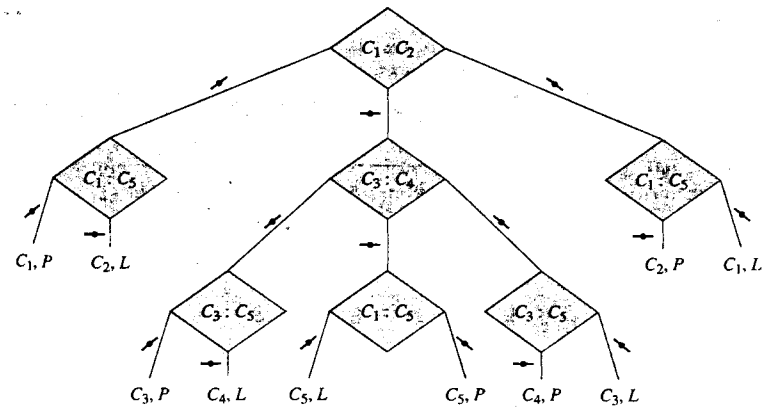


FIGURA 7.7.3 Un algoritmo para resolver el problema de las cinco monedas.

Si definimos el tiempo en el peor de los casos para el problema de pesar las monedas como el número de pesadas necesarias en el peor de los casos, es fácil determinar el tiempo en el peor de los casos a partir del árbol de decisión; el tiempo en el peor de los casos es igual a la altura del árbol. Por ejemplo, la altura del árbol de decisión de la figura 7.7.3 es 3, de modo que el tiempo en el peor de los casos para este algoritmo es igual a 3.

Podemos utilizar árboles de decisión para mostrar que el algoritmo dado en la figura 7.7.3 para resolver el juego de las cinco monedas es óptimo, es decir, que *ningún* algoritmo que resuelva el juego de las cinco monedas tiene un tiempo en el peor de los casos menor que 3.

Argumentaremos por contradicción para mostrar que ningún algoritmo que resuelva el juego de las cinco monedas tiene un tiempo en el peor de los casos menor que 3. Supongamos que existe un algoritmo que resuelve el juego de las cinco monedas en el peor de los casos en dos o menos pesadas. El algoritmo se puede describir mediante un árbol de decisión, y como el tiempo en el peor de los casos es 2 o menos, la altura del árbol de decisión es dos o menos. Como cada vértice interno tiene a lo más tres hijos, tal árbol debe tener a lo más nueve vértices terminales (véase la figura 7.7.4). Ahora, los vértices terminales corresponden a los posibles resultados. Así, un árbol de decisión de altura 2 o menos puede tener a lo más nueve resultados. Pero el juego de las cinco monedas tiene 10 resultados:

$$C_1, L, \quad C_1, P, \quad C_2, L, \quad C_2, P, \quad C_3, L,$$
$$C_3, P, \quad C_4, L, \quad C_4, P, \quad C_5, L, \quad C_5, P.$$

Esto es una contradicción. Por tanto, ningún algoritmo que resuelva el juego de las cinco monedas tiene un tiempo en el peor de los casos menor que 3, y el algoritmo de la figura 7.7.3 es óptimo.

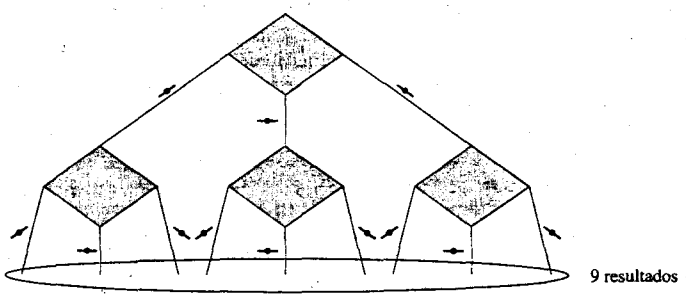


FIGURA 7.7.4 Un algoritmo para el juego de las cinco monedas que utiliza a lo más dos pesadas.

Hemos visto cómo se puede utilizar un árbol de decisión para obtener una cota inferior del tiempo necesario para resolver un problema en el peor de los casos. A veces, la cota inferior no se puede alcanzar.

Consideremos el juego con cuatro monedas (las reglas son iguales a las del juego de las cinco monedas, excepto que el número de monedas se reduce en uno). Como ahora existen ocho resultados en vez de 10, podemos concluir que cualquier algoritmo que resuelva el juego de cuatro monedas requiere al menos dos pesadas en el peor de los casos. (Ahora *no podemos* concluir que se necesitan al menos tres pesadas en el peor de los casos.) Sin embargo, un análisis más cuidadoso muestra que, de hecho, se necesitan tres pesadas.

La primera pesada compara dos monedas con dos monedas o una moneda con una moneda. La figura 7.7.5 muestra que si comenzamos comparando dos monedas con dos monedas, el árbol de decisión puede tomar en cuenta seis resultados a lo más. Como existen ocho resultados, ningún algoritmo que comience comparando dos monedas con dos monedas puede resolver el problema con dos pesadas o menos en el peor de los casos. De manera análoga, la figura 7.7.6 muestra que si comenzamos comparando una moneda con otra y las monedas se equilibran, el árbol de decisión puede tomar en cuenta sólo tres resultados. Como existen cuatro resultados posibles después de identificar dos monedas buenas, ningún algoritmo que comience comparando una moneda con otra puede resolver el problema en dos pesadas o menos en el peor de los casos. Por tanto, cualquier algoritmo que resuelva el juego de las cuatro monedas necesita al menos tres pesadas en el peor de los casos.

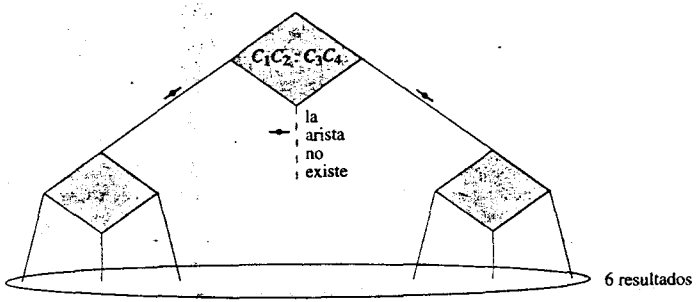


FIGURA 7.7.5 Un algoritmo para el juego de cuatro monedas que comienza comparando dos monedas con dos monedas.

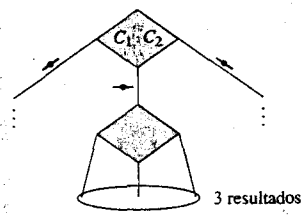


FIGURA 7.7.6 Un algoritmo para el juego de cuatro monedas que comienza comparando una moneda con otra.

Si modificamos el juego de las cuatro monedas, pidiendo sólo que identifiquemos la moneda mala (sin determinar si es más pesada o más ligera), podemos resolver el juego en dos pesadas en el peor de los casos (véase el ejercicio 1).

Ahora revisaremos el ordenamiento. Podemos utilizar árboles de decisión para estimar el tiempo necesario para ordenar en el peor de los casos.

El problema de ordenamiento se describe fácilmente: Dados n elementos

$$x_1, \quad \ldots, \quad x_n,$$

hay que ordenarlos en forma ascendente (o descendente). Restringiremos nuestra atención a los algoritmos de ordenamiento que comparan dos elementos y, con base en el resultado de la comparación, modifican la lista original.

EJEMPLO 7.7.2

La figura 7.7.7 muestra un árbol de decisión para el algoritmo de ordenamiento de a_1, a_2, a_3. Cada arista se etiqueta con el arreglo de la lista basado en la pregunta en un vértice interno. Los vértices terminales proporcionan el orden requerido.

Definimos el tiempo necesario para el ordenamiento en el peor de los casos como el número de comparaciones necesarias en el peor de los casos. Al igual que en el caso de los árboles de decisión que resuelven los juegos con monedas, la altura de un árbol de decisión que resuelve un problema de ordenamiento es igual al tiempo en el peor de los casos. Por ejemplo, el tiempo en el peor de los casos para el algoritmo dado por el árbol de decisión de la figura 7.7.7 es igual a 3. Mostraremos que este algoritmo es óptimo, es decir, que *ningún* algoritmo que ordene tres elementos tiene un tiempo menor que 3 en el peor de los casos.

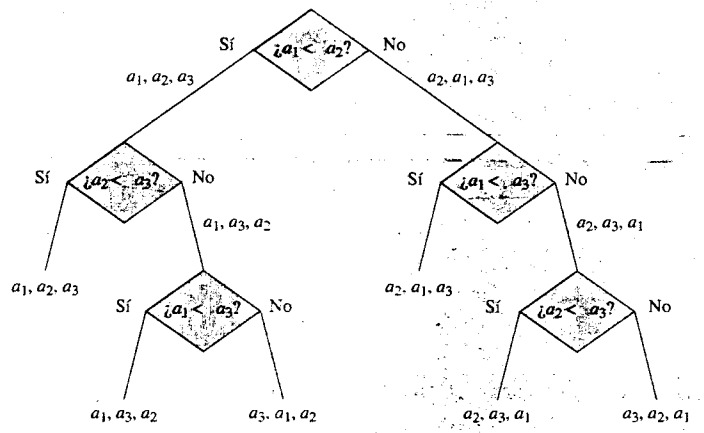


FIGURA 7.7.7 Un algoritmo para ordenar a_1, a_2, a_3.

Argumentaremos por contradicción para mostrar que ningún algoritmo que ordene tres elementos tiene un tiempo menor que 3 en el peor de los casos. Supongamos que existe un algoritmo que ordena tres elementos con dos o menos comparaciones en el peor de los casos. El algoritmo se puede describir mediante un árbol de decisión, y como el tiempo en el peor de los casos es 2 o menos, la altura del árbol de decisión es dos o menos. Como cada vértice interno tiene a lo más dos hijos, tal árbol puede tener a lo más cuatro vértices terminales (véase la figura 7.7.8). Ahora, los vértices terminales corresponden a los resultados posibles. Así, un árbol de decisión con altura 2 o menos sólo puede tomar en cuenta

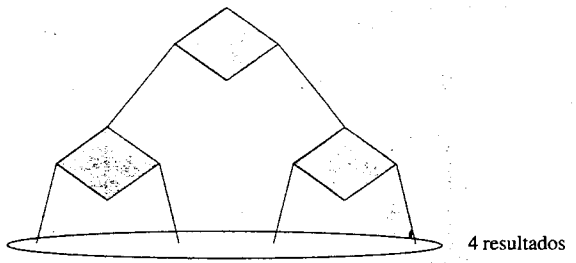
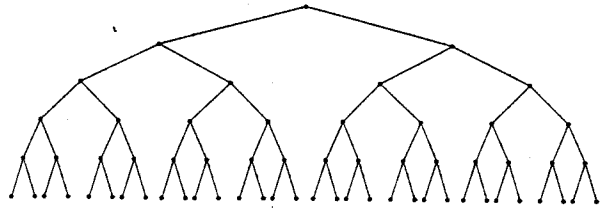


FIGURA 7.7.8 Un algoritmo de ordenamiento que realiza dos comparaciones como máximo.



Nivel	*Número de vértices*
0	1
1	2
2	4
3	8
4	16
5	32

FIGURA 7.7.9 Nivel comparado con el número máximo de vértices en ese nivel en un árbol binario.

cuatro resultados a lo más. Pero el problema de ordenar tres elementos tiene seis resultados posibles, correspondientes a las $3! = 6$ formas en que se pueden ordenar los elementos:

$$s_1, s_2, s_3, \quad s_1, s_3, s_2, \quad s_2, s_1, s_3, \quad s_2, s_3, s_1, \quad s_3, s_1, s_2, \quad s_3, s_2, s_1.$$

Esto es una contradicción. Por tanto, ningún algoritmo que ordene tres elementos tiene un tiempo menor que 3 en el peor de los casos y el algoritmo de la figura 7.7.7 es óptimo. □

Como $4! = 24$, existen 24 resultados posibles para el problema de ordenar cuatro elementos. Para acomodar 24 vértices terminales, debemos tener un árbol por lo menos de altura 5 (véase la figura 7.7.9). Por tanto, cualquier algoritmo que ordene cuatro elementos necesita al menos cinco comparaciones en el peor de los casos. El ejercicio 9 consiste en dar un algoritmo que ordene cuatro elementos utilizando cinco comparaciones en el peor de los casos.

El método del ejemplo 7.7.2 se puede utilizar para dar una cota inferior sobre el número de comparaciones neeesarias para ordenar una cantidad arbitraria de elementos en el peor de los casos.

TEOREMA 7.7.3

Si $f(n)$ es el número de comparaciones necesarias para ordenar n elementos en el peor de los casos mediante un algoritmo de ordenamiento, entonces $f(n) = \Omega(n \lg n)$.

Demostración. Sea T el árbol de decisión que representa el algoritmo para una entrada de tamaño n y sea h la altura de T. Entonces el algoritmo necesita h comparaciones en el peor de los casos, de modo que

$$h = f(n). \tag{7.7.1}$$

El árbol T tiene al menos $n!$ vértices terminales, de modo que por el teorema 7.5.6,

$$\lg n! \leq h. \tag{7.7.2}$$

El ejemplo 3.5.8 muestra que $\lg n! = \Theta(n \lg n)$; así, para alguna constante positiva C,

$$Cn \lg n \leq \lg n! \tag{7.7.3}$$

para todo entero n, salvo un número finito. Al combinar (7.7.1), (7.7.2) y (7.7.3), obtenemos

$$Cn \lg n \leq f(n)$$

para toda n, salvo un número finito. Por tanto,

$$f(n) = \Omega(n \lg n). \qquad \blacksquare$$

El teorema 5.3.10 establece que el ordenamiento por fusión (algoritmo 5.3.8) utiliza $\Theta(n \lg n)$ en el peor de los casos y, por el teorema 7.7.3, es óptimo. Se sabe que muchos otros algoritmos de ordenamiento también alcanzan la cota óptima $\Theta(n \lg n)$ de comparaciones; uno de ellos, el ordenamiento por torneo, se describe antes del ejercicio 12.

Ejercicios

1. Cuatro monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. Trace un árbol de decisión que proporcione un algoritmo para identificar a lo más en dos pesadas a la moneda mala (pero que no necesariamente determine si es más pesada o más ligera que las otras) utilizando sólo una balanza de platillos.
2. Muestre que se necesitan al menos dos pesadas para resolver el problema del ejercicio 1.
3. Ocho monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. Trace un árbol de decisión que proporcione un algoritmo para identificar a lo más en tres pesadas a la moneda mala y que determine si es más pesada o más ligera que las otras utilizando sólo una balanza de platillos.
4. Doce monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. Trace un árbol de decisión que proporcione un algoritmo para identificar a lo más en tres pesadas a la moneda mala y que determine si es más pesada o más ligera que las otras utilizando sólo una balanza de platillos.
5. ¿Cuál es el error del siguiente argumento, el cual supuestamente muestra que el juego de las doce monedas necesita al menos cuatro pesadas en el peor de los casos si comenzamos pesando cuatro monedas contra cuatro monedas?

   Si pesamos cuatro monedas contra cuatro monedas y éstas se equilibran, entonces debemos determinar la moneda mala mediante las cuatro monedas restantes. Pero el análisis de esta sección mostró que la determinación de la moneda mala entre cuatro monedas necesita al menos tres pesadas en el peor de los casos. Por tanto, en el peor de los casos, si comenzamos pesando cuatro monedas contra cuatro, el juego de las doce monedas necesita al menos cuatro pesadas.

☆ 6. Trece monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. ¿Cuántas pesadas se necesitan en el peor de los casos para determinar la moneda mala y saber si es más pesada o más ligera que las demás utilizando sólo una balanza de platillos? Demuestre su respuesta.

7. Resuelva el ejercicio 6 para el juego de 14 monedas.
8. $(3^n - 3)/2$, $n \geq 2$, monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. [Kurosaka] dio un algoritmo para determinar la moneda mala y saber si es más pesada o más ligera que las demás, utilizando sólo una balanza de platillos, en n pesadas en el peor de los casos. Demuestre que la moneda no se puede determinar e identificar como pesada o ligera en menos de n pesadas.
9. Proporcione un algoritmo que ordene cuatro elementos utilizando cinco comparaciones en el peor de los casos.
10. Utilice árboles de decisión para determinar una cota inferior sobre el número de comparaciones necesarias para ordenar cinco elementos en el peor de los casos. Proporcione un algoritmo que utilice este número de comparaciones para ordenar cinco elementos en el peor de los casos.

11. Utilice árboles de decisión para determinar una cota inferior sobre el número de comparaciones necesarias para ordenar seis elementos en el peor de los casos. Proporcione un algoritmo que utilice este número de comparaciones para ordenar seis elementos en el peor de los casos.

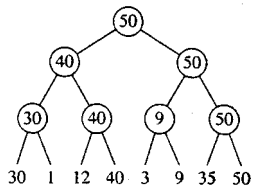
Los ejercicios 12-18 se refieren al *ordenamiento por torneo.*

Ordenamiento por torneo. Se da una sucesión

$$s_1, \ \ldots, \ s_{2^k}$$

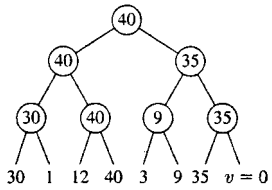
que debe ordenarse en forma creciente.

Formamos un árbol binario con vértices terminales etiquetados $s_1, \ldots, s_{2^k}$. Al margen aparece un ejemplo.



Trabajamos de izquierda a derecha para crear un padre de cada par y lo etiquetamos con el máximo de los hijos. Continuamos de esta forma hasta llegar a la raíz. En este momento se habrá determinado m, el valor máximo.

Para determinar el segundo valor más grande, primero se elige un valor v menor que todos los elementos de la sucesión. Reemplazamos el vértice terminal w que contiene a m con v. Reetiquetamos los vértices siguiendo el camino de w a la raíz, como muestra la figura al margen. En este momento se habrá determinado el segundo valor más grande. Se continúa de esta forma hasta ordenar la sucesión.



12. ¿Por qué es adecuado el nombre "torneo"?
13. Trace los dos árboles que se crearían después del árbol anexo, al aplicar el ordenamiento por torneo.
14. ¿Cuántas comparaciones necesita el ordenamiento por torneo para determinar el elemento máximo?
15. Muestre que cualquier algoritmo que determine el valor máximo entre n elementos necesita al menos $n - 1$ comparaciones.
16. ¿Cuántas comparaciones necesita el ordenamiento por torneo para determinar el segundo elemento más grande?
17. Escriba el ordenamiento por torneo como un algoritmo formal.
18. Muestre que si n es una potencia de 2, el ordenamiento por torneo necesita $\Theta(n \lg n)$ comparaciones.
19. Proporcione un ejemplo de una situación real (como la de la figura 7.7.1) que se pueda modelar mediante un árbol de decisión. Trace el árbol de decisión.
20. Trace un árbol de decisión que se pueda utilizar para determinar las personas que deben recibir un reembolso de impuestos.
21. Trace un árbol de decisión que proporcione una estrategia razonable para jugar blackjack (véase, por ejemplo, [Ainslie]).

7.8 *ISOMORFISMOS DE ÁRBOLES*

En la sección 6.6 definimos lo que quiere decir que dos gráficas sean isomorfas. (Tal vez el lector debería revisar la sección 6.6 antes de continuar.) En esta sección analizamos los árboles isomorfos, los árboles con raíz isomorfos y los árboles binarios isomorfos.

El teorema 6.6.4 establece que las gráficas simples G_1 y G_2 son isomorfas si y sólo si existe una función, f, uno a uno y sobre el conjunto de vértices de G_1 al conjunto de vértices de G_2 que preserva la relación de adyacencia, en el sentido de que los vértices v_i y v_j son adyacentes en G_1 si y sólo si los vértices $f(v_i)$ y $f(v_j)$ son adyacentes en G_2. Como un árbol (libre) es una gráfica simple, los árboles T_1 y T_2 son isomorfos si y sólo si existe una función, f, uno a uno y sobre el conjunto de vértices de T_1 al conjunto de vértices de T_2 que preserva la relación de adyacencia, en el sentido de que los vértices v_i y v_j son adyacentes en T_1 si y sólo si los vértices $f(v_i)$ y $f(v_j)$ son adyacentes en T_2.

EJEMPLO 7.8.1

La función f del conjunto de vértices del árbol T_1 de la figura 7.8.1 al conjunto de vértices del árbol T_2 que aparece en la figura 7.8.2 definida como

$$f(a) = 1, \quad f(b) = 3, \quad f(c) = 2, \quad f(d) = 4, \quad f(e) = 5$$

es una función uno a uno, sobre, que preserva la relación de adyacencia. Así, los árboles T_1 y T_2 son isomorfos. □

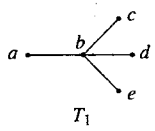


FIGURA 7.8.1 Un árbol.

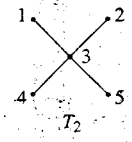


FIGURA 7.8.2 Un árbol isomorfo al árbol de la figura 7.8.1.

Como en el caso de las gráficas, podemos mostrar que dos árboles no son isomorfos si podemos exhibir un invariante no compartido por los árboles.

EJEMPLO 7.8.2

Los árboles T_1 y T_2 de la figura 7.8.3 no son isomorfos, pues T_2 tiene un vértice (x) de grado 3, pero T_1 no tiene un vértice de grado 3. □

Podemos mostrar que existen tres árboles no isomorfos con cinco vértices; estos árboles aparecen en las figuras 7.8.1 y 7.8.3.

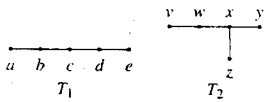


FIGURA 7.8.3 Árboles no isomorfos. T_2 tiene un vértice de grado 3, pero T_1 no.

TEOREMA 7.8.3

Existen tres árboles no isomorfos con cinco vértices.

Demostración. Daremos un argumento para mostrar que cualquier árbol con cinco vértices es isomorfo a alguno de los árboles en las figuras 7.8.1 y 7.8.3.

Si T es un árbol con cinco vértices, por el teorema 7.2.3, T tiene cuatro aristas. Si T tuviera un vértice v de grado mayor que 4, v incidiría en más de cuatro aristas. Esto implica que cada vértice en T tiene grado 4 como máximo.

Primero determinaremos todos los árboles no isomorfos con cinco vértices, en los que el grado máximo de los vértices sea 4. Luego determinaremos todos los árboles no isomorfos con cinco vértices en los cuales el grado máximo de los vértices sea 3, y así sucesivamente.

Sea T un árbol con cinco vértices y supongamos que T tiene un vértice v de grado 4. Entonces existen cuatro aristas incidentes en v y, por el teorema 7.2.3, éstas son todas las aristas. Esto implica que en este caso, T es isomorfo al árbol de la figura 7.8.1.

Supongamos que T es un árbol con cinco vértices y que el grado máximo de los vértices es 3. Sea v un vértice de grado 3. Entonces v es incidente en tres aristas, como muestra la figura 7.8.4. La cuarta arista no puede ser incidente en v, pues entonces v tendría grado 4. Así, la cuarta arista es incidente sobre uno de los vértices v_1, v_2 o v_3. Al agregar una arista incidente en alguno de los vértices v_1, v_2 o v_3, obtenemos un árbol isomorfo al árbol T_2 de la figura 7.8.3.

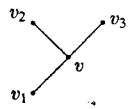


FIGURA 7.8.4
El vértice v tiene grado 3.

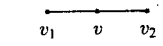


FIGURA 7.8.5
El vértice v tiene grado 2.

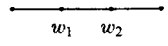


FIGURA 7.8.6
Agregado de una tercera arista a la gráfica de la figura 7.8.5.

Ahora, supongamos que T es un árbol con cinco vértices y que el grado máximo de los vértices es 2. Sea v un vértice de grado 2. Entonces v es incidente en dos aristas, como muestra la figura 7.8.5. Una tercera arista no puede ser incidente en v; así, debe ser incidente en v_1 o en v_2. Al agregar la tercera arista obtenemos la gráfica de la figura 7.8.6. Por la misma razón, la cuarta arista no puede incidir en los vértices w_1 o w_2 de la figura 7.8.6. Al agregar la última arista se obtiene un árbol isomorfo al árbol T_1 de la figura 7.8.3.

Como un árbol con cinco vertices debe tener un vértice de grado 2, hemos determinado todos los árboles no isomorfos con cinco vértices. ■

Para que dos árboles *con raíz* T_1 y T_2 sean isomorfos, debe existir una función, f, uno a uno y sobre T_1 a T_2 que preserve la relación de adyacencia y que preserve la raíz. Esta última condición significa que f(raíz de T_1) = raíz de T_2. A continuación damos la definición formal.

DEFINICION 7.8.4

Sea T_1 un árbol con raíz r_1 y sea T_2 un árbol con raíz r_2. Los árboles con raíz T_1 y T_2 son *isomorfos* si existe una función, f, uno a uno y sobre el conjunto de vértices de T_1 en el conjunto de vértices de T_2 tal que

(a) Los vértices v_i y v_j son adyacentes en T_1 si y sólo si los vértices $f(v_i)$ y $f(v_j)$ son adyacentes en T_2.

(b) $f(r_1) = r_2$.

Decimos que la función f es un *isomorfismo*.

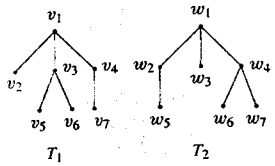


FIGURA 7.8.7
Árboles con raíz isomorfos.

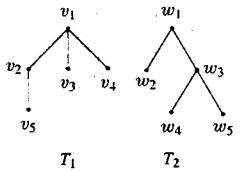


FIGURA 7.8.8
Árboles con raíz no isomorfos. (Los árboles son isomorfos como árboles *libres*.)

EJEMPLO 7.8.5

Los árboles con raíz T_1 y T_2 de la figura 7.8.7 son isomorfos. Un isomorfismo es

$$f(v_1) = w_1, \quad f(v_2) = w_3, \quad f(v_3) = w_4, \quad f(v_4) = w_2,$$
$$f(v_5) = w_7, \quad f(v_6) = w_6, \quad f(v_7) = w_5. \qquad \square$$

El isomorfismo del ejemplo 7.8.5 no es único. ¿Podría el lector determinar otro isomorfismo de los árboles con raíz de la figura 7.8.7?

EJEMPLO 7.8.6

Los árboles con raíz T_1 y T_2 de la figura 7.8.8 no son isomorfos, pues la raíz de T_1 tiene grado 3 y la raíz de T_2 tiene grado 2. Estos árboles son isomorfos como árboles *libres*. Cada uno de ellos es isomórfo al árbol T_2 de la figura 7.8.3. □

Al argumentar como en la demostración del teorema 7.8.3, podemos mostrar que existen cuatro árboles con raíz, no isomorfos, con cuatro vértices.

TEOREMA 7.8.7

Existen cuatro árboles con raíz, no isomorfos, con cuatro vértices. Estos árboles aparecen en la figura 7.8.9.

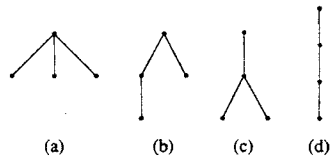


FIGURA 7.8.9 Los cuatro árboles con raíz, no isomorfos, con cuatro vértices.

Demostración. Primero determinaremos todos los árboles con raíz, no isomorfos, con cuatro vértices, en los cuales la raíz tiene grado 3; luego determinaremos los árboles con raíz, no isomorfos, con cuatro vértices en los cuales la raíz tiene grado 2, y así sucesivamente. Observamos que la raíz de un árbol con raíz y cuatro vértices no puede tener grado mayor que 3.

Un árbol con raíz con cuatro vértices en el que la raíz tiene grado 3 debe ser isomorfo al árbol de la figura 7.8.9a.

Un árbol con raíz con cuatro vértices en el que la raíz tiene grado 2 debe ser isomorfo al árbol de la figura 7.8.9b.

Sea T un árbol con raíz con cuatro vértices en el que la raíz tiene grado 1. Entonces la raíz es incidente en una arista. Las dos aristas restantes se pueden agregar de dos formas (véanse las figuras 7.8.9c y d). Por tanto, todos los árboles con raíz, no isomorfos, con cuatro vértices, aparecen en la figura 7.8.9. ■

Los árboles binarios son un tipo especial de árboles con raíz; así, un isomorfismo de árboles binarios debe preservar la relación de adyacencia y las raíces. Sin embargo, en los árboles binarios, un hijo se designa como hijo izquierdo o como hijo derecho. Necesitamos que un isomorfismo de árboles binarios preserve los hijos izquierdo y derecho. A continuación se proporciona la definición formal.

DEFINICIÓN 7.8.8

Sea T_1 un árbol binario con raíz r_1 y sea T_2 un árbol binario con raíz r_2. Los árboles binarios T_1 y T_2 son *isomorfos* si existe una función, f, uno a uno y sobre el conjunto de vértices de T_1 al conjunto de vértices de T_2 tal que

(a) Los vértices v_i y v_j son adyacentes en T_1 si y sólo si los vértices $f(v_i)$ y $f(v_j)$ son adyacentes en T_2.

(b) $f(r_1) = r_2$.

(c) v es un hijo izquierdo de w en T_1 si y sólo si $f(v)$ es un hijo izquierdo de $f(w)$ en T_2.

(d) v es un hijo derecho de w en T_1 si y sólo si $f(v)$ es un hijo derecho de $f(w)$ en T_2.

Decimos que la función f es un *isomorfismo*.

EJEMPLO 7.8.9

Los árboles binarios T_1 y T_2 de la figura 7.8.10 son isomorfos. El isomorfismo es $f(v_i) = w_i$ para $i = 1, \ldots, 4$. □

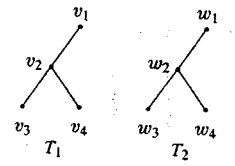


FIGURA 7.8.10 Árboles binarios isomorfos.

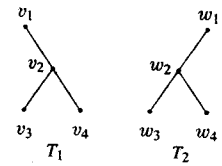


FIGURA 7.8.11 Árboles binarios no isomorfos. (Los árboles son isomorfos como árboles *con raíz* y como árboles *libres*.)

EJEMPLO 7.8.10

Los árboles binarios T_1 y T_2 de la figura 7.8.10 no son isomorfos. La raíz v_1 de T_1 tiene un hijo derecho, pero la raíz w_1 de T_1 no tiene hijo derecho. □

Los árboles T_1 y T_2 de la figura 7.8.10 *son* isomorfos como árboles con raíz y como árboles libres. Como árboles *con raíz*, cualquiera de los árboles de la figura 7.8.11 es isomorfo al árbol con raíz T de la figura 7.8.9c.

Al argumentar como en las demostraciones de los teoremas 7.8.3 y 7.8.7, podemos mostrar que existen cinco árboles binarios, no isomorfos, con tres vértices.

TEOREMA 7.8.11

Existen cinco árboles binarios no isomorfos con tres vértices. Estos cinco árboles binarios aparecen en la figura 7.8.12.

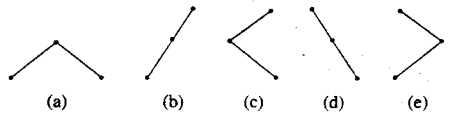


FIGURA 7.8.12 Los cinco árboles binarios, no isomorfos, con tres vértices.

Demostración. Primero determinamos todos los árboles binarios, no isomorfos, con tres vértices, en los cuales la raíz tenga grado 2. Luego determinaremos todos los árboles binarios, no isomorfos, con tres vértices, en los cuales la raíz tenga grado 1. Observemos que la raíz de cualquier árbol binario no puede tener grado mayor que 2.

Un árbol binario con tres vértices en el cual la raíz tenga grado 2 debe ser isomorfo al árbol de la figura 7.8.12a. En un árbol binario con tres vértices en el cual la raíz tenga grado 1, la raíz tiene un hijo izquierdo y no tiene hijo derecho, o bien un hijo derecho y no tiene hijo izquierdo. Si la raíz tiene un hijo izquierdo, este hijo tiene a su vez un hijo izquierdo o derecho. Obtenemos los dos árboles binarios de las figuras 7.8.12b y c. De manera similar, si la raíz tiene un hijo derecho, este hijo tiene a su vez un hijo izquierdo o un hijo derecho. Obtenemos los dos árboles binarios de la figura 7.8.12d y e. Por tanto, todos los árboles binarios no isomorfos con tres vértices aparecen en la figura 7.8.12. ■

Si S es un conjunto de árboles de un tipo particular (por ejemplo, S es un conjunto de árboles libres o S es un conjunto de árboles con raíz o S es un conjunto de árboles binarios) y definimos una relación R en S mediante la regla $T_1 R T_2$ si T_1 y T_2 son isomorfos, entonces R es una relación de equivalencia. Cada clase de equivalencia consta de un conjunto de árboles isomorfos entre sí.

En el teorema 7.8.3 mostramos que existen tres árboles libres no isomorfos con cinco vértices. En el teorema 7.8.7 mostramos que existen cuatro árboles con raíz no isomorfos con cuatro vértices. En el teorema 7.8.11 mostramos que existen cinco árboles binarios no isomorfos con tres vértices. El lector podría preguntarse si existen fórmulas para el número de árboles no isomorfos con n vértices de un tipo particular. Existen fórmulas para el número de árboles libres de n vértices no isomorfos, para el número de árboles con raíz de n vértices no isomorfos, y para el número de árboles binarios de n vértices no isomorfos. Las fórmulas para los dos primeros casos son un poco complicadas. Además, la deducción de estas fórmulas requiere técnicas más avanzadas que las desarrolladas en este libro. Las fórmulas y las demostraciones aparecen en [Deo, sección 10-3]. Deduciremos una fórmula para el número de árboles binarios con n vértices.

TEOREMA 7.8.12

Existen $C(2n, n) / (n + 1)$ *árboles binarios no isomorfos con* n *vértices.*

Demostración. Sea a_n el número de árboles binarios con n vértices. Por ejemplo, $a_0 = 1$, pues existe un árbol binario sin vértices; $a_1 = 1$ pues existe un árbol binario con un vértice; $a_2 = 2$ pues existen dos árboles binarios con dos vértices (véase la figura 7.8.13); y $a_3 = 5$, pues existen cinco árboles binarios con tres vértices (véase la figura 7.8.12).

Deduciremos una relación de recurrencia para la sucesión $a_0, a_1, \ldots$. Consideremos la construcción de un árbol binario con n vértices, $n > 0$. Un vértice debe ser la raíz. Como restan $n - 1$ vértices, si el subárbol izquierdo tiene k vértices, el subárbol derecho debe tener $n - k - 1$ vértices. Construimos un árbol binario de n vértices cuyo subárbol izquierdo tiene k vértices y cuyo subárbol derecho tiene $n - k - 1$ vértices mediante un proceso de dos pasos: Construimos el subárbol izquierdo, construimos el subárbol derecho. (La figura 7.8.14 muestra esta construcción para $n = 6$ y $k = 2$.) Por el principio de multiplicación, esta construcción se puede realizar de $a_k a_{n-k-1}$ formas. Los diversos valores de k proporciona distintos árboles binarios de n vértices, de modo que por el principio de la suma, el número total de árboles binarios de n vértices es

$$\sum_{k=0}^{n-1} a_k a_{n-k-1}.$$

Obtenemos la relación de recurrencia

$$a_n = \sum_{k=0}^{n-1} a_k a_{n-k-1}, \qquad n \geq 1.$$

Pero esta relación de recurrencia y la condición inicial $a_0 = 1$ definen la sucesión de números de Catalan (véanse los ejemplos 4.2.23 y 5.1.7). Así, a_n es igual al número de Catalan $C(2n, n) / (n + 1)$. ■

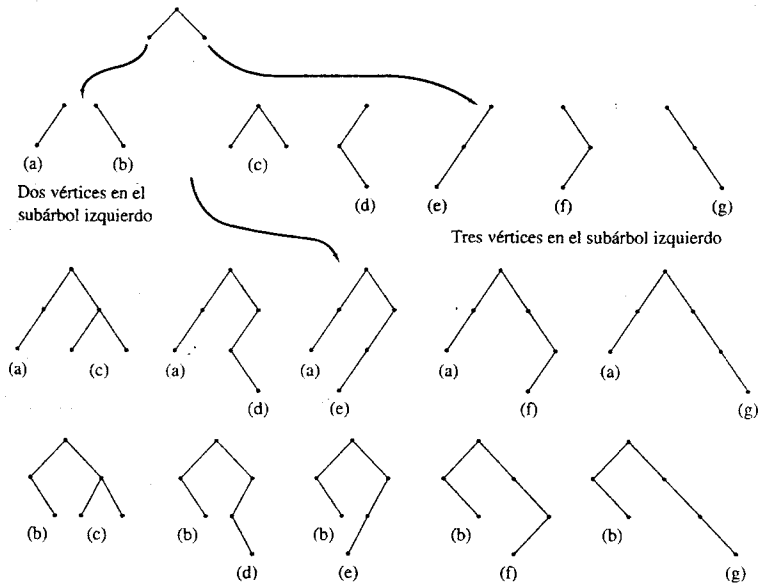


FIGURA 7.8.14 La demostración del teorema 7.8.12 para el caso de $n = 6$ vértices y $k = 2$ vértices en el subárbol izquierdo.

FIGURA 7.8.13
Los dos árboles binarios no isomorfos con dos vértices.

Al analizar los isomorfismos de gráficas en la sección 6.6, mencionamos que no existe un método rápido para decidir si dos gráficas arbitrarias son isomorfas. La situación es distinta para los árboles. Es posible determinar si dos árboles arbitrarios son isomorfos en un tiempo polinomial. Como caso particular, daremos un algoritmo de tiempo lineal para determinar si dos árboles binarios T_1 y T_2 son isomorfos. El algoritmo se basa en el recorrido en preorden (véase la sección 7.6). Primero verificamos que T_1 y T_2 sean no vacíos, después de lo cual verificamos que los subárboles izquierdos de T_1 y T_2 sean isomorfos y que los subárboles derechos de T_1 y T_2 sean isomorfos.

ALGORITMO 7.8.13 *Verificación de que dos árboles binarios sean isomorfos*

Entrada: Las raíces r_1 y r_2 de dos árboles binarios. (Si el primer árbol es vacío, r_1 tiene el valor especial *null*. Si el segundo árbol es vacío, r_2 tiene el valor especial *null*.)

Salida: **true**, si los árboles son isomorfos
false, si los árboles no son isomorfos

```
procedure bin_tree_isom (r1, r2)
1.  if r1 = null and r2 = null then
2.    return(true)
    // ahora, una o ambas raíces r1 o r2 no es null
3.  if r1 = null and r2 = null then
4.    return(true)
    // ahora, ninguna de las raíces r1 y r2 es null
5.  l c_r1 := hijo izquierdo de r1
6.  l c_r2 := hijo izquierdo de r2
7.  r c_r1 := hijo derecho de r1
8.  r c_r2 := hijo derecho de r2
9.  return(bin_tree_isom (l c_r1, l c_r2)
      and bin_tree_isom(r c_r1, r c_r2)
end bin_tree_isom
```

Como medida del tiempo necesario para el algoritmo 7.8.13, contamos el número de comparaciones con *null* en las líneas 1 y 3. Mostraremos que el algoritmo 7.8.13 es un algoritmo de tiempo lineal en el peor de los casos.

TEOREMA 7.8.14

El tiempo del algoritmo 7.8.13 en el peor de los casos es $\Theta(n)$, donde n es la cantidad total de vértices en los dos árboles.

Demostración. Sea a_n el número de comparaciones con *null* en el peor de los casos del algoritmo 7.8.13, donde n es el número total de vértices en los árboles dados como entrada. Utilizaremos la inducción matemática para demostrar que

$$a_n \leq 3n + 2, \quad \text{para } n \geq 0.$$

PASO BASE ($n = 0$). Si $n = 0$, los árboles dados como entrada al algoritmo 7.8.13 son vacíos. En este caso, existen dos comparaciones con *null* en la línea 1, después de lo cual el procedimiento concluye. Así, $a_0 = 2$ y la desigualdad es válida cuando $n = 0$.

PASO INDUCTIVO. Suponga que

$$a_k \leq 3k + 2,$$

donde $k < n$. Debemos mostrar que

$$a_n \leq 3n + 2.$$

Primero determinamos una cota superior para el número de comparaciones en el peor de los casos, cuando el número total de vértices en los árboles dados como entrada al procedimiento es $n > 0$ y ninguno de los árboles es vacío. En este caso, existen cuatro comparaciones en las líneas 1 y 3. Sea L la suma de los números de los vértices en los dos subárboles izquierdos de los árboles dados como entrada y R la suma de los números de los vértices en los dos subárboles derechos de los árboles dados como entrada. Entonces, en la línea 9, existen a lo más $a_L + a_R$ comparaciones adicionales. Por tanto, en el peor de los casos se necesitan a lo más $4 + a_L + a_R$ comparaciones. Por la hipótesis de inducción,

$$a_L \leq 3L + 2 \quad \text{y} \quad a_R \leq 3R + 2. \tag{7.8.1}$$

Ahora,

$$2 + L + R = n \tag{7.8.2}$$

pues los vértices son precisamente las dos raíces, los vértices en los subárboles izquierdos y los vértices en los subárboles derechos. Al combinar (7.8.1) y (7.8.2), obtenemos

$$4 + a_L + a_R \leq 4 + (3L + 2) + (3R + 2) = 3(2 + L + R) + 2 = 3n + 2.$$

Si alguno de los árboles es vacío, se necesitan cuatro comparaciones en las líneas 1 y 3, después de lo cual concluye el procedimiento. Así, sin importar si los árboles son vacíos o no, se necesitan a lo más $3n + 2$ comparaciones en el peor de los casos. Por tanto,

$$a_n \leq 3n + 2$$

y esto concluye el paso inductivo. Concluimos que el tiempo necesario en el peor de los casos para el algoritmo 7.8.13 es $O(n)$.

Si n es par, digamos, $n = 2k$, se puede utilizar inducción para mostrar (véase el ejercicio 24) que cuando se introducen como dato del algoritmo 7.8.13 dos árboles binarios isomorfos con k vértices, el número de comparaciones es igual a $3n + 2$. Con este resultado se puede mostrar (véase el ejercicio 25) que si n es impar, digamos $n = 2k + 1$, cuando se introducen como dato del algoritmo 7.8.13 los dos árboles binarios de la figura 7.8.15, el número de comparaciones es igual a $3n + 1$. Así, el tiempo del algoritmo 7.8.13 en el peor de los casos es $\Omega(n)$.

Como el tiempo en el peor de los casos es $O(n)$ y $\Omega(n)$, el tiempo en el peor de los casos del algoritmo 7.8.13 es $\Theta(n)$. ■

[Aho] proporciona un algoritmo para determinar si dos árboles con raíz (no necesariamente binarios) son isomorfos, cuyo tiempo en el peor de los casos es lineal con respecto del número de vértices.

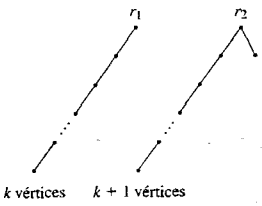
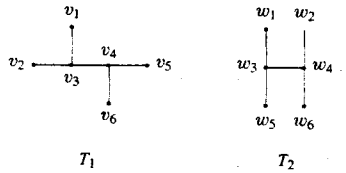


FIGURA 7.8.15
Dos árboles binarios que proporcionan el tiempo de ejecución en el peor de los casos $3n + 1$ para el algoritmo 7.8.13 cuando $n = 2k + 1$ es impar.

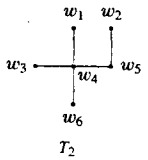
Ejercicios

En los ejercicios 1-6, determine si cada par de árboles libres son isomorfos. Si los árboles son isomorfos, especifique un isomorfismo. Si no son isomorfos, mencione un invariante que tenga uno de los árboles y el otro no.

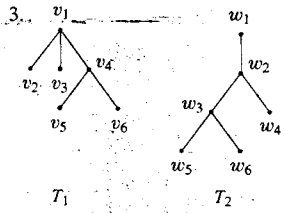
1.



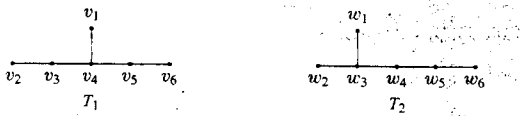
2. T_1 como en el ejercicio 1.



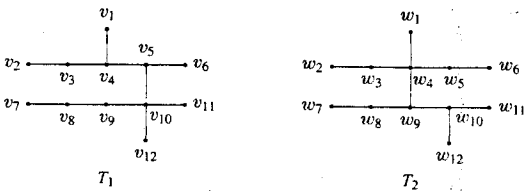
3.



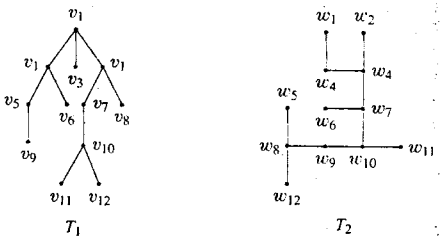
4.



5.

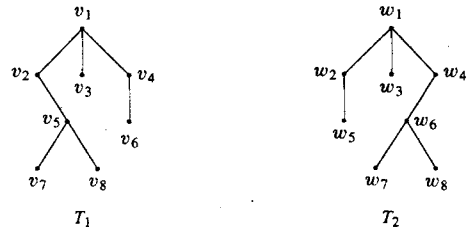


6.



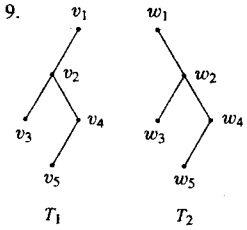
En los ejercicios 7-9, determine si cada par de árboles con raíz son isomorfos. Si los árboles son isomorfos, especifique un isomorfismo. Si no son isomorfos, mencione un invariante que tenga uno de los árboles y el otro no. Además, determine si los árboles son isomorfos como árboles libres.

7.



8. T_1 y T_2 como en el ejercicio 3

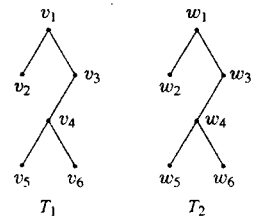
9.



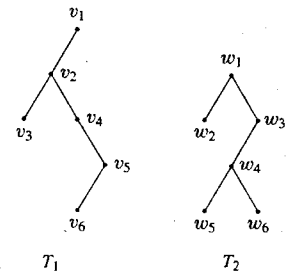
En los ejercicios 10-12, determine si cada par de árboles binarios son isomorfos. Si los árboles son isomorfos, especifique un isomorfismo. Si no son isomorfos, mencione un invariante que tenga uno de los árboles y el otro no. Además, determine si los árboles son isomorfos como árboles libres o como árboles con raíz.

10. T_1 y T_2 como en el ejercicio 9

11.



12.



13. Trace todos los árboles libres no isomorfos con tres vértices.

14. Trace todos los árboles libres no isomorfos con cuatro vértices.

15. Trace todos los árboles libres no isomorfos con seis vértices.

16. Trace todos los árboles con raíz no isomorfos con tres vértices.

17. Trace todos los árboles con raíz no isomorfos con cinco vértices.

18. Trace todos los árboles binarios no isomorfos con dos vértices.

19. Trace todos los árboles binarios no isomorfos con cuatro vértices.

20. Trace todos los árboles binarios completos no isomorfos con siete vértices. (Un árbol binario completo es un árbol binario en donde cada vértice interno tiene dos hijos.)

21. Trace todos los árboles binarios completos no isomorfos con nueve vértices.
22. Determine una fórmula para el número de árboles binarios completos no isomorfos con n vértices.
23. Determine todos los árboles de expansión (como árboles libres y no como árboles con raíz) para cada una de las gráficas de los ejercicios 7-9, sección 7.3.
24. Utilice inducción para mostrar que cuando se introducen como dato dos árboles binarios isomorfos con k vértices en el algoritmo 7.8.13, el número de comparaciones con *null* es igual a $6k + 2$.
25. Muestre que cuando se introducen como dato los dos árboles binarios de la figura 7.8.15 en el algoritmo 7.8.13, el número de comparaciones con *null* es igual a $6k + 4$.
26. Escriba un algoritmo para generar un árbol binario arbitrario con n vértices.
27. [*Proyecto*] Realice un informe acerca de las fórmulas para el número de árboles libres no isomorfos y para el número de árboles con raíz no isomorfos, con n vértices (véase [Deo]).

†7.9 ÁRBOLES DE JUEGOS

Los árboles son útiles en el análisis de juegos como el gato, el ajedrez y las damas, donde los jugadores alternan sus movimientos. En esta sección mostraremos la forma de utilizar los árboles para desarrollar estrategias para ganar juegos. Este tipo de método se utiliza para desarrollar muchos programas de computadora que permiten a los humanos jugar contra las computadoras, o incluso computadoras contra computadoras.

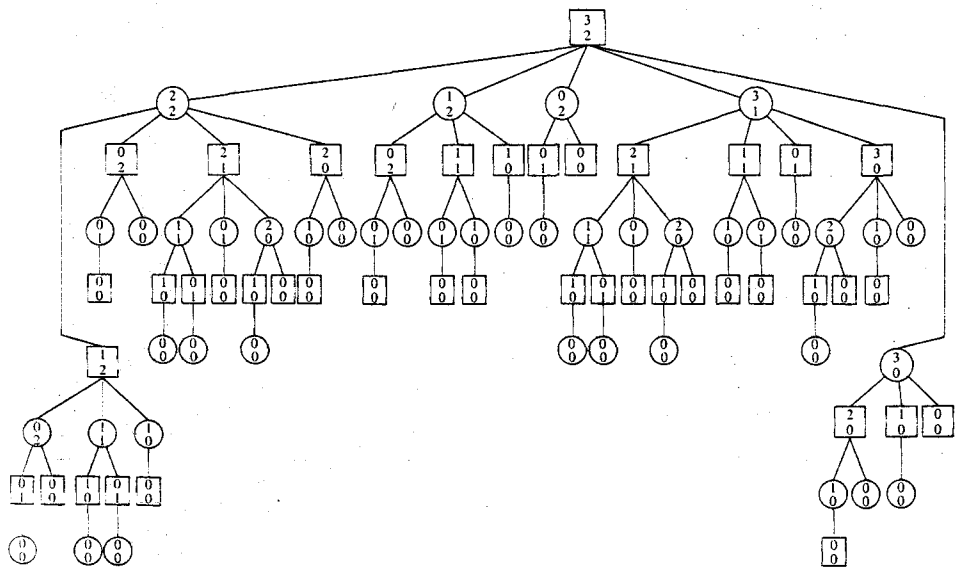


FIGURA 7.9.1 Un árbol de juego para nim. La distribución inicial es dos pilas de tres y dos fichas, respectivamente.

† Esta sección se puede omitir sin pérdida de continuidad.

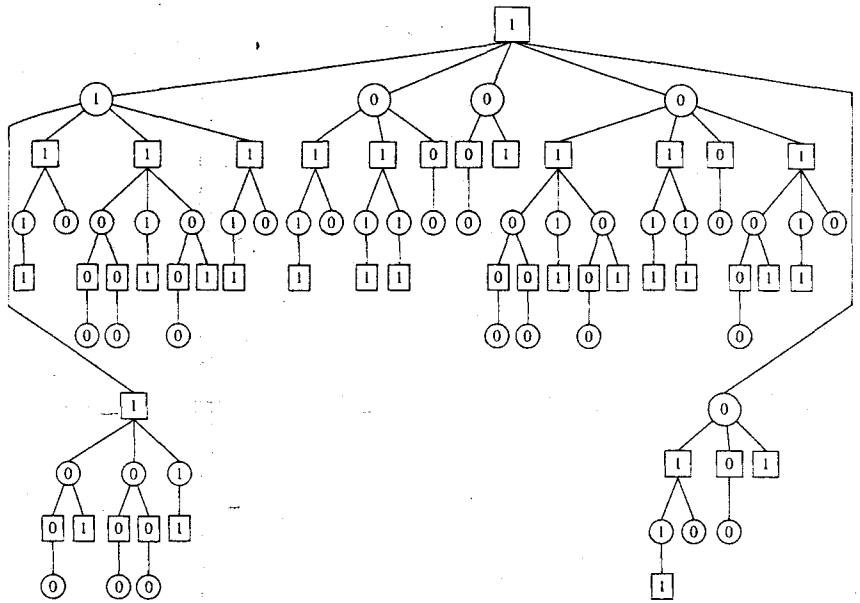


FIGURA 7.9.2 El árbol del juego de la figura 7.9.1 con los valores de todos los vértices.

Como ejemplo del punto de vista general, consideremos una versión del juego de nim. En un principio, existen n pilas, cada una de las cuales tiene un cierto número de fichas idénticas. Los jugadores alternan sus movimientos. Un movimiento consiste en retirar una o más fichas de cualquiera de las pilas. El jugador que quita la última ficha pierde. Como caso particular, considere una distribución inicial con dos pilas: una con tres fichas y la otra con dos. Todas las sucesiones de movimientos posibles se pueden enumerar en un **árbol del juego** (véase la figura 7.9.1). El primer jugador se representa mediante un cuadro y el segundo mediante un círculo. Cada vértice muestra una posición particular del juego. En nuestro juego, la posición inicial aparece como $\binom{3}{2}$. Un camino representa una serie de movimientos. Si una posición aparece dentro de un cuadro, es el movimiento del primer jugador; si una posición aparece dentro de un círculo, es el movimiento del segundo jugador. Un vértice terminal representa el final del juego. En el juego de nim, si el vértice terminal es un círculo, el primer jugador quitó la última ficha y pierde el juego. Si el vértice terminal es un cuadro, pierde el segundo jugador.

El análisis comienza con los vértices terminales. Etiquetamos cada vértice terminal con el valor de la posición para el primer jugador. Si el vértice terminal es un círculo, como ha perdido el primer jugador, esta posición no es importante para el primer jugador y le asignamos el valor 0 (véase la figura 7.9.2). Si el vértice terminal es un cuadro, como ha ganado el primer jugador, esta posición es valiosa para el primer jugador y la etiquetamos con un valor mayor que 0, digamos, 1 (véase la figura 7.9.2). En este momento, todos los vértices terminales tienen asignado un valor.

Ahora, consideremos el problema de asignar valores a los vértices internos. Supongamos, por ejemplo, que tenemos un cuadro interno, tal que todos sus hijos tienen asignado un valor. Por ejemplo, si tenemos la situación que aparece en la figura 7.9.3, el primer jugador (el cuadro) debe pasar a la posición representada por el vértice B, pues esta posición es la más valiosa. En otras palabras, el cuadro se mueve a una posición representada por un hijo con el valor máximo. Asignamos este valor máximo al vértice del cuadro.

Consideremos la situación desde el punto de vista del segundo jugador (círculo). Supongamos que tenemos la situación de la figura 7.9.4. El círculo se debe mover a la posición representada por el vértice C, pues esta posición es la menos valiosa para el cuadro y por tanto la más valiosa para el círculo. En otras palabras, el círculo se mueve a una posición representada por un hijo con el valor mínimo. Asignamos este valor mínimo al vértice del círculo. El proceso mediante el cual el círculo busca el mínimo de sus hijos y el cuadro busca el máximo de sus hijos es el **procedimiento minimax**.

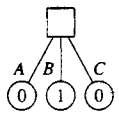


FIGURA 7.9.3 El primer jugador (cuadro) se debe mover a la posición B pues es más valiosa. Este valor máximo (1) se asigna al cuadro.

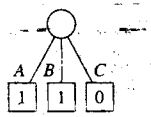


FIGURA 7.9.4 El segundo jugador (círculo) se debe mover a la posición C pues es menos valiosa (para el cuadro). Este valor mínimo (0) se asigna al círculo.

Al trabajar de abajo hacia arriba, partiendo de los vértices terminales y utilizando el procedimiento minimax, podemos asignar valores a todos los vértices del árbol del juego (véase la figura 7.9.2). Estos números representan el valor del juego, en cualquier posición, para el primer jugador. Observe que la raíz de la figura 7.9.2, la cual representa la posición original, tiene un valor de 1. Esto significa que el primer jugador siempre puede ganar, si utiliza una estrategia óptima. Esta estrategia óptima está contenida en el árbol del juego: El primer jugador siempre se mueve a una posición que maximiza el valor de los hijos. Sin importar lo que haga el segundo jugador, el primer jugador siempre puede pasar a un vértice con el valor 1. Al final, se llega a un vértice terminal con valor 1 si el primer jugador gana.

Muchos juegos interesantes, como el ajedrez, tienen árboles de juego tan grandes que no es factible utilizar una computadora para generar todo el árbol. Sin embargo, el concepto de árbol del juego sigue siendo útil para analizar tales juegos.

Al utilizar un árbol de juego, debemos utilizar una búsqueda a profundidad. Si el árbol del juego es tan grande que no sea factible llegar a un vértice terminal, limitamos el nivel hasta el cual se puede realizar la búsqueda a profundidad. La búsqueda es una **búsqueda de n niveles** si limitamos la búsqueda a n niveles debajo del vértice dado. Como los vértices del nivel más bajo podrían no ser vértices terminales, se necesita un método para asignarles un valor. Aquí hay que analizar los detalles específicos de cada juego. Se construye una **función de evaluación** E, la cual asigna a cada posición posible del juego P el valor $E(P)$ de la posición para el primer jugador. Después de asignar valores a los vértices del nivel más bajo mediante la función E, podemos aplicar el procedimiento minimax para generar los valores de los demás vértices. Ilustraremos estos conceptos mediante un ejemplo.

EJEMPLO 7.9.1

Aplicar el procedimiento minimax para determinar el valor de la raíz en el juego de gato, utilizando una búsqueda minimax a profundidad de dos niveles. Utilizar la función de evaluación E, la cual asigna a una posición el valor

$$NX - NO$$

donde NX (respectivamente, NO) es el número de renglones, columnas o diagonales que contienen una X (respectivamente, O) y que X (respectivamente, O) podría completar. Por ejemplo, la posición P de la figura 7.9.5 tiene $NX = 2$, pues X podría completar la columna o la diagonal, y $NO = 1$, pues O sólo puede completar una columna. Por tanto,

$$E(P) = 2 - 1 = 1.$$

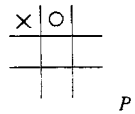


FIGURA 7.9.5
El valor de la posición P es $E(P) = NX - NO = 2 - 1 = 1$.

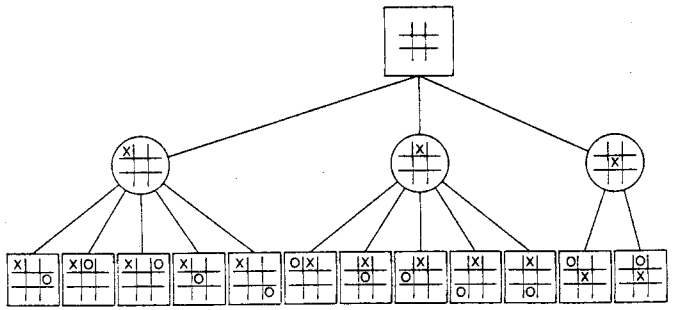


FIGURA 7.9.6 El árbol de juego del gato, hasta el nivel 2, omitiendo las posiciones simétricas.

En la figura 7.9.6 hemos trazado el árbol de juego del gato hasta el nivel 2. Hemos omitido las posiciones simétricas. Primero asignamos a los vértices del nivel 2 los valores dados por E (véase la figura 7.9.7). A continuación, calculamos los valores del círculo minimizando sobre los hijos. Por último, calculamos el valor de la raíz maximizando sobre los hijos. Con este análisis, el primer movimiento del primer jugador debe ser en el cuadro del centro. □

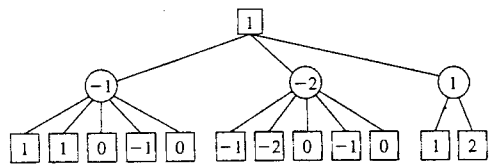


FIGURA 7.9.7 El árbol de juego de la figura 7.9.6 mostrando los valores de todos los vértices.

La evaluación de un árbol de juego, o incluso de una parte de un árbol de juego, puede consumir mucho tiempo, de modo que cualquier técnica que reduzca este esfuerzo es bienvenida. La técnica más general es la **poda alfa-beta**. En general, la poda alfa-beta permite evitar muchos vértices de un árbol de juego, y aun así determinar el valor de un vértice. El valor obtenido es igual al que se obtendría evaluando todos los vértices.

Por ejemplo, consideremos el árbol de juego de la figura 7.9.8. Suponga que queremos evaluar el vértice A mediante una búsqueda a profundidad de dos niveles. Evaluamos los hijos de izquierda a derecha. Comenzamos en la parte inferior izquierda, evaluando los vértices E, F y G. Los valores mostrados se obtienen a partir de una función de evaluación. El vértice B es 2, el mínimo de sus hijos. En este momento, sabemos que el valor x de A debe ser al menos 2, pues el valor de A es el máximo de sus hijos; es decir,

$$x \geq 2. \tag{7.9.1}$$

Esta cota inferior para A es el **valor alfa** de A. Los siguientes vértices por evaluar son H, I y J. Al evaluar I como 1, sabemos que el valor y de C no puede exceder a 1, ya que el valor de C es el mínimo de sus hijos; es decir,

$$y \leq 1. \tag{7.9.2}$$

(7.9.1) y (7.9.2) implican que, sin importar el valor de y, éste no afectará el valor de x; así, ya no tenemos que preocuparnos por el subárbol con raíz en el vértice C. Decimos que ocurre un **corte alfa**. A continuación evaluamos los hijos de D y el propio D. Por último, vemos que el valor de A es 3.

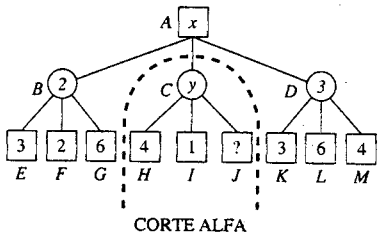


FIGURA 7.9.8 Evaluación del vértice A mediante una búsqueda a profundidad de dos niveles con un corte alfa. Un corte alfa aparece en el vértice C al evaluar el vértice I, pues el valor de I (1) es menor o igual que la estimación de la cota inferior (2) del vértice A.

En resumen, un corte alfa aparece en un vértice de cuadro v cuando un nieto w de v tiene un valor menor o igual al valor alfa de v. El subárbol cuya raíz es el padre de w se puede eliminar (podar). Esta eliminación no afectará el valor de v. Un valor alfa de un vértice v es sólo una cota inferior para el valor de v. El valor alfa de un vértice v depende del estado actual de la búsqueda y cambia al continuar ésta.

De manera análoga, un **corte beta** ocurre en un vértice de círculo v cuando un nieto w de v tiene un valor mayor o igual al valor beta de v. El subárbol cuya raíz es el padre de w se puede podar. Esta eliminación no afectará el valor de v. Un **valor beta** para un vértice v es sólo una cota superior para el valor de v. El valor beta de un vértice depende del estado actual de la búsqueda y cambia al continuar ésta.

EJEMPLO 7.9.2

Evaluar la raíz del árbol de la figura 7.9.9 utilizando la búsqueda a profundidad con una poda alfa-beta. Suponga que los hijos se evalúan de izquierda a derecha. Para cada vértice cuyo valor sea calculado, escribir el valor en el vértice. Colocar un signo de verificación en la raíz de cada subárbol podado. El valor de cada vértice terminal se escribe bajo el vértice.

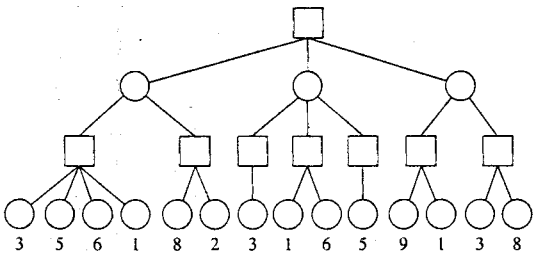


FIGURA 7.9.9 El árbol de juego para el ejemplo 7.9.2.

Primero evaluamos los vértices A, B, C y D (véase la figura 7.9.10). A continuación, vemos que el valor de E es 6. Esto produce un valor beta de 6 para F. A continuación evaluamos el vértice G. Como su valor es 8 y 8 es mayor que el valor beta de F, obtenemos un corte beta y podamos el subárbol con raíz H. El valor de F es 6. Esto produce un valor alfa de 6 para I. A continuación, evaluamos los vértices J y K. Como el valor 3 de K es menor que el valor alfa 6 de I, ocurre un corte alfa y podemos podar el subárbol con raíz L. A continuación evaluamos M, N, O, P, Q, R y S. No se puede podar más. Por último, determinamos que la raíz I tiene el valor 8. □

Se ha mostrado (véase [Pearl]) que para los árboles de juegos tales que cada padre tiene n hijos y donde se han ordenado los vértices terminales de manera aleatoria, para una cantidad dada de tiempo, el procedimiento alfa-beta permite una búsqueda a profundidad $\frac{4}{3}$ mayor que el procedimiento minimax puro, el cual evalúa cada vértice. Pearl también demostró que para tales árboles de juegos, el procedimiento alfa-beta es óptimo.

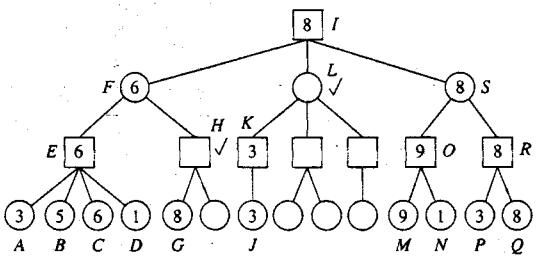


FIGURA 7.9.10 Evaluación de la raíz del árbol de juego de la figura 7.9.9 utilizando una búsqueda a profundidad con poda alfa-beta. Los vértices con un signo de verificación son las raíces de los subárboles podados. Los valores de los vértices evaluados se escriben dentro de los vértices.

Se han combinado otras técnicas con la poda alfa-beta para facilitar la búsqueda en un árbol de juego. Una idea consiste en ordenar los hijos de los vértices por evaluar de modo que se examinen primero los movimientos más promisorios (véanse los ejercicios 23-26). Otra idea consiste en permitir una búsqueda a profundidad variable, en la cual pueda retroceder la búsqueda al llegar a una posición no promisoria, medida mediante alguna función.

Algunos programas para juegos han tenido algo de éxito. Los mejores programas de backgammon y damas juegan a un nivel comparable al de los mejores jugadores humanos. Se escribió una parte de la historia en 1996 cuando el programa de ajedrez de IBM, Deep Blue, ganó el primer juego de un encuentro a seis partidas contra Garry Kasparov, con lo cual se convirtió en el primer programa de ajedrez en vencer a un campeón mundial bajo las restricciones normales de tiempo. (Los programas de computadora habían derrotado anteriormente a los mejores jugadores humanos, Kasparov incluido, en juegos realizados a mayor velocidad.) Kasparov prevaleció finalmente, ganando tres juegos y empatando en otros dos.

Ejercicios

1. Trace el árbol de juego completo para una versión del juego de nim en la cual la posición inicial conste de una pila de seis fichas y un turno consista en quitar una, dos o tres fichas. Asigne valores a todos los vértices, de modo que el árbol resultante sea análogo a la figura 7.9.2. Suponga que el último jugador que quite una ficha pierde. Jugando con una estrategia óptima, ¿quién ganará siempre: el primer jugador o el segundo? Describa una estrategia óptima para el ganador.
2. Trace el árbol de juego completo para el juego de nim en la cual la posición inicial conste de dos pilas con tres fichas cada una. Omita las posiciones simétricas. Suponga que el último jugador que quite una ficha pierde. Asigne valores a todos los vértices, de modo que el árbol resultante sea análogo a la figura 7.9.2. Jugando con una estrategia óptima, ¿quién ganará siempre: el primer jugador o el segundo? Describa una estrategia óptima para el ganador.
3. Trace el árbol de juego completo para el juego de nim en la cual la posición inicial conste de dos pilas, una con tres fichas y la otra con dos. Suponga que el último jugador que quite una ficha gana. Asigne valores a todos los vértices, de modo que el árbol resultante sea análogo a la figura 7.9.2. Jugando con una estrategia óptima, ¿quién ganará siempre: el primer jugador o el segundo? Describa una estrategia óptima para el ganador.
4. Trace el árbol de juego completo para el juego de nim en la cual la posición inicial conste de dos pilas con tres fichas cada una. Omita las posiciones simétricas. Suponga que el último jugador que quite una ficha gana. Asigne valores a todos los vértices, de modo que el árbol resultante sea análogo a la figura 7.9.2. Jugando con una estrategia óptima, ¿quién ganará siempre: el primer jugador o el segundo? Describa una estrategia óptima para el ganador.
5. Trace el árbol de juego completo para la versión del juego de nim descrita en el ejercicio 1. Suponga que el último jugador que quite una ficha gana. Asigne valores a todos los vértices, de modo que el árbol resultante sea análogo a la figura 7.9.2. Jugando con una estrategia óptima, ¿quién ganará siempre: el primer jugador o el segundo? Describa una estrategia óptima para el ganador.
6. Dé un ejemplo de un árbol de juego completo (posiblemente hipotético) en el cual un vértice terminal sea 1 si gana el primer jugador y 0 si pierde el primer jugador, con las siguientes propiedades: Existen más ceros que unos entre los vértices terminales, pero el primer jugador siempre puede ganar jugando una estrategia óptima.

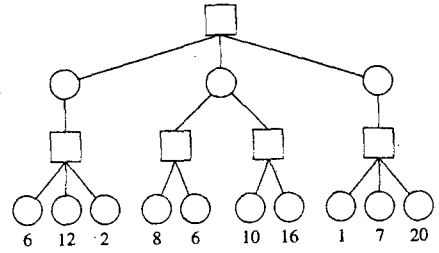
Los ejercicios 7 y 8 se refieren a nim y nim′. Nim es el juego que utiliza n pilas de fichas, descrito en esta sección, en la cual el último jugador que realice un movimiento pierde. Nim′ es el juego que utiliza n pilas de fichas, descrito en esta sección, excepto que el último jugador que realice un movimiento gana. Fijemos n pilas, cada una con un número fijo de dichas. Suponemos que al menos una pila tiene al menos dos fichas.

★ 7. Muestre que el primer jugador siempre puede ganar en el juego de nim si y sólo si el primer jugador siempre puede ganar el juego de nim′.

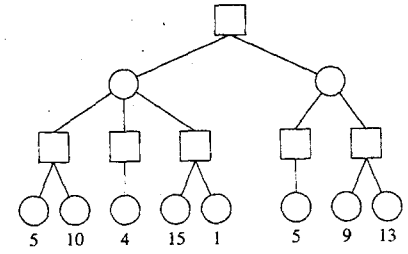
☆ 8. Dada una estrategia ganadora para un jugador particular para el juego de nim, describa una estrategia ganadora para este jugador, para nim′.

Evalúe cada vértice de cada árbol de juego. Se proporcionan los valores de los vértices terminales.

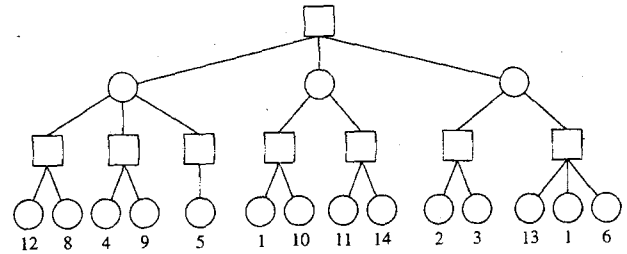
9.



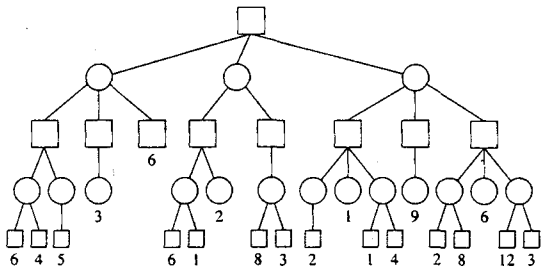
10.



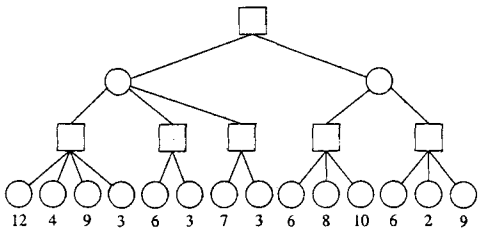
11.



12.



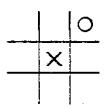
13.



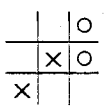
14. Evalúe la raíz de cada uno de los árboles de los ejercicios 9-13 mediante una búsqueda a profundidad con poda alfa-beta. Suponga que los hijos se evalúan de izquierda a derecha. Para cada vértice cuyo valor sea calculado, escriba el valor en el vértice. Coloque un signo de verificación en la raíz de cada subárbol podado. El valor de cada vértice terminal se escribe bajo el vértice.

En los ejercicios 15-18, determine el valor de la posición del gato mediante la función de evaluación del ejemplo 7.9.1.

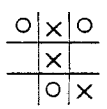
15.



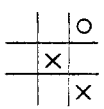
16.



17.



18.



19. Suponga que el primer jugador se mueve al cuadro central en el gato. Trace un árbol de juego de dos niveles, de modo que la raíz tenga una X en el cuadro central. Omita las posiciones simétricas. Evalúe todos los vértices mediante la función de evaluación del ejemplo 7.9.1. ¿Adónde se moverá O?

20. ¿Haría un juego perfecto de gato un programa de búsqueda de dos niveles basado en la función de evaluación E del ejemplo 7.9.1? En caso contrario, ¿podría modificar E de modo que un programa de búsqueda de dos niveles haga un juego perfecto de gato?

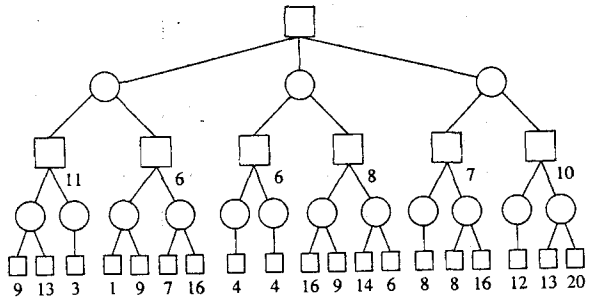
21. Escriba un algoritmo que evalúe los vértices de un árbol de juego hasta el nivel n mediante una búsqueda a profundidad. Suponga la existencia de una función de evaluación E.

22. Escriba un algoritmo que evalúe la raíz de un árbol de juego mediante una búsqueda a profundidad de n niveles con poda alfa-beta. Suponga la existencia de una función de evaluación E.

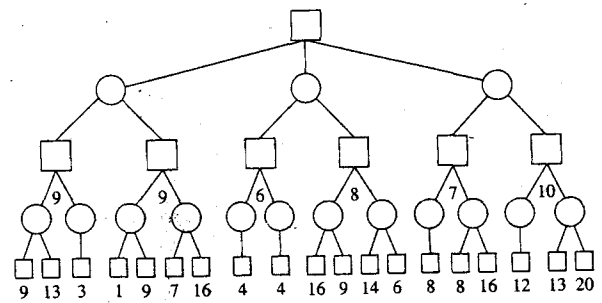
El siguiente método conduce con frecuencia a un mayor número de podas que el procedimiento minimax alfa-beta puro. Primero se realiza una búsqueda de dos niveles, evaluando los hijos de izquierda a derecha. En este momento, todos los hijos de la raíz tienen valores. A continuación, se ordenan los hijos de la raíz, con los movimientos más promisorios a la izquierda. Luego se utiliza una búsqueda a profundidad de n niveles con poda alfa-beta. Se evalúan los hijos de izquierda a derecha.

Realice este procedimiento para $n = 4$ para cada árbol de juego de los ejercicios 23-25. Coloque un signo de verificación en la raíz de cada subárbol podado. El valor de cada vértice, dado por la función de evaluación, está dado debajo del vértice.

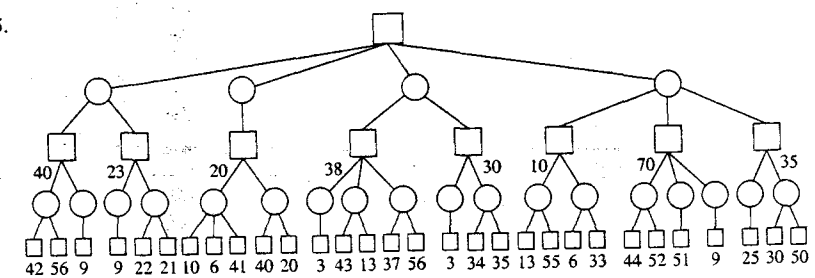
23.



24.

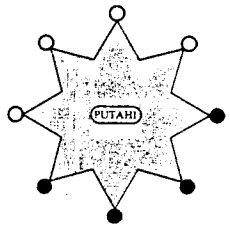


25.



26. Escriba un algoritmo para realizar el procedimiento descrito antes del ejercicio 23.

Mu Torere es un juego de dos personas jugado por los maoríes (véase [Bell]). El tablero es una estrella de ocho puntas (véase la siguiente figura) con un área circular en el centro conocida como *putahi*. El primer jugador tiene cuatro fichas negras y el segundo jugador tie-



ne cuatro fichas blancas. La posición inicial aparece en la figura. Un jugador que no pueda realizar un movimiento pierde. Los jugadores alternan sus movimientos. A lo más una ficha puede ocupar un punto de la estrella o del putahi. Un movimiento es:

(a) Un movimiento a un punto adyacente
(b) Un movimiento desde el putahi a un punto
(c) Un movimiento desde el punto al putahi, siempre que uno o ambos puntos adyacentes contengan las piezas del adversario

☆ 27. Desarrolle una función de evaluación para Mu Torere.
☆ 28. Combine la función de evaluación del ejercicio 27 con una búsqueda de dos niveles del árbol de juego para obtener un algoritmo para jugar Mu Torere. Evalúe la capacidad de juego de este algoritmo.
☆ 29. ¿Podría ganar siempre el primer jugador en Mu Torere?
☆ 30. ¿Podría empatar siempre el primer jugador en Mu Torere?
31. [*Proyecto*] Según [Nilsson], el árbol de juego completo para el ajedrez tiene más de 10^{100} vértices. Realice un informe acerca de la forma en que se obtuvo esta estimación.
☆ 32. [*Proyecto*] Desarrolle una función de evaluación para Kalah. (Véanse los detalles en [Ainslie].)
☆ 33. Desarrolle un algoritmo para jugar Kalah con base en la función de evaluación del ejercicio 32. Evalúe la capacidad de juego de este algoritmo.

NOTAS

Se recomienda la siguiente bibliografía para el tema de los árboles: [Berge; Bondy; Deo; Even, 1979; Gibbons; Harary; Knuth, 1973, Vol. 1; Liu, 1985; y Ore].

Véase [Date] para el uso de los árboles en las bases de datos jerárquicas.

[Cormen] tiene información adicional acerca de los códigos de Huffman y una demostración de que el algoritmo 7.1.8 construye un árbol de Huffman óptimo.

[Golomb, 1965] describe el retroceso y contiene varios ejemplos y aplicaciones.

Los algoritmos para los árboles de expansión mínimos y su implantación aparecen en [Tarjan].

[Baase] analiza el tiempo mínimo para el ordenamiento, así como cotas inferiores para otros problemas.

Los algoritmos de ordenamiento clásicos se estudian ampliamente en [Knuth, 1973, Vol. 3]. Véanse [Akl; Baase; Leighton; Lester; Lewis; y Quinn] para el ordenamiento mediante máquinas paralelas.

Algunas buenas referencias para los árboles de juegos son [Nievergelt; Nilsson; y Slagle]. En [Frey], el procedimiento minimax se aplica a un juego sencillo. Se analizan y comparan varios métodos para acelerar la búsqueda en el árbol de juego. Se proporcionan programas de computadora. [Berlekamp] contiene una teoría general de los juegos, así como el análisis de muchos juegos particulares.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 7.1

Árbol libre
Árbol con raíz
Nivel de un vértice en un árbol con raíz
Altura de un árbol con raíz
Árbol de definición jerárquica
Código de Huffman

Sección 7.2

Padre
Ancestro
Hijo
Descendiente
Hermano
Vértice terminal
Vértice interno
Subárbol
Gráfica acíclica
Caracterizaciones alternativas de los árboles (Teorema 7.2.3)

Sección 7.3

Árbol de expansión
Una gráfica tiene un árbol de expansión si y sólo si es conexa
Búsqueda a lo ancho
Búsqueda a profundidad
Retroceso

Sección 7.4

Árbol de expansión mínimo
Algoritmo de Prim para determinar un árbol de expansión mínimo
Algoritmo codicioso

Sección 7.5

Árbol binario
Hijo izquierdo en un árbol binario
Hijo derecho en un árbol binario
Árbol binario completo
Si T es un árbol binario completo con i vértices internos, entonces T tiene $i + 1$ vértices terminales y $2i + 1$ vértices en total.
Si un árbol binario de altura h tiene t vértices terminales, entonces $\lg t \leq h$.
Árbol de búsqueda binaria
Algoritmo para construir un árbol de búsqueda binaria

Sección 7.6

Recorrido en preorden
Recorrido en entreorden
Recorrido en posorden
Forma prefija de una expresión (notación polaca)
Forma entrefija de una expresión
Forma posfija de una expresión (notación polaca inversa)
Representación de una expresión mediante un árbol

Sección 7.7

Árbol de decisión
La altura de un árbol de decisión que representa a un algoritmo es proporcional al tiempo del algoritmo en el peor de los casos.
Cualquier algoritmo de ordenamiento necesita al menos $\Omega(n \lg n)$ comparaciones para ordenar n elementos en el peor de los casos.

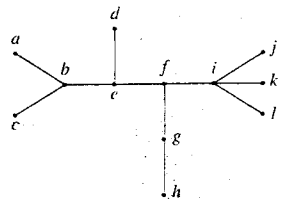
Sección 7.8

Árboles libres isomorfos
Árboles con raíz isomorfos
Árboles binarios isomorfos
El número de Catalan $C(2n, n)/(n + 1)$ es igual al número de árboles binarios con n vértices no isomorfos.
Algoritmo de tiempo lineal (algoritmo 7.8.13) para verificar si dos árboles binarios son isomorfos

Sección 7.9

Árbol de juego
Procedimiento minimax
Búsqueda de n niveles
Función de evaluación
Poda alfa-beta
Valor alfa
Corte alfa
Valor beta
Corte beta

AUTOEVALUACIÓN DEL CAPÍTULO



Sección 7.1

1. Trace el árbol libre anexo como un árbol con raíz c.
2. Determine el nivel de cada vértice en el árbol anexo con raíz en c.
3. Determine la altura del árbol anexo con raíz en c.

4. Construya un código de Huffman óptimo para el conjunto de letras de la tabla.

Letra	*Frecuencia*
A	5
B	8
C	5
D	12
E	20
F	10

Sección 7.2

5. Trace el árbol libre del ejercicio 1 como un árbol con raíz *f*. Determine

(a) El padre de *a*.

(b) Los hijos de *b*.

(c) Los vértices terminales.

(d) El subárbol con raíz en *e*.

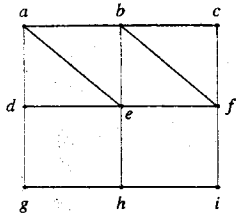
Responda verdadero o falso en los ejercicios 6-8 y explique su respuesta.

6. Si *T* es un árbol con seis vértices, entonces *T* debe tener cinco aristas.

7. Si *T* es un árbol con raíz y seis vértices, entonces la altura de *T* es a lo más 5.

8. Una gráfica acíclica con ocho vértices tiene siete aristas.

Sección 7.3



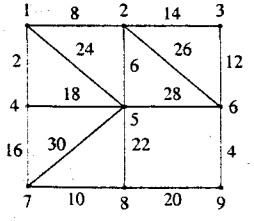
9. Utilice la búsqueda a lo ancho (algoritmo 7.3.6) con el orden de los vértices *eachgbdfi* para determinar un árbol de expansión de la gráfica anexa.

10. Utilice la búsqueda a profundidad (algoritmo 7.3.7) con el orden de los vértices *eachgbdfi* para determinar un árbol de expansión de la gráfica anexa.

11. Utilice la búsqueda a lo ancho (algoritmo 7.3.6) con el orden de los vértices *fdehagbci* para determinar un árbol de expansión de la gráfica anexa.

12. Utilice la búsqueda a profundidad (algoritmo 7.3.7) con el orden de los vértices *fdehagbci* para determinar un árbol de expansión de la gráfica anexa.

Sección 7.4



13. Determine un árbol de expansión mínimo para la gráfica anexa.

14. ¿En qué orden suma las aristas el algoritmo de Prim para la gráfica anexa, si el vértice inicial es 1?

15. ¿En qué orden suma las aristas el algoritmo de Prim para la gráfica adyacente, si el vértice inicial es 6?

16. Dé un ejemplo del uso del método codicioso que no conduzca a un algoritmo óptimo.

Sección 7.5

17. Trace un árbol binario con exactamente dos hijos izquierdos y un hijo derecho.

18. Un árbol binario completo tiene 15 vértices internos. ¿Cuántos vértices terminales tiene?

19. Coloque las palabras

WORD PROCESSING PRODUCES CLEAN MANUSCRIPTS

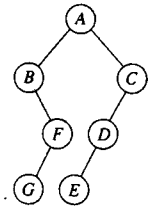
BUT NOT NECESSARILY CLEAR PROSE,

en el orden en que aparecen, en un árbol de búsqueda binaria.

20. Explique la forma en que buscaría MORE en el árbol de búsqueda binaria del ejercicio 19.

Sección 7.6

Los ejercicios 21-23 se refieren al árbol binario anexo.



21. Indique el orden de procesamiento de los vértices al utilizar el recorrido en preorden.

22. Indique el orden de procesamiento de los vértices al utilizar el recorrido en entreorden.

23. Indique el orden de procesamiento de los vértices al utilizar el recorrido en posorden.

24. Represente la expresión prefija $- * E/BD - CA$ como un árbol binario. Además, escriba la forma posfija y la forma entrefija con todos los paréntesis de la expresión.

Sección 7.7

25. Seis monedas tienen apariencia idéntica, pero una de ellas es más pesada o más ligera que las demás, las cuales pesan lo mismo. Demuestre que se necesitan al menos tres pesadas en el peor de los casos para identificar a la moneda mala y determinar si es más pesada o más ligera que las otras utilizando sólo una balanza de platillos.

26. Trace un árbol de decisión que proporcione un algoritmo para resolver el juego de monedas del ejercicio 25 a lo más en tres pesadas en el peor de los casos.

27. El profesor E. Sabic afirma haber descubierto un algoritmo que utiliza a lo más $100n$ comparaciones en el peor de los casos para ordenar n elementos, para toda $n \geq 1$. El algoritmo del profesor compara parejas de elementos y, con base en el resultado de la comparación, modifica la lista original. Proporcione un argumento que muestre que el profesor debe estar equivocado.

28. El algoritmo de *ordenamiento por inserción binaria* ordena un arreglo de tamaño n como sigue. Si $n = 1, 2$ o 3, el algoritmo utiliza un ordenamiento óptimo. Si $n > 3$, el algoritmo ordena $s_1, \ldots, s_n$ de la siguiente manera. Primero se ordenan de manera recursiva $s_1, \ldots, s_{n-1}$. Luego se utiliza la búsqueda binaria para determinar la posición correcta de s_n, después de lo cual se inserta s_n en su posición correcta. Determine el número de comparaciones utilizadas por el ordenamiento por inserción binaria en el peor de los casos, para $n = 4, 5, 6$. ¿Existe algún algoritmo que utilice menos comparaciones para $n = 4, 5, 6$?

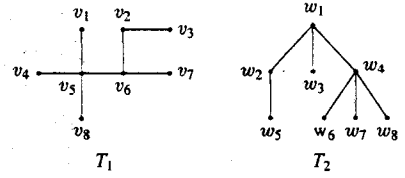
Sección 7.8

Responda verdadero o falso en los ejercicios 29 y 30 y explique sus respuestas.

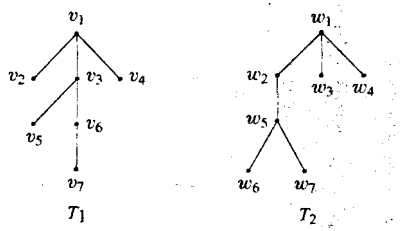
29. Si T_1 y T_2 son isomorfos como árboles con raíz, entonces T_1 y T_2 son isomorfos como árboles libres.

30. Si T_1 y T_2 son isomorfos como árboles libres, entonces T_1 y T_2 son isomorfos como árboles con raíz.

31. Determine si los siguientes árboles libres son isomorfos. Si son isomorfos, proporcione un isomorfismo. Si los árboles no son isomorfos, proporcione un invariante no compartido por los árboles.



32. Determine si los siguientes árboles con raíz son isomorfos. Si los árboles son isomorfos, proporcione un isomorfismo. Si los árboles no son isomorfos, proporcione un invariante no compartido por los árboles.

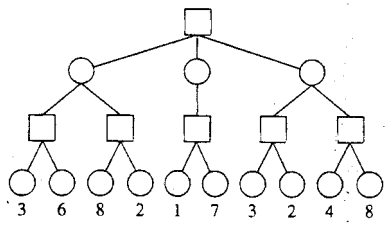


Sección 7.9

33. Determine el valor de la posición del gato mediante la función de evaluación del ejemplo 7.9.1.

34. Proporcione una función de evaluación de una posición del gato distinta a la del ejemplo 7.9.1. Intente una distinción más fina entre las posiciones con respecto de la función de evaluación del ejemplo 7.9.1.

35. Evalúe cada vértice en el siguiente árbol de juego. Los valores de los vértices terminales están dados.



36. Evalúe la raíz del árbol del ejercicio 35 mediante el procedimiento minimax con poda alfa-beta. Suponga que los hijos se evalúan de izquierda a derecha. Para cada vértice cuyo valor sea calculado, escriba el valor en el vértice. Coloque un signo de verificación en la raíz de cada subárbol podado.

8

MODELO DE REDES Y REDES DE PETRI

8.1 MODELOS DE REDES
8.2 UN ALGORITMO DE FLUJO MÁXIMO
8.3 EL TEOREMA DEL FLUJO MÁXIMO Y CORTE MÍNIMO
8.4 ACOPLAMIENTO
RINCÓN DE SOLUCIÓN DE PROBLEMAS: ACOPLAMIENTO
8.5 REDES DE PETRI
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

¡Cásate conmigo, y nunca miraré a otro caballo!

— de A Day at the Races

En este capítulo analizamos dos temas, modelos de redes y redes de Petri, que utilizan las gráficas dirigidas. La mayor parte de este capítulo se dedica al problema de maximizar el flujo que pasa a través de una red. La red podría ser una red de transporte a través de la cual fluyen las mercancías, una red de tuberías a través de las cuales fluye el petróleo, una red de computadoras a través de la cual fluyen los datos, o varias otras posibilidades. En cada caso, el problema consiste en determinar un flujo máximo. Muchos otros problemas, que aparentemente no serían problemas de flujo, pueden modelarse como problemas de flujo en una red.

La maximización del flujo en una red es un problema que pertenece a la teoría de gráficas y a la investigación de operaciones. El problema del agente de ventas viajero es otro ejemplo de un problema en teoría de gráficas e investigación de operaciones. La **investigación de operaciones** estudia la muy amplia categoría de problemas de optimización del desempeño de un sistema. Algunos problemas típicos de la investigación de operaciones son los relativos a redes, asignación de recursos y a asignación de personal.

Las redes de Petri modelan sistemas en los que el procesamiento puede ocurrir de manera concurrente. El modelo proporciona un marco de referencia para tratar cuestiones como la posible aparición de estancamientos en un sistema o el hecho de exceder la capacidad de los componentes de un sistema.

8.1 MODELOS DE REDES

Consideremos la gráfica dirigida de la figura 8.1.1, la cual representa una red de tuberías de petróleo. El petróleo se descarga en el muelle a y se bombea a través de la red hasta la refinería z. Los vértices b, c, d y e representan estaciones de bombeo intermedias. Las aristas dirigidas representan subtuberías del sistema y muestran la dirección en que puede fluir el petróleo. Las etiquetas sobre las aristas muestran las capacidades de las subtuberías. El problema consiste en determinar una forma de maximizar el flujo del muelle a la refinería y calcular el valor de este flujo máximo. La figura 8.1.1 es un ejemplo de **red de transporte**.

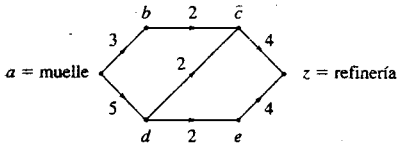


FIGURA 8.1.1 Una red de transporte.

DEFINICION 8.1.1

Una *red de transporte* (o más simple, una *red*) es una gráfica dirigida, simple, con pesos, que satisface:

(a) Un vértice fijo, la *fuente*, no tiene aristas de entrada.

(b) Un vértice fijo, el *sumidero* (o *destino*), no tiene aristas de salida.

(c) El peso C_{ij} de la arista dirigida (i, j), llamado la *capacidad* de (i, j), es un número no negativo.

EJEMPLO 8.1.2

La gráfica de la figura 8.1.1 es una red de transporte. La fuente es el vértice a y el sumidero es el vértice z. La capacidad de la arista (a, b), C_{ab}, es 3, y la capacidad de la arista (b, c), C_{bc}, es 2. □

En este capítulo, si G es una red, denotaremos la fuente por a y el sumidero por z.

Un **flujo** en una red asigna un flujo en cada arista dirigida que no excede la capacidad de dicha arista. Además, se supone que el flujo de entrada a un vértice v, que no sea la fuente ni el sumidero, es igual al flujo de salida de v. La siguiente definición precisa estas ideas.

DEFINICION 8.1.3

Sea G una red de transporte. Sea C_{ij} la capacidad de la arista dirigida (i, j). Un *flujo* F en G asigna a cada arista dirigida (i, j) un número no negativo F_{ij} tal que:

(a) $F_{ij} \leq C_{ij}$.

(b) Para cada vértice j, que no sea la fuente ni el sumidero,

$$\sum_i F_{ij} = \sum_i F_{ji}, \tag{8.1.1}$$

[En una suma como (8.1.1), a menos que se indique lo contrario, se supone que la suma se realiza sobre todos los vértices i. Además, si (i, j) no es una arista, hacemos $F_{ij} = 0$.]

F_{ij} es el *flujo en la arista* (i, j). Para cualquier vértice j,

$$\sum_i F_{ij}$$

es el *flujo de entrada a* j, y

$$\sum_i F_{ji}$$

es el *flujo de salida de* j.

La propiedad expresada por la ecuación (8.1.1) es la **conservación del flujo**. En el ejemplo del bombeo de petróleo de la figura 8.1.1, la conservación del flujo indica que el petróleo no se utiliza ni se suministra en las estaciones de bombeo b, c, d y e.

EJEMPLO 8.1.4

Las asignaciones

$$F_{ab} = 2, \quad F_{bc} = 2, \quad F_{cz} = 3, \quad F_{ad} = 3,$$
$$F_{dc} = 1, \quad F_{de} = 2, \quad F_{ez} = 2,$$

definen un flujo para la red de la figura 8.1.1. Por ejemplo, el flujo de entrada del vértice d,

$$F_{ad} = 3,$$

es igual al flujo de salida del vértice d,

$$F_{dc} + F_{de} = 1 + 2 = 3. \qquad \square$$

En la figura 8.1.2 hemos trazado de nuevo la red de la figura 8.1.1 para mostrar el flujo del ejemplo 8.1.4. Una arista e se etiqueta "x, y" si la capacidad de e es x y el flujo en e es y. Esta notación se utilizará en todo el capítulo.

Observe que en el ejemplo 8.1.4, el flujo de salida de la fuente a,

$$F_{ab} + F_{ad},$$

es igual al flujo de entrada del sumidero z,

$$F_{cz} + F_{ez},$$

ambos iguales a 5. El siguiente teorema demuestra que el flujo de salida de la fuente siempre es igual al flujo de entrada del sumidero.

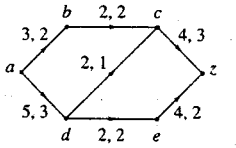


FIGURA 8.1.2
Flujo en una red. Las aristas se etiquetan con x, y para indicar la capacidad x y el flujo y.

TEOREMA 8.1.5

Dado un flujo F en una red, el flujo de salida de la fuente a es igual al flujo de entrada del sumidero z; es decir,

$$\sum_i F_{ai} = \sum_i F_{iz}.$$

Demostración. Sea V el conjunto de vértices. Tenemos

$$\sum_{j\in V}\left(\sum_{i\in V} F_{ij}\right) = \sum_{j\in V}\left(\sum_{i\in V} F_{ji}\right),$$

pues cada suma doble es

$$\sum_{e\in E} F_e,$$

donde E es el conjunto de aristas. Ahora,

$$\begin{aligned} 0 &= \sum_{j\in V}\left(\sum_{i\in V} F_{ij} - \sum_{i\in V} F_{ji}\right) \\ &= \left(\sum_{i\in V} F_{iz} - \sum_{i\in V} F_{zi}\right) + \left(\sum_{i\in V} F_{ia} - \sum_{i\in V} F_{ai}\right) \\ &\quad + \sum_{\substack{j\in V \\ j\neq a,z}}\left(\sum_{i\in V} F_{ij} - \sum_{i\in V} F_{ji}\right) \\ &= \sum_{i\in V} F_{iz} - \sum_{i\in V} F_{ai} \end{aligned}$$

pues $F_{zi} = 0 = F_{ia}$, para toda $i \in V$, y (definición 8.1.3b)

$$\sum_{i\in V} F_{ij} - \sum_{i\in V} F_{ji} = 0 \qquad \text{si } j \in V - \{a, z\}.$$

■

En vista del teorema 8.1.5, podemos enunciar la siguiente definición.

DEFINICIÓN 8.1.6

Sea F un flujo en una red G. El valor

$$\sum_i F_{ai} = \sum_i F_{iz}$$

es el *valor del flujo F.*

EJEMPLO 8.1.7

El valor del flujo en la red de la figura 8.1.2 es 5. □

El problema de una red de transporte G se puede establecer así: Determinar un flujo máximo en G; es decir, entre todos los flujos posibles en G, determinar un flujo F tal que el valor de F sea máximo. En la siguiente sección daremos un algoritmo que resuelve este problema de manera eficiente. Concluimos esta sección con algunos ejemplos más.

EJEMPLO 8.1.8 *Una red de bombeo*

La figura 8.1.3 representa una red de bombeo por medio de la cual se envía agua a dos ciudades, A y B, desde tres pozos, w_1, w_2 y w_3. Las capacidades de los sistemas intermedios aparecen sobre las aristas. Los vértices b, c y d representan estaciones de bombeo intermedias. Modelar este sistema como una red de transporte.

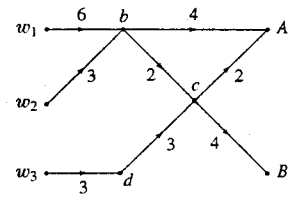


FIGURA 8.1.3 Una red de bombeo. El agua de las ciudades A y B se obtiene de los pozos w_1, w_2 y w_3. Las capacidades aparecen sobre las aristas.

Para obtener una fuente y un sumidero fijos, podemos obtener una red de transporte equivalente, uniendo las fuentes en una **superfuente** y los sumideros en un **supersumidero** (véase la figura 8.1.4). En la figura 8.1.4, ∞ representa una capacidad ilimitada.

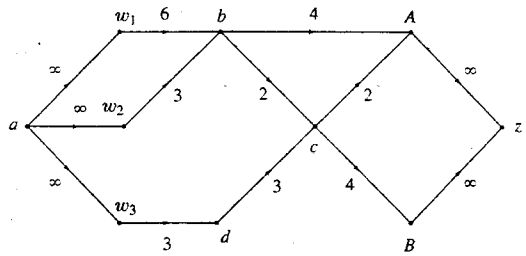


FIGURA 8.1.4 La red de la figura 8.1.3 con una fuente y un sumidero fijos. □

EJEMPLO 8.1.9 *Una red de flujo de tráfico*

Es posible ir de la ciudad A a la ciudad C directamente o pasar por la ciudad B. Durante el periodo de 6:00 a 7:00 p.m. los tiempos promedio de viaje son

A a B (15 minutos)

B a C (30 minutos)

A a C (30 minutos).

Las capacidades máximas de las carreteras son

A a B (3000 vehículos)

B a C (2000 vehículos)

A a C (4000 vehículos).

Representar el flujo de tráfico de A a C durante el periodo de 6:00 a 7:00 p. m. como una red.

Un vértice representa una ciudad en un instante dado (véase la figura 8.1.5). Una arista conecta X, t_1 con Y, t_2 si podemos salir de la ciudad X en el instante t_1 p.m. y llegar a la ciudad Y en el instante t_2 p.m. La capacidad de una arista es la capacidad de la ruta. Existen aristas de capacidad infinita que conectan a A, t_1 con A, t_2 y B, t_1 con B, t_2 para indicar que una cantidad arbitraria de autos puede esperar en la ciudad A o la ciudad B. Por último, introducimos una superfuente y un supersumidero. □

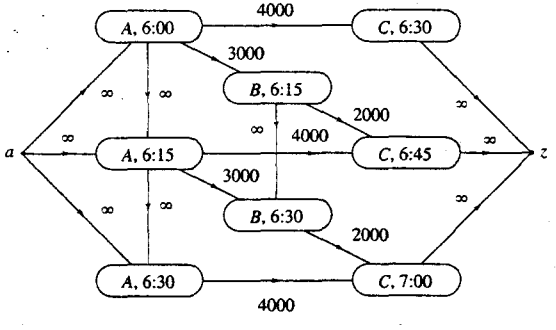


FIGURA 8.1.5 Una red que representa el flujo de tráfico de la ciudad A a la ciudad C durante el periodo de 6:00 a 7:00 p.m.

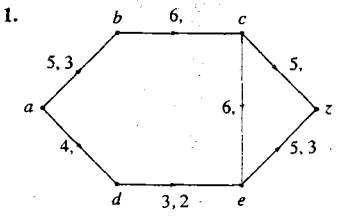
Algunas variantes del problema de flujo de redes se han utilizado en el diseño de redes de computadoras eficientes (véase [Jones; Kleinrock]). Al modelar una red de computadoras, un vértice es un mensaje o un centro de intercambio, una arista representa un canal por el cual se pueden transmitir los datos entre los vértices, un flujo es el número promedio de bits por segundo que se transmiten en un canal, y la capacidad de una arista es la capacidad del canal correspondiente.

BIBLIOTECA FACULTAD Cs. EXACTAS, INGENIERÍA Y AGRIMENSURA ROSARIO

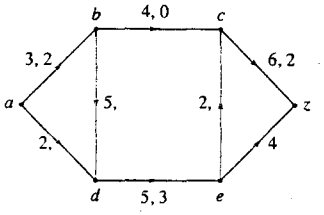
Ejercicios

En los ejercicios 1-3, escriba los flujos faltantes sobre las aristas de modo que el resultado sea un flujo en la red dada. Determine los valores de los flujos.

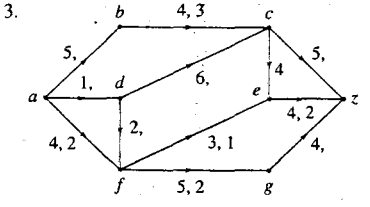
1.



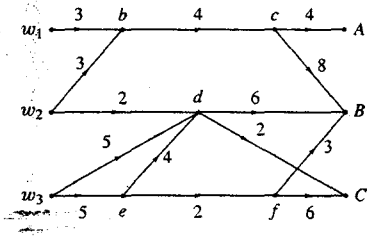
2.



3.



4. La siguiente gráfica representa una red de bombeo en la que el petróleo de tres pozos, w_1, w_2 y w_3, se entrega a tres refinerías, A, B y C. Las capacidades de los sistemas intermedios aparecen sobre las aristas. Los vértices b, c, d, e y f representan estaciones de bombeo intermedias. Modele este sistema como una red.



5. Modele el sistema del ejercicio 4 como una red, suponiendo que el pozo w_1 puede bombear a lo más 2 unidades, el pozo w_2 a lo más 4 unidades, y el pozo w_3 a lo más 7 unidades.
6. Modele el sistema del ejercicio 5 como una red, suponiendo, además de las limitaciones sobre los pozos, que la ciudad A necesita 4 unidades, que B necesita 3 unidades, y que C necesita 4 unidades.
7. Modele el sistema del ejercicio 6 como una red, suponiendo, además de las restricciones sobre los pozos y las necesidades de las ciudades, que la estación intermedia d puede bombear a lo más 6 unidades.

8. Existen dos rutas de la ciudad A a la ciudad D. Una ruta pasa por la ciudad B y la otra por la ciudad C. Durante el periodo de 7:00 a 8:00 a.m., los tiempos promedio de recorrido son

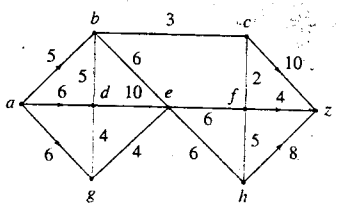
A a B (30 minutos)
A a C (15 minutos)
B a D (15 minutos)
C a D (15 minutos).

Las capacidades máximas de estas rutas son

A a B (1000 vehículos)
A a C (3000 vehículos)
B a D (4000 vehículos)
C a D (2000 vehículos).

Represente como una red el flujo de tráfico de A a D durante el periodo de 7:00 a 8:00 a.m.

9. En el siguiente sistema, queremos maximizar el flujo de a a z. Las capacidades aparecen sobre las aristas. El flujo entre dos vértices, ninguno de los cuales sea a o z, puede tener cualquier dirección. Modele este sistema como una red.



10. Proporcione un ejemplo de una red con exactamente dos flujos máximos, donde cada F_{ij} sea un entero no negativo.

11. ¿Cuál es el máximo número de aristas que puede tener una red?

8.2 *UN ALGORITMO DE FLUJO MÁXIMO*

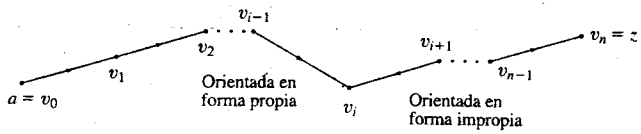
Si G es una red de transporte, un **flujo máximo** en G es un flujo con valor máximo. En general, existirán varios flujos con el mismo valor máximo. En esta sección daremos un algoritmo para determinar un flujo máximo. La idea fundamental es sencilla: comenzar con cierto flujo inicial e incrementar de manera iterativa el valor del flujo hasta que no pueda mejorarse más. El flujo resultante será entonces un flujo máximo.

Podemos considerar como flujo inicial aquél en el que el flujo en cada arista es igual a cero. Para incrementar el valor de un flujo dado, debemos determinar un camino de la fuente al sumidero e incrementar el flujo a lo largo de este camino.

En este punto, será útil introducir cierta terminología. En esta sección, G denota una red con fuente a, sumidero z y capacidad C. Por el momento, consideraremos a las aristas de G no dirigidas, y sea

$$P = (v_0, v_1, \ldots, v_n), \quad v_0 = a, \quad v_n = z,$$

un camino de a a z en esta gráfica no dirigida. (Todos los caminos de esta sección harán referencia a la gráfica no dirigida subyacente.) Si una arista e en P está dirigida de v_{i-1} a v_i, decimos que e está **orientada en forma propia (con respecto de P)**; en caso contrario, decimos que e está **orientada en forma impropia (con respecto de P)** (véase la figura 8.2.1).



Camino P

FIGURA 8.2.1 Aristas orientadas en forma propia e impropia. La arista (v_{i-1}, v_i) está orientada en forma propia, pues está orientada en la dirección de a a z. La arista (v_i, v_{i+1}) está orientada en forma impropia, pues *no* está en la dirección de a a z.

Si podemos determinar un camino P de la fuente al sumidero en donde cada arista de P esté orientada en forma propia y el flujo en cada arista es menor que la capacidad de la arista, es posible aumentar el valor del flujo.

EJEMPLO 8.2.1

Consideremos el camino de a a z de la figura 8.2.2. Todas las aristas de P están orientadas en forma propia. El valor del flujo en esta red se puede incrementar en 1, como muestra la figura 8.2.3.

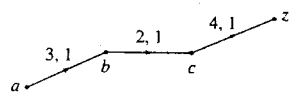


FIGURA 8.2.2 Un camino tal que todas sus aristas están orientadas en forma propia.

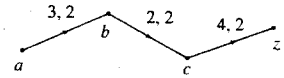


FIGURA 8.2.3 Después de incrementar el flujo de la figura 8.2.2 en 1.

□

También es posible incrementar el flujo en ciertos caminos de la fuente al sumidero que tengan aristas orientadas en forma propia e impropia. Sea P un camino de a a z y sea x un vértice en P que no sea a ni z (véase la figura 8.2.4). Existen cuatro posibilidades para

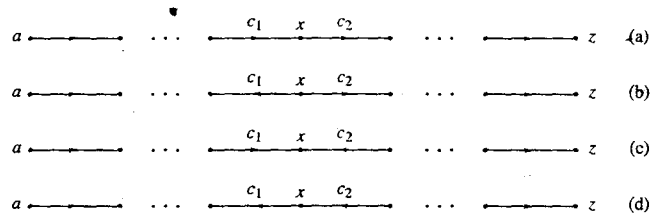


FIGURA 8.2.4 Las cuatro orientaciones posibles de las aristas incidentes en x.

las orientaciones de las aristas e_1 y e_2 incidentes en x. En el caso (a), ambas aristas están orientadas en forma propia. En este caso, si incrementamos el flujo en cada arista en Δ, el flujo de entrada en x seguirá siendo igual al flujo de salida de x. En el caso (b), si incrementamos el flujo en e_2 en Δ, debemos *disminuir* el flujo en e_1 en Δ de modo que el flujo de entrada en x siga siendo igual al flujo de salida en x. El caso (c) es análogo al caso (b), excepto que incrementamos el flujo en e_1 en Δ y disminuimos el flujo en e_2 en Δ. En el caso (d), disminuimos el flujo en ambas aristas en Δ. En cada caso, las asignaciones resultantes de las aristas dan como resultado un flujo. Por supuesto, para realizar estas alteraciones, debemos tener un flujo menor que la capacidad en una arista orientada en forma propia y un flujo distinto de cero en una arista orientada en forma impropia.

EJEMPLO 8.2.2

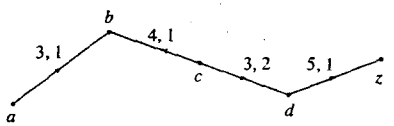


FIGURA 8.2.5 Un camino con una arista orientada en forma impropia: (c, b).

Consideremos el camino de a a z de la figura 8.2.5. Las aristas (a, b), (c, d) y (d, z) están orientadas en forma propia y la arista (c, b) está orientada en forma impropia. Disminuimos el flujo en 1 en la arista orientada en forma impropia (c, b) y aumentamos el flujo en 1 en las aristas orientadas en forma propia (a, b), (c, d) y (d, z) (véase la figura 8.2.6). El valor del nuevo flujo es 1 más que el valor del flujo original.

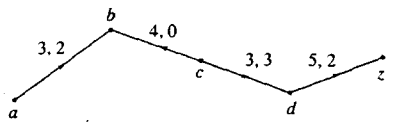


FIGURA 8.2.6 Después de incrementar en 1 el flujo de la figura 8.2.5. □

Resumimos el método de los ejemplos 8.2.1 y 8.2.2 como un teorema.

TEOREMA 8.2.3

Sea P un camino de a a z en una red G tal que:

(a) Para cada arista (i, j) de P, orientada en forma propia,

$$F_{ij} < C_{ij}.$$

(b) Para cada arista (i, j) de P, orientada en forma impropia,

$$0 < F_{ij}.$$

Sea

$$\Delta = \text{mín } X,$$

donde X consta de los números $C_{ij} - F_{ij}$ para las aristas (i, j) de P orientadas en forma propia, y F_{ij} para las aristas (i, j) de P orientadas en forma impropia. Definimos

$$F^*_{ij} = \begin{cases} F_{ij} & \text{si } (i, j) \text{ no está en } P \\ F_{ij} + \Delta & \text{si } (i, j) \text{ está orientada en forma propia en } P \\ F_{ij} - \Delta & \text{si } (i, j) \text{ está orientada en forma impropia en } P. \end{cases}$$

Entonces F^* es un flujo cuyo valor es Δ unidades mayor que el valor de F.

Demostración. (Véanse las figuras 8.2.2, 8.2.3, 8.2.5 y 8.2.6.) El argumento de que F^* es un flujo aparece antes del ejemplo 8.2.2. Como la arista (a, v) en P se incrementa en Δ, el valor de F^* es Δ unidades mayor que el valor de F. ■

En la siguiente sección mostraremos que si no existen caminos que satisfagan las condiciones del teorema 8.2.3, el flujo es máximo. Así, es posible construir un algoritmo con base en el teorema 8.2.3. La idea es

1. Iniciar con un flujo (por ejemplo, un flujo tal que el flujo en cada arista sea 0).
2. Buscar un camino que satisfaga las condiciones del teorema 8.2.3. Si no existe tal camino, se habrá terminado y el flujo es máximo.
3. Se incrementa el flujo del camino en Δ, donde Δ se define como en el teorema 8.2.3, y se regresa a la línea 2.

En el algoritmo formal, buscamos un camino que satisfaga las condiciones del teorema 8.2.3 a la vez que se lleva un registro de las cantidades

$$C_{ij} - F_{ij}, F_{ij}.$$

ALGORITMO 8.2.4 ***Determinación de un flujo máximo en una red***

Este algoritmo determina un flujo máximo en una red. La capacidad de cada arista es un entero no negativo.

Entrada: Una red con fuente a, sumidero z, capacidad C, vértices $a = v_0, \ldots, v_n = z$ y n

Salida: Un flujo máximo F

```
        procedure max_flow(a, z, C, v, n)
         // la etiqueta de v es (predecesor (v), val (v))
         // se inicia con un flujo nulo
1.       for cada arista (i, j) do
2.        F_ij := 0
3.       while true do
4.        begin
         // se eliminan todas las etiquetas
5.       for i := 0 to n do
6.         begin
7.         predecessor(v_i) := null
8.         val(vi) := null
9.         end
         // se etiqueta a
10.      predecessor(a) := −
11.      val (a) := ∞
         // U es el conjunto de vértices etiquetados no examinados
12.      U := {a}
         // se continúa hasta que z es etiquetado
13.      while val(z) = null do
14.       begin
15.       if U = ∅ then // el flujo es máximo
16.         return(F)
17.       se elige v en U
18.       U := U − {v}
19.       Δ := val (v)
20.       for cada arista (v, w) con val(w) = null do
21.        if F_vw < C_vw then
22.          begin
23.          predecessor(w) := v
24.          val(w) := min{Δ, C_vw − F_vw}
25.          U := U ∪ {w}
26.          end
27.       for cada arista (w, v) con val (w) = null do
28.        if F_wv > 0 then
29.          begin
30.          predecessor(w) := v
31.          val(w) := min{Δ, F_wv}
32.          U := U ∪ {w}
33.          end
34.       end
```



```
         // se determina un camino P de a a z para el cual se revisa el flujo
35.      w_0 := z
36.      k := 0
37.      while w_k ≠ a do
38.       begin
39.       w_{k+1} := predecessor(w_k)
40.       k := k + 1
41.       end
42.      P := (w_k, w_{k+1}, ..., w_1, w_0)
43.      Δ := val(z)
44.      for i = 1 to k do
45.       begin
46.       e := (w_i, w_{i−1})
47.       if e está orientada en forma propia en P then
48.         F_e := F_e + Δ
49.       else
50.         F_e := F_e + Δ
51.       else
52.      end
        end max_flow
```

Se deja como ejercicio demostrar que el algoritmo 8.2.4 concluye (ejercicio 19). Si se permite que las capacidades sean números racionales no negativos, el algoritmo también concluye; sin embargo, si se permiten capacidades reales no negativas arbitrarias y que las aristas de la línea 20 se revisen en cualquier orden, el algoritmo podría no concluir (véase [Ford, páginas 21-22]).

El algoritmo 8.2.4 se conoce como el **algoritmo de etiquetado**. Ilustraremos el algoritmo con dos ejemplos.

EJEMPLO 8.2.5

En este análisis, si el vértice v satisface

$$predecessor(v) = p \quad \text{y} \quad val(v) = t,$$

la etiqueta de v sobre la gráfica será (p, t).

En las líneas 1 y 2, inicializamos el flujo como 0 en cada arista (véase la figura 8.2.7). A continuación, en las líneas 5-9, hacemos todas las etiquetas iguales a *null*. Luego, en las líneas 10 y 11, etiquetamos el vértice a como $(-, \infty)$. En la línea 12 hacemos $U = \{a\}$. Luego entramos al ciclo while (línea 13).

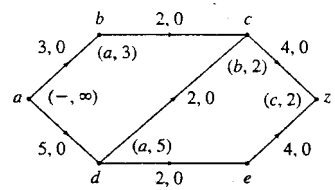


FIGURA 8.2.7 Después del primer etiquetado. El vértice v se etiqueta $(predecessor(v), val(v))$.

Como z no está etiquetado y U no es vacío, pasamos a la línea 17, donde elegimos el vértice a en U y lo eliminamos de U en la línea 18. En este momento, $U = \emptyset$. Hacemos Δ igual a ∞ [$= val(a)$] en la línea 19. En la línea 20 examinamos las aristas (a, b) y (a, d) pues b y d no están etiquetados. Para la arista (a, b) tenemos

$$F_{ab} = 0 < C_{ab} = 3.$$

En las líneas 23 y 24, etiquetamos el vértice b como $(a, 3)$, ya que

$$predecessor(b) = a$$

y

$$val(b) = \min\{\Delta, 3 - 0\} = \min\{\infty, 3 - 0\} = 3.$$

En la línea 25, agregamos b a U. De manera análoga, etiquetamos el vértice d como $(a, 5)$ y agregamos d a U. En este momento, $U = \{b, d\}$.

Ahora regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos a b. Eliminamos b de U en la línea 18. Hacemos Δ igual a 3 [$= val(b)$] en la línea 19. En la línea 20 examinamos la arista (b, c). En las líneas 23 y 24 etiquetamos el vértice c como $(b, 2)$ ya que

$$predecessor(c) = b$$

y

$$val(c) = \min\{\Delta, 2 - 0\} = \min\{3, 2 - 0\} = 2.$$

En la línea 25 agregamos c a U. En este punto, $U = \{c, d\}$.

Ahora regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos a c. Eliminamos c de U en la línea 18. Hacemos Δ igual a 2 [$= val(c)$] en la línea 19. En la línea 20 examinamos la arista (c, z). En las líneas 23 y 24 etiquetamos el vértice z como $(c, 2)$. En la línea 25 agregamos z a U. En este punto, $U = \{d, z\}$.

Ahora regresamos a la parte superior del ciclo while (línea 13). Como z está etiquetado, pasamos a la línea 35. En las líneas 35-42, seguimos los predecesores desde z para determinar el camino

$$P = (a, b, c, z)$$

de a a z. En la línea 42 hacemos Δ igual a 2. Como cada arista de P está orientada en forma propia, en la línea 48 incrementamos el flujo en cada arista de P en $\Delta = 2$ unidades, para obtener la figura 8.2.8.

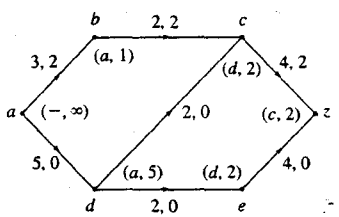


FIGURA 8.2.8 Después de incrementar el flujo en el camino (a, b, c, z) en 2 unidades y del segundo etiquetado.

Ahora regresamos a la parte superior del ciclo while (línea 3). Luego, en las líneas 5-9 hacemos todas las etiquetas iguales a *null*. Entonces, en las líneas 10 y 11, etiquetamos el vértice a como $(-, \infty)$ (véase la figura 8.2.8). En la línea 12 hacemos $U = \{a\}$. Luego entramos al ciclo while (línea 13).

Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos el vértice a en U y lo eliminamos de U en la línea 18. En las líneas 23 y 24 etiquetamos el vértice b como $(a, 1)$ y el vértice d como $(a, 5)$. Agregamos b y d a U, de modo que $U = \{b, d\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos b. Eliminamos b de U en la línea 18. En la línea 20 examinamos la arista (b, c). Como $F_{bc} = C_{bc}$, no etiquetamos el vértice c en este punto. Ahora, $U = \{d\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos el vértice d en U y lo eliminamos de U en la línea 18. En las líneas 23 y 24 etiquetamos el vértice c como $(d, 2)$ y al vértice e como $(d, 2)$. Agregamos c y e a U, de modo que $U = \{c, e\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos c en U y lo eliminamos de U en la línea 18. En las líneas 23 y 24 etiquetamos el vértice z como $(c, 2)$. Agregamos z a U, de modo que $U = \{z, e\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z está etiquetado, pasamos a la línea 35. En la línea 42 tenemos que

$$P = (a, d, c, z).$$

Como cada arista de P está orientada en forma propia, en la línea 48 incrementamos el flujo en cada arista de P en $\Delta = 2$ unidades para obtener la figura 8.2.9.

El lector debe verificar que la siguiente iteración del algoritmo produce el etiquetado de la figura 8.2.9. Al incrementar el flujo en $\Delta = 2$ unidades se obtiene la figura 8.2.10.

Ahora regresamos a la parte superior del ciclo while (línea 3). A continuación, en las líneas 5-9 hacemos todas las etiquetas iguales a *null*. Entonces, en las líneas 10 y 11 etiquetamos el vértice a como $(-, \infty)$ (véase la figura 8.2.10). En la línea 12 hacemos $U = \{a\}$. Luego entramos al ciclo while (línea 13).

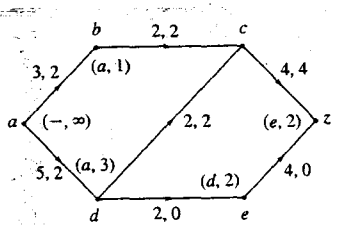


FIGURA 8.2.9 Después de incrementar el flujo en el camino (a, d, c, z) en 2 unidades y del tercer etiquetado.

Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos el vértice a en U y lo eliminamos de U en la línea 18. En las líneas 23 y 24 etiquetamos el vértice b como $(a, 1)$ y el vértice d como $(a, 1)$. Agregamos b y d a U, de modo que $U = \{b, d\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos b. Eliminamos b de U en la línea 18. En la línea 20 examinamos la arista (b, c). Como $F_{bc} = C_{bc}$, no etiquetamos el vértice c. Ahora, $U = \{d\}$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos el vértice d en U y lo eliminamos de U en la línea 18. En la línea 20 examinamos las aristas (d, c) y (d, e). Como $F_{dc} = C_{dc}$ y $F_{de} = C_{de}$, no etiquetamos el vértice c ni el vértice e. Ahora, $U = \varnothing$.

Luego regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado, pasamos a la línea 15. Como U es vacío, el algoritmo termina. El flujo de la figura 8.2.10 es máximo.

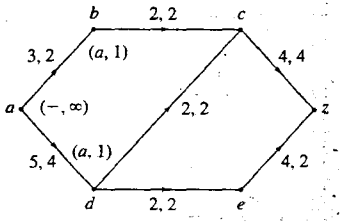


FIGURA 8.2.10 Después de incrementar el flujo en el camino (a, d, e, z) en 2 unidades y del cuarto y último etiquetado. El flujo es máximo. □

Nuestro último ejemplo muestra la forma de modificar el algoritmo 8.2.4 para generar un flujo máximo a partir de un flujo dado.

EJEMPLO 8.2.6

Reemplazar el flujo nulo de las líneas 1 y 2 del algoritmo 8.2.4 con el flujo de la figura 8.2.11 y determinar luego un flujo máximo.

Después de inicializar el flujo dado, pasamos a las líneas 5-9, donde hacemos todas las etiquetas iguales a *null*. Luego, en las líneas 10 y 11, etiquetamos el vértice a como $(-, \infty)$ (véase la figura 8.2.11). En la línea 12 hacemos $U = \{a\}$. Luego entramos al ciclo while (línea 13).

Como z no está etiquetado y U no es vacío, pasamos a la línea 17, donde elegimos el vértice a en U y lo eliminamos de U en la línea 18. En las líneas 23 y 24, etiquetamos el vértice b como $(a, 1)$ y el vértice d como $(a, 1)$. Agregamos b y d a U de modo que $U = \{b, d\}$.

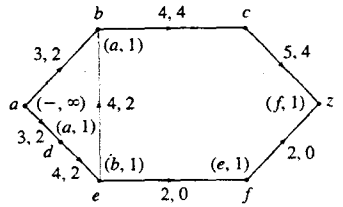


FIGURA 8.2.11 Después del etiquetado.

Ahora regresamos a la parte superior del ciclo while (línea 13). Como z no está etiquetado y U no es vacío, nos movemos a la línea 17, donde elegimos un vértice en U. Suponga que elegimos a b. Eliminamos b de U en la línea 18. En la línea 20 examinamos las aristas (b, c) y (e, b). Como $F_{bc} = C_{bc}$, no etiquetamos el vértice c. En las líneas 30 y 31, el vértice e se etiqueta $(b, 1)$ ya que

$$val(e) = \text{mín}\{val(b), F_{eb}\} = \text{mín}\{1, 2\} = 1.$$

Luego regresamos a la parte superior del ciclo while (línea 13). Finalmente etiquetamos z (véase la figura 8.2.11) y en la línea 42 determinamos el camino

$$P = (a, b, e, f, z).$$

Las aristas (a, b), (e, f) y (f, z) están orientadas en forma propia, de modo que el flujo en cada una se incrementa en 1. Como la arista (e, b) está orientada en forma impropia, su flujo se decrementa en 1 unidad. Obtenemos el flujo de la figura 8.2.12.

Otra iteración del algoritmo produce el flujo máximo de la figura 8.2.13. □

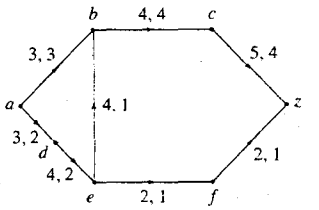


FIGURA 8.2.12 Después de incrementar el flujo del camino (a, b, e, f, z) en 1 unidad. Observe que la arista (e, b) está orientada en forma impropia, por lo que su flujo se *disminuye* en 1 unidad.

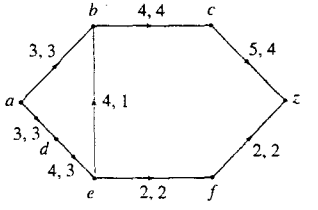
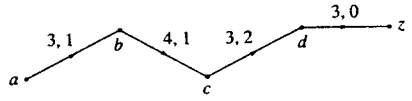


FIGURA 8.2.13 Después de incrementar el flujo del camino (a, d, e, f, z) en 1 unidad. El flujo es máximo.

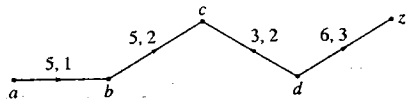
Ejercicios

En los ejercicios 1-3 se da un camino de la fuente a al sumidero z en una red. Determine el máximo incremento posible en el flujo, que se pueda obtener alterando los flujos en las aristas en el camino.

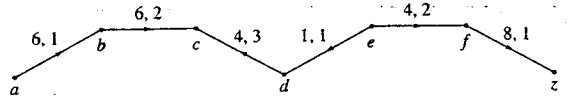
1.



2.



3.

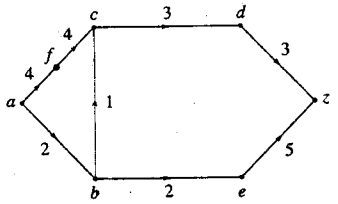


En los ejercicios 4-12, utilice el algoritmo 8.2.4 para determinar un flujo máximo en cada red.

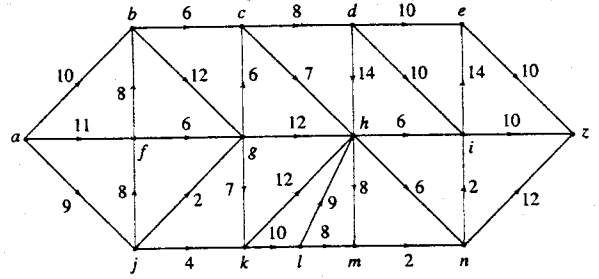
4. Figura 8.1.4

5. Figura 8.1.5

6.



7. Ejercicio 5, sección 8.1
8. Ejercicio 6, sección 8.1
9. Ejercicio 7, sección 8.1
10. Ejercicio 8, sección 8.1
11. Ejercicio 9, sección 8.1
12.



En los ejercicios 13-18, determine un flujo máximo en cada red, comenzando con el flujo dado.

13. Figura 8.1.2

14. Ejercicio 1, sección 8.1

15. Ejercicio 2, sección 8.1

16. Ejercicio 3, sección 8.1

17. La figura 8.1.4, con los flujos

$$F_{a,w_1} = 2, \quad F_{w_1,b} = 2, \quad F_{bA} = 0, \quad F_{cA} = 0, \quad F_{Az} = 0,$$
$$F_{a,w_2} = 0, \quad F_{w_2,b} = 0, \quad F_{bc} = 2, \quad F_{cB} = 4, \quad F_{Bz} = 4,$$
$$F_{a,w_3} = 2, \quad F_{w_3,d} = 2, \quad F_{dc} = 2.$$

18. La figura 8.1.4, con los flujos

$$F_{a,w_1} = 1, \quad F_{w_1,b} = 1, \quad F_{bA} = 4, \quad F_{cA} = 2, \quad F_{Az} = 6,$$
$$F_{a,w_2} = 3, \quad F_{w_2,b} = 3, \quad F_{bc} = 0, \quad F_{cB} = 1, \quad F_{Bz} = 1,$$
$$F_{a,w_3} = 3, \quad F_{w_3,d} = 3, \quad F_{dc} = 3.$$

19. Muestre que el algoritmo 8.2.4 concluye.

8.3 *EL TEOREMA DEL FLUJO MÁXIMO Y CORTE MÍNIMO*

En esta sección mostraremos que al concluir el algoritmo 8.2.4, el flujo en la red es máximo. Mientras tanto, definiremos y analizaremos los cortes en las redes.

Sea G una red y consideremos el flujo F al concluir el algoritmo 8.2.4. Algunos vértices están etiquetados y otros no. Sea P ($\overline{P}$) el conjunto de vértices etiquetados (no etiquetados). (Recuerde que $\overline{P}$ denota el complemento de P.) Entonces la fuente a está en P y el sumidero z está en $\overline{P}$. El conjunto S de aristas (v, w), con $v \in P$ y $w \in \overline{P}$ es un **corte**, y la suma de las capacidades de las aristas en S es la **capacidad del corte**. Veremos que este corte tiene una capacidad mínima y, como un corte mínimo corresponde a un flujo máximo (teorema 8.3.9), el flujo F es máximo. Comenzamos con la definición formal de corte.

En esta sección, G es una red con fuente a y sumidero z. La capacidad de la arista (i, j) es C_{ij}.

DEFINICION 8.3.1

Un *corte* $(P, \overline{P})$ en G consta de un conjunto P de vértices y el complemento $\overline{P}$ de P, con $a \in P$ y $z \in \overline{P}$.

EJEMPLO 8.3.2

Consideremos la red G de la figura 8.3.1. Si hacemos $P = \{a, b, d\}$, entonces $\overline{P} = \{c, e, f, z\}$ y $(P, \overline{P})$ es un corte en G. Como muestra la figura, a veces indicamos un corte trazando una línea punteada para separar los vértices.

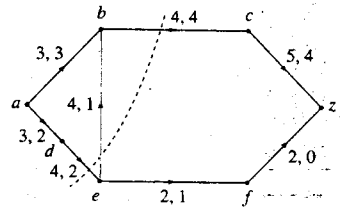


FIGURA 8.3.1 Un corte en una red. La línea punteada divide los vértices en los conjuntos $P = \{a, b, d\}$ y $\overline{P} = \{c, e, f, z\}$ produciendo el corte $(P, \overline{P})$. □

EJEMPLO 8.3.3

La figura 8.2.10 muestra el etiquetado al final del algoritmo 8.2.4 para una red particular. Si P ($\overline{P}$) denota el conjunto de vértices etiquetados (no etiquetados), obtenemos el corte que se muestra en la figura 8.3.2.

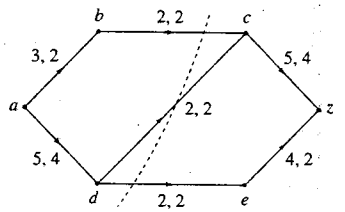


FIGURA 8.3.2 Una red al finalizar el algoritmo 8.2.4. El corte $(P, \overline{P})$, $P = \{a, b, d\}$ se obtiene al definir P como el conjunto de vértices etiquetados. □

Ahora definiremos la capacidad de un corte.

DEFINICIÓN 8.3.4

La *capacidad del corte* $(P, \overline{P})$ es el número

$$C(P, \overline{P}) = \sum_{i \in P} \sum_{j \in \overline{P}} C_{ij}.$$

EJEMPLO 8.3.5

La capacidad del corte de la figura 8.3.1 es

$$C_{bc} + C_{de} = 8.$$ □

EJEMPLO 8.3.6

La capacidad del corte de la figura 8.3.2 es

$$C_{bc} + C_{dc} + C_{de} = 6.$$ □

El siguiente teorema muestra que la capacidad de cualquier corte siempre es mayor o igual que el valor de cualquier flujo.

TEOREMA 8.3.7

Sea F un flujo en G y sea $(P, \overline{P})$ un corte en G. Entonces la capacidad de $(P, \overline{P})$ es mayor o igual que el valor de F; es decir,

$$\sum_{i \in P} \sum_{j \in \overline{P}} C_{ij} \geq \sum_{i} F_{ai}. \tag{8.3.1}$$

(La notación $\sum_i$ significa la suma sobre todos los vértices i.)

Demostración. Observe que

$$\sum_{j \in P} \sum_{i \in P} F_{ji} = \sum_{j \in P} \sum_{i \in P} F_{ij},$$

pues cada lado de la ecuación es simplemente la suma de F_{ij} sobre todas las $i, j \in P$.

Ahora,

$$\begin{aligned}
\sum_i F_{ai} &= \sum_{j \in P} \sum_i F_{ji} - \sum_{j \in P} \sum_i F_{ij} \\
&= \sum_{j \in P} \sum_{i \in P} F_{ji} + \sum_{j \in P} \sum_{i \in \overline{P}} F_{ji} - \sum_{j \in P} \sum_{i \in P} F_{ij} - \sum_{j \in P} \sum_{i \in \overline{P}} F_{ij} \\
&= \sum_{j \in P} \sum_{i \in \overline{P}} F_{ji} - \sum_{j \in P} \sum_{i \in \overline{P}} F_{ij} \leq \sum_{j \in P} \sum_{i \in \overline{P}} F_{ji} \leq \sum_{j \in P} \sum_{i \in \overline{P}} C_{ij}.
\end{aligned}$$ ■

EJEMPLO 8.3.8

En la figura 8.3.1, el valor 5 del flujo es menor que la capacidad 8 del corte. □

Un **corte mínimo** es un corte con capacidad mínima.

TEOREMA 8.3.9 *Teorema del flujo máximo y el corte mínimo*

Sea F un flujo en G y sea $(P, \overline{P})$ un corte en G. Si la igualdad se cumple en (8.3.1), *entonces el flujo es máximo y el corte es mínimo. Además, la igualdad se cumple en* (8.3.1) *si y sólo si*

(*a*) $F_{ij} = C_{ij}$ para $i \in P$, $j \in \overline{P}$

y

(*b*) $F_{ij} = 0$ para $i \in \overline{P}$, $j \in P$.

Demostración. El primer enunciado es inmediato.

La demostración del teorema 8.3.7 muestra que la igualdad se cumple precisamente cuando

$$\sum_{j\in P}\sum_{i\in\overline{P}} F_{ij} = 0 \quad \text{y} \quad \sum_{j\in P}\sum_{i\in\overline{P}} F_{ji} = \sum_{j\in P}\sum_{i\in\overline{P}} C_{ji};$$

de modo que la última afirmación también es verdadera. ■

EJEMPLO 8.3.10

En la figura 8.3.2, el valor del flujo y la capacidad del corte son ambos iguales a 6; por tanto, el flujo es máximo y el corte es mínimo. □

Podemos utilizar el teorema 8.3.9 para mostrar que el algoritmo 8.2.4 produce un flujo máximo.

TEOREMA 8.3.11

Al concluir, el algoritmo 8.2.4 *produce un flujo máximo. Además, si P (respectivamente, $\overline{P}$) es el conjunto de vértices etiquetados (respectivamente, no etiquetados) al concluir el algoritmo* 8.2.4, *el corte $(P, \overline{P})$ es mínimo.*

Demostración. Sea $\overline{P}$ el conjunto de vértices etiquetados (no etiquetados) de G al concluir el algoritmo 8.2.4. Consideremos una arista (i, j), donde $i \in P$, $j \in \overline{P}$. Como i está etiquetado, debemos tener

$$F_{ij} = C_{ij};$$

en caso contrario, habríamos etiquetado j en las líneas 23 y 24. Ahora, consideremos una arista (j, i), donde $j \in \overline{P}$, $i \in P$. Como i está etiquetado, debemos tener

$$F_{ji} = 0;$$

en caso contrario, habríamos etiquetado j en las líneas 30 y 31. Por el teorema 8.3.9, al concluir el algoritmo 8.2.4, el flujo es máximo y el corte $(P, \overline{P})$ es mínimo. ■

Ejercicios

En los ejercicios 1-3, determine la capacidad del corte $(P, \overline{P})$. Además, determine si el corte es mínimo.

1. $P = \{a, d\}$ para el ejercicio 1, sección 8.1
2. $P = \{a, d, e\}$ para el ejercicio 2, sección 8.1
3. $P = \{a, b, c, d\}$ para el ejercicio 3, sección 8.1

En los ejercicios 4-16, determine un corte mínimo en cada red.

4. Figura 8.1.1
5. Figura 8.1.4
6. Figura 8.1.5
7. Ejercicio 1, sección 8.1
8. Ejercicio 2, sección 8.1
9. Ejercicio 3, sección 8.1
10. Ejercicio 4, sección 8.1
11. Ejercicio 5, sección 8.1
12. Ejercicio 6, sección 8.1
13. Ejercicio 7, sección 8.1
14. Ejercicio 8, sección 8.1
15. Ejercicio 9, sección 8.1
16. Ejercicio 12, sección 8.2

Los ejercicios 17-22 se refieren a una red G tal que, además de tener capacidades C_{ij} dadas por enteros no negativos, tiene requisitos de flujo mínimo en una arista dados por enteros no negativos m_{ij}. Es decir, un flujo F debe satisfacer

$$m_{ij} \le F_{ij} \le C_{ij}$$

para todas las aristas (i, j).

17. Proporcione un ejemplo de una red G, en la cual $m_{ij} \le C_{ij}$ para todas las aristas (i, j), de modo que no exista un flujo.

Defina

$$C(\overline{P}, P) = \sum_{i\in\overline{P}}\sum_{j\in P} C_{ij},$$

$$m(P, \overline{P}) = \sum_{i\in P}\sum_{j\in\overline{P}} m_{ij}, \qquad m(\overline{P}, P) = \sum_{i\in\overline{P}}\sum_{j\in P} m_{ij}.$$

18. Muestre que el valor V de cualquier flujo satisface

$$m(P, \overline{P}) - C(\overline{P}, P) \le V \le C(P, \overline{P}) - m(\overline{P}, P)$$

para cualquier corte $(P, \overline{P})$.

19. Muestre que si existe un flujo en G, entonces existe un flujo máximo en G con valor

$$\text{mín}\{C(P, \overline{P}) - m(\overline{P}, P) \mid (P, \overline{P}) \text{ es un corte en } G\}.$$

20. Suponga que G tiene un flujo F. Desarrolle un algoritmo para determinar un flujo máximo en G.
21. Muestre que si existe un flujo en G, entonces existe un flujo mínimo en G con valor

$$\text{máx}\{m(P, \overline{P}) - C(\overline{P}, P) \mid (P, \overline{P}) \text{ es un corte en } G\}.$$

22. Suponga que G tiene un flujo F. Desarrolle un algoritmo para determinar un flujo mínimo en G.
23. ¿Verdadero o falso? Si F es un flujo en una red G, $(P, \overline{P})$ es un corte en G y la capacidad de $(P, \overline{P})$ excede al valor del flujo, F, entonces el corte $(P, \overline{P})$ no es mínimo y el flujo F no es máximo. Si es verdadero, demuéstrelo; en caso contrario, proporcione un contraejemplo.

8.4 ACOPLAMIENTO

En esta sección consideraremos el problema de hacer corresponder los elementos de un conjunto con los elementos de otro conjunto. Veremos que este problema se puede reducir a determinar un flujo máximo en una red. Comenzaremos con un ejemplo.

EJEMPLO 8.4.1

Supongamos que cuatro personas *A, B, C* y *D* realizan una solicitud para cinco trabajos J_1, J_2, J_3, J_4 y J_5. Suponga que el solicitante *A* está calificado para los trabajos J_2 y J_5; el solicitante *B* está calificado para los trabajos J_2 y J_5; el solicitante *C* está calificado para los trabajos J_1, J_3, J_4 y J_5; y el solicitante *D* está calificado para los trabajos J_2 y J_5. ¿Es posible hallar un trabajo para cada solicitante?

La situación se puede modelar mediante la gráfica de la figura 8.4.1. Los vértices representan a los solicitantes y los trabajos. Una arista une un solicitante con un trabajo para el cual el solicitante está calificado. Podemos mostrar que no es posible asociar un trabajo a cada solicitante, si nos fijamos en los solicitantes *A, B* y *D*, quienes son calificados para los trabajos J_2 y J_5. Si *A* y *B* reciben un trabajo, no quedaría un trabajo para *D*. Por tanto, no existe una asignación para *A, B, C* y *D*. □

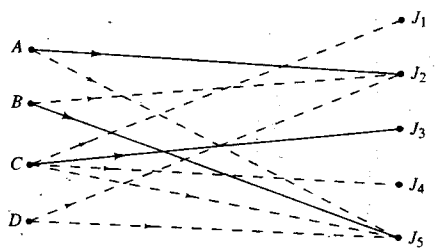


FIGURA 8.4.1 Solicitantes (*A, B, C, D*) y trabajos (J_1, J_2, J_3, J_4, J_5). Una arista une un solicitante con un trabajo para el cual está calificado. Las líneas continuas muestran un acoplamiento máximo (es decir, el mayor número de solicitantes tiene trabajo).

En el ejemplo 8.4.1, un acoplamiento consiste en hallar trabajos para las personas calificadas. Un acoplamiento máximo encuentra trabajo para el mayor número de personas. En la gráfica de la figura 8.4.1 aparece con líneas continuas un acoplamiento máximo. Un acoplamiento completo encuentra trabajo para todos. Hemos mostrado que la gráfica de la figura 8.4.1 no tiene un acoplamiento completo. A continuación damos las definiciones formales.

DEFINICIÓN 8.4.2

Sea *G* una gráfica dirigida, bipartita, con conjuntos ajenos de vértices *V* y *W* en los cuales las aristas se dirigen de los vértices en *V* a los vértices en *W*. (Cualquier vértice en *G* está en *V* o en *W*.) Un *acoplamiento* para *G* es un conjunto de aristas *E* sin vértices en común. Un *acoplamiento máximo* para *G* es un acoplamiento *E* tal que *E* contiene el número máximo de aristas. Un *acoplamiento completo* para *G* es un acoplamiento *E* con la propiedad de que si $v \in V$, entonces $(v, w) \in E$ para algún $w \in W$.

EJEMPLO 8.4.3

El acoplamiento para la gráfica de la figura 8.4.2, que aparece con líneas continuas, es un acoplamiento máximo y un acoplamiento completo. □

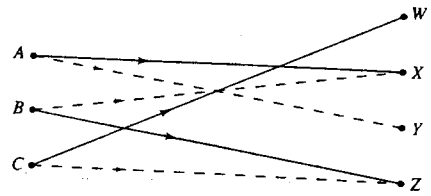


FIGURA 8.4.2 Las líneas continuas muestran un acoplamiento máximo (se utiliza el máximo número de aristas) y un acoplamiento completo (se establece una correspondencia de *A, B* y *C*).

En el siguiente ejemplo ilustraremos la forma de utilizar el problema de acoplamiento como un problema de redes.

EJEMPLO 8.4.4 ***Una red de acoplamiento***

Modele el problema de acoplamiento del ejemplo 8.4.1 como una red.

Primero asignamos a cada arista de la gráfica de la figura 8.4.1 la capacidad 1 (véase la figura 8.4.3). A continuación agregamos una superfuente *a* y aristas de capacidad 1 de *a* a cada uno de los vértices *A, B, C* y *D*. Por último, agregamos un supersumidero y aristas de capacidad 1 desde J_1, J_2, J_3, J_4 y J_5 a *z*. Una red como la de la figura 8.4.3 es una **red de acoplamiento**.

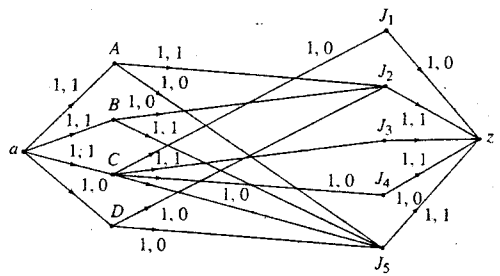


FIGURA 8.4.3 El problema de acoplamiento (figura 8.4.1) como una red de acoplamiento. □

El siguiente teorema relaciona las redes de acoplamiento y los flujos.

TEOREMA 8.4.5

Sea G una gráfica dirigida, bipartita, con conjuntos ajenos de vértices V y W en la cual las aristas están dirigidas de los vértices de V a los vértices de W. (Cualquier vértice de G está en V o en W.)

(a) Un flujo en la red de acoplamiento proporciona un acoplamiento en G. El vértice $v \in V$ se asocia con el vértice $w \in W$ si y sólo si el flujo en la arista (v, w) es 1.

(b) Un flujo máximo corresponde a un acoplamiento máximo.

(c) Un flujo con valor $|V|$ corresponde a un acoplamiento completo.

Demostración. Sea a (z) la fuente (sumidero) en la red de acoplamiento y supongamos dado un flujo.

Supongamos que la arista (v, w), $v \in V$, $w \in W$, tiene flujo 1. La única arista hacia el vértice v es (a, v). Esta arista debe tener flujo 1; así, el flujo hacia el vértice v es 1. Como el flujo de salida de v también es 1, la única arista de la forma (v, x) que tiene flujo 1 es (v, w). De manera análoga, la única arista de la forma (x, w) que tiene flujo 1 es (v, w). Por tanto, si E es el conjunto de aristas de la forma (v, w) con flujo 1, los miembros de E no tienen vértices en común, por lo que E es un acoplamiento para G.

Las partes (b) y (c) son consecuencia del hecho de que el número de vértices en V asociados mediante este acoplamiento es igual al valor del flujo correspondiente. ■

Como un flujo máximo proporciona un acoplamiento máximo, el algoritmo 8.2.4 aplicado a una red de acoplamiento produce un acoplamiento máximo. En la práctica, se puede simplificar la implantación del algoritmo 8.2.4 utilizando la matriz de adyacencia de la gráfica (véase el ejercicio 11).

EJEMPLO 8.4.6

El acoplamiento de la figura 8.4.1 se representa como un flujo en la figura 8.4.3. Como el flujo es máximo, el acoplamiento es máximo. □

Ahora analizaremos la existencia de un acoplamiento completo en una gráfica dirigida bipartita G con conjuntos de vértices V y W. Si $S \subseteq V$, sea

$$R(S) = \{ w \in W \mid v \in S \text{ y } (v, w) \text{ es una arista en } G\}.$$

Suponga que G tiene un acoplamiento completo. Si $S \subseteq V$ debemos tener

$$|S| \leq |R(S)|.$$

Ocurre que si $|S| \leq |R(S)|$ para todos los subconjuntos S de V, entonces G tiene un acoplamiento completo. Este resultado fue planteado por vez primera por el matemático inglés Philip Hall y se conoce como el **teorema de los matrimonios de Hall**, pues si V es un conjunto de hombres y W es un conjunto de mujeres, y si existe una arista de $v \in V$ en $w \in W$ si v y w son compatibles, el teorema proporciona una condición bajo la cual cada hombre puede casarse con una mujer compatible.

TEOREMA 8.4.7 *Teorema de los matrimonios de Hall*

Sea G una gráfica dirigida, bipartita, con conjuntos ajenos de vértices V y W en la cual las aristas se dirigen de los vértices en V a los vértices en W. (Cualquier vértice en G está en V o en W.) Existe un acoplamiento completo en G si y sólo si

$$|S| \leq |R(S)| \quad \text{para todo } S \subseteq V. \tag{8.4.1}$$

Demostración. Ya hemos señalado que si existe un acoplamiento completo en G, entonces se cumple la condición (8.4.1).

Supongamos que se cumple la condición (8.4.1). Sea $n = |V|$ y sea $(P, \overline{P})$ un corte mínimo en la red de acoplamiento. Si podemos mostrar que la capacidad de este corte es n, un flujo máximo tendría valor n. El acoplamiento correspondiente a este flujo máximo sería un acoplamiento completo.

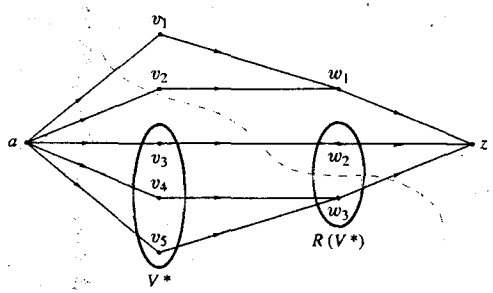


FIGURA 8.4.4 La demostración del teorema 8.4.7. $V = \{v_1, v_2, v_3, v_4, v_5\}$; $n = |V| = 5$; $W = \{w_1, w_2, w_3\}$; el corte es $(P, \overline{P})$, donde $P = \{a, v_3, v_4, v_5, w_3\}$; $V^* = V \cap P = \{v_3, v_4, v_5\}$; $R(V^*) = \{w_2, w_3\}$; $W_1 = R(V^*) \cap P = \{w_3\}$; $W_2 = R(V^*) \cap \overline{P} = \{w_2\}$; $E = \{(a, v_1), (a, v_2), (v_3, w_2), (w_3, z)\}$. La capacidad del corte es $|E| = 4 < n$. Las aristas de tipo I son (a, v_1) y (a, v_2). La arista (v_3, w_2) es la única arista de tipo II, y la arista (w_3, z) es la única arista de tipo III.

El argumento es por contradicción. Suponga que la capacidad del corte mínimo $(P, \overline{P})$ es menor que n. La capacidad de este corte es el número de aristas en el conjunto

$$E = \{(x, y) \mid x \in P, y \in \overline{P}\}$$

(véase la figura 8.4.4). Un miembro de E es de uno de los tres tipos

Tipo I: (a, v), $v \in V$.
Tipo II: (v, w), $v \in V$, $w \in W$.
Tipo III: (w, z), $w \in W$.

Estimaremos el número de aristas de cada tipo.

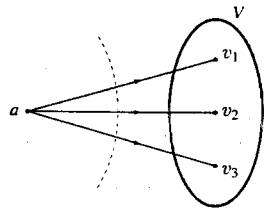


FIGURA 8.4.5 La demostración del teorema 8.4.7 para $n = 3$. Si $V \subseteq \overline{P}$, se puede ver en la figura que la capacidad del corte es n. Como estamos suponiendo que la capacidad del corte es menor que n, este caso no puede ocurrir. Por tanto, $V \cap P$ es no vacío.

Si $V \subseteq \overline{P}$, la capacidad del corte es n (véase la figura 8.4.5); así,

$$V^* = V \cap P$$

es no vacío. Esto implica que existen $n - |V^*|$ aristas en E de tipo I.

Separamos $R(V^*)$ en los conjuntos

$$W_1 = R(V^*) \cap P \quad \text{y} \quad W_2 = R(V^*) \cap \overline{P}.$$

Entonces existen al menos $|W_1|$ aristas en E de tipo III. Así, existen menos de

$$n - (n - |V^*|) - |W_1| = |V^*| - |W_1|$$

aristas de tipo II en E. Como cada miembro de W_2 contribuye a lo más con una arista de tipo II,

$$|W_2| < |V^*| - |W_1|.$$

Así,

$$|R(V^*)| = |W_1| + |W_2| < |V^*|,$$

lo cual contradice (8.4.1). Por tanto, existe un acoplamiento completo. ■

EJEMPLO 8.4.8

Para la gráfica de la figura 8.4.1, si $S = \{A, B, D\}$, tenemos que $R(S) = \{J_2, J_5\}$ y

$$|S| = 3 > 2 = |R(S)|.$$

Por el teorema 8.4.7, no existe un acoplamiento completo para la gráfica de la figura 8.4.1. □

EJEMPLO 8.4.9

Existen n computadoras y n unidades de disco. Cada computadora es compatible con $m > 0$ unidades de disco y cada unidad de disco es compatible con m computadoras. ¿Es posible asociar cada computadora con una unidad de disco compatible?

Sea V el conjunto de computadoras y W el conjunto de unidades de disco. Existe una arista de $v \in V$ en $w \in W$ si v y w son compatibles. Observe que cada vértice tiene grado

m. Sea $S = \{v_1, \ldots, v_k\}$ un subconjunto de V. Entonces existen km aristas que salen del conjunto S. Si $R(S) = \{w_1, \ldots, w_j\}$, entonces $R(S)$ recibe a lo más jm aristas de S. Por tanto,

$$km \leq jm.$$

Ahora,

$$|S| = k \leq j = |R(S)|.$$

Por el teorema 8.4.7, existe un acoplamiento completo. Así, es posible asociar cada computadora con una unidad de disco compatible. □

Ejercicios

1. Muestre que el flujo de la figura 8.4.3 es máximo, exhibiendo un corte mínimo cuya capacidad sea 3.
2. Determine el flujo que corresponde al acoplamiento de la figura 8.4.2. Muestre que este flujo es máximo, exhibiendo un corte mínimo cuya capacidad sea 3.
3. El solicitante A está calificado para los trabajos J_1 y J_4; B está calificado para los trabajos J_2, J_3 y J_6; C está calificado para los trabajos J_1, J_3, J_5 y J_6; D está calificado para los trabajos J_1, J_3 y J_4; y E está calificado para los trabajos J_1, J_3 y J_6.
   (a) Modele esta situación como una red de acoplamiento.
   (b) Utilice el algoritmo 8.2.4 para determinar un acoplamiento máximo.
   (c) ¿Existe un acoplamiento completo?
4. El solicitante A está calificado para los trabajos J_1, J_2, J_4 y J_5; B está calificado para los trabajos J_1, J_4 y J_5; C está calificado para los trabajos J_1, J_4 y J_5; D está calificado para los trabajos J_1 y J_5; E está calificado para los trabajos J_2, J_3 y J_5; y F está calificado para los trabajos J_4 y J_5. Responda las partes (a)-(c) del ejercicio 3 para esta situación.
5. El solicitante A está calificado para los trabajos J_1, J_2 y J_4; B está calificado para los trabajos J_3, J_4, J_5 y J_6; C está calificado para los trabajos J_1 y J_5; D está calificado para los trabajos J_1, J_3, J_4 y J_8; E está calificado para los trabajos J_1, J_2, J_4, J_6 y J_8; F está calificado para los trabajos J_4 y J_6; y G está calificado para los trabajos J_3, J_5 y J_7. Responda las partes (a)-(c) del ejercicio 3 para esta situación.
6. Cinco estudiantes, V, W, X, Y y Z, son miembros de cuatro comités, C_1, C_2, C_3 y C_4. Los miembros de C_1 son V, X y Y; los miembros de C_2 son X y Z; los miembros de C_3 son V, Y y Z; y los miembros de C_4 son V, W, X y Z. Cada comité debe enviar un representante a la administración. Ningún estudiante puede representar a dos comités.
   (a) Modele esta situación como una red de acoplamiento.
   (b) ¿Cuál es la interpretación de un acoplamiento máximo?
   (c) ¿Cuál es la interpretación de un acoplamiento completo?
   (d) Utilice el algoritmo 8.2.4 para determinar un acoplamiento máximo.
   (e) ¿Existe un acoplamiento completo?
7. Muestre que con un orden adecuado de los vértices, la matriz de adyacencia de una gráfica bipartita se puede escribir como

$$\begin{pmatrix} 0 & A \\ A^T & 0 \end{pmatrix},$$

donde 0 es una matriz que consta únicamente de ceros y A^T es la transpuesta de la matriz A.

En los ejercicios 8-10, G es una gráfica bipartita, A es la matriz del ejercicio 7, y F un flujo en la red de acoplamiento asociada. Etiquete cada entrada en A que represente una arista con el flujo igual a 1.

8. ¿Cuál tipo de etiquetado corresponde a un acoplamiento?
9. ¿Cuál tipo de etiquetado corresponde a un acoplamiento completo?
10. ¿Cuál tipo de etiquetado corresponde a un acoplamiento máximo?
11. Enuncie el algoritmo 8.2.4, aplicado a una red de acoplamiento, en términos de operaciones de la matriz A del ejercicio 7.

Sea G una gráfica dirigida, bipartita, con conjuntos ajenos de vértices V y W en los cuales las aristas se dirigen de los vértices en V a los vértices en W. (Cualquier vértice en G está en V o en W.) Definimos la deficiencia de G como

$$\delta(G) = \max\{\,|S| - |R(S)| \mid S \subseteq V\,\}.$$

12. Muestre que G tiene un acoplamiento completo si y sólo si $\delta(G) = 0$.
☆ 13. Muestre que el número máximo de vértices en V que se pueden asociar con vértices de W es $|V| - \delta(G)$.
14. ¿Verdadero o falso? Cualquier acoplamiento está contenido en un acoplamiento máximo. De ser verdadero, demuéstrelo; de ser falso, proporcione un contraejemplo.

RINCÓN DE SOLUCIÓN DE PROBLEMAS:

ACOPLAMIENTO

Problema

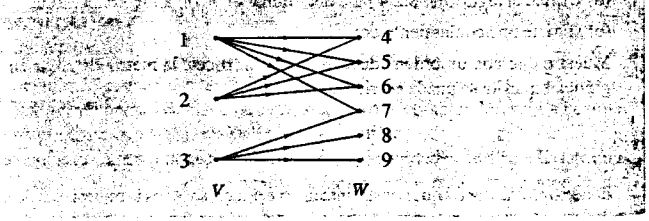
Sea G una gráfica dirigida, bipartita, con conjuntos ajenos de vértices V y W en los cuales las aristas se dirigen de los vértices en V a los vértices en W. (Cualquier vértice en G está en V o en W.) Sea M_w el grado máximo entre los vértices de W y sea m_v el grado mínimo entre los vértices de V. Mostrar que si $0 < M_w \leq m_v$, entonces G tiene un acoplamiento completo.

Para enfrentar el problema

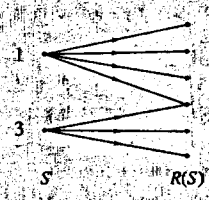
El teorema de los matrimonios de Hall (teorema 8.4.7) dice que una gráfica dirigida, bipartita con conjuntos ajenos de vértices V y W tiene un acoplamiento completo si y sólo si $|S| \leq |R(S)|$ para todo $S \subseteq V$. Así, una posible forma de resolver el problema es mostrar que la condición dada $M_w \leq m_v$ implica que $|S| \leq |R(S)|$ para todo $S \subseteq V$.

Determinación de una solución

Nuestro objetivo es demostrar que si $M_w \leq m_v$, entonces $|S| \leq |R(S)|$ para todo $S \subseteq V$. Comencemos con una gráfica ejemplo G para la cual $M_w \leq m_v$; de hecho, podemos hacer que $M_w = 2$ y $m_v = 3$:



Consideremos un subconjunto ejemplo $S = \{1, 3\}$ de V y las aristas incidentes en los vértices de S:



El hecho de que el grado mínimo de los vértices de V sea 3 significa que para *cualquier* subconjunto S de V, *al menos* tres aristas son incidentes en cada vértice de S. En general, habrá al menos $3|S| = m_v|S|$ aristas incidentes en los vértices de S. En nuestro ejemplo, $3|S| = 6$, pero en realidad hay siete aristas incidentes en los vértices de S. La expresión $m_v|S|$ siempre es una *cota inferior* para el número de aristas incidentes en los vértices de S.

El hecho de que el grado máximo de los vértices de W sea 2 significa que para *cualquier* subconjunto S de V, *a lo más* dos aristas son incidentes en cada vértice de $R(S)$. En general, habrá $2|R(S)| = M_w|R(S)|$ aristas incidentes en los vértices de $R(S)$. En nuestro ejemplo, $2|R(S)| = 12$, pero en realidad existen 10 aristas incidentes en los vértices de $R(S)$. Como las aristas incidentes en S son un subconjunto de las aristas incidentes en los vértices de $R(S)$, la expresión $M_w|R(S)|$ es siempre una *cota superior* para el número de aristas incidentes en los vértices de S.

Tenemos dos formas de estimar el número de aristas incidentes en los vértices de S. La primera forma, utilizando S, nos proporciona una cota inferior $m_v|S|$ sobre el número de aristas, y la segunda, utilizando $R(S)$, nos proporciona una cota superior $M_w|R(S)|$ sobre el número de aristas. Al comparar estas estimaciones obtenemos la desigualdad

$$m_v|S| \leq M_w|R(S)|.$$

No podemos deducir que $|S| \leq |R(S)|$, pero no hemos utilizado la hipótesis

$$M_w \leq m_v$$

aún! Al combinar las dos últimas desigualdades, tenemos

$$m_v|S| \leq M_w|R(S)| \leq m_v|R(S)|.$$

Si ahora cancelamos m_v en ambos extremos de la desigualdad, obtenemos

$$|S| \leq |R(S)|$$

que es exactamente la desigualdad que queríamos demostrar.

Solución formal

Sea $S \subseteq V$. Cada vértice en S incide en al menos m_v aristas; así, existen al menos $m_v|S|$ aristas incidentes en los vértices de S. Cada vértice en $R(S)$ incide en a lo más

M_w aristas; así, existen a lo más $M_w|R(S)|$ aristas incidentes en los vértices de $R(S)$. Esto implica que $m_v|S| \leq M_w|R(S)|$. Como $M_w \leq m_v$, tenemos que $|R(S)|M_w \leq |R(S)|m_v$. Por tanto, $m_v|S| \leq m_v|R(S)|$ y $|S| \leq |R(S)|$. Por el teorema 8.4.7, G tiene un acoplamiento completo.

Resumen de técnicas para la solución de problemas

- Analice algunas gráficas ejemplo.
- Al analizar los ejemplos, es conveniente asignar valores distintos a los parámetros del problema, de modo que pueda seguir su pista. (En nuestro ejemplo, hicimos $M_w = 2$ y $m_v = 3$.)
- Intente reducir las condiciones dadas a las de un teorema útil. (En este problema reducimos las condiciones dadas a las del teorema de los matrimonios de Hall.)
- A veces, una desigualdad se puede demostrar estimando el tamaño de cierto conjunto, de alguna forma. Si una estimación proporciona una cota superior M y otra proporciona una cota inferior m, esto implica que $m \leq M$.

Comentarios

Esta última técnica para resolver problemas proporciona un método para demostrar una desigualdad. De manera análoga, a veces se puede demostrar contando el número de elementos de cierto conjunto de dos formas distintas. Si una forma de contar da como resultado c_1 y la otra da como resultado c_2, entonces $c_1 = c_2$. Estas técnicas se utilizan ampliamente y no está de más insistir en su utilidad. Por ejemplo, en la sección 4.2 dedujimos una fórmula para $C(n, r)$ contando el número de r-permutaciones de un conjunto de n elementos de dos formas distintas.

EJERCICIO

1. Proporcione un ejemplo de una gráfica bipartita G que tenga un acoplamiento completo pero que no satisfaga la condición $M_w \leq m_v$.

$A = 1$
$B = 2$
$C = 3$
$A = A + 1$
$C = B + C$
$B = A + C$

FIGURA 8.5.1
Un programa de computadora.

8.5 *REDES DE PETRI*

Consideremos el programa de computadora de la figura 8.5.1. En general, las instrucciones se procesarían en forma secuencial; es decir, primero $A = 1$, luego $B = 2$, y así sucesivamente. Sin embargo, observe que no existe una razón lógica para evitar que las tres primeras instrucciones ($A = 1$, $B = 2$ y $C = 3$) se procesen en un orden arbitrario o de manera concurrente. Al disminuir el precio del hardware de computadora, y de los procesadores en particular, existe un creciente interés en el procesamiento concurrente, para lograr mayor velocidad y eficiencia. El uso de **redes de Petri**, gráficas modelo del procesamiento concurrente, es un método para modelar y estudiar el procesamiento concurrente.

DEFINICIÓN 8.5.1

Una *red de Petri* es una gráfica dirigida $G = (V, E)$, donde $V = P \cup T$ y $P \cap T = \emptyset$. Cualquier arista e en E es incidente en un miembro de P y un miembro de T. El conjunto P es el conjunto de *lugares* y el conjunto T es el conjunto de *transiciones*.

De manera informal, una red de Petri es una gráfica dirigida, bipartita, en la cual las dos clases de vértices se llaman lugares y transiciones. En general, en las redes de Petri se permite la existencia de aristas paralelas; sin embargo, para simplificar la exposición, aquí no permitiremos la existencia de aristas paralelas.

EJEMPLO 8.5.2

La figura 8.5.2 muestra un ejemplo de red de Petri. En general, los lugares se dibujan como círculos y las transiciones como barras o cajas rectangulares.

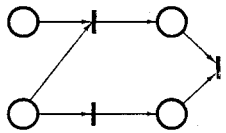


FIGURA 8.5.2 Una red de Petri. Los círculos son *lugares* y las barras son *transiciones*. □

DEFINICIÓN 8.5.3

Un *marcado* de una red de Petri asigna a cada lugar un entero no negativo. Una red de Petri con un marcado es una *red de Petri marcada* (o simplemente una red de Petri).

Si un marcado asigna el entero no negativo n al lugar p, decimos que existen n *elementos* en p. Los elementos se representan mediante puntos.

EJEMPLO 8.5.4

La figura 8.5.3 muestra un ejemplo de red de Petri marcada.

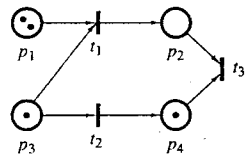


FIGURA 8.5.3 Una red de Petri marcada. □

Al modelar una situación, los lugares representan **condiciones**, las transiciones representan **eventos**, y la presencia de al menos un elemento en un lugar (condición) indica que tal condición se cumple.

EJEMPLO 8.5.5 *Modelo de red de Petri para un programa de computadora*

En la figura 8.5.4 hemos modelado el programa de computadora de la figura 8.5.1. En este caso, los eventos (transiciones) son las instrucciones, y los lugares representan las condiciones bajo las cuales se puede ejecutar una instrucción. □

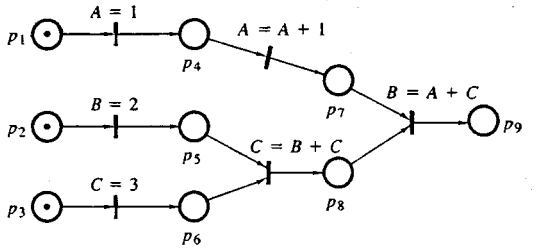


FIGURA 8.5.4 El programa de la figura 8.5.1 como una red de Petri. Los elementos indican que se cumplen las condiciones para ejecutar $A = 1$, $B = 2$ y $C = 3$.

DEFINICION 8.5.6

En una red de Petri, si una arista va del lugar p a la transición t, decimos que p es un *lugar de entrada* para la transición t. Un *lugar de salida* se define de manera análoga. Si cada lugar de entrada de una transición t tiene al menos un elemento, decimos que t está *activada*. La *descarga* de una transición elimina un elemento de cada lugar de entrada y agrega un elemento a cada lugar de salida.

EJEMPLO 8.5.7

En la red de Petri de la figura 8.5.3, los lugares p_1 y p_3 son lugares de entrada para la transición t_1. Las transiciones t_1 y t_2 están activadas, pero la transición t_3 no. Si descargamos la transición t_1, obtenemos la red de Petri marcada de la figura 8.5.5. Ahora, la transición t_3 está activada. Si entonces descargamos la transición t_3, obtenemos la red que se muestra. En este momento, ninguna transición está activada y por tanto ninguna se puede descargar. □

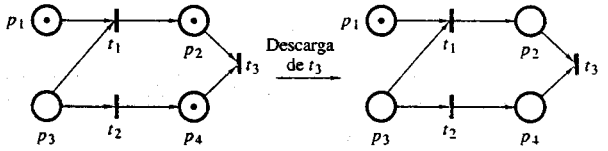


FIGURA 8.5.5 Descarga de la transición t_3.

DEFINICION 8.5.8

Si una serie de descargas transforma un marcado M en un marcado M', decimos que M' es *alcanzable* desde M.

EJEMPLO 8.5.9

En la figura 8.5.6, M'' es alcanzable desde M, descargando primero la transición t_1 y luego descargando la transición t_2. □

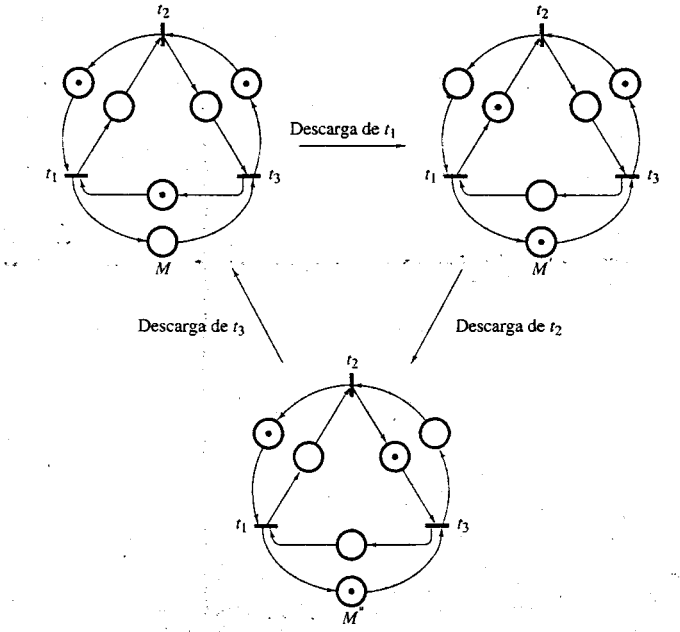


FIGURA 8.5.6 El marcado M'' es alcanzable desde M, descargando primero t_1 y luego t_2.

Al modelar una situación, la descarga de una transición simula la ocurrencia de ese evento. Por supuesto, un evento puede ocurrir sólo si se cumplen todas las condiciones para su ejecución; es decir, la transición se puede descargar sólo si está activada. Al colocar elementos en los lugares p_1, p_2 y p_3 de la figura 8.5.4, mostramos que se cumplen las condiciones para ejecutar las instrucciones $A = 1$, $B = 2$ y $C = 3$. El programa está listo para su ejecución. Como las transiciones $A = 1$, $B = 2$ y $C = 3$ están activadas, se pueden descargar en cualquier orden o de manera concurrente. La transición $C = B + C$ sólo se activa si los lugares p_5 y p_6 tienen elementos. Pero estos lugares tendrán elementos sólo si las transiciones $B = 2$ y $C = 3$ se han descargado. En otras palabras, la condición bajo la cual puede ocurrir el evento $C = B + C$ es que $B = 2$ y $C = 3$ hayan sido ejecutadas. De esta forma modelamos las secuencias válidas de ejecución de la figura 8.5.1 y la concurrencia implícita en este programa.

Entre las propiedades más importantes estudiadas en la teoría de las redes de Petri están la **supervivencia** y la **seguridad**. La supervivencia se refiere a la ausencia de estancamientos y la seguridad se relaciona con la capacidad limitada de la memoria.

EJEMPLO 8.5.10 ***Modelo de red de Petri para un sistema de cómputo compartido***

Dos personas comparten un sistema de cómputo que tiene una unidad de disco D y una impresora P. Cada persona necesita D y P. La figura 8.5.7 muestra un posible modelo de red de Petri para esta situación. El marcado indica que D y P están disponibles.

Ahora, suponga que la persona 1 solicita D y luego P (mientras tanto, la persona 2 no las solicita). Las ocurrencias de estos eventos se simulan descargando primero la transición "solicitar D" y descargando después la transición "solicitar P" para la persona 1. La red de Petri resultante aparece en la figura 8.5.8. Cuando la persona 1 termina el procesamiento y libera D y P, lo cual se simula descargando las transiciones "procesar" y luego "liberar D y P", regresamos a la red de Petri de la figura 8.5.7. Si la persona 2 solicita D y luego P (mientras que la persona 1 no las solicita), obtenemos una secuencia de descarga similar.

De nuevo, suponga que tenemos la situación de la figura 8.5.7. Ahora, suponga que la persona 1 solicita D y que la persona 2 solicita P. Después de descargar las transiciones adecuadas para simular la ocurrencia de estos eventos, obtenemos la red de Petri de la figura 8.5.9. Observe que en este punto, no se pueden descargar las transiciones. La persona 1 está esperando que la persona 2 libere a P, y la persona 2 está esperando que la persona 1 libere D. La actividad dentro del sistema se detiene. Decimos que ha ocurrido un estancamiento.

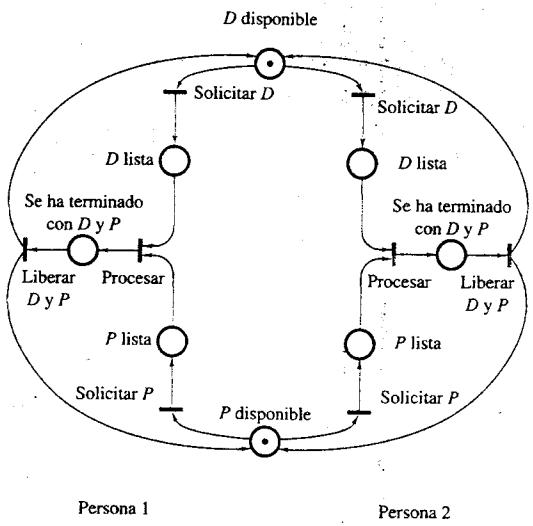


FIGURA 8.5.7 Un modelo de red de Petri para un sistema de cómputo compartido. Cada persona necesita la unidad de disco D y la impresora P. El marcado indica que D y P están disponibles.

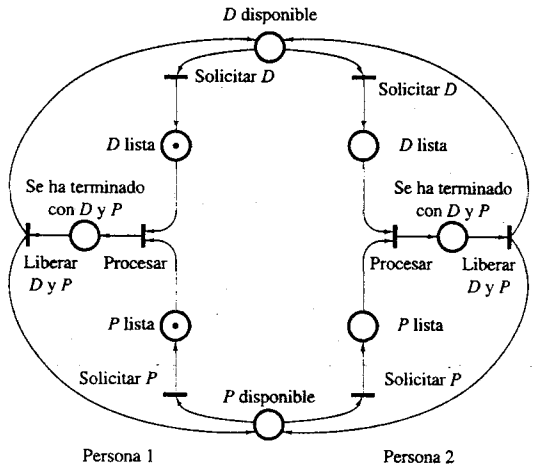


FIGURA 8.5.8 La red de Petri de la figura 8.5.7 después de descargar "solicitar D" y luego "solicitar P" para la persona 1. Después de que la persona 1 termina el procesamiento y libera D y P, lo cual se simula descargando "procesar" y luego "liberar D y P", obtenemos de nuevo la red de Petri de la figura 8.5.7.

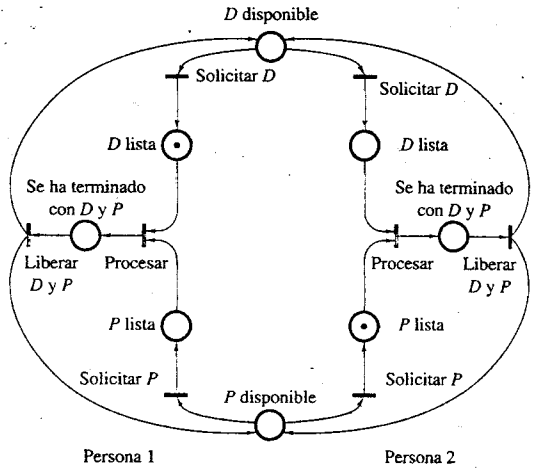


FIGURA 8.5.9 La red de Petri de la figura 8.5.7 después de descargar "solicitar D" para la persona 1 y "solicitar P" para la persona 2. En este punto, la red de Petri está estancada; es decir, ninguna transición se puede descargar.

Formalmente, decimos que una red de Petri marcada está **estancada** si ninguna transición se puede descargar. Una preocupación fundamental en los ambientes de procesamiento concurrente es la forma de evitar los estancamientos. □

El ejemplo 8.5.10 motiva la siguiente definición.

DEFINICION 8.5.11

Un marcado M de una red de Petri está *vivo* si, partiendo de M, sin importar la serie de descargas realizadas, es posible descargar cualquier transición dada mediante alguna secuencia de descargas adicionales.

Si un marcado M está vivo para una red de Petri P, entonces, sin importar la serie de descarga de transiciones, P nunca se estancará. De hecho, podemos descargar cualquier transición mediante cierta secuencia de descargas adicionales.

EJEMPLO 8.5.12

El marcado M de la red de la figura 8.5.6 está vivo. Para ver esto, observe que la única transición del marcado M que se puede descargar es t_1, la cual produce el marcado M'. La única transición del marcado M' que puede descargarse es t_2, la cual produce el marcado M''. La única transición del marcado M'' que se puede descargar es t_3, la cual nos regresa al marcado M. Así, cualquier secuencia de descarga, que parta del marcado M, produce uno de los marcados M, M' o M'', y de ahí podemos descargar cualquiera de las transiciones t_1, t_2 o t_3 procediendo como en la figura 8.5.6. Por tanto, el marcado M de la red de la figura 8.5.6 está vivo. □

EJEMPLO 8.5.13

El marcado de la figura 8.5.4 no está vivo, pues después de descargar la transición $A = 1$, ya no puede volver a descargarse. □

Si se considera que un lugar tiene capacidad limitada, el hecho de que la red sea **acotada** nos garantiza que no habrá un desbordamiento de los lugares.

DEFINICIÓN 8.5.14

Un marcado M para una red de Petri está *acotado* si existe algún entero positivo n con la propiedad de que, en cualquier secuencia de descarga, ningún lugar recibe más de n elementos. Si un marcado M está acotado y en cualquier secuencia de descarga ningún lugar recibe más de un elemento, decimos que M es un marcado *seguro*.

Si cada lugar representa un registro capaz de guardar una palabra de computadora y si un marcado inicial es seguro, tenemos garantizado que no se excederá la capacidad de memoria de los registros.

EJEMPLO 8.5.15

Los marcados de la figura 8.5.6 son seguros. El marcado M de la figura 8.5.10 no es seguro pues, como se muestra, si se descarga la transición t_1, el lugar p_2 entonces tendrá dos elementos. Al enumerar todos los marcados alcanzables desde M, se puede verificar que M está acotada y viva (véase el ejercicio 7).

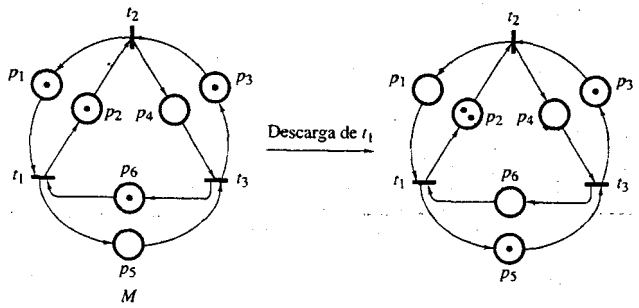


FIGURA 8.5.10 El marcado M no es seguro. Después de descargar t_1, p_2 tiene dos elementos.

Ejercicios

En los ejercicios 1-3, modele cada programa mediante una red de Petri. Proporcione un marcado que represente la situación anterior a la ejecución del programa.

1.
$A = 1$
$B = 2$
$C = A + B$
$C = C + 1$

2.
$A = 2$
$B = A + A$
$C = 3$
$D = A + A$
$C = A + B + C$

3.
$A = 1$
$S = 0$
10 $S = S + A$
$A = A + 1$
GOTO 10

4. Describa tres situaciones relacionadas con la concurrencia y que puedan modelarse como redes de Petri.
5. Proporcione un ejemplo de una red de Petri marcada en la cual se activen dos transiciones, pero que la descarga de una desactive la otra.
6. Considere el siguiente algoritmo para bañar a un león.
   1. Conseguir el león.
   2. Conseguir el jabón.
   3. Conseguir la tina de baño.
   4. Colocar agua en la tina de baño.
   5. Colocar al león en la tina de baño.

6. Bañar al león con jabón.
7. Enjuagar al león.
8. Sacar al león de la tina de baño.
9. Secar al león.

Modele este algoritmo como una red de Petri. Proporcione un marcado que represente la situación anterior a la ejecución.

7. Muestre que el marcado M de la figura 8.5.10 está vivo y acotado.

Responda las siguientes preguntas para cada una de las redes de Petri marcadas de los ejercicios 8-12.

(a) ¿Cuáles transiciones están activadas?

(b) Muestre el marcado que resulta de descargar t_1.

(c) ¿Está vivo M?

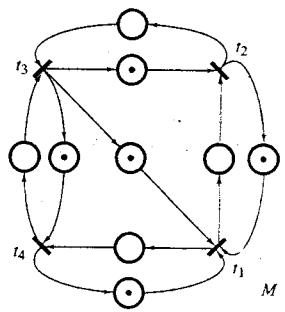
(d) ¿Es M seguro?

(e) ¿Está acotado M?

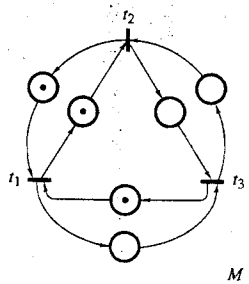
(f) Muestre o describa todos los marcados alcanzables desde M.

(g) Exhiba un marcado (distinto del marcado que coloca cero elementos en cada lugar) no alcanzable desde M.

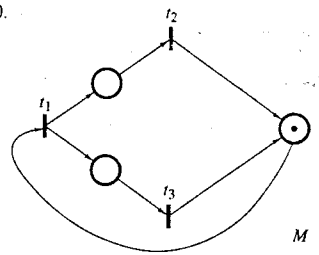
8.



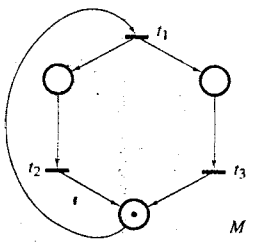
9.



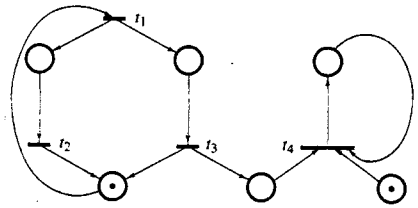
10.



11.



☆ **12.**



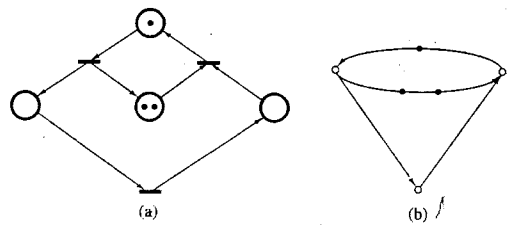
13. Proporcione un ejemplo de una red de Petri con un marcado seguro, pero no vivo.

14. Proporcione un ejemplo de una red de Petri con un marcado acotado, pero no seguro.

15. El **problema de la comida de los filósofos** (véase [Dijkstra, 1968]) se relaciona con cinco filósofos sentados ante una mesa redonda. Cada filósofo come o medita. La mesa está puesta de manera alternada, con un plato y un palillo para comida china. La comida requiere dos palillos, de modo que si cada filósofo toma el palillo a la derecha del plato, ninguno podrá comer (el sistema se estanca). Modele esta situación como una red de Petri. Su modelo debe estar vivo, de modo que el sistema no se estanque y de modo que, en cada momento, cualquier filósofo tenga la posibilidad de comer o meditar.

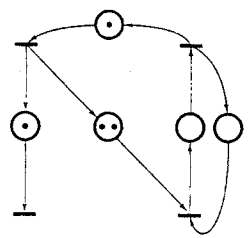
16. Desarrolle una red de Petri alternativa a la situación del ejemplo 8.5.10 que evite el estancamiento.

Si cada lugar de una red de Petri marcada P tiene una arista de entrada y una de salida, entonces podemos trazar P como una gráfica dirigida, donde los vértices corresponden a las transiciones y las aristas a los lugares. Los elementos se colocan sobre las aristas. Tal gráfica es una *gráfica marcada*. A continuación mostramos una red de Petri marcada y su representación como una gráfica marcada.

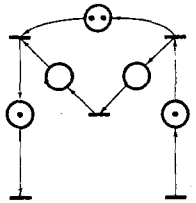


17. ¿Cuáles de las redes de Petri de los ejercicios 8-17 se pueden trazar como gráficas marcadas?

18. Trace la siguiente red de Petri marcada como una gráfica marcada.



19. Trace la siguiente red de Petri marcada como una gráfica marcada.



El *número de elementos* de un ciclo dirigido simple en una gráfica simple es el número de elementos sobre todas las aristas del ciclo.

20. Muestre que el número de elementos de un ciclo dirigido simple no cambia durante cualquier secuencia de descarga.

☆ 21. Muestre que un marcado M para una gráfica marcada G está vivo si y sólo si M coloca al menos un elemento en cada ciclo dirigido simple en G.

☆ 22. Muestre que un marcado vivo es seguro para una gráfica marcada G si y sólo si cada arista de G pertenece a un ciclo dirigido simple, con número de elementos igual a 1.

23. Proporcione un ejemplo de una gráfica marcada con un marcado no vivo, en la cual cada arista pertenezca a un ciclo dirigido simple, con número de elementos igual a 1.

24. Sea G una gráfica marcada. Muestre que cada arista en G está contenida en un ciclo dirigido simple si y sólo si cada marcado para G está acotado.

☆ 25. Sea G una gráfica dirigida tal que, si ignoramos la dirección de las aristas en G, G sea conexa como gráfica no dirigida. Muestre que G tiene un marcado vivo y seguro si y sólo si dados cualesquiera dos vértices v y w en G, existe un camino dirigido de v a w.

NOTAS

Las referencias generales con secciones acerca de los modelos de redes son [Berge; Deo; Liu, 1968, 1985; y Tucker]. La obra clásica acerca de las redes es [Ford]; muchos de los resultados de redes, en particular los primeros resultados, se deben a Ford y Fulkerson, los autores de ese libro. [Tarjan] analiza los algoritmos para el flujo en una red y los detalles de su implantación.

Las redes de Petri surgieron en la tesis doctoral de C. Petri [Petri] en 1962. Desde entonces ha habido mucha investigación acerca de sus propiedades y un gran interés en su uso para modelar sistemas reales.

El problema de determinar un flujo máximo en una red G, con fuente a, sumidero z y capacidades C_{ij}, se puede parafrasear como sigue:

$$\text{maximizar} \sum_j F_{aj}$$

sujeta a

$$0 \le F_{ij} \le C_{ij} \quad \text{para toda } i, j;$$

$$\sum_i F_{ij} = \sum_i F_{ji} \quad \text{para toda } j.$$

Tal problema es un ejemplo de **problema de programación lineal**. En un problema de programación lineal, queremos maximizar (o minimizar) una expresión lineal, como $\Sigma_j F_{aj}$, sujeta a restricciones lineales en forma de igualdades o desigualdades, como $0 \le F_{ij} \le C_{ij}$ y $\Sigma_i F_{ij} = \Sigma_i F_{ji}$. Aunque el **algoritmo simplex** es en general una manera eficiente de resolver un problema general de programación lineal, es usual que los problemas de redes de transporte se resuelvan de manera más eficiente mediante el algoritmo 8.2.4. Véase [Hillier] para una exposición del algoritmo símplex.

Suponga que para cada arista (i, j) en una red G, c_{ij} representa el costo de flujo de una unidad a través de la arista (i, j). Suponga que queremos determinar un flujo máximo, con costo mínimo

$$\sum_i \sum_j c_{ij} F_{ij}.$$

Este problema, llamado el **problema de transporte**, es nuevamente un problema de programación lineal y, como en el caso del problema del flujo máximo, se puede utilizar un algoritmo específico para obtener una solución, de manera más eficiente que el algoritmo símplex (véase [Hillier]).

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 8.1

Red (de transporte)
Fuente
Sumidero
Capacidad
Flujo en una red
Flujo en una arista
Flujo de entrada de un vértice
Flujo de salida de un vértice
Conservación del flujo
Dado un flujo F en una red, el flujo de salida de la fuente es igual al flujo de entrada en el sumidero. Este valor común es el valor del flujo F.
Superfuente
Supersumidero

Sección 8.2

Flujo máximo
Arista orientada en forma propia con respecto de un camino
Arista orientada en forma impropia con respecto de un camino
Cómo incrementar el flujo en un camino de la fuente al sumidero cuando:
  (a) para cada arista orientada en forma propia, el flujo es menor que la capacidad y
  (b) cada arista orientada en forma impropia tiene flujo positivo (véase el teorema 8.2.3)
Cómo determinar un flujo máximo en una red (algoritmo 8.2.4)

Sección 8.3

Corte en una red
Capacidad de un corte
La capacidad de cualquier corte es mayor o igual que el valor de cualquier flujo (teorema 8.3.7)
Corte mínimo
Teorema del flujo máximo y el corte mínimo (teorema 8.3.9)
Al concluir el algoritmo del flujo máximo, algoritmo 8.2.4, el conjunto de vértices etiquetados define un corte mínimo

Sección 8.4

Acoplamiento
Acoplamiento máximo
Acoplamiento completo
Red de acoplamiento
Relación entre los flujos y el acoplamiento (teorema 8.4.5)
Teorema de los matrimonios de Hall (teorema 8.4.7)

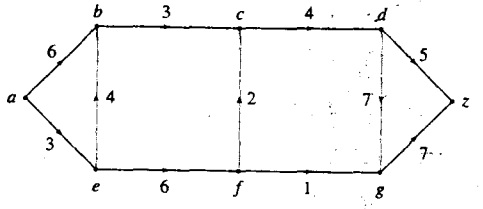
Sección 8.5

Red de Petri (marcada)
Lugar
Transición
Marcado
Condiciones y eventos
Lugar de entrada
Lugar de salida
Transición activada
Descarga de una transición activada
Marcado alcanzable
Estancamiento
Marcado vivo
Marcado acotado
Marcado seguro

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 8.1

Los ejercicios 1-4 se refieren a la siguiente red. Las capacidades aparecen sobre las aristas.



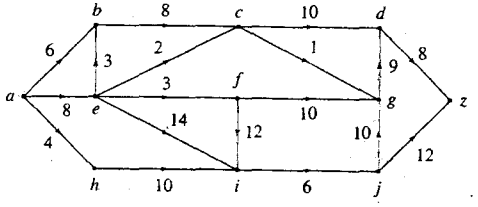
1. Explique por qué

$$F_{a,e} = 2, \quad F_{e,b} = 2, \quad F_{b,c} = 3, \quad F_{c,d} = 3, \quad F_{d,z} = 3, \quad F_{a,b} = 1,$$

con las demás $F_{x,y} = 0$, es un flujo.
2. ¿Cuál es el flujo de entrada de b?
3. ¿Cuál es el flujo de salida de c?
4. ¿Cuál es el valor del flujo F?

Sección 8.2

5. Para el flujo del ejercicio 1, determine un camino de a a z tal que (a) para cada arista orientada en forma propia, el flujo sea menor que la capacidad y (b) cada arista orientada en forma impropia tenga flujo positivo.
6. Modifique sólo los flujos en las aristas del camino del ejercicio 5 y determine un flujo con un valor mayor que F.
7. Utilice el algoritmo 8.2.4 para determinar un flujo máximo en la red del ejercicio 1 (comience con el flujo tal que el flujo en cada arista sea igual a cero).
8. Utilice el algoritmo 8.2.4 para determinar un flujo máximo en la siguiente red (comience con el flujo tal que el flujo en cada arista sea igual a cero).



Sección 8.3

9. En cada una de las partes (a)–(d), responda verdadero si la afirmación es verdadera para cualquier red; en caso contrario, responda falso.

   (a) Si la capacidad de un corte en una red es igual a Ca, entonces el valor de cualquier flujo es menor o igual a Ca.

   (b) Si la capacidad de un corte en una red es igual a Ca, entonces el valor de cualquier flujo es mayor o igual a Ca.

   (c) Si la capacidad de un corte en una red es igual a Ca, entonces el valor de algún flujo es mayor o igual a Ca.

   (d) Si la capacidad de un corte en una red es igual a Ca, entonces el valor de algún flujo es menor o igual a Ca.

10. Determine la capacidad del corte $(P, \overline{P})$ en la red del ejercicio 1, donde $P = \{a, b, e, f\}$.
11. ¿Es mínimo el corte $(P, \overline{P})$, $P = \{a, b, e, f\}$, en la red del ejercicio 1? Explique.
12. Determine un corte mínimo en la red del ejercicio 8.

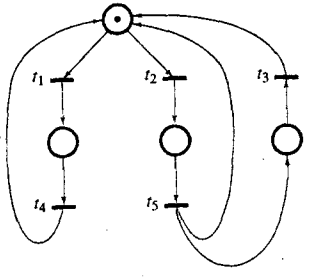
Sección 8.4

Los ejercicios 13-16 se refieren a la siguiente situación. El solicitante A está calificado para los trabajos J_2, J_4 y J_5; el solicitante B está calificado para los trabajos J_1 y J_3; el solicitante C está calificado para los trabajos J_1, J_3 y J_5; y el solicitante D está calificado para los trabajos J_3 y J_5.

13. Modele esta situación como una red de acoplamiento.
14. Utilice el algoritmo 8.2.4 para determinar un acoplamiento máximo.
15. ¿Existe un acoplamiento completo?
16. Determine un corte mínimo en la red de acoplamiento.

Sección 8.5

Los ejercicios 17-20 se refieren a la siguiente red de Petri.



17. ¿Cuáles transiciones están activadas?
18. Muestre el marcado resultante al descargar t_2.
19. ¿Es seguro el marcado?
20. ¿Está vivo el marcado?

Esta cuatro memas
Uno no puede llegar a este
estado mental

—de The Late George Apley

10

AUTÓMATAS, GRAMÁTICAS Y LENGUAJES

10.1 CIRCUITOS SECUENCIALES Y MÁQUINAS DE ESTADO FINITO
10.2 AUTÓMATAS DE ESTADO FINITO
10.3 LENGUAJES Y GRAMÁTICAS
10.4 AUTÓMATAS DE ESTADO FINITO NO DETERMINISTAS
10.5 RELACIONES ENTRE LENGUAJES Y AUTÓMATAS
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

En el capítulo 9 analizamos los circuitos combinatorios donde la salida sólo dependía de la entrada. Estos circuitos no tienen memoria. En este capítulo iniciamos el estudio de los circuitos donde la salida no sólo depende de la entrada sino también del estado del sistema en el momento en que se introduce la entrada. El estado del sistema queda determinado por el procesamiento anterior. En este sentido, estos circuitos tienen memoria. Tales circuitos se llaman *circuitos secuenciales* y tienen una importancia obvia en el diseño de computadoras.

Las máquinas de estado finito son modelos abstractos de máquinas con una memoria interna primitiva. Un autómata de estado finito es un tipo particular de máquina de estado finito que está íntimamente ligada a un tipo particular de lenguaje. En la última parte de este capítulo analizaremos con cierto detalle las máquinas de estado finito, los autómatas de estado finito y los lenguajes.

10.1 *CIRCUITOS SECUENCIALES Y MÁQUINAS DE ESTADO FINITO*

Las operaciones dentro de una computadora digital se realizan a intervalos discretos de tiempo. La salida depende del estado del sistema, así como de la entrada. Supondremos que el estado del sistema cambia solamente en los instantes $t = 0, 1, \ldots$. Una forma sencilla de introducir la secuenciación en los circuitos consiste en utilizar un **retraso unitario de tiempo**.

DEFINICIÓN 10.1.1

Un *retraso unitario de tiempo* acepta como entrada un bit x_t en el instante t y tiene como salida x_{t-1}, el bit recibido como entrada en el instante $t - 1$. El retraso unitario de tiempo aparece en la figura 10.1.1.

Como ejemplo del uso del retraso unitario de tiempo, analizaremos el **sumador en serie**.

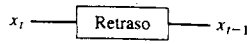


FIGURA 10.1.1
Retraso unitario de tiempo.

DEFINICIÓN 10.1.2

Un *sumador en serie* acepta como entrada dos números binarios

$$x = 0x_N x_{N-1} \cdots x_0 \quad \text{y} \quad y = 0y_N y_{N-1} \cdots y_0$$

y produce como salida la suma $z_{N+1} z_N \cdots z_0$ de x y y. Los números x y y se introducen de manera secuencial por pares, $x_0, y_0; \ldots; x_N, y_N; 0, 0$. La suma sale como $z_0, z_1, \ldots, z_{N+1}$.

EJEMPLO 10.1.3 *Circuito sumador en serie*

La figura 10.1.2 muestra un circuito que utiliza un retraso unitario de tiempo para implantar un sumador en serie.

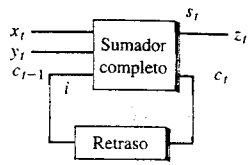


FIGURA 10.1.2
Un circuito sumador en serie.

Ahora mostraremos la forma en que el sumador en serie calcula la suma de

$$x = 010 \quad \text{y} \quad y = 011.$$

Primero hacemos $x_0 = 0$ y $y_0 = 1$. (Suponemos que en este instante $i = 0$. Esto se puede arreglar haciendo primero que $x = y = 0$.) El estado del sistema aparece en la figura 10.1.3a. A continuación, hacemos $x_1 = y_1 = 1$. El retraso unitario de tiempo envía $i = 0$ como el tercer bit al sumador completo. El estado del sistema aparece en la figura 10.1.3b. Por último, hacemos $x_2 = y_2 = 0$. Esta vez, el retraso unitario de tiempo envía $i = 1$ como tercer bit al sumador completo. El estado del sistema aparece en la figura 10.1.3c. Obtenemos la suma $z = 101$.

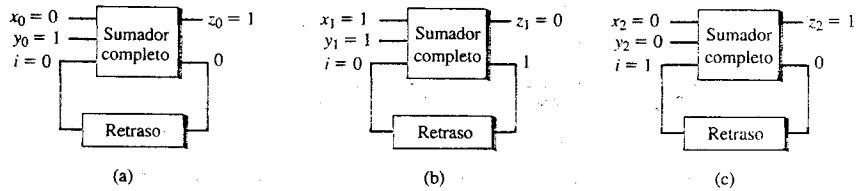


FIGURA 10.1.3 Cálculo de 010 + 011 con el circuito sumador en serie. □

Una **máquina de estado** finito es un modelo abstracto de una máquina con una memoria interna primitiva.

DEFINICION 10.1.4

Una *máquina de estado finito* M consta de

(a) Un conjunto finito I de *símbolos de entrada.*
(b) Un conjunto finito O de *símbolos de salida.*
(c) Un conjunto finito S de *estados.*
(d) Una *función de estado siguiente* f de $S \times I$ en S.
(e) Una *función de salida* g de $S \times I$ en O.
(f) Un *estado inicial* $\sigma \in S$.

Escribimos $M = (I, O, S, f, g, \sigma)$.

EJEMPLO 10.1.5

Sean $I = \{a, b\}$, $O = \{0, 1\}$ y $S = \{\sigma_0, \sigma_1\}$. Definimos el par de funciones $f: S \times I \to S$ y $g: S \times I \to O$ mediante las reglas dadas en la tabla 10.1.1.

TABLA 10.1.1

	f		g	
S \ I	a	b	a	b
σ_0	σ_0	σ_1	0	1
σ_1	σ_1	σ_1	1	0

Entonces $M = (I, O, S, f, g, \sigma_0)$ es una máquina de estado finito.

La tabla 10.1.1 se interpreta como sigue:

$$f(\sigma_0, a) = \sigma_0 \quad g(\sigma_0, a) = 0,$$
$$f(\sigma_0, b) = \sigma_1 \quad g(\sigma_0, b) = 1,$$
$$f(\sigma_1, a) = \sigma_1 \quad g(\sigma_1, a) = 1,$$
$$f(\sigma_1, b) = \sigma_1 \quad g(\sigma_1, b) = 0. \qquad \square$$

Las funciones de estado siguiente y de salida también se pueden definir mediante un **diagrama de transición**. Antes de definir este concepto, ilustraremos la forma de construirlo.

EJEMPLO 10.1.6

Dibujar el diagrama de transición para la máquina de estado finito del ejemplo 10.1.5.

El diagrama de transición es una digráfica. Los vértices son los estados (véase la figura 10.1.4). El estado inicial se indica mediante una flecha, como se muestra en la figura. Si estamos en el estado σ y la entrada i produce la salida o y nos pasa al estado σ', trazamos una arista dirigida del vértice σ al vértice σ' y la etiquetamos como i/o. Por ejemplo, si estamos en el estado σ_0, y utilizamos la entrada a, la tabla 10.1.1 nos dice que producimos la salida 0 y permanecemos en el estado σ_0. Así, trazamos un lazo dirigido sobre el vértice σ_0 y lo etiquetamos $a/0$ (véase la figura 10.1.4).

Por otro lado, si estamos en el estado σ_0 y utilizamos la entrada b, la salida es 1 y pasamos al estado σ_1. Así, trazamos una arista dirigida de σ_0 a σ_1 y la etiquetamos $b/1$. Al considerar todas las posibilidades, obtenemos el diagrama de transición de la figura 10.1.4.

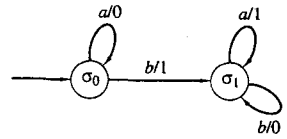


FIGURA 10.1.4 Un diagrama de transición. □

DEFINICION 10.1.7

Sea $M = (I, O, S, f, g, \sigma)$ una máquina de estado finito. El *diagrama de transición* de M es una digráfica G cuyos vértices son los miembros de S. Una flecha indica el estado inicial σ. Una arista dirigida (σ_1, σ_2) existe en G si existe una entrada i tal que $f(\sigma_1, i) = \sigma_2$. En este caso, si $g(\sigma_1, i) = o$, la arista (σ_1, σ_2) se etiqueta i/o.

Podemos considerar a la máquina de estado finito $M = (I, O, S, f, g, \sigma)$ como una computadora sencilla. Partimos del estado σ, introducimos una cadena sobre I, y producimos una cadena de salida.

DEFINICION 10.1.8

Sea $M = (I, O, S, f, g, \sigma)$ una máquina de estado finito. Una *cadena de entrada* para M es una cadena en I. La cadena

$$y_1 \cdots y_n$$

es la *cadena de salida* para M correspondiente a la cadena de entrada

$$\alpha = x_1 \cdots x_n$$

si existen estados $\sigma_0, \ldots, \sigma_n \in S$ tales que

$$\sigma_0 = \sigma$$
$$\sigma_i = f(\sigma_{i-1}, x_i) \quad \text{para } i = 1, \ldots, n;$$
$$y_i = g(\sigma_{i-1}, x_i) \quad \text{para } i = 1, \ldots, n.$$

EJEMPLO 10.1.9

Determinar la cadena de salida correspondiente a la cadena de entrada

$$aababba \tag{10.1.1}$$

para la máquina de estado finito del ejemplo 10.1.5.

Al principio, estamos en el estado σ_0. La primera entrada es a. En el diagrama de transición de M (figura 10.1.4) localizamos la arista que sale de σ_0 con la etiqueta a/x, la cual nos dice que si a es la entrada, x es la salida. En nuestro caso, 0 es la salida. La arista apunta hacia el estado siguiente, σ_0. Ahora, a es nuevamente la entrada. Como antes, producimos la salida 0 y permanecemos en el estado σ_0. Ahora, la entrada es b. En este caso, la salida es 1 y pasamos al estado σ_1. Continuamos de esta forma para determinar que la cadena de salida es

$$0011001. \tag{10.1.2}$$

□

EJEMPLO 10.1.10 *Una máquina de estado finito para el sumador en serie*

Diseñar una máquina de estado finito que realice la suma en serie.

Representaremos la máquina de estado finito mediante su diagrama de transición. Como el sumador en serie acepta pares de bits, el conjunto de entrada será

$$\{00, 01, 10, 11\}.$$

El conjunto de salida es

$$\{0, 1\}.$$

Dada una entrada xy, realizamos una de las dos acciones siguientes: sumamos x y y, o sumamos x, y y 1, según si el bit de acarreo era 0 o 1. Así, existen dos estados, que llamaremos C (acarreo) y NC (sin acarreo). El estado inicial es NC. En este momento, podemos dibujar los vértices y designar el estado inicial en nuestro diagrama de transición (véase la figura 10.1.5).

FIGURA 10.1.5
Dos estados de la máquina de estado finito para el sumador en serie.

Ahora, consideremos las entradas posibles en cada vértice. Por ejemplo, si 00 es la entrada en NC, producimos como salida 0 y permanecemos en el estado NC. Así, NC tiene un lazo etiquetado 00/0. Como otro ejemplo, si 11 es la entrada de C, calculamos $1 + 1 + 1 = 11$. En este caso, la salida es 1 y permanecemos en el estado C. Así, C tiene un lazo etiquetado 11/1. Como ejemplo final, si estamos en el estado NC y la entrada es 11, producimos como salida 0 y pasamos al estado C. Al considerar todas las posibilidades, llegamos al diagrama de transición de la figura 10.1.6.

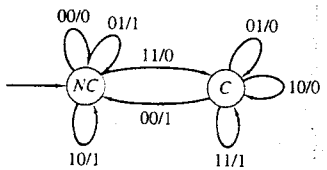


FIGURA 10.1.6 Una máquina de estado finito que realiza la suma en serie. □

EJEMPLO 10.1.11 *El flip-flop SR*

Un **flip-flop** es un componente básico de los circuitos digitales, pues sirve como una celda de memoria de un bit. El **flip-flop** SR (o **flip-flop set-reset**) se puede definir mediante la tabla siguiente

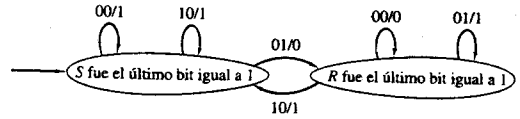


FIGURA 10.1.7 El flip-flop SR como una máquina de estado finito.

S	R	Q
1	1	No permitida
1	0	1
0	1	0
0	0	1 si S fue el último igual a 1 0 si R fue el último igual a 1

El flip-flop SR "recuerda" si S o R fue el último bit igual a 1. (Si $Q = 1$, S fue el último bit igual a 1; si $Q = 0$, R fue el último bit igual a 0.) Podemos modelar el flip-flop SR como una máquina de estado finito, definiendo dos estados: "S fue el último bit igual a 1" y "R fue el último bit igual a 1" (véase la figura 10.1.7). Definimos la entrada como los nuevos valores de S y R; la notación SR indica que $S = s$ y $R = r$. Definimos Q como la salida. De manera arbitraria hemos designado al estado inicial como "S fue el último bit igual a 1". La figura 10.1.8 muestra una implantación del flip-flop SR mediante un circuito secuencial.

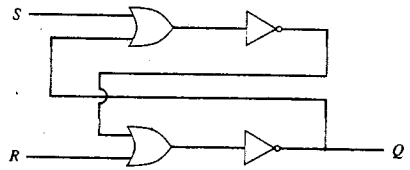


FIGURA 10.1.8 Una implantación del flip-flop SR mediante un circuito secuencial. □

Ejercicios

En los ejercicios 1-5, dibuje el diagrama de transición de la máquina de estado finito (I, O, S, f, g, σ_0).

1. $I = \{a, b\}$, $O = \{0, 1\}$, $S = \{\sigma_0, \sigma_1\}$

	f		g	
S \ I	a	b	a	b
σ_0	σ_1	σ_1	1	1
σ_1	σ_0	σ_1	0	1

2. $I = \{a, b\}$, $O = \{0, 1\}$, $S = \{\sigma_0, \sigma_1\}$

	f		g	
S \ I	a	b	a	b
σ_0	σ_1	σ_0	0	0
σ_1	σ_0	σ_0	1	1

3. $I = \{a, b\}$, $O = \{0, 1\}$, $S = \{\sigma_0, \sigma_1, \sigma_2\}$

	f		g	
S \ I	a	b	a	b
σ_0	σ_1	σ_1	0	1
σ_1	σ_2	σ_1	1	1
σ_2	σ_0	σ_0	0	0

4. $I = \{a, b, c\}$, $O = \{0, 1\}$, $S = \{\sigma_0, \sigma_1, \sigma_2\}$

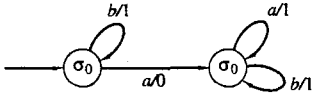
	f			g		
S \ I	a	b	c	a	b	c
σ_0	σ_0	σ_1	σ_2	0	1	0
σ_1	σ_1	σ_1	σ_0	1	1	1
σ_2	σ_2	σ_1	σ_0	1	0	0

5. $I = \{a, b, c\}$, $O = \{0, 1, 2\}$, $S = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$

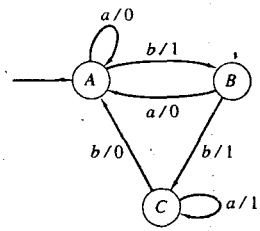
	f			g		
S \ I	a	b	c	a	b	c
σ_0	σ_1	σ_0	σ_2	1	1	2
σ_1	σ_0	σ_2	σ_2	2	0	0
σ_2	σ_3	σ_3	σ_0	1	0	1
σ_3	σ_1	σ_1	σ_0	2	0	2

En los ejercicios 6-10, determine los conjuntos I, O y S, el estado inicial y la tabla que define las funciones de estado siguiente y de salida para cada máquina de estado finito.

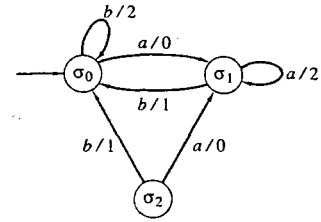
6.



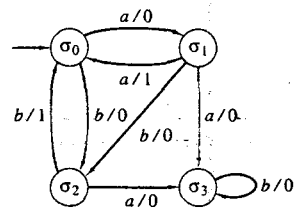
7.



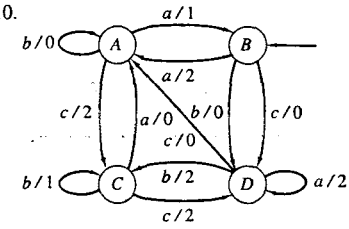
8.



9.



10.



En los ejercicios 11-20, determine la cadena de salida para la cadena de entrada y la máquina de estado finito dadas.

11. *abba*; ejercicio 1
12. *abba*; ejercicio 2
13. *aabbaba*; ejercicio 3
14. *aabbcc*; ejercicio 4
15. *aabaab*; ejercicio 5
16. *aaa*; ejercicio 6
17. *aabbabaab*; ejercicio 7
18. *baaba*; ejercicio 8
19. *bbababbabaaa*; ejercicio 9
20. *cacbccbaabac*; ejercicio 10

En los ejercicios 21-26, diseñe una máquina de estado finito con las propiedades dadas. La entrada siempre es una cadena de bits.

21. Produce la salida 1 si la entrada es un número par de bits; produce la salida 0 en caso contrario.
22. Produce la salida 1 si la entrada son k unos, donde k es un múltiplo de 3; produce la salida 0 en caso contrario.
23. Produce la salida 1 si la entrada son dos o más unos; produce la salida 0 en caso contrario.
24. Produce la salida 1 siempre que vea 101; produce la salida 0 en caso contrario.
25. Produce la salida 1 a partir de que vea 101; produce la salida 0 en caso contrario.
26. Produce la salida 1 cuando ve el primer 0 y hasta ver otro 0; a partir de ese momento, produce la salida 0; en los demás casos, produce la salida 0.
27. Sea $\alpha = x_1 \cdots x_n$ una cadena de bits. Sea $\beta = y_1 \cdots y_n$, donde

$$y_i = \begin{cases} a & \text{si } x_i = 0 \\ b & \text{si } x_i = 1 \end{cases}$$

para $i = 1, \ldots, n$. Sea $\gamma = y_n \cdots y_1$.

Muestre que si γ es la entrada de la máquina de estado finito de la figura 10.1.4, la salida es el complemento a dos de α (véase el algoritmo 9.5.16 para una descripción del complemento a dos).

Entrada	*Salida*
11	1
00	0
10	1
01	1
00	0
00	1
00	0
00	0

☆ 28. Muestre que no existe una máquina de estado finito que reciba una cadena de bits y produzca como salida 1 siempre que el número de unos de entrada sea igual al número de ceros de entrada y produzca 0 en caso contrario.

☆ 29. Muestre que no existe una máquina de estado finito que realice la multiplicación en serie. En específico, muestre que no existe una máquina de estado finito que reciba como entrada los números binarios $X = x_1 \cdots x_n$ y $Y = y_1 \cdots y_n$ como la secuencia de números de dos bits

$$x_n y_n, \quad x_{n-1} y_{n-1} \quad \ldots, \quad x_1 y_1, \quad 00, \quad \ldots, \quad 00,$$

donde existen n 00, y las salidas $z_{2n}, \ldots z_1$, donde $Z = z_1 \cdots z_{2n} = XY$. *Ejemplo*: Si tal máquina existiese, para multiplicar 101 × 1001 habría que introducir como entrada 11, 00, 10, 01, 00, 00, 00, 00. El primer par 11 es el par de los bits del extremo derecho (10<u>1</u>, 100<u>1</u>); el segundo par 00 es el siguiente par de bits (1<u>0</u>1, 10<u>0</u>1); y así sucesivamente. Rellenamos la cadena de entrada con cuatro pares de 00, la longitud del factor de mayor longitud, 1001. Como 101 × 1001 = 101101, se está afirmando que se obtiene la salida de la tabla anexa.

10.2 AUTÓMATAS DE ESTADO FINITO

Un **autómata de estado finito** es un tipo particular de máquina de estado finito. Los autómatas de estado finito tienen un interés especial, debido a su relación con los lenguajes, como veremos en la sección 10.5.

DEFINICIÓN 10.2.1

Un *autómata de estado finito* $A = (I, O, S, f, g, \sigma)$ es una máquina de estado finito en la que el conjunto de símbolos de salida es $\{0, 1\}$ y donde el estado actual determina la última salida. Aquellos estados para los cuales la última salida es 1 son los *estados de aceptación*.

EJEMPLO 10.2.2

Dibujar el diagrama de transición de la máquina de estado finito A definida mediante la siguiente tabla. El estado inicial es σ_0. Mostrar que A es un autómata de estado finito y determinar el conjunto de estados de aceptación.

	f		g	
I / S	a	b	a	b
σ_0	σ_1	σ_0	1	0
σ_1	σ_2	σ_0	1	0
σ_2	σ_2	σ_0	1	0

El diagrama de transición aparece en la figura 10.2.1. Si estamos en el estado σ_0, la última salida fue 0. Si estamos en el estado σ_1 o en el estado σ_2, la última salida fue 1; así, A es un autómata de estado finito. Los estados de aceptación son σ_1 y σ_2.

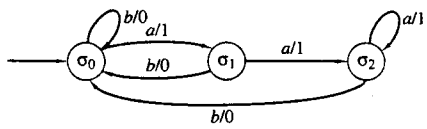


FIGURA 10.2.1 El diagrama de transición para el ejemplo 10.2.2. □

El ejemplo 10.2.2 muestra que la máquina de estado finito definida mediante un diagrama de transición será un autómata de estado finito si el conjunto de símbolos de salida es $\{0, 1\}$ y si, para cada estado σ, todas las aristas que llegan a σ tienen la misma etiqueta de salida.

Por lo general, el diagrama de transición de un autómata de estado finito se dibuja con los estados de aceptación encerrados en círculos dobles y omitiendo los símbolos de salida. Al dibujar el diagrama de transición de la figura 10.2.1 de esta forma, obtenemos el diagrama de transición de la figura 10.2.2.

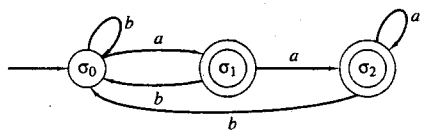


FIGURA 10.2.2 El diagrama de transición de la figura 10.2.1, con los estados de aceptación encerrados en círculos dobles y omitiendo los símbolos de salida.

EJEMPLO 10.2.3

Dibujar el diagrama de transición del autómata de estado finito de la figura 10.2.3 como el diagrama de transición de una máquina de estado finito.

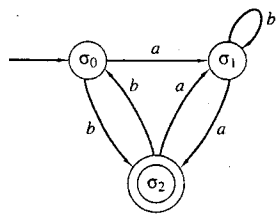


FIGURA 10.2.3 Un autómata de estado finito.

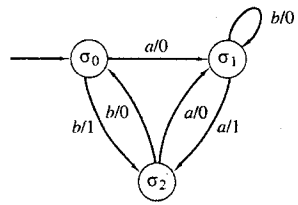


FIGURA 10.2.4 El autómata de estado finito de la figura 10.2.3, ahora como un diagrama de transición de una máquina de estado finito.

Como σ_2 es un estado de aceptación, etiquetamos todas sus aristas de entrada con la salida 1 (véase la figura 10.2.4). Los estados σ_0 y σ_1 no son de aceptación, así que etiquetamos todas sus aristas de entrada con la salida 0. Obtenemos el diagrama de transición de la figura 10.2.4. □

Como una alternativa a la definición 10.2.1, podemos considerar a un autómata de estado finito A como formado por

1. Un conjunto finito I de *símbolos de entrada.*
2. Un conjunto finito S de *estados.*
3. Una *función de estado siguiente* f de $S \times I$ en S.
4. Un subconjunto A de S de *estados de aceptación.*
5. Un estado inicial $\sigma \in S$.

Si utilizamos esta caracterización, escribimos $A = (I, S, f, A, \sigma)$.

EJEMPLO 10.2.4

El diagrama de transición del autómata de estado finito $A = (I, S, f, A, \sigma)$, donde

$$I = \{a, b\}; \quad S = \{\sigma_0, \sigma_1, \sigma_2\}; \quad A = \{\sigma_2\}; \quad \sigma = \sigma_0;$$

y f está dada por la siguiente tabla.

	f	
S \ I	a	b
σ_0	σ_0	σ_1
σ_1	σ_0	σ_2
σ_2	σ_0	σ_2

aparece en la figura 10.2.5.

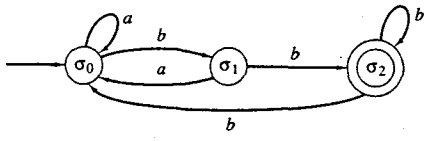


FIGURA 10.2.5 El diagrama de transición para el ejemplo 10.2.4. □

Si una cadena se utiliza como entrada de un autómata de estado finito, terminaremos en un estado de aceptación o en uno de no aceptación. La situación de este estado final determina si la cadena es **aceptada** por el autómata de estado finito.

DEFINICIÓN 10.2.5

Sea $A = (I, S, f, A, \sigma)$ un autómata de estado finito. Sea $\alpha = x_1 \cdots x_n$ una cadena sobre I. Si existen estados $\sigma_0, \ldots, \sigma_n$ tales que

(a) $\sigma_0 = \sigma$;

(b) $f(\sigma_{i-1}, x_i) = \sigma_i$ para $i = 1, \ldots, n$;

(c) $\sigma_n \in A$;

decimos que α es *aceptada por* A. La cadena nula es aceptada si y sólo si $\sigma \in A$. Sea $\mathrm{Ac}(A)$ el conjunto de cadenas aceptadas por A; decimos que A *acepta* $\mathrm{Ac}(A)$.

Sea $\alpha = x_1 \cdots x_n$ una cadena sobre I. Definimos los estados $\sigma_0, \ldots, \sigma_n$ mediante las condiciones (a) y (b) anteriores. Decimos que el camino (dirigido) $(\sigma_0, \ldots, \sigma_n)$ es el camino que *representa* a α en A.

La definición 10.2.5 implica que si el camino P representa a la cadena α en un autómata de estado finito A, entonces A acepta a α si y sólo si P termina en un estado de aceptación.

EJEMPLO 10.2.6

¿Es aceptada la cadena *abaa* por el autómata de estado finito de la figura 10.2.2?

Comenzamos en el estado σ_0. Cuando a es la entrada, pasamos al estado σ_1. Cuando b es la entrada, vamos al estado σ_0. Cuando a es la entrada, pasamos al estado σ_1. Por último, cuando se utiliza como entrada el último símbolo a, pasamos al estado σ_2. El camino $(\sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_2)$ representa la cadena *abaa*. Como el estado final σ_2 es un estado de aceptación, la cadena *abaa* es aceptada por el autómata de estado finito de la figura 10.2.2. □

EJEMPLO 10.2.7

¿Es aceptada la cadena $\alpha = abbabba$ por el autómata de estado finito de la figura 10.2.3?

El camino que representa a α termina en σ_1. Como σ_1 no es un estado de aceptación, la cadena α no es aceptada por el autómata de estado finito de la figura 10.2.3. □

Ahora daremos dos ejemplos que ilustran los problemas del diseño.

EJEMPLO 10.2.8

Diseñar un autómata de estado finito que acepte precisamente aquellas cadenas sobre $\{a, b\}$ que no tengan letras a.

La idea es utilizar dos estados:

A: Se encontró una a.

NA: No se encontró una a.

El estado NA es el estado inicial y el único estado de aceptación. Ahora es sencillo dibujar las aristas (véase la figura 10.2.6). Observe que el autómata de estado finito acepta en forma correcta la cadena vacía. □

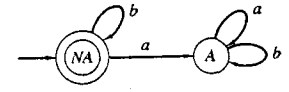


FIGURA 10.2.6
Un autómata de estado finito que acepta precisamente aquellas cadenas sobre $\{a, b\}$ que no tienen letras a.

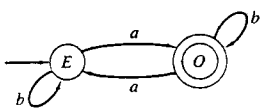


FIGURA 10.2.7
Un autómata de estado finito que acepta precisamente aquellas cadenas sobre $\{a, b\}$ que contienen un número impar de letras a.

EJEMPLO 10.2.9

Diseñar un autómata de estado finito que acepte precisamente aquellas cadenas sobre $\{a, b\}$ que contienen un número impar de letras a.

Esta vez los dos estados son

E: Se encontró un número par de a.

O: Se encontró un número impar de a.

El estado inicial es E y el estado de aceptación es O. Obtenemos el diagrama de transición de la figura 10.2.7. □

Un autómata de estado finito es en esencia un algoritmo que sirve para decidir si una cadena dada es aceptada o no. Como ejemplo, convertiremos el diagrama de transición de la figura 10.2.7 en un algoritmo.

ALGORITMO 10.2.10

Este algoritmo determina si una cadena sobre $\{a, b\}$ es aceptada por el autómata de estado finito cuyo diagrama de transición aparece en la figura 10.2.7.

Entrada: n, la longitud de la cadena ($n = 0$ designa a la cadena vacía)
$s_1 s_2 \cdots s_n$, la cadena

Salida: "Aceptar" si la cadena es aceptada
"Rechazar" si la cadena no es aceptada

```
procedure fsa (s, n)
  state := 'E'
  for i := 1 to n do
    begin
    if state = 'E' and s_i = 'a' then
      state := 'O'
    if state = 'O' and s_i = 'a' then
      state := 'E'
    end
  if state = 'O' then
    return("Aceptar")
  else
    return("Rechazar")
end fsa
```

Si dos autómatas de estado finito aceptan precisamente las mismas cadenas, decimos que los autómatas son **equivalentes**.

DEFINICIÓN 10.2.11

Los autómatas de estado finito A y A' son *equivalentes* si $\mathrm{Ac}(A) = \mathrm{Ac}(A')$.

EJEMPLO 10.2.12

Se puede verificar que los autómatas de estado finito de las figuras 10.2.6 y 10.2.8 son equivalentes (véase el ejercicio 33). □

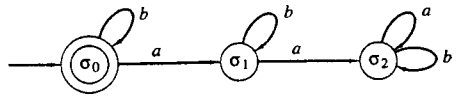


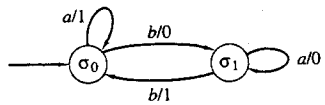
FIGURA 10.2.8 Un autómata de estado finito equivalente al de la figura 10.2.6.

Si definimos una relación R sobre un conjunto de autómatas de estado finito mediante la regla ARA' si A y A' son equivalentes (en el sentido de la definición 10.2.11), R es una relación de equivalencia. Cada clase de equivalencia consta de un conjunto de autómatas de estado finito equivalentes entre sí.

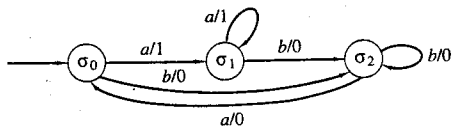
Ejercicios

En los ejercicios 1-3, muestre que cada máquina de estado finito es un autómata de estado finito y trace de nuevo el diagrama de transición como el diagrama de un autómata de estado finito.

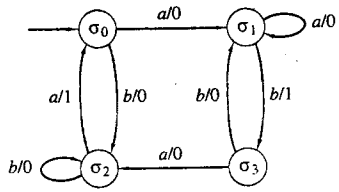
1.



2.

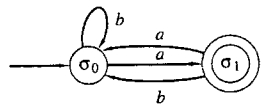


3.

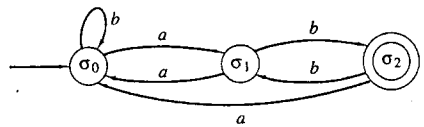


En los ejercicios 4-6, trace de nuevo el diagrama de transición del autómata de estado finito como el diagrama de transición de una máquina de estado finito.

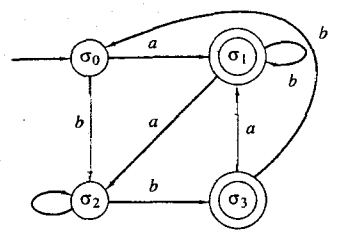
4.



5.



6.



En los ejercicios 7-9, dibuje el diagrama de transición del autómata de estado finito (I, S, f, A, σ_0).

7. $I = \{a, b\}$, $S = \{\sigma_0, \sigma_1, \sigma_2\}$, $A = \{\sigma_0\}$

	f	
S \ I	a	b
σ_0	σ_1	σ_0
σ_1	σ_2	σ_0
σ_2	σ_0	σ_2

8. $I = \{a, b\}$, $S = \{\sigma_0, \sigma_1, \sigma_2\}$, $A = \{\sigma_0, \sigma_2\}$

	f	
S \ I	a	b
σ_0	σ_1	σ_1
σ_1	σ_0	σ_2
σ_2	σ_0	σ_1

9. $I = \{a, b, c\}$, $S = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $A = \{\sigma_0, \sigma_2\}$

	f		
S \ I	a	b	c
σ_0	σ_1	σ_0	σ_2
σ_1	σ_0	σ_3	σ_0
σ_2	σ_3	σ_2	σ_0
σ_3	σ_1	σ_0	σ_1

10. Para cada uno de los autómatas de estado finito de los ejercicios 1-6, determine los conjuntos I, S y A, el estado inicial y la tabla que define la función de estado siguiente.

11. ¿Cuáles son las máquinas de estado finito de los ejercicios 1-10, sección 10.1, son autómatas de estado finito?

12. ¿Cómo debe verse la tabla de una máquina de estado finito M para que M sea un autómata de estado finito?

BIBLIOTECA FACULTAD EXACTAS, INGENIERÍA Y AGRIMENSURA ROSARIO

En los ejercicios 13-17, determine si la cadena dada es aceptada por el autómata de estado finito dado.

13. *abbaa*; figura 10.2.2
14. *abbaa*; figura 10.2.3
15. *aabaabb*; figura 10.2.5
16. *aaabbbaab*; ejercicio 5
17. *aaababbab*; ejercicio 6

18. Muestre que una cadena α sobre $\{a, b\}$ es aceptada por el autómata de estado finito de la figura 10.2.2 si y sólo si α termina en a.

19. Muestre que una cadena α sobre $\{a, b\}$ es aceptada por el autómata de estado finito de la figura 10.2.5 si y sólo si α termina en bb.

☆ 20. Caracterice las cadenas aceptadas por el autómata de estado finito de los ejercicios 1-9.

En los ejercicios 21-31, trace el diagrama de transición de un autómata de estado finito que acepte el conjunto dado de cadenas en $\{a, b\}$.

21. Número par de a
22. Exactamente una b
23. Al menos una b
24. Exactamente dos a
25. Al menos dos a
26. Contiene m letras a, donde m es un múltiplo de 3
27. Comienza con baa
☆ 28. Contiene $abba$
29. Toda b es seguida por a
★ **30.** Termina con aba
☆ 31. Inicia con ab y termina con baa

32. Escriba algoritmos, similares al algoritmo 10.2.10, que deciden si una cadena dada es aceptada o no por el autómata de estado finito de los ejercicios 1-9.

33. Proporcione un argumento formal para mostrar que los autómatas de estado finito de las figuras 10.2.6 y 10.2.8 son equivalentes.

34. Sea L un conjunto finito de cadenas sobre $\{a, b\}$. Muestre que existe un autómata de estado finito que acepta a L.

35. Sea L el conjunto de cadenas aceptadas por el autómata de estado finito del ejercicio 6. Sea S el conjunto de todas las cadenas sobre $\{a, b\}$. Diseñe un autómata de estado finito que acepte a $S - L$.

36. Sea L_i el conjunto de cadenas aceptadas por los autómatas de estado finito $A_i = (I, S_i, f_i, A_i, \sigma_i)$, $i = 1, 2$. Sea

$$A = (I, S_1 \times S_2, f, A, \sigma)$$

donde

$$f((S_1, S_2), x) = (f_1(S_1, x), f_2(S_2, x)),$$
$$A = \{(A_1, A_2) \mid A_1 \in A_1 \text{ y } A_2 \in A_2\},$$
$$\sigma = (\sigma_1, \sigma_2).$$

Muestre que $\text{Ac}(A) = L_1 \cap L_2$.

37. Sea L_i el conjunto de cadenas aceptadas por los autómatas de estado finito $A_i = (I, S_i, f_i, A_i, \sigma_i)$, $i = 1, 2$. Sea

$$A = (I, S_1 \times S_2, f, A, \sigma)$$

donde

$$f((S_1, S_2), x) = (f_1(S_1, x), f_2(S_2, x)),$$
$$A = \{(A_1, A_2) \mid A_1 \in A_1 \text{ o } A_2 \in A_2\},$$
$$\sigma = (\sigma_1, \sigma_2).$$

Muestre que $\text{Ac}(A) = L_1 \cup L_2$.

En los ejercicios 38-42, sea $L_i = \text{Ac}(A_i)$, $i = 1, 2$. Trace el diagrama de transición de los autómatas de estado finito que aceptan $L_1 \cap L_2$ y $L_1 \cup L_2$.

38. A_1 dada en el ejercicio 4; A_2 está dada en el ejercicio 5

39. A_1 dada en el ejercicio 4; A_2 está dada en el ejercicio 6

40. A_1 dada en el ejercicio 5; A_2 está dada en el ejercicio 6

41. A_1 dada en el ejercicio 6; A_2 está dada en el ejercicio 6

42. A_1 dada por la figura 10.5.7, sección 10.5; A_2 dada en el ejercicio 6

10.3 *LENGUAJES Y GRAMÁTICAS*

De acuerdo con el *Diccionario Webster*, un lenguaje es un "cuerpo de palabras y métodos de combinación de palabras utilizado y comprendido por una comunidad de tamaño considerable". Tales lenguajes se llaman **lenguajes naturales** para distinguirlos de los **lenguajes formales**, los cuales se utilizan para modelar los lenguajes naturales y para comunicarse con las computadoras. Las reglas de un lenguaje natural son muy complejas y es difícil caracterizarlas por completo. Por otro lado, es posible especificar completamente las reglas de construcción de ciertos lenguajes formales. Comenzaremos con la definición de un lenguaje formal.

DEFINICIÓN 10.3.1

Sea A un conjunto finito. Un *lenguaje* (*formal*) L sobre A es un subconjunto de A^*, el conjunto de todas las cadenas sobre A.

EJEMPLO 10.3.2

Sea $A = \{a, b\}$. El conjunto L de todas las cadenas sobre A que contiene un número impar de letras a es un lenguaje sobre A. Como vimos en el ejemplo 10.2.9, L es precisamente el conjunto de cadenas sobre A aceptadas por el autómata de estado finito de la figura 10.2.7. □

Una forma de definir un lenguaje es dar una lista de reglas que éste debe cumplir.

DEFINICIÓN 10.3.3

Una *gramática con estructura de frases* (o simplemente una *gramática*) G consta de

(a) Un conjunto finito N de *símbolos no terminales*.

(b) Un conjunto finito T de *símbolos terminales*, donde $N \cap T = \varnothing$.

(c) Un subconjunto finito P de $[(N \cup T)^* - T^*] \times (N \cup T)^*$, llamado el conjunto de *producciones*.

(d) Un *símbolo inicial* $\sigma \in N$.

Escribimos $G = (N, T, P, \sigma)$.

Por lo general, una producción $(A, B) \in P$ se escribe

$$A \to B.$$

La definición 10.3.3c establece que en la producción $A \to B$, $A \in (N \cup T)^* - T^*$ y $B \in (N \cup T)^*$; así, A debe incluir al menos un símbolo no terminal, mientras que B puede constar de cualquier combinación de símbolos terminales y no terminales.

EJEMPLO 10.3.4

Sean

$$N = \{\sigma, S\}$$
$$T = \{a, b\}$$
$$P = \{\sigma \to b\sigma, \sigma \to aS, S \to bS, S \to b\}.$$

Entonces $G = (N, T, P, \sigma)$ es una gramática. □

Dada una gramática G, podemos construir un lenguaje $L(G)$ a partir de G utilizando las producciones para deducir las cadenas que forman a $L(G)$. La idea es partir del símbolo inicial y luego utilizar varias veces las producciones hasta obtener una cadena de símbolos terminales. El lenguaje $L(G)$ es el conjunto de todas las cadenas obtenidas de esta forma. La definición 10.3.5 da los detalles formales.

DEFINICIÓN 10.3.5

Sea $G = (N, T, P, \sigma)$ una gramática.

Si $\alpha \to \beta$ es una producción y $x\,\alpha\,y \in (N \cup T)^*$, decimos que $x\,\beta\,y$ se *deriva directamente* de $x\alpha y$ y escribimos

$$x\alpha y \Rightarrow x\beta y.$$

Si $\alpha_i \in (N \cup T)^*$ para $i = 1, \ldots, n$ y α_{i+1} se deriva directamente de α_i para $i = 1, \ldots, n-1$, decimos que α_n se *deriva de* α_1 y escribimos

$$\alpha_1 \Rightarrow \alpha_n.$$

Decimos que

$$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$$

es la *derivación de* α_n (*a partir de* α_1). Por convención, cualquier elemento de $(N \cup T)^*$ se deriva de sí mismo.

El *lenguaje generado por* G, denotado $L(G)$, consta de todas las cadenas sobre T derivables de σ.

EJEMPLO 10.3.6

Sea G la gramática del ejemplo 10.3.4.

La cadena $abSbb$ se deriva directamente de $aSbb$, lo cual se escribe

$$aSbb \Rightarrow abSbb$$

utilizando la producción $S \to bS$.

La cadena $bbab$ se deriva de σ, lo cual se escribe

$$\sigma \Rightarrow bbab.$$

La derivación es

$$\sigma \Rightarrow b\sigma \Rightarrow bb\sigma \Rightarrow bbaS \Rightarrow bbab.$$

Las únicas derivaciones de σ son

$$\begin{aligned}
&\sigma \Rightarrow b\sigma \\
&\vdots \\
&\Rightarrow b^n\sigma \qquad n \geq 0 \\
&\Rightarrow b^n aS \\
&\vdots \\
&\Rightarrow b^n ab^{m-1}S \\
&\Rightarrow b^n ab^m \quad n \geq 0, \quad m \geq 1.
\end{aligned}$$

Así, $L(G)$ consta de las cadenas sobre $\{a, b\}$ que contienen precisamente una a y terminan con b. □

Una forma alternativa para establecer las producciones de una gramática es utilizar la **forma normal de Backus** (o **forma Backus-Naur**, o **FBN**). En la FBN, los símbolos no terminales comienzan por lo general con "<" y terminan con ">". La producción $S \to T$ se escribe $S ::= T$. Las producciones de la forma

$$S ::= T_1, \quad S ::= T_2, \quad \ldots, \quad S ::= T_n$$

se pueden combinar como

$$S ::= T_1 \mid T_2 \mid \cdots \mid T_n.$$

La barra "|" se lee "o".

EJEMPLO 10.3.7 *Una gramática para los enteros*

Un entero se define como una cadena que consta de un signo opcional (+ o −) seguido de una cadena de dígitos (0 a 9). La siguiente gramática genera a todos los enteros.

$$\begin{aligned}
&< \text{dígito} > ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
&< \text{entero} > ::= < \text{entero con signo} > \mid < \text{entero sin signo} > \\
&< \text{entero con signo} > ::= + < \text{entero sin signo} > \mid - < \text{entero sin signo} > \\
&< \text{entero sin signo} > ::= < \text{dígito} > \mid < \text{dígito} > < \text{entero sin signo} >
\end{aligned}$$

El símbolo inicial es < entero >.

Por ejemplo, la derivación del entero −901 es

$$\begin{aligned}
< \text{entero} > &\Rightarrow < \text{entero con signo} > \\
&\Rightarrow - < \text{entero sin signo} > \\
&\Rightarrow - < \text{dígito} > < \text{entero sin signo} > \\
&\Rightarrow - < \text{dígito} > < \text{dígito} > < \text{entero sin signo} > \\
&\Rightarrow - < \text{dígito} > < \text{dígito} > < \text{dígito} > \\
&\Rightarrow -9 < \text{dígito} > < \text{dígito} > \\
&\Rightarrow -90 < \text{dígito} > \\
&\Rightarrow -901.
\end{aligned}$$

En la notación de la definición 10.3.3, este lenguaje consta de

1. El conjunto $N = \{< \text{dígito} >, < \text{entero} >, < \text{entero con signo} >, < \text{entero sin signo} >\}$ de símbolos no terminales.
2. El conjunto $T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}$ de símbolos terminales.
3. Las producciones

$$\begin{aligned}
&< \text{dígito} > \to 0, \ldots, < \text{dígito} > \to 9, \\
&< \text{entero} > \to < \text{entero con signo} >, \\
&< \text{entero} > \to < \text{entero sin signo} >, \\
&< \text{entero con signo} > \to + < \text{entero sin signo} >, \\
&< \text{entero con signo} > \to - < \text{entero sin signo} >, \\
&< \text{entero sin signo} > \to < \text{dígito} >, \\
&< \text{entero sin signo} > \to < \text{dígito} > < \text{entero sin signo} >.
\end{aligned}$$

4. El símbolo inicial < entero >. □

Los lenguajes de computación, como FORTRAN, Pascal y C, se especifican por lo general en FBN. El ejemplo 10.3.7 muestra la forma de especificar una constante entera en un lenguaje de computación mediante su FBN.

Las gramáticas se clasifican según los tipos de producciones que las definen.

DEFINICIÓN 10.3.8

Sea G una gramática y λ la cadena nula.

(a) Si cada producción es de la forma

$$\alpha A\beta \to \alpha\delta\beta, \quad \text{donde } \alpha, \beta \in (N \cup T)^*, \quad A \in N, \quad \delta \in (N \cup T)^* - \{\lambda\}, \tag{10.3.1}$$

entonces G es una *gramática sensible al contexto* (o de *tipo* 1).

(b) Si cada producción es de la forma

$$A \to \delta, \quad \text{donde } A \in N, \quad \delta \in (N \cup T)^*, \tag{10.3.2}$$

entonces G es una *gramática libre de contexto* (o de *tipo* 2).

(c) Si cada producción es de la forma

$$A \to a \text{ o } A \to aB \text{ o } A \to \lambda, \quad \text{donde } A, B \in N, \quad a \in T,$$

decimos que G es una *gramática regular* (o de *tipo* 3).

Según (10.3.1), en una gramática sensible al contexto, podemos reemplazar A por δ si A está en el contexto de α y β. En una gramática libre de contexto, (10.3.2) establece que podemos reemplazar A por δ en cualquier momento. Una gramática regular tiene reglas de sustitución particularmente sencillas: Reemplazamos un símbolo no terminal por un símbolo terminal, por un símbolo terminal seguido de un símbolo no terminal, o por la cadena nula.

Observe que una gramática regular es una gramática libre de contexto y que una gramática libre de contexto sin producciones de la forma $A \to \lambda$ es una gramática sensible al contexto.

Algunas definiciones permiten reemplazar a por una cadena de terminales en la definición 10.3.8c; sin embargo, se puede mostrar (véase el ejercicio 32) que las dos definiciones producen los mismos lenguajes.

EJEMPLO 10.3.9

La gramática G definida por

$$T = \{a, b, c\}, \quad N = \{\sigma, A, B, C, D, E\},$$

con producciones

$$\sigma \to aAB, \quad \sigma \to aB, \quad A \to aAC, \quad A \to aC, \quad B \to Dc,$$
$$D \to b, \quad CD \to CE, \quad CE \to DE, \quad DE \to DC, \quad Cc \to Dcc,$$

y símbolo inicial σ es sensible al contexto. Por ejemplo, la producción $CE \to DE$ dice que podemos reemplazar C con D si C va seguida de E y la producción $Cc \to Dcc$ dice que podemos reemplazar C con Dc si C va seguida de c.

Podemos derivar DC de CD, pues

$$CD \Rightarrow CE \Rightarrow DE \Rightarrow DC.$$

La cadena $a^3b^3c^3$ está en $L(G)$, pues tenemos

$$\sigma \Rightarrow aAB \Rightarrow aaACB \Rightarrow aaaCCDc \Rightarrow aaaDCCc \Rightarrow aaaDCDcc$$
$$\Rightarrow aaaDDCcc \Rightarrow aaaDDDccc \Rightarrow aaabbbccc.$$

Se puede mostrar (véase el ejercicio 33) que

$$L(G) = \{a^n b^n c^n \mid n = 1, 2, \ldots\}. \qquad \square$$

Es natural permitir que el lenguaje $L(G)$ herede una propiedad de una gramática G. La siguiente definición precisa este concepto.

DEFINICIÓN 10.3.10

Un lenguaje L es *sensible al contexto* (respectivamente, *libre de contexto*, *regular*) si existe una gramática G sensible al contexto (respectivamente, libre de contexto, regular) tal que $L = L(G)$.

EJEMPLO 10.3.11

De acuerdo con el ejemplo 10.3.9, el lenguaje

$$L = \{a^n b^n c^n \mid n = 1, 2, \ldots\}$$

es sensible al contexto. Se puede mostrar (véase [Hopcroft, página 127]) que no existe una gramática libre de contexto G tal que $L = L(G)$; por tanto, L no es un lenguaje libre de contexto. $\square$

EJEMPLO 10.3.12

La gramática G definida por

$$T = \{a, b\}, \quad N = \{\sigma\},$$

con producciones

$$\sigma \to a\sigma b, \quad \sigma \to ab,$$

y símbolo inicial σ es libre de contexto. Las únicas derivaciones de σ son

$$\sigma \Rightarrow a\sigma b$$
$$\vdots$$
$$\Rightarrow a^{n-1}\sigma b^{n-1}$$
$$\Rightarrow a^{n-1}abb^{n-1} = a^n b^n.$$

Así, $L(G)$ consta de las cadenas sobre $\{a, b\}$ de la forma $a^n b^n$; $n = 1, 2, \ldots$. Este lenguaje es libre de contexto. En la sección 10.5 (véase el ejemplo 10.5.6) mostraremos que $L(G)$ no es regular. $\square$

Los ejemplos 10.3.11 y 10.3.12 muestran que el conjunto de lenguajes libres de contexto que no contienen a la cadena nula es un subconjunto propio del conjunto de lenguajes sensibles al contexto y que el conjunto de lenguajes regulares es un subconjunto propio del conjunto de lenguajes libres de contexto. También se puede mostrar que existen lenguajes que no son sensibles al contexto.

EJEMPLO 10.3.13

La gramática G definida en el ejemplo 10.3.4 es regular. Así, el lenguaje

$$L(G) = \{b^n a b^m \mid n = 0, 1, \ldots; m = 1, 2, \ldots\}$$

generado por ella es regular. $\square$

EJEMPLO 10.3.14

La gramática del ejemplo 10.3.7 es libre de contexto, pero no regular. Sin embargo, si cambiamos las producciones por

$$\begin{aligned}
&< \text{entero} > ::= + < \text{entero sin signo} > \mid - < \text{entero sin signo} > \mid \\
&\qquad 0 < \text{dígitos} > \mid 1 < \text{dígitos} > \mid \cdots \mid 9 < \text{dígitos} > \\
&< \text{entero sin signo} > ::= 0 < \text{dígitos} > \mid 1 < \text{dígitos} > \mid \cdots \mid 9 < \text{dígitos} > \\
&< \text{dígitos} > ::= 0 < \text{dígitos} > \mid 1 < \text{dígitos} > \mid \cdots \mid 9 < \text{dígitos} > \mid \lambda,
\end{aligned}$$

la gramática resultante es regular. Como el lenguaje generado no se modifica, esto implica que el conjunto de cadenas que representan a los enteros es un lenguaje regular. $\square$

El ejemplo 10.3.14 motiva la siguiente definición.

DEFINICIÓN 10.3.15

Las gramáticas G y G' son *equivalentes* si $L(G) = L(G')$.

EJEMPLO 10.3.16

Las gramáticas de los ejemplos 10.3.7 y 10.3.14 son equivalentes. □

Si definimos una relación R sobre un conjunto de gramáticas mediante la regla $G\,R\,G'$ si G y G' son equivalentes (en el sentido de la definición 10.3.15), R es una relación de equivalencia. Cada clase de equivalencia consta de un conjunto de gramáticas equivalentes entre sí.

Concluimos esta sección presentando de manera breve otro tipo de gramática que se puede utilizar para generar curvas fractales.

DEFINICIÓN 10.3.17

Una *gramática Lindenmayer interactiva libre de contexto* consta de

(a) Un conjunto finito N de *símbolos no terminales*.

(b) Un conjunto finito T de *símbolos terminales*, donde $N \cap T = \varnothing$.

(c) Un conjunto finito P de *producciones* $A \to B$, donde $A \in N \cup T$ y $B \in (N \cup T)^*$.

(d) Un *símbolo inicial* $\sigma \in N$.

La diferencia entre una gramática Lindenmayer interactiva libre de contexto y una gramática libre de contexto es que la primera permite el uso de producciones de la forma $A \to B$, donde A es un símbolo terminal o no terminal. (En una gramática libre de contexto, A debe ser no terminal.)

Las reglas para derivar cadenas en una gramática Lindenmayer interactiva libre de contexto son diferentes de las reglas para derivar cadenas en una gramática con estructura de frases (véase la definición 10.3.5). En una gramática Lindenmayer interactiva libre de contexto, para derivar la cadena β de la cadena α, hay que reemplazar *todos* los símbolos en α *de manera simultánea.* A continuación damos la definición formal.

DEFINICIÓN 10.3.18

Sea $G = (N, T, P, \sigma)$ una gramática Lindenmayer interactiva libre de contexto. Si

$$\alpha = x_1 \cdots x_n$$

y existen producciones

$$x_i \to \beta_i$$

en P, para $i = 1, \ldots, n$, escribimos

$$\alpha \Rightarrow \beta_1 \cdots \beta_n$$

y decimos que $\beta_1 \cdots \beta_n$ es *derivable de manera directa* de α. Si α_{i+1} es derivable de manera directa de α_i para $i = 1, \ldots, n-1$, decimos que α_n es *derivable de* α_1 y escribimos

$$\alpha_1 \Rightarrow \alpha_n.$$

Decimos que

$$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$$

es la *derivación de* α_n *(a partir de* α_1*)*. El *lenguaje generado por* G, denotado $L(G)$, consta de todas las cadenas sobre T derivables a partir de σ.

EJEMPLO 10.3.19 ***El copo de nieve de von Koch***

Sean

$$N = \{D\}$$
$$T = \{d, +, -\}$$
$$P = \{D \to D - D + + D - D, D \to d, + \to +, - \to -\}.$$

Consideramos a $G(N, T, P, D)$ como una gramática Lindenmayer interactiva libre de contexto. Como un ejemplo de derivación de D, tenemos

$$D \Rightarrow D - D + + D - D \Rightarrow d - d + + d - d.$$

Así, $d - d + + d - d \in L(G)$.

Ahora daremos un significado a las cadenas de $L(G)$. Interpretamos el símbolo d como una instrucción para trazar una línea recta de una longitud fija en la dirección actual; interpretamos $+$ como una instrucción para girar 60° hacia la derecha; e interpretamos $-$ como una instrucción para girar 60° hacia la izquierda. Si comenzamos del lado izquierdo y el primer movimiento es horizontal y hacia la derecha, la interpretación de la cadena $d - d + + d - d$ produce la curva de la figura 10.3.1a.

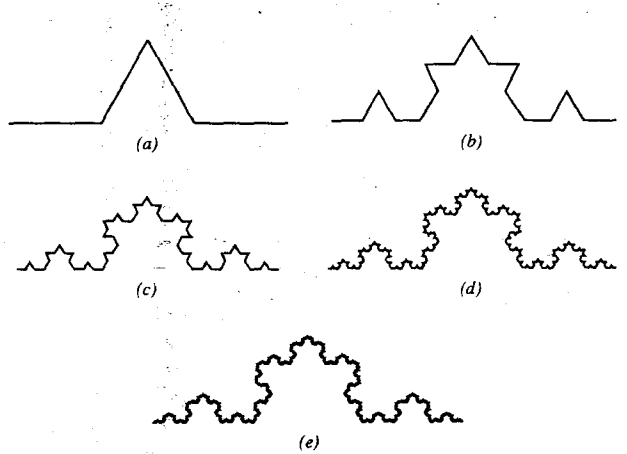


FIGURA 10.3.1 Copos de nieve de von Koch.

La siguiente cadena más larga en $L(G)$ es

$$d - d + + d - d - d - d + + d - d + + d - d + + d - d - d - d + + d - d$$

cuya derivación es

$$\begin{aligned} D &\Rightarrow D - D + + D - D \\ &\Rightarrow D - D + + D - D - D - D + + D - D + + D \\ &\quad - D + + D - D - D - D + + D - D \\ &\Rightarrow d - d + + d - d - d - d + + d - d + + d \\ &\quad - d + + d - d - d - d + + d - d. \end{aligned}$$

No es posible obtener una cadena más corta debido a que al usar las producciones debemos reemplazar *todos* los símbolos de manera simultánea (definición 10.3.18). Si reemplazamos algunas D por d y otras D por $D - D + + D - D$, no podemos derivar alguna cadena de la cadena resultante, no sólo una cadena terminal, pues d no aparece en el lado izquierdo de ninguna producción.

Al interpretar la cadena

$$d - d + + d - d - d - d + + d - d + + d - d + + d - d - d - d + + d - d$$

obtenemos la curva de la figura 10.3.1b.

Las curvas obtenidas al interpretar las cadenas en $L(G)$ aparecen en la figura 10.3.1c-e. Estas curvas se conocen como los **copos de nieve de von Koch**. □

Las curvas como el copo de nieve de von Koch se llaman **curvas fractales** (véase [Peitgen]). Una característica de las curvas fractales es que una parte del todo se parece al todo. Por ejemplo, como muestra la figura 10.3.2, cuando la parte indicada del copo de nieve de von Koch se extrae y se amplía, recuerda a la curva original.

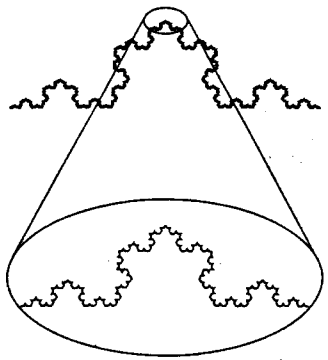


FIGURA 10.3.2 La naturaleza fractal del copo de nieve de von Koch. Cuando la parte superior del copo de nieve de von Koch se extrae y se amplía, se asemeja a la curva original.

Las gramáticas Lindenmayer interactivas libres de contexto y sensibles al contexto fueron ideadas en 1968 por A. Lindenmayer (véase [Lindenmayer]) para modelar el crecimiento de las plantas. Como lo sugiere el ejemplo 10.3.19, estas gramáticas se pueden utilizar en la graficación por computadora para generar imágenes (véase [Prusinkiewicz 1986, 1988; Smith]). Se puede mostrar (véase [Wood, página 503]) que la clase de lenguajes generados por las gramáticas Lindenmayer interactiva sensibles al contexto es exactamente igual a la clase de lenguajes generados por las gramáticas con estructura de frases.

Ejercicios

En los ejercicios 1-6, determine si la gramática dada es sensible al contexto, libre de contexto, regular, o ninguna de éstas. Indique todas las caracterizaciones que se le apliquen.

1. $T = \{a, b\}, N = \{\sigma, A\}$, con producciones

$$\sigma \to b\sigma, \quad \sigma \to aA, \quad A \to a\sigma,$$
$$A \to bA, \quad A \to a, \quad \sigma \to b,$$

y símbolo inicial σ.

2. $T = \{a, b, c\}, N = \{\sigma, A, B\}$, con producciones

$$\sigma \to AB, \quad AB \to BA, \quad A \to aA,$$
$$B \to Bb, \quad A \to a, \quad B \to b,$$

y símbolo inicial σ.

3. $T = \{a, b\}, N = \{\sigma, A, B\}$, con producciones

$$\sigma \to A, \quad \sigma \to AAB, \quad Aa \to ABa, \quad A \to aa,$$
$$Bb \to ABb, \quad AB \to ABB, \quad B \to b,$$

y símbolo inicial σ.

4. $T = \{a, b, c\}, N = \{\sigma, A, B\}$, con producciones

$$\sigma \to BAB, \quad \sigma \to ABA, \quad A \to AB, \quad B \to BA,$$
$$A \to aA, \quad A \to ab, \quad B \to b,$$

y símbolo inicial σ.

5. $<S> ::= b<S> \mid a<A> \mid a$
$<A> ::= a<S> \mid b<B>$
$<B> ::= b<A> \mid a<S> \mid b$

con símbolo inicial $<S>$.

6. $T = \{a, b\}, N = \{\sigma, A, B\}$, con producciones

$$\sigma \to AA\sigma, \quad AA \to B, \quad B \to bB, \quad A \to a,$$

y símbolo inicial σ.

En los ejercicios 7-11, muestre que la cadena dada α está en $L(G)$ para la gramática G dada, mostrando una derivación de α.

7. *bbabbab*, ejercicio 1

8. *abab*, ejercicio 2

9. *aabbaab*, ejercicio 3

10. *abbbaabab*, ejercicio 4

11. *abaabbabba*, ejercicio 5

12. Escriba las gramáticas de los ejemplos 10.3.4 y 10.3.9 y los ejercicios 1-4 y 6 en FBN.

☆ 13. Sea G la gramática del ejercicio 1. Muestre que $\alpha \in L(G)$ si y sólo si α es no nula y contiene un número par de letras a.

☆ 14. Sea G la gramática del ejercicio 5. Caracterice $L(G)$.

En los ejercicios 15-24, escriba una gramática que genere las cadenas con la propiedad dada.

15. Cadenas sobre $\{a, b\}$ que comienzan con a

16. Cadenas sobre $\{a, b\}$ que terminan con ba

17. Cadenas sobre $\{a, b\}$ que contienen ba

★ **18.** Cadenas sobre $\{a, b\}$ que no terminan con ab

19. Enteros sin ceros al principio

20. Números con punto flotante (números como .294, 89., 67.284)

21. Números exponenciales (números que incluyen a los números con punto flotante y números como 6.9E3, 8E12, 9.6E-4, 9E-10)

22. Expresiones booleanas en $X_1, \ldots, X_n$

23. Todas las cadenas sobre $\{a, b\}$

24. Cadenas $x_1 \cdots x_n$ sobre $\{a, b\}$ tales que $x_1 \cdots x_n = x_n \cdots x_1$

Se propone que cada una de las gramáticas de los ejercicios 25-31 genera el conjunto L de cadenas sobre $\{a, b\}$ que contiene la misma cantidad de letras a y letras b. Si la gramática genera L, demuestre que lo hace. Si la gramática no genera L, dé un contraejemplo y demuestre que su contraejemplo es correcto. En cada gramática, S es el símbolo inicial.

25. $S \to aSb \mid bSa \mid \lambda$

26. $S \to aSb \mid bSa \mid SS \mid \lambda$

27. $S \to aB \mid bA \mid \lambda, B \to b \mid bA, A \to a \mid aB$

28. $S \to abS \mid baS \mid aSb \mid bSa \mid \lambda$

29. $S \to aSb \mid bSa \mid abS \mid baS \mid Sab \mid Sba \mid \lambda$

30. $S \to aB \mid bA, A \to a \mid SA, B \to b \mid SB$

31. $S \to aSbS \mid bSaS \mid \lambda$

★ **32.** Sea G una gramática y λ la cadena nula. Muestre que si cada producción es de la forma

$$A \to \alpha \text{ o } A \to \alpha B \text{ o } A \to \lambda, \quad \text{donde } A, B \in N, \quad a \in T^* - \{\lambda\},$$

entonces existe una gramática regular G' con $L(G) = L(G')$.

☆ 33. Sea G la gramática del ejemplo 10.3.9. Muestre que

$$L(G) = \{a^n b^n c^n \mid n = 1, 2, \ldots\}.$$

34. Muestre que el lenguaje

$$\{a^n b^n c^k \mid n, k \in \{1, 2, \ldots\}\}$$

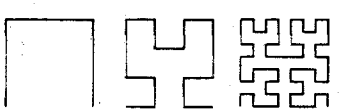
es un lenguaje libre de contexto.

35. Sean

$$N = \{S, D\}$$
$$T = \{d, +, -\}$$
$$P = \{S \to D + D + D + D,$$
$$D \to D + D - D - DD + D + D - D \mid d, + \to +, - \to -\}.$$

Considere a $G = (N, T, P, S)$ como una gramática Lindenmayer libre de contexto. Interprete el símbolo d como una instrucción para trazar una línea recta, con una longitud fija, en la dirección actual; interprete $+$ como una instrucción para girar a la derecha 90°; e interprete $-$ como una instrucción para girar a la izquierda 90°. Genere las dos cadenas de menor longitud en $L(G)$ y trace las curvas correspondientes. Estas curvas se conocen como las *islas cuadráticas de Koch.*

☆ 36. La siguiente figura



muestra las tres primeras etapas de la *curva de Hilbert.* Defina una gramática Lindenmayer libre de contexto que genere cadenas tales que, al ser interpretadas en forma adecuada, generen la curva de Hilbert.

10.4 AUTÓMATAS DE ESTADO FINITO NO DETERMINISTAS

En esta y la siguiente sección mostraremos que las gramáticas regulares y los autómatas de estado finito son esencialmente lo mismo, en el sentido de que cada una especifica un lenguaje regular. Comenzamos con un ejemplo que ilustra la forma de convertir un autómata de estado finito en una gramática regular.

EJEMPLO 10.4.1

Escribir la gramática regular dada por el autómata de estado finito de la figura 10.2.7.

Los símbolos terminales son los símbolos de entrada $\{a, b\}$. Los estados E y O se convierten en los símbolos no terminales. El estado inicial E se convierte en el símbolo inicial. Las producciones corresponden a las aristas dirigidas. Si existe una arista etiquetada x de S a S', escribimos la producción

$$S \to x\,S'.$$

En nuestro caso, obtenemos las producciones

$$E \to bE, \quad E \to aO, \quad O \to aE, \quad O \to bO. \tag{10.4.1}$$

Además, si S es un estado de aceptación, incluimos la producción

$$S \to \lambda.$$

En nuestro caso, obtenemos la producción adicional

$$O \to \lambda. \tag{10.4.2}$$

Entonces la gramática $G = (N, T, P, E)$, con $N = \{O, E\}$, $T = \{a, b\}$ y P formado por las producciones (10.4.1) y (10.4.2), genera el lenguaje $L(G)$, que es igual al conjunto de cadenas aceptadas por el autómata de estado finito de la figura 10.2.7. □

TEOREMA 10.4.2

Sea A un autómata de estado finito dado como un diagrama de transición. Sea σ el estado inicial. Sea T el conjunto de símbolos de entrada y sea N el conjunto de estados. Definimos las producciones

$$S \to x S'$$

si existe una arista etiquetada x de S a S', y

$$S \to \lambda$$

si S es un estado de aceptación. Sea G la gramática regular

$$G = (N, T, P, \sigma).$$

Entonces el conjunto de cadenas aceptadas por A es igual a $L(G)$.

Demostración. Primero mostraremos que $\text{Ac}(A) \subseteq L(G)$. Sea $\alpha \in \text{Ac}(A)$. Si α es la cadena nula, entonces σ es un estado de aceptación. En este caso, G contiene la producción

$$\sigma \to \lambda.$$

La derivación

$$\sigma \Rightarrow \lambda \tag{10.4.3}$$

muestra que $\alpha \in L(G)$.

Ahora, supongamos que $\alpha \in \text{Ac}(A)$ y que α no es la cadena nula. Entonces $\alpha = x_1 \cdots x_n$ para $x_i \in T$. Como α es aceptada por A, existe un camino $(\sigma, S_1, \ldots, S_n)$, donde S_n es un estado de aceptación, con las aristas etiquetadas en forma sucesiva como $x_1, \ldots, x_n$. Esto implica que G contiene las producciones

$$\sigma \to x_1 S_1$$

$$S_{i-1} \to x_i S_i \quad \text{para } i = 2, \ldots, n.$$

Como S_n es un estado de aceptación, G también contiene a la producción

$$S_n \to \lambda.$$

La derivación

$$\begin{aligned} \sigma &\Rightarrow x_1 S_1 \\ &\Rightarrow x_1 x_2 S_2 \\ &\vdots \\ &\Rightarrow x_1 \cdots x_n S_n \\ &\Rightarrow x_1 \cdots x_n \end{aligned} \tag{10.4.4}$$

muestra que $\alpha \in L(G)$.

Concluiremos la demostración mostrando que $L(G) \subseteq \text{Ac}(A)$. Supongamos que $\alpha \in L(G)$. Si α es la cadena nula, α debe surgir de la derivación (10.4.3), pues una derivación que comienza con cualquier otra producción daría como resultado una cadena no nula. Así, la producción $\sigma \to \lambda$ está en la gramática. Por tanto, σ es un estado de aceptación en A. Esto implica que $\alpha \in \text{Ac}(A)$.

Ahora, supongamos que $\alpha \in L(G)$ y que α no es la cadena nula. Entonces $\alpha = x_1 \cdots x_n$ para $x_i \in T$. Esto implica que existe una derivación de la forma (10.4.4). Si, en el diagrama de transición, comenzamos con σ y seguimos el camino $(\sigma, S_1, \ldots, S_n)$, podemos generar la cadena α. La última producción utilizada en (10.4.4) es $S_n \to \lambda$; así, el último estado alcanzado es un estado de aceptación. Por tanto, α es aceptada por A, de modo que $L(G) \subseteq \text{Ac}(A)$. Con esto concluimos la demostración. ■

A continuación, consideremos la situación inversa. Dada una gramática regular G, querémos construir un autómata de estado finito A de modo que $L(G)$ sea precisamente el conjunto de cadenas aceptadas por A. A primera vista, podría parecer que basta invertir el procedimiento del teorema 10.4.2. Sin embargo, el siguiente ejemplo muestra que la situación es un poco más compleja.

EJEMPLO 10.4.3

Consideremos la gramática regular definida por

$$T = \{a, b\}, \quad N = \{\sigma, C\}$$

con producciones

$$\sigma \to b\sigma, \quad \sigma \to aC, \quad C \to bC, \quad C \to b$$

y símbolo inicial σ.

Los símbolos no terminales se convierten en estados, donde σ es el estado inicial. Para cada producción de la forma

$$S \to xS'$$

trazamos una arista del estado S al estado S' y la etiquetamos x. Las producciones

$$\sigma \to b\sigma, \quad \sigma \to aC, \quad C \to bC$$

proporcionan la gráfica de la figura 10.4.1. La producción $C \to b$ es equivalente a las dos producciones

$$C \to bF, \quad F \to \lambda,$$

donde F es un símbolo no terminal adicional. Las producciones

$$\sigma \to b\sigma, \quad \sigma \to aC, \quad C \to bC, \quad C \to bF$$

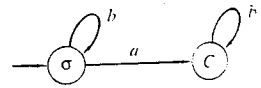


FIGURA 10.4.1
La gráfica correspondiente a las producciones $\sigma \to b\sigma$, $\sigma \to aC$, $C \to bC$.

proporcionan la gráfica de la figura 10.4.2. La producción

$$F \to \lambda$$

nos dice que F debe ser un estado de aceptación (véase la figura 10.4.2). □

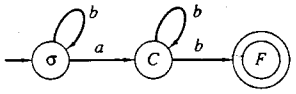


FIGURA 10.4.2 El autómata de estado finito no determinista correspondiente a la gramática $\sigma \to b\sigma$, $\sigma \to aC$, $C \to bC$, $C \to b$.

Por desgracia, la gráfica de la figura 10.4.2 no es un autómata de estado finito. Existen varios problemas. El vértice C no tiene una arista de salida con la etiqueta a, y el vértice F no tiene aristas de salida. Además, el vértice C tiene dos aristas de salida con la etiqueta b. Un diagrama como el de la figura 10.4.2 define otro tipo de autómata llamado un **autómata de estado finito no determinista**. La razón de las palabras "no determinista" es que cuando estamos en un estado donde hay varias aristas de salida, todas ellas con la misma etiqueta x, si se introduce x como entrada, tenemos una situación no determinista, pues hay que elegir el estado siguiente. Por ejemplo, si en la figura 10.4.2 estamos en el estado C y la entrada es b, podemos elegir entre dos estados siguientes: permanecer en el estado C o ir al estado F.

DEFINICIÓN 10.4.4

Un *autómata de estado finito no determinista* A consta de

(a) Un conjunto finito I de *símbolos de entrada.*

(b) Un conjunto finito S de *estados.*

(c) Una *función de estado siguiente* f de $S \times I$ en $P(S)$.

(d) Un subconjunto A de S de *estados de aceptación.*

(e) Un *estado inicial* $\sigma \in S$.

Escribimos $A = (I, S, f, A, \sigma)$.

La única diferencia entre un autómata de estado finito no determinista y un autómata de estado finito es que en este último la función de estado siguiente nos lleva a un estado definido de manera única, mientras que en un autómata de estado finito no determinista la función de estado siguiente nos lleva a un conjunto de estados.

EJEMPLO 10.4.5

Para el autómata de estado finito no determinista de la figura 10.4.2, tenemos

$$I = \{a, b\}, \quad S = \{\sigma, C, F\}, \quad A = \{F\}.$$

El estado inicial es σ y la función de estado siguiente f está dada por

S \ I	a	b
σ	$\{C\}$	$\{\sigma\}$
C	$\varnothing$	$\{C, F\}$
F	$\varnothing$	$\varnothing$

□

El diagrama de transición de un autómata de estado finito no determinista se traza de una manera análoga al de un autómata de estado finito. Trazamos una arista del estado S a cada estado en el conjunto $f(S, x)$ y los etiquetamos x.

EJEMPLO 10.4.6

El diagrama de transición del autómata de estado finito no determinista

$$I = \{a, b\}, \quad S = \{\sigma, C, D\}, \quad A = \{C, D\}$$

con estado inicial α y función de estado siguiente

S \ I	a	b
σ	$\{\sigma, C\}$	$\{D\}$
C	$\varnothing$	$\{C\}$
D	$\{C, D\}$	$\varnothing$

aparece en la figura 10.4.3.

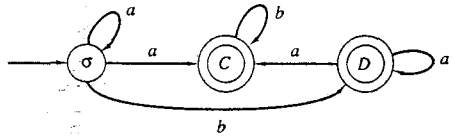


FIGURA 10.4.3 El diagrama de transición del autómata de estado finito no determinista del ejemplo 10.4.6. □

Una cadena α es aceptada por un autómata de estado finito no determinista A si existe algún camino que represente a α en el diagrama de transición de A que parta del estado inicial y llegue a un estado de aceptación. A continuación damos la definición formal.

DEFINICIÓN 10.4.7

Sea $A = (I, S, f, A, \sigma)$ un autómata de estado finito no determinista. La cadena nula es *aceptada* por A si y sólo si $\sigma \in A$. Si $\alpha = x_1 \cdots x_n$ es una cadena no nula sobre I y existen estados $\sigma_0, \ldots, \sigma_n$ tales que

(a) $\sigma_0 = \sigma$;

(b) $\sigma_i \in f(\sigma_{i-1}, x_i)$ para $i = 1, \ldots, n$;

(c) $\sigma_n \in A$;

decimos que α es *aceptada* por A. $\mathrm{Ac}(A)$ denota el conjunto de cadenas aceptadas por A y decimos que A *acepta* $\mathrm{Ac}(A)$.

Si A y A' son autómatas de estado finito no deterministas y $\mathrm{Ac}(A) = \mathrm{Ac}(A')$, decimos que A y A' son *equivalentes*.

Si $\alpha = x_1 \cdots x_n$ es una cadena sobre I y existen estados $\sigma_0, \ldots, \sigma_n$ que satisfagan las condiciones (a) y (b), el camino $(\sigma_0, \ldots, \sigma_n)$ es un *camino que representa* a α en A.

EJEMPLO 10.4.8

La cadena

$$\alpha = bbabb$$

es aceptada por el autómata de estado finito no determinista de la figura 10.4.2, pues el camino $(\sigma, \sigma, \sigma, C, C, F)$, el cual termina en un estado de aceptación, representa a α. Observe que el camino $P = (\sigma, \sigma, \sigma, C, C, C)$ también representa a α, pero P no termina en un estado de aceptación. Sin embargo, la cadena α es aceptada pues existe al menos un camino que representa a α y que termina en un estado de aceptación. Una cadena β no es aceptada si ningún camino representa a β o bien si todos los caminos que representan a β terminan en un estado que no es de aceptación. □

EJEMPLO 10.4.9

La cadena $\alpha = aabaabbb$ es aceptada por el autómata de estado finito no determinista de la figura 10.4.3. El lector debe localizar el camino que representa a α, y que termina en el estado C. □

EJEMPLO 10.4.10

La cadena $\alpha = abba$ no es aceptada por el autómata de estado finito no determinista de la figura 10.4.3. Al salir de σ, cuando la entrada es a, existen dos opciones: Ir a C o permanecer en σ. Si vamos a C, al introducir dos letras b, nuestros movimientos quedan determinados y permanecemos en C. Pero cuando se introduce la a final, no existe una arista a lo largo de la cual moverse. Por otro lado, supongamos que cuando introducimos la primera a, permanecemos en σ. Entonces, al introducir b, nos movemos a D. Pero ahora, al introducir la siguiente b, no existe una arista sobre la cual moverse. Como no existe un camino que represente a α en la figura 10.4.3, la cadena α no es aceptada por el autómata de estado finito no determinista de la figura 10.4.3. □

Formulemos la construcción del ejemplo 10.4.3 como un teorema.

TEOREMA 10.4.11

Sea $G = (N, T, P, \sigma)$ una gramática regular. Sean

$$I = T$$
$$S = N \cup \{F\}, \text{ donde } F \notin N \cup T$$
$$f(S, x) = \{S' \mid S \to xS' \in P\} \cup \{F \mid S \to x \in P\}$$
$$A = \{F\} \cup \{S \mid S \to \lambda \in P\}.$$

Entonces el autómata de estado finito no determinista $A = (I, S, f, A, \sigma)$ acepta precisamente las cadenas $L(G)$.

Demostración. La demostración es esencialmente igual a la demostración del teorema 10.4.2 y por tanto se omite. ■

Podría parecer que un autómata de estado finito no determinista es un concepto más general que el de un autómata de estado finito; sin embargo, en la siguiente sección mostraremos que dado un autómata de estado finito no determinista A, podemos construir un autómata de estado finito que es equivalente a A.

Ejercicios

En los ejercicios 1-5, trace el diagrama de transición del autómata de estado finito no determinista (I, S, f, A, σ_0).

1. $I = \{a, b\}, S = \{\sigma_0, \sigma_1, \sigma_2\}, A = \{\sigma_0\}$

S \ I	a	b
σ_0	$\varnothing$	$\{\sigma_1, \sigma_2\}$
σ_1	$\{\sigma_2\}$	$\{\sigma_0, \sigma_1\}$
σ_2	$\{\sigma_0\}$	$\varnothing$

2. $I = \{a, b\}, S = \{\sigma_0, \sigma_1, \sigma_2\}, A = \{\sigma_0, \sigma_1\}$

S \ I	a	b
σ_0	$\{\sigma_1\}$	$\{\sigma_0, \sigma_2\}$
σ_1	$\varnothing$	$\{\sigma_2\}$
σ_2	$\{\sigma_1\}$	$\varnothing$

EJEMPLO 10.5.7

Sea L el conjunto de cadenas aceptada por el autómata de estado finito A de la figura 10.5.4. Construir un autómata de estado finito que acepte las cadenas

$$L^R = \{x_n \cdots x_1 \mid x_1 \cdots x_n \in L\}.$$

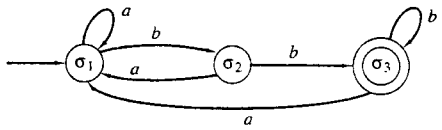


FIGURA 10.5.4 El autómata de estado finito del ejemplo 10.5.7, que acepta a L.

Queremos convertir A en un autómata de estado finito que acepte L^R. La cadena $\alpha = x_1 \cdots x_n$ es aceptada por A si existe un camino P en A que represente a α, comience en σ_1 y termina en σ_3. Si comenzamos en σ_3 y recorremos P en reversa, terminamos en σ_1 y procesamos las aristas en el orden $x_n, \ldots, x_1$. Así, sólo necesitamos invertir todas las flechas de la figura 10.5.4 y hacer de σ_3 el estado inicial y σ_1 el estado de aceptación (véase la figura 10.5.5). El resultado es un autómata de estado finito no determinista que acepta L^R.

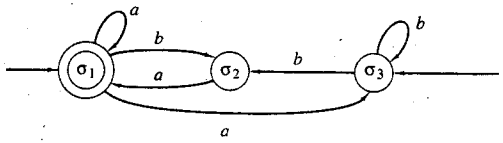


FIGURA 10.5.5 Un autómata de estado finito no determinista que acepta L^R.

Después de determinar un autómata de estado finito equivalente y eliminar los estados inalcanzables, obtenemos el autómata de estado finito equivalente de la figura 10.5.6.

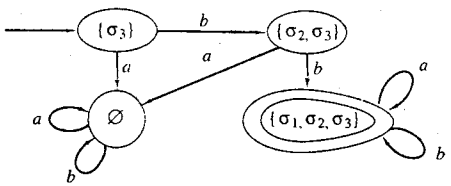


FIGURA 10.5.6 Un autómata de estado finito que acepta L^R. □

EJEMPLO 10.5.8

Sea L el conjunto de cadenas aceptadas por el autómata de estado finito A de la figura 10.5.7. Construir un autómata de estado finito no determinista que acepte las cadenas

$$L^R = \{x_n \cdots x_1 \mid x_1 \cdots x_n \in L\}.$$

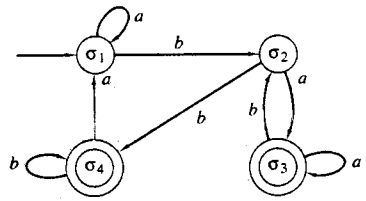


FIGURA 10.5.7 El autómata de estado finito del ejemplo 10.5.8, que acepta a L.

Si A sólo tuviera un estado de aceptación, podríamos utilizar el procedimiento del ejemplo 10.5.7 para construir el autómata de estado finito no determinista deseado. Así, primero construimos un autómata de estado finito no determinista equivalente a A con un estado de aceptación. Para hacer esto, introducimos un estado adicional σ_5. Luego hacemos que los caminos que terminan en σ_3 o σ_4 terminen de manera opcional en σ_5 (véase la figura 10.5.8). El autómata de estado finito no determinista deseado se obtiene a partir de la figura 10.5.8 por el método del ejemplo 10.5.7 (véase la figura 10.5.9). Por supuesto, si se desea, podríamos construir un autómata de estado finito equivalente. □

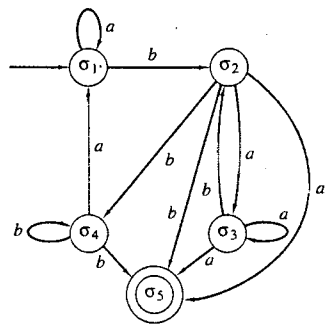


FIGURA 10.5.8 Un autómata de estado finito no determinista con un estado de aceptación equivalente al autómata de estado finito de la figura 10.5.7.

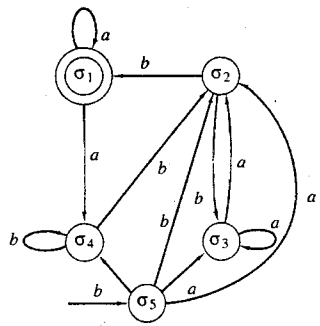


FIGURA 10.5.9 Un autómata de estado finito no determinista que acepta L^R.

Ejercicios

1. Determine los autómatas de estado finito equivalentes a los autómatas de estado finito no deterministas de los ejercicios 1-10, sección 10.4.

En los ejercicios 2-6, determine autómatas de estado finito que acepten las cadenas generadas por las gramáticas regulares.

2. La gramática del ejercicio 1, sección 10.3
3. La gramática del ejercicio 5, sección 10.3
4. $<S> ::= a<A> \mid a<B>$
   $<A> ::= a<B> \mid b<S> \mid b$
   $<B> ::= b<S> \mid b$
   con símbolo inicial $<S>$
5. $<S> ::= a<S> \mid a<A> \mid b<C> \mid a$
   $<A> ::= b<A> \mid a<C>$
   $<B> ::= a<S> \mid a$
   $<C> ::= a<B> \mid a<C>$
   con símbolo inicial $<S>$
6. $<S> ::= a<A> \mid a<B>$
   $<A> ::= b<S> \mid b$
   $<B> ::= a<B> \mid a<C>$
   $<C> ::= a<S> \mid b<A> \mid a<C> \mid a$
   con símbolo inicial $<S>$
7. Determine los autómatas de estado finito que acepten las cadenas de los ejercicios 21-29, sección 10.4.
8. Elimine los estados inalcanzables del autómata de estado finito de la figura 10.5.3, para determinar un autómata de estado finito equivalente, pero más sencillo.
9. Muestre que el autómata de estado finito no determinista de la figura 10.5.5 acepta una cadena α sobre $\{a, b\}$ si y sólo si α comienza con bb.
10. ★ Caracterice las cadenas aceptadas por los autómatas de estado finito no deterministas de las figuras 10.5.7 y 10.5.9.

En los ejercicios 11-21, determine un autómata de estado finito no determinista que acepte al conjunto dado de cadenas. Si S_1 y S_2 son conjuntos de cadenas, hacemos

$$S_1^+ = \{u_1u_2\cdots u_n \mid u_i \in S_1, n \in \{1, 2, \ldots\}\}; \quad S_1S_2 = \{uv \mid u \in S_1, v \in S_2\}.$$

11. $\mathrm{Ac}(A)^R$, donde A es el autómata del ejercicio 4, sección 10.2
12. $\mathrm{Ac}(A)^R$, donde A es el autómata del ejercicio 5, sección 10.2
13. $\mathrm{Ac}(A)^R$, donde A es el autómata del ejercicio 6, sección 10.2
14. $\mathrm{Ac}(A)^+$, donde A es el autómata del ejercicio 4, sección 10.2
15. $\mathrm{Ac}(A)^+$, donde A es el autómata del ejercicio 5, sección 10.2
16. $\mathrm{Ac}(A)^+$, donde A es el autómata del ejercicio 6, sección 10.2
17. $\mathrm{Ac}(A)^+$, donde A es el autómata de la figura 10.5.7
18. $\mathrm{Ac}(A_1)\mathrm{Ac}(A_2)$, donde A_1 es el autómata del ejercicio 4, sección 10.2, y A_2 es el autómata del ejercicio 5, sección 10.2
19. $\mathrm{Ac}(A_1)\mathrm{Ac}(A_2)$, donde A_1 es el autómata del ejercicio 5, sección 10.2, y A_2 es el autómata del ejercicio 6, sección 10.2

20. $\mathrm{Ac}(A_1)\mathrm{Ac}(A_1)$, donde A_1 es el autómata del ejercicio 6, sección 10.2
21. $\mathrm{Ac}(A_1)\mathrm{Ac}(A_2)$, donde A_1 es el autómata de la figura 10.5.7 y A_2 es el autómata del ejercicio 5, sección 10.2
22. Determine una gramática regular que genere el lenguaje L^R, donde L es el lenguaje generado por la gramática del ejercicio 5, sección 10.3.
23. Determine una gramática regular que genere el lenguaje L^+, donde L es el lenguaje generado por la gramática del ejercicio 5, sección 10.3.
24. Sea L_1 (respectivamente, L_2) el lenguaje generado por la gramática del ejercicio 5, sección 10.3 (respectivamente, ejemplo 10.5.5). Determine una gramática regular que genere el lenguaje L_1L_2.
25. ★ Muestre que el conjunto
$$L = \{x_1\cdots x_n \mid x_1\cdots x_n = x_n\cdots x_1\}$$
de cadenas sobre $\{a, b\}$ no es un lenguaje regular.
26. Muestre que si L_1 y L_2 son lenguajes regulares sobre I y S es el conjunto de todas las cadenas sobre I, entonces cada uno de los conjuntos $S - L_1$, $L_1 \cup L_2$, $L_1 \cap L_2$, L_1^+ y L_1L_2 es un lenguaje regular.
27. ☆ Muestre, mediante un ejemplo, que existen lenguajes libres de contexto L_1 y L_2 tales que $L_1 \cap L_2$ no sea libre de contexto.
28. ★ Demuestre o refute la afirmación: Si L es un lenguaje regular, también lo es
$$\{u^n \mid u \in L, n \in \{1, 2, \ldots\}\}.$$

NOTAS

Las referencias generales acerca de los autómatas, las gramáticas y los lenguajes son [Carroll; Cohen; Davis; Hopcroft; Kelley; McNaughton; Sudkamp; y Wood].

El desarrollo sistemático de la geometría fractal fue iniciado por Benoit B. Mandelbrot (véase [Mandelbrot, 1977, 1982]).

Una máquina de estado finito tiene una memoria primitiva interna, en el sentido de que recuerda el estado en el que se encuentra. Al permitir la existencia de una memoria externa, en la cual la máquina pueda leer y escribir datos, podemos definir máquinas más poderosas. También se obtienen otras mejoras permitiendo que la máquina revise la cadena de entrada en cualquier dirección, o bien permitiendo que la máquina modifique la cadena de entrada. Entonces, es posible caracterizar las clases de máquinas que aceptan lenguajes libres de contexto, lenguajes sensibles al contexto y lenguajes generados por gramáticas con estructura de frases.

Las **máquinas de Turing** forman una clase de máquinas con particular importancia. Como una máquina de estado finito, una máquina de Turing siempre se encuentra en un estado particular. Se supone que la cadena de entrada de una máquina de Turing está en una banda que es infinita en ambas direcciones. Una máquina de Turing revisa un carácter a la vez y, después de revisarlo se detiene o realiza una, ninguna o todas las acciones siguientes: modificar el carácter, moverse una posición a la izquierda o a la derecha, o cambiar estados. En particular, se puede modificar la cadena de entrada. Una máquina de Turing T acepta una cadena α si, al introducir α como entrada a T, T se detiene en un estado de aceptación. Se puede mostrar que un lenguaje L es generado por una gramática con estructura de frases si y sólo si existe una máquina de Turing que acepta L.

La importancia real de las máquinas de Turing surge de la creencia, ampliamente compartida, en el sentido de que cualquier función que se pueda calcular mediante alguna computadora digital, inclusive hipotética, se puede calcular mediante alguna máquina de Turing. Esta última afirmación se conoce como la **hipótesis de Turing** o **tesis de Church**. La tesis de Church implica que una máquina de Turing es el modelo abstracto correcto de una computadora digital. Estas ideas también sugieren la siguiente definición formal de algoritmo: Un **algoritmo** es una máquina de Turing tal que, al darle una cadena de entrada, se detendrá en algún momento.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 10.1

Retraso unitario de tiempo
Sumador en serie
Máquina de estado finito
Símbolo de entrada
Símbolo de salida
Estado
Función de estado siguiente
Función de salida
Estado inicial
Diagrama de transición
Cadenas de entrada y de salida para una máquina de estado finito
Flip-flop *SR*

Sección 10.2

Autómata de estado finito
Estado de aceptación
Cadena aceptada por un autómata de estado finito
Autómata de estado finito equivalente

Sección 10.3

Lenguaje natural
Lenguaje formal
Gramática con estructura de frases
Símbolo no terminal
Símbolo terminal
Producción
Símbolo inicial
Cadena derivable de manera directa
Cadena derivable
Derivación
Lenguaje generado por una gramática
Forma normal de Backus (= Forma de Backus-Naur = FBN)
Gramática sensible al contexto (= gramática de tipo 1)
Gramática libre de contexto (= gramática de tipo 2)
Gramática regular (= gramática de tipo 3)
Lenguaje sensible al contexto
Lenguaje libre de contexto
Lenguaje regular
Gramática Lindenmayer interactiva libre de contexto
Copo de nieve de von Koch
Curvas fractales

Sección 10.4

Dado un autómata de estado finito A, cómo construir una gramática regular G, tal que el conjunto de cadenas aceptadas por A sea igual al lenguaje generado por G (véase el teorema 10.4.2)
Autómata de estado finito no determinista
Cadena aceptada por un autómata de estado finito no determinista
Autómata de estado finito no determinista equivalente
Dada una gramática regular G, cómo construir un autómata de estado finito no determinista A tal que el lenguaje generado por G sea igual al conjunto de cadenas aceptadas por A (véase el teorema 10.4.11)

Sección 10.5

Dado un autómata de estado finito no determinista, cómo construir un autómata de estado finito determinista equivalente (véase el teorema 10.5.3)
Un lenguaje L es regular si y sólo si existe un autómata de estado finito que acepta las cadenas en L

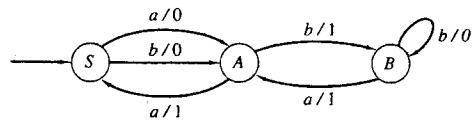
AUTOEVALUACIÓN DEL CAPÍTULO

Sección 10.1

1. Trace el diagrama de transición de la máquina de estado finito $(I, O, S, f, g, \sigma_0)$, donde $I = \{a, b\}$, $O = \{0, 1\}$ y $S = \{\sigma_0, \sigma_1\}$.

	f		g	
S \ I	a	b	a	b
σ_0	σ_1	σ_0	0	1
σ_1	σ_0	σ_1	1	0

2. Determine los conjuntos I, O y S, el estado inicial, la tabla que define el estado siguiente, y las funciones de salida para la siguiente máquina de estado finito.



3. Para la máquina de estado finito del ejercicio 1, determine la cadena de salida para la cadena de entrada *bbaa*.

4. Diseñe una máquina de estado finito cuya entrada sea una cadena de bits, tal que produzca 0 como salida a partir de que vea 001, y 1 en caso contrario.

Sección 10.2

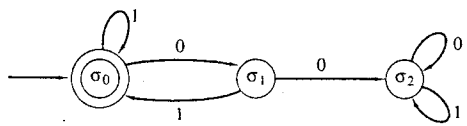
5. Trace el diagrama de transición del autómata de estado finito (I, S, f, A, S), donde $I = \{0, 1\}$, $S = \{S, A, B\}$ y $A = \{A\}$.

	f	
S \ I	**0**	**1**
S	A	S
A	S	B
B	A	S

6. ¿Es aceptada la cadena 11010 por el autómata de estado finito del ejercicio 5?

7. Dibuje el diagrama de transición de un autómata de estado finito que acepte al conjunto de cadenas sobre $\{0, 1\}$ que contengan un número par de ceros y un número impar de unos.

8. Caracterice el conjunto de cadenas aceptadas por el siguiente autómata de estado finito.



3. $I = \{a, b\}$, $S = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $A = \{\sigma_1\}$

S \ I	a	b
σ_0	$\varnothing$	$\{\sigma_3\}$
σ_1	$\{\sigma_1, \sigma_2\}$	$\{\sigma_3\}$
σ_2	$\varnothing$	$\{\sigma_0, \sigma_1, \sigma_3\}$
σ_3	$\varnothing$	$\varnothing$

4. $I = \{a, b, c\}$, $S = \{\sigma_0, \sigma_1, \sigma_2\}$, $A = \{\sigma_0\}$

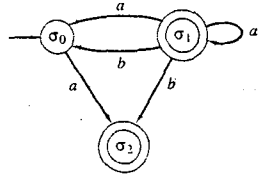
S \ I	a	b	c
σ_0	$\{\sigma_1\}$	$\varnothing$	$\varnothing$
σ_1	$\{\sigma_0\}$	$\{\sigma_2\}$	$\{\sigma_0, \sigma_2\}$
σ_2	$\{\sigma_0, \sigma_1, \sigma_2\}$	$\{\sigma_0\}$	$\{\sigma_0\}$

5. $I = \{a, b, c\}$, $S = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $A = \{\sigma_0, \sigma_3\}$

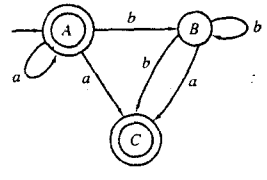
S \ I	a	b	c
σ_0	$\{\sigma_1\}$	$\{\sigma_0, \sigma_1, \sigma_3\}$	$\varnothing$
σ_1	$\{\sigma_2, \sigma_3\}$	$\varnothing$	$\varnothing$
σ_2	$\varnothing$	$\{\sigma_0, \sigma_3\}$	$\{\sigma_1, \sigma_2\}$
σ_3	$\varnothing$	$\varnothing$	$\{\sigma_0\}$

Para cada autómata de estado finito no determinista de los ejercicios 6-10, determine los conjuntos I, S y A, el estado inicial, y la tabla que define la función de estado siguiente.

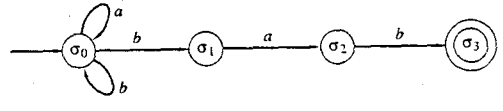
6.



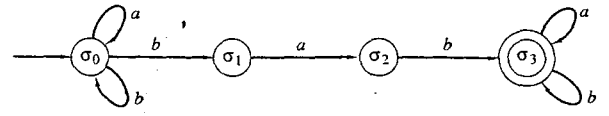
7.



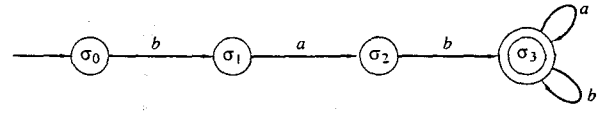
8.



9.



10.



11. Escriba las gramáticas regulares dadas por los autómatas de estado finito de los ejercicios 4-9, sección 10.2.

12. Represente las gramáticas de los ejercicios 1 y 5, sección 10.3, y el ejemplo 10.3.14 mediante autómatas de estado finito no deterministas.

13. ¿Es aceptada la cadena *bbabbb* por el autómata de estado finito no determinista de la figura 10.4.2? Demuestre su respuesta.

14. ¿Es aceptada la cadena *bbabab* por el autómata de estado finito no determinista de la figura 10.4.2? Demuestre su respuesta.

15. Demuestre que una cadena α sobre $\{a, b\}$ es aceptada por el autómata de estado finito no determinista de la figura 10.4.2 si y sólo si α contiene exactamente una a y termina con b.

16. ¿Es aceptada la cadena *aaabba* por el autómata de estado finito no determinista de la figura 10.4.3? Demuestre su respuesta.

17. ¿Es aceptada la cadena *aaaab* por el autómata de estado finito no determinista de la figura 10.4.3? Demuestre su respuesta.

18. Caracterice las cadenas aceptadas por el autómata de estado finito no determinista de la figura 10.4.3.

19. Muestre que las cadenas aceptadas por el autómata de estado finito no determinista del ejercicio 8 son precisamente aquellas cadenas sobre $\{a, b\}$ que terminan con *bab*.

☆ 20. Caracterice las cadenas aceptadas por los autómatas de estado finito no deterministas de los ejercicios 1-7, 9 y 10.

Diseñe autómatas de estado finito no deterministas que acepten las cadenas sobre $\{a, b\}$ con las propiedades dadas en los ejercicios 21-29.

21. Que comiencen con *abb* o *ba*

22. Que terminen con *abb* o *ba*

23. Que contengan *abb* o *ba*

★ **24.** Que contengan *bab* y *bb*

25. Que tengan cada *b* precedida y seguida por una *a*

26. Que comiencen con *abb* y terminen con *ab*

★ **27.** Que comiencen con *ab* pero que no terminen con *ab*

28. Que no contengan *ba* ni *bbb*

☆ 29. Que no contengan *abba* ni *bbb*

30. Escriba gramáticas regulares que generen las cadenas de los ejercicios 21-29.

10.5 RELACIONES ENTRE LENGUAJES Y AUTÓMATAS

En la sección anterior mostramos (teorema 10.4.2) que si A es un autómata de estado finito, entonces existe una gramática regular G, tal que $L(G) = \text{Ac}(A)$. Como un recíproco parcial, mostramos (teorema 10.4.11) que si G es una gramática regular, entonces existe un autómata de estado finito no determinista A tal que $L(G) = \text{Ac}(A)$. En esta sección mostramos (teorema 10.5.4) que si G es una gramática regular, entonces existe un autómata de estado finito A tal que $L(G) = \text{Ac}(A)$. Este resultado será deducido del teorema 10.4.11, mostrando que cualquier autómata de estado finito no determinista se puede convertir en un autómata de estado finito equivalente (teorema 10.5.3). Primero ilustraremos el método por medio de un ejemplo.

EJEMPLO 10.5.1

Determinar un autómata de estado finito equivalente al autómata de estado finito no determinista de la figura 10.4.2.

El conjunto de símbolos de entrada no se modifica. Los estados constan de todos los subconjuntos

$$\varnothing, \quad \{\sigma\}, \quad \{C\}, \quad \{F\}, \quad \{\sigma, C\}, \quad \{\sigma, F\}, \quad \{C, F\}, \quad \{\sigma, C, F\}$$

del conjunto original $S = \{\sigma, C, F\}$ de estados. El estado inicial es $\{\sigma\}$. Los estados de aceptación son todos los subconjuntos

$$\{F\}, \quad \{\sigma, F\}, \quad \{C, F\}, \quad \{\sigma, C, F\}$$

de S que contienen un estado de aceptación del autómata de estado finito no determinista original.

Trazamos una arista de X a Y y la etiquetamos como x si $X = \varnothing = Y$ o si

$$\bigcup_{S \in X} f(S, x) = Y.$$

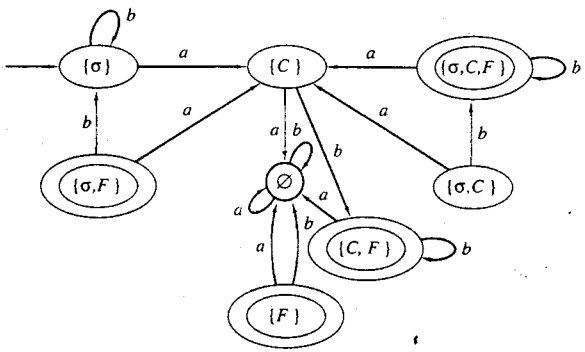


FIGURA 10.5.1 Un autómata de estado finito equivalente al autómata de estado finito no determinista de la figura 10.4.2.

Obtenemos el autómata de estado finito de la figura 10.5.1. Los estados

$$\{\sigma, F\}, \quad \{\sigma, C\}, \quad \{\sigma, C, F\}, \quad \{F\},$$

que nunca se pueden alcanzar, se pueden eliminar. Así, obtenemos el autómata de estado finito equivalente y simplificado de la figura 10.5.2.

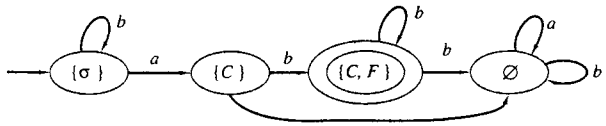


FIGURA 10.5.2 Una versión simplificada de la figura 10.5.1 (con los estados inalcanzables eliminados). □

EJEMPLO 10.5.2

El autómata de estado finito equivalente al autómata de estado finito no determinista del ejemplo 10.4.6 aparece en la figura 10.5.3.

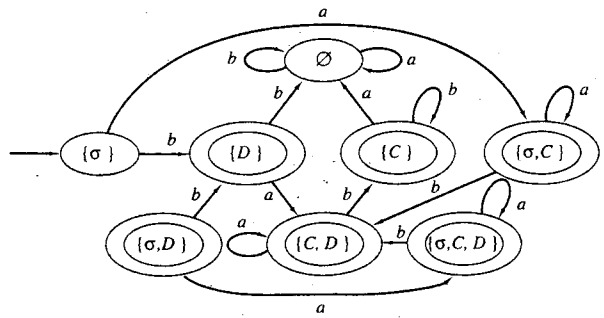


FIGURA 10.5.3 Un autómata de estado finito equivalente al autómata de estado finito no determinista de la figura 10.4.6. □

Ahora justificaremos formalmente el método de los ejemplos 10.5.1 y 10.5.2.

TEOREMA 10.5.3

Sea $A = (I, S, f, A, \sigma)$ un autómata de estado finito no determinista. Sean

(a) $S' = P(S)$.

(b) $S' = I$.

(c) $\sigma' = \{\sigma\}$.

(d) $A' = \{X \subseteq S \mid X \cap A \neq \varnothing\}$.

(e) $$f'(X, x) = \begin{cases} \varnothing & \text{si } X = \varnothing \\ \bigcup_{S \in X} f(S, x) & \text{si } X \neq \varnothing. \end{cases}$$

Entonces el autómata de estado finito $A' = (I', S', f', A', \sigma')$ es equivalente a A.

Demostración. Supongamos que la cadena $\alpha = x_1 \cdots x_n$ es aceptada por A. Entonces existen estados $\sigma_0, \ldots, \sigma_n \in S$ tales que

$$\sigma_0 = \sigma;$$
$$\sigma_i \in f(\sigma_{i-1}, x_i) \quad \text{para } i = 1, \ldots, n;$$
$$\sigma_n \in A.$$

Hacemos $Y_0 = \{\sigma_0\}$ y

$$Y_i = f'(Y_{i-1}, x_i) \quad \text{para } i = 1, \ldots, n.$$

Como

$$Y_1 = f'(Y_0, x_1) = f'(\{\sigma_0\}, x_1) = f(\sigma_0, x_1),$$

esto implica que $\sigma_1 \in Y_1$. Ahora,

$$\sigma_2 \in f(\sigma_1, x_2) \subseteq \bigcup_{S \in Y_1} f(S, x_2) = f'(Y_1, x_2) = Y_2.$$

De nuevo,

$$\sigma_3 \in f(\sigma_2, x_3) \subseteq \bigcup_{S \in Y_2} f(S, x_3) = f'(Y_2, x_3) = Y_3.$$

El argumento puede continuar (formalmente utilizaríamos la inducción) para mostrar que $\sigma_n \in Y_n$. Como σ_n es un estado de aceptación en A, Y_n es un estado de aceptación en A'. Así, en A', tenemos

$$f'(\sigma', x_1) = f'(Y_0, x_1) = Y_1$$
$$f'(Y_1, x_2) = Y_2$$
$$\vdots$$
$$f'(Y_{n-1}, x_n) = Y_n.$$

Por tanto, α es aceptada por A'.

Ahora, supongamos que la cadena $\alpha = x_1 \cdots x_n$ es aceptada por A'. Entonces existen subconjuntos $Y_0, \ldots, Y_n$ de S tales que

$$Y_0 = \sigma' = \{\sigma\};$$
$$f'(Y_{i-1}, x_i) = Y_i \quad \text{para } i = 1, \ldots, n;$$

existe un estado $\sigma_n \in Y_n \cap A$.

Como

$$\sigma_n \in Y_n = f'(Y_{n-1}, x_n) = \bigcup_{S \in Y_{n-1}} f(S, x_n),$$

existe $\sigma_{n-1} \in Y_{n-1}$ con $\sigma_n \in f(\sigma_{n-1}, x_n)$. De manera análoga, como

$$\sigma_{n-1} \in Y_{n-1} = f'(Y_{n-2}, x_{n-1}) = \bigcup_{S \in Y_{n-2}} f(S, x_{n-1}),$$

existe $\sigma_{n-2} \in Y_{n-2}$ con $\sigma_{n-1} \in f(\sigma_{n-2}, x_{n-1})$. Continuamos de esta manera para obtener

$$\sigma_i \in Y_i \quad \text{para } i = 0, \ldots, n,$$

con

$$\sigma_i \in f(\sigma_{i-1}, x_i) \quad \text{para } i = 1, \ldots, n.$$

En particular,

$$\sigma_0 \in Y_0 = \{\sigma\}.$$

Así, $\sigma_0 = \sigma$, el estado inicial en A. Como σ_n es un estado de aceptación en A, la cadena α es aceptada por A. ■

El siguiente teorema resume estos resultados y los de la sección anterior.

TEOREMA 10.5.4

Un lenguaje L es regular si y sólo si existe un autómata de estado finito que acepte precisamente las cadenas en L.

Demostración. Este teorema enuncia de otra forma los teoremas 10.4.2, 10.4.11 y 10.5.3. ■

EJEMPLO 10.5.5

Determinar un autómata de estado finito A que acepte precisamente las cadenas generadas por la gramática regular G con las producciones

$$\sigma \to b\sigma, \quad \sigma \to aC, \quad C \to bC, \quad C \to b.$$

El símbolo inicial es σ, el conjunto de símbolos terminales $\{a, b\}$, y el conjunto de símbolos no terminales es $\{\sigma, C\}$.

El autómata de estado finito no determinista A' que acepta $L(G)$ aparece en la figura 10.4.2. Un autómata de estado finito equivalente a A' aparece en la figura 10.5.1 y un autómata de estado finito simplificado equivalente A aparece en la figura 10.5.2. El autómata de estado finito A acepta precisamente las cadenas generadas por G. □

Cerramos esta sección dando algunas aplicaciones de los métodos y teoría hasta ahora desarrollados.

EJEMPLO 10.5.6 ***Un lenguaje que no es regular***

Mostrar que el lenguaje

$$L = \{a^n b^n \mid n = 1, 2, \ldots\}$$

no es regular.

Si L es regular, existe un autómata de estado finito A tal que $\text{Ac}(A) = L$. Supongamos que A tiene k estados. La cadena $\alpha = a^k b^k$ es aceptada por A. Consideremos el camino P que representa a α. Como existen k estados, algún estado σ es visitado al menos dos veces en la parte del camino que representa a a^k. Así, existe un ciclo C, tal que todas sus aristas tienen la etiqueta a y que contiene a σ. Modificamos el camino P para obtener un camino P', como sigue. Al llegar a σ en P, seguimos C. Después de regresar a σ sobre C, continuamos sobre P hasta el final. Si la longitud de C es j, el camino P' representa la cadena $\alpha' = a^{j+k} b^k$. Como P y P' terminan en el mismo estado σ' y σ' es un estado de aceptación, α' es aceptada por A. Esto es una contradicción, pues α' no es de la forma $a^n b^n$. Por tanto, L no es regular. □

11

GEOMETRÍA COMPUTACIONAL

11.1 EL PROBLEMA DEL PAR MÁS CERCANO
† 11.2 UNA COTA INFERIOR PARA EL PROBLEMA DEL PAR MÁS CERCANO
11.3 UN ALGORITMO PARA CALCULAR LA CUBIERTA CONVEXA
NOTAS
CONCEPTOS BÁSICOS DEL CAPÍTULO
AUTOEVALUACIÓN DEL CAPÍTULO

Sin embargo, demostré sin sombra de duda y con una lógica geométrica que existía una clave.

—de The Caine Mutiny

La **geometría computacional** se encarga del diseño y análisis de algoritmos para resolver problemas geométricos. Los algoritmos geométricos eficientes son muy útiles en campos como la graficación por computadora, la estadística, el procesamiento de imágenes y en el diseño con integración a muy grande escala (very-large-scale-integration, VLSI). En este capítulo presentaremos una introducción a este fascinante tema.

El problema del par más cercano proporciona un ejemplo de problema en geometría computacional: Dados n puntos en el plano, determinar el par más cercano. Además de este problema, consideraremos el problema de determinar la cubierta convexa.

11.1 *EL PROBLEMA DEL PAR MÁS CERCANO*

El **problema del par más cercano** se puede enunciar fácilmente: Dados n puntos en el plano, determinar un par más cercano (véase la figura 11.1.1). (Decimos un par más cercano pues es posible que varios pares tengan la misma distancia mínima.) Nuestra medida de distancia es la distancia euclidiana ordinaria.

Una forma de resolver este problema es enumerar la distancia entre cada par y elegir el mínimo en esta lista de distancias. Como existen $C(n, 2) = n(n-1)/2 =$

† Esta sección puede omitirse sin pérdida de continuidad.

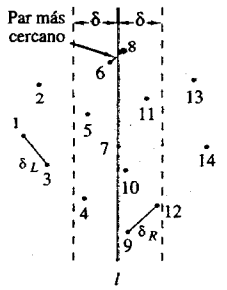


FIGURA 11.1.1
n puntos en el plano. El problema consiste en determinar un par más cercano. Para este conjunto, el par más cercano es 6 y 8. La líneal divide a los puntos en dos partes aproximadamente iguales. El par más cercano en la mitad izquierda es 1 y 3, que están a una distancia δ_L. El par más cercano en la mitad derecha es 9 y 12, que están a una distancia δ_R. Cualquier par (por ejemplo, 6 y 8) cuya distancia sea menor que $\delta = \text{mín}\{\delta_L, \delta_R\}$ debe estar en la franja vertical de ancho 2δ con centro en *l*.

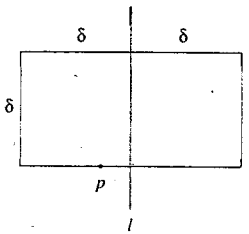


FIGURA 11.1.2
Cualquier punto *q* posterior a *p* cuya distancia a *p* sea menor que δ debe estar dentro del rectángulo.

$\Theta(n^2)$ pares, el tiempo necesario para este algoritmo "enumera todo" es $\Theta(n^2)$. Podemos hacer algo mejor: daremos un algoritmo "divide y vencerás" para obtener el par más cercano, cuyo tiempo en el peor de los casos es $\Theta(n \lg n)$. Primero analizaremos el algoritmo y luego daremos una descripción más precisa de éste mediante un seudocódigo.

Nuestro algoritmo comienza determinando una recta vertical *l* que divide a los puntos en dos conjuntos aproximadamente iguales (véase la figura 11.1.1). [Si *n* es par, dividimos los puntos en dos partes, cada una de las cuales tiene $n/2$ puntos. Si *n* es impar, separamos los puntos en dos partes, una con $(n+1)/2$ puntos y la otra con $(n-1)/2$ puntos.]

Ahora, resolvemos el problema de manera recursiva para cada una de las partes. Sea δ_L la distancia entre un par más cercano en la parte izquierda; sea δ_R la distancia entre un par más cercano en la parte derecha; y sea

$$\delta = \text{mín}\{\delta_L, \delta_R\}.$$

Por desgracia, podría ocurrir que δ no fuese la distancia entre un par más cercano en el conjunto original de puntos, pues un par de puntos, uno de la parte izquierda y el otro de la parte derecha, podrían estar a una distancia menor que δ (véase la figura 11.1.1). Así, debemos considerar las distancias entre los puntos en lados opuestos de la recta *l*.

Observemos primero que si la distancia entre un par de puntos es menor que δ, entonces los puntos deben estar en la franja vertical de ancho 2δ con centro en *l* (véase la figura 11.1.1). (Cualquier punto que no esté en esta franja estará al menos a distancia δ de cualquier punto del otro lado de *l*.) Así, podemos restringir nuestra búsqueda de un par a una distancia menor que δ a los puntos de esta franja.

Si existen *n* puntos en la franja y verificamos *todos* los pares de la franja, el tiempo necesario para procesar los puntos de la franja en el peor de los casos es $\Theta(n^2)$. En este caso, el tiempo de nuestro algoritmo en el peor de los casos es $\Omega(n^2)$, que al menos es tan malo como el de la búsqueda exhaustiva; así, debemos evitar la verificación de todos los pares de la franja.

Ordenamos los puntos de la franja en orden creciente de sus coordenadas *y* y luego analizamos los puntos en este orden. Al examinar un punto *p* de esta franja, cualquier punto *q* posterior a *p* cuya distancia a *p* sea menor que δ debe estar estrictamente dentro o en la base del rectángulo de altura δ cuya base contiene a *p* y cuyos lados verticales están a una distancia δ de *l* (véase la figura 11.1.2). (No necesitamos calcular la distancia entre *p* y los puntos por debajo de *p*. Estas distancias ya se habrán considerado anteriormente, pues estamos examinando los puntos en orden creciente de sus coordenadas *y*.) Mostraremos que este rectángulo contiene a lo más ocho puntos, incluyendo a *p*, de modo que si calculamos las distancias entre *p* y los siguientes siete puntos en la franja, podremos estar seguros de que calcularemos las distancias entre *p* y todos los puntos del rectángulo. Por supuesto, si existen menos de siete puntos después de *p* en la lista, calculamos las distancias entre *p* y los puntos restantes. Al restringir la búsqueda en la franja de esta forma, el tiempo necesario para procesar los puntos de la franja es $O(n)$. (Como a lo más existen *n* puntos en la franja, el tiempo necesario para procesar los puntos de la franja es a lo más $7n$.)

Mostraremos que el rectángulo de la figura 11.1.2 contiene a lo más ocho puntos. La figura 11.1.3 muestra el rectángulo de la figura 11.1.2 dividido en ocho cuadrados iguales. Observe que la longitud de una diagonal de un cuadrado es

$$\left(\left(\frac{\delta}{2}\right)^2 + \left(\frac{\delta}{2}\right)^2\right)^{1/2} = \frac{\delta}{\sqrt{2}} < \delta;$$

así, cada cuadrado contiene a lo más un punto. Por tanto, el rectángulo $2\delta \times \delta$ contiene a lo más ocho puntos.

EJEMPLO 11.1.1

Ahora mostraremos la forma en que el algoritmo del par más cercano determina un par más cercano para los datos de la figura 11.1.1.

El algoritmo comienza determinando una recta vertical *l* que separa a los puntos en dos partes iguales,

$$S_1 = \{1, 2, 3, 4, 5, 6, 7\}, \qquad S_2 = \{8, 9, 10, 11, 12, 13, 14\}.$$

Para estos datos, existen muchas opciones posibles para la línea divisoria. La recta particular aquí elegida pasa por el punto 7.

Ahora, resolvemos el problema de manera recursiva para S_1 y S_2. El par más cercano de puntos en S_1 es 1 y 3. Sea δ_L la distancia entre los puntos 1 y 3. El par más cercano de puntos en S_2 es 9 y 12. Sea δ_R la distancia entre los puntos 9 y 12. Sea

$$\delta = \text{mín}\{\delta_L, \delta_R\} = \delta_L.$$

Ahora ordenamos los puntos de la franja vertical de ancho 2δ con centro en *l* en orden creciente de sus coordenadas *y*:

9, 12, 4, 10, 7, 5, 11, 6, 8.

Ahora analizamos los puntos en este orden. Calculamos las distancias entre cada punto y los siguientes siete puntos, o entre cada punto y los puntos restantes si existen menos de siete puntos después de él.

Primero calculamos las distancias de 9 a cada uno de los puntos 12, 4, 10, 7, 5, 11 y 6. Como cada una de estas distancias es mayor que δ, en este punto no hemos encontrado un par más cercano.

Ahora calculamos las distancias de 12 a cada uno de los puntos 4, 10, 7, 5, 11, 6 y 8. Como cada una de estas distancias es mayor que δ, en este punto aún no hemos encontrado un par más cercano.

Luego calculamos las distancias de 4 a cada uno de los puntos 10, 7, 5, 11, 6 y 8. Como cada una de estas distancias es mayor que δ, en este punto aún no hemos encontrado un par más cercano.

Ahora calculamos las distancias de 10 a cada uno de los puntos 7, 5, 11, 6 y 8. Como la distancia entre 10 y 7 es menor que δ, hemos descubierto un par más cercano. Actualizamos δ como la distancia entre los puntos 10 y 7.

Ahora calculamos las distancias de 7 a cada uno de los puntos 5, 11, 6 y 8. Como cada una de estas distancias es mayor que δ, no hemos encontrado un par más cercano.

Ahora calculamos las distancias de 5 a cada uno de los puntos 11, 6 y 8. Como cada una de estas distancias es mayor que δ, no hemos encontrado un par más cercano.

Ahora calculamos las distancias de 11 a cada uno de los puntos 6 y 8. Como cada una de estas distancias es mayor que δ, no hemos encontrado un par más cercano.

Ahora calculamos la distancia de 6 a 8. Como la distancia entre 6 y 8 es menor que δ, hemos descubierto un par más cercano. Actualizamos δ como la distancia entre los puntos 6 y 8. Como ya no hay más puntos en la franja por considerar, el algoritmo termina. El par más cercano es 6 y 8 y la distancia entre ellos es δ. □

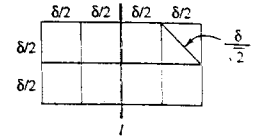


FIGURA 11.1.3
El rectángulo grande contiene a lo más ocho puntos, pues cada cuadrado contiene a lo más un punto.

Antes de enunciar formalmente el algoritmo del par más cercano, debemos resolver varios detalles técnicos.

Para terminar la recursión, verificamos la cantidad de puntos en la entrada y si existen tres o menos puntos, determinamos un par más cercano en forma directa. La separación de los datos y el uso de la recursión sólo cuando existen cuatro o más puntos nos garantiza que cada una de las dos partes contiene al menos un par de puntos y, por tanto, que existe un par más cercano en cada parte.

Antes de llamar al procedimiento recursivo, ordenamos todo el conjunto de puntos mediante sus abscisas. Esto facilita la separación de los puntos en dos partes casi iguales.

Utilizamos el ordenamiento por fusión (véase la sección 5.3) para ordenar según las coordenadas y. Sin embargo, en vez de ordenar cada vez, examinamos los puntos en la franja vertical, y suponemos, como en el ordenamiento por fusión, que cada mitad está ordenada según sus coordenadas y. Luego, basta realizar la fusión de las dos mitades para ordenar todos los puntos según sus coordenadas y.

Ahora enunciaremos formalmente el algoritmo del par más cercano. Para que nuestra descripción sea más sencilla, nuestra versión produce como salida la distancia entre un par más cercano, pero no el propio par más cercano. Dejaremos esta mejora como ejercicio (ejercicio 5).

ALGORITMO 11.1.2 ***Determinación de la distancia entre un par de puntos más cercanos***

Entrada: $p_1, \ldots, p_n$ ($n \geq 2$ puntos en el plano)

Salida: δ, la distancia entre un par de puntos más cercanos

```
procedure closest_pair (p, n)
  ordenar p_1, ..., p_n por su abscisa
  return (rec_cl_pair (p, 1, n))
end closest_pair
procedure rec_cl_pair (p, i, j)
  // La entrada es la sucesión p_i, ..., p_j de puntos en el plano
  // ordenados según su abscisa.
  // Al concluir rec_cl_pair, la sucesión queda ordenada
  // según su coordenada y.
  // rec_cl_pair regresa la distancia entre un par más cercano
  // en la entrada.
  // Denotemos la abscisa del punto p como p.x.
  // caso trivial (3 o menos puntos)
  if j - i < 3 then
    begin
    ordenar p_i, ..., p_j según su coordenada y
    determinar de manera directa la distancia δ entre un par más cercano
    return(δ)
    end
  // dividir
  k := ⌊(i + j)/2⌋
  l := p_k.x
  δ_L := rec_cl_pair (p, i, k)
  δ_R := rec_cl_pair (p, k + 1, j)
  δ := mín {δ_L, δ_R}
```



```
  // ahora, p_i, ..., p_k están ordenados según su coordenada y
  // p_{k+1}, ..., p_j están ordenados según su coordenada y
  se realiza la fusión de p_i, ..., p_k y p_{k+1}, ..., p_j según su coordenada y
  // suponga que el resultado de la fusión se guarda de nuevo en
  // p_i, ..., p_j
  // ahora, p_i, ..., p_j están ordenados según su coordenada y
  // se guardan los puntos de la franja vertical en v
  t := 0
  for k := i to j do
    if p_k.x > l - δ and p_k.x < l + δ then
      begin
      t := t + 1
      v_t := p_k
      end
  // los puntos en la franja son v_1, ..., v_t
  // se busca el par más cercano en la franja
  // se compara cada uno con los siguientes siete puntos
  for k := 1 to t - 1 do
    for s := k + 1 to mín {t, k + 7} do
      δ := mín {δ, dist (v_k, v_s)}
    return (δ)
  end rec_cl_pair
```

Mostraremos que el tiempo del algoritmo del par más cercano en el peor de los casos es $\Theta(n \lg n)$. El procedimiento *closest_pair* comienza ordenando los puntos según su abscisa. Si utilizamos un ordenamiento óptimo (por ejemplo, ordenamiento por fusión), el tiempo de ordenamiento en el peor de los casos será $\Theta(n \lg n)$. A continuación, *closest_pair* llama a *rec_cl_pair*. Sea α_n el tiempo en el peor de los casos de *rec_cl_pair* para una entrada de tamaño n. Si $n > 3$, *rec_cl_pair* se llama a sí mismo, con una entrada de tamaño $\lfloor n/2 \rfloor$ y $\lfloor (n+1)/2 \rfloor$. Cada una de las fusiones, localización de los puntos en la franja, y la verificación de las distancias en la franja tarda un tiempo $O(n)$. Así, obtenemos la recurrencia

$$a_n \leq a_{\lfloor n/2 \rfloor} + a_{\lfloor (n+1)/2 \rfloor} + cn, \qquad n > 3.$$

Ésta es la misma recurrencia satisfecha por el ordenamiento por fusión, de modo que podemos concluir que *rec_cl_pair* tiene el mismo tiempo $O(n \lg n)$ que el ordenamiento por fusión, en el peor de los casos. Como el tiempo en el peor de los casos para el ordenamiento de los puntos según su abscisa es $\Theta(n \lg n)$ y el tiempo en el peor de los casos de *rec_cl_pair* es $O(n \lg n)$, el tiempo en el peor de los casos de *closest_pair* es $\Theta(n \lg n)$. En la sección 11.2 mostraremos que cualquier algoritmo que determine un par de puntos más cercanos en el plano tiene un tiempo $\Omega(n \lg n)$ en el peor de los casos; así, nuestro algoritmo es asintóticamente óptimo.

Se puede mostrar (ejercicio 10) que existen a lo más seis puntos en el rectángulo de la figura 11.1.2 al incluir la base y excluir los otros lados. Este resultado es el mejor posible, pues podemos colocar seis puntos en el rectángulo (ejercicio 8). Al considerar las posibles posiciones de los puntos en el rectángulo, D. Lerner y R. Johnsonbaugh han demostrado que basta comparar cada punto en la franja con los siguientes tres puntos (en vez de los siguientes siete). Este resultado es el mejor posible, pues la verificación de los dos puntos siguientes no conduce a un algoritmo correcto (ejercicio 7).

Ejercicios

1. Describa la forma en que el algoritmo del par más cercano determina el par más cercano de puntos si la entrada es (8, 4), (3 11), (12, 10), (5, 4), (1, 2), (17, 10), (8, 7), (8, 9), (11, 3), (1, 5), (11, 7), (5. 9), (1. 9), (7, 6), (3, 7), (14, 7).
2. ¿Qué podría concluir acerca de la entrada del algoritmo del par más cercano si la salida es cero para la distancia entre un par más cercano?
3. Dé un ejemplo de entrada para la cual el algoritmo del par más cercano coloca algunos puntos sobre la línea divisoria l en la mitad izquierda y otros puntos sobre l en la mitad derecha.
4. Explique por qué en ciertos casos, al separar un conjunto de puntos mediante una recta vertical en dos partes casi iguales, es necesario que la línea contenga algunos de los puntos.
5. Escriba un algoritmo del par más cercano que determine un par más cercano, así como la distancia entre el par de puntos.
6. Escriba un algoritmo que determina la distancia entre un par de puntos más cercanos sobre una línea (recta).
7. Dé un ejemplo de entrada para la cual la comparación de cada punto en la franja con los siguientes dos puntos produzca una salida incorrecta.
8. Dé un ejemplo para mostrar que es posible colocar seis puntos en el rectángulo de la figura 11.1.2 al incluir la base y excluir los otros lados.
9. Al calcular las distancias entre un punto p de la franja y los puntos siguientes a él, ¿podemos dejar de calcular las distancias a p si encontramos un punto q tal que la distancia entre p y q sea mayor que δ? Explique.
10. ★ Muestre que existen a lo más seis puntos en el rectángulo de la figura 11.1.2 al incluir la base y excluir los otros lados.
11. Escriba un algoritmo de tiempo $\Theta(n \lg n)$ para determinar la distancia δ entre un par más cercano, de modo que si $\delta > 0$ también determine *todos* los pares que están a distancia δ.
12. Escriba un algoritmo de tiempo $\Theta(n \lg n)$ para determinar la distancia δ entre un par más cercano, y *todos* los pares que están a distancia menor que 2δ.

†11.2 *UNA COTA INFERIOR PARA EL PROBLEMA DEL PAR MÁS CERCANO*

En la sección 11.1 dimos un algoritmo $\Theta(n \lg n)$ para determinar la distancia entre un par más cercano entre n elementos del plano. En esta sección mostraremos que este resultado es óptimo; es decir, que cualquier algoritmo que determine la distancia entre un par más cercano entre n elementos del plano tiene un tiempo $\Omega(n \lg n)$ en el peor de los casos.

Primero mostraremos que un problema relacionado con el anterior, el de determinar si n elementos son distintos, tiene una cota inferior $\Omega(n \lg n)$ para el tiempo en el peor de los casos. Como un algoritmo del par más cercano se puede utilizar para determinar si n elementos son distintos (n elementos son distintos si y sólo si la distancia entre el par más cercano es distinta de cero), su tiempo en el peor de los casos debe ser al menos tan grande como $\Omega(n \lg n)$, la cota inferior para el problema de los elementos distintos.

† Esta sección puede omitirse sin pérdida de continuidad.

TEOREMA 11.2.1

El tiempo en el peor de los casos para un algoritmo que resuelva el problema de determinar si n números reales son distintos es $\Omega(n \lg n)$.

Demostración. Demostraremos que cualquier algoritmo que resuelva el problema de determinar si n números reales son distintos debe ordenar los números. Como el ordenamiento tiene un tiempo $\Omega(n \lg n)$ en el peor de los casos (teorema 7.7.3), cualquier algoritmo que resuelva el problema de determinar si n números reales son distintos también debe tener un tiempo $\Omega(n \lg n)$ en el peor de los casos.

Supongamos que un algoritmo que resuelva el problema de determinar si n números reales son distintos recibe la entrada

$$x_1, \quad \ldots, \quad x_n$$

donde los x_i son distintos. La salida será "Distintos". Supongamos que los elementos ordenados son

$$x_{k_1}, \quad x_{k_2}, \quad \ldots, \quad x_{k_n}. \tag{11.2.1}$$

Afirmamos que el algoritmo debe comparar x_{k_i} y $x_{k_{i+1}}$ para $i = 1, \ldots, n-1$, de modo que el algoritmo debe "conocer" el orden de la entrada. Estableceremos esta afirmación argumentando por contradicción.

Supongamos que el algoritmo no compara x_{k_j} y $x_{k_{j+1}}$ para alguna j. Alteramos la entrada original $x_1, \ldots, x_n$ cambiando el valor de x_{k_j} para $x_{k_{j+1}}$, pero dejando invariantes las demás x_i. Volvemos a ejecutar el algoritmo. El resultado de cada comparación será igual al de la ejecución original, pues la única comparación cuyo resultado cambiaría implica a x_{k_j} y $x_{k_{j+1}}$ y el algoritmo no compara este par. Así, la salida es nuevamente "Distintos". Ésta es una contradicción, pues ahora la entrada tiene números duplicados. Así, cualquier algoritmo que resuelva el problema de determinar si n números reales son distintos debe comparar x_{k_i} y $x_{k_{i+1}}$ para $i = 1, \ldots, n-1$.

Para completar la demostración, ahora mostraremos la forma de convertir un algoritmo que resuelva el problema de determinar si n números reales son distintos en un algoritmo de ordenamiento. Como el ordenamiento tiene una cota inferior $\Omega(n \lg n)$ en el peor de los casos, esto completará la demostración del teorema.

Sea A un algoritmo que resuelva el problema de determinar si n números reales son distintos. Modificamos A de la siguiente manera. Primero construimos los vértices $1, \ldots, n$. Cada vez que el algoritmo A compare x_j con x_k, colocamos una arista dirigida de j a k, si $x_j < x_k$. Para entradas distintas, si el orden es (11.2.1), hemos mostrado que A debe comparar x_{k_i} y $x_{k_{i+1}}$ para $i = 1, \ldots, n-1$. Así, existe un camino de x_{k_1} y x_{k_n} que proporciona el orden deseado. Podemos determinar este camino de la siguiente forma. Primero localizamos el único vértice sin aristas de entrada. Éste es el vértice k_1 correspondiente a x_{k_1}, el menor elemento de la lista. Eliminamos todas las aristas de salida de k_1. Repetimos este proceso; es decir, localizamos el único vértice sin aristas de entrada. Éste es el vértice k_2 que corresponde a x_{k_2}, el segundo menor elemento de la lista. Continuamos de esta forma para obtener el orden deseado. Como el ordenamiento requiere al menos $C\,n \lg n$ comparaciones (teorema 7.7.3), concluimos que nuestro algoritmo modificado realiza al menos $C\,n \lg n$ comparaciones. Como las modificaciones al algoritmo A no implican la comparación de elementos, este algoritmo modificado tiene exactamente el mismo número de comparacio-

nes que A. Así, el algoritmo A necesita al menos $C\,n \lg n$ comparaciones. Esto concluye la demostración. ■

COROLARIO 11.2.2

El tiempo en el peor de los casos para cualquier algoritmo que determine la distancia entre un par más cercano entre n elementos en el plano es $\Omega(n \lg n)$.

Demostración. Sea t_n el tiempo en el peor de los casos para un algoritmo CP (par más cercano, por sus siglas en inglés) que regrese la distancia entre un par más cercano de n puntos en el plano. Consideremos el siguiente algoritmo que resuelve el problema de determinar si existen duplicados entre n números:

```
procedure dup (x, n)
 // La entrada es x_1, ..., x_n.
 // Se transforma la entrada en puntos del plano.
 for i := 1 to n do
  a_i := (x_i, 0)
 if CP (a, n) = 0 then
  return ("Con duplicados")
 else
  return ("Sin duplicados")
end dup
```

El tiempo t'_n en el peor de los casos para *dup* es el tiempo necesario en el ciclo for, más el tiempo en el peor de los casos para CP; es decir,

$$t'_n = n + t_n.$$

Por el teorema 11.2.1,

$$Cn \lg n \leq t'_n.$$

Al combinar estos dos enunciados, obtenemos

$$\Omega(n \lg n) = C\,n \lg n - \leq t'_n - n = t_n.$$ ■

Ejercicios

1. Escriba un algoritmo que resuelva el problema de determinar si n números reales son distintos. Haga su algoritmo lo más eficiente posible.
2. El *problema del vecino más cercano* es: Dados n puntos S del plano, uno de los cuales se designa como p, y las distancias ordenadas de p a q para todo $q \neq p$, determinar un punto s en S, $s \neq p$, más cercano a p. Muestre que el tiempo en el peor de los casos para un algoritmo que resuelva este problema es $\Omega(\lg n)$.
3. El problema de todos los vecinos más cercanos es: Dados n puntos S del plano, y las distancias de p a q para todo $q \neq p$, para cada punto p en S, determinar un punto q en S, $q \neq p$, más cercano a p. Muestre que el tiempo en el peor de los casos para un algoritmo que resuelva este problema es $\Omega(n \lg n)$.
4. Suponga dada una gráfica dirigida con vértices $1, \ldots, n$, la cual contiene las aristas (p_i, p_{i+1}), $i = 1, \ldots, n-1$, para cierta permutación $p_1, \ldots, p_n$ de $1, \ldots, n$. Suponga también que si (p_i, p_j) es una arista, entonces $i < j$. (Esta situación es análoga a la de la demostración del teorema 11.2.1.) Dé un algoritmo cuya salida sea $p_1, \ldots, p_n$. [Existe un algoritmo cuyo tiempo en el peor de los casos es $\Theta(e + n)$, donde e es el número de aristas en la gráfica.]

11.3 UN ALGORITMO PARA CALCULAR LA CUBIERTA CONVEXA

Un problema fundamental en la geometría computacional es el de calcular los puntos que "acotan" a un conjunto finito de **puntos en el plano**, formalmente llamados cubierta convexa. (Véase la figura 11.3.1, donde se indican los puntos que forman la cubierta convexa.) La cubierta convexa tiene aplicaciones en muchas áreas, incluyendo la estadística, la graficación por computadora y el procesamiento de imágenes. Por ejemplo, en estadística, los puntos de un conjunto de datos que determinen la cubierta convexa podrían ser extraños, puntos poco representativos de los datos, por lo que podrían descartarse. En esta sección presentamos el algoritmo de Graham para calcular la cubierta convexa. En esta sección, un "conjunto de puntos" será un "conjunto de puntos *distintos*". Comenzamos con las definiciones.

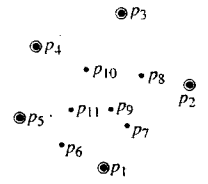


FIGURA 11.3.1
La cubierta convexa de los puntos $p_1, \ldots, p_{11}$ es p_1, p_2, p_3, p_4, p_5.

DEFINICIÓN 11.3.1

Dado un conjunto finito de puntos S del plano, un punto p en S es un *punto de la cubierta* si existe una línea (recta) L que pasa por p de modo que todos los puntos de S, excepto p, están en un lado de L (y excepto p, ninguno está en L).

EJEMPLO 11.3.2

En la figura 11.3.2, p_1 es un punto de la cubierta, pues podemos determinar una línea L_1 que pasa por p_1 tal que los demás puntos están estrictamente en un lado de L_1. El punto p_8 no es un punto de la cubierta, pues cada línea L por p_8 tiene puntos en ambos lados de L. El punto p_6 tampoco es un punto de la cubierta. Como muestra la figura 11.3.2, es posible trazar una línea L_2 por p_6 tal que un lado de L_2 no tenga puntos del conjunto, pero L_2 no cumple las condiciones de la definición 11.3.1, pues contiene puntos distintos de p_6. □

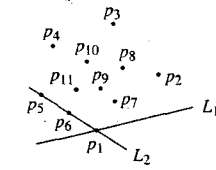


FIGURA 11.3.2
Los puntos de la cubierta son p_1, p_2, p_3, p_4, p_5. Los demás puntos no son puntos la cubierta.

La cubierta convexa de un conjunto finito de puntos S del plano consta de los puntos de la cubierta, enumerados en orden al recorrer la frontera de S. En la figura 11.3.2, la cubierta convexa es la sucesión de puntos p_1, p_2, p_3, p_4, p_5. La siguiente definición precisa el concepto de orden de los puntos de la cubierta.

DEFINICIÓN 11.3.3

La *cubierta convexa* de un conjunto finito de puntos S del plano es la sucesión $p_1, p_2, \ldots, p_n$ de puntos de la cubierta de S, enumerados en el siguiente orden. El punto p_1 es el punto con coordenada y mínima. Si existen varios puntos con la misma coordenada y mínima, p_1 es el que tiene abscisa mínima. (Observe que p_1 es un punto de la cubierta.) Para $i \geq 2$, sea α_i el ángulo que forma la horizontal con el segmento de recta p_1, p_i (véase la figura 11.3.3). Los puntos $p_2, p_3, \ldots, p_n$ se ordenan de modo que $\alpha_2, \alpha_3, \ldots, \alpha_n$ sea una sucesión creciente.

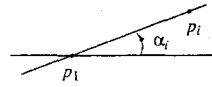


FIGURA 11.3.3
α_i es el ángulo que forma la horizontal con el segmento de recta p_1, p_i.

EJEMPLO 11.3.4

En la figura 11.3.4, el punto p_1 tiene la coordenada y mínima, de modo que es el primer punto enumerado en la cubierta convexa. Como se muestra, los ángulos que forma la horizontal con los segmentos de recta p_1, p_2; p_1, p_3; p_1, p_4; p_1, p_5 van creciendo. Así, la cubierta convexa del conjunto de puntos de la figura 11.3.4 es p_1, p_2, p_3, p_4, p_5. □

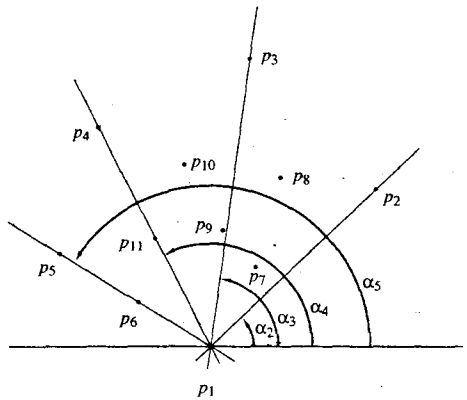


FIGURA 11.3.4 La cubierta convexa es p_1, p_2, p_3, p_4, p_5 debido a que estos son puntos de la cubierta convexa y que los ángulos correspondientes $\alpha_2, \alpha_3, \alpha_4, \alpha_5$ van creciendo, donde α_i es el ángulo que forma la horizontal con el segmento de recta p_1, p_i.

La definición 11.3.3 sugiere un algoritmo para calcular la cubierta convexa de un conjunto finito de puntos S del plano. Primero se determina el punto p_1 con coordenada y mínima. Si existen varios puntos con la misma coordenada y mínima, se elige el punto con abscisa mínima. A continuación se ordenan *todos* los puntos p en S de acuerdo con el ángulo que forma la horizontal con el segmento de recta p_1, p. Por último, se examinan los puntos, en orden, y se descartan aquellos que no están en la cubierta convexa. El resultado será la cubierta convexa. Ésta es la estrategia utilizada por el algoritmo de Graham. Para convertir esta idea en un algoritmo, hay que resolver dos aspectos fundamentales. Debemos describir una forma para comparar ángulos, y debemos desarrollar un método para verificar si los puntos están en la cubierta convexa. Primero veremos el problema de la comparación de ángulos.

Supongamos que visitamos los puntos distintos p_1, p_0, p_2 en el plano en este orden. Si después de salir de p_0 nos movemos hacia la izquierda, decimos que los puntos p_1, p_0, p_2 *forman un giro hacia la izquierda* (véase la figura 11.3.5). Más precisamente, los puntos p_1, p_0, p_2 forman un giro hacia la izquierda si el ángulo del segmento de recta p_0, p_2 al segmento de recta p_0, p_1, medido en el sentido contrario al de las manecillas del reloj, es menor de 180°. De manera análoga, si al salir de p_0 nos movemos hacia la derecha, decimos que los puntos p_1, p_0, p_2 *forman un giro hacia la derecha* (véase la figura 11.3.5); es decir, los puntos p_1, p_0, p_2 forman un giro hacia la derecha si el ángulo del segmento de recta p_0, p_2 al segmento de recta p_0, p_1, medido en el sentido contrario al de las manecillas del reloj, es mayor de 180°.

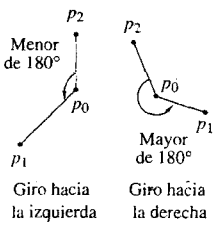


FIGURA 11.3.5
El primer giro es hacia la izquierda, pues el ángulo de p_0, p_2 a p_0, p_1 es menor de 180°. El segundo giro es hacia la derecha, pues el ángulo de p_0, p_2 a p_0, p_1 es mayor de 180°.

Podemos utilizar los métodos de la geometría analítica para deducir un criterio que permita decidir si los puntos p_1, p_0, p_2 forman un giro hacia la izquierda o hacia la derecha. Sean (x_i, y_i) las coordenadas del punto p_i, $i = 0, 1, 2$ (véase la figura 11.3.6). Supongamos primero que $x_1 < x_0$. La ecuación de la recta L que pasa por p_0 y p_1 es

$$y = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0).$$

Ahora, p_1, p_0, p_2 forman un giro hacia la izquierda precisamente cuando p_2 está arriba de L, o cuando

$$y_2 > y' = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x_2 - x_0).$$

Podemos reescribir esta última desigualdad como

$$y_2 - y_0 > \frac{y_1 - y_0}{x_1 - x_0}(x_2 - x_0).$$

Al multiplicar por $x_1 - x_0$, que es negativo, y pasar todos los términos a un lado de la desigualdad se obtiene

$$(y_2 - y_0)(x_1 - x_0) - (y_1 - y_0)(x_2 - x_0) < 0.$$

Si definimos el **producto cruz** de los puntos p_0, p_1, p_2 como

$$\text{cruz}(p_0, p_1, p_2) = (y_2 - y_0)(x_1 - x_0) - (y_1 - y_0)(x_2 - x_0),$$

hemos demostrado lo siguiente:

Si los puntos p_1, p_0, p_2 forman un giro hacia la izquierda, entonces $\text{cruz}(p_0, p_1, p_2) < 0$.

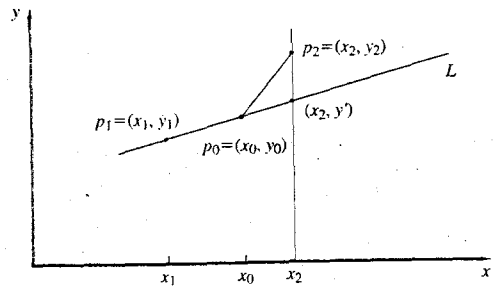


FIGURA 11.3.6 Criterio para decidir si los puntos p_1, p_0, p_2 forman un giro hacia la izquierda o hacia la derecha. La figura supone que $x_1 < x_0$. L es la recta que pasa por p_0 y p_1. Como se muestra, en este caso ocurre un giro hacia la izquierda precisamente cuando p_2 está arriba de L.

De manera análoga, podemos mostrar que

Si los puntos p_1, p_0, p_2 forman un giro hacia la derecha, entonces cruz$(p_0, p_1, p_2) > 0$,

y

Si los puntos p_1, p_0, p_2 son colineales, entonces cruz$(p_0, p_1, p_2) = 0$.

También podemos demostrar los recíprocos de estas afirmaciones. Por ejemplo, para demostrar que si cruz $(p_0, p_1, p_2) < 0$, entonces los puntos p_1, p_0, p_2 forman un giro hacia la izquierda, podemos argumentar como sigue. Supongamos que cruz$(p_0, p_1, p_2) < 0$. Como los puntos no forman un giro hacia la derecha [ya que en ese caso, cruz(p_0, p_1, p_2) sería positivo] y no son colineales [ya que en ese caso, cruz$(p_0, p_1, p_2) = 0$], deben formar un giro hacia la izquierda. Así,

Los puntos p_1, p_0, p_2 forman un giro hacia la izquierda si y sólo si cruz$(p_0, p_1, p_2) < 0$.

Los puntos p_1, p_0, p_2 forman un giro hacia la derecha si y sólo si cruz$(p_0, p_1, p_2) > 0$.

Los puntos p_1, p_0, p_2 son colineales si y sólo si cruz$(p_0, p_1, p_2) = 0$.

(11.3.1)

Hemos demostrado (11.3.1) suponiendo que $x_1 < x_0$. Si $x_1 = x_0$ o $x_1 > x_0$, la conclusión (11.3.1) sigue siendo válida (véanse los ejercicios 2 y 3). Resumimos estas conclusiones como un teorema.

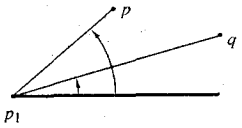


FIGURA 11.3.7
El caso $p > q$ ocurre cuando p, p_1, q forman un giro hacia la izquierda.

TEOREMA 11.3.5

Si p_0, p_1, p_2 son puntos distintos en el plano,

(a) p_1, p_0, p_2 forman un giro hacia la izquierda si y sólo si cruz$(p_0, p_1, p_2) < 0$.

(b) p_1, p_0, p_2 forman un giro hacia la derecha si y sólo si cruz$(p_0, p_1, p_2) > 0$.

(c) p_1, p_0, p_2 son colineales si y sólo si cruz$(p_0, p_1, p_2) = 0$.

Demostración. La demostración aparece antes del enunciado del teorema. ■

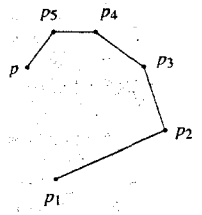


FIGURA 11.3.8
Una situación en el algoritmo de la cubierta convexa al examinar el punto p. Antes de examinar a p, la cubierta convexa de los puntos analizados hasta ese momento es p_1, p_2, p_3, p_4, p_5. Como p_4, p_5, p forman un giro hacia la izquierda, p_5 sigue estando en la cubierta convexa de los puntos analizados hasta ese momento, por lo que se le retiene. La cubierta convexa actual es $p_1, p_2, p_3, p_4, p_5, p$. El algoritmo continúa examinando al punto posterior a p.

El algoritmo de Graham comienza determinando el punto p_1 con coordenada y mínima. Si existen varios puntos con la misma coordenada y mínima, el algoritmo elige el punto con abscisa mínima. A continuación, el algoritmo ordena *todos* los puntos p en S de acuerdo con el ángulo que forma la horizontal con el segmento de recta p_1, p. Por último, se examinan los puntos, en orden, y se descartan aquellos que no están en la cubierta convexa.

Podemos utilizar el producto cruz para comparar puntos $p \neq q$ en el ordenamiento. Para comparar los puntos p y q, calculamos cruz(p_1, p, q). Si cruz$(p_1, p, q) < 0$, entonces p, p_1, q forman un giro hacia la izquierda. Con respecto de los ángulos con la horizontal, esto significa que $p > q$ (véase la figura 11.3.7). Si cruz$(p_1, p, q) > 0$, entonces $p < q$. Si cruz$(p_1, p, q) = 0$, entonces p, p_1, q son colineales. En este último caso, definimos $p > q$ si p está más lejos de p_1 que q, y $p < q$ si q está más lejos de p_1 que p.

También podemos utilizar el producto cruz para determinar si un punto no está en la cubierta convexa y que, por tanto, puede ser descartado. Al examinar los puntos en orden, conservamos los puntos que estarían en la cubierta convexa si no hay más puntos por examinar. Luego analizamos el siguiente punto p. Por ejemplo, en la figura 11.3.8, suponga que hemos conservado $p_1, \ldots, p_5$ y que el siguiente punto por examinar es p. Como p_4, p_5, p forman un giro hacia la izquierda, conservamos a p_5. Luego continuamos analizando el punto posterior a p.

Como otro ejemplo, suponga que hemos conservado a $p_1, \ldots, p_5$ (véase la figura 11.3.9). Esta vez, como p_4, p_5, p forman un giro hacia la derecha, descartamos p_5. Ahora regresamos para examinar p_3, p_4, p. Como estos puntos también forman un giro hacia la derecha, descartamos p_4. Ahora regresamos para examinar p_2, p_3, p. Como estos puntos forman un giro hacia la izquierda, conservamos a p_3. Continuamos examinando el punto posterior a p. El seudocódigo para el algoritmo de Graham aparece como el algoritmo 11.3.6.

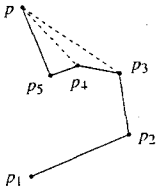


FIGURA 11.3.9
Una situación en el algoritmo de la cubierta convexa al examinar el punto p. Antes de examinar a p, la cubierta convexa de los puntos analizados hasta ese momento es p_1, p_2, p_3, p_4, p_5. Como p_4, p_5, p forman un giro hacia la derecha, descartamos p_5. Esto deja los puntos p_3, p_4, p, que también forman un giro hacia la derecha; así, también descartamos p_4. Esto deja los puntos p_2, p_3, p, que forman un giro hacia la izquierda, por lo que conservamos a p_3. La cubierta convexa actual es p_1, p_2, p_3, p. El algoritmo continúa examinando al punto posterior a p.

ALGORITMO 11.3.6 *Algoritmo de Graham para calcular la cubierta convexa*

Este algoritmo calcula la cubierta convexa de los puntos $p_1, \ldots, p_n$ del plano. Las coordenadas x y y del punto p se denotan $p.x$ y $p.y$, respectivamente.

Entrada: $p_1, \ldots, p_n$ y n

Salida: $p_1, \ldots, p_k$ (la cubierta convexa de $p_1, \ldots, p_n$) y k

```
procedure graham_scan (p, n, k)
 // caso trivial
 if n = 1 then
  begin
  k := 1
  return
  end
 // determinar el punto con coordenada y mínima
 min := 1
 for i := 2 to n do
  if p_i.y < p_min.y then
   min := 1
 // Entre todos estos puntos, determinar aquél con abscisa mínima
 // coordenada x
 for i := 1 to n do
  if p_i.y = p_min.y and p_i.x < p_min.x then
   min := i
 swap(p_1, p_min)
 // ordenar según el ángulo de la horizontal a p_1, p_i
 sort p_2, ..., p_n
 // p_0 es un punto adicional, agregado con el fin de evitar que
 // el algoritmo se repita indefinidamente
 p_0 := p_n
 // descartar los puntos que no están en la cubierta convexa
 k := 2
 for i := 3 to n do
  begin
   while p_{k-1}, p_k, p_i no giran a la izquierda do
    // descartar p_k
    k := k - 1
   k := k + 1
   swap(p_i, p_k)
   end
end graham_scan
```

EJEMPLO 11.3.7

La figura 11.3.10 muestra al algoritmo de Graham en acción.

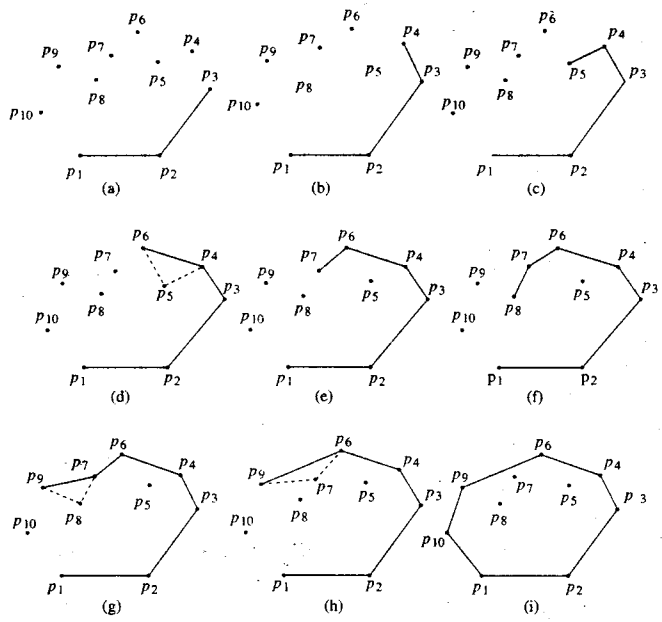


FIGURA 11.3.10 El algoritmo de Graham para calcular la cubierta convexa. El punto p_1 tiene la coordenada y mínima, y entre todos los puntos de este tipo, tiene la abscisa mínima. Los puntos ordenados según el ángulo de la horizontal con p_1, p_i son $p_2, p_3, \ldots, p_{10}$. En (a), el algoritmo comienza examinando p_1, p_2, p_3. p_1, p_2, p_3 forman un giro hacia la izquierda, de modo que se conserva a p_2. Luego en (b), se examinan p_2, p_3, p_4. Estas p_2, p_3, p_4 forman un giro hacia la izquierda, de modo que se conserva a p_3. Luego en (c), se examinan p_3, p_4, p_5. p_3, p_4, p_5 forman un giro hacia la izquierda, de modo que se conserva a p_4. Luego en (d), se examinan p_4, p_5, p_6. p_4, p_5, p_6 forman un giro hacia la derecha, de modo que se descarta a p_5. El algoritmo regresa a p_3, p_4, p_6. p_3, p_4, p_6 forman un giro hacia la izquierda, de modo que se conserva a p_4. Luego en (e), se examinan p_4, p_6, p_7. p_4, p_6, p_7 forman un giro hacia la izquierda, de modo que se conserva a p_6. Luego en (f), se examinan p_6, p_7, p_8. p_6, p_7, p_8 forman un giro hacia la izquierda, de modo que se conserva a p_7. Luego en (g), se examinan p_7, p_8, p_9. p_7, p_8, p_9 forman un giro hacia la derecha, de modo que se descarta a p_8. En (h), el algoritmo regresa a p_6, p_7, p_9. p_6, p_7, p_9 también forman un giro hacia la derecha, de modo que se descarta a p_7. El algoritmo regresa a p_4, p_6, p_9. p_4, p_6, p_9 forman un giro hacia la izquierda, de modo que se conserva a p_6. Finalmente, en (i), el algoritmo concluye examinando a p_6, p_9, p_{10}. p_6, p_9, p_{10} forman un giro hacia la izquierda, de modo que se conserva a p_9. La cubierta convexa es $p_1, p_2, p_3, p_4, p_6, p_9, p_{10}$. □

El algoritmo de Graham comienza con dos ciclos for, cada uno de los cuales tarda un tiempo $\Theta(n)$. Si utilizamos un ordenamiento óptimo, como aquél por fusión, el tiempo de ordenamiento en el peor de los casos será $\Theta(n \lg n)$.

El tiempo *total* de ejecución del último ciclo while es $O(n)$, pues un punto se puede descartar a lo más una vez y existen n puntos. El propio ciclo for se ejecuta $\Theta(n)$; así, el tiempo total para el último ciclo for y el ciclo while es $\Theta(n)$. Vemos que el tiempo de ordenamiento domina, de modo que el tiempo en el peor de los casos del algoritmo de Graham es $\Theta(n \lg n)$. El tiempo necesario después de ordenar los puntos es $\Theta(n)$, de modo que si los puntos vienen ordenados de antemano, el algoritmo de Graham puede calcular la cubierta convexa en un tiempo lineal.

Nuestro último teorema muestra que cualquier algoritmo que calcule la cubierta convexa de n puntos del plano tiene un tiempo $\Omega(n \lg n)$ en el peor de los casos; de modo que el algoritmo de Graham es óptimo.

TEOREMA 11.3.8

Cualquier algoritmo que calcule la cubierta convexa de n puntos del plano tiene un tiempo $\Omega(n \lg n)$ en el peor de los casos.

Demostración. Sea A un algoritmo que calcule la cubierta convexa de un conjunto finito de puntos del plano. Mostraremos que, en el peor de los casos, este algoritmo ocupa tanto tiempo como un algoritmo de ordenamiento y por tanto tiene la misma cota inferior $\Omega(n \lg n)$ del problema de ordenamiento.

Consideremos una sucesión arbitraria de números reales

$$y_1, \quad y_2, \quad \ldots, \quad y_n,$$

donde cada y_i está entre 0 y 1. Utilizamos el algoritmo de la cubierta convexa, el algoritmo A, para construir un algoritmo, el algoritmo B, que ordene a esta sucesión. En primer lugar, el algoritmo B proyecta estos números reales sobre el círculo unitario (véase la figura 11.3.11). A continuación, el algoritmo B llama al algoritmo A para determinar la cubierta convexa $h_1, h_2, \ldots, h_n$ de los puntos del círculo. Por último, el algoritmo B produce como salida las coordenadas y de $h_1, h_2, \ldots, h_n$ en este orden. Observe que la salida del algoritmo B es la sucesión de entrada ordenada:

$$y_1, \quad y_2, \quad \ldots, \quad y_n.$$

Como el algoritmo B es un algoritmo de ordenamiento, el teorema 7.7.3 implica que su tiempo en el peor de los casos, t_n, satisface

$$t_n \geq Cn \lg n.$$

Por otro lado, el algoritmo B consta de dos ciclos $\Theta(n)$ (uno para proyectar los puntos en el círculo unitario, y el otro para producir como salida las coordenadas y de la cubierta convexa) y la llamada al algoritmo A, que tarda un tiempo s_n, digamos, en el peor de los casos. Así,

$$t_n = 2n + s_n.$$

Por tanto,

$$s_n = t_n - 2n \geq Cn \lg n - 2n = \Omega(n \lg n)$$

y esto concluye la demostración. ■

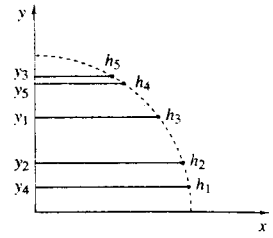


FIGURA 11.3.11
Cubos numerados sobre una mesa. Ordenamiento mediante el algoritmo de la cubierta convexa. Los puntos y_1, y_2, y_3, y_4, y_5 se proyectan primero sobre el círculo unitario. Los puntos resultantes sobre el círculo unitario se denotan h_i. Luego, se utiliza el algoritmo de la cubierta convexa para determinar la cubierta convexa h_1, h_2, h_3, h_4, h_5. Las coordenadas y de la cubierta convexa (en el orden h_1, h_2, h_3, h_4, h_5) son y_4, y_2, y_1, y_5, y_3, el orden requerido de las y.

Ejercicios

1. Sea S un conjunto finito de puntos del plano. Sea p_1 el punto con coordenada y mínima. Si varios puntos tienen la misma coordenada y mínima, elija aquél con abscisa mínima. Demuestre que p_1 está en la cubierta convexa de S.
2. Demuestre el teorema 11.3.5 cuando $x_1 = x_0$.
3. Demuestre el teorema 11.3.5 cuando $x_1 > x_0$.
4. Utilice el algoritmo de Graham para determinar la cubierta convexa de los puntos (10, 1), (7, 7), (3, 13), (6, 10), (16, 4), (10, 5), (7, 13), (13, 8), (4, 4), (2, 2), (1, 8), (10 ,13), (7, 1), (4, 8), (12, 3), (16, 10), (14, 5), (10, 9).
5. Utilice el algoritmo de Graham para determinar la cubierta convexa de los puntos (7, 8), (9, 8), (3, 11), (5, 1), (7, 11), (9, 5), (9, 1), (6, 7), (4, 5), (2, 1), (10 ,17), (7, 3), (7, 14), (4, 8), (11, 3), (10 ,12).
6. Suponga que se ha utilizado el algoritmo de Graham para determinar la cubierta convexa de un conjunto de n puntos S. Muestre que si se agrega un punto S para obtener S', la cubierta convexa de S' se puede determinar en un tiempo $\Theta(n)$.

Los ejercicios 7-10 se refieren a la *marcha de Jarvis*, otro algoritmo que calcula la cubierta convexa de un conjunto finito de puntos del plano. Comienza, como el algoritmo de Graham, determinando el punto con coordenada y mínima. Si varios puntos tienen la misma coordenada y mínima, se elige aquél con abscisa mínima. A continuación, la marcha de Jarvis determina el punto p_2 tal que el ángulo de la horizontal con el segmento p_1, p_2 sea mínimo. (En el caso de empates, se elige el punto más lejano de p_1.) Después de determinar $p_1, \ldots, p_i$, la marcha de Jarvis determina el punto p_{i+1} tal que p_{i-1}, p_i, p_{i+1} forman el menor giro hacia la izquierda. (En el caso de empates, se elige el punto más lejano de p_i.)

7. Muestre que la marcha de Jarvis realmente determina la cubierta convexa.
8. Escriba un seudocódigo para la marcha de Jarvis.
9. Determine el tiempo en el peor de los casos para la marcha de Jarvis.
10. ¿Existen conjuntos para los cuales la marcha de Jarvis sea más rápida que el algoritmo de Graham? Explique.

NOTAS

[Preparata, 1985 y Edelsbrunner] son libros de geometría computacional.

El algoritmo del par más cercano de la sección 11.1 fue ideado por M. I. Shamos y aparece en [Preparata, 1985]. [Preparata, 1985] también proporciona un algoritmo $\Theta(n \lg n)$ para determinar el par más cercano en un número arbitrario de dimensiones.

El algoritmo de Graham (véase [Graham, 1972]) que apareció en 1972 fue uno de los primeros algoritmos $\Theta(n \lg n)$ para la cubierta convexa plana. La marcha de Jarvis aparece en [Jarvis]. El cálculo de la cubierta convexa de un conjunto de puntos en un espacio de más de dos dimensiones es mucho más complejo que el cálculo de la cubierta convexa en el plano. El primer algoritmo tridimensional óptimo fue dado en 1977 por [Preparata, 1977]. En 1981, [Seidel] dio un algoritmo n-dimensional para la cubierta convexa, que es óptimo para n par. La determinación de algoritmos más eficientes para determinar la cubierta convexa en espacios de varias dimensiones es una importante área de investigación en la actualidad.

CONCEPTOS BÁSICOS DEL CAPÍTULO

Sección 11.1

Geometría computacional
Problema del par más cercano
Algoritmo del par más cercano

Sección 11.2

El tiempo en el peor de los casos de cualquier algoritmo que resuelve el problema de determinar si n números reales son distintos es $\Omega(n \lg n)$ (teorema 11.2.1).
El tiempo en el peor de los casos de cualquier algoritmo que determine la distancia entre un par más cercano entre n elementos del plano es $\Omega(n \lg n)$ (corolario 11.2.2).

Sección 11.3

Punto de la cubierta
Cubierta convexa
Producto cruz
Algoritmo de Graham para calcular la cubierta convexa
Cualquier algoritmo que calcule la cubierta convexa de n puntos del plano tiene un tiempo en el peor de los casos de $\Omega(n \lg n)$ (teorema 11.3.8).

AUTOEVALUACIÓN DEL CAPÍTULO

Sección 11.1

1. Describa la forma en que el algoritmo del par más cercano determina el par más cercano de puntos, si la entrada es como la del ejercicio 4, sección 11.3.
2. Para concluir la recursión en el algoritmo del par más cercano, si existen tres o menos puntos en la entrada podemos determinar el par más cercano en forma directa. ¿Por qué no podemos reemplazar "tres" por "dos"?
3. Muestre que existen a lo más cuatro puntos en la mitad inferior del rectángulo de la figura 11.3.3.
4. ¿Cuál sería el tiempo en el peor de los casos del algoritmo del par más cercano (algoritmo 11.1.2) si en vez de realizar la fusión de $p_i, \ldots, p_k$ y $p_{k+1}, \ldots, p_j$ utilizamos el ordenamiento por fusión para ordenar $p_i, \ldots, p_j$?

Sección 11.2

5. Muestre que el tiempo en el peor de los casos de cualquier algoritmo que determine un par más cercano entre n elementos en el espacio de dimensión d es $\Omega(n \lg n)$.
6. Muestre que el tiempo en el peor de los casos de cualquier algoritmo que determine todos los pares más cercanos entre n elementos del plano es $\Omega(n \lg n)$.
7. Enuncie y demuestre una mejor cota inferior para el tiempo en el peor de los casos de cualquier algoritmo que resuelva el problema al determinar si n números reales son todos iguales.
8. Muestre que el tiempo en el peor de los casos de cualquier algoritmo que determine todos los pares a una distancia menor o igual a 2δ, donde δ es la distancia entre un par más cercano, es $\Omega(n \lg n)$.

Sección 11.3

9. Sea S un conjunto finito de puntos en el plano. Sea p el punto en S con abscisa máxima. Si varios puntos tienen la misma abscisa máxima, elija el que tiene coordenada y máxima. Demuestre que p está en la cubierta convexa de S.
10. Sea S un conjunto finito de puntos distintos en el plano. Suponga que S contiene al menos dos puntos. Sean p y q puntos en S que están a una distancia máxima. Demuestre que p y q están en la cubierta convexa de S.
11. Utilice el algoritmo de Graham para determinar la cubierta convexa del conjunto de puntos del ejercicio 1, sección 11.1.
12. Suponga que se ha utilizado el algoritmo de Graham para determinar la cubierta convexa de un conjunto de $n \geq 2$ puntos. Muestre que si un punto distinto de p_1 del algoritmo 11.3.6 se elimina de S para obtener S', la cubierta convexa de S' se puede determinar en un tiempo $\Theta(n)$.

APÉNDICE: MATRICES

La organización de datos por renglones y columnas es una práctica común. En matemáticas, un arreglo de datos de este tipo es una **matriz**. En este apéndice resumimos algunas definiciones y propiedades elementales de las matrices. Primero damos la definición de matriz.

DEFINICION A.1

Una *matriz*

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \tag{A.1}$$

es un arreglo rectangular de datos.

Si A tiene m renglones y n columnas, decimos que el tamaño de A es m por n (lo cual se escribe $m \times n$).

Con frecuencia abreviamos la ecuación (A.1) como $A = (a_{ij})$. En esta ecuación, a_{ij} denota el elemento de A que aparece en el i-ésimo renglón y la j-ésima columna.

EJEMPLO A.2

La matriz

$$A = \begin{pmatrix} 2 & 1 & 0 \\ -1 & 6 & 14 \end{pmatrix}$$

tiene dos renglones y tres columnas, por lo que su tamaño es 2×3. Por ejemplo, si escribimos $A = (a_{ij})$, tendríamos que

$$a_{11} = 2, \quad a_{21} = -1, \quad a_{13} = 0. \qquad \square$$

DEFINICION A.3

Dos matrices A y B son *iguales*, lo que se denota $A = B$, si tienen el mismo tamaño y sus entradas correspondientes son iguales.

EJEMPLO A.4

Determinar w, x, y y z tales que

$$\begin{pmatrix} x+y & y \\ w+z & w-z \end{pmatrix} = \begin{pmatrix} 5 & 2 \\ 4 & 6 \end{pmatrix}.$$

De acuerdo con la definición A.3, como las matrices tienen el mismo tamaño, serán iguales siempre que

$$\begin{aligned} x + y &= 5 & y &= 2 \\ w + z &= 4 & w - z &= 6. \end{aligned}$$

Al resolver estas ecuaciones, obtenemos

$$w = 5, \quad x = 3, \quad y = 2, \quad z = -1. \qquad \square$$

A continuación describimos algunas operaciones que se pueden realizar con las matrices. La **suma** de dos matrices se obtiene sumando las entradas correspondientes. El **producto por un escalar** se obtiene multiplicando cada entrada de la matriz por un número fijo.

DEFINICION A.5

Sean $A = (a_{ij})$ y $B = (b_{ij})$ dos matrices $m \times n$. La suma de A y B se define como

$$A + B = (a_{ij} + b_{ij}).$$

El *producto por un escalar* de una matriz $A = (a_{ij})$ por un número c se define como

$$cA = (ca_{ij}).$$

Si A y B son matrices, definimos $-A = (-1)A$ y $A - B = A + (-B)$.

EJEMPLO A.6

Si

$$A = \begin{pmatrix} 4 & 2 \\ -1 & 0 \\ 6 & -2 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & -3 \\ 4 & 4 \\ -1 & -3 \end{pmatrix},$$

entonces

$$A + B = \begin{pmatrix} 5 & -1 \\ 3 & 4 \\ 5 & -5 \end{pmatrix}, \qquad 2A = \begin{pmatrix} 8 & 4 \\ -2 & 0 \\ 12 & -4 \end{pmatrix}, \qquad -B = \begin{pmatrix} -1 & 3 \\ -4 & -4 \\ 1 & 3 \end{pmatrix}.$$

$\square$

La multiplicación de matrices es otra importante operación matricial.

DEFINICIÓN A.7

Sea $A = (a_{ij})$ una matriz $m \times n$ y sea $B = (b_{jk})$ una matriz $n \times l$. El *producto matricial* de A y B se define como la *matriz* $m \times l$

$$AB = (c_{ik}),$$

donde

$$c_{ik} = \sum_{j=1}^{n} a_{ij} b_{jk}.$$

Para multiplicar la matriz A por la matriz B, la definición A.7 requiere que el número de columnas de A sea igual al número de renglones de B.

EJEMPLO A.8

Sean

$$A = \begin{pmatrix} 1 & 6 \\ 4 & 2 \\ 3 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 2 & -1 \\ 4 & 7 & 0 \end{pmatrix}.$$

El producto matricial AB está definido, pues el número de columnas de A es igual al número de renglones de B; ambos son iguales a 2. La entrada c_{ik} en el producto AB se obtiene mediante el i-ésimo renglón de A y la k-ésima columna de B. Por ejemplo, la entrada c_{31} se calcula mediante el tercer renglón

$$(3 \quad 1)$$

de A y la primera columna

$$\begin{pmatrix} 1 \\ 4 \end{pmatrix}$$

de B. Multiplicamos cada elemento del tercer renglón de A por cada elemento de la primera columna de B y luego sumamos los resultados para obtener

$$3 \cdot 1 + 1 \cdot 4 = 7.$$

Como el número de columnas de A es igual al número de renglones de B, los elementos se corresponden correctamente. De esta manera, obtenemos el producto

$$AB = \begin{pmatrix} 25 & 44 & -1 \\ 12 & 22 & -4 \\ 7 & 13 & -3 \end{pmatrix}.$$

□

EJEMPLO A.9

El producto matricial

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} \qquad \text{es} \qquad \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}.$$

□

DEFINICIÓN A.10

Sea A una matriz $n \times n$. Si m es un entero positivo, la m-ésima *potencia* de A se define como el producto matricial

$$A^m = \underbrace{A \cdots A}_{m\ A\text{'s}}.$$

EJEMPLO A.11

Si

$$A = \begin{pmatrix} 1 & -3 \\ -2 & 4 \end{pmatrix},$$

entonces

$$A^2 = AA = \begin{pmatrix} 1 & -3 \\ -2 & 4 \end{pmatrix}\begin{pmatrix} 1 & -3 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 7 & -15 \\ -10 & 22 \end{pmatrix},$$

$$A^4 = AAAA = A^2A^2 = \begin{pmatrix} 7 & -15 \\ -10 & 22 \end{pmatrix}\begin{pmatrix} 7 & -15 \\ -10 & 22 \end{pmatrix} = \begin{pmatrix} 199 & -435 \\ -290 & 634 \end{pmatrix}.$$

□

Ejercicios

1. Calcule la suma

$$\begin{pmatrix} 2 & 4 & 1 \\ 6 & 9 & 3 \\ 1 & -1 & 6 \end{pmatrix} + \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}.$$

En los ejercicios 2-8, sean

$$A = \begin{pmatrix} 1 & 6 & 9 \\ 0 & 4 & -2 \end{pmatrix}, \qquad B = \begin{pmatrix} 4 & 1 & -2 \\ -7 & 6 & 1 \end{pmatrix}$$

y calcule cada expresión.

2. $A + B$ 3. $B + A$ 4. $-A$ **5.** $3A$

6. $-2B$ 7. $2B + A$ **8.** $B - 6A$

En los ejercicios 9-13, calcule los productos.

9. $\begin{pmatrix} 1 & 2 & 3 \\ -1 & 2 & 3 \\ 0 & 1 & 4 \end{pmatrix}\begin{pmatrix} 2 & 8 \\ -1 & 1 \\ 6 & 0 \end{pmatrix}$ 10. $\begin{pmatrix} 1 & 6 \\ -8 & 2 \\ 4 & 1 \end{pmatrix}\begin{pmatrix} 4 & 1 \\ 7 & -6 \end{pmatrix}$

11. A^2, donde $A = \begin{pmatrix} 1 & -2 \\ 6 & 2 \end{pmatrix}$

12. $\begin{pmatrix} 2 & -4 & 6 & 1 & 3 \end{pmatrix}\begin{pmatrix} 1 \\ 3 \\ -2 \\ 6 \\ 4 \end{pmatrix}$ 13. $\begin{pmatrix} 2 & 4 & 1 \\ 6 & 9 & 3 \\ 1 & -1 & 6 \end{pmatrix}\begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix}$

14. (a) Proporcione el tamaño de cada matriz.

$$A=\begin{pmatrix}1 & 4 & 6\\ 0 & 1 & 7\end{pmatrix},\qquad B=\begin{pmatrix}1 & 4 & 7\\ 8 & 2 & 1\\ 0 & 1 & 6\end{pmatrix},\qquad C=\begin{pmatrix}4 & 2\\ 0 & 0\\ 2 & 9\end{pmatrix}$$

(b) Utilice las matrices de la parte (a) para decidir cuáles de los productos

$$A^2,\quad AB,\quad BA,\quad AC,\quad CA,\quad AB^2,\quad BC,\quad CB,\quad C^2$$

están definidos, y luego calcule estos productos.

15. Determine x, y y z de modo que se cumpla la ecuación

$$\begin{pmatrix}x+y & 3x+y\\ x+z & x+y-2z\end{pmatrix}=\begin{pmatrix}-1 & 1\\ 9 & -17\end{pmatrix}.$$

16. Determine w, x, y y z de modo que se cumpla la ecuación

$$\begin{pmatrix}2 & 1 & -1 & 7\\ 6 & 8 & 0 & 2\end{pmatrix}\begin{pmatrix}x & 2x\\ y & -y+z\\ x+w & w-2y+x\\ z & z\end{pmatrix}=-\begin{pmatrix}45 & 46\\ 3 & 87\end{pmatrix}.$$

17. Defina la matriz $n \times n$ $I_n = (a_{ij})$ como

$$a_{ij}=\begin{cases}1 & \text{si } i=j\\ 0 & \text{si } i\neq j\end{cases}.$$

La matriz I_n es la **matriz identidad** $n \times n$.

Muestre que si A es una matriz $n \times n$ (una matriz de este tipo es una **matriz cuadrada**), entonces

$$AI_n = A = I_nA.$$

Una matriz $n \times n$ A es *invertible* si existe una matriz $n \times n$ B tal que

$$AB = I_n = BA.$$

(La matriz I_n se definió en el ejercicio 17.)

18. Muestre que la matriz

$$\begin{pmatrix}2 & 1\\ 1 & 1\end{pmatrix}$$

es invertible.

☆ 19. Muestre que la matriz

$$\begin{pmatrix}a & b\\ c & d\end{pmatrix}$$

es invertible si y sólo si $ad - bc \neq 0$.

20. Suponga que queremos resolver el sistema

$$AX = C \quad \text{donde} \quad A=\begin{pmatrix}a_{11} & a_{12}\\ a_{21} & a_{22}\end{pmatrix},\qquad X=\begin{pmatrix}x\\ y\end{pmatrix},\qquad C=\begin{pmatrix}c_1\\ c_2\end{pmatrix}$$

en términos de x y y.

Muestre que si A es invertible, el sistema tiene una solución.

21. La **transpuesta** de una matriz $A = (a_{ij})$ es la matriz $A^T = (a'_{ji})$, donde $a'_{ji} = a_{ij}$. *Ejemplo*:

$$\begin{pmatrix}1 & 3\\ 4 & 6\end{pmatrix}^T=\begin{pmatrix}1 & 4\\ 3 & 6\end{pmatrix}.$$

Si A y B son matrices $m \times k$ y $k \times n$, respectivamente, muestre que

$$(AB)^T = B^TA^T.$$

REFERENCIAS

AHO, A., J. HOPCROFT, y J. ULLMAN, *The Desing and Analysis of Computer Algorithms*, Addison-Wesley, Reading, Mass., 1974.

AINSLIE, T., *Ainslie's Complete Hoyle*, Simon and Schuster, Nueva York, 1975.

AKL, S. G., *The Desing and Analysis of Parallel Algorithms*, Prentice Hall, Upper Saddle River, N. J., 1989.

APPEL, K., y W. HAKEN, "Every planar map is four-colorable", *Illinois, J. Math.*, 21 (1977), 429-567.

BAASE, S., *Computer Algorithms: Introduction to Desing and Analysis*, 2a. ed., Addisson-Wesley, Reading, Mass., 1988.

BABAI, L., y T. KUCERA, "Canonical labelling of graphs in linear average time", *Proc. 20th symposium on the Foundations of Computer Science*, 1979, 39-46.

BARKE, S. F., *The Elements of Logic*, 2a. ed., McGraw-Hill, Nueva York, 1984.

BELL, R. C., *Board and Table Games from Many Civilizations*, ed. rev., Dover, Nueva York, 1979.

BENTLEY, J., *Programming Pearls*, Addison-Wesley, Reading, Mass., 1986.

BERGE, C., *Graphs and Hypergraphs*, North-Holland, Amsterdam, 1979.

BERLEKAMP, E. R., J. H. CONWAY, y R. K. GUY, *Winning Ways*, Vols. 1 y 2, Academic Press, Nueva York, 1982.

BONDY, J. A., y U.S.R. MURTY, *Graph Theory with Applications*, American Elsevier, Nueva York, 1976.

BOOLE, G. *The Laws of Thought*, reimpreso por Dover, Nueva York, 1951.

BRASSARD, G., y P. BRATLEY, *Fundamentals of Algorithms*, Prentice Hall, Upper Saddle River, N. J., 1996.

BRUALDI, R. A., *Introductory Combinatorics*, 2a. ed., Prentice Hall, Upper Saddle River, N. J., 1992.

CARMONY, L., "Odd pie fights", *Math, Teacher*, 72 (1979), 61-64.

CARROLL, J., y D. LONG, *Theory of Finite Automata*, Prentice Hall, Upper Saddle River, N. J., 1989.

CHARTRAND, G., y L. LESNIAK, *Graphs and Diagraphs*, 2a. ed., Wadsworth, Belmont, Calif., 1986.

CHU, I. P., y R. JOHNSONBAUGH, "Tiling deficient boards with trominoes", *Math. Mag.*, 59 (1986), 34-40.

CODD, E. F., "A relational model of data for large shared databanks", *Comm, ACM,* 13 (1970), 377-387.

COHEN, D. I. A., *Introduction to Computer Theory,* Wiley, Nueva York, 1986.

COPI, I. M., *Introduction to Logic,* 7a. ed., Macmillan, Nueva York, 1986.

CORMEN, T. H., C. E. LEISERSON, y R. L. RIVEST, *Introduction to Algorithms,* MIT Press, Cambridge, Mass., 1990.

CULL. P., y E. F. ECKLUND, JR., "Towers of Hanoi and analysis of algorithms", *Amer. Math. Monthly,* 92 (1985), 407-420.

DATE, C. J., *An Introduction to Database Systems,* 4a. ed., Vol. I Addison-Wesley, Reading, Mass., 1986.

DAVIS, M. D., R. SIGAL, y E. J. WEYUKER, *Computability, Complexity, and Languages,* 2a. ed., Academic Press, San Diego, 1994.

DEO, N., *Graph Theory and Applications to Engineering and Computer Science,* Prentice Hall, Upper Saddle River, N. J., 1974.

DIJKSTRA, E. W., "A note on two problems in connexion with graphs", *Numer, Math.*, 1 (1959), 260-271

DIJKSTRA, E. W., "Cooperating sequential processes", in *Programming Languages,* F. Genuys, ed., Academic Press, Nueva York, 1968.

DOSSEY J. A., A. D. OTTO, L. E. SPENCE, y C. VANDEN EYNDEN, *Discrete Mathematics,* Scott, Foresman, Glenview, Ill., 1987.

EDELSBRUNNER, H., *Algorithms in Combinatorial Geometry,* Springer-Verlag, Nueva York, 1987.

EDGAR, W. L., *The Elements of Logic,* SRA, Chicago, 1989.

ENGLISH, E., y S. HAMILTON, "Network security under siege, the timing attack", *Computer,* (Marzo 1996), 95-97.

ERBAS, C., y M. M. TANIK, "Generating solutions to the n-queens problem using 2-circulants", *Math. Mag.*, 68 (1995), 343-356.

EVEN, S., *Algorithmic Combinatorics,* Macmillan, Nueva York, 1973.

EVEN, S., *Graph Algorithms,* Computer Science Press, Rockville, Md., 1979.

EZEKIEL, M., "The cobweb theorem", *Quart. J. Econom.,* 52 (1938), 225-280.

FORD, L. R. JR., y D. R. FULKERSON, *Flows in Networks,* Princeton University Press, Princeton, N. J., 1962.

FOWLER, P. A., "The Königsberg bridges—250 years later", *Amer. Math Monthly,* 95 (1988), 42-43.

FREY, P., "Machine-problem solving—Part 3: The alpha-beta procedure", *BYTE,* 5 (Noviembre 1980), 244-264.

FUKUNAGA, K., *Introduction to Statistical Pattern Recognition,* 2a. ed., Academic Press, Nueva York, 1990.

GALLIER, J. H., *Logic for Computer Science,* Harper & Row, Nueva York, 1986.

GARDNER, M., *Mathematical Puzzles and Diversions,* Simon and Schuter, 1959.

GARDNER, M., "A new kind of cipher that would take millions of years to break", *Sci. Amer.* (Febrero 1977), 120-124.

GARDNER, M., *Mathematical Circus,* Alfred A. Knopf, Nueva York, 1979.

GENESERETH, M. R., y N. J. NILSSON, *Logical foundations of Artificial Intelligence,* Morgan Kaufman, Los Altos, Calif., 1987.

GIBBONS, A., *Algorithmic Graph Theory,* Cambridge University Press, Cambridge, 1985.

GOLDBERG, S., *Introduction to Difference Equations,* Wiley, Nueva York, 1958.

GOLOMB, S. W., "Checker boards and polyominoes", *Amer. Math. Monthly,* 61 (1954), 675-682.

GOLOMB, S., y L. BAUMERT, "Backtrack programming", *J. ACM,* 12 (1965), 516-524.

GOSE, E., R. JOHNSONBAUGH, y S. JOST, *Pattern Recognition and Image Analysis,* Prentice Hall, Upper Saddle River, N. J., 1996.

GRAHAM, R. L., "An efficient algoritm for determining tho convex hull of a finite planar set", *Info. Proc. Lett.*, 1 (1972), 132-133.

GRAHAM, R. L., D. E. KNUTH, y O. PATASHNIK, *Concrete Mathematics: A Foundation for Computer Science,* Addison-Wesley, Reading, Mass., 1988.

GRIES, D., *The Sciencie of Programming,* Springer-Verlag, Nueva York, 1981.

HAILPERIN, T., "Boole's algebra isn't Boolean algebra", *Math. Mag.*, 54 (1981), 137-184.

HALMOS, P. R., *Naive Set Theory,* Addison-Wesley, Reading, Mass., 1969.

HELL, P., "Absolute retracts in graphs", en *Graphs and Combinatorics,* R. A. Bari y F. Harary, eds., Lecture Notes in Mathematics, Vol. 406, Springer-Verlag, Nueva York, 1974.

HILLIER, F. S., y G. J. LIEBERMAN, *Introduction to Operations Research,* Holden-Day, San Francisco, 1974.

HINZ, A. M., "The Tower of Hanoi", *Enseignement Math.*, 35 (1989), 289-321.

HOHN, F., *Applied Boolean Algebra,* 2a. ed., Macmillan, Nueva York, 1966.

HOPCROFT, J. E., y J. D. ULLMAN, *Introduction to Automata Theory, Languages, and Computation,* Addison-Wesley, Reading, Mass., 1982.

HU, T. C., *Combinatorial Algorithms,* Addison-Wesley, Reading, Mass., 1982.

JACOBS, H. R., *Geometry,* W. H. Freeman, San Francisco, 1974.

JARVIS, R. A., "On the identification of the convex hull of a finite set of points in the plane", *Info. Proc. Lett.*, 2 (1973), 18-21.

JONES, R. H., y N C. STEELE, *Mathematics in Communication Theory,* Ellis Horwood, Chichester, Inglaterra, 1989.

KELLEY, D., *Automata and Formal Languages,* Prentice Hall, Upper Saddle River, N. J., 1995.

KLEINROCK, L., *Queueing Systems,* Vol. 2: *Computer Applications,* Wiley, Nueva York, 1976.

KLINE, M., *Mathematical Thought from Ancient to Modern Times,* Oxford University Press, Nueva York, 1972.

KNUTH, D. E., *The Art of Computer Programming,* Vol. 1: *Fundamental Algorithms,* 2a. ed., Addison Wesley, Reading Mass., 1973.

KNUTH, D. E., *The Art of Computer Programming,* Vol. 3: *Sorting and Searching,* Addison Wesley, Reading Mass., 1973.

KNUTH, D. E., "Algorithms", *Sci. Amer.* (Abril 1977), 63-80.

KNUTH, D. E., *The Art of Computer Programming,* Vol. 2: *Seminumeric Algorithms,* 2a. ed., Addison-Wesley, Reading, Mass., 1981.

KNUTH, D. E., "Algorithm thinking and mathematical thinking", *Amer. Math. Monthly*, 92 (1985), 170-181.

KÖBLER, J., U. SCHÖNING, y J. TORÁN, *The Graph Isomorphism Problem: Its Structural Complexity*, Birkhäuser Verlag, Basel, Suiza, 1993.

KOHAVI, Z., *Switching and Finite Automata Theory*, 2a. ed., McGraw-Hill, Nueva York, 1978.

KÖNIG, D., *Theorie der endlichen und unendlichen Graphen*, Akademische Verlagsgesellschaft, Leipyig, 1936. (Reimpreso en 1950 por Chelsea, Nueva York.) (Traducción al inglés: *Theory of Finite and Infinite Graphs*, Birkhäuser Boston, Cambridge, Mass., 1990.)

KROENKE, D., *Database Processing*, 2a. ed., Science Research Associates, Palo Alto, Calif., 1983.

KRUSE, R. L., *Data Structures and Program Desing*, 2a. ed., Prentice Hall, Upper Saddle River, N. J., 1987.

KUROSAKA, R. T., "A ternary state of affairs", *BYTE*, 12 (Febrero 1987), 319-328.

LEIGHTON, F. T., *Introduction to Parallel Algorithms and Architectures*, Morgan Kaufmann, San Mateo, Calif., 1992.

LESTER, B. P., *The Art of Paralel Programming*, Prentice Hall, Upper Saddle River, N. J., 1993.

LEWIS, T. G., y H., EL-REWINI, *Introduction to Parallel Computing*, Prentice Hall, Upper Saddle River, N. J., 1992.

LINDENMAYER, A., "Mathematical models for cellular interaction in development", partes I y II, *J. Theoret, Biol.*, 18 (1968), 280-315.

LIPSCHUTZ, S., *Theory and Problems of Set Theory and Related Topic*, Schaum, Nueva York, 1964.

LIU, C. L., *Introduction to Combinatorial Mathematics*, McGraw-Hill, Nueva York, 1968.

LIU, C. L., *Elements of Discrete Mathematics*, 2a. ed., McGraw-Hill, Nueva York, 1985.

MANBER, U., *Introduction to Algorithms*, Addison-Wesley, Reading, Mass., 1989.

MANDELBROT, B. B., *Fractals: Form, Chance, and Dimension*, W. H. Freeman, San Francisco, 1977.

MANDELBROT, B. B., *The Fractal Geometry of Nature*, W. H. Freeman, San Francisco, 1982.

MARTIN, G. E., *Polyominoes: A Guide to Puzzles and Problems in Tiling*, Mathematical Association of America, Washington, D.C., 1991.

MCCALLA, T. R., *Digital Logic and Computer Desing*, Merrill, Nueva York, 1992.

MCNAUGHTON, R., *Elementary Computability, Formal Languages, and Automata*, Prentice Hall, Upper Saddle River, N. J., 1982.

MENDELSON, E., *Boolean Algebra and Switching Circuits*, Schaum, Nueva York, 1970.

NADLER, M., y E. P. SMITH, *Pattern Recognition Engineering*, Wiley, Nueva York, 1993.

NEWMAN, J. R., "Leonhard Euler and the Koenigsberg bridges", *Sci. Amer.* (Julio 1953), 66-70.

NIEVERGELT, J., J. C. FARRAR, y E. M. REINGOLD, *Computer Approaches to Mathematical Problems*, Prentice Hall, Upper Saddle River, N. J., 1974.

NILSSON, N. J., *Problem-Solving Methods in Artificial Intelligence*, McGraw-Hill, Nueva York, 1971.

NIVEN, I., *Mathematics of Choice*, Mathematical Association of America, Washington, D.C., 1965.

NYHOFF, L., y S. LEESTMA, *Data Structures and Program Design in Pascal*, 2a. ed., Macmillan, Nueva York, 1992.

ORE, O., *Graphs and Their Uses*, Mathematical Association of America, Washington, D.C., 1963.

PEARL, J., "The solution for the branching factor of the alpha-beta pruning algorithm and its optimality", *Comm. ACM*, 25 (1982), 559-564.

PEITGEN, H., y D. SAUPE, eds., *The Science of Fractal Images*, Springer-Verlag, Nueva York, 1988.

PETRI, C., "Kommunikation mit Automaten", Tesis doctoral. Universidad de Bonn, Bonn, Alemania Occidental, 1962 (en alemán). Traducido por C. F. Green, Jr., "Comunication with automata", Supplement to Technical Report RADC-TR-65-377, Vol. I, Rome Air Development Center, Griffiss Air Force Base, Nueva York, 1966.

PFLEEGER, C. P., *Security in Computing*, Prentice Hall, Upper Saddle River, N. J., 1989.

PREPARATA, F. P., y S. J. HONG, "Convex hulls of finite sets of points in two and three dimensions", *Comm. ACM*, 20 (1977), 87-93.

PREPARATA, F. P., y M. I. SHAMOS, *Computational Geometry*, Springer-Verlag, Nueva York, 1985.

Problem 1186, *Math. Mag.*, 58 (1985), 112-114.

PRODINGER, H., y R. TICHY, "Fibonacci numbers of graphs", *Fibonacci Quarterly*, 20 (1982), 16-21.

PRUSINKIEWICZ, P., "Graphical aplications of L-systems", *Proc. of Graphics Interface 1986—Vision Interface* (1986), 247-253.

PRUSINKIEWICZ, P., y J. HANNAN, "Applications of L-systems to computer imagery", en *Graph Grammars and Their Aplication to Computer Science; Third International Workshop*, H. Ehrig, M. Nagl, A. Rosenfeld, y G. Rosenberg, eds., Springer-Verlag, Nueva York, 1988.

QUINN, M. J., *Desing Efficient Algoritms for Parallel Computers*, McGraw-Hill, Nueva York, 1987.

READ, R. C., y D. G. CORNEIL, "The graph isomorphism disease", *J. Graph Theory*, 1 (1977), 339-363.

REINGOLD, E., J. NIEVERGELT, y N. DEO, *Combinatorial Algorithms*, Prentice Hall, Upper Saddle River, N. J., 1977.

RIORDAN, J., *An Introduction to Combinatorial Analysis*, Wiley, Nueva York, 1958.

RITTER, G. L., S. R. LOWRY, H. B. WOODRUFF, y T. L. ISENHOUR, "An aid to the superstitious", *Math. Teacher*, mayo 1977, 456-457.

ROBERTS, F. S., *Aplied Combinatorics*, Prentice Hall, Upper Saddle River, N. J., 1984.

ROBINSON, J. A., "A machine oriented logic based on the resolution principle", *J. ACM*, 12 (1965), 23-41.

ROSS, K. A., y C. R. B. WRIGHT, *Discrete Mathematics*, 3a. ed., Prentice Hall, Upper Saddle River, N. J., 1992.

SAAD, Y., y M. H. SCHULTZ, "Topological properties of hypercubes", *IEEE Trans. Computers*, 37 (1988), 867-872.

SCHWENK, A. S., "Which rectangular chessboards have a knight's tour?" *Math. Mag.*, 64 (1991), 325-332.

SEIDEL, R., "A convex hull algorithm optimal for points in even dimensions", M. S. thesis, Tech. Rep. 81-14, Dept. of Comp. Sci., Univ. of British Columbia, Vancouver, Canadá, 1981.

SHANNON, C. E., "A symbolic analysis of relay and switching circuits", *Trans. Amer. Inst. Electr. Engrs.*, 47 (1938), 713-723.

SLAGLE, J. R., *Artificial Inteligence: The Heuristic Programming Approach*, McGraw-Hill, Nueva York, 1971.

SMITH, A. R., "Plants, fractals, and formal lenguajes", *Computer Graphics*, 18 (1984), 1-10.

SOLOW, D., *How to Read and Do Proofs*, 2a. ed., Wiley, Nueva York, 1990.

STANDISH, T. A., *Data Structures, Algorithms, and Sofware Principles in C*, Addison-Wesley, Reading, Mass., 1995.

STOLL, R. R., *Set Theory and Logic*, W. H. Freeman, San Francisco, 1963.

SUDKAMP, T. A., *Languages and Machines*, Addison-Wesley, Reading, Mass., 1988.

TARJAN, R. E., *Data Structures and Network Algorithms*, Society for Industrial and Applied Mathematics, Filadelfia, 1983.

TAUBES, G., "Small army of code-breakers conquers a 129-digit giant", *Science*, 264, (1994), 776-777.

TUCKER, A., *Applied Combinatorics*, 2a. ed., Wiley, Nueva York, 1985.

ULLMAN, J. D., *Principles of Database Systems*, Computer Science Press, Rockville, Md., 1980.

VILENKIN, N. Y., *Combinatorics*, Academic Press, Nueva York, 1971.

WAGON, S., "Fourteen proofs of a result about tiling a rectangle", *Amer. Math. Monthly*, 94 (1987), 601-617. (Reimpreso en R. K. Guy y R. E. Woodrow, eds., *The Lighter Side of Mathematics*, Mathematical Association of America, Washington, D. C., 1994, 113-128).

WARD, S. A., y R. H. HALSTEAD, JR., *Computation Structures*, MIT Press, Cambridge, Mass., 1990.

WILSON, R. J., *Introduction to Graph Theory*, 2a. ed., Academic Press, Orlando, Fla., 1979.

WOOD, D., *Theory of Computation*, Harper & Row, Nueva York, 1987.

WOS, L., R. OVERBEEK, E. LUSK, y J. BOYLE, *Automated Reasoning*, Prentice Hall, Upper Saddle River, N. J., 1984.

SUGERENCIAS Y SOLUCIONES DE EJERCICIOS SELECCIONADOS

Sección 1.1

1. Es una proposición. Negación: $2 + 5 \neq 19$

4. Es una proposición. Negación: Audrey Meadows no fue la "Alicia" original en "The Honeymooners".

7. Es una proposición. Algún entero par mayor que 4 no es la suma de dos primos.

9. Verdadero

12. Verdadero

15.

p	q	$p \wedge \bar{q}$
V	V	F
V	F	V
F	V	F
F	F	F

18.

p	q	$(p \wedge q) \wedge \bar{p}$
V	V	F
V	F	F
F	V	F
F	F	F

21.

p	q	$(p \vee q) \wedge (\bar{p} \vee q) \wedge (p \vee \bar{q}) \wedge (\bar{p} \vee \bar{q})$
V	V	F
V	F	F
F	V	F
F	F	F

23. $p \wedge q$; falso

26. Hoy es lunes o está lloviendo.

29. (Hoy es lunes y está lloviendo) y no ocurre que (hace calor u hoy es lunes).

32. No. Un juez resolvió que la ley era "vaga". Se supone que la orden debería ser: "No se permitirá que una persona tenga más de tres [3] perros *o* tres [3] gatos dentro de la ciudad".

Sección 1.2

1. Si Joey estudia mucho, entonces aprobará el examen de matemáticas discretas.

4. Si Karina aprueba matemáticas discretas, entonces llevará el curso de algoritmos.

7. Si el programa es legible, entonces está bien estructurado.

8. [Para el ejercicio 1] Si Joey aprueba el examen de matemáticas discretas, entonces habrá estudiado mucho.

10. Verdadero **13.** Falso **16.** Falso

18. $p \rightarrow q$ **21.** $p \leftrightarrow (p \wedge \bar{r})$

22. Si hoy es lunes, entonces está lloviendo.

25. No ocurre que hoy sea lunes o que esté lloviendo si y sólo si hace calor.

28. Sean p: $4 < 6$ y q: $9 > 12$.
Afirmación dada: $p \rightarrow q$: falsa.
Recíproca: $q \rightarrow p$; si $9 > 12$, entonces $4 < 6$; verdadera.
Contrapositiva: $\bar{q} \rightarrow \bar{p}$; si $9 \leq 12$, entonces $4 \geq 6$; falsa.

31. Sean p: $|4| < 3$ y q: $-3 < 4 < 3$.
Afirmación dada: $q \rightarrow p$; verdadera.
Recíproca: $p \rightarrow q$; si $|4| < 3$, entonces $-3 < 4 < 3$; verdadera.
Contrapositiva: $\bar{p} \rightarrow \bar{q}$; si $|4| \geq 3$, entonces $-3 \geq 4$ o $4 \geq 3$; verdadera.

. $P \not\equiv Q$ **35.** $P \not\equiv Q$ **38.** $P \not\equiv Q$ **41.** $P \not\equiv Q$

p	q	p *impl* q	q *impl* p
V	V	V	V
V	F	F	F
F	V	F	F
F	F	V	V

Como p *impl* q es verdadera precisamente cuando q *impl* p es verdadera, p *impl* $q \equiv q$ *impl* p.

p	q	$p \to q$	$\bar{p} \vee q$
V	V	V	V
V	F	F	F
F	V	V	V
F	F	V	V

Como $p \to q$ es verdadera precisamente cuando $\bar{p} \vee q$ es verdadera, $p \to q \equiv \bar{p} \vee q$.

ción 1.3

Es una función proposicional. El dominio de discurso podrían ser todos los enteros.

Es una función proposicional. El dominio de discurso es el conjunto de todas las películas.

11 divide a 77. Verdadera

Para cada entero positivo n, n divide a 77. Falsa

Todos son más altos que todos los demás. Falsa

Alguien es mayor que otro. Verdadera

$\exists x \forall y L(x, y)$. Probablemente verdadera

$\forall x \exists y L(x, y)$. Por desgracia, probablemente falsa

Falsa. Un contraejemplo es $x = \frac{1}{2}$.

Verdadera. Por ejemplo, si $x = 0$, la proposición condicional, si $x > 1$, entonces $x^2 > x$, es verdadera, pues la hipótesis es falsa.

Falsa. Un contraejemplo es $x = 2, y = 0$.

Verdadera. Considere $x = y = 0$.

Falsa. Un contraejemplo es $x = y = 2$.

Verdadera. Considere $x = 1, y = \sqrt{8}$.

Verdadera. Considere $x = 0$. Entonces, para toda y, $x^2 + y^2 \geq 0$.

Verdadera. Para cualquier x, si hacemos $y = x - 1$, la proposición condicional, si $x < y$, entonces $x^2 < y^2$, es verdadera, pues la hipótesis es falsa.

La interpretación correcta es: Algún hombre no engaña a su esposa. Esta proposición es verdadera. Éste fue el significado que quería darle Abby. Ella recibió algunas cartas en las que las personas habían interpretado la proposición como: Ningún hombre engaña a su esposa. Bajo esta interpretación, la proposición es falsa.

Sección 1.4

1. Si tres puntos no son colineales, entonces existe exactamente un plano que los contiene.

4. Si x es un número real no negativo y n es un entero positivo, $x^{1/n}$ es el número no negativo y tal que $y^n = x$.

7. $x \cdot 0 + 0 = x \cdot 0$

pues $b + 0 = b$ para todos los números reales b

$= x \cdot (0 + 0)$

pues $b + 0 = b$ para todos los números reales b

$= x \cdot 0 + x \cdot 0$

pues $a(b + c) = ab + ac$ para todos los números reales a, b, c

Al considerar $a = b = x \cdot 0$ y $c = 0$, la ecuación anterior se convierte en $a + b = a + c$; así, $x \cdot 0 = b = c = 0$.

10. Válido $p \to q$, p, $\therefore q$ **13.** No válido $(p \vee r) \to q$, q, $\therefore \bar{p} \to r$

15. Válido. Si 64K es mejor que no tener memoria alguna, entonces compraremos una nueva computadora. Si 64K es mejor que no tener memoria alguna, entonces compraremos más memoria. Por lo tanto, si 64K es mejor que no tener memoria alguna, entonces compraremos una nueva computadora y compraremos más memoria.

18. No válido. Si no compraremos una nueva computadora, entonces 64K no es mejor que no tener memoria alguna. Compraremos una nueva computadora. Por lo tanto, 64K es mejor que no tener memoria alguna.

20. No válido **23.** No válido

26. Un análisis del argumento debe tomar en cuenta el hecho de que "nada" se utiliza de dos maneras muy diferentes.

Sección 1.5

1.

p	q	r	$p \vee q$	$\bar{p} \vee r$	$q \vee r$
V	V	V	V	V	V
V	V	F	V	F	V
V	F	V	V	V	V
V	F	F	V	F	F
F	V	V	V	V	V
F	V	F	V	V	V
F	F	V	F	V	V
F	F	F	F	V	F

2. 1. $\bar{p} \vee q \vee r$

2. $\bar{q}$
3. $\bar{r}$
4. $\bar{p} \vee r$ De 1 y 2
5. $\bar{p}$ De 3 y 4

5. Primero observemos que $p \to q$ es lógicamente equivalente a $\bar{p} \vee q$. Ahora argumentamos como sigue:

1. $\bar{p} \vee q$
2. $p \vee q$
3. q De 1 y 2

7. (Para el ejercicio 2).

1. $\bar{p} \vee q \vee r$ Hipótesis
2. $\bar{q}$ Hipótesis
3. $\bar{r}$ Hipótesis
4. p Negación de la conclusión
5. $\bar{p} \vee r$ De 1 y 2
6. $\bar{p}$ De 3 y 5

Ahora, combinamos 4 y 6 para obtener una contradicción.

Sección 1.6

1. PASO BASE. $1 = 1^2$

PASO INDUCTIVO. Supongamos que es verdadera para n.

$1 + \cdots + (2n - 1) + (2n + 1) = n^2 + 2n + 1 = (n + 1)^2$

4. PASO BASE. $1^2 = (1 \cdot 2 \cdot 3)/6$

PASO INDUCTIVO. Supongamos que es verdadera para n.

$$1^2 + \cdots + n^2 + (n+1)^2 = \frac{n(n+1)(2n+1)}{6} + (n+1)^2 = \frac{(n+1)(n+2)(2n+3)}{6}$$

7. PASO BASE. $1/(1 \cdot 3) = \frac{1}{3}$

PASO INDUCTIVO. Supongamos que es verdadera para n.

$$\frac{1}{1\cdot 3} + \cdots + \frac{1}{(2n-1)(2n+1)} + \frac{1}{(2n+1)(2n+3)} = \frac{n}{2n+1} + \frac{1}{(2n+1)(2n+3)} = \frac{n+1}{2n+3}$$

10. PASO BASE. $\cos x = \dfrac{\cos[(x/2)\cdot 2]\operatorname{sen}(x/2)}{\operatorname{sen}(x/2)}$

PASO INDUCTIVO. Supongamos que es verdadera para n. Entonces

$$\cos x + \cdots + \cos nx + \cos(n+1)x = \frac{\cos[(x/2)(n+1)]\operatorname{sen}(nx/2)}{\operatorname{sen}(x/2)} + \cos(n+1)x. \qquad (*)$$

Debemos mostrar que el lado derecho de $(*)$ es igual a

$$\frac{\cos[(x/2)(n+2)]\operatorname{sen}[(n+1)x/2]}{\operatorname{sen}(x/2)}.$$

Esto es lo mismo que mostrar que [después de multiplicar por el término $\operatorname{sen}(x/2)$]

$$\cos\left[\frac{x}{2}(n+1)\right]\operatorname{sen}\frac{nx}{2} + \cos(n+1)x \operatorname{sen}\frac{x}{2} = \cos\left[\frac{x}{2}(n+2)\right]\operatorname{sen}\left[\frac{(n+1)x}{2}\right]$$

Si $\alpha = (x/2)(n+1)$ y $\beta = x/2$, debemos mostrar que $\cos\alpha \operatorname{sen}(\alpha - \beta) + \cos 2\alpha \operatorname{sen}\beta = \cos(\alpha + \beta)\operatorname{sen}\alpha$.

Podemos verificar esta última ecuación reduciendo cada lado a términos que utilicen sólo α y β.

12. PASO BASE. $\frac{1}{2} \leq \frac{1}{2}$

PASO INDUCTIVO. Supongamos que es verdadera para n.

$$\frac{1\cdot 3\cdot 5\cdots(2n-1)(2n+1)}{2\cdot 4\cdot 6\cdots(2n)(2n+2)} \geq \frac{1}{2n}\cdot\frac{2n+1}{2n+2} = \frac{2n+1}{2n}\cdot\frac{1}{2n+2} \geq \frac{1}{2n+2}$$

15. PASO BASE $(n = 4)$. $2^4 = 16 \geq 16 = 4^2$

PASO INDUCTIVO. Supongamos que es verdadera para n.

$$(n+1)^2 = n^2 + 2n + 1 \leq 2^n + 2n + 1 \leq 2^n + 2^n \quad \text{por el ejercicio 14} \quad = 2^{n+1}$$

18. PASO BASE. $7^1 - 1 = 6$ es divisible entre 6.

PASO INDUCTIVO. Supongamos que 6 divide a $7^n - 1$. Ahora

$$7^{n+1} - 1 = 7 \cdot 7^n - 1 = 7^n - 1 + 6 \cdot 7^n.$$

Como 6 divide a $7^n - 1$ y a $6 \cdot 7^n$, divide a su suma, que es $7^{n+1} - 1$.

21. PASO BASE. $3^1 + 7^1 - 2 = 8$ es divisible entre 8.

PASO INDUCTIVO. Supongamos que 8 divide a $3^n + 7^n - 2$. Ahora

$$3^{n+1} + 7^{n+1} - 2 = 3(3^n + 7^n - 2) + 4(7^n + 1).$$

Por la hipótesis de inducción, 8 divide a $3^n + 7^n - 2$. Podemos utilizar inducción matemática para mostrar que 2 divide a $7^n + 1$ para toda $n \geq 1$ (el argumento es similar al dado en la sugerencia del ejercicio 18). Esto implica que 8 divide a $4(7^n + 1)$. Como 8 divide a $3(3^n + 7^n - 2)$ y $4(7^n + 1)$, divide a su suma, que es $3^{n+1} + 7^{n+1} - 2$.

23. En el paso inductivo, cuando se agrega la línea $(n + 1)$, debido a las hipótesis, la línea intersecará cada una de las otras n rectas. Ahora, imagine que recorremos la línea $(n + 1)$. Cada vez que pasamos por una de las regiones originales, se divide en dos regiones.

26. $\frac{5}{6} = \frac{1}{2} + \frac{1}{3} = \frac{1}{2} + \frac{1}{4} + \frac{1}{12}$

30. Podemos suponer que $p/q > 1$. Elegimos el máximo entero n tal que

$$\frac{1}{1} + \frac{1}{2} + \cdots + \frac{1}{n} \leq \frac{p}{q}.$$

Si obtenemos una igualdad, p/q está en forma egipcia, así que supongamos que

$$\frac{1}{1} + \frac{1}{2} + \cdots + \frac{1}{n} < \frac{p}{q}. \qquad (*)$$

Hacemos

$$D = \frac{p}{q} - \left(\frac{1}{1} + \cdots + \frac{1}{n}\right).$$

Es claro que $D > 0$. Como n es el máximo entero que satisface $(*)$,

$$\frac{1}{1} + \frac{1}{2} + \cdots + \frac{1}{n} + \frac{1}{n+1} \geq \frac{p}{q}.$$

Así,

$$D = \frac{p}{q} - \left(\frac{1}{1} + \cdots + \frac{1}{n}\right)$$
$$\leq \left(\frac{1}{1} + \cdots + \frac{1}{n} + \frac{1}{n+1}\right) - \left(\frac{1}{1} + \cdots + \frac{1}{n}\right)$$
$$= \frac{1}{n+1}.$$

En particular, $D < 1$. Por el ejercicio 28, D se puede escribir en forma egipcia:

$$D = \frac{1}{n_1} + \cdots + \frac{1}{n_k},$$

donde los n_i son distintos. Como

$$\frac{1}{n_i} \leq D \leq \frac{1}{1+n},$$

$n < n + 1 \leq n_i$ para $i = 1, \ldots, k$. Esto implica que

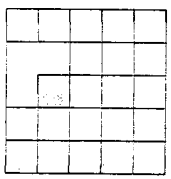
$$1, \quad 2, \quad \ldots, \quad n, \quad n_1, \quad \ldots, \quad n_k$$

son distintos. Así,

$$\frac{p}{q} = D + \frac{1}{1} + \cdots + \frac{1}{n} = \frac{1}{n_1} + \cdots + \frac{1}{n_k} + \frac{1}{1} + \cdots + \frac{1}{n}$$

se representa en forma egipcia.

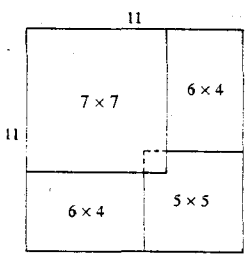
33. Un triominó puede cubrir el cuadrado a la izquierda del cuadrado faltante (que aparece sombreado) como se muestra



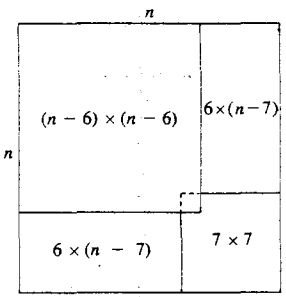
o en forma simétrica, reemplazando por "arriba" y "abajo". En el primer caso, es imposible cubrir los dos cuadrados del renglón superior, en el extremo izquierdo. En el segundo caso, es imposible cubrir los dos cuadrados del renglón inferior, en el extremo izquierdo. Por lo tanto, es imposible cubrir el tablero con triominós.

36. **PASO BASE.** ($n = 7$ u 11). El ejercicio 35 da una solución para $n = 7$.

Si $n = 11$, primero giramos el tablero de modo que el cuadrado faltante se localice en el tablero 7×7 que aparece en la siguiente figura. El ejercicio 35 muestra la forma de cubrir este subtablero. El ejercicio 34 muestra que los dos tableros 6×4 también se pueden cubrir. Es directo verificar que el tablero 5×5 con un cuadrado faltante en una esquina se puede cubrir. Hemos mostrado que cualquier tablero deficiente 11×11 se puede cubrir con triominós.



PASO INDUCTIVO. Supongamos que n es impar, $n > 11$, 3 divide a $n^2 - 1$, y que se pueden cubrir los tableros deficientes $k \times k$, donde k es impar, $n > k > 5$, y 3 divide a $k^2 - 1$. La siguiente figura muestra cómo cubrir un tablero deficiente $n \times n$. Primero giramos el tablero de modo que el cuadrado faltante se localiza en el subtablero $(n - 6) \times (n - 6)$. Ahora, $n - 6$ es impar, $n - 6 > 5$, y 3 divide a $(n - 6)^2 - 1$; así, por la hipótesis de inducción, este tablero deficiente $(n - 6) \times (n - 6)$ se puede cubrir. Como n es impar, $n - 7$ es par; así, por el ejercicio 34, los dos subtableros $6 \times (n - 7)$ se pueden cubrir. Por el ejercicio 35, el subrablero deficiente 7×7 se puede cubrir. Por lo tanto, podemos cubrir el tablero $n \times n$.



39. (a) $S_1 = 0 \neq 2$;

$$2 + \cdots + 2n + 2(n+1) = S_n + 2n + 2$$
$$= (n+2)(n-1) + 2n + 2$$
$$= (n+3)n$$
$$= S_{n+1}.$$

(b) Debemos tener $S'_n = S'_{n-1} + 2n$; así,

$$S'_n = S'_{n-1} + 2n = [S'_{n-2} + 2(n-1)] + 2n$$
$$= S'_{n-2} + 2n + 2(n-1)$$
$$= S'_{n-3} + 2n + 2(n-1) + 2(n-2) = \cdots$$
$$= S'_1 + 2[n + (n-1) + \cdots + 2]$$
$$= C + 2\left[\frac{n(n+1)}{2} - 1\right] = n^2 + n + C.$$

44. Por contradicción, supongamos que alguna afirmación $S(n)$ es falsa. Sea X el conjunto de enteros positivos n para los cuales $S(n)$ es falsa. Ahora, aplicamos el principio del buen orden.

Capítulo 1 Autoevaluación

1. Falsa

2.

p	q	r	$(p \overline{\wedge} q) \vee (p \vee \overline{r})$
V	V	V	V
V	V	F	V
V	F	V	V
V	F	F	V
F	V	V	V
F	V	F	V
F	F	V	V
F	F	F	V

3. Mi área es la administración hotelera y, o bien mi área no es la supervisión de diversiones o mi área es la cultura popular.

4. $p \vee (q \wedge \overline{r})$

5. Si Leah obtiene una buena calificación en matemáticas discretas, entonces Leah estudia mucho.

6. Recíproca: Si Leah estudia mucho, entonces Leah obtiene una buena calificación en matemáticas discretas. Contrapositiva: Si Leah no estudia mucho, entonces Leah no obtendrá una buena calificación en matemáticas discretas.

7. Verdadera **8.** $(\overline{r} \vee q) \rightarrow \overline{q}$

9. La afirmación no es una proposición. El valor de verdad no se puede determinar sin saber a cuál "equipo" se refiere.

10. La afirmación es una función proposicional. Al sustituir un equipo particular en la variable "equipo", la afirmación se convierte en una proposición.

11. Para todos los enteros positivos n, n y $n + 2$ son primos. La proposición es falsa. Un contraejemplo es $n = 7$.

12. Para algún entero positivo n, n y $n + 2$ son primos. La proposición es verdadera. Por ejemplo, si $n = 5$, n y $n + 2$ son primos.

13. Suponga que si cuatro equipos juegan siete juegos, ningún par de equipos juega al menos dos veces; o bien, en forma equivalente, si cuatro equipos juegan siete juegos, cada par de equipos juega a lo más una vez. Si los equipos son A, B, C y D, y cada par de equipos juega a lo más una vez, la mayor cantidad de juegos que se pueden realizar son:

A y B; A y C; A y D; B y C; B y D; C y D.

Así, a lo más se pueden realizar seis juegos. Ésta es una contradicción. Por lo tanto, si cuatro equipos juegan siete juegos, algún par de equipos juega al menos dos veces.

14. Los axiomas son afirmaciones que se suponen verdaderas. Las definiciones se utilizan para crear nuevos conceptos en términos de otros ya existentes.

15. En una demostración directa, no se supone la negación de la conclusión, mientras que en una demostración por contradicción, se supone la negación de la conclusión.

El argumento no es válido. Si p y r son verdaderas y q es falsa. las hipótesis son verdaderas, pero la conclusión es falsa.

$(p \vee q) \to r \equiv \overline{p \vee q} \vee r$
$\equiv \bar{p}\,\bar{q} \vee r$
$\equiv (\bar{p} \vee r)(\bar{q} \vee r)$

$(p \vee \bar{q}) \to \bar{r}s \equiv \overline{p \vee \bar{q}} \vee \bar{r}s$
$\equiv \bar{p}\,q \vee \bar{r}s$
$\equiv (\bar{p} \vee \bar{r})(\bar{p} \vee s)(q \vee \bar{r})(q \vee s)$

1. $\bar{p} \vee q$
2. $\bar{q} \vee \bar{r}$
3. $q \vee \bar{r}$
4. $\bar{p} \vee \bar{r}$ De 1 y 2
5. $\bar{r}$ De 3 y 4
1. $\bar{p} \vee q$
2. $\bar{q} \vee \bar{r}$
3. $q \vee \bar{r}$
4. r Negación de la conclusión
5. $\bar{p} \vee \bar{r}$ De 1 y 2
6. $\bar{r}$ De 3 y 5

Ahora, 4 y 6 dan una contradicción.

En los ejercicios 21-24, sólo aparece el paso inductivo.

$2 + 4 + \ldots + 2n + 2(n+1) = n(n+1) + 2(n+1)$
$= (n+1)(n+2)$

$2^2 + 4^2 + \cdots + (2n)^2 + [2(n+1)]^2$
$= \dfrac{2n(n+1)(2n+1)}{3} + [2(n+1)]^2$
$= \dfrac{2(n+1)(n+2)[2(n+1)+1]}{3}$

$\dfrac{1}{2!} + \dfrac{2}{3!} + \cdots + \dfrac{n}{(n+1)!}$
$+ \dfrac{n+1}{(n+2)!} = 1 - \dfrac{1}{(n+1)!} + \dfrac{n+1}{(n+2)!}$
$= 1 - \dfrac{1}{(n+2)!}$

$2^{n+2} = 2 \cdot 2^{n+1} < 2[1 + (n+1)2^n] = 2 + (n+1)2^{n+1}$
$= 1 + [1 + (n+1)2^{n+1}]$
$< 1 + [2^{n+1} + (n+1)2^{n+1}]$
$= 1 + (n+2)2^{n+1}$

Sección 2.1

1. $\{1, 2, 3, 4, 5, 7, 10\}$ **4.** $\{2, 3, 5\}$
7. $\emptyset$ **10.** U
13. $\{6, 8\}$ **16.** $\{1, 2, 3, 4, 5, 7, 10\}$
17. $\{(1, a), (1, b), (1, c), (2, a), (2, b), (2, c)\}$
20. $\{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c), (c, a), (c, b), (c, c)\}$
21. $\{(1, a, \alpha), (1, a, \beta), (2, a, \alpha), (2, a, \beta)\}$
24. $\{(a, 1, a, \alpha), (a, 2, a, \alpha), (a, 1, a, \beta), (a, 2, a, \beta)\}$
25. $\{\{1\}\}$
28. $\{\{a, b, c, d\}\}, \{\{a, b, c\}, \{d\}\}$,
$\{\{a, b, d\}, \{c\}\}, \{\{a, c, d\}, \{b\}\}, \{\{b, c, d\}, \{a\}\}$,
$\{\{a, b\}, \{c\}, \{d\}\}, \{\{a, c\}, \{b\}, \{d\}\}$,
$\{\{a, d\}, \{b\}, \{c\}\}$,
$\{\{b, c\}, \{a\}, \{d\}\}, \{\{b, d\}, \{a\}, \{c\}\}, \{\{c, d\}, \{a\}, \{b\}\}$,
$\{\{a, b\}, \{c, d\}\}, \{\{a, c\}, \{b, d\}\}, \{\{a, d\}, \{b, c\}\}$,
$\{\{a\}, \{b\}, \{c\}, \{d\}\}$
29. Verdadero **32.** Verdadero
33. Iguales **36.** Iguales
38. $\emptyset$, $\{a\}$, $\{b\}$, $\{a, b\}$. Todos son subconjuntos propios, excepto $\{a, b\}$.
41. $2^n - 1$ **43.** Falso. $X = \{1, 2\}$, $Y = \{2, 3\}$
46. Falso. $X = \{1, 2, 3\}$, $Y = \{2\}$, $Z = \{3\}$.
49. Falso. $X = \{1\}$, $Y = \{1, 2\}$. **52.** Verdadero
55. Verdadero **57.** $A \subseteq B$
60. $B \subseteq A$ **61.** $\{1, 4, 5\}$
65. $|A| + |B|$ cuenta los elementos de A y B, pero cuenta los elementos en $A \cap B$ dos veces.
68. P es el conjunto de números primos.

Sección 2.2

1. (a) c (b) c (c) $cddcdc$
4. (a) 12 (b) 23 (c) 7
(d) 46 (e) 1 (f) 3
(g) 3 (h) 21
8. (a) 15 (b) 155
(c) $2n + 3(n-1)n/2$ (d) Sí
(e) No
11. (a) 1, 3, 5, 7, 9, 11, 13
(b) 1, 5, 9, 13, 17, 21, 25
(c) $n_k = 2k - 1$ (c) $s_{nk} = 4k - 3$

13. (a) 88 (b) 1140
(c) 48 (d) 3168
16. $b_1 = 2, b_2 = 3, b_3 = 5, b_4 = 8, b_5 = 12, b_6 = 257$
19. Sea $s_0 = 0$. Entonces

$$\sum_{k=1}^{n} a_k b_k = \sum_{k=1}^{n} (s_k - s_{k-1}) b_k$$
$$= \sum_{k=1}^{n} s_k b_k - \sum_{k=1}^{n} s_{k-1} b_k$$
$$= \sum_{k=1}^{n} s_k b_k - \sum_{k=1}^{n} s_k b_{k+1} + s_n b_{n+1}$$
$$= \sum_{k=1}^{n} s_k (b_k - b_{k+1}) + s_n b_{n+1}.$$

22. 00, 01, 10, 11
25. 000, 010, 001, 011, 100, 110, 101, 111, 00, 01, 11, 10, 0, 1, λ
28. PASO BASE ($n = 1$). En este caso, $\{1\}$ es el único subconjunto no vacío de $\{1\}$, de modo que la suma es

$$\frac{1}{1} = 1 = n.$$

PASO INDUCTIVO. Supongamos que la afirmación es verdadera para n. Separamos los subconjuntos de $\{1, \ldots, n, n+1\}$ en dos clases:

C_1 = clase de subconjuntos no vacíos que no contienen a $n + 1$.

C_2 = clase de subconjuntos que contienen a $n + 1$.

Por la hipótesis de inducción,

$$\sum_{C_1} \frac{1}{n_1 \cdots n_k} = n.$$

Como un conjunto en C_2 consta de $n + 1$ junto con un subconjunto (vacío o no) de $\{1, \ldots, n\}$,

$$\sum_{C_2} \frac{1}{(n+1) n_1 \cdots n_k} = \frac{1}{n+1} + \frac{1}{n+1} \sum_{C_1} \frac{1}{n_1 \cdots n_k}.$$

[El término $1/(n+1)$ surge del subconjunto $\{n+1\}$.] Por la hipótesis de inducción,

$$\frac{1}{n+1} + \frac{1}{n+1} \sum_{C_1} \frac{1}{n_1 \cdots n_k}$$
$$= \frac{1}{n+1} + \frac{1}{n+1} \cdot n = 1.$$

Por lo tanto,

$$\sum_{C_2} \frac{1}{(n+1) n_1 \cdots n_k} = 1.$$

Por último,

$$\sum_{C_1 \cup C_2} \frac{1}{n_1 \cdots n_k} = \sum_{C_1} \frac{1}{n_1 \cdots n_k} + \sum_{C_2} \frac{1}{(n+1) n_1 \cdots n_k}$$
$$= n + 1.$$

Sección 2.3

1. 9 **4.** 32
7. 100010 **10.** 110010000
13. 11000 **16.** 1001000
19. 58 **22.** 2563
25. [Para el ejercicio 7] 22 **28.** FE
31. 3DBF9
33. 2010 no puede representar un número en binario, pues 2 no es un símbolo legal en binario. 2010 podría representar un número en decimal o hexadecimal.
35. 51 **38.** 4570
41. (Para el ejercicio 7) 42 **44.** (Para el ejercicio 35) 33
47. 9450 no puede representar un número en binario, pues 9, 4 y 5 no son símbolos legales en binario. 9450 no puede representar un número en octal, pues 9 no es un símbolo legal en octal. 9450 representa un número en decimal o hexadecimal.

Sección 2.4

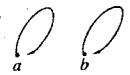
1. {(8840, Martillo), (9921, Pinzas), (452, Pintura), (22097, Tapiz)}
4. $\{(a, a), (b, b)\}$
5.

a	6
b	2
a	1
a	1

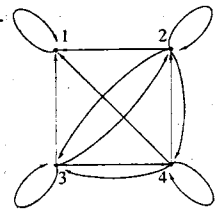
8.

Maine	Augusta
Maryland	Annapolis
Massachusetts	Boston
Michigan	Lansing
Minnesota	St. Paul
Mississippi	Jackson
Missouri	Jefferson City
Montana	Helena

9.



12.



13. $\{(a, b), (a, c), (b, a), (b, d), (c, c), (c, d)\}$

16. $\{(b, c), (c, b), (d, d)\}$

17. [Para el ejercicio 1] dominio = {8840, 9921, 452, 2207}, rango = {Martillo, Pinzas, Pintura, Tapiz}

19. $\{(1, 1), (1, 4), (2, 2), (2, 5), (3, 3), (4, 1), (4, 4), (5, 2), (5, 5)\}$

22. $\{1, 2, 3, 4, 5\}$

25. $R = R^{-1} = \{(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4), (3, 1), (3, 2), (3, 3), (4, 1), (4, 2), (5, 1)\}$
dominio R = rango R = dominio R^{-1} = rango $R^{-1} = \{1, 2, 3, 4, 5\}$

28. Antisimétrica **29.** Antisimétrica

32. Reflexiva, simétrica, antisimétrica, transitiva, orden parcial

34. Reflexiva, antisimétrica, transitiva, orden parcial

37. $R_1 \circ R_2 = \{(1, 1), (1, 2), (2, 1), (2, 2), (3, 1), (3, 2), (4, 2)\}$
$R_2 \circ R_1 = \{(1, 1), (1, 2), (3, 4), (4, 1), (4, 2)\}$

38. $\{(1, 1), (2, 2), (3, 3), (4, 4), (1, 2), (2, 3), (2, 1), (3, 2)\}$

41. $\{(1, 1), (1, 2), (2, 1), (2, 2)\}$

43. Falsa. Sean $R = \{(1, 2)\}$, $S = \{(2, 3)\}$.

46. Verdadera **49.** Verdadera **52.** Verdadera

55. Falsa. Sean $R = \{(1, 2)\}$, $S = \{(2, 1)\}$.

58. Verdadera

Sección 2.5

1. Relación de equivalencia: $[1] = [3] = \{1, 3\}$, $[2] = \{2\}$, $[4] = \{4\}$, $[5] = \{5\}$

4. Relación de equivalencia: $[1] = [3] = [5] = \{1, 3, 5\}$, $[2] = \{2\}$, $[4] = \{4\}$

7. No es una relación de equivalencia (no es transitiva ni reflexiva)

9. $\{(1, 1), (1, 2), (2, 1), (2, 2), (3, 3), (3, 4), (4, 3), (4, 4), [1] = [2] = \{1, 2\}, [3] = [4] = \{3, 4\}$

12. $\{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3), (4, 4), [1] = [2] = [3] = \{1, 2, 3\}, [4] = \{4\}$

16. $\{1\}, \{1, 3\}, \{1, 4\}, \{1, 3, 4\}$

18. (a) {San Francisco, San Diego, Los Ángeles}, {Pittsburgh, Philadelphia}, {Chicago}

21. $R = \{(x, x) \mid x \in X\}$

24. (b) $(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), (1, 10), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (9, 1), (10, 1)$

27. (a) Sólo mostraremos la simetría. Sean $(x, y) \in R_1 \cap R_2$. Entonces $(x, y) \in R_1$ y $(x, y) \in R_2$. Como R_1 y R_2 son simétricas, $(y, x) \in R_1$ y $(y, x) \in R_2$. Así, $(y, x) \in R_1 \cap R_2$ y por lo tanto, $R_1 \cap R_2$ es simétrica.

(b) A es una clase de equivalencia de $R_1 \cap R_2$ si y sólo si existen clases de equivalencia A_1 de R_1 y A_2 de R_2 tales que $A = A_1 \cap A_2$.

30. (b) Toro

31. $\rho(R_1) = \{(1, 1), (2, 2), (3, 3), (4, 4), (1, 2), (3, 4), (4, 2)\}$
$\sigma(R_1) = \{(1, 1), (2, 1), (1, 2), (3, 4), (4, 3), (4, 2), (2, 4)\}$
$\tau(R_1) = \{(1, 1), (1, 2), (3, 4), (4, 2), (3, 2)\}$
$\tau(\sigma(\rho(R_1))) = \{(x, y) \mid x, y \in \{1, 2, 3, 4\}\}$

34. Sean $(x, y), (y, z) \in \tau(R)$. Entonces $(x, y) \in R^m$ y $(y, z) \in R^n$. Así, $(x, z) \in R^{m+n}$. Por lo tanto, $(x, z) \in \tau(R)$ y $\tau(R)$ es transitiva.

37. Supongamos que R es transitiva. Si $(x, y) \in \tau(R) = \cup\{R^n\}$, entonces existen $x = x_0, \ldots, x_n = y \in X$ tales que $(x_{i-1}, x_i) \in R$ para $i = 1, \ldots, n$. Como R es transitiva, esto implica que $(x, y) \in R$. Así, $R \supseteq \tau(R)$. Como siempre tenemos que $R \subseteq \tau(R)$, esto implica que $R = \tau(R)$.

Supongamos que $\tau(R) = R$. Por el ejercicio 34, $\tau(R)$ es transitiva. Por lo tanto, R es transitiva.

38. Verdadero

41. Falso. Sean $R_1 = \{(1, 2), (2, 3)\}$, $R_2 = \{(1, 3), (3, 4)\}$.

44. Verdadero

Sección 2.6

1.

	α	β	Σ	δ
1	0	0	0	1
2	1	0	1	0
3	0	1	1	0

4.

	1	2	3	4	5
1	0	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	0
4	0	0	0	0	1
5	0	0	0	0	0

7. $R = \{(a, w), (a, y), (c, y), (d, w), (d, x), (d, y), (d, z)\}$

10. El criterio es: siempre que la entrada ij-ésima sea 1, $i \neq j$, entonces la entrada ji-ésima *no* es 1.

13. [Para el ejercicio 7]

	a	b	c	d
w	1	0	0	1
x	0	0	0	1
y	1	0	1	1
z	0	0	0	1

14. (a) $A_1 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}$

(b) $A_2 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$

(c) $A_1 A_2 = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$

(d) Cambiamos cada entrada distinta de cero de la parte (c) por 1 para obtener

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

(e) $\{(1, b), (1, a), (1, c), (2, b), (3, b)\}$

17. Cada columna que contiene un 1 en el renglón x corresponde a un elemento de la clase de equivalencia que contiene a x.

Sección 2.7

1. {(1089, Suzuki, Zamora), (5620, Kaminski, Jones), (9354, Jones, Yu), (9551, Ryan, Washington), (3600, Beaulieu, Yu), (0285, Schmidt, Jones), (6684, Manacotti, Jones)}

5. EMPLEADO [Nombre]
Suzuki, Kaminski, Jones, Ryan, Beaulieu, Schmidt, Manacotti

8. COMPRADOR [Nombre]
United Supplies, ABC Unlimited, JCN Electronics, Danny's, Underhanded Sales, DePaul University

11. TEMP := COMPRADOR [Número de parte = 20A8]
TEMP [Nombre]
Underhanded Sales, Danny's, ABC Unlimited

14. TEMP1 := COMPRADOR [Nombre = Danny's]
TEMP2 := TEMP1 [Número de parte = Parte número] PROVEEDOR
TEMP2 [Departamento]
04, 96

17. TEMP1 := COMPRADOR [Nombre = JCN Electronics]
TEMP2 := TEMP1 [Número de parte = Parte número] PROVEEDOR
TEMP3 := TEMP2 [Departamento = Departamento] DEPARTAMENTO
TEMP4 := TEMP3 [Jefe = Jefe] EMPLEADO
TEMP4 := [Nombre]
Kaminski, Schmidt, Manacotti

22. Sean R_1 y R_2 dos relaciones n-arias. Supongamos que el conjunto de elementos en la i-ésima columna de R_1 y el conjunto de elementos en la i-ésima columna de R_2 provienen de un dominio común, para $i = 1, \ldots, n$. La *unión* de R_1 y R_2 es la relación n-aria $R_1 \cup R_2$.
TEMP1 := DEPARTAMENTO [Departamento = 23]
TEMP2 := DEPARTAMENTO [Departamento = 96]
TEMP3 := TEMP1 *unión* TEMP2
TEMP4 := TEMP3 [Jefe = Jefe] EMPLEADO
TEMP4 [Nombre]
Kaminski, Schmidt, Manacotti, Suzuki

Sección 2.8

1. Es una función de X en Y; dominio = X, rango = $\{a, b, c\}$; no es uno a uno ni sobre.

4. No es una función (de X en Y).

6. Ejemplo 2.8.15

9. $f \circ g = \{(1, x), (2, z), (3, x)\}$

12. (a) $f \circ f = \{(a, a), (b, b), (c, a)\}$
$f \circ f \circ f = \{(a, b), (b, a), (c, b)\}$
(b) $f^9 = f$, $f^{623} = f$

15. El máximo común divisor de m y n debe ser 1.
En las soluciones a los ejercicios 18 y 21, $a : b$ significa guardar el elemento a en la celda b.

18. 53 : 9, 13 : 2, 281 : 6, 743 : 7, 377 : 3, 20 : 10, 10 : 0, 796 : 4

21. 714 : 0, 631 : 6, 26 : 5, 373 : 1, 775 : 8, 906 : 13, 509 : 2, 2032 : 7, 42 : 4, 4 : 3, 136 : 9, 1028 : 10

24. Durante una búsqueda, si detenemos ésta en una celda vacía, tal vez podríamos no encontrar el elemento, aunque estuviese presente. La celda podría estar vacía debido a la eliminación de un elemento. Una solución consiste en marcar las celdas eliminadas y considerarlas como no vacías durante una búsqueda.

25. Falsa. Sean $g = \{(a, x), (b, x)\}$ y $f = \{(x, 1)\}$.

28. Falsa. Sean $f = \{(a, z), (b, z)\}$ y $g = \{(1, a)\}$.

31. $g(S) = \{a\}$, $g(T) = \{a, c\}$, $g^{-1}(U) = \{1\}$, $g^{-1}(V) = \{1, 2, 3\}$

34. Si $x \in X$, entonces $x \in f^{-1}(f(\{x\}))$.
Así, $\cup\{S \mid S \in \mathcal{S}\} = X$. Supongamos que
$$a \in f^{-1}(\{y\}) \cap f^{-1}(\{z\})$$
para algunas $y, z \in Y$. Entonces $f(a) = y$ y $f(a) = z$. Así, $y = z$. Por lo tanto, $\mathcal{S}$ es una partición de X. La relación de equivalencia que genera esta partición está dada en el ejercicio 33.

[El caso: Si g es uno a uno, entonces $f \circ g$ es uno a uno implica que f es uno a uno.]

Supongamos que f no es uno a uno. Entonces existen $x_1, x_2 \in X$ distintos tales que $f(x_1) = f(x_2)$. Sea $A = \{1, 2\}$. Sea $g = \{(1, x_1), (2, x_2)\}$. En este caso, g es uno a uno, pero $f \circ g$ no lo es, y esto es una contradicción.

Si $x \in X \cap Y$, $C_{X\cap Y}(x) = 1 = 1 \cdot 1 = C_X(x)C_Y(x)$. Si $x \notin X \cap Y$, entonces $C_{X\cap Y}(x) = 0$. Como $x \notin X$ o $x \notin Y$, $C_X(x) = 0$ o $C_Y(x) = 0$. Así, $C_X(x)C_Y(x) = 0 = C_{X\cap Y}(x)$.

Si $x \in X - Y$, entonces

$$C_{X-Y}(x) = 1 = 1 \cdot [1 - 0] = C_X(x)[1 - C_Y(x)].$$

Si $x \notin X - Y$, entonces $x \notin X$ o $x \in Y$. En caso de que $x \notin X$,

$$C_{X-Y}(x) = 0 = 0 \cdot [1 - C_Y(x)] = C_X(x)[1 - C_Y(x)].$$

En caso de que $x \in Y$,

$$C_{X-Y}(x) = 0 = C_X(x)[1 - 1] = C_X(x)[1 - C_Y(x)].$$

Así, la ecuación es válida para todo $x \in U$.

$C_{X\Delta Y}(x) = C_X(x) + C_Y(x) - 2C_X(x)C_Y(x)$

Un conjunto es equivalente a sí mismo por medio de la función identidad.

Si X es equivalente a Y, existe una función, f, uno a uno y sobre X de Y. Ahora, f^{-1} es una función uno a uno y sobre de Y en X.

Si X es equivalente a Y, existe una función, f, uno a uno y sobre X en Y. Si Y es equivalente a Z, existe una función, g, uno a uno y sobre Y en Z. Ahora, $g \circ f$ es una función uno a uno y sobre g de X en Z.

Suponga que X es equivalente a $P(X)$. Entonces existe una función, f, uno a uno y sobre X en $P(X)$. Sea

$$Y = \{x \in X \mid x \notin f(x)\}.$$

Entonces $f(y) = Y$ para algún $y \in X$. Considere las posibilidades $y \in Y$ y $y \notin Y$.

f es un operador binario conmutativo.

f no es un operador binario, pues $f(x, 0)$ no está definido.

$g(x) = -x$

Cada renglón debe contener exactamente un 1 para que la relación sea una función.

Verdadero

Si n es un entero impar, $n = 2k - 1$ para algún entero k. Ahora,

$$\frac{n^2}{4} = \frac{(2k-1)^2}{4} = \frac{4k^2 - 4k + 1}{4} = k^2 - k + \frac{1}{4}.$$

Como $k^2 - k$ es un entero,

$$\left\lfloor \frac{n^2}{4} \right\rfloor = k^2 - k.$$

De esto se sigue el resultado, pues

$$\frac{n-1}{2}\frac{n+1}{2} = \frac{(2k-1)-1}{2}\frac{2k-1+1}{2}$$

$$= \frac{(2k-1)^2 - 1}{4}$$

$$= \frac{4k^2 - k}{4} = k^2 - k.$$

69. Abril y julio

Capítulo 2 Autoevaluación

1. $\varnothing$ **2.** 256, 255

3. $A \subseteq B$ **4.** Sí

5. (a) 14
(b) 18
(c) 192
(d) $a_{nk} = 4k$

6. $\sum_{k=-1}^{n-2} (n - k - 2)\, r^{k+2}$

7. (a) $b_5 = 35, b_{10} = 120$
(b) $(n + 1)^2 - 1$
(c) Sí
(d) No

8. (a) *ccddcccdd*
(b) *cccddccddc*
(c) 5
(d) 20

9. 150 **10.** 11010110, 1AE

11. 1000010 **12.** 3129

13. Reflexiva, simétrica, transitiva

14. Simétrica

15. $R = \{(1, 1), (2, 2), (3, 3), (4, 4), (1, 2), (2, 1), (2, 3)\}$

16. Todas las relaciones de los contraejemplos están definidas en $\{1, 2, 3\}$.
(a) Falsa. $R = \{(1, 1)\}$.
(b) Verdadera
(c) Verdadera
(d) Falsa. $R = \{(1, 1)\}$.

17. Sí. Es reflexiva, simétrica y transitiva.

18. $[3] = \{3, 4\}$. Existen dos clases de equivalencia.

19. $\{(a, a), (b, b), (b, d), (b, e), (d, b), (d, d), (d, e), (e, b), (e, d), (e, e), (c, c)\}$

20. (a) R es reflexiva, pues cualquier cadena de ocho bits tiene el mismo número de ceros que ella misma.

R es simétrica, pues si s_1 y s_2 tienen el mismo número de ceros, entonces s_2 y s_1 tienen el mismo número de ceros.

Para ver que R es transitiva, supongamos que s_1 y s_2 tienen el mismo número de ceros y que s_2 y s_3 tienen el mismo número de ceros. Entonces s_1 y s_3 tienen el mismo número de ceros. Por lo tanto, R es una relación de equivalencia.

(b) Hay nueve clases de equivalencia.

(c) 11111111, 01111111, 00111111, 00011111, 00001111, 00000111, 00000011, 00000001, 00000000

21. $\begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}$ **22.** $\begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$

23. $\begin{pmatrix} 1 & 1 & 0 \\ 2 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$ **24.** $\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$

25. ASIGNACIÓN [Equipo]
Blue Sox, Mutts, Jackalopes

26. JUGADOR [Nombre, Edad]
Johnsonbaugh, 22; Glover, 24; Battey, 18; Cage, 30; Homer, 37; Score, 22; Johnsonbaugh, 30; Singleton, 31

27. TEMP1 := JUGADOR [Posición]
TEMP2 := TEMP1 [Número de identificación = Número de identificación del jugador] ASIGNACIÓN
TEMP2 [Equipo]
Mutts, Jackalopes

28. TEMP1 := JUGADOR [Edad ≥ 30]
TEMP2 := TEMP1 [Número de identificación = Número de identificación del jugador] ASIGNACIÓN
TEMP2 [Equipo]
Blue Sox, Mutts

29. f no es uno a uno. f es sobre.

30. $x = y = 2.3$

31. Defina f de $X = \{1, 2\}$ en $\{3\}$ como $f(1) = f(2) = 3$. Defina g de $\{1\}$ en X como $g(1) = 1$.

32. ($a : b$ significa guardar el elemento a en la celda b.) 1 : 1 748 : 4, 18 : 5, 329 : 6, 43 : 7, 281 : 8, 620 : 9, 1141 : 10, 31 : 11, 684 : 12

Sección 3.1

1. Si $a < b$ entonces $x := a$, en caso contrario, $x := b$.
Si $c < x$ entonces $x := c$.

Sección 3.2

1. Entrada: La sucesión $s_1, s_2, \ldots, s_n$ y la longitud n de s
Salida: *small*, el menor elemento de la sucesión

```
procedure find_small(s, n)
 small := s_1
 i := 2
 while i ≤ n do
  begin
  if s_i < small then
   small := s_i
  i := i + 1
  end
 return(small)
end find_small
```

4. Entrada: La sucesión $s_1, s_2, \ldots, s_n$ y la longitud n de s
Salida: *small*, el menor elemento de la sucesión
large, el mayor elemento de la sucesión

```
procedure small_large(s, n, small, large)
 small := s_1
 large := s_1
 i := 2
 while i ≤ n do
  begin
  if s_i < small then
   small := s_i
  if s_i > large then
   large := s_i
  i := i + 1
  end
end small_large
```

7. Entrada: La sucesión $s_1, s_2, \ldots, s_n$, la longitud n de s y *key*, el valor por encontrar
Salida: El índice de la primera aparición del valor *key* en la sucesión

```
procedure find(s, n, key)
 i := 1
 while i ≤ n do
  begin
  if s_i = key then
   return(i)
  i := i + 1
  end
 return(0)
end find
```

10. Entrada: La sucesión $s_1, s_2, \ldots, s_n$ y la longitud n de s
Salida: El índice del primer elemento que sea mayor que su predecesor. Si ninguno cumple esto, la salida es 0.

```
procedure check_order(s, n)
 i := 2
 while i ≤ n do
  begin
  if s_i > s_{i-1} then
   return(i)
  i := i + 1
  end
 return(0)
end check_order
```

12. En este algoritmo, 0^k denota k ceros.

Entrada: $s_n s_{n-1} \cdots s_1$, $t_m t_{m-1} \cdots t_1$, dos números decimales; n y m

Salida: El producto de s y t

```
procedure product(s, t, n, m)
 for i := 1 to m do
  begin
  carry := 0
  for j := 1 to n do
   begin
   xy := representación decimal
    de carry + t_i s_j
   u_ij := y
   carry := x
   end
  u_{i,n+i} := carry
  end
 prod := 0
 for i := 1 to m do
  prod := prod + u_{i,n+i} ... u_{i1} 0^{i-1}
 return(prod)
end product
```

15. Entrada: A, la matriz $m \times n$ de la relación R; m y n

Salida: **true**, si R es una función
false, si R no es una función

```
procedure is_function(A, m, n)
 for i := 1 to m do
  begin
  sum := 0
  for j := 1 to n do
   sum := sum + A_ij
  if sum ≠ 1 then
   return(false)
  end
 return(true)
end is_function
```

Sección 3.3

1. $45 = 6 \cdot 7 + 3$ **4.** $221 = 17 \cdot 13 + 0$

7. Divida 90 entre 60 para obtener
$90 = 60 \cdot 1 + 30$.
Así, mcd(90, 60) = mcd(60, 30).
Divida 60 entre 30 para obtener $60 = 30 \cdot 2 + 0$.
Así, mcd(60, 30) = mcd(30, 0) = 30.
Por lo tanto, mcd(60, 90) = 30.

10. 15 **13.** 1 **16.** 1

17. Como $c \mid m$, $m = cq_1$ para algún entero q_1.
Como $c \mid n$, $n = cq_2$ para algún entero q_2. Ahora,
$$m - n = cq_1 - cq_2 = c(q_1 - q_2).$$
Por lo tanto, $c \mid (m - n)$.

20. Sea m un divisor común de a y de b. Por el teorema 3.3.4a, m divide a $a + b$. Así, m es un divisor común de a y de $a + b$.

Sea m un divisor común de a y de $a + b$. Por el teorema 3.3.4b, m divide a $(a + b) - a = b$. Así, m es un divisor común de a y b.

Como el conjunto de divisores comunes de a y $a + b$ es igual al conjunto de divisores comunes de a y b, mcd (a, b) = mcd$(a, a + b)$.

23. Si p divide a a, hemos terminado; así que podemos suponer que p no divide a a. Debemos mostrar que p divide a b. Como p es primo, mcd$(p, a) = 1$. Por el ejercicio 21, existen enteros s y t tales que
$$1 = sp + ta.$$
Al multiplicar ambos lados de esta ecuación por b, obtenemos
$$b = spb + tab.$$
Por el teorema 3.3.4c, p divide a spb y p divide a tab. Por el teorema 3.3.4a, p divide a $spb + tab = b$.

26. Entrada: a y b (enteros no negativos, no ambos iguales a cero)

Salida: El máximo común divisor de a y b.

```
procedure sub_gcd(a, b)
 while true do
  begin
  if a < b then
   swap(a, b)
  if b = 0 then
   return(a)
  a := a - b
  end
end sub_gcd
```


Sección 3.4

1. (a) En la línea 2, como $4 \neq 0$, pasamos a la línea 4. El algoritmo se llama con la entrada 3.
(b) En la línea 2, como $3 \neq 0$, pasamos a la línea 4. El algoritmo se llama con la entrada 2.
(c) En la línea 2, como $2 \neq 0$, pasamos a la línea 4. El algoritmo se llama con la entrada 1.
(d) En la línea 2, como $1 \neq 0$, pasamos a la línea 4. El algoritmo se llama con la entrada 0.
(e) En las líneas 2 y 3, como $0 = 0$, regresamos 1.
La ejecución regresa a la parte (d) en la línea 4, después de calcular 0! (= 1). Regresamos $0! \cdot 1 = 1$.
La ejecución regresa a la parte (c) en la línea 4, después de calcular 1! (= 1). Regresamos $1! \cdot 2 = 2$.
La ejecución regresa a la parte (b) en la línea 4, después de calcular 2! (= 2). Regresamos $2! \cdot 3 = 6$.
La ejecución regresa a la parte (a) en la línea 4, después de calcular 3! (= 6). Regresamos $3! \cdot 4 = 24$.

4. En la línea 1, como $5 < 0$ es falsa, pasamos a la línea 3. En la línea 3, como $b = 0$, regresamos 5.

7. (a) Entrada: n
Salida: $2 + 4 + \cdots + 2n$

```
1. procedure sum(n)
2.  if n = 1 then
3.   return(2)
4.  return(sum(n - 1) + 2n)
   end(sum)
```

(b) **PASO BASE** $(n = 1)$. Si n es igual a 1, regresamos 2 de manera correcta.

PASO INDUCTIVO. Supongamos que el algoritmo calcula en forma correcta la suma, cuando la entrada es $n - 1$. Ahora, supongamos que la entrada de este algoritmo es $n > 1$. En la línea 2, como $n \neq 1$, pasamos a la línea 4, donde llamamos al algoritmo con entrada $n - 1$. Por la hipótesis de inducción, el valor regresado $sum(n - 1)$ es igual a
$$2 + \cdots + 2(n - 1).$$
Luego, en la línea 4 regresamos
$sum(n - 1) + 2n = 2 + \cdots + 2(n - 1) + 2n$,
que es el valor correcto.

10. Entrada: n
Salida: $n!$

```
procedure factorial(n)
 fact := 1
 for i := 2 to n do
  fact := i · fact
 return(fact)
end factorial
```

13. Después de un mes, sigue habiendo una pareja, pues una pareja no puede reproducirse sino hasta transcurrir un mes. Por lo tanto, $a_1 = 1$. Después de dos meses, la pareja viva desde el principio puede reproducirse y agrega otra pareja. Por lo tanto, $a_2 = 2$. El incremento en parejas de conejos $a_n - a_{n-1}$ del mes $n - 1$ al mes n se debe a que cada pareja viva en el mes $n - 2$ produce una pareja adicional. Es decir, $a_n - a_{n-1} = a_{n-2}$. Como $\{a_n\}$ satisface las mismas relaciones de recurrencia que $\{f_n\}$ y las mismas condiciones iniciales, $a_n = f_n$, $n \geq 1$.

16. **PASO BASE** $(n = 2)$
$$f_2^2 = 4 = 1 \cdot 3 + 1 = f_1 f_3 + (-1)^2$$
PASO INDUCTIVO
$$\begin{aligned} f_n f_{n+2} + (-1)^{n+1} &= f_n(f_{n+1} + f_n) + (-1)^{n+1} \\ &= f_n f_{n+1} + f_n^2 + (-1)^{n+1} \\ &= f_n f_{n+1} + f_{n-1} f_{n+1} \\ &\quad + (-1)^n + (-1)^{n+1} \\ &= f_{n+1}(f_n + f_{n-1}) = f_{n+1}^2 \end{aligned}$$

19. **PASO BASE** $(n = 1, 2)$. f_1 (= 1) es impar y 2 no es divisible entre 3. f_2 (= 2) es par y 3 es divisible entre 3. Por lo tanto, la afirmación es verdadera para $n = 1, 2$.

PASO INDUCTIVO. Supongamos que la afirmación es verdadera para toda $k < n$. Debemos demostrar que la afirmación es verdadera para n. Podemos suponer que $n > 2$. Consideremos dos casos: $n + 1$ es divisible entre 3 y $n + 1$ no es divisible entre 3.

Si $n + 1$ es divisible entre 3, entonces n y $n - 1$ no son divisibles entre 3. Por la hipótesis de inducción, uno de los términos f_{n-1} o f_{n-2} es impar. Como $f_n = f_{n-1} + f_{n-2}$, f_n es par. Por lo tanto, si $n + 1$ es divisible entre 3, f_n es par.

Ahora supongamos que $n + 1$ no es divisible entre 3; entonces exactamente uno de los valores n o $n - 1$ es divisible entre 3. Por la hipótesis de inducción, uno de los términos f_{n-1} o f_{n-2} es impar y el otro es par. Como $f_n = f_{n-1} + f_{n-2}$, f_n es impar. Por lo tanto, si $n + 1$ no es divisible entre 3, f_n es impar.

Hemos mostrado que f_n es par si y sólo si $n + 1$ es divisible entre 3, lo cual concluye el paso inductivo.

22. Sólo mostraremos la primera fórmula. La segunda se demuestra de manera análoga.

PASO BASE
$$f_1 = 1 = 2 - 1 = f_2 - 1$$
PASO INDUCTIVO
$$\begin{aligned} \sum_{k=1}^{n+1} f_{2k-1} &= \sum_{k=1}^{n} f_{2k-1} + f_{2n+1} \\ &= (f_{2n} - 1) + f_{2n+1} = f_{2n+2} - 1 \end{aligned}$$

26. PASO BASE

$$\frac{dx}{dx} = 1 = 1x^{1-1}$$

PASO INDUCTIVO

$$\frac{dx^{n+1}}{dx} = \frac{d(x \cdot x^n)}{dx} = x\frac{dx^n}{dx} + x^n\frac{dx}{dx}$$
$$= xnx^{n-1} + x^n \cdot 1 = (n+1)x^n$$

Sección 3.5

1. $\Theta(n)$
4. $\Theta(n^2)$
7. $\Theta(n^2)$
10. $\Theta(n)$
13. $\Theta(n^2)$
16. $\Theta(n)$
19. $\Theta(n^2)$
22. $\Theta(n^3)$
25. $\Theta(\lg n)$
30. $n! = n(n-1)\cdots 2 \cdot 1$
$\leq n \cdot n \cdots n = n^n$
33. Verdadera
36. Falsa. Un contraejemplo es $f(n) = n$ y $g(n) = 2n$.
39. Verdadera
42. Falsa. Un contraejemplo es $f(n) = 1$ y $g(n) = 1/n$.
43. $f_n = \begin{cases} 1, \text{ si } n \text{ es par} \\ 0, \text{ si } n \text{ es impar} \end{cases}$
46. La desigualdad dada es equivalente, a su vez, a

$$(n/2)[(\lg n) - 1] \geq (n \lg n)/4$$
$$2[(\lg n) - 1] \geq \lg n$$
$$\lg n \geq 2.$$

La última desigualdad es una igualdad si $n = 4$. Como $\lg n$ es una función creciente, la última desigualdad es válida si $n \geq 4$. Así, la desigualdad dada es válida para $n \geq 4$.

49. (a) La suma de las áreas de los rectángulos debajo de la curva es igual a

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

Esta área es menor que el área bajo la curva, que es igual a

$$\int_1^n \frac{1}{x}\,dx = \log_e n.$$

La desigualdad dada es ahora una consecuencia inmediata.

(b) La suma de las áreas de los rectángulos cuyas bases están sobre el eje x y cuyas partes superiores están sobre la curva es igual a

$$1 + \frac{1}{2} + \cdots + \frac{1}{n-1}$$

Como esta área es mayor que el área bajo la curva, la desigualdad dada es ahora una consecuencia inmediata.

(c) La parte (a) muestra que

$$1 + \frac{1}{2} + \cdots + \frac{1}{n} = O(\log_e n).$$

Como $\log_e n = \Theta(\lg n)$ (véase el ejercicio 29),

$$1 + \frac{1}{2} + \cdots + \frac{1}{n} = O(\lg n).$$

De manera análoga, podemos concluir de la parte (b) que

$$1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Omega(\lg n).$$

Por lo tanto

$$1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\lg n).$$

52. (a) Verdadera
(b) Falsa. Un contraejemplo es
$f(n) = 1, g(n) = 2 + (-1)^n$.
(c) Falsa. Un contraejemplo es
$f(n) = n, g(n) = n^2$.
(d) Verdadera
(e) Falsa. Un contraejemplo es
$f(n) = 1, g(n) = 2 + (-1)^n$.

55. Multiplique ambos lados de la desigualdad del ejercicio 54 por $\lg e$ y utilice la fórmula del cambio de base para logaritmos.

Sección 3.6

1.

a \ b	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	–	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	2	1	2	1	2	1	2	1	2	1	2
3	0	1	2	1	2	3	1	2	3	1	2	3	1	2
4	0	1	1	2	1	2	2	3	1	2	2	3	1	2
5	0	1	2	3	2	1	2	3	4	3	1	2	3	4
6	0	1	1	1	2	2	1	2	2	2	3	3	1	2
7	0	1	2	2	3	3	2	1	2	3	3	4	4	3
8	0	1	1	3	1	4	2	2	1	2	2	4	2	5

a b	n (= Número de divisiones)
1 0	0
2 1	1
3 2	2
5 3	3
8 5	4
13 8	5

4. Utilizamos inducción sobre n para demostrar que cuando la pareja f_{n+1}, f_n se introduce como entrada en el algoritmo de Euclides se necesitan exactamente n divisiones.

PASO BASE ($n = 1$). La tabla 3.6.2 muestra que cuando la pareja f_2, f_1 se introduce como entrada en el algoritmo de Euclides, se necesita una división.

PASO INDUCTIVO. Supongamos que cuando la pareja f_{n+1}, f_n se introduce como entrada en el algoritmo de Euclides, se necesitan exactamente n divisiones. Debemos mostrar que cuando la pareja f_{n+2}, f_{n+1} se introduce como entrada en el algoritmo de Euclides, se necesitan $n + 1$ divisiones.

En la línea 7, dividimos f_{n+2} entre f_{n+1} para obtener

$$f_{n+2} = f_{n+1} + f_n.$$

Luego, el algoritmo se repite utilizando los valores de f_{n+1} y f_n. Por la hipótesis de inducción, se necesitan exactamente n divisiones adicionales. Así, se necesita un total de $n + 1$ divisiones.

Sección 3.7

1. FKKGEJAIMWQ
4. $a = c^s \bmod z = 411^{569} \bmod 713 = 500$
5. $z = pq = 17 \cdot 23 = 391$
8. $c = a^n \bmod z = 101^{31} \bmod 391 = 186$
11. Este algoritmo calcula $a^n \bmod z$.

Entrada: a, n, z
Salida: $a^n \bmod z$

```
procedure exp_mod(a, n, z)
exp := 1
x := a mod z
while n > 0 do
 begin
 if n es impar then
   exp := (exp · x) mod z
 x := (x · x) mod z
 n := n/2
 end
return(exp)
end exp_mod
```


14. Utilizamos el algoritmo de Euclides para obtener

$$660 = 22 \cdot 29 + 22, 29 = 22 \cdot 1 + 7, 22 = 3 \cdot 7 + 1.$$

Por lo tanto,

$$1 = 22 - 3 \cdot 7 = 22 - 3(29 - 22) = 4 \cdot 22 - 3 \cdot 29$$
$$= 4(660 - 22 \cdot 29) - 3 \cdot 29 = 4 \cdot 660 - 91 \cdot 29.$$

Así, $s = 91 \cdot 659 \bmod 660 = 569$.

Capítulo 3 Autoevaluación

1. En la línea 1, hacemos x igual a 12. En la línea 2, como $b > x$ (3 > 12) es falsa, pasamos a la línea 3. En la línea 3, como $c > x$ (0 > 12) es falsa, concluimos el algoritmo. El valor de x es 12, el máximo de los valores dados.

2. 1. $x := a, y := b, z := c$
2. Si $y < x$, swap (x, y)
3. Si $z < x$, swap (x, z)
4. Si $y > z$, swap (y, z)

3. 1. Si $a = b$, la salida es "No" y termina.
2. Si $a = c$, la salida es "No" y termina.
3. Si $b = c$, la salida es "No" y termina.
4. La salida es "Sí".

4. Si el conjunto S es infinito, el algoritmo no terminará jamás, de modo que no tiene carácter finito y carece de la propiedad relativa a la salida. La línea 1 no está enunciada de manera precisa, pues la *forma* de enumerar los subconjuntos de S y sus sumas no se ha especificado; así, el algoritmo no es preciso. El orden de los subconjuntos enumerados en la línea 1 depende del método utilizado para generarlos, de modo que el algoritmo carece de la propiedad de unicidad. Como la línea 2 depende del orden de los subconjuntos generados en la línea 1, aquí falla también la propiedad de unicidad.

5. En la línea 2, hacemos *large* igual a 7. En la línea 3, hacemos i igual a 2. En la línea 6, como $s_2 > large$ (9 > 7) es verdadera, hacemos *large* igual a 9. En la línea 6, como $s_3 > large$ (17 > 9) es verdadera, hacemos *large* igual a 17. En la línea 6, como $s_4 > large$ (7 > 17) es falsa, regresamos el valor 17.

6. Entrada: La matriz de tamaño $n \times n$ de una relación R y n

Salida: **true**, si R es simétrica
false, si R no es simétrica

```
procedure is_symmetric(A, n)
 for i := 1 to n − 1 do
  for j := i + 1 to n do
   if A_ij ≠ A_ji then
    return(false)
 return(true)
end is_symmetric
```

7. Entrada: La matriz A de tamaño $n \times n$ y n

Salida: A^T

```
procedure transpose(A, n)
 for i := 1 to n − 1 do
  for j := i + 1 to n do
   swap(A_ij, A_ji)
end transpose
```

8. Entrada: $s_1, \ldots, s_n$ y n

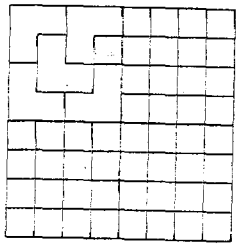
Salida: Todos los valores repetidos

```
procedure repeat(s, n)
 i := 1
 j := 2
 while j ≤ n do
  begin
  if s_i := s_j then
   begin
   print s_i
   while j ≤ n and s_i := s_j do
    j := j + 1
   end
  i := j
  j := j + 1
  end
end repeat
```

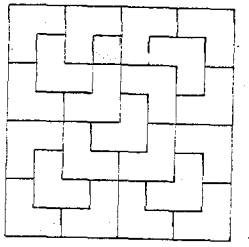
9. $333 = 24 \cdot 13 + 21$ **10.** 12

11. 2 **12.** mcd(b, r)

13. Como $n \neq 2$, pasamos de inmediato a la línea 3, donde dividimos al tablero en cuatro tableros 4×4. En la línea 4, giramos el tablero de modo que el cuadrado faltante esté en el cuadrante superior izquierdo, y llamamos al algoritmo para cubrir el subtablero superior izquierdo. La siguiente figura muestra el estado de la ejecución en este punto:



A continuación, en la línea 5, colocamos un triominó en el centro. Luego pasamos a las líneas 5–9, donde llamamos al algoritmo para cada uno de los subtableros restantes. Obtenemos el siguiente mosaico:



14. $t_4 = 3, t_5 = 5$

15. Entrada: n, un entero mayor o igual a 1

Salida: t_n

```
procedure tribonacci(n)
1. if n = 1 or n = 2 or n = 3 then
2.  return(1)
3. return(tribonacci(n − 1)
   +(tribonacci(n − 2)
   +(tribonacci(n − 3)
end tribonacci
```

16. PASO BASE ($n = 1, 2, 3$). Si $n = 1, 2, 3$, la salida de las líneas 1 y 2 tiene el valor correcto, 1. Por lo tanto, el algoritmo es correcto en estos casos.

PASO INDUCTIVO. Supongamos que $n > 3$ y que el algoritmo calcula de manera correcta t_k, si $k < n$. Como $n > 3$, pasamos a la línea 3. Luego llamamos a este algoritmo para calcular t_{n-1}, t_{n-2} y t_{n-3}. Por la hipótesis de inducción, los valores calculados son correctos. El algoritmo calcula entonces $t_{n-1} + t_{n-2} + t_{n-3}$. Pero la relación de recurrencia muestra que este valor es igual a t_n. Por lo tanto, el algoritmo calcula el valor correcto para t_n.

17. $\Theta(n^3)$ **18.** $\Theta(n^4)$ **19.** $\Theta(n^2)$

20. Entrada: A y B, matrices de $n \times n$, y n

Salida: **true**, si $A = B$; **false**, si $A \neq B$

```
procedure equal_matrix(A, B, n)
 for i := 1 to n do
  for j := 1 to n do
   if A_ij ≠ B_ij then
    return(false)
 return(true)
end equal_matrix
```

El tiempo en el peor de los casos es $\Theta(n^2)$.

21. Mostraremos que se necesitan 14 divisiones para el algoritmo de Euclides, en el peor de los casos, para números entre 0 y 1000. Por el teorema 3.6.1, si el par $a, b, a > b$, necesita 15 divisiones, entonces $a \geq f_{16} = 1597$. Por lo tanto, se necesitan a lo más 14 divisiones. Por el ejercicio 4 de la sección 3.6, cuando el par $f_{15} = 987$, $f_{14} = 610$ se utiliza como entrada para el algoritmo de Euclides, se necesitan exactamente 14 divisiones. Por lo tanto, se necesitan precisamente 14 divisiones en el algoritmo de Euclides, para el peor de los casos, para los números entre 0 y 1000.

22. El algoritmo de Euclides (algoritmo 3.3.8) necesita dos divisiones para calcular mcd(2,76652913). En la línea 4, intercambiamos a y b de modo que $a = 76652913$ y $b = 2$. Como $b \neq 0$, pasamos a la línea 7. La primera división ocurre al dividir 76652913 entre 2 para obtener

$$76652913 = 2 \cdot 38326456 + 1.$$

En las líneas 8 y 9, hacemos a igual a 2 y b igual a 1. Luego regresamos a la línea 5.

Como $b \neq 0$, pasamos a la línea 7. La segunda división ocurre cuando dividimos 2 entre 1 para obtener

$$2 = 1 \cdot 2 + 0.$$

En las líneas 8 y 9, hacemos a igual a 1 y b igual a 0. Luego regresamos a la línea 5.

Esta vez, $b = 0$, de modo que el algoritmo termina. Se realizaron dos divisiones.

23. 323 (véase el ejercicio 4, sección 3.6)

24. Como

$$\begin{aligned}\log_{3/2} \frac{2(100,000,000)}{3} &= \log_{3/2} 100^4 + \log_{3/2} \frac{2}{3} \\ &= 4 \log_{3/2} 100 - 1 \\ &= 4(11.357747) - 1 \\ &= 44.430988,\end{aligned}$$

una cota superior para el número de divisiones que necesita el algoritmo de Euclides para los enteros en el rango 0 a 100,000,000 es 44.

25. $z = pq = 13 \cdot 7 = 221$, $\phi = (p-1)(q-1) = 12 \cdot 16 = 192$

26. $ns \bmod \phi = 19 \cdot 91 \bmod 192 = 1729 \bmod 192 = 1$

27. $c = a^n \bmod z = 144^{19} \bmod 221 = 53$

28. $a = c^s \bmod z = 28^{91} \bmod 221 = 63$

Sección 4.1

1. $2 \cdot 4$ **4.** $8 \cdot 4 \cdot 5$ **7.** 8^2

10. $6 + 2$ **13.** 11 **16.** $10 \cdot 5$

19. $26^3 10^2$, $26 \cdot 25 \cdot 24 \cdot 10 \cdot 9$

22. $3 \cdot 2^6$ **25.** $2^8 - 1$ **27.** $5 \cdot 4 \cdot 3$

30. $3 \cdot 4 \cdot 3$ **33.** 5^3 **36.** $4 \cdot 3$

39. $5^3 - 4^3$ **41.** $200 - 5 + 1$ **44.** 40

47. Un número con 1 dígito contiene al 7. Los números distintos de dos dígitos, que contienen al 7, son 17, 27, . . . , 97 y 70, 71, . . . , 76,78,79. Existen 18 números de este tipo. Los números distintos de tres dígitos que contienen a 7 son 107 y 1xy, donde xy es uno de los números de dos dígitos ya mencionados. La respuesta es $1 + 18 + 19$.

50. $5 + (8 + 7 + \cdots + 1) + (7 + 6 + \cdots + 1)$

53. 10! **56.** (3!)(5!)(2!)(3!) **60.** 2^{10}

63. Contamos el número de relaciones antisimétricas en $\{1, 2, \ldots, n\}$ calculando el número de formas de construir la matriz de una relación antisimétrica.

Cada elemento de la diagonal puede ser 0 o 1. Así, existen 2^n formas de asignar valores a la diagonal.

Para i y j tales que $1 \leq i < j \leq n$, podemos asignar las entradas del renglón i, columna j y del renglón j, columna i, de tres formas:

Renglón i, columna j	*Renglón j, columna i*
0	0
1	0
0	1

Como existen $(n^2 - n)/2$ valores de i y j que satisfacen $1 \leq i < j \leq n$, podemos asignar los valores fuera de la diagonal de $3^{(n^2-n)/2}$. Por lo tanto, existen

$$2^n \cdot 3^{(n^2-n)/2}$$

relaciones antisimétricas en un conjunto de n elementos.

65. $2^5 + 2^7 - 2^4$ (De acuerdo con el ejercicio 64, el número total de posibilidades es igual al número de cadenas que comienzan con 100 + el número de cadenas que tienen el cuarto bit igual a uno menos el número de cadenas que comienzan con 100 y que tienen el cuarto bit igual a 1.)

68. $5 \cdot 4 + 3 \cdot 5 \cdot 4 - 2 \cdot 4$ (De acuerdo con el ejercicio 64, el número total de posibilidades es igual al número en que

Connie es presidente + número en que Alice tiene un puesto − número en que Connie es presidente y Alice tiene un puesto.)

Sección 4.2

1. $4! = 24$

4. *abc, acb, bac, bca, cab, cba, abd, adb, bad, bda, dab, dba, acd, adc, cad, cda, dac, dca, bcd, bdc, cbd, cdb, dbc, dcb*

7. $P(11, 3) = 11 \cdot 10 \cdot 9$ **10.** 3!

13. 4! contienen la subcadena *AE* y 4! contienen la subcadena *EA*; por lo tanto, el número total es $2 \cdot 4!$.

16. Primero contamos el número N de cadenas que contienen a la subcadena *AB* o a la subcadena *BE*. La respuesta al ejercicio será: Número total de cadenas $- N$, o bien $5! - N$.

De acuerdo con el ejercicio 64, sección 4.1, el número de cadenas que contienen a *AB* o *BE* es igual al número de cadenas que contienen a *AB* más el número de cadenas que contienen a *BE* menos el número de cadenas que contienen a *AB* y *BE*. Una cadena contiene a *AB* y *BE* si y sólo si contiene a *ABE* el número de tales cadenas es 3!. El número de cadenas que contienen a *AB* es igual al número de cadenas que contienen a $BE = 4!$. Así, el número de cadenas que contienen a *AB* o a *BE* es $4! + 4! - 3!$. La solución del ejercicio es

$$5! - (2 \cdot 4! - 3!).$$

19. $8!\ P(9, 5) = 8!(9 \cdot 8 \cdot 7 \cdot 6 \cdot 5)$

21. 10!

24. Fijamos un asiento para un jupiteriano. Existen 7! arreglos para los demás jupiterianos. Para cada uno de estos arreglos podemos colocar a los marcianos en cinco de las ocho posiciones intermedias, lo cual se puede hacer de $P(8, 5)$ formas. Así, existen $7!P(8, 5)$ arreglos de este tipo.

25. $C(4, 3) = 4$

28. $C(11, 3)$

31. $C(13, 5)$

34. Un comité que tiene a lo más un hombre tiene exactamente un hombre o ningún hombre. Existen $C(6, 1)C(7, 3)$ comités con exactamente un hombre. Existen $C(7, 4)$ comités sin hombres. Así, la respuesta es $C(6, 1)C(7, 3) + C(7, 4)$.

37. $C(10, 4)C(12, 3)C(4, 2)$

40. En primer lugar, contamos el número de cadenas de ocho bits sin dos ceros consecutivos. Separamos este problema en el problema de contar el número de cadenas de este tipo con exactamente ocho unos, con exactamente siete unos, y así sucesivamente.

Existe una cadena de ocho bits sin dos ceros consecutivos, la cual tiene exactamente ocho unos. Supongamos que una cadena de ocho bits sin ceros consecutivos tiene exactamente siete unos. El 0 puede aparecer en cualquiera de las ocho posiciones; así, existen ocho cadenas de este tipo. Supongamos que una cadena de ocho bits sin ceros consecutivos tiene exactamente seis unos. Los dos ceros deben ir en alguno de los espacios que se muestran:

$$_1_1_1_1_1_1_.$$

Así, los dos ceros se pueden colocar de $C(7, 2)$ formas. Así, existen $C(7, 2)$ cadenas de este tipo. De manera análoga, existen $C(6, 3)$ cadenas de ocho bits sin ceros consecutivos que tienen exactamente cinco unos y existen $C(5, 4)$ cadenas de ocho bits sin ceros consecutivos que tienen exactamente cuatro unos consecutivos. Si una cadena tiene menos de cuatro unos, tendrá dos ceros consecutivos. Por lo tanto, el número de cadenas de ocho bits sin ceros consecutivos es

$$1 + 8 + C(7, 2) + C(6, 3) + C(5, 4).$$

Como existen 2^8 cadenas de ocho bits, existen

$$2^8 - [1 + 8 + C(7, 2) + C(6, 3) + C(5, 4)]$$

cadenas de ocho bits que contienen al menos dos ceros consecutivos.

41. $1 \cdot 48$ (Los cuatro ases se pueden elegir de una forma y la quinta carta se puede elegir de 48 formas.)

44. En primer lugar, contamos el número de manos que contienen cartas con espadas y corazones. Como existen 26 espadas y corazones, existen $C(26, 5)$ formas de elegir cinco cartas entre estas 26. Sin embargo, $C(13, 5)$ sólo contienen espadas y $C(13, 5)$ sólo contienen corazones. Por lo tanto, existen

$$C(26, 5) - 2C(13, 5)$$

formas de elegir cinco cartas que contengan espadas y corazones.

Como existen $C(4, 2)$ formas de elegir dos palos, el número de manos que contienen cartas de exactamente dos palos es

$$C(4, 2)[C(26, 5) - 2C(13, 5)].$$

47. Existen nueve patrones consecutivos: A2345, 23456, 34567, 45678, 56789, 6789D, 789DJ, 89DJQ, 9DJQK. Para los cuatro palos posibles, existen cuatro formas de que ocurra cada patrón. Así, existen $9 \cdot 4$ manos que son consecutivas y del mismo palo.

50. $C(52, 13)$

53. $1 \cdot C(48, 9)$ (Se eligen los ases y luego se eligen las restantes 9 cartas.)

56. Existen $C(13, 4)C(13, 4)C(13, 4)C(13, 1)$ manos que contienen cuatro espadas, cuatro corazones, cuatro diamantes y un trébol. Como existen cuatro formas de elegir los tres palos para tener cuatro cartas de cada uno, existen $4C(13, 4)^3C(13, 1)$ manos que contienen cuatro cartas de tres palos y una carta del cuatro palo.

58. 2^{10} **61.** 2^9

63. $C(50, 4)$

66. $C(50, 4) - C(46, 4)$ (Número total − número de no defectuosos)

70. Ordenamos los $2n$ elementos. El primer elemento puede formar un par de $2n - 1$ formas. El siguiente elemento (aún no seleccionado) puede formar un par de $2n - 3$ formas, y así sucesivamente.

73. La solución cuenta manos *ordenadas*.

78. Utilice los teoremas 2.5.1 y 2.5.9.

Sección 4.3

1. 1357 **4.** 12435

7. [Para el ejercicio 1] En las líneas 9–14, determinamos el s_m más a la derecha, que no esté en su máximo valor. En este caso, $m = 4$. En la línea 16 incrementamos s_m. Esto hace que el último dígito sea 7. Como m es la posición extrema derecha, no hacemos nada en las líneas 18 y 19. La siguiente combinación es 1357.

9. 123, 124, 125, 126, 134, 135, 136, 145, 146, 156, 234, 235, 236, 245, 246, 256, 345, 346, 356, 456

12. 12, 21

14. Entrada: r, n

Salida: Una lista de todas las r-combinaciones de $\{1, 2, \ldots, n\}$ en orden lexicográfico creciente.

```
procedure r_comb1(r, n)
 s_0 := −1   // suponga que s_0 existe
 for i := 1 to r do
  s_i := i
 output s
 while true do
  begin
  m := r
  max_val := n
  while s_m = max_val do
   begin
   m := m − 1
   max_val := max_val − 1
   end
  if = 0 then
   return
  s_m := s_m + 1
  for j := m + 1 to r do
   s_j := s_{j−1} + 1
  output s
  end
end r_comb1
```

17. Entrada: $r, s_k, s_{k+1}, \ldots, s_n$, una cadena α, k y n

Salida: Una lista de todas las r-combinaciones de $\{s_k, s_{k+1}, \ldots, s_n\}$, cada una con α como prefijo. [Para enumerar todas las r combinaciones de $\{s_1, s_2, \ldots, s_n\}$, se llama a este procedimiento como $r_comb2(r, s, 1, n, \lambda)$, donde λ es la cadena nula.]

```
procedure r_comb2(r, s, k, n, α)
 if r = 0 then
  begin
  output α
  return
  end
 if r = n then
  begin
  output α s_k, s_{k+1}, ..., s_n
  return
  end
 // produce como salida las r-combinaciones
  que contienen a s_k
 β := α seguida de s_k
 r_comb2(r − 1, s, k + 1, n − 1, β)
 // produce como salida las r-combinaciones
  que no contienen a s_k
 r_comb2(r, s, k + 1, n − 1, α)
end r_comb2
```

Sección 4.4

1. 5! **4.** $10!/(5! \cdot 3! \cdot 2!)$

5. $C(10 + 3 - 1, 10)$

8. $C(9 + 2 - 1, 9)$

11. Cuatro, pues las posibilidades son (0, 0), (2, 1), (4, 2) y (6, 3), donde el par (r, v) denota r pelotas rojas y v pelotas verdes.

12. $C(15 + 3 - 1, 15)$

15. $C(13 + 2 - 1, 13)$

18. $C(12 + 4 - 1, 12) - C[(7 + 4 - 1, 7) + C(6 + 4 - 1, 6) + C(3 + 4 - 1, 3) + C(2 + 4 - 1, 2) - C(1 + 4 - 1, 1)]$

21. $52!/(13!)^4$ **24.** $C(20, 5)$ **27.** $C(20, 5)^2$

30. $C(15 + 6 - 1, 15)$ **33.** $C(10 + 12 - 1, 10)$

36. Aplique el resultado del ejemplo 4.4.9 a los $k - 1$ ciclos anidados interiores de ese ejemplo. A continuación, escriba el número de iteraciones para $i_1 = 1$; luego para $i_1 = 2$, y así sucesivamente. Por el ejemplo 4.4.9, esta suma es igual a $C(k + n - 1, k)$.

Sección 4.5

1. $x^4 + 4x^3y + 6x^2y^2 + 4xy^3 + y^4$

3. $C(11, 7)x^4y^7$ **6.** 5,987,520

9. $C(7, 3) + C(5, 2)$, pues

$$(a + \sqrt{ax} + x)^2(a + x)^5$$
$$= [(a + x) + \sqrt{ax}]^2(a + x)^5$$
$$= (a + x)^7 + 2\sqrt{ax}(a + x)^6 + ax(a + x)^5.$$

10. $C(10+3-1, 10)$ **13.** 1 8 28 56 70 56 28 8 1

16. [Sólo el paso inductivo] Supongamos que el teorema es verdadero para n.

$$(a+b)^{n+1} = (a+b)(a+b)^n$$

$$= (a+b)\sum_{k=0}^{n} C(n,k)a^{n-k}b^k$$

$$= \sum_{k=0}^{n} C(n,k)a^{n+1-k}b^k$$

$$+ \sum_{k=0}^{n} C(n,k)a^{n-k}b^{k+1}$$

$$= \sum_{k=0}^{n} C(n,k)a^{n+1-k}b^k$$

$$+ \sum_{k=1}^{n+1} C(n,k-1)a^{n+1-k}b^k$$

$$= C(n,0)a^{n+1}b^0 + \sum_{k=1}^{n} C(n,k)a^{n+1-k}b^k$$

$$+ C(n,n)a^0b^{n+1}$$

$$+ \sum_{k=1}^{n} C(n,k-1)a^{n+1-k}b^k$$

$$= C(n+1,0)a^{n+1}b^0$$

$$+ \sum_{k=0}^{n} [C(n,k)+C(n,k-1)]a^{n+1-k}b^k$$

$$+ C(n+1,n+1)a^0b^{n+1}$$

$$= C(n+1,0)a^{n+1}b^0$$

$$+ \sum_{k=1}^{n} C(n+1,k)a^{n+1-k}b^k$$

$$+ C(n+1,n+1)a^0b^{n+1}$$

$$= \sum_{k=0}^{n+1} C(n+1,k)a^{n+1-k}b^k$$

19. El número de soluciones enteras no negativas de

$$x_1 + x_2 + \cdots + x_{k+2} = n-k$$

es $C(k+2+n-k-1, n-k) = C(n+1, k+1)$. El número de soluciones es también el número de soluciones $C(k+1+n-k-1, n-k) = C(n,k)$ con $x_{k+2} = 0$ más el número de soluciones

$$C(k+1+n-k-1-1, n-k-1) = C(n-1, k)$$

con $x_{k+2} = 1$ más $\cdots$ más el número de soluciones $C(k+1+0-1, 0) = C(k,k)$ con $x_{k+2} = n-k$. De aquí se sigue el resultado.

22. Sean $a = 1$ y $b = 2$ en el teorema del binomio.

25. $x^3 + 3x^2y + 3x^2z + 3xy^2 + 6xyz + 3xz^2 + y^3 + 3y^2z + 3yz^2 + z^3$

28. Hacemos $a = 1$ y $b = x$ y reemplazamos n por $n-1$ en el teorema del binomio para obtener

$$(1+x)^{n-1} = \sum_{k=0}^{n-1} C(n-1,k)x^k.$$

Ahora multiplicamos por n para obtener

$$n(1+x)^{n-1} = n\sum_{k=0}^{n-1} C(n-1,k)x^k$$

$$= n\sum_{k=1}^{n} C(n-1,k-1)x^{k-1}$$

$$= \sum_{k=1}^{n} \frac{n(n-1)!}{(n-k)!\,(k-1)!}x^{k-1}$$

$$= \sum_{k=1}^{n} \frac{n!}{(n-k)!\,k!}kx^{k-1}$$

$$= \sum_{k=1}^{n} C(n,k)kx^{k-1}.$$

31. La solución es por inducción sobre k. Omitimos el paso base. Supongamos que la afirmación es verdadera para k. Después de k iteraciones, obtenemos la sucesión definida por

$$a'_j = \sum_{i=0}^{k-1} a_{i+j}\frac{B_i}{2^n}.$$

Sea $B'_0, \ldots, B'_k$ el renglón posterior a $B_0, \ldots, B_{k-1}$ en el triángulo de Pascal. Al suavizar a', mediante c para obtener a'' se tiene

$$a''j = \frac{1}{2}\left(a'_j + a'_{j+1}\right)$$

$$= \frac{1}{2^{n+1}}\left(\sum_{i=0}^{k-1} a_{i+j}B_i + \sum_{i=0}^{k-2} a_{i+j+1}B_i\right)$$

$$= \frac{1}{2^{n+1}}\left(a_jB_0 + \sum_{i=1}^{k-1} a_{i+j}B_i + \sum_{i=0}^{k-2} a_{i+j+1}B_i + a_{k+j}B_{k-1}\right)$$

$$= \frac{1}{2^{n+1}}\left(a_jB_0 + \sum_{i=1}^{k-1} a_{i+j}B_i + \sum_{i=1}^{k-1} a_{i-j}B_{i-1} + a_{k+j}B_{k-1}\right)$$

$$= \frac{1}{2^{n+1}}\left(a_jB'_0 + \sum_{i=1}^{k-1} a_{i+j}B'_i + a_{k+j}B'_k\right)$$

$$= \frac{1}{2^{n+1}}\sum_{i=0}^{k} a_{i+j}B'_i,$$

lo cual completa el paso inductivo.

Sección 4.6

1. Existen 12 nombres posibles para las 13 personas. Podemos considerar la asignación de nombres a las personas como la asignación de pichoneras a los pichones. Por el principio de la pichonera o de las casillas, algún nombre está asignado al menos a dos personas.

4. Sí. Conectamos los procesadores 1 y 2, 2 y 3, 2 y 4, 3 y 4. El procesador 5 no se conecta a los demás procesadores. Ahora, sólo los procesadores 3 y 4 están conectados directamente al mismo número de procesadores.

7. Sea a_i la posición del elemento i-ésimo no disponible. Consideremos

$$a_1, \ldots, a_{30}; \quad a_1+3, \ldots, a_{30}+3; \quad a_1+6, \ldots, a_{30}+6.$$

Estos 90 números varían entre 1 y 86. Por la segunda forma del principio de la pichonera, dos de estos números son iguales. Si $a_i = a_j + 3$, dos de ellos están a 3 artículos de distancia. Si $a_i + 3 = a_j + 6$, dos están a 6 artículos de distancia.

11. $n+1$

12. Suponga que $k \le m/2$. Es claro que $k \ge 1$. Como $m \le 2n+1$,

$$k \le \frac{m}{2} \le n + \frac{1}{2} < n+1.$$

Suponga que $k > m/2$. Entonces

$$m - k < m - \frac{m}{2} = \frac{m}{2} < n+1.$$

Como m es el máximo elemento en X, $k < m$. Así, $k+1 \le m$ y entonces $1 \le m-k$. Por lo tanto, el rango de a está contenido en $\{1, \ldots, n\}$.

13. Se puede aplicar la segunda forma del principio de la pichonera.

14. Suponga que $a_i = a_j$. Entonces $i \le m/2$ y $j > m/2$ o $j \le m/2$ e $i > m/2$. Podemos suponer que $i \le m/2$ y $j > m/2$. Ahora,

$$i + j = a_i + m - a_j = m.$$

24. Al dividir a entre b, los residuos posibles son $0, 1, \ldots, b-1$. Considere lo que ocurre después de b divisiones.

28. Suponemos que el tablero tiene tres renglones y siete columnas. Dos cuadrados de una columna que tengan el mismo color es un *par colorido*. Por el principio de la pichonera, cada columna contiene al menos un par colorido. Así, el tablero contiene siete pares coloridos, uno en cada columna. De nuevo, por el principio de la pichonera, al menos cuatro de estos siete pares coloridos tienen el mismo color, digamos rojo. Como existen tres pares de renglones y cuatro pares coloridos rojos, una tercera aplicación del principio de la pichonera muestra que al menos dos columnas contienen pares coloridos de color rojo. Estos pares coloridos determinan un rectángulo cuyas cuatro esquinas son rojas.

Capítulo 4 Autoevaluación

1. 2^4

2. $6\cdot9\cdot7 + 6\cdot9\cdot4 + 6\cdot7\cdot4 + 9\cdot7\cdot4$

3. $2^n - 2$

4. $6\cdot5\cdot4\cdot3 + 6\cdot5\cdot4\cdot3\cdot2$

5. $6!/(3!\,3!) = 20$

6. Construimos las cadenas mediante un proceso de tres pasos. Primero elegimos las posiciones para A, C y E [$C(6,3)$ formas]. Luego, colocamos A, C y E en esas posiciones. Podemos colocar a C de una forma (al final), y A y E de dos formas (AE o EA). Por último, colocamos las restantes tres letras (3! formas). Por lo tanto, el número total de cadenas es $C(6,3)\cdot 2\cdot 3!$.

7. Dos palos se pueden elegir de $C(4,2)$ formas. Podemos elegir tres cartas de un palo de $C(13,3)$ formas y elegir tres cartas del otro palo de $C(13,3)$ formas. Por lo tanto, el número total de manos es $C(4,2)C(13,3)^2$.

8. Debemos elegir tres de cuatro discos defectuosos. Así, el número total de selecciones es $C(5,3)C(95,1) + C(5,4)$.

9. 12567 **10.** 234567 **11.** 6427153

12. 631245 **13.** $8!/(3!\,2!)$

14. Contamos el número de cadenas en las que ninguna I aparece antes de ninguna L y luego las restamos del total de cadenas.

Construimos las cadenas en las que ninguna I aparece antes de alguna L mediante un proceso de dos pasos. En primer lugar, elegimos las posiciones de N, O y S; luego colocamos las I y las L. Podemos elegir las posiciones de N, O y S de $8\cdot7\cdot6$ formas. Entonces, las I y las L se pueden colocar sólo de una forma, pues las L deben aparecer primero. Así, existen $8\cdot7\cdot6$ cadenas en las que ninguna I aparece antes de alguna L.

El ejercicio 13 muestra que existen $8!/(3!\,2!)$ cadenas formadas al ordenar las letras *ILLINOIS*. Por lo tanto, existen

$$\frac{8!}{3!\,2!} - 8\cdot7\cdot6$$

cadenas formadas al ordenar las letras *ILLINOIS* en las que alguna I aparece antes de alguna L.

15. $12!/(3!)^4$

16. $C(11+4-1, 4-1)$

17. $(s-r)^4 = C(4,0)s^4 + C(4,1)s^3(-r) + C(4,2)s^2(-r)^2$

$$+ C(4,3)s(-r)^3 + C(4,4)(-r)^4$$
$$= s^4 - 4s^3r + 6s^2r^2 - 4sr^3 + r^4$$

18. $2^3 \cdot 8!/(3!\,1!\,4!)$

19. Si hacemos $a = 2$ y $b = -1$ en el teorema del binomio, obtenemos

$$1 = 1^n = [2 + (-1)]^n = \sum_{k=0}^{n} C(n,k)2^{n-k}(-1)^k.$$

20. $C(n,1) = n$

21. Si los 15 calcetines individuales juegan el papel de los pichones y los 14 tipos de pares son las pichoneras, asignamos cada calcetín (pichón) a su tipo (pichonera). Por el principio de la pichonera, alguna pichonera debe contener al menos dos pichones (los calcetines forman un par correcto).

22. Existen $3 \cdot 2 \cdot 3 = 18$ nombres posibles para las 19 personas. Podemos considerar la asignación de nombres a personas como la asignación de pichoneras a los pichones. Por el principio de la pichonera, algún nombre es asignado por lo menos a dos personas.

23. Sea a_i la posición del i-ésimo artículo disponible. Los 220 números

$$a_1, \ldots, a_{110}; \quad a_1 + 19, \ldots, a_{110} + 19$$

varían de 1 a 219. Por el principio de la pichonera, dos de ellos son iguales.

24. Cada punto tiene una abscisa que es par o impar, y una ordenada que es par o impar. Como existen cuatro posibilidades y existen cinco puntos, por el principio de la pichonera, al menos dos puntos, $p_i = (x_i, y_i)$ y $p_j = (x_j, y_j)$ satisfacen que

- x_i y x_j son ambos pares o ambos impares.

y

- y_i y y_j son ambos pares o ambos impares.

Por lo tanto, $x_i + x_j$ es par y $y_i + y_j$ es par. En particular, $(x_i + x_j)/2$ y $(y_i + y_j)/2$ son enteros. Así, el punto medio del par $p_i p_j$ tiene coordenadas enteras.

Sección 5.1

1. $a_n = a_{n-1} + 4; a_1 = 3$

4. $A_n = (1.14)A_{n-1}$ **5.** $A_0 = 2000$

6. $A_1 = 2280$, $A_2 = 2599.20$, $A_3 = 2963.088$

7. $A_n = (1.14)^n 2000$

8. Debemos tener $A_n = 4000$ o $(1.14)^n 2000 = 4000$ o $(1.14)^n = 2$. Al calcular el logaritmo de ambos lados, debemos tener $n \log 1.14 = \log 2$. Así,

$$n = \frac{\log 2}{\log 1.14} = 5.29.$$

18. Contamos el número de cadenas de n bits que no contienen al patrón 000.

- Que comienzan con 1. En este caso, si la restante cadena de $(n-1)$ bits no contiene 000, tampoco la cadena de n bits contendrá este patrón. Existen S_{n-1} de tales cadenas de $(n-1)$ bits.
- Que comienzan con 0. Hay que considerar dos casos.
  1. Que comienzan con 01. En este caso, si la restante cadena de $(n-2)$ bits no contiene a 000, tampoco lo contendrá la cadena de n bits. Existen S_{n-2} de tales cadenas de $(n-2)$ bits.
  2. Que comienzan con 00. Entonces, el tercer bit debe ser un 1, y si la restante cadena de $(n-3)$ bits no contiene a 000, tampoco lo contendrá la cadena de n bits. Existen S_{n-3} de tales cadenas de $(n-3)$ bits.

Como los casos son mutuamente excluyentes y abarcan a todas las cadenas de n bits ($n > 3$) que no contienen a 000, tenemos que $S_n = S_{n-1} + S_{n-2} + S_{n-3}$ para $n > 3$. $S_1 = 2$ (existen dos cadenas de 1 bit), $S_2 = 4$ (existen cuatro cadenas de 2 bits) y $S_3 = 7$ (existen ocho cadenas de 3 bits, pero una de ellas es 000).

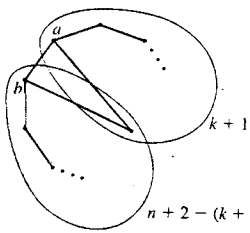
19. Existen S_{n-1} cadenas de n bits que comienzan con 1 y no contienen al patrón 00 y existen S_{n-2} cadenas de n bits que comienzan con 0 (pues el segundo bit debe ser 1) y no contienen al patrón 00. Así, $S_n = S_{n-1} + S_{n-2}$. Las condiciones iniciales son $S_1 = 2$, $S_2 = 3$.

22. $S_1 = 2$, $S_2 = 4$, $S_3 = 7$, $S_4 = 12$

25. $C_3 = 5$, $C_4 = 14$, $C_5 = 42$

29. Sea P_n el número de formas de dividir un polígono convexo de $(n+2)$ lados, $n \geq 1$, en triángulos, trazando $n-1$ líneas a través de las esquinas y de modo que no se intersequen en el interior del polígono. Observamos que $P_1 = 1$.

Supongamos que $n > 1$ y consideremos un polígono convexo de $(n+2)$ lados (véase la figura siguiente).



Elegimos una arista ab y construimos una partición del polígono mediante un procedimiento de dos pasos. En primer lugar, elegimos un triángulo al cual pertenezca el lado ab. Este triángulo divide al polígono original en dos polígonos; uno con $k+1$ lados, para cierta k tal que $1 \leq k \leq n$, y el otro con $n-k+2$ lados (véase la figura anterior). Por definición, el polígono de $(k+1)$ lados se puede separar de P_{k-1} formas y el polígono de $(n-k+2)$ lados se puede separar de P_{n-k} formas. (Para los casos degenerados $k = 1$ y $k = n$, hacemos $P_0 = 1$.) Por lo tanto, el número total de formas de separar el polígono de $(n+2)$ lados es

$$P_n = \sum_{k=1}^{n} P_{k-1} P_{n-k}.$$

Como la sucesión $P_1, P_2, \ldots$ satisface la misma relación de recurrencia que la sucesión de Catalan $C_1, C_2, \ldots$ y $P_0 = P_1 = 1 = C_0 = C_1$, esto implica que $P_n = C_n$ para toda $n \geq 1$.

33. [Para $n = 3$]

Paso 1: Mover el disco 3 del poste 1 al poste 3.
Paso 2: Mover el disco 2 del poste 1 al poste 2.
Paso 3: Mover el disco 3 del poste 3 al poste 2.
Paso 4: Mover el disco 1 del poste 1 al poste 3.
Paso 5: Mover el disco 3 del poste 2 al poste 1.
Paso 6: Mover el disco 2 del poste 2 al poste 3.
Paso 7: Mover el disco 3 del poste 1 al poste 3.

35. Sean α y β los ángulos de la figura 5.1.6. La geometría de la situación muestra que el precio tiende a estabilizarse si y sólo si $\alpha + \beta > 180°$. Esta última condición se cumple si y sólo si $-\tan\beta < \tan\alpha$. Como $b = -\tan\beta$ y $k = \tan\alpha$, concluimos que el precio se estabiliza si y sólo si $b < k$.

37. $A(2,2) = 7$, $A(2,3) = 9$

40. $A(3,n) = 2^{n+3} - 3$

43. Si $m = 0$,

$$A(m, n+1) = A(0, n+1)$$
$$= n + 2 > n + 1$$
$$= A(0,n) = A(m,n).$$

La última desigualdad es consecuencia del ejercicio 41.

44. Utilice los ejercicios 38 y 39.

47. Demostramos la afirmación mediante inducción sobre x. El propio paso inductivo requerirá inducción sobre y.

El ejercicio 44 muestra que la ecuación es válida para $x = 0, 1, 2$ y para toda y.

PASO BASE ($x = 2$). Véase el ejercicio 44.

PASO INDUCTIVO (EL CASO x IMPLICA EL CASO $x+1$).

Supongamos que $x \geq 2$ y que

$$A(x,y) = AO(x, 2, y+3) - 3 \quad \text{para toda } y \geq 0.$$

Debemos mostrar que

$$A(x+1, y) = AO(x+1, 2, y+3) - 3 \quad \text{para toda } y \geq 0.$$

Establecemos esta última ecuación mediante inducción sobre y.

PASO BASE ($y = 0$). Debemos demostrar que

$$A(x+1, 0) = AO(x+1, 2, 3) - 3$$

Ahora,

$AO(x+1, 2, 3) - 3$	
$= AO(x, 2, AO(x+1, 2, 3)) - 3$	por definición
$= AO(x, 2, 4) - 3$	por el ejercicio 46
$= A(x, 1)$	por la hipótesis de inducción sobre x
$= A(x+1, 0)$	por (5.1.11).

PASO INDUCTIVO (EL CASO y IMPLICA EL CASO $y+1$).

Supongamos que

$$A(x+1, y) = AO(x+1, 2, y+3) - 3.$$

Debemos demostrar que

$$A(x+1, y+1) = AO(x+1, 2, y+4) - 3.$$

Ahora,

$AO(x+1, 2, y+4) - 3$	
$= AO(x, 2, AO(x+1, 2, y+3)) - 3$	por definición
$= AO(x, 2, A(x+1, y) + 3) - 3$	por la hipótesis de inducción sobre y
$= A(x, A(x+1, y))$	por la hipótesis de inducción sobre x
$= A(x+1, y+1)$	por (5.1.12).

50. Suponga que tenemos n dólares. Si compramos jugo de naranja el primer día, nos sobran $n-1$ dólares, que se pueden gastar de R_{n-1} formas. De manera análoga, si el primer día compramos leche o cerveza, existen R_{n-2} formas de gastar el dinero restante. Como estos casos son ajenos entre sí, $R_n = R_{n-1} + 2R_{n-2}$.

53. $S_3 = \frac{1}{2}$, $S_4 = \frac{3}{4}$

55. Denotamos una función f de $X = \{1, \ldots, n\}$ a X como $(i_1, i_2, \ldots, i_n)$, lo cual significa que $f(k) = i_k$. El problema entonces es contar el número de formas de elegir $i_1, i_2, \ldots, i_n$ de modo que si i aparece, también lo hagan $1, 2, \ldots, i-1$.

Contaremos el número de tales funciones que tienen exactamente j unos. Tales funciones se pueden construir en dos pasos: Se eligen las posiciones de los j unos; luego se colocan los demás números. Existen $C(n,j)$ formas de colocar los unos. Los números restantes se pueden elegir de modo que si i aparece, entonces también aparecen $1, \ldots, i-1$. Existen F_{n-j} formas de elegir los números restantes, ya que los números restantes se pueden elegir de $\{2, \ldots, n\}$. Así, existen $C(n,j)F_{n-j}$ funciones del tipo deseado y que tienen exactamente j unos. Por lo tanto, el número total de

funciones de X en X que tienen la propiedad de que si i está en el rango de f, entonces también están $1, \ldots, i-1$, es

$$\sum_{j=1}^{n} C(n,j)\,F_{n-j} = \sum_{j=1}^{n} C(n,n-j)\,F_{n-j} = \sum_{j=0}^{n-1} C(n,j)\,F_j.$$

58. Utilice la ecuación (4.5.4) para escribir

$$S(k,n) = \Sigma_{i=1}^{n} S(k-1,i).$$

61. Utilizamos la terminología del ejercicio 74 de la sección 4.2. Elegimos una de las $n+1$ personas, digamos P. Existen $s_{n,j-1}$ formas de que P se siente sola. (Las otras n personas se sientan en las $k-1$ mesas restantes.) A continuación, contamos el número de arreglos en los que P no está sola. Las personas distintas de P se sientan en k mesas. Esto se puede hacer de $s_{n,k}$ formas. Ahora, P se puede sentar a la derecha de alguien de n formas. Así, existen $ns_{n,k}$ arreglos en los que P no está sola. De aquí se sigue la relación de recurrencia.

64. Sea A_n la cantidad al final de n años y sea i la tasa de interés, expresada como un decimal. El análisis posterior al ejemplo 5.1.3 muestra que

$$A_n = (1+i)^n A_0.$$

El valor de n necesario para duplicar la cantidad satisface

$$2A_0 = (1+i)^n A_0 \quad \text{o} \quad 2 = (1+i)^n.$$

Si calculamos el logaritmo natural (logaritmo de base e) de ambos lados de la ecuación, obtenemos

$$\ln 2 = n \ln(1+i).$$

Así,

$$n = \frac{\ln 2}{\ln(1+i)}.$$

Como $\ln 2 = 0.6931472\ldots$ y $\ln(1+i)$ es aproximadamente igual a i para valores pequeños de i, n es aproximadamente igual a $0.69\ldots/i$, lo cual, a su vez, es aproximadamente igual a $70/r$.

66. 1, 3, 2; 2, 3, 1; $E_3 = 2$

69. Contamos el número de permutaciones sube / baja de $1, \ldots, n$ considerando cuántas de ellas tienen n en la segunda, cuarta, ..., posiciones.

Supongamos que n está en la segunda posición. Como cualquiera de los números restantes es menor que n, cualquiera de ellos se puede colocar en la primera posición. Así, podemos elegir el número por colocar en la primera posición de $C(n-1,1)$ formas y, después de elegirlo, podemos ordenarlo de $E_1 = 1$ forma. Las últimas $n-2$ posiciones se pueden llenar de E_{n-2} formas, pues cualquier permutación sube/baja de los $n-2$ números restantes proporcionan una permutación sube/baja de $1, \ldots, n$. Así, el número de permutaciones sube/baja de $1, \ldots, n$ con n en la segunda posición es $C(n-1,1)E_1 E_{n-2}$.

Supongamos que n está en la cuarta posición. Podemos elegir los números por colocar en las primeras tres posiciones de $C(n-1,3)$ formas. Después de elegir los tres elementos, podemos ordenarlos de E_3 formas. Los últimos $n-4$ números se pueden ordenar de E_{n-4} formas. Así, el número de permutaciones sube/baja de $1, \ldots, n$, con n en la cuarta posición es $C(n-1,3)E_3 E_{n-4}$.

En general, el número de permutaciones sube/baja de $1, \ldots, n$, con n en la $(2j)$-ésima posición es

$$C(n-1, 2j-1)E_{2j-1}E_{n-2j}.$$

Al sumar sobre todas las j se obtiene la relación de recurrencia deseada.

Sección 5.2

1. Sí; orden 1

4. No

7. No

10. Sí; orden 3

11. $a_n = 2(-3)^n$

14. $a_n = 2^{n+1} - 4^n$

17. $a_n = (2^{2-n} + 3^n)/5$

20. $a_n = 2(-4)^n + 3^n(-4)^n$

23. $R_n = [(-1)^n + 2^{n+1}]/3$

26. Sea d_n la población de venados en el instante n. La condición inicial es $d_0 = 0$. La relación de recurrencia es

$$d_n = 100n + 1.2_{dn-1}, \quad n > 0.$$

$$\begin{aligned} d_n &= 100n + 1.2_{dn-1} = 100n + 1.2[100(n-1) + 1.2d_{n-2}] \\ &= 100n + 1.2 \cdot 100(n-1) + 1.2^2{}_{dn-2} \\ &= 100n + 1.2 \cdot 100(n-1) \\ &\quad + 1.2^2[100(n-2) + 1.2d_{n-3}] \\ &= 100n + 1.2 \cdot 100(n-1) \\ &\quad + 1.2^2 \cdot 100(n-2) + 1.2^3 d_{n-3} \\ &\vdots \\ &= \sum_{i=0}^{n-1} 1.2^i \cdot 100(n-i) + 1.2^n d_0 \\ &= \sum_{i=0}^{n-1} 1.2^i \cdot 100(n-i) \\ &= 100n \sum_{i=0}^{n-1} 1.2^i - 1.2 \cdot 100 \sum_{i=1}^{n-i} i \cdot 1.2^{i-1} \\ &= \frac{100n(1.2^n - 1)}{1.2-1} \\ &\quad - 120\frac{(n-1)1.2^n - n1.2^{n-1} + 1}{(1.2-1)^2}, \quad n > 0. \end{aligned}$$

29. Hacemos $b_n = a_n/n!$ para obtener $b_n = -2b_{n-1} + 3b_{n-2}$. Al resolver esta ecuación obtenemos $a_n = n!\,b_n = (n!/4)[5 - (-3)^n]$.

32. Establecemos la desigualdad mediante inducción sobre n.

Los casos base $n = 1$ y $n = 2$ se dejan al lector. Ahora supongamos que la desigualdad es verdadera para los valores menores que $n+1$. Entonces

$$\begin{aligned} f_{n+1} &= f_n + f_{n-1} \\ &\geq \left(\frac{1-\sqrt{5}}{2}\right)^{n-1} + \left(\frac{1+\sqrt{5}}{2}\right)^{n-2} \\ &= \left(\frac{1+\sqrt{5}}{2}\right)^{n-2}\left(\frac{1+\sqrt{5}}{2}+1\right) \\ &= \left(\frac{1+\sqrt{5}}{2}\right)^{n-2}\left(\frac{1+\sqrt{5}}{2}\right)^{2} \\ &= \left(\frac{1+\sqrt{5}}{2}\right)^{n}, \end{aligned}$$

y esto concluye el paso inductivo.

34. $a_n = b2^n + d4^n + 1$

37. $a_n = b/2^n + d3^n - (4/3)2^n$

40. El argumento es idéntico al dado en el teorema 5.2.11.

43. El llamado recursivo de este algoritmo para mover los $n - k_n$ discos en la parte superior del poste 1 al poste 2 requiere $T(n-k_n)$ movimientos (véase el ejemplo 5.2.4). El llamado recursivo de este algoritmo para mover los $n-k_n$ discos en la parte superior del poste 2 al poste 4 requiere de nuevo $T(n-k_n)$ movimientos. De aquí se sigue la relación de recurrencia.

46. De la desigualdad

$$\frac{k_n(k_n+1)}{2} \leq n,$$

podemos deducir $k_n \leq \sqrt{2n}$. Como

$$n - k_n \leq \frac{k_n(k_n+1)}{2},$$

esto implica que $r_n \leq k_n$. Por lo tanto,

$$\begin{aligned} T(n) &= (k_n + r_n - 1)2^{kn} + 1 \\ &< 2k_n 2^{kn} + 1 \\ &\leq 2\sqrt{2n}\,2^{\sqrt{2n}} + 1 \\ &= O(4^{\sqrt{n}}). \end{aligned}$$

Sección 5.3

1. En la línea 2, como $i > j$ $(1 > 5)$ es falso, pasamos a la línea 4, donde hacemos k igual a 3. En la línea 5, como *key* ('G') no es igual a s_3 ('J'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('G' < 'J') es verdadera, de modo que en la línea 8 hacemos j igual a 2. Luego llamamos a este algoritmo con $i = 1, j = 2$ para buscar *key* en

$$s_1 = \text{'}C\text{'}, \; s_2 = \text{'}G\text{'}.$$

En la línea 2, como $i > j$ $(1 > 2)$ es falso, pasamos a la línea 4, donde hacemos k igual a 1. En la línea 5, como *key* ('G') no es igual a s_1 ('C'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('G' < 'C') es verdadera, de modo que en la línea 10 hacemos i igual a 2. Luego llamamos a este algoritmo con $i = j = 2$ para buscar *key* en

$$s_2 = \text{'}G\text{'}.$$

En la línea 2, como $i > j$ $(2 > 2)$ es falso, pasamos a la línea 4, donde hacemos k igual a 2. En la línea 5, como *key* ('G') es igual a s_2 ('G'), regresamos 2, el índice de *key* en la sucesión s.

4. En la línea 2, como $i > j$ $(1 > 5)$ es falso, pasamos a la línea 4, donde hacemos k igual a 3. En la línea 5, como *key* ('Z') no es igual a s_3 ('J'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('Z' < 'J') es falsa, de modo que en la línea 10 hacemos i igual a 4. Luego llamamos a este algoritmo con $i = 4, j = 5$ para buscar *key* en

$$s_4 = \text{'}M\text{'}, \; s_5 = \text{'}X\text{'}.$$

En la línea 2, como $i > j$ $(4 > 5)$ es falso, pasamos a la línea 4, donde hacemos k igual a 4. En la línea 5, como *key* ('Z') no es igual a s_4 ('M'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('Z' < 'M') es falsa, de modo que en la línea 10 hacemos i igual a 5. Luego llamamos a este algoritmo con $i = j = 5$ para buscar *key* en

$$s_5 = \text{'}X\text{'}.$$

En la línea 2, como $i > j$ $(5 > 5)$ es falso, pasamos a la línea 4, donde hacemos k igual a 5. En la línea 5, como *key* ('Z') no es igual a s_5 ('X'), pasamos a la línea 7. En la línea 7, *key* $< s_k$ ('Z' < 'X') es verdadera, de modo que en la línea 10 hacemos i igual a 6. Luego llamamos a este algoritmo con $i = 6, j = 5$.

En la línea 2, como $i > j$ $(6 > 5)$ es verdadera, regresamos 0 para indicar que no hemos encontrado *key*.

7. El algoritmo B es mejor si $2 \leq n \leq 15$. (Para $n = 1$ y $n = 16$, los algoritmos requieren la misma cantidad de comparaciones.)

10. Supongamos que las sucesiones son $a_1, \ldots, a_n$ y $b_1, \ldots, b_n$.

(a) $a_1 < b_1 < a_2 < b_2 <$ (b) $a_n < b_1$

...

13. 11

17. El algoritmo 5.3.11 calcula a^n mediante la fórmula $a^n = a^m a^{n-m}$.

18. $b_n = b_{\lfloor n/2 \rfloor} + b_{\lfloor (n+1)/2 \rfloor} + 1, b_1 = 0$

19. $b_2 = 1, b_3 = 2, b_4 = 3$

20. $b_n = n - 1$

21. Demostraremos la fórmula mediante inducción matemática. El paso base, $n = 1$, ya ha sido establecido.

Supongamos que $b_k = k - 1$ para toda $k < n$. Mostraremos que $b_n = n - 1$. Ahora,

$$b_n = b_{\lfloor n/2 \rfloor} + b_{\lfloor (n+1)/2 \rfloor} + 1$$
$$= \left\lfloor \frac{n}{2} \right\rfloor - 1 + \left\lfloor \frac{n+1}{2} \right\rfloor - 1 + 1$$

por la hipótesis de inducción

$$= \left\lfloor \frac{n}{2} \right\rfloor + \left\lfloor \frac{n+1}{2} \right\rfloor - 1 = n - 1.$$

34. Si $n = 1$, entonces $i = j$ y regresamos antes de llegar a la línea 7c, 11 o 15. Por lo tanto, $b_1 = 0$. Si $n = 2$, entonces $j = i + 1$. Existe una comparación en la línea 7c y regresamos antes de llegar a la línea 11 o 15. Por lo tanto, $b_2 = 1$.

35. $b_3 = 3, b_4 = 4$

36. Cuando $n > 2$, se necesitan $b_{\lfloor (n+1)/2 \rfloor}$ comparaciones para la primera llamada recursiva y $b_{\lfloor n/2 \rfloor}$ comparaciones para la segunda llamada recursiva. Se necesitan otras dos comparaciones en las líneas 11 y 15. De aquí se sigue la relación de recurrencia.

37. Supongamos que $n = 2^k$. Entonces (5.3.12) se convierte en

$$b_{2^k} = 2b_{2^{k-1}} + 2.$$

Ahora

$$b_{2^k} = 2b_{2^{k-1}} + 2$$
$$= 2[2b_{2^{k-2}} + 2] + 2$$
$$= 2^2 b_{2^{k-2}} + 2^2 + 2 = \ldots$$
$$= 2^{k-1} b_{2^1} + 2^{k-1} + 2^{k-2} + \cdots + 2$$
$$= 2^{k-1} + 2^{k-1} + \cdots + 2$$
$$= 2^{k-1} + 2^k - 2$$
$$= n - 2 + \frac{n}{2} = \frac{3n}{2} - 2.$$

38. Utilizamos el siguiente hecho, el cual se puede verificar considerando los casos x par y x impar:

$$\left\lceil \frac{3x}{2} - 2 \right\rceil + \left\lceil \frac{3(x+1)}{2} - 2 \right\rceil = 3x - 2 \text{ para } x = 1, 2, \ldots.$$

Sea a_n el número de comparaciones necesarias para el algoritmo en el peor de los casos. Los casos $n = 1$ y $n = 2$ se pueden verificar directamente. (El caso $n = 2$ es el paso base.)

PASO INDUCTIVO. Supongamos que $a_k \le \lceil (3k/2) - 2 \rceil$ para $2 \le k < n$. Debemos mostrar que la desigualdad es válida para $k = n$.

Si n es impar, el algoritmo separa el arreglo en subclases de tamaños $(n-1)/2$ y $(n+1)/2$. Ahora,

$$a_n = a_{(n-1)/2} + a_{(n+1)/2} + 2$$
$$\le \left\lceil \frac{(3/2)(n-1)}{2} - 2 \right\rceil + \left\lceil \frac{(3/2)(n+1)}{2} - 2 \right\rceil + 2$$
$$= \frac{3(n-1)}{2} - 2 + 2 = \frac{3n}{2} - \frac{3}{2}$$
$$= \left\lceil \frac{3n}{2} - 2 \right\rceil.$$

El caso n par se analiza de manera análoga.

47. Si $n = 2^k$,

$$a_{2^k} = 3a_{2^{k-1}} + 2^k,$$

de modo que

$$a_n = a_{2^k} = 3a_{2^{k-1}} + 2^k$$
$$= 3[3a_{2^{k-2}} + 2^{k-1}] + 2^k$$
$$= 3^2 a_{2^{k-2}} + 3 \cdot 2^{k-1} + 2^k$$
$$\vdots$$
$$= 3^k a_{2^0} + 3^{k-1} \cdot 2^1 + 3^{k-2} \cdot 2^2 + \ldots + 3 \cdot 2^{k-1} + 2^k \qquad (*)$$
$$= 3^k + 2(3^k - 2^k)$$
$$= 3 \cdot 3^k - 2 \cdot 2^k$$
$$= 3 \cdot 3^{\lg n} - 2n.$$

La línea $(*)$ es consecuencia de la ecuación

$$(a - b)(a^{k-1}b^0 + a^{k-2}b^1 + \cdots + a^1 b^{k-2} + a^0 b^{k-1})$$
$$= a^k - b^k$$

con $a = 3$ y $b = 2$.

49. $b_n = b_{\lfloor (1+n)/2 \rfloor} + b_{\lfloor n/2 \rfloor} + 3$

52. $b_n = 4n - 3$

55. Mostraremos que $b_n \le b_{n+1}, n = 1, 2, \ldots$ Tenemos la relación de recurrencia

$$b_n = b_{\lfloor (1+n)/2 \rfloor} + b_{\lfloor n/2 \rfloor} + c_{\lfloor (1+n)/2 \rfloor, \lfloor n/2 \rfloor}.$$

PASO BASE. $b_2 = 2b_1 + c_{1,1} \ge 2b_1 \ge b_1$

PASO INDUCTIVO. Supongamos que la afirmación es válida para $k < n$. Si n es par, tenemos $b_n = 2b_{n/2} + c_{n/2, n/2}$, de modo que

$$b_{n+1} = b_{(n+2)/2} + b_{n/2} + c_{(n+2)/2, n/2}$$
$$\ge b_{n/2} + b_{n/2} + c_{n/2, n/2} = b_n.$$

El caso n impar es similar.

57.
```
procedure ex57(s, i, j)
  if i = j then
    return
  m := ⌊(i + j)/2⌋
  call ex57(s, i, m)
  call ex57(s, m + 1, j)
  call combine(s, i, m, j)
end ex57
```

60. Demostramos la desigualdad mediante inducción matemática.

PASO BASE. $a_1 = 0 \le 0 = b_1$

PASO INDUCTIVO. Supongamos que $a_k \le b_k$ para $k < n$. Entonces

$$a_n \le a_{\lfloor n/2 \rfloor} + a_{\lfloor (n+1)/2 \rfloor} + 2 \lg n$$
$$\le b_{\lfloor n/2 \rfloor} + b_{\lfloor (n+1)/2 \rfloor} + 2 \lg n = b_n.$$

63. Sea $c = a_1$. Si n es una potencia de m, digamos, $n = m^k$, entonces

$$a_n = a_{m^k} = a_{m^{k-1}} + d$$
$$= [a_{m^{k-2}} + d] + d$$
$$= a_{m^{k-2}} + 2d$$
$$\vdots$$
$$= a_{m^0} + kd = c + kd.$$

Un valor arbitrario de n está entre dos potencias de m, digamos

$$m^{k-1} < n \le m^k.$$

Esta última desigualdad implica que

$$k - 1 < \log_m n \le k.$$

Como la sucesión a es creciente,

$$a_{m^{k-1}} \le a_n \le a_{m^k}.$$

Ahora,

$$\Omega(\log_m n) = c + (-1 + \log_m n) \le c + (k-1)d$$
$$= a_{m^{k-1}} \le a_n$$

y

$$a_n \le a_{m^k} = c + kd$$
$$\le c + (1 + \log_m n)d = O(\log_m n).$$

Así, $a_n = \Theta(\log_m n)$. Por el ejercicio 29 de la sección 3.5, $a_n = \Theta(\lg n)$.

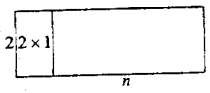
Capítulo 5 Autoevaluación

1. (a) 3, 5, 8, 12 (b) $a_1 = 3$ (c) $a_n = a_{n-1} + n$

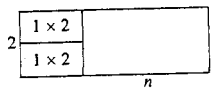
2. $A_n = (1.17)A_{n-1}, A_0 = 4000$

3. Sea X un conjunto con n elementos y sea $x \in X$. Sea k un entero fijo, $0 \le k \le n - 1$. Podemos elegir un subconjunto Y de $X - \{x\}$ con k elementos de $C(n-1, k)$ formas. Una vez hecho esto, podemos separar Y de P_k formas. Esta partición, junto con $X - Y$, es una partición de X. Como todas las particiones de X se pueden generar de esta forma, obtenemos la relación de recurrencia deseada.

4. Si el primer dominó se coloca como muestra la figura, existen a_{n-1} formas de cubrir al tablero $2 \times (n-1)$ sobrante.



Si los primeros dos dominoes se colocan como se muestra, existen a_{n-2} formas de cubrir el tablero $2 \times (n-2)$ sobrante.



Esto implica que $a_n = a_{n-1} + a_{n-2}$.

Por inspección, $a_1 = 1$ y $a_2 = 2$. Como $\{a_n\}$ satisface la misma relación de recurrencia que la sucesión de Fibonacci y $a_1 = f_1$ y $a_2 = f_2$, esto implica que $a_i = f_i$ para $i = 1, 2, \ldots$.

5. Sí

6. $a_n = 2(-2)^n - 4n(-2)^n$

7. $a_n = 3 \cdot 5^n + (-2)^n$

8. Consideremos una cadena de longitud n que contiene un número par de unos y que comienza con 0. La cadena posterior al 0 puede ser cualquier cadena de longitud $n - 1$ que contenga un número par de unos, y existen c_{n-1} de tales cadenas. Una cadena de longitud n que contiene un número par de unos y que comienza con 2 puede ir seguida de cualquier cadena de longitud $n - 1$ que contenga un número par de unos, y existen c_{n-1} de tales cadenas. Una cadena de longitud n que contiene un número par de unos y que comienza con 1 puede ir seguida de una cadena arbitraria de longitud $n - 1$ que contiene un número impar de unos. Como existen en total 3^{n-1} cadenas de longitud $n - 1$ y c_{n-1} de ellas contienen un número par de unos, existen $3^{n-1} - c_{n-1}$ cadenas de longitud $n - 1$ que contienen un número impar de unos. Esto implica que

$$c_n = 2c_{n-1} + 3^{n-1} - c_{n-1} = c_{n-1} + 3^{n-1}.$$

Una condición inicial es $c_1 = 2$, pues existen dos cadenas (0 y 2) que contienen un número par (a saber, cero) de unos.

Podemos resolver la relación de recurrencia mediante iteración:

$$c_n = c_{n-1} + 3^{n-1} = c_{n-2} + 3^{n-2} + 3^{n-1}$$
$$\vdots$$
$$= c_1 + 3^1 + 3^2 + \cdots + 3^{n-1}$$
$$= 2 + \frac{3^n - 3}{3 - 1} = \frac{3^n + 1}{2}.$$

9. $b_n = b_{n-1} + 1,\ b_0 = 0$

10. $b_1 = 1,\ b_2 = 2,\ b_3 = 3$

11. $b_n = n$

12. $n(n+1)/2 = O(n^2)$. El algoritmo dado es más rápido que la técnica directa, por lo que es preferible su uso.

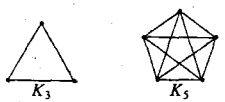
Sección 6.1

1. Como un número impar de aristas toca a algunos vértices (c y d), no existe un camino de a a a que pase por cada arista exactamente una vez.

4. $(a, c, e, b, c, d, e, f, d, b, a)$

7. $V = \{v_1, v_2, v_3, v_4\}$. $E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$. e_1 y e_6 son aristas paralelas. e_5 es un lazo. No existen vértices aislados. G no es una gráfica simple. e_1 es incidente en v_1 y v_2.

10.

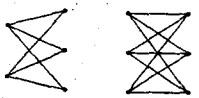


13. Bipartita. $V_1 = \{v_1, v_2, v_5\}, V_2 = \{v_3, v_4\}$.

16. No bipartita

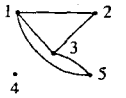
19. Bipartita. $V_1 = \{v_1\}, V_2 = \{v_2, v_3\}$.

20.



22. (b, c, a, d, e)

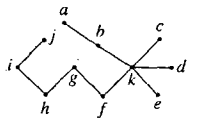
25. Dos clases



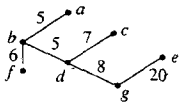
30. 00 01 10 11

30. n

36.



39.



Sección 6.2

1. Ciclo, ciclo simple

4. Ciclo, ciclo simple

7. Camino simple

10.

13.

16. Supongamos que existe tal gráfica, con vértices a, b, c, d, e, f. Supongamos que los grados de a y b son 5. Como la gráfica es simple, cada uno de los grados de c, d, e y f son por lo menos 2; así, no existe tal gráfica.

19. $(a, a), (b, c, g, b), (b, c, d, f g, b),$
$(b, c, d, e, f, g, b), (c, g, f, d, c),$
$(c, g, f, e, d, c), (d, f, e, d)$

22. Cada vértice tiene grado 4.

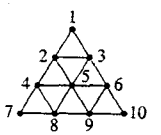
24. Existen cuatro subgráficas: el vértice v_1, sin aristas; el vértice v_2, sin aristas; los vértices v_1 y v_2, sin aristas y la gráfica original.

27. Existen 17 subgráficas.

28. No tiene ciclos de Euler

31. No tiene ciclos de Euler

34. Para



un ciclo de Euler es (10, 9, 6, 5, 9, 8, 5, 4, 8, 7, 4, 2, 5, 3, 2, 1, 3, 6, 10). El método se generaliza.

37. $m = n = 2$ o $m = n = 1$

39. d y e son los únicos vértices de grado impar.

42. El argumento es similar al de la demostración del teorema 6.2.23.

45. Verdadera. En el camino, para todas las a repetidas,

$$(\ldots, a, \ldots, b, a, \ldots)$$

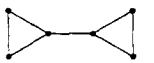
se elimina $a, \ldots, b$.

47. Supongamos que $e = (v, w)$ está en un ciclo. Entonces existe un camino P de v a w que no incluye a e. Sean x y y vértices en $G - \{e\}$. Como G es conexa, existe un camino P' en G de v a w. Reemplazamos cada ocurrencia de e en P' por P. El camino resultante de v a w está en $G - \{e\}$. Por lo tanto, $G - \{e\}$ es conexa.

50. La unión de todas las subgráficas conexas que contienen a G' es un componente.

53. Sea G una gráfica disconexa, simple, con n vértices, con el máximo número de aristas. Muestre que G tiene dos componentes. Si un componente tiene i vértices, muestre que los componentes son K_i y K_{n-i}. Utilice el ejercicio 11 de la sección 6.1, para determinar una fórmula para el número de aristas en G como función de i. Muestre que el máximo ocurre cuando $i = 1$.

55.



58. Modifique las demostraciones de los teoremas 6.2.17 y 6.2.18.

61. Utilice los ejercicios 58 y 60.

64. Primero contamos el número de caminos

$$(v_0, v_1, \ldots, v_k)$$

de longitud $k \geq 1$. El primer vértice v_0 se puede elegir de n formas. Cada vértice posterior se puede elegir de $n - 1$ formas (pues debe ser distinto de su predecesor). Así, el número de caminos de longitud k es $n(n-1)^k$.

El número de caminos de longitud k, $1 \leq k \leq n$, es

$$\sum_{k=1}^{n} n(n-1)^k = n(n-1)\frac{(n-1)^k - 1}{(n-1) - 1}$$
$$= \frac{n(n-1)\left[(n-1)^k - 1\right]}{n-2}.$$

68. Si v es un vértice en V, el camino formado por v y ninguna arista es un camino de v a v; así, vRv para cada vértice v en V. Por lo tanto, R es reflexiva.

Supongamos que vRw. Entonces existe un camino $(v_0, \ldots, v_n)$, donde $v_0 = v$ y $v_n = w$. Ahora, $(v_n, \ldots, v_0)$ es un camino de w a v, por lo que wRv. Por lo tanto, R es simétrica.

Supongamos que vRw y wRx. Entonces existe un camino P_1 de v a w y un camino P_2 de w a x. Ahora, P_1 seguido de P_2 es un camino de v a x, de modo que vRx. Por lo tanto, R es transitiva.

Como R es reflexiva, simétrica y transitiva en V, R es una relación de equivalencia en V.

70. 2

73. Sea s_n el número de caminos de longitud n de v_1 a v_1. Mostraremos que las sucesiones $s_1, s_2, \ldots$ y $f_1, f_2, \ldots$ satisfacen la misma relación de recurrencia y las mismas condiciones iniciales, lo cual implicaría que $s_n = f_n$ para $n \geq 1$.

Si $n = 1$, existe un camino de longitud 1 de v_1 a v_1, a saber, el lazo en v_1; así, $s_1 = f_1$.

Si $n = 2$, existen dos caminos de longitud 2 de v_1 a v_1: (v_1, v_1, v_1) y (v_1, v_2, v_1); así, $s_2 = f_2$.

Supongamos que $n > 2$. Consideremos un camino de longitud n de v_1 a v_1. El camino debe comenzar con el lazo (v_1, v_1) o la arista (v_1, v_2).

Si el camino comienza con el lazo, el resto del camino debe ser un camino de longitud $n - 1$ de v_1 a v_1. Como existen s_{n-1} de tales caminos, existen s_{n-1} caminos de longitud n de v_1 a v_1 que comienzan con $(v_1, v_1, \ldots)$.

Si el camino comienza con la arista (v_1, v_2), la siguiente arista en el camino debe ser (v_2, v_1). El resto del camino debe ser un camino de longitud $n - 2$ de v_1 a v_1. Como existen s_{n-2} de tales caminos, existen s_{n-2} caminos de longitud n de v_1 a v_1 que comienzan $(v_1, v_2, v_1, \ldots)$.

Como cualquier camino de longitud $n > 2$ de v_1 a v_1 comienza con el lazo (v_1, v_1) o la arista (v_1, v_2), esto implica que

$$s_n = s_{n-1} + s_{n-2}.$$

Como ahora las sucesiones $s_1, s_2, \ldots$ y $f_1, f_2, \ldots$ satisfacen la misma relación de recurrencia y las mismas condiciones iniciales, esto implica que $s_n = f_n$ para $n \geq 1$.

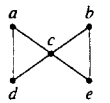
75. Supongamos que cada vértice tiene una arista de salida. Elegimos un vértice v_0. Seguimos una arista de salida de v_0 a un vértice v_1. (Por hipótesis, tal arista existe.) Continuamos siguiendo una arista de salida de v_i a un vértice v_{i+1}. Como existe un número finito de vértices, en algún momento regresaremos a un vértice ya visitado con anterioridad. En este punto, habremos descubierto un ciclo, lo cual es una contradicción. Por lo tanto, una gráfica acíclica dirigida tiene al menos un vértice sin aristas de salida.

Sección 6.3

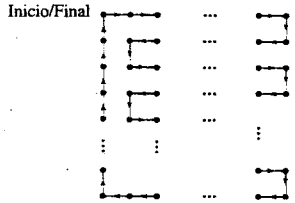
1. $(d, a, e, b, c, h, g, f, j, i, d)$

3. Tendríamos que eliminar dos aristas, cada una en b, d, i y k, lo cual deja $19 - 8 = 11$ aristas. Un ciclo hamiltoniano tendría 12 aristas.

6. $(a, b, c, j, i, m, k, d, e, f, l, g, h, a)$

9. 

2. Si n es par y $m > 1$ o si m es par y $n > 1$, existe un ciclo hamiltoniano. El diagrama muestra la solución en el caso n par.



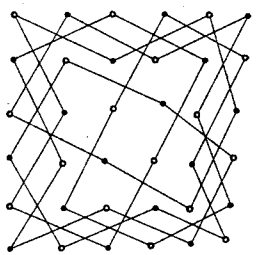
Si $n = 1$ o si $m = 1$, no existe un ciclo, y en particular, no existe un ciclo hamiltoniano. Supongamos que n y m son ambos impares y que la gráfica tiene un ciclo hamiltoniano. Como existen nm vértices, este ciclo tiene nm aristas; por lo tanto, el ciclo hamiltoniano contiene un número impar de aristas. Sin embargo, observemos que en un ciclo hamiltoniano deben aparecer tantas aristas "hacia arriba" como aristas "hacia abajo" y tantas aristas "hacia la izquierda" como aristas "hacia la derecha". Así, un ciclo hamiltoniano debe tener un número par de aristas. Esta contradicción muestra que si n y m son ambos impares, la gráfica no tiene un ciclo hamiltoniano.

5. Cuando $m = n$ y $n > 1$

8. Cualquier ciclo C del n-cubo tiene longitud par, pues los vértices en C alternan entre un número par y un número impar de unos.

Supongamos que el n-cubo tiene un ciclo simple de longitud m. Acabamos de observar que m es par. Ahora, $m > 0$ por definición. Como el n-cubo es una gráfica simple, $m \neq 2$. Por lo tanto, $m \geq 4$.

Ahora, supongamos que $m \geq 4$ y m es par. Sean G los primeros $m/2$ miembros del código de Gray G_{n-1}. Entonces $0G$, $1G^R$ describe un ciclo simple de longitud m en el n-cubo.



Sección 6.4

1. 7; (a, b, c, f) **4.** 7; (b, c, f, j)

6. El ejemplo 6.4.2 permite modelar un algoritmo.

9. Modifique el algoritmo 6.4.1 de modo que comience asignando el peso ∞ a cada arista inexistente. Luego, el algoritmo continúa como está escrito. Al terminar, $L(z)$ será igual a ∞ si no existe un camino de a a z.

Sección 6.5

1.

	a	b	c	d	e
a	0	1	1	1	1
b	1	0	1	0	0
c	1	1	0	1	1
d	1	0	1	0	1
e	1	0	1	1	0

4.

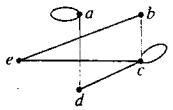
	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	1	1	0	0	0
v_2	1	0	1	0	0	0
v_3	1	1	0	0	0	0
v_4	0	0	0	0	0	0
v_5	0	0	0	0	0	1
v_6	0	0	0	0	1	0

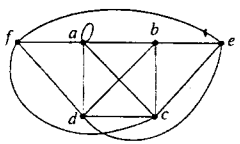
7.

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
a	1	0	1	0	1	1	0	0
b	1	1	0	0	0	0	0	0
c	0	1	0	1	1	0	1	0
d	0	0	0	1	0	1	0	1
e	0	0	1	0	0	0	1	1

10.

	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8
1	1	0	0	0	0	0	0	0
2	1	1	0	1	1	1	0	0
3	0	1	1	0	0	0	0	0
4	0	0	1	1	0	0	0	0
5	0	0	0	0	1	0	1	0
6	0	0	0	0	0	1	1	1
7	0	0	0	0	0	0	0	1

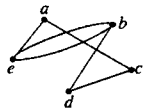
13. 

16. 

19. [Para K_5]

$$\begin{pmatrix} 4 & 3 & 3 & 3 & 3 \\ 3 & 4 & 3 & 3 & 3 \\ 3 & 3 & 4 & 3 & 3 \\ 3 & 3 & 3 & 4 & 3 \\ 3 & 3 & 3 & 3 & 4 \end{pmatrix}$$

22. La gráfica no es conexa.

25. 

28. G no es conexa.

29. Debido a la simetría de la gráfica, si v y w son vértices en K_5, existe el mismo número de caminos de longitud n de v a v como caminos de longitud n de w a w. Así, todos los elementos de la diagonal de A^n son iguales. De manera análoga, todos los elementos fuera de la diagonal de A^n son iguales.

32. Si $n \geq 2$,

$$d_n = 4a_{n-1} \qquad \text{por el ejercicio 30}$$

$$= 4\left(\frac{1}{5}\right)[4^{n-1} + (-1)^n] \qquad \text{por el ejercicio 31.}$$

La fórmula se puede verificar de manera directa para $n = 1$.

Sección 6.6

1. Las gráficas no son isomorfas, pues no tienen el mismo número de vértices.

4. Las gráficas no son isomorfas, pues G_2 tiene un vértice (c') de grado 4, pero G_1 no.

7. Las gráficas no son isomorfas, pues G_1 tiene un vértice de grado 2, pero G_2 no.

10. Las gráficas son isomorfas: $f(a) = e'$, $f(b) = c'$, $f(c) = a'$, $f(d) = g'$, $f(e) = d'$, $f(f) = f'$, $f(g) = b'$, $f(h) = h'$. Definimos

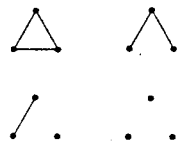
$$g((x, y)) = (f(x), f(y)).$$

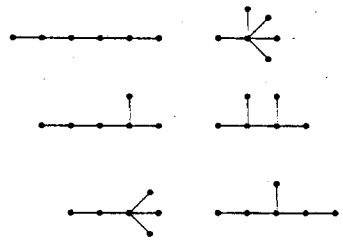
En los ejercicios 12–18, utilizamos la notación de la definición 6.6.1.

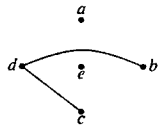
12. Si $(v_0, v_1, \ldots, v_k)$ es un ciclo simple de longitud k en G_1, entonces $(f(v_0), f(v_1), \ldots, f(v_k))$ es un ciclo simple de longitud k en G_2. [Los vértices $f(v_i)$, $i = 1, \ldots, k-1$, son distintos, pues f es uno a uno.]

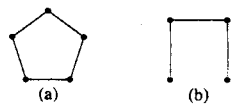
15. En la sugerencia del ejercicio 12, mostramos que si $C = (v_0, v_1, \ldots, v_k)$ es un ciclo simple de longitud k en G_1, entonces $(f(v_0), f(v_1), \ldots, f(v_k))$, que denotamos como $f(C)$, es un ciclo simple de longitud k en G_2. Sean $C_1, C_2, \ldots, C_n$ los n ciclos simples de longitud k en G_1. Entonces $f(C_1)$, $f(C_2), \ldots, f(C_n)$ son n ciclos simples de longitud k en G_2. Además, como f es uno a uno, $f(C_1), f(C_2), \ldots, f(C_n)$ son distintos.

18. La propiedad es un invariante. Si $(v_0, v_1, \ldots, v_n)$ es un ciclo de Euler en G_1, entonces, como g es sobre, $(f(v_0), f(v_1), \ldots, f(v_n))$ es un ciclo de Euler en G_2.

21. 

24. 

26. 

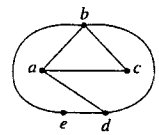
29. 

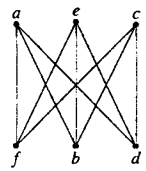
32. Definimos $g((v, w)) = (f(v), f(w))$.

33. $f(a) = a'$, $f(b) = b'$, $f(c) = c'$, $f(d) = b'$

36. $f(a) = a'$, $f(b) = b'$, $f(c) = c'$, $f(d) = a'$,

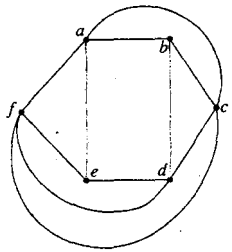
Sección 6.7

1. 

4. 

es $K_{3,3}$

6. Plana

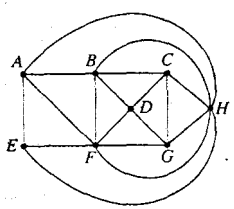


9. $2e = 2 + 2 + 2 + 3 + 3 + 3 + 4 + 4 + 5$, de modo que $e = 14$. $f = e - v + 2 = 14 - 9 + 2 = 7$

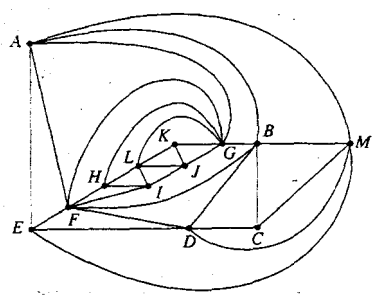
12. Una gráfica con cinco o menos vértices y un vértice de grado 2 es homeomorfa a una gráfica con cuatro o menos vértices. Tal gráfica no puede contener una copia homeomorfa de $K_{3,3}$ o de K_5.

15. Si K_5 es plana, $e \leq 3v - 6$ se convierte en $10 \leq 3 \cdot 5 - 6 = 9$.

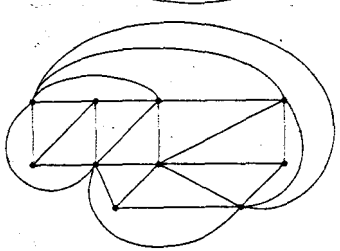
18.



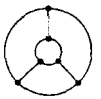
22.



25.



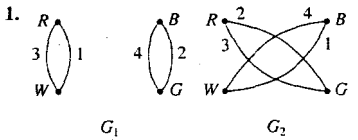
28. Contiene a



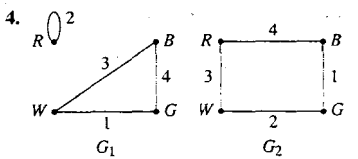
31. Suponga que G no tiene un vértice de grado 5. Muestre que $2e \geq 6v$. Ahora utilice el ejercicio 13 para obtener una contradicción.

Sección 6.8

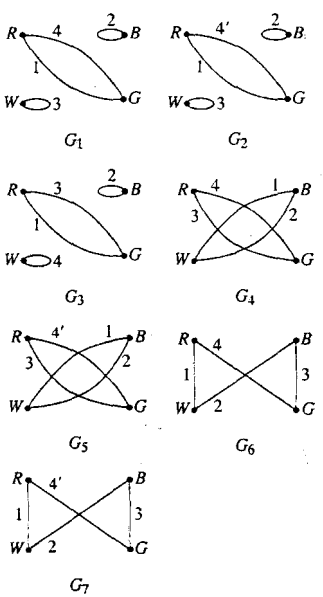
1.



4.



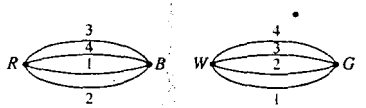
7. (a)



(b) Las soluciones son: G_1, G_5; G_1, G_7; G_2, G_4; G_2, G_6; G_3, G_6; y G_3, G_7.

13. Una arista se puede elegir de $C(2 + 4 - 1, 2) = 10$ formas. Las tres aristas etiquetadas con 1 se pueden elegir de $C(3 + 10 - 1, 3) = 220$ formas. Así, el número total de gráficas es 220^4.

15.



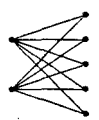
19. De acuerdo con el ejercicio 14, sin contar los lazos, cada vértice debe tener grado al menos 4. En la figura 6.8.5, sin contar los lazos, el vértice W tiene grado 3 y, por lo tanto, la figura 6.8.5 no tiene una solución a la versión modificada de Locura Instantánea. La figura 6.8.3 da una solución del juego regular de Locura Instantánea para la figura 6.8.5.

Capítulo 6 Autoevaluación

1. $V = \{v_1, v_2, v_3, v_4\}$. $E = \{e_1, e_2, e_3\}$. e_1 y e_2 son aristas paralelas. No hay lazos. v_1 es un vértice aislado. G no es una gráfica simple, e_3 es incidente en v_2 y v_4. v_2 es incidente en e_1, e_2 y e_3.

2. Existen vértices (a y e) de grado impar.

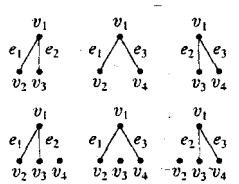
3.



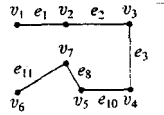
4. Si V_1 denota el conjunto de vértices que contienen un número par de unos y V_2 el conjunto de vértices que contienen un número impar de unos, cada arista es incidente en un vértice de V_1 y un vértice de V_2. Por lo tanto, el n-cubo es bipartita.

5. Es un ciclo.

6.



7.



8. No. Existen vértices de grado impar.

9. $(v_1, v_2, v_3, v_4, v_5, v_7, v_6, v_1)$

10. (000, 001, 011, 010, 110, 111, 101, 100, 000)

11. Un ciclo hamiltoniano tendría siete aristas. Supongamos que la gráfica tiene un ciclo hamiltoniano. Tendríamos que eliminar tres aristas en el vértice b y una arista en el vértice f. Esto deja $10 - 4 = 6$ aristas, que no son suficientes para un ciclo hamiltoniano. Por lo tanto, la gráfica no tiene un ciclo hamiltoniano.

12. En un ciclo hamiltoniano de peso mínimo, cada vértice debe tener grado 2. Por lo tanto, hay que incluir las aristas (a, b), (a, j), (j, i), (i, h), (g, f), (f, e) y (e, d). No podemos incluir la arista (b, h), pues entonces se formaría un ciclo. Esto implica que debemos incluir las aristas (h, g) y (b, c). Como el vértice g tiene ahora grado 2, no podemos incluir las aristas (c, g) ni (g, d). Así, debemos incluir (c, d). Éste es un ciclo hamiltoniano, y el argumento muestra que es único. Por lo tanto, es mínimo.

13. 9 **14.** 11 **15.** (a, e, f, i, g, z) **16.** 12

17.

	v_1	v_2	v_3	v_4	v_5	v_6	v_7
v_1	0	1	0	0	0	1	0
v_2	1	0	1	1	0	1	1
v_3	0	1	0	1	0	0	0
v_4	0	1	1	0	1	0	0
v_5	0	0	0	1	0	1	1
v_6	1	1	0	0	1	0	1
v_7	0	1	0	0	1	1	0

18.

	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	e_9	e_{10}	e_{11}
v_1	1	0	0	0	0	0	1	0	0	0	0
v_2	1	1	0	1	1	1	0	0	0	0	0
v_3	0	1	1	0	0	0	0	0	0	0	0
v_4	0	0	1	1	0	0	0	0	0	1	0
v_5	0	0	0	0	0	0	0	1	1	1	0
v_6	0	0	0	0	0	1	1	0	1	0	1
v_7	0	0	0	0	1	0	0	1	0	0	1

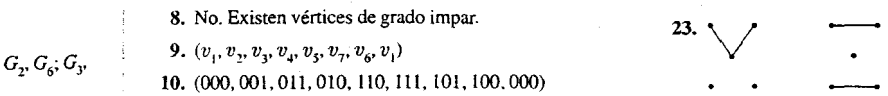
19. El número de caminos de longitud 3 de v_2 en v_3.

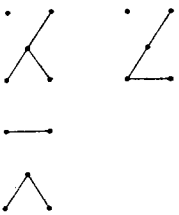
20. No. Cada arista es incidente en al menos un vértice.

21. Las gráficas son isomorfas. Los órdenes v_1, v_2, v_3, v_4, v_5 y w_3, w_1, w_4, w_2, w_5 producen matrices de adyacencia iguales.

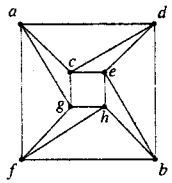
22. Las gráficas son isomorfas. Los órdenes $v_1, v_2, v_3, v_4, v_5, v_6$ y $w_3, w_6, w_2, w_5, w_1, w_4$ producen matrices de adyacencia iguales.

23.

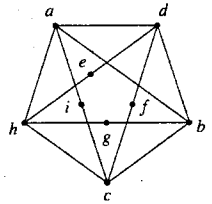




La gráfica es plana;

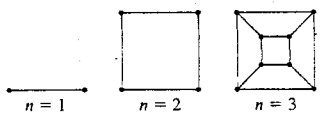


La gráfica no es plana; la siguiente subgráfica es homeomorfa a K_5:



Una gráfica simple, plana, conexa, con e aristas y v vértices satisface $e \leq 3v - 6$ (véase el ejercicio 13 de la sección 6.7). Si $e = 31$ y $v = 12$, la desigualdad no se satisface, de modo que la gráfica no puede ser plana.

Para $n = 1, 2, 3$, es posible trazar el n-cubo en el plano sin que se crucen sus aristas:



Argumentamos por contradicción para mostrar que el 4-cubo no es plano. Supongamos que el 4-cubo es plano. Como cada ciclo tiene al menos cuatro aristas, cada cara está acotada por al menos cuatro aristas. Así, el número de aristas que acotan caras es al menos $4f$. En una gráfica plana, cada arista pertenece a lo más a dos ciclos frontera. Por lo tanto, $2e \geq 4f$. Al utilizar la fórmula de Euler para gráficas, tenemos que

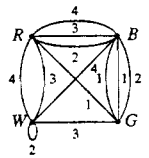
$$2e \geq 4(e - v + 2).$$

Para el 4-cubo, tenemos que $e = 32$ y $v = 16$, de modo que la fórmula de Euler queda

$$64 = 2 \cdot 32 \geq 4(32 - 16 + 2) = 72,$$

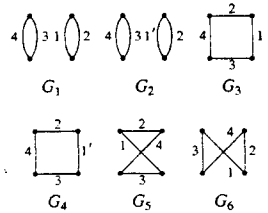
lo cual es una contradicción. Por lo tanto, el 4-cubo no es plano. El n-cubo, para $n > 4$, no es plano, pues contiene al 4-cubo.

29.



30. Véanse las sugerencias para los ejercicios 31 y 32.

31.



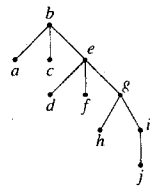
Denotamos las dos aristas incidentes en B y G etiquetadas con 1 en la gráfica del ejercicio 29 como 1 y 1′, aquí.

32. El juego del ejercicio 29 tiene cuatro soluciones. Al utilizar la notación del ejercicio 31, las soluciones son G_1, G_5; G_2, G_5; G_3, G_6; y G_4, G_6.

Sección 7.1

1. a-1; b-1; c-1; d-1; e-2; f-3; g-3; h-4; i-2; j-3; k-0

4. Altura = 4



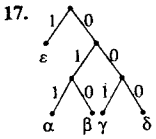
7. PEN

10. SALAD

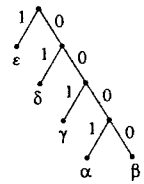
11. 0111100010

14. 0110000100100001111

17.



20. Otro árbol aparece en la sugerencia del ejercicio 17.



24. Sea T un árbol. Elegimos como raíz de T algún vértice arbitrario. Sea V el conjunto de vértices en los niveles pares y W el conjunto de vértices en los niveles impares. Como cada arista es incidente en un vértice de V y un vértice de W, T es una gráfica bipartita.

27. e, g

30. El radio es la excentricidad de un centro. No necesariamente ocurre que $2r = d$ (véase la figura 7.1.5).

Sección 7.2

1. Cronos

4. Apolo, Atenea, Hermes, Hércules

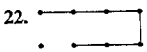
7. b; d

10. e, f, g, j; j

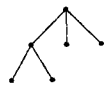
13. a, b, c, d, e

17. Son hermanos.

22.



25.



27. Un único vértice es un "ciclo" de longitud 0.

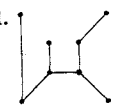
30. Cada componente de un bosque es conexo y acíclico y, por lo tanto, es un árbol.

33. Supongamos que G es conexa. Agregamos aristas paralelas hasta que la gráfica resultante G^* tenga $n - 1$ aristas. Como G^* es conexa y tiene $n - 1$ aristas, por el teorema 7.2.3, G^* es acíclica. Pero al agregar una arista paralela se forma un ciclo, lo cual es una contradicción.

36.

Sección 7.3

1.



4. El camino (h, f, e, g, b, d, c, a)

7.

10. Es claro que el problema de las dos reinas no tiene solución. Para el problema de las tres reinas, por simetría, las únicas posiciones posibles en la primera columna son arriba a la izquierda y en el segundo renglón de arriba hacia abajo. Si el primer movimiento es en la primera columna, arriba a la izquierda, el segundo movimiento debe ser a la parte inferior de la segunda columna. Ahora, ya no existe un movimiento posible para la tercera columna. Si el primer movimiento es al segundo renglón de la primera columna, no existe un movimiento posible en la segunda columna. Por lo tanto, no existe solución al problema de las tres reinas.

13. Falso. Considere K_4.

16. En primer lugar, muestre que la gráfica T construida es un árbol. Luego utilice inducción sobre el nivel de T para mostrar que T contiene a todos los vértices de G.

19. Supongamos que x es incidente en los vértices a y b. Al eliminar x de T se obtiene una gráfica disconexa, con dos componentes U y V. Los vértices a y b pertenecen a componentes distintos, digamos, $a \in U$ y $b \in V$. Existe un camino P de a a b en T'. Al movernos por P, en verdadero punto encontramos una arista $y = (v, w)$ con $v \in U$, $w \in V$. Como el hecho de agregar y a $T - \{x\}$ produce una gráfica conexa, $(T - \{x\}) \cup \{y\}$ es un árbol de expansión. Es claro que $(T' - \{y\}) \cup \{x\}$ es un árbol de expansión.

22. Supongamos que T tiene n vértices. Si se agrega una arista a T, la gráfica resultante T' es conexa. Si T' fuese acíclica, T' sería un árbol con n aristas y n vértices. Así, T' contiene un ciclo. Si T' tuviera dos o más ciclos, podríamos obtener una gráfica conexa T'' eliminando dos o más aristas de T'. Pero entonces T'' sería un árbol con n vértices y menos de $n - 1$ aristas, lo cual es imposible.

23.

	e_1	e_2	e_6	e_5	e_3	e_4	e_7	e_8
$(abca)$	1	0	0	0	1	1	0	0
$(acda)$	0	1	0	0	1	0	0	1
$(acdb)$	0	0	1	0	0	1	0	1
$(bcdeb)$	0	0	0	1	0	1	1	1

26. Entrada: Una gráfica $G = (V, E)$ con n vértices

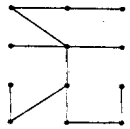
Salida: **true**, si G es conexa
false, si G no es conexa

procedure *is_connected*(V, E)
$T := bfs(V, E)$
// $T = (V', E')$
// es el árbol de expansión regresado por *bfs*
if $|V'| = n$ **then**

```
    return(true)
  else
    return(false)
end is_connected
```

Sección 7.4

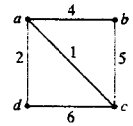
1.

4. 

10. Si v es el primer vértice examinado por el algoritmo de Prim, la arista estará en el árbol de expansión mínimo construido por el algoritmo.

13. Supongamos que G tiene dos árboles de expansión mínimales T_1 y T_2. Entonces existe una arista x en T_1 que no está en T_2. Por el ejercicio 19 de la sección 7.3, existe una arista y en T_2 que no está en T_1 y tal que $T_3 = (T_1 - \{x\}) \cup \{y\}$ y $T_4 = (T_2 - \{y\}) \cup \{x\}$ son árboles de expansión. Como x y y tienen pesos distintos, entonces T_3 o T_4 tienen peso menor que T_1, lo cual es una contradicción.

14. Falso



16. Falso. Considere K_3, con el peso de cada arista igual a 1.

20. Entrada: Las aristas E de una gráfica conexa, con pesos, con n vértices. Si e es una arista, $w(e)$ es igual al peso de e; si e no es una arista, $w(e) = \infty$ (un valor mayor que cualquier peso real).

Salida: Un árbol de expansión mínimo.

```
procedure kruskal(E, w, n)
  V' := Ø
  E' := Ø
  T' := (V', E')
  while |E'| < n - 1 do
    begin
    elegir una arista e = (v_i, v_j) de peso mínimo
      entre todas las aristas tales que al agregarse a T'
      no formen un ciclo
    E' := E' ∪ {e}
    V' := V' ∪ {v_i, v_j}
    T' := (V', E')
    end
  return(T')
end kruskal
```

23. Terminar el algoritmo de Kruskal después de k iteraciones. Esto agrupa los datos en $n - k$ clases.

27. Mostraremos que $a_1 = 7$ y $a_2 = 3$ proporciona una solución. Utilizamos inducción sobre n para mostrar que la solución codiciosa proporciona una solución óptima para $n \geq 1$. Los casos $n = 1, 2, \ldots, 8$ se pueden verificar de manera directa.

Primero mostraremos que si $n \geq 9$, existe una solución óptima que contiene al menos un 7. Sea S' una solución óptima. Supongamos que S' no contiene sietes. Como S' contiene a lo más dos unos (pues S' es óptima), S' contiene al menos tres 3. Reemplazamos tres 3 por un 7 y un 1 para obtener una solución S. Como $|S| = |S'|$, S es óptima.

Si eliminamos un 7 de S, obtenemos una solución S^* al problema $(n - 7)$. Si S^* no fuese óptima, S no podría ser óptima. Así, S^* es óptima. Por la hipótesis de inducción, la solución codiciosa GS^* al problema $(n - 7)$ es óptima, de modo que $|S^*| = |GS^*|$. Observe que 7 junto con GS^* es la solución codiciosa al problema n. Como $|GS| = |S|$, GS es óptima.

30. Mostraremos que si el algoritmo codicioso es óptimo para $n = 1, 2, \ldots, k$, donde

$$k = \sum_{i=1}^{m-1} \left(\frac{a_{i+1}}{\operatorname{mcd}(a_{i+1}, a_i)} - 1 \right) a_i,$$

entonces el algoritmo codicioso es óptimo para toda n con denominaciones

$$1 = a_1 < a_2 < \cdots < a_m.$$

El recíproco es directo.

Utilizamos inducción sobre n para demostrar que el algoritmo codicioso es óptimo para n. Los pasos base son $n = 1, \ldots, k$, los cuales se satisfacen por hipótesis.

Supongamos que $n > k$. Sea S una solución óptima para n, y sea G_n la solución codiciosa para n. Además, sean $|S|$ y $|G_n|$ los números de estampillas en estas soluciones. Afirmamos que S contiene al menos una estampilla de denominación a_m. Supongamos, por contradicción, que S no contiene una estampilla de denominación a_m. Ahora, S contiene a lo más

$$\frac{a_{i+1}}{\operatorname{mcd}(a_{i+1}, a_i)} - 1$$

estampillas de denominación a_i para $i = 1, \ldots, m - 1$. En caso contrario, podríamos reemplazar

$$\frac{a_{i+1}}{\operatorname{mcd}(a_{i+1}, a_i)}$$

estampillas de denominación a_i por

$$\frac{a_i}{\operatorname{mcd}(a_{i+1}, a_i)}$$

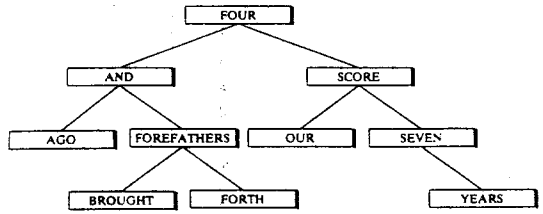
estampillas de denominación a_{i+1} y con ello reducir el número de estampillas en S. Como S es una solución óptima, esto es imposible. Por lo tanto, S contiene a lo más

$$\frac{a_{i+1}}{\operatorname{mcd}(a_{i+1}, a_i)} - 1$$

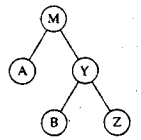
estampillas de denominación a_i para $i = 1, \ldots, m - 1$. Esto implica que podemos pagar a lo más una tarifa de k centavos. Como $n > k$, esto es una contradicción. Por lo tanto, S contiene al menos una estampilla de denominación a_m.

Sea S' igual a S, con una estampilla de denominación a_m eliminada. Entonces S' es óptima para una tarifa de $n - a_m$ centavos. Por la hipótesis de inducción, el algoritmo codicioso es óptimo para una tarifa de $n - a_m$ centavos. Por lo tanto, $|G_{n-a_m}| = |S'|$. Esto implica que $|G_n| = |S|$, con lo que concluye la demostración.

Sección 7.5



4. Falso. Considere



5.

8. $mi + 1, (m - 1)i + 1$

11. $t - 1$

14. Balanceado

17. Balanceado

18. Un árbol de altura 0 tiene un vértice, de modo que $N_0 = 1$. En un árbol binario balanceado de altura 1, la raíz debe tener al menos un hijo. Si la raíz tiene exactamente un hijo, el número de vértices estará minimizado. Por lo tanto, $N_1 = 2$. En un árbol binario balanceado de altura 2, debe existir un camino de la raíz a un vértice terminal de longitud 2. Por esto, al menos existen tres vértices. Pero para que el árbol esté balanceado, la raíz debe tener dos hijos. Por lo tanto, $N_2 = 4$.

21. Supongamos que existen n vértices en un árbol binario balanceado de altura h. Entonces

$$n \geq N_h = f_{h+2} - 1 > \left(\frac{3}{2}\right)^{h+2} - 1,$$

para $h \geq 3$. La igualdad proviene del ejercicio 20 y la última desigualdad del ejercicio 20 de la sección 3.4. Por lo tanto,

$$n + 1 > \left(\frac{3}{2}\right)^{h+2}.$$

Al obtener el logaritmo de base $\frac{3}{2}$ de cada lado, obtenemos

$$\log_{3/2}(n + 1) > h + 2.$$

Por lo tanto,

$$h < [\log_{3/2}(n + 1)] - 2 = O(\lg n).$$

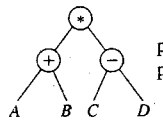
Sección 7.6

1.

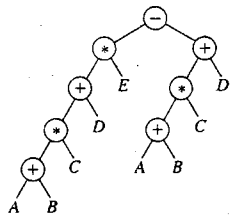
preorden	*entreorden*	*posorden*
ABDCE	*BDAEC*	*DBECA*

4.

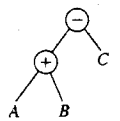
preorden	*entreorden*	*posorden*
ABCDE	*EDCBA*	*EDCBA*

6. 

prefija: $* + AB - CD$
posfija: $AB + CD - *$

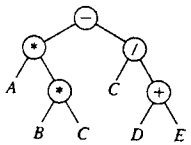
9. 

prefija: $- * + * + ABCDE + * + ABCD$
posfija: $AB + C * D + E * AB + C * D + -$

11. 

prefija: $- + ABC$
entrefija usual: $A + B - C$
entrefija con paréntesis: $((A + B) - C)$

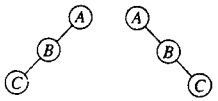
4.



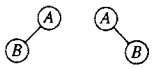
prefija: $- * A * BC/C + DE$
entrefija usual: $A * B * C - C/(D + E)$
entrefija con paréntesis: $((A * (B * C)) - (C/(D + E)))$

6. -4 **19.** 0

2.



5.



3. Entrada: *PT*, la raíz de un árbol binario

Salida: *PT*, la raíz del árbol binario modificado

```
procedure swap_children(PT)
 if PT vacío then
  return
 intercambiar los hijos izquierdo y derecho de PT
 l := hijo izquierdo de PT
 swap_children(l)
 r := hijo derecho de PT
 swap_children(r)
end swap_children
```

Un *segmento inicial* de una cadena son los primeros $i \geq 1$ caracteres, para alguna i. Sea $r(x) = 1$, para $x = A, B, \ldots, Z$; y $r(x) = -1$, para $x = +, -, *, /$. Si $x_1 \cdots x_n$ es una cadena sobre $\{A, \ldots, Z, +, -, *, /\}$, definimos

$$r(x_1 \cdots x_n) = r(x_1) + \cdots + r(x_n).$$

Entonces una cadena s es una cadena posfija si y sólo si $r(s) = 1$ y $r(s') \geq 1$, para todos los segmentos iniciales s' de s.

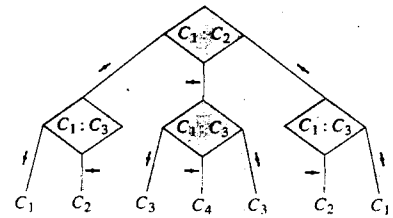
Entrada: *PT*, la raíz de un árbol no vacío

Salida: Cada nodo del árbol tiene un campo *in_cover* que es **true** si ese nodo está en la cubierta de vértices o **false** en caso contrario.

```
procedure tree_cover(PT)
 flag := false
 ptr := primer hijo de PT
 while ptr no es vacío do
  begin
  tree_cover(ptr)
  if in_cover de ptr = false then
   flag := true
  ptr := siguiente hermano de ptr
  end
 in_cover de PT := flag
end tree_cover
```

Sección 7.7

1.



4. Véase la siguiente página.

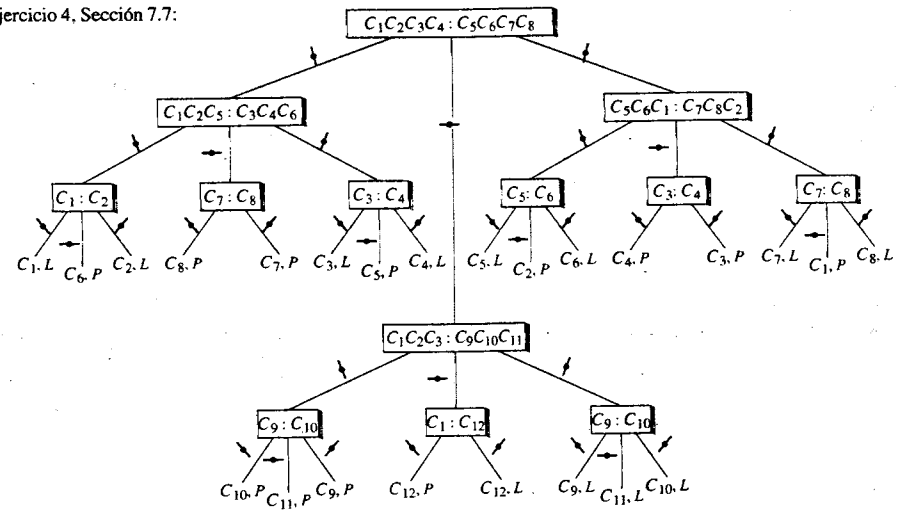
7. Existen 28 resultados posibles para el juego de las 14 monedas. Un árbol de altura 3 tiene a lo más 27 vértices terminales: así, se necesitan al menos cuatro pesadas en el peor de los casos. De hecho, existe un algoritmo que utiliza cuatro pesadas en el peor de los casos: Primero pesamos cuatro monedas contra cuatro monedas. Si las monedas no se equilibran, procedemos como en la solución dada en el ejercicio 4 (para el juego de las 12 monedas). En este caso, se necesitan a lo más tres pesadas. Si las monedas se equilibran, las desechamos; nuestro problema entonces es determinar la moneda mala entre las seis monedas restantes. El juego de 6 monedas se puede resolver con a lo más tres pesadas en el peor de los casos, lo cual, junto con la pesada inicial, requiere cuatro pesadas en el peor de los casos.

10. El análisis del árbol de decisión muestra que se necesitan al menos $\lceil \lg 5! \rceil = 7$ comparaciones para ordenar cinco elementos en el peor de los casos. El siguiente algoritmo ordena cinco elementos utilizando a lo más siete comparaciones en el peor de los casos.

Dada la sucesión $a_1, \ldots, a_5$, primero ordenamos a_1, a_2 (una comparación) y luego a_3, a_4 (una comparación). (Supongamos que $a_1 < a_2$ y $a_3 < a_4$.) Luego comparamos a_2 y a_4. (El caso $a_2 > a_4$ es simétrico, y por esta razón se omite esa parte del algoritmo.) En este punto sabemos que

$$a_1 < a_2 < a_4 \text{ y } a_3 < a_4$$

Ejercicio 4, Sección 7.7:



A continuación determinamos si a_5 está entre a_1, a_2 y a_4 comparando primero a_5 con a_2. Si $a_5 < a_2$, a continuación comparamos a_5 con a_1; pero si $a_5 > a_2$, comparamos a_5 con a_4. En cualquier caso, se necesitan otras dos comparaciones. En este momento, a_1, a_2, a_4, a_5 están ordenados. Por último, insertamos a_3 en el lugar correcto. Si primero comparamos a_3 con el segundo menor elemento de a_1, a_2, a_4, a_5, sólo se necesitará una comparación adicional, para un total de siete comparaciones. Para justificar esta última afirmación, observemos que son posibles los siguientes arreglos después de insertar a_3 en su posición correcta:

$$a_5 < a_1 < a_2 < a_4$$
$$a_1 < a_5 < a_2 < a_4$$
$$a_1 < a_2 < a_5 < a_4$$
$$a_1 < a_2 < a_4 < a_5.$$

Si a_3 es menor que el segundo elemento, sólo se necesita una comparación más (con el primer elemento) para localizar la posición correcta de a_3. Si a_3 es mayor que el segundo elemento, a lo más se necesita otra comparación para localizar la posición correcta de a_3. En los primeros tres casos, sólo necesitamos comparar a_3 con a_2 o a_5 para determinar la posición correcta de a_3, ya que sabemos que $a_3 < a_4$. En el cuarto caso, si a_3 es mayor que a_2, sabemos que va entre a_2 y a_4.

12. Podemos considerar a los números como contendientes y los vértices internos como ganadores, donde el valor mayor gana.

15. Suponga que tenemos un algoritmo que determina el valor máximo entre $x_1, \ldots, x_n$. Sean $x_1, \ldots, x_n$ los vértices de una gráfica. Existe una arista entre x_i y x_j si el algoritmo compara a x_i con x_j. La gráfica debe ser conexa. El mínimo número de aristas necesarias para unir n vértices es $n - 1$.

18. Por el ejercicio 14, el ordenamiento por torneo necesita $2^k - 1$ comparaciones para determinar el elemento máximo. Por el ejercicio 16, el ordenamiento por torneo necesita k comparaciones para determinar el segundo elemento más grande. De manera similar, el ordenamiento por torneo necesita a lo más k comparaciones para determinar el tercer elemento más grande, a lo más k comparaciones para determinar el cuarto más grande, y así sucesivamente. Así, el número total de comparaciones es a lo más

$$\begin{aligned}[2^k - 1] + (2^k - 1)k &\leq 2^k + k2^k \\ &\leq k2^k + k2^k \\ &= 2 \cdot 2^k k = 2n \lg n.\end{aligned}$$

Sección 7.8

1. Isomorfos. $f(v_1) = w_1$, $f(v_2) = w_5$, $f(v_3) = w_3$, $f(v_4) = w_4$, $f(v_5) = w_2$, $f(v_6) = w_6$.

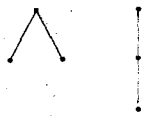
4. No son isomorfos. T_2 tiene un camino simple de longitud 2 de un vértice de grado 1 a un vértice de grado 1, pero T_1 no.

7. Isomorfos como árboles con raíz. $f(v_1) = w_1$, $f(v_2) = w_4$, $f(v_3) = w_3$, $f(v_4) = w_2$, $f(v_5) = w_6$, $f(v_6) = w_5$, $f(v_7) = w_7$, $f(v_8) = w_8$. También son isomorfos como árboles libres.

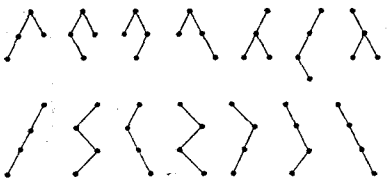
10. No son isomorfos como árboles binarios. La raíz de T_1 tiene un hijo izquierdo, pero la raíz de T_2 no. Isomorfos como árboles con raíz y como árboles libres.

13.

16.



19.



22. Sea b_n el número de árboles binarios completos, no isomorfos, con n vértices. Como cada árbol binario completo tiene un número impar de vértices, $b_n = 0$, si n es par. Mostraremos que si $n = 2i + 1$ es impar, entonces,

$$b_n = C_i,$$

donde C_i denota el i-ésimo número de Catalan.

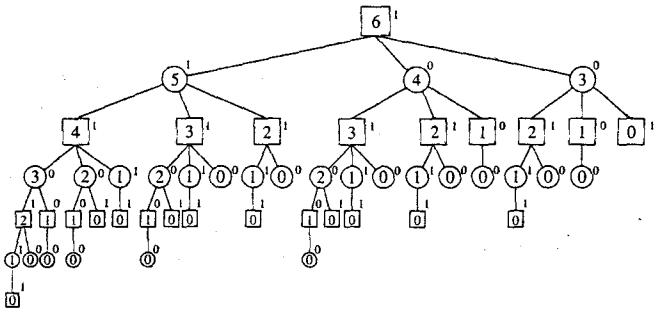
La última ecuación es consecuencia del hecho de que existe una función uno a uno y sobre del conjunto de árboles binarios con i vértices al conjunto de árboles binarios completos con $(2i + 1)$ vértices. Tal función se puede construir como sigue. Dado un árbol binario con i vértices, en cada vértice terminal agregamos dos hijos. En cada vértice con un hijo, agregamos un hijo adicional. Como el árbol que se obtiene tiene i vértices internos, existen $2i + 1$ vértices en total (teorema 7.5.4). El árbol construido es un árbol binario completo. Observe que esta función es uno a uno. Dado un árbol binario completo T' con $(2i + 1)$ vértices, si eliminamos todos los vértices terminales, obtenemos un árbol binario con i vértices T. La imagen de T es T'. Por lo tanto, la función es sobre.

25. Existen cuatro comparaciones en las líneas 1 y 3. Por el ejercicio 24; la llamada *bin_tree_isom*(lc_r_1, lc_r_2) requiere $6(k - 1) + 2$ comparaciones. La llamada *bin_tree_isom*(rc_r_1, rc_r_2) requiere cuatro comparaciones. Así, el número total de comparaciones es

$$4 + 6(k - 1) + 2 + 4 = 6k + 4.$$

Sección 7.9

1.



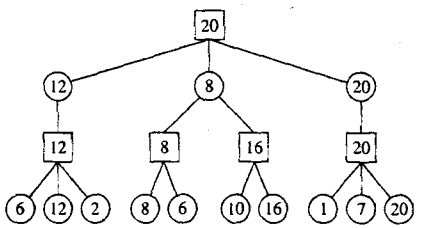
El primer jugador siempre gana. La estrategia ganadora consiste en tomar primero una ficha; y luego, sin importar lo que haga el segundo jugador, dejar una ficha.

4. El segundo jugador siempre gana. Si quedan dos pilas, deja pilas con igual número de fichas. Si queda una pila, tomarla.

7. Suponga que el primer participante puede ganar el juego de nim. El primer jugador siempre puede ganar en nim′ mediante la siguiente estrategia: Jugar nim′ exactamente como nim′ a menos que el movimiento deje un número impar de pilas con una ficha y ninguna otra pila. En este caso, dejar un número par de pilas.

Suponga que el primer jugador siempre puede ganar en nim′. El primer jugador siempre puede ganar en nim adoptando la siguiente estrategia: Jugar nim exactamente como nim′, a menos que el movimiento deje un número par de pilas con una ficha y ninguna otra pila. En este caso, dejar un número impar de pilas.

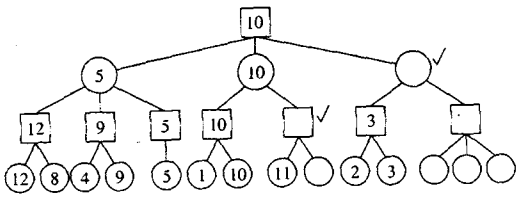
9.



BIBLIOTECA FACULTAD Cs. EXACTAS, INGENIERÍA Y AGRIMENSURA ROSARIO

12. El valor de la raíz es 3.

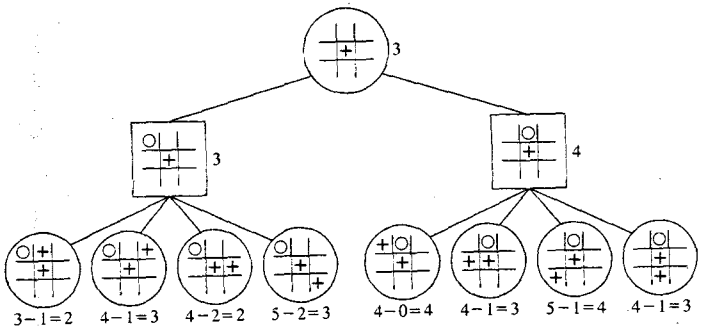
14. [Para el ejercicio 11]



15. $3 - 2 = 1$

18. $4 - 1 = 3$

19.



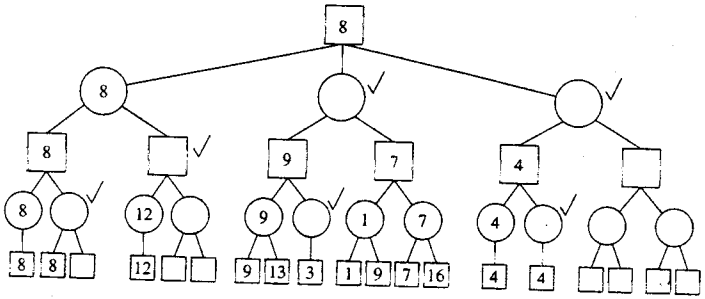
o moverá en una esquina.

22. Entrada: La raíz *PT* de un árbol de juego, el tipo *PT_type* de *PT* (*box* o *circle*, cuadro o círculo), el nivel *PT_level* de *PT*, el máximo nivel n hasta el cual realizar la búsqueda, una función de evaluación E, y un número *ab_val* (que es el valor alfa o beta del padre de *PT*). (La llamada inicial hace *ab_val* igual a ∞ si *PT* es un vértice de cuadro, o $-\infty$ si *PT* es un vértice de círculo.)

Salida: El árbol de juego con *PT* evaluado

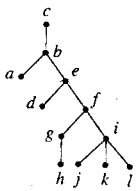
```
procedure alpha_beta_prune(PT,
 PT_type, PT_level, n, E, ab_val)
 if PT_level = n then
  begin
  contents (PT) := E(PT)
  return
  end
 if PT_type = box then
  begin
  contents (PT) := -∞
  for cada hijo C de PT
   alpha_beta_prune (C, circle,
    PT_level + 1, n, E, contents(PT))
  c_val := contents(C)
  if c_val ≥ ab_val then
   begin
   contents(PT) := ab_val
   return
   end
  if c_val > contents(PT) then
   contents(PT) := c_val
 end
 else
  begin
  contents(PT) := ∞
  for cada hijo C de PT
   alpha_beta_prune(C, box,
   PT_level + 1, n, E, contents(PT))
   c_val := contents(C)
   if c_val ≤ ab_val then
    begin
    contents(PT) := ab_val
    return
    end
   if c_val < contents(PT) then
    contents(PT) := c_val
  end
end alpha_beta_prune
```

23. Primero obtenemos los valores 6, 6, 7 para los hijos de la raíz. Luego ordenamos los hijos de la raíz de modo que el hijo del extremo izquierdo sea el primero, y utilizamos el procedimiento alfa-beta para obtener



Capítulo 7 Autoevaluación

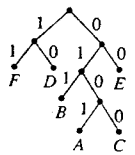
1.



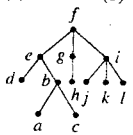
2. *a*-2, *b*-1. *c*-0, *d*-3, *e*-2, *f*-3, *g*-4, *h*-5, *i*-4, *j*-5, *k*-5, *l*-5

3. 5

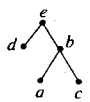
4.



5. (a) *b* (b) *a*, *c*



(c) *d*, *a*, *c*, *h*, *j*, *k*, *l* (d)

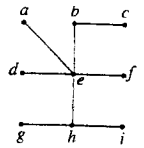


6. Verdadero. Véase el teorema 7.2.3.

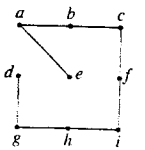
7. Verdadero. Un árbol de altura 6 o más debe tener siete o más vértices.

8. Falso.

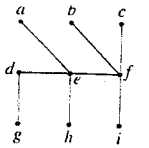
9.



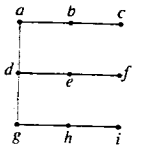
10.



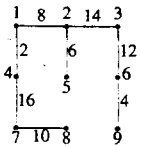
11.



12.



13.



14. (1, 4), (1, 2), (2, 5), (2, 3), (3, 6), (6, 9), (4, 7), (7, 8)

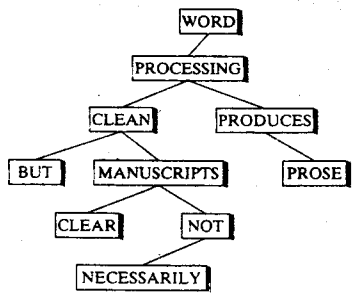
15. (6, 9), (3, 6), (2, 3), (2, 5), (1, 2), (1, 4), (4, 7), (7, 8)

16. Consideremos un "algoritmo del camino más corto" en el cual, en cada paso, elegimos una arista disponible con peso mínimo, incidente en el vértice que ha sido agregado de manera más reciente (véase el análisis antes del teorema 7.4.5).

17.

18. 16

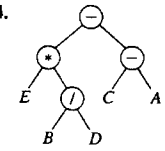
19.



20. Primero comparamos MORE con la palabra WORD en la raíz. Como MORE es menor que WORD, pasamos al hijo izquierdo. A continuación, comparamos MORE con PROCESSING. Como MORE es menor que PROCESSING, pasamos al hijo izquierdo. Como MORE es mayor que CLEAN, pasamos al hijo derecho. Como MORE es mayor que MANUSCRIPTS, pasamos al hijo derecho. Como MORE es menor que NOT, pasamos al hijo izquierdo. Como MORE es menor que NECESSARILY, intentamos ir al hijo izquierdo. Como éste no existe, concluimos que MORE no está en el árbol.

21. *ABFGCDE* **22.** *BGFAEDC* **23.** *GFBEDCA*

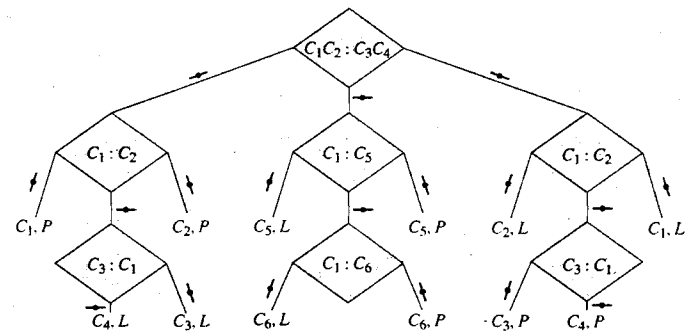
24.



posfija: $EBD/ * CA - -$
entrefija con paréntesis: $((E * (B/D)) - (C - A))$

25. Un algoritmo que requiere a lo más dos pesadas se puede representar mediante un árbol de decisión de altura a lo más 2. Sin embargo, tal árbol tiene a lo más nueve vértices terminales. Como existen 12 resultados posibles, no existe tal algoritmo. Por lo tanto, se necesitan al menos tres pesadas en el peor de los casos para identificar la moneda mala y determinar si es más pesada o más ligera.

26.



27. De acuerdo con el teorema 7.7.3, cualquier algoritmo de ordenamiento necesita al menos $Cn \lg n$ comparaciones en el peor de los casos. Como el algoritmo del profesor Sabic utiliza a lo más $100n$ comparaciones, debemos tener $Cn \lg n \leq 100n$ para todo $n \geq 1$. Si cancelamos n, obtenemos $C \lg n \leq 100$ para toda $n \geq 1$, lo cual es falso. Por lo tanto, el profesor no tiene un algoritmo de ordenamiento que utilice a lo más $100n$ comparaciones en el peor de los casos, para $n \geq 1$.

28. En el peor de los casos, se necesitan tres comparaciones para ordenar tres elementos mediante un ordenamiento óptimo (véase el ejemplo 7.7.2).

Si $n = 4$, el ordenamiento por inserción binaria ordena tres elementos (tres comparaciones en el peor de los casos) y luego inserta el cuarto elemento en la lista ordenada de tres elementos (dos comparaciones en el peor de los casos) para un total de cinco comparaciones en el peor de los casos.

Si $n = 5$, el ordenamiento por inserción binaria ordena cuatro elementos (cinco comparaciones en el peor de los casos) y luego inserta el quinto elemento en la lista ordenada de cuatro elementos (tres comparaciones en el peor de los casos) para un total de ocho comparaciones en el peor de los casos.

Si $n = 6$, el ordenamiento por inserción binaria ordena cinco elementos (ocho comparaciones en el peor de los casos) y luego inserta el sexto elemento en la lista ordenada de cinco elementos (tres comparaciones en el peor de los casos) para un total de once comparaciones en el peor de los casos.

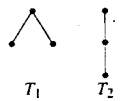
El análisis del árbol de decisión muestra que cualquier algoritmo necesita al menos cinco comparaciones en el peor de los casos para ordenar cuatro elementos. Así, el ordenamiento por inserción binaria es óptimo si $n = 4$.

El análisis del árbol de decisión muestra que cualquier algoritmo necesita al menos siete comparaciones en el peor de los casos para ordenar cinco elementos. De hecho, es posible ordenar cinco elementos mediante siete comparaciones en el peor de los casos. Así, el ordenamiento por inserción binaria no es óptimo si $n = 5$.

El análisis del árbol de decisión muestra que cualquier algoritmo necesita al menos diez comparaciones en el peor de los casos para ordenar seis elementos. De hecho, es posible ordenar seis elementos mediante diez comparaciones en el peor de los casos. Así, el ordenamiento por inserción binaria no es óptimo si $n = 6$.

29. Verdadero. Si f es un isomorfismo de T_1 y T_2 como árboles con raíz, f es también un isomorfismo de T_1 y T_2 como árboles libres.

30. Falso.



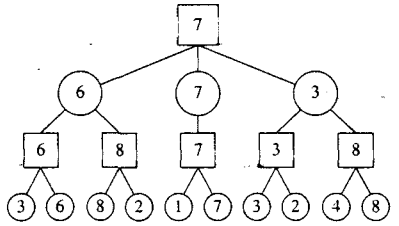
31. Isomorfos. $f(v_1) = w_6$, $f(v_2) = w_2$, $f(v_3) = w_5$, $f(v_4) = w_7$, $f(v_5) = w_4$, $f(v_6) = w_1$, $f(v_7) = w_3$, $f(v_8) = w_8$.

32. No son isomorfos. T_1 tiene un vértice (v_3) en el nivel 1, de grado 3, pero T_2 no.

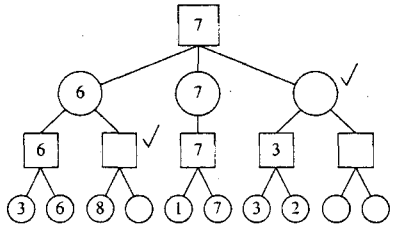
33. $3 - 1 = 2$

34. Hagamos que cada renglón, columna o diagonal que contenga una X y dos espacios en blanco cuente 1. Hagamos que cada renglón, columna o diagonal que contenga dos X y un espacio en blanco cuente 5. Hagamos que cada renglón, columna o diagonal que contenga tres X cuente 100. Hagamos que cada renglón, columna o diagonal que contenga un 0 y dos espacios en blanco cuente -1. Hagamos que cada renglón, columna o diagonal que contenga dos 0 y un espacio en blanco cuente -5. Hagamos que cada renglón, columna o diagonal que contenga tres 0 cuente -100. Sume los valores obtenidos.

35.



36.

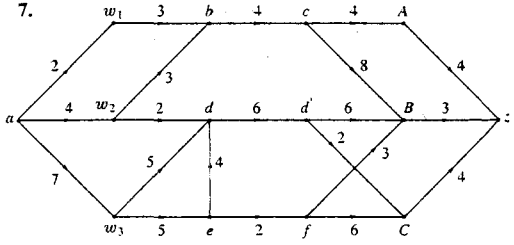


Sección 8.1

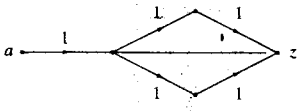
1. (b, c) es 6, 3; (a, d) es 4, 2; (c, e) es 6, 1; (c, z) es 5, 2. El valor del flujo es 5.

4. Agregar las aristas (a, w_1), (a, w_2), (a, w_3), (A, z), (B, z) y (C, z), cada una con capacidad ∞.

7.



10.

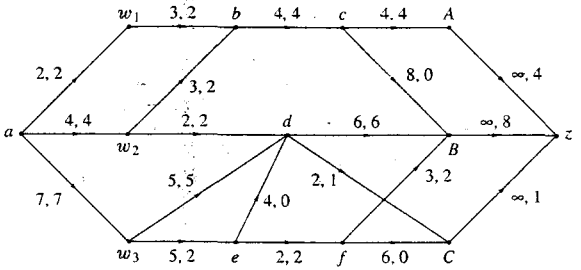


Sección 8.2

1. 1

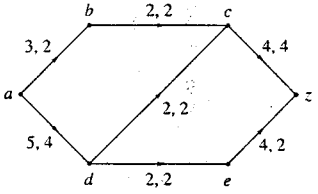
4. $(a, w_1)-6$, $(a, w_2)-0$, $(a, w_3)-3$, $(w_1, b)-6$, $(w_2, b)-0$, $(w_3, d)-3$, $(d, c)-3$, $(b, c)-2$, $(b, A)-4$, $(c, A)-2$, $(c, B)-3$, $(A, z)-6$, $(B, z)-3$

7.



10. $(a, A - 7{:}00) - 3000$, $(a, A - 7{:}15) - 3000$, $(a, A - 7{:}30) - 2000$, $(A - 7{:}00, B - 7{:}30) - 1000$, $(A - 7{:}00, C - 7{:}15) - 2000$, $(A - 7{:}15, B - 7{:}45) - 1000$, $(A - 7{:}15, C - 7{:}30) - 2000$, $(A - 7{:}30, C - 7{:}45) - 2000$, $(B - 7{:}30, D - 7{:}45) - 1000$, $(C - 7{:}15, D - 7{:}30) - 2000$, $(B - 7{:}45, D - 8{:}00) - 1000$, $(C - 7{:}30, D - 7{:}45) - 2000$, $(C - 7{:}45, D - 8{:}00) - 2000$, $(D - 7{:}45, z) - 3000$, $(D - 7{:}30, z) - 2000$, $(D - 8{:}00, z) - 3000$. El resto de las aristas tienen flujo igual a 0.

13.



16. El flujo máximo es 9.

19. Supongamos que la suma de las capacidades de las aristas incidentes en a es U. Cada iteración del algoritmo 8.2.5 incrementa el flujo en 1. Como el flujo no puede exceder a U, el algoritmo debe concluir en algún momento.

Sección 8.3

1. 8; mínimo

4. $P = \{a, b, d\}$

7. $P = \{a, d\}$

10. $P = \{a, w_1, w_2, w_3, b, d, e\}$

13. $P = \{a, w_1, w_2, w_3, b, c, d, d', e, f, A, B, C\}$

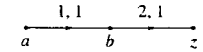
16. $P = \{a, b, c, f, g, h, j, k, l, m\}$

17.

con $C_{ab} = 1$, $C_{bz} = 2$, $m_{ab} = 1$, $m_{bz} = 2$.

20. Modifique el algoritmo 8.2.4.

23. Falso; considere el flujo

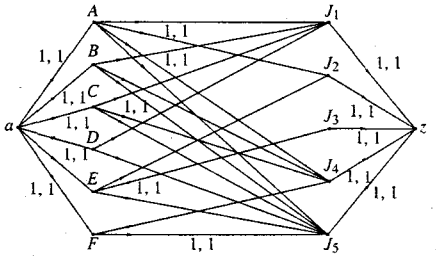


y el corte $P = \{a, b\}$.

Sección 8.4

1. $P = \{a, A, B, D, J_2, J_5\}$

4. Todas las aristas no etiquetadas son 1, 0. No existe un acoplamiento completo.



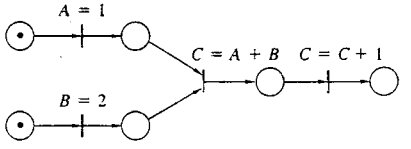
8. Cada renglón y columna tiene a lo más una etiqueta.

12. Si $\delta(G) = 0$, entonces $|S| - |R(S)| \leq 0$, para todo $S \subseteq V$. Por el teorema 8.4.7, G tiene un acoplamiento completo.

Si G tiene un acoplamiento completo, entonces $|S| - |R(S)| \leq 0$, para todo $S \subseteq V$, de modo que $\delta(G) \leq 0$. Si $S = \varnothing$, $|S| - |R(S)| = 0$, de modo que $\delta(G) = 0$.

Sección 8.5

1.

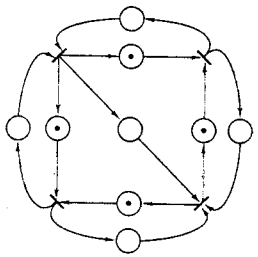


Sistemas operativos para computadoras, protocolos de comunicación, sistemas de información en oficinas, control de procesos industriales, sistemas de bases de datos (en especial, los sistemas distribuidos de bases de datos).

Sea M_1 el marcado que resulta de M al descargar t_1. La única transición activada en M_1 es t_2. Sea M_2 el marcado que resulta de M_1 al descargar t_2. La única transición activada en M_2 es t_3. Si se descarga t_3, se obtiene el marcado M. Esto implica que M está vivo y acotado.

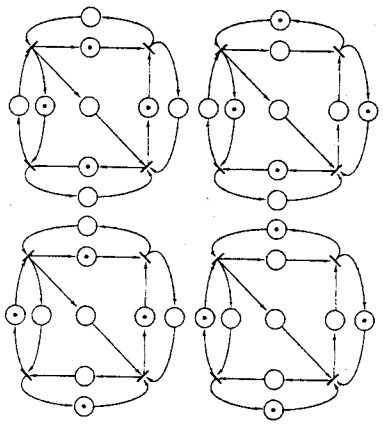
(a) t_1

(b)



(c) Sí (d) Sí (e) Sí

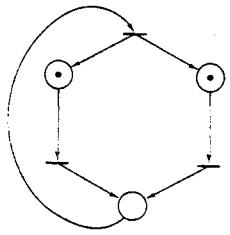
(f)



(g) Cualquier marcado que coloque dos elementos en al menos un lugar.

11. (a) t_1

(b)

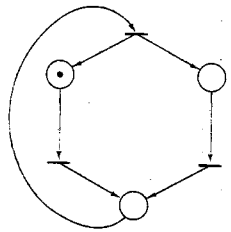


(c) Sí (d) No (e) No

(f) Designe al lugar de entrada a t_i como p_i. Los marcados alcanzables desde M son de dos tipos:

1. p_3 tiene al menos un elemento.
2. p_3 no tiene elementos y p_1 y p_2 tienen cada uno un elemento.

(g)

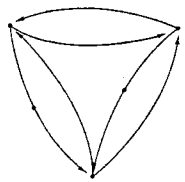


13. Figura 8.5.5

17. 8 y 9

20. Si se descarga un vértice en un ciclo simple dirigido C, se elimina un elemento de una arista en C, y se agrega un elemento a una arista en C; por lo tanto, la cantidad de elementos en C no cambia.

23.



Capítulo 8 Autoevaluación

1. En cada arista, el flujo es menor o igual que la capacidad y, excepto por la fuente y el sumidero, el flujo de entrada de cada vértice v es igual al flujo de salida de v.

2. 3 **3.** 3 **4.** 3 **5.** (a, b, e, f, g, z)

6. Modifique los flujos por $F_{a,b} = 2, F_{e,b} = 1, F_{e,f} = 1, F_{f,g} = 1, F_{g,z} = 1$.

7. $F_{a,b} = 3, F_{b,c} = 3, F_{c,d} = 4, F_{d,z} = 4, F_{a,e} = 2, F_{e,f} = 2, F_{f,c} = 2, F_{f,g} = 1, F_{g,z} = 1$, y el flujo en las demás aristas es cero.

8. $F_{a,b} = 0, F_{b,c} = 5, F_{c,d} = 5, F_{d,z} = 8, F_{e,b} = 3, F_{g,d} = 3, F_{a,e} = 8, F_{e,f} = 3, F_{f,g} = 3, F_{a,h} = 4, F_{e,i} = 2, F_{j,z} = 6, F_{h,i} = 4, F_{i,j} = 6$, y el flujo en las demás aristas es cero.

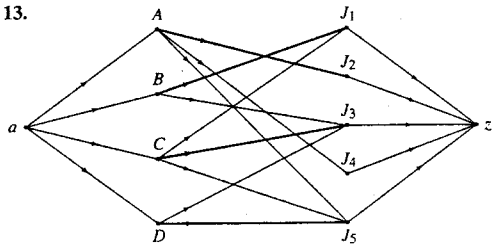
9. a—verdadero; b—falso; c—falso; d—verdadero

10. 6.

11. No. La capacidad de $(P, \bar{P})$ es 6, pero la capacidad de $(P', \bar{P}')$, $P' = \{a, b, c, e, f\}$, es 5.

12. $P = \{a, b, c, e, f, g, h, i\}$

13.

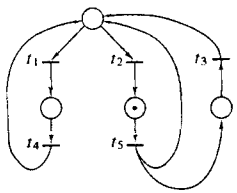


14. Véase la solución del ejercicio 13.

15. $A - J_2$, $B - J_1$, $C - J_3$, $D - J_5$, es un acoplamiento completo.

16. $P = \{a\}$ **17.** t_1 y t_2

18.



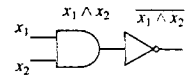
19. No

20. Sí

Sección 9.1

1. $\overline{x_1 \wedge x_2}$

x_1	x_2	$\overline{x_1 \wedge x_2}$
1	1	0
1	0	1
0	1	1
0	0	1



4.

x_1	x_2	x_3	$((x_1 \wedge x_2) \vee \overline{(x_1 \wedge x_3)}) \wedge \bar{x}_3$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	1

7. Si $x = 1$, la salida y está indeterminada: Supongamos que $x = 1$ y $y = 0$. Entonces, la entrada de la compuerta AND es 1, 0. Así, la salida de esta compuerta es 0. Como luego se realiza el NOT de esta expresión, $y = 1$. Contradicción. Se obtiene una contradicción análoga si $x = 1$ y $y = 1$.

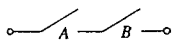
10. 0

13. 1

16. Es una expresión booleana. x_1, x_2 y x_3 son expresiones booleanas por (9.1.2). $x_2 \vee x_3$ es una expresión booleana por (9.1.3c). $(x_2 \vee x_3)$ es una expresión booleana por (9.1.3a). $x_1 \wedge (x_2 \vee x_3)$ es una expresión booleana por (9.1.3d).

19. No es una expresión booleana

22.



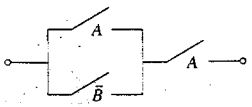
25. $(A \wedge B) \vee (C \wedge \bar{A})$

A	B	C	$(A \wedge B) \vee (C \wedge \bar{A})$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

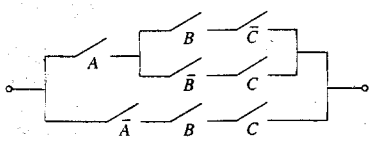
28. $(A \wedge (C \vee (D \wedge C))) \vee (B \wedge (\bar{D} \vee (C \wedge A) \vee \bar{C}))$

30.

A	B	$(A \vee \bar{B}) \wedge A$
1	1	1
1	0	1
0	1	0
0	0	0



33.



Sección 9.2

1.

x_1	x_2	$\overline{x_1 \wedge x_2}$	$\bar{x}_1 \vee \bar{x}_2$
1	1	0	0
1	0	1	1
0	1	1	1
0	0	1	1

4.

x_1	x_2	x_3	$\bar{x}_1 \vee (\bar{x}_2 \vee x_3)$	$\overline{(x_1 \wedge x_2)} \vee x_3$
1	1	1	1	1
1	1	0	0	0
1	0	1	1	1
1	0	0	1	1
0	1	1	1	1
0	1	0	1	1
0	0	1	1	1
0	0	0	1	1

6.

x_1	$x_1 \vee x_1$
1	1
0	0

9.

x_1	x_2	x_3	$x_1 \wedge \overline{(x_2 \wedge x_3)}$	$(x_1 \wedge \bar{x}_2) \vee (x_1 \wedge \bar{x}_3)$
1	1	1	0	0
1	1	0	1	1
1	0	1	1	1
1	0	0	1	1
0	1	1	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	0	0

11.

x	$\bar{\bar{x}}$
1	1
0	0

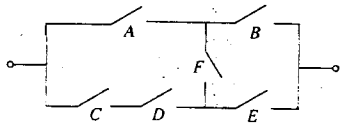
14. Falso. Sean $x_1 = 1, x_2 = 1, x_3 = 0$.

16.

a	b	c	$a \vee (b \wedge c)$	$(a \vee b) \wedge (a \vee c)$
1	1	1	1	1
1	1	0	1	1
1	0	1	1	1
1	0	0	1	1
0	1	1	1	1
0	1	0	0	0
0	0	1	0	0
0	0	0	0	0

18. Las expresiones booleanas que representan los circuitos son $(A \wedge \bar{B}) \vee (A \wedge C)$ y $A \wedge (\bar{B} \vee C)$. Las expresiones son iguales por el teorema 9.2.1c. Por lo tanto, los circuitos de conmutación son equivalentes.

21.



Sección 9.3

2. Uno puede mostrar de manera directa que las leyes asociativa y distributiva se cumplen para el mcm y el mcd. Es claro que se cumple la ley conmutativa. Para ver que se cumplen las leyes de identidad, observemos que

$$\text{mcm}(x, 1) = x \quad \text{y} \quad \text{mcd}(x, 6) = x.$$

Como

$$\text{mcm}(x, 6/x) = 6 \quad \text{y} \quad \text{mcd}(x, 6/x) = 1,$$

se cumplen las leyes de complemento. Por lo tanto, $(S, +, \cdot, ', 1, 6)$ es un álgebra booleana.

4. Sólo mostramos que

$$x \cdot (x + z) = (x \cdot y) + (x \cdot z) \quad \text{para todo } x, y, z \in S_n.$$

Ahora,

$$x \cdot (y + z) = \text{mín}\{x, \text{máx}\{y, z\}\},$$
$$(x \cdot y) + (x \cdot z) = \text{máx}\{\text{mín}\{x, y\}, \text{mín}\{x, z\}\}.$$

Supongamos que $y \le z$. (El argumento es similar si $y > z$.) Hay que considerar tres casos: $x < y$, $y \le x \le z$ y $z < x$.

Si $x < y$, obtenemos

$$\begin{aligned} x \cdot (y + z) &= \text{mín}\{x, \text{máx}\{y, z\}\} \\ &= \text{mín}\{x, z\} = x = \text{máx}\{x, x\} \\ &= \text{máx}\{\text{mín}\{x, y\}, \text{mín}\{x, z\}\} \\ &= (x \cdot y) + (x \cdot z). \end{aligned}$$

Si $y \le x \le z$, obtenemos

$$\begin{aligned} x \cdot (y + z) &= \text{mín}\{x, \text{máx}\{x, z\}\} \\ &= \text{mín}\{x, z\}\} = x = \text{máx}\{y, x\} \\ &= \text{máx}\{\text{mín}\{x, y\}, \text{mín}\{x, z\}\} \\ &= (x \cdot y) + (x \cdot z). \end{aligned}$$

Si $z < x$, obtenemos

$$\begin{aligned} x \cdot (y + z) &= \text{mín}\{x, \text{máx}\{y, z\}\} \\ &= \text{mín}\{x, z\} = z = \text{máx}\{y, z\} \\ &= \text{máx}\{\text{mín}\{x, y\}, \text{mín}\{x, z\}\} \\ &= (x \cdot y) + (x \cdot z). \end{aligned}$$

7. Si $X \cup Y = U$ y $X \cap Y = \varnothing$, entonces $Y = \bar{X}$.

8. $xy + x0 = x(x + y)y$

11. $x + y' = 1$ si y sólo si $x + y = x$.

14. $x(x + y0) = x$

15. [Para el ejercicio 12]

$$\begin{aligned} 0 &= x + y = (x + x) + y \\ &= x + (x + y) = x + 0 = x \end{aligned}$$

De manera análoga, $y = 0$.

18. [Para la parte (c)]

$$\begin{aligned} x(x + y) &= (x + 0)(x + y) \\ &= x + 0y = x + y0 = x + 0 = x \end{aligned}$$

21. Primero mostramos que si $ba = ca$ y $ba' = ca'$, entonces $b = c$. Ahora, haga $a = x$, $b = x + (y + z)$, y $c = (x + y) + z$ y utilice este resultado.

23. Si el primo p divide a n, p^2 no divide a n.

Sección 9.4

En estas sugerencias, $a \wedge b$ se escribe ab.

1. $xy \vee \bar{x}y \vee \bar{x}\bar{y}$

4. $xyz \vee xy\bar{z} \vee x\bar{y}\bar{z} \vee \bar{x}yz \vee \bar{x}y\bar{z}$

7. $xyz \vee x\bar{y}\bar{z} \vee \bar{x}y\bar{z}$

10. $wx\bar{y}z \vee wx\bar{y}\bar{z} \vee w\bar{x}yz \vee w\bar{x}y\bar{z} \vee w\bar{x}\bar{y}\bar{z}$
$\vee \bar{w}xyz \vee \bar{w}x\bar{y}z \vee \bar{w}x\bar{y}\bar{z} \vee \bar{w}\bar{x}yz \vee \bar{w}\bar{x}y\bar{z}$

11. $xy \vee x\bar{y}$ **14.** $xy\bar{z}$

17. $xyz \vee \bar{x}yz \vee xy\bar{z} \vee \bar{x}y\bar{z}$

20. 0

22. 2^{2^n}

25. [Para el ejercicio 3]

$$(\bar{x} \vee y \vee \bar{z})(x \vee \bar{y} \vee \bar{z})(x \vee \bar{y} \vee z)$$

28. [Para el ejercicio 3]

$$(\bar{x} \vee \bar{y} \vee \bar{z})(\bar{x} \vee \bar{y} \vee z)(\bar{x} \vee y \vee z)(x \vee y \vee \bar{z})(x \vee y \vee z)$$

ción 9.5

AND se puede expresar en términos de OR y NOT: $xy = \overline{\bar{x} \vee \bar{y}}$.

Un circuito combinatorio que sólo consta de compuertas AND tendría siempre 0 como salida, si todas las entradas son 0.

Utilizamos inducción sobre n para mostrar que no existe un circuito combinatorio de n compuertas que sólo conste de compuertas AND y OR y que calcule $f(x) = \bar{x}$.

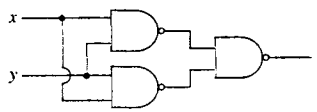
Si $n = 0$, la entrada x es igual a la salida x, de modo que es imposible que un circuito con 0 compuertas calcule f. Esto demuestra el paso base.

Ahora, supongamos que no existe un circuito combinatorio con n compuertas, que conste sólo de compuertas AND y OR y que calcule f. Consideremos un circuito combinatorio con $(n + 1)$ compuertas, que conste sólo de compuertas AND y OR. Al principio, la entrada x llega a una compuerta AND o a una compuerta OR. Supongamos que x llega al principio a una compuerta AND. (El argumento es similar si x llega al principio a una compuerta OR, por lo cual se omite.) Debido a que el circuito es combinatorio, la otra entrada de la compuerta AND es x mismo, la constante 1 o la constante 0. Si ambas entradas de la compuerta AND son el propio x, entonces la salida de la compuerta AND es igual a la entrada. En este caso, el comportamiento del circuito no se altera si eliminamos la compuerta AND y unimos x con lo que era la línea de salida de la compuerta AND. Pero ahora tenemos un circuito equivalente con n compuertas, el cual, por la hipótesis de inducción, no puede calcular f. Así, el circuito con $(n + 1)$ compuertas no puede calcular f.

Si la otra entrada de la compuerta AND es la constante 1, la salida de la compuerta AND es de nuevo igual a la entrada y podemos argumentar como en el caso anterior que el circuito de $(n + 1)$ compuertas no puede calcular f.

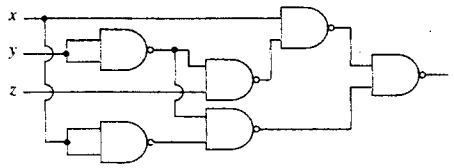
Si la otra entrada de la compuerta AND es la constante 0, la salida de la compuerta AND siempre será 0 y, en consecuencia, la modificación del valor de x no afecta la salida del circuito. En este caso, el circuito no puede calcular f. Esto concluye el paso inductivo. Por lo tanto, ningún circuito combinatorio con n compuertas, que conste sólo de compuertas AND y OR, puede calcular $f(x) = \bar{x}$. Así, {AND, OR} no es funcionalmente completo.

6.

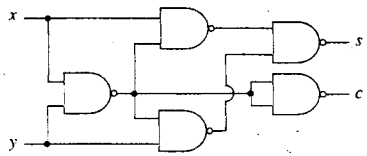


9. $y_1 = x_1x_2 \vee \overline{(x_2 \vee x_3)}$; $y_2 = \overline{x_2 \vee x_3}$

12. [Para el ejercicio 3] La forma disyuntiva normal se puede simplificar como $xy \vee x\bar{z} \vee \bar{x}\bar{y}$ y luego escribir como $x(y \vee \bar{z}) \vee \overline{x \vee y} = (x\overline{y\bar{z}}) \vee \overline{x \vee y} = \overline{x\overline{y\bar{z}}}\;\overline{x \vee y}$, lo cual produce el circuito



15.



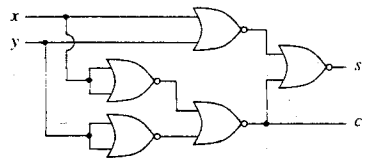
17. $xy = (x \downarrow y) \downarrow (y \downarrow y)$;
$x \vee y = (x \downarrow y) \downarrow (x \downarrow y)$; $\bar{x} = x \downarrow x$;
$x \uparrow y = [(x \downarrow x) \downarrow (y \downarrow y)] \downarrow [(x \downarrow x) \downarrow (y \downarrow y)]$

20. Como

$$\bar{x} = x \downarrow x, \quad x \vee y = (x \downarrow y) \downarrow (x \downarrow y),$$

y {NOT, OR} es funcionalmente completo, {NOR} es funcionalmente completo.

23.



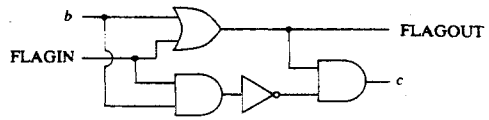
25. La tabla lógica es

x	y	z	*Salida*
1	1	1	1
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

27. La tabla lógica es

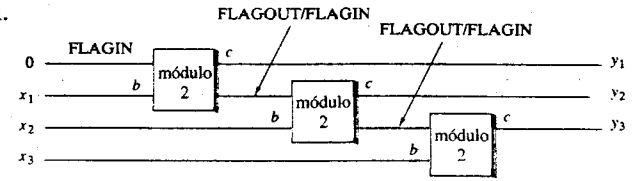
b	FLAGIN	c	FLAGOUT
1	1	0	1
1	0	1	1
0	1	1	1
0	0	0	0

Así, $c = b \oplus$ FLAGIN y FLAGOUT $= b \vee$ FLAGIN. Obtenemos el circuito



28. 010100

31.



34. Las tablas de verdad muestran que

$$\bar{x} = x \to 0, \quad x \vee y = (x \to 0) \to y.$$

Por lo tanto, una compuerta NOT se puede reemplazar por una compuerta $\to$, y una compuerta OR se puede reemplazar mediante dos compuertas $\to$. Como el conjunto {NOT, OR} es funcionalmente completo, esto implica que el conjunto $\{\to\}$ es funcionalmente completo.

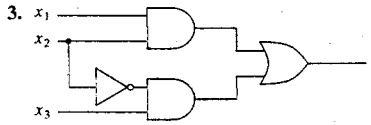
Capítulo 9 Autoevaluación

1.

x	y	z	$\overline{(x \wedge \bar{y}) \vee z}$
1	1	1	1
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	1

2. 1

3.



4. Supongamos que x es 1. Entonces la entrada superior de la compuerta OR es 0. Si y es 1, entonces la entrada inferior de la compuerta OR es 0. Como ambas entradas de la compuerta son 0, la salida y de la compuerta OR es 0, lo cual es imposible. Si y es 0, entonces la entrada inferior de la compuerta OR es 1. Como una entrada de la compuerta OR es 1, la salida y de esta compuerta es 1, lo cual es imposible. Por lo tanto, si la entrada del circuito es 1, la salida no queda determinada de manera única. Así, el circuito no es combinatorio.

5. Los circuitos son equivalentes. La tabla lógica para cualquiera de estos circuitos es

x	y	*Salida*
1	1	0
1	0	1
0	1	0
0	0	0

6. Los circuitos no son equivalentes. Si $x = 0$, $y = 1$ y $z = 0$, la salida del circuito (a) es 1, mientras que la salida del circuito (b) es 0.

7. La ecuación es verdadera. La tabla lógica para cualquiera de estas expresiones es

x	y	z	*Valor*
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	0

8. La ecuación es falsa. Si $x = 1$, $y = 0$ y $z = 1$, entonces

$$(x \wedge y \wedge z) \vee \overline{(x \vee z)} = 0,$$

pero

$$(x \wedge z) \vee (\overline{x} \wedge \overline{z}) = 1.$$

9. Leyes de acotación:

$X \cup U = U \quad X \cap \varnothing = \varnothing$ para todo $X \in S$.

Leyes de absorción:

$X \cup (X \cap Y) = X \quad X \cap (X \cup Y) = X$ para todo $X, Y \in S$.

10. $(x(x + y \cdot 0))' = (x(x + 0))'$ (Ley de acotación)
$= (x \cdot x)'$ (Ley de identidad)
$= x'$ (Ley de idempotencia)

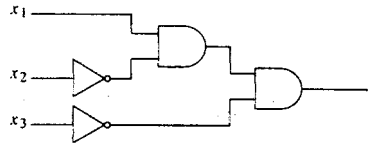
11. Dual: $(x + x(y + 1))' = x'$

$(x + x(y + 1))' = (x + x \cdot 1)'$ (Ley de acotación)
$= (x + x)'$ (Ley de identidad)
$= x'$ (Ley de idempotencia)

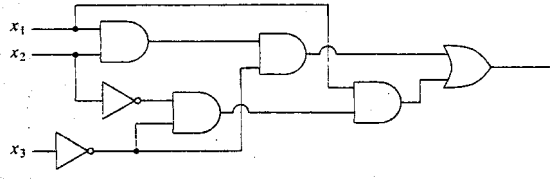
12. $\overline{\ }$ no es un operador unario en S. Por ejemplo, $\overline{[1, 2]} \notin S$.

En los ejercicios 13-16, $a \wedge b$ se escribe ab.

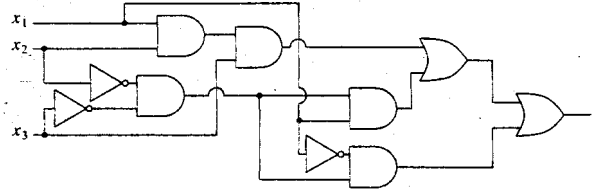
13. $x_1\overline{x}_2\overline{x}_3$



14. $x_1x_2\overline{x}_3 \vee x_1\overline{x}_2\overline{x}_3$



15. $x_1x_2x_3 \vee x_1\overline{x}_2\overline{x}_3 \vee \overline{x}_1\overline{x}_2\overline{x}_3$



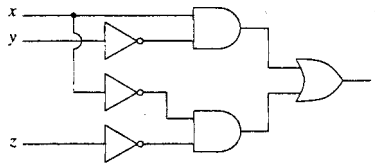
16. $x_1x_2\overline{x}_3 \vee x_1\overline{x}_2\overline{x}_3 \vee \overline{x}_1x_2x_3 \vee \overline{x}_1\overline{x}_2x_3$

17.

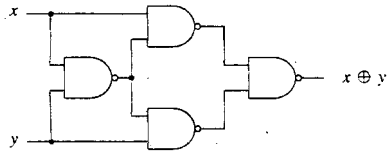
x	y	z	*Salida*
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	0
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	0

18. Forma disyuntiva normal: $x\overline{y}z \vee x\overline{y}\overline{z} \vee \overline{x}yz \vee \overline{x}\overline{y}\overline{z}$

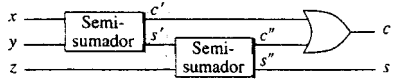
$(x\overline{y}z \vee x\overline{y}\overline{z}) \vee \overline{x}yz \vee \overline{x}\overline{y}\overline{z} = x\overline{y} \vee (\overline{x}yz \vee \overline{x}\overline{y}\overline{z})$
$= x\overline{y} \vee \overline{x}\overline{z}$



19.

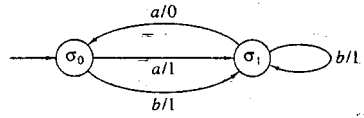


20.

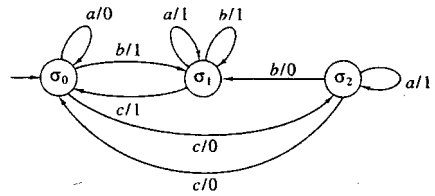


Sección 10.1

1.



4.



6. $\mathcal{I} = \{a, b\}$; $\mathcal{O} = \{0, 1\}$; $\mathcal{S} = \{\sigma_0, \sigma_1\}$; estado inicial = σ_0

$\mathcal{S}$ \ $\mathcal{I}$	a	b	a	b
σ_0	σ_1	σ_0	0	1
σ_1	σ_1	σ_1	1	1

9. $\mathcal{I} = \{a, b\}$; $\mathcal{O} = \{0, 1\}$; $\mathcal{S} = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$; estado inicial = σ_0

$\mathcal{S}$ \ $\mathcal{I}$	a	b	a	b
σ_0	σ_1	σ_2	0	0
σ_1	σ_0	σ_2	1	0
σ_2	σ_3	σ_0	0	1
σ_3	σ_1	σ_3	0	0

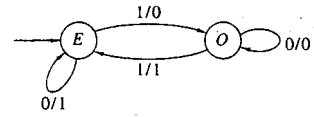
11. 1110

14. 001110

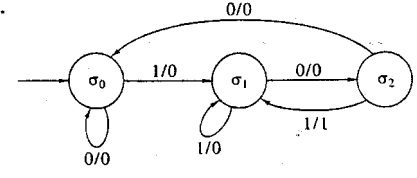
17. 001110001

20. 020022201020

21.



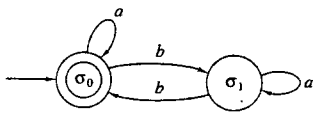
24.

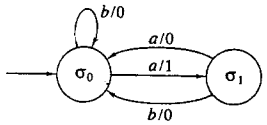


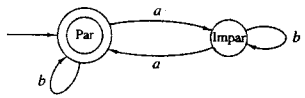
7. Cuando γ es la entrada, la máquina produce como salida x_n, $x_{n-1}, \ldots$, hasta $x_i = 1$. A partir de ese momento, la salida es $\overline{x}_i$. Sin embargo, de acuerdo con el algoritmo 9.5.16, éste es el complemento a dos de α.

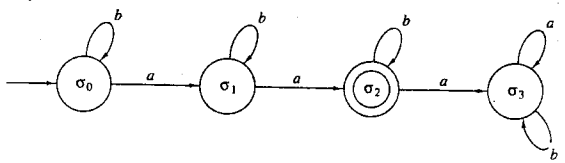
ección 10.2

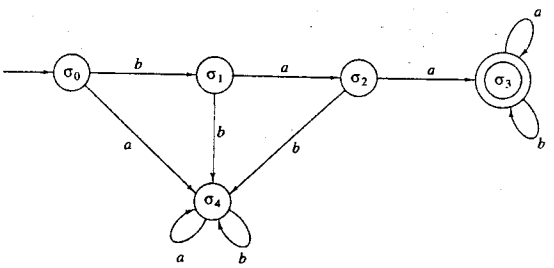
1. Todas las aristas que llegan a σ_0 producen 1 como salida y todas las aristas que llegan a σ_1 producen 0 como salida; por lo tanto, la máquina de estado finito es un autómata de estado finito.



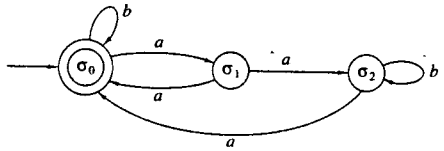








7.



10. [Para el ejercicio 1] $\mathcal{I} = \{a, b\}$; $\mathcal{S} = \{\sigma_0, \sigma_1\}$; $\mathcal{A} = \{\sigma_0\}$; estado inicial = σ_0

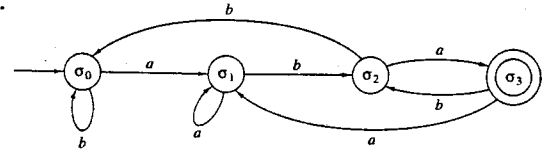
$\mathcal{S}$ \ $\mathcal{I}$	a	b
σ_0	σ_0	σ_1
σ_1	σ_1	σ_0

13. Aceptado

16. Aceptado

18. Sin importar el estado en que estemos, después de una a nos movemos a un estado de aceptación; sin embargo, después de una b nos movemos a un estado de no aceptación.

30.



32. [Para el ejercicio 1] Este algoritmo determina si una cadena sobre $\{a, b\}$ es aceptada por el autómata de estado finito cuyo diagrama de transición aparece en el ejercicio 1.

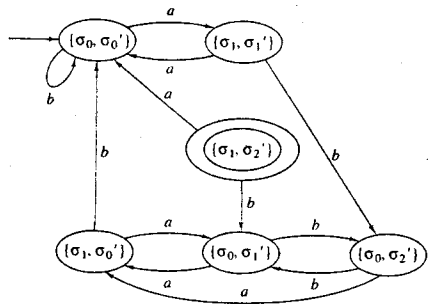
Entrada: n, la longitud de la cadena ($n = 0$ indica la cadena nula) $s_1 \cdots s_n$, la cadena

Salida: "Aceptar" si la cadena es aceptada
"Rechazar" si la cadena no es aceptada

```
procedure ex32(s, n)
 state := 'σ0'
 for i := 1 to n do
  begin
  if state = 'σ0' and si := 'b' then
   state := 'σ1'
  if state = 'σ1' and si := 'b' then
   state := 'σ0'
  end
 if state = 'σ0' then
  return("Aceptar")
 else
  return("Rechazar")
end ex32
```

35. Haga que cada estado de aceptación sea ahora de no aceptación y viceversa.

38. Utilice la construcción dada en los ejercicios 36 y 37, para obtener el siguiente autómata de estado finito que acepta $L_1 \cap L_2$. (Designamos los estados del ejercicio 5 con apóstrofos.)



El autómata de estado finito que acepta $L_1 \cup L_2$ es igual al autómata de estado finito que acepta $L_1 \cap L_2$, excepto porque el conjunto de estados de aceptación es

$$\{(\sigma_1, \sigma_0'), \quad (\sigma_1, \sigma_1'), \quad (\sigma_1, \sigma_2'), \quad (\sigma_0, \sigma_2')\}.$$

41. Utilice la construcción de los ejercicios 36 y 37.

Sección 10.3

1. Regular, libre de contexto, sensible al contexto

4. Libre de contexto, sensible al contexto

7. $\sigma \Rightarrow b\sigma \Rightarrow bb\sigma \Rightarrow bbaA \Rightarrow bbabA \Rightarrow bbabbA \Rightarrow bbabba\sigma \Rightarrow bbabbab$

10. $\sigma \Rightarrow ABA \Rightarrow ABBA \Rightarrow ABBAA$
$\Rightarrow ABBaAA \Rightarrow abBBaAA \Rightarrow abbBaAA$
$\Rightarrow abbbaAA \Rightarrow abbbaabA \Rightarrow abbbaabab$

12. [Para el ejercicio 1]

$$< \sigma > ::= b < \sigma > \mid a < A > \mid b$$
$$< A > ::= a < \sigma > \mid b < A > \mid a$$

15. $S \to aA$, $A \to aA$, $A \to bA$, $A \to a$, $A \to b$, $S \to a$

18. $S \to aA$, $S \to bS$, $S \to \lambda$, $A \to aA$, $A \to bB$, $A \to \lambda$, $B \to aA$, $B \to bS$

21. < número exponencial > ::=
< entero > E < entero > |
< número con punto flotante > |
< número con punto flotante > E < entero >

24. $S \to aSa$, $S \to bSb$, $S \to a$, $S \to b$, $S \to \lambda$

25. Si una derivación comienza con $S \Rightarrow aSb$, la cadena resultante comienza con a y termina con b. De manera análoga, si una derivación comienza con $S \Rightarrow bSa$, la cadena resultante comienza con b y termina con a. Por lo tanto, la gramática no genera la cadena $abba$.

28. Si una derivación comienza con $S \Rightarrow abS$, la cadena resultante comienza con ab. Si una derivación comienza con $S \Rightarrow baS$, la cadena resultante comienza con ba. Si una derivación comienza con $S \Rightarrow aSb$, la cadena resultante comienza con a y termina con b. Si una derivación comienza con $S \Rightarrow bSa$, la cadena resultante comienza con b y termina con a. Por lo tanto, la gramática no genera la cadena $aabbabba$.

31. La gramática genera a L, el conjunto de todas las cadenas sobre $\{a, b\}$ con igual número de letras a que de letras b.

Cualquier cadena generada por la gramática tiene igual número de letras a que de letras b, pues siempre que se utilice cualquiera de las producciones en una derivación, se agrega a la cadena igual número de letras a que de letras b.

Para demostrar el recíproco, consideremos una cadena arbitraria α en L y utilicemos inducción sobre la longitud $|\alpha|$ de α para mostrar que α es generada por la gramática. El paso base es $|\alpha| = 0$. En este caso, α es la cadena nula, y $S \Rightarrow \lambda$ es una derivación de α.

Sea α una cadena no nula, y supongamos que cualquier cadena en L cuya longitud es menor que $|\alpha|$ es generada por la gramática. Primero consideremos el caso en que α comienza con a. Entonces α se puede escribir como $\alpha = a\alpha_1 b\alpha_2$, donde α_1 y α_2 tienen igual número de letras a que de letras b. Por la hipótesis de inducción, existen derivaciones $S \Rightarrow \alpha_1$ y $S \Rightarrow \alpha_2$ de α_1 y α_2. Pero entonces

$$S \Rightarrow aSbS \Rightarrow a\alpha_1 b\alpha_2$$

es una derivación de α. De manera análoga, si α comienza con b, existe una derivación de α. Esto concluye el paso inductivo y la demostración.

32. Reemplazamos cada producción

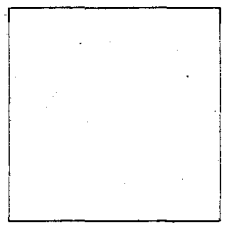
$$A \to x_1 \ldots x_n B,$$

donde $n > 1, x_i \in T$ y $B \in N$, con las producciones

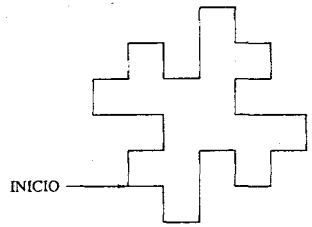
$$A \to x_1 A_1$$
$$A_1 \to x_2 A_2$$
$$\vdots$$
$$A_{n-1} \to x_n B,$$

donde $A_1, \ldots, A_{n-1}$ son símbolos no terminales adicionales.

35. $S \Rightarrow D + D + D + D \Rightarrow d + d + d + d$

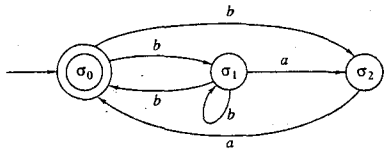


$S \Rightarrow D + D + D + D$
$\Rightarrow D + D - D - DD + D + D - D +$
$D + D - D - DD + D + D - D +$
$D + D - D - DD + D + D - D +$
$D + D - D - DD + D + D - D$
$\Rightarrow d + d - d - dd + d + d - d +$
$d + d - d - dd + d + d - d +$
$d + d - d - dd + d + d - d +$
$d + d - d - dd + d + d - d$

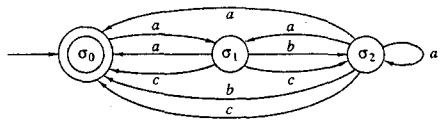


Sección 10.4

1.



4.



6. $I = \{a, b\}$; $S = \{\sigma_0, \sigma_1, \sigma_2\}$; $A = \{\sigma_1, \sigma_2\}$; estado inicial $= \sigma_0$

S \ I	a	b
σ_0	$\{\sigma_1, \sigma_2\}$	$\emptyset$
σ_1	$\{\sigma_1\}$	$\{\sigma_0, \sigma_2\}$
σ_2	$\emptyset$	$\emptyset$

9. $I = \{a, b\}$; $S = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$; $A = \{\sigma_3\}$; estado inicial $= \sigma_0$

S \ I	a	b
σ_0	$\{\sigma_0\}$	$\{\sigma_0, \sigma_1\}$
σ_1	$\{\sigma_2\}$	$\emptyset$
σ_2	$\emptyset$	$\{\sigma_3\}$
σ_3	$\{\sigma_3\}$	$\{\sigma_3\}$

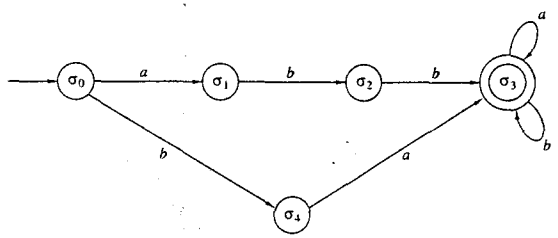
11. [Para el ejercicio 5] $N = \{\sigma_0, \sigma_1, \sigma_2\}$; $T = \{a, b\}$,

$$\sigma_0 \to a\sigma_1, \quad \sigma_0 \to b\sigma_0, \quad \sigma_1 \to a\sigma_0, \quad \sigma_1 \to b\sigma_2,$$
$$\sigma_2 \to b\sigma_1, \quad \sigma_2 \to a\sigma_0, \quad \sigma_2 \to \lambda$$

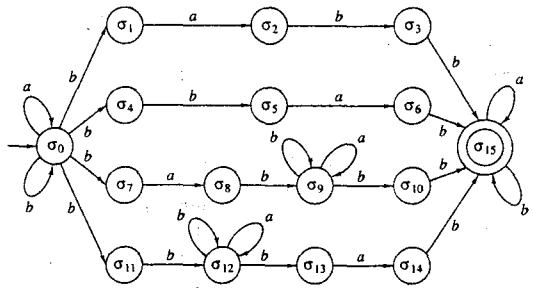
14. No. Para los tres primeros caracteres, *bba*, los movimientos están determinados y llegamos a C. A partir de C, ninguna arista contiene una a; por lo tanto, *bbabab* no es aceptada.

17. Sí. El camino $(\sigma, \sigma, \sigma, \sigma, C, C)$, que representa a la cadena *aaaab*, termina en C, que es un estado de aceptación.

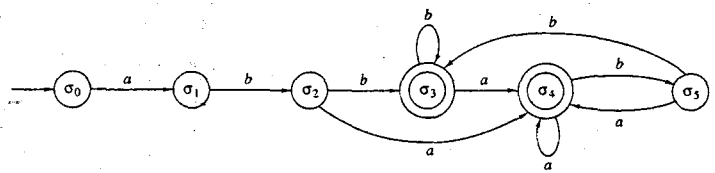
21.



24.



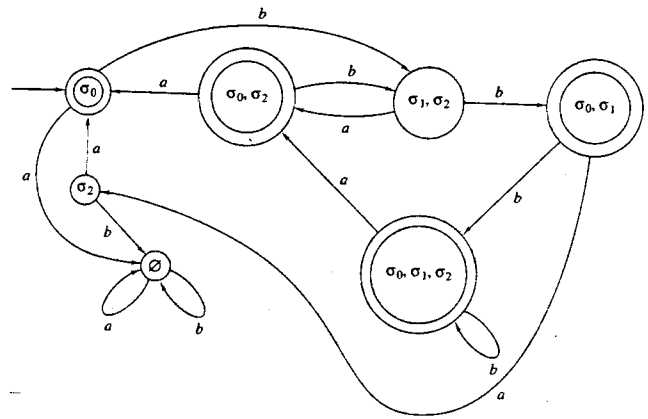
27.

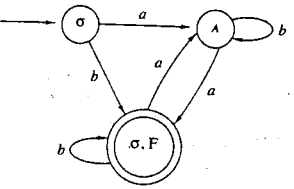


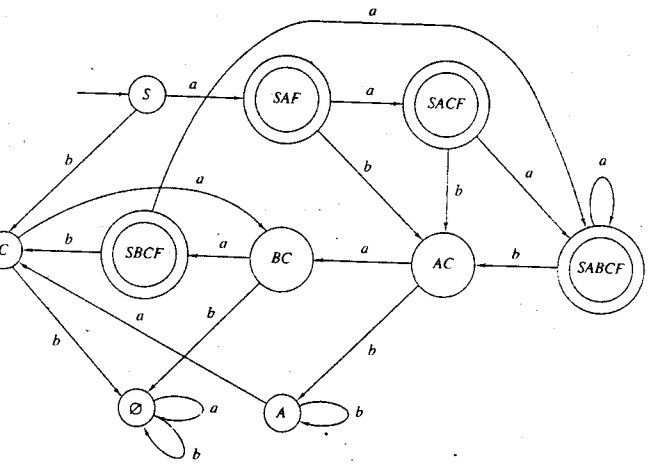
30. [Para el ejercicio 21] $\sigma_0 \to a\sigma_1$, $\sigma_0 \to b\sigma_4$, $\sigma_1 \to b\sigma_2$, $\sigma_2 \to b\sigma_3$, $\sigma_3 \to a\sigma_3$, $\sigma_3 \to b\sigma_3$, $\sigma_4 \to a\sigma_3$, $\sigma_3 \to \lambda$

ección 10.5

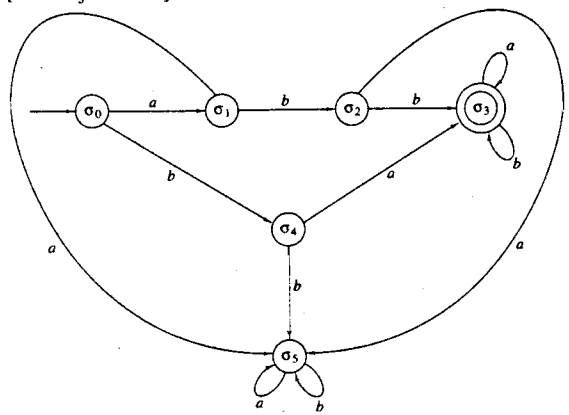
. [Para el ejercicio 1]





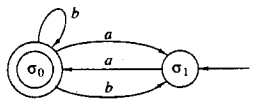


7. [Para el ejercicio 21]

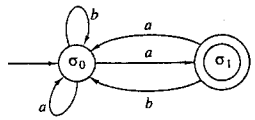


10. La figura 10.5.7 acepta la cadena ba^n, $n \geq 1$, y las cadenas que terminan en b^2 o aba^n, $n \geq 1$. Utilizamos el ejemplo 10.5.8 para ver que la figura 10.5.9 acepta la cadena a^nb, $n \geq 1$, y las cadenas que comienzan con b^2 o a^nba, $n \geq 1$.

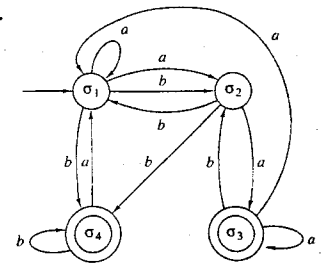
11.



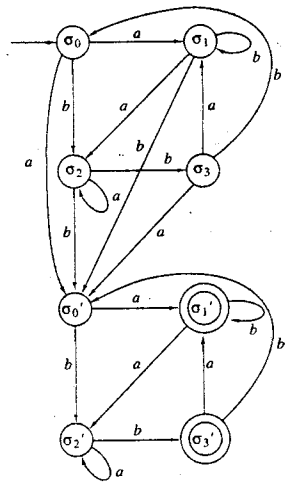
14.



17.



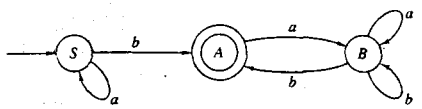
20.



22. $\sigma_0 \to a\sigma_1$, $\sigma_0 \to b\sigma_2$, $\sigma_0 \to a$, $\sigma_1 \to a\sigma_0$, $\sigma_1 \to a\sigma_2$, $\sigma_1 \to b\sigma_1$, $\sigma_1 \to b$, $\sigma_2 \to b\sigma_0$

25. Suponga que L es regular. Entonces existe un autómata de estado finito A tal que $L = \text{Ac}(A)$. Supongamos que A tiene k estados. Considere la cadena a^kbba^k y argumente como en el ejemplo 10.5.6.

28. La afirmación es falsa. Considere el lenguaje regular $L = \{a^n b \mid n \geq 0\}$, el cual es aceptado por el autómata de estado finito



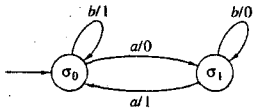
El lenguaje

$$L' = \{u^n \mid u \in L, n \in \{1, 2, \ldots\}\}$$

no es regular. Supongamos que L' es regular. Entonces existe un autómata de estado finito A que acepta a L'. En particular, A acepta a $a^n b$ para cada n. Esto implica que para n suficientemente grande, el camino que representa a $a^n b$ contiene un ciclo de longitud k. Como A acepta a $a^n b a^n b$, A también acepta a $a^{n+k} b a^n b$, lo que es una contradicción.

Capítulo 10 Autoevaluación

1.

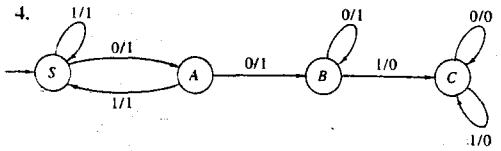


2. $I = \{a, b\}$; $O = \{0, 1\}$; $S = \{S, A, B\}$; estado inicial $= S$

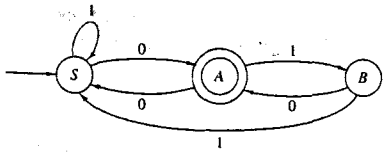
		f		g
S \ I		a	b	a b
S		A	A	0 0
A		S	B	1 1
B		A	B	1 0

3. 1101

4.

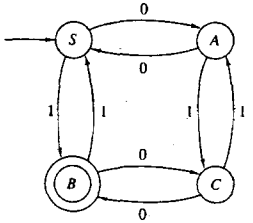


5.



6. Sí

7.



8. Cada 0 va seguido de un 1.

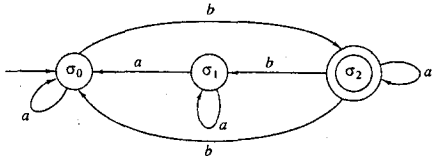
9. Libre de contexto

10. $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaaAbbbb \Rightarrow aaaaAbbbb \Rightarrow aaaabbbb$

11. $a^i b^j$, $j \leq 2 + i$, $j \geq 1$, $i \geq 0$

12. $S \rightarrow ASB$, $S \rightarrow AB$, $AB \rightarrow BA$, $BA \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$

13.



14. $I = \{a, b\}$; $S = \{\sigma_0, \sigma_1, \sigma_2\}$; $A = \{\sigma_0\}$; estado inicial $= \sigma_0$

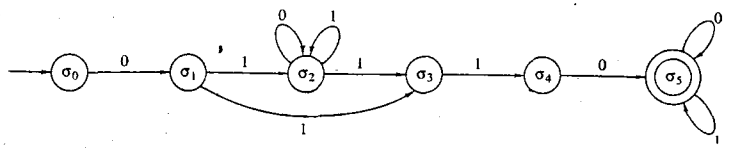
S \ I	a	b
σ_0	$\{\sigma_0, \sigma_1\}$	$\emptyset$
σ_1	$\emptyset$	$\{\sigma_2\}$
σ_2	$\{\sigma_0, \sigma_2\}$	$\{\sigma_2\}$

15. Sí, pues el camino

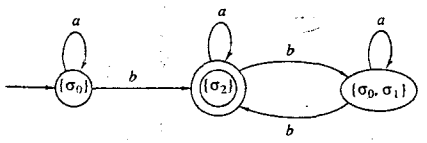
$$(\sigma_0, \sigma_0, \sigma_1, \sigma_2, \sigma_2, \sigma_2\, \sigma_2, \sigma_0)$$

representa a *aabaaba* y σ_0 es un estado de aceptación.

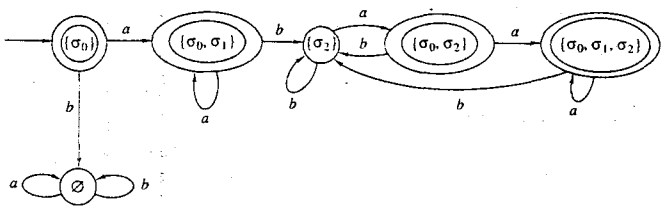
16.



17.



18.



19. Combinamos los autómatas de estado finito no deterministas que aceptan a L_1 y L_2 de la siguiente forma. Sea S el estado inicial de L_2. Para cada arista de la forma (S_1, S_2) etiquetada a en L_1, donde S_2 es un estado de aceptación, agregamos una arista (S_1, S) etiquetada a. El estado inicial del autómata de estado finito no determinista es el estado inicial de L_1. Los estados de aceptación del autómata de estado finito no determinista son los estados de aceptación de L_2.

20. Sea A' un autómata de estado finito no determinista que acepta un lenguaje regular, el cual no contiene a la cadena nula. Agregue un estado F. Para cada arista, (σ, σ') con la etiqueta a en A', donde σ' es un estado de aceptación, agregamos la arista (σ, F) con la etiqueta a. F será el único estado de aceptación. El autómata de estado finito no determinista A tiene un estado de aceptación. Afirmamos que $\text{Ac}(A) = \text{Ac}(A')$.

Mostraremos que $\text{Ac}(A) \subseteq \text{Ac}(A')$. [El argumento en el sentido $\text{Ac}(A') \subseteq \text{Ac}(A)$ es similar y se omite.] Supongamos que $\alpha \in \text{Ac}(A)$. Existe un camino

$$(\sigma_0, \sigma_1, \ldots, \sigma_{n-1}, \sigma_n)$$

que representa a α en A, con σ_n un estado de aceptación. Como $\alpha \neq \lambda$, existe un último símbolo a en α. Así, la arista (σ_{n-1}, σ_n) tiene la etiqueta a. Ahora, el camino

$$(\sigma_0, \sigma_1, \ldots, \sigma_{n-1}, F)$$

representa α en A' y termina en un estado de aceptación. Por lo tanto, $\alpha \in \text{Ac}(A')$.

Para ver que la afirmación es falsa para un lenguaje regular arbitrario, considere el lenguaje regular

$$L = \{\lambda\} \cup \{0^i \mid i \text{ es impar}\}$$

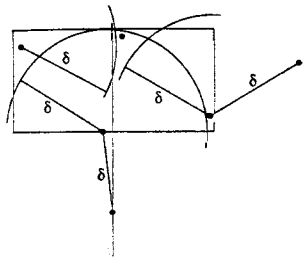
y un autómata de estado finito no determinista A con estado inicial S que acepta L. Como $\lambda \in L$, S es un estado de aceptación. Si S tiene un lazo con la etiqueta 0, entonces A acepta todas las cadenas de ceros; por lo tanto, no existe un lazo en S con la etiqueta 0. Así, existe una arista (S, S'), $S \neq S'$, con la etiqueta 0. Como $0 \in L$, S' es de aceptación. Por lo tanto, A tiene al menos dos estados de aceptación.

Sección 11.1

1. Los 16 puntos ordenados según su abscisa son: (1, 2), (1, 5), (1, 9), (3, 7), (3, 11), (5, 4), (5, 9), (7, 6), (8, 4), (8, 7), (8, 9), (11, 3), (11, 7), (12, 10), (14, 7), (17, 10), de modo que el punto divisor es (7, 6). A continuación determinamos $\delta_L = \sqrt{8}$, la distancia mínima entre los puntos del lado izquierdo (1, 2), (1, 5), (1, 9), (3, 7), (3, 11), (5, 4), (5, 9), (7, 6) y $\delta_R = 2$, la distancia mínima entre los puntos del lado derecho (8, 4), (8, 7), (8, 9), (11, 3), (11, 7), (12, 10), (14, 7), (17, 10). Así, $\delta = \text{mín}\{\delta_L, \delta_R\} = 2$. Los puntos de la franja vertical, ordenados según su coordenada y son (8, 4), (7, 6), (8, 7), (8, 9). En este caso, comparamos cada punto de la franja con todos los puntos siguientes. Las distancias de (8, 4) a (7, 6), (8, 7), (8, 9) no son menores que 2, por lo que no hay necesidad de actualizar δ. La distancia de (7, 6) a (8, 7) es $\sqrt{2}$, de modo que δ se actualiza como $\sqrt{2}$. La distancia de (7, 6) a (8, 9) y de (8, 7) a (8, 9) es mayor que $\sqrt{2}$, de modo que δ sigue siendo $\sqrt{2}$. Por lo tanto, la distancia entre el par más cercano es $\sqrt{2}$.

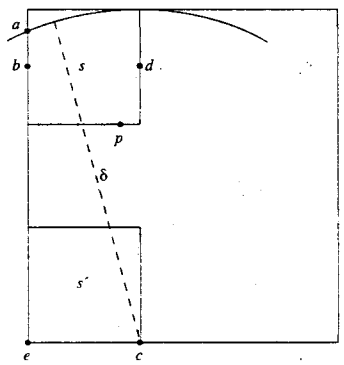
4. Considere el caso extremo en que todos los puntos están sobre una misma recta vertical.

7.



10. Sea B alguno de los cuadrados $\delta \times \delta$ izquierdo o derecho que forman al rectángulo δ por 2δ (véase la figura 11.1.2). Argumentamos por contradicción y suponemos que B contiene cuatro o más puntos. Dividimos B en cuatro cuadrados $\delta/2 \times \delta/2$, como se muestra en la figura 11.1.3. Entonces, cada uno de estos cuatro cuadrados contiene a lo más un punto, por lo que hay exactamente un punto. En lo sucesivo nos referiremos a estos cuatro cuadrados como los subcuadrados de B.

La figura



muestra la siguiente construcción. Reducimos el tamaño de los subcuadrados, de ser posible, de modo que

- Cada subcuadrado contenga un punto.
- Los subcuadrados tengan el mismo tamaño.
- Los subcuadrados sean lo más pequeños posible.

Como al menos un punto no está en una esquina de B, los subcuadrados no se colapsan en puntos, y así, al menos un punto está sobre un lado de un subcuadrado s interior a B. Elegimos tal punto y lo llamamos p. Elegimos un subcuadrado s' cercano a p. Etiquetamos los dos puntos esquina de s' del lado más alejado de p como e y c. Trazamos un círculo de radio δ con centro en c; sea a el punto (no esquina) donde este círculo corta al lado de s. Observe que este círculo corta un lado de s en un punto que no es esquina. Elegimos un punto b en s del mismo lado que a, entre a y e. Sea d el punto correspondiente en el lado opuesto de s. Ahora, la longitud del diámetro del rectángulo $R = bdce$ es menor que δ; por lo tanto, R contiene a lo más un punto. Esto es una contradicción, pues R contiene a p y al punto en s'. Por lo tanto, B contiene a lo más tres puntos.

Sección 11.2

1. Entrada: $x_1, \ldots, x_n$ y n

   Salida: "Sí" si $x_1, \ldots, x_n$ son distintos, y "No" en caso contrario

```
procedure check_distinct(x, n)
 sort x_1, ..., x_n
 for i := 1 to n − 1 do
  if x_i := x_{i+1} then
   return("No")
 return("Sí")
end check_distinct
```

4. Inicializamos una lista L como vacía. Determinamos el vértice v que no tiene aristas de entrada y lo agregamos al final de L. Eliminamos v y todas las aristas incidentes en él. Repetimos el proceso; es decir, determinamos el vértice v sin aristas de entrada y lo agregamos al final de L. Continuamos de esta forma hasta agotar los vértices. La salida es L.

Sección 11.3

1. Sea L la recta horizontal que pasa por p_1. Por la elección de p_1, ningún punto de S está arriba de L. Si p_1 es el único punto de S sobre L, p_1 es un punto de la cubierta. Si otros puntos de S están en L, todos están a la derecha de p_1 (por la elección de p_1). En este caso, si giramos L en dirección de las manecillas del reloj un poco en torno de p_1, L sólo contendrá a p_1 y todos los demás puntos de S estarán arriba de L. De nuevo, concluimos que p_1 es un punto de la cubierta.

4. Los puntos [ordenados con respecto de (7,1)] son (7, 1), (10, 1), (16, 4), (12, 3), (14, 5), (16, 10), (13, 8), (10, 5), (10, 9), (10, 13), (7, 7), (7, 13), (6, 10), (3, 13), (4, 8), (1, 8), (4, 4), (2, 2). La siguiente tabla muestra cada una de las tercias examinadas en el ciclo while, si forma un giro hacia la izquierda, y la acción por realizar con respecto de la tercia:

Tercia	*¿Giro hacia la izquierda?*	*¿Descartar el punto medio?*
(7, 1), (10, 1), (16, 4)	Sí	No
(10, 1), (16, 4), (12, 3)	Sí	No
(16, 4), (12, 3), (14, 5)	No	Sí
(10, 1), (16, 4), (14, 5)	Sí	No
(16, 4), (14, 5), (16, 10)	No	Sí
(10, 1), (16, 4), (16, 10)	Sí	No
(16, 4), (16, 10), (13, 8)	Sí	No
(16, 10), (13, 8), (10, 5)	Sí	No
(13, 8), (10, 5), (10, 9)	No	Sí
(16, 10), (13, 8), (10, 9)	No	Sí
(16, 4), (16, 10), (10, 9)	Sí	No
(16, 10), (10, 9), (10, 13)	No	Sí
(16, 4), (16, 10), (10, 13)	Sí	No
(16, 10), (10, 13), (7, 7)	Sí	No
(10, 13), (7, 7), (7, 13)	No	Sí
(16, 10), (10, 13), (7, 13)	Sí	No
(10, 13), (7, 13), (6, 10)	Sí	No
(7, 13), (6, 10), (3, 13)	No	Sí
(10, 13), (7, 13), (3, 13)	No	Sí
(16, 10), (10, 13), (3, 13)	Sí	No
(10, 13), (3, 13), (4, 8)	Sí	No
(3, 13), (4, 8), (1, 8)	No	Sí
(10, 13), (3, 13), (1, 8)	Sí	No
(3, 13), (1, 8), (4, 4)	Sí	No
(1, 8), (4, 4), (2, 2)	No	Sí
(3, 13), (1, 8), (2, 2)	Sí	No

La cubierta convexa es (7, 1), (10, 1), (16, 4), (16, 10), (10, 13), (3, 13), (1, 8), (2, 2).

7. Después de determinar $p_1, \ldots, p_i$, la marcha de Jarvis halla el punto p_{i+1} tal que p_{i-1}, p_i, p_{i+1} forman el menor giro hacia la izquierda. Esto implica que si la línea L que pasa por p_i, p_{i+1} se gira en dirección de las manecillas del reloj ligeramente en torno de p_i, L sólo contendrá a p_i y los demás puntos de S estarán en un lado de L. Así, p_i es un punto de la cubierta. Por construcción, la marcha de Jarvis determina todos los puntos de la cubierta. Así, la marcha de Jarvis realmente halla la cubierta convexa.

10. Sí. La marcha de Jarvis es más rápida cuando la mayor cantidad de puntos no están en la cubierta.

Capítulo 11 Autoevaluación

1. Los 18 puntos ordenados según su abscisa son: (1, 8), (2, 2), (3, 13), (4, 4), (4, 8), (6, 10), (7, 1), (7, 7), (7, 13), (10, 1), (10, 5), (10, 9), (10, 13), (12, 3), (13, 8), (14, 15). (16, 4), (16, 10), de modo que el punto divisor es (7, 13). A continuación determinamos $\delta_L = \sqrt{8}$, la distancia mínima entre los puntos del lado izquierdo (1, 8), (2, 2), (3, 13), (4, 4), (4, 8), (6, 10), (7, 1), (7, 7), (7, 13), y $\delta_R = \sqrt{5}$, la distancia mínima entre los puntos del lado derecho (10, 1), (10, 5), (10, 9), (10, 13), (12, 3), (13, 8), (14, 5), (16, 4), (16, 10). Así, $\delta = \text{mín}\{\delta_L, \delta_R\} = \sqrt{5}$. Los puntos de la franja vertical, ordenados según su coordenada y, son (7, 1), (7, 7), (6, 10), (7, 13). En este caso, comparamos cada punto de la franja con todos los puntos siguientes. Como ningún par está más cerca que $\sqrt{5}$, el algoritmo no actualiza a δ. Por lo tanto, la distancia entre el par más cercano es $\sqrt{5}$.

2. Si reemplazamos "tres" por "dos", cuando hay tres puntos, el algoritmo se llamaría de manera recursiva con entradas de tamaños 1 y 2. Pero un conjunto de un único punto no tiene pareja, mucho menos una pareja más cercana.

3. Cada cuadro $\delta/2 \times \delta/2$ contiene a lo más un punto, de modo que existen a lo más cuatro puntos en la mitad inferior del rectángulo.

4. $\Theta(n(\lg n)^2)$

5. Sea t_n el tiempo en el peor de los casos para un algoritmo que determina un par más cercano, entre n elementos del espacio de dimensión d. Entonces, t_n es también el tiempo asintótico en el peor de los casos para un algoritmo CP que regresa la distancia entre un par más cercano de puntos en el espacio de dimensión d, ya que podemos agregar una línea al algoritmo original que calcule y regrese la distancia entre un par más cercano. Considere el siguiente algoritmo que resuelva el problema de determinar si existen duplicados entre n números:

```
procedure dup(x, n)
 // La entrada es x_1, ..., x_n.
 // Los datos se transforman en puntos del espacio
   de dimensión d.
 for i := 1 to n do
   a_i := (x_i, 0, ..., 0)
 if CP(a, n) = 0 then
   return("Con duplicados")
 else
   return("Sin duplicados")
end dup
```

El tiempo en el peor de los casos para t'_n para *dup* es el tiempo necesario para el ciclo for más el tiempo en el peor de los casos para CP; es decir,

$$t'_n = n + 6n.$$

Por el teorema 11.2.1,

$$Cn \lg n \leq t'n.$$

Al combinar estas dos últimas afirmaciones, obtenemos

$$\Omega(n \lg n) = Cn \lg n - n \leq t'_n - n = t_n.$$

6. Podemos agregar una línea sin modificar el tiempo asintótico de un algoritmo que determine todos los pares más cercanos, de modo que determine la distancia entre un par más cercano. Por el corolario 11.2.2, esto requiere un tiempo $\Omega(n \lg n)$.

7. El tiempo en el peor de los casos de cualquier algoritmo que determine si n números reales son todos iguales es $\Omega(n)$, pues cualquier algoritmo debe examinar cada elemento al menos una vez. Esta cota inferior es justa, pues el siguiente algoritmo resuelve el problema en un tiempo $\Theta(n)$:

```
procedure all_equal(x, n)
 // determinar si x_1, ..., x_n son iguales
 for i := 1 to n - 1 do
  if x_i ≠ x_{i+1} then
   return("No todos iguales")
 return("Todos iguales")
end all_equal
```

8. La afirmación es consecuencia del hecho de que un algoritmo de este tipo se puede modificar sin alterar su tiempo asintótico en el peor de los casos, para determinar si la entrada tiene duplicados y, por el teorema 11.2.1, cualquier algoritmo que determine si existen duplicados tiene un tiempo en el peor de los casos $\Omega(n \lg n)$. Los duplicados existen si y sólo si la distancia entre cada par de salida es cero; así, sólo debemos verificar un par para ver si existen duplicados o no.

9. Sea L la recta vertical que pasa por p. Por la elección de p, ningún punto de S está a la derecha de L. Si p es el único punto de S sobre L, p es un punto de la cubierta. Si otros puntos de S están en L, todos están debajo de p. En este caso, si giramos L en dirección de las manecillas del reloj un poco en torno de p, L sólo contendrá a p y todos los demás puntos de S estarán a la izquierda de L. De nuevo, concluimos que p es un punto de la cubierta.

10. Sea L el segmento de recta que une p con q. Sea L' la recta que pasa por p perpendicular a L. No puede haber otro punto r de S sobre L' o en el lado de L' opuesto a q, ya que de existir un punto r de ese tipo, la distancia de r a q sería mayor que la distancia de p a q, lo cual es imposible. Así, p es un punto de la cubierta. De manera similar, q es un punto de la cubierta.

11. Los puntos [ordenados con respecto de (1, 2)] son (1, 2), (11, 3), (8, 4), (14, 7), (5, 4), (11, 7), (17, 10), (7, 6), (8, 7), (12, 10), (8, 9), (5, 9), (3, 7), (3, 11), (1, 5), (1, 9). La siguiente tabla muestra cada tercia examinada en el ciclo while, si realiza un giro hacia la izquierda, y la acción realizada con respecto de la tercia:

Tercia	*¿Giro hacia la izquierda?*	*¿Descartar el punto medio?*
(1, 2), (11, 3), (8, 4)	Sí	No
(11, 3), (8, 4), (14, 7)	No	Sí
(1, 2), (11, 3), (14, 7)	Sí	No
(11, 3), (14, 7), (5, 4)	Sí	No
(14, 7), (5, 4), (11, 7)	No	Sí
(11, 3), (14, 7), (11, 7)	Sí	No
(14, 7), (11, 7), (17, 10)	No	Sí
(11, 3), (14, 7), (17, 10)	No	Sí
(1, 2), (11, 3), (17, 10)	Sí	No
(11, 3), (17, 10), (7, 6)	Sí	No
(17, 10), (7, 6), (8, 7)	No	Sí
(11, 3), (17, 10), (8, 7)	Sí	No
(17, 10), (8, 7), (12, 10)	No	Sí
(11, 3), (17, 10), (12, 10)	Sí	No
(17, 10), (12, 10), (8, 9)	Sí	No
(12, 10), (8, 9), (5, 9)	No	Sí
(17, 10), (12, 10), (5, 9)	Sí	No
(12, 10), (5, 9), (3, 7)	Sí	No
(5, 9), (3, 7), (3, 11)	No	Sí
(12, 10), (5, 9), (3, 11)	No	Sí
(17, 10), (12, 10), (3, 11)	No	Sí
(11, 3), (17, 10), (3, 11)	Sí	No
(17, 10), (3, 11), (1, 5)	Sí	No
(3, 11), (1, 5), (1, 9)	No	Sí
(17, 10), (3, 11), (1, 9)	Sí	No

La cubierta convexa es (1, 2), (11, 3), (17, 10), (3, 11), (1, 9).

12. Ejecute la parte del algoritmo de Graham posterior al ordenamiento de los demás puntos.

Sección Apéndice

1. $\begin{pmatrix} 2+a & 4+b & 1+c \\ 6+d & 9+e & 3+f \\ 1+g & -1+h & 6+i \end{pmatrix}$,

2. $\begin{pmatrix} 5 & 7 & 7 \\ -7 & 10 & -1 \end{pmatrix}$

5. $\begin{pmatrix} 3 & 18 & 27 \\ 0 & 12 & -6 \end{pmatrix}$

8. $\begin{pmatrix} -2 & -35 & -56 \\ -7 & -18 & 13 \end{pmatrix}$

9. $\begin{pmatrix} 18 & 10 \\ 14 & -6 \\ 23 & 1 \end{pmatrix}$

12. (-4)

14. (a) $2 \times 3, 3 \times 3, 3 \times 2$

(b) $AB = \begin{pmatrix} 33 & 18 & 47 \\ 8 & 9 & 43 \end{pmatrix}$

$AC = \begin{pmatrix} 16 & 56 \\ 14 & 63 \end{pmatrix}$

$CA = \begin{pmatrix} 4 & 18 & 38 \\ 0 & 0 & 0 \\ 2 & 17 & 75 \end{pmatrix}$

$AB^2 = \begin{pmatrix} 177 & 215 & 531 \\ 80 & 93 & 323 \end{pmatrix}$

$BC = \begin{pmatrix} 18 & 65 \\ 34 & 25 \\ 12 & 54 \end{pmatrix}$

17. Sean $A = (b_{ij})$, $I_n = (a_{jk})$, $AI_n = (c_{ik})$. Entonces

$$c_{ik} = \sum_{j=1}^{n} b_{ij}\, a_{jk} = b_{ik} a_{kk} = b_{ik}.$$

Por lo tanto, $AI_n = A$. De manera análoga, $I_n A = A$.

20. La solución es $X = A^{-1}C$.

ÍNDICE

A

Acoplamiento, 478
  completo, 478
  máximo, 478
  red, 479
Activación de una transición, 488
Adleman, L. M., 190
Afirmación
  cuantificada existencialmente, 21
  cuantificada universalmente, 20
Aho, A., 193, 437
Ainslie, T., 429, 450
Akl, S. G., 371, 450
al-Khowārizmī, 142
Alfanumérico, 206
Álgebra booleana, 500, 512, 516
  enunciado dual, 519
  ley de involución, 518
  leyes asociativas, 516
  leyes conmutativas, 516
  leyes de absorción, 518
  leyes de acotación, 518
  leyes de complemento, 516
  leyes de De Morgan, 518
  leyes de idempotencia, 518
  leyes del 0/1, 518
Álgebra booleana (*continuación*)
  leyes del neutro, 516
  leyes distributivas, 516
Algoritmo, 589
  análisis, 166, 287
  árbol de expansión mínimo, 401, 405, 407
  búsqueda binaria, 289
  búsqueda en un árbol de expansión a lo ancho, 394
  búsqueda en un árbol de expansión en profundidad, 394
  búsqueda en una sucesión no ordenada, 176
  cálculo de un exponencial, 297
  cálculo recursivo del máximo común divisor, 161
  caminata de un robot, 162
  codicioso, 403
  comentarios, 146
  cómo cubrir un tablero deficiente con triominós, 160
  complejidad, 166
  construcción de un árbol de búsqueda binaria, 413
  construcción de un código óptimo de Huffman, 381
  de búsqueda binaria, 289
  de Euclides, 151, 155, 191; *véase también* Algoritmo, cálculo recursivo del máximo común divisor
  de Euclides, análisis, 186
  de Graham para calcular la cubierta convexa, 605
  de Kruskal, 407
  de la caminata del robot, 162
  de ordenamiento por fusión, 291, 293

Algoritmo (*continuación*)
de ordenamiento por inserción, 299
de ordenamiento por inserción binaria, 453
de ordenamiento por selección, 287
de Prim, 401, 405
de recorrido en entreorden, 418
de recorrido en posorden, 418
de recorrido en preorden, 416
del camino más corto, 338
del camino más corto de Dijkstra, 338, 394
en paralelo, 311
en serie, 311
entrada, 143
evaluación polinomial, 303
fusión de dos sucesiones, 292
generación de combinaciones, 231
generación de permutaciones, 233
marcha de Jarvis, 608
para búsqueda en una sucesión no ordenada, 176
para calcular el interés compuesto, 259
para calcular *n* factorial, 159
para construir un árbol de búsqueda binaria, 413
para construir un código de Huffman óptimo, 381
para cubrir con mosaicos un tablero deficiente con triominós, 160
para determinar el complemento a dos, 541
para determinar el elemento más grande y el más pequeño de una sucesión, 298
para determinar el más grande, 147, 148
para determinar el máximo, 145
para determinar la distancia entre un par de puntos cercanos, 596
para determinar la suma máxima de valores consecutivos, 182
para determinar un flujo máximo en una red, 466
para determinar un primo mayor que un entero dado, 149
para generar combinaciones, 231
para generar permutaciones, 233
para la búsqueda en profundidad para un árbol de expansión, 394
para la evaluación de polinomios, 303
para la fusión de dos sucesiones, 292
para resolver el problema de las cuatro reinas mediante retroceso, 396
Algoritmo (*continuación*)
para verificar si dos árboles binarios son isomorfos, 436
para verificar si se acepta una cadena, 558
para verificar si un entero positivo es primo, 149
paralelo, 311
procedimiento minimáx, 442
recursivo, 157, 258
recursivo, caso base, 160
recursivo para calcular el máximo común divisor, 161
salida, 143
seguimiento, 143
simplex, 496
solución del problema de las cuatro reinas mediante retroceso, 396
tiempo en el caso promedio, 166, 173
tiempo en el mejor de los casos, 166, 173
tiempo en el peor de los casos, 166, 173
Algoritmo de Graham para calcular la cubierta convexa, 605
tiempo en el peor de los casos, 606
Altura de un árbol, 378
Análisis de algoritmos, 166, 287
Ancestro de un vértice, 385
Antecedente, 8
Appel, K., 365, 371
Árbol, 377
algoritmo para su construcción, 413
altura, 378
ancestro, 385
binario, 408
centro, 384
con raíz, 377
de búsqueda binaria, 411
de decisión, 422
de definición jerárquica, 379
de expansión, 392
de expansión mínimo, 400
de un juego, 440
descendiente, 385
hermano, 385
hijo, 385
hoja, 385
isomorfo, 429
libre, 377
m-ario completo, 415
padre, 385

Árbol (*continuación*)
recorrido, 415
subárbol, 385
vértice de ramificación, 385
vértice interno, 385
vértice terminal, 385
Árbol binario, 408
algoritmo para verificar un isomorfismo, 436
completo, 408
equilibrado, 415
hijo derecho, 408
hijo izquierdo, 408
isomorfo, 433
Árbol con raíz, 377
isomorfo, 431
Árbol de expansión, 392
búsqueda a lo ancho, 393
búsqueda en profundidad, 394
mínimo, 400
mínimo, algoritmo, 401, 407
Árbol de un juego, 440
búsqueda en el nivel *n*, 442
corte alfa, 444
corte beta, 444
poda alfa-beta, 444
procedimiento minimáx para su evaluación, 442
valor alfa, 444
valor beta, 444
Arco, 306, 307
Argumento, 38
combinatorio, 244
deductivo, 38
falacia, 38
no válido, 38
válido, 38
Arista, 306, 307
capacidad, 456
dirigida, 93
flujo, 457
incidente, 307
orientada en forma impropia, 463
orientada en forma propia, 463
paralela, 308
peso, 309
Aristas paralelas, 308
ASCII (Código estándar americano para el intercambio de información), 379
Atkins, D., 192
Atributo, 119
Autómata de estado finito, 554, 556
equivalente, 558
Autómata de estado finito no determinista, 573, 576
equivalente, 578
Autómatas de estado finito equivalentes, 558
Axioma, 34

B

Baase, S., 193, 450
Babai, L., 353
Barker, S. F., 59
Base de datos, 119; *véase también* Base de datos relacional
consulta, 120
Base de datos relacional, 118
atributo, 119
clave, 119
operador de fusión (join), 120
operador de proyecto, 120
operador de selección, 120
Base de un sistema numérico, 84
Bell, R. C., 449
Bentley, J., 186
Berge, C., 371, 450, 496
Berlekamp, E. R., 450
Bit, 84
Biyección, 130
Bloqueo, 492
BNF (Forma normal de Backus, forma de Backus-Naur), 564
Bondy, J. A., 371, 450
Boole, G., 500, 542
Bosque, 390
Braille, L., 203
Brassard, G., 193, 302, 371
Brualdi, R. A., 253, 270, 280
Búsqueda a lo ancho, 393
para un algoritmo de árbol de expansión, 394
Búsqueda a profundidad, 394
Búsqueda en el nivel *n*, 442

C

Cadena, 700
  aceptada, 556, 578
  concatenación, 79
  derivable, 563, 569
  directamente derivable, 563, 569
  entrada, 549
  longitud, 79
  nula, 79
  salida, 549
  subcadena, 83
Cálculo de algoritmos exponenciales, 297
Camino, 306, 316
  cerrado, 328
  cruce, 364
  longitud, 309, 316
  representación, 557, 578
  simple, 319
Capacidad
  de una arista, 456
  de una corte, 474
Cara en una gráfica plana, 359
Carmony, L., 55
Carroll, J., 589
Caso base de un algoritmo recursivo, 160
Catalan, E. C., 219
Centro de un árbol, 384
Cerradura transitiva de una relación, 110
Chartrand, G., 371
Chu, I., P., 51
Ciclo, 319, 329
  de Euler, 320, 328, 331, 333
  for, 148
  fundamental, 399
  hamiltoniano, 331
  simple, 319
  while, 146
Ciclo de Euler, 320, 328, 331, 333
  dirigido, 328
Cifrar un mensaje, 189
Circuito
  combinatorio, 501
  de conmutación, 507
  en serie, 508
Circuito (*continuación*)
  en una gráfica, 319; *véase también* Ciclo
  equivalente, 512
  flip-flop, 550
  flip-flop activado-desactivado, 550
  flip-flop set-reset, SR, 550
  integrado, 536
  medio sumador. *Véase* Circuito semisumador
  paralelo, 508
  propiedades, 509
  puente, 515
  secuencial, 501, 546
  semisumador, 536
  sumador completo, 537
  sumador en serie, 547
Circuito(s) combinatorio(s), 501, 512
  equivalente(s), 512
  propiedades(s), 509
Circuito(s) de conmutación, 507
  equivalente(s), 514
Clase de equivalencia, 106
Cláusula, 42
Clave, 119
  privada, 190
  pública, 190
Cociente, 151
Codd, E. R., 119, 136
Código de Huffman, 379
  construcción del código óptimo, 381
Código Gray, 333, 351
Código Universal de Productos (UPC), 133
Coeficiente binomial, 243
Cohen, D. I. A., 589
Colisión, 128
Coloración de una gráfica, 365
Combinación, 214
  algoritmo para su generación, 231
  de orden *r*, 214
  generalizada, 235
Combinaciones generalizadas, 235
Comentario en un algoritmo, 146
Comparables, 97
Complejidad de algoritmos, 166
Complemento
  de una gráfica simple, 357

Complemento (*continuación*)
  en álgebra booleana, 517
  relativo de un conjunto, 66
Complemento a dos, 541
  algoritmo para determinar, 541
Complemento de un conjunto, 67
  relativo, 66
Componente de una gráfica, 318
Composición
  de funciones, 130
  de relaciones, 99
Compuerta, 501, 531
  AND, 501
  conjunto funcionalmente completo, 531
  inversor, 501
  NAND, 532
  NOR, 540
  NOT, 501
  OR, 501
Computadora
  en paralelo, 311
  en serie, 311
Concatenación de cadenas, 79
Conclusión, 8, 38
Condición
  inicial, 257
  necesaria, 10
  suficiente, 10
Conjunción, 2
Conjunto, 64
  ajeno, 66
  ajeno por pares, 66
  complemento, 67
  complemento relativo, 66
  diferencia, 66
  diferencia simétrica, 72
  equivalente, 135
  funcionalmente completo de compuertas, 531
  igual, 64
  independiente, 329
  intersección, 66, 68
  ley de involución, 68
  leyes asociativas, 67
  leyes conmutativas, 67
  leyes de absorción, 68
Conjunto (*continuación*)
  leyes de acotación, 68
  leyes de De Morgan, 68
  leyes de idempotencia, 68
  leyes del 0/1, 68
  leyes del complemento, 68
  leyes del neutro y del idéntico, 67
  leyes distributivas, 67
  método iterativo, 270
  método para una recurrencia homogénea lineal, 275
  nulo, 64
  partición, 69, 104, 224
  potencia, 65
  producto cartesiano, 69
  subconjunto, 65
  subconjunto propio, 65
  unión, 66, 68
  universal, 67
  universo, 67
  vacío, 64
Conjuntos
  ajenos, 66
  equivalentes, 135
  iguales, 64
  por pares, 66
Consecuente, 8
Conservación del flujo, 457
Consulta, 120
Contradicción, 36
Contraejemplo, 23
Contrapositiva, 15
Copi, I. M., 59
Copo de nieve de von Koch, 569
Cormen, T. H., 191, 193, 302, 371, 381, 414, 450
Corolario, 34
Corte, 473
  alfa, 444
  beta, 444
  capacidad, 474
  mínimo, 476
Crecimiento de poblaciones, 272, 277
Criptología, 189
Cruce, 364
Cuantificador, 18
  existencial, 21

Cuantificador (*continuación*)
universal, 20
Cubierta convexa, 601
algoritmo para calcular, 605, 608
Cubo deficiente, 54
Cubrir con mosaicos, 51, 160
Cuerpo de un lazo, 147
Cull, P., 302
Curva de Hilbert, 573

D

Date, C. J., 119, 136, 450
Davis, M. D., 59, 589
Deep Blue, 446
Deficiencia de una gráfica, 484
Definición, 34
Demostración, 34
directa, 36
indirecta, 36
por contradicción, 36
por contrapositiva, 37
resolución, 42
Demostración por resolución, 42
corrección, 45
por contradicción, 45
refutación completa, 45
Deo, N., 334, 371, 434, 440, 450, 496
Derivación, 563, 569
Descendiente de un vértice, 385
Descifrar un mensaje, 189
Desordenamiento, 287
Diagrama de transición, 549
Diámetro de una gráfica, 329
Diferencia
de conjuntos, 66
simétrica de conjuntos, 72
Digráfica, 93, 307; *véase también* Gráfica dirigida
arista dirigida, 93
de una relación, 93
lazo, 93
vértice, 93
Dijkstra, E. W., 338, 371, 495
Distancia entre vértices, 329
Disyunción, 3
Divide y vencerás, 157
Divisor, 151
Divisor común, 152
máximo, 152
Dominio
de una función, 125
de una relación, 92
del discurso, 19
Dominó, 240
Dossey, J. A., 59

E

Edelsbrunner, H., 608
Edgar, W. J., 59
Elemento, 487
conteo, 496
English, E., 192
Entrada, 143
Enunciado
de llamada, 149
de retorno, 146
dual, 519
Erbas, C., 397
Estado, 548, 556, 576
de aceptación, 554, 556, 576
inicial, 548, 556, 576
Estructura If-then, 146
Estructura If-then-else, 146
Euler, L., 320, 359
Even, S., 253, 333, 360, 362, 371, 450
Excentricidad de un vértice, 384
Excluyente, o. *Véase* O-exclusivo
Expresión
forma con todos los paréntesis, 420
forma entrefija, 419
forma posfija, 420
forma prefija, 420
Expresión booleana, 504, 542
igual, 511
Expresiones booleanas iguales, 511
Ezekiel, M., 302

F

Factorial, 157
algoritmo para calcular, 159
Factorización, 192
de enteros, 192
Falacia, 38
Fibonacci, L., 162
Flujo
algoritmo para determinar el flujo máximo, 466
conservación, 457
desde un vértice, 457
en una arista, 457
en una red, 457
hacia un vértice, 457
máximo, 462
valor, 458
Ford, L. R., 148
Forma
conjuntiva normal, 528
de Backus-Naur (FBN), 564
de una expresión con todos los paréntesis, 420
disyuntiva normal, 527
egipcia de una fracción, 53
entrefija de una expresión, 419
fuerte de la inducción matemática, 49
normal de Backus (FNB), 564
posfija de una expresión, 420
prefija de una expresión, 420
Fórmula de Euler para gráficas, 362
Fórmula de suma por partes, 83
Fowler, P. A., 320
Fractal, 570
Frey, P., 450
Fuente, 456
Fukunaga, K., 371
Función, 125
biyección, 130
booleana, 525, 542
cadena, 78
característica, 134
composición, 130
de Ackermann, 265
de disimilaridad, 310
de dispersión (hash), 127
Función (*continuación*)
del siguiente estado, 548, 556, 576
dominio, 125
evaluación, 442
imagen inversa, 133
inversa, 130
inyectiva, 129
operador binario, 131
operador unario, 131
orden, 168
proposicional, 19
rango, 125
salida, 548
sobre, 129
sucesión, 73
suprayectiva, 129
uno a uno, 129
Función proposicional, 19
dominio del discurso, 19

G

GAD (gráfica acíclica dirigida), 329
Gallier, J. H., 45
Gardner, M., 192, 193, 371
Genesereth, M. R., 45
Geometría computacional, 593
Gibbons, A., 371, 450
Goldberg, S., 302
Golomb, S. W., 51, 450
Gose, E., 371, 407
Grado
de entrada de un vértice, 328
de salida de un vértice, 328
de un vértice, 320
Graf, S., 377
Graff, M., 192
Gráfica, 306
acíclica, 387
acíclica dirigida, 329
arco, 306, 307
arista, 306, 307
aristas paralelas, 308
autocomplementaria, 357
bipartita, 312

Gráfica (*continuación*)
bipartita completa, 313
camino, 306, 316
camino cerrado, 328
camino simple, 319
ciclo, 319
ciclo de Euler, 320, 328, 331, 333
ciclo fundamental, 399
ciclo hamiltoniano, 331
ciclo simple, 319
circuito, 319
coloración, 365
complemento, 357
completa, 312
componente, 318
con pesos, 309
conexa, 316
conjunto independiente, 329
deficiencia, 484
diámetro, 329
digráfica, 307
dirigida, 307
dual, 365
fórmula de Euler, 362
gad (gráfica acíclica dirigida), 329
homeomorfa, 360
homomorfismo, 357
invariante, 353
isomorfa, 350
isomorfismo, 350
lazo, 308
marcada, 495
matriz de adyacencia, 344
matriz de incidencia, 348
matriz del ciclo fundamental, 399
no dirigida, 306
nodo, 306, 307
plana, 359
punto de articulación, 327
reducción en serie, 360
similaridad, 309
simple, 308
subgráfica, 317
vértice, 306, 307
vértice aislado, 308
Gráfica bipartita, 312
completa, 313
Gráfica dirigida, 307
ciclo de Euler dirigido, 328
Gráfica plana, 359
cara, 359
triangulación, 365
Gráfica simple, 308
autocomplementaria, 357
complemento, 357
Gráficas homeomorfas, 360
Graham, R. L., 59, 608
Gramática, 562
cadena derivable, 563
cadena derivable en forma directa, 563
con estructura de frases, 562
de tipo 1, 565
de tipo 2, 565
de tipo 3, 565
derivación, 563
equivalente, 568
lenguaje generado, 563
libre de contexto, 565
Lindenmayer, interactiva libre de contexto, 568
producción, 562
sensible al contexto, 565
símbolo inicial, 562
símbolo no terminal, 562
símbolo terminal, 562
tradicional, 565
Gramáticas equivalentes, 568
Gries, D., 59

H

Hailperin, T., 500, 542
Haken, W., 365, 371
Halmos, P. R., 136
Hall, P., 480
Hamilton, W. R., 331
Harary, F., 371, 450
Hell, P., 358
Hijo, 385
derecho, 408
izquierdo, 408

Hillier, F. S., 497
Hinz, A. M., 302
Hipercubo, 311; *véase también n*-cubo
Hipótesis, 8, 38
de Turing, 589
Hohn, F., 542
Hoja, 385
Homomorfismo de una gráfica, 357
Hopcroft, J. E., 177, 566, 589
Hu, T. C., 253

I

Identidad combinatoria, 244
Identificador en un lenguaje de programación, 113
Imagen inversa, 133
Incidente, 307
Inclusivo, o. *Véase* O-inclusivo
Índice, 73
de una sucesión, 73
Inducción, 46; *véase también* Inducción matemática
Inducción matemática, 46, 160, 258
forma fuerte, 49
paso base, 48
paso inductivo, 48
principio, 47, 48
Intersección de conjuntos, 66, 68
Invariante de una gráfica, 353
Inversor, 501
Investigación de operaciones, 455
ISBN (Número estándar internacional de un libro), 127
Islas cuadráticas de Koch, 573
Isomorfismo
de árboles, 429
de árboles binarios, 433
de árboles con raíz, 431
de gráfica, 350

J

Jacobs, H. R., 59
Jarvis, R. A., 608
Johnsonbaugh, R., 597
Jones, R. H., 460
Juego de las cinco monedas, 423
de las cuatro monedas, 424

K

Kasparov, G., 446
Kelley, D., 589
Kleinrock, L., 460
Kline, M., 59
Knuth, D. E., 59, 193, 301, 302, 450
Kobler, J., 371
Kocher, P., 192
Kohavi, Z., 542
König, D., 304, 371
Kroenke, D., 119, 136
Kruse, R. L., 302
Kurosaka, R. T., 428

L

Lazo, 93, 308
Leighton, F. T., 371, 450
Lema, 34
Lenguaje, 562
formal, 562
generado por una gramática, 563, 569
libre de contexto, 566
natural, 562
regular, 566
sensible al contexto, 566
tradicional. *Véase* Lenguaje regular
Lenstra, A., 192
Lerner, D., 597
Lester, B. P., 371, 450
Lewis, T. G., 371, 450
Ley de involución
para álgebras booleanas, 518
para conjuntos, 68
Leyes asociativas
para álgebras booleanas, 516
para conjuntos, 67
Leyes conmutativas
para álgebras booleanas, 516
para conjuntos, 67

Leyes de absorción
para álgebras booleanas, 518
para conjuntos, 68
Leyes de acotación
para álgebras booleanas, 518
para conjuntos, 68
Leyes de De Morgan
generalizadas para la lógica, 27
para álgebras booleanas, 518
para conjuntos, 68
para la lógica, 14, 27
Leyes de idempotencia
para álgebras booleanas, 518
para conjuntos, 68
Leyes de los complementos
para álgebras booleanas, 516
para conjuntos, 68
Leyes del 0/1
para álgebras booleanas, 518
para conjuntos, 68
Leyes del neutro y del idéntico
para álgebras booleanas, 516
para conjuntos, 67
Leyes distributivas
para álgebras booleanas, 516
para conjuntos, 67
Leyland, P., 192
Límite
inferior, 76
superior, 76
Lindenmayer, A., 571
Lipschutz, S., 136
Literal, 504
Liu, C. L., 59, 253, 302, 450, 496
Locura instantánea, 366
Lógica, 1
leyes de De Morgan, 14, 27
Longitud
de una cadena, 79
externa, 415
interna, 415
Longitud de un camino, 309, 316
externa, 415
interna, 415
Lucas, É., 263
Lugar, 487
de entrada para una transición, 488
de salida para una transición, 488

M

Manber, U., 193
Mandelbrot, B. B., 589
Mapa plano, 365
Máquina de estado finito, 546, 547
sumador en serie, 550
Máquina de Turing, 589
Marcado, 487
acotado, 492
alcanzable, 488
seguro, 492
Marcha de Jarvis, 608
Martin, G. E., 51, 59
Matrices iguales, 611
Matriz, 610
cuadrada, 614
de adyacencia, 344
de incidencia, 348
de un ciclo fundamental, 399
de una relación, 114
identidad, 614
igualdad, 611
invertible, 614
multiplicación, 612
potencia, 613
producto, 612
producto por un escalar, 611
suma, 611
tamaño, 610
traspuesta, 614
Máximo común divisor, 152
algoritmo para calcular, 151, 155, 161
Maxtérmino, 528
McCalla, T. R., 536, 542
McNaughton, R., 193, 589
Mendelson, E., 536, 542
Método de los vecinos más cercanos, 407
Método iterativo para resolver relaciones de recurrencia, 270
Métodos de ramificación y acotación, 371
Mintérmino, 527
Modelo de anillo para el cómputo en paralelo, 333
Modelo de malla para el cómputo en paralelo, 351
Módulo 2, 540
Mu torere, 449
Multiplicación de matrices, 612

N

n-ada, 70
n-cubo, 311, 333, 351
Nadler, M., 371
Navratilova, M., 377
Negación de una proposición, 6
Newman, J. R., 371
Nievergelt, J., 193, 450
Nilsson, N. J., 450
Nim, 441, 446
Nivel de un vértice, 378
Niven, I., 253
No comparable, 97
Nodo, 306, 307
Notación
de suma, 76
del producto, 76
O mayúscula, 168
omega, 168
polaca, 420
polaca inversa, 420
sigma, 76
theta, 168
Notación de suma, 76
índice, 76
límite inferior, 76
límite superior, 76
Notación del producto, 76
índice, 76
límite inferior, 76
límite superior, 76
Notación sigma, 76
índice, 76
límite inferior, 76
límite superior, 76
Número de Stirling
de primer tipo, 223, 269
de segundo tipo, 224, 269

Número estándar internacional de un libro (ISBN), 127
Números
armónicos, 59
de Catalan, 219, 260, 266, 435
de Euler, 270
de Schröder, 267
Nyhoff, L., 302

O

O inclusiva. *Véase* O-inclusivo
O-exclusivo, 5, 524
O-inclusivo, 5
Operador, 419
binario, 131
binario conmutativo, 135
de asignación (:=), 143
de fusión (join), 120
de igualdad (=), 146
de proyecto, 120
de selección, 120
módulo, 126
unario, 131
Operadores lógicos, 146
Operando, 419
Orden
de una función, 168
lexicográfico, 228
total, 97
Orden parcial, 97
elemento comparable, 97
elemento no comparable, 97
Ordenamiento
por fusión, 293
por inserción, 299
por inserción binaria, 453
por selección, 287
por torneo, 429
tiempo en el peor de los casos, 427
Ore, O., 371, 450

P

Padre de un vértice, 385
Palindromo, 204

Par ordenado, 69
Parámetro, 145
Partición de un conjunto, 69, 104, 224
Paso base en la inducción matemática, 48
Paso inductivo, 48
Pearl, J., 445
Peitgen, H., 570
Permutación, 210
  algoritmo para su generación, 233
  de orden *r*, 212
  desordenamiento, 287
  generalizada, 235
  sube/baja, 270
Permutaciones generalizadas, 235
Peso de una arista, 309
Petri, C., 496
Pfleeger, C. P., 193
Piso, 128
Planeación de tareas, 97
Poda alfa-beta, 444
Poliminó, 51
  de orden *s*, 51
Política para resolver colisiones, 128
Pósa, L., 336
Potencia de una matriz, 613
Premisa, 38
Preparata, F. P., 608
Primo, 2
Principio
  aditivo, 201
  de inducción matemática, 47, 48
  de la caja de zapatos, 248
  de la gaveta de Dirichlet, 248
  de la pichonera, 248
  de las casillas. *Véase* Principio de la pichonera
  de multiplicación, 198
  del buen orden para los enteros positivos, 55
Problema
  de interrupción, 177
  de la comida de los filósofos, 495
  de las cuatro reinas, 396
  de los cuatro colores, 365
  de los puentes de Königsberg, 319
  de minimización, 534
  de programación lineal, 496
Problema (*continuación*)
  de todos los vecinos más próximos, 600
  de transporte, 497
  del agente de ventas viajero, 177, 309, 332
  del ciclo hamiltoniano, 177, 331
  del vecino más cercano, 600
  intratable, 177
  sin solución, 177
Problema del par más cercano, 593
  algoritmo para su solución, 596
  cota inferior, 598
Procedimiento, 145
  de etiquetado, 467
  minimáx, 442
  parámetro, 145
Prodinger, H., 329
Producción, 562, 568
Producto
  cartesiano, 69
  cruz, 603
  por un escalar, 611
Proposición, 2
  bicondicional, 13
  compuesta, 3
  condicional, 8, 23
  conjunción, 2
  contradicción, 36
  disyunción, 3
  lógicamente equivalente, 13
  negación, 6
  o exclusivo, 5
  o inclusivo, 5
  tabla de verdad, 3
Proposición condicional, 8, 23
  antecedente, 8
  conclusión, 8
  consecuente, 8
  contrapositiva, 15
  hipótesis, 8
  recíproca, 11
  transposición, 15
Proposiciones lógicamente equivalentes, 13
Prusinkiewicz, P., 571
Punto de articulación, 327
Putahi, 449

Q

Quinn, M. J., 371, 450

R

r-combinación, 214
r-permutación, 212
Rango
  de una función, 125
  de una relación, 92
Razonamiento deductivo, 38
Read, R. C., 353
Recíproco de una proposición condicional, 11
Reconocimiento de patrones, 310
Recorrido de un árbol, 415
  en posorden, 418
  en preorden, 416
  entre orden, 418
Recorrido del caballo, 335
Red, 456
  acoplamiento, 479
  corte, 473
  de transporte, 456
  determinación de un flujo máximo, 466
  en economía, 263, 273
  flujo, 457
  flujo máximo, 462
  fuente, 456
  sumidero, 456
Red de Petri, 487
  bloqueada, 492
  elemento, 487
  lugar, 487
  marcada, 487
  marcado, 487
  transición, 487
  viva, 492
Reducción en serie de una gráfica, 360
Refutación completa, 45
Reingold, E., 193, 253
Relación, 91, 92, 125
  antisimétrica, 94
  binaria, 92
  cerradura transitiva, 110
Relación (*continuación*)
  composición, 99
  de equivalencia, 104, 220, 237, 319, 511, 512, 559, 568
  digráfica, 93
  dominio, 92
  inversa, 98
  matriz, 114
  n-aria, 118
  orden parcial, 97
  orden total, 97
  rango, 92
  recurrencia, 257
  reflexiva, 93
  simétrica, 94
  transitiva, 95
Relación de equivalencia, 104, 220, 237, 319, 511, 512, 559, 568
  clase de equivalencia, 106
Relación de recurrencia, 257
  condiciones iniciales, 257
  homogénea lineal, 274, 282
  no homogénea, 274, 282
  no lineal, 274
  solución, 270
Retraso unitario de tiempo, 547
Retroceso, 395
Riordan, J., 242, 253
Ritter, G. L., 136
Rivest, R. L., 190
Roberts, F. S., 253, 302
Robinson, J. A., 42
Ross, K. A., 59

S

Saad, Y., 371
Sabatini, G., 377
Salida, 143
Schwenk, A. S., 336
Seguimiento, 143
Seidel, R., 608
Seles, M., 377
Septominó tridimensional, 54
Serie armónica, 59
Seudocódigo, 144

Shamir, A., 190
Shamos, M. I., 608
Shannon, C. E., 500
Símbolo
de entrada, 548, 556, 576
de salida, 548
inicial, 562, 568
no terminal, 562, 568
terminal, 562, 568
Sistema
Braille, 203
consulta, 120
críptico, 189
críptico de clave pública RSA, 189
de administración de una base de datos, 119
matemático, 34
Sistema numérico, 84
base, 84
binario, 84
decimal, 84
hexadecimal, 88
octal, 91
Slagle, J. R., 450
Smith, A. R., 571
Solow, D., 59
Solución de relaciones de recurrencia, 270
método de iteración, 270
método para recurrencia homogénea lineal, 275
Stevenson, A., 142
Stoll, R. R., 136
Suavizante binomial, 247
Subárbol, 385
Subcadena, 83
Subconjunto, 65
propio, 65
Subgráfica, 317
Subsucesión, 75
Sucesión, 73, 125
creciente, 75
de Bruijn, 328
de Fibonacci, 162, 186, 257, 263, 278
de Lucas, 269
de tribonacci, 195
decreciente, 75
índice, 73
Sucesión (*continuación*)
suavizante, 247
subsucesión, 75
Sudkamp, T. A., 589
Suma
de matrices, 611
geométrica, 49
Sumidero, 456
Superfuente, 459
Supersumidero, 459

T

Tabla
de conmutación, 508
de verdad, 3
lógica, 501
Tablero deficiente, 51
Tamaño de una matriz, 610
Tarjan, R. E., 265, 450, 496
Taubes, G., 192
Techo, 128
Teorema, 34
de Kuratowski, 361
de los matrimonios de Hall, 481
del binomio, 242
del flujo máximo y corte mínimo, 476
Término no definido, 34
Tesis de Church, 589
Tiempo
de un algoritmo en el caso promedio, 166, 173
de un algoritmo en el mejor de los casos, 166, 173
de un algoritmo en el peor de los casos, 166, 173
de un ordenamiento en el peor de los casos, 427
Torneo de eliminación simple, 377, 409
Torres de Hanoi, 261, 272, 283
Transición, 487
activación, 488
activada, 488
lugar de entrada, 488
lugar de salida, 488
Trasposición, 15
Traspuesta de una matriz, 614
Triangulación de una gráfica plana, 365
Triángulo de Pascal, 244

Trominó, 51
derecho, 51
Tucker, A., 59, 242, 253, 302, 371, 496

U

Ullman, J. D., 136
Unión de conjuntos, 66, 68
Universo, 67
UPC (código universal de productos), 133

V

Valor
alfa, 444
beta, 444
de un flujo, 458
Variable
acotada, 21
de acotación, 21
libre, 21
VCR Plus+, 380
Vértice, 93, 306, 307
adyacente, 307
aislado, 308
ancestro, 385
cubierta, 422
de ramificación, 385
descendiente, 385
distancia, 329
Vértice (*continuación*)
excentricidad, 384
flujo desde un, 457
flujo hacia un, 457
grado, 320
grado de entrada, 328
grado de salida, 328
hermano, 385
hijo, 385
incidente, 307
interno, 385
nivel, 378
padre, 385
terminal, 385
Vértices
adyacentes, 307
hermanos, 385
Vilenkin, N. Y., 253
Vuelta
a la derecha, 602
a la izquierda, 602

W

Wagon, S., 330
Ward, S. A., 542
Wilson, R. J., 371
Wood, D., 571, 589
Wos, L., 45

BIBLIOTECA FACULTAD Cs. EXACTAS, INGENIERIA Y AGRIMENSURA ROSARIO

Lógica

$p \vee q$: p o q; página 3
$p \wedge q$: p y q; página 2
$\bar{p}$: no p; página 6
$p \rightarrow q$: si p, entonces q; página 8
$p \leftrightarrow q$: p si y sólo si q; página 13
$P \equiv Q$: P y Q son lógicamente equivalentes; página 13
$\forall$: para todo; página 20
$\exists$: existe; página 21
$\therefore$: por lo tanto; página 38

Notación de conjuntos

$\{x_1, \ldots, x_n\}$: conjunto que consta de los elementos $x_1, \ldots, x_n$; página 64
$\{x \mid p(x)\}$: conjunto formado por aquellos elementos x que satisfacen la propiedad $p(x)$; página 64
$x \in X$: x es un elemento de X; página 64
$x \notin X$: x no es un elemento de X; página 64
$X = Y$: igualdad de conjuntos (X y Y tienen los mismos elementos); página 64
$|X|$: número de elementos en X; página 64
$\varnothing$: conjunto vacío; página 64
$X \subseteq Y$: X es un subconjunto de Y; página 65
$\mathcal{P}(X)$: conjunto potencia de X (todos los subconjuntos de X); página 65
$X \cup Y$: X unión Y (todos los elementos en X o en Y); página 66

$\bigcup_{i=1}^{n} X_i$: unión de $X_1, \ldots, X_n$ (todos los elementos que pertenecen al menos a uno de los conjuntos $X_1, X_2, \ldots, X_n$); página 68

$\bigcup_{i=1}^{\infty} X_i$: unión de $X_1, X_2, \ldots$ (todos los elementos que pertenecen al menos a uno de los conjuntos $X_1, X_2, \ldots$); página 68

$\cup \mathcal{S}$: unión de $\mathcal{S}$ (todos los elementos que pertenecen al menos a un conjunto en $\mathcal{S}$); página 68

$X \cap Y$: X intersección Y (todos los elementos en X y en Y); página 66

$\bigcap_{i=1}^{n} X_i$: intersección de $X_1, \ldots, X_n$ (todos los elementos que pertenecen a cada uno de los conjuntos $X_1, X_2, \ldots, X_n$); página 68

$\bigcap_{i=1}^{\infty} X_i$: intersección de $X_1, X_2, \ldots$ (todos los elementos que pertenecen a cada uno de los conjuntos $X_1, X_2, \ldots$); página 68

$\cap \mathcal{S}$: intersección de $\mathcal{S}$ (todos los elementos que pertenecen a cada conjunto en $\mathcal{S}$); página 68
$X - Y$: diferencia de conjuntos (todos los elementos en X que no están en Y); página 66
$\overline{X}$: complemento de X (todos los elementos que no están en X); página 67
(x, y) : par ordenado; página 69
$(x_1, \ldots, x_n)$: n-ada; página 70
$X \times Y$: producto cartesiano de X y Y (pares (x, y) tales que $x \in X$ y $y \in Y$); página 69

BIBLIOTECA FACULTAD Cs. EXACTAS, INGENIERIA Y AGRIMENSURA ROSARIO

Relaciones

xRy : (x, y) está en R (x está relacionado con y mediante la relación R); página 92
$[x]$: clase de equivalencia que contiene a x; página 106
R^{-1} : relación inversa (todas las (y, x) con (x, y) en R); página 98
$R_2 \circ R_1$: composición de relaciones; página 99
$x \preceq y$: xRy; página 97.

Funciones

$f(x)$: valor asignado a x; página 126
$f: X \rightarrow Y$: función de X en Y; página 125
$f \circ g$: composición de f y g; página 130
f^{-1} : función inversa (todas las (y, x) con (x, y) en f); página 130
$f(n) = O(g(n))$: $|f(n)| \leq C|g(n)|$ para n suficientemente grande; página 168
$f(n) = \Omega(g(n))$: $c|g(n)| \leq |f(n)|$ para n suficientemente grande; página 168
$f(n) = \Theta(g(n))$: $c|g(n)| \leq |f(n)| \leq C|g(n)|$ para n suficientemente grande; página 168

Conteo

$C(n, r)$: número de r combinaciones de un conjunto de n elementos ($n!/[(n-r)!r!]$); página 214
$P(n, r)$: número de r permutaciones de un conjunto de n elementos ($n(n-1)\cdots(n-r+1)$); página 212

Gráficas

$G = (V, E)$: gráfica G con conjunto de vértices V y conjunto de aristas E; página 306
(v, w) : arista; página 306
$\delta(v)$: grado de un vértice v; página 320
$(v_1, \ldots, v_n)$: camino de v_1 a v_n; página 316
$(v_1, \ldots, v_n)$; $v_1 = v_n$: ciclo; página 319
K_n : gráfica completa con n vértices; página 312
$K_{m,n}$: gráfica bipartita completa con m y n vértices; página 313
$w(i, j)$: peso de la arista (i, j); página 338
F_{ij} : flujo en la arista (i, j); página 457
C_{ij} : capacidad de la arista (i, j); página 456
$(P, \bar{P})$: corte en una red; página 473